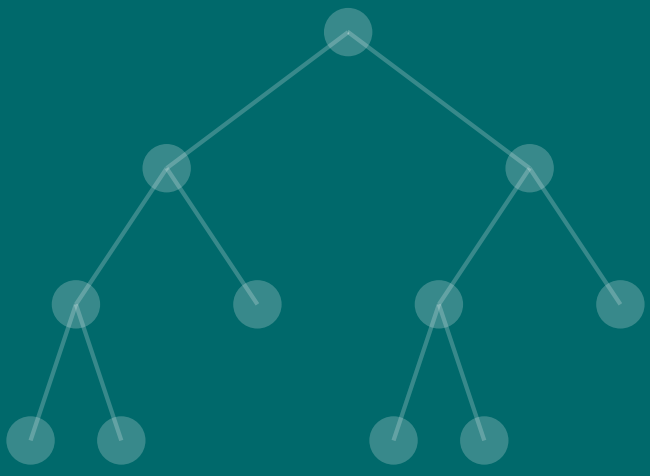
Terminale

Numérique et Sciences Informatiques

Cours • Exercices • Corrigés



Mathieu Thibaud

Édition 2026

Version du 8 novembre 2026 — v2026.11.08 (sources : commit a3b7153 + modifications non validées)

Sommaire

Présentation de l'année 4
Charte d'usage de l'intelligence artificielle 10
Panorama de l'année 12

Cours et exercices

1 Récursivité. 17
2 Programmation orientée objet et paradigmes 66
3 Structures linéaires : piles, files, listes chaînées 126
4 Processus et ordonnancement 176
5 Arbres 226
6 Bases de données et SQL 279
7 Diviser pour régner 332
8 Graphes 384
9 Réseaux 439
10 Cryptographie 480
11 Programmation dynamique 527
12 Recherche textuelle 582
13 Calculabilité et décidabilité 624
14 Systèmes sur puce et informatique embarquée 669
Problèmes de synthèse et exercices type bac 706

Préparer les épreuves du bac

Réussir la partie écrite 716
Index des exercices type bac 721
Réussir l'épreuve pratique 724
Présentation des sujets. 726
Sujet 24 — Arbre d'une expression arithmétique 727
Sujet 25 — Arbre binaire de recherche 729
Sujet 26 — Sortir d'un labyrinthe. 731
Sujet 27 — Calculatrice en notation polonaise inverse 733
Sujet 28 — Ordonnancement de tâches : le tourniquet 735
Sujet 29 — Recherche dichotomique. 737
Sujet 30 — Messages secrets 739
Sujet 31 — Recherche d'un motif. 741

Sujet 32 — Le grand escalier 743
Préparer le Grand Oral 745

Corrigés

1 Récursivité. 756
2 Programmation orientée objet et paradigmes 775
3 Structures linéaires : piles, files, listes chaînées 790
4 Processus et ordonnancement 811
5 Arbres 829
6 Bases de données et SQL 860
7 Diviser pour régner 885
8 Graphes 908
9 Réseaux 925
10 Cryptographie 937
11 Programmation dynamique 955
12 Recherche textuelle 975
13 Calculabilité et décidabilité 991
14 Systèmes sur puce et informatique embarquée 1007
Problèmes de synthèse et exercices type bac 1023
Corrigé du sujet 24 — Arbre d'une expression arithmétique 1033
Corrigé du sujet 25 — Arbre binaire de recherche 1035
Corrigé du sujet 26 — Sortir d'un labyrinthe 1038
Corrigé du sujet 27 — Calculatrice en notation polonaise inverse 1040
Corrigé du sujet 28 — Ordonnancement de tâches : le tourniquet 1043
Corrigé du sujet 29 — Recherche dichotomique 1045
Corrigé du sujet 30 — Messages secrets 1048
Corrigé du sujet 31 — Recherche d'un motif 1050
Corrigé du sujet 32 — Le grand escalier 1053

Rappels de Première et lexique

Les indispensables de Première 1056
Lexique 1066
Crédits iconographiques 1075

Présentation de l'année

Cette année prolonge et couronne ce que vous avez commencé en Première. On monte d'un cran : de nouvelles **structures de données** (arbres, graphes), des **méthodes algorithmiques** puissantes, les **bases de données**, le fonctionnement profond des **machines et des réseaux**… et même les **limites** de ce qu'un ordinateur peut calculer. Objectif : devenir autonome et rigoureux, et arriver prêt(e)s aux épreuves.

L'esprit NSI, version Terminale

- **Autonomie** : concevoir une solution, la programmer, la tester, la corriger — par vous-même.
- **Abstraction** : choisir la bonne *structure de données* et la bonne *méthode* pour chaque problème.
- **Rigueur** : justifier qu'un programme *se termine*, qu'il est *correct*, et estimer son *coût*.

I La discipline, cette année

La NSI est une **spécialité scientifique** à part entière. En Terminale, c'est **6 heures par semaine**, dont une bonne partie **sur machine**. Elle s'articule autour des mêmes grands piliers qu'en Première, poussés plus loin :

- **programmer** — avec de nouveaux styles (objet, récursif, fonctionnel) ;
- **l'algorithmique** — des méthodes puissantes (diviser pour régner, programmation dynamique, algorithmes de graphes) ;
- **les structures de données** — listes chaînées, piles, files, arbres, graphes ;
- **les données** — les bases de données relationnelles et le langage SQL ;
- **les machines et les réseaux** — systèmes d'exploitation, processus, routage, sécurité.

Environ **un tiers de l'année** reste consacré à des **projets** et à la pratique sur machine : c'est là qu'on devient vraiment autonome.

II Ce que vous allez apprendre

Le programme est dense et s'enchaîne : la **récursivité** (chapitre 1) est la clé qui ouvre les arbres, « diviser pour régner » et les graphes. L'année alterne les **algorithmes** et les **machines** : on découvre le système d'exploitation dès l'automne, les réseaux en hiver.

A. De la rentrée aux vacances de Toussaint

Chapitre 1 — La récursivité. Résoudre un problème en le ramenant à une version plus petite de lui-même : une fonction qui s'appelle elle-même. Élégant, et incontournable pour toute la suite.

Chapitre 2 — La programmation orientée objet et les paradigmes. Concevoir vos propres « objets » (leurs données *et* leurs actions) pour organiser des programmes plus gros et plus clairs.

Chapitre 3 — Les structures linéaires : piles, files, listes chaînées. Construire vos propres boîtes à ranger l'information : listes chaînées, piles (« dernier arrivé, premier servi ») et files (comme une file d'attente).

B. De la Toussaint à Noël

Chapitre 4 — Processus et ordonnancement. Comprendre comment le système d'exploitation fait tourner plusieurs programmes « en même temps » et gère les ressources de la machine.

Chapitre 5 — Les arbres. Manipuler des structures « en arborescence » (comme un arbre généalogique) et y ranger ou rechercher une information très efficacement.

Chapitre 6 — Bases de données et SQL. Interroger de vraies bases de données avec le langage SQL : retrouver, croiser et mettre à jour de très grandes quantités de données.

C. De janvier aux vacances d'hiver

Chapitre 7 — Diviser pour régner. Couper un gros problème en deux moitiés, les résoudre séparément, puis recombiner : la recette des algorithmes rapides, comme le tri fusion.

Chapitre 8 — Les graphes. Modéliser des réseaux (routes, amis, Internet) par des points reliés, et écrire des algorithmes pour s'y déplacer, détecter un cycle, trouver un chemin.

Chapitre 9 — Les réseaux. Suivre le voyage d'un message sur Internet : adressage, découpage en paquets, routage — comment les données trouvent leur chemin d'un bout à l'autre du monde.

D. De l'hiver au printemps

Chapitre 10 — Cryptographie. Protéger ces échanges : le chiffrement (symétrique et asymétrique) qui rend un message illisible pour tout autre que son destinataire.

Chapitre 11 — La programmation dynamique. *Rendre certains algorithmes récursifs spectaculairement plus rapides, en évitant de recalculer sans cesse la même chose.*

Chapitre 12 — La recherche textuelle. Trouver efficacement un motif dans un grand texte (un roman, un génome) : de la méthode naïve à l'algorithme de Boyer-Moore.

E. Au printemps

Chapitre 13 — Calculabilité et décidabilité. Toucher aux **limites** de l'informatique : existe-t-il des problèmes qu'*aucun* programme ne pourra jamais résoudre ? (Oui — et c'est vertigineux.)

Chapitre 14 — Systèmes sur puce et informatique embarquée. Découvrir comment un ordinateur entier tient sur une seule puce (dans votre téléphone, votre console, votre voiture) et ce que fait un microcontrôleur dans un système embarqué.

III Comment est organisé ce livre

Comment est organisé un chapitre

Chaque chapitre de ce livre suit le même parcours :

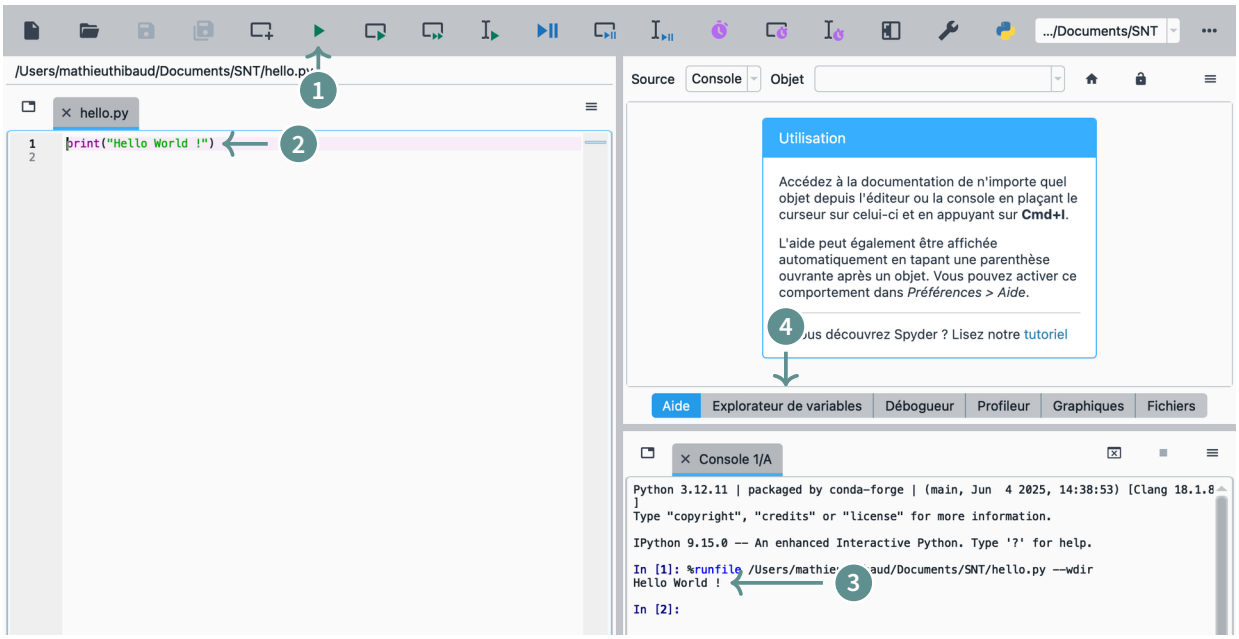
- **Activités préparatoires** : courtes, souvent sans ordinateur, à faire *avant* le cours pour se poser les bonnes questions ;
- **Cours** : les notions, des exemples corrigés et un bilan ;
- **Exercices** : du plus simple au niveau du bac, avec des coups de pouce en fin de rubrique ; environ un tiers sont marqués • essentiel (le minimum pour suivre) ; leurs corrigés sont en fin de livre ;
- **TP et projets** : un travail plus long sur machine qui réinvestit tout le chapitre ;
- **Fiche de révision** : l'essentiel du chapitre en deux pages, avec « Se tester » (questions sur le chapitre), « Et avant ? » (questions sur les chapitres précédents, pour ne pas oublier) et, souvent, un encadré « Vers le bac » (problèmes de synthèse et sujet d'épreuve pratique que vous pouvez désormais traiter) ;
- **Défis Advent of Code** : des énigmes de programmation tirées d'*Advent of Code* (`adventofcode.com`), un concours en ligne créé et animé depuis 2015 par le développeur **Eric Wastl**, qui publie chaque année, du 1er au 25 décembre, une énigme en deux parties. L'énoncé, en anglais, se lit sur le site (QR code sur chaque défi ; le navigateur peut le traduire) ; chaque défi est accompagné d'un lexique, d'un résumé, d'un petit exemple pour se tester et de coups de pouce. Ces défis sont **facultatifs** et n'utilisent que des notions déjà vues dans l'année.

Les défis n'ont volontairement pas de corrigé dans ce livre : tout le plaisir d'une énigme est de la résoudre soi-même. Le site indique si votre réponse est juste (avec un compte gratuit) ; en cas de blocage, lisez les coups de pouce, puis venez en parler en classe.
En fin de livre, **Les indispensables de Première** rappellent en quelques pages les notions de Première utiles cette année (la ligne « Prérequis » de chaque cours y renvoie), et un **lexique** donne le sens des mots importants, avec un lien vers leur définition dans le cours.

IV Travailler avec ce livre

A. Avec quel outil ?

Au lycée, on programme avec **Spyder**, un logiciel libre et gratuit. Installez-le aussi chez vous : l'installateur officiel se télécharge sur `spyder-ide.org` et contient Python, il n'y a rien d'autre à installer. En dépannage (ordinateur prêté, installation impossible), **Basthon** fonctionne dans un navigateur, sans installation (`basthon.fr`) : parfait pour un exercice court, moins pratique pour un TP ou un défi qui lit des fichiers, qu'il faut alors déposer dans la page à chaque séance.



Les repères de la capture :

1. le bouton vert ▶ **Exécuter** (ou la touche F5) : il enregistre et lance tout le fichier ;
2. **l'éditeur** (à gauche) : on y écrit le programme, enregistré dans un fichier `.py` ;
3. **la console** (en bas à droite, `In [1]:`) : on y lit les résultats, et on y tape une instruction pour tester une idée ou appeler une fonction du fichier une fois celui-ci exécuté ;
4. l'onglet **Explorateur de variables** : après l'exécution, il montre la valeur de chaque variable (pratique pour comprendre un programme qui ne fait pas ce qu'on attend).

B. Une méthode pour chaque chapitre

1. **Après chaque séance** (20 à 30 minutes) : relire le passage du cours en *retapant* les exemples dans Spyder, puis faire les exercices • essentiel indiqués par les renvois « Exercices d'application ».
2. **Bloqué ?** Chercher vraiment (5 à 10 minutes), puis lire le **coup de pouce** (▷) : d'abord la piste, ensuite seulement le début de solution.
3. **Le corrigé** se lit *après* avoir essayé : comparer, repérer son erreur (les corrigés signalent les *erreurs fréquentes*), puis refaire l'exercice sans regarder quelques jours plus tard.
4. **Deux ou trois jours après** : répondre de mémoire aux questions « Se tester » et « Et avant ? » de la fiche de révision, puis vérifier avec le lien « ▶ réponses ». Revenir régulièrement sur les chapitres passés, c'est ce qui fait tenir les connaissances jusqu'au bac.
5. **Avant un test** : la fiche de révision et sa liste « Savoir-faire » ; pour chaque case que vous ne pouvez pas cocher, refaire un exercice essentiel.

Les exercices ★★★, les défis et les encadrés « Pour les curieux » servent à aller plus loin : utiles, mais facultatifs.

C. Lire ce livre sur un ordinateur

- **Tout est cliquable** : le sommaire, le petit sommaire de chaque chapitre, « ▶ corrigé p. . . . » et « ↑ énoncé », les coups de pouce, les numéros d'exercices des renvois et les mentions « chapitre 5 ». L'**onglet de couleur** au bord de la page ramène à la page de titre du chapitre ; le **numéro de page** ramène au sommaire.
- **Les signets** (panneau latéral « Signets » ou « Table des matières » du lecteur PDF) donnent le plan détaillé : chapitre, rubrique, section.
- **Revenir où l'on était** après un lien : le lien « ↑ énoncé » ou l'onglet du chapitre, ou bien la commande « page précédente » du lecteur (Alt + ← dans Acrobat Reader sous Windows, Cmd + [dans Aperçu sur Mac).
- **Copier un programme** : cliquer sur l'icône ⧉ en haut à droite d'un bloc de code ; une page s'ouvre avec le programme prêt à être copié. Un copier-coller direct depuis le PDF emporte souvent les numéros de ligne et l'indentation est perdue.
- **Chercher un mot** : Ctrl + F (Cmd + F sur Mac).
- Pour lire confortablement, choisir l'affichage « ajuster à la largeur ». Les numéros de page du lecteur sont ceux du livre imprimé.

V Le cap : le baccalauréat

Les épreuves

- **l'épreuve de spécialité** (en **juin**, coefficient **16**), en deux parties : une **partie écrite** de 3 h 30, notée sur 12, qui porte sur l'ensemble du programme, et une **partie pratique** d'1 h sur machine, notée sur 8 (voir la deuxième partie de ce livre) ;
- **le Grand Oral** (fin juin, **après les écrits**, coefficient **8**) : vous préparez **deux questions**, dont **au moins une doit s'appuyer sur la NSI** ; le jour J, 20 minutes de préparation, puis 10 minutes d'exposé et 10 minutes d'échange avec le jury.

Comme les épreuves ont lieu **en fin d'année**, on dispose de **toute l'année** pour construire le programme. Le **troisième trimestre** sert à le **terminer et le consolider**, à s'entraîner en conditions (**bacs blancs**, pratique sur machine) et à préparer le **Grand Oral**.

VI Comment vous êtes évalué

- à chaque chapitre : un ou deux courts **tests rapides** (/10) puis un **contrôle** (/20) ;
- des **bacs blancs** pour s'entraîner en conditions ;
- des **projets et travaux sur machine**, qui comptent aussi.

L'intelligence artificielle : une charte, trois pastilles

Les assistants d'IA savent écrire du code et des explications ; le but de ce cours est que **vous** sachiez le faire, le vérifier et l'expliquer. Une **charte d'usage de l'IA**, reproduite juste après cette présentation, fixe ce qui est permis. L'usage est décidé **exercice par exercice** et signalé, dans les feuilles d'exercices, par une pastille à côté du titre ; les évaluations (tests, contrôles, sujets d'épreuve) n'en portent pas : elles se font toujours sans IA.

-  **sans IA** : exercices d'automatisme (lecture de code, questions de cours) ;
-  **IA en appui** : pour déboguer, reformuler ou vérifier, à condition d'écrire soi-même la réponse finale ;
-  **IA intégrée** : l'exercice consiste à critiquer, tester ou améliorer une réponse d'assistant.

Dans tous les cas : vous restez responsable de ce que vous rendez, vous devez pouvoir l'expliquer à l'oral, et vous dites quand l'IA a été utilisée. La complétion automatique de code dans l'éditeur compte comme un assistant.

Remarque — *Une ressource pour réviser à la maison*

Deux sites tenus par des professeurs de NSI sont d'excellentes ressources pour la terminale : claires, complètes et conformes au programme. Ils ont été une **source d'inspiration** pour la préparation de nos cours, et je vous encourage à les consulter pour réviser ou approfondir, notamment en vue de l'épreuve écrite :

- le site de **Gilles Lassus** : `glassus.github.io` ;
- le site de **David Roche** : `pixees.fr/informatiquelycee`.

Une année exigeante, mais passionnante. À la fin, vous saurez vraiment programmer — et penser comme un informaticien.

Charte d'usage de l'intelligence artificielle

Les assistants d'IA écrivent du code et des explications, souvent bien, parfois faux, toujours avec assurance. Le but de ce cours n'est pas d'obtenir une réponse : c'est que **vous** sachiez la produire, la vérifier et l'expliquer. L'usage de l'IA n'est donc ni interdit partout ni permis partout : il est **décidé exercice par exercice**, et indiqué, dans les feuilles d'exercices, par une pastille à côté du titre. Les évaluations (tests, contrôles, bacs blancs) ne portent pas de pastille : elles se font **toujours sans IA**.

Ce qui compte comme « IA » dans cette charte

- Les **assistants conversationnels** (chatbots), quel que soit le site ou l'application.
- La **complétion automatique de code** dans l'éditeur (par exemple Copilot dans VS Code) : elle compte comme un assistant, même si l'on ne « pose pas de question ».
- Les **résumés, traductions et réponses générées** affichés en tête des moteurs de recherche.
- Ne comptent pas : le cours, le manuel, la documentation officielle, un moteur de recherche classique, le correcteur orthographique.

Les trois pastilles

Pastille	Vous pouvez	Vous ne pouvez pas
 exercices d'automatisme ; règle de toutes les évaluations (tests, contrôles, bacs blancs), qui ne portent pas de pastille	Utiliser le cours et vos notes quand ils sont autorisés ; vos propres essais.	Utiliser un assistant ou la complétion de code, **ni avant ni pendant**. Pour un travail à la maison, s'en servir revient à se priver de l'entraînement que le test mesure.
 exercices du livre, préparation à la maison (pastille par défaut)	Faire expliquer un message d'erreur ; reformuler un énoncé ; demander un exemple ; **vérifier** une réponse que vous avez déjà écrite.	Faire écrire la réponse ou le code à votre place, puis le recopier. La réponse finale est **écrite par vous** et vous savez l'expliquer.
 exercices conçus avec l'IA	Tout, y compris générer une réponse : l'exercice consiste à la **critiquer**, la **comparer**, la **tester** ou l'**améliorer**.	Rendre la réponse de l'assistant telle quelle : ce qui est évalué, c'est **votre analyse**.

Trois règles, quelle que soit la pastille

1. **Vous êtes responsable de ce que vous rendez.** Une erreur de l'assistant est votre erreur.
2. **Vous devez pouvoir expliquer chaque ligne rendue**, à l'oral, sans préparation. Ce que vous ne savez pas expliquer n'est pas votre travail.
3. **Vous dites quand l'IA a été utilisée**, en une ligne en tête du travail : « IA utilisée pour : … ». Le dire n'est jamais pénalisé ; le cacher l'est.

Pourquoi ces règles

- Un assistant répond avec la même assurance quand il a raison et quand il a tort. Savoir **vérifier** une réponse (la tester, la tracer, la comparer au cours) est précisément une compétence de ce cours : c'est le sens des exercices  .
- Les épreuves du bac (partie écrite et partie pratique de l'épreuve de spécialité, Grand Oral) se passent **sans aucune aide d'IA**. S'entraîner avec une aide que vous n'aurez pas le jour J, c'est se préparer à une autre épreuve que la vôtre.
- Sur la plateforme d'entraînement de la classe, les outils d'IA sont réglés par le professeur et suivent la pastille de l'activité.

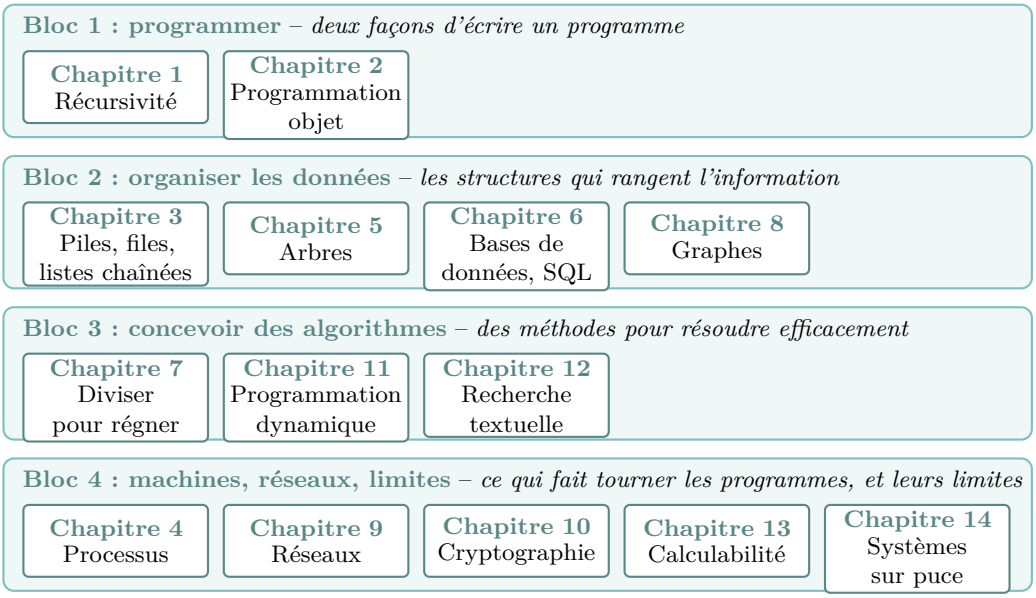
En cas de doute sur ce qui est permis pour un exercice, la règle est simple : **demandez avant**, pas après.

Panorama de l'année

Les quatorze chapitres de l'année ne sont pas indépendants : les mêmes outils (la récursivité, la pile, la file…) reviennent d'un chapitre à l'autre, et une même question — **« combien ça coûte ? »** — traverse toute l'année. Ces pages donnent une vue d'ensemble, à relire à chaque nouveau chapitre : elles aident à prendre du **recul** pour l'écrit comme pour le Grand Oral.

Le plan de l'année en quatre blocs

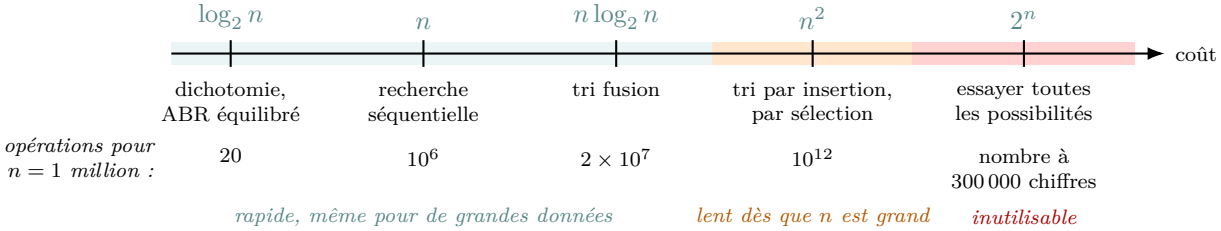
Le programme se partage en quatre grands blocs. Dans chaque bande, on lit les chapitres du bloc ; leur **numéro** indique l'ordre dans lequel on les étudie au fil de l'année.



L'année ne parcourt pas les blocs l'un après l'autre : elle **alterne**. Après les outils de base (chapitres 1 à 3), on découvre la **machine** dès l'automne (processus), puis de nouvelles structures et méthodes ; viennent ensuite le **réseau** et sa sécurité, juste après les graphes dont ils ont besoin, de nouveaux **algorithmes** au printemps, et l'on termine par les **limites** de l'informatique et les systèmes sur puce.

Le fil rouge : le coût d'un algorithme

Depuis la Première, on ne se demande pas seulement « est-ce que ça marche ? » mais aussi « **combien d'opérations** cela demande-t-il quand les données sont nombreuses ? ». On range les algorithmes sur une **échelle de coût** (n désigne la taille des données) :



Toute l'année, on cherche à **descendre cette échelle**, ou à contourner le problème quand c'est impossible. Chaque grande méthode correspond à une stratégie :

Stratégie	**L'idée**	**Chapitre**
Bien ranger les données	Avec des données organisées (triées, ou dans un arbre binaire de recherche), on élimine la moitié des candidats à chaque étape : on passe de n à $\log_2 n$.	5 Arbres (dichotomie : 1re)
Diviser pour régner	Couper le problème en deux, résoudre chaque moitié, recombiner : le tri fusion passe de n^2 à $n \log_2 n$.	7 Diviser pour régner
Ne jamais recalculer	Mémoriser les résultats des sous-problèmes : on **échange du temps contre de la mémoire**.	11 Programmation dynamique
Renoncer à l'optimum	Quand la solution exacte est hors de portée (voyageur de commerce : essayer tous les trajets), un algorithme **glouton** donne vite une solution, pas toujours la meilleure.	glouton : 1re (comparé en 11)
Faire du coût un bouclier	Le chiffrement **RSA** est sûr parce que **factoriser** un très grand nombre coûte trop cher avec les méthodes connues.	10 Cryptographie

Remarque

Le $\log_2 n$ n'est qu'un **outil de comptage** : c'est le nombre de fois où l'on peut couper n en deux avant d'arriver à 1 (à peu près le nombre de bits de n, vu en Première). Pour un million de données, $\log_2 n \approx 20$.

Chapitre par chapitre

Pour chaque chapitre : ce qu'il **réutilise**, et où il **resservira**. Les notions de Première sont signalées par « 1re ».

Chapitre	S'appuie sur...	Resservira dans...
1 Récursivité	fonctions (1re)	arbres, diviser pour régner, parcours en profondeur, programmation dynamique
2 Programmation objet	types construits (1re)	toutes les structures : classes `Pile`, `File`, `Noeud`, `Graphe`
3 Piles, files, listes chaînées	programmation objet, récursivité	parcours d'arbres et de graphes ; file des processus prêts
4 Processus	files ; arborescence de fichiers, modèle de von Neumann (1re)	arbres (l'arbre des processus) ; graphes (le cycle d'un interblocage) ; systèmes sur puce
5 Arbres	récursivité, programmation objet, piles et files ; dichotomie (1re)	index des bases de données ; graphes (un arbre est un graphe particulier)
6 Bases de données, SQL	données en table (1re) ; arbres (les index)	projets (site web adossé à une base) ; Grand Oral (données, vie privée)
7 Diviser pour régner	récursivité ; dichotomie, tris (1re)	programmation dynamique (même découpage, sans recalcul)
8 Graphes	piles et files, récursivité, programmation objet, arbres ; graphe d'attente (processus)	routage (réseaux) ; GPS
9 Réseaux	graphes pondérés (Dijkstra) ; Internet, paquets (1re)	cryptographie (sécuriser ce qui circule)
10 Cryptographie	réseaux ; coût (factorisation)	HTTPS, signature, sécurité des échanges
11 Programmation dynamique	récursivité, diviser pour régner ; glouton (1re)	problèmes d'optimisation (rendu de monnaie, sac à dos)
12 Recherche textuelle	chaînes, boucles, coût (1re)	moteurs de recherche, traitement de texte, ADN
13 Calculabilité	modèle de von Neumann : programme et données dans la même mémoire (1re)	limites de tout programme (un antivirus parfait est impossible)
14 Systèmes sur puce	modèle de von Neumann (1re) ; processus	synthèse : le téléphone, un ordinateur complet sur une puce

Remarque

Pour le **Grand Oral**, ces liens sont précieux : une bonne question en croise souvent *deux* (« Comment un message trouve-t-il son chemin sur Internet ? » = **graphes**, pour le plus court chemin, + **réseaux**, pour le routage). Voir les pages *Préparer les épreuves*, dans la deuxième partie du livre.

Une histoire de l'informatique en quelques dates

Chaque date de cette frise correspond à une notion que l'on étudie cette année, dans le chapitre indiqué à droite. Certaines sont bien plus anciennes que les ordinateurs (les graphes, les tours de Hanoï) ; la plupart sont nées entre 1936 et 1980. Le programme retient trois repères : **1936** (la machine universelle), **1948** (les premiers ordinateurs à programme enregistré, dont la puissance croît ensuite de façon exponentielle) et **1969** (le réseau qui deviendra Internet).

Au fil de ces dates, les rôles du **logiciel** et du **matériel** ont changé : en 1945, on recâblait la machine pour chaque calcul ; dès 1948, le programme enregistré en mémoire permet de tout changer par logiciel ; systèmes d'exploitation et langages font ensuite porter l'essentiel des fonctions au logiciel ; aujourd'hui, les systèmes sur puce regroupent à nouveau des circuits spécialisés (graphisme, IA, chiffrement), pilotés par des API (ch. 14).


1736 Euler

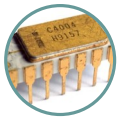

1883 Lucas (Hanoï)


1936 Turing


1945 von Neumann


1957 Bauer


1959 Dijkstra


1965/1975 Loi de Moore


1976 Diffie

Avant les ordinateurs

1736	**Euler** et les ponts de Königsberg : les **graphes** et les chemins eulériens	ch. 8
1883	**Lucas** invente les **tours de Hanoï**, un problème récursif	ch. 1

Les fondations

1936	**Turing** : la **machine universelle** et l'indécidabilité du **problème de l'arrêt**	ch. 13
1945	**von Neumann** imagine le **tri fusion**	ch. 7
1948	Premier **programme enregistré** (Manchester) : le logiciel remplace le recâblage	ch. 14
années 1950	**Bellman** fonde la **programmation dynamique**	ch. 11
1957	**Samelson** et **Bauer** formalisent le principe de la **pile**	ch. 3
1959	**Dijkstra** publie son algorithme du **plus court chemin**	ch. 8
1960	L'**arbre binaire de recherche** est formalisé	ch. 5
1960–1965	**Baran** et **Davies** : la **commutation de paquets**	ch. 9

L'informatique moderne

1965	**Moore** : les transistors d'une puce doublent chaque année (puis tous les 2 ans)	ch. 14
1965	**Dijkstra** décrit l'**interblocage** entre processus	ch. 4
1967	Le langage Simula introduit les **classes** et les **objets**	ch. 2
1969	**ARPANET**, l'ancêtre d'**Internet**, relie ses premiers ordinateurs	ch. 9
1970	**Codd** invente le **modèle relationnel** des bases de données	ch. 6
1977	**RSA** : la cryptographie à **clé publique** (Diffie et Hellman, 1976)	ch. 10
1980	**Horspool** simplifie **Boyer-Moore** (1977) : chercher un motif en lisant à l'envers	ch. 12

PREMIÈRE PARTIE

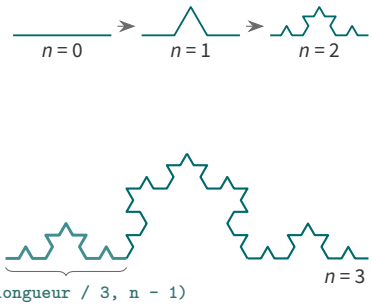
Cours et exercices

1

CHAPITRE 1

Récursivité

```
def koch(longueur, n):
    if n == 0:
        t.forward(longueur)
    else:
        koch(longueur / 3, n - 1)
        t.left(60)
        koch(longueur / 3, n - 1)
        t.right(120)
        koch(longueur / 3, n - 1)
        t.left(60)
        koch(longueur / 3, n - 1)
```



DANS CE CHAPITRE

- Activités préparatoires 18
- Cours 20
- Exercices 32
- TP et projets 48
- Fiche de révision 58
- Défi Advent of Code 60
- Corrigés 756

1

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Des appels qui s'empilent

▶ corrigé p. 756

des post-it, des poupées russes et un miroir sans fin

Durée : 25 min · Sans ordinateur, par deux

Consignes

- Matériel : une dizaine de post-it (ou de petits papiers). Les réponses s'écrivent sur le cahier.
- Règle du jeu : l'un lit le programme ligne à ligne et annonce les affichages ; l'autre gère la **pile de post-it**. À chaque **appel** de fonction, on écrit sur un post-it le nom de la fonction et la ligne où il faudra reprendre, et on le pose **sur le dessus** du tas. Quand une fonction se termine, on **retire le post-it du dessus** et on reprend là où il l'indique.

Exercice 1 *Une fonction en appelle une autre*

```
1  def fctA():
2      print("Debut fctA")
3      for i in range(3):
4          print(f"fctA {i}")
5      print("Fin fctA")
6
7  def fctB():
8      print("Debut fctB")
9      for i in range(3):
10         if i == 2:
11             fctA()
12             print("Retour dans fctB")
13         print(f"fctB {i}")
14     print("Fin fctB")
15
16 fctB()
```

1. Avec les post-it, prévoir l'affichage complet de ce programme (écrire une ligne par affichage).
2. Dessiner la pile de post-it au moment où s'affiche `fctA 1` (le dessus du tas en haut).
3. Pendant que `fctA` s'exécute, que fait `fctB` ? Quand `fctA` se termine, comment sait-on où reprendre ?
4. Les deux fonctions ont une variable `i`. Faut-il noter sur le post-it de `fctB` la valeur de *son* `i` ? Les deux `i` sont-ils la même variable ?

Exercice 2 *Les poupées russes*

Une matriochka contient une poupée plus petite, qui en contient une plus petite… jusqu'à la plus petite, qui ne s'ouvre pas. On numérote les poupées de `1` (la plus petite) à `3` (la plus grande). La fonction ci-dessous décrit comment on les ouvre toutes puis on les referme ; elle **s'appelle elle-même**.

```
1 def ouvrir(n):
2     print("J'ouvre la poupee", n)
3     if n > 1:
4         ouvrir(n - 1)
5     print("Je referme la poupee", n)
6
7 ouvrir(3)
```

5. Avec les post-it (un post-it par appel : `ouvrir(3)`, `ouvrir(2)`…), prévoir l'affichage complet.
6. Combien de post-it y a-t-il, au plus, en même temps sur le tas ? À quel moment ?
7. La ligne 2 s'exécute-t-elle quand on « descend » vers les petites poupées ou quand on « remonte » ? Et la ligne 5 ?
8. Prévoir l'affichage de `ouvrir(1)`, puis dire combien de lignes affiche `ouvrir(10)`.

Exercice 3 *Le miroir sans fin*

On retire le test `if n > 1` : la fonction devient la suivante.

```
1 def boucle():
2     print("un tres mauvais exemple")
3     boucle()
4
5 boucle()
```

9. Jouer quelques étapes avec les post-it. Que devient le tas ? Le jeu peut-il s'arrêter ?
10. Un ordinateur a une mémoire limitée (Python n'accepte qu'environ 1 000 post-it). Que va-t-il se passer selon vous ?
11. Dans `ouvrir`, quelle ligne empêchait ce problème ? Pourquoi la suite d'appels finissait-elle par s'arrêter ?

Exercice 4 *Mettre des mots*

12. Proposer sur le cahier un nom (un mot ou deux) pour chacune des notions rencontrées :
    - le tas de post-it, géré par l'ordinateur ;
    - poser un post-it sur le tas / retirer celui du dessus ;
    - une fonction qui s'appelle elle-même ;
    - le cas où la fonction ne s'appelle plus (la plus petite poupée).
13. Recopier et compléter les phrases suivantes avec vos mots.
    - Quand une fonction en appelle une autre, la première…
    - Les instructions écrites *après* l'appel s'exécutent…
    - Pour qu'une fonction qui s'appelle elle-même s'arrête, il faut…

1

COURS

Programme (BO) *« Écrire un programme récursif. Analyser le fonctionnement d'un programme récursif. Des exemples relevant de domaines variés sont à privilégier. »* La récursivité irrigue toute la fin de l'année : arbres, « diviser pour régner », retour sur trace.

Prérequis fonctions et paramètres, listes, chaînes de caractères, notion de coût. → rappels de 1re : bases de Python p. 1056, listes p. 1057, chaînes p. 1057, parcours et coût p. 1059

Objectifs *savoir écrire* une fonction récursive (méthode) et *savoir l'analyser* (trace, pile, terminaison, coût).

Situation de départ

1. Quand une fonction en appelle une autre, qui en appelle encore une autre, comment l'ordinateur sait-il ensuite où reprendre ?
2. Une poupée russe contient une poupée plus petite, qui en contient une plus petite… Peut-on résoudre un problème en le ramenant au *même* problème en plus petit ? Comment être sûr que cela s'arrête ?
3. Pourquoi un programme de quelques lignes peut-il mettre plusieurs dizaines de secondes à calculer le 40e terme d'une suite ?

I Une fonction peut en appeler une autre

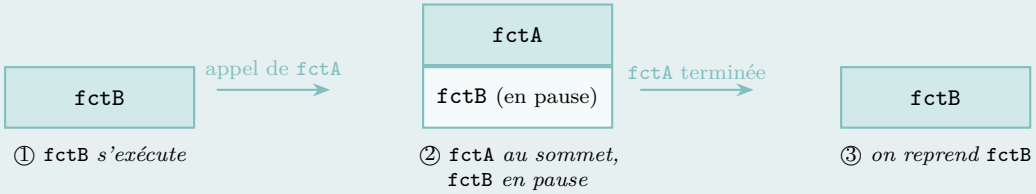
Une fonction peut en appeler une *autre*. Dans le programme ci-dessous, l'appel `fctB()` affiche `Debut fctB`, puis `fctA` (tout l'affichage de `fctA`), puis `Fin fctB` :

```
1  def fctA():
2      print("fctA")
3
4  def fctB():
5      print("Debut fctB")
6      fctA()
7      print("Fin fctB")
```

La chose essentielle à observer : l'exécution de `fctB` est **interrompue** pendant celle de `fctA`, puis **reprend exactement où elle s'était arrêtée**. Pour gérer cela, le système utilise une **pile d'exécution** (la pile, comme structure de données, sera étudiée pour elle-même au chapitre 3).

Définition 1 — Pile d'exécution

La *pile d'exécution* enregistre les fonctions en cours d'exécution. Les appels « s'empilent » les uns sur les autres. La fonction située **au sommet** est celle qui s'exécute ; toutes celles « en dessous » sont **en pause**. Quand une fonction se termine, elle est **dépilée**, et l'exécution reprend dans celle qui est désormais au sommet.



La pile ne retient pas seulement « quelle fonction » : elle conserve aussi **où reprendre** (la prochaine instruction) et la **valeur des variables locales**. Ainsi, si `fctA` et `fctB` utilisent chacune une variable `i`, ces deux variables portent le même nom mais sont **indépendantes** : chaque appel possède ses propres variables.

↻ **Retour sur l'activité.** Le tas de post-it de l'activité était exactement la pile d'exécution : le post-it du dessus est l'appel en cours, ceux du dessous sont en pause, et chacun indiquait la ligne où reprendre. Le `i` de `fctB` et celui de `fctA` étaient bien deux variables distinctes, une par post-it.

Remarque
Rien n'interdit qu'une fonction s'appelle... **elle-même**. C'est tout l'objet de ce chapitre.

II Une fonction qui s'appelle elle-même

Définition 2 — Fonction récursive

Une fonction est dite **récursive** lorsqu'elle s'appelle elle-même dans son propre corps.

Une fonction qui s'appelle elle-même *sans condition* ne s'arrête jamais : chaque appel empile un nouvel appel, **sans jamais rien dépiler**, et la pile grandit jusqu'à ce que Python s'arrête sur une erreur :

```
RecursionError: maximum recursion depth exceeded
```

Pour éviter cela, il faut une **condition d'arrêt**.

Règle 1 — La règle d'or

Toute fonction récursive **doit** contenir une **condition d'arrêt** (aussi appelée **cas de base**) : un cas simple, traité *sans* appel récursif, qui met fin à l'enchaînement des appels. Sans elle, la récursion ne s'arrête jamais.

Définition 3 — Condition d'arrêt et cas récursif

- la **condition d'arrêt** (ou **cas de base**) : le ou les cas où l'on sait répondre *directement* (`if n == 0 : return ...`) ;
- le **cas récursif** : les autres cas, où la fonction s'appelle elle-même sur un problème **plus petit**, qui se rapproche de la condition d'arrêt.

↻ **Retour sur l'activité.** La plus petite poupée russe, qui ne s'ouvre pas, était la condition d'arrêt de `ouvrir` (le test `if n > 1`). Sans elle, le « miroir sans fin » posait des post-it sans jamais en retirer : c'est le `RecursionError`.

Méthode — Les étiquettes d'étapes

Comme en 1re, un commentaire en **majuscules** nomme le **rôle** d'une ligne, avec toujours les mêmes mots : `INITIALISER`, `PARCOURIR`, `TESTER`, `METTRE A JOUR`, `RENVOYER`, `VERIFIER`. La récursivité en ajoute deux : `CAS DE BASE` (la condition d'arrêt, traitée sans appel) et `APPEL RECURSIF` (le même problème, en plus petit) ; et, quand on se sert du résultat de l'appel pour construire la réponse, `COMBINER` (dans `return n * fact(n - 1)`, l'appel et la multiplication par `n` tiennent sur la même ligne).
Avant de coder, on écrit encore, comme en 1re, **trois appels attendus**, en `assert` ou en

1

doctest : un cas **normal**, un cas **limite** (pour une fonction récursive, c'est souvent le cas de base) et un cas **piège**. Pour la factorielle de la section suivante :

```
assert fact(4) == 24    # cas normal
assert fact(0) == 1     # cas limite : le cas de base
assert fact(1) == 1     # cas piege : 1 et non 0, par un appel recursif
```

Exemple — *Prévoyez l'affichage, puis vérifiez à la machine*

▷ **Je prédis.** Qu'affiche `compte(3)` ? Cacher la correction, prévoir, puis vérifier.

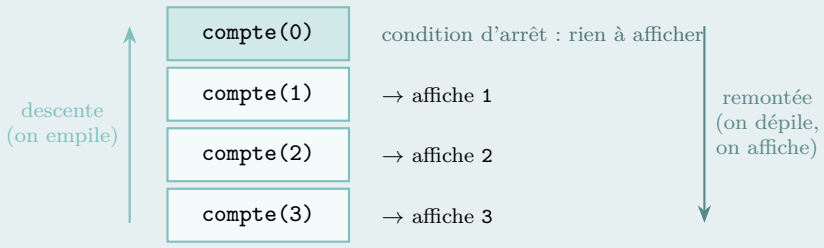
```
def compte(n):
    if n > 0:
        compte(n - 1)       # APPEL RECURSIF
        print(n)            # execute au retour de l'appel
    # CAS DE BASE : n == 0, on ne fait rien

compte(3)
```

▶ **Correction.** Suivons les appels, puis les « dépilements » :

- `compte(3)` : $3 > 0$ → appelle `compte(2)` (le `print` attendra !)
- `compte(2)` : $2 > 0$ → appelle `compte(1)`
- `compte(1)` : $1 > 0$ → appelle `compte(0)`
- `compte(0)` : condition d'arrêt, **on dépile** → retour dans `compte(1)` : affiche `1`
- on dépile → `compte(2)` : affiche `2`
- on dépile → `compte(3)` : affiche `3`

L'affichage est donc `1 2 3`. C'est contre-intuitif : le `print(n)` est *écrit après* l'appel récursif, il ne s'exécute donc qu'**au retour**, dans l'ordre inverse des appels.



Règle 2 — Avant ou après l'appel ?

Le code placé **avant** l'appel récursif s'exécute « à la descente » (dans l'ordre des appels) ; le code placé **après** s'exécute « à la remontée » (dans l'ordre inverse). Déplacer une ligne d'un côté à l'autre change complètement le résultat.

↻ **Retour sur l'activité.** Dans les poupées russes, « J'ouvre la poupee » était écrit avant l'appel et s'affichait à la descente (3, 2, 1), « Je referme la poupee » était écrit après et s'affichait à la remontée (1, 2, 3).

▶ **Exercices d'application :** exercices 1 à 4 (comprendre et analyser une récursion)

III Écrire une fonction récursive : une méthode

La récursivité prolonge une idée mathématique bien connue : le **raisonnement par récurrence**, où l'on définit un terme à partir du ou des précédents.

Règle 3 — Les trois questions

Pour écrire une fonction récursive, on se pose toujours :

1. **Quelle est la condition d'arrêt ?** (le cas de base : le cas le plus simple, réponse immédiate)
2. **Comment se rapprocher de la condition d'arrêt ?** (l'appel récursif porte sur un problème *strictement plus petit*)
3. **Comment combiner** le résultat de l'appel récursif pour obtenir la réponse ?

A. Exemple fil rouge : la factorielle

En mathématiques, $n! = n \times (n-1) \times \cdots \times 2 \times 1$, avec par convention $0! = 1$. On remarque que $n! = n \times (n-1)!$. Cette définition *est déjà récursive* :

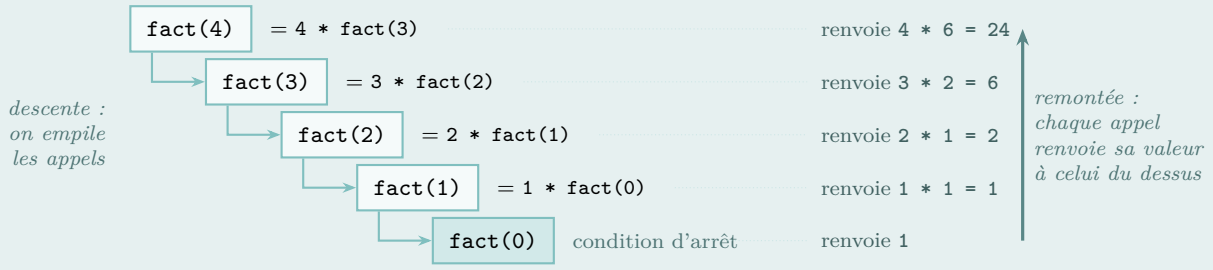
- condition d'arrêt : $0! = 1$;
- cas récursif : $n! = n \times (n-1)!$.

```
1  def fact(n):
2      if n == 0:                  # CAS DE BASE (question 1)
3          return 1
4      else:
5          return n * fact(n - 1)   # APPEL RECURSIF (2), COMBINER : x n (3)
```

▷ **Je prédis.** Que renvoie `fact(4)` ? Combien d'appels de `fact` s'empilent ?

▷ **Je vérifie.** 24 ; **cinq** appels, de `fact(4)` jusqu'au cas de base `fact(0)` (schéma ci-dessous).

La fonction **recopie mot pour mot** la définition mathématique : c'est la grande force de la récursivité.



B. Des exemples de domaines variés

Le programme demande explicitement des exemples *variés*. En voici quatre, tous bâtis sur les trois mêmes questions.

Exemple — *Somme des entiers de 1 à n*

```
1  def somme(n):
2      if n == 0:                  # CAS DE BASE
3          return 0
4      else:
5          return n + somme(n - 1)  # APPEL RECURSIF, COMBINER : + n
```

Condition d'arrêt : `somme(0) = 0`. Cas récursif : `somme(n) = n + somme(n-1)`.

1

▷ **Je prédis.** Que renvoient `somme(4)` et `somme(-1)` ?
▷ **Je vérifie.** `10` ; `somme(-1)` ne s'arrête pas : `n` vaut `-2`, `-3`... sans jamais atteindre `0`, d'où un `RecursionError`. Le cas de base doit être *atteint* (voir la terminaison, plus bas).

Exemple — *Somme des éléments d'une liste — récursion « sur les données »*
Ici, « plus petit » signifie **une liste plus courte**.

```
def somme_liste(tab):
    if tab == []:                          # CAS DE BASE : liste vide
        return 0
    else:
        return tab[0] + somme_liste(tab[1:])   # APPEL RECURSIF, COMBINER
```

▷ **Je prédis.** Que renvoie `somme_liste([4, -1, 2])` ? Combien d'appels ?
▷ **Je vérifie.** `5`, en **quatre** appels : sur `[4, -1, 2]`, `[-1, 2]`, `[2]`, puis `[]` (le cas de base).

Exemple — *Puissance x^n*

```
def puissance(x, n):
    if n == 0:                          # CAS DE BASE
        return 1
    else:
        return x * puissance(x, n - 1)  # APPEL RECURSIF, COMBINER : fois x
```

▷ **Je prédis.** Que renvoient `puissance(-2, 3)` et `puissance(0, 0)` ?
▷ **Je vérifie.** `-8` ; puis `1` : on tombe directement sur le cas de base, quel que soit `x`.

Exemple — *La légende de l'échiquier*
Sur la 1re case, on pose 1 grain de riz ; sur chaque case suivante, on *double*. Combien de grains sur la case n ?

```
def grains(case):
    if case == 1:                     # CAS DE BASE : premiere case
        return 1
    else:
        return 2 * grains(case - 1)   # APPEL RECURSIF, COMBINER : on double
```

Un problème « de la vie courante » qui donne pourtant une croissance vertigineuse : `grains(64)` dépasse 9×10^{18}.

Exemple — *La suite de Fibonacci*
La suite de **Fibonacci** est définie par $u_0 = 0$, $u_1 = 1$ et $u_n = u_{n-1} + u_{n-2}$. Écrire une fonction récursive `fib(n)` qui renvoie u_n.

▶ **Correction.** Il y a ici **deux** conditions d'arrêt ($n = 0$ et $n = 1$) et **deux** appels récursifs :

```
def fib(n):
    if n == 0:                          # CAS DE BASE (1er)
        return 0
    elif n == 1:                        # CAS DE BASE (2e)
        return 1
    else:
        return fib(n - 1) + fib(n - 2)  # APPEL RECURSIF x 2, COMBINER
```

▷ **Je prédis.** Que renvoie `fib(6)` ?
On obtient `0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55` pour `n` allant de 0 à 10.

C. Un outil Python : le paramètre par défaut

Une fonction récursive a souvent besoin d'un paramètre **de plus** que ce que l'utilisateur veut fournir : un **accumulateur**, qui transporte d'un appel à l'autre le résultat en cours de construction. Python permet de donner à ce paramètre une **valeur par défaut**, écrite dans l'en-tête avec `=` : elle est utilisée quand l'appel ne fournit pas ce paramètre.

```
1 def somme_acc(n, total=0):
2     """Renvoie total + 1 + 2 + ... + n."""
3     if n == 0:                              # CAS DE BASE
4         return total
5     else:
6         return somme_acc(n - 1, total + n)    # APPEL RECURSIF
```

L'utilisateur appelle `somme_acc(4)`, qui renvoie `10` : `total` vaut alors `0`, sans qu'il ait à le savoir. Les appels récursifs, eux, fournissent `total` : `somme_acc(3, 4)`, puis `somme_acc(2, 7)`, etc. Les paramètres qui ont une valeur par défaut s'écrivent **après** les autres.

▷ **Je prédis.** Que renvoie `somme_acc(2, 10)` ?

▷ **Je vérifie.** `13`, soit `10 + 2 + 1` : quand l'appel fournit `total`, la valeur par défaut n'est pas utilisée.

Règle 4 — Piège : une valeur par défaut modifiable

La valeur par défaut est créée **une seule fois**, quand Python lit le `def`, et non à chaque appel. Pour un nombre ou une chaîne, qu'on ne modifie jamais, c'est sans danger. Pour une **liste**, c'est un piège : la même liste sert à tous les appels qui ne la fournissent pas.

```
1 def a_rebours(n, vus=[]):             # A EVITER
2     """Renvoie la liste [n, n - 1, ..., 1]."""
3     if n == 0:                        # CAS DE BASE
4         return vus
5     else:
6         vus.append(n)                 # METTRE A JOUR
7         return a_rebours(n - 1, vus)  # APPEL RECURSIF
```

▷ **Je prédis.** Que renvoient `a_rebours(3)`, puis l'appel suivant `a_rebours(2)` ?

`a_rebours(3)` renvoie bien `[3, 2, 1]`, mais l'appel suivant `a_rebours(2)` renvoie `[3, 2, 1, 2, 1]` : la liste a gardé le résultat du premier appel. La bonne pratique : mettre `None` par défaut, et créer une liste neuve dans la fonction.

```
1 def a_rebours(n, vus=None):
2     """Renvoie la liste [n, n - 1, ..., 1]."""
3     if vus is None:
4         vus = []                      # INITIALISER : liste neuve a chaque appel de
                                        # l'utilisateur
5     if n == 0:                        # CAS DE BASE
6         return vus
7     else:
8         vus.append(n)                 # METTRE A JOUR
9         return a_rebours(n - 1, vus)  # APPEL RECURSIF
```

Cette fois, `a_rebours(2)` renvoie `[2, 1]`. On retrouvera ce schéma au chapitre *Graphes*, dans le parcours en profondeur récursif.

► **Exercices d'application :** exercices 5 à 23 (écrire des fonctions récursives : nombres, chaînes, listes, dénombrement)

1

IV Analyser une fonction récursive

Savoir écrire ne suffit pas : le programme demande de **savoir analyser**. Trois questions reviennent : *est-ce que ça s'arrête ?*, *est-ce que c'est juste ?*, *combien ça coûte ?*

A. La terminaison

Définition 4 — Variant

Un **variant** est une quantité entière **positive** qui **décroît strictement** à chaque appel récursif. S'il en existe un, la suite d'appels ne peut pas être infinie : elle atteint forcément la condition d'arrêt.

Propriété 1 — Terminaison de `somme`

Pour tout entier $n \geqslant 0$, l'appel `somme(n)` (défini plus haut) se termine.

Démonstration :
Prenons pour **variant** l'entier n passé en paramètre, et montrons qu'il remplit les trois conditions.

- **Il est positif.** On appelle `somme` au départ avec $n \geqslant 0$; comme on ne fait un appel récursif que dans le cas $n \neq 0$ (donc $n \geqslant 1$), l'appel suivant se fait avec $n - 1 \geqslant 0$. Le paramètre reste donc toujours $\geqslant 0$.
- **Il décroît strictement.** Le seul appel récursif est `somme(n-1)` : à chaque étape, le paramètre diminue exactement de 1, donc $n - 1 < n$.
- **Il atteint la condition d'arrêt.** Une suite d'entiers positifs qui décroît strictement ne peut pas être infinie. Partant de n, après exactement n appels on obtient le paramètre 0 : c'est la **condition d'arrêt**, qui renvoie 0 *sans* appel récursif et arrête la descente.

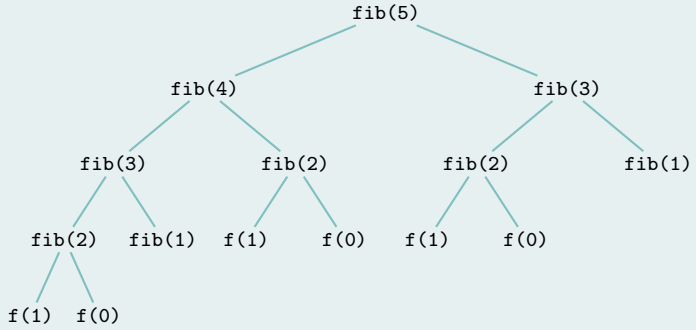
La suite des appels est donc finie : l'appel `somme(n)` se termine. □

Au bac — justifier la terminaison

On retrouve exactement le raisonnement attendu au bac : *« décrire ce qui garantit la terminaison de la fonction récursive »* (sujet Métropole 2025, jour 2 : la longueur du tableau traité diminue à chaque appel). Attention aussi à la condition d'arrêt mal choisie : `fact` écrite avec `if n == 1` au lieu de `if n == 0` *boucle* sur `fact(-1)`, `fact(-2)`... si on l'appelle sur `fact(0)`. La condition d'arrêt doit être **réellement atteinte** par le variant.

B. Le coût : l'exemple frappant de Fibonacci

La fonction `fib` écrite plus haut est correcte, mais **terriblement lente**. La raison : chaque appel déclenche **deux** appels, et les mêmes calculs sont refaits un très grand nombre de fois. Dessinons l'**arbre des appels** de `fib(5)` :



On voit que `fib(2)` est recalculé **trois** fois, `fib(3)` deux fois… Le nombre d'appels est multiplié par environ 1,6 quand n augmente de 1 (il fait plus que **doubler** quand n augmente de 2) : le coût est **exponentiel**.

Remarque

Concrètement : `fib(40)` demande plus de 330 millions d'appels (331 160 281 exactement) et met plusieurs *dizaines de secondes*, là où une version itérative (avec une boucle) répond en quelques *microsecondes*. Retenez cet ordre de grandeur : une fonction récursive peut être élégante **et** inutilisable.

Une manière efficace de calculer u_n malgré tout sera étudiée dans le chapitre de **programmation dynamique** (chapitre 11).

V Récursif ou itératif ?

Toute fonction récursive peut se réécrire de façon **itérative** (avec des boucles), et réciproquement. Le choix se fait selon la **lisibilité** et le **coût**.

Exemple — *Un grand classique du bac : le palindrome*

Écrire une fonction qui teste si une chaîne est un palindrome (mot lu pareil dans les deux sens, comme « radar »), d'abord en récursif, puis en itératif.

▶ **Correction.** **Version récursive** (on « épluche » la chaîne par les deux bouts) :

```
def tester_palindrome(chaine):
    if len(chaine) < 2:              # CAS DE BASE : 0 ou 1 caractere
        return True
    elif chaine[0] != chaine[-1]:    # TESTER : bords differents
        return False                 # RENVOYER : pas un palindrome
    else:
        return tester_palindrome(chaine[1:-1])   # APPEL RECURSIF : sans les bords
```

Version itérative (deux indices qui se rapprochent) :

```
def est_palindrome(chaine):
    i = 0                            # INITIALISER : indice de gauche
    j = len(chaine) - 1              # INITIALISER : indice de droite
    while i < j:                     # PARCOURIR
        if chaine[i] != chaine[j]:   # TESTER
            return False             # RENVOYER
        i = i + 1                    # METTRE A JOUR
        j = j - 1                    # METTRE A JOUR
    return True                      # RENVOYER
```

Les deux renvoient `True` pour `"radar"` et `False` pour `"rader"`.

▷ **Je prédis.** Que renvoient-elles pour `"abca"` ? Et pour la chaîne vide `""` ?

▷ **Je vérifie.** `False` : les bords `a` sont égaux, mais pas `b` et `c`. Puis `True` : la chaîne vide est le cas de base de la version récursive, et la boucle de la version itérative ne fait aucun tour.

	Récursif	**Itératif**
Lisibilité	colle à la définition « les deux bords + le reste »	suit un balayage classique
Mémoire	empile un appel par caractère	un seul cadre, deux indices
Terminaison	la chaîne raccourcit (variant)	`j - i` décroît

Au bac — les limites d'une fonction

Le bac aime aussi les **limites** : `tester_palindrome(53235)` *échoue*, car `len` ne s'applique pas à un entier. Une fonction n'est correcte que pour le **type de données** qu'elle attend.

Règle 5 — Comment choisir ?

- la version **récursive** gagne quand le problème est *lui-même récursif* (arbres, fractales, diviser pour régner) : le code est alors bien plus court et clair ;
- la version **itérative** gagne quand la récursion coûterait cher en appels répétés (Fibonacci naïf) ou en profondeur de pile.

▶ **Exercices d'application :** exercice 24 (récursif ou itératif ?)

VI La récursivité dans les dessins (Turtle)

La récursivité produit naturellement les **fractales** : des figures qui se répètent à toutes les échelles.

Exemple — *Testez, puis expliquez le rôle des paramètres* `longueur` *et* `n`

```
1  import turtle as t
2
3  def koch(longueur, n):
4      if n == 0:                  # CAS DE BASE : un simple segment
5          t.forward(longueur)
6      else:                       # APPEL RECURSIF x 4 : courbes 3 fois plus petites
7          koch(longueur / 3, n - 1)
8          t.left(60)
9          koch(longueur / 3, n - 1)
10         t.right(120)
11         koch(longueur / 3, n - 1)
12         t.left(60)
13         koch(longueur / 3, n - 1)
```

▶ **Correction.** `longueur` est la longueur du segment à tracer ; à chaque niveau, on la divise par 3 pour tracer quatre segments trois fois plus petits. `n` est le **niveau de détail** : pour `n = 0`, on trace un simple segment ; sinon, on remplace le segment par quatre courbes de niveau `n - 1`.



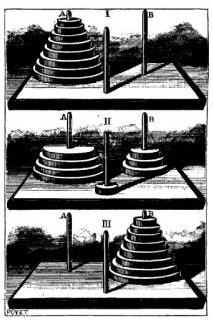
La courbe de Koch pour `n = 0, 1, 2, 3` *: chaque segment est remplacé par quatre segments trois fois plus petits.*

Le paramètre `n` est ici le **variant** : il décroît d'un cran à chaque niveau et garantit l'arrêt lorsqu'il atteint la condition d'arrêt `n == 0`.

► **Exercices d'application :** exercices 25 à 28 (dessins récursifs en Turtle)

Pour les curieux — *Un peu d'histoire* (facultatif)

En **1883**, le mathématicien français **Édouard Lucas** publie, sous le pseudonyme de « N. Claus de Siam » (anagramme de « Lucas d'Amiens »), le casse-tête des **tours de Hanoï** : déplacer une pile de disques d'un piquet à un autre, un disque à la fois, sans jamais poser un disque sur un plus petit. La solution est récursive par nature : pour déplacer n disques, on déplace les $n-1$ du dessus sur le piquet intermédiaire, puis le grand disque, puis à nouveau les $n-1$ disques par-dessus — soit $2^n - 1$ déplacements. La récursivité n'entre dans les langages de programmation qu'en **1958–1960**, avec **Lisp**, créé par **John McCarthy** (qui a aussi forgé l'expression « intelligence artificielle ») : en Lisp, presque tout s'écrit par des fonctions récursives. Algol 60 l'adopte aussitôt, puis presque tous les langages, dont Python.



Les tours de Hanoï (É. Lucas, 1883)



John McCarthy en 2006

Au bac — on vous demandera surtout de…

- **justifier** qu'une fonction est récursive, et pourquoi elle **se termine** (variant) ;
- **dérouler** une trace d'exécution / compter les appels ;
- **compléter** l'appel récursif dans une fonction donnée ;
- comparer une version **récursive** et une version **itérative**.

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise :** les fonctions, les listes et les chaînes, vues dans *Les types construits* (Première), et le raisonnement par récurrence des mathématiques.
- **Ce qui en découle :** la pile d'exécution deviendra la structure de **pile** (*Structures linéaires : piles, files, listes chaînées*) ; vous écrirez récursivement les algorithmes sur les *Arbres*, le tri fusion de *Diviser pour régner* et le parcours en profondeur des *Graphes*.
- **Pour aller plus loin :** la *Programmation dynamique* évitera les appels répétés de Fibonacci ; et vous verrez en *Calculabilité et décidabilité* qu'aucun programme ne peut décider si *tout* programme s'arrête.

Bilan — la carte mémoire

Notion	À retenir
Fonction récursive	elle s'appelle elle-même
Condition d'arrêt (cas de base)	réponse immédiate, **obligatoire**, sinon pas d'arrêt
Cas récursif	appel sur un problème **strictement plus petit**
Pile d'exécution	empile un cadre par appel ; « au sommet » = en cours
Terminaison	assurée par un **variant** (entier $\geqslant 0$ qui décroît)
Coût	attention aux **appels répétés** (Fibonacci : exponentiel)
Récursif vs itératif	récursif = lisible sur les structures récursives

Retour à la situation de départ

1. Grâce à la **pile d'exécution** : chaque appel y est empilé avec l'endroit où reprendre et ses variables locales ; quand il se termine, on le dépile et l'on reprend dans la fonction du sommet (§ I).
2. Oui : c'est une **fonction récursive**, qui s'appelle sur un problème strictement plus petit (§ II et III). Elle s'arrête si sa **condition d'arrêt** est réellement atteinte, ce que l'on justifie par un **variant** (§ IV.A).
3. La fonction `fib` naïve fait deux appels à chaque étape et recalcule sans cesse les mêmes valeurs : plus de 330 millions d'appels pour `fib(40)`, un coût **exponentiel** (§ IV.B).

Erreurs fréquentes

- **Oublier la condition d'arrêt.** La fonction s'appelle indéfiniment (`RecursionError : maximum recursion depth`). *Le réflexe :* écrire le **cas de base en premier**.
- **Une condition d'arrêt jamais atteinte.** L'appel récursif ne *rapproche* pas du cas de base (mauvais sens, argument qui ne décroît pas) : même effet qu'un oubli. *Le réflexe :* vérifier qu'un **variant** décroît strictement.
- **Oublier le `return` devant l'appel récursif.** Le résultat est calculé mais jamais renvoyé, la fonction renvoie `None`. *Le réflexe :* `return n * fact(n-1)`, et non `fact(n-1)` seul.
- **Ne pas combiner le résultat de l'appel.** On oublie l'opération qui relie le sous-problème au problème (le `n *`, le `+ 1`, la concaténation…).
- **Croire que « chaque appel repart de zéro ».** Les appels s'**empilent** ; chaque cadre possède ses **propres** variables locales. Dérouler la pile lève le doute.
- **La récursivité naïve qui recalcule.** `fib(n)` relance deux fois les mêmes calculs ⇒ coût **exponentiel**. *Le réflexe :* repérer les **appels répétés** (on apprendra à les éviter au chapitre *Programmation dynamique*).

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

- ☐ identifier le **cas de base** et le **cas récursif** d'une fonction donnée → ex. 1, 3 ;
- ☐ écrire une fonction récursive sur les **entiers** (factorielle, somme, puissance, PGCD, produit par additions) → ex. 6, 7, 10 ;
- ☐ écrire une fonction récursive sur une **chaîne** ou un **tableau** (compter, inverser, somme / maximum, recherche) → ex. 14, 15, 17 ;
- ☐ **dérouler** la trace des appels (pile d'exécution) et donner le résultat → ex. 1, 2 ;
- ☐ **compter** le nombre d'appels effectués par une fonction récursive → ex. 4, 33 ;

☐ **justifier la terminaison** en exhibant un **variant** → ex. 2, 33 ;
☐ passer d'une version **récursive** à une version **itérative** (et inversement) → ex. 24, 29 ;
☐ reconnaître le **coût** d'une récursion et le piège des **appels répétés** → ex. 4, 30.

Vers le Grand Oral

Quelques pistes de questions que ce chapitre permet de préparer (à étayer d'un exemple concret et, si possible, d'un schéma ou d'une petite démonstration) :

- **Une fonction qui s'appelle elle-même : comment peut-elle s'arrêter ?** *(cas de base, variant, pile d'appels ; plus largement, la terminaison d'un algorithme.)*
- **Récursif ou itératif : faut-il choisir ?** *(lisibilité contre coût mémoire de la pile ; le cas de Fibonacci, des appels répétés à la mémoïsation.)*
- **Où se cache la récursivité en dehors des mathématiques ?** *(fractales, arborescence de fichiers, tri fusion, retour sur trace du Sudoku, structure arborescente d'une page Web.)*
- **Quels sont les avantages et les limites du récursif en informatique ?** *(le catastrophique : Fibonacci naïf, exponentiel pour rien ; le nécessaire : les tours de Hanoï, coûteuses mais sans solution plus simple ; le génial : le tri fusion, $n \log_2 n$ au lieu de n^2.)*

1

EXERCICES

Mode d'emploi

- Niveau : ★ échauffement ★★ classique ★★★ défi (dont les extraits de bac les plus exigeants) ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- ▶ *À programmer et tester* en Python (console ou éditeur en ligne).
- **Réflexe** pour chaque fonction : **(1)** quelle est la **condition d'arrêt** (cas de base) ? **(2)** comment se **rapprocher** de la condition d'arrêt ? **(3)** comment **combiner** le résultat de l'appel récursif ?
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

Remarque Le **retour sur trace** (*backtracking*) et le **Sudoku** sont traités dans le **projet Sudoku** proposé en fin de chapitre : ils ne sont pas repris ici.

Comprendre et analyser ↑ cours

*Ici on ne programme pas (ou peu) : on **lit**, on **déroule** et on **justifie**. C'est la moitié « analyser » du programme.*

★ Exercice 1 *Avant ou après l'appel ?* ▶ corrigé p. 758 • essentiel

Sans machine, donner ce qu'affichent `a(3)` puis `b(3)`. Expliquer la différence. ▷ coup de pouce

```
1  def a(n):
2      if n > 0:
3          print(n)
4          a(n - 1)
5
6  def b(n):
7      if n > 0:
8          b(n - 1)
9          print(n)
```

★★ Exercice 2 *Dérouler une trace* ▶ corrigé p. 758 • essentiel

On donne la fonction ci-dessous.

```
1  def mystere(n):
2      if n > 1:
3          mystere(n // 2)
4      print(n)
```

1. Donner, dans l'ordre, ce qu'affiche l'appel `mystere(20)`. ▷ coup de pouce
2. Que calcule, en une phrase, la fonction `mystere` ?
3. Proposer un **variant** et justifier que la fonction **se termine** toujours.

★★ **Exercice 3** *Cherchez l'erreur* ▶ corrigé p. 758 • essentiel

Chacun de ces deux programmes est **fautif**. Expliquer précisément *pourquoi*, puis **corriger**.

▷ coup de pouce

```python
# (A) somme des entiers de 0 a n
def somme(n):
    return n + somme(n - 1)

# (B) factorielle
def fact(n):
    if n == 1:
        return 1
    else:
        return n * fact(n - 1)
```

★★ **Exercice 4** *Compter les appels* ▶ corrigé p. 758 • essentiel

On rappelle la version récursive naïve de Fibonacci ($u_0 = 0$, $u_1 = 1$, $u_n = u_{n-1} + u_{n-2}$).

```python
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

1. Dessiner l'**arbre des appels** de `fib(4)`.
2. Combien d'appels à `fib` l'appel `fib(4)` déclenche-t-il en tout ? ▷ coup de pouce
3. Expliquer, en une phrase, pourquoi ce coût devient vite **inutilisable** quand n grandit.
4. Nommer la méthode algorithmique qui permet d'éviter ces calculs répétés.

Écrire des fonctions récursives — les nombres ↑ cours

★ **Exercice 5** *Le marchand de sable* ▶ corrigé p. 759

▶ Écrire une fonction récursive `moutons(n)` qui affiche `"Encore un mouton"` n fois, puis `"Je dors !"`. Tester avec `n = 5`. ▷ coup de pouce

★ **Exercice 6** *Somme des nombres impairs* ▶ corrigé p. 759 • essentiel

▶ Écrire une fonction récursive `somme_impairs(n)` qui renvoie la somme des n premiers nombres impairs, $1 + 3 + 5 + \cdots + (2n - 1)$ (cette somme vaut 0 pour $n = 0$). Exemple : `somme_impairs(3)` renvoie `9`. Afficher ensuite `somme_impairs(n)` pour `n` allant de 1 à 6 : que remarque-t-on ?

▷ coup de pouce

★ **Exercice 7** *Multiplier avec des additions* ▶ corrigé p. 759

▶ Écrire une fonction récursive `produit(a, n)` qui renvoie $a \times n$, pour un entier `n` positif ou nul, **sans** utiliser l'opérateur `*` : uniquement des additions. Exemple : `produit(7, 6)` renvoie `42`. Combien d'appels (appel initial compris) l'appel `produit(7, 6)` effectue-t-il ? Pour calculer 3×1000, vaut-il mieux appeler `produit(3, 1000)` ou `produit(1000, 3)` ? ▷ coup de pouce

★★ **Exercice 8** *Compter les chiffres* ▶ corrigé p. 760

▶ Écrire une fonction récursive `nombre_de_chiffres(n)` qui renvoie le nombre de chiffres d'un entier `n` strictement positif. Exemple : `nombre_de_chiffres(34126)` renvoie `5`. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1

★★ **Exercice 9** *Compter les bits à 1* ▶ corrigé p. 760

▶ En s'inspirant de l'exercice précédent, écrire `nombre_de_bits(n)` qui renvoie le nombre de `1` dans l'écriture binaire d'un entier `n` positif ou nul. Exemples : `nombre_de_bits(255)` renvoie `8` et `nombre_de_bits(10)` renvoie `2` (car $10 = 1010_2$). ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★ **Exercice 10** *PGCD (algorithme d'Euclide)* ▶ corrigé p. 760 • essentiel

On rappelle que $\text{pgcd}(a, b) = \text{pgcd}(b, a \,\%\, b)$ et que $\text{pgcd}(a, 0) = a$.

▶ Écrire une fonction récursive `pgcd(a, b)` qui renvoie le PGCD de deux entiers positifs. Exemple : `pgcd(24, 18)` renvoie `6`. ▷ coup de pouce

★ **Exercice 11** *Une suite récurrente double* ▶ corrigé p. 761

Soit la suite (u_n) définie par

$$u_n = \begin{cases} a & \text{si } n = 0 \\ b & \text{si } n = 1 \\ 3\,u_{n-1} + 2\,u_{n-2} + 5 & \text{pour tout } n \geqslant 2 \end{cases}$$

▶ Écrire une fonction récursive `suite(n, a, b)` qui renvoie le terme u_n, pour des premiers termes `a` et `b` donnés en paramètres. ▷ coup de pouce

★★ **Exercice 12** *Suite de Syracuse* ▶ corrigé p. 761

À partir d'un entier $u_0 > 1$, on pose $u_{n+1} = u_n/2$ si u_n est pair, et $u_{n+1} = 3\,u_n + 1$ sinon.

▶ Écrire une fonction récursive `syracuse(u)` qui affiche les valeurs successives de la suite, de u_0 jusqu'à la première valeur égale à 1 (incluse). Exemple : `syracuse(6)` affiche `6 3 10 5 16 8 4 2 1` (une valeur par ligne). ▷ coup de pouce *La conjecture de Syracuse affirme qu'on atteint toujours 1 : elle défie encore les mathématiciens.*

★★★ **Exercice 13** *Exponentiation rapide* ▶ corrigé p. 761 ≈ 15 min

On peut calculer x^n bien plus vite en remarquant que $x^n = \left(x^{n/2}\right)^2$ si n est pair, et $x^n = x \times x^{n-1}$ si n est impair.

▶ Écrire une fonction récursive `puissance_rapide(x, n)` suivant cette idée. C'est un exemple de la méthode « **diviser pour régner** », qu'on étudiera au chapitre 7 : le nombre de multiplications passe de n à environ $\log_2 n$. ▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Chaînes de caractères et listes ↑ cours

★ **Exercice 14** *Compter une lettre* ▶ corrigé p. 761 • essentiel

▶ Écrire une fonction récursive `compter(s, c)` qui renvoie le nombre d'occurrences du caractère `c` dans la chaîne `s`. Exemple : `compter("anticonstitutionnellement", "t")` renvoie `5`.

▷ coup de pouce

★ **Exercice 15** *Inverser une chaîne* ▶ corrigé p. 762

▶ Écrire une fonction récursive `inverser(s)` qui renvoie la chaîne `s` lue à l'envers. Exemple : `inverser("récursif")` renvoie `"fisrucér"`. ▷ coup de pouce

★★ **Exercice 16** *Palindrome (contexte ADN)* ▶ corrigé p. 762

Dans cet exercice, une chaîne (par exemple un brin d'ADN écrit avec les lettres A, C, G, T) est un *palindrome* si elle se lit pareil dans les deux sens. *(En biologie, le mot a un sens un peu différent : une séquence « palindromique » est égale à son brin complémentaire lu à l'envers.)*

▶ Écrire une fonction récursive `palindrome(s)` qui renvoie `True` si la chaîne `s` est un palindrome, `False` sinon. Exemples : `palindrome("kayak")` renvoie `True`, `palindrome("GATTACA")` renvoie `False` ; tester aussi la chaîne `"GATTACATTAG"`. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 17** *Somme et maximum d'une liste* ▶ corrigé p. 762 • essentiel

▶ Écrire deux fonctions récursives sur une liste `tab` **non vide** :

a. `somme_liste(tab)` qui renvoie la somme des éléments ;
b. `maximum(tab)` qui renvoie le plus grand élément. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 18** *Recherche dans un tableau* ▶ corrigé p. 763

▶ Écrire une fonction récursive `appartient(v, t, i)` qui renvoie `True` si la valeur `v` apparaît dans le tableau `t` à partir de l'indice `i` (inclus), et `False` sinon. On suppose $0 \leqslant$ `i` $\leqslant$ `len(t)`. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 19** *Recherche dichotomique récursive* ▶ corrigé p. 763 ≈ 15 min

On dispose d'un tableau `t` **trié** par ordre croissant.

▶ Écrire une fonction récursive `dicho(t, v, gauche, droite)` qui renvoie un indice où `v` apparaît entre les indices `gauche` et `droite` **inclus**, ou `-1` si `v` n'y est pas. Appel initial : `dicho(t, v, 0, len(t) - 1)`. À chaque appel, on compare `v` à l'élément **du milieu** et on ne garde qu'une moitié : encore un « diviser pour régner » (coût $\log_2 n$). ▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Dénombrement et combinatoire ↑ cours

★★ **Exercice 20** *Mots de A et B* ▶ corrigé p. 763 • essentiel

▶ Écrire une fonction récursive `frises(n)` qui renvoie la liste de toutes les chaînes de longueur `n` composées des lettres `A` et `B`. Exemple : `frises(2)` renvoie `['AA', 'AB', 'BA', 'BB']`. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 21** *Décompositions en 1 et 2* ▶ corrigé p. 764

▶ Écrire une fonction récursive `decoupe(n)` qui affiche toutes les façons d'écrire `n` comme une somme ordonnée de `1` et de `2`. Exemple : `decoupe(3)` affiche, une décomposition par ligne : `1 + 1 + 1`, `1 + 2`, `2 + 1`. ▷ coup de pouce

a. Combien de décompositions `decoupe(n)` affiche-t-elle ? Quelle suite reconnaissez-vous ?
b. Pourquoi la valeur par défaut `""` est-elle sans danger ici, alors qu'une valeur par défaut `[]` serait un piège ?

1

★★ **Exercice 22** *Triangle de Pascal* ▶ corrigé p. 764

Le triangle de Pascal donne les coefficients binomiaux, définis récursivement par

$$\binom{n}{p} = \begin{cases} 1 & \text{si } p = 0 \text{ ou } n = p, \\ \binom{n-1}{p-1} + \binom{n-1}{p} & \text{sinon.} \end{cases}$$

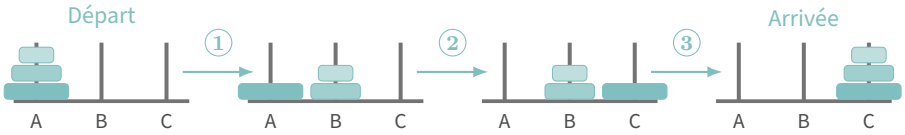
▶ Écrire une fonction récursive `pascal(n, p)` qui renvoie $\binom{n}{p}$. *(Défi : afficher les premières lignes du triangle avec une boucle.)* ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 23** *Les tours de Hanoï* ▶ corrigé p. 764 ≈ 20 min

On veut déplacer une pile de `n` disques d'un piquet `depart` vers un piquet `arrivee`, en s'aidant d'un piquet `inter`, sans jamais poser un grand disque sur un plus petit. L'idée récursive :

- déplacer les `n-1` disques du haut de `depart` vers `inter` ;
- déplacer le grand disque de `depart` vers `arrivee` ;
- déplacer les `n-1` disques de `inter` vers `arrivee`.



① déplacer les $n-1$ disques du haut (**appel récursif**) A → B ② déplacer le grand disque A → C ③ déplacer les $n-1$ disques (**appel récursif**) B → C

▶ Écrire une fonction récursive `hanoi(n, depart, inter, arrivee)` qui **affiche** la suite des déplacements (par ex. `A -> C`). Exemple d'appel : `hanoi(3, "A", "B", "C")`. Combien de déplacements sont nécessaires pour `n` disques ? *(On montrera que c'est* $2^n - 1$*.)* ▷ coup de pouce

▷ début de solution

Récursif ou itératif ? ↑ cours

★★ **Exercice 24** *Traduire dans les deux sens* ▶ corrigé p. 765 • essentiel

1. Réécrire de façon **itérative** (avec une boucle) : `factorielle`, `somme` (entiers de 0 à `n`) et `puissance`. ▷ coup de pouce
2. Réécrire de façon **récursive** la fonction itérative suivante, puis dire ce qu'elle calcule :

```
1 def f(t):
2     r = 0
3     for x in t:
4         r = r + x
5     return r
```

▷ coup de pouce

3. Pour le **palindrome**, écrire la version **itérative** `est_palindrome(s)` (deux indices qui se rapprochent). Comparer sa lisibilité à la version récursive.

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

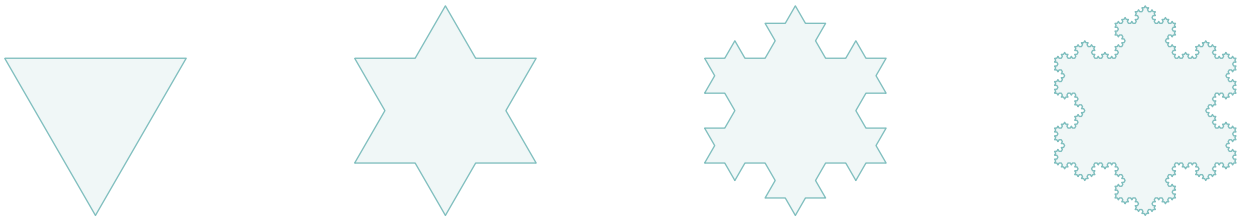
Dessins récursifs (module Turtle) ↑ cours

★★ **Exercice 25** *Carrés imbriqués* ▶ corrigé p. 766

▶ Écrire une fonction récursive `carres(cote)` qui trace un carré de côté `cote`, puis un carré plus petit à l'intérieur (par ex. de côté `cote * 0.7`), et ainsi de suite, en s'arrêtant lorsque `cote` devient inférieur à `10`. ▷ coup de pouce

★★ **Exercice 26** *Le flocon de Koch* ▶ corrigé p. 766

▶ Reprendre la fonction `koch(longueur, n)` du cours et écrire `flocon(taille, n)` qui trace le flocon complet (trois courbes de Koch formant un triangle). ▷ coup de pouce



Le flocon de Koch, aux étapes $n = 0, 1, 2$ puis 4.

★★★ **Exercice 27** *Le triangle de Sierpiński* ▶ corrigé p. 766 ≈ 30 min

Partant d'un triangle plein, on relie les milieux des côtés (ce qui délimite 4 petits triangles), on retire celui du centre, et on **recommence** sur chacun des trois triangles restants.

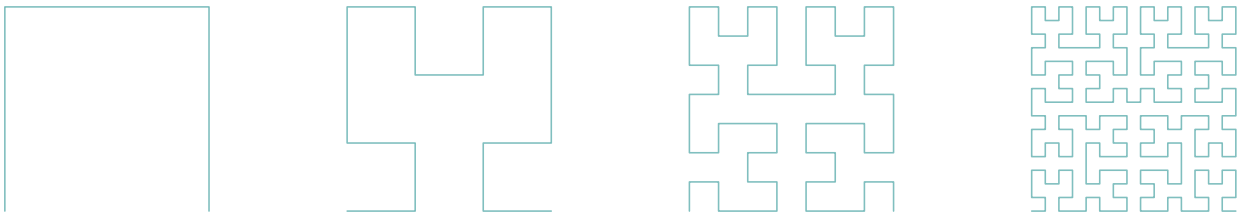


Le triangle de Sierpiński, aux ordres $n = 0, 1, 2, 3, 4$.

▶ Écrire une fonction récursive `sierpinski(a, b, c, n)` qui dessine le triangle de Sierpiński de sommets `a`, `b`, `c` (des couples de coordonnées) à l'ordre `n` (condition d'arrêt : `n == 0`, on trace le triangle plein). *(À chaque niveau, l'aire est multipliée par* $3/4$*.)* ▷ coup de pouce ▷ début de solution

★★★ **Exercice 28** *La courbe de Hilbert* ▶ corrigé p. 767 ≈ 45 min

La courbe de Hilbert est une courbe qui « remplit » un carré : à chaque niveau, elle est faite de **quatre** copies plus petites d'elle-même, reliées et pivotées.



La courbe de Hilbert, aux ordres $n = 1, 2, 3$ puis 4.

▶ Écrire une fonction récursive `hilbert(n, angle, longueur)` traçant la courbe de Hilbert d'ordre `n` (s'aider des figures : chaque ordre est fait de quatre copies de l'ordre précédent). *Défi de fin de chapitre.* ▷ coup de pouce ▷ début de solution

Vers le bac ↑cours

On ne retire des sujets que les questions qui relèvent d'un autre chapitre (programmation objet, réseaux...).

★★ Exercice 29 *Analyser et écrire des fonctions récursives (d'après Sujet zéro 2023, sujet B)*

▶ corrigé p. 767 • essentiel ≈ 20 min

Thème : l'analyse et l'écriture de programmes récursifs.

1. **a.** Expliquer en quelques mots ce qu'est une fonction récursive.

b. On considère la fonction suivante :

```
1 def compte_rebours(n):
2     """ n est un entier positif ou nul """
3     if n >= 0:
4         print(n)
5         compte_rebours(n - 1)
```

L'appel `compte_rebours(3)` affiche successivement `3`, `2`, `1`, `0`. Expliquer pourquoi le programme **s'arrête** après l'affichage de `0`. ▷ coup de pouce

2. Par convention, la factorielle de 0 vaut 1, et pour $n \geqslant 1$, $n! = n \times (n-1) \times \cdots \times 1$. **Recopier et compléter** le programme ci-dessous pour que `fact` renvoie la factorielle de `n`. Exemple : `fact(4)` renvoie `24`.

```
1 def fact(n):
2     if n == 0:
3         return ...        # a completer
4     else:
5         return ...        # a completer
```

3. La fonction `somme_entiers_rec` calcule la somme des entiers de 0 à `n`.

```
1 def somme_entiers_rec(n):
2     if n == 0:
3         return 0
4     else:
5         print(n)    # pour verification
6         return n + somme_entiers_rec(n - 1)
```

a. Écrire ce qui sera affiché dans la console lors de l'exécution de `res = somme_entiers_rec(3)`.

b. Quelle valeur sera affectée à la variable `res` ?

4. Écrire une fonction **non récursive** (itérative) `somme_entiers(n)` qui renvoie le même résultat que `somme_entiers_rec`. Exemple : `somme_entiers(4)` renvoie `10`.

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ Exercice 30 *Les scores au rugby (d'après Polynésie 2024, jour 1)*

▶ corrigé p. 768 ≈ 30 min

Thème : la récursivité. Au rugby, une équipe marque (pour simplifier) soit **3** points (pénalité), soit **5** (essai non transformé), soit **7** (essai transformé).

1. Écrire une fonction `possible_avec_penalites_seules(score)` qui renvoie `True` si le score peut être obtenu uniquement avec des pénalités (des « coups » à 3). Exemples : `possible_avec_penalites_seules(15)` renvoie `True`, et la fonction renvoie `False` pour `10`.

2. On note $f(n)$ le nombre de façons d'obtenir le score n. On admet que pour $n > 6$:

$$f(n) = f(n-3) + f(n-5) + f(n-7).$$

On donne $f(8) = 2$, $f(9) = 1$, $f(10) = 3$. On convient que $f(0) = 1$ (une seule façon d'obtenir 0 : ne rien marquer) et que $f(n) = 0$ si $n < 0$. **Déterminer les autres cas de base**, c'est-à-dire la valeur de $f(n)$ pour $1 \leqslant n \leqslant 6$. ▷ coup de pouce

3. Écrire la fonction récursive `nb_solutions(score)` qui renvoie le nombre de façons d'obtenir `score`.

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

4. Lors de l'appel de `nb_solutions(score)`, le nombre d'appels récursifs augmente très vite quand `score` grandit. **Nommer** une méthode algorithmique qui optimise le nombre d'appels.

5. **Recopier et compléter** la fonction récursive `solutions_possibles(score)` qui renvoie la liste de toutes les façons d'obtenir `score`. Une façon d'obtenir `score` est représentée par la liste des **scores successifs** du match, de `0` jusqu'à `score` : par exemple, `[0, 5, 8]` signifie un essai non transformé (5), puis une pénalité (3). Ainsi `solutions_possibles(8)` renvoie `[[0, 5, 8], [0, 3, 8]]`.

```
1  def solutions_possibles(score):
2      if score < 0:
3          resultat = []
4      elif score == 0:
5          resultat = ...                # a completer
6      else:
7          resultat = []
8          for coup in [..., 5, ...]:    # a completer
9              liste = solutions_possibles(score - coup)
10             for solution in liste:
11                 solution.append(...)  # a completer
12                 resultat.append(...)  # a completer
13     return resultat
```

Remarque : dans le sujet officiel, la dernière ligne `resultat.append(...)` *est placée au niveau de la boucle* `for coup` *; c'est une coquille (le code ne peut alors pas fonctionner). Elle est ici correctement indentée dans la boucle* `for solution`.

★★ Exercice 31 *Itinéraires dans une grille (d'après Centres étrangers 2024, groupe 1, jour 2)*

▶ corrigé p. 769 ≈ 30 min

Thème : la programmation Python et la récursivité. La partie A du sujet (programmation objet) n'est pas reprise.

On se déplace dans une grille rectangulaire, du coin en haut à gauche jusqu'au coin en bas à droite. Les seuls déplacements autorisés sont des déplacements élémentaires d'une case vers le **bas** ou d'une case vers la **droite**. Un itinéraire est noté par une suite de lettres : `D` pour un déplacement d'une case vers la droite, `B` pour un déplacement d'une case vers le bas. Le nombre de caractères `D` est la **longueur** de l'itinéraire, le nombre de caractères `B` sa **largeur**. Ainsi l'itinéraire `'DDBDBBDDDDB'` a pour longueur 7 et pour largeur 4 ; sa représentation graphique est la suivante, où `S` est la case de départ (de coordonnées $(0\,;0)$), `*` les cases visitées et `E` la case d'arrivée :

```
S * *
    * *
      *
      * * * * *
              E
```

1

Génération aléatoire d'itinéraires. On utilise la fonction `choice` du module `random` : `random.choice(sequence)` renvoie un élément choisi au hasard dans une liste non vide (et lève `IndexError` si elle est vide). On rappelle que l'opérateur `*` répète une chaîne : `"Hello world ! " * 3` vaut `'Hello world ! Hello world ! Hello world ! '`. L'algorithme proposé est le suivant :

- on initialise une variable `itineraire` à la chaîne vide, et les variables `i` et `j` à `0` ;
- tant que l'on n'est ni sur la dernière ligne ni sur la dernière colonne du tableau : on tire au sort entre un déplacement à droite ou en bas ; on concatène ce déplacement à `itineraire` ; si le déplacement est vers la droite, `j` augmente de 1, s'il est vers le bas, `i` augmente de 1 ;
- on termine le chemin par les déplacements permettant d'atteindre la case en bas à droite.

La fonction `itineraire_aleatoire(m, n)` renvoie un itinéraire aléatoire comportant `m` lettres `B` et `n` lettres `D` (grille de $m+1$ lignes et $n+1$ colonnes) ; `i` compte les `B` déjà écrits et `j` les `D`.

1. ▶ Écrire les lignes manquantes du code ci-dessous (le nombre de lignes effacées n'est pas indicatif). ▷ coup de pouce

```
1  from random import choice
2
3  def itineraire_aleatoire(m, n):
4      itineraire = ''
5      i, j = 0, 0
6      while i != m and j != n:
7          ...    # il y a plusieurs lignes
8      if i == m:
9          itineraire = itineraire + 'D' * (n - j)
10     if j == n:
11         itineraire = itineraire + 'B' * (m - i)
12     return itineraire
```

Calcul du nombre de chemins possibles. Soit m et n deux entiers naturels non nuls. On considère une grille de $m \times n$ cases (m lignes et n colonnes), et on note $N(m,n)$ le nombre de chemins distincts respectant les contraintes de l'exercice. *Attention : dans cette partie, m et n comptent des **cases**, alors que dans `itineraire_aleatoire(m, n)` ils comptaient des **déplacements** (`m` lettres `B` et `n` lettres `D`, donc une grille de $(m+1)$ lignes et $(n+1)$ colonnes). Dans une grille de $m \times n$ cases, un chemin compte $m-1$ lettres `B` et $n-1$ lettres `D`.*

2. Pour une grille de dimension $1 \times n$, justifier, éventuellement à l'aide d'un exemple, qu'il y a un seul chemin, c'est-à-dire que, pour tout entier naturel non nul n, $N(1,n) = 1$. *De même, on admet que $N(m,1) = 1$.*

3. Justifier que $N(m,n) = N(m-1,n) + N(m,n-1)$.

4. ▶ En utilisant les questions précédentes, écrire une fonction récursive `nombre_chemins(m, n)` qui renvoie le nombre de chemins possibles dans une grille rectangulaire de dimension $m \times n$.

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 32** *Les instructions d'un robot (d'après Asie 2026, jour 2)* ▶ corrigé p. 770 ≈ 45 min

Thème : la programmation Python et la récursivité. Seule la partie A du sujet est reprise (les parties B et C portent sur les réseaux).

Des robots peuvent avancer et tourner de 90 degrés à droite ou à gauche. Chaque robot reçoit ses instructions sous la forme d'une chaîne de caractères contenant des lettres et éventuellement des entiers et des parenthèses. S'il lit `'A'`, le robot avance d'un « pas » (distance fixée dans ses réglages) ; il tourne à droite s'il lit `'D'` et à gauche s'il lit `'G'`. Un entier placé dans la chaîne répète l'action ou la suite d'actions écrite entre parenthèses qui le suit : `'12A'` équivaut à `'AAAAAAAAAAAA'` et `'3(AD)'` à `'ADADAD'`. La figure ci-dessous représente le parcours du robot lorsqu'il reçoit la chaîne `'3ADA'`, c'est-à-dire la séquence `'AAADA'`.



1. Représenter le parcours effectué par le robot s'il reçoit la chaîne `'4(AG)'`.

Les premières fonctions exécutées vérifient que la chaîne transmise au robot ne comporte pas d'erreur, en particulier que : tous ses caractères sont dans `'0123456789ADG()'` ; il n'y a pas d'entier en fin de chaîne, ni d'entier juste avant une parenthèse fermante ; le parenthésage est correct (chaque parenthèse ouverte est refermée, dans le bon ordre). Dans toute cette partie, l'opérateur `in` teste la présence d'un élément dans une chaîne ; on peut le combiner avec `not` : `'a' in 'bateau'` vaut `True`, `'y' not in 'bateau'` vaut `True`.

2. Recopier et compléter la ligne 4 de la fonction `caracteres_valides`, qui renvoie un booléen indiquant si tous les caractères de la chaîne sont valides.

```
1 def caracteres_valides(chaine):
2     valides = '0123456789ADG()'
3     intrus = [c for c in chaine if c not in valides]
4     ...  # Vrai si la liste est vide, faux sinon
```

3. Recopier et compléter les lignes 4, 8 et 9 de la fonction `entiers_valides`, qui renvoie un booléen indiquant si tous les entiers sont bien placés dans la chaîne.

```
1 def entiers_valides(chaine):
2     chiffres = '0123456789'
3     if chaine[len(chaine)-1] in chiffres:
4         ...
5     for indice in range(1, len(chaine)):
6         if chaine[indice] == ')':
7             if chaine[indice - 1] in chiffres:
8                 ...
9     ...
```

Pour vérifier le parenthésage, on initialise une variable `parenthese` à `0`, puis on parcourt les caractères de `chaine` : si le caractère lu est `'('`, on augmente `parenthese` de 1 ; si c'est `')'`, on la diminue de 1 ; si `parenthese` devient strictement négative, la fonction renvoie `False`. Si le parcours s'achève, la fonction renvoie `True` si `parenthese` vaut `0`, et `False` sinon.

4. ▶ Écrire la fonction `parenthesage_correct(chaine)` qui applique cet algorithme.

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1

La fonction `lire_nombre` prend en paramètres une chaîne `chaine` et un indice `indice` ; elle renvoie la valeur du nombre dont l'écriture commence à `indice`, ainsi que l'indice de son dernier chiffre. Par exemple, `lire_nombre('AD179AGA', 2)` renvoie `(179, 4)`.

```
1  def lire_nombre(chaine, indice):
2      chiffres = "0123456789"
3      nombre = ''
4      while chaine[indice] in chiffres:
5          nombre = nombre + chaine[indice]
6          indice = indice + 1
7      return (int(nombre), indice-1)
```

5. Lors de l'appel `lire_nombre('AD179AGA', 2)`, la boucle `while` réalise trois itérations. Donner les valeurs des variables `nombre` et `indice` à la fin de chacune d'elles.

La fonction `lire_bloc` prend en paramètres une chaîne et l'indice d'une parenthèse ouvrante. Elle renvoie le contenu du bloc délimité par cette parenthèse et la parenthèse fermante associée, ainsi que l'indice de cette parenthèse fermante. Par exemple, `lire_bloc('2(AD)A', 1)` renvoie `('AD', 4)`, `lire_bloc('2(AD)3(2AGA)', 6)` renvoie `('2AGA', 11)` et `lire_bloc('2(3(AD)G)2A', 1)` renvoie `('3(AD)G', 8)`.

6. Recopier et compléter les lignes 7, 8 et 9 de la fonction `lire_bloc`.

```
1  def lire_bloc(chaine, indice):
2      indice = indice + 1
3      caractere = chaine[indice]
4      bloc = ""
5      compteur = 1
6      while compteur > 0:
7          bloc = ...
8          indice = ...
9          caractere = ...
10         if caractere == '(':
11             compteur = compteur + 1
12         if caractere == ')':
13             compteur = compteur - 1
14     return (bloc, indice)
```

On suppose déjà écrite la fonction `execute_mouvement`, qui prend en paramètre un caractère (`'A'`, `'D'` ou `'G'`) et exécute l'action correspondante du robot. La fonction **récursive** `lire_parcours` déplace le robot suivant la chaîne de caractères donnée en paramètre.

```
1  def lire_parcours(chaine):
2      chiffres = "0123456789"
3      indice = 0
4      nombre = 1
5      while indice < len(chaine):
6          car_lu = chaine[indice]
7          if car_lu in 'AGD':        # commande simple
8              for k in range(nombre):
9                  execute_mouvement(car_lu)
10             nombre = 1
11         elif car_lu in chiffres:   # repetition
12             t = ...
13             nombre = t[0]
14             indice = t[1]
15         elif car_lu == '(':        # debut d'un bloc
16             t = ...
17             bloc = t[0]
18             indice = t[1]
19             for k in range(nombre):
20                 ...
21             nombre = 1
22         indice = indice + 1
```

7. Recopier et compléter les lignes 12, 16 et 20 de la fonction `lire_parcours`. ▷ coup de pouce

★★★ **Exercice 33** *Le cadenas de l'escape game (sujet maison, façon bac)* ▶ corrigé p. 771 ≈ 60 min

Thème : la récursivité. Cet exercice n'est pas un sujet officiel : il a été écrit au format du bac pour s'entraîner.

Dans un escape game, les joueurs ont trouvé les lettres du mot qui ouvre un cadenas, mais pas leur ordre. Ils veulent donc obtenir toutes les **anagrammes** de ces lettres : tous les mots obtenus en les écrivant dans un ordre quelconque (même dépourvus de sens). Par exemple, `'LCEF'` est une anagramme de `'CLEF'`. Dans tout l'exercice, les mots sont écrits en majuscules.

1. Écrire toutes les anagrammes de `'ABC'`, en commençant par celles dont la première lettre est `A`, puis `B`, puis `C`. Combien y en a-t-il ?

2. On note $P(n)$ le nombre d'anagrammes d'un mot de n lettres **toutes différentes**. On a $P(1) = 1$. Expliquer pourquoi $P(n) = n \times P(n-1)$ pour $n \geqslant 2$ (penser au choix de la première lettre), puis écrire une fonction récursive `nb_anagrammes(n)` qui renvoie $P(n)$.

3. Le cadenas comporte 8 lettres toutes différentes, et un essai prend 2 secondes. Sachant que `nb_anagrammes(8)` renvoie `40320`, combien de temps faut-il, au pire, pour tester toutes les anagrammes ? Que penser de cette méthode si le mot était plus long ?

Pour construire les anagrammes, on utilise la fonction `sans_lettre(mot, i)`, qui renvoie le mot privé de sa lettre d'indice `i`. Par exemple, `sans_lettre('CLEF', 1)` renvoie `'CEF'`.

4. ▶ Écrire la fonction `sans_lettre` (non récursive) en utilisant des tranches de chaîne (`mot[a:b]`).

L'idée récursive est la suivante : pour chaque lettre du mot, on la place en tête, puis on la fait suivre de chacune des anagrammes des lettres restantes.

```
1 def anagrammes(mot):
2     if len(mot) <= 1:                    # condition d'arret
3         return ...
4     resultat = []
5     for i in range(len(mot)):
6         for fin in ...:
7             resultat.append(...)
8     return resultat
```

5. ▶ Recopier et compléter les lignes 3, 6 et 7 de la fonction `anagrammes`, qui renvoie la liste de toutes les anagrammes de `mot`. Par exemple, `anagrammes('AB')` renvoie `['AB', 'BA']`.

▷ coup de pouce ▷ début de solution

6. Dessiner l'arbre des appels de `anagrammes('ABC')`, en indiquant pour chaque appel son paramètre. Combien d'appels (appel initial compris) sont effectués ? En déduire, sans dessiner l'arbre, le nombre d'appels pour `anagrammes('ABCD')`.

7. Justifier que l'appel `anagrammes(mot)` se termine, quel que soit le mot.

8. On appelle `anagrammes('ALLO')`. Combien de mots la liste renvoyée contient-elle ? Combien de mots **différents** ? Expliquer la différence.

Pour vérifier si une proposition est une anagramme du bon mot, il est inutile de fabriquer toutes les anagrammes.

9. ▶ Écrire une fonction **récursive** `retirer(mot, lettre)` qui renvoie le mot privé de la **première** apparition de `lettre` (et le mot inchangé si `lettre` n'y figure pas). Par exemple, `retirer('ALLO', 'L')` renvoie `'ALO'`.

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

10. ▶ Recopier et compléter la fonction récursive `est_anagramme`, qui renvoie `True` si `mot1` est une anagramme de `mot2`, `False` sinon. Dérouler ensuite l'appel `est_anagramme('ALLO', 'LOLA')`.

```
1 def est_anagramme(mot1, mot2):
2     if mot1 == '':                       # condition d'arret
3         return ...
4     if mot1[0] not in mot2:
5         return ...
6     return est_anagramme(..., ...)
```

Critiquer une réponse d'IA ↑cours

★★ **Exercice 34** *Un test de tri écrit par un assistant d'IA* ▶ corrigé p. 773

Un élève demande à un assistant d'IA : « Écris une fonction récursive Python `est_triee(tab)` qui renvoie `True` si la liste `tab` est triée dans l'ordre croissant, `False` sinon. » Voici la réponse obtenue :

```
1 def est_triee(tab):
2     if len(tab) <= 1:                # condition d'arret : 0 ou 1 element
3         return True
4     if tab[0] > tab[1]:              # un desordre entre les deux premiers
5         return False
6     return est_triee(tab[2:])        # les deux premiers sont en ordre
```

« La condition d'arrêt est claire (une liste d'au plus un élément est triée), l'appel récursif porte sur une liste strictement plus courte, donc la fonction termine ; et l'on vérifie chaque paire d'éléments consécutifs. »

1. La réponse est-elle correcte ? ▶ Dérouler à la main `est_triee([1, 5, 2, 8])` (écrire la liste reçue par chaque appel), puis `est_triee([1, 2, 5, 8])`.
2. Localiser et corriger l'erreur. ▷ coup de pouce
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

S'entraîner à l'oral ↑ cours

★★ **Exercice 35**  *Expliquer en deux minutes* ▶ corrigé p. 773

Au Grand Oral comme devant l'examinateur de l'épreuve pratique, il faut savoir **expliquer** une notion clairement, sans lire ses notes. Choisir l'un des sujets suivants (ou le tirer au sort) :

Sujet 1. Expliquer à un camarade de Première pourquoi une fonction récursive bien écrite **termine**.

Sujet 2. Expliquer ce qu'est la **pile d'exécution** et pourquoi Python peut afficher l'erreur `RecursionError`.

Sujet 3. Expliquer, sur un exemple, pourquoi le résultat d'une fonction récursive se construit « à la remontée ».

Préparation (5 min, seul). Noter au brouillon **trois idées** et **un exemple**, rien de plus : comme au Grand Oral, ces notes servent d'aide-mémoire. **Exposé (2 min, chronométré).** Debout, face à un camarade, **sans lire ses notes** et sans rien écrire : tout passe par la parole. **Retour (2 min).** Le camarade pose une question de relance (on peut alors s'aider d'un schéma), remplit la grille ci-dessous et donne un conseil ; puis on échange les rôles. Cette grille reprend les critères d'évaluation du Grand Oral. ▷ coup de pouce

Grille entre camarades	**Oui**	**En partie**	**Pas encore**
Clair — audible, posé ; chaque mot technique est expliqué	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	☐	☐	☐
Construit — un fil conducteur, tenu en deux minutes, sans lire	☐	☐	☐
Un conseil pour la prochaine fois :			

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Écrire la suite des appels `a(3)`, `a(2)`… jusqu'à la condition d'arrêt. Pour chacun, le `print` a-t-il lieu avant de lancer l'appel suivant, ou seulement après son retour ? Même question pour `b`. ↑ énoncé

Ex. 2 q. 1 – piste Le `print` est-il placé avant ou après l'appel récursif ? Écrire d'abord la suite des valeurs de `n` à la descente, puis se demander dans quel ordre les affichages se font. ↑

Ex. 3 – piste (A) : où se trouve la condition d'arrêt ? (B) : que se passe-t-il lors de l'appel `fact(0)` ? ↑

Ex. 4 q. 2 – piste Chaque nœud de l'arbre dessiné à la question 1 est un appel : les compter tous, sans oublier l'appel initial. Repérer aussi les appels identiques répétés. ↑ énoncé

Ex. 5 – piste Se poser les trois questions du réflexe. Que faut-il afficher quand il ne reste plus de mouton ($n = 0$) ? Après avoir affiché un seul mouton, combien en reste-t-il à afficher ? ↑

Ex. 6 – piste Quel est le n-ième nombre impair (tester avec $n = 1, 2, 3$) ? La somme des n premiers impairs, c'est la somme des $n-1$ premiers, plus ce dernier nombre. Que vaut `somme_impairs(0)` ? ↑énoncé

Ex. 7 – piste Écrire $a \times n$ à l'aide de $a \times (n-1)$. Que vaut $a \times 0$? Chaque appel fait diminuer `n` de 1 : combien d'appels faut-il pour atteindre 0 ? ↑énoncé

Ex. 8 – piste Que vaut `n // 10` ? Pour quels entiers la réponse est-elle immédiate (condition d'arrêt) ? ↑

Ex. 9 – piste `n % 2` donne le dernier bit de `n` et `n // 2` le supprime. Que renvoyer pour `n == 0` ? ↑

Ex. 10 – piste Les deux égalités rappelées forment déjà la fonction : laquelle donne la condition d'arrêt, laquelle l'appel récursif ? Attention à l'ordre des deux paramètres dans l'appel. ↑

Ex. 11 – piste Il faut ici **deux** conditions d'arrêt ($n = 0$ et $n = 1$) et deux appels récursifs. Les paramètres `a` et `b` sont transmis sans changement à chaque appel. ↑énoncé

Ex. 12 – piste Quelle est la condition d'arrêt ? Le test de parité s'écrit `u % 2 == 0` ; utiliser `//` pour que les termes restent entiers. L'appel récursif porte sur le terme suivant. ↑énoncé

Ex. 13 – piste Condition d'arrêt : $x^0 = 1$. Dans le cas pair, ne faire qu'**un seul** appel récursif sur `n // 2` et ranger son résultat dans une variable avant de l'élever au carré (deux appels identiques feraient perdre tout le gain). ↑énoncé

Ex. 13 – début de solution

```
def puissance_rapide(x, n):
    if n == 0:
        return 1
    if n % 2 == 0:
        p = puissance_rapide(x, n // 2)
```

↑énoncé

Ex. 14 – piste Ne regarder que le premier caractère `s[0]` : compte-t-il pour 1 ou pour 0 ? Le reste du travail porte sur la chaîne plus courte `s[1:]`. Que renvoyer pour la chaîne vide ? ↑

Ex. 15 – piste Où doit se retrouver le premier caractère `s[0]` dans la chaîne inversée ? Le reste `s[1:]` s'inverse par l'appel récursif. ↑énoncé

Ex. 16 – piste Quels caractères doivent être égaux dans un palindrome ? Sur quelle chaîne plus courte peut-on alors reposer la question ? Quelles chaînes sont des palindromes sans rien vérifier (condition d'arrêt) ? ↑énoncé

Ex. 17 q. 2 – piste La liste étant non vide, la condition d'arrêt est une liste d'**un seul** élément. Pour le maximum : comparer `tab[0]` au maximum du reste `tab[1:]`. ↑énoncé

Ex. 18 – piste Il y a deux façons de s'arrêter : lesquelles ? Quel paramètre change d'un appel à l'autre pour se rapprocher de la fin du tableau ? ↑énoncé

Ex. 19 – piste Quand la zone comprise entre `gauche` et `droite` est-elle vide (condition d'arrêt) ? Reprendre la dichotomie de Première : la boucle `while` est remplacée par un appel récursif sur la moitié conservée. ↑énoncé

Ex. 19 – début de solution

```
def dicho(t, v, gauche, droite):
    if gauche > droite:
        return -1
    milieu = (gauche + droite) // 2
```

↑énoncé

Ex. 20 – piste Combien existe-t-il de mots de longueur `0`, et lequel ? Un mot de longueur `n` s'obtient à partir d'un mot de longueur `n - 1` : comment ? ↑énoncé

Ex. 21 – piste Une décomposition de `n` commence soit par `1`, soit par `2` : que reste-t-il alors à décomposer ? Ajouter un second paramètre `debut` qui mémorise le début déjà écrit, avec la valeur par défaut `""` (voir le cours, « Un outil Python : le paramètre par défaut »), et l'afficher quand il ne reste plus rien. ↑énoncé

Ex. 22 – piste La définition donne directement la condition d'arrêt et l'appel récursif : la traduire ligne à ligne. Pour le défi : deux boucles imbriquées, `p` allant de `0` à `n`. ↑énoncé

Ex. 23 – piste Les trois étapes de l'énoncé forment le corps de la fonction : deux appels récursifs sur `n - 1` disques et un affichage. Le piquet qui sert d'intermédiaire change selon l'étape. Condition d'arrêt : `n == 0`, il n'y a rien à faire. Pour compter : $d(n) = 2\,d(n-1) + 1$. ↑

Ex. 23 – début de solution
```
def hanoi(n, depart, inter, arrivee):
    if n > 0:
        hanoi(n - 1, depart, arrivee, inter)
```
↑énoncé

Ex. 24 q. 1 – piste Un accumulateur initialisé avec la valeur de la condition d'arrêt (`1` pour un produit, `0` pour une somme), mis à jour à chaque tour d'une boucle `for`. ↑énoncé

Ex. 24 q. 2 – piste À partir du résultat sur quel tableau plus court peut-on obtenir celui de `t` ? ↑énoncé

Ex. 25 – piste Tracer un carré (boucle de 4 côtés), se décaler un peu vers l'intérieur en levant le crayon, puis un seul appel récursif avec un côté plus petit. ↑énoncé

Ex. 26 – piste Pas de nouvelle récursion ici : trois fois « tracer une courbe de Koch, puis tourner ». De quel angle tourner pour fermer un triangle équilatéral ? ↑énoncé

Ex. 27 – piste Écrire d'abord deux fonctions utilitaires : le milieu de deux points (moyenne des coordonnées) et le tracé d'un triangle avec `goto`. Sinon, trois appels récursifs à l'ordre `n - 1`, un par triangle de coin. ↑énoncé

Ex. 27 – début de solution
```
def milieu(p, q):
    return ((p[0] + q[0]) / 2, (p[1] + q[1]) / 2)
def sierpinski(a, b, c, n):
    if n == 0:
        tracer_triangle(a, b, c)
```
↑énoncé

Ex. 28 – piste Ajouter un paramètre `angle` valant `90` ou `-90` : une copie « pivotée dans l'autre sens » s'obtient par un appel avec `-angle`. La courbe d'ordre `n` enchaîne quatre courbes d'ordre `n - 1` séparées par trois `forward` et des rotations : tourner, copie pivotée dans l'autre sens, avancer, tourner, copie, avancer, copie, tourner, avancer, copie pivotée dans l'autre sens, tourner. ↑énoncé

Ex. 28 – début de solution
```
def hilbert(n, angle, longueur):
    if n == 0:
        return
    t.left(angle)
    hilbert(n - 1, -angle, longueur)
```
↑énoncé

Ex. 29 q. 1b – piste Quel est le paramètre de l'appel qui suit l'affichage de `0` ? Le test `n >= 0` est-il alors vrai ? ↑énoncé

Ex. 30 q. 2 – piste Pour chaque n de 1 à 6, chercher à la main les combinaisons de 3, 5 et 7 dont la somme vaut n. ↑énoncé

Ex. 31 q. 1 – piste Tirer une lettre avec `choice` dans une liste de deux lettres, l'ajouter à `itineraire`, puis mettre à jour `i` ou `j` selon la lettre tirée. ↑énoncé

Ex. 32 q. 7 – piste Quelles fonctions déjà écrites renvoient un couple (valeur, indice) ? Le bloc lu doit être exécuté `nombre` fois : par quelle fonction ? C'est là qu'est la récursivité. ↑énoncé

Ex. 33 q. 5 – piste La fonction renvoie une **liste** : que doit-elle contenir pour un mot d'au plus une lettre ? La lettre `mot[i]` se place en tête ; les fins possibles sont les anagrammes de quel mot plus court ? ↑énoncé

Ex. 33 q. 5 – début de solution Ligne 3 : `return [mot]`. Ligne 6 : `for fin in anagrammes(sans_lettre(mot, i)):` ↑énoncé

Ex. 34 q. 2 – piste Lister, pour chaque appel, les couples d'éléments voisins qui ont été comparés. Tous les couples `(tab[i], tab[i + 1])` le sont-ils ? ↑énoncé

Ex. 35 – piste Structurer chaque explication en trois temps : une **définition** en une phrase, un **exemple** petit et concret déroulé à voix haute, puis le **piège** (l'erreur que l'auditeur risque de faire). Finir par une phrase qui répond à la question posée. Pour le sujet 1 : quelle quantité entière change à chaque appel, et dans quel sens ? Que se passe-t-il quand elle atteint la condition d'arrêt ? ↑énoncé

1

TP ET PROJETS

PROJET Sudoku : résoudre par retour sur trace

backtracking

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t01-projet-sudoku` — **archive .zip**

Prérequis : fonctions, listes de listes, la fonction `placement_valide` (vue en Première, à réécrire ici). **Fichiers :** `grilles_test.py`, `demo_backtracking_4x4.py` (à lancer), `terminale_squelette.py` et `generateur_squelette.py` (à compléter).

Ce que vous allez apprendre : écrire et *analyser* un programme récursif ; découvrir une méthode algorithmique fondamentale, le *retour sur trace* (*backtracking*) — essayer / revenir en arrière ; faire le lien avec l'*arbre des possibilités* et le *coût*.

1 Le problème

En Première, on a écrit `placement_valide(grille, i, j, v)` : « *a-t-on le droit d'écrire* `v` *dans cette case ?* ». On sait donc **vérifier**. Aujourd'hui on veut **résoudre** : partir d'une grille à trous et la remplir entièrement.

Le piège : dans un Sudoku difficile, on ne peut pas toujours déduire une case avec certitude. À un moment il faut **parier** sur un chiffre… et être capable de **défaire** le pari s'il mène à une impasse.

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

La grille `DIFFICILE` *(fichier* `grilles_test.py`*) : seulement 21 indices. Impossible à finir « à l'œil » sans essayer puis revenir en arrière — c'est exactement ce que fait le backtracking.*

2 Comprendre le retour sur trace — sans ordinateur

Définition 5 — L'image du labyrinthe

Vous explorez un labyrinthe avec un fil d'Ariane. À chaque carrefour, vous choisissez un couloir. Si vous arrivez dans un cul-de-sac, vous **remontez le fil** jusqu'au dernier carrefour et vous essayez un **autre** couloir. Vous finissez toujours par sortir s'il existe une sortie, parce que vous n'abandonnez jamais un carrefour sans avoir essayé **tous** ses couloirs.

Le Sudoku, c'est pareil :

- un **carrefour** = une case vide ;
- les **couloirs** = les chiffres 1 à 9 qu'on peut y écrire ;
- un **cul-de-sac** = une case suivante où **aucun** chiffre n'est possible ;
- **remonter le fil** = effacer le dernier chiffre écrit et essayer le suivant.

Exercice 1 *Sur papier.*

Sur cette mini-grille 4×4 (chiffres de 1 à 4, blocs 2×2), remplir les cases vides **dans l'ordre de lecture** (ligne par ligne, de gauche à droite), en essayant 1, puis 2… dans chacune, et noter à voix haute « j'essaie », « impasse, je reviens ». Une seule solution existe. **Combien de fois** avez-vous dû revenir en arrière ?

1			
			2
3			
			4

3 Voir le backtracking tourner

Lancer le fichier `demo_backtracking_4x4.py` (rien à écrire). Il résout la grille ci-dessus, dans le même ordre que vous, en affichant chaque essai. Les cases sont repérées par `(ligne, colonne)`, numérotées à partir de 0 :

```
-> j'ecris 2 en (0,1)
   -> j'ecris 3 en (0,2)
   X  j'efface 3 en (0,2)   (retour numero 1)
   -> j'ecris 4 en (0,2)
      ...
```

Le **décalage vers la droite** mesure la profondeur : plus on est décalé, plus on a rempli de cases.

Exercice 2 *Observer la démo.*

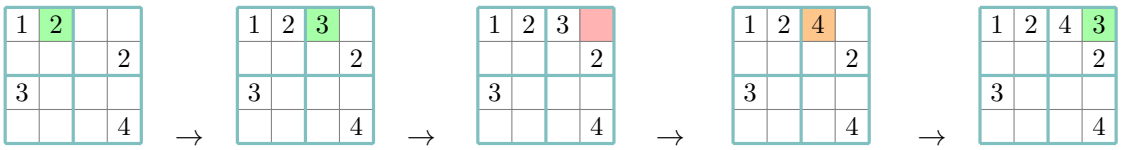
Réponses sur le cahier.

1. Combien de **retours en arrière** le programme affiche-t-il au total ?
2. Est-ce le même nombre que celui trouvé à la main ?
3. La ligne `=> grille complete !` : à quel moment apparaît-elle, et que se passe-t-il juste après ?

1

4 Le retour en arrière, pas à pas

Suivons **à la main** les premières cases vides de la mini-grille 4×4 (on parcourt de gauche à droite, de haut en bas). La case qu'on vient de remplir est en **vert**, l'impasse en **rouge**, le retour en arrière en **orange**.



Le détail des chiffres essayés à chaque étape (✓ = accepté par `placement_valide`, × = refusé) :

Étape	Case	Chiffres essayés	Ce qui se passe
1	(0,1)	1 ×, 2 ✓	on écrit **2**
2	(0,2)	1 ×, 2 ×, 3 ✓	on écrit **3**
3	(0,3)	1 ×, 2 ×, 3 ×, 4 ×	**impasse** : aucun chiffre ne passe
4	*retour* (0,2)	on efface le 3, puis 4 ✓	on écrit **4** à la place du 3
5	(0,3)	1 ×, 2 ×, 3 ✓	on écrit **3** ; la suite s'enchaîne

À l'étape 3, même le `4` est refusé : il y a déjà un `4` **dans la colonne** (case du bas). Plus aucun chiffre n'est possible en (0,3) : c'est une impasse, on **revient** à la case précédente (0,2), on efface le `3` qu'on y avait mis, et on essaie le chiffre *suivant* : `4`.

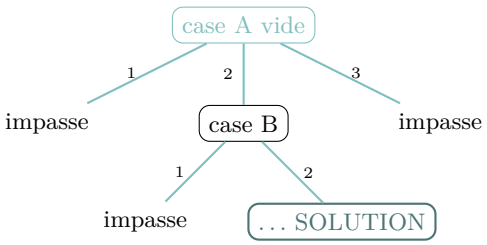
Le point-clé du backtracking

Un chiffre accepté par `placement_valide` n'est qu'un **pari local** : il respecte les règles *pour l'instant*, mais rien ne garantit qu'il soit le bon. À l'étape 2 le `3` était pourtant valide — et il a fallu l'annuler à l'étape 4. Le backtracking, c'est exactement cette capacité à **revenir défaire un pari** qui s'est révélé mauvais plus loin.

Il y a donc **deux façons d'échouer** : *tout de suite* (aucun chiffre ne passe dès la case courante, comme en (0,3)) ou *plus tard* (un chiffre passe, mais mène à une impasse quelques cases après). Le même mécanisme règle les deux.

5 L'arbre des possibilités

On peut dessiner tous les essais sous forme d'**arbre** : la racine est la grille de départ, chaque **branche** est un chiffre essayé dans la prochaine case vide, une **feuille** est soit une impasse, soit la solution.



- le backtracking = un **parcours de cet arbre**, en descendant tant qu'on peut, en remontant dès qu'on est bloqué ;
- ce type de parcours s'appelle un parcours **en profondeur d'abord** : vous le retrouverez dans les chapitres sur les arbres et les graphes.

Dessiner sur le cahier le petit arbre de la démo 4×4 (une ou deux branches suffisent) pour bien voir où sont les impasses.

6 Remettre en place la vérification

Le solveur a besoin d'une seule brique : `placement_valide(grille, i, j, v)`, qui dit si l'on a le droit d'écrire `v` dans la case `(i, j)`. Vous l'avez rencontrée en Première ; ici, vous la **réécrivez vous-même** dans `terminale_squelette.py` (parties 1 et 2), avec les fonctions de lecture qui la préparent.

Exercice 3 *Lire dans la grille.*

1. `valeurs_ligne(grille, i)` : les 9 valeurs de la ligne `i`.
2. `valeurs_colonne(grille, j)` : les 9 valeurs de la colonne `j`, **par compréhension** (on collecte `grille[i][j]` pour `i` de 0 à 8).
3. `valeurs_bloc(grille, i, j)` : les 9 valeurs du bloc 3×3 qui **contient** la case `(i, j)`. Son coin haut-gauche est `((i // 3) * 3, (j // 3) * 3)`.
4. *Sur le cahier.* Que vaut `(i // 3) * 3` pour `i` allant de 0 à 8 ? Pourquoi est-ce bien la première ligne du bloc ?

Exercice 4 *Peut-on poser un chiffre ?*

1. Écrire `placement_valide(grille, i, j, v)` : elle renvoie `False` dès que `v` figure déjà dans la ligne `i`, la colonne `j` ou le bloc de la case, et `True` sinon. *Vous pouvez utiliser vos trois fonctions de lecture et l'opérateur* `in`.
2. Écrire `grille_complete(grille)` : `True` s'il ne reste aucune case égale à `0`.
3. Écrire `grille_valide(grille)` qui, pour une grille **remplie**, renvoie `True` si aucun chiffre n'est en double (ligne, colonne ou bloc), `False` sinon. *Astuce :* pour tester la case `(i, j)` de valeur `v`, la retirer (`grille[i][j] = 0`), appeler `placement_valide`, puis la remettre.
4. *Sur le cahier.* Dans l'astuce de la question 3, que se passerait-il si l'on ne retirait pas la valeur avant d'appeler `placement_valide` ?

Tester le code : lancer `terminale_squelette.py` ; les tests des parties 1 et 2 (en bas du fichier) doivent passer, par exemple :

```
assert valeurs_colonne(FACILE, 0) == [5, 6, 0, 8, 4, 7, 0, 0, 0]
assert placement_valide(FACILE, 0, 2, 4) == True
assert placement_valide(FACILE, 0, 2, 5) == False   # 5 deja sur la ligne
assert placement_valide(FACILE, 0, 2, 8) == False   # 8 deja dans le bloc
assert grille_valide(DOUBLON_LIGNE) == False
```

7 Écrire le solveur

Dans `terminale_squelette.py` (partie 3), avec votre `placement_valide` de la section précédente, vous écrivez maintenant deux fonctions.

1

Exercice 5 `trouver_case_vide(grille)`.

Renvoie le couple `(i, j)` de la **première** case vide (parcours ligne par ligne), ou `None` s'il n'y en a plus. *Test attendu :* `trouver_case_vide(FACILE)` doit donner `(0, 2)`.

À compléter (dans `terminale_squelette.py`) :

```
def trouver_case_vide(grille):
    for i in range(9):
        for j in range(9):
            # A COMPLETER : si (i, j) est vide, renvoyer le couple (i, j)
            ...
    return None
```

Exercice 6 `resoudre(grille)` — *le cœur du sujet.*

Elle **modifie la grille sur place** et renvoie `True` si elle a trouvé une solution, `False` sinon. Voici l'algorithme, mot pour mot :

```
resoudre(grille) :
    chercher une case vide
    s'il n'y en a plus :
        la grille est finie          ->  renvoyer True

    pour chaque valeur v de 1 a 9 :
        si placement_valide(grille, i, j, v) :
            ecrire v dans la case (i, j)
            si resoudre(grille) reussit :      # appel RECURSIF sur le reste
                renvoyer True
            sinon :
                effacer la case (remettre 0)   # LE retour en arriere

    aucune valeur n'a marche         ->  renvoyer False
```

À compléter (le squelette est prêt, il vous reste les 3 lignes clés) :

```
def resoudre(grille):
    case = trouver_case_vide(grille)
    if case is None:
        return True                 # cas de base : plus de case vide
    i, j = case
    for v in range(1, 10):
        if placement_valide(grille, i, j, v):
            grille[i][j] = v
            # A COMPLETER : appel recursif resoudre(grille) ;
            #   si ca reussit -> return True
            #   sinon effacer la case : grille[i][j] = 0  (retour en arriere)
            ...
    return False
```

Les trois lignes du raisonnement récursif

- **le cas de base** (« plus de case vide → `True` ») : c'est ce qui garantit que la récursion **s'arrête** ;
- **l'appel récursif** `resoudre(grille)` : on délègue « le reste de la grille » à la même fonction ;
- `grille[i][j] = 0` : sans cette ligne, pas de retour en arrière — le programme resterait coincé sur un mauvais pari.

Où est écrit « reviens en arrière » ? Nulle part ! Le retour en arrière n'est pas une instruction : il **émerge** de trois éléments qui se combinent.

- **la boucle `pour chaque valeur v`** : dès qu'un pari échoue, on efface la case (`grille[i][j] = 0`) et la boucle passe *toute seule* au chiffre suivant — on essaie une autre piste *au même endroit* ;
- **le `renvoyer False` final** : si *aucun* chiffre ne marche à cette case, la fonction rend `False` à l'appel qui l'avait lancée — celui de la case **précédente**. Cet appel reprend alors *sa* boucle au chiffre suivant : on remonte d'un cran ;
- **la pile des appels** : chaque appel de `resoudre` en attente se souvient de sa case et du chiffre qu'il testait. C'est elle, le véritable « fil d'Ariane » : elle permet de remonter exactement au bon carrefour.

Autrement dit : la boucle explore les pistes d'une même case, le `return False` fait remonter d'une case, et la pile retient le chemin. Ensemble, ils *sont* le retour en arrière.

Exercice 7 *Tester votre solveur avec des* `assert` *(*`resoudre` *modifie la grille, on en fait une copie).*

```
def copie(grille):                      # copier une matrice : double comprehension
    return [[v for v in ligne] for ligne in grille]

g = copie(MINI)
assert resoudre(g) == True and g == SOLUTION_MINI   # 4x4
assert resoudre(copie(FACILE)) == True
assert resoudre(copie(DIFFICILE)) == True        # 21 indices : impossible "a l'oeil"
assert resoudre(copie(IMPOSSIBLE)) == False      # pas de solution -- et ca s'arrete !
```

> **Remarque** Le cas `IMPOSSIBLE` est important : il **n'a pas de solution** mais ne contient aucun doublon au départ. Un solveur correct doit renvoyer `False` **sans tourner à l'infini**. C'est la preuve que votre retour en arrière est complet.

8 Analyser

Exercice 8 *Terminaison, correction, coût.*
Réponses sur le cahier.

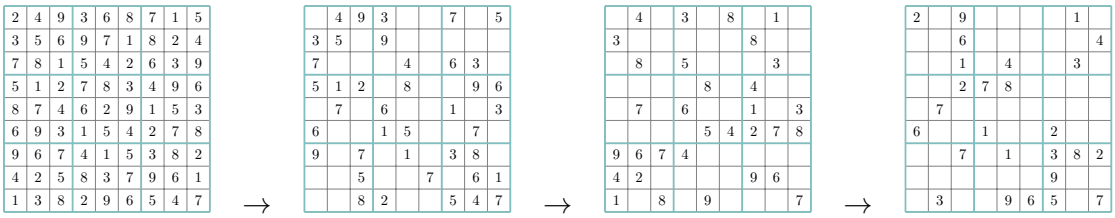
1. **Terminaison.** Pourquoi `resoudre` finit-elle toujours par s'arrêter ? *(Indice : à chaque appel récursif, il y a une case vide de moins.)*
2. **Correction.** Si `resoudre` renvoie `True`, la grille obtenue est-elle forcément valide ? Pourquoi ?
3. **Coût.** Sur `DIFFICILE`, y a-t-il eu beaucoup plus d'essais que sur `FACILE` ? *(Ajouter un compteur d'essais, comme dans la démo* 4×4*.)*
4. Le mot **récursivité** : où l'a-t-on utilisée exactement, et qu'est-ce qui joue le rôle de « fil d'Ariane » ?

9 Créer des grilles (génération)

Résoudre, c'est bien ; mais d'où viennent les grilles des journaux ? Dans cette partie, vous écrivez un programme qui **fabrique** des Sudoku jouables, avec **une seule solution**, au niveau choisi. Et l'outil sera… encore le retour sur trace.

Fichier : `generateur_squelette.py`. Il **réutilise vos fonctions** de `terminale_squelette.py` : votre solveur doit donc être terminé. Lancer le fichier après chaque étape : les tests avancent étape par étape.

On procède en quatre étapes : **(1)** fabriquer une grille **complète** ; **(2)** savoir **compter** les solutions d'une grille ; **(3)** **creuser** des trous tant que la solution reste unique ; **(4)** assembler.



complète → facile (40 indices) → moyen (32) → difficile (26)

Exercice 9 *Pourquoi pas simplement « au hasard » ?*

Idée naïve pour remplir une grille vide : on avance case par case, ligne par ligne, et dans chaque case on tire au hasard un chiffre **autorisé** (`placement_valide`), sans jamais revenir en arrière. Ci-contre, les deux premières lignes ont été remplies ainsi.

1	2	3	4	5	6	7	8	9
4	5	6	1	2	3			

1. Quels chiffres peut-on écrire dans la case colorée `(1, 6)` ? Justifier avec les trois règles.
2. Ces deux lignes contiennent-elles un doublon ? Que devient pourtant la méthode naïve ?
3. Quelle méthode, vue dans ce projet, sait sortir d'une telle impasse ?

Étape 1 — Une grille complète, différente à chaque fois

Exercice 10 *Du solveur au générateur.*

1. Dans la console, résoudre une grille **vide** avec votre solveur :

```
>>> g = grille_vide()          # 81 zeros
>>> resoudre(g)
>>> afficher(g)
```

Recommencer plusieurs fois. Que constatez-vous ? Quelle est la première ligne obtenue, et pourquoi est-ce *forcément* celle-là ? Expliquer en parlant de l'**ordre** dans lequel `resoudre` essaie les chiffres.

2. Pour obtenir une grille différente à chaque exécution, il suffit de changer *l'ordre des essais* : `random.shuffle(valeurs)` mélange la liste `valeurs` **sur place**. Compléter `resoudre_aleatoire` : le début est écrit, il vous reste le retour sur trace, en parcourant `valeurs` au lieu de `range(1, 10)`.

```
def resoudre_aleatoire(grille):
    case = trouver_case_vide(grille)
    if case is None:
        return True
    i, j = case
    valeurs = [v for v in range(1, 10)]
    random.shuffle(valeurs)          # on melange l'ordre des essais
    # A COMPLETER
```

3. `resoudre_aleatoire(grille_vide())` peut-elle renvoyer `False` ? Justifier.

Étape 2 — Compter les solutions

Un « vrai » Sudoku n'a **qu'une seule** solution : sinon le joueur devrait deviner. Pour le vérifier, il faut savoir **compter** les solutions. Or `resoudre` s'arrête à la *première* : dès qu'un appel renvoie `True`, tout remonte. La fonction `compter_solutions`, elle, explore **tout** l'arbre des essais et **additionne** ce que renvoie chaque branche.

5	3	4			8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9			1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

`DEUX_SOLUTIONS`

La grille `DEUX_SOLUTIONS` (fournie dans le fichier) est `COMPLETE_OK` dont on a vidé les quatre cases colorées `(0,3)`, `(0,4)`, `(3,3)`, `(3,4)`.

Exercice 11 *Compter au lieu de trouver.*

1. *Sur le cahier.* Combien de solutions doit trouver `compter_solutions` sur `COMPLETE_OK` ? sur `IMPOSSIBLE` ? sur `FACILE` ? Pour `DEUX_SOLUTIONS`, écrire les **deux** façons de remplir les cases colorées.

2. *Sur le cahier.* Quel est le **cas de base** ? Que renvoie-t-on quand il n'y a plus de case vide ?

3. *Sur le cahier.* Dans `resoudre`, la ligne `grille[i][j] = 0` n'est exécutée qu'en cas d'*échec*. Dans `compter_solutions`, on efface la case **toujours**, même quand la branche a donné des solutions. Donner deux raisons.

4. Écrire une première version **sans limite** (ignorer pour l'instant le paramètre `limite`), en suivant l'algorithme :

```
compter_solutions(grille) :
    chercher une case vide (i, j)
    s'il n'y en a plus :  renvoyer ...
    total <- 0
    pour chaque valeur v de 1 a 9 :
        si placement_valide(grille, i, j, v) :
            ecrire v en (i, j)
            total <- total + ...          # appel RECURSIF
            effacer la case (i, j)        # TOUJOURS
    renvoyer total
```

   Vérifier vos réponses de la question 1 dans la console. La lancer ensuite sur `grille_vide()` : que se passe-t-il ? (`Ctrl+C` pour arrêter.) *Il existe 6 670 903 752 021 072 936 960 grilles complètes…*

5. Pour tester l'unicité, on n'a pas besoin du nombre exact : dès que `total` atteint `limite`, on arrête et on renvoie `total`. Modifier votre fonction en conséquence.

   a. Pourquoi `limite = 2` suffit-il pour savoir si la solution est unique ?
   b. L'appel récursif doit recevoir `limite - total`, et non `limite`. Pourquoi ? *(Imaginer* `total` *valant déjà 1 et un appel récursif qui trouve 2 solutions.)*

   Lancer le fichier : tous les tests de l'étape 2 doivent passer (y compris « la grille est rendue intacte »).

Étape 3 — Creuser des trous

On part d'une grille complète et on vide des cases une par une, dans un ordre aléatoire. Après chaque retrait, on **garde le trou seulement si la solution reste unique** ; sinon on **rebouche**. On s'arrête quand il ne reste plus que `indices` cases remplies. Le **niveau** se règle par ce nombre d'indices : plus il en reste, plus c'est facile.

```
creuser(grille_pleine, indices) :
    grille <- une COPIE de grille_pleine
    cases  <- la liste des 81 couples (i, j), melangee
    remplies <- 81
    k <- 0
    tant que k < 81 et remplies > indices :
        (i, j) <- cases[k]
        retenir la valeur de la case (i, j), puis la vider
        si la grille a une solution unique :
            remplies <- remplies - 1      # on garde le trou
        sinon :
            remettre la valeur retenue    # on rebouche
        k <- k + 1
    renvoyer grille
```

Exercice 12 *Creuser en gardant l'unicité.*

1. *Sur le cahier.* Pourquoi travailler sur une **copie** de `grille_pleine` ? *(Penser à ce dont on aura besoin à l'étape 4.)*

2. *Sur le cahier.* Une grille a deux solutions ; on vide une case de plus. Combien de solutions a-t-elle *au moins* ? Justifier. En déduire pourquoi on teste l'unicité **après chaque retrait** (et on rebouche aussitôt), plutôt qu'une seule fois à la fin.

3. Écrire `creuser` (une ligne de code par ligne d'algorithme, ou presque). *Rappel :* `random.shuffle(cases)` mélange la liste ; la boucle `while` porte **les deux** conditions de poursuite (pas besoin de `break`).

4. *Sur le cahier.* Le programme garantit-il de laisser *exactement* `indices` cases remplies ? *(Envisager le cas où, à un moment, chacune des cases restantes est indispensable à l'unicité.)*

5. **Coût.** Au plus combien de fois `creuser` appelle-t-elle `compter_solutions` ?

Étape 4 — Assembler

Exercice 13 *La fonction* `generer(niveau)`.

1. Écrire `generer(niveau)`, qui renvoie le couple `(enonce, solution)`. Le dictionnaire `NIVEAUX` (fourni) donne le nombre d'indices à garder pour `"facile"`, `"moyen"` et `"difficile"`. Trois lignes suffisent, plus le `return`.

2. Lancer le fichier : il affiche une grille de chaque niveau. En recopier une et la résoudre à la main, ou la faire résoudre par votre voisin !

3. Le niveau « difficile » peut être **beaucoup** plus long à générer (de moins d'une seconde à une minute, selon le tirage). Expliquer pourquoi.

Le point-clé de la génération

Ce qui fait qu'une grille est un « vrai » Sudoku, ce n'est pas le nombre de trous : c'est que sa solution soit **unique**. C'est `compter_solutions` — donc encore le retour sur trace, mais en explorant *tout* l'arbre — qui le garantit à chaque retrait. Même méthode, trois usages : **trouver** une solution, en **fabriquer** une au hasard, les **compter**.

Remarque **« Niveau » : un mot piégeux.** Compter les cases vides est un *proxy* commode, mais la vraie difficulté dépend des *techniques de logique* nécessaires pour résoudre sans deviner — la mesurer précisément est un problème ouvert. Bel exemple d'un mot courant (« difficile ») dur à définir formellement.

10 Pour aller encore plus loin (au choix)

- **Trous symétriques** (comme dans les journaux) : retirer toujours `(i, j)` *et* `(8-i, 8-j)` en même temps.
- **Case la plus contrainte.** Au lieu de la *première* case vide, prendre la case vide qui a le **moins** de coups possibles. Le nombre d'essais s'effondre : c'est une idée **gloutonne** greffée sur le backtracking.

11 Bilan

- Le **retour sur trace** (*backtracking*) : essayer une piste, la poursuivre récursivement, **revenir en arrière** dès qu'elle échoue. C'est un **parcours en profondeur** de l'**arbre des essais** (un parcours que vous retrouverez avec les arbres et les graphes).
- Une fonction **récursive** a besoin d'un **cas de base** (ici : plus de case vide) pour être sûre de s'arrêter.
- La ligne qui **annule** le dernier choix (`grille[i][j] = 0`) est ce qui distingue le backtracking d'une simple boucle : c'est elle qui rend l'exploration **complète**.
- La même méthode résout des tas d'autres problèmes : labyrinthes, coloriage de cartes, placement de n reines sur un échiquier…

1

FICHE DE RÉVISION

L'essentiel

Une **fonction récursive** s'appelle elle-même sur un problème **strictement plus petit**.

- **Condition d'arrêt** (ou **cas de base**) : réponse directe, sans appel récursif ; **obligatoire**.
- **Cas récursif** : appel(s) sur un problème plus petit, qui se **rapproche** de la condition d'arrêt ; on **combine** les résultats.
- **Pile d'exécution** : chaque appel empile un cadre (ses variables locales) ; on *descend* jusqu'à la condition d'arrêt, puis on *remonte* en combinant.
- **Terminaison** : garantie par un **variant**, un entier positif qui décroît strictement à chaque appel et atteint la condition d'arrêt.

Le schéma clé — la pile d'appels de `fact(3)`

```
fact(3) = 3 * fact(2)
        = 3 * (2 * fact(1))
        = 3 * (2 * (1 * fact(0)))
        = 3 * (2 * (1 * 1))      <- condition d'arret : on remonte
        = 6
```

Trois fonctions à savoir réécrire les yeux fermés

```
1  def fact(n):                # entiers
2      if n == 0:
3          return 1
4      return n * fact(n - 1)
5
6  def somme_liste(tab):       # tab[0] + somme du reste
7      if tab == []:
8          return 0
9      return tab[0] + somme_liste(tab[1:])
10
11 def inverser(s):            # chaine a l'envers
12     if s == "":
13         return ""
14     return inverser(s[1:]) + s[0]
```

Sur ce modèle : `somme`, `puissance`, `pgcd`, *compter une lettre, maximum…*

Coût et choix récursif / itératif

La version naïve `fib(n) = fib(n - 1) + fib(n - 2)` **recalcule** sans cesse les mêmes valeurs : coût **exponentiel** (on verra au chapitre *Programmation dynamique* comment éviter ces **appels répétés**) ; `fact`, à un seul appel, a un coût **linéaire**. Le **récursif** est plus lisible sur les données récursives ; l'**itératif** (une boucle) évite le coût mémoire de la **pile**. Savoir **traduire** de l'un à l'autre.

Erreurs à éviter

- pas de condition d'arrêt, ou jamais atteinte ⇒ `RecursionError` ;
- `return` oublié devant l'appel ⇒ la fonction renvoie `None` ;
- résultat de l'appel non **combiné** (le `n *`, le `+ 1`, la concaténation oubliés).

Savoir-faire à maîtriser — je sais…

- ☐ identifier le **cas de base** et le **cas récursif** d'une fonction donnée ;
- ☐ écrire une fonction récursive sur les **entiers** (factorielle, somme, puissance, PGCD, produit par additions) ;
- ☐ écrire une fonction récursive sur une **chaîne** ou un **tableau** (compter, inverser, somme / maximum, recherche) ;
- ☐ **dérouler** la trace des appels (pile d'exécution) et donner le résultat ;
- ☐ **compter** le nombre d'appels effectués par une fonction récursive ;
- ☐ **justifier la terminaison** en exhibant un **variant** ;
- ☐ passer d'une version **récursive** à une version **itérative** (et inversement) ;
- ☐ reconnaître le **coût** d'une récursion et le piège des **appels répétés**.

Se tester — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 774

1. Quelles sont les deux parties indispensables d'une fonction récursive ?
2. `f(n)` exécute `f(n - 1)` *puis* `print(n)` lorsque `n > 0`. Qu'affiche `f(3)` ?
3. Combien d'appels (appel initial compris) déclenche `fact(5)` ? Quel est le coût ?
4. Une fonction récursive se termine mais renvoie `None` : erreur la plus probable ?
5. Pourquoi `fib(50)` naïf ne répond-il pas en un temps raisonnable ?

Et avant ? — *questions sur les chapitres précédents.* ▶ réponses p. 774

1. Pour `t = [4, 7, 2, 9]`, que valent `t[-1]`, `t[1:]` et `len(t[1:])` ? (Première)
2. Une fonction se termine par `print(res)` au lieu de `return res`. Que vaut `x` après `x = f(5)` ? (Première)
3. Deux boucles `for` imbriquées font chacune n tours. Combien de tours au total, et quel coût ? (Première)

1

DÉFI ADVENT OF CODE

DÉFI

Handy Haversacks

les sacs emboîtés — dictionnaires et récursivité

Jour 7

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t01-defi-aoc-2020-07` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/7`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Un règlement impose le contenu de chaque couleur de sac : tel sac doit contenir tant de sacs de telle couleur, qui doivent à leur tour en contenir d'autres. Avec près de six cents règles, il faut savoir quelles couleurs peuvent, au bout du compte, renfermer le sac doré du héros, puis combien de sacs ce sac doré doit contenir en tout. Les emboîtements sont trop profonds et trop nombreux pour être suivis à la main.
Ce qu'on réinvestit. La **récursivité** vue dans ce chapitre : un sac contient des sacs, qui contiennent eux-mêmes des sacs… La question posée pour un sac (« contient-il le sac doré ? », « combien de sacs contient-il ? ») se ramène à la même question posée pour les sacs qu'il contient. Une fonction récursive traduit directement ce raisonnement ; le sac vide fournit la condition d'arrêt.
La démarche de la fiche.

1. Comprendre ce qu'on cherche, puis dessiner les règles d'un petit exemple sous forme de schéma fléché.

2. Traduire les règles du fichier en un dictionnaire de dictionnaires.
3. Écrire une fonction récursive qui dit si une couleur peut contenir le sac doré : c'est la partie 1.
4. Partie 2 : une seconde fonction récursive, qui suit les règles dans l'autre sens et compte les sacs.
5. En approfondissement, inverser les règles, puis dresser l'inventaire du sac doré et compter les appels récursifs.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`bag`	sac	`rule`	règle
`contain`	contenir	`no other bags`	aucun autre sac (sac vide)
`outermost`	le plus à l'extérieur	`eventually`	au bout du compte, indirectement
`color-coded`	repéré par une couleur	`shiny gold`	or brillant

Résumé de l'énoncé (partie 1)

Contexte. Escale à l'aéroport : un nouveau règlement impose des sacs repérés par leur couleur, et chaque couleur de sac doit contenir un nombre précis de sacs d'autres couleurs. Le héros possède un sac `shiny gold` (or brillant) et se demande dans quels sacs il pourrait le ranger.
Les données. Chaque ligne du fichier est une règle qui décrit le contenu imposé d'une couleur de sac.

- Une règle a la forme `<couleur> bags contain <n> <couleur> bag(s), ...` ou bien `<couleur> bags contain no other bags.` (sac vide).
- Une couleur s'écrit en deux mots (`dull lime`) ; on lit `bag` ou `bags` selon le nombre, et la ligne se termine par un point.
- Chaque couleur possède exactement une règle.

La règle.

- Les sacs s'emboîtent : un sac contient des sacs, qui en contiennent d'autres, etc.
- On cherche les couleurs dont un sac contient au moins un sac `shiny gold`, directement ou à l'intérieur d'autres sacs, à n'importe quelle profondeur.
- Seule la possibilité compte : une couleur qui contient plusieurs sacs dorés n'est comptée qu'une fois, et les quantités ne jouent aucun rôle.
- `shiny gold` ne se compte pas lui-même.

La réponse attendue. Le nombre de ces couleurs. Par exemple, si un sac A contient un sac B, qui contient lui-même un sac doré, alors A et B comptent tous les deux : la réponse serait 2.

1

Exercice 1 *Comprendre ce qui est demandé*

But. Vérifier qu'on a compris ce qui fait compter une couleur, et repérer ce qu'il faut extraire d'une ligne.
À quoi ça sert ? La première question fait apparaître l'emboîtement à plusieurs niveaux, donc la récursivité ; la deuxième précise qu'on compte des couleurs et non des sacs ; la troisième prépare la lecture des règles.

Après lecture de l'énoncé et du résumé, vérifier sa compréhension :

1. Si un sac A contient un sac B qui contient un sac `shiny gold`, la couleur de A compte-t-elle ?
2. Une couleur dont un sac contient 5 sacs `shiny gold` compte-t-elle 5 fois ?
3. Quels morceaux d'une ligne faut-il écarter pour ne garder que des nombres et des couleurs ?

Exercice 2 *À la main, sur un petit exemple*

But. Représenter les règles par un schéma fléché, puis répondre à la partie 1 en remontant les flèches.
À quoi ça sert ? Le schéma rend visibles les emboîtements que le programme devra parcourir, et l'on obtient un exemple dont on **connaît la réponse** (3) pour tester les fonctions.

On considère les règles (inventées) suivantes :

```
pale cyan bags contain 2 shiny gold bags, 1 dull lime bag.
striped coral bags contain 1 pale cyan bag.
dull lime bags contain no other bags.
shiny gold bags contain 3 dull lime bags, 2 wavy ochre bags.
wavy ochre bags contain 4 dull lime bags.
clear navy bags contain 5 wavy ochre bags.
bold mauve bags contain 1 striped coral bag, 2 clear navy bags.
```

Faire un schéma : écrire chaque couleur, et tracer une flèche de A vers B, étiquetée n, si un sac A contient n sacs B. Répondre à la partie 1 en remontant les flèches depuis `shiny gold`. Vérification : on doit trouver `3`.

2 Programmer la partie 1

Outil Python : tester le début ou la fin d'une chaîne avec une tranche

On compare une tranche au morceau cherché : `s[:n]` contient les `n` premiers caractères, `s[-n:]` les `n` derniers.

```
reste = "no other bags."
print(reste[:8] == "no other")          # True : les 8 premiers caracteres
print(reste[-1:] == ".")                # True : le dernier caractere
print("2 dull lime bags"[:2] == "no")   # False
```

Intérêt. Pour ne pas compter les caractères à la main, écrire `reste[:len("no other")] == "no other"`. Une tranche plus longue que la chaîne ne provoque pas d'erreur : elle s'arrête à la fin.
À essayer. Que vaut `"bags"[:3] == "bag"` ? Et `"bag"[:4] == "bags"` ?

Exercice 3 *Les règles en dictionnaire*

But. Traduire chaque ligne du fichier en données Python : un dictionnaire qui associe à chaque couleur le contenu de son sac.
À quoi ça sert ? Le texte des règles est difficile à exploiter tel quel. Avec le dictionnaire `contenu`, on obtient en une instruction ce que contient un sac et en quelle quantité : les deux parties s'appuient dessus.

Écrire une fonction `lire_regle(ligne)` qui renvoie la couleur du contenant et un dictionnaire de son contenu, par exemple `("wavy ochre", {"dull lime": 4})`. Construire ensuite le dictionnaire `contenu` qui associe à chaque couleur le dictionnaire de son contenu (un dictionnaire de dictionnaires). ▷ coup de pouce

Méthode : « tous » ou « au moins un », avec un booléen drapeau

Pour savoir si **tous** les éléments d'une collection vérifient une condition, ou si **au moins un** la vérifie, on utilise un booléen appelé « drapeau » : on lui donne une valeur de départ, et la boucle la change dès qu'elle rencontre un élément décisif.

```
notes = [12, 15, 9]
tous_admis = True          # aucun contre-exemple pour l'instant
for n in notes:
    if n < 10:
        tous_admis = False
print(tous_admis)          # False : 9 ne convient pas
un_tres_bien = False       # aucun exemple pour l'instant
for n in notes:
    if n >= 15:
        un_tres_bien = True
print(un_tres_bien)        # True : 15 convient
```

Intérêt. Pour une collection vide, la boucle ne fait aucun tour : le drapeau garde sa valeur de départ (`True` pour « tous », `False` pour « au moins un »). Pour ne plus rien calculer une fois la réponse connue, on place le drapeau en tête de la condition, par exemple `if not un_tres_bien and n >= 15:` : quand le drapeau est déjà levé, Python n'évalue pas la suite du `and`.
À essayer. Un drapeau `trouve` vaut `False` avant la boucle `for x in {}:` (un sac vide). Que vaut-il après ? Qu'en déduire pour un sac vide dans `peut_contenir` ?

Exercice 4 *Peut contenir le sac doré ?*

But. Écrire une fonction récursive qui dit si un sac d'une couleur donnée peut renfermer, à une profondeur quelconque, le sac doré.
À quoi ça sert ? En l'appelant pour chaque couleur et en comptant les réponses vraies, on obtient la réponse de la partie 1. C'est l'occasion de bien identifier le cas de base et l'appel récursif.

Écrire une fonction récursive `peut_contenir(couleur)` qui renvoie `True` si un sac de cette couleur contient, directement ou non, un sac `shiny gold`. Compter les couleurs concernées. Tester sur l'exemple inventé de la fiche, puis sur vos données. ▷ coup de pouce ▷ début de solution

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Le héros se rend compte que son sac doré, une fois rempli selon le règlement, va peser lourd. On ne cherche plus les sacs qui contiennent le sac doré, mais ce que le sac doré contient ; et cette fois, les quantités comptent.
La nouvelle règle.

- On compte tous les sacs qu'un sac `shiny gold` doit contenir, à tous les niveaux d'emboîtement.
- On tient compte des quantités : si un sac contient 2 sacs B, et que chaque sac B contient d'autres sacs, il faut compter le contenu des deux sacs B.
- Le sac doré lui-même ne compte pas.

La réponse attendue. Le nombre total de sacs contenus dans le sac doré. Par exemple, si le sac doré contenait 2 sacs A, chacun contenant 3 sacs B vides, la réponse serait $2 + 2 \times 3 = 8$.

Exercice 5 *Dans l'autre sens*

But. Compter, par une seconde fonction récursive, tous les sacs contenus dans le sac doré.
À quoi ça sert ? C'est la réponse de la partie 2. Cette fois, on descend dans le contenu du sac doré et l'on tient compte des quantités : il faut bien compter chaque sac contenu, plus ce qu'il contient.

1. Dans la partie 1, on cherchait des sacs « au-dessus » du sac doré. Dans quel sens suit-on maintenant les règles ?

2. Écrire une fonction récursive `nb_sacs_dans(couleur)` qui renvoie le nombre total de sacs contenus dans un sac `couleur`. Sur l'exemple inventé de la fiche, on doit trouver `13`.

▷ coup de pouce ▷ début de solution

4 Approfondissement

★★★ **Exercice 6** *Défi : inverser les règles*

But. Inverser les règles (qui contient qui ?) pour résoudre la partie 1 en partant du sac doré.
À quoi ça sert ? Au lieu de tester toutes les couleurs, on ne visite que celles qui mènent au sac doré. On retrouve la réponse de la partie 1, ce qui valide les deux versions l'une par l'autre.

Pour chaque couleur, construire la liste des couleurs qui la contiennent **directement** (dictionnaire `contenants`). Écrire ensuite une fonction récursive `ajouter_contenants(couleur, trouves)` qui ajoute à la liste `trouves`, sans doublon, toutes les couleurs qui contiennent `couleur`, directement ou non. En déduire la réponse de la partie 1 et la comparer à celle de la version précédente.

▷ coup de pouce ▷ début de solution

★★ **Exercice 7** *Approfondissement : l'inventaire du sac doré*

But. Détailler le contenu du sac doré couleur par couleur, puis compter combien de fois la fonction récursive est appelée.
À quoi ça sert ? L'inventaire vérifie le total de la partie 2 de façon plus fine. Compter les appels montre que la récursivité naïve refait plusieurs fois les mêmes calculs, ce qui prépare la programmation dynamique.

1. À la main, sur l'exemple inventé de la fiche : combien de sacs de chaque couleur trouve-t-on, à tous les niveaux, dans un sac `shiny gold` ? Vérifier que le total redonne `13`.

2. Écrire une fonction récursive `inventaire(couleur)` qui renvoie un dictionnaire associant à chaque couleur le nombre de sacs de cette couleur contenus dans un sac `couleur`.

3. Combien de fois la fonction `nb_sacs_dans` de la partie 2 est-elle appelée pour `dull lime` sur l'exemple ? Vérifier en comptant les appels dans un dictionnaire.

4. On imagine des règles où chaque sac c_i contient un sac c_{i+1} et un sac c_{i+2}. Montrer que le nombre d'appels $A(i)$ pour c_i vérifie $A(i) = 1 + A(i+1) + A(i+2)$. Quel exemple du cours sur la récursivité retrouve-t-on ? (Le chapitre sur la programmation dynamique montrera comment éviter ces recalculs.)

▷ coup de pouce ▷ début de solution

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Couper d'abord sur `" bags contain "`. Si la suite commence par `"no other"`, le contenu est vide ; sinon, couper sur `", "` : chaque morceau commence par un nombre suivi de deux mots de couleur. ↑énoncé

Ex. 4 – piste Deux situations donnent la réponse sans appel récursif : le sac contient directement `"shiny gold"` (`True`), ou il est vide (`False`). Sinon, il suffit qu'**un** des sacs contenus puisse lui-même le contenir. ↑énoncé

Ex. 4 – début de solution Le résultat est vrai si, pour au moins un sac `x` du contenu, `x` est le sac doré **ou** `peut_contenir(x)` est vrai : c'est un « au moins un », à calculer avec un drapeau (voir l'encadré). Pour un sac vide, la boucle ne fait aucun tour et le drapeau reste à `False`. ↑énoncé

Ex. 5 – piste Un sac contenu compte-t-il lui-même, en plus de tout ce qu'il contient ? ↑énoncé

Ex. 5 – début de solution Pour un sac `A` qui contient `n` sacs `B` : ces `n` sacs comptent, et chacun d'eux apporte en plus tout son propre contenu. Le sac vide fournit le cas de base. ↑énoncé

Ex. 6 – piste Sur l'exemple inventé de la fiche, `contenants["shiny gold"]` vaut `["pale cyan"]` et `contenants["pale cyan"]` vaut `["striped coral"]`. ↑énoncé

Ex. 6 – début de solution Pour chaque couleur `c` de `contenants[couleur]` qui n'est pas encore dans `trouves` : l'ajouter, puis appeler la fonction sur `c`. La réponse est la longueur de `trouves`. ↑énoncé

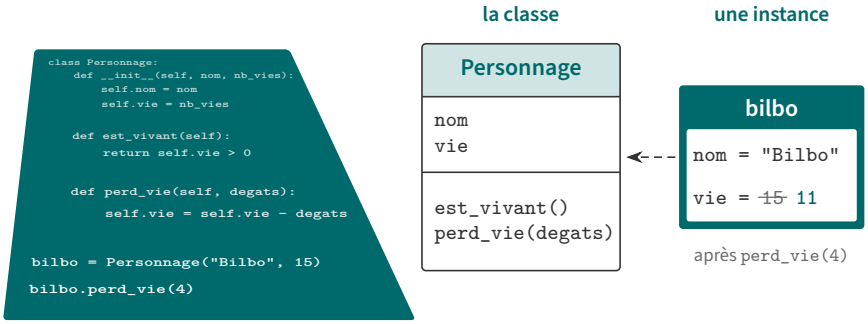
Ex. 7 – piste Un sac `A` qui contient `n` sacs `B` apporte `n` sacs `B`, plus `n` fois tout l'inventaire d'un sac `B`. ↑

Ex. 7 – début de solution Écrire une petite fonction `ajouter(total, couleur, nombre)` qui crée ou augmente l'entrée d'un dictionnaire ; `inventaire` parcourt `contenu[couleur].items()` et appelle `inventaire(x)` pour chaque sac contenu. ↑énoncé

2

CHAPITRE 2

Programmation orientée objet et paradigmes



DANS CE CHAPITRE

- Activités préparatoires . . . 67
- Cours . . . 69
- Exercices . . . 85
- TP et projets . . . 100
- Fiche de révision . . . 113
- Défis Advent of Code . . . 115
- Corrigés . . . 775

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 La fiche d'identité d'un objet

▶ corrigé p. 775

décrire ce qu'un objet sait et ce qu'il sait faire

Durée : 20 min · Sans ordinateur, par deux

Consignes

- On n'écrit **aucun code** : on décrit des objets de la vie courante **en français**, sur des fiches à deux colonnes dessinées sur le cahier : à gauche, ce que l'objet **sait** (ses caractéristiques et leur valeur) ; à droite, ce que l'objet **sait faire** (ses actions).
- Une **caractéristique** se note avec sa valeur actuelle (par exemple « couleur : verte ») ; une **action** se note par un verbe (par exemple « s'allumer »).

Exercice 1 — *Deux lampes*

Léa possède une lampe de bureau bleue, équipée d'une ampoule de 40 W ; elle est éteinte. Tom possède une lampe du **même modèle**, mais rouge, avec une ampoule de 60 W ; elle est allumée.

1. Sur le cahier, dessiner et remplir la fiche de la lampe de Léa : au moins trois caractéristiques avec leur valeur, et au moins trois actions qu'on peut lui faire faire.
2. Sans la dessiner, imaginer la fiche de la lampe de Tom. Qu'est-ce qui est **identique** sur les deux fiches ? Qu'est-ce qui **change** ?

Exercice 2 — *Le modèle commun*

3. Écrire une fiche « modèle » qui conviendrait à **toutes** les lampes de ce modèle. Qu'a-t-on gardé des fiches précédentes ? Qu'a-t-on enlevé ?
4. À l'usine, on fabrique une nouvelle lampe. Quelles informations faut-il donner au moment de la fabrication ? Quelle caractéristique a toujours la **même** valeur à la sortie de l'usine ?

Exercice 3 — *Les actions changent l'état*

5. La lampe de Léa est éteinte. On appuie **trois fois** sur son interrupteur. Quelle caractéristique a changé, et quelle est sa valeur finale ? Lesquelles n'ont pas bougé ?
6. L'action « changer l'ampoule » ne peut pas se faire sans une **information supplémentaire**. Laquelle ?
7. Classer les actions de votre fiche en deux groupes : celles qui **modifient** une caractéristique, et celles qui ne font que **donner une information** (par exemple « dire si elle éclaire »). Compléter la fiche si l'un des groupes est vide.
8. Sur la fiche d'une carte de cantine, la caractéristique « solde » vaut 12 €. Un camarade efface et écrit directement « solde : −50 ». Pourquoi est-ce gênant ? Comment l'éviter, si l'on n'a le droit de toucher au solde qu'à travers des actions ?

Exercice 4  *À vous*

9. Choisir un objet : un **réveil**, un **ascenseur** ou un **personnage de jeu vidéo**. Dessiner et remplir sur le cahier sa fiche modèle : trois caractéristiques, trois actions, dont **une** qui demande une information supplémentaire.

Exercice 5  *Ce qu'on a découvert*

10. Recopier et compléter sur le cahier, avec vos mots.

- Un objet est décrit à la fois par ...
- La fiche « modèle », commune à toutes les lampes, sert à ...
- La lampe de Léa et celle de Tom sont deux ...
- Une action peut ... une caractéristique, ou seulement ...

COURS

Programme (BO) *« Vocabulaire de la programmation objet : classes, attributs, méthodes, objets. Écrire la définition d'une classe. Accéder aux attributs et méthodes d'une classe. »* (*on n'aborde pas ici l'héritage ni le polymorphisme.*) *« Paradigmes de programmation. Distinguer sur des exemples les paradigmes impératif, fonctionnel et objet. Choisir le paradigme de programmation selon le champ d'application d'un programme. » « Modularité. Utiliser des API (Application Programming Interface) ou des bibliothèques. Exploiter leur documentation. Créer des modules simples et les documenter. »*

Prérequis fonctions et paramètres, variables, listes, dictionnaires, chaînes de caractères.
→ rappels de 1re : bases de Python p. 1056, listes p. 1057, dictionnaires p. 1058, chaînes p. 1057

Objectifs *savoir écrire* une classe (attributs, méthodes, constructeur) et *savoir l'utiliser* (créer des instances, appeler des méthodes) ; *distinguer* les paradigmes impératif, objet et fonctionnel ; *utiliser* une bibliothèque et sa documentation, *créer* un module documenté.

Situation de départ

1. Dans un jeu vidéo, des centaines de personnages ont chacun un nom et des points de vie, et savent tous attaquer ou se soigner. Comment les programmer sans recopier cent fois le même code ?
2. Pourquoi une application bancaire ne laisse-t-elle pas n'importe quelle partie du programme écrire directement un nouveau solde ?
3. Écrire une suite d'instructions exécutées l'une après l'autre : est-ce la seule façon de programmer ?

I Programmer autrement : la notion de paradigme

Depuis la classe de Première, vous programmez d'une seule manière : vous écrivez une **suite d'instructions** qui manipulent des variables, l'une après l'autre. C'est ce qu'on appelle un *style* de programmation, ou plus savamment un **paradigme**.

Définition 1 — Paradigme de programmation

Un **paradigme de programmation** est une manière de concevoir et d'organiser un programme. Un même langage (Python, par exemple) peut en autoriser **plusieurs**, et un même programme peut les **mélanger**.

Le style que vous pratiquez depuis la Première porte un nom : le paradigme **impératif**. Un programme impératif est une liste d'**ordres** donnés à la machine, qui font évoluer pas à pas l'**état** de la mémoire. Ce petit programme, qui compte les notes au-dessus de la moyenne, en réunit tous les ingrédients :

```
1 notes = [12, 8, 15, 9, 17]
2 compteur = 0                      # INITIALISER : une variable qui va changer
3 for n in notes:                   # PARCOURIR : on repete un bloc...
4     if n >= 10:                   # TESTER : ...selon une condition
5         compteur = compteur + 1   # METTRE A JOUR
```

```
print(compteur)                    # ordres executes du haut vers le bas
```

▷ **Je prédis.** Qu'affiche ce programme ?

▷ **Je vérifie.** `3` : seules les notes `12`, `15` et `17` sont supérieures ou égales à 10.

On y retrouve des **affectations** qui modifient des variables, des **boucles**, des **tests**, et un ordre d'exécution qui compte : déplacer le `print` au début changerait le résultat.

Ce n'est pas le seul, et le programme de Terminale demande d'en découvrir deux autres : le paradigme **fonctionnel** et le paradigme **objet**. Nous commencerons par le **fonctionnel**, le plus léger — il ne demande que des fonctions, que vous connaissez déjà —, avant de consacrer le cœur du chapitre au paradigme **objet**, la grande nouveauté de l'année.

II Un premier autre regard : le paradigme fonctionnel

Commençons par un paradigme qui, sur la forme, ne demande rien de neuf — il n'utilise que des **fonctions** — mais propose un regard neuf : le paradigme **fonctionnel** cherche à **bannir les effets de bord**.

Définition 2 — Effet de bord

Une fonction produit un **effet de bord** lorsqu'elle modifie quelque chose *en dehors* d'elle-même : une variable globale, une liste reçue en paramètre, un affichage, un fichier… bref, autre chose que sa seule valeur de retour.

Les effets de bord rendent les programmes difficiles à prévoir et à relire : à un instant donné, on ne sait plus quelle fonction a modifié quoi. Le paradigme fonctionnel les évite au maximum en s'appuyant sur des **fonctions pures**.

Définition 3 — Fonction pure

Une fonction est **pure** si :

- sa valeur de retour dépend **uniquement** de ses paramètres (jamais d'une variable extérieure) ;
- elle ne produit **aucun effet de bord**.

Appelée deux fois avec les mêmes arguments, elle renvoie toujours le même résultat.

Exemple — *Analysez ces deux fonctions. Laquelle est pure ?*

```
seuil = 5
def depasse_A(x):
    return x > seuil

def depasse_B(x, seuil):
    return x > seuil
```

▶ **Correction.** `depasse_A` n'est pas pure : elle dépend de `seuil`, une variable **extérieure**, et son résultat change si l'on modifie `seuil` ailleurs. `depasse_B` est pure : elle ne dépend que de ses paramètres.

Exemple — *Éviter l'effet de bord sur une liste*
La méthode `append` **modifie** la liste reçue : c'est un effet de bord. La version fonctionnelle **crée une nouvelle liste** sans toucher à l'ancienne.

```
def ajout_impur(element, tab):
    tab.append(element)          # effet de bord : tab est modifie

def ajout_pur(element, tab):
    return tab + [element]       # cree une nouvelle liste, tab intact
```

▷ **Je prédis.** Avec `t = [1, 2]`, que renvoie `ajout_pur(3, t)`, et que vaut `t` ensuite ? Même question avec `ajout_impur(3, t)`.
▷ **Je vérifie.** `ajout_pur(3, t)` renvoie `[1, 2, 3]` et `t` reste `[1, 2]`. `ajout_impur(3, t)` renvoie `None` (pas de `return`) mais `t` devient `[1, 2, 3]` : le changement est passé par l'effet de bord.

Pour les curieux (facultatif)
Certains langages sont conçus pour *favoriser* ce style (Haskell, Lisp, OCaml, Scheme ; Haskell va jusqu'à l'imposer), mais on peut programmer « fonctionnel » dans un langage généraliste comme Python. Le paradigme fonctionnel s'appuie aussi beaucoup sur des données **immuables** (tuples plutôt que listes) et sur des fonctions qui en prennent d'autres en paramètre (*ex.* une fonction qui applique une autre fonction à chaque élément d'une liste).

Remarque — *Vous en faites déjà : la création de liste par compréhension*
La **création de liste par compréhension**, vue en Première, relève de la **programmation fonctionnelle** : c'est une **expression** qui construit une **nouvelle** liste, sans modifier la liste parcourue ni aucune variable extérieure (pas d'effet de bord). Elle sait faire les deux opérations de base du style fonctionnel sur une liste : **appliquer** une fonction à chaque élément, et **garder** certains éléments.

Exemple — *Appliquer et garder avec une compréhension*
Avec `nombres = [1, 2, 3, 4, 5, 6]`, que valent les deux expressions ci-dessous ? La liste `nombres` est-elle modifiée ?

```
def carre(x):
    return x * x

def est_pair(x):
    return x % 2 == 0

[carre(x) for x in nombres]          # appliquer carre a chaque element
[x for x in nombres if est_pair(x)]  # garder les elements pairs
```

▶ **Correction.** La première vaut `[1, 4, 9, 16, 25, 36]`, la seconde `[2, 4, 6]`. Chaque compréhension crée une **nouvelle** liste : `nombres` reste `[1, 2, 3, 4, 5, 6]`.

Remarque — *Les fonctions anonymes :* `lambda`
Quand la fonction ne sert qu'une fois, on peut l'écrire directement, sans lui donner de nom, avec le mot-clé `lambda` (hérité du λ-calcul d'Alonzo Church) : `lambda x: x * x` se lit « la fonction qui, à `x`, associe `x * x` ». On l'a déjà rencontrée en Première pour **trier une table** : `sorted` reçoit, avec `key`, la fonction qui donne la valeur à comparer.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
notes = [("Ada", 15), ("Alan", 12), ("Grace", 18)]
sorted(notes, key=lambda c: c[1])   # [('Alan', 12), ('Ada', 15), ('Grace', 18)]
```

Ici, `lambda c: c[1]` équivaut à `def note(c): return c[1]`, écrite sur place. Une `lambda` ne contient qu'une **expression** (pas d'instruction, pas de `return`) : pour une fonction plus longue,

on revient à `def`.

► **Exercices d'application :** exercices 1 à 3 (fonctions pures ; défi : des fonctions qui manipulent des fonctions)

III Objets, classes et instances

Remarque

Bonne nouvelle : vous utilisez déjà des objets *sans le savoir* ! Quand vous écrivez `"le petit chat".split()` ou `ma_liste.append(3)`, le `.quelquechose()` est un appel de **méthode** sur un **objet**. Il est temps de comprendre — et d'écrire — ce qui se cache derrière.

A. L'idée : réunir des données et des comportements

Prenons un objet de la vie courante très complexe — disons le **moteur d'une voiture**. Le conducteur n'a pas besoin d'en connaître le fonctionnement interne : il dispose d'une clé, d'une pédale, d'un compteur. Toute la complexité est **enfermée dans une caisse**, et on l'utilise par quelques commandes simples.

Programmer « objet », c'est reprendre cette idée : on regroupe dans une même entité les **données** qui décrivent une chose et les **actions** qu'on peut lui appliquer, puis on l'utilise sans se soucier de l'intérieur.

Définition 4 — Objet

Un **objet** est une entité informatique qui réunit :

- des **attributs** : des données qui décrivent son *état* (un peu comme des variables attachées à l'objet) ;
- des **méthodes** : des fonctions attachées à l'objet, qui décrivent son *comportement*.

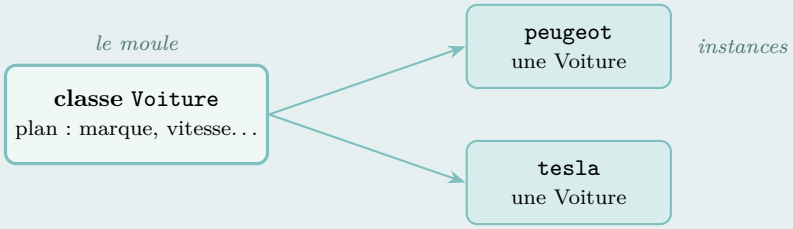
B. La classe : le plan de fabrication

On ne décrit pas chaque objet un par un : on écrit un **plan** commun à tous les objets du même genre. Ce plan s'appelle une **classe**.

Définition 5 — Classe et instance

Une **classe** est le modèle (le « plan ») qui décrit les attributs et les méthodes communs à toute une famille d'objets. Un objet construit à partir d'une classe est appelé une **instance** de cette classe.

↻ **Retour sur l'activité.** Les deux colonnes des fiches, « ce qu'il sait » et « ce qu'il sait faire », étaient les attributs et les méthodes ; la fiche « modèle » commune à toutes les lampes était une classe, et la lampe bleue de Léa et la lampe rouge de Tom en sont deux instances : mêmes attributs, valeurs différentes.



Remarque
Vous connaissez déjà des classes ! En Python, `str`, `int`, `list` sont des classes. La chaîne `"radar"` est une *instance* de `str`, la liste `[1, 2]` une *instance* de `list`. L'instruction `type("radar")` renvoie d'ailleurs `<class 'str'>`.

C. Écrire une classe (presque) vide

En Python, une classe se déclare avec le mot-clé `class`, suivi d'un nom (par convention avec une **majuscule**). On crée ensuite une instance en écrivant `NomDeLaClasse()`.

Exemple — *Saisissez, analysez et testez ce code*

```
class Personnage:
    pass          # "ne rien faire" : la classe est pour l'instant vide

gollum = Personnage()    # gollum est une instance de Personnage
bilbo = Personnage()     # bilbo en est une autre
```

▶ **Correction.** Le programme n'affiche rien. L'instruction `pass` signifie « ne rien faire » : une classe ne peut pas être totalement vide. Nous avons créé **deux** instances distinctes de `Personnage` (`print(gollum)` affiche `<__main__.Personnage object at 0x...>`).

Ces instances sont pour l'instant inutiles : donnons-leur un état.

▶ **Exercices d'application :** exercices 4 à 6 (lire une classe)

IV Les attributs : l'état d'un objet

Un **attribut** est une donnée attachée à une instance. On y accède avec la notation **pointée** : `gollum.vie` désigne l'attribut `vie` de l'instance `gollum`. Mais où et comment créer proprement cet attribut ?

A. Le constructeur `__init__` et le mot `self`

Les attributs d'une instance sont mis en place par une méthode spéciale, le **constructeur**, nommée `__init__` (deux tirets bas de chaque côté). Elle est appelée **automatiquement** au moment de la création de l'instance.

Règle 1 — Le mot `self`

Dans une classe, `self` désigne **l'instance en train d'être manipulée**. C'est **toujours** le premier paramètre d'une méthode. Écrire `self.vie = 20` crée l'attribut `vie` *de cette instance*. Quand vous écrivez ensuite `gollum.vie`, Python a remplacé `self` par `gollum`.

Exemple — *Saisissez, analysez et testez ce code*

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
class Personnage:
    def __init__(self):
        self.vie = 20

gollum = Personnage()
bilbo = Personnage()
print(gollum.vie)
```

▶ **Correction.** Le programme affiche 20 : le constructeur, appelé automatiquement à la création, fait naître tout `Personnage` avec 20 points de vie. Chaque instance possède **ses propres** attributs : `gollum.vie` et `bilbo.vie` sont indépendants.

B. Des instances qui diffèrent : les paramètres du constructeur

Ici, tous les personnages naissent avec 20 points de vie. Pour que chaque instance soit différente, on passe des **paramètres** au constructeur, exactement comme à une fonction.

Exemple — *Saisissez, analysez et testez ce code*

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
class Personnage:
    def __init__(self, nom, nb_vies):
        self.nom = nom          # attribut : le nom
        self.vie = nb_vies      # attribut : les points de vie

gollum = Personnage("Gollum", 20)
bilbo = Personnage("Bilbo", 15)
print(bilbo.nom, bilbo.vie)
```

▶ **Correction.** Le programme affiche `Bilbo 15`. Au moment de `Personnage("Bilbo", 15)`, la valeur `"Bilbo"` est reçue par le paramètre `nom` et `15` par `nb_vies`. Attention : `nom` et `nb_vies` sont de simples *paramètres* (ils ne commencent pas par `self`) ; ce sont les lignes `self.nom = ...` qui créent les *attributs*.

↻ **Retour sur l'activité.** Les informations à donner « à l'usine » (la couleur, la puissance de l'ampoule) sont les paramètres du constructeur ; l'état « éteinte » de toute lampe neuve est un attribut fixé dans `__init__` sans paramètre, comme `self.vie = 20` plus haut.

▶ **Exercices d'application :** exercices 7 **et** 8 (créer une instance, compléter un constructeur)

V Les méthodes : le comportement d'un objet

Définition 6 — Méthode

Une **méthode** est une fonction définie **à l'intérieur** d'une classe. Elle agit sur l'instance grâce à `self`. On l'appelle avec la notation pointée : `bilbo.perd_vie(4)` (`self` n'apparaît pas dans les parenthèses : seuls les autres paramètres, ici `degats`, reçoivent une valeur).

Distinguons deux rôles très courants : une méthode qui **renvoie** une information sur l'état, et une méthode qui **modifie** l'état.

Exemple — *Saisissez, analysez et testez ce code*

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
class Personnage:
    def __init__(self, nom, nb_vies):
        self.nom = nom
        self.vie = nb_vies

    def est_vivant(self):           # renvoie une information
        return self.vie > 0

    def perd_vie(self, degats):     # modifie l'etat
        self.vie = self.vie - degats

bilbo = Personnage("Bilbo", 15)
bilbo.perd_vie(4)
print(bilbo.vie, bilbo.est_vivant())
```

▶ **Correction.** Le programme affiche `11 True`. `perd_vie(4)` **modifie** l'état : `bilbo.vie` passe

de 15 à 11. `est_vivant()` ne modifie rien, elle **renvoie** une information : `11 > 0`, donc `True`.

À retenir — fonction ou méthode ?

Une **fonction** s'appelle seule : `len(bilbo_liste)`. Une **méthode** s'appelle « sur » un objet, après un point : `bilbo_liste.append(3)`. L'objet placé à gauche du point est passé, en coulisses, comme `self`.

A. Un second exemple : des objets qui interagissent

Une méthode peut recevoir en paramètre **un autre objet**. Illustrons-le avec une classe `Point` du plan.

Exemple — *La classe* `Point`

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
1  class Point:
2      def __init__(self, x, y):
3          self.x = x
4          self.y = y
5
6      def distance(self, autre):      # autre est un autre Point
7          dx = self.x - autre.x
8          dy = self.y - autre.y
9          return (dx ** 2 + dy ** 2) ** 0.5
10
11 a = Point(0, 0)
12 b = Point(3, 4)
13 print(a.distance(b))                 # affiche 5.0
```

Ici `a.distance(b)` : `self` vaut `a`, le paramètre `autre` vaut `b`.

► **Exercices d'application :** exercices 9 à 11 (écrire des méthodes ; classes `Rectangle` et `Chronometre`)

VI L'encapsulation : protéger l'état

Techniquement, rien n'empêche d'écrire directement `bilbo.vie = -999` depuis l'extérieur. Mais c'est une **mauvaise pratique** : on court-circuite la classe et on peut mettre l'objet dans un état absurde.

Définition 7 — Encapsulation

L'**encapsulation** consiste à *cacher* l'état interne d'un objet et à ne le laisser manipuler qu'à travers ses méthodes. L'utilisateur de la classe se sert des méthodes (les « boutons ») sans toucher directement aux attributs.

↻ **Retour sur l'activité.** Le camarade qui écrivait directement « solde : −50 » sur la fiche de la carte de cantine court-circuitait l'objet ; n'autoriser que les actions « recharger » et « payer », qui vérifient l'opération, c'est l'encapsulation.

Remarque

C'est l'esprit du paradigme fonctionnel (vu en début de chapitre) transposé aux objets : là, on protégeait le programme des **effets de bord** ; ici, on protège l'**état d'un objet** des modifications sauvages. Deux façons de garder la maîtrise de ses données.

Reprenons l'exemple le plus classique : un **compte bancaire**. On ne veut surtout pas qu'un programme extérieur écrive `compte.solde = 1000000` : tout dépôt doit passer par une méthode qui contrôle l'opération.

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
class CompteBancaire:
    def __init__(self, titulaire, solde_initial):
        self.titulaire = titulaire
        self._solde = solde_initial     # _solde : "ne pas toucher de l'exterieur"

    def solde(self):                    # accesseur : lire l'etat
        return self._solde

    def deposer(self, montant):
        if montant > 0:
            self._solde = self._solde + montant

    def retirer(self, montant):
        if 0 < montant <= self._solde:  # on refuse de passer en negatif
            self._solde = self._solde - montant
            return True
        return False

compte = CompteBancaire("Alice", 100)
compte.deposer(50)
compte.retirer(30)
print(compte.solde())                   # affiche 120
```

▷ **Je prédis.** On ajoute à la fin `compte.retirer(500)` puis `compte.deposer(-10)`. Que renvoie `retirer`, et que vaut alors `compte.solde()` ?

▷ **Je vérifie.** `retirer(500)` renvoie `False` (le solde ne suffit pas) et `deposer(-10)` est ignoré en silence : le solde vaut toujours `120`. Les méthodes ont protégé l'état.

Règle 2 — La convention du tiret bas

En Python, on **préfixe d'un tiret bas** (`_solde`) un attribut qui n'est pas censé être utilisé de l'extérieur. C'est une **convention** entre programmeurs : Python ne l'interdit pas techniquement, mais tout le monde comprend « ceci est interne, manipule-le par les méthodes ». Une méthode qui *lit* un attribut s'appelle un **accesseur** (ici `solde`) ; une méthode qui le *modifie*, un **mutateur**.

Au bac — où utiliser accesseurs et mutateurs ?

Les sujets de bac fournissent souvent des méthodes `get_...` (accesseurs) et `set_...` (mutateurs). Faut-il s'en servir partout ?

- **Depuis l'extérieur de la classe** (une autre classe, le programme principal) : **toujours** par les méthodes. C'est même obligatoire quand l'énoncé demande de « ne pas accéder directement aux attributs » (*ex.* une méthode de la classe `Livreur` qui reçoit un objet `Adresse` muni de l'accesseur `get_ville()` écrit `adresse.get_ville()`, et non `adresse._ville`).
- **À l'intérieur de la classe** : écrire `self.attribut` est **correct**, et c'est l'usage le plus courant (`deposer` et `retirer` ci-dessus écrivent `self._solde`). L'encapsulation sépare la classe de l'*extérieur*, pas une méthode de son propre objet : ce sont justement les méthodes de la classe qui garantissent que l'état reste cohérent. Cela vaut aussi pour un autre objet *de la même classe* (`autre.x` dans `Point.distance`).

Les sujets eux-mêmes ne sont pas homogènes : dans Asie 2025, jour 2 (classe `Date`), la méthode `nb_jours_passes` lit `self.jour` et `self.mois` directement, tandis que `nb_jours_depuis` écrit `self.get_annee()` et `other.get_annee()`. Les deux écritures sont justes : c'est une habitude de style, pas une règle.

Quand passer par le mutateur, même à l'intérieur ? Quand il contient une **vérification** : en l'appelant partout, le contrôle n'est écrit qu'une fois et s'applique à toutes les modifications.

```
class Personnage:
    def __init__(self, nom, nb_vies):
        self.nom = nom
        self._vie = nb_vies

    def set_vie(self, v):           # mutateur : le seul endroit qui verifie
        if v < 0:
            v = 0                   # jamais de vie negative
        self._vie = v

    def perd_vie(self, degats):
        self.set_vie(self._vie - degats)

    def soigne(self, soin):
        self.set_vie(self._vie + soin)
```

▷ **Je prédis.** Avec `p = Personnage("Sam", 5)`, que vaut `p._vie` après `p.perd_vie(8)`, puis après `p.soigne(3)` ?

▷ **Je vérifie.** `0`, puis `3` (et non `0`) : le mutateur a ramené -3 à `0`, et le soin repart de `0`.

Si le mutateur se contente de `self.x = v`, passer par lui à l'intérieur n'apporte rien. Dans `__init__`, on affecte directement les attributs. En Python, les programmeurs écrivent d'ailleurs rarement des `get_`/`set_` (ils utilisent des « propriétés », `@property`) [au-delà du programme] : les sujets s'en servent surtout pour faire pratiquer le vocabulaire, commun à tous les langages objet.

Ce qu'on attend de vous. Hors de la classe : les méthodes. Dans la classe : `self.attribut` est accepté, mais si le code à compléter utilise déjà les accesseurs, **imitez son style**, et si un mutateur vérifie quelque chose, passez par lui. Un correcteur ne pénalise pas `self.annee` dans une méthode de `Date`, sauf consigne contraire de l'énoncé.

A. Signaler une erreur : `raise`

Dans `deposer`, un montant négatif est ignoré **en silence** : l'appelant ne sait pas que son ordre n'a pas été exécuté. Il vaut souvent mieux **lever une erreur** (on dit aussi « lever une exception ») avec le mot-clé `raise` :

```
    def deposer(self, montant):
        if montant <= 0:
            raise ValueError("montant invalide")
        self._solde = self._solde + montant
```

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> compte.deposer(-5)
Traceback (most recent call last):
  ...
ValueError: montant invalide
```

Règle 3 — Lever une erreur

- **Syntaxe** : `raise ValueError("message")`. La fonction s'arrête **immédiatement** (les lignes suivantes ne sont pas exécutées), puis le programme s'arrête à son tour en affichant le type d'erreur et le message.
- **Types courants** : `ValueError` (valeur incorrecte : montant négatif, carte inconnue…), `IndexError` (indice hors limites, pile vide), `TypeError` (type incorrect).
- **Quand l'utiliser ?** Quand l'appelant demande quelque chose d'**impossible** : la précondition n'est pas respectée, et renvoyer une valeur « bidon » (`None`, `-1`…) risquerait de passer inaperçu.
- **Différence avec `assert`** (vu en Première) : `assert condition, "message"` sert au **programmeur** à vérifier, pendant la mise au point et dans les tests, ce qui *doit* être vrai ; il lève `AssertionError` et peut être désactivé. `raise`, lui, fait partie du **comportement prévu** de la fonction : il se documente dans la docstring, comme une sortie possible.

► **Exercices d'application :** exercice 12 (encapsulation)

VII Pour aller plus loin : les méthodes spéciales

Créons un point et tapons simplement son nom dans la console pour « le voir » :

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> p = Point(3, 4)
>>> p
<__main__.Point object at 0x7f3a...>
```

Illisible ! Par défaut, un objet ne sait pas se décrire. On corrige cela en définissant la méthode spéciale `__repr__`, qui indique **comment représenter** l'objet.

Exemple — *Un affichage lisible avec* `__repr__`

```
1  class Point:
2      def __init__(self, x, y):
3          self.x = x
4          self.y = y
5
6      def __repr__(self):
7          return f"Point({self.x}, {self.y})"
```

Désormais, taper le nom de l'objet suffit — et `print` en profite aussi :

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> p = Point(3, 4)
>>> p                       # dans la console : appelle __repr__
Point(3, 4)
>>> print(p)                # print s'en sert, faute de __str__
Point(3, 4)
```

Règle 4 — `__repr__` ou `__str__` ?

Deux méthodes gouvernent l'affichage d'un objet :

- `__repr__` est appelée quand on **tape le nom** de l'objet dans la console (et par la fonction `repr()`). C'est la représentation « officielle », sans ambiguïté : par convention, on renvoie idéalement une expression qui *recréerait* l'objet, comme `Point(3, 4)`.
- `__str__` est appelée par `print()` et `str()`. C'est la représentation « lisible », destinée à l'utilisateur final.

Règle d'or : si `__str__` n'existe pas, `print()` se rabat sur `__repr__`. Définir le seul `__repr__` suffit donc à obtenir un affichage propre *partout* — dans la console *comme* avec `print`. L'inverse est faux : `__str__` seule ne change rien quand on tape le nom dans la console. Pour simplement « voir » un objet, `__repr__` est donc le réflexe le plus direct.

Pour les curieux (facultatif)

`__init__`, `__repr__` et `__str__` font partie des méthodes dites *spéciales* (entourées de doubles tirets bas). Il en existe d'autres, hors programme, qui permettent par exemple d'utiliser `+` ou `==` sur vos propres objets. De même, l'**héritage** (créer une classe à partir d'une autre) et le **polymorphisme** sont *explicitement hors du programme* de Terminale : nous nous arrêtons ici.

▶ **Exercices d'application :** exercices 13 (défi : la classe `Fraction`) et 14 (défi : un réseau d'amis, des objets qui interagissent)

VIII Choisir — et mélanger — les paradigmes

Aucun paradigme n'est « meilleur » dans l'absolu : un bon programmeur choisit selon la situation, et peut même les combiner dans un seul programme.

Paradigme	Idée centrale	Adapté à…
Impératif	une suite d'instructions qui modifient l'état	algorithmes, calculs pas à pas
Objet	regrouper données + comportements	modéliser des entités (jeux, simulations)
Fonctionnel	des fonctions pures, sans effet de bord	transformations de données, code sûr

Exemple — *Un même programme peut mêler les trois. Repérez-les dans ce combat*

```
1  import random
2
3  class Personnage:
4      def __init__(self, nom, vie):
5          self.nom = nom
6          self.vie = vie
7      def perd_vie(self):
8          self.vie = self.vie - random.randint(1, 2)
9
10 def combat(a, b):
11     while a.vie > 0 and b.vie > 0:
12         a.perd_vie()
13         b.perd_vie()
14     if a.vie > 0:
15         return a.nom
16     elif b.vie > 0:
17         return b.nom
18     return "egalite"
```

▶ **Correction.** On modélise les combattants avec des **objets** (la classe `Personnage`), on déroule le combat avec une boucle et des conditions **impératives** (la fonction `combat`), et l'on pourrait

analyser les résultats avec des fonctions **pures**. Attention : `combat` n'est *pas* pure, car elle modifie `a.vie` et `b.vie` (effet de bord) et son résultat dépend du hasard.

A. Un même problème, plusieurs langages

Un paradigme est une **façon de penser** un programme, et chaque langage en **encourage** un (sans forcément l'imposer). Un même calcul s'écrit donc très différemment selon le langage choisi. Prenons la **factorielle** $n! = 1 \times 2 \times \cdots \times n$.

Python, ici en style **impératif** (une boucle qui met à jour un accumulateur) :

```python
def fact(n):
    r = 1                       # INITIALISER : element neutre du produit
    for k in range(2, n + 1):   # PARCOURIR
        r = r * k               # METTRE A JOUR
    return r                    # RENVOYER
```

▷ **Je prédis.** Que renvoient `fact(4)` et `fact(0)` ?
▷ **Je vérifie.** `24`, puis `1` : `range(2, 1)` est vide, la boucle ne tourne pas et l'on renvoie la valeur initiale (par convention, $0! = 1$).

OCaml, en style **fonctionnel** (une définition récursive, comme au chapitre 1, sans variable modifiée) :

```ocaml
let rec fact n =
  if n = 0 then 1 else n * fact (n - 1)
```

Prolog *(hors programme, pour la culture)*, en style **logique** : on ne décrit pas *comment* calculer, mais *ce qui est vrai*, et le langage cherche la réponse.

```prolog
fact(0, 1).
fact(N, F) :- N > 0, M is N - 1, fact(M, G), F is N * G.
```

On interroge alors : `?- fact(4, X).` et Prolog trouve `X = 24`.

Pour les curieux — *Au-delà des paradigmes : deux autres façons dont les langages diffèrent* (facultatif)
En plus du paradigme qu'ils encouragent, les langages se distinguent notamment par :

- la **façon d'être exécutés** : un langage **compilé** (C, OCaml) est entièrement traduit une fois pour toutes en un exécutable rapide ; un langage **interprété** (Python) est d'abord traduit en un code intermédiaire, qu'un programme appelé **interpréteur** exécute ensuite instruction par instruction, ce qui est plus souple mais plus lent ;
- le **typage** : **statique** (C, OCaml : le type de chaque variable est vérifié *avant* l'exécution) ou **dynamique** (Python : le type est déterminé *pendant* l'exécution).

Aucun de ces choix n'est « meilleur » : ce sont des compromis entre rapidité, sûreté et souplesse. *(Cette diversité est étudiée en détail dès la Première.)*

Pour les curieux — *Un peu d'histoire* (facultatif)
Les idées du paradigme objet naissent en **1967** avec le langage **Simula**, conçu en Norvège par **Ole-Johan Dahl** et **Kristen Nygaard** pour *simuler* des systèmes réels (files d'attente, navires) : d'où les objets. Le terme « orienté objet » est popularisé dans les années 1970 par **Alan Kay** avec **Smalltalk**. Dahl, Nygaard et Kay ont chacun reçu le prix Turing. Le paradigme fonctionnel, lui, plonge ses racines dans le **λ-calcul** d'**Alonzo Church** (1936), un modèle de calcul purement mathématique, concrétisé dès 1958 par le langage **Lisp** de John McCarthy.

▶ **Exercices d'application :** exercice 15 (choisir le bon paradigme)

IX Modularité : utiliser et créer des modules

Un vrai programme compte des milliers de lignes : on ne l'écrit pas d'un seul bloc. On le découpe en **modules**, des fichiers `.py` qui regroupent chacun les fonctions et les classes d'un même thème. Vous vous en servez déjà : `import random` ou `import math` chargent des modules écrits par d'autres.

A. Utiliser une bibliothèque et lire sa documentation

Définition 8 — Module, bibliothèque, API

Un **module** est un fichier Python que l'on peut **importer** dans un autre programme ; une **bibliothèque** est un ensemble de modules (la bibliothèque standard de Python en compte des centaines). L'**API** (*Application Programming Interface*, interface de programmation) d'une bibliothèque est la liste de ce qu'elle met à disposition (fonctions, classes, constantes) avec leur **spécification** : ce que chacune reçoit, ce qu'elle renvoie, les erreurs qu'elle peut lever.

```
import math                  # on importe tout le module...
print(math.sqrt(2))          # ... et on prefixe : math.sqrt

from random import randint   # on importe un seul nom...
de = randint(1, 6)           # ... utilisable sans prefixe
```

Pour utiliser une fonction, on n'a pas besoin de son code : sa **documentation** suffit. Dans la console, `help(nom)` affiche la docstring de la fonction, du module ou de la classe ; la documentation complète de Python est en ligne (`docs.python.org`).

```
>>> help(randint)
randint(a, b) method of random.Random instance
    Return random integer in range [a, b], including both end points.
```

Exemple — *D'après l'aide ci-dessus,* `randint(1, 6)` *peut-elle renvoyer 6 ?*

▶ **Correction.** Oui : la documentation dit « *including both end points* », les deux bornes sont incluses. `randint(1, 6)` renvoie un entier de 1 à 6, comme un dé, alors que `range(1, 6)` s'arrête à 5. Lire la documentation évite ce genre de piège.

Remarque

Une API, c'est l'**encapsulation** à l'échelle d'une bibliothèque : on se sert des fonctions par leur spécification, sans regarder l'intérieur. Les auteurs peuvent alors améliorer leur code sans casser les programmes de ceux qui l'utilisent.

B. Créer et documenter son propre module

Créer un module, c'est simplement écrire un fichier `.py`. Rangeons la classe `CompteBancaire` dans un fichier `banque.py` :

```
"""Module banque : des comptes bancaires simples."""

class CompteBancaire:
    """Un compte avec un titulaire et un solde."""

    def __init__(self, titulaire, solde_initial):
        self.titulaire = titulaire
        self._solde = solde_initial

```

```
    def solde(self):
        """Renvoie le solde du compte."""
        return self._solde

    def deposer(self, montant):
        """Ajoute montant (> 0) au solde ; ValueError sinon."""
        if montant <= 0:
            raise ValueError("montant invalide")
        self._solde = self._solde + montant

if __name__ == "__main__":    # tests : seulement si on lance banque.py
    c = CompteBancaire("Alice", 100)
    c.deposer(50)
    assert c.solde() == 150
    print("Tests du module banque : OK")
```

Un autre programme, placé **dans le même dossier**, l'utilise comme n'importe quel module :

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
from banque import CompteBancaire

compte = CompteBancaire("Bob", 20)
compte.deposer(5)
print(compte.solde())          # affiche 25
```

Règle 5 — Un module simple et documenté

- **Un thème par module**, avec un nom clair : `banque.py`, `outils.py`…
- **Une docstring** en tête du module et dans chaque fonction ou méthode : ce qu'elle reçoit, ce qu'elle renvoie, les erreurs qu'elle lève. C'est elle que `help(banque)` affichera à l'utilisateur.
- **Les tests** sous `if __name__ == "__main__":`. Ce bloc s'exécute quand on lance le fichier lui-même, mais **pas** quand on l'importe : le programme qui écrit `from banque import ...` ne voit pas les tests s'exécuter.

Au bac

Un sujet peut demander de « coder, dans un module que l'on nommera `outils`, quelques fonctions » (Antilles-Guyane 2026, jour 1), de dire quelle fonction d'un module importé (`time`, `csv`) un programme utilise (Amérique du Nord 2024, jour 1), ou de lire la documentation d'une fonction pour s'en servir. Dans tous les cas : lire la spécification, puis respecter les noms imposés.

▶ **Exercices d'application :** exercice 24 (créer et documenter un module)

X Ouverture

La programmation objet n'est pas une curiosité isolée : elle servira à **implémenter les structures de données** de toute la fin de l'année.

- **Au bac**, on vous demandera surtout de **lire** une classe donnée, d'en **lister les attributs** et leur type, de **compléter** ou **écrire** une méthode, de **créer des instances**, et de **respecter l'encapsulation**.

> **Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)
>
> - **Ce qu'il réutilise :** les fonctions de Première et la *Récursivité* (la factorielle en OCaml est une définition récursive).
> - **Ce qui en découle :** `Pile`, `File` et `Cellule` (*Structures linéaires : piles, files, listes chaînées*), le `Noeud` des *Arbres* et la classe des *Graphes* s'écriront comme des classes.
> - **Pour aller plus loin :** une table de *Bases de données et SQL* ressemblera à une collection d'objets ; SQL sera **déclaratif**, comme Prolog.

Bilan — la carte mémoire

Notion	À retenir
Classe	le plan (le « moule ») d'une famille d'objets
Instance / objet	un exemplaire construit à partir de la classe
Attribut	une donnée attachée à l'instance (`self.x`) : son **état**
Méthode	une fonction dans la classe, agissant via `self` : son **comportement**
`__init__`	le constructeur, appelé à la création ; met en place les attributs
`self`	l'instance courante ; **toujours** le 1er paramètre d'une méthode
Encapsulation	cacher l'état, n'y toucher que par des méthodes (`_attribut`)
Paradigmes	impératif / objet / fonctionnel : styles au choix, combinables
Fonction pure	résultat dépendant des seuls paramètres, sans effet de bord
Module / API	un fichier `.py` que l'on importe / ce qu'il offre, avec sa spécification

Retour à la situation de départ

1. On écrit une seule **classe** `Personnage`, le plan : chaque personnage en est une **instance**, avec ses propres **attributs** (`nom`, `vie`) mis en place par `__init__`, et des **méthodes** communes à toutes (§ III à V).
2. Par **encapsulation** : l'attribut `_solde` n'est modifié que par des méthodes (`deposer`, `retirer`) qui contrôlent l'opération et peuvent lever une erreur avec `raise` (§ VI).
3. Non : ce style est le paradigme **impératif** ; il existe aussi les paradigmes **fonctionnel** (fonctions pures, sans effet de bord) et **objet**, que l'on choisit, et que l'on mélange, selon le problème (§ I, II et VIII).

Erreurs fréquentes

- **Confondre la classe et l'instance.** La classe est le *moule* ; l'instance est l'objet fabriqué avec `Classe(...)`. *Le réflexe :* une classe, plusieurs instances.
- **Oublier `self`.** `self` est le **premier paramètre** de chaque méthode, et tout attribut se lit `self.attribut`. L'oublier ⇒ `NameError` / `TypeError`.
- **Ne pas initialiser un attribut dans `__init__`.** Y accéder ensuite lève `AttributeError`. *Le réflexe :* tous les attributs sont créés dans le constructeur.
- **Court-circuiter l'encapsulation.** Modifier directement `c._solde` depuis l'extérieur au lieu de passer par une méthode : on risque de **rompre un invariant**. *Le réflexe :* accesseur / mutateur.
- **Oublier le `return` d'une méthode** qui doit renvoyer une valeur (elle renvoie `None`), ou confondre `__repr__` (représentation, dans la console) et `__str__` (affichage).
- **Croire qu'une méthode modifie une variable extérieure.** Une méthode agit sur `self` ; ce qui est **mutable** (liste) peut changer, un nombre passé en paramètre, non.

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

- ☐ distinguer **classe** et **instance**, **attribut** et **méthode** → ex. 4, 7 ;
- ☐ écrire une classe avec `__init__`, des attributs et des méthodes → ex. 8, 10, 11 ;
- ☐ créer une instance et appeler ses méthodes → ex. 7, 9 ;
- ☐ utiliser `self` correctement → ex. 6, 8 ;
- ☐ respecter l'**encapsulation** (attribut `_x`, accesseur / mutateur) → ex. 12 ;
- ☐ écrire `__repr__` (et `__str__`) → ex. 13 ;
- ☐ lire et **compléter** une classe donnée (type bac) → ex. 16, 18 ;
- ☐ reconnaître les **paradigmes** (impératif, objet, fonctionnel) et ce qu'est une **fonction pure** → ex. 1, 2, 15 ;
- ☐ utiliser un **module** et sa **documentation** (`import`, `help`), créer un module **documenté** → ex. 24.

Vers le Grand Oral

Quelques pistes de questions que ce chapitre permet de préparer (à étayer d'un exemple concret) :

- **Qu'apporte la programmation objet par rapport à de simples fonctions ?** *(regrouper données et comportements, structurer un gros programme, réutiliser.)*
- **Encapsulation : pourquoi cacher les données ?** *(protéger un invariant, pouvoir changer l'intérieur sans casser le reste ; lien avec la fonction pure.)*
- **Impératif, objet, fonctionnel : faut-il choisir un paradigme ?** *(des styles combinables ; effet de bord contre fonction pure.)*

EXERCICES

Mode d'emploi

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 32.
- **Réflexe** pour toute classe : **(1)** quels sont les **attributs** (l'état) ? **(2)** quelles sont les **méthodes** (le comportement) ? **(3)** le premier paramètre d'une méthode est **toujours** `self`.

Le paradigme fonctionnel ↑ cours

*On commence par le paradigme le plus léger : il n'utilise que des fonctions. Une fonction est **pure** si son résultat ne dépend que de ses paramètres et qu'elle ne modifie rien d'extérieur (**aucun effet de bord**).*

★ **Exercice 1** *Pure ou pas pure ?* ▶ corrigé p. 776 • essentiel

Pour chacune de ces fonctions, dire si elle est **pure**, et justifier. ▷ coup de pouce

```
compteur = 0
def f1(x):
    return 2 * x + 1

def f2(x):
    global compteur      # f2 modifie la variable compteur exterieure
    compteur = compteur + 1
    return x

def f3(liste):
    liste.append(0)
    return liste

def f4(a, b):
    return a * b
```

★★ **Exercice 2** *Rendre une fonction pure* ▶ corrigé p. 776

On donne une fonction avec effet de bord :

```
def double_tout(tab):
    for i in range(len(tab)):
        tab[i] = tab[i] * 2      # modifie tab : effet de bord
```

▶ Réécrire une fonction `double_tout_pur(tab)` qui **renvoie une nouvelle liste** des éléments doublés, sans modifier `tab`. Vérifier sur un exemple que la liste d'origine est bien intacte.

▷ coup de pouce

```
>>> t = [1, 2, 3]
>>> double_tout_pur(t)
[2, 4, 6]
>>> t
[1, 2, 3]
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 3** *Défi — des fonctions qui manipulent des fonctions* ▶ corrigé p. 777 ≈ 20 min

En paradigme fonctionnel, une fonction est une **valeur** comme une autre : on peut la passer en paramètre à une autre fonction, ou la renvoyer. On donne :

```
def est_pair(n):
    return n % 2 == 0

def double(x):
    return 2 * x

def plus_un(x):
    return x + 1
```

1. ▶ Écrire une fonction **pure** `appliquer(f, tab)` qui renvoie la **nouvelle** liste des `f(x)` pour `x` dans `tab`. Exemple : `appliquer(abs, [-2, 3, -5])` renvoie `[2, 3, 5]`, et la liste passée en paramètre est inchangée.
   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.
2. ▶ Écrire une fonction pure `garder(test, tab)` qui renvoie la liste des éléments `x` de `tab` pour lesquels `test(x)` vaut `True`. Exemple : `garder(est_pair, [1, 2, 3, 4])` renvoie `[2, 4]`.
3. ▶ Écrire une fonction `composer(f, g)` qui **renvoie une fonction** `h` telle que `h(x)` vaut `f(g(x))`. Exemple : `composer(double, plus_un)(3)` renvoie 8. Que renvoie `composer(plus_un, double)(3)` ? ▷ coup de pouce ▷ début de solution
4. Sans définir de fonction, quelle construction vue en Première donne directement le résultat de `appliquer(abs, [-2, 3, -5])` ? et celui de `garder(est_pair, [1, 2, 3, 4])` ?

Lire et comprendre une classe ↑ cours

*Ici on ne programme pas (ou peu) : on **lit**, on **repère** attributs et méthodes, on **prévoit** l'affichage.*

★ **Exercice 4** *Lister attributs et méthodes* ▶ corrigé p. 777 • essentiel

On donne la classe suivante. ▷ coup de pouce

```
class Livre:
    def __init__(self, titre, auteur, pages):
        self.titre = titre
        self.auteur = auteur
        self.pages = pages
        self.emprunte = False

    def get_titre(self):
        return self.titre

    def emprunter(self):
        self.emprunte = True
```

1. Lister les **attributs** de la classe `Livre` en précisant leur **type**.
2. Lister les **méthodes** de la classe. Laquelle *renvoie* une information ? Laquelle *modifie* l'état ?
3. Quel est le rôle du mot `self` ?

★ **Exercice 5** *Prévoir l'affichage* ▶ corrigé p. 777

En reprenant la classe `Livre` ci-dessus, donner *sans machine* ce qu'affiche ce programme, puis vérifier. ▷ coup de pouce

```
b = Livre("1984", "Orwell", 328)
print(b.get_titre())
print(b.emprunte)
b.emprunter()
print(b.emprunte)
```

★★ **Exercice 6** *Retrouver l'erreur* ▶ corrigé p. 777

Chacun de ces trois extraits contient **une** erreur classique de débutant en POO. La repérer et la corriger. ▷ coup de pouce

```
# (a)
class Chien:
    def __init__(nom):
        self.nom = nom

# (b)
class Chat:
    def __init__(self, nom):
        nom = nom

# (c)  (la classe Chien de (a) etant corrigee)
medor = Chien("Medor")
print(nom)
```

Créer et manipuler des instances ↑ cours

★ **Exercice 7** *Créer des instances* ▶ corrigé p. 778 • essentiel

On donne la classe des points du plan.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

▶ **1.** Écrire le code qui crée deux instances : `origine` au point $(0, 0)$ et `p` au point $(3, 5)$. **2.** Afficher l'abscisse de `p` (attendu : `3`). ▷ coup de pouce

★ **Exercice 8** *Compléter un constructeur* ▶ corrigé p. 778 • essentiel

▶ Recopier et compléter le constructeur de la classe `Eleve` pour qu'elle possède trois attributs : `nom`, `classe` et une liste `notes` **vide** au départ. ▷ coup de pouce

```
class Eleve:
    def __init__(self, nom, classe):
        self.nom = ...
        self.classe = ...
        self.notes = ...      # liste vide au depart

e = Eleve("Zoe", "TG1")
print(e.nom, e.classe, e.notes)   # attendu : Zoe TG1 []
```

★★ **Exercice 9** *Ajouter des méthodes* ▶ corrigé p. 778 • essentiel

▶ En repartant de la classe `Eleve` précédente, ajouter deux méthodes :

- `ajouter_note(self, note)` qui ajoute une note à la liste `notes` ;
- `moyenne(self)` qui renvoie la moyenne des notes (on suppose la liste non vide). ▷ coup de pouce

Exemple d'utilisation :

```
>>> e = Eleve("Zoe", "TG1")
>>> e.ajouter_note(12)
>>> e.ajouter_note(16)
>>> e.moyenne()
14.0
```

2

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Écrire une classe complète ↑ cours

★★ **Exercice 10** *La classe Rectangle* ▶ corrigé p. 779 • essentiel

▶ Écrire entièrement une classe `Rectangle` avec :

- un constructeur prenant la `largeur` et la `hauteur` ;
- une méthode `aire(self)` et une méthode `perimetre(self)` ;
- une méthode `est_carre(self)` qui renvoie un booléen. ▷ coup de pouce

Exemple :

```
>>> r = Rectangle(4, 3)
>>> r.aire()
12
>>> r.perimetre()
14
>>> r.est_carre()
False
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 11** *La classe Chronometre* ▶ corrigé p. 779

▶ Écrire une classe `Chronometre` qui modélise un compteur de secondes.

- Le constructeur initialise un attribut `secondes` à `0`.
- `tic(self)` ajoute une seconde.
- `reset(self)` remet le compteur à `0`.
- `affichage(self)` renvoie une chaîne au format `"mm:ss"` (minutes et secondes sur deux chiffres).

Exemple : après 75 appels à `tic`, `affichage()` renvoie `"01:15"`. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Encapsulation ↑ cours

★★ **Exercice 12** *Corriger une mauvaise pratique* ▶ corrigé p. 780 • essentiel

Un élève manipule un compte directement par son attribut :

```
1 class Compte:
2     def __init__(self, solde):
3         self.solde = solde
4
5 c = Compte(100)
6 c.solde = c.solde - 500      # le solde devient negatif !
7 print(c.solde)
```

▶ **1.** Expliquer pourquoi manipuler `c.solde` directement est une mauvaise pratique. **2.** Réécrire la classe `Compte` en **encapsulant** le solde (attribut `_solde`) et en fournissant deux méthodes :

- `retirer(self, montant)` : effectue le retrait seulement s'il ne rend pas le solde négatif ; renvoie `True` si le retrait a eu lieu, `False` sinon ;
- `solde(self)` : renvoie le solde.

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Méthodes spéciales et objets qui interagissent ↑ cours

★★★ **Exercice 13** *Défi — la classe Fraction* ▶ corrigé p. 780 ≈ 20 min

▶ Écrire une classe `Fraction` représentant un nombre rationnel par un `numerateur` et un `denominateur`.

- Le constructeur **simplifie automatiquement** la fraction (il divise numérateur et dénominateur par leur PGCD).
- `__repr__(self)` renvoie le texte `3/4` (par exemple), pour que la fraction s'affiche rien qu'en tapant son nom dans la console.
- `fois(self, autre)` renvoie une **nouvelle** `Fraction`, produit des deux (sans modifier ni `self` ni `autre` : dans l'esprit fonctionnel). ▷ coup de pouce ▷ début de solution

Exemple :

```
>>> f = Fraction(6, 8)
>>> f
3/4
>>> print(f.fois(Fraction(2, 3)))
1/2
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 14** *Défi — un réseau d'amis* ▶ corrigé p. 781 ≈ 30 min

On modélise un petit réseau social : chaque personne est un objet de la classe `Personne`, qui possède un attribut `nom` et un attribut `amis`, liste d'**objets** `Personne` (vide à la création).

- ▶ Écrire le constructeur, puis la méthode `ajouter_ami(self, autre)` : l'amitié est **réciproque** (chacun est ajouté aux amis de l'autre), sans jamais créer de doublon, et une personne ne peut pas être son propre ami.
- ▶ Écrire la méthode `noms_amis(self)`, qui renvoie la liste des noms des amis, et la méthode `amis_communs(self, autre)`, qui renvoie la liste des noms des amis communs à `self` et `autre`.
- ▶ Écrire la méthode `amis_d_amis(self)`, qui renvoie la liste, sans doublon, des noms des amis de ses amis qui ne sont ni `self` ni déjà ses amis (les « suggestions d'amis »).
- Pourquoi la liste `amis` contient-elle des objets `Personne` plutôt que de simples noms ?

▷ coup de pouce ▷ début de solution

Exemple :

```
>>> a, b = Personne("Ada"), Personne("Bob")
>>> c, d = Personne("Chloé"), Personne("Dan")
>>> a.ajouter_ami(b); a.ajouter_ami(c); b.ajouter_ami(c); c.ajouter_ami(d)
>>> a.ajouter_ami(b)            # deja amis : rien ne change
>>> a.noms_amis(), c.noms_amis()
(['Bob', 'Chloé'], ['Ada', 'Bob', 'Dan'])
>>> a.amis_communs(b), a.amis_d_amis()
(['Chloé'], ['Dan'])
```

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Choisir un paradigme ↑cours

★ **Exercice 15** IA IA IA *Le bon style au bon endroit* ▶ corrigé p. 782

Pour chacune des situations suivantes, indiquer le paradigme *a priori* le plus naturel (impératif, objet, ou fonctionnel), et justifier en une phrase.

a) modéliser les personnages, monstres et objets d'un jeu vidéo ;
b) appliquer une même transformation à tous les éléments d'une liste, sans risque d'effet de bord ;
c) dérouler un algorithme de tri étape par étape.

▷ coup de pouce

Vers le bac ↑cours

★★ **Exercice 16** IA IA IA *Le domaine skiable (d'après Métropole 2024, jour 2)* ▶ corrigé p. 782 • essentiel ≈ 20 min

On ne traite ici que la partie « programmation objet » du sujet. Une station de ski modélise ses pistes par une classe `Piste` et son domaine par une classe `Domaine`. Le code est donné en **annexe** ci-dessous ; il se termine par la création du domaine `lievre_blanc` et de ses pistes.

1. Lister les attributs de la classe `Piste` en précisant leur type.

2. Une piste est de couleur `'noire'` si son dénivelé est $\geqslant 100$ m ; `'rouge'` s'il est < 100 et $\geqslant 70$; `'bleue'` s'il est < 70 et $\geqslant 40$; `'verte'` s'il est < 40. Écrire la méthode `set_couleur(self)` de la classe `Piste` qui affecte à l'attribut `couleur` la bonne chaîne. ▷ coup de pouce

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

3. On exécute `lievre_blanc.get_pistes()`. Parmi les propositions, indiquer le **type** de l'élément renvoyé : *(A)* une chaîne ; *(B)* un objet `Piste` ; *(C)* une liste de chaînes ; *(D)* une liste d'objets `Piste`.

4. Écrire un programme qui **ferme** toutes les pistes vertes, c'est-à-dire affecte `False` à leur attribut `ouverte`.

5. Écrire une fonction `pistes_de_couleur(lst, couleur)` qui prend une liste `lst` de pistes et une chaîne `couleur`, et renvoie la liste des **noms** des pistes de cette couleur.

Annexe — code des classes

```
class Piste:
    def __init__(self, nom, denivele, longueur):
        self.nom = nom
        self.denivele = denivele    # en metres
        self.longueur = longueur    # en kilometres
        self.couleur = ''
        self.ouverte = True

    def get_nom(self):
        return self.nom

    def set_couleur(self):
        ...   # A COMPLETER (question 2)

    def get_couleur(self):
```

```
        return self.couleur

class Domaine:
    def __init__(self, a):
        self.nom = a
        self.pistes = []

    def ajouter_piste(self, nom, denivele, longueur):
        self.pistes.append(Piste(nom, denivele, longueur))

    def get_pistes(self):
        return self.pistes

lievre_blanc = Domaine('Le Lievre blanc')
lievre_blanc.ajouter_piste('Petit Bonheur', 120, 1.5)
lievre_blanc.ajouter_piste('Foret', 105, 2.3)
lievre_blanc.ajouter_piste('Duvallon', 140, 1.2)
lievre_blanc.ajouter_piste('Les Gentianes', 85, 2.0)
lievre_blanc.ajouter_piste('Le Chamois', 55, 1.8)
lievre_blanc.ajouter_piste('Les Marmottes', 25, 0.9)
lievre_blanc.ajouter_piste('Le Jardin', 15, 0.4)
for piste in lievre_blanc.get_pistes():
    piste.set_couleur()     # une fois la question 2 traitee
```

★★ **Exercice 17** *Valeurs nutritionnelles (d'après La Réunion 2023, jour 1)* ▶ corrigé p. 783 ≈ 20 min

Pour chaque aliment, on stocke ses caractéristiques nutritionnelles (pour 100 g) dans un objet de la classe `Aliment` : énergie (kcal), protéines, glucides et lipides (en grammes).

```
class Aliment:
    def __init__(self, e, p, g, l):
        self.energie = e
        self.proteines = p
        self.glucides = g
        self.lipides = l
```

1. Le lait entier a pour valeurs : énergie `65.1`, protéines `3.32`, glucides `4.85`, lipides `3.63`. Écrire l'instruction qui crée l'instance `lait`.

2. Donner l'instruction qui permet d'obtenir la valeur `65.1` (l'énergie de `lait`).

3. Une erreur s'est glissée : la masse de protéines du lait est en réalité `3.4`. Donner l'instruction qui **modifie** cet attribut de `lait`.

4. On veut ajouter une méthode `energie_reelle` qui renvoie l'énergie d'une masse donnée (en grammes). Par exemple, `lait.energie_reelle(245)` renvoie environ `159.495` (soit $245 \times 65{,}1 \div 100$). Recopier et compléter :

```
def energie_reelle(..., masse):
    return ...
```

5. On regroupe les aliments dans un **dictionnaire** `aliments` dont les clés sont les noms : `aliments = {'lait': lait, 'pain': pain, ...}`. Donner l'instruction qui obtient la valeur énergétique réelle de 220 g de pain.

6. ▶ Écrire une fonction `plus_calorique(aliments)` qui renvoie le **nom** de l'aliment le plus énergétique (pour 100 g) du dictionnaire `aliments`. ▷ coup de pouce

   ▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 18** *La classe Date (d'après Asie 2025, jour 2)* ▶ corrigé p. 783 ≈ 30 min

On manipule des dates grâce à la classe `Date` ci-dessous.

```
class Date:
    def __init__(self, jour, mois, annee):
        self.jour = ...
        self.mois = ...
        self.annee = ...
        self.nb_jours_par_mois = [31, 28, 31, 30, 31, 30,
                                  31, 31, 30, 31, 30, 31]

    def get_annee(self):
        return ...

    def est_bissextile(self):
        ...   # a ecrire (question 4)
```

1. Recopier et compléter les trois premières lignes du constructeur (affectation des attributs `jour`, `mois` et `annee`).
2. Écrire l'instruction qui crée l'instance `d` représentant le **19 juin 2024**.
3. Compléter la méthode `get_annee` (elle renvoie l'attribut `annee`).
4. Une année est bissextile si elle est divisible par 4 **mais pas** par 100, **ou** si elle est divisible par 400. Écrire la méthode `est_bissextile(self)` qui renvoie `True` ou `False`. *(Rappel :* `a % n == 0` *teste la divisibilité de* `a` *par* `n`*.)*
5. ▶ Écrire la méthode `nb_jours_passes(self)` qui renvoie le nombre de jours écoulés depuis le **1er janvier** de l'année (jour de la date compris). On ajoutera les jours des mois précédents (via `nb_jours_par_mois`), sans oublier le **29 février** si l'année est bissextile. ▷ coup de pouce

   ▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.
6. ▶ En déduire la méthode `nb_jours_restants(self)` qui renvoie le nombre de jours restant jusqu'à la fin de l'année, **sans compter** le jour de la date (le 31 décembre, il en reste 0). *(Une année compte 365 jours, ou 366 si elle est bissextile.)*

★★ **Exercice 19** *Colorier une carte (d'après Métropole 2023, jour 1)* ▶ corrigé p. 784 ≈ 15 min

On modélise une région d'un pays par la classe `Region` ci-dessous.

```
class Region:
    def __init__(self, nom_region):
        self.nom = nom_region
        self.tab_voisines = []
        self.tab_couleurs_disponibles = ['rouge', 'vert',
            'bleu', 'jaune', 'orange', 'marron']
        self.couleur_attribuee = None
```

1. Pour chacun des noms `nom`, `tab_voisines` et `couleur_attribuee`, préciser s'il s'agit d'un **objet**, d'un **attribut**, d'une **méthode** ou d'une **classe**. Donner aussi le **type** du paramètre `nom_region`.
2. Écrire l'instruction qui crée une instance `ge` correspondant à la région « Grand Est ».
3. Écrire les trois méthodes suivantes de la classe `Region` :

- `renvoie_premiere_couleur_disponible(self)` : renvoie la première couleur de `tab_couleurs_disponibles` (supposé non vide) ;
- `renvoie_nb_voisines(self)` : renvoie le nombre de régions voisines ;
- `est_coloriee(self)` : renvoie `True` si une couleur a été attribuée à la région, `False` sinon.

▷ coup de pouce

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ Exercice 20 *Des colis à expédier (d'après Amérique du Nord 2025, jour 1)*

▶ corrigé p. 784 ≈ 60 min

Sujet repris en entier : programmation objet, puis tri, récursivité (revoir le chapitre Récursivité) et algorithme glouton.

Une entreprise gère les colis qu'elle expédie à l'aide d'une application. Chaque colis a un identifiant unique, un poids, une adresse de livraison et un état : `'préparé'`, `'transit'` ou `'livré'`. On dispose pour cela de la classe `Colis` suivante : à la création d'un colis, l'état vaut `'préparé'` et les autres valeurs sont passées en paramètres.

```
1 class Colis:
2     def __init__(self, id, poids, adresse):
3         self.id = id               # identifiant unique (str)
4         self.poids = poids         # poids en kilogrammes (float)
5         self.adresse = adresse     # adresse de destination (str)
6         self.etat = 'préparé'      # 'préparé', 'transit' ou 'livré'
```

On crée par exemple deux colis ; on accède au poids d'un colis `c` par `c.poids`, à son état par `c.etat`.

```
1 colisA = Colis('AC12', 5.0, '20 rue de la paix 57000 Metz')
2 colisB = Colis('AF34', 10.25, '32 rue du centre 57000 Metz')
```

1. Écrire la méthode `passer_transit` de la classe `Colis` qui permet de mettre l'état du colis à la valeur `'transit'`.

On dispose de la fonction `ajouter_colis` ; après les trois instructions de droite, la liste `liste_colis` contient les deux colis créés.

```
def ajouter_colis(liste, colis):
    # ajoute le colis a la fin de la liste
    liste.append(colis)
```

```
liste_colis = []
ajouter_colis(liste_colis, colisA)
ajouter_colis(liste_colis, colisB)
```

2. Dans cette question uniquement, on considère que le transporteur refuse les colis de plus de 25 kg. Recopier et modifier la fonction `ajouter_colis` pour qu'elle ajoute le colis à la liste si son poids est inférieur ou égal à 25 kg, et qu'elle affiche le message « Dépassement du poids maximal autorisé » sinon.

3. Écrire une fonction `nb_colis` qui prend en paramètre une liste d'objets de la classe `Colis` et renvoie le nombre de colis de cette liste.

4. Recopier et compléter les lignes 2 et 4 de la fonction `poids_total`, qui renvoie le poids total des colis de la liste.

```
1 def poids_total(liste):
2     total = ...
3     for c in liste:
4         total = ...
5     return total
```

5. Écrire une fonction `liste_colis_etat` qui prend en paramètres une liste d'objets de la classe `Colis` et une chaîne `statut` (parmi `'préparé'`, `'transit'` ou `'livré'`), et renvoie une nouvelle liste contenant les colis de cette liste dont l'état est `statut`.

L'entreprise tente d'optimiser, à l'aide d'un algorithme **glouton**, le chargement des colis dans un camion de capacité donnée (en kilogrammes), sans tenir compte du volume : on charge les colis par ordre **décroissant** de poids, sans dépasser la capacité du camion. Il faut donc trier la liste des colis par poids décroissants, ce que fait la fonction `tri_decroissant`.

```
def tri_decroissant(liste):
    n = len(liste)
    for i in range(n - 1):
        max_pos = i
        for j in range(i + 1, n):
            if liste[j].poids > liste[max_pos].poids:
                max_pos = j
        # Echanger les elements
        temp = liste[i]
        liste[i] = liste[max_pos]
        liste[max_pos] = temp
    return liste
```

(Le sujet officiel nomme cette variable `min_pos`*, nom trompeur ici : elle est renommée* `max_pos`*.)*

6. Donner le nom du tri utilisé dans `tri_decroissant`, ainsi que son coût dans le pire des cas.

7. Citer un autre algorithme de tri qui aurait pu être utilisé, ainsi que son coût dans le pire des cas.

La fonction récursive `chargement_glouton` a pour paramètres `liste` (une liste de colis triés par poids décroissants), `rang` (un indice compris entre `0` inclus et `len(liste)` inclus) et `capacite` (une charge en kilogrammes). Elle renvoie la liste des colis à charger selon l'algorithme glouton, en supposant que la charge restante dans le camion est `capacite` et en ne considérant que les colis de `liste` d'indice supérieur ou égal à `rang`.

```
def chargement_glouton(liste, rang, capacite):
    if rang == len(liste):
        return ...
    elif liste[rang].poids <= ...:
        return ... + chargement_glouton(liste, ..., ...)
    else:
        return chargement_glouton(liste, ..., ...)
```

8. Recopier et compléter la fonction `chargement_glouton`. ▷ coup de pouce

9. Expliquer brièvement pourquoi un appel à `chargement_glouton` peut provoquer l'erreur suivante :

```
RecursionError: maximum recursion depth exceeded while
calling a Python object.
```

10. ▶ Écrire une fonction **itérative** (sans récursivité) `chargement_glouton2(liste, capacite)`, où `liste` est une liste de colis triés par poids décroissants et `capacite` la capacité du camion en kilogrammes, qui renvoie la liste des colis à charger selon le même algorithme glouton. On pourra créer une liste `colis_a_charger`, puis parcourir tous les colis triés en ajoutant à cette liste chaque colis qui rentre encore dans le camion (le poids total ne doit pas excéder la capacité du camion). ▷ début de solution

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 21** *Puissance 4 : la grille de jeu (d'après Amérique du Nord 2026, jour 1, partie A)* ▶ corrigé p. 786 ≈ 30 min

Le jeu puissance 4 se joue à deux joueurs dans une grille de 6 lignes et 7 colonnes. Une couleur de pion, blanc ou noir, est attribuée à chaque joueur ; ils jouent à tour de rôle. À son tour, un joueur choisit une colonne dans laquelle il introduit un pion de sa couleur : la grille étant verticale, le pion tombe et bute soit sur le bas de la grille, soit sur un pion déjà placé. Le but est d'aligner au moins quatre pions de sa couleur dans n'importe quelle direction (horizontalement, verticalement, en diagonale). Le premier à y parvenir a gagné.

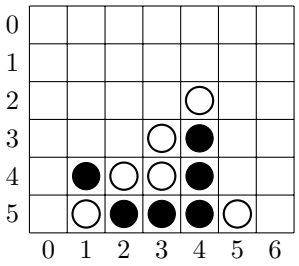


Figure 1. Une partie gagnée par le joueur blanc (numéros des lignes à gauche, des colonnes en bas).

Dans cet exercice, on programme la grille de jeu et le calcul de son **score**. La suite du sujet (l'algorithme **min-max**, qui explore un **arbre** de coups) sera traitée au chapitre *Arbres*.

Dans la suite, les deux joueurs sont notés 1 et 2 (la couleur des pions ne sert plus). L'unique attribut d'une instance de la classe `Grille` est un tableau `grille` de 6 lignes et 7 colonnes (liste de listes) contenant des entiers : `0` pour une case vide, `1` (resp. `2`) pour un pion du joueur 1 (resp. 2). Les lignes et les colonnes sont numérotées comme sur la figure 1 (ligne 0 en haut).

1. Écrire la méthode `__init__(self)` de la classe `Grille` qui définit l'attribut `grille` comme un tableau rempli de `0`.

2. La méthode `joue(self, colonne, joueur)` tente de placer un pion du joueur `joueur` dans la colonne `colonne`. Si la colonne ne contient pas déjà six pions, le pion est placé au bon endroit et la méthode renvoie `True` ; sinon, elle renvoie simplement `False`. La recherche d'une case libre se fait **de bas en haut**. Recopier et compléter les lignes 4, 6, 7 et 9.

```
1 def joue(self, colonne, joueur):
2     ligne = 5    # on part de la rangee la plus basse
3     while ligne != -1 and self.grille[ligne][colonne] != 0:
4         ligne = ...
5     if ligne != -1:
6         self.grille[ligne][colonne] = ...
7         return ...
8     else:
9         return ...
```

Voici le tableau représentant la grille de jeu à la fin du troisième coup d'une partie :

```
[[0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 1, 0, 0, 0],
 [0, 0, 1, 2, 0, 0, 0]]
```

3. Écrire le code Python qui crée cette grille de jeu en utilisant **uniquement** les méthodes de la classe `Grille`. Elle sera stockée dans une variable `jeu1`.

On suppose écrite une fonction `valeur_case(ligne, colonne)` qui renvoie le nombre d'alignements de quatre cases contenant la case `(ligne, colonne)`. Par exemple, `valeur_case(0, 1)` renvoie `4` : les quatre alignements de quatre cases contenant la case `(0, 1)` sont représentés sur la figure 2.

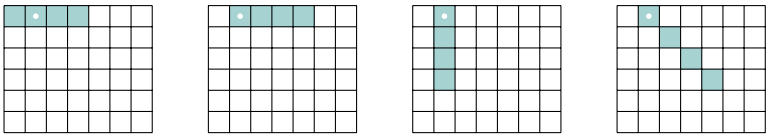


Figure 2. Les alignements contenant la case `(0, 1)` (marquée d'un point blanc).

Le **score** d'une grille s'obtient en **additionnant** les valeurs de toutes les cases occupées par un pion du joueur 2 et en **soustrayant** les valeurs de toutes les cases occupées par un pion du joueur 1 (la valeur d'une case étant donnée par `valeur_case`).

4. Donner le score de la grille `jeu1` de la question 3 : déterminer à la main, comme sur la figure 2, la valeur de chacune des trois cases occupées, puis faire le calcul.

5. Écrire la méthode `score(self)` qui renvoie le score de la grille représentée par l'objet.

▷ coup de pouce

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Critiquer une réponse d'IA ↑ cours

★★ **Exercice 22** *Une classe écrite par un assistant d'IA* ▶ corrigé p. 787

Un élève demande à un assistant d'IA : « Écris une classe Python `Panier` avec le nom du client et la liste de ses articles (vide au départ), une méthode `ajouter` et une méthode `nb_articles`. » Voici la réponse obtenue :

```
class Panier:
    def __init__(self, client, articles=[]):
        self.client = client
        self.articles = articles      # liste vide par defaut

    def ajouter(self, article):
        self.articles.append(article)

    def nb_articles(self):
        return len(self.articles)
```

« Le paramètre `articles=[]` donne une liste vide à chaque nouveau panier, sauf si l'on en fournit une ; `ajouter` l'enrichit et `nb_articles` renvoie sa longueur. »

1. La réponse est-elle correcte ? ▶ Prévoir l'affichage du scénario suivant, puis l'exécuter :

```
p1 = Panier("Alice")
p2 = Panier("Bob")
p1.ajouter("stylo")
print(p1.nb_articles(), p2.nb_articles())
```

2. Localiser et corriger l'erreur. ▷ coup de pouce
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

S'entraîner à l'oral ↑cours

★★ **Exercice 23** *Expliquer en deux minutes* ▶ corrigé p. 787

Au Grand Oral comme devant l'examinateur de l'épreuve pratique, il faut savoir **expliquer** une notion clairement, sans lire ses notes. Choisir l'un des sujets suivants (ou le tirer au sort) :

Sujet 1. Expliquer à un camarade de Première la différence entre une **classe** et une **instance**.
Sujet 2. Expliquer à quoi sert l'**encapsulation** : pourquoi passer par des méthodes plutôt que modifier directement les attributs ?
Sujet 3. Expliquer le rôle de la méthode `__init__` et du paramètre `self`.

Préparation (5 min, seul). Noter au brouillon **trois idées** et **un exemple**, rien de plus : comme au Grand Oral, ces notes servent d'aide-mémoire. **Exposé (2 min, chronométré).** Debout, face à un camarade, **sans lire ses notes** et sans rien écrire : tout passe par la parole. **Retour (2 min).** Le camarade pose une question de relance (on peut alors s'aider d'un schéma), remplit la grille ci-dessous et donne un conseil ; puis on échange les rôles. Cette grille reprend les critères d'évaluation du Grand Oral. ▷ coup de pouce

Grille entre camarades	Oui	En partie	Pas encore
Clair — audible, posé ; chaque mot technique est expliqué	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	☐	☐	☐
Construit — un fil conducteur, tenu en deux minutes, sans lire	☐	☐	☐
Un conseil pour la prochaine fois :			

Modularité ↑cours

★★ **Exercice 24** *Créer et documenter un module* ▶ corrigé p. 788

1. *Exploiter une documentation.* Dans la console, on obtient :

```
>>> from random import choice
>>> help(choice)
choice(seq) method of random.Random instance
    Choose a random element from a non-empty sequence.
```

Que peut renvoyer `choice(["pile", "face"])` ? D'après la documentation, l'appel `choice([])` respecte-t-il la spécification ? ▶ Vérifier ce qui se passe dans la console.

2. ▶ Créer un fichier `temps.py` qui commence par une docstring de module et contient deux fonctions documentées :

   - `en_secondes(h, m, s)` renvoie la durée de `h` heures, `m` minutes et `s` secondes, convertie en secondes ;
   - `en_hms(duree)` renvoie le triplet `(h, m, s)` d'une durée donnée en secondes, avec des minutes et des secondes comprises entre 0 et 59.

   Ajouter, sous `if __name__ == "__main__":`, au moins deux `assert` par fonction, par exemple `en_secondes(1, 2, 3) == 3723`. ▷ coup de pouce

   ▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

3. ▶ Dans un second fichier `trajet.py`, placé dans le même dossier, importer les deux fonctions et afficher, au format `(h, m, s)`, la durée totale d'un trajet en deux étapes : 1 h 47 min 30 s, puis 2 h 25 min 45 s.

4. Dans la console, après `import temps`, que montre `help(temps)` ? Quand on lance `trajet.py`, les tests de `temps.py` s'exécutent-ils ? Pourquoi ?

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Pour chaque fonction, se poser deux questions. Son résultat dépend-il d'autre chose que de ses paramètres ? Modifie-t-elle une variable ou un objet qui existe en dehors d'elle (variable globale, liste reçue en paramètre) ? ↑énoncé

Ex. 2 – piste Partir d'une liste vide, la remplir dans une boucle, puis la renvoyer : aucune affectation de la forme `tab[i] = ...` ne doit apparaître. ↑énoncé

Ex. 3 q. 3 – piste Pour les questions 1 et 2 : construire une nouvelle liste, sans jamais modifier `tab` ; `f` s'appelle comme n'importe quelle fonction, `f(x)`. Pour la question 3 : on peut définir une fonction **à l'intérieur** d'une autre fonction, puis la renvoyer (sans parenthèses : on renvoie la fonction, pas son résultat). ↑énoncé

Ex. 3 q. 3 – début de solution
```
def composer(f, g):
    def h(x):
        return ...
    return h
```
↑énoncé

Ex. 4 – piste Les attributs sont les noms écrits `self.... = ...` dans le constructeur ; leur type se lit sur la valeur reçue ou affectée. Les méthodes sont les fonctions définies (`def`) à l'intérieur de la classe. ↑énoncé

Ex. 5 – piste Relire le constructeur : que vaut l'attribut `emprunte` juste après la création de `b` ? Quelle méthode le modifie, et quand est-elle appelée ? ↑énoncé

Ex. 6 – piste Relire le « réflexe » du mode d'emploi. Quel est toujours le premier paramètre d'une méthode ? Comment crée-t-on un attribut dans le constructeur ? Comment accède-t-on à l'attribut d'un objet depuis l'extérieur de la classe ? ↑énoncé

Ex. 7 – piste Relire dans le cours comment on crée une instance : le constructeur est appelé automatiquement. Quelles valeurs faut-il fournir, et quel paramètre de `__init__` ne se donne jamais ? Un attribut d'un objet se lit avec la notation pointée. ↑énoncé

Ex. 8 – piste Chaque attribut `self....` reçoit le paramètre du même nom. Pour `notes`, il n'y a pas de paramètre : quelle valeur représente une liste vide ? ↑énoncé

Ex. 9 – piste La liste à modifier est l'attribut `self.notes` (méthode `append`). Pour la moyenne : somme des notes (boucle d'accumulation) divisée par leur nombre, `len(self.notes)`. ↑

Ex. 10 – piste Chaque méthode ne fait qu'un calcul sur `self.largeur` et `self.hauteur`. `est_carre` peut renvoyer directement le résultat d'une comparaison, sans `if`. ↑énoncé

Ex. 11 – piste Le nombre de minutes et le nombre de secondes restantes s'obtiennent par division entière (`//`) et reste (`%`) de `self.secondes` par 60. `f"{m:02d}"` affiche un entier sur deux chiffres. ↑énoncé

Ex. 12 – piste Que peut encore garantir la classe si n'importe qui écrit dans ses attributs ? Pour `retirer` : comparer le montant à `self._solde` **avant** de modifier quoi que ce soit. ↑

Ex. 13 – piste Écrire d'abord, à part, une fonction `pgcd(a, b)` (algorithme d'Euclide), puis simplifier dès le constructeur. `fois` construit et renvoie un **nouvel** objet `Fraction(...)`. ↑énoncé

Ex. 13 – début de solution
```
class Fraction:
    def __init__(self, num, den):
        d = pgcd(num, den)
        self.numerateur = num // d
```
↑énoncé

Ex. 14 – piste Dans `ajouter_ami`, tout faire en une fois : un test pour écarter les cas interdits (`autre is self`, `autre in self.amis` ; `is` teste s'il s'agit du **même** objet, et non de deux objets égaux), puis deux `append`, un dans chaque liste. Pour `amis_d_amis` :

deux boucles imbriquées (mes amis, puis les amis de chacun d'eux) et trois conditions avant d'ajouter un nom. ↑énoncé

Ex. 14 – début de solution
```
def ajouter_ami(self, autre):
    if autre is self or autre in self.amis:
        return
    self.amis.append(autre)
```
↑énoncé

Ex. 15 – piste Relire ce qui caractérise chaque paradigme dans le cours : des objets qui ont un état et des comportements, des fonctions pures sans effet de bord, ou une suite d'instructions qui modifient des variables. Lequel décrit le mieux chaque situation ? ↑énoncé

Ex. 16 q. 2 – piste Une cascade `if` / `elif` / `else` sur `self.denivele`, en partant du seuil le plus élevé : chaque test n'a alors besoin que d'une seule comparaison. ↑énoncé

Ex. 17 q. 6 – piste C'est une recherche de maximum (Première) : parcourir les clés du dictionnaire en mémorisant le nom du « champion » et comparer les attributs `energie` des objets associés. ↑énoncé

Ex. 18 q. 5 – piste Additionner `nb_jours_par_mois[i]` pour les mois **précédant** `self.mois` (le mois de janvier a l'indice `0`), ajouter `self.jour`, puis `1` si l'année est bissextile **et** que la date est après février. ↑énoncé

Ex. 19 q. 3 – piste Chaque méthode tient en une ligne. Que vaut `couleur_attribuee` tant qu'aucune couleur n'a été choisie ? ↑énoncé

Ex. 20 q. 8 – piste Que renvoyer quand il n'y a plus de colis à examiner (une liste !) ? Si le colis de rang `rang` est chargé, que deviennent le rang et la capacité restante pour l'appel suivant ? Et s'il ne l'est pas ? ↑énoncé

Ex. 20 q. 10 – début de solution
```
def chargement_glouton2(liste, capacite):
    colis_a_charger = []
    charge = 0
    for c in liste:
```
↑énoncé

Ex. 21 q. 5 – piste Deux boucles imbriquées sur les 6 lignes et les 7 colonnes de `self.grille` ; que faire de la valeur renvoyée par `valeur_case` selon que la case contient `0`, `1` ou `2` ? ↑

Ex. 22 q. 2 – piste La valeur par défaut `[]` est évaluée quand Python lit la ligne `def`. Combien de listes sont donc créées : une par panier, ou une seule pour tous ? ↑énoncé

Ex. 23 – piste Structurer chaque explication en trois temps : une **définition** en une phrase, un **exemple** petit et concret déroulé à voix haute, puis le **piège** (l'erreur que l'auditeur risque de faire). Finir par une phrase qui répond à la question posée. Choisir un objet de la vie courante (un chien, un compte bancaire, une voiture) et s'y tenir du début à la fin : ses attributs, ses méthodes, deux objets différents. ↑énoncé

Ex. 24 q. 2 – piste Dans `en_hms`, le nombre d'heures est le quotient `duree // 3600` ; le reste `duree % 3600` se découpe ensuite de la même façon en minutes et en secondes, avec `60`. ↑

TP ET PROJETS

PROJET

La Bataille, en objets

quatre classes qui jouent ensemble

Durée : 3 h environ (deux séances) · Sur machine, seul ou par deux

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t02-projet-poo` — **archive .zip**

But du projet

Programmer **de bout en bout** le jeu de cartes de la Bataille, puis le faire jouer **mille fois** pour répondre à de vraies questions : combien de temps dure une partie ? Avoir des As aide-t-il à gagner ? Une partie peut-elle ne jamais finir ? Le programme repose sur **quatre classes** qui interagissent, avec des attributs **encapsulés**, des **méthodes spéciales** et des **tests** ; les statistiques s'écrivent dans le style **fonctionnel**. ▶ signale ce qui est à programmer et tester.

Consignes

- L'extension est facultative.
- Le fichier `projet_poo_depart.py` est **à télécharger** (lien ci-dessus) : squelette des classes et, en fin de fichier, des **tests**. Relancer le fichier après chaque partie : chaque ligne `[A FAIRE]` doit devenir `[OK]`. On ne modifie pas les tests.
- Tous les attributs dont le nom commence par `_` sont **privés** : en dehors de leur classe, on n'y accède **que** par des méthodes.
- Usage de l'IA, indiqué sur chaque exercice :  aucune ;  en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ;  l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

Les règles (version 32 cartes)

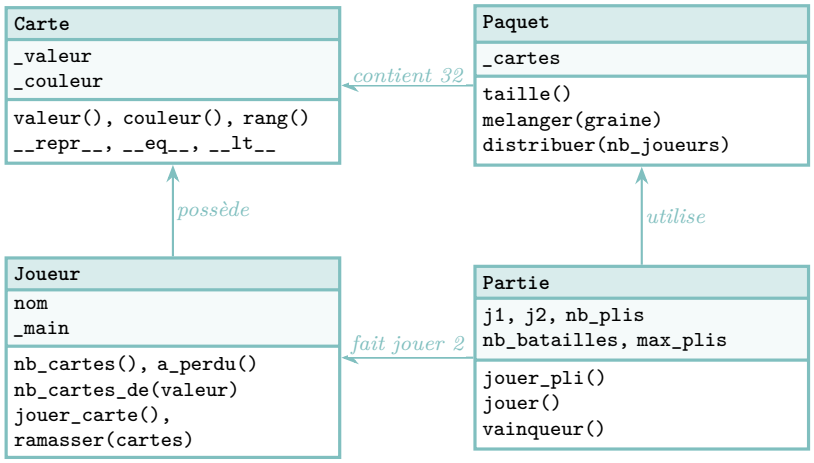
On utilise un jeu de 32 cartes : huit valeurs (`7`, `8`, `9`, `10`, `Valet`, `Dame`, `Roi`, `As`, de la plus faible à la plus forte) dans quatre couleurs (`pique`, `coeur`, `carreau`, `trefle`). La couleur ne compte pas pour comparer deux cartes.

- Le paquet est mélangé puis distribué entièrement, une carte à chacun à tour de rôle : 16 cartes par joueur, en tas, faces cachées.
- À chaque **pli**, chaque joueur retourne la carte du dessus de son tas et la pose sur le **tapis**. La plus forte l'emporte : son propriétaire ramasse **tout le tapis** et le place **sous** son tas, dans l'ordre où les cartes ont été posées (à chaque tour, la carte du premier joueur avant celle du second, cartes cachées comprises).
- En cas d'égalité, c'est la **bataille** : chacun pose une carte **face cachée**, puis on rejoue (une carte face visible chacun), et ainsi de suite jusqu'à départager. Le gagnant ramasse tout le tapis.

- Un joueur qui n'a plus de cartes a perdu, y compris au milieu d'une bataille (l'autre ramasse alors le tapis).

1 L'architecture du programme

Avant d'écrire la moindre ligne, on fixe les classes, ce qu'elles savent (attributs) et ce qu'elles savent faire (méthodes).



★ **Exercice 1** *Lire l'architecture*

1. Quelles classes créent, dans leur constructeur, des objets d'une autre classe ? Lesquels ?
2. Pourquoi `nom` n'est-il pas privé, alors que `_main` l'est ? Donner un exemple de triche (une action contraire aux règles du jeu) que l'encapsulation de `_main` rend plus difficile.
3. Ada a pour tas `Dame de coeur, 9 de pique, Roi de trefle, 7 de coeur` (carte du dessus en premier) et Alan `8 de trefle, 9 de coeur, As de pique, 10 de carreau` ; Ada est le premier joueur. Jouer la partie **à la main** : écrire les deux tas après chaque pli. Qui gagne, et en combien de plis ?

2 La classe `Carte`

★ **Exercice 2** *Une carte qui se présente*

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

▶ Dans `projet_poo_depart.py`, compléter la classe `Carte` (les listes `VALEURS`, de la plus faible à la plus forte, et `COULEURS` sont déjà définies en haut du fichier). Il faut écrire :

- le constructeur, qui lève `ValueError("carte inconnue")` avec `raise` si la valeur n'est pas dans `VALEURS` ou la couleur pas dans `COULEURS` (voir le cours, « Signaler une erreur ») ;
- les accesseurs `valeur()` et `couleur()` ;
- la méthode `rang()`, qui renvoie la position de la valeur dans `VALEURS` : 0 pour un 7, 7 pour un `As` ;
- `__repr__`, qui renvoie le texte affiché quand on tape le nom de l'objet.

On doit obtenir :

```
>>> c = Carte("Roi", "coeur")
>>> c
Roi de coeur
>>> c.rang()
6
```

Pourquoi vaut-il mieux vérifier la valeur **dès le constructeur**, plutôt qu'au moment de comparer deux cartes ?

★★ **Exercice 3** *Comparer et trier des cartes* [au-delà du programme]

Le cours a signalé qu'il existe d'autres méthodes spéciales, qui permettent d'utiliser `==` ou `<` sur nos propres objets. Écrire `a == b` déclenche l'appel `a.__eq__(b)` ; écrire `a < b` déclenche `a.__lt__(b)` (*lt* pour *less than*).

1. ▶ Avant d'écrire ces méthodes, taper les deux instructions suivantes. Noter ce que Python répond à chacune (une valeur, ou une erreur). Que compare `==` par défaut ?

```
>>> Carte("Roi", "coeur") == Carte("Roi", "coeur")
>>> sorted([Carte("Roi", "coeur"), Carte("7", "pique")])
```

2. ▶ Écrire `__eq__(self, autre)` et `__lt__(self, autre)`, qui renvoient un booléen en comparant les **rangs** des deux cartes (la couleur ne compte pas). Vérifier que le tri donne bien les cartes de la plus faible à la plus forte :

```
>>> main = [Carte("Roi", "coeur"), Carte("7", "pique"),
...         Carte("As", "pique"), Carte("9", "coeur")]
>>> sorted(main)
[7 de pique, 9 de coeur, Roi de coeur, As de pique]
```

3. Avec ces définitions, `Carte("Roi", "coeur") == Carte("Roi", "trefle")` vaut `True`. Est-ce gênant pour la Bataille ? Dans quel autre jeu (ou quelle autre situation) cette définition de l'égalité poserait-elle problème ?

3 Les classes `Paquet` et `Joueur`

★★ **Exercice 4** *Le paquet*

1. ▶ Écrire le constructeur : il fabrique les 32 cartes (une par couple valeur/couleur) dans l'attribut privé `_cartes`, une liste de `Carte`, avec deux boucles `for` imbriquées : la boucle **extérieure** sur les couleurs (`COULEURS`), la boucle **intérieure** sur les valeurs (`VALEURS`). Cet ordre est imposé : les tests avec graine en dépendent. *Pour les plus à l'aise :* le réécrire ensuite en **une ligne**, avec une liste en compréhension à deux boucles `for`.
2. ▶ Écrire `taille()`, puis la méthode `melanger(graine=None)` (paramètre par défaut : voir le chapitre *Récursivité*). Si une graine est donnée, elle appelle d'abord `random.seed(graine)` ; dans tous les cas, elle mélange ensuite les cartes avec la fonction `random.shuffle`. (Le module `random` est déjà importé.) À quoi sert la graine quand on veut tester le programme ?
3. ▶ Écrire `distribuer(nb_joueurs)` : la carte d'indice `i` va au joueur `i % nb_joueurs` ; le paquet est vide ensuite, et la méthode renvoie la liste des mains (une liste de cartes par joueur). Pour deux joueurs, la carte 0 va au joueur 0, la carte 1 au joueur 1, la carte 2 au joueur 0, etc.

★★ **Exercice 5** *Le joueur*

La main est une liste de cartes dont la carte **du dessus** est au **début**.

1. ▶ Écrire le constructeur, `nb_cartes()`, `a_perdu()`, `jouer_carte()` (retire et renvoie la carte du dessus) et `ramasser(cartes)` (ajoute les cartes reçues, dans l'ordre, **à la fin** de la main, c'est-à-dire sous le tas). Le constructeur reçoit le nom et la liste des cartes de départ.
2. Le test « Partie 3 : Joueur » du fichier `projet_poo_depart.py` ajoute une carte à la liste `cartes` **après** avoir créé le joueur, et vérifie que la main d'Ada n'a pas changé. Parmi les deux écritures suivantes, laquelle faut-il donc mettre dans le constructeur : `self._main = cartes`, ou bien `self._main = [c for c in cartes]` ? Expliquer ce qui différencie les deux.

3. ▶ Écrire `nb_cartes_de(valeur)` en **une ligne**, avec une **liste en compréhension** : on garde les cartes de `self._main` dont la valeur est celle cherchée, et on compte le résultat avec `len(...)`. Pourquoi cette méthode est-elle nécessaire pour compter les As d'un joueur depuis l'extérieur de la classe ? ▷ coup de pouce

4 La classe `Partie` : faire jouer les objets ensemble

★★★ **Exercice 6** *Un pli, avec ses batailles*

Le constructeur de `Partie` est fourni : il crée un paquet, le mélange, le distribue et crée les deux joueurs `j1` et `j2`.

▶ Écrire `jouer_pli()` qui joue un pli complet et **renvoie le joueur** qui ramasse le tapis. Une liste locale `tapis`, vide au départ, accumule toutes les cartes posées pendant le pli (visibles ou cachées). On répète les étapes suivantes tant que le pli n'est pas terminé :

a) si l'un des joueurs n'a plus de cartes (`a_perdu()`), l'autre est le gagnant du pli : c'est fini ;
b) sinon, chaque joueur pose sa carte du dessus (`jouer_carte()`) : on les ajoute au tapis. Si l'une est strictement plus forte, son propriétaire est le gagnant du pli : c'est fini ;
c) sinon (égalité), on compte une bataille de plus (`nb_batailles`) et, si les deux joueurs ont encore des cartes, chacun en pose une de plus, **face cachée** : on l'ajoute simplement au tapis sans la comparer. On revient alors à l'étape a) (la carte face visible suivante sera comparée à l'étape b)).

▷ coup de pouce

★★ **Exercice 7** *Une partie entière, et un invariant*

1. ▶ Écrire `jouer()` : tant qu'aucun des deux joueurs n'a perdu **et** que `nb_plis` est inférieur à `max_plis`, jouer un pli (`jouer_pli()`) et augmenter `nb_plis` de 1. À la fin, renvoyer `self.vainqueur()` (méthode fournie : le nom du vainqueur, ou `None` si la limite a été atteinte). `max_plis` évite de boucler indéfiniment si la partie ne finit jamais.
2. ▶ On vérifie maintenant votre trace à la main (question 3 de l'exercice 1) avec le programme. Une `Partie` distribue au hasard, mais ici on veut des tas **imposés**. On procède donc ainsi :
   a) créer une partie quelconque : `p = Partie("Ada", "Alan")` ;
   b) remplacer ses deux joueurs par ceux de l'exercice 1, en affectant les attributs `p.j1` et `p.j2` :

```
p.j1 = Joueur("Ada", [...])
p.j2 = Joueur("Alan", [...])
```

   où chaque liste contient les cartes du tas, **la carte du dessus en premier** (par exemple `Carte("Dame", "coeur")`) ;
   c) lancer `p.jouer()` puis afficher le vainqueur, `p.nb_plis` et `p.nb_batailles`.

   Le programme annonce-t-il le même vainqueur et le même nombre de plis que votre trace ? Sinon, qui a tort : la trace ou le programme ?
3. Le test « Partie 4 : jouer » du fichier vérifie qu'en fin de partie les deux joueurs ont **32 cartes au total**. Pourquoi cette propriété (un **invariant**, c'est-à-dire une propriété qui reste vraie tout au long du programme) doit-elle être vraie à la fin de chaque pli ? Donner une erreur de programmation dans `jouer_pli` qui la ferait échouer.
4. ▶ Cette fois on joue une partie « pour de vrai » : `Partie("Ada", "Alan", 2026)` mélange le paquet de façon **reproductible** (la graine `2026` fixe le mélange : tout le monde obtient la même distribution) et distribue 16 cartes à chacun. Créer cette partie, la jouer avec `jouer()` et indiquer le vainqueur et le nombre de plis (`nb_plis`). Le test « Partie 4 : jouer » attend `"Alan"` et 106 plis.

5 Mille parties : des statistiques en style fonctionnel

On veut maintenant **observer** le jeu. Dans cette partie, on filtre et on transforme la liste des résultats avec des **listes en compréhension**, comme en Première. Certaines questions ne se répondent pas avec une compréhension seule : pour une **moyenne** ou la recherche d'un **minimum**, on écrit un parcours avec une variable qui accumule ou retient le meilleur résultat ; pour **classer** les parties selon un critère (`sorted`), on introduira une fonction `lambda`, en la construisant pas à pas.

★★ **Exercice 8** *Résumer une partie*

Pour observer le jeu, il faut d'abord pouvoir jouer *une* partie et en garder les faits marquants. On écrit pour cela la fonction `resume(graine, tapis_melange=False)` (le second paramètre ne sert que dans l'extension : on le transmet simplement à `Partie`).

1. ▶ Dans `resume`, créer la partie `Partie("Ada", "Alan", graine, tapis_melange=tapis_melange)`, relever le nombre d'As du joueur `j1` (Ada) **avant** de jouer, avec `nb_cartes_de("As")`, puis jouer la partie avec `jouer()`.
2. ▶ La fonction renvoie un **dictionnaire** aux cinq clés `"graine"` (la graine reçue), `"vainqueur"` (ce que renvoie `jouer()` : un nom, ou `None` si `max_plis` est atteint), `"plis"` (`nb_plis`), `"batailles"` (`nb_batailles`) et `"as_ada"`. Pour la graine 2026, on doit obtenir :

```
>>> resume(2026)
{'graine': 2026, 'vainqueur': 'Alan', 'plis': 106, 'batailles': 8, 'as_ada': 1}
```

3. Pourquoi faut-il relever le nombre d'As **avant** d'appeler `jouer()` ? Que vaudrait-il après ?
4. `resume` est-elle une fonction **pure** ? Que deviendrait la réponse si l'on supprimait le paramètre `graine` ?

★★ **Exercice 9** *Interroger les mille parties*

On joue mille parties (graines 0 à 999) et on garde leurs résumés : `res = [resume(g) for g in range(1000)]`. `res` est une liste de 1000 dictionnaires, comme celui de l'exercice précédent. Une partie est **finie** si elle a un vainqueur ; sinon elle a atteint `max_plis`.

1. ▶ Avec une liste en compréhension, construire la liste `finies` des parties qui ont un vainqueur (ne garder que les dictionnaires `r` tels que `r["vainqueur"]` n'est pas `None`). Combien y en a-t-il ? Avec une seconde compréhension sur `finies`, construire la liste `gagnees_ada` des parties dont le vainqueur est `"Ada"` : combien Ada en gagne-t-elle, et quel pourcentage des parties finies cela représente-t-il ?
2. ▶ Avec une liste en compréhension, extraire de `finies` la liste des nombres de plis, puis en calculer la **durée moyenne** (écrire une fonction `moyenne(tab)` qui additionne les valeurs dans un accumulateur, puis divise par `len(tab)`).
3. On veut maintenant les graines des **trois** parties finies les plus longues, et celle de la plus courte.
   a) Les trois plus longues. Ici, une compréhension ne suffit plus : on ne filtre ni ne transforme `finies`, on le **classe** selon le nombre de plis de chaque partie. `sorted` accepte pour cela un paramètre `key` : une fonction qui, à partir d'un résumé `r`, renvoie la valeur à comparer. Cette fonction s'écrit avec un `lambda` : `lambda r: r["plis"]` se lit « la fonction qui, à `r`, associe `r["plis"]` » ; c'est l'équivalent, pour un usage unique, de `def cle(r): return r["plis"]`. ▶ Afficher, sous la forme (graine, nombre de plis), les trois parties les plus longues avec `sorted(finies, key=..., reverse=True)[:3]`.
   b) La plus courte. ▶ Parcourir `finies` en retenant le résumé qui a le moins de plis (recherche du minimum, comme en Première), puis afficher sa graine et son nombre de plis.

4. ▶ Recopier et compléter le tableau suivant : pour chaque nombre k d'As d'Ada au départ (clé `"as_ada"`), le nombre de parties finies avec ce k et le pourcentage d'entre elles gagnées par Ada (arrondi à l'unité à l'affichage, avec une f-string : `f"{p:.0f}"`). Pour chaque k, on construit par compréhension le groupe des parties finies correspondantes, puis le sous-groupe gagné par Ada. Conclure : avoir des As aide-t-il ? Peut-on se fier aux pourcentages de tous les groupes ?

As d'Ada au départ	0	1	2	3	4
Parties finies					
Victoires d'Ada (%)					

6 Extension : des parties qui ne finissent jamais ?

★★★ **Exercice 10** *Casser les cycles*

1. Plus d'un tiers des parties atteignent `max_plis = 2000` sans vainqueur. En augmentant `max_plis` à 20 000 pour quelques-unes, vérifier qu'elles ne finissent toujours pas. Proposer une explication : le programme est **déterministe** ; que se passe-t-il s'il retrouve exactement la même situation (mêmes tas) qu'à un pli précédent ?
2. ▶ Dans une vraie partie, les cartes ramassées ne sont pas reposées dans un ordre fixe. Ajouter à `Partie` une méthode privée `_donner(gagnant, tapis)` qui mélange le tapis si l'attribut `tapis_melange` vaut `True`, puis le fait ramasser ; l'utiliser à chaque ramassage dans `jouer_pli`.
3. ▶ Refaire les statistiques avec `tapis_melange=True` : nombre de parties finies, durée moyenne, partie la plus longue, victoires d'Ada. Les tests des parties 1 à 5 passent-ils toujours ? Pourquoi est-ce important ?
4. *(Libre.)* Autres pistes : une partie à trois joueurs ; un affichage pli par pli ; une variante où le gagnant d'une bataille choisit l'ordre du ramassage.

7 Bilan à rédiger

Exercice 11 *Ce que j'ai construit*

Rédiger, en une dizaine de lignes, un bilan qui répond aux questions suivantes.

1. Pour chacune des quatre classes, en une phrase : ce qu'elle représente et la méthode qui vous paraît la plus importante.
2. Citer un endroit du projet où l'**encapsulation** a imposé d'écrire une méthode plutôt que d'accéder directement à un attribut. Qu'y a-t-on gagné ?
3. Le projet mêle trois paradigmes. Citer pour chacun (impératif, objet, fonctionnel) un passage de votre code qui l'illustre, et dire pourquoi il convenait à cet endroit.

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 5 q. 3 – piste `[c for c in self._main if ...]` garde les cartes `c` de la main qui vérifient la condition ; la condition compare `c.valeur()` à la valeur cherchée. ↑énoncé

Ex. 6 – piste Une variable `gagnant`, qui vaut `None` tant que le pli n'est pas joué, et une boucle `while gagnant is None` ; après la boucle, `gagnant` ramasse le tapis (`ramasser`) et la méthode le renvoie. Les comparaisons s'écrivent `c2 < c1` et `c1 < c2` grâce à `__lt__`. Les cartes posées s'ajoutent au tapis dans l'ordre : carte de `j1`, puis carte de `j2`. ↑énoncé

2

TP Balles rebondissantes

la POO à la rescousse

Durée : 2 h 30 environ (deux séances) · Sur machine, par deux

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t02-tp-balles` — archive .zip

But du TP

Faire rebondir des balles dans une fenêtre : une, puis deux, puis trois… puis vingt. Sans objet, chaque balle ajoute une poignée de variables et un bloc de code recopié : on va **ressentir** cette gêne avant de la faire disparaître avec une **classe** `Balle`, qui réunit les données d'une balle et ce qu'elle sait faire. On réinvestit tout le début du chapitre : classe, instance, attributs, constructeur, méthodes, `self`. ▶ signale ce qui est à programmer et tester.

Consignes

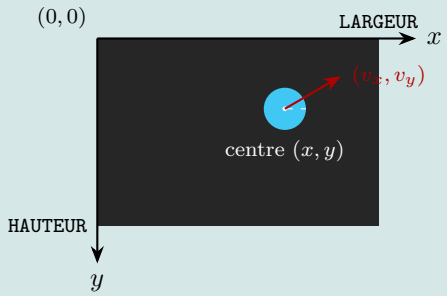
- Défi compris. Le TP est **évalué** : code rendu par le binôme, puis entretien individuel de 5 minutes (partie VI).
- Le fichier `tp_balles_depart.py` est **à télécharger** (lien ci-dessus) : la fenêtre, le dessin des disques, les boucles d'animation, les squelettes des fonctions et méthodes à écrire et, en fin de fichier, des **tests**. La constante `ANIMATION`, en haut du fichier, choisit l'animation lancée après les tests (0 : tests seuls). Relancer le fichier après chaque exercice : chaque ligne `[A FAIRE]` doit devenir `[OK]`. On ne modifie pas les tests.
- L'affichage utilise **tkinter**, livré avec Python : rien à installer.
- Usage de l'IA, indiqué sur chaque exercice :  aucune ;  en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

Lancer une fenêtre tkinter depuis Spyder

- **Une seule fois** : menu *Exécuter* → *Configuration par fichier* (`Ctrl+F6`), cocher « Exécuter dans une nouvelle console dédiée » et « Supprimer toutes les variables avant l'exécution », valider. Ensuite, `F5` comme d'habitude.
- La fenêtre s'ouvre parfois **derrière** Spyder : la chercher dans la barre des tâches.
- Tant que la fenêtre est ouverte, la console est occupée : **fermer la fenêtre** pour reprendre la main. Si la console ne répond plus : *Redémarrer le noyau* (`Ctrl+.`).

1 Une balle, sans objet

Le repère de la fenêtre et l'animation



L'origine est en **haut à gauche** et l'axe des y est dirigé **vers le bas** : $v_y > 0$, la balle descend. À chaque image, la balle avance de sa vitesse : x augmente de v_x, y de v_y (en pixels).

Pour animer, on n'écrit **pas** de boucle `while True` : tant qu'elle tourne, tkinter ne peut ni redessiner la fenêtre ni la fermer, et tout se fige. On demande plutôt à tkinter : « dans `DELAI` millisecondes, appelle `animer1` avec ces valeurs ». C'est `fenetre.after(DELAI, animer1, ...)`. Comme `animer1` se termine par cette même demande, elle est rappelée sans fin, et tkinter garde la main entre deux images. `fenetre.mainloop()` lui donne la main jusqu'à la fermeture de la fenêtre.

★ **Exercice 1** *Une balle qui rebondit*

1. ▶ Mettre `ANIMATION = 1` et lancer le fichier : la balle reste immobile. Lire `animer1` et expliquer pourquoi.
2. ▶ Compléter `animer1` pour que la balle avance de `(vx, vy)` à chaque image. Que se passe-t-il au bout de quelques secondes ?
3. Le disque a pour rayon `RAYON`. Écrire la condition qui traduit « le disque touche le bord droit », puis celles des trois autres bords. ▷ coup de pouce
4. ▶ Compléter `animer1` : quand le disque touche le bord gauche ou droit, `vx` change de signe ; le haut ou le bas, `vy` change de signe.
5. ▶ Essayer `DELAI = 5` puis `DELAI = 100`. Combien d'images par seconde dans chaque cas ? Revenir à `20`.

2 Deux balles, puis trois

★ **Exercice 2** *Deux balles, par copier-coller*

1. ▶ Écrire une fonction `animer2` et compléter `lancer2` pour animer **deux** balles, en partant d'une copie de `animer1` et de `lancer1`. Les variables de la première balle s'appellent `x1`, `y1`, `vx1`, `vy1`, `disque1` ; la seconde (couleur `"gold"`) part de $(100, 300)$ avec la vitesse $(-3, 5)$. Tester avec `ANIMATION = 2`.
2. Combien de paramètres la fonction `animer2` a-t-elle ? Combien de lignes de `animer1` ont été recopiées ?
3. On veut maintenant une **troisième** balle, et des balles de rayons différents. Sans forcément l'écrire, lister tout ce qu'il faudrait ajouter ou modifier. Combien de variables faut-il pour décrire **une** balle (position, vitesse, rayon, couleur, numéro du disque) ? pour trois ? pour vingt ?

★★ **Exercice 3** *Le piège du copier-coller*

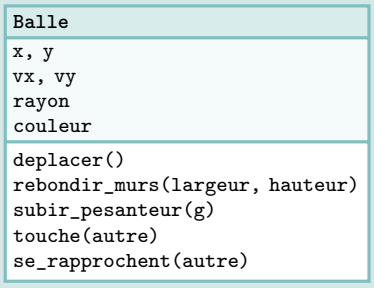
Un élève a écrit ses deux balles par copier-coller ; voici la partie qui gère les bords gauche et droit :

```
    if x1 - RAYON <= 0 or x1 + RAYON >= LARGEUR:
        vx1 = -vx1
    if x2 - RAYON <= 0 or x2 + RAYON >= LARGEUR:
        vx1 = -vx1
```

1. Repérer l'erreur. **Prédire**, sans rien lancer, ce que l'on verra à l'écran : que devient la balle 2 quand elle arrive sur un bord ? et la balle 1 **ensuite** ?
2. ▶ Introduire cette erreur dans votre `animer2`, vérifier la prédiction, puis la corriger.
3. Python n'a signalé aucune erreur. Pourquoi ce genre de faute est-il fréquent quand on recopie du code, et difficile à repérer ?
4. Les sept valeurs qui décrivent une balle sont dispersées dans sept variables, et le code qui la fait bouger est écrit à part, une fois par balle. Que faudrait-il pouvoir écrire pour dire, en une seule fois, « une balle » et « ce que fait une balle » ?

3 La POO à la rescousse

Une classe `Balle`



Une balle est **un objet** : ses six **attributs** (position, vitesse, rayon, couleur) voyagent ensemble, et ses **méthodes** disent ce qu'elle sait faire. Le code d'une méthode est écrit **une seule fois**, dans la classe ; il sert à toutes les **instances**. Dans l'appel `b.deplacer()`, le paramètre `self` de la méthode désigne l'objet `b`.

La classe ne sait rien de la fenêtre : elle ne fait que des calculs, que l'on peut **tester** avec des `assert` sans rien afficher. L'affichage (fourni) se contente de lire `b.x`, `b.y`, `b.rayon` et `b.couleur`.

★ **Exercice 4** *La classe* `Balle`

1. ▶ Compléter le constructeur `__init__(self, x, y, vx, vy, rayon, couleur)`, qui crée les six attributs.

```
>>> b = Balle(100, 50, 3, -2, 10, "gold")
>>> b.x, b.vy, b.couleur
(100, -2, 'gold')
```

2. Pendant l'exécution de `Balle(100, 50, 3, -2, 10, "gold")`, que désigne `self` ? Que vaut `self.x` à la fin du constructeur ?

★★ **Exercice 5** *Se déplacer et rebondir*

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1. ▶ Écrire la méthode `deplacer(self)` : la balle avance d'un pas (sa position augmente de sa vitesse).
2. ▶ Avant d'écrire `rebondir_murs`, écrire dans la fonction `mes_tests` trois `assert` sur cette méthode : un cas normal, un cas limite, un cas piège. Les tests en fin de fichier ne vérifient pas tout : les vôtres comptent.

3. ▶ Écrire la méthode `rebondir_murs(self, largeur, hauteur)` : si le disque touche le bord gauche ou droit de la zone **en allant vers ce bord**, `vx` change de signe ; de même pour `vy` avec le haut et le bas. La méthode ne déplace pas la balle. ▷ coup de pouce
4. Une balle de rayon 10 a son centre à 598 pixels du bord gauche d'une zone de largeur 600, et repart déjà vers la gauche (`vx = -3`). Que ferait une version de `rebondir_murs` qui ne regarde **pas** le sens de la vitesse ? Quand une balle peut-elle se retrouver dans cette situation ?

★★ **Exercice 6** *Vingt balles sans effort*

1. ▶ Écrire la fonction `balle_au_hasard(largeur, hauteur)`, qui renvoie une `Balle` tirée au hasard : rayon entier entre 8 et 25, disque **entièrement** dans la zone, `vx` et `vy` pris dans la liste `VITESSES`, couleur prise dans `COULEURS`. On utilise `random.randint(a, b)`, qui renvoie un entier entre `a` et `b` compris, et `random.choice(liste)`, qui renvoie un élément de la liste au hasard. ▷ coup de pouce
2. ▶ Écrire `creer_balles(nb, largeur, hauteur)`, qui renvoie une liste de `nb` balles au hasard, construite **par compréhension**.
3. ▶ Lancer avec `ANIMATION = 3`, puis passer à 200 balles. Combien de lignes a-t-il fallu modifier ? Comparer avec la partie II.
4. Où sont passées les variables `x1`, `y1`, `vx1`… de la partie II ? Lire la fonction fournie `animer3` : combien de paramètres a-t-elle, quel que soit le nombre de balles ?
5. Un élève écrit `return [balle_au_hasard(largeur, hauteur)] * nb`. Combien d'objets `Balle` ont été créés ? Qu'est-ce qui s'affichera à l'écran ?

★★ **Exercice 7** *Un comportement nouveau, pour toutes les balles*

1. ▶ Ajouter la méthode `subir_pesanteur(self, g)`, qui augmente `vy` de `g` (on rappelle que l'axe des y descend). Dans `animer3`, l'appeler avec `PESANTEUR` juste avant `deplacer`, puis mettre `PESANTEUR = 0.3`.
2. ▶ Laisser tourner une bonne minute. Que deviennent les balles ? Expliquer ce qui se passe quand une balle n'a presque plus de vitesse au moment où elle touche le sol.
3. ▶ Corriger `rebondir_murs` : quand la balle rebondit sur un bord, la **replacer** contre ce bord (par exemple `self.y = hauteur - self.rayon` pour le sol). Relancer les tests et l'animation.
4. Combien de méthodes a-t-il fallu modifier pour corriger les vingt balles ? Combien d'endroits du code aurait-il fallu modifier dans la version de la partie II à trois balles ?

4 Pour aller plus loin

★★★ **Exercice 8** *Défi — des balles qui se heurtent*

1. ▶ Écrire la méthode `touche(self, autre)`, qui renvoie `True` si les disques `self` et `autre` se touchent ou se chevauchent. Dans l'appel `a.touche(b)`, que désignent `self` et `autre` ? ▷ coup de pouce
2. ▶ Écrire la méthode `se_rapprochent(self, autre)`, qui renvoie `True` si la distance entre les deux centres sera plus petite au pas suivant (chaque balle avancée de sa vitesse) qu'elle ne l'est maintenant. La méthode ne déplace pas les balles.
3. ▶ Écrire la fonction `gerer_collisions(balles)` : pour chaque paire de balles qui se touchent **et** se rapprochent, on **échange** leurs vitesses (`vx` avec `vx`, `vy` avec `vy`). Chaque paire n'est examinée qu'**une fois**. Tester avec `ANIMATION = 4`. ▷ coup de pouce
4. Pourquoi faut-il tester `se_rapprochent` ? Imaginer deux balles qui se chevauchent encore à l'image qui suit l'échange. Et que se passerait-il si chaque paire était examinée deux fois, une fois comme `(a, b)` et une fois comme `(b, a)`, sans ce test ?

5. [au-delà du programme] L'échange des vitesses est exact pour deux balles de même masse qui se heurtent de face. Observer des chocs « en biais » ou entre une petite et une grosse balle : le résultat paraît-il réaliste ?

5 Bilan à rédiger

Exercice 9  *Ce que la POO a changé*

Rédiger, en une dizaine de lignes, un bilan qui répond aux questions suivantes.

1. Comparer la partie II et la partie III : nombre de variables, travail pour ajouter une balle, nombre d'endroits à modifier pour changer le comportement de toutes les balles.
2. Définir avec vos mots, en vous appuyant sur ce TP : classe, instance, attribut, méthode, `self`.
3. La classe `Balle` ne sait rien de la fenêtre. Pourquoi est-ce utile ? Qu'est-ce que cela a permis de faire dans ce TP ?

6 Évaluation : le code du binôme, puis un entretien individuel

Le rendu. Le binôme rend un seul fichier `tp_balles.py`, avec les deux noms en commentaire sur la première ligne : parties I à III au moins, la fonction `animer2` de la partie II comprise (elle reste lourde : c'est voulu), et vos `assert` dans `mes_tests`. Le code est noté sur 10, la même note pour les deux ; l'entretien, sur 10, est individuel. Note du TP : la somme des deux, sur 20.

Le code rendu par le binôme	Points
Parties I et II : une balle, puis deux, qui rebondissent correctement (exercices 1 et 2)	2
Classe `Balle` : constructeur, `deplacer`, `rebondir_murs` ; tests `[OK]` (exercices 4 et 5)	3
Vingt balles : `balle_au_hasard`, `creer_balles` par compréhension, animation (exercice 6)	2
Pesanteur et balles replacées contre les bords (exercice 7)	1
Qualité : docstrings, noms parlants, trois `assert` par méthode écrite (normal, limite, piège)	1
Défi : collisions entre balles (exercice 8)	1

L'entretien. Chacun passe **seul** un entretien de 5 minutes, noté sur 10, devant le code **rendu par le binôme**, ouvert à l'écran. Il s'agit de vérifier que chacun comprend et sait faire évoluer **tout** le code rendu, y compris les lignes écrites par son camarade : pendant l'entretien, on peut lancer le programme, mais ni IA, ni notes, ni aide du camarade. Les questions portent sur votre code et sont choisies par le professeur ; les deux membres du binôme n'ont pas les mêmes.

Question type	Ce qui est attendu	Points
Expliquer une méthode désignée par le professeur, en une phrase	2 : la phrase donne le **but** de la méthode ; 1 : description ligne à ligne exacte, sans le but ; 0 sinon.	2
Dire ce que vaut `self` (et `autre`) dans un appel du code, par exemple `balles[i].deplacer()`	2 : le bon objet, avec la raison (c'est l'objet placé avant le point) ; 1 : le bon objet sans raison ; 0 sinon.	2
Prédire l'effet d'une modification d'une ligne, **avant** de lancer	2 : effet à l'écran juste et expliqué ; 1 : effet juste mais mal expliqué, ou en partie juste ; 0 sinon.	2
Modifier en direct une méthode (2 minutes) pour une petite évolution demandée	3 : modification juste, au bon endroit, testée ; 2 : bon endroit, une erreur corrigée après une indication ; 1 : sait où intervenir sans y parvenir ; 0 sinon.	3
Donner un **avantage concret** de la classe sur les variables `x1`, `y1`… de la partie II, sur ce code	1 : un avantage précis, illustré par le code ; 0 sinon.	1

Pour s'y préparer : relire chaque méthode du fichier rendu, savoir dire ce que fait chaque ligne et pourquoi elle est là, et s'entraîner à prévoir ce que donnerait une petite modification. Un « je ne sais pas » honnête vaut mieux qu'une réponse inventée.

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 q. 3 – piste Le point le plus à droite du disque a pour abscisse `x + RAYON`. Le bord gauche est à l'abscisse 0, le bord haut à l'ordonnée 0, le bas à `HAUTEUR`. ↑énoncé

Ex. 5 q. 3 – piste « En allant vers le bord droit », c'est `self.vx > 0`. Une condition par bord, de la forme « le disque touche ce bord **et** la vitesse va vers lui », puis deux tests : l'un pour `vx` (bords gauche et droit, reliés par `or`), l'autre pour `vy`. ↑énoncé

Ex. 6 q. 1 – piste Tirer le rayon en premier : le centre doit être à au moins un rayon de chaque bord, donc `x` est entre `rayon` et `largeur - rayon`. ↑énoncé

Ex. 8 q. 1 – piste Deux disques se touchent quand la distance entre leurs centres est inférieure ou égale à la somme des rayons. Avec le théorème de Pythagore, on compare plutôt les carrés : `dx ** 2 + dy ** 2 <= (self.rayon + autre.rayon) ** 2`, sans racine carrée. ↑énoncé

Ex. 8 q. 3 – piste Deux boucles sur les indices : `i` de `0` à `n - 1`, puis `j` de `i + 1` à `n - 1`. Pour échanger deux valeurs en Python : `a.vx, b.vx = b.vx, a.vx`. ↑énoncé

FICHE DE RÉVISION

L'essentiel

- **Classe** : le *moule* (le plan). **Instance** (objet) : fabriquée par `Classe(...)`.
- **Attribut** : une donnée de l'objet. **Méthode** : une fonction définie dans la classe.
- `__init__` : le **constructeur**, il crée les attributs. `self` : l'instance courante, **1er paramètre** de toute méthode.
- **Encapsulation** : on cache l'état (attribut `_x`) et, **depuis l'extérieur**, on n'y touche que par des méthodes (**accesseur** pour lire, **mutateur** pour modifier) ; dans la classe, `self._x` est permis (on passe par le mutateur s'il vérifie quelque chose).
- `__repr__` : ce qui s'affiche quand on tape l'objet dans la console (`__str__` : pour `print`).

La classe modèle à savoir écrire

```python
class CompteBancaire:
    def __init__(self, titulaire, solde_initial):
        self.titulaire = titulaire
        self._solde = solde_initial     # attribut protege

    def solde(self):                    # accesseur : lire l'etat
        return self._solde

    def deposer(self, montant):         # mutateur : modifier l'etat
        if montant > 0:
            self._solde = self._solde + montant

    def retirer(self, montant):
        if 0 < montant <= self._solde:  # on refuse de passer en negatif
            self._solde = self._solde - montant
            return True
        return False

compte = CompteBancaire("Alice", 100)
compte.deposer(50)
compte.retirer(30)
print(compte.solde())                   # affiche 120
```

Les paradigmes (culture)

Impératif : une suite d'instructions qui modifient l'état. **Objet** : des objets qui portent données *et* comportements. **Fonctionnel** : des **fonctions pures** (résultat dépendant des seuls paramètres, **sans effet de bord**). Ces styles se **combinent**. *(Héritage et polymorphisme : hors programme.)*

Erreurs à éviter

- confondre la **classe** (moule) et l'**instance** (objet) ;
- oublier `self` (paramètre **et** `self.attribut`) ;
- attribut non initialisé dans `__init__` ⇒ `AttributeError` ;
- toucher `_solde` directement au lieu de passer par une méthode ;
- `return` oublié dans une méthode ⇒ `None`.

Savoir-faire à maîtriser — je sais…

- ☐ distinguer **classe** et **instance**, **attribut** et **méthode** ;
- ☐ écrire une classe avec `__init__`, des attributs et des méthodes ;
- ☐ créer une instance et appeler ses méthodes ;
- ☐ utiliser `self` correctement ;
- ☐ respecter l'**encapsulation** (attribut `_x`, accesseur / mutateur) ;
- ☐ écrire `__repr__` (et `__str__`) ;
- ☐ lire et **compléter** une classe donnée (type bac) ;
- ☐ reconnaître les **paradigmes** (impératif, objet, fonctionnel) et ce qu'est une **fonction pure** ;
- ☐ utiliser un **module** et sa **documentation** (`import`, `help`), créer un module **documenté**.

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 789

1. Dans `v = Voiture("Renault")`, qui est la classe, qui est l'instance ?
2. Après `c = CompteBancaire("Bo", 10)`, `c.retirer(20)` puis `c.deposer(5)`, que vaut `c.solde()` ?
3. Dans `__init__`, un élève écrit `nom = nom`. Quelle erreur apparaît ensuite ?
4. Une fonction `f(t)` qui fait `t.append(0)` puis renvoie `t` est-elle pure ?
5. Quel paradigme choisir pour modéliser les personnages d'un jeu, chacun avec son état ?

Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 789

1. Écrire la condition d'arrêt et le cas récursif de `puissance(x, n)`, pour `n` entier positif. (ch. 1)
2. `somme_liste(tab)` s'appelle sur `tab[1:]`. Quel variant garantit qu'elle se termine ? (ch. 1)
3. Avec `d = {"Ada": 15, "Alan": 12}`, comment lire la note d'Alan ? ajouter `"Grace"` avec 18 ? (Première)

DÉFIS ADVENT OF CODE

DÉFI 1 Toboggan Trajectory

la descente en luge — grille et modulo

Jour 3

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t02-defi-aoc-2020-03` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/3`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Une luge descend une pente boisée en ligne droite, et l'on veut savoir combien d'arbres elle va percuter. La carte de la forêt compte plusieurs centaines de lignes et, en théorie, elle se prolonge sans fin vers la droite : impossible de suivre la trajectoire à la main, et impossible de stocker une carte infinie. Le programme doit donc retrouver, pour chaque case atteinte, la case correspondante de la carte réelle.

Ce qu'on réinvestit. L'**encapsulation** vue dans ce chapitre. Le calcul qui ramène une colonne quelconque dans la carte réelle (un modulo) sera caché dans une classe `Foret`, derrière une méthode `est_arbre(x, y)` (exercice « Cacher le modulo dans une classe »). L'utilisateur de la classe croit manipuler une forêt infinie, alors que l'objet ne stocke que le motif de la carte : c'est exactement le rôle d'une interface, qui sépare ce qu'on fait de la façon dont c'est fait.

La démarche de la fiche.

1. Comprendre le déplacement et la répétition de la carte, puis suivre la luge à la main

sur une petite carte.
2. Lire la carte dans une liste de chaînes.
3. Écrire une fonction qui compte les arbres pour une pente quelconque : elle donne la partie 1.
4. Cacher le calcul du modulo dans une classe `Foret` (encapsulation).
5. Partie 2 : appeler cette fonction pour cinq pentes et multiplier les résultats.
6. En approfondissement, dessiner le trajet sur la carte.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`toboggan`	luge	`slope`	pente (déplacement)
`tree`	arbre	`open square`	case libre
`grid`	grille	`repeats to the right`	se répète vers la droite
`right 3, down 1`	3 à droite, 1 en bas	`top-left corner`	coin en haut à gauche
`encounter`	rencontrer	`bottom`	bas (de la carte)

Résumé de l'énoncé (partie 1)

Contexte. Toujours en route vers l'aéroport, le héros dévale une pente boisée en luge, sans pouvoir vraiment la diriger. Il dispose d'une carte de la forêt vue de dessus et veut savoir combien d'arbres il va percuter en suivant une trajectoire fixée.
Les données. Le fichier est cette carte. Chaque ligne du fichier représente une rangée de la pente (la première ligne est en haut), et chaque caractère une case : `.` désigne une case libre et `#` un arbre. Toutes les lignes ont la même longueur.
La règle.

- La forêt est bien plus large que la carte : le même motif se répète indéfiniment vers la **droite** (mais pas vers le bas). Autrement dit, après la dernière colonne, on retrouve la première.
- On part de la case en haut à gauche (ligne 0, colonne 0).
- À chaque pas, on se déplace de 3 colonnes vers la droite et de 1 ligne vers le bas. Par exemple, sur une carte large de 5 cases, le 2e pas mène à la colonne 6 de la forêt, qui correspond à la colonne 1 de la carte.
- On regarde la case atteinte à chaque pas ; la case de départ ne compte pas.
- On s'arrête quand on dépasse la dernière ligne de la carte.

La réponse attendue. Le nombre d'arbres sur les cases atteintes. Par exemple, si les cases atteintes contiennent, dans l'ordre, `.`, `#`, `#`, `.`, `#`, la réponse serait 3.

Exercice 1  *Comprendre ce qui est demandé*

But. Vérifier qu'on sait suivre la luge, en particulier quand elle sort de la carte par la droite.
À quoi ça sert ? La première question fait découvrir le calcul de colonne (le modulo) qu'utilisera le programme ; la deuxième fixe le nombre de tours de la boucle ; la troisième explique pourquoi on ne recopie pas la carte.

Après lecture de l'énoncé et du résumé, vérifier sa compréhension :

1. Sur une carte de largeur 7, dans quelle colonne de la carte réelle arrive-t-on au 4e pas ?
2. Sur une carte de 9 lignes, combien de cases visite-t-on en plus de la case de départ ?
3. Pourquoi est-il maladroit de recopier la carte vers la droite avant de commencer ?

Exercice 2 *À la main, sur un petit exemple*

But. Suivre la trajectoire à la main sur une carte de neuf lignes.
À quoi ça sert ? On obtient un exemple dont on **connaît la réponse** (2) pour tester la fonction de comptage ; dessiner la carte prolongée aide aussi à voir que la colonne réelle se calcule par un reste.

On considère la carte (inventée) suivante, large de 7 cases :

```
..#....
#...#..
.#...#.
...#..#
#.#....
..#.#..
.#...#.
#......
...##..
```

Recopier la carte sur le cahier, prolongée vers la droite, et marquer les cases visitées avec le déplacement de la partie 1. Vérification : on doit rencontrer 2 arbres.

2 Programmer la partie 1

Exercice 3 *Lire la carte*

But. Ranger la carte dans une liste de chaînes `carte`, une chaîne par ligne.
À quoi ça sert ? Avec cette liste, la case de la ligne `y` et de la colonne `x` s'obtient directement par `carte[y][x]`. Afficher hauteur et largeur vérifie la lecture et donne la largeur, indispensable pour le modulo.

Construire la liste `carte` des lignes de `input.txt` (des chaînes, sans le retour à la ligne final). Afficher la hauteur et la largeur de la carte. ▷ coup de pouce

Exercice 4 *Compter les arbres*

But. Écrire une fonction qui compte les arbres rencontrés pour une pente quelconque `(dx, dy)`.
À quoi ça sert ? Appelée avec la pente `(3, 1)`, elle donne la réponse de la partie 1. Avoir mis la pente en paramètres, plutôt que d'écrire 3 et 1 en dur, permettra de la réutiliser telle quelle dans la partie 2.

Écrire une fonction `compter_arbres(carte, dx, dy)` qui renvoie le nombre d'arbres rencontrés en se déplaçant de `dx` cases vers la droite et `dy` vers le bas à chaque pas. Tester sur l'exemple inventé de la fiche, puis sur vos données. ▷ coup de pouce ▷ début de solution

★★ Exercice 5 *Cacher le modulo dans une classe*

But. Écrire une classe `Foret` qui donne l'illusion d'une forêt infinie.
À quoi ça sert ? C'est l'encapsulation du chapitre : l'utilisateur de la classe demande si une case est un arbre sans savoir que la carte est répétée par un modulo. Le comptage des arbres s'écrit alors sans aucun calcul de colonne.

Écrire une classe `Foret` dont le constructeur reçoit la liste `carte`, avec deux méthodes :

- `hauteur(self)` : renvoie le nombre de lignes de la forêt ;
- `est_arbre(self, x, y)` : renvoie `True` si la case de la colonne `x` et de la ligne `y` de la forêt infinie est un arbre, `False` sinon.

Écrire ensuite `compter_arbres_foret(foret, dx, dy)`, qui fait le même travail que `compter_arbres` à partir d'un objet `Foret`, **sans** utiliser `%` ni `len(carte[0])`. Vérifier qu'elle donne les mêmes résultats. ▷ coup de pouce

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Le héros veut comparer plusieurs trajectoires avant de choisir la moins dangereuse. La carte et la façon de compter les arbres ne changent pas ; seule la pente (le déplacement à chaque pas) varie.
La nouvelle règle.

- Refaire le comptage pour cinq pentes : (1 à droite, 1 en bas), (3, 1), (5, 1), (7, 1) et (1 à droite, **2** en bas).
- Pour la dernière pente, on descend de deux lignes à chaque pas : on saute donc une ligne sur deux.

La réponse attendue. Le **produit** des cinq nombres d'arbres obtenus. Par exemple, si les cinq pentes rencontraient 2, 3, 1, 4 et 2 arbres, la réponse serait 48.

Exercice 6 *Plusieurs pentes*

But. Appeler la fonction de comptage pour les cinq pentes et multiplier les résultats.
À quoi ça sert ? C'est la réponse de la partie 2. On mesure l'intérêt d'une fonction bien paramétrée : la partie 2 ne demande que quelques lignes de plus.

Utiliser la fonction précédente pour toutes les pentes demandées et calculer la réponse. Sur l'exemple inventé de la fiche, on doit trouver 72. ▷ coup de pouce ▷ début de solution

4 Approfondissement

Méthode : d'une chaîne à une liste et retour

Une chaîne n'est pas modifiable : `ligne[2] = "X"` provoque une erreur. On la convertit en liste de caractères par une compréhension de liste, puis, une fois la liste modifiée, on reconstruit une chaîne avec une boucle d'accumulation, en partant de la chaîne vide.

```
ligne = "..#.."
cases = [c for c in ligne]        # ['.', '.', '#', '.', '.']
cases[2] = "X"                    # une liste, elle, se modifie
texte = ""
for c in cases:
    texte = texte + c             # on ajoute les caracteres un a un
print(texte)                      # ..X..
```

Intérêt. La liste sert de brouillon modifiable ; la chaîne n'est reconstruite qu'une fois, à la fin.
À essayer. Que se passe-t-il si l'on exécute directement `ligne[2] = "X"` ? Écrire une fonction `recoller(morceaux, sep)` qui renvoie les chaînes de la liste `morceaux` mises bout à bout, séparées par `sep` : `recoller(["a", "b", "c"], "-")` doit renvoyer `"a-b-c"` (pas de séparateur à la fin).

★★★ **Exercice 7**  *Défi : visualiser le trajet*

But. Afficher la carte avec le trajet de la luge, en marquant les cases visitées.
À quoi ça sert ? Voir le trajet permet de repérer d'un coup d'œil une erreur de déplacement (une ligne sautée, un modulo oublié). C'est aussi l'occasion de contourner le fait qu'une chaîne n'est pas modifiable.

Écrire une fonction qui affiche la carte en remplaçant chaque case visitée par `O` (case libre) ou `X` (arbre). Les chaînes n'étant pas modifiables, comment s'y prendre ? ▷ coup de pouce

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste `f.read().split()` donne directement la liste des lignes sans les `\n`. ↑ énoncé

Ex. 4 – piste Inutile de recopier la carte vers la droite : la colonne `x` d'une carte infinie correspond à la colonne `x % largeur` de la carte réelle. ↑ énoncé

Ex. 4 – début de solution Une boucle `for y in range(dy, len(carte), dy)` donne les lignes visitées (la case de départ, en ligne 0, n'est pas comptée) ; la colonne `x` augmente de `dx` à chaque tour. ↑ énoncé

Ex. 5 – piste Le constructeur range la carte et sa largeur dans des attributs ; le modulo n'apparaît plus que dans `est_arbre`. ↑ énoncé

Ex. 6 – piste Ranger les pentes dans une liste de tuples `(dx, dy)` et accumuler dans une variable initialisée à `1` (et non à `0`, puisqu'on multiplie). ↑ énoncé

Ex. 6 – début de solution Si la réponse de la partie 2 est fausse alors que la partie 1 est juste, tester la pente qui descend de plus d'une ligne : votre boucle saute-t-elle bien des lignes ? ↑ énoncé

Ex. 7 – piste Convertir chaque ligne en liste avec `[c for c in ligne]`, la modifier, puis reconstruire la chaîne avec une boucle d'accumulation (voir l'encadré « Méthode »). ↑ énoncé

2

DÉFI 2 Rain Risk

la navigation — programmation orientée objet

Jour 12

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t02-defi-aoc-2020-12` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/12`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Un navire reçoit plus de sept cents ordres de navigation : se décaler, pivoter, avancer. Il faut savoir où il se trouve à la fin, mesuré par sa distance au point de départ. Chaque ordre est simple, mais leur enchaînement est impossible à suivre de tête ; la partie 2 change en plus le sens de la plupart des ordres.
Ce qu'on réinvestit. La **programmation orientée objet** vue dans ce chapitre. Le navire a un *état* (sa position, son orientation) que chaque ordre modifie : une classe regroupe cet état dans des attributs, et les actions possibles dans des méthodes (tourner, exécuter un ordre, calculer la distance). Le programme principal se réduit alors à « créer un navire, lui donner les ordres un par un ».
La démarche de la fiche.

1. Vérifier sa compréhension des ordres, puis suivre le navire à la main sur un quadrillage.
2. Écrire la classe `Navire`, puis une méthode pour pivoter par quarts de tour.
3. Écrire la méthode qui exécute un ordre et l'appliquer à tout le fichier : réponse de la partie 1.
4. Pour la partie 2, ajouter un point de repère qui reprend le rôle de l'orientation, puis

réorganiser le code par héritage.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`ship`	navire	`facing`	orienté vers
`north / south`	nord / sud	`east / west`	est / ouest
`turn left / right`	tourner à gauche / droite	`degrees`	degrés
`forward`	en avant	`Manhattan distance`	distance de Manhattan
`waypoint`	point de repère (partie 2)	`relative to`	par rapport à

Résumé de l'énoncé (partie 1)

Contexte. Le ferry est pris dans une tempête et son ordinateur de navigation a produit une liste d'ordres très tortueuse. Vous aidez le capitaine à savoir où ces ordres mènent le bateau, pour estimer l'écart avec la route prévue.
Les données. Chaque ligne du fichier est un ordre : **une lettre** (l'action) immédiatement suivie d'**un entier** (la quantité), par exemple `N4` ou `R90`. Les ordres s'exécutent dans l'ordre des lignes.
La règle.

- Au départ, le navire est à l'origine et pointe vers l'est.
- `N`, `S`, `E`, `W` déplacent le navire de la quantité indiquée vers le nord, le sud, l'est ou l'ouest, **sans** changer la direction vers laquelle il pointe : il se décale comme un crabe.
- `L` et `R` le font pivoter sur place vers la gauche ou la droite du nombre de degrés indiqué (toujours un multiple de 90 dans les données). Il ne se déplace pas.
- `F` le fait avancer de la quantité indiquée, droit devant lui, dans la direction vers laquelle il pointe.

La réponse attendue. La distance de Manhattan entre l'arrivée et le départ, c'est-à-dire $|x| + |y|$ (comme dans une ville quadrillée, on ne coupe pas en diagonale). Par exemple, avec les ordres `F2`, `R90`, `F3`, le navire avance de 2 vers l'est, pivote vers le sud, avance de 3 : il finit en $(2, -3)$ et la réponse serait $2 + 3 = 5$.

Exercice 1 *Comprendre ce qui est demandé*

But. Vérifier sur trois questions courtes qu'on distingue bien se décaler, pivoter et avancer.
À quoi ça sert ? La confusion entre `N` (se décaler vers le nord) et « avancer en pointant vers le nord » est l'erreur la plus fréquente ; le sens des rotations en est une autre.

Questions de vérification, à traiter sur le cahier :

1. Le navire tourné vers l'est exécute `N4` : vers où est-il tourné ensuite ?
2. En partant de l'est, quelle orientation obtient-on après `R270` ? Après `L180` ?
3. Un navire termine à 3 unités à l'ouest et 5 au sud : quelle réponse ? Pourquoi des valeurs absolues ?

Exercice 2 *À la main, sur un petit exemple*

But. Suivre le navire ordre par ordre sur un quadrillage.
À quoi ça sert ? On obtient un exemple dont on **connaît la réponse** (6) et une position intermédiaire : si le programme se trompe, on pourra afficher la position après chaque ordre et repérer le premier écart.

On considère les instructions (inventées) suivantes, une par ligne dans le fichier :

```
N4  F6  L90  F3  R180  W2  F5
```

Suivre le navire sur un quadrillage (est vers la droite, nord vers le haut). Vérification : après `F3`, le navire est à 6 unités à l'est et 7 au nord ; la réponse finale est `6`.

2 Programmer la partie 1

Exercice 3 *La classe* `Navire`

But. Écrire la classe qui représente le navire : sa position, son orientation et le calcul de la distance.
À quoi ça sert ? Toute la suite manipule cet objet. Représenter l'orientation par un vecteur `(dx, dy)` plutôt que par une lettre rend l'ordre `F` immédiat : on ajoute k fois ce vecteur à la position.

Écrire une classe `Navire` dont le constructeur initialise la position `x`, `y` (est et nord) et l'orientation sous forme d'un vecteur `dx`, `dy` (vers l'est : `(1, 0)`). Ajouter une méthode `distance(self)` qui renvoie la distance de Manhattan à l'origine.

Exercice 4 *Tourner par quarts de tour*

But. Faire pivoter le vecteur d'orientation par quarts de tour, dans les deux sens.
À quoi ça sert ? C'est la partie délicate des ordres `L` et `R`. Une seule formule (le quart de tour à gauche) suffit pour tous les angles ; elle resservira telle quelle pour faire tourner le waypoint en partie 2.

Écrire une méthode `tourner_gauche(self)` qui fait tourner l'orientation d'un quart de tour vers la gauche. Vérifier sur papier : est → nord → ouest → sud → est. En déduire `tourner(self, sens, angle)`, où `sens` vaut `"L"` (gauche) ou `"R"` (droite) et `angle` vaut 90, 180 ou 270.

▷ coup de pouce ▷ début de solution

Exercice 5 *Exécuter le fichier*

But. Traduire une ligne du fichier en action sur le navire, puis exécuter tout le fichier.
À quoi ça sert ? Cette méthode réunit les briques précédentes et donne la réponse de la partie 1. Le programme principal se réduit à une boucle sur les lignes.

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
1 ligne = "1721\n"
2 print(ligne.strip())                  # 1721
3 print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Écrire une méthode `executer(self, instruction)` qui reçoit une chaîne comme `"F6"` et met à jour le navire. Lire le fichier, exécuter toutes les instructions et afficher la distance. Tester sur l'exemple inventé de la fiche, puis sur vos données.

▷ coup de pouce

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. En relisant le manuel, on découvre que la plupart des ordres ne concernent pas le navire lui-même, mais un point de repère (*waypoint*) qu'il poursuit.
La nouvelle règle.

- Le waypoint est repéré **par rapport au navire** ; au départ, il est à 10 unités à l'est et 1 au nord du navire.
- `N`, `S`, `E`, `W` déplacent le waypoint (pas le navire).
- `L`, `R` font tourner le waypoint autour du navire, du nombre de degrés indiqué.
- `F` k déplace le navire k fois du vecteur navire $\to$ waypoint ; le waypoint suit le navire et garde sa position relative.

La réponse attendue. La même qu'en partie 1 : la distance de Manhattan entre l'arrivée du navire et son départ. Avec les ordres `F2`, `R90`, `F3`, le navire va en $(20, 2)$, le waypoint pivote en $(1, -10)$, puis le navire finit en $(23, -28)$: la réponse serait 51.

Exercice 6 *Le waypoint*

But. Adapter le navire pour qu'il suive un waypoint, et exécuter les ordres avec leur nouveau sens.
À quoi ça sert ? Le waypoint joue exactement le rôle du vecteur d'orientation de la partie 1 : en le voyant, on réutilise la rotation déjà écrite et l'on obtient la réponse de la partie 2 avec peu de code nouveau.

Écrire une classe `NavireRepere` (ou ajouter les attributs `wx`, `wy`) et une méthode `executer2`. Sur l'exemple inventé de la fiche, on doit trouver `70`. ▷ coup de pouce ▷ début de solution

Outil Python : l'héritage (`class Fille(Mere)` et `super()`)

Une classe peut **hériter** d'une autre : elle reçoit automatiquement tous ses attributs et méthodes, et l'on n'écrit que ce qui change. Une méthode réécrite dans la classe fille *remplace* celle de la classe mère ; `super()` permet d'appeler la version de la classe mère.

```python
class Animal:
    def __init__(self, nom):
        self.nom = nom

    def presenter(self):
        return f"Je suis {self.nom}"

    def cri(self):
        return "..."


class Chat(Animal):                 # Chat herite de Animal
    def __init__(self, nom):
        super().__init__(nom)       # appelle le constructeur de Animal
        self.vies = 9

    def cri(self):                  # redefinition de la methode
        return "Miaou"


c = Chat("Tom")
print(c.presenter())    # Je suis Tom : methode heritee, non reecrite
print(c.cri())          # Miaou : methode redefinie
```

```
print(c.vies)          # 9
```

Intérêt. Ne pas recopier le code commun à deux classes proches : une correction faite dans la classe mère profite aux deux.
À essayer. Supprimer la ligne `super().__init__(nom)`, puis exécuter `c.presenter()`. Que se passe-t-il, et pourquoi ?

★★★ Exercice 7 — *Défi : héritage*

But. Écrire la classe de la partie 2 comme une classe fille de `Navire`.
À quoi ça sert ? L'héritage évite de recopier ce qui est commun aux deux navires (position, distance, rotation) : on n'écrit que ce qui change, et une correction faite une fois profite aux deux classes.

Écrire `NavireRepere` comme une classe fille de `Navire` qui ne redéfinit que ce qui change. Quelles méthodes peut-on hériter sans les réécrire ? ▷ coup de pouce

4 Approfondissement

Outil Python : les nombres complexes (`complex`, `1j`)

Python sait calculer avec les nombres complexes : `3 + 2j` a pour partie réelle 3 et pour partie imaginaire 2 (Python écrit `j` ce que l'on note i en mathématiques, avec $i^2 = -1$). On peut voir un complexe comme un point ou un vecteur du plan : partie réelle = abscisse (est), partie imaginaire = ordonnée (nord).

```
z = 3 + 2j              # partie reelle 3, partie imaginaire 2
print(z.real, z.imag)   # 3.0 2.0
print(z + (1 - 1j))     # (4+1j) : on additionne coordonnee par coordonnee
print(1j * 1j)          # (-1+0j) : j * j = -1
print(z * 1j)           # (-2+3j)
print(abs(z))           # 3.605... : longueur du vecteur (pas Manhattan !)
```

Intérêt. Multiplier par `1j` fait tourner un vecteur d'un quart de tour vers la gauche : $(x + yi) \times i = -y + xi$, c'est exactement la formule $(d_x, d_y) \mapsto (-d_y, d_x)$. Position et direction deviennent chacune un seul nombre.

★★ Exercice 8 — *Approfondissement : naviguer avec des nombres complexes*

But. Refaire les deux parties en représentant position et direction par des nombres complexes.
À quoi ça sert ? Une représentation bien choisie simplifie le code : une rotation devient une multiplication. Comparer avec la version objet montre qu'il n'y a pas une seule bonne modélisation.

1. Recopier et exécuter le code de l'encadré. Calculer à la main $1 \times i$, puis $i \times i$, $(-1) \times i$ et $(-i) \times i$: quelles orientations successives retrouve-t-on ? Par quel nombre faut-il multiplier pour un quart de tour vers la droite ?

2. Sur l'exemple inventé de la fiche, la position et la direction sont des complexes ; au départ `position = 0` et `direction = 1`. Que valent-elles après `N4`, `F6`, `L90` puis `F3` ?

3. Écrire une fonction `trajet_complexe(instructions, partie)` qui traite les deux parties ; vérifier qu'on obtient `6` et `70` sur l'exemple. ▷ coup de pouce ▷ début de solution

4. Pourquoi `abs(position)` ne donne-t-il pas la réponse ? Comparer cette version avec la classe `Navire` : longueur, lisibilité, facilité à corriger.

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 4 – piste Écrire les quatre vecteurs `(1, 0)`, `(0, 1)`, `(-1, 0)`, `(0, -1)` dans l'ordre et chercher comment on passe de l'un au suivant : les coordonnées s'échangent, et l'une change de signe. ↑énoncé

Ex. 4 – début de solution Un quart de tour à gauche transforme `(dx, dy)` en `(-dy, dx)`. Tourner à droite de 90° revient à tourner à gauche de 270°, et `angle // 90` donne le nombre de quarts de tour. ↑énoncé

Ex. 5 – piste Lire les lignes avec `readlines()` et les nettoyer avec `strip` ; `instruction[0]` donne la lettre, `int(instruction[1:])` la valeur. ↑énoncé

Ex. 6 – piste Le waypoint est repéré **par rapport au navire** : il joue exactement le rôle du vecteur d'orientation de la partie 1, mais avec une longueur quelconque. La méthode de rotation se réutilise telle quelle. ↑énoncé

Ex. 6 – début de solution Sur l'exemple, après `N4` et `F6` le navire est à 60 est, 30 nord ; après `L90` le waypoint est à `(-5, 10)` par rapport au navire. ↑énoncé

Ex. 7 – piste `class NavireRepere(Navire):` puis `super().__init__()` dans le constructeur. ↑énoncé

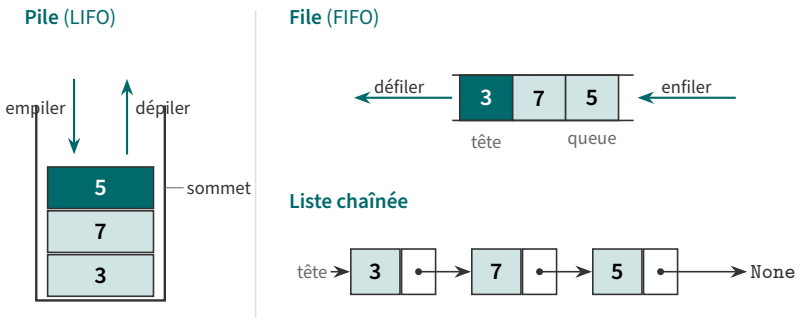
Ex. 8 q. 3 – piste Un dictionnaire associe à chaque lettre `N`, `S`, `E`, `W` son vecteur unitaire : `1j`, `-1j`, `1`, `-1`. ↑énoncé

Ex. 8 q. 3 – début de solution `L` k : multiplier la direction par `1j ** (k // 90)` ; `R` : par `(-1j) ** (k // 90)`. En partie 2, la variable `direction` joue le rôle du waypoint (au départ `10 + 1j`) : `N`, `S`, `E`, `W` la modifient au lieu de modifier la position. ↑énoncé

3

CHAPITRE 3

Structures linéaires : piles, files, listes chaînées



DANS CE CHAPITRE

- Activités préparatoires 127
- Cours 129
- Exercices 142
- TP et projets 155
- Fiche de révision 162
- Défis Advent of Code 164
- Corrigés 790

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Par où entre-t-on, par où sort-on ?

▶ corrigé p. 790

un tas, une rangée, et le bouton « Annuler »

Durée : 20 min · Sans ordinateur, par deux

Consignes

- Matériel : huit cartes numérotées de `1` à `8` (ou huit post-it).
- On **joue** chaque question avec les cartes, **puis** on note le résultat sur le cahier. Un joueur lit les instructions, l'autre manipule ; on échange les rôles à chaque partie.
- « Poser x » : ajouter la carte x ; « prendre » : retirer une carte en respectant la règle, et noter son numéro.

Exercice 1 — *Règle du tas*

Les cartes forment un **tas** posé sur la table, comme des assiettes : on ne peut poser une carte que **sur le dessus**, et on ne peut prendre que **celle du dessus**.

1. Poser `1`, `2`, `3`, `4` dans cet ordre, puis tout reprendre une par une. Dans quel ordre les cartes sortent-elles ?
2. Repartir d'un tas vide et jouer : *poser 5, poser 2, prendre, poser 7, poser 4, prendre, prendre.* Noter sur le cahier les cartes prises, dans l'ordre, puis dessiner le tas final (vertical, le dessus en haut).
3. Avec un tas de huit cartes, combien de cartes faut-il retirer avant de pouvoir prendre celle du **fond** ?

Exercice 2 — *Règle de la rangée*

Les cartes sont maintenant posées en **rangée** : on pose toujours une nouvelle carte **à gauche**, et l'on prend toujours la carte **la plus à droite**.

4. Poser `1`, `2`, `3`, `4`, puis tout reprendre. Dans quel ordre les cartes sortent-elles ?
5. Rejouer la séquence de la question 2 : *poser 5, poser 2, prendre, poser 7, poser 4, prendre, prendre.* Noter les cartes prises, dans l'ordre, puis dessiner la rangée finale (horizontale : on pose à gauche, on prend à droite).
6. Laquelle des deux règles **renverse** l'ordre des cartes ? Laquelle le **conserve** ? Citer une situation de la vie courante pour chacune.

Exercice 3 — *Le bouton « Annuler »*

Dans un éditeur de texte, on tape successivement les mots `Le`, `chat`, `dort`, `bien` : chaque mot tapé est une carte. Le raccourci `Ctrl+Z` **annule** le dernier mot tapé ; `Ctrl+Y` **refait** le dernier mot annulé.

7. On appuie deux fois sur `Ctrl+Z`. Quel texte reste-t-il ? Quels mots ont été annulés, et dans quel ordre ?
8. On appuie ensuite une fois sur `Ctrl+Y`. Quel mot revient ? Pour ranger les mots annulés, faut-il un tas ou une rangée ? Combien de paquets de cartes l'éditeur doit-il tenir en tout ?

9. Repartir du texte `Le chat dort bien`, appuyer deux fois sur `Ctrl+Z`, puis taper le mot `mal`. Serait-il logique de pouvoir encore « refaire » `bien` ? Que doit-il arriver au paquet des mots annulés ?

Exercice 4  *Ce qu'on a découvert*

10. Recopier et compléter sur le cahier, avec vos mots.

- Avec la règle du tas, la carte qui sort est toujours ...
- Avec la règle de la rangée, la carte qui sort est toujours ...
- Les deux paquets contiennent les mêmes cartes ; ce qui les distingue, c'est ...

COURS

Programme (BO) *« Structures de données, interface et implémentation. Spécifier une structure de données par son interface. Distinguer interface et implémentation. Écrire plusieurs implémentations d'une même structure de données. » « Listes, piles, files : structures linéaires. Dictionnaires, index et clé. Distinguer des structures par le jeu des méthodes qui les caractérisent. Choisir une structure de données adaptée à la situation à modéliser. Distinguer la recherche d'une valeur dans une liste et dans un dictionnaire. »*

Prérequis fonctions, boucles, listes et dictionnaires Python ; notion de classe et de méthode (chapitre *Programmation objet*). → fiches de révision : ch. 2 p. 113 → rappels de 1re : bases de Python p. 1056, listes p. 1057, dictionnaires p. 1058

Objectifs *distinguer* l'interface d'une structure de son implémentation ; *connaître* le jeu de méthodes qui caractérise une pile et une file ; *utiliser* ces structures pour résoudre un problème (parenthésage, parcours…) ; *comprendre* le rôle des listes chaînées comme brique d'implémentation.

Situation de départ

1. Comment la commande « Annuler » d'un éditeur sait-elle quelle action défaire en premier ? Et dans quel ordre une imprimante partagée imprime-t-elle les documents qu'elle reçoit ?
2. Un éditeur de code signale que `([)]` est mal écrit, alors qu'il contient autant de symboles ouvrants que de fermants. Comment le vérifie-t-il ?
3. Pourquoi deux programmes qui offrent exactement les mêmes commandes peuvent-ils être l'un rapide, l'autre très lent ?

I Interface et implémentation : le point de vue de l'année

Jusqu'ici, quand vous aviez besoin de ranger plusieurs données, vous preniez une `list` Python sans vous demander *comment* elle est fabriquée à l'intérieur de la machine. C'est exactement l'idée que le programme de Terminale veut rendre explicite.

Définition 1 — Type abstrait de données

Un **type abstrait** de données décrit une structure par les **opérations** qu'on peut lui appliquer (son *quoi*), **sans dire comment** elles sont réalisées (son *comment*). La liste des opérations, avec leur nom et leur effet, s'appelle l'**interface** de la structure.

Définition 2 — Implémentation

Une **implémentation** est une réalisation concrète de la structure dans un langage donné (avec un tableau, une liste chaînée…) qui **respecte l'interface**. Une même interface admet en général **plusieurs** implémentations.

Exemple — *Un distributeur automatique de boissons*
L'**interface**, ce sont les boutons : « choisir une canette », « insérer une pièce », « rendre la monnaie ». L'**implémentation**, c'est toute la mécanique cachée derrière la façade. Deux distributeurs de marques différentes offrent *la même interface* (les mêmes boutons) avec *deux mécaniques* totalement différentes. L'utilisateur, lui, n'a pas besoin de savoir laquelle.

Règle 1 — Ce qu'on vous demandera cette année

Le plus souvent, vous **utiliserez** une structure à travers son interface, en **admettant** simplement qu'une implémentation existe. Savoir *quelle* méthode appeler, et dans quel ordre, compte davantage que savoir la reprogrammer. On écrira tout de même *une* implémentation possible, pour lever le mystère.

Remarque
C'est le prolongement direct du chapitre *Programmation objet* : une structure de données s'écrit naturellement comme une **classe**, dont les **méthodes** forment l'interface et dont les **attributs** (cachés par *encapsulation*) constituent l'implémentation.

Les trois structures de ce chapitre sont dites **linéaires** : leurs éléments sont rangés les uns à la suite des autres, comme sur une file d'attente. Ce qui les distingue, ce n'est pas leur contenu, c'est **leur jeu de méthodes** — en particulier *par où* on entre et *par où* on sort.

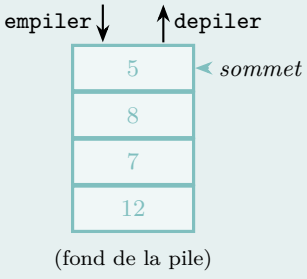
II La pile (LIFO)

Définition 3 — Pile

Une **pile** (*stack*) est une structure linéaire dans laquelle on ne peut ajouter ou retirer un élément que d'un seul côté, appelé le **sommet**. Le dernier élément entré est donc le premier à sortir : on parle de politique **LIFO** (*Last In, First Out*).

↻ **Retour sur l'activité.** La « règle du tas » de l'activité était celle d'une pile : la carte du dessus est le sommet, les cartes ressortent dans l'ordre inverse de leur arrivée (`4`, `3`, `2`, `1`), et il fallait retirer sept cartes pour atteindre le fond d'un tas de huit.

L'image classique est une **pile d'assiettes** : on ne peut poser ou prendre qu'*en haut* de la pile ; l'assiette du fond est la dernière accessible.



A. L'interface d'une pile

Méthode	Effet
`est_vide()`	renvoie `True` si la pile ne contient aucun élément
`empiler(x)`	pose `x` au sommet de la pile
`depiler()`	retire le sommet et le renvoie
`sommet()`	renvoie le sommet *sans* le retirer

Règle 2

On n'accède **jamais** directement à un élément « du milieu » d'une pile : la seule porte d'entrée et de sortie est le sommet. Vouloir atteindre le fond oblige à tout dépiler.

Exemple — *Une suite d'opérations*

On part d'une pile vide `p`. La colonne de droite montre la pile après chaque instruction. Pour gagner de la place, la pile y est **écrite en ligne**, le **sommet à droite** (dessinée verticalement, ce serait le sommet en haut).

▷ **Je prédis.** Cacher les deux colonnes de droite, prévoir ce que renvoie chaque instruction et le contenu de la pile, puis vérifier.

Instruction	Renvoie	Pile écrite en ligne (sommet à droite)
`p.empiler(5)`	—	5
`p.empiler(8)`	—	5 ; 8
`p.empiler(3)`	—	5 ; 8 ; 3
`p.sommet()`	`3`	5 ; 8 ; 3
`p.depiler()`	`3`	5 ; 8
`p.depiler()`	`8`	5
`p.est_vide()`	`False`	5

► **Exercices d'application :** exercices 1 **et** 2 (opérations et trace d'une pile)

B. À quoi sert une pile en informatique ?

La pile n'est pas un jouet : le principe LIFO se retrouve partout dès qu'il faut **se souvenir de la dernière chose à traiter**.

- **La pile d'appels.** Quand une fonction en appelle une autre, la machine *empile* l'endroit où revenir ; au `return`, elle *dépile*. C'est exactement ce qui fait fonctionner la **récursivité** (voir le chapitre correspondant) : chaque appel récursif est empilé, puis les résultats se dépilent en sens inverse.
- **La fonction « Annuler » (`Ctrl+Z`).** Chaque action est empilée ; annuler, c'est dépiler la dernière.
- **Le bouton « Précédent » d'un navigateur.** Les pages visitées sont empilées ; « précédent » dépile la dernière.
- **L'évaluation d'expressions** par un compilateur ou une calculatrice, et la vérification du parenthésage, que nous détaillons maintenant.

↺ **Retour sur l'activité.** Le bouton « Annuler » de l'activité demandait deux piles : celle des mots tapés, que `Ctrl+Z` dépile, et celle des mots annulés, que `Ctrl+Y` dépile à son tour ; taper un nouveau mot vide la seconde.

C. Exemple phare : vérifier un parenthésage

Problème. Une expression comme `([]{})` est bien parenthésée : chaque symbole ouvrant est refermé par le bon symbole fermant, dans le bon ordre. Au contraire `([)]` et `(()` sont incorrectes. Comment le vérifier automatiquement ?

Idée. À chaque symbole **ouvrant**, on l'**empile** : c'est une promesse « il faudra me refermer ». À chaque symbole **fermant**, on **dépile** : le sommet doit être l'ouvrant *correspondant*. Une pile convient parfaitement, car c'est **toujours la dernière ouverture** qui doit être refermée en premier — du LIFO pur.

L'algorithme ci-dessous n'utilise **que l'interface** de la pile : on **admet** qu'une classe `Pile` est disponible, sans savoir comment elle est faite.

```
def parenthesage_correct(expr):
    p = Pile()                          # INITIALISER : on admet que Pile existe
    correspond = {')': '(', ']': '[', '}': '{'}
    for c in expr:                      # PARCOURIR
        if c in "([{":                  # TESTER : une ouvrante est mise en attente
            p.empiler(c)                # METTRE A JOUR
        elif c in ")]}":                # TESTER : une fermante solde une ouvrante
            if p.est_vide():            # TESTER : rien a refermer
                return False            # RENVOYER : incorrect
            if p.depiler() != correspond[c]:   # TESTER : mauvais appariement
                return False            # RENVOYER : incorrect
    return p.est_vide()                 # RENVOYER : correct si tout est referme
```

▷ **Je prédis.** Que renvoient `parenthesage_correct("(()")` et `parenthesage_correct("())")` ?
▷ **Je vérifie.** `False` dans les deux cas, mais pas au même endroit : pour `"(()"`, la boucle se termine avec une `(` encore dans la pile, c'est le dernier `return` qui renvoie `False` ; pour `"())"`, la seconde `)` trouve la pile vide.

Exemple — *Dérouler l'algorithme sur `([]{})`, puis sur `([)]`*

▶ **Correction.** On suit la pile symbole après symbole.

Symbole	Action	Pile écrite en ligne (sommet à droite)
`(`	empiler	`(`
`[`	empiler	`( [`
`]`	dépiler `[`, correspond	`(`
`{`	empiler	`( {`
`}`	dépiler `{`, correspond	`(`
`)`	dépiler `(`, correspond	(vide)
fin : la pile est vide		**correct**

Sur `([)]`, au symbole `)` on dépile `[` : cela ne correspond pas à `(`, donc **incorrect**. C'est précisément le cas qu'un simple **comptage** ne détecterait pas : `([)]` contient autant de `(` que de `)`, et autant de `[` que de `]` ; seule la pile vérifie l'**ordre** des fermetures.

Pour les curieux (facultatif)
Ce même principe (empiler les ouvertures, dépiler aux fermetures) est utilisé par votre éditeur de code pour surligner les parenthèses appariées, et par tout **compilateur** pour vérifier la syntaxe des accolades et parenthèses.

▶ **Exercices d'application :** exercices 3 à 10 (parenthésage ; renverser ; taille et copie d'une pile ; balises ; notation polonaise inverse)

III La file (FIFO)

Définition 4 — File

Une **file** (*queue*) est une structure linéaire dans laquelle on **ajoute d'un côté** (la queue) et on **retire de l'autre** (la tête). Le premier élément entré est donc le premier à sortir : politique **FIFO** (*First In, First Out*).

↻ **Retour sur l'activité.** La « règle de la rangée » (poser à gauche, prendre à droite) était celle d'une file, dessinée comme ci-dessous, tête à droite : les cartes ressortent dans leur ordre d'arrivée (`1`, `2`, `3`, `4`).

L'image est la **file d'attente** devant un guichet : on rejoint la file par la **queue** (ici à gauche) et l'on est servi à la **tête** (ici à droite), « premier arrivé, premier servi ».



A. L'interface d'une file

Méthode	Effet
`est_vide()`	renvoie `True` si la file ne contient aucun élément
`enfiler(x)`	ajoute `x` à la queue (en fin de file)
`defiler()`	retire l'élément de tête et le renvoie
`tete()`	renvoie l'élément de tête *sans* le retirer

À retenir — pile ou file : la seule différence qui compte

Piles et files offrent presque les mêmes opérations (ajouter, retirer, tester si vide). Ce qui les **distingue**, c'est *où* le retrait a lieu : au **même** bout que l'ajout pour la pile (LIFO), à l'**autre** bout pour la file (FIFO). C'est un exemple parfait de ce que dit le BO : « distinguer des structures par le jeu des méthodes qui les caractérisent ».

Exemple — *Une suite d'opérations*

On part d'une file vide `f`.

▷ **Je prédis.** Cacher les deux colonnes de droite, prévoir ce que renvoie chaque instruction et le contenu de la file, puis vérifier.

Instruction	Renvoie	File (tête à droite)
`f.enfiler(22)`	—	22
`f.enfiler(19)`	—	19 ; 22
`f.enfiler(7)`	—	7 ; 19 ; 22
`f.tete()`	22	7 ; 19 ; 22
`f.defiler()`	22	7 ; 19
`f.defiler()`	19	7

Comparez avec la pile de la partie précédente : sur les *mêmes* ajouts, la pile aurait ressorti 7 d'abord, la file ressort 22 d'abord.

B. À quoi sert une file en informatique ?

Le principe FIFO s'impose dès qu'il faut **traiter les demandes dans l'ordre d'arrivée**, sans favoritisme.

- **Les files d'impression.** Les documents envoyés à une imprimante partagée sont imprimés dans l'ordre où ils sont arrivés.
- **L'ordonnancement des processus** par le système d'exploitation : les tâches en attente du processeur sont gérées dans des files (vous le verrez au chapitre *Processus*).
- **Les tampons (*buffers*).** Les octets reçus du réseau, les caractères tapés au clavier, les images d'une vidéo en *streaming* sont mis en file en attendant d'être consommés dans l'ordre.
- **Le parcours en largeur** d'un arbre ou d'un graphe (*BFS*) utilise une file pour visiter les sommets niveau par niveau (vous l'étudierez aux chapitres *Arbres* et *Graphes*). Symétriquement, le parcours en profondeur utilise, lui, une *pile*.

▶ **Exercices d'application :** exercices 11 à 14 (opérations et trace d'une file ; ordonnancement ; file de priorité) et 17 à 19 (piles et files ensemble)

IV Une implémentation possible (à admettre le plus souvent)

Jusqu'ici nous avons *utilisé* piles et files sans les construire. Levons le voile : grâce au chapitre *Programmation objet*, une pile s'écrit en quelques lignes comme une **classe** qui cache une `list` Python.

```
class Pile:
    def __init__(self):
        self._contenu = []              # attribut cache (encapsulation)

    def est_vide(self):
        return self._contenu == []

    def empiler(self, x):
        self._contenu.append(x)         # ajout en fin de liste = sommet

    def depiler(self):
        if not self.est_vide():         # pile vide : on ne fait rien
            return self._contenu.pop()  # retire et renvoie le dernier

    def sommet(self):
        if not self.est_vide():
            return self._contenu[-1]
```

▷ **Je prédis.** Après `p = Pile()`, `p.empiler(4)`, `p.empiler(9)`, `p.depiler()` et `p.empiler(1)`, que vaut `p._contenu` ? Que renvoient ensuite trois appels `p.depiler()` ?
▷ **Je vérifie.** `[4, 1]` : le sommet est la *fin* de la liste. Les trois appels renvoient `1`, `4`, puis `None` : la pile est vide, la méthode ne fait rien.

De la même façon, une **file** se construit en cachant une `list` : on ajoute à la queue avec `append`, et on retire la tête, c'est-à-dire le *premier* élément, avec `pop(0)`.

```
class File:
    def __init__(self):
        self._contenu = []              # la tete est au debut de la liste

    def est_vide(self):
        return self._contenu == []

    def enfiler(self, x):
        self._contenu.append(x)         # ajout en fin de liste = queue
```

```
10
11      def defiler(self):
12          if not self.est_vide():          # file vide : on ne fait rien
13              return self._contenu.pop(0)  # retire et renvoie le premier = tete
14
15      def tete(self):
16          if not self.est_vide():
17              return self._contenu[0]
```


▷ **Je prédis.** Après `f = File()`, `f.enfiler(4)`, `f.enfiler(9)`, `f.defiler()` et `f.enfiler(1)`, que vaut `f._contenu` ? Que renvoie `f.tete()` ?
▷ **Je vérifie.** `[9, 1]` et `9` : la tête est le *début* de la liste Python. Dessinée tête à droite comme dans ce cours, la file s'écrit 1 ; 9. Avec la pile ci-dessus, les mêmes opérations laissaient `1` au sommet.

Remarque — *Une interface volontairement minimale*
Le test `if not self.est_vide():` évite l'erreur de Python (`IndexError`) quand on dépile une pile vide ou qu'on défile une file vide : la méthode ne fait alors rien et renvoie `None`. Ce garde-fou ne dispense pas de tester `est_vide()` *avant* de retirer un élément : un `None` inattendu cache souvent une erreur de raisonnement.
Il n'y a pas non plus de méthode `taille` : on n'en a pas besoin pour utiliser une pile ou une file, et l'on sait la **reconstruire** avec les seules méthodes de l'interface, tout comme une copie de la structure (feuille, exercices **7**, **8** et **16**).

Remarque — *Attention au coût de* `pop(0)`
Cette file « fonctionne », mais elle est **sournoisement lente**. Retirer le *dernier* élément d'une `list` avec `pop()` est immédiat ; retirer le *premier* avec `pop(0)`, en revanche, oblige Python à **décaler d'une case vers la gauche tous les éléments qui restent**. Sur une file de n éléments, **chaque `defiler` coûte donc n opérations**, contre une seule pour `depiler` sur une pile. Autrement dit, cette implémentation est parfaite pour une pile mais médiocre pour une file. Une *vraie* file efficace s'appuie sur une autre implémentation — une liste chaînée ou deux piles. Belle illustration du chapitre : une **même interface** peut cacher des implémentations aux **coûts très différents**.

Règle 3 — La notation O : un ordre de grandeur du coût

Pour comparer des coûts sans compter chaque opération, on note :

- **$O(1)$** (on lit « grand O de 1 ») un coût **constant**, qui ne dépend pas de la taille n des données : `depiler` sur la pile ci-dessus ;
- **$O(n)$** un coût **linéaire**, au plus proportionnel à n quand n devient grand : `defiler` avec `pop(0)`, ou le parcours d'une liste de n éléments.

On écrit donc que `depiler` est en $O(1)$ et que ce `defiler` est en $O(n)$. On rencontrera dans les chapitres suivants d'autres ordres de grandeur, comme $O(\log n)$ (arbres) ou $O(n \log n)$ et $O(n^2)$ (tris), que le chapitre *Diviser pour régner* rassemblera.

Règle 4 — L'essentiel à retenir

Ces implémentations ne sont **qu'un choix parmi d'autres**. Ce qui compte, et ce que le bac vérifie, c'est de savoir **utiliser** l'interface (`empiler`, `depiler`, `enfiler`, `defiler`, `est_vide`...). Dans la grande majorité des exercices, on écrit simplement `p = Pile()` ou `f = File()` en **admettant** que la classe existe.

Remarque — *Un « faux ami » : la `list` de Python*
La `list` de Python *n'est pas* le type abstrait « liste » de ce chapitre : c'est un **tableau dynamique**. Elle permet certes de *simuler* une pile (`append`/`pop`) ou une file, mais elle offre bien plus d'opérations (accès à n'importe quel indice). Restreindre son usage aux seules méthodes de l'interface, c'est justement respecter la structure choisie.

► **Exercices d'application :** exercices 15 **et** 20 (une file écrite par un assistant d'IA ; une file avec deux piles), puis 7, 8 et 16 (taille et copie avec la seule interface)

3

V Les listes chaînées : l'autre grande implémentation

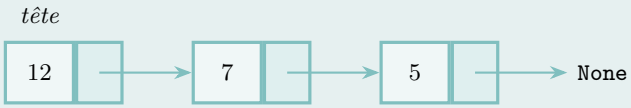
Comment ranger des éléments *à l'intérieur* de la machine ? Il existe deux grandes familles.

Définition 5 — Tableau

Un **tableau** range ses éléments dans des cases mémoire **contiguës** (les unes à côté des autres). On accède directement à la case d'indice i, mais sa **taille est fixée** : insérer un élément au milieu oblige à tout décaler.

Définition 6 — Liste chaînée

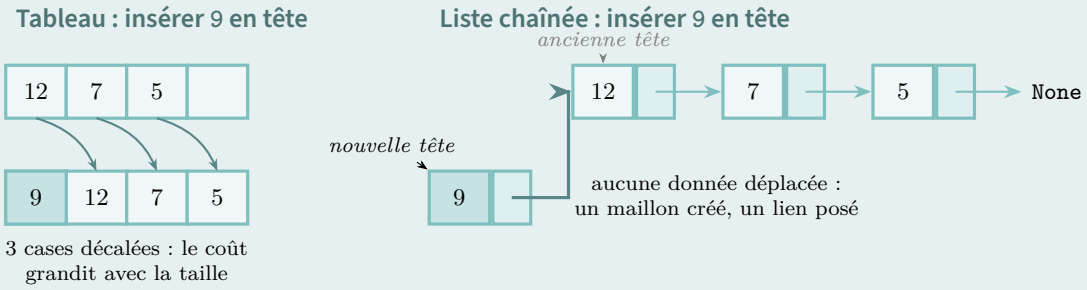
Une **liste chaînée** range ses éléments dans des **maillons** dispersés en mémoire. Chaque maillon contient une **valeur** et un **lien** (l'adresse) vers le maillon **suivant**. Un lien spécial (`None`) marque la fin. On accède à la structure par son **premier** maillon, appelé la **tête**.



A. Pourquoi cette structure ? Le compromis

	Tableau	Liste chaînée
Accès à l'élément d'indice i	direct, rapide	lent : suivre les liens un à un
Taille	fixée à l'avance	grandit à volonté
Insérer / supprimer en tête	coûteux (tout décaler)	rapide (rebrancher un lien)

Insérer un maillon ne demande que de **rebrancher deux liens**, sans déplacer aucune donnée — ce qui rend la liste chaînée idéale pour implémenter une pile ou une file où l'on ajoute et retire sans cesse à une extrémité.



Remarque — *À nouveau le faux ami* `list`
Le « tableau » de ce comparatif n'est pas non plus la `list` de Python. Celle-ci est un **tableau dynamique** (un tableau qui sait s'agrandir), et surtout elle **n'est pas** une liste chaînée, *malgré son nom* : une `list` Python ne contient aucun « lien vers le suivant », ses éléments restent rangés de façon **contiguë** en mémoire. C'est d'ailleurs pour cela que `pop(0)` y est coûteux (il faut tout décaler), alors que retirer la tête d'une *vraie* liste chaînée est immédiat.

B. À quoi servent les listes chaînées en informatique ?

- **Implémenter les piles, les files et les listes** : c'est leur premier usage. Empiler, c'est ajouter un maillon en tête ; dépiler, c'est avancer la tête d'un cran.
- **En langage C**, il n'existe pas de « liste » toute prête comme en Python. Le programmeur construit lui-même une liste chaînée : une `struct` qui contient une valeur et un **pointeur** vers la `struct` suivante, chaque maillon étant réservé en mémoire par `malloc`. C'est un exercice fondateur de tout cours de C.
- **La gestion de la mémoire** par le système : les blocs de mémoire libres sont souvent chaînés entre eux dans une *liste de blocs libres*.

À retenir — une liste chaînée en Python : la classe `Cellule`

En Python, on peut représenter un maillon par un petit objet (chapitre *Programmation objet*) : une classe `Cellule` avec deux attributs, `valeur` (l'élément) et `suivante` (la cellule d'après, ou `None` à la fin). On construit alors une liste en emboîtant les cellules : `Cellule(1, Cellule(2, Cellule(3, None)))`. Pour les piles et les files, il suffit le plus souvent d'**admettre** l'existence d'une implémentation ; en revanche, il faut savoir **parcourir** une liste chaînée construite avec `Cellule`, en suivant les liens d'une cellule à la suivante.

Exemple — *Parcourir une liste chaînée*
Avec la classe `Cellule` ci-dessous, écrire une fonction `afficher(lst)` qui affiche, dans l'ordre, les valeurs de la liste chaînée `lst` (rien pour la liste vide `None`).

```
class Cellule:
    def __init__(self, v, s):
        self.valeur = v
        self.suivante = s
```

▶ **Correction.** On part de la première cellule et on suit les liens jusqu'à `None`, en affichant chaque valeur rencontrée :

```
def afficher(lst):
    c = lst                    # INITIALISER : la cellule courante
    while c is not None:       # PARCOURIR : jusqu'a la fin (None)
        print(c.valeur)
        c = c.suivante         # METTRE A JOUR : on avance d'un maillon
```

Pour `lst = Cellule(4, Cellule(7, Cellule(2, None)))`, la boucle fait trois tours et affiche 4, 7 puis 2. Le même schéma (`c = lst`, `while c is not None:`, `c = c.suivante`) sert à tous les parcours : calculer la longueur, chercher une valeur, faire une somme, accéder au n-ième élément…

▶ **Exercices d'application :** exercices 21 à 26 (listes chaînées)

C. Le type abstrait « liste » et son interface

La liste chaînée est une *implémentation*. Comme pour la pile et la file, on peut décrire la **liste** par sa seule **interface**, sans dire comment elle est programmée.

Définition 7 — Interface d'une liste

Une **liste** est une suite finie d'éléments, que l'on manipule par cinq opérations :

- `liste_vide()` renvoie une liste vide ;
- `est_vide(L)` renvoie `True` si la liste `L` est vide ;
- `ajouter_tete(x, L)` renvoie la liste dont le premier élément est `x`, suivi des éléments de `L` ;
- `tete(L)` renvoie le premier élément d'une liste `L` non vide ;
- `queue(L)` renvoie la liste `L` privée de son premier élément (`L` non vide).

Les noms changent selon les sujets (`construire`, `premier`, `reste`, `car` et `cdr` en Lisp…) : suivez l'énoncé. Ce qui caractérise la liste, c'est ce **jeu de méthodes** : on n'accède qu'au premier élément, et `ajouter_tete` **renvoie** une nouvelle liste au lieu de modifier `L`, alors que `empiler` **modifie** la pile. La classe `Cellule` en fournit une implémentation, où chaque opération ne coûte que $O(1)$:

```
def liste_vide():
    return None

def est_vide(L):
    return L is None

def ajouter_tete(x, L):
    return Cellule(x, L)      # une cellule creee, rien n'est recopie

def tete(L):
    return L.valeur

def queue(L):
    return L.suivante
```

Exemple — *Que vaut* `tete(queue(L))` *pour* `L = ajouter_tete(4, ajouter_tete(7, ajouter_tete(2, liste_vide())))` *?*

▶ **Correction.** On lit de l'intérieur vers l'extérieur : on ajoute `2`, puis `7` devant, puis `4` devant ; `L` est la liste `4, 7, 2`. `queue(L)` est la liste `7, 2`, et sa tête vaut `7`.

▶ **Exercices d'application :** exercice 28 (l'interface d'une liste)

VI Choisir une structure : liste ou dictionnaire ?

Depuis la Première, vous connaissez deux façons de ranger des données : la **liste** Python (un tableau), dont les éléments sont repérés par un **indice** entier de `0` à $n-1$, et le **dictionnaire**, qui associe chaque **valeur** à une **clé** (`annuaire["Lina"]`). Elles ne se valent pas quand il faut **chercher**.

Règle 5 — Rechercher dans une liste ou dans un dictionnaire

- `x in liste` compare `x` aux éléments **un par un** : jusqu'à n comparaisons si `x` est absent. Le coût est **linéaire**, en $O(n)$.
- `c in dico` ne parcourt pas les clés : Python calcule directement, à partir de la clé, l'endroit où elle serait rangée (on parle de *hachage* [au-delà du programme]). Le coût reste à peu près **constant**, en $O(1)$, quelle que soit la taille du dictionnaire.
- Attention : ce sont les **clés** qui se cherchent vite. Chercher une **valeur** (`v in dico.values()`) parcourt toutes les valeurs : $O(n)$, comme pour une liste.

Exemple — *Mesurer la différence*

▷ **Je prédis.** Laquelle des deux durées affichées est la plus petite ? Que devient chacune si `n` est multiplié par 10 ?

```
1  import time
2
3  n = 1000000
4  liste = [b for b in range(n)]
5  dico = {b: True for b in range(n)}
6
7  debut = time.perf_counter()
8  trouve = -1 in liste             # -1 est absent : le pire cas
9  print(time.perf_counter() - debut)
10 debut = time.perf_counter()
11 trouve = -1 in dico
12 print(time.perf_counter() - debut)
```

► **Correction.** Sur un ordinateur ordinaire, la recherche dans la liste prend de l'ordre de 10 millisecondes, et dix fois plus si n est multiplié par 10 ; la recherche dans le dictionnaire prend environ une microseconde, et ne change presque pas avec n. Les valeurs exactes dépendent de la machine, pas leur rapport.

Règle 6 — Choisir la structure adaptée à la situation

On part de ce que le programme doit faire **le plus souvent** : garder un **ordre** et accéder par la **position** → une liste ; **retrouver** une donnée par une clé (un nom, un numéro), souvent → un dictionnaire ; servir le **dernier** arrivé → une pile ; servir le **premier** arrivé → une file.

► **Exercices d'application :** exercice 29 (liste ou dictionnaire ?)

VII Ouverture

Piles et files ne sont pas des curiosités isolées : ce sont les **outils de base** de la fin de l'année.

- Au **bac**, on vous donne l'**interface** d'une pile ou d'une file et l'on demande de **tracer** une suite d'opérations, de **choisir** la bonne structure pour un problème, ou d'**écrire un court algorithme** qui n'utilise que ces méthodes — rarement de reprogrammer la structure elle-même.

Pour les curieux — *Un peu d'histoire* (facultatif)

L'idée de **pile** apparaît dès **1946** chez **Alan Turing**, qui décrit, pour gérer le retour des sous-programmes, deux opérations qu'il nomme joliment « *bury* » (enterrer) et « *unbury* » (déterrer). Elle est redécouverte et formalisée en Allemagne vers **1955–1957** par **Klaus Samelson** et **Friedrich L. Bauer**, sous le nom de *Kellerprinzip* (« principe du cellier »), et même brevetée. Les **listes chaînées**, elles, sont inventées vers **1955–1956** par **Allen Newell**, **Cliff Shaw** et **Herbert Simon** pour leur langage *IPL* de traitement de l'information, une idée reprise dès **1958** par **John McCarthy** dans le langage **Lisp** (les fameuses « cellules » `cons`). Trois de ces chercheurs — Bauer, Newell, Simon — ont marqué durablement l'informatique ; Simon a même reçu le prix Nobel d'économie *et* le prix Turing.



Friedrich L. Bauer, co-inventeur de la pile

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise :** les classes (*Programmation orientée objet et paradigmes*) et la pile d'appels de la *Récursivité*, qui est une pile.
- **Ce qui en découle :** les *Arbres* et les *Graphes* se parcourront avec une **file** (en largeur) ou une **pile** (en profondeur).
- **Pour aller plus loin :** le système rangera les processus prêts dans une file (*Processus et ordonnancement*) ; en *Réseaux*, une file modélisera les paquets en attente dans un routeur.

Bilan — la carte mémoire

Notion	À retenir
Type abstrait / interface	la structure définie par son **jeu de méthodes**, pas par sa réalisation
Implémentation	réalisation concrète respectant l'interface (plusieurs possibles)
Pile (LIFO)	ajout et retrait au **même** bout (le sommet) ; `empiler` / `depiler`
File (FIFO)	ajout à la queue, retrait à la tête ; `enfiler` / `defiler`
Usage type pile	pile d'appels, `Ctrl+Z`, « précédent », parenthésage
Usage type file	impression, ordonnancement, tampons, parcours en largeur
Tableau	cases contiguës, accès direct, taille fixe
Liste chaînée	maillons `valeur + lien`, taille libre, insertion rapide
Liste (interface)	`liste_vide`, `est_vide`, `ajouter_tete`, `tete`, `queue`
$O(1)$ / $O(n)$	coût constant / coût proportionnel à la taille n
Liste ou dictionnaire	`x in liste` en $O(n)$; `cle in dico` en $O(1)$
Réflexe d'exercice	**admettre** l'implémentation, **utiliser** l'interface

Retour à la situation de départ

1. « Annuler » range les actions dans une **pile** (LIFO) : la dernière action faite est la première défaite (§ II.B). L'imprimante range les documents dans une **file** (FIFO) : premier arrivé, premier imprimé (§ III.B).
2. Avec une **pile** : chaque ouvrant est empilé, chaque fermant dépile le sommet, qui doit être l'ouvrant correspondant. Un simple comptage ne vérifie pas l'**ordre** des fermetures (§ II.C).

3. Une même **interface** admet plusieurs **implémentations**, aux coûts très différents : une file qui cache une `list` paie n opérations à chaque `pop(0)`, alors qu'une liste chaînée retire sa tête immédiatement (§ I, IV et V).

Erreurs fréquentes

- **Confondre pile (LIFO) et file (FIFO).** La pile ressort le **dernier** entré, la file le **premier**. *Le réflexe :* pile = Ctrl+Z ; file = file d'attente.
- **Dépiler / défiler une structure vide.** Toujours tester `est_vide` avant `depiler` / `defiler`.
- **Croire que la `list` Python est une liste chaînée.** C'est un **tableau dynamique** (accès direct par indice) ; une liste chaînée suit des **liens** de maillon en maillon.
- **Défiler avec `pop(0)`.** Décaler tout le tableau coûte **O(n)** : mauvaise file. *Le réflexe :* liste chaînée ou deux piles.
- **Perdre la suite d'une liste chaînée.** En insérant / supprimant, mettre à jour le lien `suivante` **dans le bon ordre**, sinon on perd des maillons.
- **Ouvrir l'implémentation quand l'énoncé donne l'interface.** On **admet** l'implémentation et on **utilise** seulement les opérations fournies.

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

☐ distinguer **pile (LIFO)** et **file (FIFO)** et en donner des usages concrets → ex. 17, 18 ;
☐ utiliser l'**interface** d'une pile (`empiler / depiler / est_vide / sommet`) et d'une file (`enfiler / defiler / est_vide / tete`) → ex. 1, 6, 11 ;
☐ **dérouler** l'état d'une pile ou d'une file après une suite d'opérations → ex. 2, 12, 18 ;
☐ appliquer une pile à un problème (**parenthésage**, évaluation, *annuler*) → ex. 3, 4, 10 ;
☐ écrire une fonction qui n'utilise **que l'interface** et rend la structure **intacte** (taille, copie) → ex. 7, 8, 16 ;
☐ parcourir une **liste chaînée** (longueur, occurrences, n-ième élément, copie) → ex. 21, 22, 23, 25 ;
☐ raisonner sur le **coût** des opérations (dont `pop(0)` en O(n)) → ex. 13, 14 ;
☐ distinguer **interface** et **implémentation** → ex. 15, 20 ;
☐ utiliser l'**interface d'une liste** (`ajouter_tete / tete / queue / est_vide`) et l'implémenter avec `Cellule` → ex. 28 ;
☐ distinguer la **recherche** d'une valeur dans une **liste** et dans un **dictionnaire**, et choisir la structure adaptée → ex. 17, 29.

Vers le Grand Oral

Quelques pistes de questions que ce chapitre permet de préparer (à étayer d'un exemple concret) :

- **Pile ou file : comment le choix d'une structure change un algorithme ?** *(parcours en profondeur avec une pile, en largeur avec une file.)*
- **Pourquoi séparer l'interface de l'implémentation ?** *(abstraction : changer l'intérieur sans casser le code qui l'utilise.)*
- **La liste de Python est-elle une liste chaînée ?** *(tableau dynamique, coût de `pop(0)`, ce qu'est une vraie file.)*

EXERCICES

Mode d'emploi

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 32.
- **Réflexe** : on **utilise** une structure par son **interface** (`empiler`, `depiler`, `enfiler`, `defiler`, `est_vide`…) et l'on **admet** qu'une classe l'implémente. On écrit `p = Pile()` ou `f = File()` sans la reprogrammer.
- **Convention** de ces exercices : pile dessinée **verticalement** (sommet en haut), file dessinée **horizontalement** avec la **tête à droite** (on enfile à gauche, on défile à droite).

Piles ↑ cours

★ **Exercice 1** *La bonne opération sur une pile* ▶ corrigé p. 791 • essentiel

On dispose d'une pile `p`. Pour chaque phrase, écrire l'**appel de méthode** correspondant.

▷ coup de pouce

1. ajouter la valeur `7` au sommet de la pile `p` ;
2. retirer et récupérer l'élément au sommet de `p` ;
3. savoir si la pile `p` est vide ;
4. consulter le sommet de `p` *sans* le retirer.

★ **Exercice 2** *Trace d'une pile* ▶ corrigé p. 791 • essentiel

On part d'une pile `p` vide. Donner, pour chaque appel qui renvoie une valeur, la **valeur renvoyée**, puis représenter le **contenu final** de la pile (sommet en haut). ▷ coup de pouce

```
p.empiler(4)
p.empiler(7)
p.empiler(2)
p.depiler()
p.empiler(9)
p.sommet()
p.depiler()
```

★★ **Exercice 3** *Parenthésage à la main* ▶ corrigé p. 791 • essentiel

On reprend l'algorithme du cours : on **empile** chaque symbole ouvrant, et à chaque symbole fermant on **dépile** en vérifiant la correspondance. Pour chacune des expressions suivantes, dérouler l'algorithme (contenu de la pile étape par étape) et conclure : **correcte** ou **incorrecte** (et pourquoi).

▷ coup de pouce

a) `{[()]}`
b) `(]`
c) `([]`
d) `())(`

★★ **Exercice 4** *Programmer le parenthésage* ▶ corrigé p. 792

On admet disponible une classe `Pile` d'interface `empiler`, `depiler`, `est_vide`.

▶ Écrire la fonction `parenthesage_correct(expr)` qui renvoie `True` si la chaîne `expr` est correctement parenthésée (avec `()`, `[]` et `{}`), `False` sinon. La tester sur les quatre exemples précédents. ▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 5** *Renverser avec une pile* ▶ corrigé p. 792 • essentiel

▶ En utilisant **uniquement** une pile (interface `empiler`/`depiler`/`est_vide`), écrire une fonction `renverser(tab)` qui renvoie une **nouvelle** liste contenant les éléments de `tab` dans l'ordre inverse. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

```
>>> renverser([1, 2, 3, 4])
[4, 3, 2, 1]
```

★★ **Exercice 6** *La structure de pile (d'après Asie 2024, jour 1)* ▶ corrigé p. 792 ≈ 15 min

On dispose d'une classe `Pile` dont les objets représentent une pile, munie des méthodes :

- `estVide()` : renvoie `True` si la pile est vide ;
- `empiler(e)` : ajoute l'élément `e` au sommet de la pile ;
- `depiler()` : renvoie la valeur du sommet et l'enlève de la pile.

1. On exécute les instructions suivantes :

```
essai = Pile()
essai.empiler(3)
essai.empiler(2)
essai.empiler(10)
elt = essai.depiler()
elt = essai.depiler()
```

Donner la valeur finale de `elt`, puis représenter l'état de la pile `essai`.

2. Expliquer, avec le mot **LIFO**, pourquoi c'est cette valeur qui est obtenue.

3. ▶ Écrire une fonction `vider(p)` qui dépile et affiche un à un **tous** les éléments d'une pile `p` jusqu'à ce qu'elle soit vide. Dans quel ordre les éléments empilés `3, 2, 10` sont-ils affichés ?

▷ coup de pouce

★★ **Exercice 7** *La taille d'une pile* ▶ corrigé p. 793 • essentiel

La classe `Pile` du cours n'a **pas** de méthode `taille` : on ne dispose que de `est_vide`, `empiler`, `depiler` et `sommet`.

1. Un élève propose la fonction suivante.

```
1 def taille(p):
2     n = 0
3     while not p.est_vide():
4         p.depiler()
5         n = n + 1
6     return n
```

On empile `4`, `7` puis `2` dans une pile vide `p`, puis on exécute deux fois de suite `print(taille(p))`. Qu'affiche le programme ? Expliquer, avec l'expression **effet de bord**, pourquoi cette fonction ne convient pas.

2. ▶ Écrire une fonction `taille(p)` qui renvoie le nombre d'éléments de la pile `p`, en n'utilisant **que** les méthodes de l'interface, et qui laisse `p` **dans son état d'origine** (mêmes éléments, même ordre). ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

3. Avec la pile `p` de la question 1, représenter `p` et la pile auxiliaire au moment où l'on a fini de compter, puis à la fin de l'appel.

★★ **Exercice 8** *Copier une pile* ▶ corrigé p. 793

On veut écrire une fonction `copier_pile(p)` qui renvoie une **nouvelle** pile contenant les mêmes éléments que `p`, **dans le même ordre** (même sommet), en n'utilisant que `est_vide`, `empiler`, `depiler` et `sommet`. À la fin, `p` doit avoir retrouvé son état d'origine.

1. Pourquoi l'instruction `q = p` ne fabrique-t-elle pas une copie ? Que devient `p` si l'on exécute ensuite `q.depiler()` ?

2. Un élève dépile `p` jusqu'à ce qu'elle soit vide en empilant chaque élément dans une nouvelle pile `q`. Pour la pile `p` contenant `4`, `7`, `2` (`2` au sommet), représenter `p` et `q` à la fin. Relever les **deux** défauts de cette méthode.

3. ▶ Écrire la fonction `copier_pile(p)`. La tester en vérifiant que `p` et sa copie se dépilent dans le même ordre, et que dépiler la copie ne modifie pas `p`. ▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 9** *Défi — des balises bien imbriquées* ▶ corrigé p. 794 ≈ 20 min

Dans un document, les balises s'écrivent `<b>` (ouvrante) et `</b>` (fermante), comme en HTML. Une liste de balises est **bien imbriquée** si chaque fermante correspond à la dernière ouvrante non encore fermée.

▶ On donne une liste de balises, par exemple `["<b>", "<i>", "</i>", "</b>"]` (bien imbriquée) ou `["<b>", "<i>", "</b>", "</i>"]` (mal imbriquée). Écrire `balises_ok(liste)` qui renvoie un booléen, en s'inspirant du parenthésage (une balise fermante `</x>` doit correspondre à l'ouvrante `<x>` au sommet de la pile). ▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 10** *Défi — notation polonaise inverse* ▶ corrigé p. 794 ≈ 20 min

En **notation polonaise inverse** (NPI), on écrit les opérateurs *après* leurs opérandes : `"3 4 +"` vaut 7, et `"5 1 2 + 4 * +"` vaut 17. On évalue avec une pile : chaque **nombre** est empilé ; à chaque **opérateur**, on dépile les *deux* derniers nombres, on applique l'opération et on empile le résultat.

▶ Écrire `evaluer_npi(expr)` qui renvoie la valeur (un entier) de l'expression `expr`, une chaîne dont les éléments sont séparés par des espaces (on se limite à `+`, `-`, `*`). Les nombres lus sont des chaînes, à convertir avec `int` avant de les empiler. ▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Files ↑cours

★ **Exercice 11** *La bonne opération sur une file* ▶ corrigé p. 794

On dispose d'une file `f`. Pour chaque phrase, écrire l'**appel de méthode** correspondant. ▷ coup de pouce

1. ajouter la valeur 7 à la queue de la file `f` ;
2. retirer et récupérer l'élément de tête de la file `f` ;
3. savoir si la file `f` est vide ;
4. consulter la tête de `f` *sans* la retirer.

★ **Exercice 12** *Trace d'une file* ▶ corrigé p. 795 • essentiel

On part d'une file `f` vide. Même travail (la **tête est à droite** : on défile à droite). ▷ coup de pouce

```
f.enfiler(4)
f.enfiler(7)
f.enfiler(2)
f.defiler()
f.enfiler(9)
f.tete()
f.defiler()
```

★★ **Exercice 13** *Ordonnancer des processus avec une file (d'après Amérique du Nord 2024, jour 1)* ▶ corrigé p. 795 • essentiel ≈ 30 min

Un système d'exploitation partage le processeur entre plusieurs **processus** à l'aide d'une `File`, selon la méthode du *tourniquet* : à chaque cycle, on défile un processus, on l'exécute pendant un cycle, et s'il n'est pas terminé on le remet en fin de file. On dispose des deux classes suivantes.

```
1  class File:
2      def __init__(self):
3          self.contenu = []
4      def enfile(self, element):
5          self.contenu.append(element)      # ajout en queue
6      def defile(self):
7          return self.contenu.pop(0)        # retire et renvoie la tete
8      def est_vide(self):
9          return self.contenu == []
10
11 class Processus:
12     def __init__(self, nom, duree):
13         self.nom = nom
14         self.reste = duree                # nombre de cycles restants
15     def execute(self):
16         self.reste = self.reste - 1       # consomme un cycle
17     def est_fini(self):
18         return self.reste == 0
```

1. Donner l'unique attribut de la classe `File` et son type. Rappeler pourquoi `defile` (qui utilise `pop(0)`) est plus coûteux que `enfile`.

2. Le code `f = File()` puis `print(f.defile())` provoque une erreur. Expliquer laquelle et pourquoi, puis réécrire la méthode `defile` pour qu'elle renvoie `None` lorsque la file est vide.

3. ▶ Recopier et compléter la classe `Ordonnanceur` ci-dessous (l'attribut `temps` est le cycle courant). ▷ coup de pouce

```
1 class Ordonnanceur:
2     def __init__(self):
3         self.temps = 0
4         self.file = File()
```

```
    def ajoute_nouveau_processus(self, proc):
        ...                                 # (question 3)

    def tourniquet(self):
        self.temps = self.temps + 1
        if not self.file.est_vide():
            proc = ...                      # (a) le processus elu
            ...                             # (b) l'executer un cycle
            if not proc.est_fini():
                ...                         # (c) le remettre dans la file
            return proc.nom
        else:
            return None
```

4. On crée `o = Ordonnanceur()`, ainsi que `p1 = Processus("p1", 2)` et `p2 = Processus("p2", 1)`, que l'on ajoute dans cet ordre avec `o.ajoute_nouveau_processus`. On appelle ensuite `o.tourniquet()` **trois** fois. Donner les trois noms renvoyés et l'état final de la file.

★★ **Exercice 14** *Une file de priorité de tâches (d'après Métropole 2025, jour 1)* ▶ corrigé p. 796 ≈30 min

Pour ordonnancer des tâches, on stocke des tuples `(tache, priorite)` dans une `File` d'interface : `enfiler(e)` (ajoute `e` en fin de file), `defiler()` (retire et renvoie la tête), `examiner()` (renvoie la tête sans la retirer), `est_vide()`. La file est rangée de la plus **haute** priorité (en tête) à la plus basse.

On considère la file `f` suivante **(tête à droite)** :

`(<t5>, 1) ; (<t4>, 1) ; (<t2>, 3) ; (<t1>, 3) ; (<t3>, 4)`

1. Donner la valeur de `f.defiler()[0]`, puis représenter le contenu de `f` après cette instruction.

2. En repartant de la file `f` d'origine, donner la valeur de `f.examiner()[1]`, et l'état de `f` après.

3. On veut insérer un tuple `(t, p)` à sa place. Le principe : tant que la tête de `f` est *plus prioritaire* que `p`, on la défile vers une file auxiliaire `f_aux` ; on y enfile ensuite `(t, p)` ; on transvase le reste de `f` dans `f_aux` ; enfin on renvoie `f_aux`. ▶ Recopier et compléter :

▷ coup de pouce

```
def ajouter_file_prio(f, t, p):
    f_aux = File()
    while (not f.est_vide()) and f.examiner()[1] > p:
        f_aux.enfiler(...)
    f_aux.enfiler(...)
    while not f.est_vide():
        f_aux.enfiler(...)
    return f_aux
```

4. En repartant de la file `f` d'origine, représenter la file obtenue par `f = ajouter_file_prio(f, <t6>, 2)` puis `f = ajouter_file_prio(f, <t7>, 4)`.

5. Donner le **coût d'exécution**, dans le pire des cas, de `ajouter_file_prio` en fonction du nombre `m` d'éléments de `f`.

6. *(Pomodoro)* On admet une classe `Tache` munie de `avancer(minutes)` (fait progresser la tâche) et de `est_terminee()` (booléen). ▶ Écrire une fonction `planning(f)` qui : tant que `f` n'est pas vide, défile le tuple de tête, **avance de 25** minutes sa tâche, ajoute cette **tâche** (l'objet `Tache`, pas le tuple) à une liste résultat, puis, si elle n'est pas terminée, la **ré-insère** avec sa priorité : `f = ajouter_file_prio(f, tache, priorite)` (la fonction renvoie une **nouvelle** file) ; la fonction renvoie la liste des tâches dans l'ordre où elles ont avancé.

★★ **Exercice 15** *Une file écrite par un assistant d'IA* ▶ corrigé p. 796

Un élève demande à un assistant d'IA : « Écris une classe Python `File` (structure FIFO) avec les méthodes `est_vide`, `enfiler`, `defiler` et `tete`, à partir d'une liste Python. » Voici la réponse obtenue :

```
class File:
    def __init__(self):
        self.contenu = []            # la tete est au debut de la liste

    def est_vide(self):
        return self.contenu == []

    def enfiler(self, x):
        self.contenu.append(x)       # on ajoute en queue

    def defiler(self):
        return self.contenu.pop()    # on retire l'element de tete

    def tete(self):
        return self.contenu[0]
```

« Les éléments entrent par la queue avec `append` et sortent par la tête avec `pop` : le premier entré est le premier sorti. »

1. La réponse est-elle correcte ? Dérouler à la main la séquence `f = File()`, `f.enfiler(7)`, `f.enfiler(19)`, `f.enfiler(22)`, puis `f.tete()` et `f.defiler()` (dessiner la file horizontalement, tête à droite, comme dans tout le chapitre).
2. Localiser et corriger l'erreur. ▷ coup de pouce
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

★★ **Exercice 16** *Copier une file* ▶ corrigé p. 797

On dispose d'une classe `File` d'interface `est_vide`, `enfiler`, `defiler` et `tete` (sans méthode `taille`). On veut une fonction `copier_file(f)` qui renvoie une **nouvelle** file contenant les mêmes éléments que `f`, dans le même ordre, et qui laisse `f` dans son état d'origine.

1. On défile `f` jusqu'à ce qu'elle soit vide en enfilant chaque élément dans une nouvelle file `copie`. L'ordre est-il conservé ? Comparer avec la même méthode appliquée à une pile (exercice 8, question 2). Que manque-t-il encore ?

2. Pour ne pas vider `f`, un élève propose de réenfiler dans `f` chaque élément juste après l'avoir défilé (et de l'enfiler aussi dans `copie`). Expliquer pourquoi sa boucle `while not f.est_vide():` ne s'arrête jamais.

3. ▶ Écrire la fonction `copier_file(f)`. ▷ coup de pouce

   ▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Piles et files ensemble ↑ cours

★ **Exercice 17** *LIFO ou FIFO ?* ▶ corrigé p. 797 • essentiel

Pour chaque situation, indiquer si la structure adaptée est une **pile** (LIFO) ou une **file** (FIFO), et justifier en une courte phrase. ▷ coup de pouce

a) la fonction « Annuler » (`Ctrl+Z`) d'un traitement de texte ;
b) les documents en attente sur une imprimante partagée ;
c) les appels de fonctions récursives gérés par la machine ;
d) les caractères tapés au clavier en attente d'être traités ;
e) le bouton « page précédente » d'un navigateur.

★ **Exercice 18** *Même séquence, deux structures* ▶ corrigé p. 797

On applique la *même* suite d'ajouts `1, 2, 3, 4` à une pile et à une file, puis on retire **tous** les éléments un par un. ▷ coup de pouce

1. Dans quel ordre les valeurs sortent-elles de la **pile** ?
2. Dans quel ordre sortent-elles de la **file** ?
3. Quelle structure *inverse* l'ordre des éléments ? Quelle structure le *conserve* ?

★★ **Exercice 19** *Piles et files pour parcourir un graphe (d'après Polynésie 2024, jour 2)* ▶ corrigé p. 797 ≈ 20 min

Un graphe est donné par un dictionnaire d'adjacence (les clés sont les sommets, la valeur est la liste des voisins) :

```
graphe = {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'D'],
          'D': ['B', 'C']}
```

On admet une **file** d'interface `enfiler(f, e)` / `defiler(f)` / `est_vide(f)`. Le parcours suivant explore le graphe à partir d'un sommet :

```
def parcours(graphe, depart):
    f = File()
    enfiler(f, depart)
    visite = [depart]
    while not est_vide(f):
        s = defiler(f)
        for v in graphe[s]:
            if v not in visite:
                visite.append(v)
                enfiler(f, v)
    return visite
```

1. Indiquer la signification des acronymes **LIFO** et **FIFO**, et préciser lequel désigne une file.
2. Dérouler `parcours(graphe, 'A')` : donner l'ordre dans lequel les sommets entrent dans `visite`.
3. ▶ Traduire en une fonction Python `parcours2(graphe, depart)` l'algorithme suivant, en utilisant la classe `Pile` du cours (`empiler`, `depiler`, `est_vide`) : ▷ coup de pouce

```
créer une pile p ; empiler depart dans p
créer une liste visite vide
tant que p n'est pas vide :
    x = dépiler p
    si x n'est pas dans visite :
        ajouter x à visite
        pour chaque voisin v de x : empiler v dans p
renvoyer visite
```

4. Donner un **résultat possible** de l'appel `parcours2(graphe, 'A')`.

*On reconnaît les deux parcours nommés dans le cours : avec une file, le parcours **en largeur** ; avec une pile, le parcours **en profondeur**. Ils seront étudiés en détail aux chapitres Arbres et Graphes.*

★★★ Exercice 20 — *Défi — une file avec deux piles*

▶ corrigé p. 798 ≈ 15 min

On ne dispose que de **piles**. On veut fabriquer une file en utilisant **deux piles** `entree` et `sortie` : `enfiler` empile dans `entree` ; `defiler` dépile dans `sortie` — et si `sortie` est vide, on y transvase d'abord *tout* le contenu de `entree`.

1. Sur un exemple (enfiler `1, 2, 3` puis défiler deux fois), expliquer pourquoi on récupère bien `1` puis `2` (comportement FIFO).
2. Pourquoi le transvasement d'une pile dans l'autre *remet-il* les éléments dans le bon ordre ?

▷ coup de pouce ▷ début de solution

Listes chaînées ↑ cours

Rappel — la classe `Cellule`

Une liste chaînée est faite de **cellules** (maillons). On les représente par la classe suivante ; une liste est soit `None` (liste vide), soit une `Cellule` dont l'attribut `suivante` contient le reste de la liste (définition **récursive**).

```
1  class Cellule:
2      def __init__(self, v, s):
3          self.valeur = v       # l'element
4          self.suivante = s     # la cellule suivante, ou None a la fin
5
6  # la liste 1, 2, 3 :
7  lst = Cellule(1, Cellule(2, Cellule(3, None)))
```

★★ Exercice 21 — *Longueur d'une liste (avec une boucle)*

▶ corrigé p. 798 • essentiel

▶ Écrire `longueur(lst)` qui renvoie le nombre de cellules de la liste chaînée `lst`, à l'aide d'une boucle `while` qui parcourt les cellules en suivant les liens. *(La liste vide a pour longueur `0`.)*

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ Exercice 22 — *Nombre d'occurrences*

▶ corrigé p. 798

▶ Écrire `occurrences(x, lst)` qui renvoie le nombre de fois que la valeur `x` apparaît dans la liste chaînée `lst`. L'écrire au choix récursivement ou avec une boucle.

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ Exercice 23 — *Le n-ième élément*

▶ corrigé p. 799

▶ Écrire `nieme(n, lst)` qui renvoie la valeur de la cellule d'indice `n` (le premier élément a l'indice `0`). Si `n` est supérieur ou égal à la longueur de la liste, la fonction doit lever `IndexError("indice invalide")`.

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 24** *Le collier de bonbons : maillons chaînés (d'après Amérique du Nord 2025, jour 2)* ▶ corrigé p. 799 ≈ 45 min

Un « collier » de bonbons est une liste chaînée **circulaire** et **doublement** chaînée : chaque bonbon est un maillon possédant trois informations — une `valeur`, un `pred` (le bonbon précédent) et un `succ` (le suivant), tous deux des objets `Bonbon`.

```
class Bonbon:
    def __init__(self, valeur):
        self.pred = None
        self.valeur = valeur
        self.succ = None
```

On construit un collier de trois bonbons de valeurs `0`, `1`, `2` :

```
zero = Bonbon(0)
un = Bonbon(1)
deux = Bonbon(2)
zero.succ = un ;   un.succ = deux ;   deux.succ = zero
zero.pred = deux ; un.pred = zero ;   deux.pred = un
```

1. Donner le terme correspondant à `pred`, `valeur` et `succ` dans le vocabulaire de la programmation orientée objet.

2. Déterminer les valeurs de `a` et `b` après :

```
a = zero.succ.valeur
b = un.succ.succ.pred.valeur
```

3. ▶ Écrire `creer_collier(n)` qui crée un collier de `n` bonbons de valeurs `0, 1, ..., n-1`, correctement chaîné dans les deux sens **et refermé** (le dernier a pour successeur le premier), puis renvoie le premier bonbon (valeur `0`). On suppose `n` $\geqslant 1$. ▷ coup de pouce

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

4. Écrire les **deux instructions** qui « mangent » le bonbon `deux`, c'est-à-dire le **retirent** du collier en reliant directement son prédécesseur et son successeur.

5. Quelle égalité caractérise un collier réduit à **un seul** bonbon `b` ? ▷ coup de pouce

6. *(défi ★★★)* ▶ Écrire `dernier_chaine(n)` qui construit le collier de `n` bonbons (avec `creer_collier`), puis les « mange » de proche en proche en partant du bonbon de valeur `0` (celui que renvoie `creer_collier`) — à chaque étape, le bonbon courant **mange son successeur** (on le retire du collier), puis on passe au bonbon suivant — jusqu'à ce qu'il n'en reste qu'**un**, dont on renvoie la `valeur`. ▷ coup de pouce

★★ **Exercice 25** *Copier une liste chaînée* ▶ corrigé p. 800

1. On exécute le programme suivant. Que vaut `lst.valeur` à la fin ? Pourquoi ?

```
lst = Cellule(1, Cellule(2, Cellule(3, None)))
autre = lst
autre.valeur = 10
```

2. ▶ Écrire une fonction **récursive** `copier_liste(lst)` qui renvoie une **nouvelle** liste chaînée, faite de **nouvelles** cellules, contenant les mêmes valeurs que `lst` dans le même ordre. ▷ coup de pouce

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

3. Vérifier qu'après `c = copier_liste(lst)`, l'instruction `c.valeur = 10` ne modifie pas `lst`.

★★★ **Exercice 26** *Défi — insertion dans une liste triée* ▶ corrigé p. 801 ≈ 20 min

▶ Écrire `inserer(x, lst)` qui prend un entier `x` et une liste chaînée `lst` **triée par ordre croissant**, et renvoie une **nouvelle** liste triée où `x` a été inséré à sa place. Ainsi, insérer `3` dans `1, 2, 5, 8` donne `1, 2, 3, 5, 8`. ▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

S'entraîner à l'oral ↑ cours

★★ **Exercice 27** *Expliquer en deux minutes* ▶ corrigé p. 801

Au Grand Oral comme devant l'examinateur de l'épreuve pratique, il faut savoir **expliquer** une notion clairement, sans lire ses notes. Choisir l'un des sujets suivants (ou le tirer au sort) :

Sujet 1. Expliquer à un camarade de Première la différence entre une **pile** et une **file**, avec pour chacune un exemple de la vie courante et un usage en informatique.

Sujet 2. Expliquer la différence entre **interface** et **implémentation** : pourquoi peut-on utiliser une pile sans savoir comment elle est programmée ?

Sujet 3. Expliquer pourquoi ajouter un élément en tête d'une **liste chaînée** est rapide, alors qu'accéder à son millième élément est lent.

Préparation (5 min, seul). Noter au brouillon **trois idées** et **un exemple**, rien de plus : comme au Grand Oral, ces notes servent d'aide-mémoire. **Exposé (2 min, chronométré).** Debout, face à un camarade, **sans lire ses notes** et sans rien écrire : tout passe par la parole. **Retour (2 min).** Le camarade pose une question de relance (on peut alors s'aider d'un schéma), remplit la grille ci-dessous et donne un conseil ; puis on échange les rôles. Cette grille reprend les critères d'évaluation du Grand Oral. ▷ coup de pouce

Grille entre camarades	**Oui**	**En partie**	**Pas encore**
Clair — audible, posé ; chaque mot technique est expliqué	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	☐	☐	☐
Construit — un fil conducteur, tenu en deux minutes, sans lire	☐	☐	☐
Un conseil pour la prochaine fois :			

Interface d'une liste, liste ou dictionnaire ↑ cours

★★ **Exercice 28** *L'interface d'une liste* ▶ corrigé p. 802

On dispose de l'interface du cours : `liste_vide()`, `est_vide(L)`, `ajouter_tete(x, L)`, `tete(L)` et `queue(L)`.

1. On pose `L = ajouter_tete(3, ajouter_tete(8, ajouter_tete(1, liste_vide())))`. Écrire les éléments de `L` dans l'ordre, puis donner `tete(L)`, `tete(queue(L))` et `est_vide(queue(queue(queue(L))))`.

2. ▶ En n'utilisant **que** l'interface, écrire une fonction récursive `appartient(x, L)` qui renvoie `True` si `x` est un élément de `L`, `False` sinon. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

3. ▶ Toujours avec la seule interface, écrire `renverser(L)` qui renvoie une **nouvelle** liste contenant les éléments de `L` dans l'ordre inverse. ▷ coup de pouce

4. ▶ Implémenter les cinq opérations de l'interface avec la classe `Cellule` du rappel, puis tester les questions 1 à 3.

5. Après `M = ajouter_tete(9, L)`, la liste `L` a-t-elle changé ? Comparer avec `p.empiler(9)` sur une pile `p`.

★★ Exercice 29 — *Liste ou dictionnaire ?* ▶ corrigé p. 803

Le portique d'un lycée doit vérifier, à chaque passage, que le numéro de badge présenté est autorisé. Il y a n badges autorisés.

1. On range les numéros autorisés dans une liste `autorises`. Combien de comparaisons fait, au pire, le test `b in autorises` ? Quand ce pire cas se produit-il ?

2. On range plutôt les badges dans un dictionnaire `badges` dont les **clés** sont les numéros (et les valeurs, le nom de l'élève). Que dire du coût du test `b in badges` ?

3. ▶ Recopier et compléter la fonction suivante, qui mesure le temps d'un test pour un badge absent (le pire cas), puis l'appeler pour $n = 10\,000$, $100\,000$ et $1\,000\,000$. Que constate-t-on quand n est multiplié par 10 ?

```
import time

def mesurer(n):
    """Renvoie le couple (temps liste, temps dictionnaire), en secondes."""
    autorises = [b for b in range(n)]
    badges = {b: "inconnu" for b in range(n)}
    debut = time.perf_counter()
    trouve = -1 in autorises
    t_liste = time.perf_counter() - debut
    ...                                 # a completer pour le dictionnaire
    return (t_liste, t_dico)
```

4. Le surveillant veut aussi savoir si une élève nommée `"Lina"` a un badge. Le test `"Lina" in badges.values()` est-il aussi rapide que `b in badges` ? Pourquoi ?

5. Le portique doit en plus afficher, à la fin de la journée, les passages **dans l'ordre** où ils ont eu lieu. Quelle structure choisir pour mémoriser les passages ? Et pour les badges autorisés ?

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Relire l'interface d'une pile (cours, ou « Réflexe » du mode d'emploi) : quatre méthodes, une par phrase. Lesquelles modifient la pile, lesquelles la laissent intacte ? ↑ énoncé

Ex. 2 – piste Tenir un tableau à trois colonnes : instruction, valeur renvoyée, contenu de la pile après l'instruction. Seuls `depiler` et `sommet` renvoient une valeur : lequel des deux ne retire rien ? ↑ énoncé

Ex. 3 – piste À la lecture d'un symbole fermant, deux situations font échouer : lesquelles ? Et que doit contenir la pile une fois toute l'expression lue ? ↑ énoncé

Ex. 4 – piste Associer à chaque fermante son ouvrante, par exemple avec un dictionnaire. Il y a trois cas d'échec : une fermante alors que la pile est vide, une mauvaise correspondance, une pile non vide à la fin. ↑ énoncé

Ex. 4 – début de solution

```
def parenthesage_correct(expr):
    p = Pile()
```

```
for c in expr:
    if c in "([{":
        p.empiler(c)
```
↑énoncé

Ex. 5 – piste Tout empiler, puis dépiler tant que la pile n'est pas vide, en ajoutant chaque élément dépilé à la nouvelle liste. ↑énoncé

Ex. 6 q. 3 – piste Une boucle `while` qui tourne tant que la pile n'est pas vide ; dans le corps, un seul `depiler`. ↑énoncé

Ex. 7 q. 2 – piste Les éléments dépilés doivent être rangés quelque part pour être remis ensuite : dans une seconde pile, par exemple. Dans quel ordre en ressortent-ils ? ↑énoncé

Ex. 8 q. 3 – piste Un seul transvasement renverse l'ordre ; deux transvasements le rétablissent. Pendant le second, chaque élément retiré de la pile auxiliaire peut être empilé **deux fois** : dans `p` et dans la copie. ↑énoncé

Ex. 8 q. 3 – début de solution
```
def copier_pile(p):
    aux = Pile()
    while not p.est_vide():
        aux.empiler(p.depiler())
    q = Pile()
```
↑énoncé

Ex. 9 – piste Une balise fermante commence par `"</"` : il suffit de comparer ses deux premiers caractères, `b[:2]`, à `"</"`. On empile les ouvrantes telles quelles ; pour une fermante `b`, l'ouvrante attendue au sommet s'obtient par concaténation : `"<" + b[2:]`. Il reste à reprendre l'algorithme du parenthésage. ↑énoncé

Ex. 9 – début de solution
```
def balises_ok(liste):
    p = Pile()
    for b in liste:
        if b[:2] != "</":
            p.empiler(b)
```
↑énoncé

Ex. 10 – piste `expr.split()` découpe la chaîne en une liste d'éléments. Lors d'un opérateur, le premier nombre dépilé est l'opérande de **droite** : attention à l'ordre pour `-`. À la fin, la pile ne contient plus qu'un nombre : le résultat. ↑énoncé

Ex. 10 – début de solution
```
def evaluer_npi(expr):
    p = Pile()
    for jeton in expr.split():
        if jeton in "+-*":
            b = p.depiler()
            a = p.depiler()
```
↑énoncé

Ex. 11 – piste Même travail qu'à l'exercice 1, avec l'interface d'une file : `enfiler`, `defiler`, `est_vide`, `tete`. Par quel bout entre-t-on dans une file, par quel bout en sort-on ? ↑énoncé

Ex. 12 – piste Même tableau qu'à l'exercice 2. On enfile à gauche et l'on défile à droite : l'élément qui sort est toujours le plus ancien encore présent. ↑énoncé

Ex. 13 q. 3 – piste Chaque ligne à compléter appelle une méthode de `self.file` ou de `proc` : relire les méthodes des deux classes données et choisir la bonne. ↑énoncé

Ex. 14 q. 3 – piste Dans les deux boucles, l'élément à enfiler dans `f_aux` est celui que l'on vient de retirer de `f` : par quel appel l'obtient-on ? Entre les deux boucles, on enfile le nouveau tuple. ↑énoncé

Ex. 15 q. 2 – piste Sur une liste Python, quel élément `pop()` sans argument retire-t-il : le premier ou le dernier ? ↑énoncé

Ex. 16 q. 3 – piste Une file auxiliaire garde les éléments retirés, le temps de les remettre dans `f` une fois la copie terminée. Contrairement à une pile, un transvasement de file à file conserve l'ordre. ↑énoncé

Ex. 17 – piste Pour chaque situation, se demander quel élément doit être traité en premier : le plus récent (pile) ou le plus ancien (file) ? ↑énoncé

Ex. 18 – piste Dessiner les deux structures après les quatre ajouts (pile verticale, sommet en haut ; file horizontale, tête à droite), puis retirer les éléments un par un en notant chaque valeur qui sort. ↑énoncé

Ex. 19 q. 3 – piste S'inspirer de la fonction `parcours` donnée : remplacer la file par une pile, puis traduire l'algorithme ligne à ligne (une ligne d'algorithme, une instruction Python). ↑

Ex. 20 q. 2 – piste Dessiner les deux piles verticalement après chaque opération. Que devient l'ordre des éléments quand on vide une pile dans une autre ? ↑énoncé

Ex. 20 q. 2 – début de solution Après les trois enfilages, `entree` contient `1`, `2`, `3` (`3` au sommet) et `sortie` est vide. Le premier `defiler` transvase donc tout : `sortie` contient alors `3`, `2`, `1`, avec `1` au sommet… ↑énoncé

Ex. 21 – piste Quelle variable sert de « curseur » sur les cellules, et comment avance-t-elle d'une cellule à la suivante ? Quand faut-il s'arrêter ? ↑énoncé

Ex. 22 – piste Avec une boucle : même parcours que pour la longueur, mais le compteur n'augmente que sous condition. En récursif : condition d'arrêt `lst is None`, puis l'appel porte sur `lst.suivante`. ↑énoncé

Ex. 23 – piste Combien de fois faut-il suivre le lien `suivante` pour atteindre l'indice `n` ? Que vaut le curseur si la liste est trop courte ? L'erreur se lève avec `raise IndexError("indice invalide")` (voir le cours du chapitre *Programmation orientée objet*, « Signaler une erreur »). ↑énoncé

Ex. 24 q. 3 – piste Q3 : créer le premier bonbon, puis, dans une boucle, chaque nouveau bonbon en le reliant au précédent dans les deux sens (une variable retient le précédent). Après la boucle, refermer le collier entre le dernier et le premier. ↑énoncé

Ex. 24 q. 5 – piste Q5 : que valent alors `b.succ` et `b.pred` ? ↑énoncé

Ex. 24 q. 6 – piste Q6 : une boucle qui tourne tant que le collier n'est pas réduit à un bonbon (question 5) ; à chaque tour : retirer le successeur du bonbon courant (question 4), puis avancer. ↑énoncé

Ex. 25 q. 2 – piste Condition d'arrêt : la liste vide. Sinon, la copie commence par une nouvelle cellule : quelle valeur contient-elle, et que doit valoir sa cellule suivante ? ↑énoncé

Ex. 26 – piste Une fonction récursive convient bien. Deux cas immédiats : la liste est vide, ou `x` est inférieur ou égal à la première valeur ; on crée alors une cellule en tête. Sinon, on garde la première valeur, suivie de l'insertion de `x` dans le reste. ↑énoncé

Ex. 26 – début de solution

```
def inserer(x, lst):
    if lst is None or x <= lst.valeur:
        return Cellule(x, lst)
```

↑énoncé

Ex. 27 – piste Structurer chaque explication en trois temps : une **définition** en une phrase, un **exemple** petit et concret déroulé à voix haute, puis le **piège** (l'erreur que l'auditeur risque de faire). Finir par une phrase qui répond à la question posée. Pour le sujet 1 : faire entrer trois valeurs dans l'ordre $1, 2, 3$, puis dire laquelle sort en premier dans chaque structure. Pour le sujet 3 : que connaît-on, au départ, d'une liste chaînée ? ↑énoncé

Ex. 28 q. 2 – piste Condition d'arrêt : la liste vide ne contient rien. Sinon, soit la tête est `x`, soit il faut chercher dans la queue. ↑énoncé

Ex. 28 q. 3 – piste Partir d'une liste vide `r` et, en parcourant `L` du début à la fin, ajouter chaque élément en tête de `r` : le premier élément lu finit en dernier. ↑énoncé

TP ET PROJETS

TP Piles et files au travail ▶ corrigé p. 803

deux implémentations, quatre applications

Durée : 3 h environ (deux séances) · Sur machine, seul ou par deux

Fichiers à télécharger : `maththibaud.github.io/informatique-en-autonomie/f/t03-tp-structures-lineaires` — **archive .zip**

But du TP

Construire soi-même une pile et une file à partir d'une **liste chaînée**, **mesurer** ce que l'on gagne, puis **mettre ces structures au service** de quatre programmes réels : un vérificateur de code et de pages HTML, une calculatrice qui respecte les priorités, les boutons « Annuler / Rétablir » d'un éditeur, et la simulation de la file d'attente d'un guichet. ▶ signale ce qui est à programmer et tester.

Consignes

- Les parties 2 à 5 sont indépendantes une fois la partie 1 terminée.
- Le fichier `tp_structures_lineaires_depart.py` est **à télécharger** (lien ci-dessus) : il contient les classes du cours, des fonctions toutes prêtes et, en fin de fichier, des **tests**. Relancer le fichier après chaque partie : chaque ligne `[A FAIRE]` doit devenir `[OK]`.
- Dans les parties 2 à 5, on n'utilise **que l'interface** des piles et des files (`empiler`, `depiler`, `sommet`, `enfiler`, `defiler`, `tete`, `est_vide`, `taille`). C'est une interface **étendue** : le cours n'a pas de méthode `taille`, le TP en ajoute une, de coût constant grâce à l'attribut `_taille`.
- Usage de l'IA, indiqué sur chaque exercice :  aucune ;  en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ;  l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

1 Deux implémentations pour une même interface

Le cours a donné une pile et une file qui cachent une `list` Python (classes `PileListe` et `FileListe` du fichier fourni) et a signalé que `pop(0)` rendait la file lente. On construit ici l'autre implémentation, avec la classe `Cellule` du cours.

★ **Exercice 1**  *Une pile chaînée*

Dans `PileChainee`, l'attribut `_tete` désigne la cellule du **sommet** (ou `None` si la pile est vide) et `_taille` le nombre d'éléments.

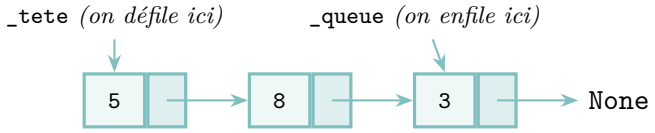


La pile obtenue après `empiler(5)`, `empiler(8)`, `empiler(3)`.

1. Dessiner la pile après un `empiler(7)`, puis après un `depiler()`. Combien de liens faut-il modifier à chaque fois ? Ce nombre dépend-il de la taille de la pile ?
2. ▶ Compléter les cinq méthodes de `PileChainee`. `depiler` et `sommet` lèvent `IndexError("pile vide")` sur une pile vide (comme `PileListe` ; `raise` : voir le chapitre *Programmation orientée objet*). ▷ coup de pouce
3. ▶ Vérifier que `tester_pile(PileListe)` et `tester_pile(PileChainee)` renvoient `"OK"`. Pourquoi la **même** fonction de test convient-elle aux deux classes ?
4. ▶ Ajouter une méthode `__repr__` pour que `print(p)` affiche `sommet -> [3, 8, 5]`.

★★ Exercice 2 *Une file chaînée à deux entrées*

Pour une file, on retire en tête et l'on ajoute en queue. On garde donc **deux** attributs : `_tete` (la première cellule, celle qu'on défilera) et `_queue` (la dernière, derrière laquelle on enfile).



1. Dessiner la file après `enfiler(1)`, puis après `defiler()`. Que se passerait-il, pour `enfiler`, si l'on n'avait *pas* l'attribut `_queue` ?
2. Deux cas particuliers demandent de l'attention : enfiler dans une file **vide**, et défiler le **dernier** élément. Que doivent valoir `_tete` et `_queue` dans chacun de ces cas ?
3. ▶ Compléter les cinq méthodes de `FileChainee` (`tete` renvoie l'élément de tête sans le retirer ; `defiler` et `tete` lèvent `IndexError("file vide")` sur une file vide) et vérifier que `tester_file(FileChainee)` renvoie `"OK"`. Ce test enfile de nouveau après avoir vidé la file : quel oubli détecte-t-il ? ▷ coup de pouce

★★ Exercice 3 *Mesurer le gain*

La fonction fournie `chronometrer_file(Classe, n)` renvoie le temps (en secondes) mis pour enfiler puis défiler `n` entiers.

1. ▶ Recopier et compléter sur le cahier le tableau suivant (deux chiffres significatifs suffisent).

`n`	10 000	50 000	100 000	200 000
`FileListe` (s)				
`FileChainee` (s)				

2. Quand `n` double, par combien le temps est-il à peu près multiplié pour chaque classe ? Relier ces observations au coût d'un `defiler` dans chaque implémentation (voir le cours).
3. Les programmes des parties suivantes n'ont pas besoin de savoir quelle implémentation est utilisée. Expliquer en une phrase pourquoi c'est un avantage.

2 Un vérificateur pour de vrais programmes

Le cours vérifie le parenthésage d'une expression courte et répond seulement « correct » ou « incorrect ». Un éditeur de code fait mieux : il travaille sur un **programme entier** et **indique où** est l'erreur.

★★ **Exercice 4** *Localiser l'erreur dans un programme Python*

On veut écrire `verifier_code(code)` qui renvoie `None` si les `()`, `[]` et `{}` du programme `code` (une chaîne pouvant contenir des `"\n"`) sont bien appariés, et sinon **un message** parmi les trois formes suivantes :

```
ligne 1, colonne 26 : ')' ferme '[' ouvert ligne 1, colonne 21
ligne 1, colonne 5 : '(' jamais ferme
ligne 1, colonne 6 : ')' ne ferme rien
```

Les lignes et les colonnes sont numérotées **à partir de 1**. On renvoie le message de la **première** erreur rencontrée ; si plusieurs symboles restent ouverts à la fin du code, on signale le **dernier** ouvert (celui du sommet de la pile).

Deux difficultés nouvelles : un symbole écrit **dans une chaîne** (`print("(")`) ou **dans un commentaire** (`# voir (2)`) ne compte pas ; et, pour signaler où se trouve un symbole ouvrant jamais refermé, il faut se souvenir de **sa** position. Attention : un `#` écrit dans une chaîne ne commence pas un commentaire, et un guillemet écrit dans un commentaire n'ouvre pas de chaîne. On ne gère ni les chaînes triples `"""..."""` ni les guillemets précédés d'un `\`.

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1. Que faut-il empiler, au lieu du seul symbole ouvrant, pour pouvoir écrire ces messages ? Proposer un **tuple**.
2. On parcourt le code caractère par caractère en tenant à jour `ligne` et `colonne`, ainsi que deux indicateurs : `dans_chaine` (vaut `None`, ou le guillemet `"` ou `'` qui a ouvert la chaîne en cours) et `dans_commentaire` (booléen). À quel caractère chacun de ces deux indicateurs se « remet-il à zéro » ?
3. ▶ Écrire `verifier_code`. Les tests de la partie 2 du fichier fourni doivent passer. Essayer aussi la fonction sur un de vos propres programmes, puis après y avoir supprimé une parenthèse.

★★★ **Exercice 5** *Défi — une vraie page HTML*

Une page Web réelle contient des balises avec **attributs** (`<h1 class="titre">`) et des balises **orphelines**, qui n'ont pas de fermante (`<br>`, `<img src="a.png">`…). La fonction fournie `extraire_balises(html)` s'occupe de ces détails : elle renvoie la liste des noms de balises dans l'ordre, une fermante étant précédée de `/`.

```
>>> extraire_balises('<p class="a">Un <b>mot</b><br></p>')
['p', 'b', '/b', 'br', '/p']
```

▶ Écrire `verifier_html(html)` qui renvoie `True` si les balises de la page sont bien imbriquées, puis la tester sur une page HTML écrite en SNT ou téléchargée (affichage du code source dans le navigateur, puis copier-coller dans une chaîne `"""..."""`).

3 Une calculatrice qui respecte les priorités

En notation polonaise inverse (NPI), on écrit l'opérateur *après* ses deux opérandes : `"3 4 2 * +"` signifie $3 + 4 \times 2$. L'évaluation se fait avec une pile de nombres (feuille d'exercices). On construit ici la calculatrice complète : on **traduit** d'abord l'expression usuelle en NPI, puis on l'**évalue**.

★★ **Exercice 6** *Une évaluation qui ne plante pas*

Rappel (défi de la feuille d'exercices). L'expression est une chaîne dont les jetons sont séparés par des espaces : `expr.split()` en donne la liste. On parcourt les jetons avec une pile de nombres : un **nombre** est empilé ; un **opérateur** dépile deux nombres, fait le calcul et empile le résultat ; à la fin, le résultat est le nombre resté dans la pile.

Ce qui change par rapport à la feuille. On traite les **quatre** opérations `+`, `-`, `*`, `/` ; les nombres sont convertis avec `float`, donc le résultat est un flottant (`17.0`) ; surtout, une expression **mal formée** ne doit plus provoquer une erreur obscure de pile vide, mais un message qui dit ce qui ne va pas.

1. À la main, écrire l'état de la pile (sommet à droite) après chaque jeton de `"7 2 - 3 *"`. Quel est le résultat ?
2. Faire le même travail pour `"3 +"`, puis pour `"3 4 5 +"`. Pour chacune, à quel moment précis voit-on que l'expression est mal formée, et que contient alors la pile ? En déduire les **deux vérifications** que la fonction doit faire, et l'endroit du programme où placer chacune.
3. ▶ Écrire `evaluer_npi(expr)`, qui :
   - renvoie la valeur d'une expression correcte, en utilisant pour chaque opérateur la fonction fournie `calculer(a, op, b)`, où `a` est l'opérande de **gauche** et `b` celui de **droite** ;
   - lève `ValueError` avec le message `"il manque un operande pour '+'"` (en y écrivant l'opérateur concerné) si un opérateur arrive alors que la pile contient moins de deux nombres ;
   - lève `ValueError` avec le message `"expression incomplete : il reste 2 nombres"` (en y écrivant le nombre d'éléments restants) si, à la fin, la pile ne contient pas exactement **un** nombre.

   Exemples attendus :

```
>>> evaluer_npi("10 4 /")
2.5
>>> evaluer_npi("3 +")
ValueError: il manque un operande pour '+'
>>> evaluer_npi("3 4 5 +")
ValueError: expression incomplete : il reste 2 nombres
```

   Les autres erreurs ne sont pas à traiter : un jeton qui n'est pas un nombre (`"3 x +"`) fait déjà échouer `float` avec une `ValueError`, et `calculer` lève elle-même `ZeroDivisionError` pour une division par zéro. Les tests `Partie 3 : evaluer_npi` du fichier fourni doivent passer.

▷ coup de pouce

4. Que se passe-t-il si l'on appelle `evaluer_npi("")` (chaîne vide) ? Le message obtenu est-il adapté ?

★★★ **Exercice 7** *De l'écriture usuelle à la NPI*

Pour traduire une expression usuelle (dite *infixe*), on la lit de gauche à droite, jeton par jeton (la fonction fournie `decouper` produit la liste des jetons), avec une **liste de sortie** et une **pile d'opérateurs** :

- un **nombre** va directement dans la sortie ;
- une parenthèse `(` est empilée ;
- une parenthèse `)` : on dépile les opérateurs vers la sortie jusqu'à trouver la `(`, que l'on dépile et que l'on jette ;
- un **opérateur** `op` : tant que le sommet de la pile est un opérateur de priorité **supérieure ou égale** à celle de `op` (`*` et `/` valent 2, `+` et `-` valent 1, dictionnaire `PRIORITE`), on le dépile vers la sortie (une `(` au sommet arrête la boucle : elle reste dans la pile) ; puis on empile `op` ;
- à la fin, on vide la pile dans la sortie.

1. À la main, recopier et compléter sur le cahier la trace pour `2 * (3 + 4) - 5` :

Jeton	Pile d'opérateurs écrite en ligne (sommet à droite)	Sortie
2	(vide)	2
*	*	2
(		
3		
+		
4		
)		
-		
5		
fin		

2. Pourquoi la règle dit-elle « supérieure **ou égale** » ? Traduire `10 - 4 - 3` deux fois : avec la règle ci-dessus, puis en remplaçant « supérieure ou égale » par « strictement supérieure ». Évaluer les deux expressions NPI obtenues : laquelle donne le bon résultat, et pourquoi ?
3. ▶ Écrire `infixe_vers_npi(expr)` qui renvoie la chaîne NPI, jetons séparés par un espace : `infixe_vers_npi("(3 + 4) * 2")` renvoie `"3 4 + 2 *"`. Les jetons produits par `decouper` sont des chaînes ; un jeton qui n'est ni un opérateur ni une parenthèse est un nombre. La fonction `calculatrice` fournie enchaîne alors traduction et évaluation : vérifier que `calculatrice("12*(3+4)-(5+5)*2")` vaut bien 64.

> **Remarque**
> Cette méthode, publiée par Edsger Dijkstra en 1961, est surnommée « algorithme de la gare de triage » : les opérateurs patientent sur une voie de garage (la pile) avant de rejoindre le train de sortie. Les calculatrices et les compilateurs utilisent toujours cette idée.

4 Les boutons « Annuler » et « Rétablir »

★★ **Exercice 8** *Un mini-éditeur*

La classe `Editeur` possède un attribut `texte` et **deux piles** : `_annuler` contient les états précédents du texte, `_retablir` les états qui viennent d'être annulés.

- `ecrire(s)` ajoute `s` à la fin du texte ; `effacer(k)` supprime les `k` derniers caractères ;
- avant chacune de ces deux actions, l'état courant est empilé dans `_annuler`, et la pile `_retablir` est **vidée** ;
- `annuler()` revient à l'état précédent ; `retablir()` refait ce qui vient d'être annulé ; ni l'une ni l'autre ne fait quoi que ce soit si sa pile est vide.

1. Avant de coder : dans `annuler()`, l'état courant ne disparaît pas, il est rangé quelque part. Où ? Même question pour `retablir()`.
2. ▶ Compléter la classe et faire passer les tests de la partie 4.
3. Pourquoi faut-il vider `_retablir` lors d'une nouvelle action ? Donner un exemple de texte absurde obtenu si on l'oublie.
4. On sauvegarde ici le texte **entier** à chaque action. Pour un document de 100 000 caractères modifié 1 000 fois, combien de caractères les piles contiennent-elles environ ? Proposer une autre chose à empiler pour économiser la mémoire.

5 Simuler la file d'attente d'un guichet

Un guichet sert les clients dans leur ordre d'arrivée : c'est une **file**. On représente chaque client par un couple `(arrivée, durée)` en minutes. La simulation avance minute par minute ; à chaque minute `t` :

a) les clients qui arrivent à la minute `t` sont enfilés ;
b) si le guichet est libre (il se libère à la minute `libre_a`, et `libre_a <= t`) et que la file n'est pas vide, on défile un client : son attente vaut `t - arrivée`, et le guichet sera libre à `t + durée`.

La simulation s'arrête quand tous les clients sont arrivés **et** que la file est vide.

★ **Exercice 9** *À la main*

On utilise les dix clients de la liste `CLIENTS` du fichier fourni : `(0, 4)`, `(1, 3)`, `(2, 2)`, `(3, 5)`, `(5, 1)`, `(6, 3)`, `(6, 2)`, `(9, 4)`, `(12, 1)`, `(13, 2)`. Recopier et compléter sur le cahier le tableau des cinq premiers clients, avec un seul guichet.

Client	Arrivée	Début du service	Attente	Fin du service
1	0	0	0	4
2	1			
3	2			
4	3			
5	5			

★★ **Exercice 10** *Programmer la simulation*

1. ▶ Écrire `simuler(clients)` pour **un** guichet ; elle renvoie le triplet `(attente moyenne, attente maximale, longueur maximale de la file)`. La longueur de la file se mesure à la **fin** de chaque minute, après le service ; la simulation commence à `t = 0`. Vérifier qu'on obtient `(7.9, 12, 6)` sur `CLIENTS`, et que vos cinq premières attentes calculées à la main sont bien celles du programme.
2. ▶ Ajouter un paramètre `nb_guichets` (valeur par défaut `1`) : on garde **une seule file** commune, et une **liste** `libre_a` d'une case par guichet. Avec deux guichets, on doit obtenir `(0.9, 3, 2)`. ▷ coup de pouce

★★ **Exercice 11** *Quand la file explose*

La fonction fournie `generer_clients(n, proba, duree_max, graine)` fabrique `n` clients : à chaque minute, un client arrive avec la probabilité `proba`, et son service dure entre 1 et `duree_max` minutes (en moyenne 3,5 minutes pour `duree_max = 6`). La `graine` rend le tirage reproductible : avec la même graine, tout le monde obtient les mêmes clients.

1. ▶ Avec `n = 500`, `duree_max = 6` et `graine = 42`, recopier et compléter le tableau (attente moyenne arrondie au dixième).

`proba`	Charge = `proba` ×3,5	Attente moy., 1 guichet	Attente moy., 2 guichets
0,20	0,70		
0,25			
0,30			
0,35			

2. La **charge** est le nombre moyen de minutes de travail qui arrivent au guichet par minute. Que se passe-t-il pour l'attente quand la charge dépasse 1 ? Expliquer pourquoi, sans calcul.
3. *(Pour aller plus loin.)* ▶ Tracer, avec `matplotlib`, l'attente moyenne en fonction de `proba` (de 0,05 à 0,40 par pas de 0,01) pour un et deux guichets. Un responsable veut une attente moyenne inférieure à 5 minutes avec un seul guichet : jusqu'à quelle fréquence d'arrivée peut-il tenir ?

6 Bilan à rédiger

Exercice 12  *Ce que j'ai construit*

Rédiger, en une dizaine de lignes, un bilan qui répond aux questions suivantes.

1. Dans chacune des quatre applications, quelle structure a-t-on utilisée (pile ou file) et pourquoi celle-ci plutôt que l'autre ? Répondre par une phrase par application, en utilisant les mots LIFO et FIFO.
2. Citer une situation du TP où l'on a changé l'**implémentation** sans toucher au programme qui **utilise** la structure. Qu'est-ce que cela a changé, et qu'est-ce qui n'a pas changé ?
3. Quelle partie a été la plus difficile ? Quel test (ou quelle trace à la main) vous a permis de trouver une erreur ?

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 q. 2 – piste Empiler, c'est créer une cellule *devant* l'ancienne tête : `Cellule(x, self._tete)`. Dépiler, c'est mémoriser la valeur de la tête, puis avancer la tête d'une cellule. Ne pas oublier de tenir `_taille` à jour. ↑ énoncé

Ex. 2 q. 3 – piste Enfiler : créer `c = Cellule(x, None)` ; si la file est vide, `c` devient la tête, sinon on l'accroche derrière l'ancienne queue (`self._queue.suivante = c`) ; dans les deux cas, `c` devient la queue. Défiler : si la tête devient `None`, la queue doit aussi redevenir `None`. ↑ énoncé

Ex. 6 q. 3 – piste La vérification de `p.taille()` se fait *avant* de dépiler les deux opérandes. Le premier nombre dépilé est l'opérande de **droite** : c'est important pour `-` et `/`. Pour lever l'erreur : `raise ValueError("...")`. ↑ énoncé

Ex. 10 q. 2 – piste À chaque minute, après les arrivées, parcourir les guichets : chaque guichet libre prend le client en tête de file, s'il y en a un. ↑ énoncé

FICHE DE RÉVISION

L'essentiel — deux structures par leur interface

	Pile (LIFO)	**File (FIFO)**
Principe	dernier entré, premier sorti	premier entré, premier sorti
Ajouter	`empiler(x)`	`enfiler(x)`
Retirer	`depiler()` (le sommet)	`defiler()` (la tête)
Consulter	`sommet()`	`tete()`
Tester	`est_vide()`	`est_vide()`
Image	Ctrl+Z, pile d'assiettes	file d'attente, impression

*On **admet** en général l'implémentation : on programme **avec** l'interface.*

Exemple phare de pile : le bon parenthésage

On **empile** chaque symbole ouvrant ; à chaque symbole fermant, on **dépile** et le sommet retiré doit être l'ouvrant **correspondant**. L'expression est correcte si l'on ne dépile jamais une pile vide et si la pile finit **vide**.

Listes chaînées

Une suite de **maillons** `(valeur, suivante)` ; le dernier pointe sur `None`. Pas d'accès direct : on **suit les liens**.

```
class Cellule:
    def __init__(self, v, s):
        self.valeur = v       # l'element
        self.suivante = s     # la cellule suivante, ou None a la fin

def longueur(lst):
    n = 0
    c = lst
    while c is not None:
        n = n + 1
        c = c.suivante        # on avance d'une cellule
    return n
```

Faux ami : la `list` *de Python est un **tableau dynamique**, pas une liste chaînée.*

Coût — le piège de la file

$O(1)$: coût constant ; $O(n)$: coût proportionnel à la taille n (§ IV). Défiler avec `liste.pop(0)` décale tout le tableau : $\mathbf{O(n)}$. Une vraie file efficace se fait avec une **liste chaînée** ou **deux piles**.

L'interface d'une liste — et le choix d'une structure

`liste_vide()`, `est_vide(L)`, `ajouter_tete(x, L)` (renvoie une nouvelle liste), `tete(L)`, `queue(L)` ; avec `Cellule` : `None`, `L is None`, `Cellule(x, L)`, `L.valeur`, `L.suivante` (§ V.C). Chercher : `x in liste` en $\mathbf{O(n)}$, `cle in dico` en $\mathbf{O(1)}$ — mais `v in dico.values()` en $O(n)$ (§ VI).

Erreurs à éviter

- confondre **LIFO** (pile) et **FIFO** (file) ;
- dépiler / défiler sans tester `est_vide` ;
- oublier que `pop(0)` pour défiler coûte O(n) : correct (c'est la `File` simple du cours), mais à éviter pour une **grande** file (liste chaînée, deux piles) ;
- perdre un maillon en oubliant de recoller `suivante`.

Savoir-faire à maîtriser — je sais…

- ☐ distinguer **pile (LIFO)** et **file (FIFO)** et en donner des usages concrets ;
- ☐ utiliser l'**interface** d'une pile (`empiler / depiler / est_vide / sommet`) et d'une file (`enfiler / defiler / est_vide / tete`) ;
- ☐ **dérouler** l'état d'une pile ou d'une file après une suite d'opérations ;
- ☐ appliquer une pile à un problème (**parenthésage**, évaluation, *annuler*) ;
- ☐ écrire une fonction qui n'utilise **que l'interface** et rend la structure **intacte** (taille, copie) ;
- ☐ parcourir une **liste chaînée** (longueur, occurrences, n-ième élément, copie) ;
- ☐ raisonner sur le **coût** des opérations (dont `pop(0)` en O(n)) ;
- ☐ distinguer **interface** et **implémentation** ;
- ☐ utiliser l'**interface d'une liste** (`ajouter_tete / tete / queue / est_vide`) et l'implémenter avec `Cellule` ;
- ☐ distinguer la **recherche** d'une valeur dans une **liste** et dans un **dictionnaire**, et choisir la structure adaptée.

Se tester — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 810

1. Que signifient LIFO et FIFO ? Laquelle décrit une file ?
2. Pile vide : `empiler(1)`, `empiler(2)`, `empiler(3)`, `depiler()`. Que renvoie alors `sommet()` ?
3. Quel est le coût de `defiler` codé par `liste.pop(0)` sur une file de n éléments ?
4. Une boucle `while c is not None` ne s'arrête jamais. Quelle ligne a-t-on oubliée ?
5. Quelle structure pour le bouton « page précédente » d'un navigateur ?

Et avant ? — *questions sur les chapitres précédents.* ▶ réponses p. 810

1. Pourquoi l'attribut `_contenu` de la classe `Pile` commence-t-il par un tiret bas ? (ch. 2)
2. `compte(n)` appelle `compte(n - 1)` tant que `n > 0`. Combien d'appels sont empilés au plus pour `compte(3)` ? (ch. 1)
3. Quel est, dans le pire cas, le coût de la recherche d'une valeur dans un tableau non trié de n éléments ? (Première)

Vers le bac

- Problème de synthèse n° 1 — Dérécursiver avec une pile, p. 706
- Problème de synthèse n° 2 — Annuler et rétablir, p. 707
- Problème de synthèse n° 3 — Listes chaînées récursives, p. 707
- Épreuve pratique : sujet 27 — Calculatrice en notation polonaise inverse, p. 733

DÉFIS ADVENT OF CODE

DÉFI 1 Operation Order

les devoirs de maths — piles et analyse d'expressions

Jour 18

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t03-defi-aoc-2020-18` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/18`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Le fichier contient plusieurs centaines de calculs avec des additions, des multiplications et des parenthèses imbriquées, mais les règles de priorité ne sont pas les règles habituelles. On veut la somme de tous les résultats. Python ne peut pas faire le calcul à notre place (il applique ses propres priorités) : il faut écrire soi-même un programme qui **évalue** une expression.

Ce qu'on réinvestit. Les **piles** vues dans ce chapitre. Comme pour la vérification d'un parenthésage, une pile permet de traiter les parenthèses imbriquées : à chaque parenthèse ouvrante, on met de côté le calcul en cours (on l'empile) pour calculer l'intérieur, et à la parenthèse fermante on le reprend (on le dépile). La récursivité, qui utilise en coulisse la pile des appels, offre une autre solution.

La démarche de la fiche.

1. Comprendre la règle de calcul sur des questions courtes, puis calculer un petit exemple

à la main.
2. Découper chaque ligne en une liste de symboles (nombres, opérateurs, parenthèses).
3. Évaluer une liste de symboles avec la règle de la partie 1 (pile ou récursivité) : la somme sur le fichier est la réponse.
4. Adapter le programme à la nouvelle priorité de la partie 2.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`homework`	devoirs à la maison	`expression`	expression (calcul)
`addition / multiplication`	addition / multiplication	`parentheses`	parenthèses
`evaluate`	calculer, évaluer	`precedence`	priorité (des opérations)
`left-to-right`	de gauche à droite	`regardless of`	quel que soit
`resulting value`	valeur obtenue	`sum`	somme

Résumé de l'énoncé (partie 1)

Contexte. En 2020, les elfes vous ont laissé partir en vacances sous les tropiques. Pendant un vol, un enfant assis à côté de vous vous demande de l'aide pour ses devoirs de calcul... mais dans son école, les règles de priorité ne sont pas celles que vous connaissez. Il faut donc refaire tous ses calculs avec les règles de son école.
Les données. Chaque ligne du fichier est une expression du cahier. Elle contient des entiers positifs (d'un seul chiffre dans les données), les opérateurs `+` (addition) et `*` (multiplication), et des parenthèses, éventuellement imbriquées. Les nombres et les opérateurs sont séparés par des espaces ; une parenthèse est collée au nombre qui la suit ou la précède, comme dans `(2 + 4)`.
La règle.

- Les parenthèses gardent leur rôle : leur contenu se calcule d'abord, en commençant par les plus intérieures.
- En revanche, `*` n'est **pas** prioritaire sur `+` : les deux opérateurs ont la même priorité.
- En dehors des parenthèses, on effectue donc les opérations dans l'ordre où on les lit, de gauche à droite. Autrement dit, `2 + 3 * 4` se calcule comme `(2 + 3) * 4`.

La réponse attendue. La somme des valeurs de toutes les lignes. Par exemple, un fichier de deux lignes `1 + 2 * 3` (qui vaut $3 \times 3 = 9$) et `2 * (1 + 1)` (qui vaut 4) donnerait 13.

Exercice 1 *Vérifier sa compréhension*

But. S'entraîner à calculer avec la règle « de gauche à droite », parenthèses comprises.
À quoi ça sert ? On doit désapprendre un réflexe (la multiplication d'abord) : mieux vaut le faire sur des calculs courts avant de programmer. La dernière question vérifie qu'on n'aura pas de souci de taille d'entiers.

1. Que valent `2 * 3 + 4` et `2 + 3 * 4` avec cette règle ?
2. Que vaut `2 * (3 + 4) * 2` ? Et `((1 + 2))` ?
3. La somme finale dépasse largement 10^9 : est-ce un problème en Python ?

Exercice 2 *À la main, sur un petit exemple*

But. Calculer à la main trois expressions avec parenthèses.
À quoi ça sert ? On obtient la valeur attendue de chaque ligne (et leur somme, 106) : ce seront les tests du programme. On voit aussi pourquoi `eval` ne convient pas.

On considère le fichier (inventé) suivant :

```
3 * (2 + 4) + 5
2 + 3 * 4 + (1 * 6)
(4 + 5 * (2 + 1)) * 2 + 3
```

Calculer chaque ligne avec la règle de l'énoncé, en écrivant les étapes comme sur le site. Vérification : les trois valeurs ont pour somme `106`. Quelle valeur Python donne-t-il pour la deuxième ligne avec `eval` ? Pourquoi ne peut-on donc pas utiliser `eval` directement ?

2 Programmer la partie 1

Méthode : reconnaître un chiffre, ignorer un espace

Un caractère `c` est un chiffre exactement quand `c in "0123456789"` vaut `True`. Pour ignorer les espaces, il suffit de ne rien faire quand `c == " "` : on construit une nouvelle chaîne en n'y ajoutant que les autres caractères.

```
print("7" in "0123456789")   # True
print("+" in "0123456789")   # False
s = "3 * (2 + 4)"
sans_espaces = ""
for c in s:
    if c != " ":
        sans_espaces += c
print(sans_espaces)          # 3*(2+4)
print(s)                     # s est inchange : 3 * (2 + 4)
```

Intérêt. Ces deux tests suffisent pour découper une ligne caractère par caractère. Question : recopier et exécuter ce code ; que valent `"12" in "0123456789"` et `"13" in "0123456789"` ? Pourquoi faut-il tester les caractères **un par un** ?

Exercice 3 *Découper une ligne en symboles*

But. Transformer une ligne de texte en une liste de symboles exploitables : entiers, opérateurs et parenthèses.
À quoi ça sert ? Le calcul est bien plus simple sur une liste comme `[3, '*', '(', 2, '+', 4, ')']` que sur une chaîne où il faut se soucier des espaces et des parenthèses collées. Les deux parties utiliseront ce même découpage.

Écrire une fonction `decouper(ligne)` qui renvoie la liste des *symboles* de la ligne : les nombres (convertis en `int`), `'+'`, `'*'`, `'('` et `')'`. Par exemple `decouper("3 * (2 + 4)")` doit renvoyer `[3, '*', '(', 2, '+', 4, ')']`. ▷ coup de pouce

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Exercice 4 *Évaluer sans priorité*

But. Calculer la valeur d'une liste de symboles avec la règle de la partie 1.
À quoi ça sert ? C'est le cœur du défi : appliquée à chaque ligne du fichier, cette fonction donne les valeurs dont la somme est la réponse. La vraie difficulté est de suspendre le calcul en cours à chaque parenthèse ouvrante, ce que permettent une pile ou un appel récursif.

Écrire une fonction `evaluer(symboles)` qui renvoie la valeur de l'expression avec la règle de la partie 1. Tester sur les trois lignes de l'exemple, puis calculer la somme sur `input.txt`. ▷ coup de pouce

▷ début de solution

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Vous aviez mal compris : dans cette école, il existe bien une priorité entre les opérations, mais elle est à l'envers de la vôtre.
La nouvelle règle.

- Les parenthèses passent toujours en premier.
- Ensuite, l'**addition est prioritaire** sur la multiplication : on effectue toutes les additions avant les multiplications.
- À priorité égale, on calcule de gauche à droite.
- Ainsi `2 * 3 + 4` vaut maintenant $2 \times 7 = 14$, alors qu'il valait 10 en partie 1.

La réponse attendue. La somme des valeurs de toutes les lignes, calculées avec cette nouvelle règle.

Exercice 5 *Changer la priorité*

But. Adapter l'évaluation pour que `+` soit calculé avant `*`.
À quoi ça sert ? Le découpage et le traitement des parenthèses restent valables ; il suffit de changer la façon de combiner les opérations. Une méthode bien choisie (gare de triage avec une table de priorités) résout même les deux parties avec le même code.

Seule la priorité entre `+` et `*` change. Adapter le programme. Sur l'exemple inventé de la fiche, on doit trouver les valeurs `33`, `50` et `135`, de somme `218`. ▷ coup de pouce ▷ début de solution

4 Approfondissement

★★ Exercice 6 *Approfondissement 1 : la descente récursive*

But. Évaluer les expressions de la partie 2 avec trois fonctions qui s'appellent mutuellement.
À quoi ça sert ? Décrire la forme des expressions par des définitions récursives, puis écrire une fonction par définition, est la méthode utilisée par les interpréteurs de langages, Python compris ; elle traite naturellement les parenthèses et les priorités.

En partie 2, une expression est un **produit de sommes** : `2 + 3 * 4 + (1 * 6)` se lit $(2+3) \times (4 + (1 \times 6))$. On peut décrire toutes les expressions par trois définitions qui s'appellent mutuellement :

- un *produit* est une somme, éventuellement suivie de `* somme`, `* somme`...
- une *somme* est un facteur, éventuellement suivi de `+ facteur`, `+ facteur`...
- un *facteur* est un nombre, ou bien un produit entre parenthèses.

1. À la main : découper la deuxième ligne de l'exemple en sommes, puis chaque somme en facteurs. Faire de même pour la troisième ligne. Où intervient la définition récursive du facteur ?

2. Écrire trois fonctions `facteur(symboles, i)`, `somme(symboles, i)` et `produit(symboles, i)` qui lisent la liste à partir de l'indice `i` et renvoient le couple `(valeur, indice du premier symbole non lu)`. Vérifier qu'on retrouve 33, 50 et 135. ▷ coup de pouce ▷ début de solution

3. Quelle est la complexité de cette méthode ? Comment l'adapter à la partie 1 (une seule priorité) ?

★★★ Exercice 7 *Approfondissement 2 : la notation postfixe*

But. Évaluer une expression écrite en notation postfixe, puis traduire une expression ordinaire dans cette notation.
À quoi ça sert ? Sans parenthèses ni priorités, l'évaluation n'utilise plus qu'une seule pile ; on sépare ainsi le problème en deux étapes simples (traduire, puis calculer), comme le font certaines calculatrices et machines virtuelles.

En **notation postfixe** (ou « polonaise inverse »), on écrit chaque opérateur *après* ses deux opérandes, et l'on n'a plus besoin de parenthèses : `(3 + 4) * 2` s'écrit `3 4 + 2 *`. Certaines calculatrices fonctionnent ainsi.

1. Évaluer à la main `3 4 + 2 *` en utilisant une pile : on lit de gauche à droite ; un nombre est empilé ; un opérateur dépile deux valeurs et empile le résultat. Dessiner la pile après chaque symbole.

2. Écrire `evaluer_postfixe(expression)` qui évalue une liste comme `[3, 4, '+', 2, '*']`. ▷ coup de pouce

3. L'algorithme de la gare de triage peut, au lieu de calculer, **écrire** l'expression en notation postfixe : un nombre va directement dans la liste de sortie, un opérateur dépilé y est ajouté au lieu d'être appliqué. Écrire `postfixe(symboles, priorite)`. Écrire à la main la notation postfixe de la deuxième ligne de l'exemple pour chacune des deux parties : qu'est-ce qui change ? ▷ coup de pouce ▷ début de solution

4. Combien de piles au total pour calculer une expression de cette façon ? Quelle est la complexité ?

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Parcourir la chaîne caractère par caractère, sans rien faire pour un espace, en accumulant les chiffres consécutifs dans une chaîne `nombre` : un nombre est terminé dès qu'on rencontre autre chose qu'un chiffre. ↑ énoncé

Ex. 4 – piste Deux méthodes possibles. **Avec une pile** : quand on rencontre `'('`, on empile le résultat partiel et l'opérateur en attente, puis on repart de zéro ; quand on rencontre `')'`, on dépile et on combine. **Récursivement** : une fonction lit l'expression de gauche à droite et s'appelle elle-même dès qu'elle rencontre une parenthèse ouvrante ; elle renvoie la valeur *et* l'indice où elle s'est arrêtée. ↑ énoncé

Ex. 4 – début de solution Version récursive : `evaluer_depuis(symboles, i)` garde une variable `total` et un opérateur courant `op` (au départ `'+'` et `total = 0`). Pour chaque symbole : un nombre, ou une sous-expression obtenue par appel récursif après `'('`, est combiné à `total` avec `op` ; `'+'` ou `'*'` met à jour `op` ; `')'` ou la fin de la liste arrête la boucle et l'on renvoie `(total, i)`. ↑

Ex. 5 – piste Avec la méthode récursive, il suffit de repérer qu'une expression devient un *produit de sommes* : une fonction calcule une somme (de nombres ou de parenthèses), une autre fait le produit de ces sommes. ↑ énoncé

Ex. 5 – début de solution Algorithme de la gare de triage (Dijkstra) : une pile d'opérateurs et une pile de valeurs. On donne à chaque opérateur une priorité (un dictionnaire `{'+': 2, '*': 1}`). Avant d'empiler un opérateur, on applique tous ceux du sommet de pile de priorité supérieure ou égale ; une `')'` applique les opérateurs jusqu'à la `'('` correspondante. Avec `{'+': 1, '*': 1}`, le même code résout la partie 1. ↑énoncé

Ex. 6 q. 2 – piste Chaque fonction suit sa définition : `somme` appelle `facteur`, puis, tant que le symbole courant est `'+'`, saute ce symbole et appelle à nouveau `facteur`. `produit` fait de même avec `somme` et `'*'`. ↑énoncé

Ex. 6 q. 2 – début de solution `facteur` : si `symboles[i]` vaut `'('`, appeler `produit(symboles, i + 1)`, qui renvoie `(v, j)` avec `symboles[j] == ')'` ; renvoyer alors `(v, j + 1)`. Sinon renvoyer `(symboles[i], i + 1)`. ↑énoncé

Ex. 7 q. 2 – piste Une seule pile (une liste avec `append` et `pop`) ; un symbole est un opérateur s'il vaut `'+'` ou `'*'`, sinon c'est un nombre. Attention à l'ordre : le premier élément dépilé est l'opérande de **droite**. ↑énoncé

Ex. 7 q. 3 – piste Reprendre le code de la gare de triage : là où l'on appliquait un opérateur dépilé, on l'ajoute à la fin de la liste `sortie`. ↑énoncé

Ex. 7 q. 3 – début de solution Partie 1 : `2 3 + 4 * 1 6 * +`. Il reste à trouver celle de la partie 2 et à vérifier que `evaluer_postfixe(postfixe(...))` redonne bien `50`. ↑

DÉFI 2 Crab Cups

les gobelets du crabe — listes chaînées

Jour 23

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t03-defi-aoc-2020-23` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/23`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Des gobelets numérotés sont disposés en cercle ; à chaque coup, trois gobelets sont déplacés selon une règle précise. On veut connaître l'ordre des gobelets après un grand nombre de coups. En partie 1 (9 gobelets, 100 coups), une simulation directe suffit ; en partie 2 (un million de gobelets, dix millions de coups), il faut une structure de données où chaque coup coûte un temps constant.

Ce qu'on réinvestit. Les **listes chaînées** de ce chapitre : un cercle de gobelets est une liste chaînée circulaire, où chaque gobelet connaît seulement celui qui le suit. Déplacer trois gobelets revient alors à changer quelques liens, sans décaler les autres. On verra qu'un simple tableau suffit à représenter cette liste chaînée, et l'on comparera les **complexités** des deux méthodes.

La démarche de la fiche.

1. Comprendre la règle d'un coup (en particulier le choix de la destination) et jouer quelques coups à la main.
2. Simuler le jeu avec une liste Python : cela suffit pour la partie 1.
3. Évaluer le coût de cette méthode et constater qu'elle est bien trop lente pour la partie 2.

4. Réécrire la simulation avec un tableau des successeurs (une liste chaînée), puis mesurer le gain.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`cup`	gobelet	`label`	étiquette, numéro
`clockwise`	dans le sens des aiguilles d'une montre	`circle`	cercle
`current cup`	gobelet courant	`pick up`	ramasser, retirer
`destination cup`	gobelet de destination	`move`	coup
`wrap around`	repartir de l'autre bout	`immediately`	juste (après)

Résumé de l'énoncé (partie 1)

Contexte. Le crabe vous défie maintenant à un jeu de bonneteau : il déplace des gobelets numérotés disposés en cercle, et vous devez prédire où ils seront à la fin. Comme ses déplacements suivent une règle fixe, on peut les simuler.
Les données. Le fichier est une simple suite de 9 chiffres, de 1 à 9, chacun une fois : les étiquettes des gobelets dans l'ordre du cercle, dans le sens horaire. Après le dernier chiffre, on revient au premier. Le premier gobelet de la suite est le gobelet « courant » au début du jeu.
La règle. Un coup se déroule ainsi :

- Retirer du cercle les 3 gobelets qui suivent le gobelet courant (dans le sens horaire).
- Choisir la destination : c'est l'étiquette du courant moins 1 ; si cette étiquette fait partie des gobelets retirés, on enlève encore 1, et ainsi de suite. Si l'on passe sous la plus petite étiquette, on repart de la plus grande.
- Replacer les 3 gobelets retirés, dans le même ordre, juste après la destination.
- Le nouveau gobelet courant est celui qui suit alors l'ancien courant.

Autrement dit, le crabe déplace un paquet de trois gobelets vers le plus grand numéro inférieur disponible.
La réponse attendue. Après 100 coups, les étiquettes lues dans le sens horaire à partir du gobelet 1, sans lui, écrites en une seule chaîne. Par exemple, si le cercle lu à partir du 1 était alors 1, 4, 7, 2, 9, 3, 5, 8, 6, la réponse serait `47293586`.

Exercice 1 *Vérifier sa compréhension*

But. Maîtriser le choix de la destination et l'ordre des étapes d'un coup.
À quoi ça sert ? Le choix de la destination, avec ses sauts et son retour à la plus grande étiquette, est la partie de la règle la plus souvent mal programmée ; ces questions en traitent les cas limites.

1. Avec 9 gobelets, le courant porte 2 et les gobelets retirés sont 1, 9 et 8 : quelle est la destination ?
2. Le gobelet courant peut-il être la destination ? Un gobelet retiré ?
3. Le nouveau courant se choisit-il avant ou après avoir replacé les trois gobelets ? Cela change-t-il quelque chose ?

Exercice 2 *À la main, sur un petit exemple*

But. Jouer trois coups à la main sur un cercle de 9 gobelets.
À quoi ça sert ? On vérifie qu'on applique correctement la règle, et l'état connu après le premier coup servira à tester la fonction `jouer` coup par coup.

On part de l'étiquetage (inventé) `527164839`. Jouer le premier coup à la main. Vérification : en lisant le cercle à partir du `5`, on obtient `5 6 4 2 7 1 8 3 9`. Quel est le nouveau gobelet courant ? Jouer encore deux coups.

2 Programmer la partie 1

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Méthode : chercher un élément dans une liste

Pour trouver la position d'un gobelet, on parcourt la liste avec un indice jusqu'à le rencontrer. Une boucle `while` s'arrête dès qu'il est trouvé, sans `break`.

```
def indice(liste, x):
    """Indice de la premiere occurrence de x, ou -1 si x est absent."""
    i = 0
    while i < len(liste) and liste[i] != x:
        i += 1
    if i == len(liste):
        return -1
    return i

g = [5, 6, 4, 2, 7, 1, 8, 3, 9]
print(indice(g, 1))        # 5
k = indice(g, 4)
g = g[:k + 1] + [0] + g[k + 1:]    # insertion juste apres le 4
print(g)                   # [5, 6, 4, 0, 2, 7, 1, 8, 3, 9]
```

La plus grande étiquette, elle, n'a pas besoin d'être cherchée : les étiquettes sont exactement les entiers de 1 à n, elle vaut donc `len(g)`.
Intérêt et limite. La recherche parcourt la liste : elle coûte jusqu'à n comparaisons. Question : que renvoie `indice(g, 10)` ? Pourquoi faut-il écrire le test `i < len(liste)` *avant* le test `liste[i] != x` ?

Exercice 3 *Une première simulation avec une liste*

But. Simuler le jeu avec une liste Python et écrire la réponse sous forme de chaîne.
À quoi ça sert ? C'est la réponse de la partie 1. Cette version simple servira ensuite de référence pour vérifier la version rapide de la partie 2.

Écrire une fonction `jouer(gobelets, nb_coups)` qui simule le jeu sur une liste Python d'entiers et renvoie la liste des gobelets après `nb_coups` coups, puis une fonction `reponse(gobelets)` qui renvoie la chaîne des étiquettes lues après le gobelet 1. Sur l'exemple inventé de la fiche, on doit obtenir `53269478` après 10 coups et `98357642` après 100 coups. ▷ coup de pouce

Exercice 4 *Combien ça coûte ?*

But. Estimer la complexité d'un coup avec une liste Python, puis le temps nécessaire pour la partie 2.
À quoi ça sert ? Avant d'écrire du code, on prévoit si la méthode est utilisable : ce calcul montre qu'il faut changer de structure de données, et indique quelles opérations il faut rendre plus rapides.

Quelle est la complexité d'un coup avec cette méthode, en fonction du nombre n de gobelets ? Lire la partie 2 (une fois débloquée) : combien d'opérations élémentaires faudrait-il environ ?
▷ coup de pouce

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Le crabe sort beaucoup plus de gobelets et joue beaucoup plus longtemps : un million de gobelets et dix millions de coups.
La nouvelle règle.

- Après vos 9 chiffres, le cercle est complété par les étiquettes $10, 11, \ldots$ jusqu'à 1 000 000, dans l'ordre ; après le gobelet 1 000 000, on revient au premier de vos chiffres.
- Le gobelet courant de départ est toujours le premier de vos chiffres.
- On joue 10 000 000 coups avec exactement les mêmes règles (la plus grande étiquette vaut maintenant 1 000 000).

La réponse attendue. Le produit des deux étiquettes qui suivent le gobelet 1. Par exemple, si ces deux gobelets portaient les étiquettes 12 et 1000, la réponse serait 12000.

★★ Exercice 5 *Une liste chaînée dans un tableau*

But. Représenter le cercle par le tableau des successeurs, où chaque coup ne modifie que trois cases.
À quoi ça sert ? Un coup devient une opération en temps constant : dix millions de coups deviennent faisables en quelques secondes, ce qui donne la réponse de la partie 2. Vérifier d'abord les résultats de la partie 1 garantit que la nouvelle version applique bien la même règle.

Réécrire la simulation avec un **tableau des successeurs** (lire l'encadré), vérifier qu'elle redonne les résultats de la partie 1, puis répondre à la partie 2. Sur l'exemple inventé de la fiche, on doit trouver `28997820592`. Le calcul peut prendre une dizaine de secondes. ▷ coup de pouce ▷ début de solution

Le tableau des successeurs

Une **liste chaînée** n'a pas besoin d'objets `Cellule` (la classe du cours) : comme les étiquettes sont les entiers de 1 à n, on peut ranger dans une liste `suivant` de taille $n+1$ l'étiquette qui suit chaque gobelet dans le cercle : `suivant[x]` est le gobelet juste après `x` (la case `0` ne sert pas). Pour le cercle `3 1 2` : `suivant[3] = 1`, `suivant[1] = 2` et `suivant[2] = 3` (le dernier pointe vers le premier : la liste est *circulaire*).

Avantages. Trouver le gobelet d'étiquette `x` ne demande aucune recherche : c'est la case `x`. Retirer ou insérer un morceau de la chaîne revient à modifier quelques cases, sans décaler les autres éléments. Un coup se fait donc en temps constant, $O(1)$, quel que soit n.

Pour lire le cercle à partir du gobelet `1`, on suit les flèches : `x = suivant[1]`, puis `x = suivant[x]`, jusqu'à revenir à `1`.

Outil Python : mesurer un temps (`time.perf_counter`)

`perf_counter()` renvoie un temps en secondes, très précis ; la différence entre deux appels donne une durée. On s'en sert ici uniquement pour **comparer** des programmes et vérifier une complexité.

```
1 from time import perf_counter
2 debut = perf_counter()
3 total = 0
4 for k in range(1_000_000):
5     total += k
6 print(f"{perf_counter() - debut:.3f} s")    # duree de la boucle
```

Question : mesurer la durée de la boucle ci-dessus avec 10^6 puis 10^7 tours. Le temps est-il proportionnel au nombre de tours ?

★★★ Exercice 6 *Défi : aller plus vite*

But. Mesurer les temps des deux versions et les comparer aux complexités prévues.
À quoi ça sert ? Confronter une prédiction théorique à des mesures est la seule façon de savoir si une analyse de complexité est juste ; on apprend aussi quelques réglages qui accélèrent une boucle très longue.

Avec `perf_counter`, comparer les deux versions (liste et tableau des successeurs) : jouer 100 coups avec 10^4, puis 10^5 gobelets étiquetés dans l'ordre. Les mesures confirment-elles les complexités trouvées plus haut ? Peut-on ensuite réduire le temps de la version chaînée en évitant de recalculer des valeurs dans la boucle ? ▷ coup de pouce

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Le plus simple est de garder le gobelet courant toujours en tête de liste : après chaque coup, faire « tourner » la liste d'un cran (le premier élément passe à la fin). Les trois gobelets ramassés sont alors aux indices 1, 2 et 3. ↑ énoncé

Ex. 4 – piste Chercher le gobelet de destination (parcours de la liste), retirer et insérer des éléments dans une liste Python : chacune de ces opérations est en $O(n)$. ↑ énoncé

Ex. 5 – piste Un coup ne modifie que **trois** cases du tableau `suivant`. Les écrire sur un dessin avant de coder : le successeur du courant, celui du dernier gobelet ramassé, celui de la destination. ↑ énoncé

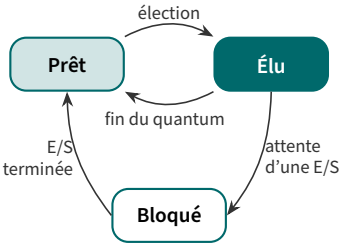
Ex. 5 – début de solution Avec `a = suivant[courant]`, `b = suivant[a]`, `c = suivant[b]` : le courant doit maintenant pointer vers `suivant[c]` ; `c` doit pointer vers l'ancien successeur de la destination ; la destination doit pointer vers `a`. Attention à l'ordre des affectations. ↑ énoncé

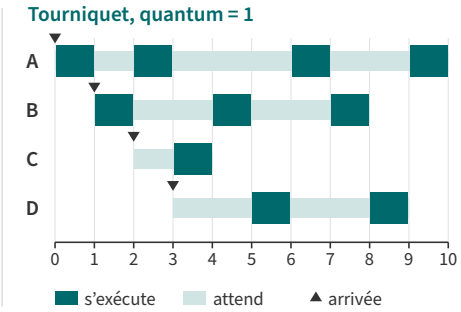
Ex. 6 – piste Dans une boucle de dix millions de tours, chaque accès à une variable globale ou chaque appel de fonction compte : placer toute la boucle dans une fonction et utiliser des variables locales. ↑énoncé

4

CHAPITRE 4

Processus et ordonnancement





DANS CE CHAPITRE

- Activités préparatoires . . . 177
- Cours . . . 179
- Exercices . . . 191
- TP et projets . . . 206
- Fiche de révision . . . 213
- Défis Advent of Code . . . 215
- Corrigés . . . 811

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 L'ordonnanceur, c'est vous

▶ corrigé p. 811

partager un seul processeur entre plusieurs tâches

Durée : 20 min · Sans ordinateur, par groupes de trois

Consignes

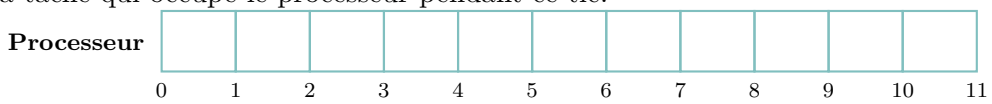
- Rôles : un **processeur** (il ne peut tenir qu'**une seule carte** à la fois, et avance d'une unité de travail par « tic ») ; un **ordonnanceur** (il décide quelle carte donner au processeur) ; un **gardien du temps** (il annonce les tics $0, 1, 2\ldots$ et remplit les frises). On change de rôle à chaque partie.
- Une tâche n'existe qu'à partir de son instant d'**arrivée** ; elle est finie quand elle a reçu autant de tics que sa **durée**.

A	B	C	D
encoder une vidéo	ouvrir un message	clic de souris	analyse antivirus
arrivée : **0**	arrivée : **1**	arrivée : **2**	arrivée : **3**
durée : **5**	durée : **2**	durée : **1**	durée : **3**

Exercice 1 *Première règle : chacun son tour, jusqu'au bout*

L'ordonnanceur applique la règle « **premier arrivé, premier servi** » : il donne le processeur à la tâche arrivée le plus tôt et la laisse jusqu'à ce qu'elle soit **finie**.

1. Jouer la partie. Recopier la frise ci-dessous sur le cahier et écrire dans chaque case la lettre de la tâche qui occupe le processeur pendant ce tic.

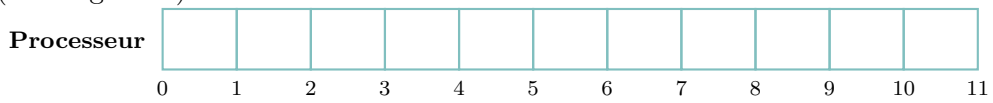


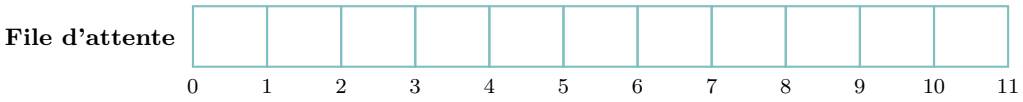
2. À quel instant chacune des tâches A, B, C et D se termine-t-elle ?
3. Le **temps d'attente** d'une tâche est le nombre de tics pendant lesquels elle était arrivée mais n'avait pas le processeur. Combien de tics le clic de souris C a-t-il attendu, pour une durée de 1 ?
4. Du point de vue de l'utilisateur qui vient de cliquer, que s'est-il passé à l'écran ?

Exercice 2 *Deuxième règle : un tic chacun, puis au bout de la file*

Nouvelle règle, le **tourniquet** : les tâches arrivées font la **queue** (une file). L'ordonnanceur donne le processeur à la tâche en tête de file pour **un seul tic** ; si elle n'est pas finie, elle repart **au bout de la file**. Si une tâche arrive au même instant, elle se place dans la file *avant* celle qui revient.

5. Jouer la partie. Recopier les deux frises sur le cahier : sur la première, noter la tâche qui occupe le processeur ; sur la seconde, noter dans chaque case le contenu de la file *à la fin* du tic (tête à gauche).





6. Combien de tics le clic de souris C attend-il maintenant ? Et B ?
7. Quelle tâche y perd ? Pourquoi est-ce acceptable pour elle ?
8. Avec cette règle, l'ordonnanceur intervient beaucoup plus souvent. Quel inconvénient cela peut-il avoir dans une vraie machine ?

Exercice 3 *Deux tâches, deux ressources*

Deux nouvelles cartes : **P** (« imprimer un scan ») et **Q** (« numériser puis imprimer »). Sur la table, deux objets uniques : l'**imprimante** et le **scanner** (une gomme et un taille-crayon font l'affaire). Un objet pris n'est rendu que quand la tâche est finie. Deux élèves du groupe jouent chacun une tâche (l'un P, l'autre Q) : ils exécutent leur script **une ligne chacun, en alternance**, en commençant par P ; le troisième observe et note qui tient quoi.

Script de P	Script de Q
1. prendre l'imprimante	1. prendre le scanner
2. prendre le scanner	2. prendre l'imprimante
3. travailler	3. travailler
4. rendre les deux objets	4. rendre les deux objets

9. Jouer la scène. Que se passe-t-il à la ligne 2 de P, puis à la ligne 2 de Q ?
10. Faire sur le cahier un petit schéma : qui tient quel objet, qui attend quel objet (une flèche par relation).
11. La situation peut-elle se débloquer toute seule ? Pourquoi ?
12. Proposer une modification d'**un seul** des deux scripts qui rend ce blocage impossible.

Exercice 4 *Ce qu'on a découvert*

13. Recopier et compléter sur le cahier, avec vos mots.
    - Avec un seul processeur, il faut un programme qui…
    - Dans les exercices 1 et 2, une tâche pouvait attendre **le processeur**. Dans l'exercice 3, elle attendait…
      En quoi ces deux attentes sont-elles différentes ?
    - Quand chacun tient ce que l'autre attend,…

COURS

Programme (BO) *« Gestion des processus et des ressources par un système d'exploitation. Décrire la création d'un processus, l'ordonnancement de plusieurs processus par le système. Mettre en évidence le risque de l'interblocage (*deadlock*). »* Commentaire : *« À l'aide d'outils standard, il s'agit d'observer les processus actifs ou en attente sur une machine. Une présentation débranchée de l'interblocage peut être proposée. »*

Prérequis la notion de **programme** et de **langage machine** (Première), l'**arborescence** des fichiers (Première), pour la relation père/fils entre processus, un peu de **files** (la file d'attente de l'ordonnanceur). → fiches de révision : ch. 3 p. 162 → rappels de 1re : architecture et OS p. 1065

Objectifs *distinguer* un programme d'un processus ; *décrire* les trois états d'un processus et les transitions entre eux ; *comprendre* le rôle d'ordonnanceur du système ; *dérouler* un ordonnancement et le représenter par un chronogramme ; *reconnaître* et *expliquer* une situation d'interblocage.

Situation de départ

1. Votre ordinateur semble faire tourner des dizaines de programmes à la fois, avec un seul cœur de processeur. Comment est-ce possible ?
2. Quand plusieurs programmes réclament le processeur en même temps, lequel doit passer en premier : le premier arrivé, le plus court, ou chacun un peu, à tour de rôle ?
3. Comment deux programmes peuvent-ils rester figés pour toujours, chacun attendant l'autre ?

I Le problème : un seul processeur, mille programmes

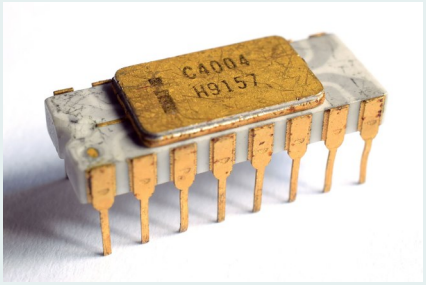
En cet instant précis, votre ordinateur « fait tourner » des dizaines de programmes : le navigateur, la messagerie, l'horloge, l'antivirus, le système lui-même… Pourtant, un cœur de processeur ne sait faire qu'**une seule chose à la fois**. Comment l'illusion tient-elle ?

La réponse tient en un mot : la **vitesse**. Le système donne le processeur à un programme pour une fraction de milliseconde, le retire, le donne à un autre, et recommence — des milliers de fois par seconde. L'œil humain n'y voit que du feu : tout *semble* simultané. Ce tour de passe-passe a un chef d'orchestre : le **système d'exploitation** (*operating system*, OS).

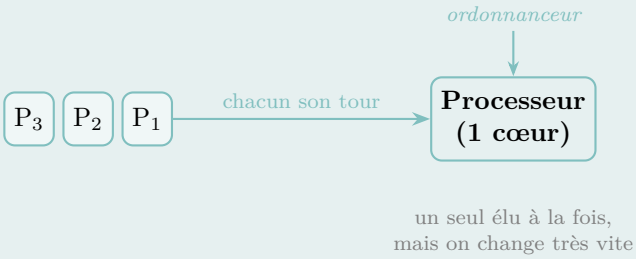
Règle 1 — Le fil rouge de ce chapitre

Le **système d'exploitation** est le programme qui gère la machine et fait tourner tous les autres. Son défi central : partager un **unique processeur** (et des ressources limitées) entre de **nombreux programmes** qui le réclament tous. Ce chapitre raconte comment il s'y prend — en **ordonnançant** les programmes (leur donner le processeur chacun son tour) — et ce qui arrive quand cela tourne mal — l'**interblocage**.

Parmi les grandes fonctions d'un OS : gérer les processus (notre sujet), la mémoire, les fichiers, les périphériques, les droits des utilisateurs, et la sécurité. Nous nous concentrons ici sur la **gestion des processus**.



La ressource à partager : **le processeur** (ici l'Intel 4004, 1971, l'un des tout premiers).



II Programme et processus : ne pas confondre

Définition 1 — Processus

Un **processus** est un **programme en cours d'exécution**. Il ne faut pas le confondre avec le programme lui-même : le programme est un *texte* inerte (le « code source » ou le fichier exécutable), rangé sur le disque ; le processus est ce texte *en train de vivre* dans la mémoire, avec ses données, sa position courante, ses ressources.

Exemple

Une **recette de cuisine** écrite sur une fiche, c'est le *programme*. Le **cuisinier en train de la réaliser** dans sa cuisine, c'est le *processus*. Deux cuisiniers peuvent suivre la *même* recette en même temps : un seul programme, mais **deux processus** distincts, chacun avec ses casseroles (ses données) et son avancement propre.

► **Exercices d'application :** exercice 1 (programme ou processus ?)

III La vie d'un processus : trois états

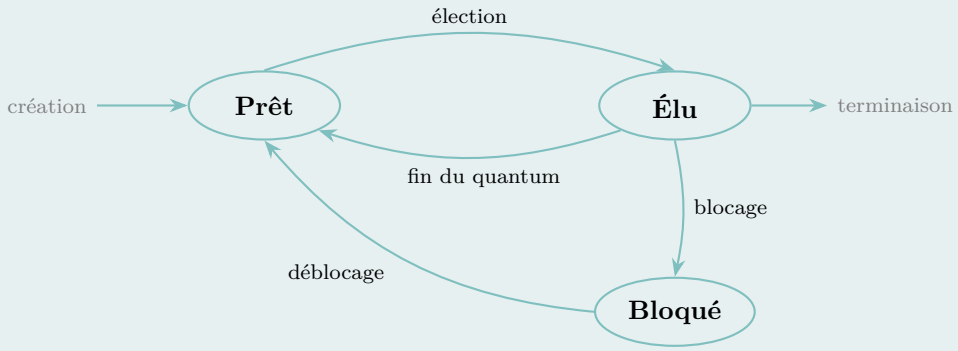
À un instant donné, un processus se trouve dans l'un de **trois états**. C'est le cœur du chapitre — apprenez ce schéma par cœur.

Définition 2 — Les trois états d'un processus

- **Élu** : le processus *utilise le processeur*, il s'exécute. Il n'y en a **qu'un seul** dans cet état à la fois (par cœur de processeur).
- **Prêt** : le processus *pourrait* s'exécuter, mais il attend son tour car le processeur est occupé par un autre.
- **Bloqué** : le processus attend une **ressource** (une donnée du disque, une saisie clavier, la fin d'un calcul…) ; il ne peut pas avancer, même si le processeur se libérait.

↻ **Retour sur l'activité.** Les deux attentes de la dernière question de l'activité sont ces deux états : une carte qui attendait *le processeur* était **prête** ; une tâche qui attendait l'imprimante ou le scanner était **bloquée**, et lui donner le processeur n'aurait servi à rien.

Les passages d'un état à l'autre portent des noms précis :



- **Élection** (prêt → élu) : le système choisit ce processus et lui donne le processeur.
- **Fin du quantum / réquisition** (élu → prêt) : son temps de parole est écoulé, le système lui reprend le processeur pour le donner à un autre.
- **Blocage** (élu → bloqué) : le processus demande une ressource indisponible et doit attendre.
- **Déblocage** (bloqué → prêt) : la ressource arrive ; le processus ne repart pas *directement* en exécution, il redevient « prêt » et attend son tour.

Remarque

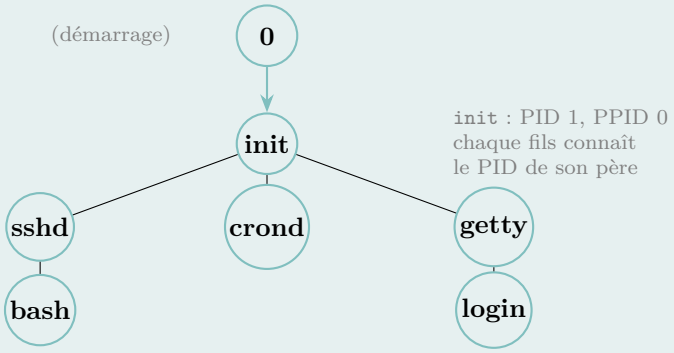
Deux transitions sont **impossibles** et constituent des pièges classiques : on ne passe **jamais** de « bloqué » directement à « élu » (on repasse toujours par « prêt »), et un processus ne peut se **terminer** ou **libérer une ressource** que depuis l'état « élu » (il faut le processeur pour exécuter la moindre instruction, y compris « je libère »).

► **Exercices d'application :** exercices 2 à 4 (les trois états ; l'automate)

IV Créer des processus : une histoire de famille

Un processus peut en créer un autre par un appel système (`fork` sous les systèmes de type Unix). Le créateur est le **père**, le créé est le **fils**. Comme chaque processus a un père (sauf le tout premier), la structure obtenue est un **arbre**, comme l'arborescence des fichiers vue en Première (les arbres seront étudiés en détail au chapitre suivant).

Chaque processus porte un identifiant unique, le **PID** (*Process IDentifier*), et connaît le PID de son père, le **PPID**. Au démarrage, le système crée un premier processus « venu de rien » (PID 0), qui engendre le processus `init` (PID 1), racine de tous les autres.



Lorsqu'un processus se termine (ou est tué) avant ses fils, ceux-ci ne disparaissent pas : chacun devient **orphelin** et est **adopté par `init`**, son PPID devient alors 1.

Remarque

L'appel `fork` **duplique** le processus qui l'exécute : après un `fork`, il y a *deux* processus (le père **et** le fils) qui poursuivent tous deux la suite du programme. Conséquence surprenante : deux `fork` à la suite donnent $2 \times 2 = 4$ processus, trois `fork` en donnent $2^3 = 8$… Chaque `fork` **double** le nombre de processus. Le code suivant affiche donc « bonjour » **quatre** fois :

```
1 from os import fork
2 fork()            # 1 processus -> 2
3 fork()            # 2 processus -> 4
4 print("bonjour")  # execute par les 4 processus
```

▷ Je prédis. Combien de « bonjour » s'afficheraient si l'on ajoutait un second `print("bonjour")` *entre* les deux `fork()` ?

▷ Je vérifie. **Six** : deux processus exécutent le `print` du milieu, puis quatre celui de la fin ($2 + 4$).

► Exercices d'application : exercices 5 à 7 (arbre de processus, PID/PPID, `fork`)

V Observer et gérer les processus : le terminal

Le système garde la trace de tous les processus. On peut les **observer** et **agir** sur eux avec quelques commandes tapées dans un **terminal** (sous Linux ou macOS). La commande reine est `ps` (*process status*) : avec les options `-ef`, elle liste *tous* les processus, un par ligne, avec leur **PID** et leur **PPID**.

```
$ ps -ef
UID     PID  PPID  STIME  TTY    CMD
root      1     0  09:14  ?      /sbin/init
mathis  843     1  09:15  tty1   -bash
mathis  901   843  09:20  tty1   python3 tri.py
mathis  902   843  09:21  tty1   ps -ef
```

En rapprochant les colonnes **PID** et **PPID**, on reconstitue l'arbre des processus : ici, `python3 tri.py` (PID 901) a pour père le terminal `-bash` (PID 843), lui-même fils de `init`. La commande `ps` apparaît dans sa propre sortie (PID 902) : au moment de l'affichage, elle est elle-même un processus en cours.

Commande	Rôle	Exemple
`ps`	les processus du terminal courant	`ps`
`ps -ef` *ou* `ps aux`	**tous** les processus (PID, PPID…)	`ps -ef`
`pstree`	les processus sous forme d'**arbre**	`pstree`
`top` *ou* `htop`	liste **rafraîchie en temps réel** (CPU, mémoire)	`top`
`kill <PID>`	demande l'arrêt d'un processus	`kill 901`
`kill -9 <PID>`	**force** l'arrêt (signal non ignorable)	`kill -9 901`
`<commande> &`	lance en **arrière-plan** (rend la main)	`python3 tri.py &`
`jobs`	les tâches lancées depuis ce terminal	`jobs`
`nice` / `renice`	règle la **priorité** d'un processus	`nice -n 10 gros_calcul`

Au bac — lire une sortie de `ps`

Ces noms ne sont pas à réciter par cœur, mais savoir **lire une sortie de `ps`** pour en tirer un arbre de PID, et connaître `top` et `kill`, sont des capacités attendues. Le « kill » de `kill -9` est trompeur : la commande *envoie un signal* au processus ; `-9` est le signal qu'on ne peut pas ignorer, d'où l'arrêt forcé. Ces commandes se pratiquent dans la partie A du TP proposé en fin de chapitre.

VI L'ordonnancement : le chef d'orchestre à l'œuvre

Quand plusieurs processus sont « prêts », lequel élire ? C'est la mission de l'**ordonnanceur** (*scheduler*), un rouage du système.

Définition 3 — Ordonnancement

L'**ordonnancement** est la stratégie par laquelle le système choisit, parmi les processus prêts, celui qui obtient le processeur, et pour combien de temps. Le petit intervalle de temps accordé à un processus s'appelle un **quantum**. Passer d'un processus à un autre s'appelle une **commutation de contexte**.

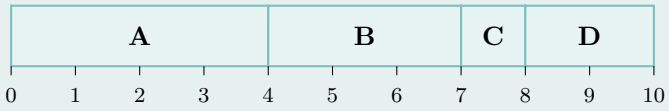
Pour comparer les stratégies, prenons quatre processus qui arrivent à des instants différents et demandent des durées de calcul différentes :

Processus	Instant d'arrivée	Durée de calcul
A	0	4
B	1	3
C	2	1
D	3	2

On mesure la qualité d'un ordonnancement par le **temps d'attente** de chaque processus (le temps passé à ne *pas* s'exécuter alors qu'il était arrivé), et on en prend la moyenne. On utilise aussi le **temps de séjour** d'un processus, égal à son instant de fin moins son instant d'arrivée (certains sujets l'appellent *temps de rotation* ou *temps d'exécution*). Les deux sont liés : temps d'attente = temps de séjour − durée de calcul.

A. Premier arrivé, premier servi (FCFS)

La stratégie la plus simple : on sert les processus dans leur **ordre d'arrivée**, chacun jusqu'au bout, sans jamais l'interrompre.

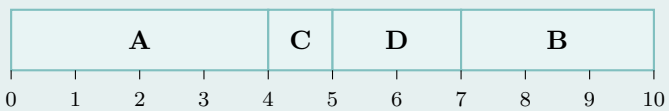


Temps d'attente : A= 0, B= 3, C= 5, D= 5. **Moyenne** = 3,25. Défaut criant : le petit C (durée 1) poireaute 5 unités derrière les gros — comme une caisse express bloquée par un caddie plein.

B. Le plus court d'abord (SJF)

À chaque fois que le processeur se libère, on élit le processus prêt dont la **durée est la plus courte**.

▷ **Je prédis.** Avant de regarder le chronogramme : à $t = 4$, A se termine ; lequel de B, C, D est élu, et dans quel ordre passent les deux autres ?



Temps d'attente : A= 0, C= 2, D= 2, B= 6. **Moyenne** = 2,50. En faisant passer les courts devant, on **réduit le temps d'attente moyen**. Quand tous les processus sont présents dès le départ, on peut démontrer que c'est même le meilleur choix possible [au-delà du programme] ; avec des arrivées échelonnées, comme ici, ce n'est plus garanti. Rançon : un processus long (ici B) peut attendre longtemps — c'est le risque de « famine ».

Au bac — variante préemptive : le plus court temps restant d'abord

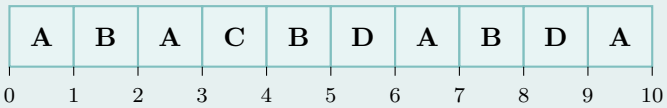
Ci-dessus, un processus élu s'exécute jusqu'au bout. Dans la variante **préemptive** (en anglais *SRTF*, *shortest remaining time first*), on refait le choix **à chaque unité de temps** : on élit le processus prêt dont la **durée restante** est la plus courte, quitte à **interrompre** celui qui s'exécute quand un processus plus court arrive. Une règle de départage est nécessaire en cas d'égalité (souvent : le premier arrivé). Cette variante figure dans des sujets de bac ; l'énoncé en rappelle alors les règles.

C. Le tourniquet (round-robin)

Pour que *personne* ne monopolise le processeur, on fixe un **quantum** (ici 1 unité) et on fait **tourner** les processus prêts dans une **file** (la structure FIFO du chapitre 3) : chacun s'exécute un quantum, puis repart en fin de file s'il n'a pas fini.

Convention : un processus qui arrive à l'instant où un quantum se termine entre dans la file **avant** le processus qui vient d'être interrompu.

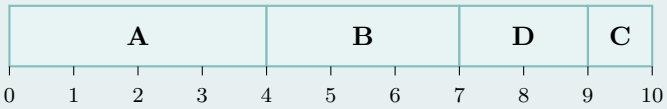
▷ **Je prédis.** Avant de regarder le chronogramme : quel processus est élu à $t = 1$, puis à $t = 2$? Appliquer la convention.



Temps d'attente : A= 6, B= 4, C= 1, D= 4. **Moyenne** = 3,75. En moyenne, c'est *moins* bon que le plus court d'abord ; mais chaque processus reçoit **très vite** un peu de processeur — l'ordinateur reste **réactif** (votre souris ne « gèle » pas pendant qu'un gros calcul tourne). C'est la stratégie des systèmes interactifs.

D. Par priorités

Chaque processus reçoit une **valeur de priorité**. Dans les sujets de bac, la valeur la plus **petite** est la plus **urgente** (vérifiez toujours la convention de l'énoncé). Chaque fois que le processeur se libère, on élit, parmi les processus prêts, celui dont la valeur est la plus petite ; à valeur égale, le **premier arrivé**. Sans préemption, l'élu s'exécute jusqu'au bout. Reprenons les quatre processus, avec les valeurs A= 3, B= 1, C= 2 et D= 1 :



À $t = 0$, A est seul : il est élu malgré sa valeur 3. À $t = 4$, B, C et D sont prêts ; B et D ont la valeur 1, et B est arrivé le premier. Temps d'attente : A= 0, B= 3, D= 4, C= 7. **Moyenne** = 3,50. Le petit C, peu prioritaire, passe en dernier : si des processus urgents arrivaient sans cesse, il pourrait attendre indéfiniment (**famine**). Pour l'éviter, on peut faire **remonter la priorité** d'un processus qui attend depuis trop longtemps. Dans la variante **préemptive**, un processus plus urgent qui arrive interrompt l'élu ; comme pour le plus court d'abord, l'énoncé en rappelle alors les règles.

Propriété 1 — Il n'y a pas d'ordonnancement parfait

Aucune stratégie n'est la meilleure sur tous les critères : le *plus court d'abord* optimise le temps d'attente moyen mais peut affamer les longs ; le *tourniquet* garantit la réactivité mais multiplie les commutations (qui coûtent du temps) ; le *premier arrivé* est simple mais pénalise les petits ; les *priorités* servent d'abord l'urgent, au risque d'affamer les autres. Choisir un ordonnancement, c'est choisir un **compromis**.

↻ **Retour sur l'activité.** En jouant l'ordonnanceur, vous l'avez constaté : avec « premier arrivé, premier servi », le clic de souris C attendait 5 tics derrière l'encodage vidéo et l'écran semblait figé ; avec le tourniquet, il n'attendait plus qu'un tic, au prix de nombreuses commutations.

Règle 2 — En quel langage écrit-on un *vrai* ordonnanceur ?

En exercice (et au bac), on programme un ordonnanceur en **Python**. C'est parfait pour *comprendre l'algorithme*… mais en vérité, **aucun système réel n'utiliserait Python pour cela**, et il faut savoir pourquoi.

Un ordonnanceur vit au cœur du système (le *noyau*), s'exécute des **milliers de fois par seconde**, et doit **manipuler directement le matériel** (sauvegarder et restaurer les registres du processeur à chaque commutation). Il lui faut donc être **extrêmement rapide** et **proche de la machine**. Python, lui, est un langage **interprété**, **de très haut niveau**, avec un ramasse-miettes (un mécanisme automatique qui libère la mémoire des données devenues inutiles, en anglais *garbage collector*, pratique mais qui interrompt le programme à des moments imprévisibles) : commode pour nous, mais bien trop lent et trop éloigné du matériel pour ce rôle.

Les vrais ordonnanceurs sont donc écrits en **C** — le langage des noyaux Linux, Windows et macOS — et leurs parties les plus basses (la **commutation de contexte**) en **assembleur**, le seul langage qui parle directement au processeur. **On fait du Python ici non pas pour *fabriquer* un ordonnanceur, mais pour en *simuler* et *comprendre* la logique** — la file, le quantum, le choix du prochain élu — sans se noyer dans la gestion du matériel.

► **Exercices d'application :** exercices 8 **et** 18 (ordonnancer à la main : trois stratégies ; chronogrammes en série, priorités comprises)

VII L'interblocage : quand tout le monde s'attend

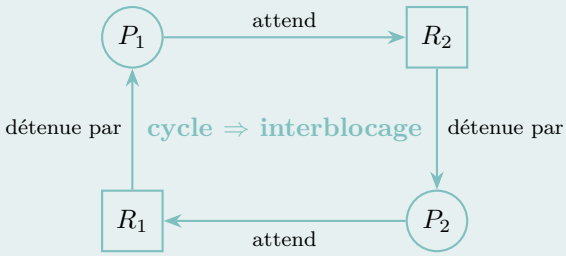
Le partage des ressources peut mener à une panne subtile où des processus s'attendent *mutuellement*, pour toujours.

Le scénario. Deux processus P_1, P_2 ; deux ressources R_1, R_2 (une imprimante et un scanner, disons), utilisables par un seul processus à la fois.

1. P_1 obtient R_1 ; P_2 obtient R_2.
2. P_1 demande alors R_2 … mais R_2 est prise par P_2 : P_1 se **bloque**.
3. P_2 demande alors R_1 … mais R_1 est prise par P_1 : P_2 se **bloque**.

Chacun retient une ressource que l'autre attend, et aucun ne peut la libérer puisque, bloqué, il ne s'exécute plus. La situation est figée pour toujours : c'est l'**interblocage** (*deadlock*).

On le visualise avec un **graphe d'attente** : une flèche « $P \to R$ » signifie « P attend R », une flèche « $R \to P$ » signifie « R est détenue par P ». Un **cycle** dans ce graphe est la signature d'un interblocage (on apprendra à détecter un cycle au chapitre 8, *Graphes*).



▷ **Je prédis.** Et si P_2 demandait, lui aussi, R_1 *puis* R_2, dans le même ordre que P_1 : un interblocage serait-il encore possible ?
▷ **Je vérifie.** Non : le premier qui obtient R_1 obtiendra aussi R_2 ; l'autre attend R_1 sans rien détenir. Pas de cycle dans le graphe d'attente, donc pas d'interblocage.

↻ **Retour sur l'activité.** Les scripts de P et Q jouaient exactement ce scénario, et le schéma « qui tient, qui attend » du cahier était ce graphe d'attente, avec son cycle. La situation ne pouvait pas se débloquer seule ; faire prendre les deux objets dans le même ordre aux deux scripts l'empêche (voir la remarque ci-dessous).

Propriété 2 — Les quatre conditions de Coffman

Un interblocage ne peut survenir que si les **quatre** conditions suivantes sont réunies *en même temps* :

1. **exclusion mutuelle** : une ressource ne sert qu'un processus à la fois ;
2. **détention et attente** : un processus garde ses ressources tout en en réclamant d'autres ;
3. **non-préemption** : on ne peut pas retirer de force une ressource à un processus ;
4. **attente circulaire** : il existe un cycle de processus attendant chacun une ressource tenue par le suivant.

Casser *n'importe laquelle* de ces conditions suffit à empêcher tout interblocage.

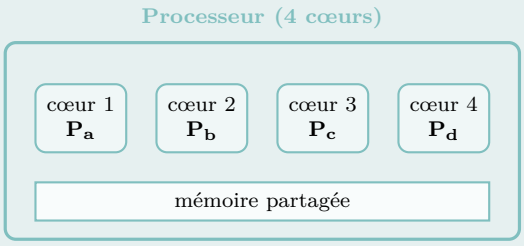
Remarque
Que faire face à un interblocage ? On peut le **détecter** (chercher un cycle dans le graphe d'attente) puis le **briser** en tuant l'un des processus fautifs (qui libère alors ses ressources). Ou mieux, l'**éviter** en amont : par exemple imposer que tout processus demande les ressources **toujours dans le même ordre** (jamais R_2 avant R_1) — ce qui casse l'attente circulaire. « Éteindre et rallumer » l'ordinateur, c'est d'ailleurs souvent tuer les processus bloqués…

► **Exercices d'application :** exercices 10 **et** 11 (interblocage)

VIII Et aujourd'hui : les processeurs multi-cœurs [au-delà du programme]

Depuis le début de ce chapitre, nous avons supposé **un seul cœur** : un seul processus élu à la fois, et l'illusion du parallélisme grâce à la vitesse de commutation. Or, depuis le milieu des années 2000, presque tous les processeurs sont **multi-cœurs** (on les retrouvera, intégrés à une seule puce, au chapitre 14 sur les systèmes sur puce).

Un **cœur** (*core*) est un processeur complet à lui seul, capable d'exécuter *son* processus. Un processeur « quadricœur » en contient quatre : il peut donc élire **jusqu'à quatre processus *vraiment* en même temps** — ce n'est plus une illusion, c'est du **vrai parallélisme**.



Quatre cœurs exécutent quatre processus simultanément, en partageant la mémoire.

Pourquoi ce virage ? Pendant des décennies, on accélérait un cœur unique en augmentant sa **fréquence** (les gigahertz). Mais on a heurté un mur : plus de fréquence, c'est plus de **chaleur** et de **consommation**. Plutôt que de faire courir un cœur toujours plus vite, on a choisi d'en **multiplier** le nombre.

Cela change peu ce que nous avons appris : sur *chaque* cœur, les états, le quantum et l'ordonnancement restent valables. L'ordonnanceur gagne juste une mission de plus — **répartir les processus prêts entre les cœurs** (l'équilibrage de charge). En revanche, comme les cœurs **partagent la mémoire**, la synchronisation devient plus délicate (les interblocages restent possibles, et il faut garder les mémoires caches cohérentes).

Conséquence pour qui programme : pour tirer parti de N cœurs, un programme doit être **découpé pour s'exécuter en parallèle** (en plusieurs *fils d'exécution*, ou *threads*) ; sinon un seul cœur travaille pendant que les autres se tournent les pouces. C'est pourquoi « savoir paralléliser » est devenu essentiel. Aujourd'hui, un simple téléphone compte couramment 6 à 8 cœurs, un ordinateur portable de 8 à 16.

IX Ouverture

Les processus ne sont pas un monde à part : ce chapitre réutilise les outils des chapitres précédents et en annonce d'autres.

- Au **bac**, on vous demandera surtout de **dérouler** un ordonnancement (chronogramme, temps d'attente), de suivre les **états** d'un processus, de lire une sortie de `ps`, et de **repérer** un interblocage dans une situation décrite.

X Un peu d'histoire : du traitement par lots au multitâche

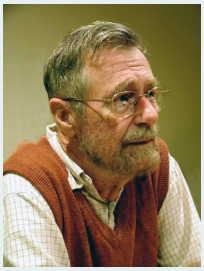


Grace Hopper en 1984

Les tout premiers ordinateurs (années 1950) exécutaient les programmes **un par un**, à la file (*traitement par lots*) : pendant qu'un programme attendait une lecture sur bande, le processeur, hors de prix, restait *oisif*. Quel gâchis ! Les programmeurs de l'époque, comme l'Américaine **Grace Hopper** (1906–1992), officier de la marine et pionnière qui écrit en 1952 l'un des tout premiers **compilateurs** puis inspire le langage COBOL, confient leurs programmes à un opérateur et attendent le résultat… parfois jusqu'au lendemain.

L'idée de partager le processeur entre plusieurs tâches — le **temps partagé** (*time-sharing*) — naît au début des années 1960 (système CTSS au MIT, 1961, puis l'ambitieux Multics). En 1969, aux laboratoires Bell, **Ken Thompson** et **Dennis Ritchie** en tirent un système plus simple et élégant, **Unix**, dont descendent aujourd'hui Linux, macOS, Android et iOS. Toute la gestion des processus de ce cours vient de là.

Cette histoire montre comment les rôles du **matériel** et du **logiciel** ont évolué. Tant que le processeur était la ressource la plus chère, on organisait le travail des humains autour de lui (files de cartes perforées, opérateurs). À mesure que le matériel est devenu rapide et bon marché, c'est le **logiciel**, le système d'exploitation, qui a pris en charge le partage du processeur, au service de dizaines d'utilisateurs puis de dizaines d'applications.



Edsger W. Dijkstra

Côté théorie, le Néerlandais **Edsger Dijkstra** (déjà croisé avec l'algorithme de la gare de triage, et qu'on retrouvera pour les plus courts chemins) pose en 1965 les outils de la synchronisation (les *sémaphores*) et illustre l'interblocage par son célèbre « dîner des philosophes » [au-delà du programme]. En 1971, **Edward Coffman** énonce les quatre conditions qui portent son nom. La prochaine fois que votre téléphone jongle avec vingt applications sans sourciller, vous saurez quel héritage se cache derrière cette fluidité.

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise :** la **file** (*Structures linéaires : piles, files, listes chaînées*) : le tourniquet n'est qu'un enfiler/défiler répété.
- **Ce qui en découle :** la relation père/fils de `fork` formera un arbre (*Arbres*) ; détecter un interblocage, c'est chercher un cycle, ce que vous programmerez en *Graphes*.
- **Pour aller plus loin :** vous verrez en *Systèmes sur puce et informatique embarquée* un téléphone ordonnancer ses processus sur les cœurs d'une seule puce.

Bilan — la carte mémoire

Notion	À retenir
Processus	un **programme en cours d'exécution** (*pas* le code source)
Système d'exploitation	le **chef d'orchestre** : partage processeur et ressources
Trois états	**élu** (s'exécute) / **prêt** (attend son tour) / **bloqué** (attend une ressource)
Transitions	élection, fin de quantum, blocage, déblocage ; jamais bloqué → élu
Création	`fork` : père/fils ⇒ **arbre** de processus ; identifiants **PID** / **PPID**
Observer / gérer	`ps -ef` (PID/PPID), `pstree`, `top` ; `kill <PID>` pour arrêter
Ordonnancement	choisir le prochain élu ; **quantum**, commutation de contexte
Stratégies	premier arrivé / plus court d'abord (attente min.) / tourniquet (réactif) / priorités (l'urgent d'abord)
Interblocage	cycle d'attente mutuelle ; **graphe d'attente** ; conditions de Coffman
En vrai	les vrais ordonnanceurs sont en **C** (+ assembleur) ; en Python on *simule*

Retour à la situation de départ

1. Le système d'exploitation donne le processeur à chaque **processus** pendant un **quantum**, puis le lui reprend, des milliers de fois par seconde : un seul processus est **élu** à la fois, les autres sont **prêts** ou **bloqués** (§ I, III et VI).
2. C'est le rôle de l'**ordonnanceur**, et aucune stratégie n'est parfaite : le plus court d'abord réduit le temps d'attente moyen mais peut affamer les longs, le tourniquet garde l'ordinateur réactif (§ VI).
3. C'est un **interblocage** : chacun détient une ressource que l'autre attend. On le repère par un **cycle** dans le graphe d'attente (§ VII).

Erreurs fréquentes

- **Confondre programme et processus.** Le programme est un fichier ; le processus est son exécution vivante. Un même programme peut donner *plusieurs* processus.
- **Croire que plusieurs processus s'exécutent « vraiment » en même temps** sur un cœur. Un seul est **élu** à la fois ; l'illusion vient de la vitesse de commutation.
- **Faire passer un processus de « bloqué » à « élu ».** Impossible : il repasse d'abord par « prêt ».
- **Oublier qu'il faut être élu pour libérer une ressource.** Un processus bloqué ne rend donc rien — c'est ce qui rend l'interblocage définitif.
- **Croire qu'un interblocage vient d'un « bug »** dans un seul programme. Il naît de l'**interaction** entre processus et de l'ordre des demandes de ressources.

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

☐ **distinguer** un programme d'un processus, et citer les grandes fonctions d'un système d'exploitation → ex. 1 ;
☐ **nommer** les trois états et **compléter** le schéma des transitions → ex. 2, 3, 4 ;
☐ **décrire** la création d'un processus (`fork`, père et fils) et compter les processus créés → ex. 7 ;
☐ **lire** une sortie de `ps`, en déduire un arbre de processus et des PID / PPID, et connaître `top` / `kill` → ex. 5, 6, 12 ;
☐ **dérouler** un ordonnancement (premier arrivé, plus court d'abord, tourniquet, par priorités) sous forme de **chronogramme** et en calculer les temps d'attente et le **temps d'attente moyen** → ex. 8, 12, 18 ;
☐ **reconnaître** un interblocage, le représenter par un **graphe d'attente** et proposer un moyen de l'éviter → ex. 10, 11.

Vers le Grand Oral

- **Comment un ordinateur donne-t-il l'illusion de tout faire à la fois avec un seul processeur ?** *(les états, l'ordonnancement, la commutation de contexte à grande vitesse.)*
- **Réactivité ou efficacité : comment un système choisit-il qui s'exécute ?** *(comparer plus court d'abord et tourniquet ; le compromis ; les chronogrammes.)*
- **Peut-on prouver qu'un système ne se figera jamais ?** *(l'interblocage ; les quatre conditions de Coffman ; l'ordre total sur les ressources.)*
- **En quoi le partage du temps a-t-il transformé l'informatique ?** *(du traitement par lots au temps partagé ; Unix ; l'héritage jusqu'à nos téléphones.)*

EXERCICES

Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

Mode d'emploi

Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 32.
Le symbole ▶ signale un exercice *à programmer et tester* en Python. Pour tout ordonnancement, on soigne le **chronogramme** et on justifie les **états**.

Programme, processus, états ↑cours

★ **Exercice 1** *Programme ou processus ?* ▶ corrigé p. 812 • essentiel

Pour chaque affirmation, dire si elle décrit un *programme* ou un *processus*, et justifier.

▷ coup de pouce

1. Le fichier `jeu.py` rangé sur le disque dur.
2. Deux fenêtres du même navigateur ouvertes en même temps.
3. La recette de cuisine imprimée sur une fiche.
4. Ce qui possède un PID et un état à un instant donné.

★ **Exercice 2** *Les trois états* ▶ corrigé p. 812

1. Nommer les trois états possibles d'un processus et, en une phrase chacun, ce qu'ils signifient.
2. Nommer la transition correspondante : (a) le système donne le processeur à un processus prêt ; (b) un processus demande une donnée sur le disque ; (c) le temps de parole d'un processus est écoulé ; (d) la donnée attendue arrive enfin.
3. Citer une transition **impossible** entre deux états, et expliquer pourquoi.

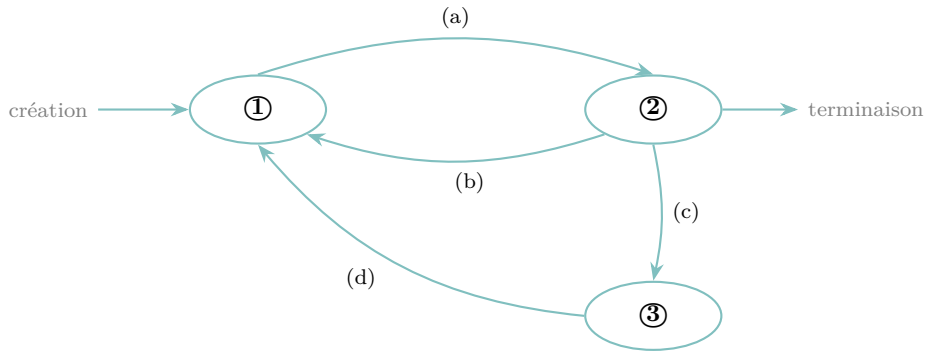
▷ coup de pouce

★★ **Exercice 3** *Le fil d'une vie* ▶ corrigé p. 812

Un processus P démarre, s'exécute, demande une lecture sur le disque, l'obtient un peu plus tard (mais le processeur est alors occupé), reprend, puis se termine. Donner la **suite ordonnée des états** traversés par P (de sa création à sa fin), en nommant chaque transition. ▷ coup de pouce

★ **Exercice 4** *Compléter l'automate des états* ▶ corrigé p. 812 • essentiel

Recopier le schéma ci-dessous et le compléter : nommer les trois **états** ①, ②, ③ et les quatre **transitions (a)**, **(b)**, **(c)**, **(d)**, en choisissant dans les étiquettes proposées. ▷ coup de pouce

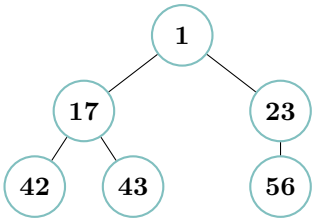


Étiquettes. États : *Élu*, *Prêt*, *Bloqué*. Transitions : *élection*, *blocage*, *déblocage*, *fin du quantum*.

Arbre de processus, PID et PPID ↑ cours

★ **Exercice 5** *Lire un arbre de processus* ▶ corrigé p. 813 • essentiel

On donne la hiérarchie de processus suivante (chaque nœud est étiqueté par son PID) :



1. Donner le PPID du processus 42, puis celui du processus 23.
2. Combien le processus 17 a-t-il de fils ? Les citer.
3. Le processus 1 a-t-il un PPID ? Que représente-t-il ?
4. On `kill` le processus 17. Que deviennent les processus 42 et 43 ? Préciser leur nouveau PPID.

▷ coup de pouce

★★ **Exercice 6** *Reconstruire l'arbre* ▶ corrigé p. 813

Une commande `ps` donne le tableau suivant. Dessiner l'arbre des processus correspondant.

▷ coup de pouce

PID	PPID	Commande
1	0	`init`
8	1	`bash`
9	1	`sshd`
40	8	`python`
41	8	`vim`
57	9	`bash`

★★ **Exercice 7** *L'appel* `fork` ▶ corrigé p. 813

L'appel `fork` **duplique** le processus qui l'exécute : après un `fork`, le père *et* le fils poursuivent tous deux la suite du programme.

1. On exécute le programme suivant. Combien de processus existent après la ligne 2 ? après la ligne 3 ? Combien de fois « A » est-il affiché ?

```
1 from os import fork
2 fork()          # ligne 2
3 fork()          # ligne 3
4 print("A")      # ligne 4
```

2. Dessiner l'arbre des processus créés (on note P_0 le processus de départ).

3. Combien de processus existeraient si l'on ajoutait un troisième `fork()` avant le `print` ? Donner la formule générale pour n appels `fork` successifs.

▷ coup de pouce

Ordonnancer à la main ↑cours

★★ **Exercice 8** *Trois stratégies, un même jeu de processus* ▶ corrigé p. 813 • essentiel

Quatre processus arrivent et demandent le processeur :

Processus	A	B	C	D
Instant d'arrivée	0	1	2	4
Durée	3	4	2	1

Pour chacune des trois stratégies ci-dessous, tracer le **chronogramme** (de $t = 0$ à $t = 10$) et calculer le **temps d'attente moyen** (rappel : temps d'attente = temps de séjour − durée, où le temps de séjour — parfois appelé temps de rotation — vaut instant de fin − instant d'arrivée).

1. **Premier arrivé, premier servi** (chacun jusqu'au bout, dans l'ordre d'arrivée).
2. **Plus court d'abord** (à chaque libération, on élit le processus prêt le plus court).
3. **Tourniquet** de quantum 1 (chacun un cycle, puis en fin de file s'il n'a pas fini). On applique la convention du cours : un processus qui arrive à l'instant où un quantum se termine entre dans la file **avant** le processus qui vient d'être interrompu.
4. Comparer les trois moyennes. Laquelle minimise l'attente ? Laquelle rend le système le plus *réactif* ? ▷ coup de pouce

Programmer un ordonnanceur ↑cours

★★★ **Exercice 9** *Le tourniquet en Python* (d'après Polynésie 2023, jour 1) ▶ corrigé p. 814 ≈ 45 min

On simule la file d'attente par une liste, et chaque processus par un objet de la classe :

```
class Processus:
    def __init__(self, pid, duree):
        self.pid = pid
        self.duree = duree
        self.reste_a_faire = duree      # cycles restant a executer
        self.etat = "Prêt"
```

On considère trois processus, dans cet ordre d'arrivée : PID 11 (durée 4), PID 20 (durée 2), PID 32 (durée 3).

1. Avec un quantum de **1** cycle, donner la suite des PID dans l'ordre de leur exécution : `11, 20, 32, 11, ...`
2. Donner cette suite pour un quantum de **2** cycles. ▷ coup de pouce
3. ▶ Recopier et compléter les trois méthodes de la classe `Processus` :

```
    def execute_un_cycle(self):
        ........................        # un cycle de moins a faire

    def change_etat(self, nouvel_etat):
        ........................

    def est_termine(self):
        ........................        # True si reste_a_faire vaut 0
```

4. ▶ Recopier et compléter la fonction `tourniquet`, qui renvoie la liste des PID dans l'ordre d'exécution (la file est ici une liste Python défilée par `pop(0)` : choix simple mais coûteux, en $O(n)$, cf. chapitre *Structures linéaires*) : ▷ coup de pouce ▷ début de solution

```python
def tourniquet(liste_attente, quantum):
    ordre_execution = []
    while liste_attente != []:
        processus = liste_attente.pop(0)
        processus.change_etat("En cours d'exécution")
        compteur = 0
        while ................. and .................:
            ordre_execution.append(.................)
            processus.execute_un_cycle()
            compteur = compteur + 1
        if ................. :
            processus.change_etat("Suspendu")
            liste_attente.append(processus)
        else:
            processus.change_etat("Terminé")
    return ordre_execution
```

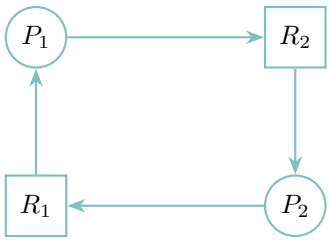
Le sujet de bac nomme les états à sa façon : « En cours d'exécution » correspond à l'état *élu* du cours, « Suspendu » à l'état *prêt* (processus remis en file).

5. ▶ Créer la liste d'attente des trois processus ci-dessus et vérifier, avec `tourniquet(liste_attente, 1)`, la suite de la question 1. Recréer ensuite une liste de **nouveaux** processus (la fonction a vidé la liste et terminé les anciens) et vérifier, avec un quantum de 2, la suite de la question 2.

Interblocage ↑cours

★★ **Exercice 10** *Lire un graphe d'attente* ▶corrigé p. 815 • essentiel

Dans le graphe d'attente ci-dessous, une flèche $P \to R$ se lit « P attend R » et $R \to P$ se lit « R est détenue par P ».



1. Quelle ressource P_1 détient-il ? Laquelle attend-il ?
2. Y a-t-il un interblocage ? Justifier en une phrase (parler de **cycle**).
3. Proposer une action concrète pour **sortir** de cette situation. ▷coup de pouce

★★★ **Exercice 11** *Éviter l'interblocage* ▶corrigé p. 815 ≈15 min

Deux processus partagent une imprimante I et un scanner S. P_1 a besoin des deux dans l'ordre I puis S ; P_2 en a besoin dans l'ordre S puis I.

1. Décrire un enchaînement précis d'événements qui mène à un interblocage.
2. Parmi les **quatre conditions de Coffman** [au-delà du programme] (exclusion mutuelle, détention et attente, non-préemption, attente circulaire), laquelle casse-t-on si l'on impose que *tous* les processus demandent les ressources **dans le même ordre** (I avant S) ? Justifier qu'alors l'interblocage devient impossible. ▷coup de pouce ▷début de solution

Exercices type bac ↑cours

★★ **Exercice 12** *Type bac — Observer puis ordonnancer des processus* (d'après Amérique du Nord 2023, jour 1) ▶ corrigé p. 815 ≈30 min

Partie A — Processus. La commande `ps` tapée dans un terminal donne la liste des processus du système. La commande `ps -eo user,pid,ppid,time,cmd` affiche, pour tous les processus, les colonnes suivantes : `USER` (l'utilisateur qui exécute le processus), `PID` (l'identifiant du processus), `PPID` (l'identifiant du processus parent), `TIME` (le temps d'utilisation du processeur par le processus) et `CMD` (la commande ou l'application à l'origine du processus). Voici un extrait de l'écran après exécution de la commande :

```
USER      PID   PPID  TIME      CMD
root      1     0     00:00:02  /sbin/init
kerneoops 1567  1     00:00:00  /usr/sbin/kerneloops
user01    1611  1     00:00:00  /lib/systemd/systemd --user
user01    1752  1611  00:00:00  /usr/libexec/gnome-session-binary --systemd-
user01    1766  1752  00:00:00  /usr/libexec/at-spi-bus-launcher --launch-im
user01    1770  1611  00:00:51  /usr/bin/gnome-shell
user01    5151  1632  00:00:00  /usr/libexec/gvfsd-network --spawner :1.2 /o
user01    5168  1632  00:00:00  /usr/libexec/gvfsd-dnssd --sp
user01    5348  1770  00:00:01  /snap/vlc/2344/usr/bin/vlc
user01    5468  1770  00:00:01  /snap/gimp/393/usr/bin/gimp
user01    5564  1611  00:00:00  /usr/bin/python3 /usr/bin/gnome-terminal --w
user01    5566  5564  00:00:00  /usr/bin/gnome-terminal.real --wait
user01    5571  1611  00:00:00  /usr/libexec/gnome-terminal-server
user01    5589  5571  00:00:00  bash
user01    5596  5589  00:00:01  /usr/bin/python3 /home/user01/.local/bin/tho
user01    5603  5596  00:00:00  /usr/bin/python3 -u -B -m thonny.plugins.cpy
user01    5615  1611  00:00:00  /usr/bin/python3 /usr/bin/gnome-terminal --w
user01    5617  5615  00:00:00  /usr/bin/gnome-terminal.real --wait
user01    5622  5571  00:00:00  bash
user01    5629  5622  00:00:00  ps -eo user,pid,ppid,time,cmd
```

En utilisant cet extrait :

1. Déterminer le nom de l'application associée au processus dont l'identifiant est 5468.
2. Déterminer l'identifiant du processus qui a sollicité le processeur le plus longtemps.
3. Déterminer l'identifiant du processus qui a le plus d'enfants.
4. Déterminer le nom du programme dont est issue la commande `ps`.
5. Déterminer la succession des identifiants des processus qui ont permis de générer le processus associé à la ligne de commande `ps -eo user,pid,ppid,time,cmd`, en partant du processus initial de PID 1.

Partie B — Ordonnancement. Étant donné un instant initial $t_0 = 0$, un processus est caractérisé par sa **durée d'exécution** (en unités de temps) et son **instant d'arrivée** (l'instant où il est créé par le système d'exploitation).

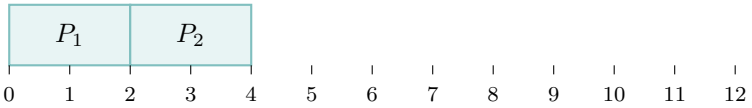
6. Les trois états principaux d'un processus sont « prêt », « bloqué » et « élu ». Donner la définition de chacun de ces trois états.

Le système utilise un ordonnancement par **tourniquet** : le processus élu dispose d'un temps prédéfini appelé *quantum*, et s'exécute soit jusqu'à ce qu'il soit terminé (durée restante inférieure ou égale au quantum), soit pendant la durée du quantum (il retourne alors à l'état prêt et réintègre la file d'attente en dernière position), soit jusqu'à ce qu'il se bloque de lui-même faute de ressource disponible.

7. On considère trois processus, avec un quantum de 2 unités de temps :

Processus	Durée d'exécution	Instant d'arrivée
P_1	5	0
P_2	3	1
P_3	4	5

Recopier et compléter le chronogramme d'exécution des processus. ▷ coup de pouce



4

★★ **Exercice 13** *Type bac — Tourniquet cycle par cycle et interblocage* (d'après Amérique du Nord 2024, jour 1) ▶ corrigé p. 816 ≈ 45 min

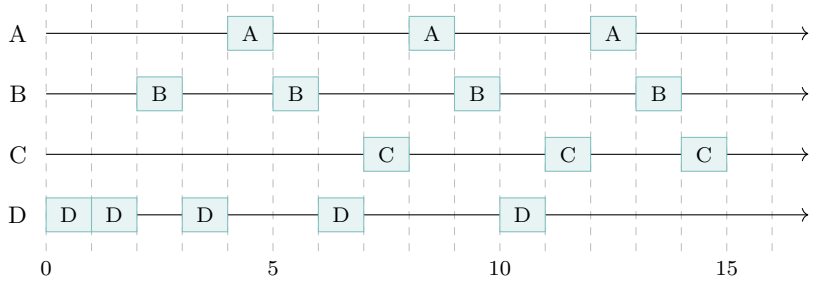
On s'intéresse aux processus et à leur ordonnancement au sein d'un système d'exploitation, sur un **monoprocesseur**.

1. Citer les trois états dans lesquels un processus peut se trouver.

On simule l'ordonnancement avec des objets. On dispose d'une classe `Processus` : `p = Processus(nom, duree)` crée un processus de nom `nom` et de durée `duree` (en cycles d'ordonnancement) ; `p.execute_un_cycle()` exécute le processus pendant un cycle ; `p.est_fini()` renvoie `True` si le processus est terminé, `False` sinon. Pour simplifier, on ne s'intéresse pas aux ressources qu'un processus pourrait acquérir ou libérer.

2. Citer les deux seuls états possibles pour un processus dans ce contexte.

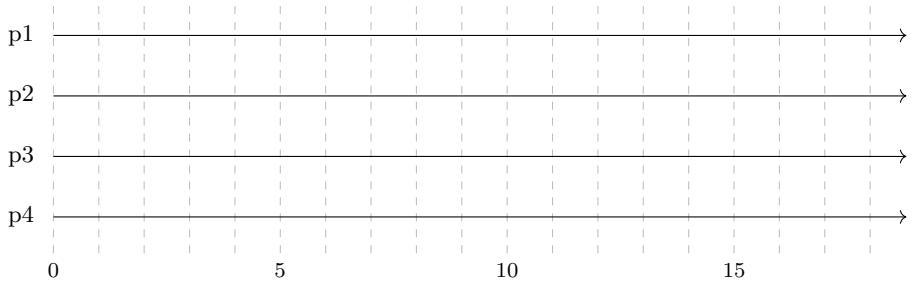
On ordonnance les processus selon une méthode du type tourniquet telle qu'**à chaque cycle** : si un nouveau processus est créé, il est mis dans la file d'attente ; ensuite, on défile un processus de la file d'attente et on l'exécute pendant un cycle ; si le processus exécuté n'est pas terminé, on le replace dans la file. Par exemple, avec les processus A (créé au cycle 2, durée 3), B (cycle 1, durée 4), C (cycle 4, durée 3) et D (cycle 0, durée 5), on obtient le chronogramme :



On décrit maintenant d'autres processus et leur moment de création ; le dictionnaire `depart_proc` associe à un cycle le processus créé à ce moment (un seul processus peut être créé lors d'un cycle donné) :

```
p1 = Processus("p1", 4)
p2 = Processus("p2", 3)
p3 = Processus("p3", 5)
p4 = Processus("p4", 3)
depart_proc = {0: p1, 1: p3, 2: p2, 3: p4}
```

3. Recopier et compléter le chronogramme ci-dessous pour les processus `p1`, `p2`, `p3` et `p4`.



L'ordonnancement est mis en place par les classes `File` et `Ordonnanceur` déjà écrites dans l'exercice « Ordonnancer des processus avec une file » du chapitre *Structures linéaires* (exercice 13, même sujet de bac). On les reprend telles qu'elles ont été complétées, avec deux précisions : `defile` renvoie `None` si la file est vide, et `tourniquet` renvoie le nom du processus élu (ou `None` si la file est vide) ; ici, la méthode qui exécute un processus pendant un cycle s'appelle `execute_un_cycle`. Rappel de l'interface utile :

```
ordo = Ordonnanceur()                # ordo.temps vaut 0, file vide
ordo.ajoute_nouveau_processus(p)     # enfile le processus p
nom = ordo.tourniquet()              # un cycle : temps + 1, elit, execute, remet en file
ordo.file.est_vide()                 # True s'il ne reste aucun processus en attente
```

4. ▶ Écrire un programme qui, à partir des variables `p1`, `p2`, `p3`, `p4` et `depart_proc` définies précédemment :

   - crée un ordonnanceur ;
   - à chaque cycle, lui ajoute le processus de `depart_proc` créé à ce cycle, s'il y en a un ;
   - affiche le nom du processus élu par `tourniquet` ;
   - s'arrête quand il n'y a plus de processus à créer ni à exécuter.

▷ coup de pouce ▷ début de solution

Dans la situation donnée en exemple (premier chronogramme), les processus utilisent en fait des ressources : un fichier commun, le clavier, le processeur graphique (GPU) et le port 25000 de la connexion Internet. Voici, dans l'ordre, ce que fait chaque processus (une ligne par cycle d'exécution) :

A	B	C	D
acquérir le GPU faire des calculs libérer le GPU	acquérir le clavier acquérir le fichier libérer le clavier libérer le fichier	acquérir le port faire des calculs libérer le port	acquérir le fichier faire des calculs acquérir le clavier libérer le clavier libérer le fichier

5. Montrer que l'ordre d'exécution donné en exemple aboutit à une situation d'**interblocage**.

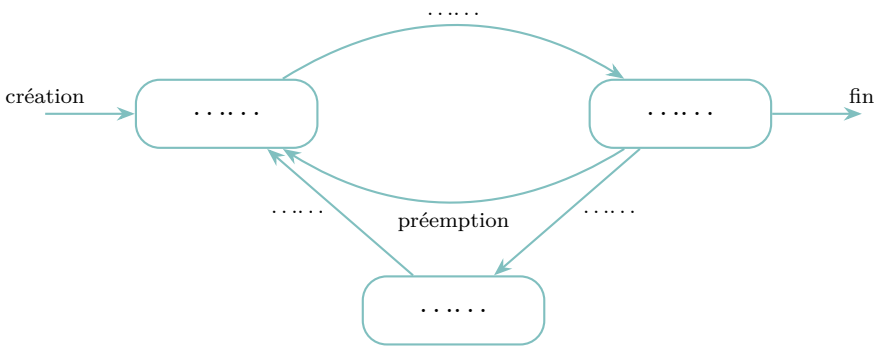
▷ coup de pouce

★★★ **Exercice 14** IA IA IA *Type bac — Une application de streaming* (d'après Amérique du Nord 2026, jour 2) ▶ corrigé p. 816 ≈ 45 min

Un système d'exploitation exécute plusieurs applications « à la fois » en répartissant le temps de calcul du processeur entre les différents processus, qui s'exécutent chacun à leur tour très rapidement. Même si un seul processus utilise réellement le processeur à un instant donné, cette alternance rapide donne l'impression que tout s'exécute en même temps. Lorsque le système interrompt un processus pour en exécuter un autre, on parle de *préemption*.

Partie A

1. Recopier et compléter le schéma ci-dessous avec les termes : « élu », « prêt », « bloqué », « élection », « blocage », « déblocage ».



Une application de streaming musical se compose de trois processus : P1 gère l'interface utilisateur (listes de morceaux, boutons lecture-pause…) ; P2 télécharge la musique et l'écrit dans une mémoire cache locale ; P3, le *lecteur audio*, décode la musique et envoie le flux au système pour lecture. Dans la situation suivante : l'utilisateur a mis l'application en arrière-plan, cachant l'interface ; P2 attend un accès à la carte Wi-Fi pour recharger des données ; P3 décode de la musique et l'envoie au système.

2. Indiquer l'état de chacun des trois processus P1, P2 et P3 dans cette situation.

3. Expliquer, de façon générale, quand un interblocage de processus peut survenir.

On considère quatre ressources : un processeur graphique (GPU), un microphone (MIC), une caméra (CAM) et un processeur dédié au calcul (CAL), chacune utilisable par un seul processus à la fois. Quatre processus P1, P2, P3 et P4 les demandent dans l'ordre suivant :

P1	P2	P3	P4
demander MIC	demander CAL	demander CAM	demander GPU
demander CAL	demander MIC	libérer CAM	demander CAL
libérer CAL	libérer MIC	demander CAL	demander CAM
libérer MIC	libérer CAL	demander MIC	libérer CAM
demander CAM	demander CAM	libérer MIC	libérer CAL
demander GPU	libérer CAM	libérer CAL	demander MIC
libérer CAM		demander GPU	libérer GPU
libérer GPU		libérer GPU	libérer MIC

4. Les processus s'exécutent de manière concurrente. Justifier qu'une situation d'interblocage peut se produire.

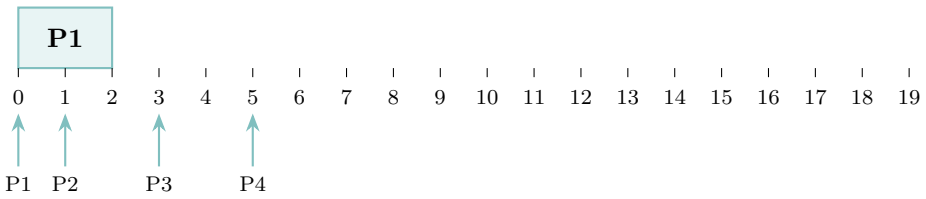
▷ coup de pouce

Partie B

On s'intéresse à un ordonnanceur de type tourniquet dont le *quantum* vaut 2 ms. Chaque processus défilé de la file d'attente s'exécute pendant une durée au plus égale au quantum : s'il termine avant ou à la fin du quantum, un autre processus est défilé et bénéficie d'un nouveau quantum ; sinon, il est remis en queue de la file d'attente. D'autres processus peuvent être ajoutés à la file au fur et à mesure de leur arrivée.

Processus	Instant d'arrivée	Temps d'exécution total
P1	0 ms	6 ms
P2	1 ms	4 ms
P3	3 ms	5 ms
P4	5 ms	3 ms

5. Recopier et compléter le chronogramme suivant, en indiquant quel processus utilise le processeur à chaque instant, de 0 ms à la fin de l'exécution de tous les processus (P1 s'exécute déjà entre 0 et 2 ms ; les flèches indiquent les instants d'arrivée).



En Python, chaque processus est un dictionnaire de clés `'nom'`, `'arrivee'` et `'temps'` (temps d'exécution *restant*, en ms) :

```
P1 = {'nom': 'P1', 'arrivee': 0, 'temps': 6}
P2 = {'nom': 'P2', 'arrivee': 1, 'temps': 4}
P3 = {'nom': 'P3', 'arrivee': 3, 'temps': 5}
P4 = {'nom': 'P4', 'arrivee': 5, 'temps': 3}
quantum = 2
```

On dispose d'une implémentation de file : `creer_file_vide()` renvoie une file vide ; `est_vide(f)` renvoie un booléen ; `enfiler(f, e)` ajoute `e` en queue de `f` (et renvoie `None`) ; `defiler(f)` enlève l'élément de tête de `f` et le renvoie.

6. Donner les instructions qui créent une file `fp` contenant les processus `P1`, `P2`, `P3` et `P4` classés par ordre d'arrivée, le premier arrivé étant en tête de la file.

La fonction `execute_un_processus` réalise une étape du tourniquet : elle prend une file de processus non vide `file_d_attente` et l'instant `t` de début de l'étape ; elle extrait de la file le processus à exécuter, le remet si besoin dans la file avec son temps d'exécution restant, et renvoie l'instant où l'étape se termine (fin du processus ou fin du quantum).

7. ▶ Recopier et compléter le code de la fonction `execute_un_processus`. ▷ coup de pouce ▷ début de solution

```
def execute_un_processus(file_d_attente, t):
    processus = defiler(file_d_attente)
    if processus['temps'] ... quantum:
        processus['temps'] = ...
        ...
        return t + ...
    else:
        return t + ...
```

On suppose désormais que tous les processus sont déjà arrivés (ils sont tous dans la file prise en entrée) : on ne tient plus compte des instants d'arrivée, et l'exécution commence à l'instant 0. La fonction `execute_tous_processus` simule le tourniquet jusqu'à ce que tous les processus soient terminés et renvoie l'instant où se termine la dernière exécution.

8. ▶ Recopier et compléter le code de la fonction `execute_tous_processus`.

```
def execute_tous_processus(file_d_attente):
    t = 0
    while ...
        t = ...
    return t
```

★★★ **Exercice 15** *Type bac — Des files de priorité pour l'ordonnanceur* (d'après Asie 2025, jour 2) ▶ corrigé p. 817 ≈ 60 min

On souhaite élaborer un programme système qui gère l'ordre d'exécution des processus sur le processeur.

1. Donner le nom de ce type de programme.

2. Donner les différents états possibles d'un processus.

Chaque processus dispose d'une **valeur de priorité**. Un processus est prioritaire sur un autre si sa valeur de priorité est **plus petite**. Ainsi, pour rendre un processus moins prioritaire, on augmente sa valeur de priorité (par exemple de 2 à 3).

Fonctionnement du programme. On dispose d'une liste dont les éléments sont des files de processus : la première file contient les processus de valeur de priorité 0 (la plus élevée), la seconde ceux de valeur 1, etc.

- **À l'arrivée d'un nouveau processus :** lui attribuer la valeur de priorité 0 et le placer dans la file correspondante (la première file de la liste).
- **À chaque cycle d'horloge :**
  - s'il n'y a pas de processus en cours d'exécution et s'il reste des processus en attente : sélectionner un processus le plus prioritaire (plus petite valeur de priorité) dans l'une des files non vides, et l'élire comme processus en cours d'exécution ;
  - sinon, si un processus est en cours d'exécution :
    - si le processus a terminé son exécution, le retirer du processeur ;
    - sinon, incrémenter son temps d'utilisation, puis :
      - si des processus **au moins aussi prioritaires** (valeur de priorité inférieure ou égale) attendent : retirer le processus en cours du processeur, **augmenter sa valeur de priorité de** 1 (il devient moins prioritaire) et le mettre dans la file correspondant à sa nouvelle valeur de priorité, puis élire un processus le plus prioritaire parmi les files non vides ;
      - sinon, augmenter sa valeur de priorité de 1 et continuer à exécuter le processus en cours.

3. Parmi les structures *liste*, *file* et *pile*, donner la plus adaptée pour stocker les processus d'une même priorité.

Un processus est représenté par une classe `Processus` d'attributs `PID` (identifiant), `priorite`, `temps_utilisation` (temps passé sur le processeur) et `temps_CPU` (temps nécessaire à son exécution).

4. Recopier et compléter le constructeur de la classe `Processus` :

```
class Processus:
    ...(self, ..., priorite, temps_CPU):
        ... priorite = priorite
        ... PID = ...
        self.temps_utilisation = 0
        self.temps_CPU = temps_CPU
```

5. On considère les trois processus suivants (à l'appel, on peut **nommer** les arguments, `PID=1` par exemple : c'est plus lisible, et l'ordre n'importe plus) :

```
P1 = Processus(PID=1, priorite=0, temps_CPU=10)
P2 = Processus(PID=2, priorite=0, temps_CPU=7)
P3 = Processus(PID=3, priorite=0, temps_CPU=5)
```

On a donc `liste_files = [[P3, P2, P1], [], []]` (dans chaque file, la tête est à droite). Recopier et compléter la simulation suivante, où `CPU` désigne le processus en cours d'exécution :

```
Cycle 1: CPU=P1     liste_files=[[P3, P2], [], []]
Cycle 2: CPU=P2     liste_files=[[P3], [P1], []]
Cycle 3: CPU=P3     liste_files=[[], [...], []]
Cycle 4: CPU=P3     liste_files=[[], [...], []]
Cycle 5: CPU=...    liste_files=[[], [...], [...]]
```

Pour les questions 6 et 7, on dispose d'un processus qui nécessite un temps d'utilisation de 1000 pour se terminer, et d'un grand nombre de processus dont le temps d'utilisation est de 4 ; chaque processus court terminé est remplacé par un nouveau processus similaire.

6. Expliquer pourquoi le processus long risque de ne **jamais** se terminer avec ce programme (indiquer notamment sa priorité au bout de quelque temps).

Pour régler ce phénomène, on ajoute au processus l'attribut `temps_d_attente`, et on définit une constante `Max_Temps`, le temps maximal qu'un processus attend avant de voir sa priorité remonter : à chaque cycle, le `temps_d_attente` augmente, et s'il dépasse `Max_Temps`, la **valeur de priorité** du processus **diminue** (il redevient plus prioritaire).

7. Expliquer pourquoi le processus long ne risque plus de ne jamais se terminer.

8. ▶ Écrire une fonction `meilleur_priorite(liste_files)` qui renvoie `None` s'il n'y a plus de processus, et sinon la valeur de priorité de l'un des processus les plus prioritaires de la liste des files. Par exemple, si `p1`, `p2`, `p3` sont des instances de `Processus` et `liste_files = [[], [p2], [p3, p1]]`, l'appel `meilleur_priorite(liste_files)` renvoie `1`.

   ▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

9. ▶ Écrire une fonction `prioritaire(liste_files)` qui renvoie `None` si aucune file ne contient de processus, et sinon renvoie le processus **en tête de la première file non vide** (rappel : la tête est à droite), en le **supprimant** de cette file. On pourra utiliser `liste.pop(i)`, qui renvoie l'élément d'indice `i` de la liste tout en le supprimant.

10. ▶ Écrire une fonction `gerer(p, liste_files)` qui prend le processus en cours d'exécution `p` (ou `None`) et la liste des files d'attente, qui implémente **un cycle** du programme décrit en début d'énoncé, et qui renvoie le processus en cours d'exécution à l'issue de ce cycle (ou `None`). ▷ coup de pouce ▷ début de solution

Critiquer une réponse d'IA ↑cours

★★ **Exercice 16** *Les états d'un processus expliqués par un assistant d'IA* ▶ corrigé p. 819

Un élève demande à un assistant d'IA : « Explique-moi les trois états d'un processus et les passages de l'un à l'autre. » Voici la réponse obtenue :

Réponse de l'assistant

« Un processus est un programme en cours d'exécution. Il passe par trois états. **Élu** : il utilise le processeur ; il n'y a qu'un seul processus élu à la fois par cœur. **Prêt** : il pourrait s'exécuter mais attend son tour, le processeur étant occupé. **Bloqué** : il attend une ressource (lecture disque, saisie clavier…) et ne peut pas avancer même si le processeur se libère. Les transitions sont les suivantes : l'ordonnanceur *élit* un processus prêt (prêt → élu) ; quand son quantum est écoulé, il est *réquisitionné* (élu → prêt) ; s'il demande une ressource indisponible, il se

> *bloque* (élu → bloqué) ; enfin, dès que la ressource attendue arrive, le processus bloqué *reprend immédiatement le processeur* (bloqué → élu), puisqu'il a déjà assez attendu. Un processus ne peut se terminer que depuis l'état élu. »

1. La réponse est-elle correcte ? Pour le vérifier, considérer un cœur où P_1 est élu et P_2 bloqué en attente d'une lecture disque ; la lecture se termine. Que se passerait-il si la description de l'assistant était vraie ? Est-ce compatible avec « un seul élu à la fois » ?
2. Localiser et corriger la phrase erronée (on redessinera l'automate des états).
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

▷ coup de pouce

S'entraîner à l'oral

↑ cours

★★ **Exercice 17** *Expliquer en deux minutes* ▶ corrigé p. 819

Au Grand Oral comme devant l'examinateur de l'épreuve pratique, il faut savoir **expliquer** une notion clairement, sans lire ses notes. Choisir l'un des sujets suivants (ou le tirer au sort) :

Sujet 1. Expliquer à un camarade la différence entre un **programme** et un **processus**.
Sujet 2. Expliquer comment un ordinateur qui n'a qu'un seul processeur donne l'impression d'exécuter plusieurs programmes en même temps.
Sujet 3. Expliquer ce qu'est un **interblocage**, avec un exemple concret.

Préparation (5 min, seul). Noter au brouillon **trois idées** et **un exemple**, rien de plus : comme au Grand Oral, ces notes servent d'aide-mémoire. **Exposé (2 min, chronométré).** Debout, face à un camarade, **sans lire ses notes** et sans rien écrire : tout passe par la parole. **Retour (2 min).** Le camarade pose une question de relance (on peut alors s'aider d'un schéma), remplit la grille ci-dessous et donne un conseil ; puis on échange les rôles. Cette grille reprend les critères d'évaluation du Grand Oral.

▷ coup de pouce

Grille entre camarades	**Oui**	**En partie**	**Pas encore**
Clair — audible, posé ; chaque mot technique est expliqué	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	☐	☐	☐
Construit — un fil conducteur, tenu en deux minutes, sans lire	☐	☐	☐
Un conseil pour la prochaine fois :			

Entraînement

↑ cours

Des séries courtes pour rendre une méthode automatique : même consigne, données différentes. Les réponses sont brèves dans le corrigé.

★ **Exercice 18** *Chronogrammes en série* ▶ corrigé p. 820 • essentiel

Pour chacun des trois jeux de données ci-dessous, tracer le **chronogramme**, puis calculer le **temps d'attente** de chaque processus et le **temps d'attente moyen** (temps d'attente = instant de fin − instant d'arrivée − durée).

▷ coup de pouce

a) Premier arrivé, premier servi.

Processus	P1	P2	P3	P4
Instant d'arrivée	0	1	2	4
Durée	5	2	3	1

b) Tourniquet de quantum 2. On applique la convention du cours : un processus qui arrive à l'instant où un quantum se termine entre dans la file **avant** le processus qui vient d'être interrompu.

Processus	R1	R2	R3	R4
Instant d'arrivée	0	1	2	4
Durée	4	3	2	1

c) Par priorités, sans préemption. Chaque fois que le processeur se libère, on élit, parmi les processus déjà arrivés, celui dont la **valeur de priorité** est la plus petite (la valeur 1 est la plus urgente) ; à valeur égale, on élit le premier arrivé. L'élu s'exécute jusqu'au bout.

Processus	S1	S2	S3	S4
Instant d'arrivée	0	1	2	3
Durée	3	2	4	1
Valeur de priorité	3	1	2	1

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Relire la définition du cours. Pour chaque phrase, se demander s'il s'agit d'un texte inerte, ou de quelque chose qui est en train de s'exécuter (et qui a donc un état à un instant donné). ↑énoncé

Ex. 2 – piste Pour la question 2, se demander pour chaque événement dans quel état se trouve le processus juste avant, et dans quel état juste après : la transition est la flèche qui relie les deux. Pour la question 3, qui décide de donner le processeur à un processus ? ↑énoncé

Ex. 3 – piste Repérer dans l'énoncé chaque événement (démarrage, demande de lecture, arrivée de la donnée, fin) et chercher, pour chacun, la flèche de l'automate des états qu'il déclenche. Se demander ce que devient P quand sa donnée arrive alors que le processeur est occupé. ↑énoncé

Ex. 4 – piste Commencer par l'état où arrive la flèche « création » : un processus qui vient d'être créé s'exécute-t-il aussitôt, ou attend-il son tour ? Puis, pour chaque flèche, chercher l'événement qui la déclenche. ↑énoncé

Ex. 5 – piste Le PPID d'un processus est le PID de son père, c'est-à-dire du nœud situé juste au-dessus de lui dans l'arbre. Pour la question 4, relire dans le cours ce que devient un processus dont le père disparaît. ↑énoncé

Ex. 6 – piste La colonne PPID donne le père de chaque processus : partir de la racine (le processus dont le père n'est pas dans le tableau), puis placer sous chaque nœud tous les processus dont le PPID est son PID. ↑énoncé

Ex. 7 q. 3 – piste Après un `fork`, *chacun* des processus existants exécute la ligne suivante, donc aussi le `fork` suivant. Compter les processus ligne par ligne et observer comment ce nombre évolue à chaque `fork`. ↑énoncé

Ex. 8 q. 4 – piste Tracer d'abord le chronogramme, puis remplir un petit tableau par processus : instant de fin, temps de séjour, temps d'attente. Pour le plus court d'abord, ne comparer que les processus *déjà arrivés* au moment où le processeur se libère. ↑énoncé

Ex. 9 q. 2 – piste Question 2 : simuler la file à la main : écrire son contenu après chaque passage au processeur, sans oublier qu'un processus qui n'a pas fini repart *en queue* de file. ↑

Ex. 9 q. 4 – piste Question 4 : se demander dans quels *deux* cas le processus doit rendre le processeur : ce sont les deux conditions de la boucle intérieure. Le test qui suit la boucle décide s'il faut remettre le processus dans la file. ↑énoncé

Ex. 9 q. 4 – début de solution Question 3 : `execute_un_cycle` diminue `self.reste_a_faire` de 1 et `est_termine` renvoie une comparaison avec 0. Question 4 : la boucle intérieure commence par `while compteur < quantum and ...` ; le test qui la suit porte sur `processus.est_termine()`. ↑énoncé

Ex. 10 q. 3 – piste Relire le sens des deux sortes de flèches, puis suivre les flèches en partant de P_1 : où arrive-t-on ? Pour la question 3, se demander ce qui pourrait obliger l'un des processus à rendre sa ressource. ↑énoncé

Ex. 11 q. 2 – piste Question 1 : faire avancer P_1 et P_2 en alternance (une demande chacun) et dessiner le graphe d'attente au fur et à mesure. Question 2 : pour chacune des quatre conditions, se demander si elle peut encore être réalisée quand tout le monde demande I avant S. ↑énoncé

Ex. 11 q. 2 – début de solution Début du scénario : P_1 demande I et l'obtient ; puis P_2 demande S et l'obtient ; puis… (continuer avec la deuxième demande de chacun). ↑

Ex. 12 q. 7 – piste Partie A : pour les questions 3 et 5, n'utiliser que la colonne PPID (compter ses répétitions ; remonter de père en père). Question 7 : écrire le contenu de la file d'attente à chaque fin de quantum, en y ajoutant d'abord les processus arrivés entre-temps. ↑énoncé

Ex. 13 q. 4 – piste Question 3 : à chaque cycle, enfiler d'abord le processus créé à ce cycle, puis défiler et exécuter la tête de file. Question 4 : se demander quand le programme doit s'arrêter : il faut qu'il ne reste *rien à créer* et *rien dans la file*. ↑énoncé

Ex. 13 q. 4 – début de solution Question 4 : commencer par `ordo = Ordonnanceur()`, puis une boucle `while` qui tourne tant que `ordo.temps` n'a pas dépassé le dernier cycle de création *ou* que la file n'est pas vide ; dans la boucle, tester `if ordo.temps in depart_proc:`. ↑énoncé

Ex. 13 q. 5 – piste Question 5 : suivre le premier chronogramme cycle par cycle en notant, à chaque cycle, l'instruction exécutée et qui détient quelle ressource. Chercher deux processus qui attendent chacun une ressource tenue par l'autre. ↑énoncé

Ex. 14 q. 4 – piste Question 4 : il suffit d'*un* scénario : chercher deux processus qui détiennent chacun une ressource que l'autre demande ensuite. ↑énoncé

Ex. 14 q. 7 – piste Question 5 : écrire le contenu de la file à chaque fin de quantum, arrivées comprises. Question 7 : comparer le temps restant au quantum ; il y a deux cas, selon que le processus termine ou non pendant ce quantum. ↑énoncé

Ex. 14 q. 7 – début de solution Question 7 : le test est `if processus['temps'] > quantum:` ; dans ce cas, on retire un quantum au temps restant et on remet le processus en queue de file. Question 8 : la boucle tourne tant que la file n'est pas vide, et `t` reçoit la valeur renvoyée par `execute_un_processus`. ↑

Ex. 15 q. 10 – piste Question 5 : appliquer la règle cycle par cycle en notant la priorité de chaque processus. Question 8 : la valeur de priorité d'un processus est l'indice de sa file. Question 9 : réutiliser `meilleur_priorite` ; dans la simulation, la tête de chaque file est à droite. Question 10 : suivre l'énoncé ligne à ligne, un `if` par cas. ↑

Ex. 15 q. 10 – début de solution Question 8 : `for i in range(len(liste_files)):` puis renvoyer `i` dès qu'une file est non vide. Question 10 : commencer par `if p is None:` (processeur libre : on élit un processus avec `prioritaire`), puis traiter le cas où `p` a terminé. ↑énoncé

Ex. 16 q. 3 – piste Reprendre l'automate des états du cours et confronter, une par une, chaque transition décrite par l'assistant. Pour la question 1, décrire l'état de P_1 et de P_2 juste après la fin de la lecture. ↑énoncé

Ex. 17 – piste Structurer chaque explication en trois temps : une **définition** en une phrase, un **exemple** petit et concret déroulé à voix haute, puis le **piège** (l'erreur que l'auditeur risque de faire). Finir par une phrase qui répond à la question posée. Sujet 2 : nommer

les trois états d'un processus et dire qui décide du passage de l'un à l'autre. Sujet 3 : chercher deux processus et deux ressources. ↑ énoncé

Ex. 18 – piste Avancer dans le temps : à chaque fois que le processeur se libère (ou qu'un quantum se termine), écrire la liste des processus déjà arrivés et non terminés, puis appliquer la règle de la stratégie pour choisir l'élu. Pour le tourniquet, écrire le contenu de la file à chaque fin de quantum. Remplir enfin un petit tableau : instant de fin, temps d'attente. ↑ énoncé

TP ET PROJETS

TP Observer, ordonnancer, bloquer ▶ corrigé p. 820

les processus dans un terminal, puis en Python

Durée : 2 h 30 à 3 h (deux séances) · Sur machine, par deux

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t04-tp-processus` — **archive .zip**

Consignes

- Partie A : un **terminal** Linux (celui du lycée, ou **JSLinux** dans le navigateur : `bellard.org/jslinux`, machine « Alpine Linux », console). Parties B et C : Python (éditeur au choix).
- Fichier à télécharger (lien ci-dessus) : `tp_processus_depart.py` (squelettes des fonctions de la partie B et tests automatiques).
- Niveau : ★ échauffement ★★ classique ★★★ défi (dont les extraits de bac les plus exigeants) ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme. Le symbole ▶ signale ce qui est à taper, programmer ou tester.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

But du TP

Mettre en pratique **tout le chapitre** : **observer** de vrais processus (PID, PPID, arrière-plan, `kill`), **programmer un simulateur d'ordonnancement** (premier arrivé, plus court d'abord, tourniquet) qui trace un **diagramme de Gantt** et calcule les temps moyens, puis **provoquer, guérir et éviter un interblocage** avec deux fils d'exécution et deux verrous. *Produit final* : un simulateur qui compare les stratégies sur un même jeu de processus, et un bilan chiffré.

Partie A — Observer et gérer les processus dans un terminal

★ **Exercice 1** *Premiers pas : qui suis-je ?*

▶ Ouvrir un terminal et taper les commandes suivantes, une par une.

```
$ ps
$ echo $$
$ ps -o pid,ppid,comm
```

1. `echo $$` affiche le PID du **shell** (l'interpréteur de commandes dans lequel on tape). Le retrouver dans la sortie de `ps`.
2. Dans la sortie de `ps -o pid,ppid,comm`, quel est le PPID de la ligne `ps` ? Expliquer pourquoi la commande `ps` apparaît **dans sa propre sortie**.

3. Retaper `ps -o pid,ppid,comm` : le PID de `ps` a-t-il changé ? Pourquoi ?

★ Exercice 2 — *Lire une sortie de* `ps -ef`

Sur une autre machine, on a obtenu la sortie suivante (colonnes utiles : **PID**, **PPID** et **CMD**, la commande lancée).

```
$ ps -ef
UID     PID  PPID  STIME  TTY    CMD
root      1     0  09:14  ?      /sbin/init
root    412     1  09:14  ?      /usr/sbin/sshd
root    455     1  09:14  ?      /usr/sbin/cron
mathis  843     1  09:15  tty1   -bash
mathis  870   412  09:16  ?      firefox
mathis  872   870  09:16  ?      firefox --tab
mathis  901   843  09:20  tty1   python3 tri.py
mathis  902   843  09:21  tty1   ps -ef
```

1. Dessiner l'**arbre des processus** (un nœud par PID, avec le nom de la commande), de racine `init`.
2. Qui est le père de `python3 tri.py` ? Depuis quoi l'utilisateur l'a-t-il lancé ?
3. Quels processus sont des **frères** de `ps -ef` ? Quel processus a le plus de descendants ?
4. `init` a pour PPID 0 : que signifie ce 0 ?

★ Exercice 3 — *Premier plan, arrière-plan,* `kill`

La commande `sleep 300` ne fait rien pendant 300 secondes : un processus « bloqué » idéal pour l'expérience.

1. ▶ Taper `sleep 5`. Peut-on taper une autre commande pendant ces 5 secondes ? On dit que `sleep` s'exécute au **premier plan**.

2. ▶ Taper `sleep 300 &`, puis `sleep 400 &`, puis `jobs` et `ps -o pid,ppid,comm`. Que change le `&` ? Quel est le PPID des deux `sleep` ? Comparer avec `echo $$`.

3. ▶ Arrêter le premier `sleep` avec `kill <PID>` (en remplaçant `<PID>` par son numéro), puis le second avec `kill %2` (`%2` désigne la tâche numéro 2 de `jobs`). Vérifier avec `jobs` et `ps`.

4. Recopier le **tableau-mémo** des commandes ; le compléter maintenant pour les quatre premières, et pour `top` à l'exercice suivant :

Commande	Rôle	Exemple tapé
`ps -o pid,ppid,comm`		
`<commande> &`		
`jobs`		
`kill <PID>`		
`top`		

★★ Exercice 4 — *Une famille de processus, et un glouton*

1. ▶ Taper `sh` : cela lance un **nouveau** shell, fils du premier. Taper `echo $$`, puis `sleep 500 &`, puis `ps -o pid,ppid,comm`. Écrire la **chaîne** des PID depuis le premier shell jusqu'au `sleep` (père → fils → petit-fils). Taper ensuite `exit` pour revenir au premier shell. *(Si la commande* `pstree` *existe sur la machine, l'essayer.)*

2. ▶ Lancer `yes > /dev/null &` (`yes` écrit des « y » sans fin, envoyés à la poubelle `/dev/null`), puis `top`. Quel processus est en tête ? Quel pourcentage du processeur utilise-t-il ? Quitter `top` avec la touche `q`.

3. Dans `top`, la colonne `S` (ou `STAT`) donne l'état : `R` (*running*) ou `S` (*sleeping*). Associer chacune de ces deux lettres à un état du cours (élu/prêt, ou bloqué). Dans quel état est `sleep 500` ? et `yes` ?

4. ▶ Arrêter `yes` avec `kill`, et vérifier dans `top` que le processeur est libéré. **Ne jamais** tuer un processus dont on ne connaît pas le rôle (surtout pas le PID 1 !).

Partie B — Programmer un simulateur d'ordonnancement

Un processus est représenté par un triplet `(nom, arrivee, duree)`. Un **chronogramme** est une liste dont la case d'indice `t` contient le nom du processus élu pendant l'unité de temps $[t\,;\,t+1[$, ou `"."` si le processeur n'a rien à faire. Par exemple, pour le jeu du cours `JEU_COURS = [("A", 0, 4), ("B", 1, 3), ("C", 2, 1), ("D", 3, 2)]`, la stratégie « premier arrivé, premier servi » donne :

```
["A", "A", "A", "A", "B", "B", "B", "C", "D", "D"]
```

On rappelle : **temps de séjour** = instant de fin − instant d'arrivée ; **temps d'attente** = temps de séjour − durée. Lancer le fichier `tp_processus_depart.py` affiche l'état des tests (`OK`, `ECHEC`, `ERREUR`) : les faire passer un à un.

★ Exercice 5 — *Premier arrivé, premier servi*

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1. ▶ Compléter `fcfs(procs)`. Les processus sont servis par ordre d'arrivée, chacun jusqu'au bout. **Attention** : si le prochain processus n'est pas encore arrivé, le processeur attend (on ajoute des `"."`). Exemple : `fcfs([("X", 2, 1)])` renvoie `[".", ".", "X"]`. ▷ coup de pouce

2. ▶ Compléter `gantt_ligne(chrono)`, qui renvoie le chronogramme sous forme de chaîne (`"AAAABBBCDD"`). Comparer au chronogramme du cours.

★ Exercice 6 — *Mesurer : temps de séjour et d'attente*

1. ▶ Compléter `statistiques(chrono, procs)`, qui renvoie un dictionnaire `nom -> (sejour, attente)`. L'instant de fin d'un processus est `i + 1`, où `i` est le **dernier** indice où il apparaît dans le chronogramme. Exemple : pour FCFS sur `JEU_COURS`, `C` a pour valeur `(6, 5)`.

2. ▶ Compléter `moyennes(stats)`, qui renvoie le couple (séjour moyen, attente moyenne). Retrouver l'attente moyenne 3,25 du cours.

★★ **Exercice 7** *Le plus court d'abord*

▶ Compléter `sjf(procs)` (version **non préemptive**) : chaque fois que le processeur se libère, on élit, parmi les processus **déjà arrivés** et non traités, celui de plus petite durée, et on l'exécute jusqu'au bout. S'il n'y a aucun processus prêt, on ajoute un `"."`. Vérifier qu'on retrouve `"AAAACDDBBB"` et l'attente moyenne 2,5 du cours.

★★ **Exercice 8** *Le tourniquet*

1. ▶ Compléter `tourniquet(procs, quantum)`. La file des processus prêts est une liste : on retire en tête (`pop(0)` : choix simple mais coûteux, en $O(n)$, cf. chapitre *Structures linéaires*), on ajoute en queue (`append`). L'élu s'exécute `min(quantum, reste)` unités. **Convention du cours** : les processus arrivés *pendant* ce quantum entrent dans la file **avant** que l'élu, s'il n'a pas fini, n'y retourne. ▷ coup de pouce

2. Vérifier : quantum 1 → `"ABACBDABDA"` (le chronogramme du cours) ; quantum 2 → `"AABBCAADDB"`.

3. ▶ Compléter `commutations(chrono)` : le nombre de fois où le processeur passe d'un processus à un **autre** processus (les `"."` ne comptent pas). Exemple : `commutations([c for c in "AAB..BCA"])` vaut 3. Combien de commutations pour le tourniquet de quantum 1 sur `JEU_COURS` ? et pour FCFS ?

★★ **Exercice 9** *Un vrai diagramme de Gantt*

▶ Compléter `gantt(chrono, procs)`, qui renvoie une chaîne de plusieurs lignes : une ligne par processus, avec `#` quand il est élu, `.` quand il **attend** (arrivé mais pas fini), une espace sinon ; puis un axe gradué de 5 en 5. On l'affiche avec `print(gantt(tourniquet(JEU_COURS, 2), JEU_COURS))` :

```
A  |##...##   |
B  | .##.....#|
C  |  ..#     |
D  |   ....## |
    0    5    10
```

Que représente le nombre de `.` d'une ligne ? ▷ coup de pouce

★★ **Exercice 10** *Campagne de mesures*

On travaille maintenant sur le jeu `JEU_TP` du fichier (six processus, dont deux arrivent tard) :

Processus	A	B	C	D	E	F
Arrivée	0	1	2	4	18	19
Durée	8	2	5	1	3	1

1. ▶ Afficher le diagramme de Gantt de FCFS. Que se passe-t-il entre les instants 16 et 18 ?

2. ▶ Recopier et compléter le tableau (valeurs arrondies au centième) :

Stratégie	**Séjour moyen**	**Attente moyenne**	**Commutations**
Premier arrivé (FCFS)			
Plus court d'abord (SJF)			
Tourniquet, quantum 1			
Tourniquet, quantum 2			
Tourniquet, quantum 4			

3. ▶ Écrire une boucle qui affiche, pour un quantum allant de 1 à 8, l'attente moyenne et le nombre de commutations du tourniquet. Décrire l'évolution. Avec un quantum de 8, à quelle stratégie le tourniquet ressemble-t-il ici ?

4. Dans un vrai système, chaque commutation de contexte **coûte** du temps (sauvegarder puis restaurer les registres). Expliquer pourquoi on ne choisit ni un quantum minuscule, ni un quantum énorme.

★★★ **Exercice 11** *Défi : le plus court temps restant d'abord*

Variante **préemptive** du plus court d'abord (*SRTF*) : **à chaque unité de temps**, on élit le processus prêt dont le temps **restant** est le plus petit — quitte à interrompre l'élu si un plus court arrive. **À égalité**, on élit le premier arrivé.

1. ▶ Écrire `srtf(procs)`. Sur `JEU_COURS`, on doit obtenir `"AACAADDBBB"`.

2. ▶ Comparer son attente moyenne sur `JEU_TP` à celles du tableau. Quel processus en fait les frais ? Quel risque du cours retrouve-t-on ?

Partie C — Provoquer, guérir et éviter un interblocage [au-delà du programme]

En Python, le module `threading` permet de lancer plusieurs **fils d'exécution** (*threads*) qui avancent « en même temps », comme des processus. Un **verrou** (`threading.Lock()`) modélise une ressource qu'un seul fil peut détenir : `with verrou:` attend de l'obtenir, puis le libère automatiquement à la fin du bloc.

★ **Exercice 12** *Provoquer un interblocage (sans rester bloqué !)*

▶ Recopier et exécuter le programme suivant.

```
import threading, time

imprimante = threading.Lock()
scanner = threading.Lock()

def p1():
    with imprimante:
        print("P1 tient l'imprimante, demande le scanner")
        time.sleep(0.1)
        with scanner:
            print("P1 imprime et scanne")

def p2():
    with scanner:
        print("P2 tient le scanner, demande l'imprimante")
        time.sleep(0.1)
        with imprimante:
            print("P2 imprime et scanne")

t1 = threading.Thread(target=p1, daemon=True)
t2 = threading.Thread(target=p2, daemon=True)
t1.start()
t2.start()
t1.join(timeout=2)      # on attend P1 au plus 2 secondes
t2.join(timeout=2)
if t1.is_alive() and t2.is_alive():
    print("Au bout de 4 s, P1 et P2 sont toujours bloqués : interblocage !")
else:
    print("Tout s'est bien terminé.")
```

1. Quels messages s'affichent ? Les lignes `"... imprime et scanne"` apparaissent-elles ?

2. Dessiner le **graphe d'attente** au moment du blocage, et y repérer le cycle.
3. À quoi sert `time.sleep(0.1)` ? Relancer après l'avoir supprimé des deux fonctions (plusieurs fois) : l'interblocage se produit-il toujours ? Qu'en déduire sur ce type de panne ?
4. Sans `timeout` ni `daemon=True`, le programme ne s'arrêterait jamais. Expliquer le rôle de chacun.

★★ **Exercice 13** *Guérir : abandonner au bout d'un délai*

La méthode `verrou.acquire(timeout=1)` tente d'obtenir le verrou pendant au plus 1 seconde : elle renvoie `True` si c'est réussi, `False` sinon ; on libère alors avec `verrou.release()`.

1. ▶ Écrire une fonction `tache(nom, premier, second, journal)` qui prend le verrou `premier` (avec `with`), attend 0,1 s, puis **tente** d'obtenir `second` pendant 1 s : en cas de succès, ajouter à la liste `journal` le message `nom + " a les deux ressources"` puis libérer `second` ; sinon ajouter `nom + " abandonne"`.

2. ▶ Lancer deux fils : `tache("P1", imprimante, scanner, journal)` et `tache("P2", scanner, imprimante, journal)` (utiliser `args=(...)` dans `threading.Thread`), attendre leur fin avec `join()` et afficher `journal`. Relancer cinq fois : le résultat est-il toujours le même ?

3. Laquelle des quatre conditions de Coffman [au-delà du programme] l'abandon casse-t-il ?

★★ **Exercice 14** *Éviter : toujours le même ordre*

▶ Relancer l'exercice précédent en faisant demander à **P2 aussi** l'imprimante **avant** le scanner. Que contient le journal ? Combien de temps dure l'exécution (mesurer avec `time.time()` avant et après) ? Expliquer pourquoi, avec un **ordre fixe** d'acquisition, un cycle d'attente est impossible.

★★ **Exercice 15** *Détecter : chercher un cycle dans le graphe d'attente*

On décrit une situation par deux dictionnaires : `detient` associe à chaque ressource le processus qui la détient, `attend` associe à un processus la ressource qu'il attend.

```
detient = {"R1": "P1", "R2": "P2", "R3": "P3"}
attend = {"P1": "R2", "P2": "R3", "P3": "R1"}
```

1. Dessiner le graphe d'attente. Y a-t-il interblocage ?
2. ▶ Écrire `detecte_interblocage(detient, attend, depart)` qui suit les flèches à partir du processus `depart` (processus → ressource attendue → processus qui la détient → ...) et renvoie `True` si l'on revient sur un processus déjà rencontré, `False` si la chaîne s'arrête (processus qui n'attend rien, ou ressource libre).
3. ▶ Tester sur l'exemple, puis après avoir supprimé l'attente de `P3`.

★★★ **Exercice 16** *Défi : le dîner des philosophes*

Cinq philosophes sont assis autour d'une table ronde ; entre deux voisins, une seule fourchette (5 en tout). Pour manger, le philosophe `i` doit tenir les fourchettes `i` et `(i + 1) % 5`.

1. ▶ Programmer 5 fils (une liste de 5 verrous) : chaque philosophe essaie 3 fois de manger en prenant d'abord la fourchette `i`, en attendant 0,05 s, puis la fourchette `(i + 1) % 5` avec `acquire(timeout=0.5)`. Compter les repas réussis et les abandons.
2. ▶ Imposer l'ordre : chacun prend d'abord la fourchette de **plus petit numéro**. Comparer. Quel philosophe change de comportement ?

Bilan du TP

À rédiger (une dizaine de lignes)

1. Avec vos mesures, classer les stratégies selon l'attente moyenne sur `JEU_TP`. Laquelle choisir pour un serveur de calcul ? pour un ordinateur de bureau ? Justifier.
2. Expliquer le compromis du quantum (réactivité contre coût des commutations).
3. Un interblocage est-il une erreur dans *un* programme ? Citer les trois attitudes possibles vues dans la partie C (le subir, le guérir, l'éviter) et ce qu'elles coûtent.
4. Votre simulateur est écrit en Python : pourquoi un vrai ordonnanceur ne l'est-il pas ?

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 5 q. 1 – piste Écrire d'abord une fonction `trier_par_arrivee(procs)` qui renvoie une copie de la liste triée par instant d'arrivée (tri par insertion de 1re, en comparant les éléments d'indice `1` des triplets). Puis garder une variable `t` (l'instant courant) : tant que `t` est inférieur à l'arrivée du processus, ajouter `"."` et avancer `t`. ↑énoncé

Ex. 8 q. 1 – piste Utiliser un dictionnaire `reste` (nom → durée restant à faire) et la liste `a_venir = trier_par_arrivee(procs)`. Écrire une boucle `while a_venir != [] or file != []` ; au début de chaque tour, faire entrer dans la file tous les processus de `a_venir` dont l'arrivée est $\leqslant$ `t`. ↑énoncé

Ex. 9 – piste Pour chaque processus, calculer d'abord son instant de fin ; puis parcourir `t` de `0` à `len(chrono) - 1` et choisir le caractère à ajouter. Pour aligner, écrire une fonction `completer(texte, n)` qui ajoute des espaces à droite de `texte` tant qu'il a moins de `n` caractères : `completer(nom, 3)` pour les noms ; pour l'axe, `completer(str(t), 5)` pour `t` allant de 0 à `len(chrono)` de 5 en 5. ↑énoncé

FICHE DE RÉVISION

L'essentiel

- Un **processus** est un **programme en cours d'exécution** (avec son état, sa mémoire). Le **système d'exploitation** donne à un seul processeur l'illusion de tout faire « en même temps » en **basculant** très vite entre les processus.
- **Automate des trois états** d'un processus : **élu** (il s'exécute), **prêt** (il attend le processeur), **bloqué** (il attend une ressource / une entrée-sortie). Transitions : élection, fin de quantum, blocage, déblocage.
- **Ordonnancer** = choisir quel processus prêt devient élu. Politiques classiques :
  - **premier arrivé, premier servi** (FCFS) : simple, mais un long processus bloque les autres ;
  - **plus court d'abord** (SJF) : réduit le temps d'attente moyen (il le minimise quand tous les processus sont présents au départ) ;
  - **tourniquet** (*round-robin*) : chacun son tour, par tranches de temps (**quantum**) : réactif ;
  - **par priorités** : l'urgent d'abord (au bac, la plus petite valeur ; à égalité, le premier arrivé) ; risque de famine.
- Un **chronogramme (diagramme de Gantt)** montre qui s'exécute à chaque instant ; on en déduit le **temps d'attente** de chaque processus (temps de rotation − durée) et leur **moyenne**.
- **Interblocage** (*deadlock*) : des processus s'attendent **mutuellement** (cycle dans le graphe d'attente) et sont bloqués pour toujours.

Observer et gérer les processus (Unix)

- `ps`, `ps -ef`, `pstree` : lister les processus (avec leur **PID** et **PPID**) ; `top` / `htop` : en temps réel.
- `kill PID` : demander l'arrêt ; `kill -9 PID` : forcer. `&` : lancer en arrière-plan ; `nice` : changer la priorité.
- `fork()` **duplique** le processus (père/fils) : n appels successifs $\Rightarrow 2^n$ processus. Chaque processus a un **PID** unique et connaît celui de son père (**PPID**).

À retenir aussi

- **Les 4 conditions de Coffman** [au-delà du programme] (toutes nécessaires à un interblocage) : exclusion mutuelle, détention et attente, non-préemption, attente circulaire. En casser *une* suffit à l'éviter.
- Un *vrai* ordonnanceur est écrit en **C** (et un peu d'assembleur pour la commutation de contexte) : en Python on **simule** l'algorithme, on ne le fabrique pas.
- Les processeurs **multi-cœurs** [au-delà du programme] exécutent réellement plusieurs processus en parallèle ; l'ordonnanceur répartit la charge sur les cœurs.

Erreurs à éviter

- confondre **prêt** (attend le processeur) et **bloqué** (attend une ressource) ;
- oublier qu'au tourniquet, un processus non terminé **repart en fin de file** après son quantum ;
- confondre **PID** (le processus lui-même) et **PPID** (son père) ;
- croire qu'un interblocage se « débloque tout seul » : non, il faut casser le cycle.

Savoir-faire à maîtriser — je sais…

- ☐ **distinguer** un programme d'un processus, et citer les grandes fonctions d'un système d'exploitation ;
- ☐ **nommer** les trois états et **compléter** le schéma des transitions ;
- ☐ **décrire** la création d'un processus (`fork`, père et fils) et compter les processus créés ;
- ☐ **lire** une sortie de `ps`, en déduire un arbre de processus et des PID / PPID, et connaître `top` / `kill` ;
- ☐ **dérouler** un ordonnancement (premier arrivé, plus court d'abord, tourniquet, par priorités) sous forme de **chronogramme** et en calculer les temps d'attente et le **temps d'attente moyen** ;
- ☐ **reconnaître** un interblocage, le représenter par un **graphe d'attente** et proposer un moyen de l'éviter.

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 828

1. Quelle différence entre l'état **prêt** et l'état **bloqué** ?
2. Un processus bloqué reçoit sa donnée. Dans quel état passe-t-il ?
3. Tourniquet de quantum 1 : A (durée 2) est entré dans la file avant B (durée 1). Ordre d'exécution ?
4. Combien de processus après 3 appels `fork()` successifs ?
5. Comment reconnaître un interblocage sur un graphe d'attente, et comment en sortir ?

Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 828

1. File vide `f` : `enfiler(4)`, `enfiler(9)`, `enfiler(1)`, `defiler()`. Que renvoie alors `tete()` ? (ch. 3)
2. Écrire le constructeur d'une classe `Processus` qui reçoit un `pid` et le range dans un attribut. (ch. 2)
3. Que se passe-t-il quand on appelle une fonction récursive sans condition d'arrêt ? (ch. 1)

Vers le bac

- Épreuve pratique : sujet 28 — Ordonnancement de tâches : le tourniquet, p. 735

DÉFIS ADVENT OF CODE

DÉFI 1 Seating System

la salle d'attente — automate cellulaire et copie de grilles

Jour 11

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t04-defi-aoc-2020-11` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/11`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. On connaît le plan d'une salle d'attente (environ 90 lignes de 95 cases) et une règle qui dit, pour chaque siège, s'il va se remplir ou se vider au tour suivant. Il faut simuler les tours jusqu'à ce que plus rien ne bouge, puis compter les sièges occupés. Chaque tour examine des milliers de cases, et il faut souvent une bonne centaine de tours avant la stabilité : seul un programme peut le faire.
Ce qu'on réinvestit. La notion d'**effet de bord** vue au chapitre 2 : la fonction `tour` doit renvoyer une nouvelle grille sans modifier l'ancienne. Le défi fait aussi écho à ce chapitre : toutes les cases changent « en même temps », et lire un état pendant qu'on le modifie donne des résultats incohérents, comme deux processus qui accèdent sans précaution à une même ressource.
La démarche de la fiche.

1. Comprendre la règle (voisins, seuils, changement simultané), puis simuler trois tours à

la main sur une petite grille.
2. Lire la grille, puis compter les voisins occupés d'une case.
3. Écrire une fonction `tour` sans effet de bord et la répéter jusqu'à stabilité : réponse de la partie 1.
4. Pour la partie 2, changer seulement la façon de compter les voisins et le seuil, en passant une fonction en paramètre.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`seat layout`	plan des sièges	`floor`	sol (pas un siège)
`empty / occupied`	libre / occupé	`adjacent`	voisin, adjacent
`simultaneously`	en même temps	`round`	tour (de règles)
`become`	devenir	`stop changing`	cesser de changer

Résumé de l'énoncé (partie 1)

Contexte. Vous attendez un ferry dans une salle encore vide et vous voulez prévoir où les voyageurs vont s'asseoir, pour choisir la meilleure place. Leur comportement est très prévisible : on le simule comme un *automate cellulaire*, où l'état de chaque case au tour suivant ne dépend que de son état et de celui de ses voisines.
Les données. Le fichier est le plan de la salle : toutes les lignes ont la même longueur, et chaque caractère est une case. `.` représente le sol, `L` un siège libre, `#` un siège occupé. Au départ, tous les sièges sont libres.
La règle.

- Les voisins d'une case sont les 8 cases qui l'entourent (même ligne, même colonne et diagonales), sans sortir du plan : une case du bord en a donc moins.
- À chaque tour, **toutes** les cases changent en même temps, d'après l'état du tour précédent.
- Un siège libre dont aucun voisin n'est occupé devient occupé (on aime être tranquille).
- Un siège occupé qui a au moins 4 voisins occupés se libère (trop de monde).
- Dans les autres cas, rien ne change ; le sol ne change jamais.
- On répète les tours jusqu'à ce qu'un tour ne modifie plus aucune case.

La réponse attendue. Le nombre de sièges occupés quand plus rien ne bouge. Par exemple, une salle de 3×3 sièges tous libres se remplit entièrement au premier tour ; au deuxième, seuls les 4 coins restent occupés, puis plus rien ne change : la réponse serait 4.

Exercice 1 *Comprendre ce qui est demandé*

But. Vérifier sur trois questions courtes qu'on a compris la notion de voisin et les seuils de la règle.
À quoi ça sert ? La troisième question annonce le piège principal du défi : une grille modifiée pendant qu'on la parcourt ne donne pas le bon résultat.

Questions de vérification, à traiter sur le cahier :

1. Combien de voisins a une case du bord qui n'est pas un coin ?
2. Un siège occupé qui a exactement 3 voisins occupés change-t-il d'état ? Et un siège libre qui en a 1 ?
3. Pourquoi ne peut-on pas modifier la grille au fur et à mesure qu'on la parcourt ?

Exercice 2 *À la main, sur un petit exemple*

But. Simuler à la main trois tours sur une grille de 5×5.
À quoi ça sert ? On obtient un exemple dont on **connaît la réponse** (11), et surtout les grilles intermédiaires : si le programme se trompe, on pourra comparer tour par tour pour trouver l'erreur.

On considère la grille (inventée) suivante :

```
L.LLL
LLLL.
L.L.L
LLLLL
..L.L
```

Dessiner la grille après un tour, après deux tours, puis après trois tours. Vérification : le quatrième tour ne change plus rien, et l'on compte alors `11` sièges occupés.

2 Programmer la partie 1

Exercice 3 *Lire la grille*

But. Transformer le fichier en une liste de listes de caractères.
À quoi ça sert ? Toute la simulation travaille sur cette structure : `grille[i][j]` donne la case de la ligne `i` et de la colonne `j`. Contrairement à une chaîne, une liste peut être modifiée case par case.

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Écrire une fonction `lire_grille(nom_fichier)` qui renvoie une liste de listes de caractères (une liste par ligne). Afficher le nombre de lignes et de colonnes.

Exercice 4 *Compter les voisins occupés*

But. Compter les sièges occupés autour d'une case, sans sortir de la grille.
À quoi ça sert ? C'est la seule information dont la règle a besoin pour décider du sort d'un siège. En faire une fonction à part permettra, en partie 2, de la remplacer par une autre façon de compter sans toucher au reste.

Écrire une fonction `voisins_occupes(grille, i, j)` qui renvoie le nombre de sièges occupés autour de la case (i, j). La tester sur une case du bord et sur une case du centre. ▷ coup de pouce

Méthode : compter des éléments avec un compteur

On parcourt la liste et l'on ajoute 1 à un compteur chaque fois que l'élément convient. Pour une grille (liste de listes), on ajoute les comptes de chaque ligne.

```
def compter(ligne, x):
    """Renvoie le nombre d'elements de ligne egaux a x."""
    total = 0
    for c in ligne:
        if c == x:
            total += 1
    return total
```

```
8
9  print(compter(["#", ".", "L", "#"], "#"))    # 2
10 print(compter("#.L##", "#"))                  # 3 : fonctionne aussi sur une chaine
11 grille = [["#", "L"], ["#", "#"]]
12 total = 0
13 for ligne in grille:
14     total += compter(ligne, "#")
15 print(total)                                  # 3 : somme des comptes de chaque ligne
```

Intérêt. Le schéma « compteur initialisé à 0, une boucle, un test » sert partout. Son coût est proportionnel à la longueur de ce qu'on parcourt : compter les sièges occupés d'une grille de $L \times C$ cases coûte $O(L \cdot C)$.
À essayer. Que renvoie `compter([1, 2, 1], 3)` ? Et `compter([], "#")` ?

4

Exercice 5 *Un tour, puis jusqu'à stabilité*

But. Calculer l'état de toute la grille au tour suivant, puis répéter jusqu'à stabilité.
À quoi ça sert ? C'est le cœur de la partie 1 : le nombre de sièges occupés dans la grille finale est la réponse. Écrire `tour` sans effet de bord garantit que toutes les cases changent bien en même temps.

Écrire une fonction `tour(grille)` qui renvoie la **nouvelle** grille sans modifier l'ancienne, puis répéter les tours tant que la grille change. Tester sur l'exemple inventé de la fiche, puis sur vos données. ▷ coup de pouce ▷ début de solution

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. En observant mieux, on comprend que les voyageurs ne regardent pas seulement les places juste à côté : ils regardent aussi plus loin, à travers les cases de sol.
La nouvelle règle.

- Dans chacune des 8 directions, on prend en compte le **premier siège visible** (libre ou occupé), en sautant les cases de sol.
- S'il n'y a que du sol jusqu'au bord dans une direction, cette direction ne compte pas.
- Un siège occupé se libère désormais s'il voit au moins **5** sièges occupés.
- La règle pour s'asseoir (aucun siège occupé visible) et le déroulement des tours ne changent pas.

La réponse attendue. Comme en partie 1 : le nombre de sièges occupés quand plus rien ne bouge.

★★ **Exercice 6** *Ce que l'on voit*

But. Compter les sièges visibles dans les 8 directions, puis rendre `tour` indépendante de la façon de compter.
À quoi ça sert ? En passant la fonction de comptage et le seuil en paramètres, une seule fonction `tour` sert aux deux parties : on ne réécrit que ce qui change vraiment.

Écrire une fonction `visibles_occupes` (mêmes paramètres que `voisins_occupes`), puis adapter `tour` pour qu'elle reçoive en paramètres la fonction de comptage et le seuil. Sur l'exemple inventé de la fiche, on doit trouver `9`. ▷ coup de pouce ▷ début de solution

Outil Python : mesurer une durée (module `time`)

La fonction `time.perf_counter()` renvoie un instant en secondes (avec beaucoup de décimales). La différence entre deux appels donne la durée écoulée entre les deux.

```python
import time

debut = time.perf_counter()      # instant de depart (en secondes)
total = 0
for i in range(1000000):
    total += i
duree = time.perf_counter() - debut
print(f"{duree:.3f} s")          # duree ecoulee, arrondie a 3 decimales
```

Intérêt. Vérifier *expérimentalement* une complexité : si doubler la taille des données double le temps, le programme est linéaire ; s'il le multiplie par 4, il est quadratique. Les durées varient d'une machine à l'autre : seuls les rapports comptent.
À essayer. Recopier et exécuter ce code, puis remplacer `1000000` par `2000000`. Comment évolue la durée ?

★★★ **Exercice 7**  *Défi : précalculer*

But. Calculer une fois pour toutes, au début, quels sièges chaque siège peut voir, puis mesurer le gain.
À quoi ça sert ? Une recherche refaite à l'identique à chaque tour est du temps perdu : la précalculer est une optimisation classique, et la mesure avec `time` montre si elle en vaut la peine.

Les voisins visibles d'un siège ne dépendent pas de l'état des sièges, seulement de la position du sol. Les calculer une seule fois au début (dictionnaire case → liste de cases) et mesurer le gain de temps avec le module `time`. ▷ coup de pouce

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 4 – piste Deux boucles sur les décalages `di` et `dj` dans `[-1, 0, 1]`, sans oublier d'exclure le cas `(0, 0)` et de vérifier qu'on ne sort pas de la grille. ↑énoncé

Ex. 5 – piste Attention : toutes les cases changent **en même temps**. Si l'on modifie la grille pendant qu'on la parcourt, les cases suivantes voient un mélange d'ancien et de nouvel état. Sur l'exemple, la première ligne deviendrait `#.#L#` au lieu de `#.###` après un tour. ↑énoncé

Ex. 5 – début de solution `copie = grille[:]` ne suffit pas : les lignes restent partagées. Construire une nouvelle liste de listes, par exemple par compréhension : `[ligne[:] for ligne in grille]`, ou remplir une grille neuve case par case. Deux listes de listes se comparent directement avec `==`. ↑énoncé

Ex. 6 – piste Pour chacune des 8 directions `(di, dj)`, on avance pas à pas tant qu'on est dans la grille et sur du sol ; on s'arrête au premier siège rencontré. ↑énoncé

Ex. 6 – début de solution Une fonction est une valeur comme une autre en Python : `tour(grille, visibles_occupes, 5)` est un appel tout à fait valide, et dans `tour` on appelle simplement le paramètre reçu. ↑énoncé

Ex. 7 – piste Le sol ne change jamais : la recherche dans les 8 directions peut être faite une fois pour toutes. ↑énoncé

DÉFI 2

Crab Combat

la bataille contre le crabe — files et récursivité

Jour 22

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t04-defi-aoc-2020-22` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/22`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Deux joueurs s'affrontent à un jeu de cartes proche de la bataille, jusqu'à ce que l'un des deux ait toutes les cartes. On veut le score du gagnant. Une partie dure des centaines de manches, et en partie 2 chaque manche peut déclencher une sous-partie complète : seule une simulation par programme permet d'aller au bout.
Ce qu'on réinvestit. Les **files** vues au chapitre 3 : chaque paquet de cartes est une file (on prend dessus, on remet dessous), comme celle des processus prêts dans l'ordonnancement en tourniquet de ce chapitre. La partie 2 réinvestit la **récursivité** du chapitre 1 : chaque sous-partie est un appel récursif, avec sa propre mémoire des situations déjà vues.
La démarche de la fiche.

1. Comprendre les règles d'une manche et du score, puis jouer quelques manches à la main.
2. Lire les deux paquets dans deux files.
3. Simuler la partie et calculer le score du gagnant (partie 1).
4. Écrire la version récursive du jeu, avec une mémoire des situations déjà vues par partie (partie 2).

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`deck`	paquet de cartes	`round`	manche, tour
`draw`	tirer, piocher	`top / bottom`	dessus / dessous (du paquet)
`higher-valued`	de plus grande valeur	`winner`	gagnant
`score`	score	`sub-game`	sous-partie (partie 2)
`recursive`	récursif	`infinite game`	partie sans fin

Résumé de l'énoncé (partie 1)

Contexte. Sur le radeau, l'ennui guette ; heureusement, un petit crabe est monté à bord et accepte de jouer aux cartes avec vous, à un jeu proche de la bataille. Avant de jouer, vous voulez prévoir l'issue de la partie à partir des paquets distribués.
Les données. Le fichier contient deux blocs séparés par une ligne vide : le paquet du joueur 1 (vous), puis celui du joueur 2 (le crabe). Chaque bloc commence par une ligne d'en-tête, puis donne une carte par ligne, la carte du **dessus** en premier. Les cartes sont des entiers, tous différents.
La règle.

- À chaque manche, chaque joueur retire sa carte du dessus ; la plus forte gagne la manche (il ne peut pas y avoir d'égalité).
- Le gagnant place les deux cartes sous son paquet : la sienne d'abord, puis celle de l'adversaire.
- La partie s'arrête quand un joueur a toutes les cartes.
- Score du gagnant : la carte du dessous est multipliée par 1, celle juste au-dessus par 2, etc., jusqu'à la carte du dessus, multipliée par le nombre de cartes ; on additionne ces produits. Plus une carte est haut dans le paquet, plus elle compte.

La réponse attendue. Le score du gagnant. Par exemple, un paquet gagnant 5, 3, 9 (dessus en premier) vaudrait $5 \times 3 + 3 \times 2 + 9 \times 1 = 30$.

Exercice 1 *Vérifier sa compréhension*

But. Vérifier sa lecture des règles d'une manche et du calcul du score.
À quoi ça sert ? La deuxième question évite l'erreur la plus fréquente (multiplier la carte du dessus par 1 au lieu du nombre de cartes) ; la troisième fournit un contrôle simple du programme : le nombre total de cartes ne doit jamais changer.

1. Une manche peut-elle se terminer par une égalité ?
2. Le gagnant a 10 cartes, la carte du dessus vaut 4 : que rapporte-t-elle au score ?
3. Le nombre total de cartes change-t-il au cours de la partie ?

Exercice 2 *À la main, sur un petit exemple*

But. Jouer les premières manches d'une petite partie en écrivant les paquets.
À quoi ça sert ? On voit concrètement où vont les cartes (sous le paquet, dans quel ordre) ; les valeurs données (18 manches, score 198) serviront à tester le programme.

On considère les paquets (inventés) suivants, le dessus étant écrit en premier :

```
Joueur 1 : 6 2 8 1
Joueur 2 : 4 7 3 5
```

Jouer les trois premières manches à la main en écrivant les deux paquets après chaque manche. Combien de cartes chaque joueur a-t-il alors ? Vérification : la partie complète dure `18` manches et le score du gagnant vaut `198`.

2 Programmer la partie 1

Méthode : une file avec une liste Python

Un paquet de cartes est une **file** : on retire au début (le dessus), on ajoute à la fin (le dessous). Une simple liste Python suffit : `pop(0)` retire et renvoie le premier élément, `append` ajoute à la fin.

```
paquet = [6, 2, 8, 1]       # dessus du paquet a gauche
x = paquet.pop(0)           # retire et renvoie 6
paquet.append(4)            # ajoute a la fin : [2, 8, 1, 4]
paquet.append(7)
paquet.append(3)
print(len(paquet), paquet[0])   # 6 2 : longueur, premier element
print(paquet[:2])           # [2, 8] : nouvelle liste des deux premiers
if paquet:                  # une liste vide vaut False
    print("non vide")
```

Intérêt et limite. C'est la file du cours, sans rien à programmer. Mais `pop(0)` décale tous les autres éléments d'une case : $O(n)$ opérations pour n éléments. Avec au plus 50 cartes, c'est négligeable ; pour de très longues files, on utiliserait la classe `File` du cours (deux piles) ou un indice de tête qui avance au lieu de décaler la liste, pour retirer en $O(1)$. Question : que contient `paquet` à la fin du code ? Que se passe-t-il si l'on appelle `pop(0)` sur une liste vide ?

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Exercice 3 *Une file pour chaque joueur*

But. Lire les deux paquets et les ranger chacun dans une file, dessus en premier.
À quoi ça sert ? Les deux opérations du jeu (prendre dessus, remettre dessous) sont exactement celles d'une file : avec la bonne structure, la simulation s'écrit en quelques lignes.

Lire le fichier et ranger les cartes de chaque joueur dans une **file** : on retire au début (dessus du paquet), on ajoute à la fin (dessous). Utiliser une liste Python (encadré ci-dessus), ou une classe `File` du cours.

▷ coup de pouce

Rappel : parcourir avec l'indice

Pour connaître à la fois la **position** et la **valeur** de chaque élément, on parcourt les indices avec `range(len(...))`.

```
paquet = [8, 6, 7]
for i in range(len(paquet)):
    print(i, paquet[i])    # 0 8, puis 1 6, puis 2 7
```

Intérêt. L'indice `i` dit où se trouve la carte dans le paquet : c'est lui qui fixe son coefficient dans le score. Question : écrire une boucle qui affiche `paquet[i] * (3 - i)` pour la liste `[8, 6, 7]`, puis qui calcule la somme de ces trois produits dans une variable `total`.

Exercice 4 *Jouer et compter*

But. Simuler la partie jusqu'au bout, puis calculer le score du paquet gagnant.
À quoi ça sert ? C'est la réponse de la partie 1. Séparer le jeu (`combat`) du calcul du score (`score`) permettra de réutiliser `score` telle quelle en partie 2.

Écrire une fonction `combat(p1, p2)` qui joue la partie et renvoie le paquet du gagnant, puis une fonction `score(paquet)`. Tester sur l'exemple, puis sur vos données. ▷ coup de pouce

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Le crabe a perdu et propose une revanche, à une version « récursive » du jeu : certaines manches se décident en jouant une partie plus petite.
La nouvelle règle.

- Avant chaque manche : si la même configuration des deux paquets est déjà apparue **dans cette partie**, le joueur 1 gagne aussitôt la partie. Cela empêche une partie de durer indéfiniment.
- Sinon, chacun tire sa carte du dessus.
- Si chaque joueur a encore au moins autant de cartes que la valeur qu'il vient de tirer, le gagnant de la manche est celui d'une sous-partie, jouée avec les mêmes règles et des copies de ses n cartes suivantes (n = valeur qu'il a tirée). Par exemple, un joueur qui tire un 3 et a encore au moins 3 cartes entre dans la sous-partie avec une copie de ses 3 cartes du dessus.
- Sinon, la plus forte carte gagne, comme en partie 1.
- Le gagnant de la manche range les deux cartes sous son paquet comme avant : la sienne d'abord, même si elle est la plus faible.

La réponse attendue. Le score du gagnant de la partie principale, calculé comme en partie 1.

Outil Python : les ensembles (`set`)

Un **ensemble** est une collection **sans doublon** et **sans ordre** : pas d'indice ; on peut seulement ajouter, retirer, tester l'appartenance et parcourir (dans un ordre quelconque). Ses éléments doivent être **non modifiables** : nombres, chaînes, **tuples** — pas de liste.

```
vus = set()                # ensemble vide (attention : {} est un dictionnaire vide)
vus.add(5)                 # ajout ; ajouter une valeur deja presente ne change rien
vus.add(8)
print(8 in vus)            # True : test d'appartenance
print(len(vus))            # 2
vus.remove(8)              # retrait (erreur si absent : tester avant avec in)
a = {1, 2, 3}              # ensemble ecrit directement
b = set([2, 3, 4, 4])      # a partir d'une liste : {2, 3, 4}
print(len(b))              # 3 : le doublon a disparu
for x in a:                # parcours, dans un ordre quelconque
    print(x)
pairs = {n for n in range(10) if n % 2 == 0}   # ensemble en comprehension
```

Deux opérateurs construisent un **nouvel** ensemble à partir de deux autres, sans les modifier : la **réunion** `a | b` (éléments présents dans `a` ou dans `b`) et l'**intersection** `a & b` (éléments présents dans les deux).

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)               # {1, 2, 3, 4} : reunion, sans doublon
print(a & b)               # {2, 3} : intersection
a = a | {7}                # pour agrandir a, on le remplace par la reunion
```

Dans ce défi, pour mémoriser l'état des deux paquets, on le convertit en un couple de tuples :

```
p1, p2 = [6, 2], [4, 7]
deja_vus = set()
etat = (tuple(p1), tuple(p2))            # ((6, 2), (4, 7))
print(etat in deja_vus)                  # False
deja_vus.add(etat)
print(((6, 2), (4, 7)) in deja_vus)      # True
```

Intérêt. Avec une liste, `x in liste` compare `x` à chaque élément l'un après l'autre : jusqu'à n comparaisons. Un ensemble range ses éléments selon une valeur calculée à partir d'eux (une *fonction de hachage*, le même principe que les clés d'un dictionnaire) : Python va directement là où `x` devrait se trouver. Le test `x in ensemble` prend donc un temps qui **ne dépend pas** du nombre d'éléments (**temps constant**, en moyenne). C'est aussi ce qui rend `a & b` rapide : Python parcourt un des deux ensembles et teste chaque élément dans l'autre en temps constant, au lieu de comparer chaque élément de `a` à chaque élément de `b`. Cet encadré servira dans plusieurs défis des chapitres suivants.

À essayer. Que se passe-t-il si l'on essaie `deja_vus.add(p1)` ? Que renvoie `set("banane")` ? Que valent `set("chat") & set("chien")` et `set("chat") | set("chien")` ? Et `a & set()` ?

★★ Exercice 5 — *Le combat récursif*

But. Écrire la version récursive du jeu, où une sous-partie est un appel de la même fonction.
À quoi ça sert ? C'est la réponse de la partie 2 : la règle « une manche se décide par une partie plus petite » se traduit directement par un appel récursif. Le point délicat est de donner à chaque partie sa propre mémoire des situations déjà vues.

Écrire une fonction récursive `combat_recursif(p1, p2)` qui renvoie le numéro du gagnant et son paquet. Sur l'exemple inventé de la fiche, le joueur 1 gagne avec un score de `151`. Bien relire la règle qui empêche une partie de durer indéfiniment : elle s'applique à **chaque** partie et sous-partie séparément. ▷ coup de pouce ▷ début de solution

★★★ Exercice 6 — *Défi : mesurer*

But. Mesurer le nombre d'appels récursifs et la profondeur de récursion atteinte.
À quoi ça sert ? Ces mesures montrent le coût réel de la partie 2 et expliquent pourquoi Python n'atteint jamais sa limite de récursion ici : c'est le même raisonnement que pour prouver qu'une fonction récursive se termine.

Compter le nombre total d'appels de `combat_recursif` et la profondeur maximale de récursion atteinte sur vos données. Pourquoi la profondeur reste-t-elle faible ? ▷ coup de pouce

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Le fichier contient deux blocs séparés par une ligne vide : après `strip`, `texte.split("\n\n")` les sépare, puis `split("\n")` découpe chaque bloc en lignes (la première, `Player 1:`, ne contient pas de carte). Avec `paquet = [6, 2, 8, 1]`, `paquet.pop(0)` renvoie `6` et `paquet.append(x)` ajoute à la fin. ↑énoncé

Ex. 4 – piste La boucle tourne `while p1 and p2` : une file vide vaut `False`. Pour le score, la carte du dessous est multipliée par 1 : en parcourant le paquet par indice (`for i in range(len(paquet))`), la carte d'indice `i` est multipliée par `len(paquet) - i` ; on accumule les produits dans une variable `total`. ↑énoncé

Ex. 5 – piste La mémoire des situations déjà vues est un **ensemble** créé au début de chaque appel de la fonction (donc propre à chaque sous-partie). Une liste n'est pas utilisable comme élément d'un ensemble : y ranger le couple `(tuple(p1), tuple(p2))`. ↑énoncé

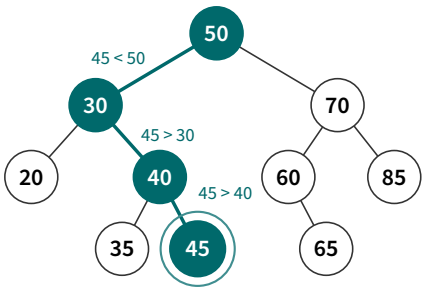
Ex. 5 – début de solution Pour une sous-partie, on construit de **nouveaux** paquets (copies) : la tranche `p1[:n]` est une nouvelle liste qui contient les `n` cartes du dessus, sans modifier `p1`. La fonction renvoie un couple `(gagnant, paquet)` ; dans la sous-partie, seul le gagnant compte pour décider de la manche en cours. ↑

Ex. 6 – piste À chaque appel récursif, le nombre total de cartes en jeu diminue strictement. ↑énoncé

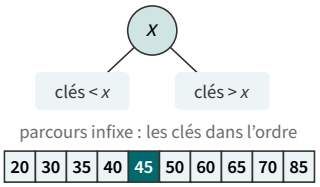
5

CHAPITRE 5

Arbres



Arbre binaire de recherche



DANS CE CHAPITRE

- Activités préparatoires . . . 227
- Cours . . . 229
- Exercices . . . 241
- TP et projets . . . 255
- Fiche de révision . . . 267
- Défis Advent of Code . . . 269
- Corrigés . . . 829

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Un tournoi, des dossiers, une famille

▶ corrigé p. 829

trois situations, une même forme

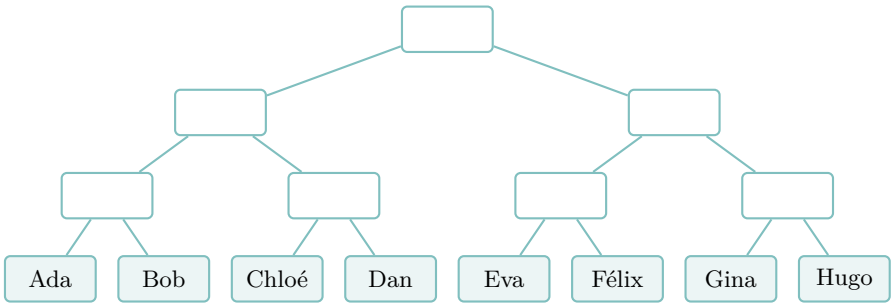
Durée : 20 min · Sans ordinateur, par deux

Consignes

- Pour chaque situation, on **dessine** d'abord, on répond ensuite, sur le cahier. Aucun mot de vocabulaire n'est attendu avant l'exercice 4 : c'est à vous d'inventer ces mots.

Exercice 1 *Le tournoi*

Huit joueurs s'affrontent dans un tournoi à élimination directe : chaque match oppose deux joueurs, le perdant est éliminé. Résultats : au premier tour, Ada bat Bob, Dan bat Chloé, Eva bat Félix et Hugo bat Gina ; au deuxième tour, Ada bat Dan et Hugo bat Eva ; en finale, Hugo bat Ada.



1. Recopier le tableau du tournoi sur le cahier et remplir chaque case vide avec le nom du vainqueur du match correspondant.
2. Combien de matchs ont été joués en tout ? Combien de tours ?
3. Même question avec 16 joueurs, puis avec 64 joueurs. Quelle relation entre le nombre de joueurs et le nombre de matchs ?
4. Combien de matchs Hugo a-t-il joués ? Et un joueur éliminé au premier tour ?

Exercice 2 *Les dossiers*

Sur l'ordinateur d'un élève : le dossier `Documents` contient les dossiers `NSI` et `Photos`, ainsi que le fichier `CV.pdf`. Le dossier `NSI` contient les dossiers `Cours` et `Projets`. `Cours` contient `arbres.pdf` ; `Projets` contient `jeu.py` et `notes.txt` ; `Photos` contient `plage.jpg`.

5. Représenter ce contenu par un dessin sur le cahier, en plaçant `Documents` tout en haut et en reliant chaque élément à ce qu'il contient directement.
6. Un camarade propose de tout ranger dans la liste `["Documents", "NSI", "Photos", "CV.pdf", "Cours", ...]`. Quelle information perd-on ?
7. Combien de « marches » faut-il descendre pour aller de `Documents` à `jeu.py` ? Quels éléments sont les plus loin de `Documents` ?
8. Quelle différence voyez-vous entre ce dessin et celui du tournoi, quant au nombre de traits qui partent d'un même élément vers le bas ?

Exercice 3 *La famille*

L'arbre généalogique d'Ada remonte vers ses ancêtres : Ada a deux parents, chacun a deux parents, et ainsi de suite.

9. Combien Ada a-t-elle de grands-parents ? d'arrière-grands-parents ? Combien d'ancêtres à la 10e génération au-dessus d'elle (si tous sont distincts) ?

10. Dans un arbre généalogique, Ada est souvent dessinée tout en bas. Si l'on dessine Ada tout en haut, comme `Documents` et le vainqueur du tournoi, quel point commun apparaît avec le tournoi ?

Exercice 4 *Inventer le vocabulaire*

11. Ces dessins ressemblent à un arbre retourné. Sur le cahier, proposer un nom pour chacune des notions suivantes (un mot ou deux), en vous inspirant de l'image de l'arbre ou de celle de la famille.

a) l'élément tout en haut, au-dessus duquel il n'y a rien ;
b) un élément tout en bas, en dessous duquel il n'y a rien ;
c) pour un élément : celui qui est juste au-dessus / ceux qui sont juste en dessous ;
d) un élément avec tout ce qui se trouve en dessous de lui ;
e) le nombre total d'éléments du dessin ;
f) le nombre d'étages du dessin.

Exercice 5 *Ce qu'on a découvert*

12. Recopier et compléter sur le cahier, avec vos mots.

- Dans les trois situations, chaque élément (sauf celui du haut) ...
- Une simple liste ne suffit pas, car ...
- Le tournoi et la famille ont une particularité que les dossiers n'ont pas : ...

COURS

Programme (BO) *« Arbres : structures hiérarchiques. Arbres binaires : nœuds, racines, feuilles, sous-arbres gauches, sous-arbres droits. Identifier des situations nécessitant une structure de données arborescente. Évaluer quelques mesures des arbres binaires (taille, encadrement de la hauteur, etc.). » « Algorithmes sur les arbres binaires et sur les arbres binaires de recherche. Calculer la taille et la hauteur d'un arbre. Parcourir un arbre de différentes façons (ordres infixe, préfixe ou suffixe ; ordre en largeur d'abord). Rechercher une clé dans un arbre de recherche, insérer une clé. »*

Prérequis la **récursivité** (indispensable ici !), les classes et objets (chapitre *Programmation objet*), les piles et les files, la recherche par dichotomie. → fiches de révision : ch. 1 p. 58, ch. 2 p. 113, ch. 3 p. 162 → rappels de 1re : dichotomie p. 1060

Objectifs *reconnaître* une situation qui appelle un arbre ; *maîtriser* le vocabulaire ; *calculer* taille et hauteur ; *implémenter* un arbre binaire (de plusieurs façons) et le *parcourir* ; *comprendre* l'arbre binaire de recherche, y *rechercher* et y *insérer* une valeur.

Situation de départ

1. Comment représenter dans un programme les dossiers d'un ordinateur, qui contiennent des fichiers et d'autres dossiers ? Pourquoi une simple liste de noms ne suffit-elle pas ?
2. Comment retrouver une valeur parmi un million en une vingtaine d'étapes, tout en pouvant en ajouter de nouvelles à tout moment ?
3. Pour afficher un arbre généalogique génération par génération, ou branche par branche, dans quel ordre faut-il visiter les personnes ?

I Pourquoi un arbre ? Des situations où rien d'autre ne convient

Depuis le début de l'année, vos structures étaient **linéaires** : listes, piles, files rangent les données « en ligne », les unes derrière les autres. Or beaucoup d'informations sont naturellement **hiérarchiques** : elles se ramifient. Une liste ne sait pas exprimer cela.

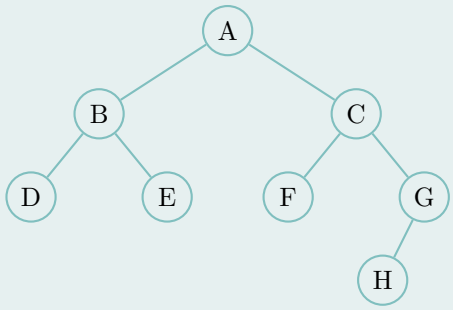
Exemple — *Quatre situations où l'arbre s'impose*

- **L'arborescence des fichiers.** Un dossier contient des fichiers *et* d'autres dossiers, qui contiennent à leur tour… Impossible de représenter fidèlement cette imbrication avec une simple liste de noms : on perdrait « qui est dans qui ».
- **Un arbre de décision** (diagnostic médical, identification d'une plante, un « akinator ») : chaque question mène, selon la réponse, à une nouvelle question ou à une conclusion. La ramification *est* le raisonnement.
- **La recherche rapide** dans une grande collection triée : un *arbre binaire de recherche* (fin de ce cours) retrouve une valeur en un nombre d'étapes proportionnel à sa **hauteur**, pas à sa taille — c'est l'idée de la dichotomie, structurée. Les **bases de données** indexent ainsi des millions de lignes.
- **La compression** (codage de Huffman), l'**évaluation d'expressions** par un compilateur, la structure d'une **page web** (le DOM), un **arbre de jeu** aux échecs… : partout où il y a une hiérarchie ou un choix qui se ramifie, il y a un arbre.

Remarque
Le point commun ? Une **relation de descendance** : un élément en « commande » plusieurs autres, qui en commandent d'autres. C'est ce qu'une structure linéaire ne peut pas capturer, et ce qu'un arbre exprime naturellement.

II Vocabulaire : de la racine aux feuilles

Voici un arbre. On le dessine, par tradition informatique, **la racine en haut** et les feuilles en bas.



Définition 1 — Le vocabulaire de l'arbre

- chaque élément est un **nœud** (ici `A`, `B`, ..., `H`) ;
- le nœud du sommet, sans père, est la **racine** (`A`) ;
- `B` et `C` sont les **fils** de `A` ; `A` est leur **père** ;
- un nœud sans fils est une **feuille** (`D`, `E`, `F`, `H`) ;
- le lien entre deux nœuds est une **arête** (ou *branche*) ;
- tout nœud est la racine d'un **sous-arbre** (celui de `C` contient `C`, `F`, `G`, `H`).

↻ **Retour sur l'activité.** Les mots inventés dans l'activité ont maintenant leur nom officiel : l'élément du haut (`Documents`, le vainqueur du tournoi) est la **racine**, ceux du bas (les fichiers, les joueurs du premier tour) sont des **feuilles**, et « un élément avec tout ce qui est en dessous de lui » est un **sous-arbre**.

Définition 2 — Arbre binaire

Un arbre est **binaire** si chaque nœud possède **au plus deux** fils, appelés **fils gauche** et **fils droit** (l'un des deux, ou les deux, peuvent manquer). L'arbre ci-dessus est binaire. Dans la suite, sauf mention contraire, « arbre » signifiera « arbre binaire ».

↻ **Retour sur l'activité.** Le tournoi et la famille de l'activité étaient des arbres binaires (au plus deux traits vers le bas à chaque élément) ; l'arborescence des dossiers ne l'était pas, puisque `Documents` a trois fils.

À retenir — un arbre, c'est récursif

Regardez bien : **un arbre est soit vide, soit un nœud (la racine) muni de deux sous-arbres** (gauche et droit), qui sont eux-mêmes des arbres... On retrouve la structure du chapitre *Récursivité* : un cas de base (l'arbre vide) et un cas qui se ramène à des cas plus petits. C'est pourquoi **presque tous les algorithmes sur les arbres seront récursifs**.

► **Exercices d'application :** exercice 1 (vocabulaire).

III Mesurer un arbre : taille, profondeur, hauteur

Définition 3 — Taille

La **taille** d'un arbre est son **nombre de nœuds**. L'arbre ci-dessus a pour taille 8. L'arbre vide a pour taille 0.

Définition 4 — Profondeur d'un nœud

La **profondeur** d'un nœud est le nombre d'**arêtes** du chemin qui va de la racine jusqu'à lui. La racine est à la profondeur 0, ses fils à 1, etc. (Dans notre exemple, `H` est à la profondeur 3.) C'est la convention de Knuth (le « niveau » d'un nœud) et celle des sujets de bac ; on rencontre parfois l'autre, qui compte les nœuds (racine à la profondeur 1) : **suivez l'énoncé**.

Définition 5 — Hauteur

La **hauteur** d'un arbre est le **nombre de nœuds** du plus long chemin de la racine à une feuille — autrement dit la profondeur maximale de ses nœuds, **plus 1**. Dans notre exemple, le chemin `A--C--G--H` donne une hauteur de 4.

Règle 1 — Attention : deux conventions pour la hauteur !

Faut-il compter les **nœuds** ou les **arêtes** du chemin ? Les deux conventions existent :

- en comptant les **nœuds**, un arbre réduit à sa seule racine a une hauteur de **1** (et l'arbre vide, 0) ;
- en comptant les **arêtes**, le même arbre a une hauteur de **0** (et l'arbre vide, par convention, -1).

Aucune n'est « la bonne » : il faut juste **préciser laquelle on utilise**. **Au bac, l'énoncé le dit toujours.** Dans ce cours, nous comptons les **nœuds** (racine seule → hauteur 1, arbre vide → hauteur 0) : c'est la définition de **Donald Knuth** (*The Art of Computer Programming*, vol. 3), pour qui la hauteur est la longueur du plus long chemin de la racine jusqu'à un sous-arbre **vide**. Si un énoncé adopte l'autre convention, **adaptez-vous** : toutes les hauteurs diminuent de 1 (l'exemple ci-dessus aurait une hauteur de 3).

↻ **Retour sur l'activité.** L'arbre du tournoi à huit joueurs a pour taille 15 (les 8 joueurs et les 7 cases de vainqueurs) et pour hauteur 4 ; les « marches » comptées de `Documents` à `jeu.py`, au nombre de 3, donnent la profondeur de `jeu.py`.

Pour les curieux — *Comme les étages d'un immeuble* (facultatif)
Cette querelle vous rappellera quelque chose. En France, on entre au **rez-de-chaussée** (l'étage « 0 »), et le premier étage est *au-dessus* : on compte les **volées d'escalier** (les arêtes). Aux États-Unis, le rez-de-chaussée *est* le « *first floor* » : on compte les **niveaux** (les nœuds). Bref, le même bâtiment a « deux hauteurs » selon le pays — exactement comme nos arbres. Un Américain et un Français se donnent rendez-vous « au premier étage » : l'un attend l'autre un étage plus bas. Précisez toujours votre convention !

À retenir — encadrement de la hauteur d'un arbre binaire de n nœuds

Avec la convention du cours (hauteur comptée en nœuds), un arbre binaire de taille $n \geqslant 1$ et de hauteur h vérifie

$$\log_2(n+1) \leqslant h \leqslant n, \qquad \text{autrement dit} \qquad h \leqslant n \leqslant 2^h - 1.$$

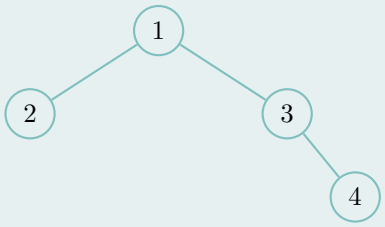
- **Au plus** n : chaque niveau contient au moins un nœud. La hauteur vaut n pour un arbre **filiforme** (chaque nœud a au plus un fils), dégénéré en une sorte de liste.
- **Au moins** $\log_2(n+1)$: le niveau de profondeur k contient au plus 2^k nœuds, donc un arbre de hauteur h en a au plus $1 + 2 + 4 + \cdots + 2^{h-1} = 2^h - 1$. Cette borne est atteinte par l'arbre **parfait**, dont tous les niveaux sont pleins.

Exemple : avec $n = 15$ nœuds, la hauteur est comprise entre 4 ($2^4 - 1 = 15$) et 15 ; avec $n = 1\,000$, entre 10 ($2^{10} - 1 = 1\,023 \geqslant 1\,000$, alors que $2^9 - 1 = 511$ ne suffit pas) et 1 000. Avec la convention des arêtes, on retranche 1 aux deux bornes.
Cette hauteur gouverne la rapidité des recherches : plus l'arbre est équilibré, plus elle est proche de $\log_2(n)$, et plus il est efficace.

► **Exercices d'application :** exercice 26 (encadrer la hauteur).

IV Implémenter un arbre binaire (de plusieurs façons)

Python ne fournit pas d'arbre « tout prêt » : on l'implémente. Comme pour les piles et les files, une **même structure** admet **plusieurs implémentations** — et les sujets de bac en utilisent plusieurs. En voici deux, sur le même petit arbre :



A. Par une classe (programmation objet)

C'est l'implémentation de référence : un **nœud** est un objet qui connaît sa **valeur** et ses deux sous-arbres. L'arbre vide est représenté par `None`.

```
class Noeud:
    def __init__(self, valeur, gauche=None, droite=None):
        self.valeur = valeur
        self.gauche = gauche      # un Noeud, ou None (sous-arbre vide)
        self.droite = droite      # un Noeud, ou None

# l'arbre ci-dessus :
a = Noeud(1, Noeud(2), Noeud(3, None, Noeud(4)))
```

Remarque

C'est exactement l'idée annoncée dans le chapitre *Programmation objet* : « une classe `Noeud` avec un sous-arbre gauche et un sous-arbre droit ». Une **feuille** est simplement un `Noeud` dont les deux fils valent `None`. Grâce aux **paramètres par défaut** `gauche=None` et `droite=None` (voir le chapitre *Récursivité*), on l'écrit simplement `Noeud(2)` ; la valeur par défaut `None` ne se modifie pas, elle est donc sans danger.

B. Par un tuple (ou une liste) imbriqué

Plus léger, sans POO : un arbre est un **triplet** `(valeur, sous-arbre gauche, sous-arbre droit)`, et l'arbre vide est `None`.

```
# (valeur, gauche, droite) ; None pour un sous-arbre vide
a = (1, (2, None, None), (3, None, (4, None, None)))
```

Remarque

On accède alors à la valeur par `a[0]`, au fils gauche par `a[1]`, au fils droit par `a[2]`. Certains sujets préfèrent une *liste* `[valeur, gauche, droite]`, ou encore un **dictionnaire**. Peu importe : ce qui compte, c'est de savoir **où sont la valeur et les deux sous-arbres**. Nous prendrons l'implémentation par **classe** pour la suite.

► **Exercices d'application :** exercice 2 (construire un arbre à partir d'une description).

V Parcourir un arbre

Parcourir un arbre, c'est visiter tous ses nœuds une fois chacun. Comme un arbre est récursif, un parcours l'est aussi : on traite la racine, et on parcourt (récursivement) les deux sous-arbres. Toute la question est : **dans quel ordre** ?

A. Les parcours en profondeur

On plonge dans un sous-arbre avant d'explorer l'autre. Selon le moment où l'on « visite » la racine, on distingue trois parcours.

Parcours	Ordre	Utile pour…
préfixe	racine, puis gauche, puis droite	copier/recréer l'arbre
infixe	gauche, puis racine, puis droite	**trier** (dans un ABR, voir plus bas)
suffixe	gauche, puis droite, puis racine	évaluer, supprimer l'arbre

```
def prefixe(a):
    if a is not None:
        print(a.valeur)          # CAS DE BASE : arbre vide, on ne fait rien
        prefixe(a.gauche)        # visiter la racine
        prefixe(a.droite)        # APPEL RECURSIF : tout le sous-arbre gauche
                                 # APPEL RECURSIF : puis tout le sous-arbre droit
```

Pour l'**infixe** et le **suffixe**, il suffit de **déplacer** la ligne `print` : au milieu (entre les deux appels) pour l'infixe, à la fin pour le suffixe.

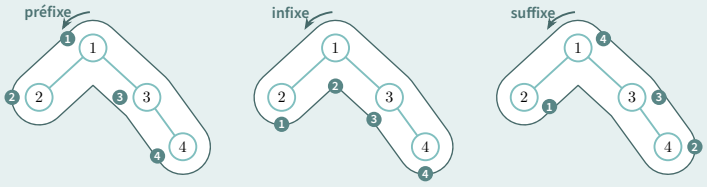
Exemple — *Les trois parcours de notre arbre*

Sur l'arbre `a = Noeud(1, Noeud(2), Noeud(3, None, Noeud(4)))` dessiné plus haut :

▷ **Je prédis.** Cacher la liste ci-dessous. Dans quel ordre les parcours préfixe, infixe et suffixe visitent-ils les nœuds de `a` ?

- préfixe : `1, 2, 3, 4` (racine d'abord) ;
- infixe : `2, 1, 3, 4` (la racine `1` apparaît après son sous-arbre gauche) ;
- suffixe : `2, 4, 3, 1` (la racine `1` en dernier).

Une astuce pour retrouver ces trois ordres sans se tromper : on fait le **tour de l'arbre** en longeant son contour, en partant de la gauche de la racine. On note un nœud quand on passe à sa **gauche** (préfixe), **sous** lui (infixe) ou à sa **droite** (suffixe). Les numéros donnent l'ordre de visite :



B. Le parcours en largeur

Ici, on visite l'arbre **niveau par niveau**, de gauche à droite (la racine, puis ses fils, puis les petits-fils…). Ce parcours n'est *pas* récursif : il s'appuie sur une **file** (voir le chapitre *Structures linéaires*). On enfile la racine, puis, tant que la file n'est pas vide, on défile un nœud, on le visite et on enfile ses fils.

```
1  def largeur(a):
2      if a is None:                # TESTER : arbre vide, rien a visiter
3          return
4      f = File()                   # INITIALISER : une file (enfiler/defiler/est_vide)
5      f.enfiler(a)                 # INITIALISER : la racine attend son tour
6      while not f.est_vide():      # PARCOURIR : tant qu'il reste des noeuds
7          n = f.defiler()
8          print(n.valeur)          # visiter
9          if n.gauche is not None: # TESTER
10             f.enfiler(n.gauche)  # METTRE A JOUR : le fils passe en fin de file
11         if n.droite is not None: # TESTER
12             f.enfiler(n.droite)  # METTRE A JOUR
```

▷ **Je prédis.** Dans quel ordre `largeur(a)` affiche-t-elle les nœuds de l'arbre `a` précédent ? Et ceux de l'arbre `A…H` du début du chapitre, comparés au parcours préfixe ?

▷ **Je vérifie.** `1, 2, 3, 4` : ici, le même ordre que le préfixe, par hasard. Sur l'arbre `A…H`, la largeur donne `A, B, C, D, E, F, G, H` (niveau par niveau) et le préfixe `A, B, D, E, C, F, G, H` : le préfixe termine le sous-arbre de `B` avant de passer à `C`.

À retenir — largeur et file, profondeur et pile

Retenez le duo : parcours en **largeur** ↔ **file** ; parcours en **profondeur** ↔ récursivité (donc, en coulisses, une **pile**). Les deux structures du chapitre « structures linéaires » resservent ici.

► **Exercices d'application :** exercices 3 à 7 et 24 (les quatre parcours ; un arbre irrégulier ; retrouver le parcours, retrouver l'arbre ; programmer les parcours en profondeur et en largeur ; parcours en série).

VI Calculer sur un arbre : taille et hauteur

Les formules récursives découlent directement de la définition « racine + deux sous-arbres ».

Propriété 1 — Taille et hauteur, récursivement

Pour un arbre `a` de sous-arbres gauche `g` et droit `d`, on définit la **taille** par :

$$\texttt{taille}(a) = \begin{cases} 0 & \text{si } a \text{ est vide,} \\ 1 + \texttt{taille}(g) + \texttt{taille}(d) & \text{sinon,} \end{cases}$$

et la **hauteur** par :

$$\texttt{hauteur}(a) = \begin{cases} 0 & \text{si } a \text{ est vide,} \\ 1 + \max\left(\texttt{hauteur}(g), \texttt{hauteur}(d)\right) & \text{sinon.} \end{cases}$$

```python
def taille(a):
    if a is None:                                   # CAS DE BASE : arbre vide
        return 0
    return 1 + taille(a.gauche) + taille(a.droite)  # APPEL RECURSIF x 2, COMBINER

def hauteur(a):
    if a is None:                                   # CAS DE BASE : arbre vide
        return 0
    return 1 + max(hauteur(a.gauche), hauteur(a.droite))  # APPEL RECURSIF x 2, COMBINER
```

▷ **Je prédis.** Que renvoient `taille(a)` et `hauteur(a)` pour `a = Noeud(1, Noeud(2), Noeud(3, None, Noeud(4)))` ? Et `hauteur(Noeud(5))` ?

▷ **Je vérifie.** `4` et `3` (le plus long chemin, `1`, `3`, `4`, compte trois nœuds). Une feuille seule a une hauteur de `1` avec la convention du cours (on compte les nœuds) ; ce serait `0` en comptant les arêtes.

Au bac — hauteur : et avec l'autre convention ?

Si l'énoncé compte les **arêtes**, seul le **cas de base** change : l'arbre vide a pour hauteur -1, donc `return -1` au lieu de `return 0`. Ainsi une feuille a bien une hauteur de $1 + \max(-1, -1) = 0$. Lisez toujours la définition donnée par le sujet avant d'écrire `hauteur`.

Remarque

Chaque appel se ramène à des sous-arbres *strictement plus petits* : le cas de base (l'arbre vide) finit toujours par être atteint. La terminaison est garantie — comme pour toute belle récursion sur un arbre.

► **Exercices d'application :** exercices 8 à 10 (taille, hauteur et nombre de feuilles, en récursif).

VII L'arbre binaire de recherche (ABR) : chercher et insérer vite

Un arbre quelconque ne facilite pas la recherche. Mais si l'on **range** les valeurs avec méthode, la recherche devient **très rapide**.

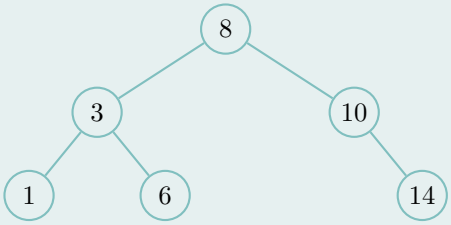
Définition 6 — Arbre binaire de recherche

Un **arbre binaire de recherche** (ABR) est un arbre binaire où, pour **chaque** nœud :

- toutes les valeurs de son sous-arbre **gauche** sont **strictement plus petites** que sa valeur ;
- toutes les valeurs de son sous-arbre **droit** sont **plus grandes ou égales** à sa valeur.

Autrement dit : `gauche` $<$ `nœud` $\leq$ `droite`. Une valeur **égale** à celle du nœud (un doublon)

se range donc **à droite** : c'est la convention de tout ce chapitre.



A. Rechercher une valeur dans un ABR

Règle 2 — Chercher dans un ABR

Pour chercher une valeur `x`, on part de la racine et, à chaque nœud, on compare : si `x` est plus petit, on descend à **gauche** ; s'il est plus grand, à **droite** ; sinon, on l'a trouvé. À chaque étape, on **élimine tout un sous-arbre** : si l'arbre est **équilibré**, c'est environ la **moitié** de ce qui reste, comme dans la **recherche dichotomique** vue en Première (le chapitre 7 en fera un exemple de « diviser pour régner »), mais incarnée dans une structure de données (si l'arbre est très déséquilibré, on n'élimine presque rien à chaque étape).

```
1  def recherche(a, x):
2      if a is None:                    # CAS DE BASE : sous-arbre vide
3          return False                 # RENVOYER : x est absent
4      if x == a.valeur:                # TESTER : trouve
5          return True
6      if x < a.valeur:                 # TESTER : de quel cote descendre ?
7          return recherche(a.gauche, x)   # APPEL RECURSIF : a gauche, les plus petits
8      else:
9          return recherche(a.droite, x)   # APPEL RECURSIF : a droite, les plus grands (ou egaux)
```

▷ **Je prédis.** Sur l'ABR dessiné ci-dessus, que renvoient `recherche(a, 6)` et `recherche(a, 7)` ? Quels nœuds chaque appel visite-t-il ?

▷ **Je vérifie.** `True` en visitant `8`, `3`, `6`. `False` en visitant les **mêmes** nœuds : après `6`, on descend à droite sur un sous-arbre vide. On ne parcourt jamais tout l'arbre.

À retenir — le coût d'une recherche dans un ABR

La recherche coûte au plus **la hauteur** de l'arbre. Pour un ABR équilibré de n valeurs, cela fait environ $\log_2(n)$ comparaisons : chercher parmi un **million** de valeurs ne demande qu'une **vingtaine** d'étapes ! C'est cette efficacité qui fait des arbres le cœur des **index de bases de données**. Petit bonus : un **parcours infixe** d'un ABR affiche ses valeurs **déjà triées**.

► **Exercices d'application :** exercice 11 (rechercher dans un ABR).

B. Insérer une valeur dans un ABR

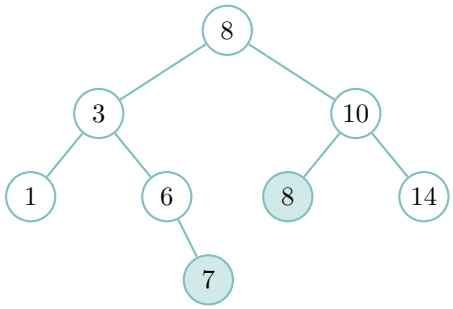
Chercher, c'est bien ; mais comment **construire** un ABR ? En y insérant les valeurs une à une, sans jamais casser la règle « à gauche les plus petits, à droite les autres ». Bonne nouvelle : l'insertion n'est qu'une **recherche qui va jusqu'au bout**.

Règle 3 — Insérer dans un ABR

Pour insérer une valeur `x`, on descend depuis la racine **exactement comme pour la chercher** : à gauche si `x` est plus petit que la valeur du nœud, à droite sinon (donc aussi en cas d'**égalité**, selon notre convention). On s'arrête quand on tombe sur un **sous-arbre vide** : c'est la place de `x`, qui y devient une nouvelle **feuille**. On ne déplace jamais les nœuds déjà en place.

Exemple — *Insérer `7` puis `8` dans l'ABR précédent*

`7` : $7 < 8$ (gauche), $7 > 3$ (droite), $7 > 6$ (droite) : `7` devient le fils droit de `6`.
`8` : $8 = 8$, donc à droite (convention), puis $8 < 10$ (gauche) : ce second `8` devient le fils gauche de `10`.



Le parcours infixe donne `1, 3, 6, 7, 8, 8, 10, 14` : toujours trié, doublon compris.

Le code est récursif, comme la recherche. La fonction **renvoie l'arbre** obtenu : c'est indispensable, car lorsque l'arbre est vide (`None`), on ne peut rien modifier — on doit *créer* un nœud et le rendre à l'appelant, qui l'accroche à sa place.

```
def inserer(a, x):
    """Insere x dans l'ABR a et renvoie l'arbre obtenu."""
    if a is None:                          # CAS DE BASE : place vide
        return Noeud(x)                    # x devient une feuille
    if x < a.valeur:                       # TESTER
        a.gauche = inserer(a.gauche, x)    # APPEL RECURSIF : plus petit, a gauche
    else:
        a.droite = inserer(a.droite, x)    # APPEL RECURSIF : plus grand ou egal, a droite
    return a                               # RENVOYER : l'arbre, a raccrocher par l'appelant
```

On construit alors un ABR en partant de l'arbre vide :

```
a = None
for v in [8, 3, 10, 1, 6, 14]:
    a = inserer(a, v)          # on recupere TOUJOURS l'arbre renvoye
```

On obtient exactement l'ABR dessiné plus haut.

Remarque — *Coût, et importance de l'ordre d'insertion*

Comme la recherche, l'insertion coûte au plus **la hauteur** de l'arbre. Mais la forme de l'arbre dépend de l'**ordre** dans lequel on insère : avec `4, 2, 6, 1, 3, 5, 7`, on obtient un arbre équilibré de hauteur 3 ; avec `1, 2, 3, 4, 5, 6, 7` (déjà trié !), chaque valeur part à droite de la précédente et l'on obtient un arbre « en peigne » de hauteur 7, aussi lent qu'une simple liste. (Il existe des ABR qui se rééquilibrent tout seuls, comme les arbres AVL ou rouge-noir [au-delà

du programme].)

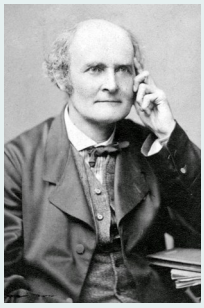
Au bac — les doublons : lire la convention de l'énoncé

Ce cours range une valeur **égale** au nœud dans le sous-arbre **droit**. Ce n'est qu'une convention : certains sujets de bac interdisent les doublons (on n'insère pas une valeur déjà présente, comme dans le TP proposé en fin de chapitre), d'autres les placent à gauche. Dans tous les cas, la recherche et l'insertion suivent le même principe ; il suffit de **lire la définition donnée** et de l'appliquer de façon cohérente.

► **Exercices d'application :** exercices 12, 13 **et** 25 (insérer à la main ; programmer l'insertion ; insertions en série).

Pour les curieux — *Un peu d'histoire… et de botanique* (facultatif)

En informatique, l'arbre est le seul à **pousser la tête en bas** : racine en haut, feuilles en bas — un botaniste s'en évanouirait. La structure est ancienne : le mot « arbre » entre en mathématiques en **1857** avec le Britannique **Arthur Cayley**, qui s'en sert ensuite, en **1874–1875**, pour *compter* les molécules chimiques possibles (vues comme des arbres d'atomes). Mais l'**arbre binaire de recherche** est formalisé autour de **1960** par plusieurs chercheurs indépendamment (dont **P. F. Windley**, **A. D. Booth**, **T. N. Hibbard**). En **1952**, un étudiant du MIT, **David Huffman**, invente pour un devoir noté un codage par arbre qui reste, soixante-dix ans plus tard, au cœur des formats de compression (ZIP, JPEG…) : le plus rentable des devoirs maison. Enfin, les **B-arbres** (Bayer et McCreight, **1970**) organisent aujourd'hui les données de la quasi-totalité des bases de données et systèmes de fichiers de la planète.



Arthur Cayley (1821–1895), qui a donné son nom mathématique à l'arbre



*Et la nature s'y met aussi : à **Baïes** (Italie), un figuier sauvage pousse **la tête en bas** depuis la voûte d'une salle romaine, sa racine au plafond et ses branches feuillues vers le sol — un véritable arbre informatique !*

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise :** la *Récursivité* (taille, hauteur, parcours), la classe `Noeud` (*Programmation orientée objet et paradigmes*), la file et la pile (*Structures linéaires : piles, files, listes chaînées*) ; l'ABR incarne *La recherche dichotomique* (Première).
- **Déjà croisé :** l'arbre des processus créé par `fork` (*Processus et ordonnancement*).
- **Pour aller plus loin :** les arbres organiseront les index des *Bases de données et SQL* ; en *Graphes*, vous verrez qu'un arbre est un graphe connexe sans cycle.

Bilan — la carte mémoire

Notion	À retenir
Arbre	structure **hiérarchique** : une racine, des sous-arbres (récursif)
Nœud / racine / feuille	élément / nœud du sommet / nœud sans fils
Arbre binaire	chaque nœud a **au plus deux** fils (gauche, droit)
Taille	nombre de nœuds : $1 + \texttt{taille}(g) + \texttt{taille}(d)$
Hauteur	plus long chemin racine→feuille (préciser la **convention** !)
Implémentations	classe `Noeud`, ou triplet `(valeur, g, d)`… (vide = `None`)
Parcours profondeur	préfixe / infixe / suffixe (récursifs)
Parcours largeur	niveau par niveau, avec une **file**
ABR	gauche $<$ nœud $\leq$ droite (doublon à droite)
Recherche, insertion	on descend d'un seul côté ; coût = hauteur ($\approx \log_2(n)$ si équilibré)
Insérer dans un ABR	descendre comme pour chercher ; `x` devient une **feuille**

Retour à la situation de départ

1. Par un **arbre** : chaque dossier est un **nœud** dont les fils sont ses fichiers et ses sous-dossiers. Une liste, structure linéaire, perdrait la relation « qui est dans qui » (§ I et II).
2. Avec un **arbre binaire de recherche** (à gauche les valeurs plus petites, à droite les autres) : chercher ou insérer coûte au plus la **hauteur**, soit environ $\log_2(n)$ comparaisons, une vingtaine pour un million de valeurs si l'arbre est équilibré (§ VII).
3. Génération par génération, c'est le parcours en **largeur**, avec une **file** ; branche par branche, ce sont les parcours en **profondeur** (préfixe, infixe, suffixe), récursifs (§ V).

Erreurs fréquentes

- **Oublier le cas de l'arbre vide.** Toute fonction récursive sur un arbre commence par `if a is None:` … *Le réflexe :* l'arbre vide est le cas de base.
- **Confondre taille, hauteur et profondeur.** La **taille** compte les nœuds ; la **hauteur** est la plus longue descente. *Le réflexe :* fixer la **convention** de hauteur (arbre vide = 0 ici) et s'y tenir.
- **Mélanger les parcours. préfixe** (racine avant), **infixe** (racine au milieu), **suffixe** (racine après) ; **largeur** = niveau par niveau avec une **file**.
- **Chercher partout dans un ABR.** On exploite `gauche` $<$ `nœud` $\leq$ `droite` pour n'explorer **qu'un** côté ; sinon on perd tout l'intérêt.
- **Perdre le nœud inséré.** Écrire `inserer(a.gauche, x)` sans réaffecter : quand `a.gauche` vaut `None`, le nouveau nœud n'est accroché nulle part. *Le réflexe :* `a.gauche = inserer(a.gauche, x)`, et `a = inserer(a, x)` à l'appel.
- **Hésiter sur un doublon.** Appliquer la convention de l'énoncé ; ici, une valeur égale va **à droite**.
- **Oublier de combiner / renvoyer** dans la récursion (comme en récursivité) : `return 1 + taille(g) + taille(d)`.
- **Croire qu'un ABR est toujours équilibré.** Le coût d'une recherche est la **hauteur** : $\log_2 n$ *si* l'arbre est équilibré, mais jusqu'à n s'il est « en peigne ».

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

☐ employer le **vocabulaire** (racine, nœud, feuille, sous-arbre, taille, hauteur) → ex. 1, 2 ;

☐ calculer **taille**, **hauteur**, nombre de feuilles par une fonction **récursive** → ex. 8, 9, 10 ;

☐ **encadrer** la hauteur d'un arbre binaire de n nœuds, de l'arbre filiforme à l'arbre parfait → ex. 26 ;

☐ écrire et reconnaître les **quatre parcours** (préfixe, infixe, suffixe, largeur / file) → ex. 3 à 7, 24 ;

☐ manipuler un arbre via la classe `Noeud` ou une **interface** donnée → ex. 2, 17, 19 ;

☐ **rechercher** et **insérer** dans un **ABR** (à la main et en Python, doublon à droite), et relier infixe ↔ tri → ex. 11, 12, 13, 16, 25 ;

☐ justifier le **coût** d'une recherche ou d'une insertion dans un ABR (la hauteur) → ex. 11, 13, 16, 26.

Vers le Grand Oral

Quelques pistes de questions que ce chapitre permet de préparer (à étayer d'un exemple concret) :

- **Pourquoi un arbre binaire de recherche accélère-t-il une recherche ?** *(hauteur $\approx \log_2 n$, même idée que la dichotomie ; la limite de l'arbre déséquilibré.)*
- **Récursivité et arbres : un mariage naturel ?** *(un arbre est défini récursivement ; presque tout algorithme sur les arbres l'est aussi.)*
- **À quoi servent les arbres dans la vraie vie ?** *(arborescence de fichiers, code de Huffman, index d'une base de données, DOM d'une page Web, arbre de décision.)*

EXERCICES

Mode d'emploi

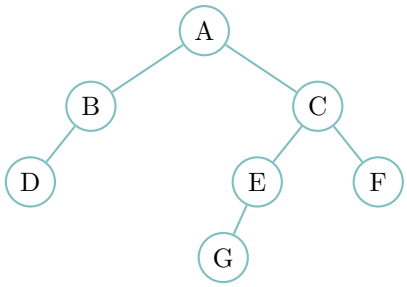
- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 32.
- On **admet** l'implémentation par la classe `Noeud` ci-dessous ; l'arbre vide est `None`.
- **Convention** de hauteur : on compte les **nœuds** (racine seule → hauteur 1, arbre vide → 0), sauf mention contraire.

```
1  class Noeud:
2      def __init__(self, valeur, gauche=None, droite=None):
3          self.valeur = valeur
4          self.gauche = gauche        # un Noeud, ou None
5          self.droite = droite        # un Noeud, ou None
```


Lire un arbre ↑cours

★ **Exercice 1** *Vocabulaire* ▶ corrigé p. 831 • essentiel

On considère l'arbre binaire suivant. ▷ coup de pouce



1. Donner la **racine**, puis la liste des **feuilles**.
2. Donner la **taille** et la **hauteur** de l'arbre.
3. Quel est le **père** de `G` ? Quels sont les **fils** de `C` ?
4. Donner la **profondeur** du nœud `E`.
5. Décrire le **sous-arbre** de racine `C` (ses nœuds).

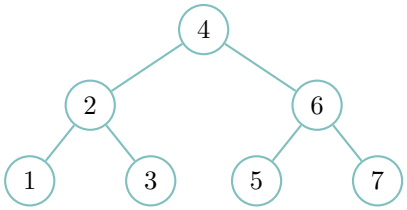
★ **Exercice 2** *Construire à partir d'une description* ▶ corrigé p. 831

▶ Dessiner l'arbre `a = Noeud(5, Noeud(3, Noeud(2)), Noeud(8, None, Noeud(9)))`, puis donner sa taille et sa hauteur. ▷ coup de pouce

Parcourir un arbre ↑ cours

★ **Exercice 3** *Les quatre parcours* ▶ corrigé p. 831 • essentiel

On considère l'arbre suivant.



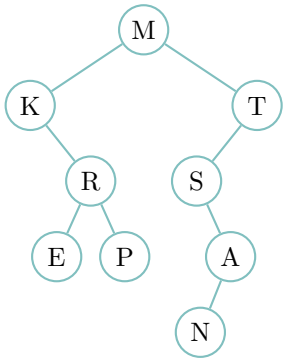
Donner l'ordre de visite des nœuds pour les parcours : **préfixe**, **infixe**, **suffixe**, et **en largeur**. Que remarque-t-on pour le parcours infixe ?

▷ coup de pouce

5

★★ **Exercice 4** *Parcours d'un arbre irrégulier* ▶ corrigé p. 832 • essentiel

L'arbre de l'exercice précédent était **complet** : tous ses niveaux étaient remplis. Celui-ci ne l'est pas : certains nœuds n'ont qu'un fils (à gauche ou à droite), et les branches n'ont pas toutes la même longueur.

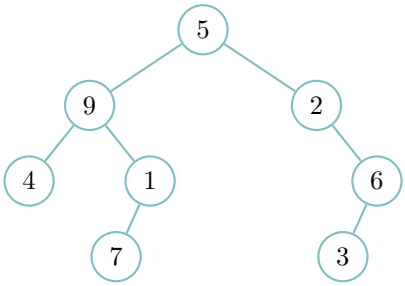


1. Donner l'ordre de visite des nœuds pour les parcours **préfixe**, **infixe**, **suffixe** et **en largeur**.
2. Le premier nœud visité par le parcours infixe est-il une feuille ? Et le dernier ?
3. Sur un arbre non vide **quelconque**, pour chacun des trois parcours en profondeur, le premier nœud visité est-il toujours une feuille ? Et le dernier ? Justifier.

▷ coup de pouce

★★ **Exercice 5** *Retrouver le parcours, retrouver l'arbre* ▶ corrigé p. 832

1. On considère l'arbre ci-dessous.



Parmi les cinq suites suivantes, quatre sont ses parcours préfixe, infixe, suffixe et en largeur ; la cinquième n'est **aucun** d'eux. Associer chaque suite à son parcours, puis expliquer pourquoi la suite restante n'est pas un parcours, sans recalculer les quatre parcours en entier.

a) 4 7 1 9 3 6 2 5 **b)** 5 9 2 4 1 6 7 3 **c)** 5 9 1 4 7 2 6 3

d) 4 9 7 1 5 2 3 6 **e)** 5 9 4 1 7 2 6 3

2. On ne dispose plus du dessin. Un arbre binaire, dont les nœuds portent des lettres toutes différentes, a pour parcours **préfixe** `S T U V W X Y Z` et pour parcours **infixe** `U T V S X Z Y W`. Dessiner cet arbre, puis donner son parcours **suffixe**.

3. Montrer, à l'aide d'un exemple, que le parcours préfixe **seul** ne suffit pas à retrouver un arbre.

▷ coup de pouce

★★ **Exercice 6** *Programmer les parcours en profondeur* ▶ corrigé p. 833

▶ Écrire une fonction `prefixe(a)` qui **affiche** les valeurs de `a` dans l'ordre préfixe. Indiquer ensuite la (seule) ligne à déplacer pour obtenir le parcours **infixe**, puis le parcours **suffixe**.

▷ coup de pouce

★★★ **Exercice 7** *Défi — parcours en largeur* ▶ corrigé p. 833 ≈ 20 min

▶ Écrire une fonction `largeur(a)` qui affiche les valeurs de `a` **niveau par niveau**, de gauche à droite. On s'appuiera sur une `File` (interface `enfiler`, `defiler`, `est_vide`). *(On admet la classe* `File`*.)*

▷ coup de pouce ▷ début de solution

Mesurer un arbre ↑ cours

★ **Exercice 8** *Taille* ▶ corrigé p. 833

▶ Écrire une fonction récursive `taille(a)` qui renvoie le nombre de nœuds de l'arbre `a`. *(Rappel : l'arbre vide* `None` *a pour taille* `0`*.)*

▷ coup de pouce

★★ **Exercice 9** *Hauteur* ▶ corrigé p. 834 • essentiel

▶ Écrire une fonction récursive `hauteur(a)` qui renvoie la hauteur de `a` (convention : compter les nœuds ; l'arbre vide a pour hauteur `0`).

▶ Un sujet de bac définit la hauteur en comptant les **arêtes** (une feuille seule a pour hauteur 0). Écrire `hauteur_aretes(a)` selon cette convention. Quelle valeur faut-il donner à l'arbre vide ?

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 10** *Nombre de feuilles* ▶ corrigé p. 834

▶ Écrire une fonction `nb_feuilles(a)` qui renvoie le nombre de feuilles de `a`. *(Un nœud est une feuille si ses deux fils sont* `None`*.)*

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Arbres binaires de recherche ↑ cours

★★ **Exercice 11** *Rechercher dans un ABR* ▶ corrigé p. 834 • essentiel

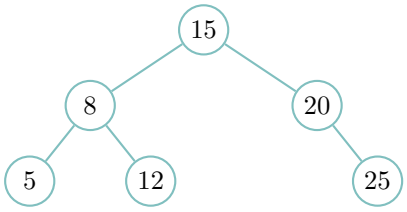
▶ Écrire une fonction récursive `recherche(a, x)` qui renvoie `True` si la valeur `x` est présente dans l'**ABR** `a`, `False` sinon, en profitant de l'ordre de l'ABR (on ne parcourt **pas** tout l'arbre). Combien de nœuds, au maximum, la recherche visite-t-elle dans un ABR de hauteur `h` ?

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 12** *Insérer dans un ABR* ▶ corrigé p. 835 • essentiel

On part de l'arbre binaire de recherche suivant.



1. Redessiner l'arbre après insertion, **dans cet ordre**, des valeurs `10`, `18` puis `2`. ▷ coup de pouce
2. Quel parcours de l'ABR obtenu affiche les valeurs dans l'ordre **croissant** ?

5

★★ **Exercice 13** *Programmer l'insertion dans un ABR* ▶ corrigé p. 835

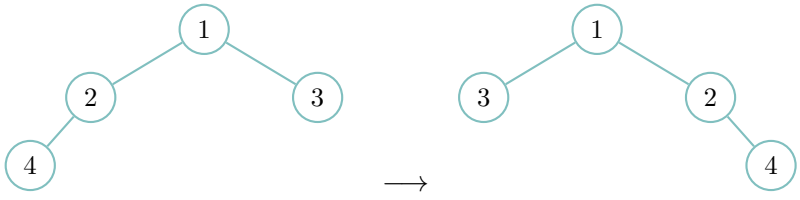
On reprend la classe `Noeud` du cours. Convention du cours : une valeur **égale** à celle du nœud va **à droite**.

1. ▶ Écrire une fonction récursive `inserer(a, x)` qui insère `x` à sa place dans l'ABR `a` et **renvoie** l'arbre obtenu. ▷ coup de pouce ▷ début de solution
   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.
2. ▶ Partant de l'arbre vide, insérer dans cet ordre `5`, `3`, `8`, `3`, `9`, `5`. Dessiner l'ABR obtenu : où sont placés les deux doublons ? Vérifier que le parcours infixe donne les valeurs triées.
3. ▶ En déduire une fonction `tri_abr(t)` qui renvoie une nouvelle liste contenant les valeurs de la liste `t` triées (doublons compris), à l'aide d'un ABR. ▷ coup de pouce
4. Dessiner l'ABR obtenu en insérant `1, 2, 3, 4, 5, 6, 7` dans cet ordre, puis celui obtenu avec `4, 2, 6, 1, 3, 5, 7`. Donner la hauteur de chacun. Combien de nœuds, au pire, une recherche visite-t-elle dans chacun ?

Pour aller plus loin ↑ cours

★★★ **Exercice 14** *Défi — l'arbre miroir* ▶ corrigé p. 836 ≈ 15 min

▶ Écrire une fonction `miroir(a)` qui renvoie un **nouvel** arbre, image de `a` dans un miroir : à chaque nœud, les sous-arbres gauche et droit sont échangés (récursivement). ▷ coup de pouce ▷ début de solution



▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 15** *Défi — maximum d'un arbre* ▶ corrigé p. 837 ≈ 15 min

▶ Écrire une fonction `maximum(a)` qui renvoie la plus grande valeur d'un arbre `a` **non vide** (attention : arbre *quelconque*, pas forcément un ABR — il faut donc bien tout explorer).

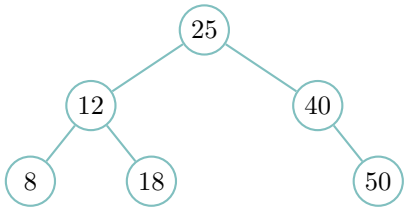
▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Vers le bac ↑cours

★★ **Exercice 16** *ABR : mesures, insertion, recherche (d'après Métropole 2023, jour 2)* ▶ corrigé p. 837 • essentiel ≈ 20 min

On considère l'arbre binaire de recherche suivant.



1. Donner la **taille** et la **hauteur** de cet ABR.
2. Le redessiner après l'ajout des valeurs `15` puis `30`, dans cet ordre.
3. Pour obtenir les valeurs dans l'ordre croissant, quel parcours faut-il utiliser ? *(A) largeur ; (B) préfixe ; (C) infixe ; (D) suffixe.*
4. On dispose d'une classe `ABR` avec les méthodes `est_vide()`, `racine()`, `sg()` (sous-arbre gauche), `sd()` (sous-arbre droit). ▶ Recopier et compléter les lignes 5, 7 et 9 de la méthode récursive `present`, qui renvoie `True` si `x` est dans l'arbre. ▷ coup de pouce

```
1 def present(self, x):
2     if self.est_vide():
3         return False
4     elif self.racine() == x:
5         return ...              # ligne 5
6     elif self.racine() < x:
7         return self.sd(). ...   # ligne 7
8     else:
9         return ...              # ligne 9
```

★★ **Exercice 17** *Un arbre binaire par une classe (d'après Sujet zéro 2023, sujet B)* ▶ corrigé p. 837 ≈ 20 min

On donne la classe `ArbreBinaire` suivante : chaque objet a une `valeur` et deux sous-arbres `enfant_gauche` et `enfant_droit` (`None` au départ).

```
1  class ArbreBinaire:
2      def __init__(self, valeur):
3          self.valeur = valeur
4          self.enfant_gauche = None
5          self.enfant_droit = None
6
7      def insert_gauche(self, valeur):
8          if self.enfant_gauche is None:
9              self.enfant_gauche = ArbreBinaire(valeur)
10         else:
11             new_node = ArbreBinaire(valeur)
12             new_node.enfant_gauche = self.enfant_gauche
13             self.enfant_gauche = new_node
14
15     def insert_droit(self, valeur):
16         if self.enfant_droit is None:
17             self.enfant_droit = ArbreBinaire(valeur)
18         else:
19             new_node = ArbreBinaire(valeur)
20             new_node.enfant_droit = self.enfant_droit
21             self.enfant_droit = new_node
```

```
22
23      def get_valeur(self):
24          return self.valeur
25
26      def get_gauche(self):
27          return self.enfant_gauche
28
29      def get_droit(self):
30          return self.enfant_droit
```

1. Donner un exemple d'**attribut** et un exemple de **méthode** de cette classe.

2. On exécute les instructions suivantes. Dessiner l'arbre obtenu.

```
1 r = ArbreBinaire(15)
2 r.insert_gauche(6)
3 r.insert_droit(18)
4 r.get_gauche().insert_droit(10)
```

3. ▶ Écrire une fonction `taille(arbre)` qui renvoie le nombre de nœuds de l'arbre, en utilisant **uniquement** les accesseurs `get_gauche` et `get_droit` (l'arbre vide est `None`). ▷ coup de pouce

   ▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

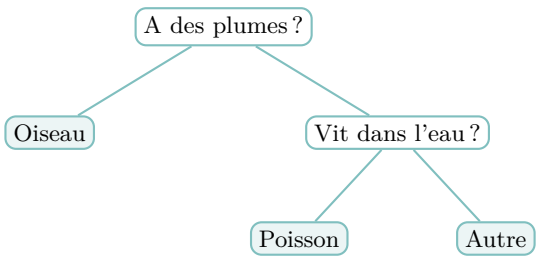
★★ Exercice 18

Un arbre de décision (d'après Amérique du Nord 2025, jour 1) ▶ corrigé p. 838 ≈ 30 min

Attention : dans cet exercice, la classe `Noeud` *n'est pas celle du mode d'emploi.* Un arbre de décision est représenté par deux classes : les **nœuds** portent une question, les **feuilles** portent un résultat.

```
1 class Noeud:
2     def __init__(self, question, sioui, sinon):
3         self.question = question
4         self.sioui = sioui        # un Noeud ou une Feuille
5         self.sinon = sinon        # un Noeud ou une Feuille
6
7 class Feuille:
8     def __init__(self, resultat):
9         self.resultat = resultat  # une chaine de caracteres
```

1. ▶ Écrire le code Python qui construit l'arbre de décision ci-dessous et l'affecte à une variable `arbre`.



   (Convention : la branche de ***gauche*** *correspond à la réponse « oui », celle de* ***droite*** *à « non ».)*

2. On veut une méthode `est_resultat()` : elle renvoie `True` pour une `Feuille`, `False` pour un `Noeud`. ▶ Écrire cette méthode dans **chacune** des deux classes.

3. ▶ Écrire une méthode `nb_resultats()` dans **chacune** des deux classes, renvoyant le **nombre de résultats** (diagnostics) atteignables : pour une `Feuille`, il vaut `1` ; pour un `Noeud`, c'est la somme de ceux de ses deux sous-arbres `sioui` et `sinon`. ▷ coup de pouce

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

4. ▶ *(Défi)* Écrire une fonction `diagnostic(arbre)` qui pose les questions à l'utilisateur (fonction `input`) en descendant l'arbre, et renvoie le résultat de la feuille atteinte. *(On répondra* `"oui"` *ou* `"non"`*.)* ▷ coup de pouce

★★ Exercice 19 *Un arbre représenté par des listes (d'après Amérique du Sud 2023, jour 1)*

▶ corrigé p. 839 ≈ 30 min

Ici, un arbre (pas forcément binaire) est représenté par une liste `[e, lst_sa]` où `e` est l'étiquette du nœud et `lst_sa` la **liste de ses sous-arbres**. L'arbre vide est `[]`, une feuille est `[e, []]`.

```
1 n2 = [2, []]
2 n5 = [5, []]
3 n8 = [8, []]
4 n1 = [1, [n2, n5]]
5 a  = [4, [n1, n8]]
```

1. Dessiner l'arbre représenté par la variable `a`.

2. Le **poids** d'un arbre est la somme des étiquettes de ses nœuds ; on le calcule par un parcours **en largeur** avec une file, manipulée par les fonctions `creer_file()`, `enfiler(file, x)`, `defiler(file)` (retire et renvoie l'élément de tête) et `est_vide(file)`. ▶ Recopier et compléter les lignes 7, 8 et 11. ▷ coup de pouce

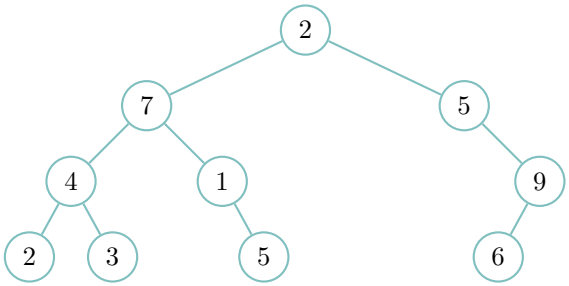
```
1  def poids(arbre):
2      if arbre == []:
3          return 0
4      p = 0
5      file = creer_file()
6      enfiler(file, arbre)
7      while ... :                    # ligne 7
8          arbre = ...                # ligne 8
9          for sous_arbre in arbre[1]:
10             enfiler(file, sous_arbre)
11         p = ...                    # ligne 11
12     return p
```

3. *(défi ★★★)* ▶ Un arbre est un **mobile** si *tous* ses sous-arbres ont le **même poids** et sont eux-mêmes des mobiles (l'arbre vide et une feuille sont des mobiles). Écrire une fonction récursive `est_mobile(arbre)` qui renvoie un booléen ; on pourra utiliser `poids`. ▷ coup de pouce

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 20** *La plus grande somme racine-feuille (d'après Amérique du Sud 2022, jour 2)* ▶ corrigé p. 839 ≈ 20 min

Dans un arbre binaire étiqueté par des entiers, un **chemin racine-feuille** part de la racine et descend, de fils en fils, jusqu'à une feuille. Sa **somme** est la somme des étiquettes rencontrées. On cherche la **plus grande** de ces sommes.



1. Déterminer « à la main » la plus grande somme racine-feuille de cet arbre.

2. ▶ Écrire une fonction récursive `somme_max(a)` qui renvoie la plus grande somme racine-feuille d'un arbre `a` **non vide** (implémentation par la classe `Noeud`). ▷ coup de pouce ▷ début de solution

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 21** *Puissance 4 : l'arbre de coups et l'algorithme min-max (d'après Amérique du Nord 2026, jour 1, parties B et C)* ▶ corrigé p. 840 ≈ 45 min

Suite de l'exercice « Puissance 4 : la grille de jeu » du chapitre POO. Une grille de puissance 4 (6 lignes, 7 colonnes) est un objet de la classe `Grille` ; les deux joueurs sont notés 1 et 2. On dispose des méthodes suivantes :

- `joue(self, colonne, joueur)` place un pion du joueur dans la colonne et renvoie `True`, ou renvoie `False` si la colonne est pleine ;
- `score(self)` renvoie le score de la grille : plus il est grand, plus la position est favorable au joueur 2 ; plus il est petit (négatif), plus elle est favorable au joueur 1.

On veut programmer l'algorithme **min-max**, qui permet à un joueur d'optimiser ses chances de victoire : à chaque coup possible, on associe un score calculé en fonction de tous les coups futurs possibles, les siens mais aussi ceux de l'adversaire ; on choisit le coup de meilleur score. Pour simuler l'ensemble des coups possibles, on utilise un **arbre** dont les nœuds correspondent à des coups : chaque fils d'un nœud est un coup possible juste après le coup du nœud.

Partie 1 — L'arbre de coups

On associe un arbre de coups à un joueur et à une grille de jeu. Il simule tous les coups possibles pour le joueur et son adversaire, et calcule le score de chaque coup selon l'algorithme min-max :

- on ne calcule qu'un nombre maximal donné de coups à l'avance, noté `niveau_max` : c'est la profondeur maximale de l'arbre (la racine est au niveau 0). `niveau_max` est une constante accessible comme variable globale ;
- si un nœud correspond à un coup **gagnant**, son score vaut $-(100+10\times(\texttt{niveau_max}-\texttt{niveau}))$ pour le joueur 1 et $100+10\times(\texttt{niveau_max}-\texttt{niveau})$ pour le joueur 2, où `niveau` est la profondeur du nœud dans l'arbre ;
- si un nœud est au niveau `niveau_max`, son score est le score de la grille (méthode `score` de la classe `Grille`) ;
- dans tous les autres cas, le score d'un nœud est le **minimum** des scores de ses fils s'il s'agit de coups du joueur 1, et le **maximum** des scores de ses fils s'il s'agit de coups du joueur 2.

À son tour, le joueur 1 choisit donc le coup de niveau 1 de plus **petit** score, et le joueur 2 celui de plus **grand** score.

Attention : dans cet exercice, la classe `Noeud` *n'est pas celle du mode d'emploi.* Pour construire l'arbre, on utilise une classe `Noeud` à trois attributs : `colonne` (soit `-1`, soit le numéro de la colonne où le coup est joué, de 0 à 6), `score` (le score associé au coup) et `suivants` (la liste des nœuds fils). La racine ne représente pas un coup : son attribut `colonne` vaut `-1` et ses fils représentent tous les premiers coups possibles du joueur 1.

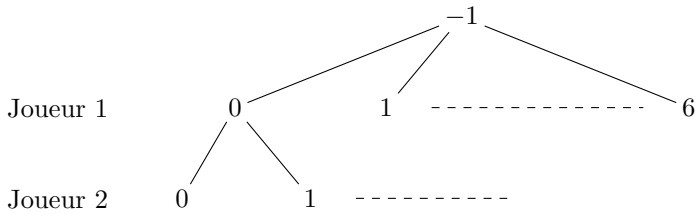


Figure 1. Schéma partiel de l'arbre de coups pour `niveau_max = 2` (valeur de l'attribut `colonne`).

1. Recopier et compléter le constructeur de la classe `Noeud`, qui crée un nœud symbolisant un coup joué dans la colonne `colonne`, de score nul et sans aucun fils.

```
1 class Noeud:
2     def __init__(self, colonne):
3         ...
4         ...
5         ...
```

2. Écrire une méthode `colonne_score_min(self)` de la classe `Noeud` qui renvoie le couple `(colonne, score)` du fils de **plus petit score** (l'un d'eux en cas d'égalité). On suppose que le nœud a au moins un fils.

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

On suppose écrite de même une méthode `colonne_score_max` (fils de score le plus grand), et ajoutées à la classe `Grille` les méthodes suivantes :

- `gagnant(self)` renvoie le numéro du joueur gagnant (1 ou 2) s'il existe un alignement de quatre de ses pions dans la grille, ou `0` s'il n'y a aucun gagnant (on suppose qu'il ne peut y avoir qu'un gagnant) ;
- `copie_grille(self)` renvoie une grille de jeu, copie exacte de la grille représentée par l'objet : modifier la copie ne modifie pas l'original.

3. La méthode `calcule_score(self, niveau, joueur, grille)` de la classe `Noeud` attribue un score au nœud conformément à l'algorithme min-max : `niveau` est la profondeur du nœud, `joueur` le numéro du joueur qui joue les coups étudiés à partir de ce nœud, et `grille` un objet `Grille` représentant le jeu juste après le coup correspondant au nœud. Recopier et compléter ce code. ▷ coup de pouce ▷ début de solution

```
1  def calcule_score(self, niveau, joueur, grille):
2      g = grille.gagnant()
3      if g == 1:
4          self.score = ...
5      elif g == 2:
6          self.score = ...
7      elif niveau == niveau_max:
8          self.score = ...
9      else:
10         for colonne in range(7):
```

```
            grille2 = grille.copie_grille()
            if grille2.joue(colonne, joueur):
                nouveau_noeud = ...
                self.suivants.append(...)
                nouveau_noeud.calcule_score(...)
        if joueur == 1:
            self.score = ...
        else:
            self.score = ...
```

4. Indiquer pourquoi il n'est pas réaliste d'utiliser cet algorithme pour explorer l'ensemble des parties, ce qui correspond à `niveau_max = 42`.

Partie 2 — Choix du meilleur coup à jouer

5. ▶ Utiliser ce qui précède et la classe `Noeud` pour écrire une fonction `choisit_coup(grille, joueur)` qui prend en paramètres une grille de jeu et le numéro d'un joueur, et renvoie le numéro de la colonne où ce joueur devrait jouer pour optimiser ses chances de victoire selon l'algorithme min-max. ▷ coup de pouce

Critiquer une réponse d'IA ↑ cours

★★ **Exercice 22** *Un test d'ABR proposé par un assistant d'IA* ▶ corrigé p. 841

Un élève demande à un assistant d'IA : « Écris une fonction Python `est_abr(a)` qui renvoie `True` si l'arbre binaire `a` (classe `Noeud` du cours, attributs `valeur`, `gauche`, `droite`) est un arbre binaire de recherche. » Voici la réponse obtenue :

```
def est_abr(a):
    if a is None:
        return True                    # l'arbre vide est un ABR
    if a.gauche is not None and a.gauche.valeur >= a.valeur:
        return False
    if a.droite is not None and a.droite.valeur < a.valeur:
        return False
    return est_abr(a.gauche) and est_abr(a.droite)
```

« Cette fonction vérifie, pour chaque nœud, que son fils gauche est plus petit et son fils droit plus grand ou égal, puis fait de même récursivement dans les deux sous-arbres : c'est exactement la définition d'un ABR. »

1. La réponse est-elle correcte ? ▶ La tester sur l'arbre `Noeud(8, Noeud(3, Noeud(1), Noeud(12)), Noeud(10))` : le dessiner d'abord, dire s'il est un ABR, puis comparer avec ce que renvoie la fonction.
2. Localiser et corriger l'erreur. ▷ coup de pouce
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

S'entraîner à l'oral ↑cours

★★ **Exercice 23** *Expliquer en deux minutes* ▶ corrigé p. 842

Au Grand Oral comme devant l'examinateur de l'épreuve pratique, il faut savoir **expliquer** une notion clairement, sans lire ses notes. Choisir l'un des sujets suivants (ou le tirer au sort) :

Sujet 1. Expliquer à un camarade de Première ce qu'est un **arbre binaire de recherche** et pourquoi y chercher une valeur est rapide.

Sujet 2. Expliquer les trois parcours en profondeur (**préfixe**, **infixe**, **suffixe**) sur un petit arbre, et ce que donne le parcours infixe d'un ABR.

Sujet 3. Expliquer pourquoi un ABR peut être très efficace, ou pas plus efficace qu'une liste, selon l'**ordre** dans lequel on insère les valeurs.

Préparation (5 min, seul). Noter au brouillon **trois idées** et **un exemple**, rien de plus : comme au Grand Oral, ces notes servent d'aide-mémoire. **Exposé (2 min, chronométré).** Debout, face à un camarade, **sans lire ses notes** et sans rien écrire : tout passe par la parole. **Retour (2 min).** Le camarade pose une question de relance (on peut alors s'aider d'un schéma), remplit la grille ci-dessous et donne un conseil ; puis on échange les rôles. Cette grille reprend les critères d'évaluation du Grand Oral. ▷ coup de pouce

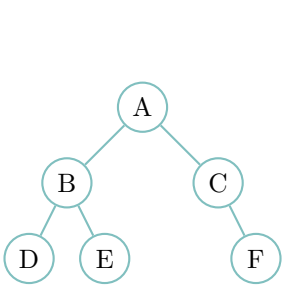
Grille entre camarades	**Oui**	**En partie**	**Pas encore**
Clair — audible, posé ; chaque mot technique est expliqué	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	☐	☐	☐
Construit — un fil conducteur, tenu en deux minutes, sans lire	☐	☐	☐
Un conseil pour la prochaine fois :			

Entraînement ↑cours

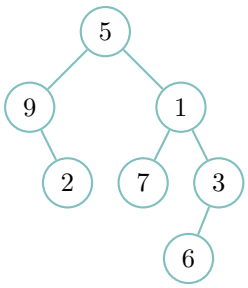
Des séries courtes pour rendre une méthode automatique : même consigne, données différentes. Les réponses sont brèves dans le corrigé.

★ **Exercice 24** *Parcours en série* ▶ corrigé p. 842 • essentiel

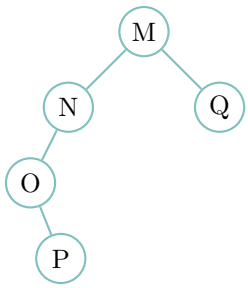
Pour chacun des trois arbres ci-dessous, donner l'ordre de visite des nœuds pour les parcours **préfixe**, **infixe**, **suffixe** et **en largeur**. ▷ coup de pouce



a)



b)



c)

★ **Exercice 25** *Insertions en série dans un ABR* ▶ corrigé p. 843 • essentiel

Pour chaque suite de valeurs, partir d'un ABR **vide**, insérer les valeurs **dans l'ordre donné** (convention du cours : une valeur égale à celle d'un nœud va à droite), puis dessiner l'ABR obtenu et donner sa **hauteur**. ▷ coup de pouce

a) `7, 3, 9, 1, 5, 8`
b) `10, 15, 12, 20, 5, 13`
c) `4, 2, 6, 4, 5`
d) `1, 2, 3, 4, 5`

Encadrer la hauteur ↑ cours

★★ **Exercice 26** *Encadrer la hauteur* ▶ corrigé p. 843

On compte la hauteur en nœuds (convention du cours) sauf à la question 5.

1. Un arbre binaire a 15 nœuds. Quelle est sa plus petite hauteur possible ? sa plus grande ? Dessiner, sur le cahier, la forme d'un arbre de 15 nœuds dans chacun des deux cas (les valeurs n'importent pas).

2. Combien de nœuds peut contenir, au plus, un arbre binaire de hauteur 5 ? au moins ?

3. Un ABR contient un million de valeurs. Entre quelles bornes sa hauteur est-elle comprise ? En déduire le nombre maximal de nœuds visités par une recherche si l'arbre a la plus petite hauteur possible, puis s'il a la plus grande. ▷ coup de pouce

4. On insère les valeurs $1, 2, 3, \ldots, 100$, dans cet ordre, dans un ABR vide. Quelle est sa hauteur ? Proposer un ordre d'insertion des valeurs 1 à 7 qui donne un ABR de hauteur minimale.

5. Reprendre la question 1 avec la convention qui compte les **arêtes**.

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Une feuille est un nœud sans fils. Pour la hauteur, chercher le plus long chemin de la racine à une feuille et appliquer la convention du mode d'emploi (on compte les nœuds). Pour la profondeur, relire la définition du cours : que compte-t-on depuis la racine ? ↑ énoncé

Ex. 2 – piste Le premier paramètre de `Noeud` est la valeur, le deuxième le sous-arbre gauche, le troisième le sous-arbre droit (absent ou `None` : pas de fils de ce côté). Dessiner d'abord la racine, puis traiter chaque sous-arbre de la même façon. ↑ énoncé

Ex. 3 – piste En préfixe, en infixe et en suffixe, la racine est visitée avant, entre ou après ses deux sous-arbres, et cette règle s'applique à **chaque** sous-arbre. En largeur, on lit l'arbre niveau par niveau, de gauche à droite. ↑ énoncé

Ex. 4 – piste Q1 : un nœud sans fils gauche (comme `K`) a un sous-arbre gauche **vide** : en infixe, on le visite donc tout de suite, puis on parcourt son sous-arbre droit. Avancer lentement, nœud par nœud, sans se fier à la position sur le dessin. Q3 : en préfixe, un nœud est-il visité avant ou après ses fils ? Et en suffixe ? ↑ énoncé

Ex. 5 – piste Q1 : où la racine `5` apparaît-elle dans chacun des quatre parcours ? Cela sépare déjà les suites en groupes. Q2 : la première lettre du préfixe est la racine ; dans l'infixe, elle sépare les nœuds du sous-arbre gauche (à sa gauche) de ceux du sous-arbre droit (à sa droite). On recommence ensuite sur chaque sous-arbre. ↑ énoncé

Ex. 6 – piste Condition d'arrêt : l'arbre vide, on n'affiche rien. Sinon, trois instructions : afficher la racine, parcourir le sous-arbre gauche, parcourir le sous-arbre droit. Dans quel ordre pour le préfixe ? ↑ énoncé

Ex. 7 – piste Pas de récursivité ici. On enfile la racine ; puis, tant que la file n'est pas vide, on défile un nœud, on l'affiche et on enfile ses fils non vides, le gauche d'abord. ↑énoncé

Ex. 7 – début de solution
```
def largeur(a):
    if a is None:
        return
    f = File()
    f.enfiler(a)
    while not f.est_vide():
```
↑énoncé

Ex. 8 – piste Condition d'arrêt : l'arbre vide. Sinon, comment obtenir la taille d'un arbre à partir de sa racine et des tailles de ses deux sous-arbres ? ↑énoncé

Ex. 9 – piste Condition d'arrêt : l'arbre vide. Sinon, la hauteur se déduit de la plus **grande** des hauteurs des deux sous-arbres (fonction `max`). Pour les arêtes : quelle valeur donner à l'arbre vide pour qu'une feuille obtienne la hauteur 0 avec la même formule ? ↑énoncé

Ex. 10 – piste Deux cas immédiats : l'arbre vide, et le nœud qui est lui-même une feuille. Dans les autres cas, que faire des nombres de feuilles des deux sous-arbres ? ↑énoncé

Ex. 11 – piste Comparer `x` à la racine : trois cas. Un seul appel récursif, du côté où `x` peut encore se trouver. Pour le coût : combien de nœuds au plus sur un chemin de la racine à une feuille ? ↑énoncé

Ex. 12 q. 1 – piste Pour chaque valeur, partir de la racine et descendre : à gauche si la valeur est plus petite que le nœud, à droite sinon (y compris en cas d'égalité : convention du cours), jusqu'à trouver une place vide. Une nouvelle valeur devient toujours une feuille. ↑énoncé

Ex. 13 q. 1 – piste Q1 : trois cas : arbre vide (on renvoie un nouveau `Noeud(x)`) ; `x` plus petit que la racine (on insère à gauche) ; sinon (on insère à droite). Ne pas oublier de **réaffecter** le sous-arbre modifié, puis de renvoyer `a`. ↑énoncé

Ex. 13 q. 1 – début de solution Q1 :
```
if x < a.valeur:
    a.gauche = inserer(a.gauche, x)
```
↑énoncé

Ex. 13 q. 3 – piste Q3 : insérer toutes les valeurs, puis écrire une variante du parcours infixe qui **renvoie une liste** : `infixe(a.gauche) + [a.valeur] + infixe(a.droite)`. ↑

Ex. 14 – piste Que vaut le miroir de l'arbre vide (condition d'arrêt) ? Sinon, on construit un **nouveau** `Noeud` de même valeur : que doit-on placer à sa gauche, et à sa droite ? ↑énoncé

Ex. 14 – début de solution
```
def miroir(a):
    if a is None:
        return None
    return Noeud(a.valeur, ..., ...)
```
↑énoncé

Ex. 15 – piste Prendre le `max` de la racine et des maxima des deux sous-arbres **non vides** : tester `a.gauche is not None` avant d'appeler `maximum(a.gauche)`, et de même à droite. ↑

Ex. 15 – début de solution
```
def maximum(a):
    m = a.valeur
    if a.gauche is not None:
        m = max(m, maximum(a.gauche))
```
↑énoncé

Ex. 16 q. 4 – piste Ligne 7 : si la racine est plus petite que `x`, dans quel sous-arbre `x` peut-il encore se trouver ? On y relance la même méthode. Ligne 9 : même raisonnement de l'autre côté. ↑énoncé

Ex. 17 q. 3 – piste Même structure que la fonction `taille` du cours : seule change la façon d'obtenir les sous-arbres (des appels de méthodes au lieu d'attributs). ↑énoncé

Ex. 18 q. 3 – piste Q3 : chaque classe a sa propre version de la méthode. Dans la classe `Noeud`, sur quels objets faut-il appeler la méthode `nb_resultats` ? Où se trouve alors la condition d'arrêt de cette récursion ? ↑énoncé

Ex. 18 q. 4 – piste Q4 : une boucle `while` qui descend tant que l'objet courant n'est pas un résultat (méthode `est_resultat`) ; selon la réponse donnée à la question du nœud courant, on passe à `sioui` ou à `sinon`. ↑énoncé

Ex. 19 q. 2 – piste Q2 : reprendre le schéma du parcours en largeur du cours : quelle condition fait tourner la boucle ? D'où vient l'arbre traité à chaque tour ? Où se trouve l'étiquette dans la liste `[e, lst_sa]` ? ↑énoncé

Ex. 19 q. 3 – piste Q3 : parcourir la liste des sous-arbres : chacun doit avoir le même poids que le premier, **et** être lui-même un mobile (appel récursif). Si aucun ne fait défaut, renvoyer `True`. ↑énoncé

Ex. 20 q. 2 – piste La plus grande somme vaut `a.valeur` plus la meilleure des sommes des sous-arbres **non vides**. Condition d'arrêt : la feuille, dont la somme est sa seule valeur. Attention au nœud qui n'a qu'un seul fils. ↑énoncé

Ex. 20 q. 2 – début de solution
```
def somme_max(a):
    if a.gauche is None and a.droite is None:
        return a.valeur
    if a.gauche is None:
        return a.valeur + somme_max(a.droite)
```
↑énoncé

Ex. 21 q. 3 – piste Q3 : les trois premiers cas sont des scores directs : relire les règles de la partie 1. Dans la boucle : on crée le nœud du coup joué dans `colonne`, on l'ajoute aux fils, puis l'appel récursif se fait au niveau suivant, pour l'**autre** joueur, sur `grille2`. ↑

Ex. 21 q. 3 – début de solution Q3 : l'appel récursif s'écrit
`nouveau_noeud.calcule_score(niveau + 1, 3 - joueur, grille2)`
(`3 - joueur` vaut 2 si `joueur` vaut 1, et inversement). ↑énoncé

Ex. 21 q. 5 – piste Q5 : créer la racine (colonne `-1`), lui faire calculer son score au niveau `0`, puis choisir parmi ses fils selon le joueur : `colonne_score_min` ou `colonne_score_max`. ↑énoncé

Ex. 22 q. 2 – piste Relire la définition du cours, mot à mot, puis la comparer à ce que teste réellement chacun des deux `if`. ↑énoncé

Ex. 23 – piste Structurer chaque explication en trois temps : une **définition** en une phrase, un **exemple** petit et concret déroulé à voix haute, puis le **piège** (l'erreur que l'auditeur risque de faire). Finir par une phrase qui répond à la question posée. Dessiner mentalement un arbre de trois à cinq nœuds et le décrire à voix haute (« la racine porte 8, son fils gauche… ») : c'est lui qui servira d'exemple. Sujet 3 : à quoi le nombre de comparaisons d'une recherche est-il lié ? ↑énoncé

Ex. 24 – piste Même méthode qu'aux exercices 3 et 4. En profondeur, la racine est visitée avant, entre ou après ses deux sous-arbres, et cette règle s'applique à **chaque** sous-arbre : terminer tout le sous-arbre gauche avant de passer au droit. En largeur, lire l'arbre niveau par niveau, de gauche à droite. ↑énoncé

Ex. 25 – piste Insérer les valeurs une par une. Chacune part de la racine et descend : à gauche si elle est plus petite que le nœud, à droite sinon, jusqu'à une place vide, où elle devient une feuille. ↑énoncé

Ex. 26 q. 3 – piste Chercher la plus petite puissance de 2 qui dépasse 1 000 000 : $2^{10} = 1\,024$, donc 2^{20} dépasse un million. ↑énoncé

TP ET PROJETS

TP 1 Un arbre binaire de recherche, de A à Z

▶ corrigé p. 844

construire, chercher, mesurer, indexer

Durée : 3 h environ (deux séances) · Sur machine, seul ou par deux

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t05-tp-arbres` — **archive .zip**

But du TP

Programmer une classe `ABR` complète (insertion, recherche, minimum et maximum, parcours infixe), puis **mesurer** ce que le cours affirme : un ABR rempli au hasard reste bas, un ABR rempli avec des valeurs triées devient un « peigne » aussi lent qu'une liste. On termine par une application : l'**index** des mots d'un texte, comme à la fin d'un livre. ▶ signale ce qui est à programmer et tester.

Consignes

- Le fichier `tp_arbres_depart.py` est **à télécharger** (lien ci-dessus) : la classe `Noeud` et les fonctions `taille` et `hauteur` du cours, le squelette des classes et, en fin de fichier, des **tests**. Relancer le fichier après chaque partie : chaque ligne `[A FAIRE]` doit devenir `[OK]`.
- **Convention** de hauteur du cours : on compte les nœuds (racine seule → hauteur 1, arbre vide → 0).
- Usage de l'IA, indiqué sur chaque exercice : IA aucune ; IA en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; IA l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

1 Construire un ABR

On enveloppe l'arbre dans une classe `ABR` : l'attribut privé `_racine` désigne le nœud racine (ou `None`), et `_taille` compte les valeurs. L'utilisateur ne manipule jamais les nœuds : il appelle `inserer`, `contient`, `infixe`… Les valeurs sont **sans doublon** : c'est une **variante** de la convention du cours (où une valeur égale est rangée à droite). Ici, comme dans un ensemble, insérer une valeur déjà présente ne fait rien.

★ **Exercice 1** IA *L'ordre d'insertion compte*

1. Dessiner l'ABR obtenu en insérant, dans cet ordre, `15, 8, 20, 5, 12, 25, 10, 18, 2` dans un arbre vide. Donner sa taille et sa hauteur.
2. Dessiner l'ABR obtenu en insérant les **mêmes** valeurs dans l'ordre croissant `2, 5, 8, 10, 12, 15, 18, 20, 25`. Taille ? Hauteur ? Pourquoi parle-t-on de « peigne » ?
3. Proposer un ordre d'insertion de ces neuf valeurs qui donne un arbre de hauteur 4 **différent** de celui de la question 1.

★★ **Exercice 2** *Insérer sans récursivité*

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

▶ Compléter la méthode `inserer(x)` avec une boucle `while` : si l'arbre est vide, le nouveau nœud devient la racine ; sinon, on descend depuis la racine, à gauche si `x` est plus petit que la valeur du nœud courant, à droite s'il est plus grand, jusqu'à trouver une place libre (`None`) où l'on accroche `Noeud(x)`. La méthode renvoie `True` si `x` a été ajouté, `False` s'il était déjà présent (l'arbre est alors inchangé). Penser à mettre `_taille` à jour. ▷ coup de pouce

```
>>> a = ABR()
>>> for v in [15, 8, 20, 5, 12, 25, 10, 18, 2]:
...     a.inserer(v)
>>> a.taille(), a.hauteur()
(9, 4)
>>> a.inserer(12)
False
```

Le test vérifie aussi que `_taille` est égal à `taille(a._racine)`. Quel est l'intérêt de tenir à jour l'attribut `_taille` plutôt que d'appeler la fonction récursive `taille` à chaque fois ?

2 Chercher

★★ **Exercice 3** *Chercher en comptant ses pas*

1. ▶ Écrire la méthode `contient(x)`, **itérative**, qui renvoie le couple `(présent, visites)` : un booléen, et le nombre de nœuds visités pendant la recherche.

```
>>> a.contient(15), a.contient(10), a.contient(13)
((True, 1), (True, 4), (False, 3))
```

2. Justifier, sur l'arbre de la question 1 de l'exercice 1, que `a.contient(13)` visite 3 nœuds. Pourquoi le nombre de visites ne dépasse-t-il jamais la hauteur de l'arbre ?

★ **Exercice 4** *Le plus petit et le plus grand*

▶ Écrire `minimum()` et `maximum()`, qui lèvent `ValueError("arbre vide")` sur un arbre vide (avec `raise`, comme au chapitre *Programmation orientée objet*). Où se trouve la plus petite valeur d'un ABR ? Est-ce forcément une feuille ? ▷ coup de pouce

3 Le parcours infixe trie

★★ **Exercice 5** *Infixe et tri par ABR*

1. ▶ Écrire la méthode `infixe()`, qui renvoie la **liste** des valeurs dans l'ordre du parcours infixe. On pourra définir, *à l'intérieur* de la méthode, une fonction récursive `parcours(n)` qui ajoute les valeurs du sous-arbre de racine `n` à une liste `res`. ▷ coup de pouce
2. Sur l'arbre de l'exercice 1, que renvoie `infixe()` ? Expliquer, à l'aide de la définition d'un ABR, pourquoi le parcours infixe donne **toujours** les valeurs dans l'ordre croissant.
3. ▶ En déduire une fonction `tri_abr(tab)` de **trois lignes** (sans compter la ligne `def`) qui trie une liste. Que deviennent les doublons ?

```
>>> tri_abr([5, 3, 9, 3, 1, 7])
[1, 3, 5, 7, 9]
```

4. ▶ Tester `tri_abr` sur 200 listes aléatoires en comparant son résultat à `sorted(set(t))`, la liste triée sans doublon (`set(t)` est l'**ensemble** des valeurs de `t`, sans répétition : voir l'encadré du défi *Crab Combat*, chapitre 4).

4 Mesurer la hauteur

Le cours affirme qu'un ABR de n valeurs « bien rempli » a une hauteur proche de $\log_2(n)$, mais qu'un ABR déséquilibré peut atteindre une hauteur n. Vérifions-le.

★★ **Exercice 6** *Au hasard ou dans l'ordre*

1. ▶ Écrire `hauteur_triee(n)`, qui renvoie la hauteur de l'ABR obtenu en insérant `0, 1, ..., n-1` dans l'ordre, et `hauteur_aleatoire(n, essais=20)`, qui renvoie la hauteur **moyenne** sur `essais` ABR obtenus en insérant ces mêmes entiers dans un ordre aléatoire (`random.shuffle`).

▷ coup de pouce

2. ▶ Recopier et compléter sur le cahier le tableau suivant (la hauteur triée pour $n = 10\,000$ se déduit sans calcul).

n	10	100	1 000	10 000
$\log_2(n)$ (calculatrice)	3,3			
hauteur moyenne, au hasard				
hauteur, valeurs triées				

3. Que vaut à peu près le rapport entre la hauteur moyenne au hasard et $\log_2(n)$? Comment évolue la hauteur au hasard quand n est multiplié par 10 ?
4. ▶ Tracer avec `matplotlib` trois courbes en fonction de n : la hauteur avec des valeurs triées pour n parmi `10, 20, 50, 100, 200, 500, 1000, 2000` ; la hauteur moyenne au hasard pour les mêmes valeurs, plus `5000` et `10000` ; enfin $\log_2(n)$, approché par une fonction `log2(n)` qui compte le nombre de divisions par 2 nécessaires pour passer de n à 1 ou moins. Les échelles logarithmiques (`plt.xscale("log")` et `plt.yscale("log")`) rendent le graphique lisible.
5. Le fichier de départ commence par `sys.setrecursionlimit(20000)`. Sans cette ligne, quel calcul échouerait, et pourquoi ? *(Penser à la forme de l'arbre et à la fonction* `hauteur`*.)*

★★ **Exercice 7** *Ce que cela coûte vraiment*

▶ On insère les entiers de 0 à 999, une fois dans un ordre aléatoire (arbre `ar`), une fois dans l'ordre (arbre `at`). Pour chaque arbre, calculer le nombre **moyen** et le nombre **maximal** de nœuds visités par `contient(v)` quand `v` parcourt `range(1000)`. Comparer avec la recherche dans une liste non triée, puis avec la recherche dichotomique dans une liste triée (Première). Conclure : quand un ABR est-il intéressant ?

5 Application : l'index d'un texte

À la fin d'un livre, l'**index** donne, pour chaque mot important, les pages où il apparaît, **par ordre alphabétique**. On le construit avec un ABR dont les clés sont des **mots** (les chaînes se comparent avec `<` dans l'ordre du dictionnaire) et dont chaque nœud (classe fournie `NoeudIndex`) porte aussi la liste `lignes` des numéros de lignes où le mot apparaît.

★★ **Exercice 8** *Construire et interroger l'index*

1. ▶ Compléter la méthode `ajouter(mot, ligne)` de la classe `Index` : on descend dans l'ABR comme pour `inserer` ; si le mot est absent, on crée son nœud (et l'on incrémente `_nb_mots`) ; on ajoute ensuite `ligne` à la liste du nœud, sauf si ce numéro y figure déjà (les lignes arrivent dans l'ordre : il suffit de regarder le dernier).

▷ coup de pouce

2. ▶ Écrire `lignes(mot)` (recherche itérative) et `afficher()` (parcours infixe), puis construire l'index du texte fourni avec `idx = indexer(TEXTE)`.

```
>>> idx.lignes("hauteur")
[6, 9, 12]
>>> idx.afficher()
a : 9
arbre : 1, 6, 8
arbres : 11
...
```

3. Combien de mots différents l'index contient-il ? Comparer la hauteur de l'index à $\log_2$ de ce nombre. Les mots d'un texte arrivent-ils « au hasard » ou « triés » ?
4. Où les mots `à`, `élimine` et `équilibré` apparaissent-ils dans l'affichage ? Expliquer (`ord("é")` et `ord("z")` aideront). Proposer une façon d'obtenir un vrai ordre alphabétique.

5

★★★ **Exercice 9** *Défi — toutes les valeurs d'un intervalle*

1. ▶ Écrire la méthode `entre(a, b)` de la classe `ABR`, qui renvoie le couple formé de la liste croissante des valeurs `v` telles que `a <= v <= b`, et du nombre de nœuds visités. Elle ne doit **pas** explorer les sous-arbres qui ne peuvent rien contenir d'utile : quand la valeur d'un nœud est inférieure ou égale à `a`, son sous-arbre gauche est inutile ; quand elle est supérieure ou égale à `b`, son sous-arbre droit l'est.

```
>>> a.entre(9, 19)
([10, 12, 15, 18], 6)
```

2. ▶ Sur les arbres `ar` et `at` de l'exercice 7, comparer le nombre de nœuds visités par `entre(500, 509)`.
3. ▶ *(Application.)* Écrire, dans la classe `Index`, une méthode `commencant_par(prefixe)` qui renvoie les mots de l'index commençant par `prefixe`, dans l'ordre, en élaguant de la même façon : c'est le principe de l'**autocomplétion**. Par exemple, `commencant_par("arb")` renvoie `['arbre', 'arbres']`.

6 Bilan à rédiger

Exercice 10 *Ce que j'ai mesuré*

Rédiger, en une dizaine de lignes, un bilan qui répond aux questions suivantes.

1. De quoi dépend le coût d'une recherche dans un ABR ? Citer ses mesures pour $n = 1\,000$ (hauteur, nombre moyen de visites) dans les deux cas.
2. Dans quelles situations réelles risque-t-on d'insérer des valeurs déjà triées ? Comment une base de données pourrait-elle s'en protéger ? *(Penser à la hauteur de l'arbre : quelle forme faudrait-il lui garantir, quel que soit l'ordre des insertions ?)*
3. Quelles propriétés de l'ABR l'index a-t-il utilisées ? Citer deux méthodes du TP et la propriété dont chacune dépend.

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 2 – piste Garder une variable `n` sur le nœud courant. À chaque tour : si `x == n.valeur`, renvoyer `False` ; si `x < n.valeur` et que `n.gauche` est `None`, on accroche le nouveau nœud à gauche ; sinon on descend `n = n.gauche`. Même chose à droite. Un booléen `place`, qui passe à `True` quand le nœud est accroché, sert de condition à la boucle : `while not place`. ↑énoncé

Ex. 4 – piste Depuis la racine, descendre toujours du même côté tant que c'est possible. ↑énoncé

Ex. 5 q. 1 – piste `parcours(n)` : si `n` n'est pas `None`, parcourir le sous-arbre gauche, ajouter `n.valeur` à `res`, puis parcourir le sous-arbre droit. La méthode appelle `parcours(self._racine)` puis renvoie `res`. ↑énoncé

Ex. 6 q. 1 – piste Pour `hauteur_aleatoire` : à chaque essai, construire `valeurs = [i for i in range(n)]`, la mélanger, l'insérer dans un nouvel `ABR`, puis ajouter sa hauteur à un total. ↑énoncé

Ex. 8 q. 1 – piste Écrire la boucle `while n.valeur != mot` : à chaque tour, si le fils où il faut descendre est `None`, on y crée d'abord un `NoeudIndex(mot)`, puis on descend. En sortie de boucle, `n` est le nœud du mot. ↑énoncé

TP 2 Le morpion imbattable ▶ corrigé p. 850

arbres de jeu et algorithme minimax

Durée : 3 h environ (deux séances) · Sur machine, seul ou par deux

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t05-tp-morpion-minimax` — **archive .zip**

But du TP

Programmer un joueur de morpion qui **ne perd jamais**. L'idée : depuis la position courante, la machine imagine **toutes** les suites possibles de la partie. Ces suites forment un **arbre**, l'**arbre de jeu**, que l'on parcourt récursivement en supposant que l'adversaire joue, lui aussi, parfaitement : c'est l'algorithme **minimax**. On réinvestit tout le chapitre : vocabulaire des arbres (racine, fils, feuilles, hauteur), parcours en profondeur, fonctions récursives. ▶ signale ce qui est à programmer et tester.

Consignes

- Les exercices marqués « défi » sont pour aller plus loin.
- Le fichier `tp_morpion_minimax_depart.py` est **à télécharger** (lien ci-dessus) : l'affichage d'une grille, la fonction `adversaire`, une fonction `partie` pour jouer contre la machine, les squelettes des fonctions à écrire et, en fin de fichier, des **tests**. Relancer le fichier après chaque **section** du TP : chaque ligne `[A FAIRE]` doit devenir `[OK]`. Certains tests explorent tout l'arbre du jeu : ils prennent quelques secondes.
- **Conventions** du cours : la **profondeur** d'un nœud compte les arêtes (racine à la profondeur 0) ; la **hauteur** d'un arbre compte les nœuds (racine seule → hauteur 1).
- Usage de l'IA, indiqué sur chaque exercice :  aucune ;  en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ;

l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

1 Représenter le jeu

Rappel des règles : deux joueurs, `X` et `O`, posent à tour de rôle leur symbole dans une case vide d'une grille 3×3 ; `X` commence toujours. Le premier qui aligne trois symboles (ligne, colonne ou diagonale) gagne ; si la grille est pleine sans alignement, la partie est nulle.

La grille en Python

0	1	2
3	4	5
6	7	8

Les cases sont numérotées de 0 à 8, ligne par ligne (figure de gauche). Une grille est une **liste de 9 caractères** : `"X"`, `"O"`, ou `"."` pour une case vide (constante `VIDE`). La fonction fournie `depuis_texte` fabrique une grille à partir d'une chaîne, ce qui évite d'écrire de longues listes :

```
>>> g = depuis_texte("XOXOOX...")
>>> g
['X', 'O', 'X', 'O', 'O', 'X', '.', '.', '.']
```

X	O	X
O	O	X

Les huit alignements gagnants sont donnés par la liste `LIGNES` de triplets de numéros de cases : `(0, 1, 2)`, `(3, 4, 5)`, `(6, 7, 8)` pour les lignes, puis les trois colonnes et les deux diagonales.

★ **Exercice 1** *Lire une grille*

On considère les grilles `"XO.OX...X"`, `"XOX.OX.O."` et `"XOXOOX..."`.

1. Dessiner chacune des trois grilles.
2. Pour chacune, dire si un joueur a gagné et, si oui, lequel et grâce à quel triplet de `LIGNES`.
3. Justifier que, dans une partie qui n'est pas finie, c'est à `X` de jouer si et seulement si la grille contient autant de `X` que de `O`. À qui est-ce le tour dans la troisième grille ?
4. Écrire les huit triplets de `LIGNES`.

★ **Exercice 2** *Les règles en Python*

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1. ▶ Écrire `coups_possibles(grille)`, qui renvoie la liste croissante des numéros des cases vides. Une liste par compréhension suffit.

```
>>> coups_possibles(depuis_texte("XOXOOX..."))
[6, 7, 8]
```

2. ▶ Écrire `gagnant(grille)`, qui renvoie `"X"` ou `"O"` si ce joueur a aligné trois symboles, et `None` sinon. ▷ coup de pouce
3. ▶ Écrire `est_finie(grille)`, qui renvoie `True` si quelqu'un a gagné ou si la grille est pleine.

★★ **Exercice 3** *Jouer sans abîmer la grille*

1. On propose la version suivante de `jouer`. Qu'affiche la console à la fin de ces instructions ? Pourquoi la grille `g` a-t-elle changé ?

```
1 def jouer_faux(grille, case, joueur):
2     nouvelle = grille
3     nouvelle[case] = joueur
4     return nouvelle
```

```
>>> g = depuis_texte("XOXOOX...")
>>> h = jouer_faux(g, 8, "X")
>>> g
```

2. ▶ Écrire `jouer(grille, case, joueur)`, qui renvoie une **nouvelle** grille dans laquelle `joueur` a joué dans `case`, **sans modifier** `grille`. La compréhension `[c for c in grille]` fabrique une copie de la liste.
3. Dans la suite, depuis une même position, on va essayer **tous** les coups possibles l'un après l'autre. Expliquer pourquoi `jouer_faux` rendrait ces essais faux.

2 L'arbre de jeu

L'arbre de jeu

À partir d'une position, on construit un arbre :

- la **racine** est la position de départ ;
- les **fils** d'un nœud sont les positions obtenues en jouant chacun des coups possibles (le joueur change à chaque niveau) ;
- les **feuilles** sont les parties **finies** (victoire ou grille pleine).

Un chemin de la racine à une feuille raconte une partie. La profondeur d'un nœud est le nombre de coups joués depuis la racine. Ce n'est **pas** un arbre binaire : un nœud peut avoir jusqu'à 9 fils.

★★ **Exercice 4** *Un petit arbre de jeu, à la main*

On part de la grille `"XOXOOX..."` ; c'est à `X` de jouer.

1. Dessiner sur le cahier l'arbre de jeu complet : chaque nœud est une petite grille, chaque arête est étiquetée par le numéro de la case jouée.
2. Donner la taille de cet arbre, sa hauteur et son nombre de feuilles. Combien de parties différentes peuvent se jouer à partir de cette position ?
3. Pour chaque feuille, indiquer le résultat de la partie : victoire de `X`, victoire de `O`, ou nul.
4. Une feuille est-elle forcément à la profondeur maximale ? Justifier sur l'exemple.

★ **Exercice 5** *L'arbre complet du morpion*

On part maintenant de la grille **vide**.

1. Combien la racine a-t-elle de fils ? Et chacun de ces fils ? Quelle est la hauteur de l'arbre de jeu complet ?
2. Si aucune partie ne s'arrêtait avant que la grille soit pleine, combien y aurait-il de parties ? *(Exprimer le résultat avec une factorielle, puis le calculer.)*
3. Toujours sous cette hypothèse, combien l'arbre aurait-il de nœuds ? *(Faire la somme niveau par niveau :* $1 + 9 + 9 \times 8 + \dots$*)*
4. Les vrais nombres sont-ils plus grands ou plus petits ? Pourquoi ? On les calculera par programme à l'exercice 8.

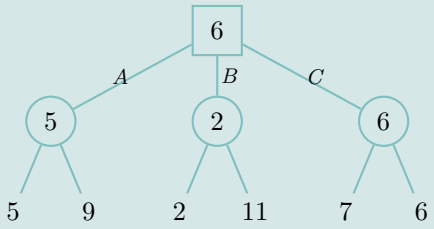
3 L'algorithme minimax

Le principe de minimax

On donne un **score** à chaque feuille, toujours du point de vue de `X` : $+1$ si `X` a gagné, -1 si `O` a gagné, 0 pour un nul. `X` cherche donc à obtenir le score le plus **grand** (on l'appelle MAX), `O` le plus **petit** (MIN). La **valeur** d'un nœud se calcule en remontant depuis les feuilles :

- la valeur d'une **feuille** est son score ;
- si c'est à MAX de jouer, la valeur du nœud est le **maximum** des valeurs de ses fils ;
- si c'est à MIN de jouer, c'est le **minimum** des valeurs de ses fils.

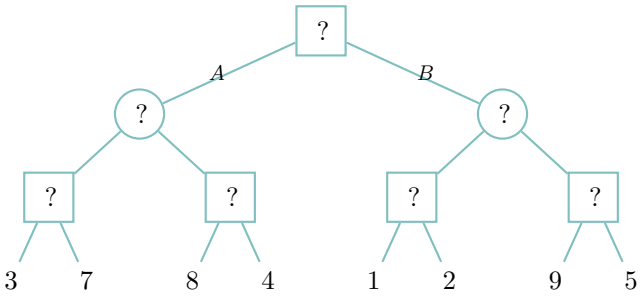
On écrit aussi « min-max », comme dans l'exercice 21 du chapitre (Puissance 4). La valeur de la racine est ce que MAX est **sûr** d'obtenir, quoi que fasse l'adversaire ; le bon coup est celui qui mène au fils de meilleure valeur. On suppose ainsi que l'adversaire joue parfaitement : c'est le pire cas, donc le plus prudent.



Exemple : MAX (carré) a trois coups `A`, `B`, `C` ; MIN (rond) répond. Les nœuds MIN valent $\min(5,9) = 5$, $\min(2,11) = 2$, $\min(7,6) = 6$; la racine vaut $\max(5,2,6) = 6$: MAX joue `C`. La meilleure feuille (11) est sous `B`, mais MIN ne laissera jamais MAX l'atteindre : jouer `B` rapporterait 2.

★ Exercice 6 *Remonter les valeurs*

1. Recopier l'arbre ci-dessous et calculer la valeur de chaque nœud. MAX joue à la racine, puis MIN, puis MAX.



2. Quel coup MAX doit-il jouer ? Quel score est-il sûr d'obtenir ?

3. La plus grande feuille sous `A` vaut 8 et la plus grande sous `B` vaut 9. Un joueur « optimiste » qui choisirait la branche contenant la meilleure feuille aurait-il raison ici ? Et dans l'exemple de l'encadré ?

4. Reprendre l'arbre de jeu de l'exercice 4 : calculer la valeur de chaque nœud et en déduire le coup que doit jouer `X`.

★★ Exercice 7 *Programmer minimax*

1. ▶ Écrire `score_final(grille)`, qui renvoie le score d'une partie finie : 1, -1 ou 0.

2. ▶ Écrire la fonction **récursive** `minimax(grille, joueur)`, qui renvoie la valeur de la position `grille` quand c'est à `joueur` de jouer. Quelle est la condition d'arrêt (le cas de base) ?

▷ coup de pouce

```
>>> minimax(depuis_texte("XOXOOX..."), "X")
1
>>> minimax(depuis_texte("X..OO.X.."), "X")
0
```

3. Comparer la première valeur avec celle trouvée à la main à l'exercice 6. Dessiner la seconde grille : pourquoi `X` ne peut-il pas espérer mieux qu'un nul ?
4. ▶ Que renvoie `minimax(grille_vide(), "X")` ? Que peut-on en conclure sur le jeu de morpion ? Mesurer le temps de calcul avec `time.perf_counter()`.
5. Dans quel ordre les nœuds de l'arbre de jeu sont-ils visités par `minimax` ? Quel parcours du cours reconnaît-on, et dans quel ordre (préfixe, infixe ou suffixe) la valeur d'un nœud est-elle calculée par rapport à celles de ses fils ?

★★ Exercice 8 *Compter les positions*

1. ▶ Écrire `minimax_compte(grille, joueur)`, qui fait le même calcul que `minimax` mais renvoie le couple `(score, visites)`, où `visites` est le nombre de positions examinées (la position courante comprise). Sur la grille de l'exercice 4, on doit retrouver la taille de l'arbre.

▷ coup de pouce

2. ▶ Écrire `nb_parties(grille, joueur)`, qui renvoie le nombre de feuilles de l'arbre de jeu, c'est-à-dire le nombre de parties possibles.
3. ▶ Calculer ces deux nombres pour la grille vide et les comparer aux estimations de l'exercice 5.
4. Le morpion ne compte en réalité que 5 478 grilles différentes que l'on peut atteindre en jouant. Pourquoi `minimax` examine-t-il beaucoup plus de positions ? Donner deux suites de coups différentes qui mènent à la même grille. *(On verra au chapitre Programmation dynamique comment éviter ces calculs répétés.)*

★★ Exercice 9 *Choisir son coup, puis jouer*

1. ▶ Écrire `meilleur_coup(grille, joueur)`, qui renvoie la case à jouer : celle dont le fils a la meilleure valeur minimax pour `joueur` (la plus grande pour `X`, la plus petite pour `O`). En cas d'égalité, on garde la **première** case trouvée.

▷ coup de pouce

```
>>> meilleur_coup(depuis_texte("XOXOOX..."), "X")     # gagner
8
>>> meilleur_coup(depuis_texte("X..OO.X.."), "X")     # bloquer
5
>>> meilleur_coup(depuis_texte("XX..O...."), "O")     # bloquer, cote O
2
```

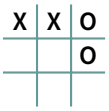
2. ▶ Lancer `partie()` (vous jouez `X`) puis `partie("O")` (la machine commence). Essayer de battre la machine. Y arrive-t-on ?
3. Le premier coup de la machine, quand elle commence, est nettement plus long que les suivants. Expliquer.

4 Gagner vite, perdre tard

★★ **Exercice 10** *Une machine qui ne se presse pas*



grille `"XOOX....."`



grille `"XXO..O..."`

1. Sur la grille de gauche, c'est à `X` de jouer. Quel coup gagne immédiatement ? ▶ Que renvoie `meilleur_coup` sur cette grille ? Calculer la valeur minimax de chaque coup possible et expliquer ce choix. La machine se trompe-t-elle ?
2. Sur la grille de droite, c'est encore à `X` de jouer. Montrer que `X` a perdu si `O` joue bien. ▶ Quel coup la machine choisit-elle ? Pourquoi est-ce un mauvais choix face à un humain, qui peut se tromper ?
3. On modifie le score des feuilles pour récompenser une victoire **rapide** : `X` gagne $\to 10+v$, `O` gagne $\to -(10+v)$, nul $\to 0$, où v est le nombre de cases encore vides. ▶ Modifier `score_final` en conséquence, relancer les tests, puis rejouer les deux grilles. Qu'est-ce qui a changé, et pourquoi ?
4. Avec ce nouveau score, `minimax(grille_vide(), "X")` donne-t-il toujours le même résultat ? Le signe de la valeur garde-t-il le même sens ?

5 Pour aller plus loin

★★★ **Exercice 11** *Défi — l'élagage alpha-bêta*

Reprenons l'exemple de l'encadré de la partie 3, en parcourant les fils de gauche à droite. Après `A`, MAX est sûr d'obtenir au moins 5. Dans `B`, la première feuille vaut 2 : MIN pourra donc ramener `B` à 2 **ou moins**, et MAX ne choisira jamais `B`. Inutile de regarder la feuille 11 : on **élague** cette branche. Le résultat est le même, avec moins de calculs.

Pour programmer cette idée, on transmet deux bornes dans la récursion : `alpha`, ce que MAX est déjà sûr d'obtenir ailleurs, et `beta`, ce que MIN est déjà sûr d'obtenir ailleurs. Dès que `alpha >= beta`, le nœud courant ne sera jamais atteint en jouant bien : on arrête d'examiner ses fils.

1. Sur l'arbre de l'exercice 6, parcouru de gauche à droite, quelles feuilles l'élagage permet-il de ne pas examiner ?
2. ▶ Écrire `alphabeta(grille, joueur, alpha, beta)`, qui renvoie le couple `(score, visites)`. L'appel initial est `alphabeta(grille, joueur, -100, 100)`. Sans `break` : la boucle sur les coups est un `while` dont la condition contient `alpha < beta`. ▷ coup de pouce
3. ▶ Comparer le nombre de positions examinées par `alphabeta` et par `minimax_compte` sur la grille vide. Le score est-il le même ?
4. ▶ L'efficacité de l'élagage dépend de l'ordre dans lequel on essaie les coups. Essayer d'abord le centre, puis les coins, puis les bords (ordre `[4, 0, 2, 6, 8, 1, 3, 5, 7]`) : combien de positions examine-t-on alors ? Pourquoi est-ce mieux ?

★★★ **Exercice 12** *Défi — quand l'arbre est trop grand*

Aux échecs, une position offre en moyenne une trentaine de coups et une partie en compte souvent plus de 80 : l'arbre de jeu a bien plus de nœuds qu'il n'y a d'atomes dans l'univers. On ne peut pas descendre jusqu'aux feuilles. On s'arrête donc à une **profondeur limite** et l'on **estime** la position atteinte avec une **fonction d'évaluation**. C'est la méthode de l'exercice 21 du chapitre (Puissance 4). Ce défi suppose faite la partie 4 (scores $\pm(10+v)$).

1. ▶ Écrire `evaluer(grille)`, qui renvoie le nombre d'alignements de `LIGNES` encore possibles pour `X` (sans aucun `O`) moins le nombre d'alignements encore possibles pour `O` (sans aucun `X`). Que vaut-elle après un premier `X` au centre ? dans un coin ? sur un bord ?
2. Pourquoi une victoire doit-elle toujours valoir plus que n'importe quelle évaluation ? Vérifier que c'est le cas avec les scores de la partie 4.
3. ▶ Écrire `minimax_limite(grille, joueur, profondeur)` : comme `minimax`, mais lorsque `profondeur` vaut 0 (et que la partie n'est pas finie), on renvoie `evaluer(grille)` ; chaque appel récursif se fait avec `profondeur - 1`. Écrire ensuite `meilleur_coup_limite(grille, joueur, profondeur)`.
4. ▶ Dans `partie`, remplacer l'appel à `meilleur_coup` par `meilleur_coup_limite(grille, joueur, 1)`, jouer `X` et battre la machine. Recommencer avec une profondeur 2. À partir de quelle profondeur la machine devient-elle, d'après vous, imbattable ?

6 Bilan à rédiger

Exercice 13  *Ce que fait vraiment la machine*

Rédiger, en une dizaine de lignes, un bilan qui répond aux questions suivantes.

1. Qu'est-ce qu'un arbre de jeu ? Que représentent ses nœuds, ses arêtes et ses feuilles ? Quelle est sa hauteur pour le morpion ?
2. Décrire en trois phrases l'algorithme minimax : condition d'arrêt, cas MAX, cas MIN. Pourquoi est-il naturellement récursif ?
3. La machine du TP « réfléchit »-elle ? Que fait-elle exactement, et pourquoi cette méthode ne suffit-elle pas aux échecs ? Citer les deux améliorations vues dans les défis.

Un peu d'histoire

En 1928, le mathématicien **John von Neumann** démontre le *théorème du minimax* : dans un jeu à deux joueurs où l'un gagne exactement ce que l'autre perd, chacun a une stratégie qui lui garantit le meilleur résultat possible face à un adversaire parfait. En 1950, **Claude Shannon**, le père de la théorie de l'information, publie un article sur la programmation d'un ordinateur pour jouer aux échecs : il y propose exactement la méthode du défi 2, une recherche minimax à profondeur limitée avec une fonction d'évaluation (matériel, mobilité des pièces…). L'élagage alpha-bêta est imaginé à la fin des années 1950, notamment par John McCarthy, Allen Newell et Herbert Simon. Le 11 mai 1997, l'ordinateur **Deep Blue** d'IBM, qui examinait environ 200 millions de positions par seconde avec ces techniques, bat le champion du monde Garry Kasparov dans un match en six parties. Kasparov réclama une revanche ; IBM démonta la machine.

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 2 q. 2 – piste Parcourir les triplets avec `for a, b, c in LIGNES:` ; il y a un gagnant si la case `a` n'est pas vide et si les cases `a`, `b` et `c` contiennent le même symbole. Penser au cas de la grille vide : trois cases `"."` identiques ne font pas un gagnant ! ↑ énoncé

Ex. 7 q. 2 – piste Condition d'arrêt : si la partie est finie, renvoyer `score_final(grille)`. Sinon, chercher le maximum (pour `"X"`) ou le minimum (pour `"O"`) des valeurs des fils, comme dans une recherche de maximum : partir de `valeur = -100` pour `"X"` (`100` pour `"O"`), puis, pour chaque case `c` de `coups_possibles(grille)`, calculer `s = minimax(jouer(grille, c, joueur), adversaire(joueur))` et remplacer `valeur` par `max(valeur, s)` (ou `min(valeur, s)`). ↑ énoncé

Ex. 8 q. 1 – piste Une feuille renvoie `(score_final(grille), 1)`. Sinon, partir de `visites = 1` et, pour chaque coup, récupérer le couple `s, v` renvoyé par l'appel récursif : mettre à jour `valeur` avec `s` comme dans `minimax`, et ajouter `v` à `visites`. ↑énoncé

Ex. 9 q. 1 – piste Initialiser `meilleur` avec le premier coup possible et `meilleur_score` avec sa valeur, puis parcourir tous les coups : la valeur du coup `c` est `minimax(jouer(grille, c, joueur), adversaire(joueur))`. Attention : après le coup de `joueur`, c'est à l'adversaire de jouer. ↑énoncé

Ex. 11 q. 2 – piste Cas où `joueur` vaut `"X"` : `valeur = -100` ; tant qu'il reste des coups et que `alpha < beta`, calculer le couple `(s, v)` du fils, ajouter `v` aux visites, puis `valeur = max(valeur, s)` et `alpha = max(alpha, valeur)`. Cas de `"O"` : symétrique, avec `100`, `min` et `beta`. ↑énoncé

FICHE DE RÉVISION

L'essentiel

- **Vocabulaire : racine** (en haut), **nœud**, **feuille** (sans fils), **sous-arbre** gauche / droit. Un arbre est **défini récursivement** : vide, ou un nœud reliant deux sous-arbres.
- **Taille** = nombre de nœuds. **Profondeur** d'un nœud = nombre d'arêtes depuis la racine (racine = 0).
- **Hauteur** = nombre de nœuds du plus long chemin racine → feuille : arbre vide 0, racine seule 1 (convention du cours). Autre convention, en arêtes : racine seule 0, arbre vide -1 (`return -1` dans `hauteur`). **Suivez l'énoncé.**
- **Parcours en profondeur** (récursifs) : **préfixe** (racine, g, d), **infixe** (g, racine, d), **suffixe** (g, d, racine).
- **Parcours en largeur** : niveau par niveau, à l'aide d'une **file**.
- **ABR** (arbre binaire de recherche) : `gauche` $<$ `nœud` $\leq$ `droite` (un **doublon** va **à droite** : convention du cours ; lire celle de l'énoncé). L'**infixe** d'un ABR donne les valeurs **triées**.
- **Insérer** dans un ABR : descendre comme pour chercher, jusqu'à un sous-arbre vide ; la valeur y devient une **feuille**.

Les fonctions à savoir écrire (récursives)

```python
class Noeud:
    def __init__(self, valeur, gauche=None, droite=None):
        self.valeur = valeur
        self.gauche = gauche
        self.droite = droite

def taille(a):
    if a is None:
        return 0
    return 1 + taille(a.gauche) + taille(a.droite)

def hauteur(a):
    if a is None:
        return 0
    return 1 + max(hauteur(a.gauche), hauteur(a.droite))

def recherche(a, x):        # dans un ABR : on ne descend que d'un cote
    if a is None:
        return False
    if x == a.valeur:
        return True
    if x < a.valeur:
        return recherche(a.gauche, x)
    return recherche(a.droite, x)

def inserer(a, x):          # dans un ABR ; renvoie l'arbre obtenu
    if a is None:
        return Noeud(x)
    if x < a.valeur:
        a.gauche = inserer(a.gauche, x)
    else:                   # plus grand ou egal : a droite
        a.droite = inserer(a.droite, x)
    return a
```

Coût

Recherche ou insertion dans un ABR = la **hauteur** : $\approx \log_2 n$ si l'arbre est **équilibré**, mais jusqu'à n s'il est « en peigne ». C'est l'idée de la **dichotomie** portée sur une structure.

Erreurs à éviter

- oublier le cas `a is None` (cas de base) ;
- confondre taille / hauteur / profondeur, ou changer de convention de hauteur ;
- chercher dans *tout* l'ABR au lieu d'un seul côté ;
- écrire `inserer(a.gauche, x)` sans réaffecter `a.gauche = ...` (le nœud créé est perdu) ;
- oublier de **combiner** les résultats des sous-arbres.

Savoir-faire à maîtriser — je sais…

- ☐ employer le **vocabulaire** (racine, nœud, feuille, sous-arbre, taille, hauteur) ;
- ☐ calculer **taille**, **hauteur**, nombre de feuilles par une fonction **récursive** ;
- ☐ **encadrer** la hauteur d'un arbre binaire de n nœuds, de l'arbre filiforme à l'arbre parfait ;
- ☐ écrire et reconnaître les **quatre parcours** (préfixe, infixe, suffixe, largeur / file) ;
- ☐ manipuler un arbre via la classe `Noeud` ou une **interface** donnée ;
- ☐ **rechercher** et **insérer** dans un **ABR** (à la main et en Python, doublon à droite), et relier infixe ↔ tri ;
- ☐ justifier le **coût** d'une recherche ou d'une insertion dans un ABR (la hauteur).

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 859

1. Qu'est-ce qu'une feuille ? Que vaut la taille d'un arbre ?
2. Donner le parcours infixe de `Noeud(5, Noeud(2), Noeud(8, Noeud(6)))`.
3. Combien de comparaisons au plus pour chercher dans un ABR « en peigne » de n nœuds ? Et s'il est équilibré ?
4. `taille` écrite sans le test `a is None` : que se passe-t-il ?
5. Quelle structure pour parcourir un arbre niveau par niveau ?

Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 859

1. Citer les trois états d'un processus. Quelle transition entre eux est impossible ? (ch. 4)
2. Qu'est-ce qu'une fonction pure ? (ch. 2)
3. Dans une fonction récursive, `print(n)` est-il exécuté dans le même ordre s'il est placé avant ou après l'appel ? (ch. 1)

Vers le bac

- Problème de synthèse n° 4 — Trier avec un arbre binaire de recherche, p. 708
- Problème de synthèse n° 5 — La largeur d'un arbre, p. 709
- Épreuve pratique : sujet 24 — Arbre d'une expression arithmétique, p. 727
- Épreuve pratique : sujet 25 — Arbre binaire de recherche, p. 729

DÉFIS ADVENT OF CODE

DÉFI 1 Binary Boarding

les cartes d'embarquement — binaire et dichotomie

Jour 5

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t05-defi-aoc-2020-05` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/5`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Chaque carte d'embarquement porte un code de dix lettres qui désigne un siège de l'avion. Il faut décoder plusieurs centaines de ces codes pour trouver le plus grand numéro de siège, puis le seul siège libre : celui du héros. Décoder un code à la main prend une minute ; en décoder des centaines demande un programme.
Ce qu'on réinvestit. Les **arbres binaires** vus dans ce chapitre. Un code de place décrit un chemin depuis la racine d'un arbre binaire complet : chaque lettre choisit le fils gauche ou le fils droit, c'est-à-dire la moitié basse ou la moitié haute des places restantes, et la feuille atteinte est le siège. On découvrira que ce chemin n'est autre que l'écriture binaire du numéro, et qu'il s'agit d'une dichotomie.
La démarche de la fiche.

1. Comprendre le découpage en moitiés, puis décoder un code à la main.
2. Reconnaître que le code est un nombre écrit en binaire.
3. Écrire la fonction de décodage et chercher le plus grand numéro : c'est la partie 1 ; la

réécrire par dichotomie.
4. Partie 2 : trier les numéros et trouver le trou.
5. En approfondissement, trouver le trou sans trier.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`boarding pass`	carte d'embarquement	`seat`	siège, place
`row / column`	rang / colonne	`seat ID`	numéro de siège
`front / back`	avant / arrière	`lower / upper half`	moitié basse / haute
`binary space partitioning`	partition binaire de l'espace	`sanity check`	vérification de bon sens

Résumé de l'énoncé (partie 1)

Contexte. Le héros est enfin dans l'avion… mais il a perdu sa carte d'embarquement. Il photographie celles des autres passagers pour retrouver sa place par élimination. La première étape est de savoir lire ces cartes.
Les données. Chaque ligne du fichier est le code de place imprimé sur une carte : 10 lettres. Les 7 premières (`F` ou `B`) donnent le rang, numéroté de 0 à 127 ; les 3 dernières (`L` ou `R`) donnent la colonne, de 0 à 7.
La règle.

- Pour le rang, on part de l'intervalle complet des rangs, de 0 à 127.
- Chaque lettre coupe l'intervalle restant en deux moitiés égales : `F` garde la moitié basse, `B` la moitié haute. Après la septième lettre, il ne reste qu'un rang.
- On procède de même pour la colonne, de 0 à 7, avec `L` (moitié basse) et `R` (moitié haute) : après trois lettres, il ne reste qu'une colonne.
- Chaque siège a un numéro : rang $\times 8$ + colonne.

La réponse attendue. Le plus grand numéro de siège présent dans le fichier. Par exemple, le code `FFFFFFBRLL` désigne le rang 1 et la colonne 4, donc le siège numéro $1 \times 8 + 4 = 12$.

Exercice 1 *Comprendre ce qui est demandé*

But. Vérifier qu'on sait interpréter un code dans les cas extrêmes.
À quoi ça sert ? Les cas extrêmes donnent les bornes des rangs et des numéros ; la dernière question prépare l'idée qu'un code et un numéro se correspondent un à un, ce qui justifiera la lecture en binaire.

Après lecture de l'énoncé et du résumé, vérifier sa compréhension :

1. Quel rang donne `FFFFFFF` ? Et `BBBBBBB` ?
2. Quels sont le plus petit et le plus grand numéro de siège possibles ?
3. Deux codes différents peuvent-ils donner le même numéro de siège ?

Exercice 2 *À la main, sur un petit exemple*

But. Décoder deux codes à la main en suivant la méthode de l'énoncé, puis comparer avec l'écriture binaire.
À quoi ça sert ? On obtient des valeurs connues (`75`, `2`, `602`) pour tester le programme, et l'on soupçonne déjà un raccourci : le rang et la colonne se lisent peut-être directement en binaire.

Décoder à la main le code `BFFBFBBLRL` en suivant la méthode de l'énoncé (diviser l'intervalle par deux à chaque lettre). On doit trouver le rang `75`, la colonne `2` et le numéro `602`.

Faire de même avec `FFBBFBFRLR`, puis comparer : écrire `75` et `2` en binaire. Que remarque-t-on ?

2 Programmer la partie 1

Exercice 3 *Reconnaître le binaire*

But. Montrer que le code de place est l'écriture binaire du numéro de siège.
À quoi ça sert ? Ce constat remplace sept puis trois découpages par une simple conversion de base : la fonction de décodage tiendra en quelques lignes.

1. Remplacer dans `BFFBFBBLRL` chaque `F` et chaque `L` par `0`, chaque `B` et chaque `R` par `1`. Que vaut le nombre binaire obtenu ? Le comparer au numéro de siège.

2. Expliquer pourquoi « multiplier le rang par 8 puis ajouter la colonne » revient à recoller les deux écritures binaires.

▷ coup de pouce

Outil Python : conversions binaires (`int(..., 2)`, `bin`) et traduction lettre par lettre

Python sait lire et écrire des nombres en base 2. Une chaîne n'étant pas modifiable, pour « traduire » ses lettres on construit une **nouvelle** chaîne en la parcourant, avec un dictionnaire qui donne la traduction de chaque lettre.

```
print(int("1011", 2))          # 11 : lit une chaine ecrite en base 2
print(bin(11))                 # 0b1011 : ecriture binaire (une chaine)
traduction = {"F": "0", "B": "1"}
mot = "FBFB"
binaire = ""
for lettre in mot:
    binaire = binaire + traduction[lettre]
print(binaire)                 # 0101 : nouvelle chaine
print(mot)                     # FBFB : mot n'a pas change
```

Intérêt. La conversion et la traduction prennent un temps proportionnel au nombre de caractères : un seul parcours suffit.
À essayer. Que vaut `int("1111111111", 2)` ? Quel lien avec le plus grand numéro de siège possible ?

Exercice 4 *Décoder et chercher le maximum*

But. Écrire la fonction qui décode un code, puis parcourir le fichier pour trouver le plus grand numéro.
À quoi ça sert ? C'est la réponse de la partie 1. La liste des numéros obtenus resservira telle quelle pour chercher la place libre en partie 2.

Écrire une fonction `numero_siege(code)` qui renvoie le numéro de siège, puis le programme qui donne la réponse sur `input.txt`. Vérifier `numero_siege("BFFBFBBLRL") == 602`. Sur la liste (inventée) `BFFBFBBLRL`, `FBBFFFBRRR`, `BBFBFFFLLR`, `FFBBFBFRLR`, `BFBBBFFLRR`, on doit trouver `833`.

▷ coup de pouce

Exercice 5 *Le lien avec la dichotomie*

But. Réécrire le décodage en suivant littéralement l'énoncé : deux bornes et un intervalle divisé par deux à chaque lettre.
À quoi ça sert ? Comparer les deux versions montre que chaque lettre est un choix gauche ou droite dans un arbre binaire, comme dans une recherche dichotomique. Le nombre de lettres nécessaire est le nombre de divisions par deux, d'où le lien avec le logarithme en base 2.

Écrire une seconde version `numero_siege_dicho(code)` qui suit littéralement l'énoncé : deux bornes `bas` et `haut`, et chaque lettre garde une moitié. Vérifier qu'elle donne les mêmes résultats que la première. Combien de lettres faudrait-il pour un avion de 1024 rangs ? ▷ coup de pouce

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Il reste à trouver la place du héros, puisque c'est la seule dont la carte manque. On utilise les mêmes numéros de siège que dans la partie 1.
La nouvelle règle.

- L'avion est plein, sauf quelques sièges tout à l'avant et tout à l'arrière qui n'existent pas : ces numéros-là manquent aussi dans la liste, mais ce ne sont pas des places.
- Le siège du héros est absent de la liste, mais les numéros juste avant et juste après le sien y figurent.

La réponse attendue. Le numéro du siège du héros. Par exemple, si les numéros présents étaient 40, 41, 43 et 44, sa place serait la 42.

Exercice 6 *La place libre*

But. Trouver le seul numéro manquant entre deux numéros présents.
À quoi ça sert ? C'est la réponse de la partie 2. Une fois les numéros triés, le trou se voit en comparant chaque numéro au suivant.

Trouver la place du héros en suivant les indications de la partie 2. ▷ coup de pouce ▷ début de solution

4 Approfondissement

Outil Python : les ensembles (`set`) — rappel

Tout est présenté dans l'encadré « Outil Python : les ensembles » du défi *Crab Combat* (chapitre 4). On n'utilise ici que `set()` (ensemble vide), `add`, `remove` et le test `x in ensemble`, en temps constant (en moyenne). Les éléments doivent être non modifiables (nombres, chaînes, tuples).
À essayer. Construire l'ensemble des numéros de la liste inventée de la partie 1 (les cinq codes de l'exercice « Décoder et chercher le maximum ») en partant de `set()` et en ajoutant chaque numéro avec `add`, puis tester si `602` et `603` en font partie.

★★ **Exercice 7** *La place libre avec un ensemble*

But. Trouver la place libre sans trier, en testant l'appartenance de chaque candidat à un ensemble.
À quoi ça sert ? C'est une autre méthode sans tri, qui ne demande aucune astuce de calcul : on teste chaque candidat, et chaque test coûte un temps constant. Comparer les complexités des méthodes, c'est choisir en connaissance de cause.

1. Répondre à la question « À essayer » de l'encadré (le code complet est dans le défi *Crab Combat*, chapitre 4).
2. À la main : les numéros présents sont $\{12, 13, 14, 16, 17\}$. Parcourir les entiers de 12 à 17 et dire, pour chacun, s'il est dans l'ensemble. Quelle place est libre ?
3. Écrire `place_libre_ensemble(numeros)` qui trouve la place libre sans trier, puis vérifier qu'elle donne le même résultat que l'exercice de la partie 2.
4. Comparer les complexités des deux méthodes (tri, ensemble).

▷ coup de pouce ▷ début de solution

★★★ **Exercice 8** *Défi : sans tri*

But. Trouver la place libre en un seul parcours, sans trier, à l'aide d'un calcul de somme.
À quoi ça sert ? Le tri coûte plus qu'un parcours : un raisonnement mathématique permet ici de descendre à un coût linéaire, avec très peu de mémoire.

Trouver la place libre en temps linéaire sans trier ni utiliser d'ensemble, en comparant la somme des numéros présents à une somme connue. ▷ coup de pouce

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Multiplier par $8 = 2^3$ décale l'écriture binaire de trois rangs vers la gauche, ce qui laisse exactement la place des 3 bits de la colonne. ↑ énoncé

Ex. 4 – piste `int("1011", 2)` convertit une chaîne écrite en binaire en entier. Pour traduire les lettres, parcourir le code avec un dictionnaire `{"F": "0", "B": "1", "L": "0", "R": "1"}` (voir l'encadré). Pour le maximum, une variable qui garde le plus grand numéro vu jusqu'ici. ↑ énoncé

Ex. 5 – piste Le milieu est `(bas + haut) // 2` ; une lettre « basse » modifie `haut`, une lettre « haute » modifie `bas`. ↑ énoncé

Ex. 6 – piste Trier la liste des numéros avec un tri écrit par vous (le tri par insertion de Première), puis la parcourir en comparant chaque numéro au suivant. ↑ énoncé

Ex. 6 – début de solution Dans la liste triée, le trou se repère au premier indice `k` tel que `tries[k + 1] != tries[k] + 1` : la place cherchée est alors `tries[k] + 1`. ↑ énoncé

Ex. 7 – piste Les seules places candidates sont les entiers compris entre le plus petit numéro présent `mini` et le plus grand `maxi`, à calculer par une boucle : `range(mini, maxi)`. ↑ énoncé

Ex. 7 – début de solution On parcourt ces entiers et l'on renvoie le premier qui n'est pas dans l'ensemble `presents` (rempli avec `add`) : chaque test coûte un temps constant. ↑ énoncé

Ex. 8 – piste La somme des entiers de a à b vaut $\frac{(a+b)(b-a+1)}{2}$. ↑ énoncé

DÉFI 2 Custom Customs

la douane — ensembles, union et intersection

Jour 6

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t05-defi-aoc-2020-06` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/6`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Des passagers, répartis en groupes de voyage, ont rempli un formulaire de 26 questions fermées. Pour chaque groupe, il faut compter les questions auxquelles quelqu'un a répondu oui (partie 1), puis celles auxquelles tout le monde a répondu oui (partie 2), et faire la somme sur plusieurs centaines de groupes. Compter à la main sans doublon ni oubli est fastidieux ; avec la bonne structure de données, le programme est très court.

Ce qu'on réinvestit. Les **ensembles** et leurs opérations : « au moins une personne » correspond à la réunion, « toutes les personnes » à l'intersection. Un ensemble peut se représenter par un arbre binaire de recherche, vu dans ce chapitre : tester l'appartenance d'une valeur coûte alors la hauteur de l'arbre. Le type `set` de Python, lui, utilise une table de hachage, comme les dictionnaires.

La démarche de la fiche.

1. Comprendre ce qu'on compte dans un groupe, puis le calculer à la main sur un petit fichier.
2. Découper le fichier en groupes, puis en personnes.
3. Réunir les réponses de chaque groupe et sommer : c'est la partie 1.
4. Partie 2 : remplacer la réunion par l'intersection.
5. En approfondissement, résoudre les deux parties avec un dictionnaire de compteurs.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`customs`	douane	`declaration form`	formulaire de déclaration
`yes-or-no question`	question fermée (oui/-non)	`group`	groupe
`anyone`	au moins une personne	`everyone`	tout le monde
`duplicate`	doublon	`sum of those counts`	somme de ces nombres

Résumé de l'énoncé (partie 1)

Contexte. Pendant le vol, on distribue des formulaires de douane comportant 26 questions fermées, repérées par les lettres de `a` à `z`. Le héros aide les autres passagers, regroupés par groupes de voyage, à les remplir. On veut résumer leurs réponses groupe par groupe.
Les données. Le fichier recense les réponses.

- Les groupes sont séparés par une **ligne vide** ; à l'intérieur d'un groupe, il y a une ligne par personne.
- Une ligne contient, sans doublon et dans un ordre quelconque, les lettres des questions où la personne a répondu oui. Une question absente de la ligne correspond donc à un non.

La règle.

- Pour chaque groupe, on compte les lettres différentes cochées par **au moins une** personne du groupe.
- Une même lettre cochée par plusieurs personnes du groupe ne compte qu'une fois.
- On fait ce calcul pour chaque groupe séparément, puis on additionne.

La réponse attendue. La somme de ces nombres sur tous les groupes. Par exemple, pour un groupe formé des lignes `ab` et `bc`, les lettres cochées sont `a`, `b`, `c` : ce groupe compte pour 3.

Exercice 1 *Comprendre ce qui est demandé*

But. Vérifier qu'on a compris ce qu'on compte dans un groupe et comment le fichier délimite les groupes.
À quoi ça sert ? La première question montre que les doublons ne comptent pas, la deuxième que l'ordre est sans importance (deux propriétés exactes d'un ensemble) ; la troisième prépare le découpage du fichier en groupes.

Après lecture de l'énoncé et du résumé, vérifier sa compréhension :

1. Un groupe formé des deux lignes `ab` et `ba` compte pour combien ?
2. L'ordre des lettres dans une ligne a-t-il une importance ? Et l'ordre des personnes dans un groupe ?
3. Peut-on traiter le fichier ligne par ligne sans perdre l'information « fin de groupe » ?

Exercice 2 *À la main, sur un petit exemple*

But. Découper un petit fichier en groupes et calculer, pour chacun, l'ensemble des lettres cochées.
À quoi ça sert ? On obtient un exemple dont on **connaît la réponse** (15) pour tester le programme ; écrire les ensembles à la main fait apparaître l'opération à programmer.

```
xyz
yz

m
m
m

pqr
qrs
rst

abcd

hk
kh
h
```

On considère le fichier (inventé) suivant.

1. Combien contient-il de groupes, et combien de personnes dans chaque groupe ?
2. Pour chaque groupe, écrire l'ensemble des lettres demandé par la partie 1 et son cardinal.

Vérification : la somme doit valoir 15.

2 Programmer la partie 1

Exercice 3 *Découper en groupes*

But. Transformer le fichier en une liste de groupes, chaque groupe étant la liste des réponses de ses membres.
À quoi ça sert ? Les deux parties posent leur question groupe par groupe : avec cette liste, il suffira d'écrire une fonction qui traite un groupe, puis de l'appliquer à chacun.

Lire tout le fichier en une chaîne et construire une liste de groupes, chaque groupe étant la liste des lignes de ses membres. Afficher le nombre de groupes. ▷ coup de pouce

Outil Python : les ensembles (`set`) — rappel

Tout est présenté dans l'encadré « Outil Python : les ensembles » du défi *Crab Combat* (chapitre 4) : ensemble vide `set()`, ajout avec `add`, retrait avec `remove`, test `x in ensemble` en temps constant, éléments non modifiables (tuples), réunion `a | b` et intersection `a & b`. Un ensemble ne contient jamais deux fois le même élément.
À essayer. Que renvoie `len(set("banane"))` ? Après `e = set("abc")`, puis `e.add("d")`, `e.add("a")` et `e.remove("b")`, que contient `e` ? Que valent `set("abc") | set("bcd")` et `set("abc") & set("bcd")` ?

Exercice 4 *Réunir les réponses*

But. Calculer, pour un groupe, le nombre de lettres cochées par au moins une personne, puis faire la somme sur tous les groupes.
À quoi ça sert ? C'est la réponse de la partie 1. La réunion d'ensembles élimine d'elle-même les doublons, sans aucun test à écrire.

Écrire une fonction `nb_oui_quelquun(groupe)` qui renvoie le nombre de lettres présentes chez au moins un membre, puis faire la somme sur tout le fichier. Tester sur l'exemple inventé de la fiche, puis sur vos données. ▷ coup de pouce ▷ début de solution

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Le héros a mal lu la consigne : ce qui compte n'est pas ce que quelqu'un a coché, mais ce que tout le groupe a coché. Le fichier et le découpage en groupes restent les mêmes.
La nouvelle règle.

- Pour chaque groupe, on compte les lettres cochées par **toutes** les personnes du groupe.
- Une lettre oubliée par une seule personne du groupe ne compte plus.

La réponse attendue. La somme de ces nombres sur tous les groupes. Par exemple, le groupe formé des lignes `ab` et `bc` ne compte plus que pour 1 : seule la lettre `b` est cochée par les deux personnes.

Exercice 5 *Changer d'opération*

But. Reconnaître l'opération sur les ensembles qui traduit « toutes les personnes », puis adapter la fonction.
À quoi ça sert ? C'est la réponse de la partie 2, obtenue en changeant une seule opération. Attention au point de départ du calcul, qui ne peut plus être l'ensemble vide.

1. Quelle opération sur les ensembles correspond à la nouvelle question ?
2. Écrire la fonction correspondante et faire la somme. Sur l'exemple inventé de la fiche, on doit trouver `9`.

▷ coup de pouce ▷ début de solution

4 Approfondissement

★★★ Exercice 6 *Défi : sans ensembles*

But. Résoudre les deux parties avec un dictionnaire de compteurs au lieu d'ensembles.
À quoi ça sert ? Un même dictionnaire répond aux deux questions à la fois : on voit qu'une structure qui garde un peu plus d'information (combien de personnes, et pas seulement « oui ou non ») peut servir à plusieurs usages.

Résoudre les deux parties sans ensemble, avec un dictionnaire qui compte, pour chaque lettre, le nombre de membres du groupe qui l'ont cochée. Comment lit-on alors les deux réponses dans ce dictionnaire ?

▷ coup de pouce

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Couper d'abord aux lignes vides (`"\n\n"`), puis couper chaque bloc avec `split()`. ↑énoncé

Ex. 4 – piste `set("pqr")` donne l'ensemble `{"p", "q", "r"}`. Les doublons disparaissent d'eux-mêmes dans un ensemble. ↑énoncé

Ex. 4 – début de solution Partir d'un ensemble vide `set()`, puis, pour chaque membre, le remplacer par sa réunion avec l'ensemble des lettres de ce membre : `reunion = reunion | set(personne)`. ↑énoncé

Ex. 5 – piste Les lettres communes à deux ensembles `a` et `b` forment leur intersection `a & b`. Il suffit de remplacer `|` par `&` dans la fonction de la partie 1… ou presque (voir le coup de pouce suivant). ↑énoncé

Ex. 5 – début de solution Attention au point de départ : partir de l'ensemble vide ne marche plus ici. Partir plutôt de l'ensemble des réponses du **premier** membre du groupe. ↑

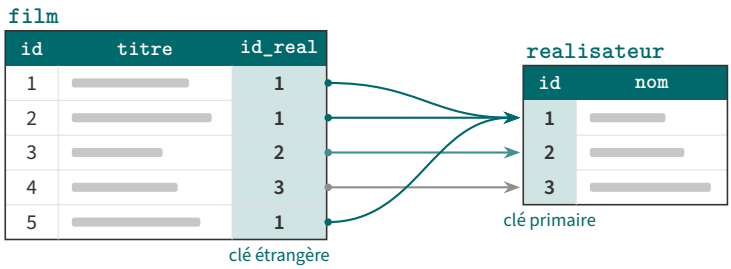
Ex. 6 – piste Une lettre compte pour la partie 2 si son compteur est égal au nombre de membres du groupe.

↑ énoncé

6

CHAPITRE 6

Bases de données et SQL



DANS CE CHAPITRE

- Activités préparatoires 280
- Cours 282
- Exercices 297
- TP et projets 308
- Fiche de révision 319
- Défis Advent of Code 321
- Corrigés 860

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Le grand tableau du secrétariat

▶ corrigé p. 860

quand un seul tableau ne suffit plus

Durée : 25 min · Sans ordinateur, par deux

Consignes

- On répond sur le cahier : on repère, on compte, on recopie. Aucun mot de vocabulaire n'est attendu avant l'exercice 4.

Exercice 1 *Le tableau tout-en-un*

Le secrétariat d'un lycée note tout dans un seul tableau, partagé entre plusieurs personnes. Voici un extrait :

	Nom	Prénom	Classe	Prof. principal	Salle	Matière	Note	Date
1	Martin	Léa	TNSI1	M. Rossi	B12	NSI	15	12/09
2	Bianchi	Hugo	TNSI1	M. Rossi	B12	NSI	11	12/09
3	Martin	Léa	TNSI2	Mme Costa	C04	NSI	17	12/09
4	Martin	Léa	TNSI1	M. Rosi	B12	Maths	13	15/09
5	Bianci	Hugo	TNSI1	M. Rossi	B21	Maths	9	15/09
6	Durand	Noé	TNSI2	Mme Costa	C04	NSI	23	12/09
7	Durand	Noé	T NSI 2	Mme Costa	C04	Maths	abs	2026-09-15
8	Silva	Inès	TNSI2	Mme Costa	C04	NSI	14	12/09

1. Combien de fois le nom du professeur principal de la TNSI1 est-il écrit ? Et la salle de la TNSI1 ?
2. Repérer dans le tableau **au moins cinq** anomalies (noter le numéro de ligne et la colonne), et les classer : faute de frappe, valeur impossible, même chose écrite de deux façons, information contradictoire.
3. Les lignes 1 et 3 parlent-elles de la même élève ? Les lignes 1 et 4 ? Comment en être sûr ?
4. La TNSI1 déménage en salle B15. Combien de cases faut-il modifier ? Que se passe-t-il si l'on en oublie une ?
5. Zoé Petit arrive en TNSI1 ; elle n'a encore aucune note. Comment l'inscrire dans ce tableau ?
6. On efface la ligne 8 (la note avait été saisie par erreur). Quelle autre information disparaît avec elle ?

Exercice 2 *Deux secrétaires, un seul fichier*

Le fichier du tableau est rangé sur un disque partagé. À 10 h 00, Mme Leroy l'ouvre et ajoute la note de Zoé. À 10 h 01, M. Giraud ouvre **le même fichier** et corrige « Bianci » en « Bianchi ». Mme Leroy enregistre à 10 h 05, M. Giraud à 10 h 06.

7. Que contient le fichier à 10 h 07 ? Quel travail a été perdu, et pourquoi ?
8. Imaginer une règle que le logiciel devrait appliquer pour que cela n'arrive pas.

Exercice 3 *Découper le tableau*

On décide de ranger les informations dans **trois** tableaux plus petits, chacun ne parlant que d'une seule chose. Corriger au passage les fautes de frappe et les écritures différentes d'une même chose ; une valeur impossible ne peut pas être corrigée sans information : écrire « ? » à sa place (à vérifier).

9. Sur le cahier, construire et remplir les trois tableaux suivants (une ligne par classe, une ligne par élève, une ligne par note) ; seules leurs colonnes sont données.

CLASSE		
Code	**Prof. princ.**	**Salle**

ÉLÈVE			
...	**Nom**	**Prénom**	**Classe**

NOTE			
Élève	**Matière**	**Note**	**Date**

10. Dans ÉLÈVE, deux élèves ont les mêmes nom et prénom. Inventer une **première colonne** (titre « ... ») qui permette de les distinguer à coup sûr. Que met-on alors dans la colonne « Élève » de NOTE, et pourquoi pas le nom ?

11. Reprendre les questions 4, 5 et 6 avec ces trois tableaux : que deviennent les problèmes ?

12. Écrire deux **règles** qu'un logiciel pourrait vérifier tout seul à chaque saisie, pour refuser les anomalies de l'exercice 1.

Exercice 4 *Mettre des mots*

13. Sur le cahier, proposer un nom pour chacune des notions suivantes.

a) un des trois petits tableaux (CLASSE, ÉLÈVE, NOTE) ;
b) une colonne / une ligne d'un de ces tableaux ;
c) la colonne inventée qui distingue chaque élève à coup sûr ;
d) la colonne « Élève » de NOTE, qui renvoie vers une ligne d'ÉLÈVE ;
e) une règle comme « une note est comprise entre 0 et 20 » ;
f) le logiciel qui range les tableaux, vérifie les règles et gère les modifications simultanées.

14. Recopier et compléter : « Un simple fichier partagé ne sait pas garantir que les données restent ..., ni que plusieurs personnes puissent ... »

COURS

Programme (BO) *« Identifier les concepts définissant le modèle relationnel » (relation, attribut, domaine, clef primaire, clef étrangère, schéma relationnel ; contraintes d'intégrité de domaine, de relation et de référence). « Savoir distinguer la structure d'une base de données de son contenu. Repérer des anomalies dans le schéma d'une base de données » (redondances, anomalies d'insertion, de suppression, de mise à jour). « Identifier les services rendus par un système de gestion de bases de données relationnelles : persistance des données, gestion des accès concurrents, efficacité de traitement des requêtes, sécurisation des accès. Identifier les composants d'une requête. Construire des requêtes d'interrogation à l'aide des clauses du langage SQL :* `SELECT`, `FROM`, `WHERE`, `JOIN`. *Construire des requêtes d'insertion et de mise à jour à l'aide de :* `UPDATE`, `INSERT`, `DELETE`. »

Prérequis la notion de **type** (domaine) ; les tableaux et dictionnaires (comme analogies) ; un peu de logique (**et**, **ou**, **non**). → rappels de 1re : dictionnaires p. 1058, données en tables p. 1062, binaire et booléens p. 1061

Objectifs *comprendre* pourquoi une base de données s'impose ; *modéliser* des données en relationnel (relations, clés) ; *connaître* le rôle d'un SGBD ; *écrire* des requêtes SQL d'interrogation **et** de mise à jour.

Situation de départ

1. Un lycée note tout (élèves, classes, notes) dans un seul tableau partagé. Pourquoi, au bout de quelques semaines, y trouve-t-on des fautes de frappe, des doublons et des informations contradictoires ?
2. Quand deux personnes modifient le même fichier en même temps, l'une écrase le travail de l'autre. Comment une banque ou un site marchand l'évitent-ils ?
3. Les films sont rangés dans un tableau, leurs réalisateurs dans un autre. Comment obtenir, en une seule demande, le titre de chaque film avec le nom de son réalisateur ?

I Pourquoi une base de données ? Quand le tableur ne suffit plus

Pour ranger quelques informations, un tableur (type Excel) ou un fichier texte peuvent suffire. Mais dès que les données deviennent **nombreuses**, **partagées** et **précieuses**, ces solutions craquent : une même information recopiée à plusieurs endroits finit par se contredire, deux personnes qui modifient le fichier en même temps écrasent le travail l'une de l'autre, la recherche devient lente. Une banque (un virement doit être fait en entier ou pas du tout : c'est une **transaction**), un site marchand (le dernier article en stock ne doit pas être vendu deux fois), un réseau social (des milliards de lignes interrogées en une fraction de seconde) ou le carnet de notes d'un lycée (pas de note pour un élève qui n'existe pas, chacun ne voit que ce qui le concerne) ne peuvent pas s'en contenter.

Règle 1 — Ce qu'un simple fichier ne sait pas faire

Une base de données assure quatre choses qu'un tableur néglige : la **cohérence** (des règles empêchent les données absurdes), les **accès concurrents** (plusieurs utilisateurs à la fois, sans se marcher dessus), l'**efficacité** (recherche rapide même sur d'énormes volumes) et la **sécurité** (droits d'accès, sauvegarde, transactions).

Pour les curieux (facultatif)
Vous en utilisez sans le savoir des dizaines par jour : votre téléphone stocke ses contacts, ses messages, ses photos dans des bases de données. La quasi-totalité des applications du monde repose dessus.

II Le modèle relationnel

En **1970**, un chercheur d'IBM, **Edgar F. Codd**, propose d'organiser les données en **tableaux** reliés entre eux : c'est le **modèle relationnel**, encore ultra-dominant aujourd'hui.

Définition 1 — Relation, attribut, enregistrement, domaine

Une **relation** (ou **table**) est un tableau à deux dimensions. Chaque **colonne** est un **attribut**, qui porte un **nom** et un **domaine** (son type : entier, texte, date…). Chaque **ligne** est un **enregistrement** : une entité décrite par une valeur pour chaque attribut.

Prenons une base de données de **films**. Voici la relation `film` :

id	titre	id_real	annee	note
1	Inception	1	2010	9
2	Interstellar	1	2014	9
3	Amélie	2	2001	8
4	Parasite	3	2019	10
5	Dunkerque	1	2017	7

Définition 2 — Schéma

Le **schéma** d'une relation donne le nom de la relation et, pour chaque attribut, son nom et son domaine. On le note ainsi (la clé primaire est soulignée) :

`film` (`id` : entier, `titre` : texte, `id_real` : entier, `annee` : entier, `note` : entier)

Le **schéma d'une base** est l'ensemble des schémas de ses relations.

Règle 2 — Structure et contenu d'une base

La **structure** d'une base, c'est son schéma : les relations, leurs attributs, leurs domaines et leurs clés. Elle est fixée à la création des tables (on verra `CREATE TABLE` au § V) et change rarement. Le **contenu**, ce sont les enregistrements (les lignes) présents à un instant donné : il change sans cesse, à chaque ajout, modification ou suppression. Ajouter un film change le contenu ; ajouter un attribut `duree` à la relation `film` change la structure.

Remarque
Une relation est un **ensemble** d'enregistrements : l'ordre des lignes n'a aucune importance, et il ne peut pas y avoir deux lignes totalement identiques.

► **Exercices d'application :** exercice 1 (lire un schéma)

III Les contraintes d'intégrité : garder des données saines

Le modèle relationnel impose des **règles** qui empêchent les incohérences. Ce sont les **contraintes d'intégrité**.

A. Contrainte de domaine

Règle 3

Chaque valeur d'un attribut doit appartenir à son **domaine**, c'est-à-dire à son **type**. Un attribut `annee` de type *entier* doit refuser la valeur `"hier"`. Bien choisir les domaines, c'est déjà interdire une foule d'erreurs.

Pour les curieux — *SQLite, un SGBD au typage souple* (facultatif)
La plupart des SGBD (PostgreSQL, MySQL…) refusent `"hier"` dans une colonne `INTEGER`. **SQLite, lui, peut l'accepter** : le type déclaré n'est qu'une *préférence* (on parle d'**affinité**). Il convertit `'2024'` en l'entier `2024`, mais range `'hier'` tel quel, comme du texte, sans erreur. Pour obtenir un vrai contrôle de domaine en SQLite, on ajoute une contrainte explicite, par exemple `annee INTEGER CHECK (typeof(annee) = 'integer')`, ou l'on déclare la table `STRICT` (SQLite 3.37 et plus).

Remarque — *Les principaux types SQL*
En SQL (ici en SQLite), les domaines les plus courants sont : `INTEGER` (entier), `REAL` (nombre à virgule), `TEXT` (chaîne de caractères) et `BLOB` (données binaires). Un **booléen** se code par un `INTEGER` (`0`/`1`) et une **date** par un `TEXT` (ou un entier). *Attention aux « faux nombres »* : un code postal (`06000`) ou un numéro de téléphone se stockent en `TEXT`, pas en `INTEGER` — sinon le `0` initial disparaît et `06000` devient `6000` !

B. Clé primaire : identifier sans ambiguïté

Définition 3 — Clé primaire

La **clé primaire** d'une relation est un attribut (ou un groupe d'attributs) dont la valeur **identifie de façon unique** chaque enregistrement. Son **rôle** est donc de *distinguer sans ambiguïté* les lignes ; le SGBD garantit qu'**aucune** clé primaire ne se répète (et qu'elle n'est jamais vide). C'est la **contrainte d'entité**.

Dans `film`, deux films peuvent partager le même titre ou la même année : c'est pourquoi on ajoute un attribut `id` (un numéro unique) qui sert de clé primaire.

Règle 4 — Question de bac : « tel attribut peut-il être clé primaire ? »

Un attribut ne peut être clé primaire que si ses valeurs sont **toujours différentes** d'une ligne à l'autre (et jamais vides). C'est pourquoi le `nom`, le `titre` ou un numéro de `téléphone` font en général de **mauvaises** clés (homonymes, valeurs partagées ou manquantes) : on leur préfère un **identifiant** dédié. Plusieurs attributs peuvent convenir (les *clés candidates*) ; on en choisit **une**.

C. Clé étrangère : relier les tables

Dans `film`, l'attribut `id_real` ne contient pas le *nom* du réalisateur, mais un **numéro** qui **renvoie** à une autre table, `realisateur` :

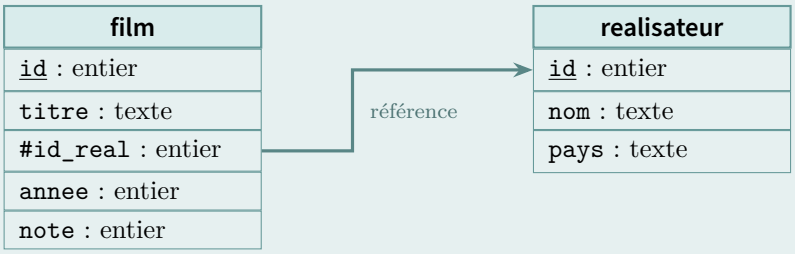
id	nom	pays
1	Nolan	Royaume-Uni
2	Jeunet	France
3	Bong Joon-ho	Corée du Sud

Définition 4 — Clé étrangère

Une **clé étrangère** est un attribut d'une relation dont la valeur doit être celle d'une **clé primaire** dans une autre (ou la même) relation. Son **rôle** est de *relier deux tables* et de garantir que la référence **existe** vraiment. Ici, `film.id_real` est une clé étrangère qui référence `realisateur.id`.

↻ **Retour sur l'activité.** La colonne inventée dans l'activité pour distinguer les deux Léa Martin était une **clé primaire** de ÉLÈVE ; la colonne « Élève » de NOTE, qui renvoie vers une ligne d'ÉLÈVE plutôt que de recopier le nom, était une **clé étrangère**.

On représente souvent le schéma de la base par un dessin : une boîte par relation, la clé primaire soulignée, la clé étrangère précédée d'un `#`, et une flèche qui va de la clé étrangère vers la clé primaire qu'elle référence.



Règle 5 — Contrainte de référence

Une clé étrangère ne peut désigner qu'une valeur **existante**. Impossible d'ajouter un film dont le `id_real` vaut `7` si aucun réalisateur n'a l'`id 7`. Impossible, aussi, de supprimer le réalisateur `1` tant que des films le référencent. La base reste **cohérente**.

Règle 6 — Question de bac : différence entre clé primaire et clé étrangère

La **clé primaire** *identifie* de façon unique les lignes de **sa** table (unicité garantie). La **clé étrangère** *pointe* vers la clé primaire d'une **autre** table pour les relier (elle, peut se répéter : un même réalisateur a plusieurs films). En résumé : la primaire *distingue*, l'étrangère *relie*.

À retenir — pourquoi deux tables plutôt qu'une ? Les anomalies d'un mauvais schéma

On aurait pu écrire le nom et le pays du réalisateur en toutes lettres dans chaque film. Mais alors « Nolan, Royaume-Uni » serait recopié 3 fois : c'est une **redondance**. Elle entraîne trois **anomalies** :

- **anomalie de mise à jour** : corriger le pays de Nolan oblige à modifier toutes ses lignes ; en oublier une rend la base incohérente (une faute de frappe, « Nollan », crée même un réalisateur fantôme) ;
- **anomalie d'insertion** : impossible d'enregistrer un réalisateur qui n'a encore aucun film, sans inventer un film ;
- **anomalie de suppression** : supprimer le seul film de Bong Joon-ho efface du même coup tout ce que la base savait de lui.

En **séparant** en deux tables reliées par une clé étrangère, chaque information n'est écrite **qu'une fois** : les trois anomalies disparaissent. Repérer ces anomalies dans un schéma, puis le découper, c'est tout l'art du modèle relationnel.

6

► **Exercices d'application :** exercice 24 (un schéma mal conçu : anomalies, structure et contenu)

D. D'autres contraintes utiles

Au-delà des trois grandes contraintes (domaine, entité, référence), on peut en imposer d'autres à un attribut :

- `NOT NULL` : l'attribut ne peut pas être **vide** (obligatoire) ;
- `UNIQUE` : toutes ses valeurs doivent être **différentes**, sans pour autant être la clé primaire (*ex.* une adresse e-mail) ;
- une **contrainte utilisateur** (ou « métier ») : une règle propre au contexte, par exemple « une `note` est comprise entre `0` et `20` » ou « un `age` est positif ».

Règle 7 — Récapitulatif : les contraintes d'intégrité

de domaine (bon type) ; **d'entité** (clé primaire unique) ; **de référence** (clé étrangère existante) ; et les contraintes complémentaires (`NOT NULL`, `UNIQUE`, règles métier). Toutes servent le même but : garder des données **cohérentes**.

↻ **Retour sur l'activité.** Les « règles vérifiées à chaque saisie » imaginées dans l'activité sont des contraintes d'intégrité : « abs » dans la colonne Note viole la contrainte de domaine, une note de 23 une contrainte utilisateur.

► **Exercices d'application :** exercices 2 **et** 3 (clés, contraintes, faire évoluer le modèle)

IV Le SGBD : le chef d'orchestre

On ne manipule jamais les fichiers d'une base « à la main » : on passe par un **logiciel** spécialisé.

Définition 5 — Système de gestion de bases de données (SGBD)

Un **SGBD** est le logiciel qui **stocke**, **protège** et **interroge** une base de données. On ne lui parle pas dans un langage de programmation classique, mais avec un langage dédié : **SQL**.

Règle 8 — Les grandes fonctions d'un SGBD

- **Persistance** : les données survivent à l'extinction de la machine.
- **Cohérence** : il fait respecter toutes les contraintes d'intégrité.
- **Efficacité** : il recherche vite, même dans des milliards de lignes (grâce à des *index* — souvent des arbres, cf. chapitre *Arbres* !).
- **Sécurité** : droits d'accès par utilisateur, sauvegardes. Quand la base contient des **données personnelles**, protéger les accès est aussi une obligation légale : on ne collecte que ce qui est nécessaire et l'on en garde le contrôle (le RGPD dans l'Union européenne ; à Monaco, la loi n° 1.565 du 3 décembre 2024, contrôlée par l'Autorité de protection des données personnelles, APDP).
- **Accès concurrents** : plusieurs utilisateurs en même temps, sans conflit, grâce aux **transactions** (un problème de ressource partagée, comme entre les processus du chapitre 4).

↻ **Retour sur l'activité.** Les deux secrétaires qui enregistraient le même fichier à une minute d'intervalle se heurtaient au problème des **accès concurrents** : la correction de M. Giraud effaçait la note saisie par Mme Leroy. Un SGBD empêche une modification d'en écraser une autre.

Au bac — « intérêt d'un SGBD » / « deux services rendus »

Il suffit d'en citer deux parmi la liste ci-dessus. Réponses attendues typiques : il **garantit la cohérence** des données (contraintes d'intégrité), il gère les **accès concurrents** de plusieurs utilisateurs, il assure la **persistance** et la **sécurité**, et il permet d'**interroger efficacement** de gros volumes. Là où un simple fichier laisserait apparaître des incohérences, le SGBD les *empêche*.

Remarque — *Les transactions : le « tout ou rien »*
Une **transaction** regroupe des opérations qui doivent réussir **ensemble** ou **pas du tout**. Le virement bancaire (débit *et* crédit) en est l'exemple parfait : si une panne survient au milieu, la transaction est **annulée** entièrement, comme si rien ne s'était passé. On résume ces garanties par l'acronyme **ACID**.

Pour les curieux (facultatif)
Des SGBD célèbres : **PostgreSQL**, **MySQL**, **Oracle**, **SQLite**. Ce dernier, minuscule et sans serveur, est embarqué dans presque toutes les applications et tous les téléphones. C'est celui que nous utiliserons (via *DB Browser* ou un site en ligne).

V SQL : interroger une base de données

SQL (*Structured Query Language*) est le langage universel des bases relationnelles. Une instruction SQL s'appelle une **requête**. Commençons par la plus importante : **lire** des données.

A. Créer et remplir une table

Pour mémoire, on crée une table avec `CREATE TABLE` et on y ajoute des lignes avec `INSERT`.

```sql
CREATE TABLE film (
    id INTEGER PRIMARY KEY,
    titre TEXT,
    id_real INTEGER,
    annee INTEGER,
    note INTEGER
);

INSERT INTO film (id, titre, id_real, annee, note)
VALUES (1, 'Inception', 1, 2010, 9);
```

Remarque — *Déclarer une clé étrangère*
Dans `CREATE TABLE`, on déclare la clé étrangère avec `REFERENCES` : `id_real INTEGER REFERENCES realisateur(id)`. Le SGBD fait alors respecter la contrainte de référence. Attention, **SQLite** ne vérifie les clés étrangères que si on le lui demande, au début de chaque session, par `PRAGMA foreign_keys = ON;` : sans cette ligne, il accepte un `id_real` qui ne désigne aucun réalisateur.

Remarque
En SQL, les mots-clés (`SELECT`, `FROM`…) s'écrivent traditionnellement en **majuscules** (ce n'est pas obligatoire) et les chaînes de caractères entre **apostrophes simples** : `'Inception'`.

B. `SELECT … FROM` : choisir des colonnes

Règle 9 — La requête de base

`SELECT` liste les **colonnes** à récupérer ; `FROM` indique la **table**. L'étoile `*` signifie « toutes les colonnes ».

▷ Je prédis. Cacher la table résultat, la prévoir, puis vérifier.

```sql
SELECT titre, annee
FROM film;
```

donne :

titre	annee
Inception	2010
Interstellar	2014
Amélie	2001
Parasite	2019
Dunkerque	2017

C. `WHERE` : filtrer les lignes

Règle 10

`WHERE` garde uniquement les lignes qui **satisfont une condition**. On combine les conditions avec `AND`, `OR`, `NOT`, et on compare avec `=`, `<>` (différent), `<`, `>`, `<=`, `>=`.

▷ Je prédis. Cacher la table résultat, la prévoir, puis vérifier.

```sql
SELECT titre
FROM film
WHERE annee > 2010 AND note >= 9;
```

renvoie `Interstellar` et `Parasite`.

> **Remarque** — *Tester une case **vide** :* `IS NULL`
> Un attribut peut être **vide** (valeur `NULL` — par exemple une date de retour non encore renseignée). On ne la teste **pas** avec `= NULL` (qui ne fonctionne pas en SQL), mais avec `IS NULL` (case vide) ou `IS NOT NULL` (case renseignée) :
>
> ```sql
> SELECT titre
> FROM film
> WHERE note IS NULL;      -- les films dont la note n'est pas renseignee
> ```

D. `ORDER BY` : trier le résultat

```sql
SELECT titre, note
FROM film
ORDER BY note DESC;
```

trie les films de la meilleure note à la moins bonne (`DESC` = décroissant ; `ASC` = croissant, par défaut).

▷ **Je prédis.** Quel film arrive en tête ? Lequel en dernier ?
▷ **Je vérifie.** `Parasite` (10) en tête, `Dunkerque` (7) en dernier. `Inception` et `Interstellar` ont la même note : leur ordre entre eux n'est pas garanti.

E. `DISTINCT` : supprimer les doublons

```sql
SELECT DISTINCT id_real
FROM film;
```

renvoie chaque réalisateur **une seule fois**, même s'il a plusieurs films.

▷ **Je prédis.** Combien de lignes contient le résultat ? Et sans `DISTINCT` ?
▷ **Je vérifie.** 3 lignes (`1`, `2`, `3`) ; sans `DISTINCT`, 5 lignes : le `1` de Nolan apparaît trois fois.

F. Les fonctions d'agrégation

Elles **résument** une colonne en *une* valeur : `COUNT` (compter), `SUM` (somme), `AVG` (moyenne), `MIN`, `MAX`.

```sql
SELECT COUNT(*)                  -- nombre de films : 5
FROM film;

SELECT AVG(note)                 -- note moyenne
FROM film;

SELECT MAX(note)                 -- meilleure note avant 2015
FROM film
WHERE annee < 2015;
```

▷ **Je prédis.** Que renvoient la deuxième et la troisième requête ?
▷ **Je vérifie.** `8.6` (la somme `43` divisée par `5`), puis `9` : `Parasite` (`10`) date de 2019, il est écarté par le `WHERE` *avant* le calcul du maximum.

> **Pour les curieux** — *Au-delà du strict programme* (facultatif)
> Les clauses présentées d'ici aux jointures (`LIKE`, `LIMIT`, l'alias `AS`, `GROUP BY` et `HAVING`) ne sont pas exigées au bac, mais elles sont très courantes et rendent SQL vraiment puissant. Les fonctions d'agrégation ci-dessus, elles, sont au programme, à condition de les utiliser **sans** `GROUP BY` ni `HAVING`.

G. `LIKE` : rechercher un motif

`LIKE` compare du texte à un **motif** : `%` remplace n'importe quelle suite de caractères, `_` un seul.

▷ Je prédis. Cacher la table résultat, la prévoir, puis vérifier.

```sql
SELECT titre
FROM film
WHERE titre LIKE 'In%';
```

renvoie les titres commençant par « In » : `Inception`, `Interstellar`.

H. Alias `AS` et `LIMIT`

`AS` renomme une colonne dans le résultat ; `LIMIT` borne le nombre de lignes.

```sql
SELECT titre AS film, note AS etoiles
FROM film
ORDER BY note DESC
LIMIT 3;
```

donne les **3** meilleurs films, avec des colonnes renommées.

▷ Je prédis. Quelle table résultat (en-têtes compris) ?
▷ Je vérifie. Deux colonnes nommées `film` et `etoiles` ; trois lignes : `Parasite | 10`, puis `Inception | 9` et `Interstellar | 9`, dans un ordre non garanti entre ces deux-là.

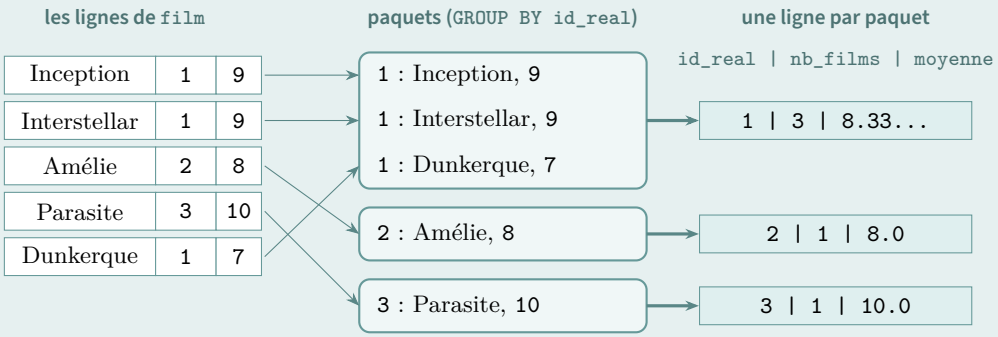
I. `GROUP BY` : regrouper puis résumer

`GROUP BY` forme des **paquets** de lignes partageant une même valeur, et applique un agrégat à **chaque** paquet.

▷ Je prédis. Cacher la table résultat, la prévoir, puis vérifier.

```sql
SELECT id_real, COUNT(*) AS nb_films, AVG(note) AS moyenne
FROM film
GROUP BY id_real;
```

donne, **pour chaque réalisateur**, son nombre de films et sa note moyenne :



On peut filtrer *après* regroupement avec `HAVING` :

```sql
SELECT id_real, COUNT(*) AS nb
FROM film
GROUP BY id_real
HAVING COUNT(*) >= 2;
```

ne garde que les réalisateurs ayant **au moins 2** films.

▷ **Je prédis.** Quelle table résultat ?
▷ **Je vérifie.** Une seule ligne, `1 | 3` : seul le réalisateur `1` (Nolan) a au moins deux films.

> **Pour les curieux** (facultatif)
> Retenez la différence : `WHERE` filtre les lignes *avant* le regroupement, `HAVING` filtre les paquets *après*.

J. Les jointures : croiser deux tables

Notre requête `GROUP BY` affiche des `id_real` (`1`, `2`…), pas des noms. Pour afficher le **nom** du réalisateur, il faut **recoller** les deux tables : c'est une **jointure**.

Règle 11 — `JOIN ... ON`

`FROM A JOIN B ON A.cle_etrangere = B.cle_primaire` fusionne chaque ligne de `A` avec la ligne de `B` qui lui correspond, selon l'égalité indiquée après `ON`.

▷ **Je prédis.** Cacher la table résultat, la prévoir, puis vérifier.

```sql
SELECT film.titre, realisateur.nom
FROM film
JOIN realisateur ON film.id_real = realisateur.id;
```

associe à chaque film le nom de son réalisateur :

titre	nom
Inception	Nolan
Interstellar	Nolan
Amélie	Jeunet
Parasite	Bong Joon-ho
Dunkerque	Nolan

À retenir — préfixer un attribut par sa table

Quand un nom d'attribut existe dans les deux tables (ici `id`), on le **préfixe** par le nom de sa table (`realisateur.id`) pour lever l'ambiguïté. On peut évidemment combiner une jointure avec `WHERE`, `ORDER BY`, `GROUP BY`… et joindre **trois** tables ou plus.

► **Exercices d'application :** exercices 4 à 14 (interroger, agréger, regrouper, jointures)

VI SQL : modifier une base de données

Au-delà de la lecture, SQL sait aussi **écrire**. Trois instructions, à manier avec prudence.

Règle 12 — Les trois commandes de mise à jour

- `INSERT INTO ... VALUES ...` : **ajouter** une ligne ;
- `UPDATE ... SET ... WHERE ...` : **modifier** des lignes existantes ;
- `DELETE FROM ... WHERE ...` : **supprimer** des lignes.

```sql
UPDATE film
SET note = 8
WHERE titre = 'Dunkerque';

DELETE FROM film
WHERE annee < 2005;
```

6

▷ **Je prédis.** Après ces deux requêtes, combien de lignes reste-t-il dans `film`, et quelle est la note de `Dunkerque` ?
▷ **Je vérifie.** `4` lignes (seul `Amélie`, de 2001, est supprimé) ; `Dunkerque` a désormais la note `8`.

Règle 13 — Le `WHERE` qui sauve des vies

N'oubliez jamais le `WHERE` dans un `UPDATE` ou un `DELETE` ! `DELETE FROM film;` (sans condition) efface **tous** les films d'un coup. C'est la requête la plus redoutée des informaticiens.

► **Exercices d'application :** exercices 15 **et** 16 (insérer, modifier, supprimer ; transactions et ACID)

A. Injection SQL : ne jamais coller une saisie dans une requête

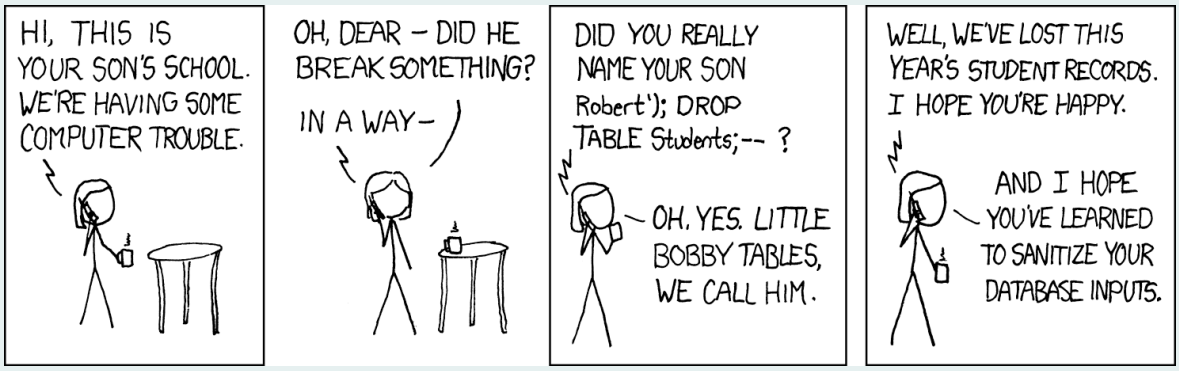
Dans une vraie application (un site web, le logiciel de vie scolaire…), personne ne tape les requêtes : un **programme** les fabrique à partir de ce que l'utilisateur a saisi. En Python, le module `sqlite3` envoie une requête à la base avec `connexion.execute(requete)`. La tentation est grande de coller la saisie dans le texte de la requête :

```python
titre = input("Titre du film : ")
requete = "SELECT titre, note FROM film WHERE titre = '" + titre + "'"
resultat = connexion.execute(requete).fetchall()
```

▷ **Je prédis.** Un utilisateur tape `x' OR '1'='1`. Quelle requête est envoyée au SGBD ? Que renvoie-t-elle ?
▷ **Je vérifie.** La requête devient `SELECT titre, note FROM film WHERE titre = 'x' OR '1'='1'`. La condition `'1'='1'` est toujours vraie : on obtient **tous** les films, alors qu'aucun ne s'appelle « x ».

Définition 6 — Injection SQL

Une **injection SQL** consiste à saisir un texte qui, une fois collé dans une requête, en **change le sens** : une apostrophe ferme la chaîne prévue par le programmeur, et la suite est lue comme du SQL. C'est l'une des failles les plus exploitées du Web : lire des données interdites, se connecter sans mot de passe, modifier ou détruire des tables.



Randall Munroe, « Exploits of a Mom », xkcd n° 327 (2007), `https://xkcd.com/327/` — licence CC BY-NC 2.5

La blague expliquée — « Little Bobby Tables »

L'école appelle une mère : elle a « des problèmes informatiques ». Son fils s'appellerait-il vraiment `Robert'); DROP TABLE Students;--` ? « Oui, on l'appelle le petit Bobby Tables. » Résultat : les fiches des élèves de l'année sont perdues. Le logiciel de l'école enregistrait chaque nouvel élève avec une requête fabriquée par collage, du type `INSERT INTO Students (nom) VALUES ('nom');`. Avec le prénom de Bobby, le SGBD reçoit :

```
INSERT INTO Students (nom) VALUES ('Robert'); DROP TABLE Students;--');
```

- l'apostrophe après `Robert` **ferme la chaîne** : pour le SGBD, le nom est simplement « Robert » ;
- `);` **termine** l'`INSERT`, qui est correct et s'exécute ;
- `DROP TABLE Students;` est une **seconde requête**, qui supprime la table entière, structure et contenu [au-delà du programme] : pire qu'un `DELETE` sans `WHERE` ;
- `--` ouvre un **commentaire** SQL : la fin prévue par le programmeur, `');`, est ignorée, donc aucune erreur de syntaxe ne trahit l'attaque.

La mère conclut qu'il faut « nettoyer » les saisies. Mieux encore : ne **jamais** les coller dans une requête.

Règle 14 — Requêtes paramétrées

On écrit un point d'interrogation `?` à la place de chaque valeur, et on passe les valeurs **à part**, dans un tuple :

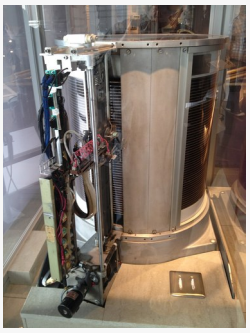
```
requete = "SELECT titre, note FROM film WHERE titre = ?"
resultat = connexion.execute(requete, (titre,)).fetchall()
```

`sqlite3` transmet la valeur comme une simple **donnée**, jamais interprétée comme du SQL. Avec la saisie `x' OR '1'='1`, on cherche un film dont le titre est exactement ce texte : aucune ligne. Bobby, lui, serait enregistré sous son nom complet, apostrophe comprise, et la table survivrait. Donc : **jamais** de valeur saisie collée dans une requête (ni avec `+`, ni avec une f-string), toujours un `?` par valeur.

Remarque

`connexion.execute` refuse d'exécuter deux instructions à la fois : l'attaque de Bobby échouerait telle quelle avec `sqlite3`. Mais l'injection `x' OR '1'='1`, qui reste dans une seule requête, fonctionne : seul le `?` protège vraiment. C'est un aspect concret de la **sécurité** des accès, l'un des services d'un SGBD (§ IV). On le met en pratique dans la partie 6 du TP « La ligue de futsal des quartiers ».

Pour les curieux — *Un peu d'histoire… et d'humour* (facultatif)



Le mécanisme du disque IBM 350 (1956)

Avant de ranger les données, il a fallu pouvoir les **retrouver vite**. Jusque dans les années 1950, elles dorment sur des cartes perforées ou des bandes magnétiques, qu'il faut dérouler du début. En **1956**, IBM commercialise le **premier disque dur**, l'**IBM 350** du système *RAMAC* : cinquante plateaux de 61 cm de diamètre pour… 5 millions de caractères (moins de 4 Mo), loués plus de 3 000 dollars par mois. Pour la première fois, on accède **directement** à n'importe quel enregistrement. Le modèle relationnel naît ensuite en **1970** sous la plume d'**Edgar F. Codd**, chez IBM — une idée si féconde qu'elle lui vaudra le prix **Turing** en 1981. Dans la foulée, **Donald Chamberlin** et **Raymond Boyce** conçoivent un langage pour l'interroger : *SEQUEL*, vite rebaptisé **SQL**. D'où l'éternelle querelle : faut-il prononcer « *ess-cue-elle* » ou « *sequel* » ? Un demi-siècle plus tard, personne n'a tranché — c'est la seule dispute plus vieille que celle des accolades. Codd et ses collègues cherchaient alors à gérer les **grandes masses de données** des entreprises (le prototype d'IBM s'appelait *System R*) ; depuis, SQL règne sans partage : les données de la planète, de votre banque à votre application de messagerie, s'expriment dans ce langage vieux d'un demi-siècle et jamais détrôné.

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise :** un enregistrement ressemble à un objet, ses colonnes à des attributs (*Programmation orientée objet et paradigmes*) ; on prolonge *Les données en tables* (Première).
- **Déjà croisé :** les index sont souvent des arbres de recherche (*Arbres*) ; les transactions règlent un partage de ressource, comme en *Processus et ordonnancement*.
- **Pour aller plus loin :** un réseau social se modélisera parfois mieux par des *Graphes* ; `LIKE` annonce la *Recherche textuelle*.

Bilan — la carte mémoire

Notion / clause	À retenir
Relation / attribut / enregistrement	table / colonne / ligne
Domaine	le type d'un attribut (contrainte de domaine)
Clé primaire	identifie chaque ligne de façon **unique**
Clé étrangère	référence une clé primaire d'une autre table
SGBD	logiciel qui stocke, protège, interroge (persistance, cohérence, …)
`SELECT ... FROM ... WHERE`	choisir des colonnes, filtrer des lignes
`ORDER BY` / `DISTINCT`	trier / enlever les doublons
`COUNT/SUM/AVG/MIN/MAX`	résumer une colonne en une valeur
`GROUP BY` / `HAVING`	regrouper, filtrer les groupes (au-delà du BO)
`JOIN ... ON`	croiser deux tables par une clé
`INSERT` / `UPDATE` / `DELETE`	ajouter / modifier / supprimer (**penser au `WHERE` !**)
Requête paramétrée (`?`)	ne jamais coller une saisie dans une requête (injection SQL)

Retour à la situation de départ

1. Les mêmes informations y sont recopiées à plusieurs endroits, et rien ne contrôle la saisie. Le **modèle relationnel** découpe les données en **relations** reliées par des **clés étrangères**, où chaque information n'est écrite qu'une fois, et les **contraintes d'intégrité** refusent les valeurs absurdes (§ II et III).
2. Elles confient leurs données à un **SGBD**, qui gère les **accès concurrents** grâce aux **transactions** (« tout ou rien ») et garantit la cohérence des données (§ IV).
3. Par une requête SQL avec **jointure** (`JOIN ... ON`), qui recolle chaque film à la ligne de `realisateur` désignée par sa clé étrangère `id_real` (§ V.J).

Erreurs fréquentes

- **Oublier le `WHERE` d'un `UPDATE` / `DELETE`.** Sans lui, on modifie ou supprime **toute la table**. *Le réflexe :* « le `WHERE` qui sauve des vies ».
- **Oublier la condition de jointure.** `FROM a, b` sans `WHERE`/`ON` produit le **produit cartésien** (toutes les combinaisons).
- **Confondre `WHERE` et `HAVING`.** `WHERE` filtre les **lignes** (avant regroupement), `HAVING` filtre les **groupes** (après `GROUP BY`).
- **Confondre clé primaire et clé étrangère.** La clé primaire **identifie** une ligne ; la clé étrangère **référence** la clé primaire d'une autre table.
- **Croire que `ORDER BY` / `DISTINCT` changent les données.** Ils ne changent que l'**affichage** du résultat, pas la table.
- **Détails de syntaxe.** Chaînes entre **apostrophes** (`'Ada'`), test d'égalité avec **un seul** `=`.

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

- ☐ lire un **schéma relationnel** (clés primaire et étrangère) → ex. 1, 2, 3 ;
- ☐ distinguer la **structure** d'une base de son **contenu**, et repérer les **anomalies** d'un schéma (redondance ; insertion, suppression, mise à jour) → ex. 3, 24 ;
- ☐ écrire `SELECT` / `FROM` / `WHERE` (`AND`, `OR`, `NOT`), `ORDER BY`, `DISTINCT` → ex. 4, 5, 6 ;
- ☐ utiliser les **agrégats** (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) → ex. 8, 9, 10 ; pour aller plus loin, regrouper avec `GROUP BY` / `HAVING` → ex. 11 ;
- ☐ écrire une **jointure** (`JOIN ... ON`) entre deux tables → ex. 12, 13 ;
- ☐ **mettre à jour** une base (`INSERT`, `UPDATE`, `DELETE`) sans oublier le `WHERE` → ex. 15, 18 ;
- ☐ expliquer le rôle d'un **SGBD** et d'une **transaction** (propriétés ACID) → ex. 2, 16.

Vers le Grand Oral

Quelques pistes de questions que ce chapitre permet de préparer (à étayer d'un exemple concret) :

- **Pourquoi répartir les données en plusieurs tables ?** *(éviter la redondance, garantir la cohérence, relier par des clés.)*
- **Qu'est-ce qu'une transaction, et pourquoi les propriétés ACID ?** *(un virement bancaire en tout-ou-rien ; les accès simultanés.)*
- **SQL : comment retrouver une information dans des millions de lignes ?** *(langage déclaratif « je décris ce que je veux » ; les index, qui sont des arbres.)*

EXERCICES

Mode d'emploi

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 32.
- Écrire chaque requête SQL **sur plusieurs lignes** (`SELECT...` / `FROM...` / `WHERE...`).
- **Tester ses requêtes** dans la console SQL de **Basthon** : `console.basthon.fr/?kernel=sql`. Trois bases sont fournies (`musique.sql`, `pays.sql`, `bibliotheque.sql`), à télécharger sur `maththibaud.github.io/informatique-en-autonomie/f/t06-exercices-bdd-sql` : copier-coller le script de la base voulue, l'exécuter, puis interroger. Chaque exercice indique la base à charger.

Modèle relationnel ↑cours

On travaillera ici sur la base `bibliotheque`, dont voici le schéma (clé primaire soulignée, clé étrangère notée `#`) : `auteur(id, nom, pays)`

`livre(id, titre, #id_auteur, annee, genre)`
`adherent(id, nom, prenom)`
`emprunt(id, #id_livre, #id_adherent, date_emprunt)`

★ Exercice 1 — *Lire un schéma* ▶ corrigé p. 862 • essentiel

1. Combien de relations comporte la base ? Combien d'attributs pour la relation `emprunt` ?
2. Donner la clé primaire de `livre`, puis les clés étrangères de `emprunt`.
3. Quel est le **rôle** de l'attribut `id_auteur` dans la relation `livre` ?
4. Proposer un **domaine** (type) raisonnable pour `annee` et pour `date_emprunt`.

▷ coup de pouce

★ Exercice 2 — *Clés et contraintes* ▶ corrigé p. 862

1. Expliquer pourquoi on préfère un attribut `id` plutôt que `titre` comme clé primaire de `livre`.
2. Le SGBD refuse d'insérer dans `emprunt` une ligne dont l'`id_livre` vaut `99`. Quelle **contrainte** est en cause ? L'expliquer.
3. Rappeler la **différence** entre une clé primaire et une clé étrangère.

▷ coup de pouce

★★ Exercice 3 — *Faire évoluer le modèle* ▶ corrigé p. 862

On souhaite désormais enregistrer, pour chaque livre, sa **maison d'édition** (nom, ville). Proposer une évolution du schéma qui **évite de recopier** ces informations dans chaque livre (on précisera la nouvelle relation, sa clé primaire, et la clé étrangère ajoutée). ▷ coup de pouce

Les trois bases fournies ↑cours

Base `musique` :

artiste		
id	**nom**	**pays**
1	Daft Punk	France
2	Stromae	Belgique
3	Adele	Royaume-Uni
4	Angèle	Belgique

chanson					
id	**titre**	**id_artiste**	**annee**	**duree**	**streams**
1	One More Time	1	2000	320	450
2	Get Lucky	1	2013	369	900
3	Instant Crush	1	2013	337	600
4	Alors on danse	2	2009	216	700
5	Papaoutai	2	2013	232	850
6	Formidable	2	2013	197	500
7	Someone Like You	3	2011	285	950
8	Hello	3	2015	295	1000
9	Balance ton quoi	4	2018	191	400
10	Bruxelles je t'aime	4	2021	200	300

(`duree` en secondes, `streams` en millions.)

Base `pays` (`population` en millions d'habitants, `superficie` en milliers de km^2) :

id	nom	continent	capitale	population	superficie
1	France	Europe	Paris	68	552
2	Allemagne	Europe	Berlin	84	358
3	Italie	Europe	Rome	59	301
4	Espagne	Europe	Madrid	48	506
5	Japon	Asie	Tokyo	125	378
6	Chine	Asie	Pékin	1412	9597
7	Inde	Asie	New Delhi	1417	3287
8	Brésil	Amérique	Brasilia	215	8516
9	Canada	Amérique	Ottawa	39	9985
10	Mexique	Amérique	Mexico	128	1964
11	Égypte	Afrique	Le Caire	109	1002
12	Nigéria	Afrique	Abuja	219	924
13	Australie	Océanie	Canberra	26	7692
14	Maroc	Afrique	Rabat	37	447

Base `bibliotheque` : schéma donné au début (relations `auteur`, `livre`, `adherent`, `emprunt`). *Charger* `bibliotheque.sql` *pour la découvrir.*

Interroger : `SELECT` et `WHERE` ↑cours

★ **Exercice 4** *Sélections simples* ▶ corrigé p. 863 • essentiel [base pays — Basthon]

Écrire une requête qui affiche :

1. le `nom` de tous les pays d'`Europe` ;
2. le `nom` et la `capitale` des pays de plus de `100` millions d'habitants ;
3. le `nom` des pays d'`Asie` dont la superficie dépasse `1000`.

▷ coup de pouce

★★ **Exercice 5** *Conditions composées* ▶ corrigé p. 863 • essentiel [base pays — Basthon]

Écrire une requête qui affiche :

1. le `nom` des pays d'`Afrique` **ou** d'`Océanie` ;
2. la `capitale` des pays dont la population est comprise entre `50` et `150` millions ;
3. le `nom` des pays qui ne sont **pas** en `Europe`.

▷ coup de pouce

Trier, dédoublonner, rechercher un motif ↑ cours

★ **Exercice 6** *Trier et dédoublonner* ▶ corrigé p. 863 • essentiel [base pays — Basthon]

Écrire une requête qui affiche :

1. le `nom` de tous les pays, triés par population **décroissante** ;
2. la liste des `continents`, **sans doublon**.

▷ coup de pouce

★★ **Exercice 7** `LIMIT` *et* `LIKE` *[au-delà du programme]* ▶ corrigé p. 863 [base pays — Basthon]

Écrire une requête qui affiche :

1. le `nom` et la `superficie` des **3** pays les plus vastes ;
2. le `nom` des pays commençant par la lettre `A` ;
3. la `capitale` des pays dont le nom de capitale contient un **espace**.

▷ coup de pouce

Agréger ↑ cours

★ **Exercice 8** *Fonctions d'agrégation* ▶ corrigé p. 864 • essentiel [base pays — Basthon]

Écrire une requête qui affiche :

1. le **nombre** de pays de la base ;
2. la population **moyenne** ;
3. la plus grande `superficie`.

▷ coup de pouce

★★ **Exercice 9** *Agrégats et filtre* ▶ corrigé p. 864 [base pays — Basthon]

Écrire une requête qui affiche :

1. la population **totale** des pays d'`Asie` ;
2. la plus petite **et** la plus grande population, en **une** requête.

▷ coup de pouce

★ **Exercice 10** *Agréger la base musique* ▶ corrigé p. 864 [base `musique` — Basthon]

Écrire une requête qui affiche :

1. le nombre de chansons sorties en `2013` ;
2. la durée **moyenne** des chansons ;
3. le total des `streams` de la base.

▷ coup de pouce

Regrouper : `GROUP BY` [au-delà du programme] ↑cours

★★★ **Exercice 11** *Regrouper par continent* [au-delà du programme] ▶ corrigé p. 864 ≈ 15 min [base `pays` — Basthon]

Écrire une requête qui affiche :

1. le **nombre de pays** par continent ;
2. la population **totale** par continent, triée de la plus grande à la plus petite ;
3. les continents comptant **au moins 4** pays.

▷ coup de pouce ▷ début de solution

Croiser des tables : les jointures ↑cours

★★ **Exercice 12** *Jointure de deux tables* ▶ corrigé p. 864 • essentiel [base `bibliotheque` — Basthon]

Écrire une requête qui affiche :

1. le `titre` de chaque livre **avec le nom** de son auteur ;
2. les `titre`s des livres écrits par `Camus` ;
3. les `titre`s des livres dont l'auteur est du `Royaume-Uni`.

▷ coup de pouce

★★★ **Exercice 13** *Jointure de trois tables* ▶ corrigé p. 865 ≈ 15 min [base `bibliotheque` — Basthon]

Écrire une requête qui affiche :

1. pour chaque emprunt, le `nom` et le `prenom` de l'adhérent **avec le titre** du livre emprunté ;
2. [au-delà du programme] le `nom` de chaque adhérent **ayant emprunté au moins un livre**, avec son nombre d'emprunts (avec `GROUP BY`).

▷ coup de pouce ▷ début de solution

★★★ **Exercice 14** *Jointure et agrégat (musique)* [au-delà du programme] ▶ corrigé p. 865 ≈ 10 min [base `musique` — Basthon]

Afficher le `nom` de chaque artiste **belge** avec son **total de streams**.

▷ coup de pouce ▷ début de solution

Modifier la base ↑cours

★★ **Exercice 15** *Ajouter, modifier, supprimer* ▶ corrigé p. 865 • essentiel [base `bibliotheque` — Basthon]

1. ajouter l'auteur de nom `'Hugo'` (`id` 6, pays `'France'`) ;
2. remplacer le `genre` du livre d'`id 1` par `'Dystopie'` ;
3. supprimer l'emprunt d'`id 6`. *(Que se passerait-il sans le* `WHERE` *?)*

▷ coup de pouce

★★ **Exercice 16** *Le virement bancaire : transactions et ACID* ▶ corrigé p. 865 • essentiel

Une banque stocke ses comptes dans la relation `compte(id, titulaire, solde)`, avec la contrainte `CHECK (solde >= 0)` : un solde ne peut pas être négatif. Au départ, le compte `1` (Léa) contient `100` € et le compte `2` (Noé) `30` €.

Un SGBD garantit qu'une **transaction** (un groupe d'opérations, ouvert par `BEGIN TRANSACTION;` et validé par `COMMIT;`) respecte quatre propriétés, résumées par l'acronyme **ACID** :

- **Atomicité** : toutes les opérations de la transaction sont faites, ou **aucune** (en cas d'échec, `ROLLBACK` annule tout) ;
- **Cohérence** : la transaction fait passer la base d'un état qui respecte les contraintes à un autre état qui les respecte ;
- **Isolation** : des transactions exécutées en même temps ne se perturbent pas, comme si elles avaient lieu l'une après l'autre ;

- **Durabilité** : une fois validée, une transaction n'est plus perdue, même en cas de panne.

1. Écrire les **deux** requêtes `UPDATE` qui réalisent un virement de `50` € du compte `1` vers le compte `2`.
2. Sans transaction, le serveur tombe en panne juste après la première requête. Quel est alors l'état de la base ? Quelle propriété aurait évité ce problème ?
3. On recommence en entourant les deux requêtes de `BEGIN TRANSACTION;` et `COMMIT;`. Que contient la base après le `COMMIT` ? Juste après, une coupure de courant survient : quelle propriété garantit que le virement n'est pas perdu ?
4. On tente maintenant de virer `80` € du compte `1` vers le compte `2`, dans une transaction. Laquelle des deux requêtes échoue, et pourquoi ? Que devient l'autre requête ? Quelle propriété est en jeu ?
5. Le compte `1` contient maintenant `50` €. Léa et son père, chacun depuis son téléphone, retirent **au même moment** `20` € de ce compte. Chaque application **lit** le solde, calcule le nouveau solde, puis l'**écrit**. Quel solde final attend-on ? Montrer qu'avec l'ordre « lecture de Léa, lecture du père, écriture de Léa, écriture du père », le compte finit à `30` €. Qu'est-ce que la banque a perdu ? Quelle propriété le SGBD doit-il assurer ?

▷ coup de pouce

Vers le bac ↑ cours

★★ **Exercice 17** *Location entre particuliers (d'après Polynésie 2023, jour 2)* ▶ corrigé p. 866 ≈ 30 min

Un site de location d'objets entre particuliers utilise le schéma relationnel suivant. La table `Possede` relie un membre aux objets qu'il propose ; sa clé primaire est le **couple** (`id_membre`, `id_objet`). `Membre (id_membre, nom, prenom, cp)`

`Objet (id_objet, type, tarif)`
`Reservation (id_reservation, #id_objet, #id_membre, date_location, date_retour)`
`Possede (#id_membre, #id_objet)`

1. Écrire une requête affichant le `nom` et le `prenom` des membres dont le code postal `cp` est `'69003'`.
2. Écrire une requête qui **modifie** le `tarif` de l'objet d'`id_objet 1` pour le passer à `15`.
3. Écrire une requête qui **ajoute** le membre Wendie Renard (`id_membre 6`, `cp '69100'`).
4. On exécute `DELETE FROM Membre WHERE nom = 'Ali' AND prenom = 'Mohamed';`. Ce membre a déjà effectué des réservations et propose des objets. Expliquer pourquoi cette requête **produit une erreur**.

▷ coup de pouce

5. Mohamed Ali a pour `id_membre 1`. Proposer la **suite de requêtes** permettant de le supprimer **correctement** de la base.

▷ coup de pouce

6. *(jointure)* Écrire une requête comptant le **nombre de réservations** réalisées par le membre Fernando Alonso.
7. *(jointure)* Écrire une requête donnant les `nom` et `prenom` des membres qui possèdent un objet de type `'appareil à raclette'`.

▷ coup de pouce

★★ Exercice 18 *Location de véhicules (d'après La Réunion 2023, jour 2)*

▶ corrigé p. 866 ≈ 30 min

Schéma de la base : `vehicule(id_vehicule, immatriculation, marque, modele, type, carburant)`
`utilisateur(id_utilisateur, nom, prenom, permis, adresse, ville)`
`location(id_location, #id_utilisateur, #id_vehicule, date_debut, date_fin)`

Extrait de la relation `vehicule` :

id_vehicule	immatriculation	marque	modele	type	carburant
1	AB-135-YZ	Peugeot	208	citadine	diesel
2	EC-246-TP	Renault	Zoe	citadine	electrique
3	LC-231-MG	Tesla	Model X	SUV	electrique
4	ML-128-VM	Citroën	C3	citadine	essence
5	CL-142-CE	Citroën	C5	berline	diesel
6	JL-526-LM	Peugeot	508	berline	diesel

1. Donner le **résultat** de :

```sql
SELECT id_vehicule
FROM vehicule
WHERE type = 'citadine';
```

2. Écrire une requête affichant les `immatriculation` des véhicules **diesel**.

3. Écrire une requête calculant le **nombre** de véhicules de la table.

4. Citer, **en justifiant**, les clés étrangères de la relation `location`.

5. Expliquer pourquoi la requête suivante **produit une erreur** (l'`id_location 1` existe déjà) :

▷ coup de pouce

```sql
INSERT INTO location
VALUES (1, 132, 4, '2022-05-10', '2022-05-12');
```

6. Louise `DUBOIS` (`id_utilisateur 133`) habite au numéro `50`, et non `52`, de la rue de la Liberté. Écrire la requête qui **corrige** son adresse, en la remplaçant par `'50 rue de la Liberté'`.

7. *(jointure)* Écrire une requête listant le `modele` et l'`immatriculation` des véhicules dont la location a débuté le `'2022-06-21'`, ainsi que le `nom` et le `prenom` des utilisateurs correspondants.

▷ coup de pouce

★★ Exercice 19 *Podcasts de radio (d'après Centres étrangers 2023, jour 1)*

▶ corrigé p. 867 ≈ 30 min

Extraits de `podcast` (clé primaire `id_podcast`, `#id_emission` clé étrangère vers `emission`) :

id_podcast	theme	annee	id_emission
1	L'enseignement supérieur est-il juste ?	2022	10081
3	Travailleurs de plateformes	2021	10175
4	Souveraineté numérique française	2019	10183
5	Dans le cloud en Islande	2019	10212

La relation `emission(id_emission, nom, radio, animateur)` décrit les émissions ; la relation `description` contient un `id_description`, un `resume`, une `duree` (en minutes) et l'`id_podcast` du podcast décrit.

1. Écrire le **schéma relationnel** de la relation `description` (attributs et types probables, clé primaire soulignée, clé étrangère).

▷ coup de pouce

2. Écrire ce qu'**affiche** :

```sql
SELECT theme, annee
FROM podcast
WHERE id_emission = 10081;
```

3. Écrire une requête affichant les `theme` des podcasts de l'année `2019`.

4. Décrire ce que renvoie `SELECT DISTINCT theme FROM podcast;`.

5. Le nom de l'animateur de l'émission « Le Temps du débat » doit devenir « Emmanuel L. ». Écrire la requête de **mise à jour**.

6. Écrire une requête **ajoutant** l'émission « Hashtag » (radio « France Inter », animateur « Mathieu V. », `id_emission` `12850`).

7. *(jointure)* Écrire une requête listant le `theme`, le `nom` de l'émission et le `resume` des podcasts dont la `duree` est strictement inférieure à `5` minutes. ▷ coup de pouce

★★ **Exercice 20** *Vols spatiaux (d'après Métropole 2023, jour 2)* ▶ corrigé p. 867 ≈ 30 min

La base comporte les relations suivantes ; extrait de `Fusee` ci-dessous.

`Astronaute (id_astronaute, nom, prenom, nationalite, nb_vols)`
`Fusee (id_fusee, modele, constructeur, nb_places)`
`Vol (id_vol, #id_fusee, date)`
`Equipe (#id_vol, #id_astronaute)`

id_fusee	modele	constructeur	nb_places
1	Falcon 9	SpaceX	6
2	Starship	SpaceX	100
3	Soyouz	TsSKB Progress	2
4	SLS	Boeing	6

1. Donner la **définition** d'une clé primaire.

2. On tente d'insérer un astronaute avec `id_astronaute = 1` (déjà attribué à PESQUET). Expliquer pourquoi le SGBD **renvoie une erreur**.

3. Écrire le **schéma relationnel** de la table `Fusee` en précisant le **domaine** de chaque attribut.

4. Écrire le **résultat** de `SELECT COUNT(*) FROM Fusee WHERE constructeur = 'SpaceX';`.

5. Écrire une requête affichant le `modele` et le `constructeur` des fusées ayant **au moins 4 places**.

6. Écrire une requête affichant les `nom` et `prenom` des astronautes dans l'**ordre alphabétique** du nom.

7. *(jointure)* Écrire une requête donnant le `nom` et le `prenom` des astronautes ayant décollé le `'25/10/2022'`.

▷ coup de pouce

★★ **Exercice 21** *Ludothèque (d'après Métropole 2025, jour 2)* ▶ corrigé p. 868 ≈ 30 min

Une ludothèque (un seul exemplaire par jeu) utilise le schéma (`#` = clé étrangère). Quand un jeu est emprunté et pas encore rendu, `dateRendu` vaut `NULL`. `jeu(nomJeu, editeur, anneeSortie, ageMinimum, categorie)`
`adherent(idAdherent, nom, prenom, dateNaissance, adresse)`
`emprunt(idEmprunt, #nomJeu, #idAdherent, dateEmprunt, dateRendu)`

1. Expliquer pourquoi on ne peut pas prendre l'attribut `nom` comme clé primaire de `adherent`.
2. Décrire ce que renvoie `SELECT nomJeu, editeur FROM jeu ORDER BY nomJeu;`.
3. Écrire une requête affichant le `nomJeu` et la `categorie` des jeux sortis **à partir de 2010 et** dont l'`ageMinimum` est strictement inférieur à `10`.
4. Écrire une requête affichant le `nomJeu` des jeux **en cours d'emprunt** (non encore rendus).
   ▷ coup de pouce
5. *(jointure)* Écrire une requête affichant le `nom` et le `prenom` des adhérents ayant emprunté le jeu « Catan ».
6. Le jeu « Catan » (emprunt d'`idEmprunt 1538`) a été rendu le `'2025-06-03'`. Écrire la requête de **mise à jour** correspondante.
7. *(modélisation)* La ludothèque ajoute une relation `evenement(nom, dateEvenement, heure)` et une relation `participation` reliant les adhérents aux événements auxquels ils participent. Proposer les **clés étrangères** de `participation` en précisant les attributs référencés.
   ▷ coup de pouce

Critiquer une réponse d'IA ↑ cours

★★ **Exercice 22** *Une requête proposée par un assistant d'IA* ▶ corrigé p. 868 [base bibliotheque — Basthon]

Sur la base `bibliotheque` (schéma de la section 1), un élève demande à un assistant d'IA : « Écris la requête SQL qui compte le nombre d'emprunts effectués par l'adhérente dont le nom est `'Durand'`. » Voici la réponse obtenue :

```sql
SELECT COUNT(*)
FROM emprunt
JOIN adherent
WHERE adherent.nom = 'Durand';
```

« On joint la table des emprunts à celle des adhérents, on ne garde que les lignes de l'adhérente Durand, et `COUNT(*)` compte les lignes restantes. »

1. La réponse est-elle correcte ? La vérifier sur une mini-base : `adherent` contient `(1, 'Durand', 'Léa')`, `(2, 'Martin', 'Noé')`, `(3, 'Petit', 'Zoé')` ; `emprunt` contient cinq lignes, dont deux d'`id_adherent 1`, deux d'`id_adherent 2` et une d'`id_adherent 3`. Quel résultat attend-on ? Quel résultat renvoie la requête ? (On pourra créer ces deux tables dans une console SQL **vierge**, sans charger `bibliotheque.sql`, et tester.)
2. Localiser et corriger l'erreur.
   ▷ coup de pouce
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

S'entraîner à l'oral ↑cours

★★ **Exercice 23** *Expliquer en deux minutes* ▶ corrigé p. 868

Au Grand Oral comme devant l'examinateur de l'épreuve pratique, il faut savoir **expliquer** une notion clairement, sans lire ses notes. Choisir l'un des sujets suivants (ou le tirer au sort) :

Sujet 1. Expliquer à un camarade pourquoi une médiathèque gère ses emprunts avec une **base de données relationnelle** plutôt qu'avec un tableur.
Sujet 2. Expliquer le rôle d'une **clé primaire** et celui d'une **clé étrangère**.
Sujet 3. Expliquer ce que fait une **jointure**, sur l'exemple de deux tables.

Préparation (5 min, seul). Noter au brouillon **trois idées** et **un exemple**, rien de plus : comme au Grand Oral, ces notes servent d'aide-mémoire. **Exposé (2 min, chronométré).** Debout, face à un camarade, **sans lire ses notes** et sans rien écrire : tout passe par la parole. **Retour (2 min).** Le camarade pose une question de relance (on peut alors s'aider d'un schéma), remplit la grille ci-dessous et donne un conseil ; puis on échange les rôles. Cette grille reprend les critères d'évaluation du Grand Oral. ▷ coup de pouce

Grille entre camarades	Oui	En partie	Pas encore
Clair — audible, posé ; chaque mot technique est expliqué	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	☐	☐	☐
Construit — un fil conducteur, tenu en deux minutes, sans lire	☐	☐	☐
Un conseil pour la prochaine fois :			

Concevoir un bon schéma ↑cours

★★ **Exercice 24** *Un schéma mal conçu : anomalies, structure et contenu* ▶ corrigé p. 869

Le club de lecture d'un lycée enregistre ses emprunts dans une **seule** relation : `emprunt_tout(id_emprunt, titre, auteur, pays_auteur, nom_adherent, ville_adherent, date)`

id_emprunt	titre	auteur	pays_auteur	nom_adherent	ville_adherent	date
1	L'Étranger	Camus	France	Durand	Monaco	2026-09-02
2	La Peste	Camus	France	Martin	Menton	2026-09-05
3	L'Étranger	Camus	France	Martin	Menton	2026-09-20
4	1984	Orwell	Royaume-Uni	Petit	Nice	2026-10-01

1. Quelles informations sont **recopiées** plusieurs fois dans cette table ? Comment appelle-t-on ce défaut ?

2. Martin déménage à Nice. Quelles lignes faut-il modifier ? Que se passe-t-il si l'on en oublie une ? Nommer cette anomalie.

3. Le club achète un livre que personne n'a encore emprunté. Peut-on l'enregistrer dans cette table ? Et un nouvel adhérent qui n'a encore rien emprunté ? Nommer cette anomalie.

4. On supprime l'emprunt `4`, rendu depuis longtemps. Quelles informations disparaissent en même temps ? Nommer cette anomalie.

5. Proposer un découpage en plusieurs relations (schéma relationnel, clés primaires soulignées, clés étrangères précédées de `#`) qui fait disparaître ces anomalies. ▷ coup de pouce

6. **Structure ou contenu ?** Pour chaque opération sur la base découpée, dire si elle modifie la **structure** de la base ou son **contenu** : *(a)* un adhérent emprunte un livre ; *(b)* on ajoute un attribut `email` aux adhérents ; *(c)* on corrige le titre d'un livre ; *(d)* on crée une relation `editeur` ; *(e)* on supprime tous les emprunts de 2025.

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Chaque ligne du schéma est une relation ; ses attributs sont les noms entre parenthèses. La clé primaire est soulignée, une clé étrangère est précédée de `#`. Pour un domaine, se demander quel genre de valeur l'attribut contient : un nombre entier, un texte, une date ? ↑énoncé

Ex. 2 – piste Question 1 : deux livres différents peuvent-ils porter le même titre ? Question 2 : l'attribut `id_livre` d'un emprunt doit correspondre à une ligne existante de quelle relation ? ↑énoncé

Ex. 3 – piste Les informations d'un éditeur ne doivent figurer qu'**une** fois dans la base. Comment le modèle relationnel relie-t-il deux relations ? ↑énoncé

Ex. 4 – piste Chaque requête suit le même schéma : `SELECT` (les colonnes voulues), `FROM` (la table), `WHERE` (la condition que doivent vérifier les lignes). Les chaînes de caractères s'écrivent entre apostrophes ; deux conditions exigées ensemble se relient par `AND`. ↑énoncé

Ex. 5 – piste Question 2 : « compris entre » cache **deux** comparaisons ; quel opérateur logique les relie ? Question 3 : le cours donne un opérateur de différence, et aussi `NOT`. ↑énoncé

Ex. 6 – piste Question 1 : `ORDER BY` trie par ordre croissant si l'on ne précise rien ; quel mot-clé ajouter pour l'ordre décroissant ? Question 2 : quel mot-clé, placé juste après `SELECT`, supprime les doublons ? ↑énoncé

Ex. 7 – piste Question 1 : trier d'abord, puis ne garder que les premières lignes. Questions 2 et 3 : dans un motif `LIKE`, le symbole `%` remplace n'importe quelle suite de caractères ; où le placer ? ↑énoncé

Ex. 8 – piste Une fonction d'agrégation (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) se place dans le `SELECT` et renvoie une seule valeur pour toute la table. `COUNT(*)` compte les lignes. ↑énoncé

Ex. 9 – piste Le `WHERE` s'applique **avant** l'agrégat : on filtre les lignes, puis on calcule. Un même `SELECT` peut contenir plusieurs agrégats séparés par des virgules. ↑énoncé

Ex. 10 – piste Mêmes outils qu'à l'exercice 8, sur la table `chanson`. Question 1 : le `WHERE` garde d'abord les chansons de 2013, puis l'on compte les lignes restantes. ↑énoncé

Ex. 11 – piste `GROUP BY continent` forme un groupe par continent : les agrégats du `SELECT` se calculent alors **groupe par groupe**. Question 3 : on filtre des **groupes**, pas des lignes. ↑énoncé

Ex. 11 – début de solution Question 1 :

```
SELECT continent, COUNT(*)
FROM pays
GROUP BY continent;
```

Les questions 2 et 3 partent de la même structure. ↑énoncé

Ex. 12 – piste Quelle clé étrangère de `livre` référence la relation `auteur` ? C'est elle qui donne la condition du `ON`. Préfixer les attributs par le nom de leur table en cas d'ambiguïté. ↑

Ex. 13 – piste `emprunt` est au centre : elle contient une clé étrangère vers chacune des deux autres relations, d'où deux `JOIN ... ON` successifs. Question 2 : il faut regrouper par adhérent. ↑énoncé

Ex. 13 – début de solution

```
SELECT adherent.nom, adherent.prenom, livre.titre
FROM emprunt
JOIN adherent ON emprunt.id_adherent = adherent.id ...
```

↑énoncé

Ex. 14 – piste Trois étapes : joindre `chanson` et `artiste`, ne garder que les artistes belges, regrouper par artiste pour faire la somme. ↑énoncé

Ex. 14 – début de solution
```
SELECT artiste.nom, SUM(chanson.streams)
    FROM chanson
    JOIN artiste ON chanson.id_artiste = artiste.id ...
```
↑énoncé

Ex. 15 – piste Une instruction par question : `INSERT INTO ... VALUES`, `UPDATE ... SET ... WHERE`, `DELETE FROM ... WHERE`. Les chaînes de caractères s'écrivent entre apostrophes. ↑énoncé

Ex. 16 – piste Question 1 : `UPDATE compte SET solde = solde - ... WHERE ...`. Question 5 : faire un tableau à deux colonnes (Léa, père) et noter, ligne après ligne, la valeur lue puis la valeur écrite par chacun. ↑énoncé

Ex. 17 q. 4 – piste Question 4 : ce membre apparaît-il dans d'autres relations ? Que garantit une contrainte de clé étrangère ? ↑énoncé

Ex. 17 q. 5 – piste Question 5 : dans quel ordre supprimer : les lignes qui **référencent** ce membre, ou le membre lui-même ? ↑énoncé

Ex. 17 q. 7 – piste Question 7 : `Possede` fait le lien entre `Membre` et `Objet` : il faut donc deux jointures. ↑énoncé

Ex. 18 q. 5 – piste Question 5 : quel est le rôle de `id_location` dans la relation, et quelle propriété doit-il vérifier ? ↑énoncé

Ex. 18 q. 7 – piste Question 7 : les informations demandées sont réparties dans trois relations : laquelle contient les deux clés étrangères ? ↑énoncé

Ex. 19 q. 1 – piste Question 1 : repérer l'attribut qui identifie chaque description, puis celui qui fait référence à une autre relation. ↑énoncé

Ex. 19 q. 7 – piste Question 7 : `description` est reliée à `podcast` par `id_podcast`, et `podcast` à `emission` par `id_emission` : ce sont ces attributs qui servent dans les conditions de jointure. ↑énoncé

Ex. 20 – piste Question 7 : la date est dans `Vol`, les noms dans `Astronaute` ; quelle relation fait le lien entre les deux ? ↑énoncé

Ex. 21 q. 4 – piste Question 4 : une valeur `NULL` ne se teste pas avec `=` : quel opérateur SQL du cours teste l'absence de valeur ? ↑énoncé

Ex. 21 q. 7 – piste Question 7 : une participation associe **un** adhérent et **un** événement : comment chacun est-il identifié dans sa relation ? ↑énoncé

Ex. 22 q. 2 – piste Sans condition `ON`, avec quelles lignes de `adherent` chaque emprunt est-il associé ? Combien de lignes la jointure produit-elle avant le `WHERE` ? ↑énoncé

Ex. 23 – piste Structurer chaque explication en trois temps : une **définition** en une phrase, un **exemple** petit et concret déroulé à voix haute, puis le **piège** (l'erreur que l'auditeur risque de faire). Finir par une phrase qui répond à la question posée. Garder le même exemple pour tout l'exposé (des livres, des adhérents, des emprunts) et dire pour chaque notion quel problème concret elle évite. ↑énoncé

Ex. 24 q. 5 – piste Chaque information ne doit être écrite qu'**une** fois. Un auteur a un pays, un adhérent a une ville, et un emprunt relie un livre à un adhérent, à une date. ↑énoncé

TP ET PROJETS

TP 1 La ligue de futsal des quartiers

▶ corrigé p. 870

modéliser, créer, interroger une base SQLite, puis la piloter depuis Python

Durée : 2 séances (environ 3 h) · Par deux, sur machine

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t06-tp-bdd-sql` — **archive .zip**

Consignes

- Outils : **DB Browser for SQLite** (onglets « Structure de la base », « Parcourir les données », « Exécuter le SQL ») et **Python** (Thonny, IDLE ou VS Code).
- Fichiers à télécharger (lien ci-dessus) : la base `tp_bdd_sql.db`, le script `tp_bdd_sql_creation.py` qui la fabrique (le relancer **remet la base dans son état de départ**) et le fichier `tp_bdd_sql_depart.py` de la partie 6.
- Écrire chaque requête **sur plusieurs lignes** (`SELECT...` / `FROM...` / `WHERE...`), l'exécuter, puis noter sur le cahier (ou dans un fichier texte) le résultat demandé. ▶ signale ce qui se fait sur machine.
- Dans DB Browser, une modification n'est enregistrée dans le fichier qu'après « **Écrire les modifications** » (Ctrl+S).
- Usage de l'IA, indiqué sur chaque exercice : IA aucune ; IA en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; IA l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

But du TP

Une ligue amateur (fictive) organise un championnat de futsal entre six quartiers de Monaco. On va **modéliser** ses données, **créer** des tables avec leurs contraintes et les **mettre à l'épreuve**, **interroger** la base de la saison (sélections, agrégats, regroupements, jointures), la **mettre à jour** après une journée de matchs, et enfin écrire en Python le programme qui calcule le **classement du championnat** — le produit final du TP.

Partie 1 — Modéliser (sur papier, 20 min)

★ **Exercice 1** IA *Du cahier des charges au schéma*

Le secrétaire de la ligue décrit ses besoins : « Chaque **équipe** a un nom (deux équipes n'ont jamais le même), un quartier et une année de création. Chaque **joueur** a un nom, un prénom, une année de naissance, un poste (gardien, défenseur, milieu ou attaquant) et joue dans **une seule** équipe. Une **rencontre** a lieu lors d'une journée, à une date, entre une équipe qui reçoit et une équipe qui se déplace ; on note le score une fois le match joué. Enfin, on veut savoir **qui a marqué chaque but**, et à quelle minute. »

1. Quelles relations (tables) proposez-vous ? Pour chacune, donner ses attributs, sa clé primaire et ses clés étrangères, dans la notation du cours.
2. Pourquoi ne pas écrire le **nom** de l'équipe dans chaque ligne de la table des joueurs ?

3. La relation des rencontres fait référence **deux fois** à la même relation. Laquelle, et pourquoi ?
4. Avant que le match soit joué, que met-on dans les cases du score ?

Le schéma retenu par la ligue

```
equipe(id, nom, quartier, annee_creation)
joueur(id, nom, prenom, annee_naissance, poste, #id_equipe)
rencontre(id, journee, date_match, #id_domicile, #id_exterieur, buts_dom,
buts_ext)
but(id, #id_rencontre, #id_joueur, minute)
```

`id_domicile` et `id_exterieur` référencent `equipe.id` ; un score non encore connu vaut `NULL`. Les dates sont du texte `'AAAA-MM-JJ'`. Comparer avec votre proposition avant de continuer.

Partie 2 — Créer des tables avec leurs contraintes

★ **Exercice 2** *Deux tables dans une base vierge*

► Dans DB Browser, créer une **nouvelle** base `essai.db` (« Nouvelle base de données »), puis, dans l'onglet « Exécuter le SQL », écrire et exécuter les deux `CREATE TABLE` des relations `equipe` et `joueur`, en traduisant **toutes** les règles suivantes :

- chaque `id` est la clé primaire (`INTEGER PRIMARY KEY`) ;
- le nom d'une équipe est **obligatoire** et **unique** ; son quartier est obligatoire ;
- le nom et le prénom d'un joueur sont obligatoires ; son année de naissance est comprise entre 1950 et 2015 ; son poste est l'un des quatre mots `'gardien'`, `'defenseur'`, `'milieu'`, `'attaquant'` ;
- `id_equipe` est obligatoire et **référence** `equipe(id)`.

▷ coup de pouce Insérer ensuite deux équipes et deux joueurs de votre choix, et vérifier dans « Parcourir les données » qu'ils sont bien là.

★★ **Exercice 3** *La table des rencontres*

► Toujours dans `essai.db`, écrire le `CREATE TABLE rencontre`. Les deux équipes sont obligatoires et référencent `equipe(id)` ; les deux scores sont positifs ou nuls ; et une équipe ne peut pas jouer **contre elle-même**. ▷ coup de pouce

★ **Exercice 4** *Comparer avec la vraie base*

► Ouvrir `tp_bdd_sql.db`. Dans l'onglet « Structure de la base » (ou avec la requête `SELECT sql FROM sqlite_master;`), lire les `CREATE TABLE` de la ligue et les comparer aux vôtres. Noter sur le cahier les différences éventuelles.

Partie 3 — Mettre les contraintes à l'épreuve

★★ **Exercice 5** *Le SGBD monte la garde*

▶ Sur `tp_bdd_sql.db`, exécuter les instructions suivantes **une par une**. Pour chacune, noter sur le cahier si elle est **acceptée** ou **refusée**, et quelle contrainte du cours est en jeu (domaine, entité, référence, `NOT NULL`, `UNIQUE`, contrainte métier).

	Instruction
a	`INSERT INTO equipe VALUES (1, 'Les Pingouins', 'Fontvieille', 2020);`
b	`INSERT INTO equipe VALUES (7, 'Condamine FC', 'La Condamine', 2020);`
c	`INSERT INTO equipe (id, nom, annee_creation) VALUES (7, 'Saint-Roman FC', 2020);`
d	`INSERT INTO joueur VALUES (37, 'Durand', 'Léa', 2007, 'attaquant', 9);`
e	`INSERT INTO joueur VALUES (37, 'Durand', 'Léa', 2007, 'avant-centre', 2);`
f	`INSERT INTO rencontre VALUES (16, 6, '2026-10-10', 3, 3, NULL, NULL);`
g	`DELETE FROM equipe WHERE id = 6;`
h	`INSERT INTO equipe VALUES (8, 'Saint-Roman FC', 'Saint-Roman', 'hier');`

1. Pourquoi l'instruction g est-elle refusée, alors qu'elle ne fait que **supprimer** ?
2. L'instruction h est acceptée ! Que montre-t-elle sur la façon dont SQLite applique la contrainte de **domaine** ? Proposer une contrainte `CHECK` qui l'aurait empêchée.
3. ▶ Écrire la requête qui supprime l'équipe insérée en h (et seulement elle).

Partie 4 — Interroger la base de la saison

Pour chaque requête : l'écrire sur plusieurs lignes, l'exécuter, puis noter le nombre de lignes obtenues et la première ligne du résultat. *Après quatre journées, la cinquième n'a pas encore été jouée.*

★ **Exercice 6** *Sélectionner, filtrer, trier*

Q1. Le nom et le quartier des équipes, de la plus ancienne à la plus récente.
Q2. Le prénom, le nom et l'année de naissance des joueurs nés en 2008 ou après, du plus jeune au plus âgé.
Q3. Les gardiens nés avant 2000 (nom, prénom, numéro de leur équipe).
Q4. Les différents postes, chacun une seule fois.
Q5. Les joueurs dont le nom se termine par la lettre `i` (avec `LIKE`). [au-delà du programme]
Q6. Les rencontres (numéro, journée, date) dont le score n'est pas encore connu.

★★ **Exercice 7** *Agréger et regrouper*

Q7. En une seule requête : le nombre de joueurs, l'année de naissance du plus âgé et celle du plus jeune.
Q8. Le nombre total de buts marqués pendant la saison, puis la moyenne de buts par match joué. Vérifier le total en comptant les lignes de la table `but`.
Q9. Le nombre de joueurs pour chaque poste (`GROUP BY`). [au-delà du programme]
Q10. Le nombre de buts marqués lors de chaque journée (`GROUP BY`). [au-delà du programme]

★★ **Exercice 8** *Croiser les tables*

Q11. Le prénom, le nom et le poste des joueurs du `'Rocher Club'`, triés par nom.

Q12. **Le classement des buteurs** : les cinq meilleurs buteurs avec leur prénom, leur nom, le nom de leur équipe et leur nombre de buts (en cas d'égalité, par ordre alphabétique des noms). [au-delà du programme] ▷ coup de pouce

Q13. Les équipes dont les joueurs ont marqué **au moins** 10 buts, avec ce nombre (`GROUP BY ... HAVING`). [au-delà du programme]

★★★ **Exercice 9** *Les résultats d'une journée*

Q14. Afficher les résultats de la journée 3 sous la forme : *nom de l'équipe qui reçoit, buts_dom, buts_ext, nom de l'équipe qui se déplace.* ▷ coup de pouce

Q15. **Contrôle de cohérence.** Dans une base saine, pour chaque rencontre jouée, le nombre de lignes de la table `but` est égal à `buts_dom + buts_ext`. Écrire une requête qui affiche les rencontres pour lesquelles ce n'est **pas** le cas. Que doit-elle renvoyer ici ? [au-delà du programme]

Partie 5 — Mettre la base à jour

★★ **Exercice 10** *Le week-end de la journée 5*

▶ Écrire et exécuter les requêtes qui traduisent les nouvelles suivantes. **Relire chaque `WHERE` avant d'exécuter.**

1. La rencontre 14 (Larvotto Sporting – Condamine FC) s'est terminée sur le score de 1 à 1 : buts de Gianni Esposito (joueur 36) à la 12e minute et de Diego Carvalho (joueur 11) à la 29e. Mettre à jour le score, puis insérer les deux buts en **une seule** instruction `INSERT`. ▷ coup de pouce
2. La rencontre 15 est reportée au 11 octobre 2026.
3. Le Rocher Club a été créé en 1993, et non en 1995.
4. Yanis Moreau (joueur 6) quitte la ligue. Tenter de le supprimer de la table `joueur`. Que se passe-t-il, et pourquoi ? Combien de lignes de `but` le concernent ?
5. Raphaël Petit quitte lui aussi la ligue : le supprimer (en le désignant par son nom **et** son prénom). Pourquoi cette fois la suppression réussit-elle ?
6. Que ferait l'instruction `DELETE FROM joueur;` ? *(Ne pas l'exécuter !)*

Relancer la requête de contrôle de cohérence : elle ne doit toujours rien renvoyer. Puis **écrire les modifications** (Ctrl+S) et **fermer** la base dans DB Browser avant de passer à Python (un fichier ouvert en écriture par un logiciel peut être *verrouillé* pour les autres).

Partie 6 — Piloter la base depuis Python

Le module `sqlite3` (fourni avec Python) permet d'envoyer des requêtes SQL à une base et de récupérer les résultats sous forme de **liste de tuples**. Ouvrir `tp_bdd_sql_depart.py` (dans le même dossier que la base). On y trouve :

```
connexion = sqlite3.connect("tp_bdd_sql.db")
connexion.execute("PRAGMA foreign_keys = ON")   # sinon : cles etrangeres ignorees !

def equipes_creees_avant(annee):
    requete = """SELECT nom, quartier
                 FROM equipe
                 WHERE annee_creation < ?
                 ORDER BY annee_creation"""
    return connexion.execute(requete, (annee,)).fetchall()
```

Le point d'interrogation `?` est un **paramètre** : `sqlite3` le remplace par la valeur du tuple `(annee,)`. `fetchall()` renvoie toutes les lignes, `fetchone()` la première seulement.

★ **Exercice 11** *Premières requêtes depuis Python*

▶ Dans la console, que renvoie `equipes_creees_avant(2005)` ?

▶ Compléter `joueurs_de(nom_equipe)` (liste des triplets `(prenom, nom, poste)` des joueurs de l'équipe, triés par nom) et `nb_buts(nom, prenom)` (un **entier**). Lancer le fichier : les tests `test_joueurs_de` et `test_nb_buts` doivent afficher `OK`. ▷ coup de pouce

★★ **Exercice 12** *Pourquoi le point d'interrogation ?*

Un camarade construit plutôt la requête en y collant directement le nom : `requete = f"... WHERE equipe.nom = '{nom_equipe}'"`.

1. ▶ Essayer sa version avec `joueurs_de("x' OR '1'='1")`. Combien de joueurs obtient-on ? Écrire la condition `WHERE` réellement envoyée au SGBD.
2. Ce piège s'appelle une **injection SQL**. Pourquoi est-il dangereux sur un site web où l'utilisateur tape lui-même le texte ? Que fait le `?` de différent ?

★★ **Exercice 13** *Le classement du championnat*

▶ Compléter `classement()` : écrire la requête qui sélectionne `id_domicile`, `id_exterieur`, `buts_dom`, `buts_ext` des rencontres **déjà jouées**, puis, pour chacune, mettre à jour le dictionnaire des deux équipes (`J`, `G`, `N`, `P`, buts pour `bp`, buts contre `bc`, points `pts` : victoire 3, nul 1, défaite 0). Le tri final est fourni. Décommenter `afficher_classement()` et recopier le podium sur le cahier (rang, équipe, points, différence de buts). ▷ coup de pouce Pourquoi les tests vérifient-ils que la somme des buts pour est égale à la somme des buts contre ?

★★★ **Exercice 14** *Saisir un résultat en « tout ou rien »*

On veut saisir les deux autres matchs de la journée 5 **depuis Python**, sans risquer une base incohérente (des buts enregistrés mais un score faux, ou l'inverse).

▶ Compléter `saisir_resultat(id_rencontre, buts)`, où `buts` est une liste de couples `(id_joueur, minute)` : à l'intérieur du bloc `with connexion:`, insérer chaque but, **compter** les buts de chaque équipe (le joueur doit appartenir à l'une des deux équipes de la rencontre, sinon lever une erreur avec `raise ValueError(...)`), puis mettre à jour le score. Le bloc `with connexion:` est une **transaction** : si une erreur survient, **tout** ce qui a été fait dans le bloc est annulé.

1. Tester d'abord ces deux appels (le joueur 1 joue pour Fontvieille, le joueur 99 n'existe pas) :
   ```
   saisir_resultat(15, [(30, 14), (1, 20)])
   saisir_resultat(15, [(30, 14), (99, 20)])
   ```
   Dans chaque cas, quelle erreur obtient-on ? Combien de buts de la rencontre 15 sont enregistrés ensuite ? Quelle propriété des transactions est illustrée ?
2. Saisir pour de bon : rencontre 13 avec `[(6, 5), (23, 11), (5, 18), (24, 30), (23, 38)]` et rencontre 15 avec `[(30, 14), (29, 33)]`. Donner les scores obtenus, puis le **classement final** de la saison et le meilleur buteur.

Pour aller plus loin

★★★ **Exercice 15** *Le classement en une seule requête SQL* *[au-delà du programme]*

Obtenir les points et la différence de buts de chaque équipe **sans Python**. Idée : fabriquer une table intermédiaire qui contient **deux lignes par rencontre jouée** (une par équipe, avec ses buts « pour » et « contre ») grâce à `UNION ALL`, puis regrouper par équipe. Pour les points, SQL possède une expression conditionnelle : `CASE WHEN pour > contre THEN 3 WHEN pour = contre THEN 1 ELSE 0 END`. Comparer avec le classement obtenu en Python.

Bilan à rédiger

En une dizaine de lignes, sur le cahier ou dans un fichier :

- citer **trois** erreurs de saisie que les contraintes de la base ont empêchées pendant le TP, et la contrainte responsable de chacune ;
- expliquer la différence entre `WHERE` et `HAVING` à l'aide d'une requête du TP ;
- expliquer pourquoi on passe les valeurs avec `?` plutôt qu'en les collant dans le texte de la requête ;
- recopier le classement final et dire quel « service » du SGBD a garanti qu'aucun match n'a été à moitié saisi.

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 2 – piste Mots-clés utiles : `NOT NULL`, `UNIQUE`, `CHECK (...)`, `REFERENCES equipe(id)`. Dans un `CHECK`, on peut écrire `x BETWEEN a AND b` et `x IN ('...', '...')`. ↑énoncé

Ex. 3 – piste La dernière règle porte sur **deux** attributs : on l'écrit après la liste des attributs, sous la forme `CHECK (... <> ...)`. ↑énoncé

Ex. 8 q. 12 – piste Trois tables : `but`, `joueur`, `equipe`. On regroupe par joueur (`GROUP BY joueur.id`), on trie par nombre de buts décroissant, puis `LIMIT 5`. ↑énoncé

Ex. 9 q. 14 – piste Il faut joindre la table `equipe` **deux fois** : une fois pour l'équipe qui reçoit, une fois pour celle qui se déplace. On la renomme à chaque fois avec un alias : `JOIN equipe AS d ON ...`, `JOIN equipe AS e ON ...`. ↑énoncé

Ex. 10 q. 1 – piste On peut écrire plusieurs n-uplets après `VALUES`, séparés par des virgules : `VALUES (...), (...)`. ↑énoncé

Ex. 11 – piste Pour `nb_buts`, la requête `SELECT COUNT(*) ...` renvoie une seule ligne d'une seule colonne : `connexion.execute(...).fetchone()[0]`. ↑énoncé

Ex. 13 – piste La boucle du fichier de départ, `for (dom, ext, bd, be) in connexion.execute(requete):`, donne pour chaque rencontre l'équipe qui reçoit (`dom`), l'équipe qui se déplace (`ext`) et les buts de chacune (`bd`, `be`). Une même rencontre compte pour **deux** équipes : pour l'équipe qui reçoit, les buts « pour » sont `bd` et les buts « contre » `be` ; pour l'autre, c'est l'inverse. On peut parcourir les deux cas avec `for (equipe, pour, contre) in ((dom, bd, be), (ext, be, bd)):`. ↑énoncé

TP 2 SQL Murder Mystery ▶ corrigé p. 877

une enquête à résoudre en SQL

Durée : 1 séance (environ 2 h) · Par deux, sur machine

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t06-tp-murder-mystery` — **archive .zip**

Consignes

- Trois façons de jouer, au choix :
  - **en ligne**, sans rien installer : sur `https://mystery.knightlab.com/`, chaque cadre de code de la page accepte n'importe quelle requête (bouton *Run*) ; rien n'est enregistré d'une visite à l'autre ;
  - avec **DB Browser for SQLite** : ouvrir la base `sql-murder-mystery.db`, puis onglet « Exécuter le SQL » ;
  - avec **Python** : le fichier `tp_murder_depart.py`, placé dans le même dossier que la base, contient la connexion et une fonction `afficher(requete)` qui exécute une requête et en affiche le résultat.
- Fichiers à télécharger (lien ci-dessus) : la base `sql-murder-mystery.db`, sa licence `sql-murder-mystery_LICENCE.txt` et `tp_murder_depart.py`.
- Tenir un **carnet d'enquête** (cahier ou fichier texte) : chaque requête utile, ce qu'elle a appris, les numéros (`id`) des personnes repérées. ▶ signale ce qui se fait sur machine.
- La solution de l'enquête se trouve facilement sur Internet : la chercher, c'est se priver du plaisir de la trouver.
- Niveau : ★ échauffement ★★ classique ★★★ défi (dont les extraits de bac les plus exigeants) ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

But du TP

Un meurtre a été commis à *SQL City*, une ville imaginaire. L'inspecteur vous avait confié le rapport de police… que vous avez égaré. Vous vous souvenez seulement qu'il s'agissait d'un **meurtre** (*murder*), commis le **15 janvier 2018** à **SQL City**. Tout le reste se trouve dans la base de données de la ville, et vous n'avez qu'un seul outil : **SQL**. On va d'abord **explorer** une base inconnue (ses tables, ses clés, son schéma), puis **recouper** des indices à coups de `WHERE`, d'agrégats et de jointures, et enfin **valider** sa réponse avec un `INSERT`.

Petit lexique — les données sont en anglais

Les noms des tables, des attributs et les textes de la base sont en anglais.

crime scene report	rapport de police (scène de crime)	*witness*	témoin
murder, robbery, theft	meurtre, vol avec violence, vol	*interview, transcript*	audition, déposition écrite
drivers license	permis de conduire	*plate number*	numéro d'immatriculation
car make, car model	marque, modèle de la voiture	*gender* : *male, female*	sexe : homme, femme
hair color, eye color	couleur des cheveux, des yeux	*height*	taille en **pouces** (1 pouce = 2,54 cm)
income, annual income	revenu, revenu annuel	*ssn*	numéro de sécurité sociale
gym, membership	salle de sport, abonnement	*check in, check out*	arrivée, départ
event, to attend	événement, assister à	*last, somewhere*	dernier, quelque part
street, Dr, Ave	rue, *Drive*, *Avenue*	*to hire*	engager (quelqu'un)

Partie 1 — Explorer la base avant d'enquêter (25 min)

Un enquêteur sérieux commence par savoir **où** chercher. Toute base SQLite contient une table spéciale qui la décrit, `sqlite_master` : on y trouve la liste des tables et, pour chacune, l'instruction `CREATE TABLE` qui l'a créée.

★ **Exercice 1** *Combien de tables, combien de lignes ?*

► Exécuter la requête suivante, qui affiche le nom de chaque table de la base :

```sql
SELECT name
FROM sqlite_master
WHERE type = 'table';
```

1. Combien la base compte-t-elle de tables ?
2. ► Pour chaque table, écrire la requête qui donne son nombre de lignes, et noter les résultats dans un tableau du carnet. Quelle table contient le plus de lignes ? Laquelle est vide ?
3. D'après son nom, à quoi la table vide pourrait-elle servir ? (On le vérifiera à la fin de la partie 2.)

★★ **Exercice 2** *Lire la structure et dessiner le schéma*

► Afficher l'instruction de création d'une table, par exemple `person` :

```sql
SELECT sql
FROM sqlite_master
WHERE name = 'person';
```

(Dans DB Browser, l'onglet « Structure de la base » montre la même chose.) Faire de même pour les autres tables.

1. Pour chaque table, donner sa **clé primaire**, s'il y en a une. Quelles tables n'en ont pas ? Dans ces tables, deux lignes pourraient-elles être exactement identiques ? Que dit le cours d'une relation à ce sujet ?
2. Lister toutes les **clés étrangères**, sous la forme `table.attribut` → `table.attribut` référencé.
3. Dans la table `person`, l'attribut `ssn` est-il une clé primaire ou une clé étrangère ? Et dans la table `income` ?
4. Dessiner le **schéma relationnel** de la base dans la notation du cours : une boîte par relation, la clé primaire soulignée, un `#` devant chaque clé étrangère, une flèche de la clé étrangère vers la clé qu'elle référence. Quelle table est au centre du schéma ? Quelles tables ne sont reliées à aucune autre ?
5. Les dates sont stockées comme des **entiers**, par exemple `20170101`. Comment s'écrit le 15 janvier 2018 ? Pourquoi ce format permet-il de comparer deux dates avec `<` ou de chercher toutes les dates d'un mois avec `>=` et `<=` ?

★ **Exercice 3** *Premiers coups de sonde*

► Avant de chercher le rapport, prendre la mesure de la table `crime_scene_report` :

1. les différents types de crimes, chacun une seule fois (combien y en a-t-il ?) ;
2. le nombre de crimes enregistrés à `SQL City` ;
3. en une seule requête, la date du plus ancien et celle du plus récent rapport.

Partie 2 — L'enquête (55 min)

Les questions sont de moins en moins guidées. Noter au fur et à mesure, dans le carnet, les **indices** et les **numéros** des personnes : ils resserviront.

★ **Exercice 4** *Le rapport perdu*

▶ Retrouver le rapport du meurtre du 15 janvier 2018 à SQL City. Quelle table interroger ? Sur quels attributs porte la condition ? Écrire la requête, puis traduire la description du rapport dans le carnet.

▷ coup de pouce

★★ **Exercice 5** *Les deux témoins*

1. ▶ **Le premier témoin.** Avec les attributs de la table `person`, comment traduire « habiter la *dernière* maison d'une rue » ? Trouver ce témoin **avec une fonction d'agrégation**, en deux requêtes. Noter son nom et son `id`.

▷ coup de pouce

2. ▶ **Le second témoin.** On ne connaît que son prénom et sa rue. Que renvoie la condition `name = 'Annabel'` ? Pourquoi ? Trouver ce témoin en affichant les habitants de sa rue **triés par nom**. Noter son nom et son `id`.
3. **Autre méthode** [au-delà du programme] : l'opérateur `LIKE` du cours, où `%` remplace n'importe quelle suite de caractères. Écrire la condition qui trouve directement les personnes dont le nom **commence** par `Annabel` et qui habitent cette rue.

★★ **Exercice 6** *Les dépositions*

1. ▶ Afficher les dépositions (`transcript`) des deux témoins à partir de leurs `id`.
2. ▶ Écrire une **seule** requête, avec une jointure, qui affiche pour les deux témoins leur **nom** à côté de leur déposition.
3. Traduire les deux dépositions et dresser la **liste des indices** sur le meurtrier. Pour chaque indice, indiquer la table (et l'attribut) où il pourra être vérifié.

▷ coup de pouce

★★★ **Exercice 7** *Le suspect*

Recouper maintenant les indices, une table après l'autre.

1. ▶ Combien de passages à la salle de sport ont été enregistrés le jour indiqué par le second témoin ?
2. ▶ Parmi les personnes venues ce jour-là, afficher l'identifiant d'abonnement, le nom et le `person_id` de celles qui ont le type d'abonnement décrit par le premier témoin. En lisant le résultat, ne garder que les abonnements dont le numéro correspond à l'indice. Combien de suspects reste-t-il ?
3. ▶ Pour départager les suspects, afficher leur sexe et le numéro d'immatriculation de leur voiture. Qui est le meurtrier ? Que remarque-t-on pour l'autre suspect ?
4. **Défi** [au-delà du programme] : écrire une **seule** requête, avec trois jointures et `LIKE`, qui renvoie directement le nom du meurtrier.

▷ coup de pouce

★ **Exercice 8** *Valider la réponse*

La base vérifie elle-même la réponse. ▶ Exécuter, en remplaçant le texte par le prénom et le nom trouvés (majuscules comprises, entre apostrophes simples) :

```
1 INSERT INTO solution VALUES (1, 'Prénom Nom');
2
3 SELECT value
4 FROM solution;
```

(Sur le site, écrire les deux instructions dans le même cadre.)

1. Traduire le message obtenu.

2. Combien de lignes la table `solution` contient-elle maintenant ? Que vaut sa colonne `user` ? On a pourtant inséré la valeur 1… L'énigme sera résolue à l'exercice 11.

Partie 3 — Le vrai commanditaire (25 min)

★★ **Exercice 9** *Ce que sait le meurtrier*

▶ Afficher la déposition du meurtrier, la traduire, puis dresser la liste des indices sur la personne qui l'a engagé. Pour chacun, indiquer la table et l'attribut qui permettront de le vérifier. Convertir en centimètres l'intervalle de taille donné.

★★★ **Exercice 10** *Démasquer la commanditaire*

▶ Sans aide cette fois : retrouver la personne qui a commandité le meurtre, puis valider la réponse comme à l'exercice 8.

1. Commencer par les indices de la table `drivers_license`. Combien de permis correspondent ?
2. Vérifier ensuite la présence au concert. Comment écrire « en décembre 2017 » avec les dates entières de la base ?
3. **Défi** (proposé par le site) : n'utiliser que **deux** requêtes en tout pour cette partie, la lecture de la déposition comprise.
4. [au-delà du programme] Avec `GROUP BY` et `HAVING`, combien de personnes sont allées **exactement trois fois** au concert en décembre 2017 ? Pourquoi cet indice ne suffit-il pas à lui seul ?

▷ coup de pouce

Partie 4 — Comment la base vérifie-t-elle la réponse ? (10 min)

★★ **Exercice 11** *Le déclencheur caché*

Au-delà des tables, une base peut contenir des **déclencheurs** (*triggers*) [au-delà du programme] : des instructions que le SGBD exécute **automatiquement** quand un événement se produit, par exemple après chaque `INSERT` dans une table.

▶ Afficher le déclencheur de la base :

```sql
SELECT sql
FROM sqlite_master
WHERE type = 'trigger';
```

1. Après quel événement ce déclencheur s'exécute-t-il ? Quelles sont les deux instructions qu'il exécute alors ? Expliquer le mystère de l'exercice 8 (la colonne `user`).
2. La fonction `hex` écrit un texte en **hexadécimal**, un octet (deux chiffres hexadécimaux) par caractère. Pourquoi les auteurs ont-ils écrit les noms des coupables de cette façon plutôt qu'en clair ?
3. À l'aide d'une table ASCII, décoder les quatre premiers octets du premier code. Le nom est-il vraiment *secret* ? Quelle est la différence entre **coder** et **chiffrer** ?

Bilan à rédiger

En une dizaine de lignes, dans le carnet :

- la liste des clauses SQL utilisées pendant l'enquête, en séparant celles du programme de celles qui vont au-delà ;
- le « chemin » suivi dans le schéma pour aller du rapport de police au meurtrier, avec les clés étrangères empruntées ;
- deux attributs de la base dont le **domaine** a dû être compris pour avancer (dates entières, taille en pouces, texte sensible aux majuscules…) ;
- quelle requête a coûté le plus d'essais, et pourquoi.

Pour aller plus loin

★★★ **Exercice 12**  *L'enquête rejouée en Python, sans* `LIKE`

▶ Dans `tp_murder_depart.py`, la fonction `afficher(requete, parametres)` accepte des requêtes **paramétrées** : chaque `?` de la requête est remplacé par une valeur du tuple `parametres`. Pour récupérer les lignes elles-mêmes, on écrit `connexion.execute(requete, parametres).fetchall()`, qui renvoie une liste de tuples.

Écrire un script qui retrouve le meurtrier **sans lire aucun résultat à la main** et **sans** `LIKE` : une requête paramétrée par la date et le type d'abonnement renvoie les candidats ; une boucle ne garde que ceux dont l'identifiant d'abonnement commence par les bons caractères (avec une **tranche**) ; une seconde requête paramétrée par l'`id` de chaque candidat vérifie sa plaque. Le script affiche le nom du coupable.

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 4 – piste Trois conditions reliées par `AND`. La date est un entier : on l'écrit sans apostrophes. Le texte, lui, s'écrit entre apostrophes et en respectant les majuscules : `'SQL City'`. ↑énoncé

Ex. 5 q. 1 – piste Première requête : le plus grand numéro (`address_number`) parmi les habitants de la rue. Seconde requête : la personne de cette rue qui habite à ce numéro. Autre méthode : trier les habitants de la rue par numéro décroissant (`ORDER BY ... DESC`) et lire la première ligne. ↑énoncé

Ex. 6 – piste La table `interview` n'a pas de nom de personne, seulement `person_id` : c'est une clé étrangère vers `person.id`. D'où `FROM person JOIN interview ON interview.person_id = person.id`. ↑énoncé

Ex. 7 – piste Le chemin dans le schéma : `get_fit_now_check_in` → `get_fit_now_member` (par `membership_id`) → `person` (par `person_id`) → `drivers_license` (par `license_id`). Quand deux tables ont un attribut de même nom (`id`), on le préfixe par le nom de sa table. ↑énoncé

Ex. 10 – piste Une jointure `person` – `drivers_license` (par `license_id`), puis une seconde avec `facebook_event_checkin` (par `person_id`). Les conditions sur la taille : `height >= ... AND height <= ...`. On lit ensuite combien de fois chaque nom apparaît. ↑

FICHE DE RÉVISION

L'essentiel — le modèle relationnel

- Une **base relationnelle** = des **tables** (relations). Une table a des **attributs** (colonnes) ; une ligne est un **enregistrement**.
- **Clé primaire** : identifie de façon **unique** chaque ligne. **Clé étrangère** : référence la clé primaire d'une **autre** table (elle relie les tables).
- Un **SGBD** gère persistance, cohérence, efficacité (index), sécurité et accès **simultanés**. Une **transaction** regroupe des opérations qui réussissent **ensemble** ou **pas du tout** (virement bancaire) ; on résume ses garanties par l'acronyme **ACID**.

Interroger — le mémo SQL

```sql
SELECT colonnes            -- ou * ; SELECT DISTINCT : sans doublons
FROM table
WHERE condition            -- AND / OR / NOT, =, <, >, IS NULL, LIKE
ORDER BY colonne [DESC];   -- trier l'affichage
```

Agréger : `COUNT`, `SUM`, `AVG`, `MIN`, `MAX` (au programme) ; pour aller plus loin, `GROUP BY` (regrouper) et `HAVING` (filtrer les groupes), non exigés au bac.

```sql
SELECT categorie, COUNT(*)
FROM jeu
GROUP BY categorie
HAVING COUNT(*) >= 3;
```

Joindre deux tables par une clé :

```sql
SELECT film.titre, realisateur.nom
FROM film
JOIN realisateur ON film.id_real = realisateur.id;
```

Mettre à jour (attention au `WHERE` !)

```sql
INSERT INTO Membre (id_membre, nom, prenom, cp)
VALUES (6, 'Renard', 'Paul', '69003');
UPDATE Objet SET tarif = 15 WHERE id_objet = 2;
DELETE FROM emprunt WHERE dateRendu IS NOT NULL;
```

Un `UPDATE` / `DELETE` *sans* `WHERE` *touche* ***toute*** *la table.*

Erreurs à éviter

- `WHERE` oublié dans `UPDATE` / `DELETE` ;
- jointure sans condition ⇒ produit cartésien ;
- `WHERE` (lignes) confondu avec `HAVING` (groupes) ;
- apostrophes autour des chaînes ; `=` et non `==`.

Savoir-faire à maîtriser — je sais…

- ☐ lire un **schéma relationnel** (clés primaire et étrangère) ;
- ☐ distinguer la **structure** d'une base de son **contenu**, et repérer les **anomalies** d'un schéma (redondance ; insertion, suppression, mise à jour) ;
- ☐ écrire `SELECT` / `FROM` / `WHERE` (`AND`, `OR`, `NOT`), `ORDER BY`, `DISTINCT` ;
- ☐ utiliser les **agrégats** (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) ; pour aller plus loin, regrouper avec `GROUP BY` / `HAVING` ;
- ☐ écrire une **jointure** (`JOIN ... ON`) entre deux tables ;
- ☐ **mettre à jour** une base (`INSERT`, `UPDATE`, `DELETE`) sans oublier le `WHERE` ;
- ☐ expliquer le rôle d'un **SGBD** et d'une **transaction** (propriétés ACID).

Se tester — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 883

1. À quoi sert une **clé étrangère** ?
2. Que fait la requête `DELETE FROM emprunt;` ?
3. Écrire la requête qui compte les films de la table `film` sortis après 2010.
4. Une jointure **sans** condition entre une table de 10 lignes et une de 5 lignes donne combien de lignes ?
5. Que garantit une transaction lors d'un virement bancaire ?

Et avant ? — *questions sur les chapitres précédents.* ▶ réponses p. 883

1. Quelle est la hauteur d'un arbre réduit à sa racine, avec la convention du cours ? Et celle de l'arbre vide ? (ch. 5)
2. Pour vérifier un parenthésage, pourquoi une pile plutôt qu'une file ? (ch. 3)
3. Combien d'appels (appel initial compris) déclenche `fib(4)` dans la version récursive naïve ? (ch. 1)

DÉFIS ADVENT OF CODE

DÉFI 1 Passport Processing

les passeports — dictionnaires et validation

Jour 4

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t06-defi-aoc-2020-04` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/4`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Un scanner d'aéroport a lu quelques centaines de passeports, chacun décrit par une suite de champs (année de naissance, taille, couleur des yeux…). Il faut compter les passeports acceptables : d'abord ceux qui ont tous les champs nécessaires, puis ceux dont chaque valeur est plausible. Le fichier est long et ses passeports s'étalent sur un nombre variable de lignes : le travail consiste surtout à bien le découper, puis à appliquer des règles précises sans en oublier.

Ce qu'on réinvestit. La notion de **domaine** vue dans ce chapitre : chaque champ d'un passeport n'est valide que si sa valeur appartient à un domaine précis (une année entre deux bornes, une couleur parmi une liste fixée, un numéro d'exactement neuf chiffres). Vérifier un passeport revient à vérifier, champ par champ, l'appartenance à ces domaines, comme une base de données vérifie ses contraintes avant d'accepter une ligne.

La démarche de la fiche.

1. Comprendre ce qu'est un passeport complet, puis trier à la main un petit fichier.

2. Découper le fichier en passeports, puis chaque passeport en un dictionnaire.
3. Vérifier la présence des champs et compter : c'est la partie 1.
4. Partie 2 : écrire une petite fonction testée par champ, puis les combiner.
5. En approfondissement, décrire la forme d'une valeur par un motif.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`field`	champ	`required`	obligatoire
`missing`	manquant	`optional`	facultatif
`batch file`	fichier de lot	`blank line`	ligne vide
`key:value pair`	paire clé :valeur	`at least / at most`	au moins / au plus

Résumé de l'énoncé (partie 1)

Contexte. Arrivé à l'aéroport, le héros s'aperçoit qu'il a emporté une carte d'identité du pôle Nord, et non son passeport. La file devant les scanners automatiques est interminable, car ces machines vérifient mal les documents. Il faut écrire le programme qui décide si un document est acceptable.
Les données. Le fichier contient les données lues par un scanner.

- Les passeports sont séparés par une **ligne vide** ; un même passeport peut s'étaler sur une ou plusieurs lignes.
- Un passeport est une suite de champs `cle:valeur`, séparés par des espaces ou des retours à la ligne, dans un ordre quelconque. Par exemple, `byr:1985` indique une année de naissance 1985.
- Il existe huit clés : `byr` (année de naissance), `iyr` (année de délivrance), `eyr` (année d'expiration), `hgt` (taille), `hcl` (couleur des cheveux), `ecl` (couleur des yeux), `pid` (numéro de passeport) et `cid` (pays).

La règle.

- En partie 1, un passeport est valide si les sept premières clés (toutes sauf `cid`) sont présentes.
- `cid` peut manquer : c'est ce qui laisse passer la carte du héros, à laquelle il manque justement ce champ.
- Les valeurs ne sont pas encore examinées : seule compte la présence des clés.

La réponse attendue. Le nombre de passeports valides. Par exemple, un passeport qui a les sept premières clés mais pas `cid` compte ; un passeport qui a `cid` mais pas `hgt` ne compte pas.

Exercice 1 *Comprendre ce qui est demandé*

But. Vérifier qu'on a compris ce qui rend un passeport valide en partie 1, et repérer la difficulté de la lecture du fichier.
À quoi ça sert ? Les deux premières questions fixent la règle (présence des clés, pas leur valeur ; `cid` facultatif) ; la troisième montre qu'il faut découper le fichier par passeport, et non par ligne.

Après lecture de l'énoncé et du résumé, vérifier sa compréhension :

1. Un passeport qui possède les huit champs mais contient `byr:abc` est-il valide en partie 1 ?
2. Un passeport de sept champs, parmi lesquels `cid`, est-il valide ?

3. Lire le fichier ligne par ligne rend-il le découpage en passeports facile ? Pourquoi ?

Exercice 2 — *À la main, sur un petit exemple*

But. Repérer les passeports d'un petit fichier et dire lesquels sont complets.
À quoi ça sert ? On obtient un exemple dont on **connaît la réponse** (4), sur lequel on testera chaque étape du programme : nombre de blocs, dictionnaires, comptage.

On considère le fichier (inventé) suivant :

```
byr:1985 hgt:172cm ecl:grn
pid:004512398 iyr:2015 eyr:2027 hcl:#a97842

iyr:2018 hcl:#18171d byr:2001
ecl:blu eyr:2029 pid:765432109

hgt:64in pid:12345678 cid:99 ecl:hzl
hcl:#cc33ff byr:1950 iyr:2012 eyr:2023

eyr:2031 ecl:oth byr:1999 iyr:2014 pid:000000042 hcl:#0a0b0c hgt:190cm

hcl:#623a2f pid:543210987 eyr:2025 byr:1932
iyr:2019 ecl:amb hgt:158cm cid:7
```

Combien de passeports contient-il ? Lesquels sont valides selon la partie 1 ? Vérification : on doit en trouver 4.

2 Programmer la partie 1

Exercice 3 — *Séparer les passeports*

But. Découper le texte du fichier en une liste de blocs, un bloc par passeport.
À quoi ça sert ? C'est la difficulté principale de la lecture : un passeport s'étale sur un nombre variable de lignes. Une fois les blocs isolés, chacun se traite indépendamment des autres.

Lire tout le fichier en une chaîne et construire la liste des blocs de texte, un bloc par passeport. Afficher le nombre de blocs. ▷ coup de pouce

Exercice 4 — *Un passeport, un dictionnaire*

But. Transformer un bloc de texte en un dictionnaire qui associe chaque clé à sa valeur.
À quoi ça sert ? Le dictionnaire permet de demander directement « la clé `byr` est-elle présente ? » ou « que vaut `hgt` ? », sans chercher dans le texte. Les deux parties travaillent sur ces dictionnaires.

Écrire une fonction `lire_passeport(bloc)` qui renvoie un dictionnaire, par exemple `{"byr": "1985", "hgt": "172cm", ...}`. Garder les valeurs sous forme de chaînes. ▷ coup de pouce

Rappel : « tous » avec un booléen drapeau

La méthode est présentée dans l'encadré « Méthode : « tous » ou « au moins un », avec un booléen drapeau » du défi *Handy Haversacks* (chapitre 1) : pour vérifier que **toutes** les valeurs conviennent, on part d'un booléen à `True`, que la boucle passe à `False` au premier contre-exemple.
À essayer. Écrire `complet(passeport)` avec un drapeau `tout_present`. Que renvoie-t-elle si la liste des clés obligatoires est vide ?

Exercice 5 *Compter les passeports complets*

But. Vérifier qu'un passeport possède toutes les clés obligatoires, puis compter les passeports complets.
À quoi ça sert ? C'est la réponse de la partie 1. La fonction `complet` resservira en partie 2 : un passeport incomplet y est refusé avant même qu'on examine les valeurs.

Écrire `complet(passeport)` qui vérifie la présence des champs obligatoires, puis compter. Tester sur l'exemple inventé de la fiche, puis sur vos données. ▷ coup de pouce

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Des documents absurdes passent encore : il faut maintenant contrôler aussi les valeurs. Le fichier et le découpage en passeports ne changent pas.
La nouvelle règle. Un passeport est valide si les sept clés sont présentes **et** si chaque valeur respecte sa règle :

- `byr` est entre 1920 et 2002, `iyr` entre 2010 et 2020, `eyr` entre 2020 et 2030 (quatre chiffres, bornes comprises) ;
- `hgt` est un nombre suivi de `cm` (de 150 à 193) ou de `in` (de 59 à 76) ;
- `hcl` est `#` suivi d'exactement six caractères parmi `0-9` et `a-f` ; `ecl` vaut exactement l'un de `amb blu brn gry grn hzl oth` ;
- `pid` compte exactement neuf chiffres (zéros de tête compris) ; `cid` est ignoré.

Il suffit qu'une seule valeur sorte de sa règle pour que le passeport soit refusé.
La réponse attendue. Le nombre de passeports valides selon ces règles. Par exemple, `hgt:180cm` est accepté, mais ni `hgt:180` (pas d'unité) ni `hgt:80in` (au-delà de 76 pouces).

Méthode : tester la forme d'une chaîne (boucle drapeau, tranches)

Pour savoir si une chaîne ne contient que des chiffres, on la parcourt avec un booléen drapeau (« tous ») ; les comparaisons de tranches testent le début ou la fin. Tout cela renvoie un booléen qui décrit la forme de la chaîne, sans la convertir.

```
def que_des_chiffres(s):
    """Renvoie True si s est non vide et ne contient que des chiffres."""
    if len(s) == 0:
        return False
    ok = True
    for c in s:
        if c not in "0123456789":
            ok = False
    return ok

print(que_des_chiffres("2019"))    # True : que des chiffres
print(que_des_chiffres("-12"))     # False : le signe n'est pas un chiffre
print(que_des_chiffres(""))        # False : chaine vide
taille = "172cm"
print(taille[-2:] == "cm")       # True : les 2 derniers caracteres
print(taille[:1] == "#")         # False : le premier caractere
print(taille[:-2], taille[-2:]) # 172 cm : tranches avec indices negatifs
```

Intérêt. Vérifier la forme d'une chaîne **avant** d'appeler `int`, qui provoque une erreur sur `"abc"` ou `"170cm"`.
À essayer. Que renvoient `int("0042")` et `que_des_chiffres("0042")` ?

Outil Python : ranger des fonctions dans un dictionnaire

En Python, une fonction est une valeur comme une autre : on peut la ranger dans une variable, une liste ou un dictionnaire, puis l'appeler plus tard.

```python
def double(x):
    return 2 * x


def carre(x):
    return x * x


operations = {"double": double, "carre": carre}   # sans parentheses
for nom, f in operations.items():
    print(nom, f(5))      # double 10, puis carre 25
```

Intérêt. Une seule boucle applique à chaque champ la règle qui lui correspond ; ajouter une règle revient à ajouter une ligne au dictionnaire.
À essayer. Que contient le dictionnaire si l'on écrit `{"double": double(5)}` ?

Exercice 6 *Une fonction par règle*

But. Écrire et tester une fonction de validation par champ, puis les combiner pour valider un passeport entier.
À quoi ça sert ? Une petite fonction testée par règle est bien plus facile à mettre au point qu'une seule longue condition : en cas de mauvaise réponse, les `assert` désignent la règle fautive. Leur combinaison donne la réponse de la partie 2.

1. Pour chaque champ, écrire une fonction de validation : `valide_byr(valeur)`, `valide_hgt(valeur)`, etc. Pour chacune, écrire au moins un `assert` sur une valeur **acceptée** et un sur une valeur **refusée**, en visant les valeurs limites : par exemple `assert valide_byr("2002")` et `assert not valide_byr("2003")`.

2. Écrire `valide(passeport)` qui combine le tout, puis compter. Sur l'exemple inventé de la fiche, on doit trouver 2 passeports valides.

▷ coup de pouce ▷ début de solution

4 Approfondissement

★★★ **Exercice 7** *Défi : décrire la forme par un motif*

But. Écrire une seule fonction générale qui vérifie la forme d'une chaîne d'après un motif.
À quoi ça sert ? Plusieurs règles de la partie 2 ne diffèrent que par la forme attendue : une fonction générale évite de répéter la même boucle. Les `assert` déjà écrits garantissent que la réécriture ne change pas les résultats.

Plusieurs règles décrivent la *forme* d'une chaîne, caractère par caractère. On la résume par un **motif** : dans un motif, `9` désigne un chiffre quelconque, `h` un caractère hexadécimal (`0` à `9` ou `a` à `f`) et tout autre caractère doit être recopié tel quel. Par exemple `"#hhhhhh"` décrit `hcl` et `"99h99"` accepte `"08a30"` mais ni `"8a30"` ni `"08g30"`.

1. Écrire `respecte(motif, valeur)` qui renvoie `True` si `valeur` a la forme décrite par `motif`.

2. Réécrire `valide_hcl`, `valide_pid` et la vérification des années avec `respecte`. Les tests écrits plus haut doivent toujours passer.

▷ coup de pouce ▷ début de solution

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Une ligne vide, c'est deux retours à la ligne consécutifs : `texte.split("\n\n")`. ↑énoncé

Ex. 4 – piste `bloc.split()` sans argument coupe à la fois aux espaces et aux retours à la ligne. Chaque morceau se coupe ensuite en deux au `:`. ↑énoncé

Ex. 5 – piste Ranger les clés obligatoires dans une liste et tester chacune avec l'opérateur `in`, qui teste la présence d'une **clé** dans un dictionnaire. ↑énoncé

Ex. 6 – piste Penser aux chaînes pièges : une taille sans unité, un identifiant trop court, une couleur en majuscules… La fonction `que_des_chiffres` de l'encadré « Méthode » et les tranches `valeur[:-2]` et `valeur[-2:]` sont utiles. ↑énoncé

Ex. 6 – début de solution Un dictionnaire peut associer chaque nom de champ à sa fonction de validation : `regles = {"byr": valide_byr, ...}`. On parcourt alors `regles.items()` et on appelle `fonction(passeport[cle])`. ↑énoncé

Ex. 7 – piste Commencer par comparer les longueurs, puis parcourir les deux chaînes en parallèle avec un indice `i`. ↑énoncé

Ex. 7 – début de solution Les tests de la partie 2 servent ici de filet de sécurité : on modifie le code, on relance les `assert`. ↑énoncé

DÉFI 2 Ticket Translation

les billets de train — dictionnaires et ensembles

Jour 16

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t06-defi-aoc-2020-16` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/16`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. On dispose des règles de validité d'une vingtaine de champs et de plus de deux cents billets de train dont on ne lit que les nombres. Il faut d'abord repérer les valeurs impossibles, puis, en partie 2, retrouver à quel champ correspond chaque position du billet. Croiser des milliers de valeurs avec toutes les règles, puis déduire l'attribution des champs, demande un programme organisé.
Ce qu'on réinvestit. Ce défi fait écho aux **contraintes de domaine** vues dans ce chapitre : chaque champ d'un billet n'accepte que certaines valeurs, comme un attribut de base de données, et l'on s'en sert pour écarter les enregistrements incohérents. On utilise aussi des dictionnaires, des booléens drapeaux (« au moins un », « tous ») et des ensembles pour raisonner par élimination.
La démarche de la fiche.

1. Vérifier sa compréhension des règles, puis repérer à la main les valeurs non valides d'un petit fichier.
2. Lire les trois blocs du fichier, puis tester chaque valeur contre les règles : réponse de la partie 1.
3. Écarter les billets non valides, puis attribuer les champs aux positions par élimination : réponse de la partie 2.
4. En défi, réfléchir à ce qu'il faudrait faire si l'élimination se bloquait.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`ticket`	billet	`field`	champ
`range`	intervalle	`inclusive`	bornes comprises
`nearby`	voisin, proche	`invalid`	non valide
`scanning error rate`	taux d'erreur de lecture	`comma-separated`	séparé par des virgules
`determine`	déterminer	`position`	position, rang

Résumé de l'énoncé (partie 1)

Contexte. Votre trajet passe par un train à grande vitesse, mais le billet est écrit dans une langue inconnue : vous ne pouvez lire que les nombres. Vous avez rassemblé dans un fichier les règles de validité des différents champs d'un billet, votre billet, et ceux d'autres voyageurs aperçus à la gare.
Les données. Le fichier a trois blocs séparés par une ligne vide.

- D'abord les règles, une par ligne, de la forme `nom: a-b or c-d` : la valeur de ce champ doit être dans l'un des deux intervalles, bornes comprises.
- Puis votre billet : la ligne `your ticket:` suivie d'une ligne.
- Enfin les billets voisins : la ligne `nearby tickets:` suivie d'une ligne par billet.

Un billet est une suite d'entiers séparés par des virgules. Les champs sont dans le même ordre sur tous les billets, mais on ne sait pas quel champ est à quelle position.
La règle.

- Une valeur est non valide si elle ne respecte **aucune** règle : elle ne peut correspondre à aucun champ, quelle que soit sa position.
- Il suffit qu'elle respecte une seule règle (n'importe laquelle) pour ne pas être comptée.
- On examine seulement les billets voisins : votre propre billet est laissé de côté.

La réponse attendue. La somme de toutes les valeurs non valides des billets voisins. Par exemple, avec les deux règles `1-3 or 7-9` et `4-6 or 10-12` et les billets voisins `2,5, 8,13` et `20,4`, seules 13 et 20 ne respectent aucune règle : la réponse serait $13 + 20 = 33$.

Exercice 1 *Comprendre ce qui est demandé*

But. Vérifier sur trois questions courtes qu'on a compris quand une valeur est valide.
À quoi ça sert ? Elles visent trois pièges : les bornes comprises, une valeur non valide égale à 0 (invisible dans la somme mais pas pour la partie 2), une valeur compatible avec plusieurs règles.

Questions de vérification, à traiter sur le cahier :

1. Pour la règle `2-6 or 9-12`, les valeurs 6, 7 et 9 sont-elles acceptées ?
2. Une valeur non valide égale à `0` change-t-elle la somme ? Son billet est-il valide pour autant ?
3. Une valeur peut-elle respecter plusieurs règles à la fois ?

Exercice 2 *À la main, sur un petit exemple*

But. Repérer à la main les valeurs non valides d'un petit fichier.
À quoi ça sert ? On obtient un exemple dont on **connaît la réponse** (46). Le même fichier sert d'exemple pour toute la partie 2, et la remarque sur le quatrième billet prépare son premier piège.

On considère le fichier (inventé) suivant :

```
poids: 1-4 or 8-12
taille: 3-6 or 10-15
depart: 5-11 or 14-20

your ticket:
9,14,7

nearby tickets:
3,13,10
2,21,10
12,15,11
7,0,13
4,13,10
25,4,4
3,6,11
```

Repérer les valeurs non valides. Vérification : la réponse est `46`. Que remarque-t-on sur le quatrième billet voisin ?

2 Programmer la partie 1

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
1 ligne = "1721\n"
2 print(ligne.strip())                  # 1721
3 print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Exercice 3 *Lire le fichier*

But. Transformer les trois blocs du fichier en données Python : un dictionnaire de règles, votre billet, la liste des billets voisins.
À quoi ça sert ? Tout le défi travaille sur ces trois objets : une fois le fichier lu, on n'a plus à manipuler de texte.

Écrire une fonction `lire(nom_fichier)` qui renvoie trois objets : un dictionnaire `regles` (nom du champ → liste de deux couples de bornes), votre billet et la liste des billets voisins (listes d'entiers). ▷ coup de pouce

Rappel : « au moins un » et « tous » avec un booléen drapeau

La méthode est présentée dans l'encadré « Méthode : « tous » ou « au moins un », avec un booléen drapeau » du défi *Handy Haversacks* (chapitre 1) : pour « au moins un », le drapeau part de `False` et passe à `True` au premier exemple ; pour « tous », il part de `True` et passe à `False` au premier contre-exemple.
À essayer. Pour une liste vide, que vaut à la fin un drapeau « au moins un » ? un drapeau « tous » ? Pourquoi ces deux résultats sont-ils cohérents ?

Exercice 4 *Valeurs non valides*

But. Tester si une valeur respecte au moins une règle, puis additionner les valeurs qui n'en respectent aucune.
À quoi ça sert ? Cette somme est la réponse de la partie 1, et la fonction `valable_quelque_part` servira de nouveau en partie 2 pour trier les billets.

Écrire une fonction `respecte(valeur, bornes)` puis une fonction `valable_quelque_part(valeur, regles)`. En déduire la réponse de la partie 1 : la somme des valeurs non valides (l'énoncé en ligne l'appelle « taux d'erreur »). Tester sur l'exemple inventé de la fiche, puis sur vos données.
▷ coup de pouce

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Il s'agit maintenant de déchiffrer votre billet : savoir à quel champ correspond chaque position.
La nouvelle règle.

- Écarter d'abord les billets voisins qui contiennent au moins une valeur non valide (même si cette valeur vaut 0).
- Avec les billets restants, déterminer la position de chaque champ : un champ ne peut être à une position que si **toutes** les valeurs des billets restants à cette position respectent sa règle. Une seule attribution est possible.
- Repérer les champs dont le nom commence par `departure`.

La réponse attendue. Le produit des valeurs de **votre** billet pour les champs dont le nom commence par `departure`. Par exemple, s'il n'y avait que deux tels champs et que votre billet portait 7 et 11 à leurs positions, la réponse serait 77.

Exercice 5 — *Ne garder que les bons billets*

But. Ne conserver que les billets voisins dont toutes les valeurs sont valides.
À quoi ça sert ? Seuls ces billets sont fiables pour déduire la position des champs : un seul billet faux suffirait à fausser l'attribution.

Construire la liste des billets voisins valides. Attention : un billet peut contenir une valeur non valide qui ne change pas la somme de la partie 1. Sur l'exemple inventé de la fiche, il reste 4 billets valides.

Outil Python : les ensembles (`set`) — rappel

Tout est présenté dans l'encadré « Outil Python : les ensembles » du défi *Crab Combat* (chapitre 4) : `set()`, `add`, `remove`, test `x in ensemble` en temps constant, éléments non modifiables (tuples). Dans ce défi, ces opérations suffisent.
À essayer. Que contient `{1, 2, 2, 3}` ? Que se passe-t-il si l'on appelle `remove` pour retirer un élément absent ? Comment l'éviter avec un test `in` ?

Outil Python : tester le début d'une chaîne avec une tranche

`s[:n]` est la chaîne des `n` premiers caractères de `s`. Pour savoir si `s` commence par `debut`, on compare `s[:len(debut)]` à `debut`.

```
1 nom = "departure track"
2 debut = "departure"
3 print(nom[:len(debut)] == debut)      # True
4 print("arrival"[:6] == "depart")      # False
```

Intérêt. `len(debut)` évite de compter les lettres à la main. Une tranche plus longue que la chaîne ne provoque pas d'erreur : elle s'arrête à la fin de la chaîne.
À essayer. Que vaut `"dep"[:9]` ? En déduire la valeur de `"dep"[:9] == "departure"`.

★★ Exercice 6 — *Élimination par contraintes*

But. Déduire la position de chaque champ, d'abord en listant les champs possibles par position, puis par élimination.
À quoi ça sert ? C'est le cœur de la partie 2 : une fois chaque champ placé, on lit sur votre billet les valeurs des champs `departure` et l'on obtient la réponse.

Pour chaque position, construire l'**ensemble** des champs compatibles avec *toutes* les valeurs des billets valides à cette position. Puis : tant qu'il reste des positions non attribuées, trouver une position qui n'a plus qu'un seul champ possible, l'attribuer, et retirer ce champ des autres ensembles. Sur l'exemple, la position 0 est compatible avec `poids` et `taille`, la position 1 avec `taille` seulement ; à la fin, `depart` est en position 2 et vaut 7 sur votre billet. ▷ coup de pouce

▷ début de solution

★★★ Exercice 7 — *Défi : et si l'élimination bloquait ?*

But. Imaginer un cas où l'élimination ne suffit pas, et la méthode qu'il faudrait alors employer.
À quoi ça sert ? La méthode de la fiche fonctionne parce que les données s'y prêtent ; savoir repérer ses limites conduit au retour sur trace, une méthode générale de recherche de solution.

Inventer un petit exemple où, à un moment, aucune position n'a un seul champ possible alors qu'une attribution existe. Comment faudrait-il alors chercher la solution ? ▷ coup de pouce

Coups de pouce *après avoir vraiment cherché : la* ***piste*** *d'abord, le* ***début de solution*** *si besoin.*

Ex. 3 – piste `open(...).read().strip().split("\n\n")` sépare les trois blocs, car ils sont séparés par une ligne vide. ↑énoncé

Ex. 4 – piste `a <= valeur <= b` s'écrit tel quel en Python. Les deux fonctions sont des « au moins un » : un drapeau initialisé à `False` dans chacune. ↑énoncé

Ex. 6 – piste Un dictionnaire `possibles` associant à chaque position un `set` de noms de champs. Avant d'appeler `remove`, tester avec `in` que le champ est encore dans l'ensemble. Pour récupérer l'unique élément d'un ensemble, une boucle `for` suffit. ↑énoncé

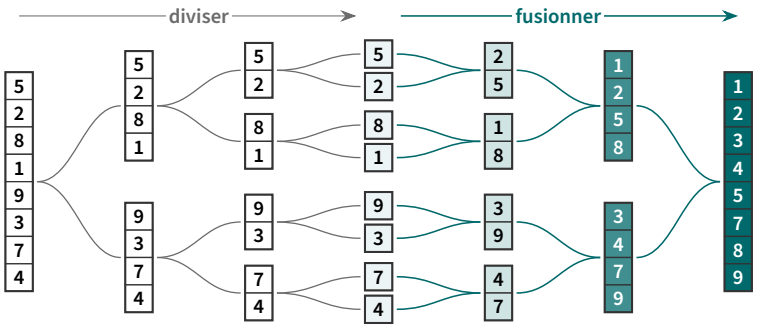
Ex. 6 – début de solution La boucle principale est un `while` sur le nombre de positions déjà attribuées. À chaque tour, on cherche une position `p` telle que `len(possibles[p]) == 1`. ↑énoncé

Ex. 7 – piste Penser au retour sur trace : on essaie un choix, et on revient en arrière s'il mène à une impasse. ↑énoncé

7

CHAPITRE 7

Diviser pour régner



DANS CE CHAPITRE

- Activités préparatoires 333
- Cours 335
- Exercices 346
- TP et projets 359
- Fiche de révision 372
- Défis Advent of Code 374
- Corrigés 885

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Couper en deux, encore et encore

▶ corrigé p. 885

un dictionnaire, deux paquets de cartes, une équipe

Durée : 30 min · Sans ordinateur, par deux

Consignes

- Matériel : un jeu de huit cartes numérotées de 1 à 8 (ou huit post-it). L'exercice 3 se fait par quatre. Les réponses s'écrivent sur le cahier.
- On **manipule** et on **compte** d'abord ; aucun vocabulaire n'est attendu avant l'exercice 4.

Exercice 1 *Le dictionnaire de 2000 pages*

On cherche le mot « algorithme » dans un dictionnaire papier de 2 000 pages.

1. Si l'on tourne les pages une à une depuis le début, combien de pages faut-il regarder, au pire ?
2. Stratégie de Léa : ouvrir *au milieu*, regarder si le mot est avant ou après, puis recommencer *sur la moitié qui reste*. Recopier et compléter le tableau sur le cahier (arrondir à l'entier supérieur).

Coups d'œil	0	1	2	3	4	5	6	7	8	9	10	11
Pages	2 000	1 000										

3. Combien de coups d'œil faut-il à Léa, au pire, pour tomber sur la bonne page ? Et avec un dictionnaire deux fois plus épais (4 000 pages) ?
4. Jeu, par deux : l'un pense à un nombre entre 1 et 100, l'autre le devine ; on ne répond que « plus grand », « plus petit » ou « gagné ». Jouer trois parties en appliquant la stratégie de Léa. Noter le nombre de questions à chaque partie. Combien en faut-il au plus ?

Exercice 2 *Fusionner deux paquets triés*

Poser, faces visibles, deux petits paquets **déjà triés** (la plus petite carte au-dessus, ici à gauche) :

paquet G : **2** **5** **6** **8** paquet D : **1** **3** **4** **7**

5. On veut former un seul paquet trié. Règle : on ne regarde que les **deux cartes du dessus** ; on prend la plus petite et on la pose sur le paquet final. Réaliser la fusion et noter l'ordre des cartes prises.
6. Combien de **comparaisons** (« laquelle est la plus petite ? ») a-t-on faites ? Pourquoi la dernière carte ne coûte-t-elle aucune comparaison ?
7. Deux paquets triés de 50 cartes chacun : combien de comparaisons au plus faut-il pour les fusionner ? Justifier en remarquant que chaque carte n'est prise qu'une fois.

Exercice 3 *Trier en équipe*

Le paquet mélangé | 7 | 3 | 6 | 1 | 8 | 2 | 5 | 4 | est confié à une équipe. Chacun applique la **même consigne** :

Consigne de l'équipe

Si tu reçois **une seule carte**, rends-la telle quelle : elle est triée. Sinon, **coupe** ton paquet en deux moitiés, fais trier chaque moitié par un camarade *qui suit la même consigne*, puis **fusionne** les deux moitiés triées qu'on te rend (comme dans l'exercice 2).

8. Jouer la scène à quatre (un même élève peut traiter plusieurs paquets l'un après l'autre). Dessiner sur le cahier tous les paquets formés en coupant (de haut en bas), puis tous les paquets formés en fusionnant (de bas en haut).

9. Combien d'étages de « coupes » faut-il pour arriver à des paquets d'une carte ? Et pour 16 cartes ? pour 1 024 cartes ?

10. À chaque étage de fusions, chaque carte est prise une fois : combien de comparaisons au plus, *pour tout l'étage*, avec 8 cartes ? En déduire un maximum pour tout le tri.

11. En Première, le tri par sélection de n cartes demande $\frac{n(n-1)}{2}$ comparaisons. Recopier et compléter le tableau, en ordre de grandeur, pour le tri en équipe : « nombre de cartes $\times$ nombre d'étages ».

Nombre de cartes n	8	16	1 024
Tri par sélection : $\frac{n(n-1)}{2}$	28		
Tri en équipe : $n\times$ (nombre d'étages)			

Exercice 4 *Mettre des mots*

12. La consigne de l'équipe se fait en trois temps. Donner un verbe pour chacun :

- partager le paquet en deux moitiés ;
- faire trier chaque moitié, de la même façon ;
- recoller les deux moitiés triées en un seul paquet.

13. Dans la stratégie du dictionnaire, on ne garde qu'une seule moitié. Lequel des trois temps n'a alors presque rien à faire ? Pourquoi ?

14. Recopier et compléter avec ses propres mots : « La consigne de l'équipe s'applique à elle-même sur des paquets plus petits, comme une fonction… ; elle s'arrête quand… »

COURS

Programme (BO) *« Écrire un algorithme utilisant la méthode "diviser pour régner". »* Le BO cite deux exemples : *« la rotation d'une image bitmap d'un quart de tour avec un coût en mémoire constant »* et *« le tri fusion, qui permet d'exploiter la récursivité et d'exhiber un algorithme de coût en $n \log_2 n$ dans le pire des cas »*.

Prérequis récursivité (condition d'arrêt, variant, coût), listes et tableaux, tris déjà vus (insertion, sélection), notion de coût et de logarithme base 2. → fiches de révision : ch. 1 p. 58 → rappels de 1re : listes p. 1057, tris p. 1060, dichotomie p. 1060

Objectifs *reconnaître* un problème qui se prête à « diviser pour régner », *écrire* l'algorithme récursif correspondant, et *analyser* son coût (passage de n^2 à $n \log_2 n$, de n à $\log_2 n$).

Situation de départ

1. Quelqu'un pense à un nombre entre 1 et un million ; vous ne pouvez lui demander que « plus grand ? » ou « plus petit ? ». Combien de questions vous faut-il, au plus ?
2. Trier un million de valeurs avec le tri par insertion de Première demande de l'ordre de 10^{12} opérations. Peut-on faire beaucoup mieux, et comment ?
3. Qu'ont en commun ces deux stratégies, et pourquoi s'écrivent-elles naturellement avec une fonction qui s'appelle elle-même ?

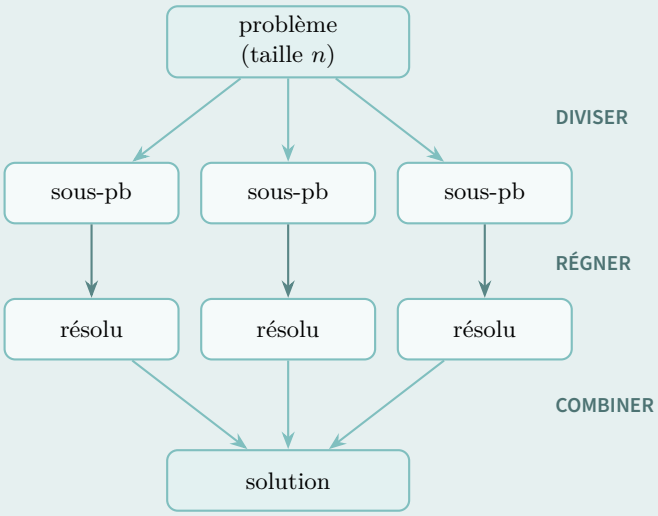
I Un paradigme : diviser pour régner

Pour chercher un mot dans un dictionnaire de 2 000 pages, on ouvre *au milieu* et on **recommence sur la moitié qui reste** : une dizaine de coups d'œil suffit. Pour trier un paquet de cartes à plusieurs, on le coupe en deux, on fait trier chaque moitié, puis on **fusionne** les deux moitiés triées. Ces deux stratégies relèvent de la même méthode, l'une des plus puissantes de l'algorithmique : **diviser pour régner** (en anglais *divide and conquer*).

Définition 1 — Diviser pour régner

Diviser pour régner est une **méthode de conception d'algorithmes** en trois temps :

- **Diviser** : découper le problème en sous-problèmes *du même type*, mais **plus petits** ;
- **Régner** : résoudre chaque sous-problème (le plus souvent **récursivement**) ;
- **Combiner** : recoller les solutions des sous-problèmes en une solution du problème de départ.



Remarque

Comme les sous-problèmes sont « du même type mais plus petits », on les résout en **rappelant la même fonction** : les algorithmes « diviser pour régner » sont donc presque toujours **récursifs**. On retrouve la mécanique du chapitre **Récursivité** : une **condition d'arrêt** (un problème si petit qu'on sait répondre directement) et un **cas récursif** qui se rapproche de cette condition d'arrêt.

Méthode — Les étiquettes d'étapes

Dans les exemples et les corrigés, un commentaire en **majuscules** nomme le **rôle** des lignes importantes, toujours avec les mêmes mots. Pour un algorithme « diviser pour régner » : `CAS DE BASE` (la condition d'arrêt), `DIVISER` (découper le problème), `APPEL RECURSIF` (régner sur un morceau), `COMBINER` (recoller les solutions). Devant un nouveau problème, on répond à ces quatre questions avant d'écrire la moindre ligne.

II Un premier exemple : la recherche dichotomique

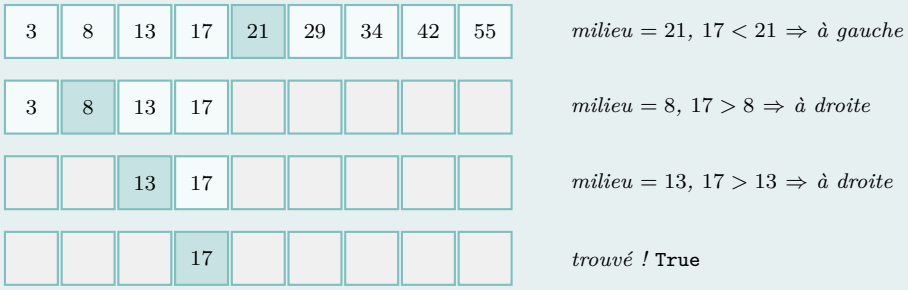
Reprenons le dictionnaire. On dispose d'un tableau **déjà trié** et on veut savoir si une valeur `cible` s'y trouve.

- **Diviser** : on compare `cible` à l'élément *du milieu* ; cela désigne la moitié (gauche *ou* droite) où chercher.
- **Régner** : on cherche dans cette moitié, *de la même façon*.
- **Combiner** : rien à recoller ici — la réponse de la moitié *est* la réponse. (La phase « combiner » est parfois triviale.)

Exemple — *Lisez et testez cette fonction. Sur quel tableau doit-on l'appeler ?*

```
1  def recherche(tab, cible, debut, fin):
2      if debut > fin:                                  # CAS DE BASE : intervalle vide
3          return False
4      milieu = (debut + fin) // 2                      # DIVISER
5      if tab[milieu] == cible:                         # TESTER
6          return True                                  # RENVOYER : trouve
7      elif cible < tab[milieu]:
8          return recherche(tab, cible, debut, milieu - 1)   # APPEL RECURSIF : moitie gauche
9      else:
10         return recherche(tab, cible, milieu + 1, fin)     # APPEL RECURSIF : moitie droite
```

▶ **Correction.** On doit l'appeler sur un tableau **trié** dans l'ordre croissant : sinon, comparer `cible` au milieu ne dit pas de quel côté chercher. Le premier appel porte sur **tout** le tableau : pour chercher `17` dans `t`, on écrit `recherche(t, 17, 0, len(t) - 1)`. À chaque appel, l'**intervalle de recherche est coupé en deux**.



▷ **Je prédis.** Sur ce même tableau `t`, que renvoie `recherche(t, 20, 0, 8)`, et combien d'appels à `recherche` sont effectués en tout ?

▷ **Je vérifie.** `False`, après **5** appels : on regarde `21`, `8`, `13`, `17`, puis l'intervalle devient vide (`debut = 4 > fin = 3`). L'appel de la condition d'arrêt compte aussi.

Propriété 1 — Coût de la recherche dichotomique

Sur un tableau de taille n, la recherche dichotomique effectue au plus environ $\log_2 n$ comparaisons.

Démonstration :
À chaque appel, la taille de l'intervalle est **divisée par deux**. Partant de n, on obtient successivement n, $n/2$, $n/4$, … On atteint la condition d'arrêt (intervalle vide ou de taille 1) après le nombre de divisions par 2 nécessaires pour descendre de n à 1 : ce nombre est, par définition, $\log_2 n$. □

À retenir — un coût $\log_2 n$ au lieu de n

L'ordre de grandeur est spectaculaire : dans un tableau d'**un million** d'éléments, une recherche naïve (élément par élément) demande jusqu'à un million de comparaisons, là où la dichotomie en demande environ 20 (car $2^{20} \approx 10^6$). On passe d'un coût n à un coût $\log_2 n$: c'est exactement le gain que le programme demande de « montrer ». C'est aussi l'idée de la recherche dans un arbre binaire de recherche (chapitre 5), où chaque comparaison élimine tout un sous-arbre.

↻ **Retour sur l'activité.** La stratégie de Léa dans le dictionnaire était une recherche dichotomique : 2 000 pages, puis 1 000, 500… 11 coups d'œil suffisent ($2^{11} = 2\,048$), un seul de plus pour 4 000 pages, et 7 questions au plus pour un nombre entre 1 et 100. Le temps « combiner » n'y a rien à faire.

Remarque — *Un « diviser pour régner » un peu particulier*
Ici, on ne conserve qu'**une** moitié : la recherche dichotomique ne fait donc qu'**un seul** appel récursif (certains parlent alors de « *diminuer* pour régner »). Le tri fusion de la section suivante, lui, résout **les deux** moitiés : c'est un « diviser pour régner » complet.

► **Exercices d'application :** exercices 1 **et** 2 (recherche dichotomique à la main ; maximum d'un tableau)

7

III L'exemple phare : le tri fusion

On sait déjà trier (tri par insertion, tri par sélection, vus en Première), mais ces tris coûtent n^2 : sur un million de valeurs, c'est 10^{12} opérations, inutilisable. Le **tri fusion** (*merge sort*), imaginé par **John von Neumann** en **1945**, applique « diviser pour régner » et fait **bien mieux**.

Un peu d'histoire. Mathématicien d'origine hongroise, von Neumann collabore alors au laboratoire de **Los Alamos** (projet Manhattan) et participe à la conception de l'**EDVAC**, l'un des premiers ordinateurs à programme enregistré, dont il décrit l'organisation, toujours celle de nos ordinateurs (« architecture de von Neumann »). Pour montrer qu'elle sait faire autre chose que du calcul, il écrit l'un des tout premiers programmes de tri : un tri fusion.



John von Neumann (photo de son badge de Los Alamos)

Règle 1 — Tri fusion : les trois temps

- **Diviser** : couper le tableau en **deux moitiés**.
- **Régner** : trier **récursivement** chacune des deux moitiés.
- **Combiner** : **fusionner** les deux moitiés triées en un seul tableau trié.

La condition d'arrêt est immédiate : un tableau de 0 ou 1 élément est **déjà trié**.

A. Le cœur du tri : fusionner deux listes triées

Tout repose sur une opération simple. On dispose de **deux listes déjà triées** et on veut les réunir en une seule liste triée. On avance avec un « doigt » sur chaque liste, et on recopie à chaque fois **le plus petit des deux éléments pointés**.

Exemple — *Fusionnez à la main* `[4, 12, 23, 56]` *et* `[3, 32, 35, 42, 57]`

▶ Correction.

tête de G	tête de D	on recopie…
4	3	$3 < 4$: on prend 3
4	32	$4 < 32$: on prend 4
12	32	on prend 12
23	32	on prend 23
56	32	$32 < 56$: on prend 32
56	35	on prend 35
56	42	on prend 42
56	57	on prend 56
G vide		on recopie la fin de D : 57

Résultat : `[3, 4, 12, 23, 32, 35, 42, 56, 57]`.

En Python :

```
def fusion(gauche, droite):
    resultat = []                                  # INITIALISER
    i = j = 0                                      # INITIALISER : un doigt par liste
    while i < len(gauche) and j < len(droite):     # PARCOURIR
        if gauche[i] <= droite[j]:                 # TESTER : le plus petit des deux
            resultat.append(gauche[i])             # METTRE A JOUR
            i = i + 1
        else:
            resultat.append(droite[j])             # METTRE A JOUR
            j = j + 1
    resultat = resultat + gauche[i:]               # METTRE A JOUR : reste de gauche (ou rien)
    resultat = resultat + droite[j:]               # METTRE A JOUR : reste de droite (ou rien)
    return resultat                                # RENVOYER
```

▷ **Je prédis.** Que renvoie `fusion([2, 7], [2, 3, 9])` ? Lequel des deux `2` est recopié en premier ?
▷ **Je vérifie.** `[2, 2, 3, 7, 9]`. Le `2` de `gauche` passe en premier : le test est `<=`, pas `<`. Quand la boucle s'arrête (`gauche` épuisée après le `7`), le `9` restant est recopié par `droite[j:]`.

À retenir — le coût linéaire de la fusion

Fusionner deux listes dont les tailles totalisent n coûte de l'ordre de n opérations : chaque élément est recopié **une seule fois**. Retenez ce « coût linéaire de la fusion » : il explique le coût final du tri.

↻ **Retour sur l'activité.** La fusion des paquets G et D de l'activité suivait exactement cette règle : 7 comparaisons pour 8 cartes, la dernière se recopiant sans comparaison, et au plus 99 comparaisons pour deux paquets de 50 cartes.

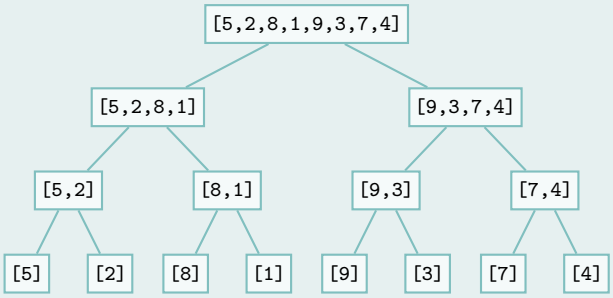
B. Le tri fusion complet

Il ne reste qu'à écrire les trois temps. La condition d'arrêt tient sur `if len(tab) <= 1`.

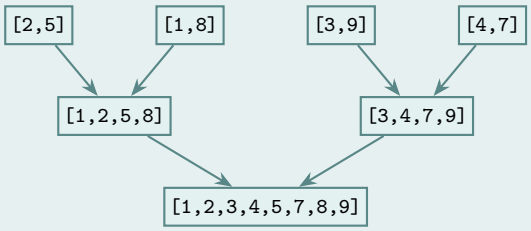
```
def tri_fusion(tab):
    if len(tab) <= 1:                  # CAS DE BASE : deja trie
        return tab
    milieu = len(tab) // 2             # DIVISER
    gauche = tri_fusion(tab[:milieu])  # APPEL RECURSIF : regner sur la 1re moitie
    droite = tri_fusion(tab[milieu:])  # APPEL RECURSIF : regner sur la 2e moitie
    return fusion(gauche, droite)      # COMBINER
```

▷ **Je prédis.** Lors de l'appel `tri_fusion([6, 1, 4])`, que valent `gauche` et `droite` juste avant la fusion ?
▷ **Je vérifie.** `milieu = 3 // 2 = 1` : `gauche = [6]` et `droite = [1, 4]`. La moitié droite est la plus longue, et elle revient **déjà triée** de son appel récursif ; la fusion donne `[1, 4, 6]`.

Suivons l'algorithme sur `[5, 2, 8, 1, 9, 3, 7, 4]`. On **descend** en coupant en deux (diviser), jusqu'aux singletons ; puis on **remonte** en fusionnant (combiner).



Descente (diviser) : on coupe en deux jusqu'aux singletons.



Remontée (combiner) : on fusionne deux à deux les moitiés triées.

Pour les curieux (facultatif)
Cette version renvoie une **nouvelle liste** (claire, mais elle recopie les données). On peut aussi écrire des versions qui rangent les valeurs directement dans le tableau donné au lieu de renvoyer une nouvelle liste : c'est le cas du *tri rapide*, vraiment « en place » (il n'utilise presque aucune mémoire en plus), et du *Tim sort* des TP proposés en fin de chapitre (qui modifie le tableau donné, mais recopie des morceaux pendant ses fusions).

C. Terminaison

Propriété 2

Pour tout tableau, l'appel `tri_fusion(tab)` se termine.

Démonstration :
Prenons pour **variant** la longueur du tableau traité. Elle est un entier positif. À chaque appel récursif, on travaille sur une **moitié** : comme `milieu = len(tab)//2` avec `len(tab) >= 2` dans le cas récursif, chaque moitié a une longueur *strictement inférieure* à celle du tableau de départ. Le variant décroît donc strictement et finit par atteindre 0 ou 1 : la condition d'arrêt, qui renvoie sans appel récursif. □

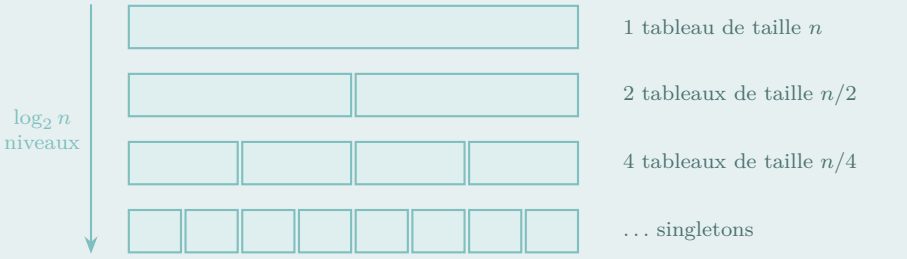
► **Exercices d'application :** exercices 3 à 5, 13 et 14 (tri fusion : l'arbre des appels, compter les appels, écrire la fusion ; arbres d'appels et fusions en série)

IV Analyser le coût : pourquoi $n \log_2 n$?

C'est le point que le programme demande d'« exhiber ». Reprenons l'arbre des appels. Il a deux dimensions.

- **La hauteur.** À chaque niveau, on coupe les tableaux en deux. Partir de n et couper en deux jusqu'à des singletons prend $\log_2 n$ **niveaux** (mêmes divisions par 2 que pour la dichotomie).
- **Le travail par niveau.** À un niveau donné, les fusions traitent *en tout* les n éléments : le coût cumulé des fusions d'un niveau est de l'ordre de n.

En multipliant, *« $\log_2 n$ niveaux, chacun coûtant n »*, on obtient un coût total de l'ordre de $\boxed{n \log_2 n}$, et cela **même dans le pire des cas** — contrairement au tri rapide que l'on verra plus loin.



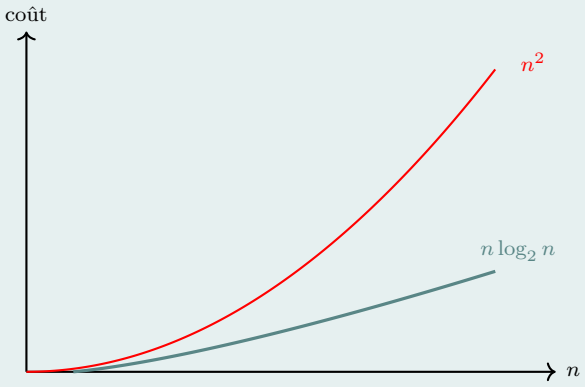
Chaque ligne *totalise une largeur n : coût $\approx n$ par niveau, $\times$ $\log_2 n$ niveaux.*

↻ **Retour sur l'activité.** Le tri en équipe de l'activité était un tri fusion, la consigne jouant le rôle de la fonction récursive : 3 étages de coupes pour 8 cartes, au plus 8 comparaisons par étage de fusions, soit environ $8 \times 3 = 24$ contre 28 pour le tri par sélection ; avec 1 024 cartes, 10 240 contre 523 776.

A. Comparaison n^2 contre $n \log_2 n$

Le gain n'a l'air de rien sur un petit tableau ; il devient énorme quand n grandit.

n	n^2 (insertion)	$n \log_2 n$ (fusion)	rapport
10	100	≈ 33	$3\times$
1 000	1 000 000	$\approx 10\,000$	$100\times$
10^6	10^{12}	$\approx 2 \times 10^7$	$50\,000\times$



Règle 2 — À retenir sur le coût

Diviser pour régner transforme souvent un coût **quadratique** (n^2) en coût **quasi-linéaire** ($n \log_2 n$), ou un coût **linéaire** (n) en coût **logarithmique** ($\log_2 n$). C'est la raison d'être de la méthode.

Règle 3 — Les classes de coût et la notation O

On note **O($f(n)$)** (« grand O de $f(n)$ ») un coût **au plus proportionnel** à $f(n)$ quand la taille n des données devient grande : les constantes et les termes plus petits ne comptent pas. Le chapitre *Structures linéaires* a introduit O(1) et O(n) ; voici la liste complète, avec les mots qu'emploient les sujets de bac.

Coût	**Notation**	**Exemple**
constant	O(1)	dépiler ; accéder à `t[i]`
logarithmique	O($\log_2 n$)	recherche dichotomique ; recherche dans un ABR équilibré
linéaire	O(n)	parcours d'une liste ; `x in liste`
quasi-linéaire	O($n \log_2 n$)	tri fusion
quadratique	O(n^2)	tri par insertion ou par sélection (pire cas)
cubique	O(n^3)	trois boucles imbriquées de n tours
exponentiel	O(2^n)	tours de Hanoï ($2^n - 1$ déplacements)

Le tri fusion est donc en O($n \log_2 n$) dans tous les cas, le tri par insertion en O(n^2) dans le pire des cas.

V D'autres visages de « diviser pour régner »

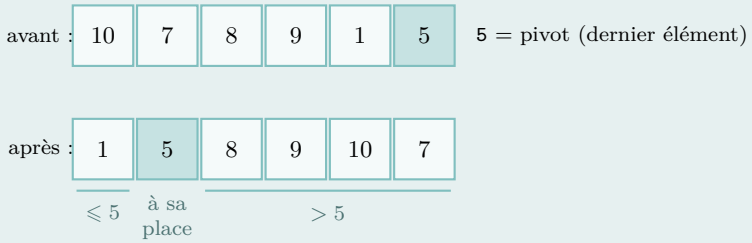
Le paradigme dépasse largement le tri. En voici trois illustrations à connaître.

A. Le tri rapide (*quicksort*)

Autre tri « diviser pour régner », mais qui divise **autrement** : on choisit un élément **pivot**, on **partitionne** le tableau (les plus petits que le pivot à gauche, les plus grands à droite), puis on trie récursivement chaque côté. Ici l'effort est dans le « diviser » (la partition), et le « combiner » est gratuit.

```
1 def tri_rapide(tab, bas, haut):
2     if bas < haut:                      # TESTER : sinon CAS DE BASE, rien a faire
3         p = partition(tab, bas, haut)   # DIVISER : le pivot va a sa position finale
4         tri_rapide(tab, bas, p - 1)     # APPEL RECURSIF : a gauche du pivot
5         tri_rapide(tab, p + 1, haut)    # APPEL RECURSIF : a droite du pivot
```

La **partition** range les éléments autour du pivot ; celui-ci se retrouve à sa **place définitive**, les petits à gauche, les grands à droite. On trie ensuite *récursivement* chaque zone.



Remarque

En moyenne, le tri rapide coûte $n \log_2 n$ et il est **très rapide en pratique** (il trie sur place, sans recopier). Mais si le pivot est mal choisi (par exemple sur un tableau déjà trié), les deux côtés sont très déséquilibrés et le coût **dégénère en** n^2 : c'est là toute la différence avec le tri fusion, garanti en $n \log_2 n$. On l'étudie dans le TP **« Tri rapide »**, en fin de chapitre.

B. Le *Tim sort* : le tri de Python

L'algorithme utilisé par `sorted()` et `list.sort()` en Python s'appelle **Tim sort** (Tim Peters, 2002). C'est un **mélange malin** de tri fusion et de tri par insertion : il repère les portions déjà triées (*runs*), trie les petits morceaux par insertion (rapide sur peu d'éléments), puis les fusionne comme le tri fusion. Un **TP** lui est consacré.

C. La rotation d'une image, exemple du BO

Le BO cite la **rotation d'un quart de tour d'une image** carrée. On découpe l'image en **quatre quadrants**, on fait tourner récursivement chacun, puis on les **repositionne** en tournant (le quadrant haut-gauche va en haut-droite, etc.). En permutant directement les pixels, cette approche atteint un **coût mémoire constant** — la précision demandée par le programme.



Au bac — on vous demandera surtout de…

- **identifier** les trois temps (diviser / régner / combiner) sur un algorithme donné ;
- **compléter** une fonction récursive (souvent l'appel récursif ou la fusion) ;
- **dérouler** un tri fusion ou une recherche dichotomique sur un petit tableau ;
- **justifier** la terminaison (variant = taille du tableau) et **comparer** les coûts n^2, $n\log_2 n$, $\log_2 n$.

Exemples de sujets : *tri fusion* (Liban 2023, jour 2), *recherche dichotomique* récurrente sur de nombreux sujets.

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise :** la *Récursivité* (condition d'arrêt, arbre des appels) et les tris de coût n^2 vus dans *Les algorithmes de tri* (Première).
- **Un écho :** la dichotomie, vue dans *La recherche dichotomique* (Première), et la recherche dans un ABR (*Arbres*) suivent déjà cette idée ; comparez-la à la stratégie gloutonne vue dans *Les algorithmes gloutons* (Première).
- **Pour aller plus loin :** quand les sous-problèmes se répéteront, on mémorisera : ce sera la *Programmation dynamique*.

Bilan — la carte mémoire

Notion	À retenir
Diviser pour régner	diviser · régner (récursif) · combiner
Lien avec la récursivité	sous-problèmes « du même type, plus petits » $\Rightarrow$ récursif
Recherche dichotomique	tableau **trié**, on coupe en deux : coût $\log_2 n$
Tri fusion	diviser en 2 · trier chaque moitié · **fusionner**
Fusion de 2 listes triées	le plus petit des deux « têtes » ; coût $\approx n$
Coût du tri fusion	$n\log_2 n$ **même dans le pire des cas**
Tri rapide (quicksort)	pivot + partition ; $n\log_2 n$ en moyenne, n^2 au pire
Terminaison	variant = **taille** du tableau (décroît)

Retour à la situation de départ

1. Par **recherche dichotomique** : chaque question élimine la moitié des nombres restants ; une vingtaine de questions suffisent, car $2^{20} \approx 10^6$. Le coût passe de n à $\log_2 n$ (§ II).
2. Oui : le **tri fusion** coupe le tableau en deux, trie récursivement chaque moitié puis **fusionne** les deux moitiés triées. Il coûte $n\log_2 n$, environ 2×10^7 opérations pour un million de valeurs, même dans le pire des cas (§ III et IV).
3. Toutes deux appliquent la méthode **diviser pour régner** : diviser en sous-problèmes du même type mais plus petits, les résoudre (régner), puis combiner. Comme les sous-problèmes sont du même type, l'algorithme est **récursif**, avec une condition d'arrêt (§ I).

Erreurs fréquentes

- **Dichotomie sur un tableau non trié.** L'algorithme **ne fonctionne pas** : le tri est une **hypothèse** indispensable.
- **Erreur d'indices `deb` / `fin` / `mil`.** Mauvaise mise à jour (± 1 oublié) $\Rightarrow$ boucle infinie ou dépassement. *Le réflexe :* `mil = (deb + fin) // 2`, puis `deb = mil + 1` ou `fin = mil - 1`.
- **Oublier le cas de base** de la récursion (liste de taille 0 ou 1 déjà triée).
- **Fusion incomplète.** Après la boucle, **recopier** ce qui reste dans la liste non épuisée.
- **Croire le tri rapide toujours en $n \log_2 n$.** C'est vrai en **moyenne** ; au **pire** il est en n^2. Le tri fusion, lui, est en $n \log_2 n$ **même au pire**.
- **Confondre variant et invariant.** Le **variant** (ici la taille) prouve la **terminaison** ; l'invariant prouve la **correction**.

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

☐ reconnaître le schéma **diviser / régner / combiner** → ex. 2, 6 ;

☐ **dérouler** une recherche dichotomique et compter les tours → ex. 1, 11 ;

☐ justifier la **terminaison** (variant) et le **coût logarithmique** → ex. 1, 6 ;

☐ expliquer et dérouler le **tri fusion** (arbre des appels, coût $n \log_2 n$) → ex. 3, 4 ;

☐ **fusionner** deux listes déjà triées → ex. 5 ;

☐ comparer les **coûts** ($\log_2 n$, n, $n \log_2 n$, n^2) → ex. 4, 9.

Vers le Grand Oral

Quelques pistes de questions que ce chapitre permet de préparer (à étayer d'un exemple concret) :

- **Diviser pour régner : pourquoi couper en deux rend-il plus rapide ?** *(la recherche dichotomique en $\log_2 n$; le tri fusion en $n \log_2 n$ contre n^2.)*
- **Le tri fusion est-il toujours meilleur que les tris quadratiques ?** *($n \log_2 n$ même au pire, mais un coût mémoire supplémentaire.)*
- **Quels sont les avantages et les limites du récursif en informatique ?** *(du catastrophique — Fibonacci naïf — au génial — le tri fusion — en passant par les tours de Hanoï, coûteuses mais sans solution plus simple.)*

EXERCICES

Mode d'emploi

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 32.
- **Réflexe** « diviser pour régner » : **(1)** comment **diviser** en sous-problèmes plus petits ? **(2)** quelle est la **condition d'arrêt** ? **(3)** comment **combiner** les résultats ?
- Le *tri rapide*, le *Tim sort* et la *rotation d'image* sont traités dans les **TP** proposés en fin de chapitre.

Comprendre et dérouler ↑ cours

★ **Exercice 1** *Recherche dichotomique à la main* ▶ corrigé p. 887 • essentiel

On cherche dans le tableau trié `tab = [3, 8, 12, 19, 25, 31, 40, 42, 55, 60, 71]`.

1. Donner, dans l'ordre, les valeurs du milieu examinées lors de la recherche de `40`. Combien de valeurs du milieu sont examinées ?
2. Même question pour la recherche de `7` (absent). Comment sait-on qu'il est absent ?
3. Sur un tableau trié de 1000 valeurs, combien de valeurs du milieu sont examinées au maximum ? (donner un ordre de grandeur, en justifiant par les divisions par 2).

▷ coup de pouce

★ **Exercice 2** *Maximum d'un tableau par diviser pour régner* ▶ corrigé p. 887 • essentiel

Pour trouver le maximum d'un tableau **non vide** `t`, on peut procéder ainsi : si `t` n'a qu'un élément, c'est lui le maximum ; sinon, on coupe `t` en deux moitiés, on cherche le maximum de chaque moitié, et on garde le plus grand des deux.

```
1  def maximum(t):
2      if len(t) == 1:                # condition d'arret
3          return ...                 # a completer
4      milieu = len(t) // 2
5      m_g = maximum(...)             # a completer
6      m_d = maximum(...)             # a completer
7      if m_g > m_d:
8          return m_g
9      else:
10         return m_d
```

1. Repérer dans ce code les trois temps : **diviser**, **régner**, **combiner**.
2. ▶ Compléter les trois lignes manquantes, puis tester la fonction sur `[4, 9, 2, 7]`.
3. Dessiner l'arbre des appels de `maximum([4, 9, 2, 7])` en notant, à côté de chaque appel, la valeur qu'il renvoie.

▷ coup de pouce

★★ **Exercice 3** *Tri fusion : l'arbre des appels* ▶ corrigé p. 888 • essentiel

On trie `[8, 3, 5, 1, 9, 2]` par tri fusion (on coupe à `milieu = len(tab) // 2`).

1. Dessiner l'arbre de la phase « diviser » (jusqu'aux listes de 1 élément). ▷ coup de pouce
2. Détailler la **remontée** : donner chaque liste obtenue par fusion, jusqu'à la liste triée finale. ▷ coup de pouce
3. ▶ Compléter les deux appels récursifs de la fonction ci-dessous, puis la tester. ▷ coup de pouce

```python
def tri_fusion(tab):
    if len(tab) <= 1:
        return tab
    milieu = len(tab) // 2
    gauche = ...                    # a completer
    droite = ...                    # a completer
    return fusion(gauche, droite)   # fusion : cf. cours
```

★ **Exercice 4** *Compter les appels du tri fusion* ▶ corrigé p. 889

On reprend la fonction `tri_fusion` de l'exercice précédent.

1. Dessiner l'arbre des appels de `tri_fusion([6, 2, 7, 1])`. Combien d'appels à `tri_fusion` en tout (appel initial compris) ?
2. Pour une liste de 8 éléments, combien d'appels y a-t-il à chaque niveau de l'arbre ? Combien de niveaux sous l'appel initial ? Combien d'appels en tout ?
3. Pour une liste de $1024 = 2^{10}$ éléments, combien de niveaux de découpe y a-t-il sous l'appel initial ? Quel lien avec $\log_2$?

▷ coup de pouce

★★ **Exercice 5** *Écrire la fusion de deux listes triées* ▶ corrigé p. 889 • essentiel

1. Fusionner **à la main** les listes triées `[1, 4, 9]` et `[2, 3, 10, 12]` : à chaque étape, noter les deux éléments comparés et celui qui est recopié. Combien de comparaisons ?
2. ▶ Écrire une fonction `fusion(a, b)` qui prend deux listes **triées** et renvoie une **nouvelle** liste triée contenant tous leurs éléments, sans utiliser `sort` ni `sorted`. La tester sur l'exemple de la question 1, puis avec une liste vide. ▷ coup de pouce ▷ début de solution
   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.
3. Justifier que le nombre de comparaisons effectuées par `fusion(a, b)` est au plus `len(a) + len(b) - 1`. Quel est donc le coût de la fusion ? ▷ coup de pouce

Vers le bac ↑ cours

★★ **Exercice 6** *Coder un texte avec « diviser pour régner »* (d'après Métropole 2025, jour 2) ▶ corrigé p. 890 ≈ 45 min

Pour **compresser** un texte, on donne aux caractères **fréquents** un code binaire **court**. L'algorithme de **Shannon-Fano** est un « diviser pour régner » :

- on classe les caractères par effectif **décroissant** ;
- **diviser** : on coupe la liste en **deux groupes** d'effectif total à peu près égal (fonction `separe` ci-dessous) ; le premier groupe reçoit le bit `"1"`, le second le bit `"0"` ;
- **régner** : on recommence sur chaque groupe, jusqu'à ce qu'un groupe ne contienne **qu'un** caractère (condition d'arrêt).

On représente les données par une liste de couples `(caractère, effectif)`. On **admet** la fonction `separe`, qui coupe la liste triée en deux groupes d'effectifs à peu près équilibrés :

```python
def separe(tab):
    total = 0
    for (_, effectif) in tab:
        total = total + effectif
    k = 0
    cumul = tab[0][1]
    while cumul < total / 2:        # on coupe des que le cumul
        k = k + 1                   # atteint la moitie du total
        cumul = cumul + tab[k][1]
    return tab[:k + 1], tab[k + 1:]
```

On considère le texte dont les effectifs sont :

`tab = [("A", 15), ("B", 7), ("C", 6), ("D", 6), ("E", 5)]`.

1. Que renvoie `separe(tab)` ? (donner les deux groupes) Quel bit (`"1"` ou `"0"`) commence alors le code de `"A"` ? celui de `"C"` ? ▷ coup de pouce
2. ▶ Compléter la fonction récursive `shannon`, qui renvoie le code binaire d'un `symbole` : ▷ coup de pouce

```
def shannon(symbole, tab):
    if len(tab) == 1:               # condition d'arret : un seul caractere
        return ""
    groupe1, groupe2 = separe(tab)
    if symbole in [c for (c, _) in groupe1]:
        return "1" + ...            # a completer
    else:
        return "0" + ...            # a completer
```

3. ▶ Vérifier que l'on obtient les codes `A = 11`, `B = 10`, `C = 011`, `D = 010`, `E = 00`. **Dérouler à la main** le calcul de `shannon("C", tab)`.
4. Observer les cinq codes : montrer qu'**aucun n'est le début d'un autre**. Pourquoi est-ce indispensable pour **décoder** un texte sans ambiguïté ?
5. En quoi cet algorithme relève-t-il de « diviser pour régner » ? Identifier les trois temps.
6. **Terminaison** : justifier que la fonction `shannon` *s'arrête toujours* (quel « variant » diminue strictement à chaque appel récursif ?). ▷ coup de pouce
7. ▶ Écrire une fonction `encode_shannon(texte, tab)` qui renvoie le **texte codé** : la concaténation des codes `shannon` de chacun de ses caractères. Exemple : `encode_shannon("CAD", tab)` renvoie `"011"+"11"+"010" = "01111010"`. ▷ coup de pouce

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

> **Remarque** Ce sujet (Métropole 2025, jour 2) montre que « diviser pour régner » ne sert pas qu'à **trier** : il structure aussi la **compression** des données.

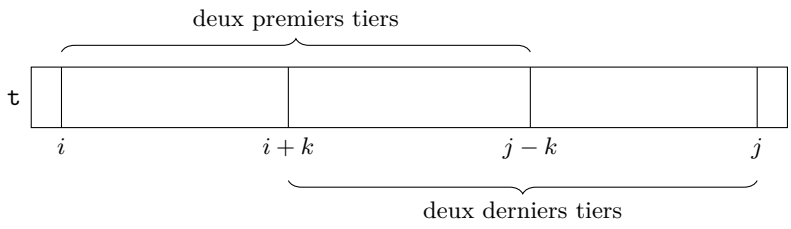
★★ Exercice 7 *Le tri de Stooge* (d'après Amérique du Nord 2024, jour 2)

▶ corrigé p. 891 ≈ 45 min

On se penche sur un algorithme pour trier un tableau appelé le **tri de Stooge**. Pour trier les éléments situés entre les indices i et j, où $i < j$, dans un tableau `t` par ce tri, on procède ainsi :

- si les éléments d'indice i et j sont mal placés, on les échange ;
- s'il y a au moins trois éléments de l'indice i à l'indice j inclus :
  - on trie les deux premiers tiers de cette portion avec cette méthode ;
  - on trie les deux derniers tiers de cette portion avec cette méthode ;
  - on trie à nouveau les deux premiers tiers de cette portion avec cette méthode.

Pour réaliser ce découpage en tiers, on considère l'entier `k` défini par l'expression `(j - i + 1) // 3`, et on considère les indices intermédiaires `i + k` et `j - k`.



Voici le code partiel du tri de Stooge en Python, qui trie les éléments d'un tableau par ordre croissant :

```
1 def triStooge(tab, i, j):
2     if tab[i] > tab[j]:
3         echange(tab, i, j)
4     if (j - i) > 1:
5         k = (j - i + 1) // 3
6         triStooge(...)
7         triStooge(...)
8         triStooge(...)
```

1. ▶ Écrire la fonction `echange(tab, i, j)` qui prend en arguments une liste Python `tab` et deux indices `i`, `j`, et réalise **sur place** l'échange des valeurs de `tab` à ces indices. La fonction ne renvoie rien.

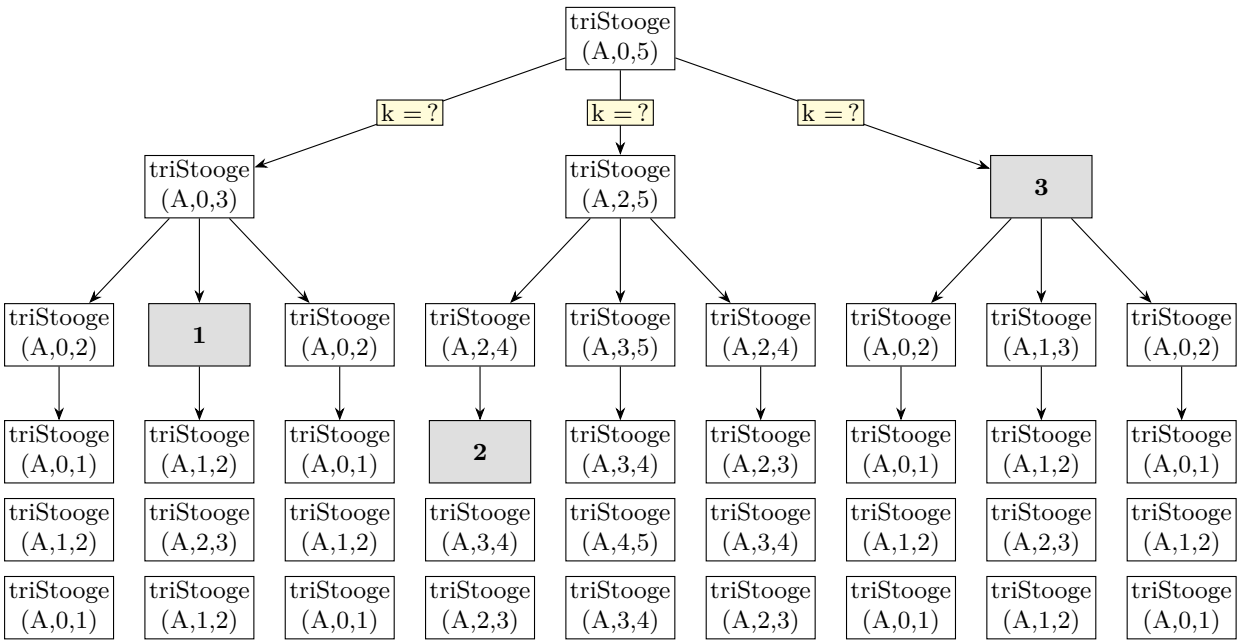
2. ▶ Finaliser le programme précédent en complétant les lignes 6, 7 et 8. ▷ coup de pouce

3. Indiquer, en le justifiant, si cet algorithme est itératif ou récursif.

Soit l'appel `triStooge(A, 0, 5)` avec `A = [5, 6, 4, 2, 3, 1]`.

4. Déterminer la valeur prise par `k` lors de ce premier appel. Une justification est attendue.

La figure ci-dessous présente l'arbre (incomplet) des appels récursifs effectués depuis l'appel `triStooge(A, 0, 5)`.



Arbre des appels récursifs pour `triStooge(A, 0, 5)` *(au dernier niveau, les trois appels issus d'un même nœud sont empilés verticalement).*

5. Dénombrer les appels récursifs effectués lors du tri, sans compter l'appel initial. ▷ coup de pouce

6. Déterminer les appels effectués dans les cases **1**, **2** et **3** de cet arbre des appels.

7. Dans cette question, `A = [5, 6, 4, 2]`. Recopier le tableau ci-dessous et le compléter (remplacer les `??`). ▷ coup de pouce

Appel	Valeur de A avant l'appel	Valeur de A après l'appel
`triStooge(A,0,3)`	`[5, 6, 4, 2]`	`??`
`??`	`[2, 6, 4, 5]`	`??`
`triStooge(A,1,3)`	`[2, 4, 6, 5]`	`??`
`??`	`[2, 4, 5, 6]`	`[2, 4, 5, 6]`

On montre que le coût en temps dans le pire des cas du tri de Stooge est de l'ordre de n^e, avec e environ égal à $\frac{8}{3}$.

8. Donner un algorithme de tri dont le coût est strictement meilleur.

★★★ **Exercice 8** *L'élément absolument majoritaire* (d'après Asie 2024, jour 2) ▶ corrigé p. 892 ≈ 60 min

On cherche à déterminer, s'il existe, l'élément **absolument majoritaire** d'une liste : un élément qui apparaît dans *strictement plus de la moitié* des emplacements de la liste.

Par exemple, la liste `[1, 4, 1, 6, 1, 7, 2, 1, 1]` admet `1` comme élément absolument majoritaire, car il apparaît 5 fois sur 9 éléments. En revanche, la liste `[1, 4, 6, 1, 7, 2, 1, 1]` n'admet pas d'élément absolument majoritaire : le plus fréquent est `1`, mais il n'apparaît que 4 fois sur 8, ce qui ne fait pas plus que la moitié.

1. Déterminer les effectifs possibles d'un élément absolument majoritaire dans une liste de taille 10.

Partie A — Calcul des effectifs sans dictionnaire. On peut déterminer l'éventuel élément absolument majoritaire d'une liste en calculant l'effectif de chacun de ses éléments.

2. ▶ Écrire une fonction `effectif` qui prend en paramètres une valeur `val` et une liste `lst` et qui renvoie le nombre d'apparitions de `val` dans `lst`. Il ne faut pas utiliser la méthode `count`. ▷ coup de pouce

3. Déterminer le nombre de comparaisons entre `val` et un élément de la liste effectuées par l'appel `effectif(1, [1, 4, 1, 6, 1, 7, 2, 1, 1])`.

4. ▶ En utilisant la fonction `effectif`, écrire une fonction `majo_abs1` qui prend en paramètre une liste `lst` et qui renvoie son élément absolument majoritaire s'il existe, `None` sinon.

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

5. On suppose que `majo_abs1` renvoie son résultat dès qu'un élément convient. Déterminer le nombre de comparaisons entre éléments de la liste effectuées par l'appel `majo_abs1([1, 4, 1, 6, 1, 7, 2, 1, 1])`.

Partie B — Calcul des effectifs dans un dictionnaire. Un autre algorithme consiste à stocker l'effectif partiel de chaque élément déjà rencontré dans un dictionnaire.

6. ▶ Recopier et compléter les lignes 3, 4, 5 et 7 de la fonction `eff_dico` suivante, qui prend en paramètre une liste `lst` et renvoie un dictionnaire dont les clés sont les éléments de `lst` et les valeurs les effectifs de ces éléments dans `lst`.

```
1 def eff_dico(lst):
2     dico_sortie = {}
3     for ......... :
4         if ... in dico_sortie:
5             ...
6         else:
7             ...
8     return dico_sortie
```

7. ▶ En utilisant la fonction `eff_dico`, écrire une fonction `majo_abs2` qui prend en paramètre une liste `lst` et qui renvoie son élément absolument majoritaire s'il existe, `None` sinon.

Partie C — Par la méthode « diviser pour régner ». Un dernier algorithme consiste à partager la liste en deux listes, à déterminer les éventuels éléments absolument majoritaires de chacune des deux listes, puis à **combiner** les résultats pour obtenir, s'il existe, l'élément absolument majoritaire de la liste initiale. On considère `lst` une liste de taille n.

8. Déterminer l'élément absolument majoritaire de `lst` si $n = 1$. C'est le cas de base (condition d'arrêt).

On suppose que l'on a partagé `lst` en deux listes : `lst1 = lst[:n//2]` (les `n//2` premiers éléments de `lst`) et `lst2 = lst[n//2:]` (les autres éléments).

9. Si ni `lst1` ni `lst2` n'admet d'élément absolument majoritaire, expliquer pourquoi `lst` n'admet pas d'élément absolument majoritaire. ▷ coup de pouce

10. Si `lst1` admet un élément absolument majoritaire `maj1`, donner un algorithme pour vérifier si `maj1` est l'élément absolument majoritaire de `lst`.

11. ▶ Recopier et compléter les lignes 4, 11, 13, 15 et 17 de la fonction récursive `majo_abs3` qui implémente l'algorithme précédent. On pourra utiliser la fonction `effectif` de la question 2.

▷ coup de pouce ▷ début de solution

```
1  def majo_abs3(lst):
2      n = len(lst)
3      if n == 1:
4          return ...
5      else:
6          lst_g = lst[:n//2]
7          lst_d = lst[n//2:]
8          maj_g = majo_abs3(lst_g)
9          maj_d = majo_abs3(lst_d)
10         if maj_g is not None:
11             eff = ......
12             if eff > n/2:
13                 return ...
14         if maj_d is not None:
15             eff = ......
16             if eff > n/2:
17                 return ...
```

★★ **Exercice 9** *Les deux points les plus proches* (d'après Polynésie 2023, jour 1) ▶ corrigé p. 893 ≈ 60 min

L'objectif est de trouver les deux points les plus proches dans un nuage de points dont on connaît les coordonnées dans un repère orthogonal. On rappelle que la distance entre deux points $A(x_A ; y_A)$ et $B(x_B ; y_B)$ est donnée par la formule :

$$AB = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}.$$

Les coordonnées d'un point sont stockées dans un tuple de deux nombres réels. Le nuage de points est représenté en Python par une liste de n tuples, n étant le nombre total de points. On suppose qu'il n'y a pas de points confondus et qu'il y a au moins deux points dans le nuage. Pour calculer la racine carrée, on utilise la fonction `sqrt` du module `math` :

```
>>> from math import sqrt
>>> sqrt(16)
4.0
```

1. Questions générales.

a) Donner le rôle de l'instruction `from math import sqrt`.

b) Expliquer le résultat suivant :

```
>>> 0.1 + 0.2 == 0.3
False
```

c) Expliquer l'erreur suivante :

```
>>> point_A = (3, 4)
>>> point_A[0]
3
>>> point_A[0] = 2
Traceback (most recent call last):
    File "<console>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

2. On définit la classe `Segment` ci-dessous :

```
1 from math import sqrt
2 class Segment:
3     def __init__(self, point1, point2):
4         self.p1 = point1
5         self.p2 = point2
6         self.longueur = ..............  # a completer
```

a) Recopier et compléter la ligne 6 du constructeur de la classe `Segment`.

La fonction `liste_segments` ci-dessous prend en paramètre une liste de points et renvoie une liste contenant les objets `Segment` qu'il est possible de construire à partir de ces points. Les segments $[AB]$ et $[BA]$ étant confondus, on n'ajoute qu'un seul objet dans la liste.

```
1 def liste_segments(liste_points):
2     n = len(liste_points)
3     segments = []
4     for i in range(..............):
5         for j in range(..............., n):
6             # On construit le segment a partir des points i et j.
7             seg = ..............................
8             segments.append(seg)  # On l'ajoute a la liste
9     return segments
```

b) ▶ Recopier la fonction sans les commentaires et compléter le code manquant.

▷ coup de pouce

c) Donner, en fonction de n, la longueur de la liste `segments`. Le résultat peut être laissé sous la forme d'une somme.

d) Donner, en fonction de n, la complexité en temps de la fonction `liste_segments`.

3. On veut maintenant écrire la fonction de recherche des deux points les plus proches par la méthode **diviser pour régner**. On dispose de deux fonctions `moitie_gauche` (respectivement `moitie_droite`) qui prennent en paramètre une liste et renvoient chacune une nouvelle liste contenant la moitié gauche (respectivement droite) de la liste de départ. Si le nombre d'éléments est impair, l'élément du centre se trouve dans la partie gauche.

```
>>> liste = [1, 2, 3, 4]
>>> moitie_gauche(liste)
[1, 2]
>>> moitie_droite(liste)
```

```
[3, 4]
>>> liste = [1, 2, 3, 4, 5]
>>> moitie_gauche(liste)
[1, 2, 3]
>>> moitie_droite(liste)
[4, 5]
```

▶ Écrire la fonction `plus_court_segment` qui prend en paramètre une liste d'objets `Segment` et renvoie l'objet `Segment` dont la longueur est la plus petite. On procédera de la façon suivante :

- tester si le cas de base (condition d'arrêt) est atteint, c'est-à-dire si la liste contient un seul segment ;
- découper la liste en deux listes de tailles égales (à une unité près) ;
- appeler récursivement la fonction pour rechercher le minimum dans chacune des deux listes ;
- comparer les deux valeurs récupérées et renvoyer la plus petite des deux.

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

4. On considère les trois points $A(3;4)$, $B(2;3)$ et $C(-3;-1)$.

a) Donner l'instruction Python permettant de construire la variable `nuage_points` contenant les trois points A, B et C.

b) ▶ En utilisant les fonctions de l'exercice, écrire les instructions Python qui affichent les coordonnées des deux points les plus proches du nuage de points `nuage_points`.

★★★ **Exercice 10** *Comparer deux classements* (sujet maison, façon bac) ▶ corrigé p. 894 ≈ 60 min

Cet exercice n'est pas un sujet officiel : il a été écrit au format du bac pour s'entraîner.

Une plateforme d'écoute musicale veut mesurer à quel point le goût d'un auditeur s'écarte du classement officiel des meilleures ventes. Les chansons sont désignées par leur **rang officiel** (1 pour la plus vendue, 2 pour la suivante, etc.). Le classement personnel d'un auditeur est la liste de ces rangs, dans l'ordre où il les préfère. Par exemple, l'auditeur dont le classement est `[2, 4, 1, 3, 5]` préfère la chanson de rang officiel 2, puis celle de rang 4, etc.

On appelle **inversion** d'une liste `t` tout couple d'indices `(i, j)` tel que `i < j` et `t[i] > t[j]` : deux chansons que l'auditeur range dans l'ordre inverse du classement officiel. Plus il y a d'inversions, plus l'auditeur est en désaccord avec le classement officiel.

Partie A — Une méthode directe.

1. Donner les inversions de `[2, 4, 1, 3, 5]` (sous la forme de couples de *valeurs*). Quel classement de cinq chansons ne contient aucune inversion ? Lequel en contient le plus, et combien ?

2. ▶ Écrire une fonction `inversions_naif(t)` qui renvoie le nombre d'inversions de la liste `t` en testant tous les couples d'indices `(i, j)` avec `i < j`. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

3. Donner, en fonction de la longueur n de la liste `t`, le nombre de comparaisons `t[i] > t[j]` effectuées par la fonction `inversions_naif`. Quel est le coût de cette fonction ? Estimer ce nombre pour une liste d'un million de chansons.

Partie B — Par la méthode « diviser pour régner ». On coupe la liste `t` en deux moitiés `gauche = t[:milieu]` et `droite = t[milieu:]`, avec `milieu = len(t) // 2`. Une inversion de `t` est alors soit une inversion de `gauche`, soit une inversion de `droite`, soit une inversion **croisée** : un couple formé d'un élément `a` de `gauche` et d'un élément `b` de `droite` avec `a > b`.

4. On prend `t = [5, 2, 6, 1, 4, 3]`. Donner `gauche` et `droite`, puis compter les inversions de `gauche`, celles de `droite` et les inversions croisées. En déduire le nombre d'inversions de `t`.

5. a) Justifier brièvement que le nombre d'inversions croisées ne change pas si l'on trie chacune des deux moitiés.

b) On suppose donc `gauche` et `droite` **triées**, et on les fusionne comme dans le tri fusion. Au moment où l'on place `droite[j]` dans le résultat parce que `droite[j] < gauche[i]`, justifier que `droite[j]` forme une inversion croisée avec exactement `len(gauche) - i` éléments de `gauche`.

▷ coup de pouce

6. ▶ Recopier et compléter la fonction `fusion_compte`, qui renvoie la liste fusionnée **et** le nombre d'inversions croisées.

```
def fusion_compte(gauche, droite):
    resultat = []
    croisees = 0
    i = j = 0
    while i < len(gauche) and j < len(droite):
        if gauche[i] <= droite[j]:
            resultat.append(gauche[i])
            i = i + 1
        else:
            resultat.append(droite[j])
            croisees = ...                    # a completer
            j = j + 1
    resultat = resultat + gauche[i:]
    resultat = resultat + droite[j:]
    return ...                                # a completer
```

7. ▶ Recopier et compléter la fonction récursive `tri_compte`, qui renvoie la liste `t` triée **et** son nombre d'inversions. Identifier dans ce code les trois temps « diviser », « régner » et « combiner ».

▷ coup de pouce ▷ début de solution

```
def tri_compte(t):
    if len(t) <= 1:                           # condition d'arret
        return ...                            # a completer
    milieu = len(t) // 2
    gauche, inv_g = ...                       # a completer
    droite, inv_d = ...                       # a completer
    fusionnee, inv_c = fusion_compte(gauche, droite)
    return ...                                # a completer
```

8. Dérouler l'appel `tri_compte([3, 1, 2])` : donner chaque appel récursif et la valeur qu'il renvoie.

9. Justifier que `tri_compte` se termine. Le coût de `tri_compte` est du même ordre que celui du tri fusion : lequel ? Comparer avec la Partie A pour un million de chansons (on prendra $\log_2(10^6) \approx 20$).

Critiquer une réponse d'IA ↑cours

★★ **Exercice 11** *Une recherche dichotomique écrite par un assistant d'IA* ▶ corrigé p. 896

Un élève demande à un assistant d'IA : « Écris une fonction récursive Python de recherche dichotomique qui renvoie `True` si `cible` est dans le tableau trié `tab`, entre les indices `debut` et `fin` inclus. » Voici la réponse obtenue :

```
def recherche(tab, cible, debut, fin):
    if debut > fin:                    # intervalle vide : absent
        return False
    milieu = (debut + fin) // 2
    if tab[milieu] == cible:
        return True
    elif cible < tab[milieu]:
        return recherche(tab, cible, debut, milieu)      # moitie gauche
    else:
        return recherche(tab, cible, milieu + 1, fin)    # moitie droite
```

« À chaque appel, on compare la cible à l'élément du milieu et on ne garde que la moitié utile : l'intervalle est divisé par deux, d'où un coût en $\log_2 n$. »

1. La réponse est-elle correcte ? ▶ Dérouler à la main `recherche([3, 8, 12, 19, 25], 12, 0, 4)`, puis `recherche([3, 8, 12, 19, 25], 5, 0, 4)` (noter à chaque appel les valeurs de `debut`, `fin` et `milieu`).
2. Localiser et corriger l'erreur. ▷ coup de pouce
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

S'entraîner à l'oral ↑cours

★★ **Exercice 12** *Expliquer en deux minutes* ▶ corrigé p. 896

Au Grand Oral comme devant l'examinateur de l'épreuve pratique, il faut savoir **expliquer** une notion clairement, sans lire ses notes. Choisir l'un des sujets suivants (ou le tirer au sort) :

Sujet 1. Expliquer les trois étapes de la méthode « **diviser pour régner** » sur l'exemple du tri fusion.

Sujet 2. Expliquer à un parent pourquoi le tri fusion est bien plus rapide que le tri par insertion sur un million de valeurs.

Sujet 3. Expliquer pourquoi la recherche dichotomique est un « diviser pour régner » et pourquoi elle exige un tableau **trié**.

Préparation (5 min, seul). Noter au brouillon **trois idées** et **un exemple**, rien de plus : comme au Grand Oral, ces notes servent d'aide-mémoire. **Exposé (2 min, chronométré).** Debout, face à un camarade, **sans lire ses notes** et sans rien écrire : tout passe par la parole. **Retour (2 min).** Le camarade pose une question de relance (on peut alors s'aider d'un schéma), remplit la grille ci-dessous et donne un conseil ; puis on échange les rôles. Cette grille reprend les critères d'évaluation du Grand Oral. ▷ coup de pouce

Grille entre camarades	Oui	En partie	Pas encore
Clair — audible, posé ; chaque mot technique est expliqué	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	☐	☐	☐
Construit — un fil conducteur, tenu en deux minutes, sans lire	☐	☐	☐
Un conseil pour la prochaine fois :			

Entraînement ↑cours

Des séries courtes pour rendre une méthode automatique : même consigne, données différentes. Les réponses sont brèves dans le corrigé.

★ **Exercice 13** *Arbres d'appels du tri fusion en série* ▶ corrigé p. 897 • essentiel

Pour chacune des listes ci-dessous, dessiner l'**arbre des appels** de `tri_fusion` (coupe à `milieu = len(tab) // 2`, jusqu'aux listes d'un élément), compter les appels (appel initial compris), puis donner, dans l'ordre de la remontée, chaque **fusion** effectuée et la liste obtenue. ▷ coup de pouce

a) `[7, 2, 9, 4]`
b) `[6, 1, 8, 3, 5]`
c) `[4, 10, 2, 7, 1, 9, 3]`

★ **Exercice 14** *Fusions à la main en série* ▶ corrigé p. 897 • essentiel

Fusionner **à la main** chacun des couples de listes triées ci-dessous : à chaque étape, noter les deux éléments comparés et celui qui est recopié. Donner la liste obtenue et le nombre de comparaisons. ▷ coup de pouce

a) `[2, 5, 8]` et `[1, 6, 7]`
b) `[1, 2, 3]` et `[4, 5]`
c) `[3, 9]` et `[1, 4, 10, 11]`

7

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Noter à chaque étape les bornes `g` et `d` de la zone de recherche et l'indice du milieu `(g + d) // 2`, puis ne garder que la moitié où la valeur peut encore se trouver. Pour la question 3 : combien de fois peut-on diviser 1000 par 2 avant d'arriver à 1 ? ↑énoncé

Ex. 2 – piste Que vaut le maximum d'un tableau d'un seul élément ? Les deux appels récursifs portent sur les deux moitiés de `t`, à écrire avec des tranches autour de `milieu`. Pour l'arbre, noter les valeurs renvoyées en remontant, des feuilles vers la racine. ↑énoncé

Ex. 3 q. 1 – piste Q1 : chaque liste de longueur au moins 2 est coupée en `tab[:milieu]` et `tab[milieu:]` ; attention, pour une longueur impaire, la moitié gauche est la plus courte. ↑énoncé

Ex. 3 q. 2 – piste Q2 : lire l'arbre de bas en haut : chaque nœud reçoit la fusion des deux listes triées de ses fils. ↑énoncé

Ex. 3 q. 3 – piste Q3 : que faut-il trier récursivement ? Exprimer chacune des deux moitiés avec une tranche de `tab` autour de `milieu`. ↑énoncé

Ex. 4 – piste Un appel sur une liste d'au moins deux éléments lance exactement deux appels, sur des listes deux fois plus courtes. Compter niveau par niveau : combien d'appels au niveau suivant pour chaque appel du niveau courant ? ↑énoncé

Ex. 5 q. 2 – piste Q2 : garder un indice par liste. Tant que **les deux** listes ont encore des éléments, comparer leurs « têtes » et recopier la plus petite. Que reste-t-il à faire quand l'une des deux est épuisée ? ↑énoncé

Ex. 5 q. 2 – début de solution Q2 :

```
resultat = []
i, j = 0, 0
while i < len(a) and j < len(b):
    if a[i] <= b[j]: ...
```

↑énoncé

Ex. 5 q. 3 – piste Q3 : chaque comparaison est suivie de la recopie d'un élément ; combien d'éléments au plus sont recopiés avant que l'une des listes soit vide ? ↑énoncé

Ex. 6 q. 1 – piste Q1 : calculer d'abord l'effectif total, puis cumuler les effectifs dans l'ordre jusqu'à atteindre la moitié de ce total. ↑énoncé

Ex. 6 q. 2 – piste Q2 : le code du symbole, c'est le bit de son groupe suivi du code du symbole *à l'intérieur de ce groupe* : sur quelle liste faire l'appel récursif ? ↑énoncé

Ex. 6 q. 6 – piste Q6 : comparer la longueur de `tab` à celle du groupe passé à l'appel récursif : chacun des deux groupes peut-il être vide ? ↑énoncé

Ex. 6 q. 7 – piste Q7 : partir d'une chaîne vide et la compléter en parcourant le texte caractère par caractère. ↑énoncé

Ex. 7 q. 2 – piste Q2 : relire la figure : entre quels indices se trouvent les « deux premiers tiers » ? les « deux derniers tiers » ? Dans quel ordre la méthode les trie-t-elle ? ↑énoncé

Ex. 7 q. 5 – piste Q5 : combien d'appels un nœud qui n'est pas une feuille déclenche-t-il ? Compter niveau par niveau. ↑énoncé

Ex. 7 q. 7 – piste Q7 : calculer `k` pour l'appel `triStooge(A,0,3)` et en déduire ses trois appels (lignes 2 à 4 du tableau). Parmi ces trois appels, la valeur « après » d'une ligne est la valeur « avant » de la suivante ; la valeur « après » de la ligne 1 est celle de `A` une fois les trois appels terminés. ↑énoncé

Ex. 8 q. 2 – piste Q2 : un compteur initialisé à 0 et une boucle sur les éléments de `lst`. ↑énoncé

Ex. 8 q. 9 – piste Q9 : raisonner par l'absurde : si un élément apparaît au plus la moitié des fois dans `lst1` **et** dans `lst2`, que dire de son effectif dans `lst` ? ↑énoncé

Ex. 8 q. 11 – piste Q11 : un candidat majoritaire d'une moitié doit être recompté dans la liste **entière** `lst`, pas seulement dans sa moitié. ↑énoncé

Ex. 8 q. 11 – début de solution Q11, ligne 4 : une liste d'un seul élément a cet élément pour majoritaire, `return lst[0]`. Ligne 11 : `eff = effectif(maj_g, lst)`. ↑énoncé

Ex. 9 q. 2b – piste Q2 b : pour ne construire $[AB]$ qu'une fois, l'indice `j` doit toujours être strictement plus grand que `i` ; que doit valoir `i` au plus ? ↑énoncé

Ex. 9 q. 3 – piste Q3 : on compare les segments par leur attribut `longueur`, mais on renvoie l'**objet** `Segment` lui-même. ↑énoncé

Ex. 9 q. 3 – début de solution Q3 :

```
def plus_court_segment(segments):
    if len(segments) == 1:
        return segments[0]
    seg_g = plus_court_segment(moitie_gauche(segments)) ...
```
↑

Ex. 10 q. 2 – piste Q2 : deux boucles imbriquées, la seconde commençant juste après l'indice de la première. ↑énoncé

Ex. 10 q. 5 – piste Q5 b) : `gauche` est triée : comparer `droite[j]` à `gauche[i]`, `gauche[i+1]`, … D'autre part, les éléments `gauche[0]`, …, `gauche[i-1]` déjà placés sont-ils plus grands ou plus petits que `droite[j]` ? ↑énoncé

Ex. 10 q. 7 – piste Q7 : le nombre total d'inversions est la somme de trois nombres : relire l'introduction de la Partie B. ↑énoncé

Ex. 10 q. 7 – début de solution Q7 : condition d'arrêt : une liste d'au plus un élément est triée et n'a aucune inversion, `return t, 0`. Puis `gauche, inv_g = tri_compte(t[:milieu])`. ↑énoncé

Ex. 11 q. 2 – piste Suivre `debut`, `fin` et `milieu` lors de la recherche de `5` : que se passe-t-il quand `debut == fin` et que la cible est plus petite que `tab[milieu]` ? ↑énoncé

Ex. 12 – piste Structurer chaque explication en trois temps : une **définition** en une phrase, un **exemple** petit et concret déroulé à voix haute, puis le **piège** (l'erreur que l'auditeur risque de faire). Finir par une phrase qui répond à la question posée. Sujet 2 : comparer deux nombres frappants plutôt que deux formules ; combien de fois peut-on couper un million en deux avant d'arriver à 1 ? ↑énoncé

Ex. 13 – piste Même méthode qu'à l'exercice 3 : chaque liste d'au moins deux éléments est coupée en `tab[:milieu]` et `tab[milieu:]` (pour une longueur impaire, la moitié gauche est la plus courte). Pour la remontée, chaque nœud reçoit la fusion des deux listes triées de ses fils, en commençant par le bas de l'arbre. ↑énoncé

Ex. 14 – piste Garder un doigt sur la tête de chaque liste : comparer les deux têtes, recopier la plus petite et avancer dans sa liste. Quand une liste est épuisée, que reste-t-il à faire, et faut-il encore comparer ? ↑énoncé

TP ET PROJETS

TP 1 Pourquoi le tri fusion coûte $n \log_2 n$? ▶ corrigé p. 898

Remarque On cherche à **établir** (et pas seulement à admettre) le coût du tri fusion. On raisonne en comptant le **nombre d'opérations élémentaires** (comparaisons, ajouts) en fonction de la taille n de la liste. On rappelle les deux fonctions du cours (la fin de `fusion` est écrite ici avec deux boucles, pour bien compter) :

```
1  def fusion(gauche, droite):        # gauche et droite sont deja triees
2      resultat = []
3      i = j = 0
4      while i < len(gauche) and j < len(droite):
5          if gauche[i] <= droite[j]:
6              resultat.append(gauche[i])
7              i = i + 1
8          else:
9              resultat.append(droite[j])
10             j = j + 1
11     while i < len(gauche):         # reste de gauche
12         resultat.append(gauche[i])
13         i = i + 1
14     while j < len(droite):         # reste de droite
15         resultat.append(droite[j])
16         j = j + 1
17     return resultat
18
19 def tri_fusion(tab):
20     if len(tab) <= 1:              # condition d'arret : deja trie
21         return tab
22     milieu = len(tab) // 2
23     gauche = tri_fusion(tab[:milieu])
24     droite = tri_fusion(tab[milieu:])
25     return fusion(gauche, droite)
```

1 Étape 1 — le coût d'*une* fusion

1. Dans `fusion`, chaque tour de boucle (des trois boucles `while`) exécute **exactement un** `append`. Si `gauche` et `droite` ont pour tailles p et q, combien y a-t-il d'éléments dans `resultat` à la fin ? En déduire le nombre total d'`append`.

2. Conclure : fusionner deux listes triées dont les tailles totalisent n éléments coûte de l'ordre de combien d'opérations ? Ce coût est-il **linéaire** ou **quadratique** ?

2 Étape 2 — l'arbre des appels récursifs

Pour simplifier, on suppose que n est une **puissance de 2** : $n = 2^k$. À chaque appel, `tri_fusion` coupe la liste en **deux moitiés**, jusqu'à obtenir des listes de taille 1.

1. Recopier et compléter sur le cahier le tableau décrivant les appels, niveau par niveau (le niveau 0 est l'appel initial) :

Niveau	Nombre de sous-listes	Taille de chaque sous-liste
0	1	n
1	2	$n/2$
2	…	…
i	…	…

2. À un niveau i donné, on fusionne toutes les sous-listes de ce niveau. En utilisant l'étape 1, montrer que le **coût total des fusions d'un niveau** vaut environ n, *et que ce coût est le même à tous les niveaux.*

3. Jusqu'à quel niveau descend-on ? (Autrement dit : pour quelle valeur de i les sous-listes ont-elles une taille de 1 ?) En déduire, en fonction de n, le **nombre de niveaux où ont lieu des fusions** (le niveau des listes de taille 1 n'en fait aucune).

4. Rassembler : coût total = (coût d'un niveau) × (nombre de niveaux). En déduire le coût du tri fusion en fonction de n.

3 Étape 3 — interpréter le résultat

1. Pour $n = 1024 = 2^{10}$, donner l'ordre de grandeur de $n \log_2 n$ et de n^2. Combien de fois le tri fusion est-il plus rapide, environ ?

2. Ce raisonnement donne le coût **dans tous les cas** (on n'a fait aucune hypothèse sur l'ordre des éléments). Pourquoi est-ce un avantage du tri fusion sur le tri rapide (dont le coût peut atteindre n^2 au pire) ?

> **Remarque** — *Aller plus loin* — hors programme On peut résumer tout le raisonnement par une **relation de récurrence** sur le coût $T(n)$:
>
> $$T(n) = \underbrace{2\,T(n/2)}_{\text{trier les deux moitiés}} + \underbrace{c\,n}_{\text{la fusion}} .$$
>
> « Dérouler » cette relation revient exactement à parcourir l'arbre des appels de l'étape 2, et redonne $T(n)$ de l'ordre de $n \log_2 n$.

TP 2 Tri rapide (*Quicksort*) ▶ corrigé p. 899

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t07-tp-tri-rapide` — **archive .zip**

Mode d'emploi

- **Objectif** : implémenter le **tri rapide**, un autre algorithme « diviser pour régner », puis le **comparer au tri fusion** et analyser leurs performances.
- Prérequis : le **cours** « Diviser pour régner » (paradigme, tri fusion, coût).
- Niveau : ★ échauffement ★★ classique ★★★ défi.
- ▶ *À programmer et tester* en Python (console ou éditeur en ligne).
- **Fichier à télécharger** (lien ci-dessus) : `tri_rapide.py` (la **trame** des fonctions à écrire **et** un jeu de tests — `python3 tri_rapide.py`).
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

1 Le principe

Le tri rapide applique « diviser pour régner », mais divise **autrement** que le tri fusion : au lieu de couper au milieu, il choisit un élément **pivot** et range les autres autour de lui.

- **Diviser** : choisir un **pivot**, puis **partitionner** — les éléments *inférieurs ou égaux* au pivot à gauche, les *supérieurs* à droite. Le pivot est alors à sa **position définitive**.
- **Régner** : trier récursivement le sous-tableau de gauche, puis celui de droite.
- **Combiner** : rien à faire ! Une fois les deux côtés triés autour du pivot, le tableau entier est trié.

Exemple
Pour `[10, 7, 8, 9, 1, 5]` avec le pivot `5` (dernier élément), après partition on obtient par exemple `[1, 5, 8, 9, 10, 7]` : le pivot `5` est en position `1`, tout ce qui est à sa gauche lui est $\leqslant$, tout ce qui est à sa droite lui est $>$.

2 Implémentation du tri rapide

★ **Exercice 1** *La fonction* `partition`

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Écrivez la fonction `partition(tab, bas, haut)` qui partitionne les éléments de `tab` d'indices `bas` à `haut` (inclus) et renvoie l'indice final du pivot. Suivez ces étapes :

1. choisir le **dernier** élément `tab[haut]` comme pivot ;
2. poser `i = bas - 1` : `i` sera l'indice du dernier élément $\leqslant$ pivot déjà rangé à gauche ;
3. parcourir les indices `j` de `bas` à `haut - 1` (on ne parcourt pas le pivot) ;
4. chaque fois que `tab[j]` $\leqslant$ pivot : augmenter `i` de 1, *puis* échanger `tab[i]` et `tab[j]` ;
5. à la fin, échanger `tab[i + 1]` et `tab[haut]` (le pivot vient juste après `i`) et renvoyer `i + 1`.

★★ **Exercice 2** *La fonction récursive* `tri_rapide`

En utilisant `partition`, écrivez la fonction récursive `tri_rapide(tab, bas, haut)` qui trie `tab` entre les indices `bas` et `haut`. *Réflexe « diviser pour régner » :* quelle est la **condition d'arrêt** ? sur quels sous-tableaux se font les deux appels récursifs ?

▶ Testez :

```
tab = [10, 7, 8, 9, 1, 5]
tri_rapide(tab, 0, len(tab) - 1)
print("Tableau trie :", tab)   # Doit afficher [1, 5, 7, 8, 9, 10]
```

★★ **Exercice 3** *Rappel : le tri fusion*

Voici une implémentation du tri fusion qui modifie `tab` directement, au lieu de renvoyer un nouveau tableau comme dans le cours ; elle recopie toutefois les deux moitiés dans `gauche` et `droite`. Relisez-la et vérifiez qu'elle correspond bien aux trois temps du cours.

```
def tri_fusion(tab):
    if len(tab) > 1:
        milieu = len(tab) // 2
        gauche = tab[:milieu]
        droite = tab[milieu:]
        tri_fusion(gauche)
        tri_fusion(droite)
        i = 0                    # indice dans gauche
        j = 0                    # indice dans droite
        k = 0                    # indice dans tab
        while i < len(gauche) and j < len(droite):   # fusion
            if gauche[i] <= droite[j]:
                tab[k] = gauche[i]
                i = i + 1
            else:
                tab[k] = droite[j]
                j = j + 1
            k = k + 1
        while i < len(gauche):   # reste de gauche
            tab[k] = gauche[i]
            i = i + 1
            k = k + 1
        while j < len(droite):   # reste de droite
            tab[k] = droite[j]
            j = j + 1
            k = k + 1
```

3 Comparaison des performances

★★ **Exercice 4** *Mesurer et tracer*

1. Écrivez `tableau_aleatoire(n)` qui renvoie un tableau de `n` entiers aléatoires entre `0` et `n` (module `random`).
2. À l'aide du module `time`, mesurez le temps d'exécution des deux tris pour des tailles croissantes ($n = 100, 1000, 10000, 100000$).
3. Tracez un graphique comparatif avec `matplotlib`.

```
import time
import random
import matplotlib.pyplot as plt

tailles = [100, 1000, 10000, 100000]
temps_rapide = []
```

```python
temps_fusion = []
for n in tailles:
    t1 = tableau_aleatoire(n)
    t2 = [x for x in t1]     # copie : les deux tris recoivent le meme tableau
    debut = time.time()
    tri_rapide(t1, 0, len(t1) - 1)
    temps_rapide.append(time.time() - debut)
    debut = time.time()
    tri_fusion(t2)
    temps_fusion.append(time.time() - debut)

plt.plot(tailles, temps_rapide, label="Tri rapide")
plt.plot(tailles, temps_fusion, label="Tri fusion")
plt.xlabel("Taille du tableau")
plt.ylabel("Temps d'execution (secondes)")
plt.legend()
plt.show()
```

4 Analyser les résultats

★ **Exercice 5** *Lire le graphique*

- Quel algorithme est le plus rapide pour les **petites** tailles de tableau ?
- Et pour les **grandes** tailles ?

★★ **Exercice 6** *L'importance du pivot*

Testez différentes stratégies de choix du pivot et comparez : premier élément, dernier élément, élément **du milieu** (`tab[(bas + haut) // 2]`), élément **au hasard**. Astuce : échangez d'abord l'élément choisi avec `tab[haut]`, puis appelez la même `partition`. Que se passe-t-il pour le tri rapide sur un tableau **déjà trié** si le pivot est le dernier élément ? (Essayez $n = 500$ puis $n = 2\,000$: quel message Python affiche-t-il ?)

Coût des deux tris

- **Tri rapide** : $O(n \log_2 n)$ en moyenne et dans le meilleur cas, mais $O(n^2)$ dans le **pire cas** (pivot mal choisi, tableau déjà trié). Avantage : il trie **sur place**, sans recopier.
- **Tri fusion** : **toujours** $O(n \log_2 n)$, y compris au pire cas, mais il faut de la **mémoire supplémentaire** pour la fusion.

★★★ **Exercice 7** *Questions de réflexion*

- Pourquoi le choix du pivot est-il déterminant pour les performances du tri rapide ?
- Dans quel cas préférer le tri fusion au tri rapide ?
- Pourquoi le tri rapide est-il *souvent* plus rapide que le tri fusion **en pratique**, malgré un pire cas moins bon ? (indice : « sur place », constantes cachées, cache mémoire.)

★★★ **Exercice 8** *Pour aller plus loin : un tri rapide qui renvoie une nouvelle liste*

1. ▶ Écrivez une fonction récursive `tri_rapide_liste(t)` qui renvoie une **nouvelle** liste triée, sans modifier `t` et sans fonction `partition` : on prend pour pivot le dernier élément, on construit **par compréhension** la liste `petits` des autres éléments `<=` pivot et la liste `grands` des éléments `>` pivot, puis on combine.
2. Comparez avec `tri_rapide` : quelle version est la plus facile à lire ? Laquelle utilise le plus de mémoire, et pourquoi ?

Jeu de tests — à faire passer

Le fichier `tri_rapide.py` contient la **trame** des fonctions `partition`, `tri_rapide` et `tableau_aleatoire` (à compléter), la fonction `tri_fusion` de l'exercice 3 (fournie) **et** une batterie de tests. Lancez-le : `python3 tri_rapide.py` affiche, pour chaque fonction, `OK` si elle est juste, `A FAIRE ou ECHEC` sinon. Vérifications essentielles :

```
# partition : le pivot est bien place, gauche <= pivot < droite
t = [10, 7, 8, 9, 1, 5]
p = partition(t, 0, len(t) - 1)      # p doit valoir 1
# t[0], ..., t[p - 1] sont <= t[p] et t[p + 1], ... sont > t[p]

tab = [5, 4, 3, 2, 1]
tri_rapide(tab, 0, len(tab) - 1)     # tab doit valoir [1, 2, 3, 4, 5]
tab = [3, 3, 3]
tri_rapide(tab, 0, len(tab) - 1)     # doublons : [3, 3, 3]
```

Le script teste aussi les cas limites (tableau vide, un seul élément) et compare le résultat à `sorted()` sur **200 tableaux aléatoires** : il suffit d'une seule différence pour que le test affiche `A FAIRE ou ECHEC`.

7

TP 3 Le *Tim sort*, le tri de Python

▶ corrigé p. 902

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t07-tp-tim-sort` — **archive .zip**

Mode d'emploi

- **Objectif** : découvrir et implémenter le **Tim sort**, l'algorithme de tri utilisé par Python, qui **combine** tri par insertion et tri fusion.
- Prérequis : le **cours** « Diviser pour régner » (tri fusion, fusion de deux listes triées).
- Niveau : ★ échauffement ★★ classique ★★★ défi.
- ▶ *À programmer et tester* en Python (console ou éditeur en ligne).
- **Fichier à télécharger** (lien ci-dessus) : `tim_sort.py` (la **trame** à trous des fonctions **et** un jeu de tests — `python3 tim_sort.py`).
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

1 Le principe

Le **Tim sort** est le tri utilisé par défaut en Python pour `sorted()` et `list.sort()`. Conçu par **Tim Peters** en 2002 (l'auteur du *Zen de Python*), il est redoutablement efficace sur les données *réelles* car il exploite les portions **déjà triées**.

L'idée : au lieu de découper jusqu'à des singletons comme le tri fusion, on s'arrête plus tôt et on trie de **petits blocs** avec le tri par insertion (très rapide sur peu d'éléments et sur des données presque triées), puis on **fusionne** ces blocs comme le tri fusion.

Tim sort : les deux temps

1. découper le tableau en **blocs** (appelés *runs*) de taille fixe (ici `64`) et trier chaque bloc par **insertion** ;
2. **fusionner** les blocs deux à deux, comme dans le tri fusion, jusqu'à n'en avoir plus qu'un.

Remarque

Rappel des coûts : le **tri par insertion** est en $O(n^2)$ en général, mais $O(n)$ si le tableau est **déjà presque trié** — d'où son intérêt sur les petits blocs. Le **tri fusion** est toujours en $O(n \log_2 n)$.

2 Les briques de base

★ **Exercice 1** *Le tri par insertion sur un sous-tableau*

Complétez `tri_insertion(tab, debut, fin)`, qui trie `tab` entre les indices `debut` et `fin` inclus :

```
def tri_insertion(tab, debut, fin):
    for i in range(debut + 1, fin + 1):
        cle = tab[i]                    # element a inserer
        j = i - 1
        while j >= debut and tab[j] > ...:     # Completer la condition
            tab[j + 1] = ...
            j = j - 1
        tab[j + 1] = ...
```

▶ Testez :

```
tab = [12, 11, 13, 5, 6]
tri_insertion(tab, 0, len(tab) - 1)
print("Tableau trie :", tab)    # Doit afficher [5, 6, 11, 12, 13]
```

★★ **Exercice 2** *Fusionner deux sous-tableaux triés*

Complétez `fusion_en_place(tab, debut, milieu, fin)`, qui fusionne les deux sous-tableaux triés `tab[debut..milieu]` et `tab[milieu+1..fin]`. *C'est la « fusion » du cours, dont le résultat est rangé directement dans* `tab` *(les deux moitiés sont tout de même recopiées dans* `gauche` *et* `droite`*).*

```
def fusion_en_place(tab, debut, milieu, fin):
    nb_gauche = milieu - debut + 1
    nb_droite = fin - milieu
    gauche = tab[debut:milieu + 1]
    droite = tab[milieu + 1:fin + 1]
    i = 0
    j = 0
    k = debut
    while i < nb_gauche and j < nb_droite:
        if gauche[i] <= droite[j]:
            tab[k] = ...
            i = i + 1
        else:
            tab[k] = ...
            j = j + 1
        k = k + 1
    while i < nb_gauche:
        tab[k] = ...
        i = i + 1
        k = k + 1
```

```
    while j < nb_droite:
        tab[k] = ...
        j = j + 1
        k = k + 1
```

▶ Testez :

```
tab = [11, 12, 13, 5, 6, 7]
fusion_en_place(tab, 0, 2, 5)
print("Tableau fusionne :", tab)    # Doit afficher [5, 6, 7, 11, 12, 13]
```

3 Assembler le Tim sort

★★ **Exercice 3** *La fonction* `tim_sort`

Complétez `tim_sort` en utilisant `tri_insertion` et `fusion_en_place`. Le paramètre `taille_bloc` (taille des blocs ou *runs*, 64 par défaut) permet de tester sur de petits tableaux ; la fonction trie `tab` sur place et le renvoie :

```
def tim_sort(tab, taille_bloc=64):
    n = len(tab)
    for debut_bloc in range(0, n, taille_bloc):     # 1. trier chaque bloc
        fin_bloc = min(debut_bloc + taille_bloc - 1, n - 1)
        tri_insertion(tab, ..., ...)                # Completer les arguments
    taille = taille_bloc                            # 2. fusionner deux a deux
    while taille < n:
        for debut in range(0, n, 2 * taille):
            milieu = min(debut + taille - 1, n - 1)
            fin = min(debut + 2 * taille - 1, n - 1)
            if milieu < fin:
                fusion_en_place(tab, ..., ..., ...) # Completer les arguments
        taille = 2 * taille
    return tab
```

▶ Testez :

```
tab = [5, 21, 7, 23, 19, 100, 8, 12, 22, 15]
tim_sort(tab)
print("Trie :", tab)    # [5, 7, 8, 12, 15, 19, 21, 22, 23, 100]
```

4 Comparaison et analyse

★★ **Exercice 4** *Trois tris en course*

Reprenez `tableau_aleatoire`, `tri_rapide` et `tri_fusion` (TP « Tri rapide » ci-dessus), puis comparez les temps du **Tim sort**, du tri rapide et du tri fusion pour $n = 100, 1000, 10000, 100000$.

```
import time
import random
import matplotlib.pyplot as plt

tailles = [100, 1000, 10000, 100000]
temps_tim = []
temps_rapide = []
temps_fusion = []
for n in tailles:
    t1 = tableau_aleatoire(n)
    t2 = [x for x in t1]      # copies : les trois tris recoivent le meme tableau
    t3 = [x for x in t1]
    debut = time.time()
    tim_sort(t1)
    temps_tim.append(time.time() - debut)
    debut = time.time()
```

```
    tri_rapide(t2, 0, len(t2) - 1)
    temps_rapide.append(time.time() - debut)
    debut = time.time()
    tri_fusion(t3)
    temps_fusion.append(time.time() - debut)

plt.plot(tailles, temps_tim, label="Tim sort")
plt.plot(tailles, temps_rapide, label="Tri rapide")
plt.plot(tailles, temps_fusion, label="Tri fusion")
plt.xlabel("Taille du tableau")
plt.ylabel("Temps d'execution (secondes)")
plt.legend()
plt.show()
```

★ **Exercice 5** *Lire les résultats*

- Quel algorithme est le plus rapide pour les **petites** tailles ? pour les **grandes** ?
- Refaites la mesure sur un tableau **déjà trié** (par exemple `[k for k in range(n)]`). *Attention :* sur un tel tableau, notre `tri_rapide` (pivot = dernier élément) dépasse la profondeur de récursion autorisée par Python dès $n \approx 1\,000$ (`RecursionError`) : c'est un résultat, pas une erreur de votre code. Retirez-le de la course pour cette mesure, puis répondez : que constatez-vous pour le Tim sort ?

★★★ **Exercice 6** *Questions de réflexion*

- Pourquoi le Tim sort utilise-t-il le tri par **insertion** pour les petits blocs ?
- En quoi la taille des *runs* influence-t-elle les performances ?
- Le vrai Tim sort détecte des *runs* de taille **variable** (les portions déjà triées des données réelles). En quoi est-ce plus malin que notre version à *runs* de taille fixe ?

Jeu de tests — à faire passer

Le fichier `tim_sort.py` contient la **trame à trous** de `tri_insertion`, `fusion_en_place` et `tim_sort` (les mêmes que ci-dessus) **et** une batterie de tests. Lancez-le : `python3 tim_sort.py` affiche, pour chaque fonction, `OK` si elle est juste, `A FAIRE ou ECHEC` sinon. Vérifications essentielles :

```
b = [5, 2, 9, 1, 3]
tri_insertion(b, 0, len(b) - 1)      # b doit valoir [1, 2, 3, 5, 9]

m = [1, 4, 7, 2, 3, 9]               # deux moities triees
fusion_en_place(m, 0, 2, 5)          # m doit valoir [1, 2, 3, 4, 7, 9]

tim_sort([5, 4, 3, 2, 1], 2)         # blocs de 2 : [1, 2, 3, 4, 5]
```

Le script compare aussi `tim_sort` à `sorted()` sur **200 tableaux aléatoires** (avec de petits blocs pour forcer les fusions) : il suffit d'une seule différence pour que le test affiche `A FAIRE ou ECHEC`.

TP 4 Faire tourner une image ▶ corrigé p. 905

rotation d'un quart de tour « diviser pour régner »

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t07-tp-rotation-image` — **archive .zip**

Mode d'emploi

- **Objectif** : faire pivoter une image d'un **quart de tour** avec un algorithme récursif « diviser pour régner », **en place** (sans deuxième image) — l'exemple cité par le programme.
- Prérequis : le **cours** « Diviser pour régner », et savoir qu'une image est un **tableau de pixels**.
- Niveau : ★ échauffement ★★ classique ★★★ défi.
- ▶ *À programmer et tester* en Python avec la bibliothèque **Pillow** (`PIL`), console ou éditeur en ligne. On travaille sur le fichier à télécharger `joconde.png`, une image **carrée** de 256 × 256 pixels.
- **Fichier à télécharger** (lien ci-dessus) : `rotation_image.py` (la **trame à trous** de la fonction `rotation` **et** un jeu de tests — `python3 rotation_image.py`).
- Usage de l'IA, indiqué sur chaque exercice :  aucune ;  en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ;  l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

1 Une image, c'est un tableau de pixels

Une image numérique est une **grille de pixels**. Pour une image carrée de côté `n`, on la représente par un **tableau à deux dimensions** : `image[i][j]` est le pixel de la ligne `i`, colonne `j` (un triplet rouge-vert-bleu).

Le code suivant charge l'image fournie et construit ce tableau `image`. *À recopier tel quel :*

```
from PIL import Image

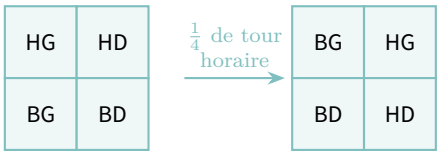
im = Image.open("joconde.png").convert("RGB")
n = im.width                                  # image carree n x n (ici 256)
pixels = [p for p in im.getdata()]            # tous les pixels, ligne par ligne
image = [pixels[k*n:(k+1)*n] for k in range(n)] # tableau 2D : image[i][j]
```

2 L'idée « diviser pour régner »

Pour tourner l'image d'un quart de tour (sens des aiguilles d'une montre) :

- **Diviser** : couper l'image carrée en **quatre quadrants** de taille moitié (haut-gauche, haut-droite, bas-gauche, bas-droite).
- **Régner** : faire tourner **récursivement** chacun des quatre quadrants.
- **Combiner** : **permuter** les quatre blocs d'un quart de tour (le bloc haut-gauche va en haut-droite, etc.).

La **condition d'arrêt** : un quadrant de **1 pixel** n'a rien à faire (il est déjà « tourné »).



Permutation des blocs (rotation horaire), après que chaque bloc a été tourné sur lui-même.

Remarque

On ne crée **aucune** image supplémentaire : tous les pixels sont déplacés *à l'intérieur* du tableau `image`. Le coût en **mémoire** reste donc **constant** (à part la pile des appels) — c'est précisément la propriété que le programme met en avant.

3 À vous de jouer

★ **Exercice 1** *Sur papier : une petite image 4×4*

On note les pixels par des nombres. Faites tourner cette image d'un **quart de tour horaire** et écrivez sur le cahier le tableau obtenu.

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

★★ **Exercice 2** *Compléter la fonction récursive*

Complétez la fonction `rotation` ci-dessous : les **deux appels récursifs** manquants, puis les **deux affectations** manquantes de la permutation des blocs.

```
def rotation(image, ligne, col, taille):
    if taille == 1:              # condition d'arret : un seul pixel
        return
    demi = taille // 2
    # 1. tourner recursivement les 4 quadrants
    rotation(image, ligne,        col,        demi)   # haut-gauche
    rotation(image, ...,          ...,        demi)   # haut-droite  <- a completer
    rotation(image, ...,          ...,        demi)   # bas-gauche   <- a completer
    rotation(image, ligne + demi, col + demi, demi)   # bas-droite
    # 2. permuter les blocs (rotation horaire) : HG -> HD -> BD -> BG -> HG
    for i in range(demi):
        for j in range(demi):
            hg = image[ligne + i][col + j]
            hd = image[ligne + i][col + demi + j]
            bd = image[ligne + demi + i][col + demi + j]
            bg = image[ligne + demi + i][col + j]
            image[ligne + i][col + demi + j]        = hg    # HD <- HG
            image[ligne + demi + i][col + demi + j] = ...   # a completer
            image[ligne + demi + i][col + j]        = ...   # a completer
            image[ligne + i][col + j]               = bg    # HG <- BG
```

★ **Exercice 3** *Lancer et enregistrer le résultat*

1. Complétez l'**appel initial** pour tourner l'image entière :

```
rotation(image, ..., ..., ...)   # a completer
```

2. Recopiez ce code qui reconstruit et enregistre l'image tournée, puis ▶ vérifiez le fichier obtenu.

```
plat = []                         # on remet les pixels a plat
for ligne in image:
    for pixel in ligne:
        plat.append(pixel)
sortie = Image.new("RGB", (n, n))
```

```
6 sortie.putdata(plat)
7 sortie.save("joconde_tournee.png")
```




À gauche l'image de départ, à droite le résultat attendu après un quart de tour horaire.

★★ **Exercice 4** *Pourquoi « en place » ?*

- Combien d'images supplémentaires cet algorithme crée-t-il ? Que vaut donc son coût **en mémoire** ?
- Où sont stockés les **4 pixels** `hg`, `hd`, `bd`, `bg` le temps de l'échange ? Pourquoi cette petite « réserve » ne remet-elle pas en cause la mémoire constante ?

★★★ **Exercice 5** *Pour aller plus loin*

- Cet algorithme suppose que `n` est une **puissance de 2**. Pourquoi ? Que faudrait-il gérer si ce n'était pas le cas (essayez de couper une image 6×6) ?
- Comment modifieriez-vous la permutation pour tourner dans le sens **anti-horaire** ?
- *Bonus* : en comptant les déplacements de pixels, l'algorithme est en $O(n^2 \log_2 n)$ en **temps** (plus que la simple recopie en $O(n^2)$). Son intérêt est donc surtout la **mémoire** : expliquez en une phrase ce compromis.

★★ **Exercice 6** *La simple recopie*

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1. Sur l'image 4×4 de l'exercice 1, de quelle case de l'image de départ vient le pixel ligne `0`, colonne `0` de l'image tournée ? et celui ligne `1`, colonne `3` ? Déduisez-en, pour une image $n \times n$, la ligne et la colonne de départ du pixel ligne `i`, colonne `j` de l'image tournée.
2. ▶ Écrivez une fonction `rotation_copie(image)` qui **renvoie** une nouvelle image tournée d'un quart de tour horaire, construite **par compréhension**, sans modifier `image`. Vérifiez sur l'image 4×4 qu'on obtient le même résultat qu'avec la fonction récursive.
3. Comparez les deux méthodes : nombre d'écritures de pixels, mémoire utilisée.

Jeu de tests — à faire passer

Le fichier `rotation_image.py` travaille sur une image représentée par un **tableau 2D** (sans `PIL`), pour pouvoir être testé partout. Il contient la fonction `rotation` **à trous** (la même que dans l'exercice 2) **et** une batterie de tests. Complétez les trous puis lancez-le : `python3 rotation_image.py` affiche, pour chaque test, `OK` s'il passe, `A FAIRE ou ECHEC` sinon. Idée des tests :

```
# une rotation d'un quart de tour horaire
img = [["A", "B"],
       ["C", "D"]]
rotation(img, 0, 0, 2)
# img doit valoir [["C", "A"],
#                  ["D", "B"]]
```

Le script vérifie aussi l'égalité avec une rotation horaire de référence (programmée sans « diviser pour régner ») pour $n = 2,4,8$, et que **quatre rotations** redonnent l'image initiale. Une fois les tests passés, appliquez votre fonction à la vraie image `joconde.png` (exercice 3).

FICHE DE RÉVISION

L'essentiel — trois temps

Diviser en sous-problèmes plus petits, **régner** (les résoudre, souvent récursivement), **combiner** les résultats. Exemples phares : **dichotomie** et **tri fusion**.

Recherche dichotomique (tableau trié)

```python
def recherche(tab, cible, debut, fin):
    if debut > fin:                 # condition d'arret
        return False
    milieu = (debut + fin) // 2
    if tab[milieu] == cible:
        return True
    elif cible < tab[milieu]:
        return recherche(tab, cible, debut, milieu - 1)    # a gauche
    else:
        return recherche(tab, cible, milieu + 1, fin)      # a droite
```

Premier appel : `recherche(t, 17, 0, len(t) - 1)`. **Terminaison** : la taille de l'intervalle (le **variant**) décroît strictement. **Coût logarithmique** : on **divise par deux** à chaque appel $\Rightarrow$ dans un million de valeurs, ≈ 20 comparaisons ($\log_2$).

Tri fusion

On coupe en deux, on trie **récursivement** chaque moitié, on **fusionne** :

```python
def fusion(gauche, droite):
    resultat = []
    i = j = 0
    while i < len(gauche) and j < len(droite):
        if gauche[i] <= droite[j]:
            resultat.append(gauche[i])
            i = i + 1
        else:
            resultat.append(droite[j])
            j = j + 1
    resultat = resultat + gauche[i:]    # reste de gauche (ou rien)
    resultat = resultat + droite[j:]    # reste de droite (ou rien)
    return resultat
```

`tri_fusion(tab)` : condition d'arrêt `len(tab) <= 1` ; sinon `milieu = len(tab) // 2`, deux appels sur `tab[:milieu]` et `tab[milieu:]`, puis `fusion`. **Coût $n \log_2 n$ même au pire** : $\log_2 n$ niveaux $\times$ un coût n de fusion par niveau.

Comparer les coûts

Coût	Exemple	$n = 10^6$
$O(\log_2 n)$	recherche dichotomique	≈ 20
$O(n)$	parcours, fusion	10^6
$O(n \log_2 n)$	tri fusion	$\approx 2 \cdot 10^7$
$O(n^2)$	tri par sélection / insertion	10^{12}

Au bac : logarithmique, linéaire, quasi-linéaire, quadratique (§ IV).

Erreurs à éviter

- dichotomie sur un tableau **non trié** ;
- bornes mal mises à jour $\Rightarrow$ récursion infinie ;
- fusion : oublier de **recopier le reste** ;
- croire le tri rapide toujours en $n \log_2 n$ (pire cas n^2).

Savoir-faire à maîtriser — je sais…

- ☐ reconnaître le schéma **diviser / régner / combiner** ;
- ☐ **dérouler** une recherche dichotomique et compter les tours ;
- ☐ justifier la **terminaison** (variant) et le **coût logarithmique** ;
- ☐ expliquer et dérouler le **tri fusion** (arbre des appels, coût $n \log_2 n$) ;
- ☐ **fusionner** deux listes déjà triées ;
- ☐ comparer les **coûts** ($\log_2 n$, n, $n \log_2 n$, n^2).

- ☐ expliquer et dérouler le **tri fusion** (arbre des appels, coût $n \log_2 n$) ;
- ☐ **fusionner** deux listes déjà triées ;
- ☐ comparer les **coûts** ($\log_2 n$, n, $n \log_2 n$, n^2).

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 907

1. Quels sont les trois temps du paradigme « diviser pour régner » ?
2. Que renvoie `fusion([1, 6], [2, 3, 9])` ?
3. Combien de comparaisons, au plus, pour une recherche dichotomique dans 1000 valeurs triées ?
4. Sans la recopie des restes dans `fusion`, que renvoie `fusion([1, 6], [2, 3, 9])` ?
5. Pour **une seule** recherche dans un tableau **non trié** de n valeurs : trier puis dichotomie, ou parcours ?

Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 907

1. Écrire la requête qui affiche les titres de la table `film`, de la meilleure note à la moins bonne. (ch. 6)
2. Premier arrivé, premier servi : A arrive à 0 (durée 3), B arrive à 1 (durée 2). Temps d'attente de B ? (ch. 4)
3. Pourquoi une version récursive consomme-t-elle plus de mémoire que sa version itérative ? (ch. 1)

Vers le bac

- Épreuve pratique : sujet 29 — Recherche dichotomique, p. 737

DÉFIS ADVENT OF CODE

DÉFI 1 Report Repair

la note de frais — parcours de listes et complexité

Jour 1

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t07-defi-aoc-2020-01` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/1`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Une note de frais contient environ deux cents dépenses ; parmi elles, deux (puis trois) ont pour somme 2020, et il faut les retrouver. Essayer toutes les paires à la main est hors de portée : il y en a près de vingt mille. Un programme les essaie toutes sans peine, mais la vraie question du défi est ailleurs : **combien d'opérations** faut-il, et peut-on faire mieux ?

Ce qu'on réinvestit. La **complexité** et la comparaison de plusieurs méthodes, comme dans ce chapitre. La méthode directe (deux boucles imbriquées) coûte $O(n^2)$; un ensemble permet de n'utiliser qu'une seule boucle ; enfin, trier la liste puis la parcourir par les deux bouts donne une troisième méthode. Ce tri coûte $O(n \log n)$ grâce à un algorithme « diviser pour régner » comme le tri fusion du cours : c'est lui qui rend la troisième méthode intéressante.

La démarche de la fiche.

1. Lever les ambiguïtés de l'énoncé, puis trouver la paire à la main sur un petit exemple.
2. Lire le fichier dans une liste d'entiers.

3. Écrire la solution directe par double boucle, puis évaluer son coût.
4. Partie 2 : passer des paires aux triplets en ajoutant une boucle.
5. En approfondissement, obtenir les mêmes réponses plus vite, avec un ensemble puis avec un tri.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`expense report`	note de frais	`entry`	ligne, valeur du fichier
`accounting`	comptabilité	`sum to`	avoir pour somme
`fix up`	corriger	`multiply`	multiplier
`puzzle input`	vos données (fichier personnel)	`star`	étoile (une par partie réussie)

Résumé de l'énoncé (partie 1)

Contexte. Avant de partir en vacances, vous devez régler un dernier problème : les elfes de la comptabilité ont une note de frais qui ne tombe pas juste et vous demandent de les aider à la corriger. Il faut retrouver dans cette note les dépenses qui posent problème.
Les données. Le fichier est la note de frais. Chaque ligne contient **un seul entier positif** : le montant d'une dépense. Il y a environ deux cents lignes ; leur ordre ne joue aucun rôle.
La règle.

- Parmi toutes les dépenses, exactement **deux** lignes ont des valeurs dont la somme vaut **2020**. Autrement dit, il faut trouver la seule paire de lignes qui « complètent » 2020.
- Les deux lignes doivent être **différentes** : une valeur présente une seule fois dans le fichier ne peut pas être associée à elle-même.
- Il n'y a qu'une seule paire qui convient : dès qu'on l'a trouvée, on peut s'arrêter.

La réponse attendue. Le **produit** des deux valeurs trouvées (et non leur somme, qui vaut toujours 2020). Par exemple, pour les dépenses 1000, 300, 1020, 45, la paire est formée de 1000 et 1020 : la réponse serait 1020000.

Exercice 1 *Vérifier sa compréhension*

But. Lever deux ambiguïtés de l'énoncé avant de programmer.
À quoi ça sert ? La première question fixe une contrainte du programme (ne jamais associer une ligne à elle-même) ; la seconde dit qu'on a le droit de réordonner la liste, par exemple de la trier, sans changer la réponse : l'approfondissement 2 s'en servira.

1. Si le fichier contient une seule fois la valeur `1010`, peut-elle former la paire avec elle-même ?
2. L'ordre des lignes dans le fichier a-t-il une importance pour la réponse ?

Exercice 2 *À la main, sur un petit exemple*

But. Trouver la paire à la main sur six nombres.
À quoi ça sert ? On dispose ainsi d'un exemple dont on **connaît la réponse** (`980896`) : chaque fonction écrite ensuite sera d'abord testée dessus, avant d'être lancée sur les vraies données.

On considère le fichier (inventé) suivant :

```
1010
812
1500
1208
33
487
```

Trouver à la main la réponse attendue. Vérification : on doit obtenir `980896`. Pourquoi la ligne `1010` ne donne-t-elle pas de solution à elle seule ?

2 Programmer la partie 1

Exercice 3 *Lire le fichier*

But. Transformer le fichier texte en une liste Python d'entiers, `nombres`.
À quoi ça sert ? Toutes les méthodes du défi travaillent sur cette liste. Les lignes lues sont des chaînes : sans conversion en entiers, la somme `"1010" + "1010"` donnerait `"10101010"` et non 2020. Afficher la longueur vérifie qu'aucune ligne n'a été perdue.

Écrire les lignes qui lisent `input.txt` et construisent la liste `nombres` des entiers qu'il contient. Afficher sa longueur pour vérifier.

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Exercice 4 *Une première solution*

But. Écrire la méthode la plus directe : essayer toutes les paires de lignes.
À quoi ça sert ? Elle donne la réponse de la partie 1. Elle sert aussi de **référence** : juste mais lente, elle permettra de vérifier les méthodes plus rapides des approfondissements.

Écrire une fonction `paire_2020(nombres)` qui renvoie le produit des deux valeurs recherchées. Tester sur l'exemple inventé de la fiche, puis sur vos données. ▷ coup de pouce

Exercice 5 *Combien d'opérations ?*

But. Évaluer le coût de la double boucle en fonction du nombre n de dépenses.
À quoi ça sert ? C'est cette estimation qui dit si la méthode reste utilisable quand n grandit, et qui servira de point de comparaison pour les autres méthodes du défi.

Le fichier contient environ 200 nombres. Combien de paires la fonction précédente examine-t-elle au pire ? Quelle est sa complexité en fonction de n ?

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Les elfes sont ravis, mais il leur reste une ancienne note de frais à vérifier selon une autre règle. Le fichier est le même ; seul le nombre de dépenses à associer change.
La nouvelle règle.

- Il faut cette fois trouver les **trois** lignes différentes dont les valeurs ont pour somme 2020.
- Comme dans la partie 1, une ligne ne peut servir qu'une fois, et il n'y a qu'un seul triplet qui convient.

La réponse attendue. Le produit de ces trois valeurs. Par exemple, pour les dépenses 1000, 700, 320, 55, le triplet est formé de 1000, 700 et 320 : la réponse serait 224000000.

Exercice 6 *La partie 2*

But. Passer des paires aux triplets en ajoutant une boucle à la solution de la partie 1.
À quoi ça sert ? On obtient la réponse de la partie 2 et l'on voit le prix à payer : chaque boucle imbriquée de plus multiplie le coût par n, ce qui motive la recherche de méthodes plus économes.

Adapter le programme pour répondre à la nouvelle question. Sur l'exemple inventé de la fiche, on doit trouver `24106500`. ▷ coup de pouce

4 Approfondissement

Outil Python : les ensembles (`set`) — rappel

Tout est présenté dans l'encadré « Outil Python : les ensembles » du défi *Crab Combat* (chapitre 4). On n'utilise ici que `set()` (ensemble vide), `add`, `remove` et le test `x in ensemble`, en temps constant (en moyenne). Les éléments doivent être non modifiables (nombres, chaînes, tuples).
À essayer. Avec `vus = {812, 1208}`, que valent `812 in vus` et `2020 - 812 in vus` ? Sur un million d'éléments, la différence de temps avec une liste est énorme.

★★ Exercice 7 *Approfondissement 1 : une seule boucle grâce à un ensemble*

But. Remplacer la boucle intérieure par un test d'appartenance à un ensemble.
À quoi ça sert ? Le test `x in ensemble` se fait en temps constant (en moyenne) : la partie 1 passe de $O(n^2)$ à $O(n)$, la partie 2 de $O(n^3)$ à $O(n^2)$. Le choix de la structure de données compte souvent plus que la vitesse de la machine.

1. Recopier puis exécuter le code de l'encadré du défi *Crab Combat* (chapitre 4). Que se passe-t-il si l'on ajoute deux fois `1208` à un ensemble ?
2. Écrire `paire_rapide(nombres)` qui résout la partie 1 avec une **seule** boucle et un ensemble `vus`. Quelle est sa complexité ? ▷ coup de pouce ▷ début de solution
3. En déduire une solution de la partie 2 en $O(n^2)$ au lieu de $O(n^3)$. ▷ coup de pouce

★★ **Exercice 8** *Approfondissement 2 : trier puis rapprocher deux indices*

But. Résoudre le même problème sans ensemble, en triant la liste puis en rapprochant deux indices.
À quoi ça sert ? Une fois la liste triée, chaque comparaison permet d'écarter définitivement un élément. On dispose alors de trois méthodes pour la même réponse, et l'on peut comparer leurs coûts : c'est l'objet du défi.

Une autre idée, sans ensemble, réinvestit le **tri** : avec le tri fusion de ce chapitre, il coûte $O(n \log n)$. On trie la liste, puis on place un indice `g` au début et un indice `d` à la fin.

1. Sur la liste triée `[33, 487, 812, 1010, 1208, 1500]`, on a `g = 0` et `d = 5` : la somme vaut $33 + 1500 = 1533$, trop petite. Faut-il avancer `g` ou reculer `d` ? Justifier, puis dérouler la méthode à la main jusqu'à trouver la paire.

2. Écrire `paire_triee(nombres, cible)` qui applique cette méthode (on travaille sur une copie triée, obtenue avec **votre** fonction `tri_fusion` du cours, qui renvoie une nouvelle liste) et renvoie le produit, ou `None` s'il n'y a pas de paire. ▷ coup de pouce ▷ début de solution

3. Quelle est la complexité totale (tri compris) ? Comparer avec les deux méthodes précédentes.

4. Adapter la méthode pour résoudre la partie 2 en $O(n^2)$, sans ensemble : trier la liste **une seule fois**, puis, pour chaque `x`, rapprocher deux indices parmi les éléments qui suivent `x`.

7

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 4 – piste Deux boucles imbriquées sur les **indices** `i` et `j`, avec `j > i` pour ne jamais prendre deux fois la même ligne. ↑ énoncé

Ex. 6 – piste Une boucle de plus. Combien de triplets pour $n = 200$? C'est encore rapide. ↑ énoncé

Ex. 7 q. 2 – piste Pour chaque nombre `x`, la seule valeur qui le complète est `2020 - x`. Il suffit de savoir si on l'a déjà rencontrée. ↑ énoncé

Ex. 7 q. 2 – début de solution On parcourt les nombres un par un ; **avant** d'ajouter `x` à l'ensemble `vus`, on regarde si `2020 - x` y est déjà. Tester avant d'ajouter évite d'utiliser deux fois la même ligne. ↑ énoncé

Ex. 7 q. 3 – piste Généraliser la fonction avec un paramètre `cible` : pour chaque `x`, chercher une paire de somme `2020 - x` parmi les nombres **suivants**. ↑ énoncé

Ex. 8 q. 2 – piste Tant que `g < d` : si la somme est trop petite, la seule façon de l'augmenter est d'avancer `g` ; si elle est trop grande, reculer `d`. ↑ énoncé

Ex. 8 q. 2 – début de solution Si `t[g] + t[d] < cible`, alors `t[g]` associé à n'importe quel élément situé avant `d` donne aussi une somme trop petite : `t[g]` ne peut faire partie d'aucune paire, on peut l'écarter. ↑ énoncé

DÉFI 2 Encoding Error

la faille du chiffrement — fenêtre glissante et complexité

Jour 9

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t07-defi-aoc-2020-09` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/9`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Un fichier contient un millier de nombres, dont chacun devrait pouvoir s'écrire comme la somme de deux nombres parmi les 25 qui le précèdent. Il faut repérer le premier nombre qui viole cette règle, puis retrouver une suite de nombres consécutifs dont la somme vaut ce nombre fautif. À la main, tester des milliers de paires et de tranches est impossible ; un programme le fait en quelques instants, à condition d'être bien conçu.
Ce qu'on réinvestit. Ce défi prolonge le défi précédent (*Report Repair*, recherche d'une paire de somme donnée) et surtout l'analyse de **complexité** de ce chapitre. La première solution qui vient à l'esprit essaie toutes les tranches : elle est quadratique. Comme le tri fusion détrône les tris quadratiques, une meilleure idée — deux indices qui ne font qu'avancer, une *fenêtre glissante* — permet un parcours linéaire. Cette fenêtre glissante reviendra au chapitre 12, avec la recherche naïve d'un motif dans un texte.
La démarche de la fiche.

1. Vérifier sa compréhension de la règle, puis l'appliquer à la main sur un petit exemple dont on connaît la réponse.
2. Écrire une fonction qui teste si une cible est la somme de deux éléments d'une fenêtre, puis faire glisser cette fenêtre : réponse de la partie 1.
3. Pour la partie 2, écrire une première solution qui essaie toutes les tranches, puis une

solution linéaire à deux indices.
4. En approfondissement, mesurer les durées pour confirmer les complexités, et accélérer la partie 1 avec un ensemble.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`preamble`	préambule	`previous`	précédent
`sum of any two`	somme de deux quelconques	`different`	différent
`weakness`	faiblesse, faille	`valid / invalid`	valide / invalide
`contiguous set`	suite contiguë	`at least two`	au moins deux

Résumé de l'énoncé (partie 1)

Contexte. Le héros branche son ordinateur sur une prise de données de l'avion. Il en sort une longue suite de nombres, chiffrée avec un vieux procédé qui comporte une faille. Dans ce procédé, chaque nombre doit pouvoir s'obtenir à partir des nombres reçus juste avant lui : un nombre qui ne le peut pas trahit l'erreur, et c'est lui qu'on cherche.
Les données. Le fichier contient **un entier positif par ligne**, dans l'ordre de réception. Seule la position compte : le nombre de la ligne 30 est testé à l'aide des nombres des lignes qui le précèdent immédiatement.
La règle.

- Les 25 premiers nombres forment le *préambule* (le petit exemple de l'énoncé en utilise 5). Ils ne sont pas testés : ils servent seulement de point de départ.
- Chaque nombre suivant doit être la somme de deux nombres de **valeurs différentes** pris parmi les 25 nombres qui le précèdent immédiatement. Deux nombres égaux ne forment donc pas une paire valable.
- On regarde ainsi une *fenêtre* de 25 nombres qui glisse d'un cran à chaque nouveau nombre : le plus ancien en sort, le nombre qu'on vient de tester y entre. Un nombre sorti de la fenêtre ne peut plus servir.
- Le premier nombre pour lequel aucune paire ne convient est le nombre fautif.

La réponse attendue. Ce premier nombre qui ne respecte pas la règle. Par exemple, avec un préambule de 3 et la suite 2, 5, 4, 7, 9, 30 : 7 vaut $2+5$, 9 vaut $5+4$, mais 30 ne s'obtient pas avec 4, 7 et 9 : la réponse serait 30.

Exercice 1 *Comprendre ce qui est demandé*

But. Vérifier sur trois cas simples qu'on a bien compris la règle : valeurs différentes, contenu exact de la fenêtre.
À quoi ça sert ? Ces trois questions correspondent aux trois erreurs les plus fréquentes du programme : accepter deux valeurs égales, décaler la fenêtre d'un cran, utiliser un nombre déjà sorti.

À traiter sur le cahier :

1. La fenêtre contient `3 7 10 7 2`. Le nombre suivant `14` est-il valide ?
2. Avec un préambule de longueur p, quels indices forment la fenêtre du nombre d'indice i ?
3. La paire peut-elle utiliser un nombre sorti de la fenêtre ?

Exercice 2 *À la main, sur un petit exemple*

But. Faire glisser à la main une fenêtre de 5 nombres sur une liste de quinze nombres.
À quoi ça sert ? On obtient un exemple dont on **connaît la réponse** (22) : le programme devra la retrouver avant d'être lancé sur les vraies données. Écrire les fenêtres successives aide aussi à trouver les bonnes bornes de la tranche.

On considère la liste (inventée) suivante, avec un préambule de longueur 5 :

```
6  3  7  11  1  13  24  8  25  33  37  62  70  58  22
```

Vérifier à la main les nombres situés après le préambule, en écrivant à chaque fois la fenêtre des 5 nombres précédents et une paire qui convient. Vérification : le premier nombre invalide est 22.

2 Programmer la partie 1

Exercice 3 *Somme de deux*

But. Écrire la fonction qui dit si une cible est la somme de deux éléments distincts d'une fenêtre.
À quoi ça sert ? C'est la brique de base de la partie 1 : la fonction suivante l'appellera une fois pour chaque nombre à tester. Les `assert` garantissent qu'elle est juste avant de s'en servir.

Écrire une fonction `somme_de_deux(fenetre, cible)` qui renvoie `True` si deux éléments de la liste `fenetre` ont pour somme `cible`, en respectant la règle des valeurs différentes. L'essayer avec des `assert` sur des cas simples.

▷ coup de pouce

Exercice 4 *La fenêtre glissante*

But. Faire glisser la fenêtre sur toute la liste et renvoyer le premier nombre invalide.
À quoi ça sert ? Cette fonction donne directement la réponse de la partie 1. Le nombre trouvé sert ensuite de cible pour toute la partie 2 ; le paramètre `p` permet de tester sur l'exemple ($p = 5$) avant les vraies données ($p = 25$).

Écrire une fonction `premier_invalide(nombres, p)` où `p` est la longueur du préambule. Tester avec `p = 5` sur l'exemple inventé de la fiche, puis avec la bonne longueur sur vos données.

▷ coup de pouce

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Pour exploiter la faille, on se sert du nombre fautif trouvé en partie 1 : on ne cherche plus une paire, mais une **suite contiguë** de nombres du fichier, c'est-à-dire des nombres qui se suivent sans trou.
La nouvelle règle.

- Trouver une suite d'**au moins deux** nombres consécutifs du fichier dont la somme vaut exactement le nombre fautif. Le nombre fautif seul ne compte donc pas comme une suite.
- Dans cette suite, repérer le plus petit et le plus grand nombre. Attention : ce ne sont pas forcément le premier et le dernier de la suite.

La réponse attendue. La somme du plus petit et du plus grand nombre de cette suite. Par exemple, si le nombre fautif était 18 et que le fichier contenait 2, 5, 4, 7 à la suite, la suite serait 2, 5, 4, 7 et la réponse $2 + 7 = 9$.

Exercice 5 *Une première solution*

But. Trouver la suite contiguë en essayant toutes les positions de début et de fin possibles.
À quoi ça sert ? Cette méthode simple donne une première réponse juste à la partie 2. Elle servira aussi de référence pour vérifier, et chronométrer, la méthode plus rapide de l'exercice suivant.

Écrire une fonction `suite_naive(nombres, cible)` qui essaie toutes les positions de début et de fin et renvoie la somme du plus petit et du plus grand nombre de la suite trouvée. Sur l'exemple inventé de la fiche, on doit trouver `12`. ▷ coup de pouce

Exercice 6 *Deux indices*

But. Remplacer la double boucle par deux indices qui ne reculent jamais, en justifiant que c'est correct.
À quoi ça sert ? On passe d'une méthode quadratique à une méthode linéaire : c'est exactement le gain de complexité étudié dans ce chapitre. La justification (les nombres sont positifs) est aussi importante que le code.

Les nombres de vos données sont tous positifs. Expliquer pourquoi on peut alors trouver la suite avec seulement deux indices `debut` et `fin` qui ne font qu'avancer, en tenant à jour la somme de la tranche. Programmer cette méthode et comparer sa complexité à celle de l'exercice précédent.

▷ coup de pouce ▷ début de solution

4 Approfondissement

Outil Python : mesurer une durée (`time`) et tirer au hasard (`random`)

Le module `time` fournit un chronomètre précis ; le module `random` tire des nombres au hasard, utile pour fabriquer de grosses données de test.

```python
import random
import time


def somme_entiers(n):
    """Somme des entiers de 0 a n - 1, calculee par une boucle."""
    total = 0
    for k in range(n):
        total += k
    return total


x = random.randint(1, 6)          # entier au hasard entre 1 et 6 (bornes comprises)
liste = [random.randint(1, 1000) for _ in range(5)]
depart = time.perf_counter()      # top depart (en secondes)
total = somme_entiers(10**6)
duree = time.perf_counter() - depart
print(f"{duree:.4f} s")           # 4 chiffres apres la virgule
```

Intérêt. Vérifier expérimentalement une complexité : si l'on double n, une méthode linéaire prend environ deux fois plus de temps, une méthode quadratique environ quatre fois plus. Une mesure isolée varie : la répéter.
À essayer. Mesurer `somme_entiers(10**6)` puis `somme_entiers(2 * 10**6)` : le rapport des durées est-il proche de 2 ?

★★★ Exercice 7 — *Défi : mesurer*

But. Chronométrer les deux méthodes de la partie 2 sur des données de plus en plus grosses.
À quoi ça sert ? Une complexité annoncée se vérifie par l'expérience : si l'écart entre les deux méthodes grandit avec la taille des données, l'analyse théorique est confirmée.

Mesurer avec le module `time` la durée des deux méthodes de la partie 2 sur vos données, puis sur une liste aléatoire de 10 000 nombres positifs, en prenant pour cible la somme (calculée par une boucle) d'une tranche choisie à l'avance, par exemple des indices 3 000 à 3 049 : on est ainsi sûr qu'une suite existe. Les mesures sont-elles cohérentes avec les complexités annoncées ? ▷ coup de pouce

Outil Python : les ensembles (`set`) — rappel

Tout est présenté dans l'encadré « Outil Python : les ensembles » du défi *Crab Combat* (chapitre 4) : ensemble vide `set()`, ajout avec `add`, retrait avec `remove`, test `x in ensemble` en temps constant, éléments non modifiables (tuples). Un ensemble ne contient jamais deux fois le même élément.
À essayer. Avec `fenetre = [3, 7, 10, 7, 2]`, que vaut `set(fenetre)` ? Combien d'éléments ?

★★ Exercice 8 — *Approfondissement 1 : la partie 1 avec un ensemble*

But. Tester une paire de somme donnée avec une seule boucle, grâce à un ensemble.
À quoi ça sert ? La partie 1 passe de p^2 à p tests par nombre. L'idée — chercher directement le complément `cible - x` au lieu d'essayer toutes les paires — resservira chaque fois qu'on cherche deux valeurs liées par une somme.

1. À la main : avec la fenêtre `3 7 10 7 2` et la cible `13`, calculer pour chaque `x` la valeur `13 - x` et dire si elle est dans la fenêtre. Recommencer avec la cible `14` : pourquoi le complément `7` de `7` ne convient-il pas ?
2. Écrire `somme_de_deux_ensemble(fenetre, cible)` avec une seule boucle et un ensemble, puis l'utiliser pour retrouver `22` sur l'exemple inventé de la fiche.
3. Pour $p = 25$, combien de tests environ fait chaque version pour un nombre ? En déduire les complexités totales.

▷ coup de pouce ▷ début de solution

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Deux boucles sur les indices, avec `j > i`, comme pour le jour 1, et un test sur les valeurs des deux éléments. ↑énoncé

Ex. 4 – piste Pour le nombre d'indice `i`, la fenêtre est la tranche `nombres[i - p:i]`. Faire un petit dessin pour vérifier les bornes. ↑énoncé

Ex. 5 – piste Une suite contiguë, c'est une tranche `nombres[debut:fin]`. Attention : la suite réduite au seul nombre invalide ne compte pas. Le plus petit et le plus grand nombre de la tranche se trouvent avec une boucle, comme une recherche de maximum. ↑énoncé

Ex. 6 – piste Si la somme courante est trop petite, on ajoute le nombre suivant (`fin` avance) ; si elle est trop grande, on retire le premier (`debut` avance). ↑énoncé

Ex. 6 – début de solution Chaque indice avance au plus n fois : la méthode est linéaire. La boucle `while` s'arrête quand la somme est atteinte avec au moins deux nombres. ↑

Ex. 7 – piste `time.perf_counter()` avant et après l'appel ; la différence donne la durée en secondes. ↑

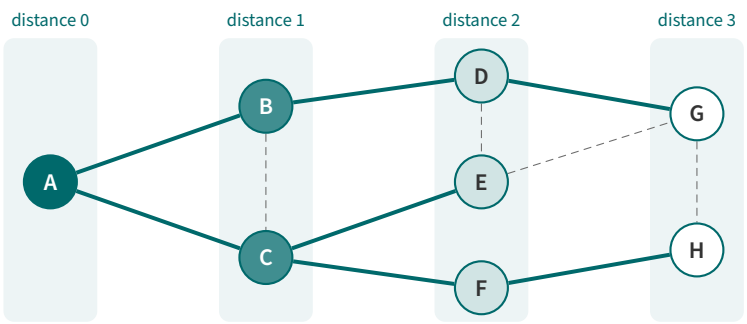
Ex. 8 – piste Pour chaque `x` de la fenêtre, la seule valeur qui le complète est `cible - x`. ↑énoncé

Ex. 8 – début de solution Construire `valeurs = set(fenetre)`, puis tester `cible - x in valeurs` sans oublier la règle des valeurs différentes : `cible - x != x`. ↑

8

CHAPITRE 8

Graphes



DANS CE CHAPITRE

- Activités préparatoires . . . 385
- Cours . . . 388
- Exercices . . . 405
- TP et projets . . . 419
- Fiche de révision . . . 425
- Défis Advent of Code . . . 427
- Corrigés . . . 908

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Sans lever le crayon

▶ corrigé p. 908

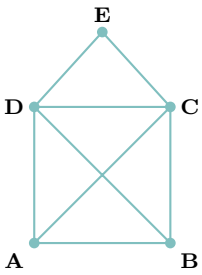
trois dessins, sept ponts, une règle à découvrir

Durée : 35 min · Sans ordinateur, par deux

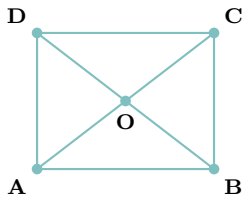
Consignes

- Matériel : un crayon et une gomme. Les réponses s'écrivent sur le cahier.
- Règle du jeu : **repasser chaque trait d'un dessin une fois et une seule, sans lever le crayon**. On a le droit de passer plusieurs fois par un même point (marqué d'un gros point), mais jamais deux fois sur le même trait.

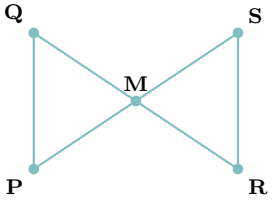
Exercice 1 *Trois dessins*



(a) la maison



(b) l'enveloppe fermée



(c) le nœud papillon

1. Pour chaque dessin, essayer de le tracer sans lever le crayon (au brouillon). Si c'est possible, noter la suite des points visités (par exemple `A-B-...`) ; sinon, écrire « pas trouvé ». Répondre pour (a), (b) et (c).
2. Pour le dessin (a), un camarade affirme : « il faut partir de `A` ou de `B` ». Essayer en partant de `E`. Qu'observe-t-on ?
3. Pour le dessin (c), peut-on revenir à son point de départ ? Depuis n'importe quel point ?

Exercice 2 *Compter les traits*

Pour chaque point, on compte le nombre de traits qui y aboutissent.

4. Recopier et compléter le tableau sur le cahier.

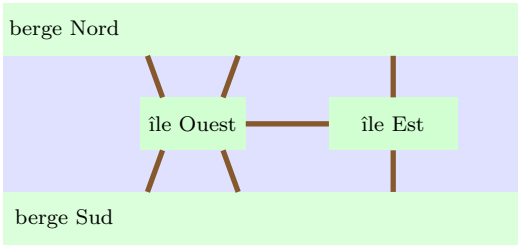
Dessin	**nombre de traits en chaque point**	**nombre de points « impairs » (où aboutit un nombre impair de traits)**	**possible ?**	**retour au départ ?**
(a) la maison				
(b) l'enveloppe				
(c) le papillon				

5. Quand on **passe** par un point (on y arrive puis on en repart), combien de traits utilise-t-on ? Que peut-on en déduire pour un point qui n'est ni le départ ni l'arrivée ?

6. Proposer une règle : d'après le nombre de points « impairs », quand le dessin est-il possible ? quand peut-on revenir au départ ?

Exercice 3 *Les sept ponts de Königsberg*

Au XVIIIe siècle, la ville de Königsberg est traversée par une rivière formant deux îles, reliées aux berges et entre elles par **sept ponts**. Les habitants se demandaient : *peut-on se promener en traversant chaque pont une fois et une seule ?*



7. Chercher quelques minutes une telle promenade sur la carte.

8. La forme des îles et la longueur des ponts n'ont aucune importance. Redessiner sur le cahier la situation avec seulement des **points** (un par terre : `N`, `S`, `O`, `E`) et des **traits** (un par pont).

9. Compter les traits en chaque point. À l'aide de la règle trouvée à l'exercice 2, la promenade est-elle possible ?

10. La ville décide de construire un huitième pont. Entre quelles terres le placer pour que la promenade devienne possible ? D'où faudra-t-il alors partir ?

Exercice 4 *Mettre des mots*

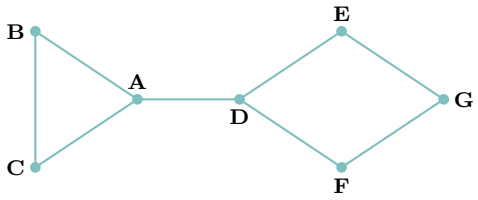
11. Proposer un nom pour chacune des notions rencontrées :

- un dessin fait de points reliés par des traits ;
- un point / un trait ;
- le nombre de traits qui aboutissent à un point ;
- remplacer la carte par des points et des traits.

12. *Défi.* Peut-on dessiner une figure qui a **exactement un** point « impair » ? Essayer, puis expliquer.

Exercice 5 *Visiter tous les points*

On ne cherche plus à passer par tous les **traits**, mais par tous les **points**. Le dessin ci-dessous est le plan d'un musée : chaque point est une salle, chaque trait un couloir. Un gardien part de la salle `A` et doit passer dans **toutes** les salles ; il a le droit d'emprunter plusieurs fois le même couloir.



13. Un premier gardien applique la règle : « depuis la salle où je suis, je prends toujours le couloir qui mène à la salle de plus petite lettre ». Écrire les dix premières salles de sa tournée, en commençant par `A`. Qu'observe-t-on ?

14. Un deuxième gardien **coche** sur le plan chaque salle où il passe, et prend toujours le couloir qui mène à la salle **non cochée** de plus petite lettre. Écrire sa tournée depuis `A`. Que se passe-t-il ?

15. Au moment où le deuxième gardien est bloqué, de quoi aurait-il dû se souvenir pour repartir ?

16. **Comment visiter toutes les salles sans tourner en rond ni rester bloqué ?** Proposer une méthode « papier-crayon » : ce que l'on note au fur et à mesure, et comment on choisit la salle suivante.

17. L'appliquer depuis `A` : dans quel ordre les salles sont-elles visitées ? Comment sait-on que la visite est terminée ?

COURS

Programme (BO) *« Graphes : structures relationnelles. Sommets, arcs, arêtes, graphe orienté ou non orienté. Modéliser des situations sous forme de graphes. Écrire les implémentations correspondantes d'un graphe : matrice d'adjacence, liste de successeurs / de prédécesseurs. Passer d'une représentation à une autre. Parcourir un graphe en largeur, en profondeur. Repérer la présence d'un cycle. Chercher un chemin dans un graphe. »*

Prérequis les **piles** et les **files** (indispensables pour les parcours !), les **dictionnaires** et les listes de listes, la **récursivité**, les **classes**, la notion de **coût** d'un algorithme. Les **arbres** (un arbre est un graphe particulier). → fiches de révision : ch. 1 p. 58, ch. 2 p. 113, ch. 3 p. 162, ch. 5 p. 267 → rappels de 1re : listes p. 1057, dictionnaires p. 1058, parcours et coût p. 1059

Objectifs *reconnaître* une situation qui appelle un graphe et la *modéliser* ; *maîtriser* le vocabulaire ; *implémenter* un graphe de trois façons et *passer* de l'une à l'autre ; *parcourir* un graphe en largeur (file) et en profondeur (pile) ; *trouver* un chemin, le plus court chemin, détecter un cycle.

Situation de départ

1. Comment ranger dans un programme un plan de métro, un réseau d'amis ou le Web, où n'importe quel élément peut être relié à n'importe quel autre ?
2. Comment une application de transport trouve-t-elle le trajet qui passe par le moins de stations, sans essayer tous les chemins possibles ?
3. Comment explorer un labyrinthe ou un réseau sans tourner en rond indéfiniment ?

I Pourquoi un graphe ? Parce que le monde est un réseau

Regardez autour de vous. Le plan du métro : des stations, reliées par des lignes. Vos amitiés : des personnes, reliées par des « amis ». Le Web : des milliards de pages, reliées par des liens. Un GPS : des carrefours, reliés par des routes. Internet : des routeurs, reliés par des câbles. Le cerveau : des neurones, reliés par des synapses. Une molécule : des atomes, reliés par des liaisons.

À chaque fois, la même idée revient : **des objets, et des relations entre ces objets**. C'est *exactement* cela, un graphe. Aucune autre structure de ce cours ne capture aussi bien cette réalité : une liste range « en ligne », un arbre range « en hiérarchie », mais seul le graphe autorise **n'importe quelle relation entre n'importe quels objets** — des boucles, des raccourcis, des allers-retours. C'est la structure la plus générale, et sans doute la plus utile de toute l'informatique.

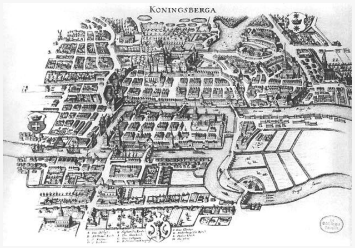
Règle 1 — La grande idée

Un **graphe** modélise un ensemble d'**objets** (les **sommets**) et les **relations** qui les lient deux à deux (les **arêtes**). Dès qu'un problème parle de « qui est relié à qui », il parle de graphes.

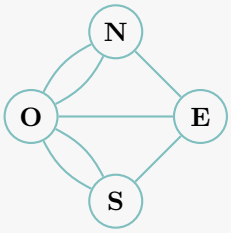
A. L'acte de naissance : les sept ponts de Königsberg (1736)

Pour les curieux — *Un peu d'histoire* (facultatif)

Au XVIIIe siècle, la ville de Königsberg est traversée par une rivière formant deux îles, reliées aux berges et entre elles par **sept ponts**. Les habitants se demandaient si l'on pouvait se promener en traversant chaque pont **une fois et une seule**. En 1736, **Leonhard Euler** règle la question en **oubliant la carte** : peu importe la forme des îles ou la longueur des ponts, seul compte **ce qui est relié à quoi**. Il remplace la ville par quatre points (les berges `N` et `S`, les îles `O` et `E`) et sept traits (les ponts) : c'est le **premier graphe de l'histoire**, et l'acte de naissance de la **théorie des graphes**. En comptant les ponts qui partent de chaque terre, il *prouve* que la promenade est impossible (voir le théorème d'Euler, à la fin de la section II). Euler (1707–1783), Suisse alors membre de l'Académie des sciences de Saint-Pétersbourg, est l'un des mathématiciens les plus féconds de l'histoire : ses œuvres complètes remplissent plus de soixante-dix volumes.



Königsberg et ses ponts (gravure de 1652)



Le graphe d'Euler



Euler (par J. E. Handmann, 1753)

À retenir — la leçon d'Euler : modéliser

Retenez la leçon d'Euler, car c'est **toute** la démarche de ce chapitre : *modéliser*, c'est jeter les détails inutiles pour ne garder que la structure des relations. Une fois le bon graphe dessiné, le problème est souvent à moitié résolu.

B. Des situations où le graphe s'impose

Exemple — *Quatre problèmes, un seul outil*

- **Le GPS et le plus court chemin.** Carrefours = sommets, routes = arêtes (*pondérées* par les distances ou les temps). Trouver son itinéraire, c'est chercher un chemin de coût minimal dans un graphe.
- **Les réseaux sociaux.** Personnes = sommets, relations = arêtes. « Combien d'intermédiaires entre deux inconnus ? », « qui suggérer comme ami ? », « qui est la personne la plus *centrale* ? » : autant de questions de graphes.
- **Le Web et Google.** Pages = sommets, liens hypertextes = arcs (*orientés* !). Le classement des résultats de Google repose sur un parcours de cet immense graphe.
- **La planification de tâches.** Étapes d'une recette, d'un chantier, d'un programme = sommets ; « il faut faire ceci *avant* cela » = arcs. On ordonne le travail en parcourant le graphe des dépendances.

Nous n'étudierons qu'une petite partie de cette théorie immense : comment **représenter** un graphe en Python, et comment le **parcourir** pour répondre à des questions concrètes.

II Le vocabulaire des graphes

Un graphe est un ensemble de **sommets** (*vertices* ou *nodes* en anglais) reliés par des **arêtes** ou des **arcs** (*edges*). Il peut être **non orienté** ou **orienté**.

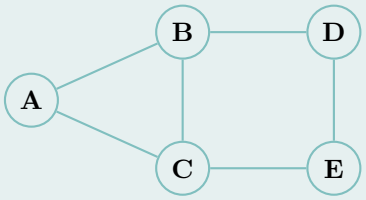
Définition 1 — Graphe, ordre, taille

Un **graphe** $G = (S,A)$ est la donnée d'un ensemble fini S de **sommets** et d'un ensemble A d'**arêtes** (des paires de sommets, en non orienté) ou d'**arcs** (des couples ordonnés, en orienté).

- l'**ordre** du graphe est son nombre de sommets, souvent noté $n = |S|$;
- la **taille** du graphe est son nombre d'arêtes, souvent noté $m = |A|$.

Deux sommets reliés par une arête sont dits **adjacents** (ou **voisins**).

A. Graphe non orienté



Dans un graphe **non orienté**, une arête se parcourt **dans les deux sens**. Une **chaîne** est une suite de sommets reliés de proche en proche, par exemple `A -- C -- E -- D` ; sa **longueur** est le nombre d'arêtes, ici 3.

Les sommets `B` et `C` sont **adjacents** à `A` : ce sont ses **voisins**. Le **degré** d'un sommet est son nombre de voisins ($\deg(\texttt{A}) = 2$, $\deg(\texttt{B}) = 3$).

↻ **Retour sur l'activité.** Les points et les traits de l'activité étaient des **sommets** et des **arêtes**, et le « nombre de traits en un point » était le **degré**. Remplacer la carte de Königsberg par quatre points et sept traits, c'était **modéliser**.

Exemple concret : le graphe des amis sur Facebook est **non orienté** — si vous êtes ami avec quelqu'un, la réciproque est vraie.

Propriété 1 — Lemme des poignées de main

Dans un graphe non orienté, la **somme des degrés** de tous les sommets est égale au **double** du nombre d'arêtes :

$$\sum_{s \in S} \deg(s) = 2\,m.$$

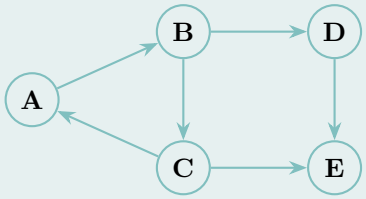
En conséquence, le nombre de sommets de degré **impair** est toujours **pair**.

Démonstration :
Chaque arête relie exactement deux sommets ; elle est donc comptée **deux fois** dans la somme des degrés (une fois à chaque extrémité). La somme des degrés vaut donc $2m$. Comme $2m$ est pair, la somme des degrés est paire : les sommets de degré impair doivent se « compenser » deux par deux, donc ils sont en nombre pair. □

↻ **Retour sur l'activité.** Voilà pourquoi le défi de l'activité était impossible : aucune figure ne peut avoir **exactement un** point « impair », puisque ces points sont toujours en nombre pair.

Pour les curieux (facultatif)
Joli nom : si, dans une soirée, chaque poignée de main relie deux personnes, alors le nombre total de mains serrées (comptées par personne) est pair — et il y a toujours un nombre pair d'invités ayant serré un nombre impair de mains !

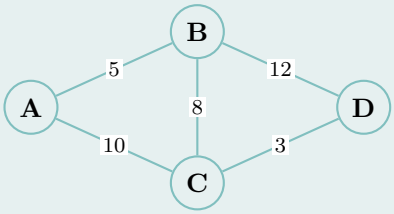
B. Graphe orienté



Dans un graphe **orienté**, un **arc** ne se parcourt que dans le sens de la flèche. Un **chemin** est une suite de sommets reliés par des arcs *dans le bon sens*, par exemple `A` → `B` → `D` → `E`. Les voisins (ou **successeurs**) de `B` sont `C` et `D` (mais pas `A` !).

Exemple concret : le graphe des abonnements sur X (Twitter) ou Instagram est **orienté** : on peut « suivre » quelqu'un sans être suivi en retour.

C. Graphe pondéré

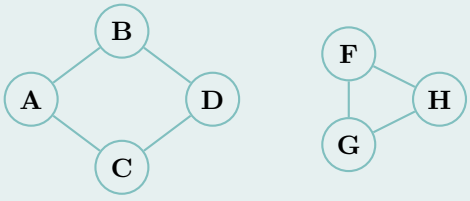


Un graphe est **pondéré** (ou valué) si chaque arête porte une valeur numérique — son **poids** (coût, distance, temps, débit…). Exemples : les **distances** entre villes sur une carte routière, le **coût** d'une liaison entre routeurs (on le retrouvera au chapitre *Réseaux*, avec le protocole OSPF).

D. Connexité, cycle

Définition 2 — Connexité

Un graphe est **connexe** s'il est « d'un seul tenant » : n'importe quelle paire de sommets peut être reliée par une chaîne. Sinon, il se décompose en plusieurs **composantes connexes** (ses « morceaux »).



Ce graphe n'est **pas** connexe : aucune chaîne ne relie `A` à `F`. Il a **deux** composantes connexes : {A,B,C,D} et {F,G,H}.

Définition 3 — Cycle

Un **cycle** est une chaîne (non orientée) ou un chemin (orienté) qui **revient à son point de départ** sans réemprunter deux fois la même arête. Un graphe connexe **sans** cycle est exactement un… **arbre** : voilà le lien avec le chapitre *Arbres*.

E. Quelques graphes particuliers

Définition 4 — Sous-graphe, sommet isolé, graphe complet

- un **sous-graphe** de G s'obtient en ne gardant qu'une partie des sommets de G et une partie des arêtes qui les relient (les composantes connexes en sont des exemples) ;
- un sommet est **isolé** si son degré est nul (aucun voisin) ;
- un graphe est **complet** si **toute** paire de sommets est reliée par une arête. Un graphe complet à n sommets possède alors $\dfrac{n(n-1)}{2}$ arêtes (chacun des n sommets est relié aux $n-1$ autres, et chaque arête est comptée deux fois).

F. Chaînes eulériennes : le théorème d'Euler [au-delà du programme]

Revenons à l'énigme fondatrice des **sept ponts de Königsberg** (§ I). On peut maintenant l'énoncer proprement.

Définition 5 — Chaîne et cycle eulériens

Une **chaîne eulérienne** est une chaîne qui emprunte **chaque arête du graphe une fois et une seule**. Si, de plus, elle revient à son point de départ, c'est un **cycle eulérien**.

La question de Königsberg était donc : *ce graphe admet-il une chaîne eulérienne ?* Euler y répond **complètement**, et sa réponse ne dépend que des **degrés** des sommets.

Théorème 1 — Euler, 1736 ; Hierholzer, 1873

Soit G un graphe connexe.

- G admet un **cycle eulérien** si et seulement si **tous** ses sommets sont de degré **pair** ;
- G admet une **chaîne eulérienne** (sans forcément revenir au départ) si et seulement s'il a **exactement zéro ou deux** sommets de degré **impair** (et, s'il y en a deux, ce sont nécessairement le départ et l'arrivée).

Pour les curieux — *L'idée* (facultatif)

Le sens « facile » est intuitif : chaque fois qu'on **traverse** un sommet intermédiaire, on « consomme » deux arêtes (une pour entrer, une pour sortir). Un sommet par lequel on ne fait que passer doit donc avoir un degré **pair**. Seuls le départ et l'arrivée peuvent avoir un degré impair — et ils coïncident dans le cas d'un cycle, qui exige alors **tous** les degrés pairs. C'est ce sens que démontre Euler en 1736 ; la réciproque (si les degrés conviennent et que le graphe est connexe, la chaîne existe) ne sera démontrée qu'en 1873, par Carl **Hierholzer**.

Exemple — *Königsberg tranché*
Dans le graphe des ponts, les quatre terres ont pour degrés 3,3,3,5 : **quatre** sommets de degré impair. Comme il en faudrait au plus deux, **aucune** chaîne eulérienne n'existe : la promenade rêvée par les habitants est bel et bien **impossible**. Euler l'a démontré sans essayer un seul trajet — juste en comptant des degrés.

↻ **Retour sur l'activité.** La règle conjecturée dans l'activité est ce théorème : la maison (deux points impairs, `A` et `B`) se trace en partant de `A` ou de `B`, l'enveloppe fermée (quatre points impairs) est impossible, et le nœud papillon (aucun point impair) se trace en revenant au départ, d'où que l'on parte.

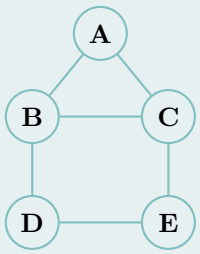
Pour les curieux (facultatif)
Ce théorème n'est **pas au programme** : on le donne pour la culture, et parce qu'il montre magnifiquement la puissance de la modélisation par graphes. À ne pas confondre avec le parcours d'un graphe (passer par tous les **sommets**) : ici on veut passer par toutes les **arêtes**.

► **Exercices d'application :** exercices 1 à 3 (vocabulaire et modélisation)

III Modéliser un graphe en Python

Pour représenter un graphe, il faut se donner une convention répondant à deux questions : *qui sont les sommets ?* et *pour chaque sommet, quels sont ses voisins ?* (et éventuellement le poids de l'arête). Il existe pour cela **trois représentations classiques** — et savoir **passer de l'une à l'autre** est une capacité attendue au bac.

Dans toute cette partie, on illustre avec ce petit graphe non orienté, appelé G :



A. Représentation par matrice d'adjacence

Principe. On **numérote** les sommets (ici par ordre alphabétique : A= 0, B= 1…). On construit un tableau carré M où $M[i][j] = 1$ si les sommets de rangs i et j sont **voisins**, et 0 sinon. Ce tableau est la **matrice d'adjacence**.

$$\begin{array}{c} \\ \texttt{A} \\ \texttt{B} \\ \texttt{C} \\ \texttt{D} \\ \texttt{E} \end{array} \begin{array}{c} \begin{array}{ccccc} \texttt{A} & \texttt{B} & \texttt{C} & \texttt{D} & \texttt{E} \end{array} \\ \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{pmatrix} \end{array}$$

À retenir — matrice symétrique ou non

Pour un graphe **non orienté**, $M[i][j] = M[j][i]$: la matrice est **symétrique** (l'arête B–C donne un 1 de part et d'autre de la diagonale). Pour un graphe **orienté**, elle ne l'est pas forcément. Pour un graphe **pondéré**, on remplace les 1 par les poids.

En Python, une matrice est naturellement une **liste de listes** :

```
#      A  B  C  D  E
M = [ [0, 1, 1, 0, 0],   # A
      [1, 0, 1, 1, 0],   # B
      [1, 1, 0, 0, 1],   # C
      [0, 1, 0, 0, 1],   # D
      [0, 0, 1, 1, 0] ]  # E
```

Propriété 2 — Coût de la matrice d'adjacence

Pour un graphe à n sommets :

- **mémoire** : $O(n^2)$ (on stocke $n \times n$ cases, y compris tous les zéros) ;
- tester si deux sommets sont **voisins** : $O(1)$ (un simple accès `M[i][j]`) ;
- lister les voisins d'un sommet : $O(n)$ (il faut parcourir toute sa ligne).

B. Représentation par liste d'adjacence

Principe. À chaque sommet, on associe la **liste de ses voisins**. On utilise un **dictionnaire** : les clés sont les sommets, les valeurs sont les listes de voisins. (Pour un graphe orienté, on stocke la liste des *successeurs*, ou celle des *prédécesseurs*.)

```
G = {'A': ['B', 'C'],
     'B': ['A', 'C', 'D'],
     'C': ['A', 'B', 'E'],
     'D': ['B', 'E'],
     'E': ['C', 'D']}
```

Propriété 3 — Coût de la liste d'adjacence

Pour un graphe à n sommets et m arêtes :

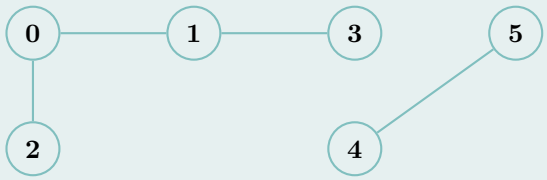
- **mémoire** : $O(n+m)$ — on ne stocke **que** les arêtes présentes, pas les zéros. C'est bien plus économique quand le graphe a **peu d'arêtes** (graphe « creux ») ;
- accéder à la liste des voisins d'un sommet : $O(1)$ (accès direct par la clé) ; la **parcourir** (pour énumérer ces voisins) : $O(\deg)$, où deg est le degré du sommet ;
- tester si deux sommets sont voisins : $O(\deg)$ (il faut parcourir la liste des voisins).

Règle 2 — Quelle représentation choisir ?

Matrice : rapide pour tester une arête, mais gourmande en mémoire — idéale si le graphe est **dense** (beaucoup d'arêtes). **Liste** : économe en mémoire, rapide pour lister les voisins — idéale si le graphe est **creux** (peu d'arêtes), ce qui est le cas le plus fréquent (un réseau social : chacun connaît quelques centaines de personnes, pas des millions).

C. Représentation par liste de listes

Quand les sommets sont **numérotés** de 0 à $n-1$, on peut se passer du dictionnaire : la liste d'adjacence devient une simple **liste de listes**, où la case d'indice i contient la liste des voisins du sommet i. C'est la convention de plusieurs sujets de bac. Par exemple, le graphe



se représente par `adj = [[1, 2], [0, 3], [0], [1], [5], [4]]` : le sommet 0 a pour voisins 1 et 2, le sommet 1 a pour voisins 0 et 3, etc.

Règle 3 — Passer d'une représentation à l'autre — capacité attendue

On sait convertir chaque représentation en une autre : parcourir la **matrice** ligne par ligne pour bâtir les listes de voisins ; ou parcourir un **dictionnaire** d'adjacence pour remplir une matrice de 0 et de 1. C'est un exercice classique du bac (voir la feuille).

► **Exercices d'application :** exercices 4 à 7 (représenter un graphe)

IV Une classe Graphe

Manipuler directement le dictionnaire `G` fonctionne, mais rien n'empêche alors d'oublier un sens d'une arête et d'obtenir un graphe incohérent. On **encapsule** donc la représentation dans une **classe** (chapitre 2) : l'utilisateur ne touche plus au dictionnaire, il passe par quelques méthodes (même démarche que pour les piles et les files). On se limite ici aux graphes **non orientés**.

```
class Graphe:
    # graphe non oriente, stocke par listes d'adjacence
    def __init__(self, sommets):
        self.sommets = sommets
        self.adj = {}                 # sommet -> liste de ses voisins
        for s in sommets:
            self.adj[s] = []

    def ajoute_arete(self, a, b):
        self.adj[a].append(b)
        self.adj[b].append(a)         # non oriente : dans les deux sens

    def voisins(self, s):
        return self.adj[s]

    def sont_voisins(self, a, b):
        return b in self.adj[a]
```

La méthode `ajoute_arete` est la seule à modifier le graphe, et elle met **toujours** à jour les deux listes : la symétrie est garantie par construction.

Pour construire le graphe G de la partie précédente, on part de la **liste de ses arêtes** :

```
g = Graphe(['A', 'B', 'C', 'D', 'E'])
aretes = [('A', 'B'), ('A', 'C'), ('B', 'C'), ('B', 'D'), ('C', 'E'), ('D', 'E')]
for (a, b) in aretes:
    g.ajoute_arete(a, b)
```

▷ **Je prédis.** Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> g.voisins('B')
['A', 'C', 'D']
>>> g.sont_voisins('D', 'E'), g.sont_voisins('A', 'D')
(True, False)
```

On retrouve la ligne `B` de la matrice d'adjacence, et le dictionnaire `g.adj` est exactement le dictionnaire `G` écrit plus haut.

Remarque
Pour un graphe **orienté**, il suffirait de retirer la deuxième ligne de `ajoute_arete` : on n'ajoute alors `b` que comme successeur de `a`.

► **Exercices d'application :** exercice 8 (la classe `Graphe`)

V Parcourir un graphe

Parcourir un graphe, c'est visiter **tous ses sommets** de proche en proche, à partir d'un sommet de départ. C'est l'opération fondamentale : trouver un chemin, la sortie d'un labyrinthe, tester la connexité, détecter un cycle… tout repose sur un parcours.

Règle 4 — Attention

Parcourir un graphe n'est pas la même chose que balayer bêtement le dictionnaire ou la matrice ! Un **parcours** suit les **arêtes**, de voisin en voisin.

8

A. Le schéma commun à tous les parcours

Tous les parcours suivent le même squelette :

- on part d'un sommet et on le met dans une structure d'attente `S` ;
- **tant que** `S` n'est pas vide : on en **retire** un sommet (on le « visite »), et on **ajoute** à `S` ses voisins pas encore rencontrés.

Dans les codes qui suivent, des commentaires en majuscules repèrent ces étapes, toujours avec les mêmes mots : `INITIALISER` (la structure d'attente et la mémoire des sommets vus), `PARCOURIR`, `TESTER`, `METTRE A JOUR`, `RENVOYER` (et `APPEL RECURSIF` pour la version récursive).

Règle 5 — La différence cruciale avec les arbres

Dans un arbre, on ne peut jamais revenir sur ses pas : les sous-arbres sont disjoints. Dans un graphe, un voisin peut **déjà avoir été vu** (par un cycle, un raccourci…). Il est donc **indispensable de mémoriser les sommets déjà découverts**, sinon le parcours tourne en rond **indéfiniment**.

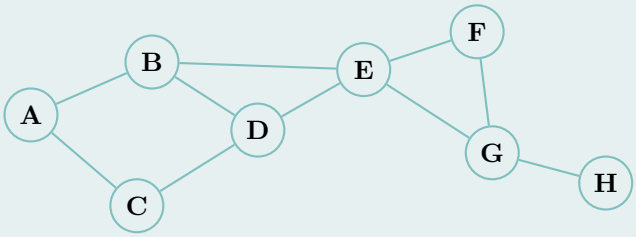
↻ **Retour sur l'activité.** Dans l'activité du musée, le premier gardien tournait en rond faute de cocher les salles visitées, et le second restait bloqué faute de liste des salles à visiter : la mémoire des sommets découverts et la structure d'attente `S` corrigent exactement ces deux défauts.

Et voici la clé de tout le chapitre : **le choix de la structure `S` change la nature du parcours**. C'est là que les piles et les files du début d'année reviennent en force.

Règle 6 — File ou pile : les deux parcours

- `S` est une **file** (FIFO) ⇒ on visite d'abord les sommets **les plus proches** du départ ⇒ **parcours en largeur** (*BFS*, Breadth First Search).
- `S` est une **pile** (LIFO) ⇒ on file « le plus loin possible » avant de revenir en arrière ⇒ **parcours en profondeur** (*DFS*, Depth First Search).

Pour la suite, on utilise ce graphe à 8 sommets (les voisins sont toujours donnés par ordre alphabétique) :



B. Le parcours en largeur (BFS)

On explore le graphe « par cercles concentriques » : d'abord le départ, puis **tous ses voisins** (distance 1), puis les voisins de ceux-ci (distance 2), etc. On tient à jour :

- une liste `traites` : les sommets déjà visités (renvoyée à la fin) ;
- une liste `decouverts` : tous les sommets déjà rencontrés (pour ne pas les remettre en file) ;
- une **file** `en_attente` : les sommets découverts mais pas encore visités. On utilise la structure abstraite `File` du chapitre *Structures linéaires* : `enfiler`, `defiler`, `est_vide`. Comme au chapitre *Arbres*, on **admet** qu'une implémentation existe : seule compte ici l'**interface**.

```
1  def BFS(g, depart):
2      """Parcours en largeur de g depuis depart ;
3         renvoie la liste des sommets dans l'ordre de visite."""
4      traites = []                          # INITIALISER
5      decouverts = [depart]                 # INITIALISER : sommets deja vus
6      en_attente = File()                   # INITIALISER : la file (interface admise)
7      en_attente.enfiler(depart)
8      while not en_attente.est_vide():      # PARCOURIR : tant que la file n'est pas vide
9          sommet = en_attente.defiler()     # on defile (FIFO)
10         for voisin in g.voisins(sommet):  # PARCOURIR : les voisins
11             if voisin not in decouverts:  # TESTER : AVANT d'enfiler
12                 decouverts.append(voisin) # METTRE A JOUR
13                 en_attente.enfiler(voisin)# METTRE A JOUR : on enfile
14         traites.append(sommet)            # METTRE A JOUR : sommet visite
15     return traites                        # RENVOYER
```

▷ **Je prédis.** Sur le graphe ci-dessus, dans quel ordre `BFS(g, 'B')` visite-t-il les sommets ?
▷ **Je vérifie.** `['B', 'A', 'D', 'E', 'C', 'F', 'G', 'H']` : on visite `B`, puis ses voisins `A, D, E`, puis les nouveaux voisins de ceux-ci, etc.

Voici l'état de la file `en_attente` après chaque tour de boucle (à gauche, le sommet défilé et visité ; la tête de la file est à gauche, les sommets qui viennent d'être enfilés sont foncés) :

défile		file après le tour
B	enfile A, D, E	A D E
A	enfile C	D E C
D	rien à enfiler	E C
E	enfile F, G	C F G
C	rien à enfiler	F G
F	rien à enfiler	G
G	enfile H	H
H	rien à enfiler	*file vide : fin*

Remarque — *Pourquoi* `decouverts` *?*
Un même sommet peut être le voisin de plusieurs autres. Sans la liste `decouverts`, on l'enfilerait plusieurs fois. Le test `if voisin not in decouverts`, fait **avant d'enfiler**, garantit que chaque sommet n'entre **qu'une fois** dans la file. Tester seulement `traites` ne suffirait pas : un sommet déjà *en attente* (découvert mais pas encore visité) serait enfilé une seconde fois.

Remarque — *Et si l'on écrivait la file avec une* `list` *?*
On pourrait remplacer `File()` par une `list` Python, en enfilant avec `append` et en défilant avec `pop(0)`. Le parcours serait juste, mais **lent** sur un grand graphe : chaque `pop(0)` décale tous les éléments restants et coûte $O(n)$ (voir le chapitre *Structures linéaires*). D'où l'intérêt de raisonner avec l'**interface** `File`, quitte à choisir ensuite une implémentation efficace (liste chaînée, deux piles).

1. Application vedette : le plus court chemin

Le BFS découvre les sommets par distance croissante. Donc, si l'on **mémorise le « parent »** de chaque sommet (celui qui l'a fait découvrir), on peut ensuite **remonter** de l'arrivée jusqu'au départ : on obtient un chemin, et — miracle du BFS — c'est le **plus court** (en nombre d'arêtes).

```
1  def plus_court_chemin(g, depart, arrivee):
2      """Renvoie un plus court chemin de depart a arrivee (liste de sommets),
3         ou None s'il n'y en a pas."""
4      decouverts = [depart]                    # INITIALISER
5      en_attente = File()                      # INITIALISER
6      en_attente.enfiler(depart)
7      parent = {depart: None}                  # INITIALISER : depart n'a pas de parent
8      while not en_attente.est_vide():         # PARCOURIR
9          sommet = en_attente.defiler()
10         if sommet == arrivee:                # TESTER : arrive, on reconstruit
11             chemin = [arrivee]
12             while parent[chemin[0]] is not None:
13                 chemin = [parent[chemin[0]]] + chemin
14             return chemin                    # RENVOYER
15         for voisin in g.voisins(sommet):     # PARCOURIR
16             if voisin not in decouverts:     # TESTER
17                 decouverts.append(voisin)    # METTRE A JOUR
18                 en_attente.enfiler(voisin)   # METTRE A JOUR
19                 parent[voisin] = sommet      # METTRE A JOUR : qui l'a decouvert
20     return None                              # RENVOYER : arrivee inaccessible
```

▷ **Je prédis.** Sur le graphe à 8 sommets, que renvoie `plus_court_chemin(g, 'A', 'H')` ?
▷ **Je vérifie.** `['A', 'B', 'E', 'G', 'H']` (4 arêtes). `E` est découvert depuis `B`, défilé avant `D` : c'est `B` qui devient son parent.

Propriété 4 — Pourquoi c'est le plus court

Le BFS visite tous les sommets à distance k **avant** ceux à distance $k+1$. Si l'arrivée est à distance k du départ, elle sera donc découverte comme voisine d'un sommet à distance $k-1$. En remontant les parents, on traverse exactement une « couche » à chaque étape : le chemin obtenu a la longueur minimale k. C'est pour cela que le BFS est au cœur du calcul d'itinéraires sur graphe non pondéré.

C. Le parcours en profondeur (DFS)

Ici, on va **le plus loin possible** avant de faire demi-tour : on visite un voisin, puis **son** premier voisin, et ainsi de suite jusqu'à l'impasse ; alors on **revient au dernier embranchement** pour explorer une autre branche. C'est exactement la stratégie « toujours à droite » dans un labyrinthe.

Ce parcours s'écrit **naturellement de façon récursive** : visiter un sommet, c'est se visiter soi-même puis lancer le parcours sur chaque voisin non encore visité.

```
def DFS(g, sommet, traites=None):
    if traites is None:                 # INITIALISER (premier appel)
        traites = []
    traites.append(sommet)              # METTRE A JOUR : sommet visite
    for voisin in g.voisins(sommet):    # PARCOURIR
        if voisin not in traites:       # TESTER
            DFS(g, voisin, traites)     # APPEL RECURSIF
    return traites                      # RENVOYER
```

Le paramètre `traites` a la valeur par défaut `None`, remplacée par une liste neuve au premier appel : c'est la bonne pratique vue au chapitre *Récursivité* (une valeur par défaut `[]` serait partagée d'un appel de l'utilisateur à l'autre).

▷ **Je prédis.** Sur le graphe précédent, dans quel ordre `DFS(g, 'A')` visite-t-il les sommets ?

▷ **Je vérifie.** `['A', 'B', 'D', 'C', 'E', 'F', 'G', 'H']` : depuis `A` on plonge par `B`, puis `D`, puis `C` (impasse), on remonte, puis `E`, `F`, `G`, `H`.

1. Version itérative, avec une pile

Le DFS s'écrit aussi **sans récursion**, en remplaçant la **file** du BFS par une **pile** (structure abstraite `Pile`, elle aussi admise : `empiler`, `depiler`, `est_vide`) : le dernier sommet empilé repart en premier.

```
def DFS_iteratif(g, depart):
    traites = []                                # INITIALISER
    en_attente = Pile()                         # INITIALISER : la pile (interface admise)
    en_attente.empiler(depart)
    while not en_attente.est_vide():            # PARCOURIR
        sommet = en_attente.depiler()           # on depile (LIFO)
        if sommet not in traites:               # TESTER : deja traite, on l'ignore
            traites.append(sommet)              # METTRE A JOUR : sommet visite
            for voisin in g.voisins(sommet):    # PARCOURIR : les voisins
                if voisin not in traites:       # TESTER
                    en_attente.empiler(voisin)  # METTRE A JOUR : on empile
    return traites                              # RENVOYER
```

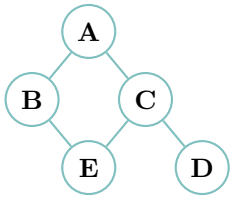
▷ **Je prédis.** Sur le graphe précédent, dans quel ordre `DFS_iteratif(g, 'A')` visite-t-il les sommets ? Est-ce l'ordre du DFS récursif ?

▷ **Je vérifie.** `['A', 'C', 'D', 'E', 'G', 'H', 'F', 'B']` : le **dernier** voisin empilé ressort le premier, d'où un ordre différent du DFS récursif — mais c'est bien, lui aussi, un parcours en profondeur.

Règle 7 — Où placer le test : file ou pile, ce n'est pas pareil

- **En largeur (file)** : on teste `decouverts` **avant d'enfiler**. Chaque sommet n'entre qu'une fois dans la file, et l'ordre « par couches » est respecté.
- **En profondeur itératif (pile)** : on fait l'inverse, **exprès**. Un sommet peut être empilé plusieurs fois (par plusieurs voisins) ; on le marque `traites` seulement **quand on le dépile**, et on ignore les exemplaires suivants. C'est ce qui garantit que l'on plonge toujours depuis le sommet le plus *récent* : si l'on marquait les sommets dès l'empilement, le parcours obtenu ne serait plus vraiment « en profondeur » (exemple ci-dessous).

Exemple — *Marquer à l'empilement ou au dépilement ?*
Arêtes `A--B`, `A--C`, `B--E`, `C--D`, `C--E`, départ `A`, voisins pris par ordre alphabétique. Dans le tableau, la pile est écrite en ligne, **sommet à droite** ; chaque ligne correspond à un sommet dépilé.



marqué à l'empilement		marqué au dépilement (`DFS_iteratif`)	
dépile	**pile ensuite**	**dépile**	**pile ensuite**
`A`	`B C` (`B`, `C` marqués)	`A`	`B C`
`C`	`B D E` (`D`, `E` marqués)	`C`	`B D E`
`E`	`B D` (`B` déjà marqué)	`E`	`B D B` (`B` pas encore traité)
`D`	`B`	`B`	`B D`
`B`	(vide)	`D`	`B`
		`B`	(vide) : déjà traité, ignoré
ordre : `A, C, E, D, B`		ordre : `A, C, E, B, D`	

En marquant dès l'empilement (à gauche), `E` ne peut pas empiler `B`, déjà marqué mais jamais visité : on repart de `D`. Or un vrai parcours en profondeur, arrivé en `E`, continue vers son voisin `B` avant de revenir en arrière : c'est l'ordre `A, C, E, B, D` que donne bien `DFS_iteratif` (à droite).

À retenir — une file pour la largeur, une pile pour la profondeur

BFS et DFS partagent le même squelette, mais pas la même structure : le BFS range les sommets en attente dans une **file** (FIFO, premier arrivé, premier sorti), le DFS dans une **pile** (LIFO, dernier arrivé, premier sorti). Ce seul choix explique leurs comportements opposés : la file sert d'abord les sommets découverts le plus tôt, donc les plus **proches** du départ (cercles concentriques) ; la pile sert d'abord le dernier découvert, donc on s'**enfonce** le plus loin possible. Toute la puissance du chapitre *Structures linéaires* est là : la structure de données choisie *dicte* le comportement de l'algorithme. Il n'existe pas « un seul » BFS ni « un seul » DFS : ce qui les caractérise est la **méthode de découverte**, pas l'ordre exact des voisins.

D. Deux applications directes : chemin et cycle

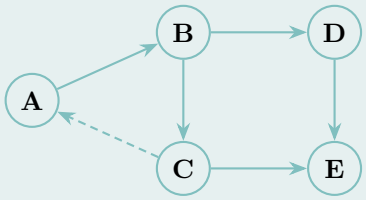
- **Existe-t-il un chemin de `u` à `v` ?** On lance un parcours depuis `u` ; il existe un chemin si et seulement si `v` figure dans les sommets visités. (Un parcours depuis `u` explore **toute** la composante connexe de `u`.)
- **Le graphe est-il connexe ?** Il l'est si et seulement si un parcours depuis *n'importe quel* sommet visite **tous** les sommets.
- **Y a-t-il un cycle ?** On le repère aussi au cours d'un parcours… mais le critère **n'est pas le même** selon que le graphe est orienté ou non : c'est l'objet du paragraphe suivant.

1. Repérer un cycle : deux critères, selon l'orientation

Graphe non orienté. Au cours d'un parcours, si l'on retombe sur un sommet **déjà découvert** qui n'est pas le sommet d'où l'on vient d'arriver (son « parent »), c'est qu'un cycle boucle par là. On exclut le parent parce qu'une arête se lit dans les deux sens : l'arête `A -- B` fait apparaître `A` parmi les voisins de `B`, sans former de cycle pour autant.

Graphe orienté : ce critère est faux. Reprenons le graphe orienté de la partie II (rappelé ci-dessous) et oublions un instant l'arc `C→A`, en pointillés. Un parcours en profondeur depuis `A` découvre `B`, `C`, puis `E` (par l'arc `C→E`) ; il revient en arrière, explore `D`, et l'arc `D→E` le fait retomber sur `E`, déjà découvert, alors qu'il arrive de `D`. Le critère non orienté crierait au cycle… à tort : `B→C→E` et `B→D→E` sont deux chemins qui **se rejoignent**, mais aucun arc ne permet de repartir

de `E` pour revenir en arrière. Même piège dans la recette de l'exercice 16 du chapitre : la tâche `(f)` est atteinte à la fois depuis `(e)` et depuis `(d)`, sans le moindre cycle.



Pour s'en sortir, on distingue, pendant un parcours en **profondeur**, trois états pour chaque sommet :

- **non vu** : le parcours ne l'a pas encore atteint ;
- **en cours** : on a commencé à l'explorer, mais pas fini d'explorer ses successeurs. Il est encore dans la **pile d'appels** : c'est un sommet du chemin qui mène du départ au sommet actuel ;
- **terminé** : tous ses successeurs ont été explorés ; on est sorti de son appel.

Règle 8 — Cycle dans un graphe orienté

Au cours d'un parcours en profondeur d'un graphe **orienté**, il y a un cycle lorsqu'un arc mène à un sommet **en cours** : on peut alors suivre le chemin en cours jusqu'au sommet actuel, puis revenir au point de départ par cet arc. Un arc qui mène à un sommet **terminé** ne signale **pas** de cycle (deux chemins qui se rejoignent).

Exemple — *Les deux situations sur le même graphe*

Sans l'arc `C`→`A`. `A`, `B`, `C` passent « en cours », puis `E`, qui n'a aucun successeur : `E` est aussitôt « terminé », puis `C`. On revient à `B`, qui explore `D` ; l'arc `D`→`E` mène à un sommet **terminé** : pas de cycle. Finalement, tout est terminé : le graphe n'a pas de cycle.

Avec l'arc `C`→`A`. Quand on explore `C`, les sommets `A`, `B`, `C` sont « en cours ». L'arc `C`→`A` mène à un sommet **en cours** : il y a un cycle, `A`→`B`→`C`→`A`.

En Python, avec le dictionnaire `G` des successeurs et un dictionnaire `etat` (un sommet absent de `etat` n'a pas encore été vu) :

```
def cycle_depuis(G, sommet, etat):
    """Parcours en profondeur du graphe oriente G depuis sommet ;
       renvoie True si l'on retombe sur un sommet en cours (cycle)."""
    etat[sommet] = 'en cours'                    # METTRE A JOUR : entre dans la pile d'appels
    for voisin in G[sommet]:                     # PARCOURIR
        if voisin not in etat:                   # TESTER : jamais vu, on plonge
            if cycle_depuis(G, voisin, etat):    # APPEL RECURSIF
                return True                      # RENVOYER
        elif etat[voisin] == 'en cours':         # TESTER
            return True                          # RENVOYER : retour sur le chemin en cours, cycle
        # sinon voisin est 'termine' : deja explore, pas de cycle par la
    etat[sommet] = 'termine'                     # METTRE A JOUR : successeurs tous explores
    return False                                 # RENVOYER

def a_un_cycle(G):
    etat = {}                                    # INITIALISER
    for s in G:                                  # PARCOURIR : relancer depuis chaque sommet non vu
        if s not in etat and cycle_depuis(G, s, etat):
            return True                          # RENVOYER
    return False                                 # RENVOYER
```

Sur le graphe orienté ci-dessus :

▷ Je prédis. Cacher les réponses, prévoir chaque résultat, puis vérifier.

```
>>> G = {'A': ['B'], 'B': ['C', 'D'], 'C': ['A', 'E'], 'D': ['E'], 'E': []}
>>> a_un_cycle(G)
True
>>> G['C'] = ['E']                  # on retire l'arc C -> A
>>> a_un_cycle(G)
False
```

Remarque

Ce critère « sommet en cours » permet enfin de programmer la détection d'interblocage vue au chapitre *Processus* : le **graphe d'attente** des processus y est un graphe **orienté**, et un cycle y signale un **interblocage**.

[au-delà du programme] **Et pour un graphe pondéré ?** Le BFS trouve le plus court chemin en *nombre d'arêtes*, mais ignore les poids. Pour le plus court chemin en *distance* (un vrai GPS), on utilise l'**algorithme de Dijkstra** (1959), qui visite les sommets par coût total croissant à l'aide d'une file de priorité. C'est l'objet du projet proposé en fin de chapitre — mais vous en connaissez déjà l'idée directrice. On le retrouvera au chapitre 9, derrière le protocole de routage OSPF.

► **Exercices d'application :** exercices 9 à 14 (parcours à la main et programmés ; l'ordre des voisins ; vrai ou faux parcours, arbre du parcours en largeur ; chemins, connexité) et 22 (parcours à la main en série) ; pour les cycles orientés, voir aussi l'exercice type bac 16 (questions 4 et 5)

8

Pour les curieux — *Un peu d'histoire… et beaucoup de génie* (facultatif)

La théorie des graphes naît avec Euler et les ponts de Königsberg, en 1736 (§ I). Deux siècles plus tard, en **1956**, le Néerlandais **Edsger Dijkstra** cherche un joli exemple pour montrer les capacités d'un nouvel ordinateur. Assis à la **terrasse d'un café d'Amsterdam** avec sa fiancée, **sans papier ni crayon**, il conçoit *en vingt minutes* son fameux algorithme du plus court chemin — publié en 1959 dans un article de **trois pages**. « L'une des raisons pour lesquelles il est si élégant, dira-t-il, c'est que je l'ai conçu sans stylo. » Aujourd'hui, cet algorithme guide chacun de vos trajets GPS.

Et l'histoire continue : en **1967**, le psychologue **Stanley Milgram** met en évidence le phénomène du « petit monde » — les fameux **six degrés de séparation** qui relieraient deux humains quelconques. En **1998**, deux étudiants, **Larry Page** et **Sergueï Brin**, modélisent le Web comme un gigantesque graphe orienté : leur algorithme **PageRank** deviendra… **Google**. Des ponts de Königsberg à votre moteur de recherche, c'est la même idée qui travaille : *ce qui compte, ce sont les relations.*

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise :** la **file** (parcours en largeur) et la **pile** (parcours en profondeur) de *Structures linéaires : piles, files, listes chaînées* ; la *Récursivité* pour le parcours en profondeur ; la classe `Graphe` (*Programmation orientée objet et paradigmes*).
- **Déjà croisé :** un arbre est un graphe connexe sans cycle (*Arbres*) ; un cycle dans le graphe d'attente signale un interblocage (*Processus et ordonnancement*).
- **Pour aller plus loin :** l'algorithme de Dijkstra sera au cœur du routage OSPF (*Réseaux*).

Bilan — la carte mémoire

Notion	À retenir
Graphe	des **sommets** reliés par des **arêtes** (relations)
Orienté / non orienté	arcs à sens unique / arêtes à double sens
Ordre / taille	nombre de sommets n / nombre d'arêtes m
Voisins, degré	sommets adjacents ; nombre de voisins
Poignées de main	$\sum \deg(s) = 2m$ (nb. de sommets de degré impair : pair)
Chaîne / chemin, cycle	suite de sommets reliés ; qui revient au départ
Connexe	« d'un seul tenant » (sinon : composantes connexes)
Matrice d'adjacence	tableau $n \times n$ de 0/1 ; mémoire $O(n^2)$; arête en $O(1)$
Liste d'adjacence	dictionnaire sommet $\to$ voisins ; mémoire $O(n+m)$
BFS (largeur)	une **file** (FIFO), cercles concentriques, **plus court chemin**
DFS (profondeur)	une **pile** (LIFO, ou la récursivité), « le plus loin d'abord »
Mémoriser les visités	**obligatoire** sur un graphe (cycles !)
Cycle, non orienté	retomber sur un sommet déjà découvert, autre que le parent
Cycle, orienté	retomber (en profondeur) sur un sommet **en cours**

Retour à la situation de départ

1. Par un **graphe** : des **sommets** reliés par des **arêtes** (ou des arcs, s'il est orienté), représenté en Python par une **matrice d'adjacence** ou une **liste d'adjacence**, un dictionnaire qui associe à chaque sommet ses voisins (§ I à III).
2. Elle lance un **parcours en largeur**, avec une **file**, depuis la station de départ : les sommets sont découverts par distance croissante, et en remontant les « parents » on obtient un **plus court chemin** en nombre d'arêtes (§ V.B).
3. En **mémorisant les sommets déjà découverts** : un parcours, en largeur ou en profondeur, visite ainsi chaque sommet une seule fois. Retomber sur un sommet déjà vu (autre que le parent, ou « en cours » s'il est orienté) révèle un **cycle** (§ V).

Erreurs fréquentes

- **Oublier de mémoriser les sommets visités.** Sur un graphe (contrairement à un arbre), le parcours **boucle à l'infini**. *Le réflexe :* une liste `decouverts` / `traites`, toujours.
- **Confondre file et pile. File** (FIFO) = BFS = largeur ; **pile** (LIFO) = DFS = profondeur. Le squelette est le même, c'est la structure qui change tout.
- **Défiler avec `pop(0)` sur une `list`.** Le parcours reste juste, mais chaque `pop(0)` coûte $O(n)$ (chapitre *Structures linéaires*). *Le réflexe :* raisonner avec l'interface `File` (`enfiler`/`defiler`/`est_vide`).
- **Appliquer à un graphe orienté le critère de cycle du non orienté.** Retomber sur un sommet *déjà découvert* ne prouve rien en orienté (deux chemins peuvent se rejoindre) : il faut retomber sur un sommet **en cours**.
- **Confondre arête et arc.** Non orienté $\to$ **arête** (deux sens), matrice **symétrique** ; orienté $\to$ **arc** (un sens).
- **Croire que la matrice est toujours le bon choix.** Pour un graphe **creux**, la liste d'adjacence est bien plus économe ($O(n+m)$ contre $O(n^2)$).

- **Enfiler un sommet déjà en attente (BFS).** En largeur, on teste l'appartenance à `decouverts` *avant* d'enfiler, pas seulement à `traites`. (En profondeur itératif, c'est l'inverse : on teste `traites` au moment de dépiler, voir le cours.)
- **Oublier la double insertion** dans `ajoute_arete` d'un graphe non orienté (il faut ajouter `a` chez `b` **et** `b` chez `a`).

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

☐ employer le **vocabulaire** (sommet, arête/arc, voisin, degré, chaîne/chemin, cycle, connexe) → ex. 1, 3 ;

☐ **modéliser** une situation concrète par un graphe orienté ou non, pondéré ou non (réseau, plan, grille ou labyrinthe) → ex. 2, 3, 15, 23 ;

☐ écrire une **matrice d'adjacence**, une **liste d'adjacence** (de successeurs, de prédécesseurs), et **passer** de l'une à l'autre → ex. 4, 5, 6 ;

☐ écrire / compléter une **classe `Graphe`** → ex. 8, 10 ;

☐ dérouler et coder un **parcours en largeur** (file) et en **profondeur** (pile ou récursif) → ex. 9 à 12, 22 ;

☐ trouver un **chemin**, un **plus court chemin** (BFS), tester la **connexité** → ex. 13, 14, 23 ;

☐ repérer un **cycle**, en distinguant graphe non orienté (sommet déjà découvert, autre que le parent) et orienté (sommet **en cours**) → ex. 16 (q. 4, 5, 7), 22 c.

Vers le Grand Oral

- **Comment un GPS calcule-t-il le plus court chemin ?** *(modélisation en graphe pondéré ; BFS pour le nombre d'étapes, Dijkstra pour la distance ; l'anecdote du café d'Amsterdam.)*
- **Pourquoi dit-on que nous sommes tous à « six poignées de main » ?** *(graphe des relations, phénomène du petit monde, distance dans un graphe.)*
- **Largeur ou profondeur : comment le choix d'une structure de données change-t-il un algorithme ?** *(file vs pile ; un même squelette, deux comportements.)*
- **En quoi le problème des sept ponts a-t-il fondé une science ?** *(l'abstraction d'Euler ; modéliser, c'est jeter l'inutile.)*

EXERCICES

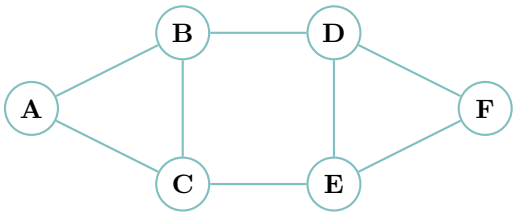
Mode d'emploi

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 32.
- On dispose de la classe `Graphe` du cours (graphes non orientés) : constructeur `Graphe(sommets)`, méthodes `ajoute_arete(a, b)`, `voisins(s)`, `sont_voisins(a, b)`.

```
class Graphe:
    def __init__(self, sommets):
        self.sommets = sommets
        self.adj = {}
        for s in sommets:
            self.adj[s] = []
    def ajoute_arete(self, a, b):
        self.adj[a].append(b)
        self.adj[b].append(a)
    def voisins(self, s):
        return self.adj[s]
    def sont_voisins(self, a, b):
        return b in self.adj[a]
```


Vocabulaire et modélisation ↑ cours

★ **Exercice 1** IA IA IA ▶ corrigé p. 910 • essentiel On considère le graphe non orienté ci-dessous.



1. Donner la liste des voisins de `B`, puis le degré de chaque sommet.
2. Donner une chaîne de longueur 4 reliant `A` à `F`.
3. Ce graphe est-il connexe ? Justifier.
4. Donner un cycle de ce graphe.

▷ coup de pouce

★ **Exercice 2** IA IA IA ▶ corrigé p. 911 Dans une classe, on relève les binômes de travail : Alice travaille avec Bilal et Chloé ; Bilal avec Alice, Chloé et Dora ; Chloé avec Alice, Bilal et Émile ; Dora avec Bilal ; Émile avec Chloé.

1. Dire si le graphe qui modélise ces binômes est orienté ou non orienté, et pourquoi.
2. Dessiner le graphe (un sommet par initiale).
3. Qui a le plus de partenaires ? Traduire cette question en vocabulaire de graphe.

▷ coup de pouce

★ **Exercice 3** IA IA IA ▶ corrigé p. 911 • essentiel Sur un réseau social, on note « $X \to Y$ » lorsque « X suit Y ». On relève : `A`→`B`, `A`→`C`, `B`→`C`, `C`→`A`, `D`→`C`.

1. Dessiner le graphe **orienté** correspondant.
2. Donner les **successeurs** de `A`, puis les **prédécesseurs** de `C`.
3. Existe-t-il un chemin de `D` vers `B` ? de `B` vers `D` ? Que remarque-t-on ?
4. Ce graphe possède-t-il un cycle (au sens orienté) ?

▷ coup de pouce

Représenter un graphe ↑ cours

★ **Exercice 4** IA IA IA ▶ corrigé p. 911 • essentiel On reprend le graphe non orienté de l'exercice 1 (sommets `A` à `F`, ordre alphabétique).

1. Écrire sa **matrice d'adjacence**. Vérifier qu'elle est symétrique.
2. Écrire sa **liste d'adjacence** (dictionnaire Python).

▷ coup de pouce

★ **Exercice 5** IA IA IA ▶ corrigé p. 911 Construire (dessiner) le graphe correspondant à chacune des représentations suivantes.

1. $M = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{pmatrix}$ (sommets `A`, `B`, `C`, `D`, `E`)
2. la liste de listes `adj = [[1, 2], [0, 2, 3], [0, 1], [1, 4], [3]]` (sommets 0 à 4).

▷ coup de pouce

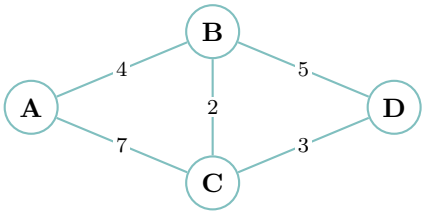
★★ **Exercice 6** IA IA IA ▶ ▶ corrigé p. 912 On numérote les sommets de 0 à $n-1$. Un graphe est donné soit par sa matrice `M` (liste de listes de 0/1), soit par ses listes d'adjacence `adj` (liste de listes d'entiers).

1. Écrire `matrice_vers_listes(M)` qui renvoie les listes d'adjacence correspondantes.
2. Écrire `listes_vers_matrice(adj)` qui renvoie la matrice d'adjacence correspondante.
3. Tester que les deux fonctions sont bien « inverses » l'une de l'autre sur un exemple.
4. Un graphe **orienté** est donné par le dictionnaire `succ` de ses **listes de successeurs**, par exemple `succ = {'A': ['B', 'C'], 'B': ['C'], 'C': ['A'], 'D': ['C']}`. Écrire `predecesseurs(succ)` qui renvoie le dictionnaire des **listes de prédécesseurs** (pour l'exemple : `{'A': ['C'], 'B': ['A'], 'C': ['A', 'B', 'D'], 'D': []}`).

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 7** IA IA IA ▶ corrigé p. 913 On considère le graphe **pondéré** suivant (les poids sont des distances en km).



1. Proposer une **matrice de poids** pour ce graphe (mettre 0 sur la diagonale et là où il n'y a pas d'arête).

2. Donner, en énumérant les chemins, **le ou les** trajets les plus courts (en km) de `A` à `D`, et leur longueur.
▷ coup de pouce

La classe `Graphe` ↑cours

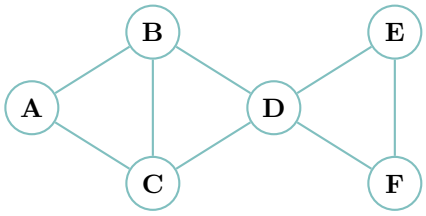
★★ **Exercice 8** ▶ ▶ corrigé p. 913 • essentiel En utilisant **uniquement** l'interface de la classe `Graphe` (l'attribut `sommets` et les méthodes `voisins` et `sont_voisins`), écrire les fonctions suivantes.

1. `degre(g, s)` : renvoie le degré du sommet `s`.
2. `est_isole(g, s)` : renvoie `True` si `s` n'a aucun voisin.
3. `nb_aretes(g)` : renvoie le nombre d'arêtes du graphe.
▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Parcourir un graphe ↑cours

★★ **Exercice 9** ▶ ▶ corrigé p. 913 • essentiel On considère ce graphe (les voisins seront toujours pris par **ordre alphabétique**).



1. Donner « à la main » le parcours en **largeur** (BFS) depuis `A`, en écrivant à chaque étape le contenu de la file.
2. Donner « à la main » le parcours en **profondeur** (DFS) depuis `A`.
3. Les deux parcours visitent-ils les mêmes sommets ? Dans le même ordre ?

▷ coup de pouce

★★ **Exercice 10** ▶ ▶ corrigé p. 914

1. Programmer la classe `Graphe`, puis les fonctions `BFS(g, depart)` et `DFS(g, sommet)` (récursive) du cours. Pour la file du BFS, reprendre la classe `File` du chapitre *Structures linéaires*.
▷ coup de pouce
2. Construire le graphe de l'exercice précédent et vérifier vos réponses aux traces à la main.
3. Programmer aussi `DFS_iteratif(g, depart)` (avec une pile). Donne-t-il le même ordre que le DFS récursif ? Expliquer.
▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 11** ▶ ▶ corrigé p. 914 • essentiel On a construit un graphe non orienté `g` avec la classe `Graphe`, en ajoutant les arêtes dans un ordre quelconque. Voici son attribut `g.adj` : c'est **dans cet ordre** que `g.voisins(s)` renvoie les voisins (ils ne sont donc plus rangés par ordre alphabétique).

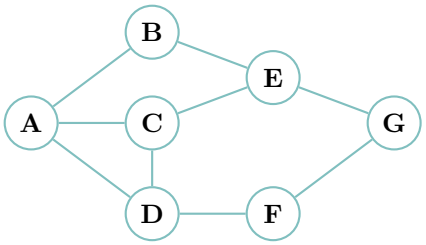
```
g.adj = {'A': ['D', 'B'], 'B': ['E', 'A', 'C'], 'C': ['B', 'F'],
         'D': ['A', 'E'], 'E': ['D', 'G', 'B'], 'F': ['C', 'G'],
         'G': ['F', 'E']}
```

1. Dessiner le graphe. Combien a-t-il d'arêtes ?

2. Donner à la main le résultat de `BFS(g, 'A')` (fonction du cours), en écrivant le contenu de la file après chaque sommet défilé.
3. Donner à la main le résultat de `DFS(g, 'A')` (version récursive du cours).
4. Donner à la main le résultat de `DFS_iteratif(g, 'A')`, en écrivant le contenu de la pile après chaque sommet dépilé (sommet de la pile à droite, comme dans le cours). Pourquoi certains sommets sont-ils dépilés sans être traités ?
5. On range maintenant chaque liste de voisins par ordre alphabétique. Les ordres de visite changent-ils ? Qu'est-ce qui, en revanche, ne change pas dans le parcours en largeur ?
6. ▶ Vérifier les réponses avec les fonctions de l'exercice précédent (on peut affecter directement `g.adj`).

▷ coup de pouce

★★ **Exercice 12**  ▶ corrigé p. 915 On considère le graphe non orienté ci-dessous. On ne précise pas dans quel ordre la méthode `voisins` renvoie les voisins d'un sommet : selon cet ordre, un même parcours lancé depuis `A` peut donner des résultats différents.



1. Pour chacune des suites suivantes, dire si elle peut être le résultat de `BFS(g, 'A')`, de `DFS(g, 'A')` (récursif), ou d'aucun des deux, pour un ordre des voisins bien choisi. Justifier.

a) `A C B D E F G` **b)** `A B E C D F G` **c)** `A D F G E C B`
d) `A B C D F E G` **e)** `A B E G F C D`

2. Les voisins sont maintenant pris par ordre alphabétique. Dérouler `BFS(g, 'A')` en notant, pour chaque sommet découvert, le sommet depuis lequel il a été découvert (son **parent**). Dessiner les seules arêtes « sommet – parent » : on obtient un arbre, l'**arbre du parcours en largeur**. Que représente la profondeur d'un sommet dans cet arbre ? En déduire un plus court chemin de `A` à `G`. Est-ce le seul ?

▷ coup de pouce

Chemins, cycles, connexité ↑ cours

★★ **Exercice 13**  ▶ ▶ corrigé p. 915 • essentiel À l'aide d'un parcours, écrire :

1. `existe_chemin(g, u, v)` : renvoie `True` s'il existe une chaîne de `u` à `v`. ▷ coup de pouce
2. `est_connexe(g)` : renvoie `True` si le graphe est connexe. ▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 14**  ▶ ▶ corrigé p. 916 ≈ 30 min On veut le **plus court chemin** en nombre d'arêtes.

1. En vous inspirant du BFS, écrire `plus_court_chemin(g, depart, arrivee)` qui renvoie la liste des sommets d'un plus court chemin (ou `None` s'il n'en existe aucun). ▷ coup de pouce
▷ début de solution
2. Écrire `distances(g, depart)` qui renvoie un dictionnaire donnant, pour chaque sommet, sa distance (nombre d'arêtes) au départ. ▷ coup de pouce

3. Sur un graphe de votre choix, vérifier que `len(plus_court_chemin(...)) - 1` vaut bien la distance renvoyée.

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Vers le bac ↑cours

★★ **Exercice 15** IA IA IA *Un cercle d'amis* (d'après Amérique du Nord 2024, jour 1, ex. 2) ▶ corrigé p. 917 ≈ 45 min

On modélise un groupe de huit personnes (Anas, Emma, Gabriel, Jade, Lou, Milo, Nina, Yanis) par un graphe : les sommets sont les personnes, les arêtes les liens d'amitié.

- Gabriel est ami avec Jade, Yanis, Nina et Milo ;
- Jade est amie avec Gabriel, Yanis, Emma et Lou ;
- Yanis est ami avec Gabriel, Jade, Emma, Nina, Milo et Anas ;
- Emma est amie avec Jade, Yanis et Nina ;
- Nina est amie avec Gabriel, Yanis et Emma ;
- Milo est ami avec Gabriel, Yanis et Anas ;
- Anas est ami avec Yanis et Milo ;
- Lou est amie avec Jade.

Partie A — Matrice d'adjacence

1. Dessiner ce graphe (chaque personne par l'initiale de son prénom, entourée d'un cercle).
2. Recopier et compléter la matrice d'adjacence (un 1 pour un lien d'amitié, un 0 sinon).

```
# sommets :     G, J, Y, E, N, M, A, L
matrice_adj = [[0, 1, 1, 0, 1, 1, 0, 0],   # G
               [.....................],   # J
               ...                    ]
```

On dispose de `sommets = ['G','J','Y','E','N','M','A','L']` et d'une fonction `position(l, s)` qui renvoie la position de `s` dans la liste `l` (ou `None` s'il est absent).

3. Indiquer les retours de `position(sommets, 'G')` et `position(sommets, 'Z')`.
4. Recopier et compléter la fonction `nb_amis` qui renvoie le nombre d'amis du sommet `s` (ou `None` s'il est absent). ▷ coup de pouce

```
def nb_amis(L, m, s):
    pos_s = ...
    if pos_s == None:
        return ...
    amis = 0
    for i in range(len(m)):
        amis += ...
    return ...
```

5. Indiquer le retour de `nb_amis(sommets, matrice_adj, 'G')`.

Partie B — Dictionnaire de listes d'adjacence

6. Dans un dictionnaire `{c : v}`, que représentent `c` et `v` ici ?
7. Recopier et compléter le dictionnaire `graphe` de listes d'adjacence des amis.
8. Écrire `nb_amis(d, s)` qui renvoie le nombre d'amis de `s` (on suppose `s` présent dans `d`).

Milo se fâche avec Gabriel et Yanis, et Anas avec Yanis ; le nouveau graphe est :

```
graphe = {'G': ['J', 'Y', 'N'],       'J': ['G', 'Y', 'E', 'L'],
          'Y': ['G', 'J', 'E', 'N'],  'E': ['J', 'Y', 'N'],
          'N': ['G', 'Y', 'E'],       'M': ['A'],
          'A': ['M'],                 'L': ['J']}
```

Le « cercle d'amis » d'une personne est l'ensemble des personnes atteignables depuis elle ; on l'obtient par un **parcours en profondeur**.

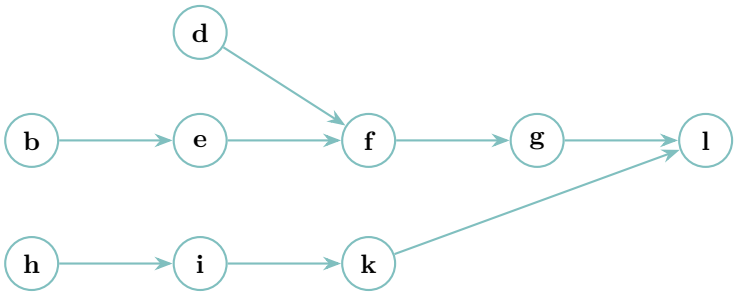
9. Donner le cercle d'amis de Lou.
10. Recopier et compléter la fonction de parcours en profondeur. ▷ coup de pouce

```
def parcours_en_profondeur(d, s, visites=None):
    if visites is None:
        visites = []
    ...
    for v in d[s]:
        ...
            parcours_en_profondeur(d, v, visites)
    return visites
```

★★ Exercice 16 *La recette du pain : un graphe de tâches* (d'après Asie 2024, jour 1, ex. 2)

▶ corrigé p. 918 ≈ 30 min

La fabrication d'un pain se décompose en tâches ; un **arc** `(i)`→`(j)` signifie « la tâche `(i)` doit être faite avant la tâche `(j)` ». On donne une partie du graphe des dépendances :



1. S'agit-il d'un graphe orienté ou non orienté ?
2. D'après le graphe, dire si l'on peut faire : `(f)` puis `(g)` ? `(g)` puis `(f)` ? `(b)` puis `(e)` ?
3. Donner toutes les tâches à réaliser **nécessairement avant** de pouvoir faire `(l)`. ▷ coup de pouce
4. Ce graphe contient-il un cycle ? Qu'est-ce que cela garantit sur la faisabilité de la recette ?
5. Un élève lance un parcours en profondeur depuis `(b)`, puis depuis `(d)`. Il applique la règle : « si l'on retombe sur un sommet déjà découvert qui n'est pas celui d'où l'on vient, il y a un cycle ». En arrivant de `(d)` sur `(f)`, déjà découvert, il conclut à un cycle. Expliquer son erreur et donner le bon critère pour un graphe orienté. ▷ coup de pouce

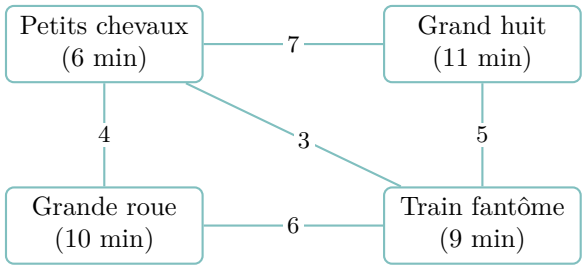
On considère maintenant un autre graphe de tâches, numérotées de 0 à 4, donné par sa matrice d'adjacence (`M[i][j] = 1` si la tâche `i` doit être faite avant la tâche `j`).

```
M = [ [0, 1, 0, 0, 0],
      [0, 0, 1, 0, 0],
      [0, 0, 0, 1, 0],
      [0, 1, 0, 0, 1],
      [0, 0, 0, 0, 0] ]
```

6. Dessiner le graphe associé (les sommets sont leurs indices 0 à 4).
7. Déterminer s'il est possible de trouver un ordre de réalisation des tâches respectant toutes les dépendances. Si oui, le donner ; si non, expliquer pourquoi. *(On parle d'un* tri topologique *du graphe.)* ▷ coup de pouce

★★ **Exercice 17** *Un parc d'attractions* (d'après Pays étrangers 2025, sujet PE2, ex. 3 (partie A)) ▶ corrigé p. 918 ≈ 30 min

Un parc d'attractions est modélisé par un graphe : chaque sommet est une attraction (de durée donnée), chaque arête porte la durée (en minutes) du trajet entre deux attractions.



Les attractions sont des objets de la classe suivante ; les voisines sont stockées sous forme de couples `(attraction, durée_du_trajet)`.

```
class Attraction:
    def __init__(self, nom, duree):
        self.nom = nom
        self.duree = duree
        self.voisines = []

a1 = Attraction("Grand huit", 11)
a2 = Attraction("Petits chevaux", 6)
a3 = Attraction("Train fantome", 9)
a4 = Attraction("Grande roue", 10)
a1.voisines = [(a2, 7), (a3, 5)]
a2.voisines = [(a1, 7), (a3, 3), (a4, 4)]
a3.voisines = [(a1, 5), (a2, 3), (a4, 6)]
a4.voisines = ...
```

1. La grande roue est ralentie : sa durée passe à 12 minutes. Écrire la ligne de code qui réalise cette modification.
2. Donner et expliquer la valeur de `a2.voisines[2][1]`. ▷ coup de pouce
3. Recopier et compléter la dernière ligne (`a4.voisines = ...`).
4. Expliquer pourquoi cette modélisation utilise un graphe **non orienté**.

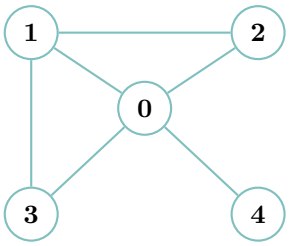
Une **balade** est un chemin du graphe ; sa durée est la somme des durées de ses sommets *et* de ses arêtes. On modélise une balade par la liste des attractions parcourues.

5. Calculer la durée de la balade `[a1, a2, a3]` et expliquer le calcul.
6. Expliquer pourquoi `[a2, a1, a4, a3]` n'est pas une balade.
7. ▶ Écrire `sont_voisines(x, y)` qui renvoie `True` si les attractions `x` et `y` sont voisines. ▷ coup de pouce
8. ▶ Écrire `est_balade(tab)` qui renvoie `True` si la liste `tab` d'attractions est une balade (chaque attraction voisine de la suivante). ▷ coup de pouce

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 18** *Un réseau d'ordinateurs* (d'après Centres étrangers 2024, groupe 1, jour 1, ex. 1) ▶ corrigé p. 919 ≈ 30 min

Un réseau de 5 ordinateurs numérotés 0 à 4 est représenté par des **listes de voisins**.



1. Recopier et compléter la variable `voisins` (la case d'indice `i` contient la liste des voisins de `i`).

```
voisins = [[1, 2, 3, 4],
           [0, 2, 3],
           [0, 1],
           [...],
           [...]]
```

2. On ajoute un ordinateur 5, accessible **seulement** depuis 0 et 2. Dessiner le nouveau graphe et donner la nouvelle variable `voisins`.
3. ▶ Écrire `voisin_alea(voisins, s)` qui renvoie un voisin de `s` choisi au hasard. *(On dispose de la fonction* `random.randrange(n)`*, qui renvoie un entier de* 0 *inclus à* `n` *exclu.)*

▷ coup de pouce

Au début, un virus n'est présent que sur un ordinateur. À chaque étape, chaque ordinateur contaminé contamine **tous** ses voisins non encore contaminés.

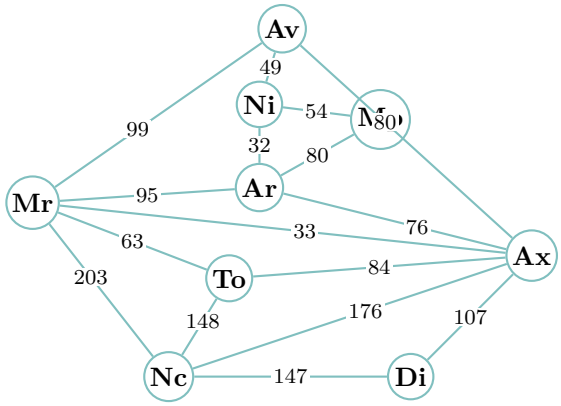
4. ▶ Écrire une fonction `temps_propagation(voisins, s)` qui renvoie le nombre d'étapes nécessaires pour que le virus, parti de l'ordinateur `s`, contamine **tout** le réseau. ▷ coup de pouce

▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 19** *Une agence de voyages* (d'après Polynésie 2024, jour 2, ex. 1) ▶ corrigé p. 920 ≈ 45 min

Une agence de voyages relie des villes ; le graphe pondéré ci-dessous donne les distances (en km).



1. Déterminer le plus court chemin (en km) de `Mp` à `Nc` et sa longueur. ▷ coup de pouce

2. On veut aller de `Mp` à `Nc` en **visitant le moins de villes possible**. Déterminer les deux chemins possibles.

On oublie désormais les distances (graphe non pondéré). On implémente le graphe par un dictionnaire de listes d'adjacence (sommets de type `str`).

3. Donner l'implémentation Python `G` de ce graphe (une paire clé/valeur par ligne).
4. Rappeler la signification de `LIFO` et `FIFO`. Lequel désigne une file ?

On donne la fonction `parcours` (la file utilise `creerFile`, `estVide`, `enfiler`, `defiler`) :

```
def parcours(graphe, sommet):
    f = creerFile()
    enfiler(f, sommet)
    visite = [sommet]
    while not estVide(f):
        s = defiler(f)
        for v in graphe[s]:
            if not (v in visite):
                visite.append(v)
                enfiler(f, v)
    return visite
```

5. S'agit-il d'un parcours en largeur ou en profondeur ? Justifier.
6. ▶ En modifiant `parcours`, écrire `distance(graphe, sommet)` qui renvoie un dictionnaire donnant, pour chaque sommet, sa distance (nombre d'arêtes) au sommet de départ.

▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

7. ▶ Écrire une fonction `parcours2(G, s)` réalisant un parcours en **profondeur** à l'aide d'une **pile**. Donner un résultat possible de `parcours2(G, 'Av')`. ▷ coup de pouce

Critiquer une réponse d'IA ↑ cours

★★ **Exercice 20** *Un parcours en largeur écrit par un assistant d'IA* ▶ corrigé p. 921

Un élève demande à un assistant d'IA : « Écris en Python un parcours en largeur d'un graphe `g` (dictionnaire des listes d'adjacence) depuis un sommet `depart`, qui renvoie la liste des sommets dans l'ordre où ils sont traités. » Voici la réponse obtenue :

```
def parcours_largeur(g, depart):
    en_attente = [depart]          # file des sommets decouverts
    decouverts = [depart]
    traites = []
    while en_attente != []:
        sommet = en_attente.pop()  # on retire le prochain sommet
        for voisin in g[sommet]:
            if voisin not in decouverts:
                decouverts.append(voisin)
                en_attente.append(voisin)
        traites.append(sommet)
    return traites
```

« On découvre d'abord tous les voisins du départ, puis les voisins des voisins, etc. : la liste `en_attente` joue le rôle de la file du parcours en largeur, et `decouverts` évite de traiter deux fois un même sommet. »

1. La réponse est-elle correcte ? ▶ Dérouler la fonction à la main sur le graphe `G = {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'E'], 'D': ['B'], 'E': ['C']}` depuis `'A'` (noter l'état de `en_attente` à chaque tour) et comparer avec l'ordre attendu d'un parcours en largeur.

2. Localiser et corriger l'erreur. ▷ coup de pouce
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

S'entraîner à l'oral ↑ cours

★★ **Exercice 21** IA IA IA *Expliquer en deux minutes* ▶ corrigé p. 921

Au Grand Oral comme devant l'examinateur de l'épreuve pratique, il faut savoir **expliquer** une notion clairement, sans lire ses notes. Choisir l'un des sujets suivants (ou le tirer au sort) :

Sujet 1. Expliquer la différence entre le parcours **en largeur** et le parcours **en profondeur** d'un graphe, et la structure (pile ou file) utilisée par chacun.

Sujet 2. Expliquer à un camarade de Première comment un **graphe** modélise un réseau routier ou un réseau social, avec le vocabulaire de base.

Sujet 3. Expliquer comment choisir entre une **matrice d'adjacence** et une **liste de successeurs** pour représenter un graphe.

Préparation (5 min, seul). Noter au brouillon **trois idées** et **un exemple**, rien de plus : comme au Grand Oral, ces notes servent d'aide-mémoire. **Exposé (2 min, chronométré).** Debout, face à un camarade, **sans lire ses notes** et sans rien écrire : tout passe par la parole. **Retour (2 min).** Le camarade pose une question de relance (on peut alors s'aider d'un schéma), remplit la grille ci-dessous et donne un conseil ; puis on échange les rôles. Cette grille reprend les critères d'évaluation du Grand Oral. ▷ coup de pouce

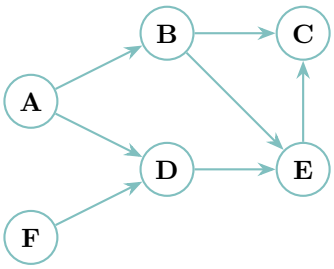
Grille entre camarades	Oui	En partie	Pas encore
Clair — audible, posé ; chaque mot technique est expliqué	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	☐	☐	☐
Construit — un fil conducteur, tenu en deux minutes, sans lire	☐	☐	☐
Un conseil pour la prochaine fois :			

Entraînement ↑ cours

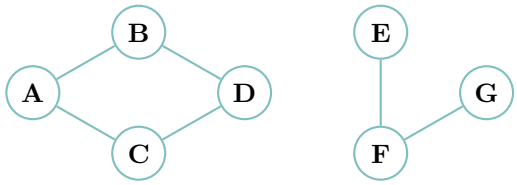
Des séries courtes pour rendre une méthode automatique : même consigne, données différentes. Les réponses sont brèves dans le corrigé.

★ **Exercice 22** IA IA IA *Parcours à la main en série* ▶ corrigé p. 922 • essentiel Pour chacun des trois graphes ci-dessous, donner l'ordre de visite du **parcours en largeur** depuis `A`, en écrivant le contenu de la file après chaque sommet défilé, puis l'ordre de visite du **parcours en profondeur** récursif (fonction `DFS` du cours) depuis `A`. Les voisins sont toujours pris par **ordre alphabétique**. Répondre enfin à la question propre à chaque graphe. ▷ coup de pouce

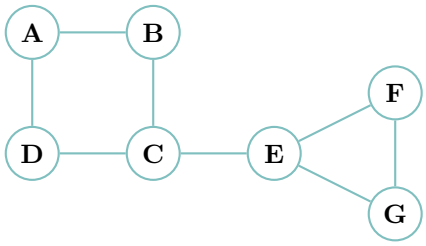
a) **Un graphe orienté** : on ne suit un arc que dans le sens de la flèche (les voisins d'un sommet sont ses successeurs). Quel sommet n'est jamais visité, et pourquoi ?



b) Un graphe non connexe. Quels sommets ne sont pas visités ? Comment compléter le travail pour visiter tous les sommets du graphe ?

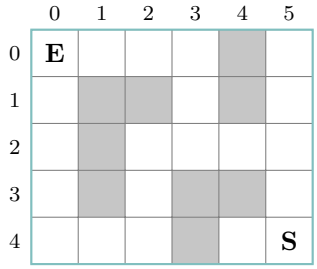


c) Un graphe avec des cycles. Pourquoi aucun des deux parcours ne tourne-t-il en rond ? En déroulant le parcours en profondeur, noter le **parent** de chaque sommet (le sommet depuis lequel il a été découvert) : à quels moments le critère du cours (un voisin déjà découvert, autre que le parent) révèle-t-il un cycle ? Quels cycles sont ainsi repérés ?



Modéliser une grille ↑cours

★★ **Exercice 23** *Sortir d'un labyrinthe* (type bac) ▶ corrigé p. 922 ≈ 40 min Un labyrinthe est dessiné sur une grille de 5 lignes et 6 colonnes. Les cases grisées sont des murs ; on se déplace d'une case libre à une case libre voisine **en haut, en bas, à gauche ou à droite** (jamais en diagonale). L'entrée `E` est la case de la ligne 0 et de la colonne 0, la sortie `S` celle de la ligne 4 et de la colonne 5. En Python, le labyrinthe est une liste de chaînes (`'#'` pour un mur, `'.'` pour une case libre) : la case de la ligne `i` et de la colonne `j` est `lab[i][j]`, et on la repère par le couple `(i, j)`.




```
lab = ["....#.",
       ".##.#.",
       ".#....",
       ".#.##.",
       "...#.."]
```

On modélise le labyrinthe par un graphe : les **sommets** sont les cases libres, et une **arête** relie deux cases libres voisines.

1. Pourquoi peut-on choisir un graphe **non orienté** ? Combien ce graphe a-t-il de sommets ? d'arêtes ?

2. Quelles cases n'ont qu'un seul voisin ? Que représentent-elles dans le labyrinthe ? Le graphe contient-il un cycle ? Si oui, en donner un.

3. ▶ Écrire une fonction `voisines(lab, i, j)` qui renvoie la liste des cases libres voisines de la case `(i, j)`, dans l'ordre **haut, bas, gauche, droite**, sans jamais sortir de la grille. Par exemple, `voisines(lab, 2, 3)` renvoie `[(1, 3), (2, 2), (2, 4)]` et `voisines(lab, 0, 0)` renvoie `[(1, 0), (0, 1)]`.

4. ▶ Écrire une fonction `construire_graphe(lab)` qui renvoie le graphe du labyrinthe, objet de la classe `Graphe` dont les sommets sont les couples `(i, j)` des cases libres. Les arêtes sont ajoutées avec `ajoute_arete`, et chacune **une seule fois**.

5. Dérouler à la main un parcours en largeur depuis l'entrée et donner la **distance** de chaque case libre à l'entrée (nombre de déplacements), en l'écrivant dans la case sur une copie de la grille. En déduire la longueur d'un plus court chemin de `E` à `S`, puis un tel chemin. Est-il unique ?

6. On suppose que `g.voisins(s)` renvoie les voisines dans l'ordre haut, bas, gauche, droite. On lance le parcours en profondeur récursif `DFS(g, (0, 0))` du cours, en notant le **parent** de chaque case (la case depuis laquelle elle a été découverte). En remontant les parents depuis `S`, quel chemin obtient-on ? Quelle est sa longueur ? Quel parcours faut-il utiliser pour obtenir à coup sûr un plus court chemin, et pourquoi ?

7. ▶ Vérifier la réponse à la question 1 en comptant les sommets et les voisins du graphe renvoyé par `construire_graphe(lab)`, puis la réponse à la question 5 à l'aide des fonctions `plus_court_chemin` et `distances` de l'exercice 14.

▷ coup de pouce

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

8

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Le degré d'un sommet est son nombre de voisins. Une chaîne de longueur 4 emprunte 4 arêtes. Pour la connexité : peut-on aller de n'importe quel sommet à n'importe quel autre ? Un cycle est une chaîne qui revient à son point de départ sans réutiliser d'arête. ↑

Ex. 2 – piste Si Alice travaille avec Bilal, Bilal travaille-t-il forcément avec Alice ? Pour la question 3, quel mot du vocabulaire désigne le nombre de voisins d'un sommet ? ↑énoncé

Ex. 3 – piste Chaque relevé « $X \to Y$ » donne une flèche de X vers Y. Les successeurs de `A` sont les sommets vers lesquels partent ses flèches ; les prédécesseurs de `C`, ceux d'où partent les flèches qui arrivent sur `C`. Un chemin doit suivre le sens des flèches. ↑énoncé

Ex. 4 – piste Matrice : une ligne et une colonne par sommet, dans l'ordre alphabétique ; la case (ligne X, colonne Y) vaut 1 si X et Y sont reliés. Liste d'adjacence : un dictionnaire qui associe à chaque sommet la liste de ses voisins. ↑énoncé

Ex. 5 – piste Matrice : un 1 en ligne i, colonne j indique une arête entre le i-ième et le j-ième sommet ; la matrice étant symétrique, il suffit de lire les cases situées au-dessus de la diagonale. Liste de listes : `adj[i]` contient les voisins du sommet i. ↑énoncé

Ex. 6 – piste Question 1 : pour chaque ligne `i` de `M`, garder les indices `j` tels que `M[i][j]` vaut 1. Question 2 : partir d'une matrice $n \times n$ remplie de 0, puis placer les 1. ↑énoncé

Ex. 6 – début de solution
```
def matrice_vers_listes(M):
    adj = []
    for i in range(len(M)):
        voisins_i = [] ...
```
↑énoncé

Ex. 7 q. 2 – piste Lister tous les chemins de `A` à `D` qui ne repassent jamais par un même sommet, puis additionner les poids de chacun. ↑énoncé

Ex. 8 q. 3 – piste Que renvoie `g.voisins(s)`, et comment en déduire le degré ? Pour `nb_aretes`, additionner les degrés de tous les sommets (`g.sommets`) : attention, chaque arête est alors comptée deux fois. ↑énoncé

Ex. 9 – piste BFS : écrire la file à chaque étape ; on défile en tête, on enfile en queue les voisins **non encore découverts**, par ordre alphabétique. DFS : aller le plus loin possible avant de revenir en arrière. ↑énoncé

Ex. 10 q. 1 – piste Q1 : relire les deux codes du cours. Le BFS tient trois choses à jour : la file `en_attente`, la liste `decouverts` (testée **avant** d'enfiler un voisin) et la liste `traites`. Le DFS récursif transmet sa liste `traites` à chaque appel ; que vaut ce paramètre au premier appel ? ↑énoncé

Ex. 10 q. 3 – piste Q3 : reprendre le code du BFS en remplaçant la file par une pile, avec la règle du cours : on ne marque un sommet comme traité qu'au moment où on le **dépile**, et l'on ignore ceux qui sont déjà traités. Dans quel ordre les voisins d'un sommet sortent-ils de la pile ? ↑énoncé

Ex. 11 – piste Les fonctions du cours ne trient rien : elles parcourent `g.voisins(s)` dans l'ordre de la liste. Pour le DFS itératif, les voisins sont empilés dans l'ordre de la liste : le dernier de la liste ressort donc le premier. Q5 : à quelle distance de `A` se trouve chaque sommet ? ↑énoncé

Ex. 12 – piste En largeur, les sommets sortent de la file par distance croissante au départ, couche par couche ; et, dans une couche, l'ordre suit celui des parents dans la couche précédente. En profondeur, après un sommet, on visite forcément l'un de **ses** voisins non encore visités, s'il en reste : on ne revient en arrière qu'à l'impasse. ↑énoncé

Ex. 13 q. 1 – piste Q1 : quels sommets un parcours lancé depuis `u` visite-t-il ? ↑énoncé

Ex. 13 q. 2 – piste Q2 : un parcours depuis n'importe quel sommet doit visiter tous les sommets. ↑énoncé

Ex. 14 q. 1 – piste Q1 : pendant le BFS, mémoriser le `parent` de chaque sommet découvert (le sommet depuis lequel on l'a découvert), puis remonter de l'arrivée jusqu'au départ. ↑énoncé

Ex. 14 q. 1 – début de solution Q1 :

```
parent = {depart: None}
en_attente = File()
en_attente.enfiler(depart)
while not en_attente.est_vide():
```

`s = en_attente.defiler()` … À la fin, remonter avec `s = parent[s]` et renvoyer la liste obtenue, remise dans l'ordre du départ vers l'arrivée. ↑énoncé

Ex. 14 q. 2 – piste Q2 : un sommet découvert depuis `s` est à la distance de `s` plus 1. ↑énoncé

Ex. 15 q. 4 – piste Q4 : la ligne `pos_s` de la matrice ne contient que des 0 et des 1 : que représente la somme de ses éléments ? ↑énoncé

Ex. 15 q. 10 – piste Q10 : marquer d'abord `s` comme visité, puis ne relancer le parcours que sur les voisins **pas encore** visités. ↑énoncé

Ex. 16 q. 3 – piste Q3 : remonter les arcs depuis `(l)`, puis depuis chacun de ses prédécesseurs, et ainsi de suite. ↑énoncé

Ex. 16 q. 5 – piste Q5 : au moment où l'on arrive de `(d)`, l'exploration de `(f)` est-elle encore en cours, ou déjà terminée ? Peut-on revenir de `(f)` vers `(d)` en suivant les flèches ? ↑

Ex. 16 q. 7 – piste Q7 : chercher un cycle dans le graphe dessiné à la question 6 : que se passe-t-il si des tâches dépendent les unes des autres en boucle ? ↑énoncé

Ex. 17 q. 2 – piste Q2 : `a2.voisines[2]` est un couple : que contient sa composante d'indice 1 ? ↑énoncé

Ex. 17 q. 7 – piste Q7 : parcourir les couples de `x.voisines` : seule leur première composante sert ici. ↑énoncé

Ex. 17 q. 8 – piste Q8 : réutiliser `sont_voisines` ; jusqu'à quel indice la boucle doit-elle aller pour que « la suivante » existe toujours ? ↑énoncé

Ex. 18 q. 3 – piste Q3 : tirer au hasard un **indice** valide de la liste `voisins[s]`. ↑énoncé

Ex. 18 q. 4 – piste Q4 : c'est un parcours en largeur « par vagues » : chaque vague de nouveaux contaminés correspond à une étape. ↑énoncé

Ex. 18 q. 4 – début de solution Q4 :

```
contamines = [s]
```

`frontiere = [s]`
`etapes = 0`
`while len(contamines) < len(voisins):` … Construire la nouvelle frontière (la vague suivante) à partir des voisins de la frontière courante. ↑énoncé

Ex. 19 q. 1 – piste Q1 : `Nc` n'est relié qu'à `Mr`, `To`, `Ax` et `Di` : chercher d'abord la meilleure distance de `Mp` à chacune de ces villes. ↑énoncé

Ex. 19 q. 6 – piste Q6 : remplacer la liste `visite` par un dictionnaire : un sommet découvert depuis `s` est à la distance de `s` plus 1. ↑énoncé

Ex. 19 q. 6 – début de solution Q6 : `dist = {sommet: 0}` remplace `visite = [sommet]` ; dans la boucle :

```
if not (v in dist):
    dist[v] = dist[s] + 1 ...
```

↑énoncé

Ex. 19 q. 7 – piste Q7 : même structure que `parcours`, en remplaçant la file par une pile ; un sommet n'est marqué visité qu'au moment où on le dépile. ↑énoncé

Ex. 20 q. 2 – piste Sans argument, `pop()` retire-t-il le premier ou le dernier élément de la liste ? Quelle structure obtient-on alors ? ↑énoncé

Ex. 21 – piste Structurer chaque explication en trois temps : une **définition** en une phrase, un **exemple** petit et concret déroulé à voix haute, puis le **piège** (l'erreur que l'auditeur risque de faire). Finir par une phrase qui répond à la question posée. Prévoir un graphe de quatre sommets que l'on sait décrire à voix haute (« A est relié à B et à C… ») et l'utiliser comme exemple. Sujet 1 : dans quel ordre les sommets sont-ils découverts ? ↑énoncé

Ex. 22 – piste En largeur : on défile le sommet de tête, on enfile ses voisins **non encore découverts**, par ordre alphabétique, puis on note la file. En profondeur : on visite un sommet, puis on relance le parcours sur son premier voisin non encore visité ; on ne revient en arrière qu'à l'impasse. Dans les deux cas, un sommet déjà découvert n'est jamais repris. ↑énoncé

Ex. 23 – piste Q3 : les quatre cases candidates sont `(i - 1, j)`, `(i + 1, j)`, `(i, j - 1)` et `(i, j + 1)` ; on garde celles dont les indices restent dans la grille (`0 <= a < len(lab)` et `0 <= b < len(lab[0])`) **et** qui ne sont pas un mur. Tester les indices **avant** de lire `lab[a][b]`. Q4 : pour ne pas ajouter deux fois l'arête entre deux cases voisines, tester `sont_voisins` avant `ajoute_arete`, ou ne regarder que la voisine de droite et celle du bas de chaque case. ↑énoncé

TP ET PROJETS

PROJET L'algorithme de Dijkstra

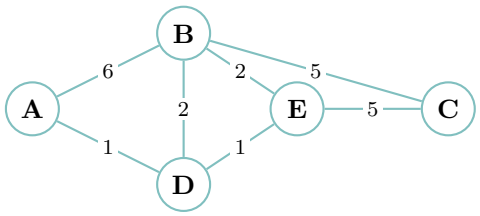
Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t08-projet-dijkstra` — **archive .zip**

But du projet

Projet [au-delà du programme] : l'algorithme de Dijkstra n'est pas exigible au bac. Il prolonge le parcours en largeur du cours et éclaire le routage (protocole OSPF) du chapitre *Réseaux*. Comprendre, dérouler *à la main*, puis **programmer** l'algorithme de Dijkstra, qui calcule le plus court chemin dans un graphe **pondéré** (à poids positifs) — le cœur de tout GPS. ▶ signale ce qui est à programmer et tester. Un fichier `dijkstra_graphes.py` est **à télécharger** (lien ci-dessus) : les graphes y sont déjà saisis, avec la **trame** des fonctions à compléter (sections « Programmer l'algorithme » et « Dijkstra avec une matrice d'adjacence ») et des **tests** ; une fois vos fonctions écrites, il affiche les distances et les chemins pour vérifier vos calculs sans tout retaper.

1 Le problème : et si les arêtes ont un coût ?

Dans le cours, le parcours en largeur (BFS) trouve le plus court chemin en **nombre d'arêtes**. Mais sur une vraie carte, une route peut être longue et une autre courte : les arêtes ont un **poids** (distance, temps, péage…). Le BFS ne sait pas en tenir compte : le trajet avec le moins de villes traversées n'est pas forcément le plus rapide !



Le graphe pondéré `G` *du projet (poids = distances).*

Aller de `A` *à* `C` *:* `A -- B -- C` traverse peu de villes mais coûte $6 + 5 = 11$. On verra que le vrai plus court chemin coûte… bien moins.

Remarque
En 1956, **Edsger Dijkstra** a trouvé la solution en vingt minutes, à la terrasse d'un café d'Amsterdam, *sans papier ni crayon*. Voici son idée.

2 L'idée de Dijkstra

On part du sommet `depart` et on fait grandir, petit à petit, un « nuage » de sommets dont on connaît **avec certitude** la distance minimale au départ.

Le principe, en trois idées

1. On garde pour chaque sommet une **distance provisoire** (au début : 0 pour le départ, $+\infty$ pour les autres).
2. À chaque tour, on **fixe définitivement** le sommet non encore traité qui a la **plus petite** distance provisoire (le plus proche : on est *sûr* qu'on ne fera pas mieux).
3. Depuis ce sommet `u` tout juste fixé, on **met à jour** ses voisins : si passer par `u` raccourcit le chemin vers un voisin `v`, on améliore la distance de `v`. Cette étape s'appelle un **relâchement** :

```
si distance[u] + poids(u,v) < distance[v] :
    distance[v] = distance[u] + poids(u,v)
```

On recommence jusqu'à avoir fixé tous les sommets.

> **Remarque**
> Pour **retrouver le chemin** (et pas seulement sa longueur), on note au passage, pour chaque sommet, le **prédécesseur** par lequel on l'a amélioré. On remontera ensuite ces prédécesseurs de l'arrivée jusqu'au départ (exactement comme pour le plus court chemin en largeur).

3 Dérouler à la main

Exercice 1 *Dérouler à la main*

On applique Dijkstra au graphe `G` ci-dessus, au départ de `A`. Recopier sur le cahier et compléter le tableau : à chaque ligne, on **fixe** le sommet non traité de plus petite distance (colonne « fixé », on le marque $*$), puis on met à jour ses voisins. (∞ = pas encore atteint.)

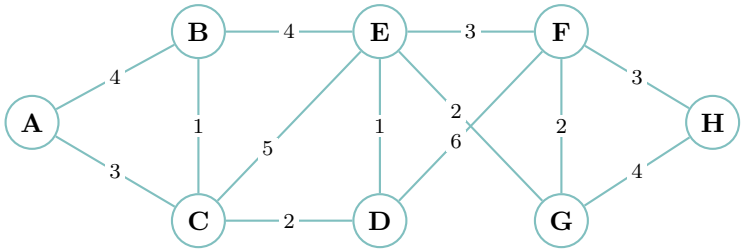
Étape	Sommet fixé	A	B	C	D	E
init	—	0	∞	∞	∞	∞
1	A (0)	0*	…	…	…	…
… (une ligne par étape, jusqu'à l'étape 5)						

1. Compléter le tableau jusqu'à ce que tous les sommets soient fixés.
2. En déduire la **distance minimale** de `A` à chaque sommet.
3. Donner le **plus court chemin** de `A` à `C`, et sa longueur. Comparer avec le trajet `A -- B -- C`.

4 Un graphe plus lourd

Exercice 2 *Un graphe plus lourd*

Voici un graphe pondéré à **huit** sommets. On va dérouler Dijkstra au départ de `A`, puis **vérifier avec l'ordinateur**.



1. Dérouler Dijkstra au départ de `A` : recopier sur le cahier le tableau ci-dessous et le compléter, une ligne par étape jusqu'à ce que les huit sommets soient fixés (marquer $*$ le sommet fixé à chaque étape ; ∞ = non atteint).

Ét.	Fixé	A	B	C	D	E	F	G	H
init	—	0	∞	∞	∞	∞	∞	∞	∞
1	A (0)	0*	…	…	…	…	…	…	…
… (une ligne par étape, jusqu'à l'étape 8)									

2. En déduire la distance minimale de `A` à chaque sommet, et un plus court chemin de `A` à `H`. Est-il unique ?
3. ▶ **Vérification par l'ordinateur** (*à faire après avoir complété* `dijkstra` *et* `chemin`*, section « Programmer l'algorithme »*). Dans le fichier `dijkstra_graphes.py` (fourni), le gros graphe est **déjà saisi** (variable `G_gros`). Quand vous aurez complété `dijkstra` et `chemin` dans ce fichier, exécutez-le et comparez l'affichage à votre tableau. Tout doit coïncider !

> **Remarque**
> Le graphe est saisi dans le fichier à partir de la **liste des arêtes** (`aretes`), et la fonction `construire_graphe` le transforme en dictionnaire : on évite ainsi de recopier à la main un long dictionnaire (et les erreurs qui vont avec).

5 Programmer l'algorithme

On représente le graphe pondéré par un **dictionnaire** : à chaque sommet, la liste de ses couples `(voisin, poids)`. Dans le fichier fourni, le graphe `G` du projet s'appelle `G_petit` :

```
G_petit = {'A': [('B', 6), ('D', 1)],
           'B': [('A', 6), ('D', 2), ('E', 2), ('C', 5)],
           'C': [('B', 5), ('E', 5)],
           'D': [('A', 1), ('B', 2), ('E', 1)],
           'E': [('D', 1), ('B', 2), ('C', 5)]}
```

Exercice 3 *La fonction* `dijkstra`

▶ Compléter la fonction `dijkstra` (elle est déjà recopiée, avec ses trous, dans `dijkstra_graphes.py`). Elle renvoie deux dictionnaires : `distance` (distance minimale du départ à chaque sommet) et `precedent` (le prédécesseur de chaque sommet sur un plus court chemin).

Outil Python : l'infini `float('inf')`

Au départ, aucun sommet (sauf le départ) n'a encore été atteint : sa distance est « infinie ». Python dispose d'un nombre spécial, `float('inf')`, plus grand que tous les nombres : `float('inf') > 10**100` vaut `True`, et `float('inf') + 5` vaut encore `float('inf')`. Ainsi, le premier chemin trouvé vers un sommet est toujours meilleur que « pas de chemin », sans cas particulier à écrire. On l'utilisera de nouveau au chapitre *Programmation dynamique*.

```python
def dijkstra(graphe, depart):
    distance = {}
    precedent = {}
    for s in graphe:
        distance[s] = float('inf')              # +infini : pas encore atteint
        precedent[s] = None
    distance[depart] = ...                      # (a)
    a_traiter = [s for s in graphe]             # sommets pas encore fixes
    while a_traiter != []:
        # 1) choisir le sommet non traite de plus petite distance
        u = a_traiter[0]
        for s in a_traiter:
            if distance[s] < distance[u]:
                u = ...                         # (b)
        a_traiter = [s for s in a_traiter if s != u]   # on retire u
        # 2) relacher les aretes issues de u
        for (v, poids) in graphe[u]:
            if distance[u] + poids < ... :      # (c)
                distance[v] = ...               # (d)
                precedent[v] = ...              # (e)
    return distance, precedent
```

1. Compléter les cinq trous `(a)` à `(e)`.
2. Tester : `dijkstra(G_petit, 'A')` doit redonner les distances trouvées à la main, et le test `dijkstra` du fichier doit afficher `OK`.

Exercice 4 *Reconstruire le chemin*

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

▶ Pour obtenir le **chemin** lui-même, écrire (dans le fichier, à la place de `# A COMPLETER`) la fonction `chemin(precedent, depart, arrivee)` qui reconstruit la liste des sommets, du départ à l'arrivée, en remontant les prédécesseurs (renvoyer `None` si l'arrivée n'a pas été atteinte). Vérifier que le chemin de `A` à `C` est bien celui trouvé à la main.

6 Dijkstra avec une matrice d'adjacence

On peut tout aussi bien coder Dijkstra à partir d'une **matrice d'adjacence pondérée** `M` : `M[u][v]` contient le poids de l'arête `u`–`v`, et `0` s'il n'y a pas d'arête. Les sommets sont alors des **indices** $0, 1, 2, \ldots$ L'algorithme est **le même** ; seule change la façon de lire les voisins : au lieu de parcourir une liste, on parcourt la **ligne `u`** de la matrice.

Exercice 5 *Version avec une matrice d'adjacence*

▶ Compléter la fonction ci-dessous (six trous `(a)` à `(f)`), elle aussi déjà recopiée dans `dijkstra_graphes.py`. Ici `distance`, `precedent` et `traite` sont des **listes** indexées par les sommets.

```
def dijkstra_matrice(M, depart):
    n = len(M)
    distance = [float('inf')] * n     # +infini partout
    distance[depart] = ...            # (a)
    precedent = [None] * n
    traite = [False] * n
    for etape in range(n):
        # 1) sommet non traite de plus petite distance
        u = -1
        for s in range(n):
            if not traite[s] and (u == -1 or distance[s] < distance[u]):
                u = ...               # (b)
        traite[u] = True
        # 2) relacher : on parcourt la ligne u de la matrice
        for v in range(n):
            if M[...][...] != 0 and not traite[v]:   # (c) : l'arete u--v existe-t-elle ?
                if distance[u] + M[u][v] < ... :      # (d)
                    distance[v] = ...                 # (e)
                    precedent[v] = ...                # (f)
    return distance, precedent
```

1. Compléter les six trous `(a)` à `(f)`.
2. Quelle est la seule vraie différence avec la version par listes ? Montrer la ligne du code où elle apparaît.
3. ▶ Tester avec le fichier `dijkstra_graphes.py` fourni (la matrice du gros graphe y est déjà construite) et vérifier qu'on obtient **exactement** les mêmes distances que la version par listes.

> **Remarque**
> Les deux versions donnent le même résultat : ce sont deux **implémentations** du même algorithme. Le choix dépend de la représentation du graphe — matrice (pratique si le graphe est « dense ») ou listes (économe si le graphe est « creux »), comme dans le cours.

7 Pour aller plus loin [au-delà du programme]

Pourquoi Dijkstra donne-t-il la bonne réponse ?

L'algorithme repose sur une affirmation forte : *quand on fixe le sommet* `u` *de plus petite distance provisoire, cette distance est déjà la distance minimale réelle.* Voici pourquoi (on suppose **tous les poids positifs**).

Démonstration :
Supposons le contraire : soit `u` le **premier** sommet fixé dont la distance provisoire $d[u]$ dépasserait la vraie distance minimale $\delta(u)$. Considérons un *vrai* plus court chemin du départ à `u`, et notons `y` le premier sommet de ce chemin **pas encore fixé**, et `x` son prédécesseur (déjà fixé, donc de distance correcte). Quand on a fixé `x`, on a relâché l'arête `x`→`y`, donc

$$d[y] \leq d[x] + \text{poids}(x,y) = \delta(x) + \text{poids}(x,y) = \delta(y) \leq \delta(u).$$

(La dernière inégalité vient de ce que `y` est *sur* un plus court chemin vers `u` et que les poids sont positifs : aller jusqu'à `y` coûte au plus ce que coûte aller jusqu'à `u`.) Mais on a choisi `u` comme

sommet de plus petite distance provisoire, donc $d[u] \leq d[y] \leq \delta(u)$. On aurait donc $d[u] \leq \delta(u)$, ce qui **contredit** $d[u] > \delta(u)$. L'hypothèse est absurde : la distance de `u` est bien correcte au moment où on le fixe. □

> **Remarque**
> La positivité des poids est **essentielle** : avec des poids négatifs, un détour pourrait « rapporter » et l'argument s'effondre. Dijkstra ne fonctionne donc que pour des poids ≥ 0.

Efficacité

Telle qu'écrite, à chaque tour on cherche le minimum en parcourant tous les sommets restants : le coût est en $O(n^2)$. En rangeant les sommets dans une **file de priorité** (rangée dans une structure appelée *tas*), on descend à $O((n+m)\log n)$ — indispensable sur les graphes routiers à des millions de sommets.

> **Remarque**
> Si **toutes** les arêtes ont le même poids, Dijkstra fixe les sommets dans l'ordre du **parcours en largeur** (à l'ordre près entre sommets situés à la même distance) : le BFS est le cas particulier de Dijkstra où tous les poids valent 1.

FICHE DE RÉVISION

L'essentiel

- Un **graphe** = des **sommets** reliés par des **arêtes**. **Non orienté** : arêtes à double sens (matrice **symétrique**) ; **orienté** : **arcs** ; **pondéré** : un **poids** par arête.
- **Voisins** d'un sommet = ses adjacents ; **degré** = nombre de voisins. **Chaîne** / **chemin** = suite de sommets reliés ; **cycle** : revient au départ. **Connexe** : d'un seul tenant.
- **Trois représentations**, et savoir **passer de l'une à l'autre** : **matrice d'adjacence** ($n \times n$, `M[i][j] = 1` si voisins ; mémoire $O(n^2)$, tester une arête en $O(1)$) ; **liste d'adjacence** (dictionnaire sommet $\to$ liste des voisins ; mémoire $O(n+m)$, idéale si peu d'arêtes) ; **liste de listes** (sommets 0 à $n-1$).
- Un **parcours** doit **mémoriser les sommets déjà découverts** (sinon il tourne en rond sur un cycle). **File** (FIFO) $\Rightarrow$ **largeur** (BFS) ; **pile** (LIFO) ou récursivité $\Rightarrow$ **profondeur** (DFS). On utilise les interfaces `File` / `Pile` du chapitre *Structures linéaires* (une `list` avec `pop(0)` marche, mais chaque `pop(0)` coûte $O(n)$).
- Le **BFS** donne un **plus court chemin** (en nombre d'arêtes) : mémoriser le **parent** de chaque sommet découvert, puis **remonter** depuis l'arrivée.
- **Chemin** de `u` à `v` : il en existe un si et seulement si un parcours depuis `u` visite `v`. Graphe **connexe** : un parcours visite **tous** les sommets.
- **Cycle** : en **non orienté**, on retombe sur un sommet déjà découvert qui n'est pas le parent ; en **orienté**, il faut retomber, lors d'un parcours en profondeur, sur un sommet **en cours** (encore dans la pile d'appels) — un sommet **terminé** ne signale pas de cycle.

Le code à savoir écrire

```
class Graphe:                       # non oriente
    def __init__(self, sommets):
        self.sommets = sommets
        self.adj = {}                # sommet -> liste de ses voisins
        for s in sommets:
            self.adj[s] = []

    def ajoute_arete(self, a, b):
        self.adj[a].append(b)
        self.adj[b].append(a)        # non oriente : dans les deux sens

    def voisins(self, s):
        return self.adj[s]

def BFS(g, depart):                 # largeur : une FILE
    traites = []
    decouverts = [depart]
    en_attente = File()             # la file (interface admise)
    en_attente.enfiler(depart)
    while not en_attente.est_vide():
        sommet = en_attente.defiler()       # on defile (FIFO)
        for voisin in g.voisins(sommet):
            if voisin not in decouverts:    # tester AVANT d'enfiler
                decouverts.append(voisin)
                en_attente.enfiler(voisin)  # on enfile
        traites.append(sommet)
    return traites

def DFS(g, sommet, traites=None):   # profondeur : RECURSIF
```

```
    if traites is None:
        traites = []
    traites.append(sommet)
    for voisin in g.voisins(sommet):
        if voisin not in traites:
            DFS(g, voisin, traites)
    return traites
```

Erreurs à éviter

- oublier de **mémoriser les sommets découverts** → boucle infinie sur un cycle ;
- oublier la **double insertion** dans `ajoute_arete` d'un graphe non orienté ;
- en largeur, tester `traites` au lieu de `decouverts` → un sommet enfilé deux fois ;
- en orienté, conclure à un cycle dès qu'on retombe sur un sommet déjà découvert (il faut un sommet **en cours**) ;
- défiler avec `pop(0)` sur une `list` sans voir que chaque appel coûte $O(n)$.

Savoir-faire à maîtriser — je sais…

- ☐ employer le **vocabulaire** (sommet, arête/arc, voisin, degré, chaîne/chemin, cycle, connexe) ;
- ☐ **modéliser** une situation concrète par un graphe orienté ou non, pondéré ou non (une grille ou un labyrinthe : cases libres = sommets, passages = arêtes) ;
- ☐ écrire une **matrice d'adjacence**, une **liste d'adjacence** (de successeurs, de prédécesseurs), et **passer** de l'une à l'autre ;
- ☐ écrire / compléter une **classe `Graphe`** ;
- ☐ dérouler et coder un **parcours en largeur** (file) et en **profondeur** (pile ou récursif) ;
- ☐ trouver un **chemin**, un **plus court chemin** (BFS), tester la **connexité** ;
- ☐ repérer un **cycle**, en distinguant graphe non orienté (sommet déjà découvert, autre que le parent) et orienté (sommet **en cours**).

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 924

1. Pourquoi la matrice d'adjacence d'un graphe **non orienté** est-elle symétrique ?
2. Arêtes `A-B`, `A-C`, `B-D`, `C-D` ajoutées dans cet ordre : que renvoie `DFS(g, 'A')` ?
3. Mémoire d'une matrice d'adjacence ? d'une liste d'adjacence (n sommets, m arêtes) ?
4. Que fait un parcours qui ne mémorise pas les sommets visités, sur un graphe à cycle ?
5. Pour un plus court chemin en nombre d'arêtes : parcours en largeur ou en profondeur ?

Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 924

1. Quel est le coût du tri fusion d'un tableau de n valeurs ? celui d'une recherche dichotomique ? (ch. 7)
2. Donner le parcours préfixe de `Noeud(1, Noeud(2, Noeud(4)), Noeud(3))`. (ch. 5)
3. Dans la méthode `voisins(self, s)` de la classe `Graphe`, que désigne `self` ? (ch. 2)

Vers le bac

- Problème de synthèse n° 6 — Une médiathèque, de la table à l'objet, p. 709
- Problème de synthèse n° 7 — Ordonnancer des tâches qui dépendent les unes des autres, p. 710
- Épreuve pratique : sujet 26 — Sortir d'un labyrinthe, p. 731

DÉFIS ADVENT OF CODE

DÉFI 1 Jurassic Jigsaw

le puzzle d'images — grilles, rotations et symétries

Jour 20

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t08-defi-aoc-2020-20` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/20`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Une image a été découpée en une centaine de petits carrés, mélangés, tournés et retournés au hasard. Pour la reconstituer, il faut trouver quelles tuiles se touchent, en comparant leurs bords dans toutes les orientations possibles : des dizaines de milliers de comparaisons, impossibles à faire à la main.
Ce qu'on réinvestit. Les **graphes** de ce chapitre : les tuiles en sont les sommets, et deux tuiles qui partagent un bord sont reliées par une arête. Le degré d'un sommet dit alors où est la tuile : 2 voisines pour un coin, 3 pour un bord, 4 à l'intérieur. La partie 1 se résout ainsi sans reconstituer l'image ; l'approfondissement assemble l'image par un **parcours** de ce graphe.
La démarche de la fiche.

1. Comprendre comment deux tuiles se raccordent, et le vérifier à la main sur deux petites tuiles.

2. Lire les tuiles dans un dictionnaire, puis extraire les quatre bords de chacune.
3. Compter, pour chaque bord, les autres tuiles qui le possèdent : les tuiles à deux bords libres sont les coins (partie 1).
4. Pour la partie 2, orienter et assembler les tuiles, puis chercher un motif dans l'image obtenue.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`tile`	tuile, morceau d'image	`reassemble`	réassembler
`rotated / flipped`	tourné / retourné (en miroir)	`border / edge`	bord
`line up`	coïncider, s'aligner	`outermost`	le plus à l'extérieur
`corner`	coin	`ID number`	numéro d'identification
`sea monster`	monstre marin (motif à chercher)	`roughness`	rugosité (partie 2)

Résumé de l'énoncé (partie 1)

Contexte. Les messages du satellite contiennent en fait une photo, mais l'appareil l'a envoyée découpée en petits carrés (des « tuiles »), mélangés et, pire, tournés ou retournés au hasard. Il faut reconstituer ce puzzle. Pour commencer, il suffit de trouver les quatre tuiles des coins.
Les données. Le fichier est une suite de blocs séparés par une ligne vide. Chaque bloc est une tuile : une ligne `Tile <numéro>:` qui donne son numéro, suivie d'une grille carrée de 10×10 caractères, `.` (blanc) ou `#` (noir). Le nombre de tuiles est un carré parfait : l'image complète est un carré de tuiles.
La règle.

- Chaque tuile a pu être tournée d'un nombre quelconque de quarts de tour et/ou retournée en miroir.
- Deux tuiles voisines dans l'image ont, une fois bien orientées, des bords identiques : c'est ce qui permet de les raccorder.
- Les bords extérieurs de l'image ne correspondent à aucune autre tuile.
- En pratique, deux tuiles non voisines n'ont jamais de bord commun : un bord partagé désigne donc sans ambiguïté deux voisines.

La réponse attendue. Le produit des numéros des quatre tuiles placées aux coins de l'image. Par exemple, si les coins portaient les numéros 2, 3, 5 et 7, la réponse serait $2 \times 3 \times 5 \times 7 = 210$.

Exercice 1 *Vérifier sa compréhension*

But. Comprendre comment se comparent deux bords et ce que révèle le nombre de bords libres d'une tuile.
À quoi ça sert ? La deuxième question donne la clé de la partie 1 : on reconnaît un coin à son nombre de bords sans correspondant, ce qui évite d'assembler l'image.

1. Deux bords qui se correspondent peuvent apparaître « à l'envers » l'un par rapport à l'autre : pourquoi ?
2. Combien de bords sans correspondant a une tuile de coin ? une tuile du bord de l'image ? une tuile intérieure ?
3. Faut-il reconstituer toute l'image pour la partie 1 ?

Exercice 2 *À la main, sur un petit exemple*

But. Écrire les bords de deux petites tuiles et trouver celui qu'elles partagent.
À quoi ça sert ? On fait à la main exactement ce que fera le programme : extraire quatre bords, et les comparer à l'endroit et à l'envers. Le résultat connu (bord droit de A, bord gauche de B) servira de test.

Voici deux tuiles inventées de taille 5×5 :

```
Tuile A
#.#..
#.#..
#...#
.#.##
#....
```

```
Tuile B
.#...
##.##
###.#
....#
.#.##
```

1. Écrire les quatre bords de chaque tuile sous forme de chaînes, en lisant toujours de gauche à droite (haut, bas) ou de haut en bas (gauche, droite).
2. Montrer qu'un seul bord de A coïncide avec un bord de B, à condition de lire l'un des deux à l'envers. Vérification : il s'agit du bord droit de A et du bord gauche de B.
3. Pour placer B à droite de A, B doit-il être tourné, retourné, ou laissé tel quel ?

2 Programmer la partie 1

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Exercice 3 *Lire les tuiles*

But. Ranger chaque tuile, sous forme de liste de chaînes, dans un dictionnaire indexé par son numéro.
À quoi ça sert ? Toute la suite manipule les tuiles par leur numéro (la réponse est un produit de numéros) ; vérifier que leur nombre est un carré parfait permet aussi de contrôler la lecture.

Construire un dictionnaire `tuiles` qui associe à chaque numéro (un entier) la tuile sous forme de liste de chaînes. Vérifier le nombre de tuiles : est-ce un carré parfait ? ▷ coup de pouce

Outils Python : lire une chaîne à l'envers, construire une colonne, comparer deux chaînes

`s[::-1]` est la tranche qui parcourt `s` avec un pas de -1 : c'est la chaîne à l'envers. Pour obtenir une **colonne** d'une tuile, on part d'une chaîne vide et on lui ajoute, ligne après ligne, le caractère voulu. Enfin `min(a, b)` et `<` comparent deux chaînes dans l'ordre « alphabétique » des caractères (l'ordre de leurs codes : `#` vient avant `.`).

```
tuile = ["#.#", "..#", "##."]
print(tuile[0][::-1])              # "#.#" lu a l'envers : "#.#"
print("..#"[::-1])                 # "#.."
gauche = ""
for ligne in tuile:
    gauche += ligne[0]             # premier caractere de chaque ligne
```

```
print(gauche)                    # "#.#"
print(min("..#", "#.."))         # "#.." car "#" < "."
```

Intérêt. Avec `min(b, b[::-1])`, un bord et son inverse donnent **la même** chaîne : on peut alors les ranger sous une seule clé de dictionnaire. Question : écrire le bord droit de cette tuile avec une boucle. Que vaut `min("##.", ".##")` ?

Exercice 4 *Les bords*

But. Extraire les quatre bords d'une tuile, puis compter pour chacun les autres tuiles qui le possèdent.
À quoi ça sert ? Ce comptage construit en fait les arêtes du graphe des tuiles : un bord possédé par une autre tuile est une liaison avec une voisine, un bord possédé par aucune autre est un bord de l'image.

Écrire une fonction `bords(tuile)` qui renvoie la liste `[haut, bas, gauche, droite]` des quatre bords, lus comme dans l'exercice « À la main, sur un petit exemple ». Puis, pour chaque tuile, compter combien de ses bords ne se retrouvent, à l'endroit ou à l'envers, chez aucune *autre* tuile.

▷ coup de pouce

Exercice 5 *Les coins*

But. Repérer les tuiles qui ont exactement deux bords libres et multiplier leurs numéros.
À quoi ça sert ? C'est la réponse de la partie 1. Ces quatre coins seront aussi le point de départ de l'assemblage de la partie 2.

Une tuile de coin a exactement deux bords qui ne correspondent à aucune autre tuile. Repérer les quatre coins et calculer la réponse.

▷ coup de pouce

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site. Cette partie est **longue** : la découper en étapes et tester chacune.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Il faut maintenant reconstituer vraiment l'image, puis y chercher des monstres marins, dont le dessin (un motif de `#`) est donné sur le site.
La nouvelle règle.

- Assembler l'image en orientant chaque tuile pour que ses bords coïncident avec ceux de ses voisines.
- Retirer le bord (première et dernière lignes et colonnes) de chaque tuile, puis recoller les tuiles : chacune devient un carré 8×8.
- Chercher le motif dans l'une des 8 orientations de l'image : chaque `#` du motif doit tomber sur un `#` de l'image, les autres cases du motif sont quelconques.
- Une seule orientation de l'image contient des monstres.

La réponse attendue. Le nombre de `#` de l'image qui n'appartiennent à aucun monstre. Par exemple, si l'image contenait 100 `#` dont 30 couverts par des monstres, la réponse serait 70.

★★ Exercice 6 *Assembler l'image*

But. Écrire les fonctions qui tournent et retournent une tuile, puis placer toutes les tuiles dans la bonne orientation.
À quoi ça sert ? Sans image assemblée, impossible de chercher le monstre. C'est l'étape la plus longue : les huit orientations, obtenues avec seulement deux fonctions, servent aussi bien pour les tuiles que pour l'image entière.

Écrire `rotation(tuile)` (quart de tour) et `symetrie(tuile)` (miroir), puis `orientations(tuile)` qui renvoie les huit positions possibles (encadré « Les huit orientations d'une grille carrée », section « Pour y arriver »). Assembler ensuite toute l'image en suivant l'encadré « Stratégie d'assemblage pas à pas ». ▷ coup de pouce ▷ début de solution

Outil Python : les ensembles (`set`), et rappel sur le drapeau

Pour tester si **toutes** les cases d'une liste sont des `#`, on utilise un booléen « drapeau » (méthode du défi *Handy Haversacks*, chapitre 1) : il part de `True` et la boucle le passe à `False` au premier contre-exemple. Un **ensemble** est une collection sans doublon et sans ordre ; on y ajoute avec `add`, on réunit deux ensembles avec `a | b`, et le test `x in ensemble` est très rapide (encadré « les ensembles » du défi *Crab Combat*, chapitre 4).

```
image = ["#.#", "###"]
cases = [(0, 0), (1, 1)]
tout_diese = True
for i, j in cases:
    if image[i][j] != "#":
        tout_diese = False
print(tout_diese)                    # True
couvertes = set()                    # ensemble vide (attention : {} est un dict)
couvertes.add((0, 0))
couvertes = couvertes | {(0, 0), (1, 1)}   # reunion : (0, 0) n'est pas compte deux fois
print(len(couvertes))                # 2
print((1, 1) in couvertes)           # True
```

Intérêt. Si deux monstres se chevauchaient, une case commune ne serait comptée qu'une fois dans l'ensemble. Question : que vaudrait `tout_diese` avec la liste de cases `[(0, 0), (0, 1)]` ?

★★ Exercice 7 *Chercher le motif*

But. Recoller l'image sans les bords des tuiles, y repérer toutes les positions du monstre, et compter les `#` restants.
À quoi ça sert ? C'est la réponse de la partie 2. Ranger les cases couvertes dans un ensemble garantit qu'une case commune à deux monstres n'est retirée qu'une fois.

Retirer les bords de chaque tuile, recoller l'image, puis chercher le motif dans chacune de ses huit orientations. Calculer la réponse demandée. ▷ coup de pouce

4 Pour y arriver : orientations et assemblage

Les huit orientations d'une grille carrée

Un carré peut être tourné de 0, 1, 2 ou 3 quarts de tour : cela fait 4 positions. On peut aussi le retourner (miroir gauche-droite) avant de le tourner : 4 positions de plus. Toute autre transformation (miroir haut-bas, miroir selon une diagonale) redonne l'une de ces 8 positions : par exemple, le miroir haut-bas est un miroir gauche-droite suivi d'un demi-tour. Il suffit donc de deux fonctions, `rotation` et `symetrie`, pour toutes les obtenir.

Stratégie d'assemblage pas à pas

1. Prendre un coin trouvé en partie 1 et chercher l'orientation où ses deux bords libres sont en haut et à gauche : il sera la case $(0, 0)$.
2. Remplir la première ligne de gauche à droite : pour la case suivante, chercher parmi les tuiles non placées celle qui, dans l'une de ses 8 orientations, a un bord gauche égal au bord droit de la tuile précédente.
3. Pour les lignes suivantes, chaque tuile doit avoir son bord haut égal au bord bas de la tuile du dessus.
4. Grâce à la partie 1, on sait que chaque bord intérieur n'est partagé que par deux tuiles : à chaque étape, il y a donc une seule tuile possible.

5 Approfondissement

★★ **Exercice 8** *Approfondissement 1 : assembler par un parcours*

But. Assembler l'image en partant de n'importe quelle tuile et en propageant le placement de voisine en voisine.

À quoi ça sert ? C'est un parcours du graphe des tuiles, avec une pile comme dans le cours : la méthode n'a plus besoin de commencer par un coin, et l'on reconnaît un algorithme général derrière un problème de puzzle.

La stratégie du défi part d'un coin et remplit l'image ligne par ligne. On peut aussi partir de **n'importe quelle** tuile, laissée telle quelle à la position $(0, 0)$, et propager : pour chaque tuile placée, on cherche ses voisines par ses quatre bords, on les oriente et on les place à côté. Les positions peuvent devenir négatives ; on « recadre » à la fin.

1. À la main, avec les tuiles A et B de la fiche : A est placée telle quelle en $(0, 0)$. À quelle position va B, et dans quelle orientation ? Quelle serait la position de **A**, et son orientation, si l'on avait placé B en premier, telle quelle, en $(0, 0)$?

2. Écrire `assembler_parcours(tuiles)` qui utilise une **pile** des tuiles placées mais pas encore traitées, et un dictionnaire `place` qui associe à chaque position `(ligne, colonne)` la tuile orientée. Vérifier qu'on obtient la même réponse à la partie 2. ▷ coup de pouce ▷ début de solution

3. Cette méthode suppose qu'aucun bord n'est un palindrome (une chaîne égale à son inverse). Pourquoi ? La méthode du défi a-t-elle le même défaut ?

4. Comparer les deux méthodes : nombre de tuiles examinées, longueur du code. Quel parcours de graphe reconnaît-on ?

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Les tuiles sont séparées par une ligne vide : `split("\n\n")`. La première ligne d'un bloc donne le numéro : une tranche enlève le texte `Tile` et le `:` final. ↑ énoncé

Ex. 4 – piste Une boucle qui ajoute `ligne[0]` à une chaîne, vide au départ, donne le bord gauche ; `chaine[::-1]` lit une chaîne à l'envers. ↑ énoncé

Ex. 5 – piste Construire un dictionnaire qui associe à chaque bord, mis sous une forme *canonique* (le plus petit, dans l'ordre alphabétique, entre le bord et son inverse : `min(b, b[::-1])`), la liste des tuiles qui le possèdent. Un bord de tuile est « libre » si cette liste n'a qu'un élément. ↑ énoncé

Ex. 6 – piste Tester `rotation` sur la tuile A : quatre rotations successives doivent redonner A, et `orientations(A)` doit contenir 8 tuiles toutes différentes. ↑ énoncé

Ex. 6 – début de solution Pour une liste de chaînes `t` de taille n, la ligne `j` de la tuile tournée d'un quart de tour dans le sens horaire est formée des caractères `t[n - 1 - i][j]` pour `i` de `0` à `n - 1`, ajoutés un par un à une chaîne vide au départ. ↑

Ex. 7 – piste Stocker le motif comme une liste de décalages `(ligne, colonne)` des cases `#` ; le motif est présent à la position `(i, j)` si (avec un drapeau) toutes les cases `(i + dl, j + dc)` de l'image sont des `#`. Une seule orientation de l'image contient des motifs. ↑énoncé

Ex. 8 q. 2 – piste Pour le côté droit d'une tuile `t` placée en `(l, c)` : la voisine (si elle existe et n'est pas encore placée) va en `(l, c + 1)`, dans l'orientation dont le bord **gauche** est égal au bord droit de `t`. Idem pour les trois autres côtés. ↑énoncé

Ex. 8 q. 2 – début de solution Une liste `COTES` de quadruplets `(dl, dc, mon_bord, son_bord)`, par exemple `(0, 1, droite, gauche)`, évite d'écrire quatre fois le même code (les fonctions sont des valeurs comme les autres). Recadrage : soustraire à toutes les positions le plus petit numéro de ligne et le plus petit numéro de colonne, trouvés par une boucle sur les clés de `place`. ↑

DÉFI 2 Allergen Assessment

les allergènes — ensembles, intersections et élimination

Jour 21

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t08-defi-aoc-2020-21` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/21`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. On dispose d'une liste d'aliments : pour chacun, ses ingrédients (dans une langue inconnue) et quelques-uns des allergènes qu'il contient. On veut en déduire quels ingrédients sont sans danger, puis quel ingrédient contient chaque allergène. Le raisonnement est celui d'une enquête par recoupements, mais avec des dizaines de lignes et des centaines d'ingrédients : il faut un programme.
Ce qu'on réinvestit. Les **graphes** de ce chapitre : relier chaque allergène aux ingrédients qui peuvent le contenir donne un graphe, et l'élimination de la partie 2 retire des arêtes jusqu'à ce qu'il n'en reste qu'une par allergène. Pour trouver ces arêtes, on utilise les opérations sur les **ensembles** : l'intersection garde ce qui est commun à plusieurs lignes, la réunion rassemble les suspects.
La démarche de la fiche.

1. Comprendre ce qu'une ligne permet de déduire (et ce qu'elle ne permet pas), puis mener l'enquête à la main sur un petit fichier.
2. Lire chaque aliment comme deux ensembles : ses ingrédients et ses allergènes.
3. Calculer les candidats de chaque allergène, puis compter les ingrédients sans danger (partie 1).
4. Éliminer pas à pas pour attribuer chaque allergène à un ingrédient (partie 2).

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`ingredient`	ingrédient	`allergen`	allergène
`food`	aliment	`contains`	contient
`listed / marked`	indiqué, mentionné	`safe`	sans danger
`exactly one`	exactement un	`zero or one`	zéro ou un
`dangerous`	dangereux	`alphabetically`	par ordre alphabétique

Résumé de l'énoncé (partie 1)

Contexte. Pour rejoindre votre île, vous construisez un radeau et devez emporter des provisions. Problème : les étiquettes des aliments sont écrites dans une langue inconnue ; seuls certains allergènes sont indiqués dans une langue que vous comprenez. Avant de choisir vos provisions, vous voulez repérer les ingrédients qui sont certainement sans danger.
Les données. Chaque ligne du fichier décrit un aliment. Elle commence par sa liste d'ingrédients (des mots illisibles), séparés par des espaces, puis vient `(contains` suivi de quelques allergènes séparés par des virgules, et une parenthèse fermante. Par exemple, `kesa zuli brak (contains milk)` : trois ingrédients et un allergène indiqué.
La règle.

- Chaque allergène se trouve dans exactement un ingrédient ; un ingrédient contient zéro ou un allergène.
- Si un allergène est indiqué sur une ligne, l'ingrédient qui le contient figure forcément sur cette ligne.
- Mais un allergène peut être présent dans un aliment sans y être indiqué : l'absence de mention ne prouve rien.
- Un ingrédient est sans danger s'il ne peut contenir aucun des allergènes, compte tenu de toutes les lignes.

La réponse attendue. Le nombre total d'apparitions des ingrédients sans danger : un

ingrédient présent sur trois lignes compte trois fois. Par exemple, avec les deux lignes `rel mux tap (contains soy)` et `mux dov (contains soy)`, seul `mux` peut contenir `soy` ; `rel`, `tap` et `dov` sont sans danger et apparaissent une fois chacun : la réponse serait 3.

Exercice 1 *Vérifier sa compréhension*

But. Distinguer ce qu'une ligne prouve et ce qu'elle ne prouve pas.
À quoi ça sert ? Ces trois questions contiennent toute la logique du défi : la première conduit à l'intersection des lignes, les deux suivantes évitent d'éliminer un ingrédient à tort.

1. Un allergène est indiqué sur deux lignes : où peut se trouver l'ingrédient qui le contient ?
2. Un ingrédient absent d'une ligne qui indique `milk` peut-il contenir `milk` ?
3. Un ingrédient présent sur une ligne qui n'indique pas `milk` peut-il contenir `milk` ?

Exercice 2 *À la main, sur un petit exemple*

But. Mener l'enquête à la main sur cinq aliments.
À quoi ça sert ? On découvre la méthode (garder les ingrédients communs à toutes les lignes d'un allergène) avant de la programmer, et l'on obtient une réponse connue, `7`, pour tester le programme.

On considère le fichier (inventé) suivant :

```
brak zuli monto pirf (contains gluten, milk)
zuli kesa tolu (contains gluten)
monto tolu brak vemi (contains eggs)
tolu pirf zuli (contains gluten, eggs)
kesa zuli brak (contains milk)
```

1. Pour l'allergène `gluten`, quels ingrédients figurent sur *toutes* les lignes qui le mentionnent ? Faire de même pour `eggs` et `milk`.
2. En déduire les ingrédients qui ne peuvent contenir aucun allergène, et la réponse de la partie 1. Vérification : on doit trouver `7`.
3. L'ingrédient `monto` figure sur une ligne qui mentionne `milk`. Pourquoi est-il pourtant sans danger ?

2 Programmer la partie 1

Outil Python : les ensembles (`set`) — rappel

Tout est présenté dans l'encadré « Outil Python : les ensembles » du défi *Crab Combat* (chapitre 4) : `set()`, `add`, `remove`, test `x in ensemble` en temps constant, parcours avec `for`, éléments non modifiables (tuples), **réunion** `a | b` (éléments présents dans l'un ou dans l'autre) et **intersection** `a & b` (éléments présents dans les deux). Ces deux opérateurs renvoient un nouvel ensemble, sans modifier `a` ni `b`.
À essayer. Avec `a = set("brak zuli monto".split())` et `b = {"zuli", "kesa", "brak"}`, que valent `a & b`, `b & a` et `a | b` ? Écrire une boucle qui construit l'ensemble des éléments de `b` absents de `a`. Que vaut `len(a & set())` ?

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Exercice 3 *Lire le fichier*

But. Transformer chaque ligne en un couple de deux ensembles : ingrédients et allergènes.
À quoi ça sert ? Les ensembles rendent les recoupements immédiats (une intersection avec `&`) et toutes les questions suivantes partent de cette liste `aliments`.

Construire la liste `aliments` : chaque aliment est un couple `(ingredients, allergenes)` de deux ensembles de chaînes. ▷ coup de pouce

Exercice 4 *Les candidats*

But. Associer à chaque allergène l'ensemble des ingrédients qui peuvent le contenir.
À quoi ça sert ? Ce dictionnaire est le graphe allergènes–ingrédients du défi : il donne les suspects de la partie 1 et sert de point de départ à l'élimination de la partie 2.

Écrire une fonction `candidats(aliments)` qui renvoie un dictionnaire associant à chaque allergène l'ensemble des ingrédients qui peuvent le contenir. Tester sur l'exemple. ▷ coup de pouce

Exercice 5 *Compter*

But. Rassembler tous les suspects, puis compter les apparitions des autres ingrédients.
À quoi ça sert ? C'est la réponse de la partie 1. Le piège est de compter chaque ingrédient une seule fois au lieu de compter chacune de ses apparitions.

Calculer l'ensemble des ingrédients « suspects » (réunion des candidats), puis compter les apparitions des autres ingrédients dans le fichier. ▷ coup de pouce

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Il reste à identifier précisément les ingrédients dangereux, pour pouvoir les éviter.
La nouvelle règle.

- Déterminer, pour chaque allergène, l'ingrédient qui le contient ; avec les données fournies, la solution est unique.
- Ranger ces ingrédients dans l'ordre alphabétique de leur **allergène** (et non de leur propre nom).

La réponse attendue. La liste de ces ingrédients, séparés par des virgules sans espace. Par exemple, si `soy` était dans `mux` et `fish` dans `dov`, la réponse serait `dov,mux`, car `fish` vient avant `soy`.

Méthode : trier puis assembler des chaînes

Trier. Les chaînes se comparent avec `<` dans l'ordre alphabétique (`"eggs" < "gluten"` vaut `True`) : le **tri par insertion** du cours de première les range donc dans l'ordre alphabétique, sans rien changer à son code.
Assembler. Pour coller des chaînes en intercalant un séparateur, on part du premier mot et l'on ajoute, pour chacun des suivants, le séparateur puis le mot : il n'y a pas de virgule en trop à la fin.

```
mots = ["tolu", "zuli", "brak"]
texte = mots[0]
for i in range(1, len(mots)):
    texte = texte + "," + mots[i]
print(texte)                    # tolu,zuli,brak
```

Question : que se passe-t-il si la liste `mots` est vide ? Que faut-il changer pour obtenir `tolu - zuli - brak` ?

Exercice 6 *Attribuer chaque allergène*

But. Réduire les candidats par élimination jusqu'à un seul ingrédient par allergène, puis écrire la réponse dans le bon ordre.
À quoi ça sert ? C'est la réponse de la partie 2 : en termes de graphe, on retire des arêtes jusqu'à ce que chaque allergène ne soit plus relié qu'à un ingrédient.

Déterminer quel ingrédient contient chaque allergène, puis construire la chaîne demandée. Sur l'exemple inventé de la fiche, on doit obtenir `tolu,zuli,brak`. ▷ coup de pouce ▷ début de solution

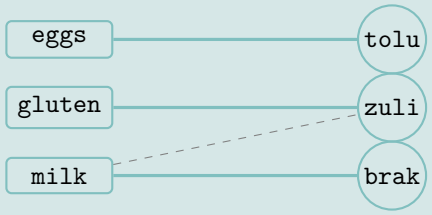
★★★ Exercice 7 *Défi : et si ça bloquait ?*

But. Chercher un cas où la méthode d'élimination ne permet plus d'avancer.
À quoi ça sert ? Le programme marche sur les données du défi, mais rien ne garantit qu'il marche toujours : construire un contre-exemple permet de connaître les limites d'un algorithme et d'imaginer une méthode plus générale.

Inventer un petit fichier pour lequel la méthode d'élimination ne suffit pas : à un moment, aucun allergène n'a un seul candidat. Que faudrait-il faire alors ? ▷ coup de pouce

Un peu d'histoire : Philip Hall et le « problème des mariages » (1935)

Le graphe du défi. Dessinons les allergènes d'un côté, les ingrédients de l'autre, et relions chaque allergène à ses candidats. Toutes les arêtes vont d'un groupe à l'autre : c'est un graphe **biparti**. Choisir un ingrédient par allergène, deux allergènes n'ayant jamais le même ingrédient, revient à garder des arêtes **sans sommet commun** : on appelle cela un **couplage**. Sur le petit exemple de la fiche :



en trait plein : le couplage trouvé par élimination

Le théorème. En 1935, le mathématicien anglais **Philip Hall** (1904–1982), spécialiste d'algèbre à Cambridge, publie un article de cinq pages, *On Representatives of Subsets*. Il y répond à une question simple : à quelle condition peut-on choisir, dans chaque ensemble d'une liste, un élément différent pour chacun ? Réponse : il faut et il suffit que **tout groupe de k ensembles contienne, à eux tous, au moins k éléments différents**. La condition est

évidemment nécessaire (trois allergènes qui n'ont que deux candidats à eux trois ne peuvent pas être servis) ; le tour de force de Hall est de prouver qu'elle suffit. En 1950, Paul Halmos et Herbert Vaughan le reformulent avec des jeunes filles qui veulent chacune épouser un garçon qu'elle connaît : depuis, on l'appelle le **théorème des mariages**.

Et aujourd'hui. En 1962, David Gale et Lloyd Shapley étudient la version où chacun classe ses partenaires par préférence (« mariages stables ») ; leur algorithme sert à répartir des étudiants entre des formations (la procédure Parcoursup s'en inspire) et a valu à Shapley le prix Nobel d'économie 2012, partagé avec Alvin Roth.

Sources : P. Hall, « On Representatives of Subsets », *J. London Math. Soc.*, 10 (1935), p. 26–30 ; P. Halmos et H. Vaughan, « The Marriage Problem », *Amer. J. Math.*, 72 (1950) ; D. Gale et L. Shapley, « College Admissions and the Stability of Marriage », *Amer. Math. Monthly*, 69 (1962).

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Après `strip`, `ligne.split(" (contains ")` sépare les deux parties ; il reste à enlever la parenthèse finale (le dernier caractère) avec la tranche `[:-1]` puis à couper selon `", "`. ↑énoncé

Ex. 4 – piste L'ingrédient qui contient un allergène est présent sur chaque ligne où cet allergène est indiqué : il est donc dans l'**intersection** (opérateur `&`) des ensembles d'ingrédients de ces lignes. ↑énoncé

Ex. 5 – piste Réunion de plusieurs ensembles : partir de `set()` et le remplacer, pour chaque ensemble de candidats `e`, par `suspects | e`. Pour compter, parcourir les ingrédients de chaque ligne et ajouter 1 à un compteur pour chacun qui n'est pas dans `suspects` ; attention, on compte chaque apparition sur chaque ligne. ↑énoncé

Ex. 6 – piste C'est le même raisonnement qu'un sudoku : un allergène qui n'a plus qu'un seul candidat est résolu, et cet ingrédient peut être retiré des candidats de tous les autres allergènes. ↑énoncé

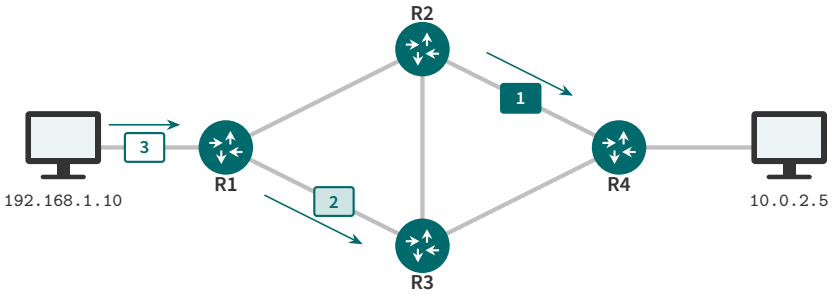
Ex. 6 – début de solution Tant que le dictionnaire des candidats n'est pas vide : chercher un allergène dont l'ensemble a une seule valeur, le retirer du dictionnaire (`del`) en notant l'ingrédient dans un dictionnaire `attribution`, puis retirer cet ingrédient (`remove`, après avoir testé `in`) de tous les autres ensembles. Pour la réponse, trier les **allergènes** (tri par insertion), puis assembler les ingrédients correspondants. ↑énoncé

Ex. 7 – piste Essayer avec deux allergènes indiqués toujours ensemble, sur les mêmes lignes : le problème a-t-il encore une seule solution ? Dans les cas où la solution est unique mais où l'élimination bloque, on peut tester les possibilités une à une (retour sur trace). ↑énoncé

9

CHAPITRE 9

Réseaux



DANS CE CHAPITRE

- Activités préparatoires 440
- Cours 442
- Exercices 454
- TP et projets 469
- Fiche de révision 472
- Défi Advent of Code 474
- Corrigés 925

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Le voyage d'un paquet

▶ corrigé p. 925

découper, aiguiller, contourner, recoller

Durée : 30 min · Sans ordinateur, par deux

Consignes

- On suit les paquets du doigt sur le plan, on compte, on corrige des tableaux. Les réponses s'écrivent sur le cahier. Aucun mot de vocabulaire n'est attendu avant l'exercice 5.

Exercice 1 *Découper le message*

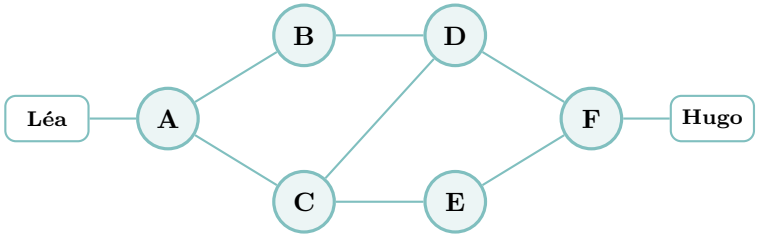
Léa veut envoyer à Hugo le message `RENDEZ-VOUS 18H PORT HERCULE` (28 caractères, espaces compris). Le réseau ne transporte que de **petits paquets** : chacun contient au plus 6 caractères du message, et porte une **étiquette** avec l'expéditeur, le destinataire et un numéro.

De : Léa À : Hugo n°1	RENDEZ

1. Combien de paquets faut-il ? Écrire le contenu du paquet n°3 et celui du dernier.
2. Pourquoi chaque paquet porte-t-il l'adresse du destinataire, et pas seulement le premier ?
3. À quoi sert le numéro ?

Exercice 2 *Les élèves-routeurs*

Les paquets traversent six **routeurs** A, B, C, D, E, F. Chacun ne connaît que **ses voisins directs** et possède une petite **table** qui lui dit à quel voisin transmettre un paquet selon sa destination.



Routeur A	
Pour aller vers	**Envoyer à**
réseau de Léa	Léa (direct)
réseau de Hugo	B

Routeur B	
Pour aller vers	**Envoyer à**
réseau de Léa	A
réseau de Hugo	D

Routeur C	
Pour aller vers	**Envoyer à**
réseau de Léa	A
réseau de Hugo	D

Routeur D	
Pour aller vers	**Envoyer à**
réseau de Léa	B
réseau de Hugo	F

Routeur E	
Pour aller vers	**Envoyer à**
réseau de Léa	C
réseau de Hugo	F

Routeur F	
Pour aller vers	**Envoyer à**
réseau de Léa	D
réseau de Hugo	Hugo (direct)

4. Suivre le paquet n°1 de Léa jusqu'à Hugo : par quels routeurs passe-t-il ? Combien de « sauts » de routeur à routeur ?
5. Le routeur B connaît-il le chemin complet jusqu'à Hugo ? Que sait-il exactement ?
6. Hugo répond « OK ». Par où passe sa réponse ? Est-ce forcément le même chemin qu'à l'aller ?
7. Trouver un autre chemin de A à F avec le même nombre de sauts.

Exercice 3 *La panne*

Pendant l'envoi, le câble entre B et D est **coupé**. Les paquets n°1 et 2 sont déjà passés ; le paquet n°3 arrive en B.

8. Que peut faire B avec sa table actuelle ?

9. B corrige sa table au plus vite : « réseau de Hugo → envoyer à A ». Mais A n'a pas encore changé la sienne. Suivre le paquet n°3 : que se passe-t-il ?

10. Pour éviter qu'un paquet tourne en rond pour toujours, on écrit sur chaque paquet un **compteur** qui vaut 8 au départ ; chaque routeur lui retire 1 et jette le paquet si le compteur atteint 0. Que devient le paquet n°3 ?

11. Quelles tables faut-il corriger pour que les paquets suivants arrivent à Hugo, et pour que ses réponses reviennent à Léa ? Écrire les nouvelles lignes. Quel est le nouveau chemin de Léa à Hugo, et combien de sauts ?

12. Sur Internet, il y a des millions de routeurs et des pannes à chaque instant. Peut-on corriger les tables « à la main », comme ici ? Que faudrait-il ?

Exercice 4 *Recoller les morceaux*

Hugo reçoit, dans cet ordre, les paquets n°2, n°1, n°5 et n°4.

13. Pourquoi les paquets peuvent-ils arriver dans le désordre ?

14. Reconstituer ce que Hugo peut lire. Que lui manque-t-il, et comment le sait-il ?

15. Que devrait faire Hugo pour obtenir le message complet ? Et que pourrait faire Léa si elle n'avait aucune nouvelle de Hugo ?

Exercice 5 *Mettre des mots*

16. Proposer, sur le cahier, un nom pour chacune des notions suivantes :

- un morceau du message, avec son étiquette ;
- l'étiquette (expéditeur, destinataire, numéro, compteur) ;
- la machine qui choisit à quel voisin transmettre ;
- le tableau « pour aller vers…, envoyer à… » ;
- la façon de faire voyager un message en morceaux indépendants ;
- les règles qui permettent aux routeurs de mettre leurs tables à jour tout seuls.

17. Recopier et compléter la phrase : « Découper en paquets rend le réseau…, car… »

COURS

Programme (BO) *« Protocoles de routage. Identifier, suivant le protocole de routage utilisé, la route empruntée par un paquet. »* Commentaire du programme : *« En mode débranché, les tables de routage étant données, on se réfère au nombre de sauts (protocole RIP) ou au coût des routes (protocole OSPF). Le lien avec les algorithmes de recherche de chemin sur un graphe est mis en évidence. »*

Prérequis la notion de **réseau** et d'**adresse IP** (Première), les **graphes** (chemins, plus court chemin), la notion de **coût**. L'algorithme de **Dijkstra** (projet du chapitre *Graphes*) éclaire OSPF, mais n'est **pas exigé** (voir la section dédiée, au-delà du programme). → fiches de révision : ch. 8 p. 425 → rappels de 1re : Web et réseau p. 1064

Objectifs *distinguer* machine, switch et routeur ; *comprendre* le voyage d'un paquet à travers les *couches* ; *lire* et *utiliser* une table de routage **donnée** ; *identifier* la route empruntée par un paquet selon le protocole (sauts pour RIP, coûts pour OSPF), et *compléter* la table d'un petit réseau ; *comparer* les choix de RIP et d'OSPF.

Situation de départ

1. Votre message traverse une dizaine de machines avant d'arriver à l'autre bout du monde. Comment chacune sait-elle à qui le transmettre, sans connaître le trajet complet ?
2. Si un câble est coupé pendant l'envoi, pourquoi la suite du message arrive-t-elle quand même ?
3. Entre une route qui traverse peu de machines et une route plus longue mais faite de liaisons rapides, laquelle choisir ?

I Le problème : relier des milliards de machines

Vous envoyez un message ; une fraction de seconde plus tard, il s'affiche à l'autre bout du monde, après avoir traversé une dizaine de machines qu'il n'a jamais « vues ». Comment ce petit paquet de données a-t-il **trouvé son chemin** parmi le milliard de routes possibles, sans que personne ne lui dise *a priori* par où passer ?

C'est toute la question de ce chapitre. **Internet** n'est pas une machine géante : c'est un **réseau de réseaux**, une immense mosaïque de petits réseaux locaux reliés entre eux par des **routeurs**. Nous allons suivre « le voyage d'un paquet » de bout en bout, et découvrir comment les routeurs, chacun ne connaissant que ses voisins, parviennent collectivement à l'acheminer.

Règle 1 — Le fil rouge : le voyage d'un paquet

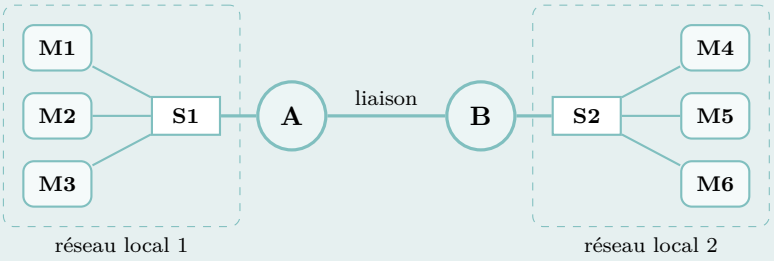
Un message est découpé en petits **paquets**. Chaque paquet voyage *indépendamment*, de routeur en routeur, chacun décidant **localement** vers qui le transmettre ensuite, grâce à sa **table de routage**. Comprendre le réseau, c'est comprendre comment ces tables se remplissent — c'est le rôle des **protocoles de routage** (RIP, OSPF), qui sont, au fond, des algorithmes de plus court chemin dans un graphe.

II Les acteurs : machines, switchs, routeurs

Rappel de Première (chapitre Le Web*) : cette partie reprend le vocabulaire déjà vu, pour le fixer avant d'aller plus loin.*

Définition 1 — Réseau local, switch, routeur

Un **réseau local** (LAN) regroupe des machines proches partageant la même **adresse réseau**. Un **switch** (commutateur) relie les machines *à l'intérieur* d'un même réseau local. Un **routeur** relie *plusieurs réseaux locaux entre eux* : il possède plusieurs **interfaces** (cartes réseau), une par réseau auquel il est raccordé. **Internet = l'interconnexion de tous ces réseaux par des routeurs.**



Dans un réseau local, un paquet destiné à une machine *du même réseau* y va directement via le switch. Mais s'il vise une machine d'un *autre* réseau, le switch le remet au **routeur**, qui devra l'aiguiller de proche en proche jusqu'au bon réseau.

► **Exercices d'application :** exercice 1 (machines, switchs, routeurs)

III Les adresses IP : reconnaître son réseau

Chaque interface possède une **adresse IP** (en IPv4 : quatre octets, soit quatre nombres de 0 à 255, par exemple `172.16.5.3` — donc 32 bits en tout, d'où l'intérêt de l'écriture binaire vue en Première). Cette adresse se découpe en deux parties : une partie **réseau** (commune à toutes les machines du réseau local) et une partie **hôte** (qui identifie la machine dans ce réseau). Où se situe la frontière ? C'est le rôle du **masque**.

Définition 2 — Notation CIDR (le `/n`)

La notation **CIDR** (*Classless Inter-Domain Routing*) écrit le masque sous la forme `/n` : les `n` **premiers bits** de l'adresse forment la partie **réseau** (ils sont à 1 dans le masque), les $32 - n$ suivants forment la partie **hôte** (à 0). Deux machines sont dans **le même réseau local** si et seulement si leurs `n` premiers bits coïncident. Dans ce réseau :

- l'adresse dont **tous** les bits d'hôte valent 0 est l'**adresse du réseau** ;
- l'adresse dont **tous** les bits d'hôte valent 1 est l'**adresse de diffusion** (*broadcast*) ;
- ces deux-là étant réservées, le nombre de machines adressables est $\mathbf{2^{32-n} - 2}$.

L'intérêt de CIDR, c'est que `n` peut prendre **n'importe quelle valeur** de 0 à 32 (pas seulement 8, 16 ou 24) : on ajuste finement la taille d'un réseau à ses besoins.

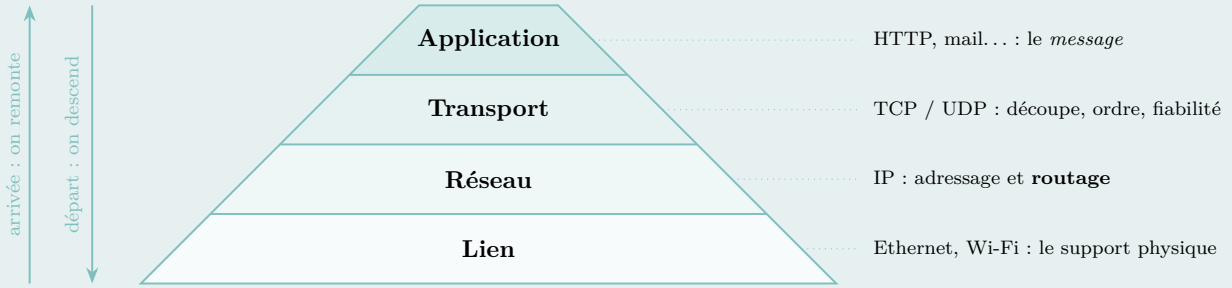
Exemple

Avec `172.16.5.3/16`, le `/16` dit que les 16 premiers bits (`172.16`) forment l'adresse réseau : adresse du réseau `172.16.0.0`, diffusion `172.16.255.255`, et $2^{32-16} - 2 = 2^{16} - 2 = 65\,534$ machines possibles. Toute machine `172.16.•.•` est dans ce réseau ; `172.18.1.1/16` (réseau `172.18`) est dans un *autre* réseau — il faudra un routeur. Avec un `/24` (ex. `192.168.1.0/24`), il ne reste que 8 bits d'hôte, donc $2^8 - 2 = 254$ machines ; avec un `/30`, $2^2 - 2 = 2$ machines seulement (juste ce qu'il faut pour relier deux routeurs).

► **Exercices d'application :** exercices 2 à 4 et 18 (lire une adresse IP ; adressage IP ; dimensionner des sous-réseaux ; adresses de réseau en série)

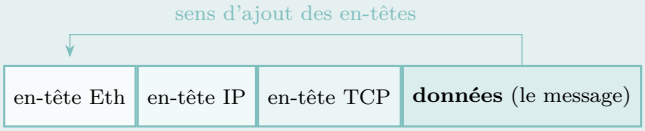
IV Le voyage d'un paquet : le modèle en couches

Comment une même donnée peut-elle à la fois s'adresser à *une application* précise, être acheminée d'un *bout à l'autre* d'Internet, et circuler sur *un câble* local ? En empilant plusieurs **couches**, chacune ne s'occupant que d'une chose. C'est le modèle **TCP/IP**.



À l'émission, la donnée **descend** les couches : chacune ajoute son **en-tête** (son étiquette) — c'est l'**encapsulation**, comme des enveloppes emboîtées. À l'arrivée, la donnée **remonte** : chaque couche retire son en-tête et transmet le reste au-dessus.

↻ **Retour sur l'activité.** L'étiquette collée sur chaque paquet de l'activité (expéditeur, destinataire, numéro, compteur) était un **en-tête**. Celui de la couche Réseau (IP) porte bien les adresses, et le compteur qui faisait jeter le paquet tournant en rond entre A et B y existe vraiment.



Encapsulation : en descendant les couches, chaque en-tête s'ajoute **à gauche** du bloc précédent. À l'arrivée, on les retire un à un.

Ce chapitre se concentre sur la couche **Réseau (IP)** : c'est là que se joue le **routage**.

► **Exercices d'application :** exercice 5 (des enveloppes emboîtées : les couches)

V La commutation de paquets

Rappel de Première (chapitre Le Web*) : le découpage en paquets a déjà été rencontré ; on en tire ici les conséquences pour le routage.*

Plutôt que de réserver une ligne entière pour toute la durée d'un échange (*commutation de circuits*, comme l'ancien téléphone), Internet **découpe** les messages en **paquets** indépendants qui empruntent le réseau chacun de leur côté (**commutation de paquets**).

Propriété 1 — Pourquoi des paquets ?

La commutation de paquets **partage** efficacement les liens entre tous les échanges, et surtout elle est **robuste** : si un routeur ou un câble tombe, les paquets suivants *contournent* la panne par un autre chemin. C'est cette robustesse qui a fondé Internet.

↻ **Retour sur l'activité.** Le message de Léa, découpé en 5 paquets numérotés, voyageait ainsi : chaque paquet porte l'adresse de Hugo parce qu'il voyage seul, son numéro permet de recoller les morceaux arrivés dans le désordre, et après la coupure du câble B–D les paquets suivants passaient par C, toujours en 3 sauts.

VI Le routage : la table de routage

Un routeur qui reçoit un paquet consulte sa **table de routage** : pour chaque réseau de destination, elle indique **par où sortir** (quelle interface, ou quel routeur voisin) et une **métrique** (le « coût » de la route ; plus elle est petite, mieux c'est). Un réseau directement raccordé a une métrique **nulle**. *Au bac, selon les énoncés, cette métrique est notée* 0, 1 *ou « – » : suivez l'énoncé.*

Exemple

Table de routage (simplifiée) d'un routeur A relié aux réseaux `R1` et `R3`, et au routeur G :

Réseau à atteindre	Sortir par	Métrique
R1	interface eth0	0
R3	interface eth2	0
R2	routeur G	1

Lecture de la dernière ligne : « pour atteindre R2, envoie le paquet au routeur G, qui saura continuer ».

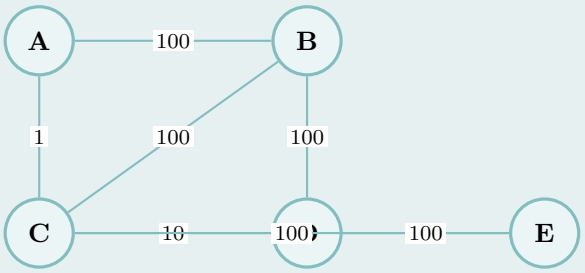
Comment remplir ces tables ? Deux façons : le **routage statique** (chaque ligne écrite à la main — possible seulement pour de tout petits réseaux) et le **routage dynamique**, où des **protocoles** découvrent les routes automatiquement. C'est ce dernier qui nous intéresse.

↻ **Retour sur l'activité.** Chaque élève-routeur tenait une table de routage : B ne connaissait pas le chemin jusqu'à Hugo, seulement le voisin à qui transmettre (D). Après la panne, corriger les tables à la main était faisable pour six routeurs, impossible pour des millions : c'est le travail des protocoles de routage dynamique.

Règle 2 — Un réseau de routeurs = un graphe

Modélisons : chaque **routeur** est un **sommet**, chaque **liaison** entre routeurs est une **arête**. Trouver la meilleure route, c'est chercher un **plus court chemin** dans ce graphe. Les protocoles de routage sont donc des **algorithmes de graphes** — ceux du chapitre *Graphes* !

Prenons ce réseau pour toute la suite (les nombres sur les liens seront expliqués plus loin) :



chaque routeur porte aussi son réseau local (non dessiné) ; nombres = débits (Mbit/s)

► **Exercices d'application :** exercice 6 (lire une table de routage)

VII RIP : compter les sauts

Le protocole **RIP** (*Routing Information Protocol*) mesure une route par son **nombre de sauts** : le nombre de routeurs à traverser. Chaque routeur envoie **périodiquement** (toutes les 30 s) à ses voisins la liste des réseaux qu'il connaît et à quelle distance ; de proche en proche, chacun complète sa table en gardant, pour chaque destination, la route au **plus petit nombre de sauts**.

Définition 3 — Métrique de RIP

Métrique RIP = **nombre de sauts** (routeurs traversés). C'est un **vecteur de distance** : chaque routeur ne connaît que « à quelle distance » est chaque réseau, sans voir toute la carte. RIP s'appuie sur l'algorithme de **Bellman-Ford**, un algorithme de programmation dynamique (on verra cette méthode au chapitre 11).

Exemple

Table de routage de **A** par RIP (destinations = réseaux locaux de B, C, D, E) :

▷ **Je prédis.** Avant de lire la table : pour atteindre le réseau de E, vers quel voisin A envoie-t-il le paquet, et en combien de sauts ?

Réseau	Sortir vers	Sauts
réseau de B	B	1
réseau de C	C	1
réseau de D	B (ou C)	2
réseau de E	C	2

Pour atteindre C, RIP choisit le lien **direct** A–C (1 saut). Pour D, deux routes ont 2 sauts, A–B–D et A–C–D : elles sont **à égalité**, l'une ou l'autre convient (un énoncé de bac précise alors laquelle retenir, ou accepte les deux) ; pour E, A–C–E (2 sauts).

Exemple — *un réseau plus grand*

Voici un réseau à **six** routeurs (les nombres sont des débits, ignorés par RIP) :

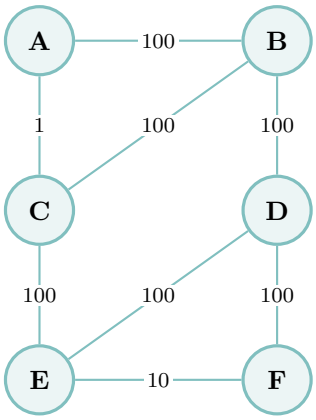


Table de routage **RIP** de A (on compte les sauts en remontant de proche en proche) :

Réseau	Sortir vers	Sauts
réseau de B	B	1
réseau de C	C	1
réseau de D	B	2 (A–B–D)
réseau de E	C	2 (A–C–E)
réseau de F	B (ou C)	3 (A–B–D–F ou A–C–E–F)

Pour F, deux routes de 3 sauts sont à égalité : l'une ou l'autre convient.

À retenir — les limites de RIP

RIP est simple, mais il a deux gros défauts : ses messages périodiques **encombrent** le réseau, et il est **plafonné à 15 sauts** (au-delà, la destination est réputée inaccessible). Surtout, il ne regarde que le *nombre* de routeurs, jamais leur *vitesse* : un saut par une liaison poussive compte autant qu'un saut par la fibre. D'où OSPF.

► **Exercices d'application :** exercices 7 **et** 19 (construire une table RIP ; tables RIP en série)

VIII OSPF : le coût des liens

OSPF (*Open Shortest Path First*) ne compte pas les sauts : il additionne le **coût** des liaisons traversées. Le coût d'une liaison dépend de son **débit** (le nombre de bits par seconde qu'elle peut transporter) : plus c'est rapide, moins c'est cher.

Définition 4 — Métrique d'OSPF

Le **coût** d'une liaison se calcule à partir de son débit :

$$\text{coût} = \frac{10^8}{\text{débit (en bits/s)}}.$$

La **métrique d'une route** est la **somme des coûts** des liaisons traversées. Chaque routeur connaît la carte complète du réseau (*état de liens*) et en déduit les routes de coût minimal (en coulisses, par l'algorithme de **Dijkstra** : voir la dernière section sur le routage, au-delà du programme). Sur les petits réseaux du bac, on trouve ces routes en comparant les coûts totaux de quelques chemins.

Au bac — lire la formule de coût donnée par l'énoncé

Le 10^8 **n'est qu'une convention** (une « bande passante de référence », ici 100 Mbit/s) : selon les énoncés, on rencontre aussi 10^9 ou 10^{10}, ce qui ne change pas *quelle* route est la meilleure, seulement l'échelle des coûts. Dans la vraie norme OSPF, le coût est de plus **arrondi à un entier** (avec un minimum de 1). Il faut donc toujours **lire la formule donnée dans l'énoncé** avant de calculer.

Exemple

Sur nos liens (en Mbit/s, soit $\times 10^6$ bits/s), le coût vaut 10^8/débit :

Débit	100 Mbit/s	10 Mbit/s	1 Mbit/s
Coût $= 10^8$/débit	1	10	100

Les liens à 100 Mbit/s coûtent donc 1, le lien A–C (à seulement 1 Mbit/s) coûte 100, et C–D (à 10 Mbit/s) coûte 10. Table de routage de **A** par OSPF :

▷ **Je prédis.** Avant de lire la table : pour atteindre le réseau de C, A emprunte-t-il encore le lien direct A–C ? Quel est le coût de la meilleure route ?

Réseau	Sortir vers	Coût
réseau de B	B	1
réseau de C	B	2 (A–B–C)
réseau de D	B	2 (A–B–D)
réseau de E	B	3 (A–B–C–E ou A–B–D–E)

Pour E, deux routes coûtent 3 ; elles partent toutes deux vers B, la table est donc la même.

Exemple — *le réseau à six routeurs, en OSPF*

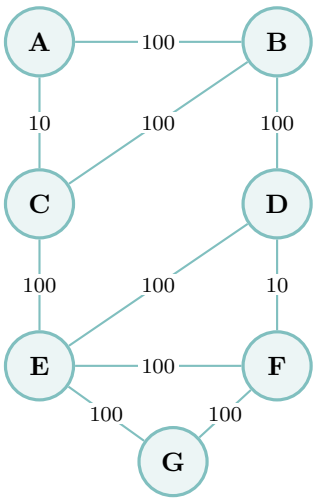
Reprenons le réseau à six routeurs ci-dessus. Les liens à 100 Mbit/s coûtent 1 ; le lien A–C (1 Mbit/s) coûte 100 ; le lien E–F (10 Mbit/s) coûte 10. Table **OSPF** de A :

Réseau	Sortir vers	Coût
réseau de B	B	1 (A–B)
réseau de C	B	2 (A–B–C)
réseau de D	B	2 (A–B–D)
réseau de E	B	3 (A–B–C–E ou A–B–D–E)
réseau de F	B	3 (A–B–D–F)

On retrouve le **désaccord** avec RIP : pour C et E, RIP passait par le lien direct A–C (lent), OSPF fait le détour rapide par B. Pour F, OSPF passe par A–B–D–F (coût 3) ; RIP hésitait entre deux routes de 3 sauts : s'il retient A–B–D–F, les deux protocoles s'accordent.

Exemple — *un réseau à sept routeurs, entièrement en OSPF*

Voici un réseau plus étendu ; les nombres sont les **débits** (Mbit/s). On applique OSPF (coût $= 10^8$/débit) depuis A.



Coûts : les liens à 100 Mbit/s valent 1, les liens A–C et D–F (10 Mbit/s) valent 10. Table **OSPF** de A :

Réseau	Sortir vers	Coût
réseau de B	B	1 (A–B)
réseau de C	B	2 (A–B–C)
réseau de D	B	2 (A–B–D)
réseau de E	B	3 (A–B–C–E ou A–B–D–E)
réseau de F	B	4 (A–B–C–E–F ou A–B–D–E–F)
réseau de G	B	4 (A–B–C–E–G ou A–B–D–E–G)

Plusieurs routes sont à égalité, mais elles partent toutes vers B : la table de A n'en dépend pas. On voit bien OSPF **additionner les coûts** le long de chaque route et retenir la moins chère : jamais il n'emprunte les liens à 10 (A–C, D–F), toujours il reste sur la fibre à 100, quitte à traverser plus de routeurs.

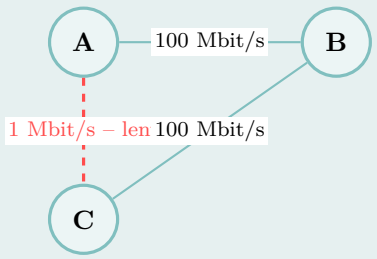
► **Exercices d'application :** exercices 8 **et** 9 (du débit au coût ; table OSPF ; duel avec RIP)

IX RIP contre OSPF : le duel du lien lent

Comparons les deux tables de A. Elles **diffèrent** là où le lien direct A–C entre en jeu :

Pour atteindre	RIP choisit	OSPF choisit
le réseau de C	**A–C** (1 saut)	A–B–C (coût 2)
le réseau de E	A–C–E (2 sauts)	A–B–C–E (coût 3)

RIP fonce sur le lien **direct** A–C … sans voir que c'est une liaison **poussive** à 1 Mbit/s ! OSPF, lui, préfère faire **un détour par B** sur des liaisons rapides : la route est plus longue en nombre de routeurs, mais bien plus rapide en réalité.



RIP : A–C (1 saut)
OSPF : A–B–C (rapide)

Propriété 2 — La leçon

Compter les sauts (RIP) n'est pas compter le temps (OSPF). En pénalisant les liaisons lentes par un coût élevé, OSPF trouve les routes réellement les plus rapides. C'est pourquoi les grands réseaux lui donnent la préférence.

9

X Dans les coulisses d'OSPF : l'algorithme de Dijkstra [au-delà du programme]

Le programme officiel est clair : au bac, **les tables de routage sont données**, ou se lisent « à l'œil » sur un petit réseau, en comparant le nombre de sauts (RIP) ou le coût total (OSPF) de quelques routes. C'est exactement ce que nous avons fait plus haut. Mais un vrai routeur OSPF, devant un réseau de plusieurs centaines de routeurs, ne peut pas « regarder » : il calcule. Et ce calcul, c'est l'**algorithme de Dijkstra**, que vous avez programmé dans le projet proposé en fin de chapitre *Graphes*. Ce paragraphe, **au-delà du programme**, le rappelle pour comprendre ce que fait vraiment OSPF.

Le principe de Dijkstra (rappel du projet *Graphes*)

On part du routeur de départ et on fait grandir, petit à petit, un « nuage » de sommets dont on connaît **avec certitude** la distance minimale (ici : le coût minimal) au départ.

1. On garde pour chaque sommet une **distance provisoire** (au début : 0 pour le départ, $+\infty$ pour les autres).
2. À chaque tour, on **fixe définitivement** le sommet non encore traité qui a la **plus petite** distance provisoire : il entre dans le nuage.
3. Depuis ce sommet `u` tout juste fixé, on **met à jour** ses voisins : si passer par `u` raccourcit le chemin vers un voisin `v`, on améliore la distance de `v` et on note `u` comme **prédécesseur** de `v`. C'est le **relâchement** :

```
si distance[u] + coût(u,v) < distance[v] :
    distance[v] = distance[u] + coût(u,v)
```

On recommence jusqu'à avoir fixé tous les sommets.

Exemple — *Dijkstra sur le réseau du cours, depuis A*
Coûts OSPF : A–B, B–C, B–D, C–E, D–E valent 1 ; C–D vaut 10 ; A–C vaut 100. À chaque ligne, on fixe le sommet marqué $*$ (en cas d'égalité, on prend l'ordre alphabétique) ; entre parenthèses, le prédécesseur.

Fixé	A	B	C	D	E
—	0	∞	∞	∞	∞
A	0*	1 (A)	100 (A)	∞	∞
B		1*	2 (B)	2 (B)	∞
C			2*	2 (B)	3 (C)
D				2*	3 (C)
E					3*

Au deuxième tour, la distance provisoire de C tombe de 100 (lien direct, lent) à 2 (par B) : c'est le relâchement qui « découvre » le détour rapide. Pour remplir la colonne « sortir vers » de la table, on **remonte les prédécesseurs** jusqu'au départ : E ← C ← B ← A, donc on sort vers **B**. On retrouve exactement la table OSPF de A donnée plus haut.

Pour les curieux (facultatif)
Si tous les liens avaient le même coût 1, Dijkstra fixerait les sommets dans l'ordre d'un parcours en largeur, et la table obtenue serait celle de RIP (nombre de sauts), au choix près entre routes de même longueur. RIP, lui, ne fonctionne pas ainsi : chaque routeur n'a pas la carte complète, il ne connaît que ce que ses voisins lui annoncent (algorithme de **Bellman-Ford**, réparti entre les routeurs).

► **Exercices d'application :** exercice 10 (au-delà du programme : dérouler Dijkstra pour obtenir une table OSPF)

XI TCP et UDP : deux façons de transporter

Juste au-dessus d'IP, la couche **Transport** offre deux services au choix. **TCP** garantit que *tout* arrive, *dans l'ordre*, sans perte (il numérote les paquets, les réordonne, redemande ceux qui manquent grâce à des **accusés de réception**) : idéal pour un fichier ou une page web. [au-delà du programme] **UDP** envoie sans garantie ni accusé de réception, mais **sans attendre** : idéal quand la vitesse prime sur la perfection (appel vidéo, jeu en ligne, où un paquet en retard est inutile de toute façon).

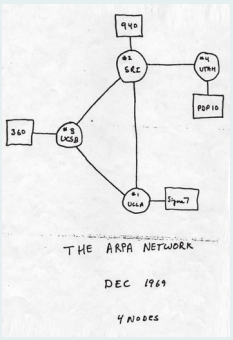
XII Ouverture

Ce chapitre est l'application la plus directe du chapitre *Graphes* : un réseau de routeurs *est* un graphe, et le routage est une recherche de plus court chemin.

- Au **bac**, on vous demandera surtout de manipuler des **adresses IP** (notation CIDR, appartenance à un réseau), de **lire** ou **compléter** une table de routage, de calculer des **coûts OSPF** et de **déterminer** la route suivie par un paquet selon RIP ou selon OSPF.

XIII Un peu d'histoire : d'ARPANET à Internet

Dans la première moitié des années 1960, l'idée de **commutation de paquets** germe indépendamment chez **Paul Baran** (États-Unis) et **Donald Davies** (Royaume-Uni) : découper l'information en morceaux pour bâtir un réseau qui *survive* à la panne d'un nœud. En 1969 naît **ARPANET**, ancêtre d'Internet, reliant quatre universités américaines. Le croquis ci-contre, dessiné en décembre 1969, montre ce réseau tout entier : quatre nœuds seulement, à UCLA, au Stanford Research Institute (SRI), à Santa Barbara (UCSB) et à l'université de l'Utah. Le tout premier message, envoyé de UCLA vers SRI le 29 octobre 1969, devait être « LOGIN »… le système a planté après « LO ».



Le réseau ARPANET en décembre 1969

En France, **Louis Pouzin** conçoit à partir de 1971 le réseau **Cyclades**, qui invente le **datagramme** : le réseau se contente d'acheminer chaque paquet indépendamment, au mieux, et ce sont les machines situées aux extrémités qui vérifient que tout est bien arrivé. En 1974, **Vint Cerf** et **Bob Kahn** conçoivent, en reprenant cette idée, **TCP/IP**, le langage commun qui permet à des réseaux *différents* de se comprendre — l'acte de naissance d'Internet, officiellement adopté le 1er janvier 1983. Aujourd'hui, ces mêmes principes acheminent chaque jour des milliards de paquets : à chaque page ouverte, un petit algorithme de plus court chemin travaille pour vous.



Louis Pouzin, père de Cyclades

9

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise :** les *Graphes* : RIP compte les sauts, OSPF additionne les coûts des liens (Dijkstra) pour trouver un plus court chemin.
- **De la Première :** les adresses IP, le binaire et le bit alterné, vu dans *Le Web : réseaux et interactions homme-machine* (Première), que prolongent les accusés de réception de TCP.
- **Pour aller plus loin :** un paquet peut être lu en chemin : vous verrez en *Cryptographie* comment HTTPS le protège.

Bilan — la carte mémoire

Notion	À retenir
Internet	un **réseau de réseaux** reliés par des **routeurs**
Switch / routeur	switch = dans un LAN ; routeur = entre plusieurs LAN
Adresse IP / masque	`/n` = nb de bits de l'**adresse réseau** ; même réseau ?
Couches (TCP/IP)	Application / Transport / Réseau (IP) / Lien ; **encapsulation**
Commutation de paquets	message découpé ; robuste (contourne les pannes)
Table de routage	par où sortir + métrique ; statique ou **dynamique**
RIP	métrique = **nombre de sauts** ; vecteur de distance ; ≤ 15 sauts
OSPF	métrique = **somme des coûts**, coût $= 10^8$/débit
Dijkstra [au-delà du programme]	le calcul qui se cache derrière OSPF (nuage + relâchement)
RIP vs OSPF	RIP ignore la vitesse ; OSPF évite les liens lents

Retour à la situation de départ

1. Chaque **routeur** consulte sa **table de routage**, qui indique, pour chaque réseau de destination, par où sortir : il décide localement du saut suivant. Les tables se remplissent automatiquement grâce aux **protocoles de routage** (§ VI à VIII).
2. Grâce à la **commutation de paquets** : le message est découpé en paquets indépendants, et une fois les tables mises à jour, les paquets suivants **contournent** la panne par un autre chemin (§ V).
3. Cela dépend du protocole : **RIP** compte les **sauts** et prend la route qui traverse le moins de routeurs ; **OSPF** additionne les **coûts** des liaisons (10^8/débit) et préfère les liaisons rapides, même au prix d'un détour (§ VII à IX).

Erreurs fréquentes

- **Confondre switch et routeur.** Le switch relie des machines *d'un même* réseau ; le routeur relie *plusieurs* réseaux.
- **Se tromper de métrique.** RIP compte les **sauts** (des entiers, 1 par routeur) ; OSPF additionne des **coûts** (10^8/débit).
- **Croire que le chemin le plus court en sauts est le plus rapide.** Faux : un seul saut par une liaison lente peut être pire que trois sauts par la fibre.
- **Oublier la métrique** 0 des réseaux directement raccordés.
- **Inverser le sens du coût.** Plus le débit est *grand*, plus le coût est *petit* (10^8/débit).

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

☐ **distinguer** machine, switch et routeur, et reconnaître un réseau local à son adresse → ex. 1, 2, 3 ;

☐ **expliquer** l'encapsulation en couches et le rôle de la couche IP → ex. 5 ;

☐ **lire** et **utiliser** une table de routage donnée, et la **compléter** sur un petit réseau (réseau, sortie, métrique) → ex. 6, 7 ;

☐ **identifier la route RIP** (nombre de sauts) sur un réseau donné → ex. 7, 12 ;

☐ **calculer les coûts** (10^8/débit), **identifier la route OSPF** (coût total minimal), puis **comparer** à RIP → ex. 8, 9, 12 ;

☐ [au-delà du programme] **dérouler Dijkstra** pour retrouver une table OSPF → ex. 10.

Vers le Grand Oral

- **Comment un message trouve-t-il son chemin sur Internet ?** *(paquets, tables de routage, routage de proche en proche.)*
- **Compter les sauts ou compter le temps : comment choisir une route ?** *(RIP vs OSPF ; le lien lent ; Dijkstra sur le graphe des routeurs.)*
- **Pourquoi découper un message en paquets rend-il Internet indestructible ?** *(commutation de paquets ; contournement des pannes ; l'héritage d'ARPANET.)*
- **En quoi le routage est-il un problème de graphes ?** *(routeurs = sommets, liaisons = arêtes, plus court chemin ; lien avec Dijkstra.)*

EXERCICES

Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

Mode d'emploi

Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 32.
On soigne les **tables de routage** (destination, sortie, métrique). Rappels : RIP = nombre de sauts ; OSPF = somme des coûts, coût = 10^8/débit.

Machines, switchs, routeurs ↑cours

★ **Exercice 1** *Qui fait quoi ?* ▶ corrigé p. 926 • essentiel

1. Quelle est la différence entre un **switch** et un **routeur** ?
2. Deux machines du *même* réseau local doivent-elles passer par un routeur pour communiquer ?
3. Qu'est-ce qu'une **interface** d'un routeur ? Pourquoi un routeur en a-t-il plusieurs ?
4. Compléter : « Internet est l'interconnexion de ... par des ... ».

▷ coup de pouce

Adressage IP ↑cours

★ **Exercice 2** *Lire une adresse IP* ▶ corrigé p. 926

On considère la machine d'adresse `192.168.1.37/24`.

1. Combien d'octets, et donc de bits, compte une adresse IPv4 ?
2. Écrire cette adresse en binaire (un groupe de 8 bits par octet).
3. Que signifie `/24` ? Quelle partie de l'adresse désigne le réseau, laquelle désigne la machine ?
4. Pourquoi l'écriture `192.168.1.300` ne peut-elle pas être une adresse IPv4 ?

▷ coup de pouce

★★ **Exercice 3** *Même réseau ou pas ?* ▶ corrigé p. 927 • essentiel

Rappels : avec un masque `/n`, les `n` premiers bits donnent la partie « réseau ». L'adresse dont tous les bits d'hôte valent 0 est l'**adresse du réseau** ; celle dont ils valent tous 1 est l'**adresse de diffusion** (*broadcast*).

1. Les machines `192.168.16.12/24` et `192.168.16.240/24` sont-elles dans le même réseau local ? Et `192.168.16.12/24` et `192.168.17.3/24` ?
2. Pour le réseau de la machine `192.168.16.12/24`, donner : l'adresse du réseau, l'adresse de diffusion, et le nombre maximal de machines connectables.
3. Même question (adresse réseau et nombre de machines) pour `172.16.5.3/16`. ▷ coup de pouce

★★ **Exercice 4** *Dimensionner les réseaux d'une entreprise* ▶ corrigé p. 927

L'entreprise **CaféNet** raccorde plusieurs sites. Pour chacun, on veut choisir le masque `/n` donnant le **plus petit réseau** qui contient *toutes* ses machines, afin de **gaspiller le moins d'adresses possible** (rappel : un réseau `/n` accueille $2^{32-n} - 2$ machines). Voici les besoins :

Site	Nombre de machines
Siège	100
Agence Nord	50
Agence Sud	20
Boutique	10
Liaison entre deux routeurs	2

1. Pour chaque site (sauf la liaison entre routeurs, traitée à la question 2), donner le masque `/n` le plus adapté et le nombre d'adresses de machines alors disponibles.

2. Pour la liaison entre deux routeurs, on choisit un `/30`. Combien d'adresses de machines offre-t-il ? Pourquoi est-ce exactement ce qu'il faut ?

3. Si l'on attribuait paresseusement un `/24` à *chaque* site, combien d'adresses de machines seraient inutilisées au Siège ? à la Boutique ? Que penser de ce choix ? ▷ coup de pouce

Le modèle en couches ↑ cours

★★ **Exercice 5** *Des enveloppes emboîtées* ▶ corrigé p. 927 • essentiel

Depuis son ordinateur portable relié en Wi-Fi, Léa ouvre une page web : son navigateur envoie une requête **HTTP** à un serveur situé à l'autre bout d'Internet. Le paquet traverse **trois routeurs** avant d'arriver.

1. Ranger les quatre couches du modèle TCP/IP de la plus **haute** à la plus **basse** : Lien, Application, Réseau, Transport. Associer à chacune l'un des protocoles HTTP, TCP, IP, Wi-Fi.

2. Qu'est-ce que l'**encapsulation** ? Dessiner le bloc qui quitte l'ordinateur de Léa : la requête HTTP et les trois en-têtes ajoutés, dans l'ordre où ils apparaissent de gauche à droite.

3. Chaque routeur doit choisir par où renvoyer le paquet. Quel en-tête lit-il pour cela, et quelle information y trouve-t-il ? A-t-il besoin de lire la requête HTTP ?

4. D'un routeur à l'autre, le paquet change de support (Wi-Fi, puis câbles Ethernet, puis fibre). Quel en-tête est remplacé à chaque lien ? Quelles informations de l'en-tête IP restent les mêmes du début à la fin du trajet ?

5. À l'arrivée sur le serveur, dans quel ordre les en-têtes sont-ils retirés ? À quel programme la requête est-elle finalement remise ?

6. Léa quitte le Wi-Fi et passe en 4G pendant le chargement. Pourquoi son navigateur n'a-t-il pas besoin d'être modifié ? En déduire l'intérêt du découpage en couches. ▷ coup de pouce

Tables de routage ↑ cours

★ **Exercice 6** IA IA IA *Lire une table* ▶ corrigé p. 928

Un routeur A a la table de routage suivante :

Réseau	Sortir par	Métrique
N1	eth0	0
N3	eth2	0
N2	routeur G	1

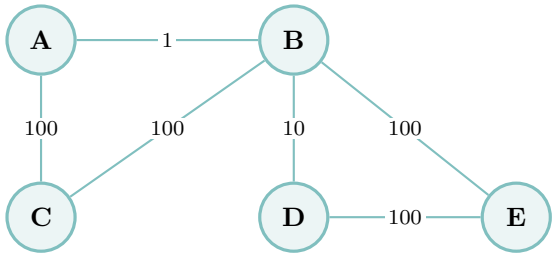
1. Quels réseaux sont *directement* reliés à A ? Comment le voit-on ?
2. Un paquet arrive en A, destiné au réseau N2 : que fait A ?
3. Que signifie une métrique de 0 ?

▷ coup de pouce

Le protocole RIP ↑ cours

★★ **Exercice 7** IA IA IA *Construire une table RIP* ▶ corrigé p. 928 • essentiel

On considère le réseau ci-dessous (les nombres sont les débits en Mbit/s ; on les **ignore** pour RIP). Chaque routeur dessert son propre réseau local.



1. Donner, pour le routeur **A**, le nombre de sauts vers chacun des routeurs B, C, D, E (métrique RIP).
2. En déduire la table de routage RIP de A (colonnes : réseau visé, on sort *vers* quel voisin, nombre de sauts).
3. Quelle est la limite de RIP en nombre de sauts, et pourquoi ses messages périodiques posent-ils problème sur un grand réseau ?

▷ coup de pouce

Le protocole OSPF ↑ cours

★ **Exercice 8** IA IA IA *Du débit au coût* ▶ corrigé p. 928

En OSPF, le coût d'une liaison vaut 10^8/débit, le débit étant exprimé en bit/s (1 Mbit/s = 10^6 bit/s, 1 Gbit/s = 10^9 bit/s).

1. Calculer le coût d'une liaison à 1 Gbit/s, à 50 Mbit/s et à 20 Mbit/s.
2. Classer ces trois liaisons de la plus intéressante à la moins intéressante pour OSPF.
3. Un chemin emprunte une liaison à 1 Gbit/s puis deux liaisons à 50 Mbit/s. Quel est son coût total ?

▷ coup de pouce

★★ **Exercice 9** *Coûts, table OSPF, et duel avec RIP* ▶ corrigé p. 928 • essentiel

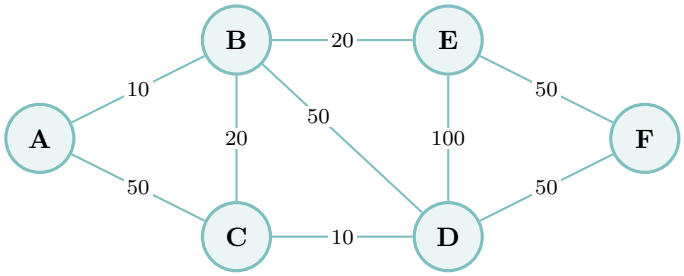
On reprend **le réseau de l'exercice 7**, mais cette fois les nombres sont les **débits** qui servent à calculer les coûts OSPF (coût $= 10^8$/débit).

1. Calculer le coût de chaque type de liaison : 100 Mbit/s, 10 Mbit/s, 1 Mbit/s.
2. Donner la table de routage **OSPF** de A (réseau visé, sortie, coût total).
3. **Comparer** avec la table RIP de A : pour quelles destinations les deux protocoles *ne choisissent-ils pas la même route* ? Expliquer pourquoi. ▷ coup de pouce

★★★ **Exercice 10** *Dijkstra derrière OSPF* *[au-delà du programme]* ▶ corrigé p. 929 ≈ 45 min

*Cet exercice va **au-delà du programme officiel** : au bac, les tables de routage sont données, ou se lisent sur un petit réseau. Il sert à comprendre ce que fait vraiment un routeur OSPF.*

Voici un réseau de six routeurs ; les nombres sont les **débits** des liaisons, en Mbit/s. On rappelle : coût $= 10^8$/débit (débit en bit/s).



1. Calculer le coût OSPF de chaque liaison et le reporter sur un schéma.
2. Dérouler l'algorithme de Dijkstra au départ de `A` : recopier et compléter un tableau (une ligne par sommet fixé ; colonnes `A` à `F` ; distance provisoire et prédécesseur ; ∞ = pas encore atteint). ▷ coup de pouce
3. En déduire la table de routage OSPF de `A` (réseau visé, sortir vers, coût total). ▷ coup de pouce
4. Par quelle route RIP enverrait-il un paquet de `A` vers le réseau de `B` ? Et OSPF ? Expliquer.
5. La liaison `A`–`C` tombe en panne. Dérouler à nouveau Dijkstra depuis `A` et donner la nouvelle table OSPF de `A`.
6. ▶ Vérifier vos tableaux avec la fonction `dijkstra` écrite dans le projet proposé en fin de chapitre *Graphes*.

Exercices type bac ↑cours

★★★ **Exercice 11** IA IA IA *Réseau d'un établissement* (d'après La Réunion 2023, jour 2) ▶ corrigé p. 929 ≈ 60 min

Un ensemble scolaire « Établissement » regroupe plusieurs sites (l'Administration, l'école primaire, le collège, le lycée général, le lycée professionnel). Les postes de chaque site sont reliés entre eux par des commutateurs (switchs), eux-mêmes reliés à des routeurs. R1, R2, R3, R4, R5 et R6 sont les routeurs de l'établissement ; la figure 1 montre leurs liaisons et le routeur qui dessert trois des secteurs.

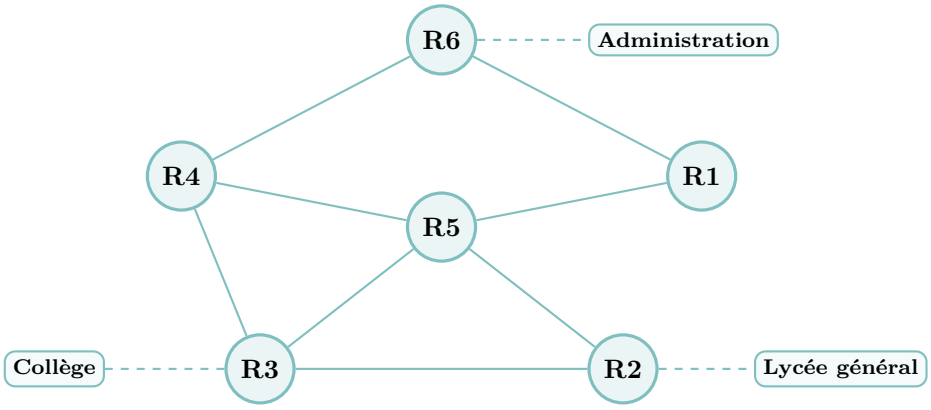


Figure 1 : les routeurs de l'établissement (en pointillés : le routeur qui dessert un secteur)

Rappels. Une adresse IPv4 est composée de 4 octets, soit 32 bits, notée `A.B.C.D`. La notation `A.B.C.D/n` signifie que les n premiers bits de l'adresse représentent la partie « réseau », les bits suivants la partie « hôte » ; les n premiers bits du masque valent donc 1, les autres 0. L'adresse dont tous les bits de la partie hôte sont à 0 est l'*adresse du réseau* ; celle dont tous les bits de la partie hôte sont à 1 est l'*adresse de diffusion*.

1. Le poste PC LG 03 de la salle informatique du Lycée général a pour adresse IPv4 `192.168.162.4`. Recopier et compléter son écriture binaire : `11000000.10101000.xxxxxxxx.xxxxxxxx`.

2. Le poste PC Admin 02 du secteur « Administration » a pour adresse IPv4 `192.168.16.12/24`.

   (a) Donner l'adresse du réseau local dédié au secteur « Administration » et son masque de sous-réseau.

   (b) Donner l'adresse de diffusion (*broadcast*) de ce réseau.

   (c) Donner le nombre maximal de machines que l'on peut connecter sur ce réseau.

3. L'administrateur s'intéresse à la transmission de paquets du secteur « Administration » vers le secteur « Collège », donc aux chemins entre les routeurs R6 et R3. Le protocole **RIP** construit les tables de routage en indiquant, pour chaque routeur, la distance en nombre de sauts qui le sépare d'un autre routeur.

   (a) On donne ci-dessous les tables de routage des routeurs R1, R2, R3 et R4 obtenues par ce protocole. Donner les tables de routage des routeurs R5 et R6.

Table de R1

Dest.	passe par	Dist.
R2	R5	2
R3	R5	2
R4	R6	2
R5	R5	1
R6	R6	1

Table de R2

Dest.	passe par	Dist.
R1	R5	2
R3	R3	1
R4	R3	2
R5	R5	1
R6	R5	3

Table de R3

Dest.	passe par	Dist.
R1	R5	2
R2	R2	1
R4	R4	1
R5	R5	1
R6	R4	2

Table de R4

Dest.	passe par	Dist.
R1	R6	2
R2	R3	2
R3	R3	1
R5	R5	1
R6	R6	1

(b) On souhaite transmettre un paquet de données depuis le poste PC Admin 01 vers le poste PC CLG 01. Déterminer le parcours emprunté par ce paquet, en utilisant les tables de routage.

(c) À la suite d'une panne, le routeur R4 est déconnecté. Déterminer alors une nouvelle route pouvant être empruntée par les données du secteur « Administration » vers le Collège en effectuant le moins de sauts possible.

4. Le routeur R4 est réparé et reconnecté au réseau. On applique désormais le protocole **OSPF**, qui attribue un coût à chaque liaison afin de privilégier les routes plus rapides. Le coût d'une liaison est coût $= \dfrac{10^8}{d}$, où d est le débit en bit/s.

(a) Recopier et compléter le tableau suivant :

Liaison	**Débit** (bit/s)	**Coût**
Ethernet		10
Fast-Ethernet	10^8	
Fibre	10^9	

(b) Recopier le schéma ci-dessous et indiquer le coût de chacune des liaisons connues.

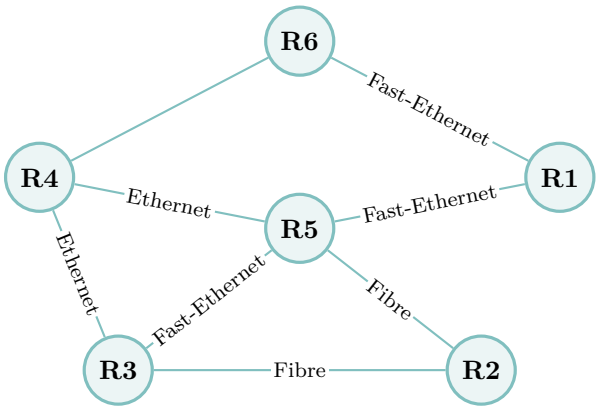


Figure 2 : types des liaisons entre les routeurs

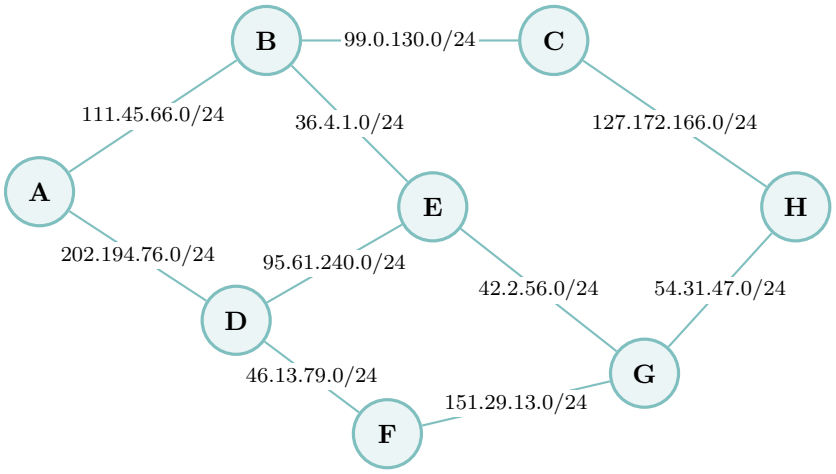
(c) On sait que le parcours R6 – R4 – R5 – R2 a un coût de 11,1. Déterminer le type de liaison entre R6 et R4.

(d) On souhaite acheminer un paquet de données du secteur « Administration » vers le réseau du Lycée général en utilisant le protocole OSPF. Déterminer la route empruntée par un paquet allant du poste PC Admin 01 vers le poste PC LG 01. ▷ coup de pouce

▷ début de solution

★★ **Exercice 12** *Des adresses, RIP puis OSPF* (d'après Centres étrangers 2023, jour 1) ▶ corrigé p. 931 ≈ 30 min

Voici un réseau dans lequel A, B, C, D, E, F, G et H sont des routeurs ; chaque liaison est étiquetée par l'adresse du réseau qui relie les deux routeurs.



Les adresses IPv4 sont composées de 4 octets, notées `X1.X2.X3.X4` en décimal. La notation CIDR `X1.X2.X3.X4/n` signifie que les n premiers bits de poids forts représentent la partie « réseau », les suivants la partie « hôte ». Tous les hôtes d'un même réseau local ont la même partie réseau ; l'adresse dont tous les bits de la partie hôte sont à 0 est l'*adresse du réseau*.

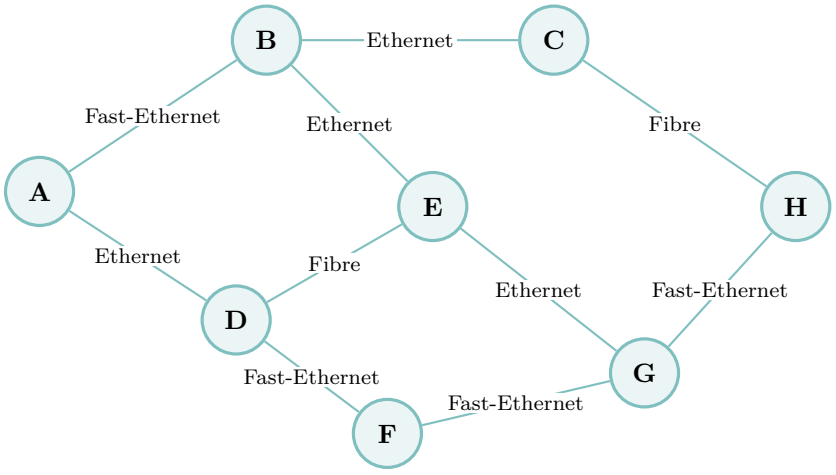
1. (a) `10100100.10110010.XXXXXXXX.XXXXXXXX` est la conversion en binaire de l'adresse `164.178.2.13`. Terminer cette conversion en remplaçant les deux octets `XXXXXXXX` par leur valeur binaire.

   (b) Donner, en justifiant, l'adresse du réseau auquel appartient la machine d'adresse `164.178.2.13/24`.

   Le protocole RIP cherche à minimiser le nombre de routeurs traversés (nombre de sauts).

2. Donner tous les chemins optimaux pour un paquet émis par A à destination de G en suivant le protocole RIP.

Voici maintenant le type de connexion entre les routeurs :



On travaille avec le protocole OSPF. Le coût d'une liaison est donné par coût $= 10^9/BP$, où BP est la bande passante en bit/s, estimée selon le type de connexion : Ethernet 10^8, Fast-Ethernet 10^9, Fibre 10^{10}.

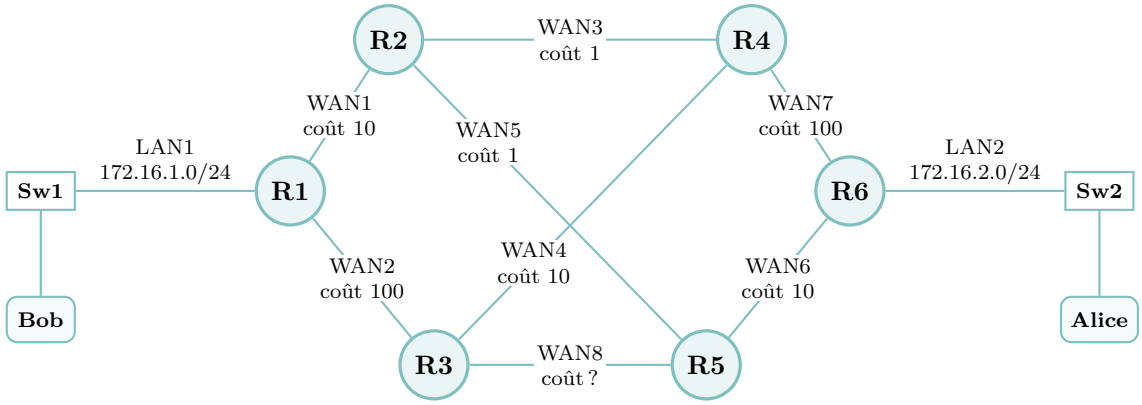
3. (a) Dessiner le schéma du réseau en remplaçant le type de connexion par le coût (on se limitera aux noms des routeurs et aux coûts).
   (b) Donner le chemin suivi par un paquet émis par A à destination de G en respectant le protocole OSPF.
   (c) Donner ce chemin si le routeur F est en panne.

▷ coup de pouce

★★ Exercice 13 *Bob écrit à Alice* (d'après Sujet zéro 2023, sujet A)

▶ corrigé p. 931 ≈ 30 min

Bob et Alice communiquent au travers du réseau ci-dessous, dont le protocole de routage est OSPF, qui minimise le coût des communications (LAN : réseau local ; WAN : réseau étendu ; R : routeur ; Sw : switch).



Une adresse IPv4 est composée de quatre octets, soit 32 bits. La notation `/n` signifie que les n premiers bits de l'adresse forment la partie « réseau » et les suivants la partie « machine ». L'adresse dont tous les bits machine sont à 0 est l'adresse du réseau, celle dont ils sont tous à 1 l'adresse de diffusion : ces deux adresses ne peuvent pas être attribuées à des machines. Le choix des routes est uniquement basé sur OSPF, avec un débit de référence de 10 000 Mbit/s :

$$\text{coût} = \frac{\text{débit de référence}}{\text{débit du réseau concerné}}.$$

1. La configuration IP partielle suivante a été affichée sur l'un des ordinateurs : IP hôte `172.16.2.3`, IP passerelle `172.16.2.253`. Indiquer, en justifiant, si cette configuration appartient à l'ordinateur de Bob ou à celui d'Alice.

2. Le réseau WAN8 a un débit de 1 000 Mbit/s. Calculer le coût correspondant.

3. On donne les tables de routage des routeurs R1 à R5 (« Pass. » désigne la passerelle, c'est-à-dire le routeur suivant ; « – » signale un réseau directement relié). Écrire la table de routage du routeur R6.

Routeur R1

Dest.	Pass.	Coût
LAN1	–	–
LAN2	R2	21
WAN1	–	–
WAN2	–	–
WAN3	R2	10
WAN4	R2	11
WAN5	R2	10
WAN6	R2	11
WAN7	R2	11
WAN8	R2	11

Routeur R2

Dest.	Pass.	Coût
LAN1	R1	10
LAN2	R5	11
WAN1	–	–
WAN2	R1	10
WAN3	–	–
WAN4	R4	1
WAN5	–	–
WAN6	R5	1
WAN7	R4	1
WAN8	R5	1

Routeur R3

Dest.	Pass.	Coût
LAN1	R4	21
LAN2	R5	20
WAN1	R4	11
WAN2	–	–
WAN3	R4	10
WAN4	–	–
WAN5	R5	10
WAN6	R5	10
WAN7	R4	10
WAN8	–	–

Routeur R4

Dest.	Pass.	Coût
LAN1	R2	11
LAN2	R2	12
WAN1	R2	1
WAN2	R3	10
WAN3	–	–
WAN4	–	–
WAN5	R2	1
WAN6	R2	2
WAN7	–	–
WAN8	R2	2

Routeur R5

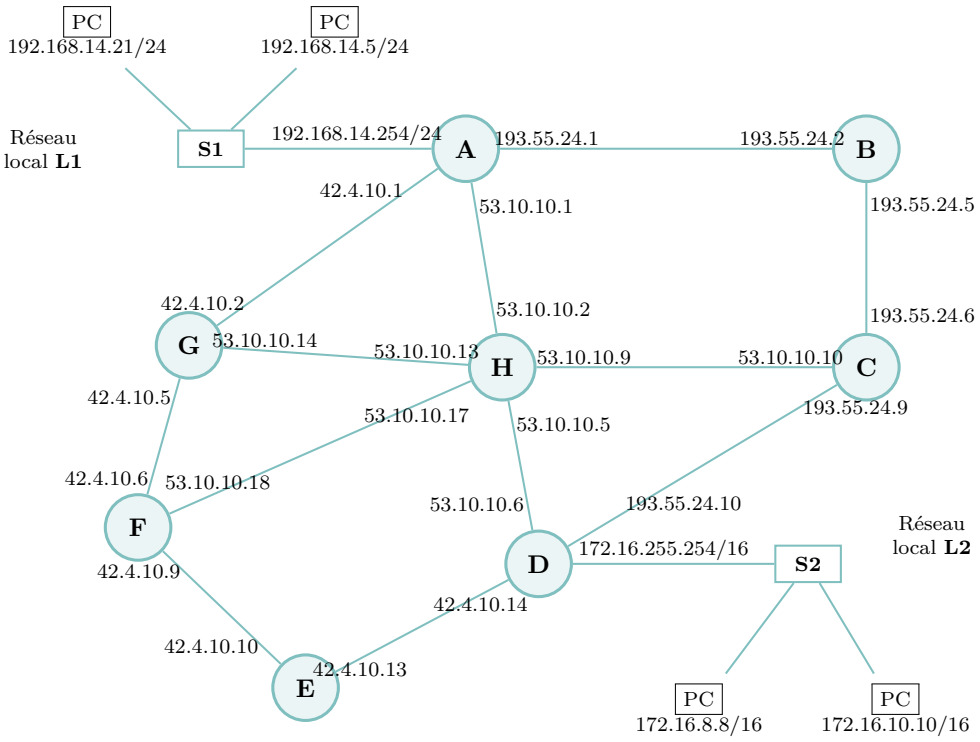
Dest.	Pass.	Coût
LAN1	R2	11
LAN2	R6	10
WAN1	R2	1
WAN2	R3	10
WAN3	R2	1
WAN4	R2	2
WAN5	–	–
WAN6	–	–
WAN7	R2	2
WAN8	–	–

4. Bob envoie un message à Alice. Énumérer dans l'ordre tous les routeurs par lesquels transitera ce message.

5. Un routeur tombe en panne : le nouveau coût de la route entre Bob et Alice est de 111. Déterminer le nom du routeur en panne. ▷ coup de pouce

★★★ **Exercice 14** *Du réseau local L1 au réseau local L2* (d'après Centres étrangers 2024, groupe 1, jour 1) ▶ corrigé p. 932 ≈ 45 min

Rappels. Une adresse IPv4 est composée de 4 octets, soit 32 bits, notée `a.b.c.d` (« notation décimale pointée »). La notation CIDR `a.b.c.d/n` signifie que les n premiers bits de l'adresse représentent la partie réseau, les bits suivants la partie machine. L'adresse dont tous les bits machine sont à 0 est l'*adresse du réseau* ; celle dont ils sont tous à 1 est l'*adresse de diffusion*. On considère le réseau suivant :



Partie A — Adresses IP

1. Les machines du réseau local L1 ont un masque de 24 bits en notation CIDR, soit `255.255.255.0` en notation décimale pointée. Donner le masque en notation décimale pointée des machines du réseau L2 (masque de 16 bits).

Concernant le réseau local L2 :

2. Donner l'adresse du réseau.

3. Donner l'adresse de diffusion.

4. Donner le nombre maximum de machines pouvant être connectées à ce réseau.

Partie B — Protocoles de routage. Voici des extraits des tables de routage des routeurs :

Routeur	Réseau destinataire	Passerelle	Interface
A	L2	53.10.10.2	53.10.10.1
B	L2	193.55.24.6	193.55.24.5
C	L2	193.55.24.10	193.55.24.9
D	L2	Connecté	172.16.255.254
E	L2	42.4.10.14	42.4.10.13
F	L2	42.4.10.10	42.4.10.9
G	L2	53.10.10.13	53.10.10.14
H	L2	53.10.10.6	53.10.10.5

5. À l'aide de ces extraits, donner un chemin (c'est-à-dire nommer les routeurs traversés) suivi par un message envoyé du réseau L1 vers le réseau L2.

La liaison entre les routeurs H et D est rompue.

6. Sachant que le protocole RIP est utilisé (distance en nombre de sauts), donner les nouveaux chemins que pourra suivre un message allant de L1 vers L2.

7. Choisir un des chemins de la question précédente. Donner les routeurs dont la règle de routage vers L2 est *obligatoirement* modifiée. Après avoir examiné tous les routeurs, écrire les règles de routage modifiées en conséquence.

La liaison entre H et D est rétablie. Pour tenir compte du débit des liaisons, on utilise désormais OSPF, avec coût $= 10^9/BP$, où BP est la bande passante en bit/s. Voici les bandes passantes des liaisons :

Liaison	Bande passante	Liaison	Bande passante	Liaison	Bande passante
A-B	1 Gbit/s	B-C	1 Gbit/s	D-E	10 Gbit/s
A-H	1 Gbit/s	C-H	100 Mbit/s	E-F	10 Gbit/s
A-G	1 Gbit/s	C-D	1 Gbit/s	F-H	1 Gbit/s
D-H	100 Mbit/s	G-H	1 Gbit/s	F-G	10 Gbit/s

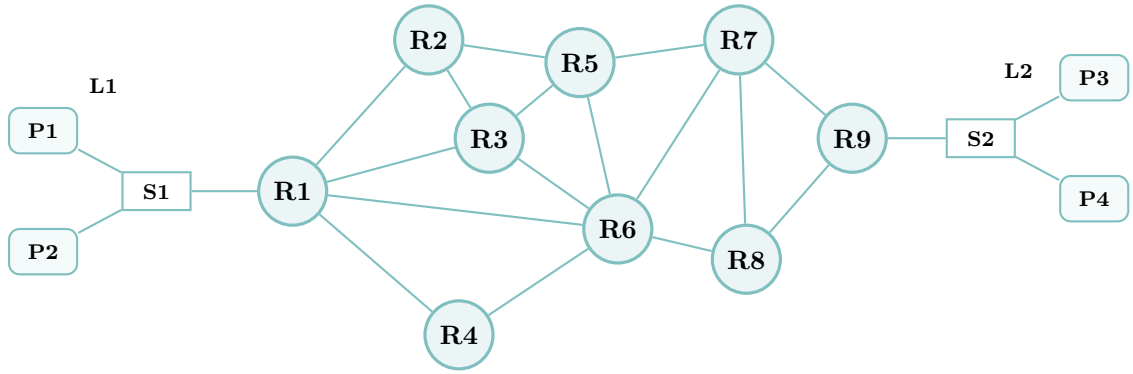
8. Calculer le coût des liaisons pour les trois valeurs de bande passante du tableau.

9. Déterminer alors le chemin suivi par un message allant de L1 vers L2, et donner son coût.

10. La liaison entre les routeurs G et F est rompue. Déterminer le nouveau chemin suivi par un message allant de L1 vers L2, et donner son coût. ▷ coup de pouce ▷ début de solution

★★ **Exercice 15** *Le réseau d'une société* (d'après Métropole septembre 2024, jour 1) ▶ corrigé p. 932 ≈ 30 min

Le réseau informatique d'une société est constitué de routeurs interconnectés par des fibres optiques. Il comporte deux réseaux locaux : L1, relié au routeur R1, et L2, relié au routeur R9.



Partie A. Les adresses IP sont notées `X1.X2.X3.X4`, et `X1.X2.X3.X4/n` signifie que les n premiers bits de poids forts représentent la partie réseau. Toutes les machines d'un réseau local ont la même partie réseau. Voici les adresses IPv4 du réseau de la société :

Nom	Adresses IPv4
R1	192.168.1.1/24, 192.168.2.1/24, 192.168.3.1/24, 192.168.4.1/24, 192.168.5.1/24
R2	192.168.2.2/24, 192.168.7.1/24, 192.168.8.1/24
R3	192.168.3.2/24, 192.168.7.2/24, 192.168.9.1/24, 192.168.10.1/24
R4	192.168.5.2/24, 192.168.6.1/24
R5	192.168.8.2/24, 192.168.9.2/24, 192.168.11.1/24, 192.168.12.1/24
R6	192.168.4.2/24, 192.168.6.2/24, 192.168.10.2/24, 192.168.11.2/24, 192.168.13.1/24, 192.168.14.1/24
R7	192.168.12.2/24, 192.168.13.2/24, 192.168.15.1/24, 192.168.16.1/24
R8	192.168.14.2/24, 192.168.15.2/24, 192.168.17.1/24
R9	192.168.16.2/24, 192.168.17.2/24, 192.168.18.1/24
P1	192.168.1.10
P2, P3, P4	non fournies

1. À partir des adresses des interfaces et de celle du portable P1, donner une adresse possible pour le portable P2.

2. Donner l'adresse du réseau local L2 ainsi que le nombre d'adresses possibles pour les portables P3 et P4.

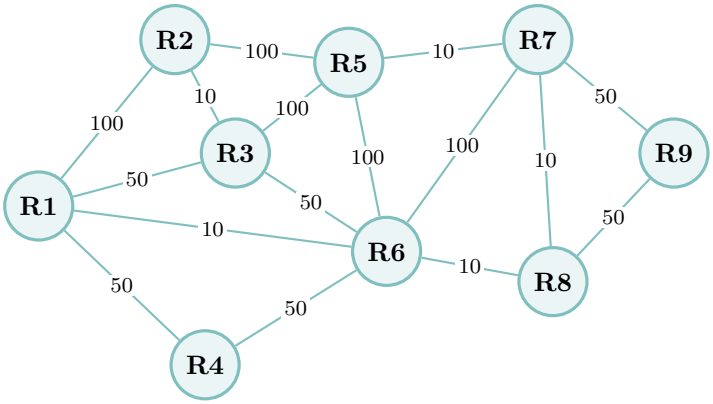
Partie B. On suppose que le protocole de routage RIP est utilisé.

3. Recopier et compléter, en ajoutant autant de lignes que nécessaire, la table de routage simplifiée du routeur R1.

Destination	Suivant	Nombre de sauts
R2	R2	1
R3		

4. L'ordinateur P1 envoie un paquet de données à l'ordinateur P3. Donner l'un des chemins empruntés par le paquet ainsi que le nombre de sauts.

Partie C. On utilise maintenant le protocole OSPF. Le poids de chaque arête ci-dessous est la bande passante de la liaison, en mégabits par seconde (Mbit/s), et le coût d'une liaison est $C = \dfrac{10^8}{BP}$, où BP est la bande passante en bits par seconde.



5. Déterminer la route empruntée par un paquet allant de l'ordinateur P1 à l'ordinateur P3. Préciser le coût de ce trajet. ▷ coup de pouce

Critiquer une réponse d'IA ↑ cours

★★ **Exercice 16** *RIP et OSPF expliqués par un assistant d'IA* ▶ corrigé p. 933

Un élève demande à un assistant d'IA : « Quelle est la différence entre RIP et OSPF, et comment calcule-t-on la métrique d'une route en OSPF ? » Voici la réponse obtenue :

Réponse de l'assistant

« RIP et OSPF sont deux protocoles de routage dynamique : ils remplissent automatiquement les tables de routage. RIP mesure une route par son nombre de sauts (le nombre de routeurs traversés) ; chaque routeur échange périodiquement sa table avec ses voisins (algorithme de Bellman-Ford), et une destination à plus de 15 sauts est considérée comme inaccessible. OSPF, lui, connaît la carte complète du réseau et applique l'algorithme de Dijkstra : la métrique d'une route est la somme des coûts des liaisons traversées. Le coût d'une liaison se calcule à partir de son débit par la formule coût = débit/10^8 (débit en bits par seconde) : une liaison à 100 Mbit/s coûte donc 1. Ainsi, OSPF tient compte de la vitesse des liaisons, contrairement à RIP qui traite de la même façon une fibre et une liaison lente. »

1. La réponse est-elle correcte ? La vérifier par un calcul : avec la formule de l'assistant, quel est le coût d'une liaison à 1 Mbit/s ? à 100 Mbit/s ? Laquelle OSPF préférerait-il alors ?
2. Localiser et corriger l'erreur, puis refaire le calcul.
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ? ▷ coup de pouce

S'entraîner à l'oral ↑cours

★★ **Exercice 17** *Expliquer en deux minutes* ▶ corrigé p. 933

Au Grand Oral comme devant l'examinateur de l'épreuve pratique, il faut savoir **expliquer** une notion clairement, sans lire ses notes. Choisir l'un des sujets suivants (ou le tirer au sort) :

Sujet 1. Expliquer comment un paquet trouve son chemin à travers plusieurs réseaux grâce aux **tables de routage**.

Sujet 2. Expliquer la différence entre les protocoles **RIP** et **OSPF**, avec un exemple où ils ne choisissent pas la même route.

Sujet 3. Expliquer à un camarade de Première pourquoi un **protocole de routage** est nécessaire, au lieu de remplir les tables à la main.

Préparation (5 min, seul). Noter au brouillon **trois idées** et **un exemple**, rien de plus : comme au Grand Oral, ces notes servent d'aide-mémoire. **Exposé (2 min, chronométré).** Debout, face à un camarade, **sans lire ses notes** et sans rien écrire : tout passe par la parole. **Retour (2 min).** Le camarade pose une question de relance (on peut alors s'aider d'un schéma), remplit la grille ci-dessous et donne un conseil ; puis on échange les rôles. Cette grille reprend les critères d'évaluation du Grand Oral. ▷ coup de pouce

Grille entre camarades	Oui	En partie	Pas encore
Clair — audible, posé ; chaque mot technique est expliqué	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	☐	☐	☐
Construit — un fil conducteur, tenu en deux minutes, sans lire	☐	☐	☐
Un conseil pour la prochaine fois :			

Entraînement ↑cours

Des séries courtes pour rendre une méthode automatique : même consigne, données différentes. Les réponses sont brèves dans le corrigé.

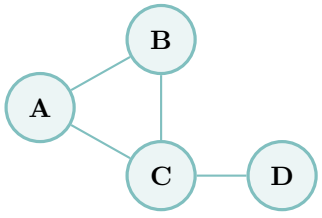
★ **Exercice 18** *Adresses de réseau en série* ▶ corrigé p. 934 • essentiel

Pour chacune des machines ci-dessous, donner l'**adresse du réseau**, l'**adresse de diffusion** et le **nombre maximal de machines** de ce réseau. ▷ coup de pouce

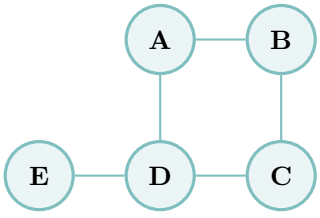
a) `192.168.42.17/24`
b) `172.20.3.200/16`
c) `10.4.7.19/8`
d) `192.168.43.250/24`
e) Les machines des questions a) et d) peuvent-elles communiquer sans passer par un routeur ?

★ **Exercice 19** *Tables RIP en série* ▶ corrigé p. 934 • essentiel

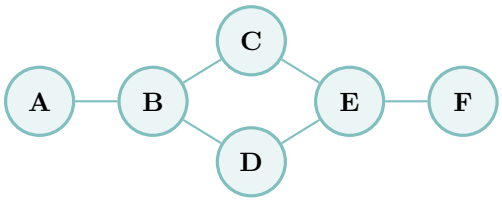
Dans chacun des réseaux ci-dessous, chaque routeur dessert son propre réseau local. Donner la **table de routage RIP** du routeur **A** (colonnes : destination, sortir vers, nombre de sauts) : pour chaque routeur de destination, le voisin vers lequel A envoie les paquets et le nombre de sauts. En cas d'égalité, donner les deux possibilités. ▷ coup de pouce



a)



b)



c)

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Relire dans le cours le rôle de chaque appareil : lequel travaille à l'intérieur d'un réseau local, lequel fait passer les paquets d'un réseau à un autre ? Pour la question 3 : sur combien de réseaux un routeur est-il branché ? ↑énoncé

Ex. 2 – piste Chaque nombre de l'adresse occupe un octet. Pour la conversion, décomposer chaque nombre en somme de puissances de 2 (128, 64, 32…, 1). Le `/24` indique combien de bits, en partant de la gauche, forment la partie réseau. ↑énoncé

Ex. 3 q. 3 – piste Avec un masque `/24`, comparer les trois premiers octets ; avec `/16`, les deux premiers. Pour le nombre de machines, compter les bits d'hôte et ne pas oublier les deux adresses réservées. ↑énoncé

Ex. 4 q. 3 – piste Pour chaque site, chercher la plus petite puissance de 2 qui, diminuée de 2, atteint le nombre de machines : $2^4 - 2 = 14$, $2^5 - 2 = 30$, $2^6 - 2 = 62$, $2^7 - 2 = 126$. Le nombre de bits d'hôte donne alors n. ↑énoncé

Ex. 5 q. 6 – piste Question 2 : en descendant, chaque couche ajoute son en-tête **à gauche** de ce qu'elle reçoit (schéma du cours). Questions 3 et 4 : un routeur travaille à la couche **Réseau** ; ce qui change d'un lien à l'autre, c'est le support physique. ↑énoncé

Ex. 6 – piste Une métrique de 0 correspond à un réseau atteint par une interface du routeur lui-même. Pour les autres lignes, la colonne « Sortir par » indique à qui le routeur confie le paquet. ↑énoncé

Ex. 7 q. 3 – piste Ne regarder que les liaisons, pas les débits : pour chaque destination, chercher le chemin qui traverse le moins de liaisons, puis noter le *premier* voisin de ce chemin. ↑

Ex. 8 – piste Convertir d'abord chaque débit en bit/s, puis diviser 10^8 par ce débit. Plus le coût est petit, plus la liaison est intéressante ; le coût d'un chemin est la somme des coûts de ses liaisons. ↑énoncé

Ex. 9 q. 3 – piste Écrire d'abord le coût de chaque liaison sur le schéma. Pour chaque destination, comparer les coûts totaux de plusieurs chemins possibles ; pour la question 3, regarder de près le lien A–B. ↑énoncé

Ex. 10 q. 2 – piste Question 2 : reprendre le principe rappelé dans le cours : à chaque tour, fixer le sommet non traité de plus petite distance provisoire, puis relâcher ses liaisons (`si distance[u] + coût(u,v) < distance[v]`…). ↑énoncé

Ex. 10 q. 3 – piste Question 3 : la colonne « sortir vers » est le **premier** routeur du chemin : remonter les prédécesseurs depuis la destination jusqu'à `A`. ↑énoncé

Ex. 11 q. 4d – piste Question 1 : convertir chaque octet en somme de puissances de 2. Question 3 : pour R5 et R6, compter les sauts sur la figure 1 en s'inspirant des tables données. Question 4 : le coût d'Ethernet donne son débit ; pour la liaison R6–R4, retrancher du coût total les coûts des liaisons déjà connues. ↑énoncé

Ex. 11 q. 4d – début de solution Question 4 (c) : écrire 11,1 = coût(R6–R4) + coût(R4–R5) + coût(R5–R2), lire les deux derniers coûts sur le schéma complété, puis retrouver le type de liaison dans le tableau. ↑énoncé

Ex. 12 q. 3c – piste Question 2 : lister les chemins de A à G et garder *tous* ceux qui ont le moins de sauts. Question 3 : calculer le coût de chaque type de connexion avec la formule de l'énoncé (attention : 10^9 ici), puis comparer les coûts totaux des chemins. ↑énoncé

Ex. 13 q. 5 – piste Question 1 : comparer l'adresse de l'hôte, et celle de sa passerelle (le routeur de son réseau local), aux adresses des réseaux LAN1 et LAN2 indiquées sur le schéma. Question 4 : partir du routeur de Bob et suivre, de table en table, la colonne « Pass. » vers le LAN d'Alice. Question 5 : pour chaque routeur du chemin, se demander quel serait le coût de la meilleure route sans lui. ↑énoncé

Ex. 14 q. 10 – piste Questions 2 à 4 : avec un masque `/16`, les deux derniers octets désignent la machine. Question 5 : partir du routeur relié à L1 et suivre les passerelles du tableau. Question 7 : un routeur doit changer sa règle si sa passerelle actuelle vers L2 n'est plus sur le nouveau chemin. Questions 9 et 10 : calculer le coût total de chaque chemin possible et garder le plus petit. ↑énoncé

Ex. 14 q. 10 – début de solution Question 8 : 1 Gbit/s $= 10^9$ bit/s donne un coût de $10^9/10^9 = 1$; faire de même avec 10^8 bit/s et 10^{10} bit/s. Question 9 : écrire ces coûts sur chaque liaison, puis comparer par exemple les chemins qui passent par H avec ceux qui passent par G puis F. ↑énoncé

Ex. 15 q. 5 – piste Question 1 : P2 est dans le même réseau que l'interface du routeur à laquelle il est relié. Question 3 : compter les liaisons sur le schéma, en notant le premier routeur du plus court chemin. Question 5 : calculer le coût de chaque liaison, puis comparer les coûts totaux des chemins possibles. ↑énoncé

Ex. 16 q. 3 – piste Appliquer la formule de l'assistant aux deux débits et se demander si le résultat va dans le bon sens : une liaison rapide doit-elle coûter plus ou moins cher qu'une liaison lente ? ↑énoncé

Ex. 17 – piste Structurer chaque explication en trois temps : une **définition** en une phrase, un **exemple** petit et concret déroulé à voix haute, puis le **piège** (l'erreur que l'auditeur risque de faire). Finir par une phrase qui répond à la question posée. Comparer le trajet d'un paquet à celui d'un voyageur qui ne lit que les panneaux du prochain carrefour. Sujet 2 : quelle grandeur chaque protocole cherche-t-il à rendre minimale ? ↑

Ex. 18 – piste Avec un masque `/24`, les trois premiers octets forment la partie réseau ; avec `/16`, les deux premiers ; avec `/8`, le premier. L'adresse du réseau met tous les octets de la partie machine à 0, l'adresse de diffusion les met tous à 255. Pour le nombre de machines, compter les bits de la partie machine, sans oublier les deux adresses réservées. ↑énoncé

Ex. 19 – piste Pour chaque destination, chercher le chemin qui traverse le moins de liaisons : le nombre de sauts est le nombre de liaisons de ce chemin, et la colonne « sortir vers » contient le **premier** routeur du chemin, qui est toujours un voisin de A. ↑énoncé

TP ET PROJETS

TP Construire, router et pinguer un réseau ▶ corrigé p. 934

Simulateur Filius (en français)

Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

But du TP

Utiliser **Filius** pour construire un réseau, configurer les adresses IP, remplir **à la main les tables de routage** des routeurs, puis faire circuler un paquet d'un bout à l'autre avec `ping` — et l'observer. On va du réseau local le plus simple jusqu'au **routage manuel** entre trois routeurs.

Prise en main. Filius a deux modes (boutons du haut) : le mode **construction** (marteau) pour poser les composants et les relier avec le **Câble**, et le mode **simulation** (flèche verte) pour lancer les échanges. Les composants (barre de gauche) sont l'**Ordinateur**, le **Switch**, le **Routeur** et le **Câble** :

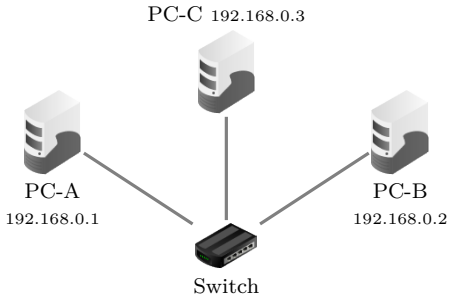


Un **double-clic** sur un composant ouvre sa configuration. **Pour taper des commandes sur une machine** : en mode simulation, double-cliquer dessus, ouvrir « **Installation des logiciels** » et installer la « **Ligne de commande** » ; on y tape ensuite `ping`, `route` (affiche la table de routage), `ipconfig`, `traceroute`. La fenêtre « **Afficher les échanges de données** » (clic droit sur une machine) montre les paquets couche par couche.

Partie 1 — Un réseau local

Exercice 1 *Construire et adresser*

▶ En mode **construction**, poser trois **Ordinateurs** reliés à un même **Switch**. Double-cliquer sur chacun pour saisir son **adresse IP** et le **masque** `255.255.255.0` ; laisser la passerelle vide.



1. Justifier, à partir des adresses et du masque, que les trois machines sont dans **le même réseau local**.
2. Combien de machines au maximum ce réseau `/24` peut-il contenir ?

Exercice 2 *Pinguer et observer*

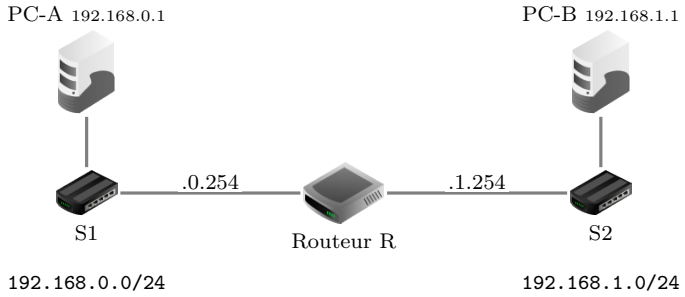
▶ Passer en **simulation**. Sur PC-A, installer la **Ligne de commande**, l'ouvrir et taper : `ping 192.168.0.2`.

1. Décrire le trajet du paquet (quels composants traverse-t-il ?). Le `ping` réussit-il ?
2. ▶ Taper `ping 192.168.0.9` (machine inexistante). Que se passe-t-il, et pourquoi ?

Partie 2 — Deux réseaux, un routeur, une passerelle

Exercice 3 *Relier deux réseaux*

▶ Dans un **nouveau projet** Filius, construire deux réseaux locaux reliés par **un Routeur**, comme sur le schéma (PC-B a ici l'adresse `192.168.1.1`). À sa création, Filius demande « **Sélectionnez le nombre d'interfaces du routeur** » : en mettre **2**. Configurer chaque interface via l'onglet « **Gérer les connexions** ».



Interface du routeur côté réseau 0 : `192.168.0.254/24` ; côté réseau 1 : `192.168.1.254/24`.

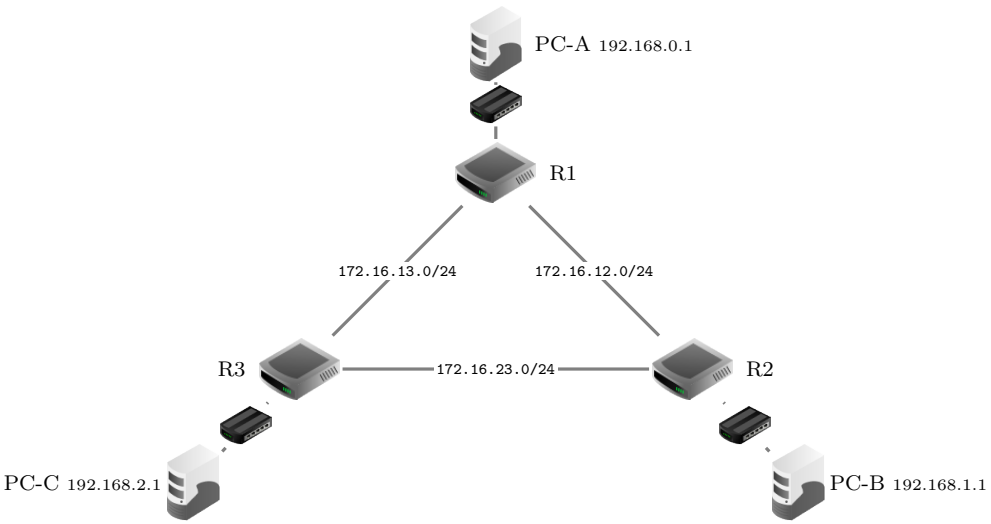
Exercice 4 *La passerelle est indispensable*

▶ En simulation, installer la Ligne de commande sur PC-A et taper `ping 192.168.1.1`.

1. Sans passerelle, Filius répond « `destination non joignable` ». Pourquoi ? *(Le paquet doit **quitter** le réseau local.)*
2. ▶ Régler la **Passerelle** de PC-A sur `192.168.0.254` (l'interface du routeur de *son* côté), celle de PC-B sur `192.168.1.254`. Relancer le `ping` : il doit réussir. Décrire le trajet complet.

Partie 3 — Le routage manuel entre trois routeurs

On relie **trois** réseaux locaux par **trois routeurs** en triangle. Chaque routeur doit **savoir vers qui envoyer** un paquet destiné à un réseau qui ne lui est pas directement rattaché.



Plan d'adressage (chaque routeur a **trois** interfaces). Réseaux locaux : A `192.168.0.0/24` (sur R1), B `192.168.1.0/24` (sur R2), C `192.168.2.0/24` (sur R3).

Routeur	vers son réseau local	vers un routeur	vers un routeur
R1	`192.168.0.254`	`172.16.12.1` (R2)	`172.16.13.1` (R3)
R2	`192.168.1.254`	`172.16.12.2` (R1)	`172.16.23.2` (R3)
R3	`192.168.2.254`	`172.16.13.3` (R1)	`172.16.23.3` (R2)

Passerelles : PC-A → `192.168.0.254`, PC-B → `192.168.1.254`, PC-C → `192.168.2.254`.

Exercice 5 *Construire et adresser*

▶ Réaliser ce réseau : 3 **Routeurs** (à 3 interfaces chacun), 3 **Switchs**, 3 **Ordinateurs**. Configurer toutes les interfaces (« Gérer les connexions ») et toutes les passerelles selon le plan.

Exercice 6 *Remplir les tables de routage à la main*

▶ Sur *chaque* routeur, ouvrir la configuration, aller sur « **Table de routage** », puis **décocher « Routage automatique »** et **décocher « Afficher toutes les lignes »** (pour ne voir que les entrées manuelles). Avec le bouton « **Nouvelle ligne** », ajouter les **deux** routes vers les réseaux qui ne sont pas directement rattachés au routeur. Les colonnes sont : **IP de destination**, **Masque**, **Passerelle suivante**, **Via l'interface**.

Table de routage de R1 (à recopier et compléter sur le cahier avant de la saisir dans Filius — elle doit atteindre les réseaux B et C) :

IP de destination	Masque	Passerelle suivante	Via l'interface
`192.168.1.0`	`255.255.255.0`		
`192.168.2.0`	`255.255.255.0`		

Tables de R2 et R3 : les construire de même (deux lignes chacune, pour les deux réseaux locaux manquants), sur le cahier, puis les saisir.

Exercice 7 *Pinguer de bout en bout, et vérifier*

▶ En simulation, installer la Ligne de commande sur PC-A, puis :

1. `ping 192.168.1.1` (PC-A vers PC-B). Décrire la suite des routeurs traversés. Ralentir la vitesse (curseur en haut, ≈ 50%) pour bien voir le paquet avancer.

2. Taper `route` sur PC-A, puis `traceroute 192.168.2.1` (vers PC-C). Que montrent ces commandes ?

3. ▶ **Expérience de reroutage.** Sur R1, **modifier** la route vers `192.168.1.0` pour qu'elle sorte non plus vers R2, mais vers **R3** (passerelle suivante `172.16.13.3`, via l'interface `172.16.13.1`). Relancer `ping 192.168.1.1` depuis PC-A : le paquet passe désormais par **R1 → R3 → R2**. Vérifier avec `traceroute`. Qu'a-t-on montré sur le rôle de la table de routage ?

Bilan du TP

Un **switch** suffit dans un réseau local ; changer de réseau exige un **routeur** et une **passerelle**. Avec plusieurs routeurs, chacun ne décide qu'**une étape** grâce à sa **table de routage** : en la remplissant (à la main ici, automatiquement par RIP ou OSPF en vrai) on trace la route des paquets — et en la modifiant, on la change, comme on vient de le voir.

Crédits : icônes issues du logiciel libre Filius (licence GNU GPL v2 ou v3), `www.lernsoftware-filius.de`.

FICHE DE RÉVISION

L'essentiel

- **Internet = un réseau de réseaux** : des réseaux locaux (LAN) reliés entre eux par des **routeurs**. Un **switch** relie les machines *d'un même* réseau ; un **routeur** relie *plusieurs* réseaux (il a une interface par réseau).
- **Adresse IP + masque** (`/n`) : les n premiers bits désignent le **réseau**, le reste la **machine**. Deux machines sont sur le même réseau si elles ont la même **adresse réseau**.
- Un message est découpé en **paquets** (commutation de paquets) : chacun voyage **indépendamment**, de routeur en routeur ; le réseau **survit aux pannes** (on contourne).
- **Modèle en couches TCP/IP** : Application / Transport / Réseau (IP) / Lien. À chaque couche, on **encapsule** (on ajoute un en-tête).
- Chaque routeur possède une **table de routage** : pour chaque destination, *par où sortir* et à quel **coût** (métrique). Elle peut être **statique** (à la main) ou **dynamique** (calculée par un protocole).
- **Deux protocoles de routage** :
  - **RIP** : métrique = **nombre de sauts** (routeurs traversés). Vecteur de distance (Bellman-Ford). Simple, mais **ignore la vitesse** des liens. Plafond 15 sauts.
  - **OSPF** : métrique = **somme des coûts**, avec coût = 10^8/débit. Choisit la route la plus **rapide**. En coulisses, chaque routeur applique l'algorithme de **Dijkstra** au graphe des routeurs [au-delà du programme].

Le réflexe : routage = graphes

- Les routeurs sont les **sommets**, les liaisons les **arêtes** (pondérées par la métrique).
- **RIP** construit un plus court chemin *en nombre d'arêtes* ; **OSPF** un plus court chemin *pondéré* (par l'algorithme de Dijkstra, [au-delà du programme]). D'où le lien direct avec le chapitre **Graphes**.
- **Le piège du lien lent** : une route en *un* saut par une liaison lente peut être pire qu'une route en *trois* sauts par la fibre ⇒ RIP et OSPF ne choisissent pas toujours la même route.

Erreurs à éviter

- confondre **switch** (un réseau) et **routeur** (plusieurs réseaux) ;
- confondre les métriques : RIP compte des **sauts** (entiers), OSPF additionne des **coûts** (10^8/débit) ;
- croire que « le moins de sauts » = « le plus rapide » (faux : c'est le piège du lien lent) ;
- inverser le sens du coût OSPF : plus le débit est **grand**, plus le coût est **petit** ;
- appliquer 10^8/débit sans lire l'énoncé : le 10^8 n'est qu'une **convention** (on rencontre aussi 10^9, 10^{10}, des coûts arrondis…) : suivez la formule donnée.

Savoir-faire à maîtriser — je sais…

☐ **distinguer** machine, switch et routeur, et reconnaître un réseau local à son adresse ;

☐ **expliquer** l'encapsulation en couches et le rôle de la couche IP ;

☐ **lire** et **utiliser** une table de routage donnée, et la **compléter** sur un petit réseau (réseau, sortie, métrique) ;

☐ **identifier la route RIP** (nombre de sauts) sur un réseau donné ;

- ☐ **calculer les coûts** (10^8/débit), **identifier la route OSPF** (coût total minimal), puis **comparer** à RIP ;
- ☐ [au-delà du programme] **dérouler Dijkstra** pour retrouver une table OSPF.

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 936

1. Les machines `10.0.3.7/24` et `10.0.4.7/24` sont-elles sur le même réseau ?
2. Combien de machines au plus sur un réseau `/24` ?
3. Coût OSPF d'une liaison à 10 Mbit/s ?
4. Vrai ou faux : la route qui a le moins de sauts est toujours la plus rapide.
5. Quel protocole tient compte du débit des liaisons, RIP ou OSPF ?

Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 936

1. Arêtes `A-B`, `B-C`, `A-D`, `D-C` : ordre d'un parcours en largeur depuis `A`, voisins pris par ordre alphabétique ? (ch. 8)
2. Écrire la requête qui passe à 8 la note du film d'identifiant 5 dans la table `film`. (ch. 6)
3. Les paquets en attente dans un routeur sont traités dans l'ordre d'arrivée : quelle structure linéaire ? (ch. 3)

DÉFI ADVENT OF CODE

DÉFI Lobby Layout

le carrelage hexagonal — ensembles, coordonnées et automate

Jour 24

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t09-defi-aoc-2020-24` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/24`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Un sol est pavé de carreaux hexagonaux, blancs d'un côté et noirs de l'autre. Le fichier indique des centaines de carreaux à retourner, chacun désigné par un chemin à suivre depuis le centre. On veut compter les carreaux noirs, puis faire évoluer le motif pendant 100 jours. Le pavage est infini et les chemins sont longs : il faut un programme, et surtout une bonne façon de repérer une case hexagonale.
Ce qu'on réinvestit. La notion de **voisins dans un graphe**, vue au chapitre 8 : chaque carreau du pavage a exactement six voisins. Un bon système de **coordonnées** transforme chaque direction en un déplacement à additionner, et un **ensemble** ne garde que les carreaux noirs d'un pavage infini. La partie 2 fait évoluer tous les carreaux en même temps, comme les sièges du défi *Seating System* (chapitre 4).
La démarche de la fiche.

1. Comprendre comment se lisent les chemins et quand un carreau finit noir, puis suivre

quelques chemins sur un dessin.
2. Découper une ligne en directions, puis choisir des coordonnées pour calculer la case d'arrivée.
3. Retourner les carreaux dans un ensemble de carreaux noirs : sa taille est la réponse de la partie 1.
4. Faire évoluer le pavage jour après jour en comptant les voisins noirs (partie 2).

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`hexagonal tile`	carreau hexagonal	`hex grid`	grille hexagonale
`flip`	retourner	`white / black side`	face blanche / noire
`reference tile`	carreau de référence	`step`	pas, déplacement
`neighbor`	voisin	`delimiter`	séparateur
`east / west`	est / ouest	`northeast, southwest...`	nord-est, sud-ouest...

Résumé de l'énoncé (partie 1)

Contexte. Enfin arrivé à l'hôtel, vous trouvez le hall en travaux : il faut poser un carrelage hexagonal formant un motif précis. Chaque carreau est blanc d'un côté et noir de l'autre. Les ouvriers ont une liste des carreaux à retourner, mais ils ont besoin de vous pour savoir à quoi ressemblera le motif.
Les données. Le sol est un pavage infini de carreaux hexagonaux, disposés en rangées horizontales : chaque carreau a six voisins, dans les directions `e` (est), `se` (sud-est), `sw` (sud-ouest), `w` (ouest), `nw` (nord-ouest) et `ne` (nord-est). Chaque ligne du fichier est un chemin : une suite de ces directions collées, sans séparateur, comme `seswnee`.
La règle.

- Au départ, tous les carreaux sont blancs.
- Pour chaque ligne, on part toujours du même carreau de référence et l'on suit les directions une à une, de voisin en voisin.
- Le carreau d'arrivée est retourné : blanc devient noir, noir devient blanc. Les carreaux traversés en chemin ne changent pas.
- Plusieurs lignes peuvent désigner le même carreau par des chemins différents : il est alors retourné plusieurs fois.

La réponse attendue. Le nombre de carreaux noirs une fois toutes les lignes traitées. Par exemple, avec les trois lignes `ee`, `w` et `ee`, le carreau désigné par `ee` est retourné deux fois et redevient blanc ; seul celui désigné par `w` est noir : la réponse serait 1.

Exercice 1 *Vérifier sa compréhension*

But. Comprendre comment se lisent les directions et ce que deviennent les carreaux désignés plusieurs fois.
À quoi ça sert ? La première question prépare le découpage des lignes ; la deuxième montre qu'il faut repérer les carreaux par une position, et non par le chemin qui y mène.

1. Pourquoi `ne` ne peut-il pas être lu comme `n` puis `e` ?
2. Deux lignes différentes peuvent-elles désigner le même carreau ? Un carreau désigné trois fois finit-il noir ou blanc ?
3. Le carreau de référence peut-il être retourné ?

Exercice 2 *À la main, sur un petit exemple*

But. Découper quelques chemins et les suivre sur un dessin de grille hexagonale.
À quoi ça sert ? On découvre que des chemins différents peuvent arriver au même carreau, et l'on obtient une réponse connue (`6`) pour tester le programme.

On considère le fichier (inventé) suivant :

```
eee
nwwsw
seswnee
wwnw
enwsw
swsenw
nenenw
ewnwse
```

Découper chaque ligne en directions (par exemple `seswnee` donne `se`, `sw`, `ne`, `e`). Sur un dessin de grille hexagonale, repérer les carreaux désignés par `enwsw` et `ewnwse`. Vérification : à la fin, `6` carreaux sont noirs.

2 Programmer la partie 1

Exercice 3 *Découper une ligne*

But. Transformer une ligne comme `seswnee` en liste de directions.
À quoi ça sert ? Les directions sont collées et n'ont pas toutes la même longueur : une fois la liste obtenue, suivre le chemin devient une simple boucle.

Écrire une fonction `directions(ligne)` qui renvoie la liste des directions d'une ligne. Tester sur `seswnee`.

▷ coup de pouce

Exercice 4 *Des coordonnées*

But. Associer à chaque direction un déplacement, puis calculer la position du carreau désigné par une ligne.
À quoi ça sert ? C'est le point clé du défi : avec de bonnes coordonnées, deux chemins qui mènent au même carreau donnent la même position, et l'on peut alors repérer chaque carreau sans dessiner la grille.

Avant de coder, choisir un système de coordonnées (lire l'encadré « Trois systèmes de coordonnées possibles », section « Pour y arriver ») et écrire, pour chacune des six directions, le déplacement correspondant. Vérifier sur un dessin que `nw` puis `se`, ou `e`, `nw` puis `sw`, ramènent au point de départ. Écrire alors une fonction `position(ligne)` qui renvoie les coordonnées du carreau désigné.

▷ coup de pouce

Méthode : retourner un carreau avec un ensemble (`set`)

Les ensembles sont présentés dans l'encadré « Outil Python : les ensembles » du défi *Crab Combat* (chapitre 4) ; rappel : éléments non modifiables, donc une position `(x, y)` convient. On ne garde que les carreaux **noirs** : retourner un carreau, c'est l'ajouter à l'ensemble s'il n'y est pas, l'en retirer sinon.

```
1  def retourner(noirs, p):
2      """Retourne le carreau p : blanc -> noir, noir -> blanc."""
3      if p in noirs:
4          noirs.remove(p)
5      else:
6          noirs.add(p)
7
8  noirs = set()               # ensemble vide (attention : {} est un dict)
9  noirs.add((3, 0))
10 print((3, 0) in noirs)      # True
11 retourner(noirs, (1, 1))    # (1, 1) absent : il est ajoute
12 retourner(noirs, (3, 0))    # (3, 0) present : il est retire
```

```
print(noirs)                # {(1, 1)}
```

Intérêt. Le test `p in noirs` se fait en temps constant ; avec une liste, chaque test `p in liste` parcourrait toute la liste. Question : que contient `noirs` si l'on appelle trois fois `retourner(noirs, (0, 0))` à partir d'un ensemble vide ?

Exercice 5 *Retourner les carreaux*

But. Tenir à jour l'ensemble des carreaux noirs en traitant toutes les lignes.
À quoi ça sert ? C'est la réponse de la partie 1 (la taille de l'ensemble) ; cet ensemble est aussi l'état de départ de la partie 2.

Gérer l'ensemble `noirs` des coordonnées des carreaux noirs et répondre à la partie 1.

▷ coup de pouce

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Le motif n'est pas figé : il évolue de jour en jour selon une règle simple. On part du carrelage obtenu en partie 1.
La nouvelle règle.

- Chaque jour, on compte les voisins noirs de chaque carreau dans l'état de la veille, puis tous les carreaux changent **en même temps**.
- Un carreau noir ayant 0 ou plus de 2 voisins noirs devient blanc.
- Un carreau blanc ayant exactement 2 voisins noirs devient noir.
- Les autres carreaux ne changent pas.

La réponse attendue. Le nombre de carreaux noirs après 100 jours. Par exemple, si seuls deux carreaux voisins étaient noirs, ils le resteraient (un voisin noir chacun) et les deux carreaux qui les touchent tous les deux deviendraient noirs : 4 carreaux noirs le lendemain.

Méthode : compter avec un dictionnaire

Pour compter les apparitions de chaque valeur, on associe à chaque valeur rencontrée son nombre d'apparitions. À la première rencontre, la clé n'existe pas encore : on la crée avec la valeur `1`.

```
compte = {}
for case in [(0, 1), (2, 0), (0, 1)]:
    if case in compte:
        compte[case] += 1
    else:
        compte[case] = 1
print(compte)               # {(0, 1): 2, (2, 0): 1}
print((5, 5) in compte)     # False : case jamais rencontree
```

Intérêt. Une seule lecture de la liste suffit, et le test `case in compte` se fait en temps constant. Question : que se passerait-il sans le test, avec seulement `compte[case] = compte[case] + 1` ?

★★ **Exercice 6** *Un automate cellulaire*

But. Calculer l'ensemble des carreaux noirs du lendemain, puis répéter cent fois.
À quoi ça sert ? C'est la réponse de la partie 2. Comme le pavage est infini, toute la difficulté est de n'examiner que les carreaux susceptibles de changer, et de compter leurs voisins noirs efficacement.

Écrire une fonction `jour_suivant(noirs)` qui renvoie le nouvel ensemble des carreaux noirs. Sur l'exemple inventé de la fiche, on doit trouver `9` carreaux noirs après un jour, `10` après deux jours et `53` après dix jours. ▷ coup de pouce ▷ début de solution

4 Pour y arriver : repérer des cases hexagonales

Trois systèmes de coordonnées possibles

Dans une grille hexagonale dont les lignes sont horizontales (comme ici), chaque case a deux voisins sur sa ligne et quatre sur les lignes voisines. Plusieurs repérages sont possibles :

- **Coordonnées « doublées ».** On numérote les lignes y ; sur une ligne, les colonnes x avancent de 2 en 2, et les lignes voisines sont décalées d'un cran. Une case a donc des coordonnées (x, y) avec $x + y$ pair.
- **Coordonnées axiales.** Deux axes obliques à 60° : chaque déplacement modifie une ou deux coordonnées de ± 1.
- **Coordonnées cubiques.** Trois coordonnées (x, y, z) liées par $x + y + z = 0$: chaque déplacement augmente l'une de 1 et en diminue une autre de 1. C'est symétrique et facile à vérifier, au prix d'une coordonnée en trop.

Dans tous les cas, deux chemins qui mènent à la même case doivent donner exactement les mêmes coordonnées : c'est ce qui permet d'utiliser un ensemble.

5 Approfondissement

Méthode : additionner deux positions coordonnée par coordonnée

Une position à 2 ou à 3 coordonnées se range dans un tuple. Pour lui ajouter un déplacement, on parcourt les indices des coordonnées avec une **compréhension de liste**, puis on reconvertit le résultat en tuple pour pouvoir le ranger dans un ensemble.

```
pos = (1, -2, 1)
pas = (0, 1, -1)
somme = [pos[k] + pas[k] for k in range(len(pos))]
print(somme)                          # [1, -1, 0]
nouvelle = tuple(somme)               # (1, -1, 0) : rangeable dans un ensemble
```

Intérêt. Grâce à `len(pos)`, le même code additionne des positions à 2 ou à 3 coordonnées. Question : que se passe-t-il si `pos` vaut `(1, 2, 3)` et `pas` vaut `(10, 20)` ?

★★ **Exercice 7** *Approfondissement 1 : d'autres systèmes de coordonnées*

But. Résoudre le défi avec d'autres repérages des cases hexagonales, et les comparer.
À quoi ça sert ? Le choix d'une représentation conditionne la simplicité du code : on voit qu'une même fonction peut servir pour plusieurs systèmes, et que les coordonnées cubiques donnent en plus une formule de distance.

1. **Coordonnées cubiques.** On repère une case par (x, y, z) avec $x + y + z = 0$; on choisit `e` $= (1, -1, 0)$ et `ne` $= (1, 0, -1)$. Trouver les quatre autres déplacements, sachant que `w` est l'opposé de `e` et que faire `e` puis `nw` revient à faire `ne`. Calculer à la main la position de `seswnee`.

2. Écrire une fonction `position_systeme(ligne, deplacements)` qui fonctionne avec n'importe quel dictionnaire de déplacements (2 ou 3 coordonnées). Vérifier qu'on trouve encore `6` carreaux noirs sur l'exemple inventé de la fiche. ▷ coup de pouce

3. **Coordonnées doublées.** Avec `e` $= (2, 0)$ et `ne` $= (1, -1)$, trouver les autres déplacements. Montrer que $x + y$ est toujours pair. Retrouver le résultat de la partie 1.

4. **Distance.** En coordonnées cubiques, on admet que le nombre minimal de pas pour aller du centre à la case (x, y, z) est $\max(|x|, |y|, |z|)$. Vérifier sur `eee` et `seswnee`. Quel est, sur l'exemple, le carreau noir le plus éloigné du centre ? ▷ coup de pouce

5. Comparer les trois systèmes : nombre de coordonnées, facilité de vérification, calcul des voisins pour la partie 2.

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Parcourir la chaîne avec un indice `i` dans une boucle `while` : si le caractère est `n` ou `s`, la direction occupe deux caractères, sinon un seul. ↑ énoncé

Ex. 4 – piste Un dictionnaire `DEPLACEMENTS` qui associe à chaque direction un couple `(dx, dy)` rend la fonction très courte. ↑ énoncé

Ex. 5 – piste Retourner un carreau, c'est l'ajouter à l'ensemble s'il n'y est pas, l'en retirer sinon : c'est la fonction `retourner` de l'encadré. ↑ énoncé

Ex. 6 – piste La grille est infinie : on ne la stocke pas. Seuls les carreaux noirs et leurs voisins peuvent changer ; ce sont donc les seuls à examiner. ↑ énoncé

Ex. 6 – début de solution Construire un dictionnaire `nb_voisins_noirs` : pour chaque carreau noir, ajouter 1 à chacun de ses six voisins. Le nouvel ensemble se construit ensuite à partir de ce dictionnaire et de l'ancien ensemble ; attention aux carreaux noirs qui n'ont aucun voisin noir (ils n'apparaissent pas dans le dictionnaire). ↑ énoncé

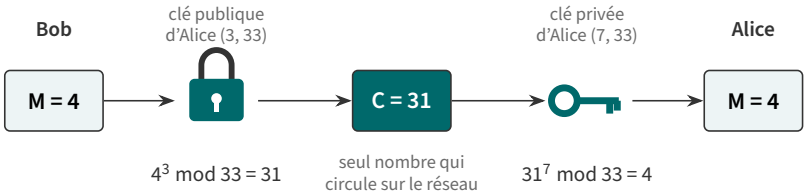
Ex. 7 q. 2 – piste Partir de la liste `[0] * len(deplacements["e"])` et, à chaque pas, additionner coordonnée par coordonnée (voir l'encadré). Renvoyer un tuple, pour pouvoir le ranger dans un ensemble. ↑ énoncé

Ex. 7 q. 4 – piste Un pas modifie deux coordonnées d'une unité chacune, en sens contraires : il ne peut faire baisser $\max(|x|, |y|, |z|)$ que d'au plus 1. ↑ énoncé

10

CHAPITRE 10

Cryptographie



DANS CE CHAPITRE

- Activités préparatoires 481
- Cours 483
- Exercices 495
- TP et projets 508
- Fiche de révision 514
- Défis Advent of Code 516
- Corrigés 937

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Casser le code de César

▶ corrigé p. 937

Durée : 15 à 20 min · Par deux, sans ordinateur ni calculatrice

Consignes

- Vous êtes l'**attaquant** : vous ne connaissez pas la clé, mais vous voulez lire le message.
- Répondre sur le cahier. Travail **sans IA** : ni assistant, ni complétion de code ; vous devez pouvoir expliquer chaque réponse.

Le message intercepté

Les espions du roi ont intercepté ce message. On sait seulement que chaque lettre a été **décalée** d'un même nombre k de rangs dans l'alphabet (après `Z`, on repart à `A`) ; les espaces et la ponctuation n'ont pas été modifiés.

```
FY XPDDLRP DPNCPE L PEP TYEPCNPAEP ALC WPD PDATZYD OF CZT. APCDZYYP YP NZYYLTE WL
NWP, XLTD NSLNFY DLTE BFP WPD WPEECPD ZYE PEP OPNLWPPD. OLYD FYP WLYRFP, NPCELTYPD
WPEECPD CPGTPYYPYE MTPY AWFD DZFGPYE BFP WPD LFECPD : PY QCLYNLTD, WL WPEECP P PDE
OP WZTY WL AWFD QCPBFPYEP. WP OPNLWLRP YP NSLYRP ALD NPD QCPBFPYNPD, TW WPD OPAWLNP
DPFWPXPYE. TW DFQQTE OZYN OP NZXAEPC WPD WPEECPD OF EPIEP NSTQQCP AZFC CPECZFGPC WL
NWP PE WTCP NP XPDDLRP DLYD W'LGZTC ULXLTD CPNF.
```

Réglette (rang de chaque lettre) :

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

Première idée : tout essayer

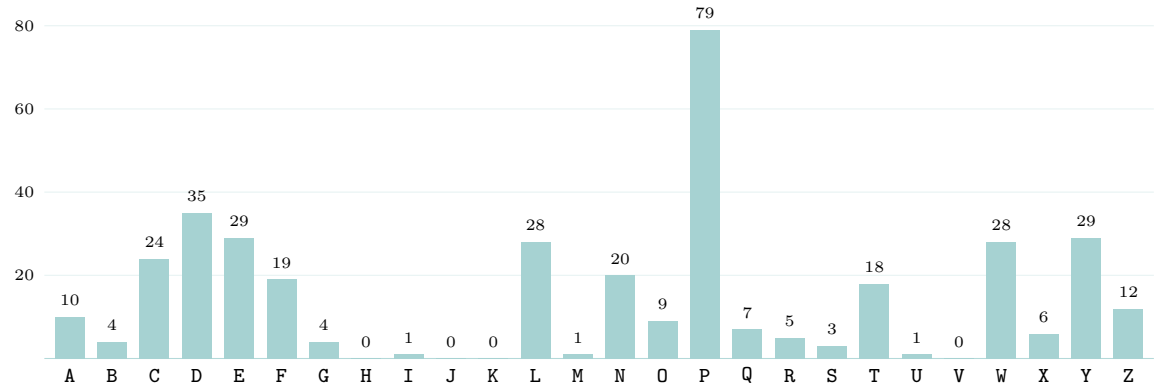
1. Combien de valeurs de k différentes faut-il envisager ? Essayer $k = 1$ puis $k = 2$ sur le premier mot `FY` : obtient-on un mot français ?

Essayer toutes les clés à la main sur le message entier serait long. Cherchons une méthode plus maligne.

Deuxième idée : compter les lettres

2. Dans la **première phrase** seulement (jusqu'au premier point), compter combien de fois apparaissent les lettres `P`, `D` et `L`.

Pour gagner du temps, voici le décompte de **toutes** les lettres du message (372 lettres en tout) :



Et voici, pour comparaison, la fréquence des lettres les plus courantes dans un long texte en français (valeurs approchées) :

Lettre	E	S	A	I	T	N	R	U	L	O
Fréquence (%)	14,7	7,9	7,6	7,5	7,2	7,1	6,6	6,3	5,5	5,4

Toutes les autres lettres sont en dessous de 4 %.

3. Quelle lettre est de loin la plus fréquente dans le message chiffré ? Et dans un texte français ? Que peut-on en conclure ?

4. En déduire la clé k (le décalage entre les deux lettres, à lire sur la réglette).

5. Vérifier votre hypothèse avec la deuxième lettre la plus fréquente du message : que devient-elle quand on la décale de k rangs *en arrière* ? Est-ce cohérent avec le tableau du français ?

6. Déchiffrer la première phrase du message.

7. *Pour les plus rapides* : déchiffrer la suite. Que dit le message ?

Ce qu'on a découvert

8. Vous avez lu le message **sans connaître la clé** au départ. Parmi les verbes *chiffrer*, *déchiffrer* et *décrypter*, lequel décrit ce que vous venez de faire ? Lequel décrit ce que fera le destinataire, qui connaît k ? Lequel décrit ce qu'avait fait l'expéditeur ?

9. Le décalage change les lettres, mais que conserve-t-il du message ? Que devrait, au contraire, faire un bon chiffrement ?

COURS

Programme (BO) *« Sécurisation des communications. Décrire les principes de chiffrement symétrique (clef partagée) et asymétrique (avec clef privée/clef publique). Décrire l'échange d'une clef symétrique en utilisant un protocole asymétrique pour sécuriser une communication HTTPS. »*

Prérequis la **division euclidienne** et le **reste** (opérateur `%`), la notion de **coût** d'un algorithme (croissance exponentielle), les **réseaux** et le fait qu'un paquet peut être **intercepté**. → fiches de révision : ch. 9 p. 472 → rappels de 1re : bases de Python p. 1056, Web et réseau p. 1064

Objectifs *chiffrer* et *déchiffrer* avec un code symétrique ; *comprendre* l'idée du chiffrement asymétrique et *dérouler RSA* à la main ; *expliquer* l'échange de clé et le protocole **HTTPS**.

Situation de départ

1. Votre mot de passe traverse une dizaine de machines inconnues avant d'arriver au site. Comment reste-t-il secret, alors que n'importe qui peut intercepter les paquets ?
2. Deux personnes qui ne se sont jamais rencontrées peuvent-elles convenir d'un code secret en ne se parlant qu'en public ?
3. Que garantit exactement le petit cadenas affiché par votre navigateur ?

I Le problème : se parler en secret au milieu de tous

Vous tapez un mot de passe, vous payez en ligne, vous envoyez un message : ces données traversent une dizaine de machines que vous ne connaissez pas, sur un réseau où **n'importe quel paquet peut être intercepté** (chapitre précédent). Et pourtant, presque personne ne peut les lire. Comment garder un secret quand le message passe *sous les yeux de tous* ?

C'est la question de la **cryptographie**, l'art de rendre un message illisible pour tout autre que son destinataire. Une science aussi vieille que l'écriture, qui a fait basculer des guerres (Enigma), qui protège aujourd'hui plus de 90 % du trafic d'Internet, et qui repose sur une poignée d'idées d'une élégance stupéfiante.

Définition 1 — Le vocabulaire du secret

Chiffrer, c'est transformer un message **clair** en un message **chiffré** (illisible) à l'aide d'une **clé**. **Déchiffrer**, c'est faire l'opération inverse quand on possède la clé. **Décrypter** (ou « casser »), c'est retrouver le message clair *sans* posséder la clé : c'est le travail de l'**attaquant** (la **cryptanalyse**).

↻ **Retour sur l'activité.** Dans l'activité, vous avez **décrypté** le message intercepté, sans connaître la clé ; son destinataire, qui connaît k, l'aurait simplement **déchiffré**.

Remarque

On dit *chiffrer* / *déchiffrer*, jamais « crypter » (qui n'a pas de sens : on ne peut pas rendre secret sans clé). En revanche *décrypter* existe : c'est réussir à lire sans la clé.

Règle 1 — Le fil rouge : la clé, et rien que la clé

Tout au long du chapitre, une seule question : **qui possède la clé ?** Nous verrons deux grandes familles. Dans le chiffrement **symétrique**, une *même* clé secrète chiffre et déchiffre. Dans le chiffrement **asymétrique**, il y a *deux* clés jumelles : une publique (connue de tous) et une privée (jamais partagée). Comprendre la cryptographie moderne, c'est comprendre pourquoi il a fallu inventer la seconde pour résoudre le talon d'Achille de la première : **comment échanger la clé ?**

Propriété 1 — Principe de Kerckhoffs (1883)

La sécurité d'un système ne doit reposer **que sur le secret de la clé**, jamais sur le secret de l'algorithme. L'algorithme peut (et doit) être **public** : c'est en le publiant qu'on le fait éprouver par le monde entier. *Un secret partagé par des milliers de personnes n'est plus un secret ; une clé, si.*

II Le chiffrement symétrique : une seule clé

Définition 2 — Chiffrement symétrique

Un chiffrement est **symétrique** lorsque **la même clé** sert à chiffrer et à déchiffrer. Alice et Bob doivent donc **partager un secret commun** (la clé) avant de communiquer.



Ève écoute la ligne : elle voit `U8#z!`, mais sans K elle ne lit rien.

A. Le chiffre de César : le décalage

Le plus vieux code connu : Jules César décalait chaque lettre de **3 rangs** dans l'alphabet. Avec une clé k, la lettre de rang x (A = 0, B = 1, …, Z = 25) devient la lettre de rang `(x + k) % 26` (le reste de la division par 26 fait « boucler » l'alphabet : après Z, on repart à A).

Exemple

Avec $k = 3$: `BONJOUR` devient `ERQMRXU`. Pour déchiffrer, on décale de 3 *en arrière* (soit `(x - 3) % 26`).

```
1  def cesar(message, k):
2      resultat = ""                                  # INITIALISER
3      for lettre in message:                         # PARCOURIR
4          rang = ord(lettre) - ord("A")              # A->0, B->1, ...
5          nouveau = (rang + k) % 26                  # decalage circulaire
6          resultat = resultat + chr(nouveau + ord("A"))  # METTRE A JOUR
7      return resultat                                # RENVOYER
8
9  # chiffrer avec k, dechiffrer avec -k (ou 26-k)
```

▷ **Je prédis.** Que renvoient `cesar("XYZ", 3)` et `cesar("ABC", -3)` ?
▷ **Je vérifie.** `"ABC"` : après `Z`, le `% 26` fait repartir à `A` ; puis `"XYZ"` : en Python, le reste est toujours positif (`-3 % 26` vaut 23), le décalage négatif marche donc sans précaution.

Propriété 2 — Un code très faible

La clé de César est un nombre entre 0 et 25 : il n'y a que **26 clés possibles**. Un attaquant les essaie *toutes* en une fraction de seconde (**attaque par force brute**). Pire : même sans force brute, une **analyse des fréquences** suffit. En français, la lettre la plus fréquente est le **E** ; la lettre la plus fréquente du texte chiffré est donc, très probablement, un E décalé — ce qui révèle la clé.

↻ **Retour sur l'activité.** C'est exactement l'attaque menée dans l'activité : `P`, de loin la lettre la plus fréquente du message, ne pouvait être qu'un `E` décalé, d'où $k = 11$, et la deuxième, `D`, redonne bien un `S`. Le décalage déplace les fréquences sans les changer : un bon chiffrement doit les brouiller, ce que commence à faire Vigenère (ci-dessous).

B. Le chiffrement affine : un peu d'arithmétique

On généralise César : la lettre de rang x devient `(a * x + b) % 26`. La clé est le couple (a,b).

Règle 2 — La condition sur a

Pour que le déchiffrement soit possible, il faut que a soit **premier avec** 26 (leur PGCD vaut 1). Sinon, deux lettres différentes pourraient se chiffrer pareil : impossible de revenir en arrière. Les valeurs possibles de a sont donc : $1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25$.

Pour déchiffrer, on a besoin de l'**inverse modulaire** de a : le nombre a^{-1}, entre 0 et 25, tel que `(a * a_inv) % 26` vaut 1. Par exemple, l'inverse de 5 est 21 (car $5 \times 21 = 105 = 4 \times 26 + 1$). En Python : `pow(5, -1, 26)` renvoie `21`. On retiendra ce mot *inverse modulaire* : il reviendra, central, dans RSA.

C. Vigenère : le « chiffre indéchiffrable »

Faiblesse commune de César et affine : une lettre donnée est *toujours* chiffrée de la même façon. Blaise de Vigenère (XVIe siècle) a l'idée d'utiliser une **clé-mot** répétée : chaque lettre du message est décalée selon la lettre correspondante de la clé.

Exemple

Message `NSIROCKS`, clé `CLE` répétée en `CLECLECL`. On additionne les rangs modulo 26 ($N + C$, $S + L$, …) : on obtient `PDMTZGMD`. Avec la même clé, `ANANAS` devient `CYEPLW` : la même lettre du clair (les trois `A`) donne cette fois *trois* lettres chiffrées différentes (`C`, `E`, `L`).

Réputé incassable pendant trois siècles, il fut finalement percé par **Charles Babbage** (vers 1854) puis **Friedrich Kasiski** (1863) : la clé étant *répétée*, des motifs réapparaissent périodiquement et trahissent sa longueur.

D. Le masque jetable : le seul code inviolable

Poussons l'idée de Vigenère à l'extrême : et si la clé était **aussi longue que le message**, **totalement aléatoire**, et **utilisée une seule fois** ? On obtient le **masque jetable** (*one-time pad*). En informatique, on ne décale plus les lettres : on combine chaque caractère du message avec la clé par un **OU exclusif** (XOR, noté `^`), l'opération booléenne vue en Première, appliquée ici bit à bit.

Propriété 3 — Le XOR est son propre inverse

Le XOR a une propriété magique : $(m \oplus k) \oplus k = m$. La *même* opération, avec la *même* clé, chiffre *et* déchiffre. Une seule fonction suffit.

```
def chiffre(message, masque):
    resultat = ""                                # INITIALISER
    for i in range(len(message)):                # PARCOURIR
        code = ord(message[i]) ^ ord(masque[i])  # XOR caractere par caractere
        resultat = resultat + chr(code)          # METTRE A JOUR
    return resultat                              # RENVOYER
# chiffre(chiffre(msg, masque), masque) == msg   (le XOR s'annule)
```

▷ **Je prédis.** Que valent `12 ^ 10`, puis `(12 ^ 10) ^ 10` ?
▷ **Je vérifie.** `6` : bit à bit, `1100` XOR `1010` donne `0110` (et non $12 - 10 = 2$) ; puis `12` : refaire le XOR avec la même clé annule le premier.

Règle 3 — Pourquoi c'est *prouvé* inviolable

Supposons qu'Ève intercepte le chiffré de `LUNDI`. Comme le masque est aléatoire et aussi long que le message, *tout* clair de 5 lettres est possible : `MARDI`, `JEUDI`, `STYLO`… Pour chacun de ces clairs, il existe exactement un masque qui donne le chiffré intercepté, et rien ne distingue le vrai des autres. Ève n'apprend **rien**. C'est la **sécurité parfaite** (ou inconditionnelle : aucune puissance de calcul n'y change rien), démontrée par **Claude Shannon** (1949).

Remarque
Le masque jetable est inviolable… en théorie. En pratique, il est peu commode : il faut une clé *aussi longue que tout ce qu'on veut s'envoyer*, vraiment aléatoire, et jamais réutilisée (réutiliser un masque casse tout). Le « téléphone rouge » Moscou–Washington l'utilisait pendant la Guerre froide.

E. AES : le standard d'aujourd'hui

Les codes modernes ne travaillent plus lettre par lettre mais **bit par bit, par blocs**. Le plus utilisé est l'**AES** (*Advanced Encryption Standard*, 2001) :

- il chiffre des **blocs de 128 bits** rangés dans une matrice 4×4 d'octets ;
- une clé de **128, 192 ou 256 bits** pilote une série de brouillages (substitutions, décalages, mélanges) ;
- pour l'AES-256, une attaque par force brute demanderait 2^{256} essais — un nombre à **78 chiffres**, hors de portée de toute machine ;
- aucune attaque efficace n'est connue à ce jour. C'est un chiffrement symétrique… et il est **excellent**.

Règle 4 — Le talon d'Achille du symétrique

Le chiffrement symétrique est **rapide** et **robuste** : parfait pour de gros volumes. Mais il a un défaut fatal : Alice et Bob doivent **déjà partager la clé secrète**. Comment l'échanger sur un réseau où tout est intercepté, *sans s'être jamais rencontrés* ? C'est le **problème de l'échange de clé** — et pendant des siècles, on a cru qu'il était insoluble.

► **Exercices d'application :** exercices 1 à 5 (César, affine, Vigenère, masque jetable)

III Le chiffrement asymétrique : deux clés jumelles

En **1976**, **Whitfield Diffie** et **Martin Hellman** (Stanford) publient une idée qui retourne 2000 ans de cryptographie : et si les deux clés étaient **différentes** ?

Un peu d'histoire. Passionné de cryptographie, à une époque où ce domaine était presque entièrement réservé aux militaires, **Whitfield Diffie** avait sillonné les États-Unis au début des années 1970 pour rencontrer les rares chercheurs qui s'y intéressaient ; c'est ainsi qu'il rejoint Martin Hellman à Stanford. Leur article de 1976, *New Directions in Cryptography*, fonde la cryptographie à clé publique et leur vaut le prix Turing en 2015.



Whitfield Diffie

Définition 3 — Chiffrement asymétrique (à clé publique)

Chaque personne possède une **paire** de clés jumelles :

- une **clé publique**, qu'elle distribue à qui veut (comme un cadenas ouvert) ;
- une **clé privée**, qu'elle garde secrète et ne partage *jamais*.

Un message chiffré avec la clé publique ne peut être déchiffré que par la clé privée associée — et, pour les systèmes comme RSA (§ IV), *réciproquement*. Surtout : **connaître la clé publique ne permet pas de retrouver la clé privée**.



Bob veut écrire à Alice ? Il prend le cadenas public d'Alice (que tout le monde peut copier), ferme la boîte avec, et l'envoie. Seule Alice, qui a la clé privée, peut l'ouvrir — même Bob ne le pourrait plus !

A. Deux clés aux rôles interchangeables

Appelons les deux clés d'Alice A (privée) et B (publique). Elles vérifient deux propriétés remarquables :

- on ne peut pas déduire l'une de l'autre ;
- ce qui est chiffré avec l'une se déchiffre avec l'*autre*, **dans les deux sens** (c'est le cas de RSA, étudié ci-dessous ; ce n'est pas vrai de tous les systèmes asymétriques).

Cette symétrie ouvre *deux* usages selon la clé avec laquelle on *commence* :

On chiffre avec…	**On déchiffre avec…**	**But**
la clé **publique** du destinataire	sa clé **privée**	**confidentialité** (seul lui lira)
sa **propre** clé privée	la clé **publique** de l'expéditeur	**authentification** (signature)

B. L'échange de clé de Diffie-Hellman

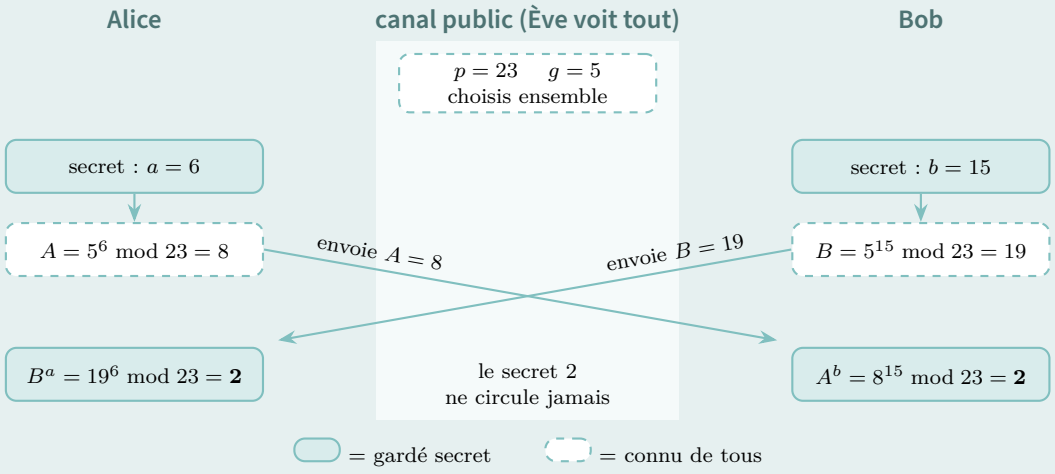
Avant même RSA, Diffie et Hellman résolvent le fameux problème de l'échange de clé : permettre à Alice et Bob de fabriquer une **clé secrète commune** *en public*, sans se l'être jamais transmise.

Règle 5 — L'image des pots de peinture

Alice et Bob conviennent *publiquement* d'une couleur commune. Chacun y ajoute *en secret* sa propre couleur, et s'échange le mélange (impossible à « dé-mélanger »). Enfin, chacun rajoute sa couleur secrète au mélange reçu : ils obtiennent la **même** couleur finale, qu'Ève, elle, ne peut pas reconstituer. La « peinture » mathématique est l'**exponentiation modulaire**.

Exemple

Publics : $p = 23$, $g = 5$. Alice choisit en secret $a = 6$ et publie $A = 5^6 \bmod 23 = 8$ (on note $x \bmod n$ le reste de la division de x par n, `x % n` en Python). Bob choisit $b = 15$ et publie $B = 5^{15} \bmod 23 = 19$. Chacun calcule alors le secret commun : Alice fait $B^a = 19^6 \bmod 23 = 2$; Bob fait $A^b = 8^{15} \bmod 23 = 2$. **Même résultat :** 2, sans jamais l'avoir transmis ! Ève connaît p,g,A,B mais ne peut pas retrouver a ni b (c'est le *problème du logarithme discret*, très difficile).



C. Authentification : la signature numérique

Le chiffrement asymétrique ne fait pas que cacher : il **prouve l'identité**. Si Alice chiffre un message avec sa clé **privée** (que personne d'autre ne possède), alors *n'importe qui* peut le déchiffrer avec sa clé **publique** — et si ça marche, c'est la **preuve** que le message vient bien d'Alice. C'est le principe de la **signature numérique**, qui résout le problème de l'**authentification** : être sûr que l'interlocuteur est bien celui qu'il prétend être.

► **Exercices d'application :** exercice 6 (symétrique ou asymétrique ?)

IV RSA : le chiffrement asymétrique, en vrai

Diffie et Hellman avaient l'*idée* du chiffrement à clé publique, mais pas de recette concrète. Elle vient un an plus tard.

A. Une histoire à rebondissement

Pour les curieux — *La double invention* (facultatif)

En **1977** au MIT, **Ron Rivest**, **Adi Shamir** et **Leonard Adleman** cherchent à prouver qu'une telle recette est *impossible*… et démontrent l'inverse : ils inventent **RSA** (leurs initiales). Coup de théâtre : on apprendra en **1997** que le Britannique **Clifford Cocks**, au service secret GCHQ, avait trouvé exactement le même système **quatre ans plus tôt** (1973), mais classé « Secret Défense ». Il fut le véritable inventeur… que le monde ignora près de vingt-cinq ans.

B. Rappel : l'arithmétique modulaire

Faire des calculs **modulo** n, c'est ne garder que le **reste** de la division par n. On écrit $15 \equiv 1\ [7]$ (lire « 15 est congru à 1 modulo 7 ») car $15 = 2 \times 7 + 1$. RSA repose entièrement là-dessus.

C. Étape par étape : la fabrication des clés

Règle 6 — Générer une paire de clés RSA

1. Choisir deux **grands nombres premiers** p et q (dans la réalité : environ 300 chiffres chacun, pour un n de 2 048 bits, soit 617 chiffres).
2. Calculer $n = p \times q$. Ce nombre n sera public.
3. Calculer $\varphi = (p-1)(q-1)$, puis choisir e **premier avec** φ.
   La **clé publique** est le couple (e, n).
4. Calculer d, l'**inverse modulaire** de e : le nombre tel que $e \times d \equiv 1\ [\varphi]$.
   La **clé privée** est le couple (d, n).

Exemple — *Fabrication complète, avec de petits nombres*

Prenons $p = 3$ et $q = 11$.

- $n = 3 \times 11 = 33$.
- $\varphi = (3-1)(11-1) = 2 \times 10 = 20$. On choisit $e = 3$ (premier avec 20).
- **Clé publique :** $(3, 33)$, diffusée à tous.
- On cherche d tel que $3d \equiv 1\ [20]$. Comme $3 \times 7 = 21 = 20 + 1$, on a $d = 7$. En Python : `pow(3, -1, 20)` renvoie 7.
- **Clé privée :** $(7, 33)$, gardée secrète.

D. Chiffrer et déchiffrer

Règle 7 — Les deux opérations RSA

Pour un message-nombre M (avec $0 \leq M < n$) :

$$\text{chiffrer :} \quad C = M^e \bmod n \qquad\qquad \text{déchiffrer :} \quad M = C^d \bmod n$$

Exemple — *Bob écrit à Alice*
Bob veut envoyer le nombre $M = 4$. Il a la clé publique d'Alice (3,33) :

$$C = 4^3 \bmod 33 = 64 \bmod 33 = 31.$$

Il transmet **31**. Alice reçoit 31 et applique sa clé privée (7,33) :

$$M = 31^7 \bmod 33 = 27\,512\,614\,111 \bmod 33 = 4.$$

Elle retrouve 4, le message de Bob. Ève, qui a vu passer 31 et connaît (3,33), ne peut pas résoudre $x^3 \equiv 31\ [33]$ efficacement.

Le principe est toujours le même, quels que soient p, q et e : d'autres jeux de clés sont fabriqués et vérifiés en Python dans le TP proposé en fin de chapitre.

E. Pourquoi c'est sûr, et pourquoi ça marche

Règle 8 — La sécurité repose sur la factorisation

La clé publique donne n. Pour retrouver la clé privée d, il faudrait connaître $\varphi = (p-1)(q-1)$, donc **retrouver** p **et** q à partir de n : **factoriser** n. Or multiplier deux grands premiers est *facile*, mais pour l'opération inverse (factoriser), on ne connaît **aucun algorithme efficace** : les meilleures méthodes connues demandent un temps qui explose avec la taille de n. Avec un n de plusieurs centaines de chiffres, aucune machine actuelle n'y parvient. Le plus grand n factorisé (2020) avait 250 chiffres.

Démonstration : ([au-delà du programme] pourquoi le déchiffrement retombe sur M)
Comme $e \times d \equiv 1\ [\varphi]$, le **petit théorème de Fermat** (et sa généralisation d'Euler) permet de montrer que $M^{ed} \equiv M\ [n]$. Or déchiffrer, c'est calculer $C^d = (M^e)^d = M^{ed} \equiv M\ [n]$: on retombe exactement sur le message. Les rôles de e et d étant symétriques ($M^{ed} = M^{de}$), on peut aussi bien chiffrer avec la privée et déchiffrer avec la publique — c'est ce qui autorise la signature. □

F. RSA en Python

Python fait tous ces calculs grâce à la fonction `pow` « à trois arguments » : `pow(M, e, n)` calcule M^e mod n par **exponentiation modulaire rapide**, sans jamais construire le nombre géant M^e ; et `pow(e, -1, phi)` calcule l'inverse modulaire d. Sur l'exemple ci-dessus, `pow(3, -1, 20)` renvoie `7`, `pow(4, 3, 33)` renvoie `31` (chiffrement) et `pow(31, 7, 33)` renvoie `4` (déchiffrement).

Règle 9 — En quel langage écrit-on la *vraie* cryptographie ?

Les programmes Python de ce chapitre **simulent** l'algorithme pour le comprendre. La cryptographie réelle (celle de votre navigateur) est écrite en **C** et en **assembleur**, pour la vitesse et pour maîtriser chaque détail : une simple variation du *temps* de calcul peut fuiter la clé (attaque par *canal auxiliaire*). On ne « bricole » jamais sa propre crypto en production : on utilise des bibliothèques éprouvées.

G. RSA est-il éternel ? (l'ombre quantique)

Pour les curieux (facultatif)

Deux évènements pourraient faire tomber RSA : (1) la découverte d'un algorithme *classique* rapide de factorisation (régulièrement annoncée, régulièrement démentie) ; (2) surtout, l'**ordinateur quantique**. L'**algorithme de Shor** (1994) factorise en temps *polynomial* sur une telle machine : le jour où un ordinateur quantique assez grand existera, RSA s'effondrera. C'est pourquoi les chercheurs préparent déjà la **cryptographie post-quantique**.

► **Exercices d'application :** exercices 7 à 10 (RSA, inverse modulaire, Diffie-Hellman)

V HTTPS : le mariage des deux mondes

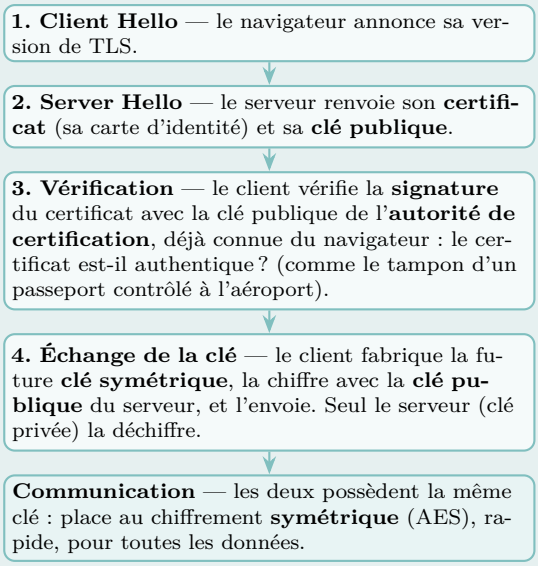
Aujourd'hui, le petit cadenas de votre navigateur signifie **HTTPS** : le trafic est chiffré. Mais avec quoi ? Ni l'un ni l'autre seul :

- le **symétrique** (AES) est rapide mais suppose une clé déjà partagée ;
- l'**asymétrique** (RSA) résout l'échange de clé mais est **lent** (gros calculs).

Règle 10 — L'idée de HTTPS : le meilleur des deux

On utilise l'**asymétrique** *juste au début*, pour échanger en sécurité une **clé de session symétrique**. Puis, tout le reste de la communication (les vraies données, en volume) est chiffré avec cette clé **symétrique**, bien plus rapide. HTTPS = **TLS** (l'échange de clé, asymétrique) + HTTP chiffré (les données, symétrique).

A. La poignée de main TLS (*handshake*)



À retenir — le rôle du certificat

Le **certificat** et l'**autorité de certification** apportent l'**authentification** : sans eux, un attaquant pourrait se placer au milieu (*man in the middle*) en présentant *sa* clé publique à la place de celle du serveur. Le certificat, signé par une autorité de confiance, garantit que la clé publique est bien celle du bon serveur.

► **Exercices d'application :** exercice 11 (le cadenas HTTPS du navigateur)

VI Ouverture

La cryptographie avance avec ses attaquants : chaque progrès du calcul oblige à revoir les tailles de clés et les algorithmes.

- **Une course sans fin.** Les clés grandissent avec la puissance de calcul : un n de 512 bits a été factorisé en 1999, et l'on recommande aujourd'hui au moins 2 048 bits pour RSA ; contre l'ordinateur quantique, des algorithmes dits **post-quantiques** sont déjà normalisés [au-delà du programme].
- Au **bac**, on vous demandera surtout de **programmer** ou **dérouler** un chiffrement simple (César, Vigenère, `XOR`), de distinguer chiffrement **symétrique** et **asymétrique**, et d'expliquer le rôle de chacun dans **HTTPS**.

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise :** les *Réseaux* : un paquet traverse des routeurs inconnus ; HTTPS chiffre le message avant qu'il ne descende les couches TCP/IP.
- **Une question de coût :** la sécurité de RSA parie que factoriser reste difficile ; vous relierez cela à « P versus NP » en *Calculabilité et décidabilité*.
- **Pour aller plus loin :** vous retrouverez le module AES intégré aux puces en *Systèmes sur puce et informatique embarquée*.

Bilan — la carte mémoire

Notion	À retenir
Chiffrer / déchiffrer / décrypter	avec clé / avec clé / **sans** clé (l'attaquant)
Kerckhoffs	la sécurité repose sur la **clé**, pas sur l'algorithme (public)
Symétrique	**une** clé partagée ; rapide ; ex. César, Vigenère, masque, **AES**
Masque jetable	clé aléatoire aussi longue, jetable ⇒ **inviolable** (Shannon)
Défaut du symétrique	**échanger la clé** sur un canal non sûr
Asymétrique	**2** clés jumelles : **publique** (chiffre) / **privée** (déchiffre)
Diffie-Hellman	fabriquer une clé commune **en public** (exponentiation mod.)
Authentification	chiffrer avec sa clé **privée** = **signer**
RSA	$n = pq$; $\varphi = (p-1)(q-1)$; (e,n) publique, (d,n) privée
RSA (formules)	$C = M^e \bmod n$; $M = C^d \bmod n$; `d = pow(e,-1,phi)`
Sécurité de RSA	difficulté de **factoriser** n ; menace : **Shor** (quantique)
HTTPS	asymétrique pour **échanger la clé** + symétrique pour **les données**

Retour à la situation de départ

1. Il est **chiffré** : seul le détenteur de la **clé** peut le déchiffrer, et l'attaquant qui intercepte le chiffré ne lit rien — à condition que la clé résiste à la force brute et que le code ne trahisse pas les fréquences des lettres (§ I et II).

2. Oui : l'échange de **Diffie-Hellman** fabrique une clé commune en public. Plus généralement, le chiffrement **asymétrique** chiffre avec la clé **publique** du destinataire, et seule sa clé **privée** déchiffre, comme dans **RSA** (§ III et IV).
3. Il signale **HTTPS** : le **certificat** authentifie le serveur, l'asymétrique sert à échanger une clé de session, puis toutes les données sont chiffrées en **symétrique** (AES), bien plus rapide (§ V).

Erreurs fréquentes

- **Dire « crypter ».** On dit *chiffrer* / *déchiffrer* (avec clé) et *décrypter* (sans clé, l'attaquant).
- **Croire que César ou Vigenère sont sûrs.** 26 clés pour César ; Vigenère tombe par l'analyse des motifs répétés.
- **Confondre les deux clés dans le bon sens.** Pour la *confidentialité*, on chiffre avec la clé **publique** du *destinataire*. Pour *signer*, on chiffre avec sa *propre* clé **privée**.
- **Oublier la condition « e premier avec φ »** (et « a premier avec 26 » pour l'affine) : sans elle, pas d'inverse modulaire, donc pas de déchiffrement.
- **Calculer φ avec n.** C'est $\varphi = (p-1)(q-1)$, pas $n-1$.
- **Penser que HTTPS n'utilise que RSA.** RSA sert *seulement* à échanger la clé ; les données sont chiffrées en **symétrique** (AES), car c'est rapide.

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

☐ **chiffrer** et **déchiffrer** par décalage (César), par le masque jetable / XOR → ex. 1, 2, 5 ;
☐ **expliquer** la différence symétrique / asymétrique et le **problème de l'échange de clé** → ex. 6, 10 ;
☐ **dérouler RSA** : calculer n, φ, choisir e, trouver d, chiffrer et déchiffrer un nombre → ex. 7, 8, 9 ;
☐ **expliquer** l'authentification (signer avec la clé privée) et le rôle d'un certificat → ex. 6, 11 ;
☐ **décrire** le fonctionnement de **HTTPS** (asymétrique pour la clé, symétrique pour les données) → ex. 11, 14.

Vers le Grand Oral

- **Comment garder un secret quand tout le monde peut lire le message ?** *(chiffrement, clé, Kerckhoffs ; du César au masque jetable inviolable.)*
- **Comment deux inconnus peuvent-ils fabriquer un secret commun en public ?** *(le coup de génie de Diffie-Hellman ; l'échange de clé.)*
- **Pourquoi peut-on publier une clé sans compromettre le secret ?** *(clé publique / privée ; RSA ; la difficulté de factoriser.)*
- **Que signifie vraiment le petit cadenas de mon navigateur ?** *(HTTPS = TLS + AES ; certificat et autorité de certification.)*
- **L'ordinateur quantique va-t-il casser Internet ?** *(algorithme de Shor ; factorisation ; cryptographie post-quantique.)*

Un peu d'histoire

Pour les curieux (facultatif)

La cryptographie a changé le cours de l'Histoire. Pendant la Seconde Guerre mondiale, l'armée allemande chiffrait ses ordres avec la machine **Enigma**, réputée inviolable : ses rotors, qui tournent à chaque lettre frappée, changent la substitution à chaque caractère, avec environ 10^{20} réglages possibles. À **Bletchley Park**, une équipe menée par **Alan Turing** construisit une machine électromécanique, la « Bombe », pour la casser : les historiens estiment que cet exploit a raccourci la guerre de *deux ans*. Turing, père de l'informatique théorique, fut aussi l'un des plus grands cryptanalystes de l'Histoire. Dès **1883**, **Auguste Kerckhoffs** avait énoncé le principe qui gouverne encore toute la discipline : ne comptez jamais sur le secret de la méthode, seulement sur celui de la clé.



Une machine Enigma à trois rotors

EXERCICES

Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

Rappel des rangs : A=0, B=1, …, Z=25. *On travaille toujours en majuscules, sans accents.* Niveau : ★ échauffement ★★ classique ★★★ défi (dont les extraits de bac les plus exigeants) ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.

Chiffrements symétriques ↑cours

★ Exercice 1 *Le chiffre de César, à la main* ▶ corrigé p. 938 • essentiel

1. Chiffrer le mot `NSI` avec la clé $k = 4$.
2. Le mot `KHOOR` a été chiffré avec la clé $k = 3$. Le déchiffrer.
3. Combien y a-t-il de clés possibles pour le chiffre de César ? En déduire pourquoi ce code est très facile à casser par un ordinateur.
4. On intercepte le mot chiffré `YLUKLGCVBZ`, obtenu avec une clé de César inconnue. Expliquer la stratégie de l'**attaque par force brute** pour retrouver le message clair (on ne demande pas de tout dérouler).

▷ coup de pouce

★ Exercice 2 *Le chiffre de César, en Python* ▶ corrigé p. 938

Recopier et compléter la fonction `cesar` qui chiffre `message` avec la clé `k`.

```
1  def cesar(message, k):
2      resultat = ""
3      for lettre in message:
4          rang = ord(lettre) - ord("A")
5          nouveau = ...                  # (a) decaler de k, modulo 26
6          resultat = resultat + ...      # (b) reconvertir en lettre
7      return resultat
```

1. Compléter les lignes `(a)` et `(b)`.
2. Comment appeler `cesar` pour *déchiffrer* un message chiffré avec la clé `k` ? (Deux façons possibles.)

▷ coup de pouce

★★ Exercice 3 *Le chiffrement affine* ▶ corrigé p. 938

Chaque lettre de rang x devient `(a * x + b) % 26`. La clé est le couple (a,b).

1. Pourquoi a doit-il être **premier avec** 26 ? Donner la liste de toutes les valeurs possibles de a. ▷ coup de pouce
2. Chiffrer `NSI` avec la clé $(a,b) = (3,1)$.
3. Pour déchiffrer, on a besoin de l'**inverse modulaire** de a. Vérifier que 9 est l'inverse de 3 modulo 26 (c'est-à-dire que `(3 * 9) % 26` vaut 1).
4. En déduire la formule de déchiffrement, puis déchiffrer `ODZ` chiffré avec (3,1). ▷ coup de pouce

★★ **Exercice 4** *Vigenère : la clé qui se répète* ▶ corrigé p. 939

On chiffre en additionnant, modulo 26, chaque lettre du message avec la lettre correspondante de la clé, **répétée** autant de fois que nécessaire.

1. Chiffrer `ANANAS` avec la clé `NSI` (répétée en `NSINSI`).
2. Dans le mot chiffré, une même lettre du clair (par exemple les trois `A`) donne-t-elle toujours la même lettre chiffrée ? Comparer avec César sur ce point.
3. Le chiffre de Vigenère fut réputé « indéchiffrable » pendant trois siècles. Quelle propriété de la clé (le fait qu'elle se *répète*) a finalement permis de le casser ? ▷ coup de pouce

★ **Exercice 5** *Le masque jetable et le XOR* ▶ corrigé p. 939 • essentiel

En informatique, le masque jetable combine chaque caractère du message avec la clé par un **OU exclusif** (XOR, noté `^`).

```
1 def chiffre(message, masque):
2     resultat = ""
3     for i in range(len(message)):
4         code = ord(message[i]) ^ ord(masque[i])
5         resultat = resultat + chr(code)
6     return resultat
```

1. On sait que $65 \text{ ^ } 66 = 3$. Que vaut $3 \text{ ^ } 66$? Qu'illustre ce calcul sur le XOR ?
2. En déduire pourquoi la *même* fonction `chiffre` sert aussi à **déchiffrer**.
3. À quelles conditions (longueur, hasard, réutilisation) le masque jetable est-il **inviolable** ? Citer le nom du chercheur qui l'a démontré.
4. Pourquoi, malgré cette perfection théorique, n'utilise-t-on pas le masque jetable partout ? ▷ coup de pouce

Chiffrement asymétrique et RSA ↑ cours

★★ **Exercice 6** *Symétrique ou asymétrique ?* ▶ corrigé p. 939 • essentiel

1. Expliquer en une phrase la différence entre un chiffrement **symétrique** et un chiffrement **asymétrique**.
2. Citer le principal **défaut** du chiffrement symétrique, que l'asymétrique vient résoudre.
3. Alice veut envoyer un message confidentiel à Bob. Avec *quelle* clé, et de *qui*, doit-elle chiffrer ? Avec quelle clé Bob déchiffre-t-il ? ▷ coup de pouce
4. Alice veut maintenant **prouver** que c'est bien elle qui écrit (signature). Avec quelle clé chiffre-t-elle alors ?

★★ **Exercice 7** *RSA à la main — fabriquer les clés (1)* ▶ corrigé p. 940 • essentiel

Alice choisit les nombres premiers $p = 3$ et $q = 11$.

1. Calculer $n = p \times q$, puis $\varphi = (p-1)(q-1)$.
2. Alice choisit $e = 3$. Vérifier que e est bien premier avec φ. Donner la **clé publique**.
3. Trouver d, l'inverse modulaire de e (le nombre tel que $3d \equiv 1\ [20]$). Donner la **clé privée**.

▷ coup de pouce

4. Bob envoie le nombre $M = 5$. Calculer le message chiffré $C = M^e \bmod n$.
5. Écrire le calcul que fait Alice pour déchiffrer C (on admettra qu'il redonne bien 5).

★★ **Exercice 8** *RSA à la main — trouver d (2)* ▶ corrigé p. 940

Cette fois $p = 7$ et $q = 11$, et Alice choisit $e = 7$.

1. Calculer n et φ.
2. Pour trouver d tel que $7d \equiv 1\ [60]$, on peut tester $d = 1,2,3,\ldots$ jusqu'à ce que $7d$ soit égal à un multiple de 60 augmenté de 1. Montrer que $d = 43$ convient (calculer 7×43).
3. Donner la clé publique et la clé privée.
4. Bob envoie $M = 2$. Calculer $C = 2^7 \bmod 77$.

▷ coup de pouce

★★ **Exercice 9** *L'inverse modulaire en Python* ▶ corrigé p. 940

En Python, on calcule d par `d = pow(e, -1, phi)`.

1. Que renvoie `pow(3, -1, 20)` ? `pow(7, -1, 60)` ?
2. Que calcule l'appel `pow(M, e, n)` ? Pourquoi l'écrit-on ainsi plutôt que `(M ** e) % n` ?

▷ coup de pouce

3. Compléter la fonction de génération de clés : ▷ coup de pouce

```
def generer_cles(p, q, e):
    n = ...
    phi = ...
    d = ...
    return (e, n), (d, n)   # (cle publique, cle privee)
```

★★ **Exercice 10** *L'échange de clé de Diffie-Hellman* ▶ corrigé p. 940

Alice et Bob veulent fabriquer une clé secrète commune *en public*. Ils conviennent publiquement de $p = 23$ et $g = 5$.

1. Alice choisit en secret $a = 4$ et publie $A = g^a \bmod p$. Calculer A. ▷ coup de pouce
2. Bob choisit en secret $b = 3$ et publie $B = g^b \bmod p$. Calculer B.
3. Alice calcule $B^a \bmod p$; Bob calcule $A^b \bmod p$. Vérifier qu'ils obtiennent **le même** secret.
4. Ève a intercepté p, g, A et B. Pourquoi ne peut-elle pas retrouver le secret commun aussi facilement qu'Alice et Bob ? ▷ coup de pouce

HTTPS ↑cours

★ **Exercice 11** *Le petit cadenas du navigateur* ▶ corrigé p. 941 • essentiel

1. HTTPS combine un chiffrement **asymétrique** et un chiffrement **symétrique**. Lequel sert à *échanger la clé*, lequel sert à chiffrer *les données* ? Pourquoi ce partage des rôles ?
2. Remettre dans l'ordre les étapes de la poignée de main (*handshake*) TLS :
   - ☐ le client fabrique une clé symétrique, la chiffre avec la clé publique du serveur et l'envoie ;
   - ☐ le serveur envoie son certificat et sa clé publique ;
   - ☐ les deux échangent ensuite leurs données chiffrées en symétrique (AES) ;
   - ☐ le client vérifie le certificat (signé par une autorité de certification).
3. À quoi sert le **certificat** ? Que risquerait-on sans lui ?

▷ coup de pouce

Vers le bac ↑cours

★★ **Exercice 12** *Type bac — La méthode du masque jetable* (d'après Métropole 2025, jour 2) ▶ corrigé p. 941 • essentiel ≈ 60 min

Partie A — Le masque jetable. On attribue à chaque lettre son rang (0 à 25). Pour chiffrer, on additionne le rang de chaque lettre du message avec celui de la lettre correspondante du masque ; si le résultat dépasse 25, on retranche 26 (calcul « modulo 26 »). Exemple : `HELLO` avec le masque `WMCKL` donne `DQNVZ`.

1. Chiffrer le message `LIBRE` à l'aide du masque (la clé) `EYQMT`.

On dispose de la variable `alphabet` (liste des 26 lettres dans l'ordre) accessible partout.

2. Écrire une fonction `indice(L, element)` qui renvoie l'indice de `element` dans la liste `L` (on suppose `element` présent une seule fois). Par exemple `indice(alphabet, 'K')` renvoie `10`.

▷ coup de pouce

3. Écrire `lettres_vers_indices(chaine)` qui renvoie la liste des indices des lettres. Par exemple `lettres_vers_indices('HELLO')` renvoie `[7, 4, 11, 11, 14]`. ▷ début de solution

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

On dispose aussi de `indices_vers_lettres` (réciproque, non demandée). Voici la fonction `chiffrement`, incomplète :

```
def chiffrement(msg, cle):
    assert len(cle) >= len(msg), "impossible"
    indices_msg = lettres_vers_indices(msg)
    indices_cle = lettres_vers_indices(cle)
    n = len(msg)
    indices_msg_chiffre = []
    for k in range(n):
        ind = ...              # (a)
        if ind >= 26:
            ind = ...          # (b)
        indices_msg_chiffre.append(ind)
    msg_chiffre = indices_vers_lettres(...)   # (c)
    return msg_chiffre
```

4. Recopier et compléter les lignes `(a)`, `(b)` et `(c)`.
5. Indiquer, en justifiant, ce que l'on observe lors de l'appel `chiffrement('RESEAU', 'GFTZ')`.
▷ coup de pouce
6. Déchiffrer le message `GMEDH` avec la clé `FVEIT`. Expliquer la méthode de déchiffrement quand on connaît la clé.
▷ coup de pouce
7. Adapter la fonction `chiffrement` pour obtenir une fonction `dechiffrement` (indiquer les lignes à modifier).

Partie B — Sécurisation des communications.

8. Expliquer la différence entre un algorithme de chiffrement symétrique et asymétrique.
9. Bob envoie sa clé publique à Alice, qui chiffre son message avec cette clé et l'envoie à Bob. Indiquer comment Bob déchiffre le message reçu.
10. Expliquer comment une tierce personne pourrait se faire passer pour Alice sans que Bob s'en aperçoive.
11. Expliquer brièvement le fonctionnement du protocole **HTTPS**, et pourquoi on ne se contente pas d'un chiffrement asymétrique pour toute la communication.

★★★ **Exercice 13** IA IA IA *Type bac — Le chiffrement de Polybe* (d'après Centres étrangers 2026, groupe 1, jour 1) ▶ corrigé p. 942 ≈ 60 min

Le chiffrement de Polybe range les 26 lettres et les 10 chiffres dans une grille 6×6. Chaque caractère est remplacé par le couple (numéro de ligne, numéro de colonne) de sa case. Exemple de grille :

	1	2	3	4	5	6
1	Q	7	A	X	2	J
2	9	E	H	0	R	M
3	L	Z	4	W	D	O
4	6	V	N	B	8	K
5	P	Y	1	S	T	F
6	G	C	3	I	U	5

Ainsi `N` (ligne 4, colonne 3) donne (4,3), et `NSI` se chiffre en (4,3) (5,4) (6,4).

1. Déchiffrer, avec la grille ci-dessus, le message (6,2) (3,6) (3,5) (2,2).

Pour construire une grille, on choisit une **clé** (des lettres et des chiffres, tous différents), on l'écrit d'abord, puis on complète avec les lettres restantes dans l'ordre alphabétique, enfin les chiffres 0 à 9. Avec la clé `2048ALGORITHMES`, l'ordre d'insertion est `2048ALGORITHMESBCDFJKNPQUVWXYZ135679`.

2. Chiffrer `BAC` avec la clé `SECURITY1024`.
▷ coup de pouce
3. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

On donne la fonction `generer_ordre` :

```python
def generer_ordre(cle):
    ordre_insertion = cle
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
    for lettre in alphabet:
        if lettre not in ordre_insertion:
            ordre_insertion = ordre_insertion + lettre
    return ordre_insertion
```

4. Quel est le résultat de `generer_ordre('AXU7')` ?

5. Écrire `grille_vide(n)` renvoyant un tableau de `n` lignes de `n` chaînes vides. Par exemple `grille_vide(3)` renvoie `[["","",""], ["","",""], ["","",""]]`.

6. Recopier et compléter les lignes `(a)`, `(b)`, `(c)` de `generer_grille` : ▷ coup de pouce

```python
def generer_grille(cle):
    ordre_insertion = generer_ordre(cle)
    grille = grille_vide(6)
    indice = 0
    for i in range(...):        # (a)
        for j in range(...):    # (b)
            grille[i][j] = ...  # (c)
            indice = indice + 1
    return grille
```

7. Écrire la fonction `generer_dico(cle)` renvoyant le dictionnaire qui associe à chaque caractère le couple (numéro de ligne, numéro de colonne) de sa case, numérotés **à partir de** 1 comme dans l'exemple (4,3) ci-dessus, et non à partir de 0 comme les indices de `grille`. Puis écrire `chiffrer(cle, message)` renvoyant la liste des couples. ▷ début de solution

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

8. Alice et Bob changent de clé chaque jour et souhaitent se l'échanger par un chiffrement **asymétrique**. Expliquer la différence entre chiffrement symétrique et asymétrique.

★ Exercice 14

Type bac — Transmettre une clé de session (d'après Métropole 2024, jour 2) ▶ corrigé p. 943 ≈ 15 min

Bob a mis en place une base de données pour gérer sa collection de CD. Cette base est hébergée sur un serveur auquel il accède depuis un client installé sur son ordinateur personnel. Pour sécuriser la connexion, un algorithme de chiffrement **symétrique** est utilisé.

1. Expliquer brièvement ce qu'est un algorithme de chiffrement symétrique.

La clé de chiffrement, notée C, est choisie aléatoirement par le serveur à chaque connexion d'un client. Pour que le chiffrement et le déchiffrement puissent se faire sans problème, le serveur doit envoyer au client la clé C de façon sécurisée.

2. Rappeler brièvement ce qu'est un algorithme de chiffrement asymétrique.

On suppose à présent que Bob possède une clé publique et une clé privée, et que sa clé publique est connue du serveur.

3. Proposer une solution pour que le serveur envoie la clé C à l'ordinateur de Bob de façon sécurisée, c'est-à-dire pour que **seul Bob** puisse déchiffrer la clé envoyée.

▷ coup de pouce

★★★ **Exercice 15** *Type bac — Les mots de passe d'un hôpital* (d'après Polynésie 2024, jour 2) ▶ corrigé p. 944 ≈ 45 min

Pour utiliser les ordinateurs d'un hôpital, tout le personnel doit saisir son identifiant puis son mot de passe. Pour davantage de sécurité, ce dernier doit être fort et changé régulièrement. On appelle « mot de passe fort » une chaîne de caractères composée : d'au minimum 12 caractères ; d'au moins 2 majuscules ; d'au moins 2 chiffres ; d'au moins 2 symboles parmi `#@!?%<>=€$+-*/&`. Par exemple, un médecin utilise le mot de passe fort `@20!HôPiTaL&24#`.

On dispose des méthodes `isalpha` (le caractère est-il une lettre ?), `isupper` (est-il une majuscule ?) et `isdigit` (est-il un chiffre ?) :

```
>>> 'A'.isalpha()
True
>>> 'a'.isupper()
False
>>> '5'.isdigit()
True
```

Tous les symboles sont rangés dans une variable globale de type `str` :

```
1 liste_symboles = '#@!?%<>=€$+-*/&'
```

1. ▶ La fonction `mdp_fort` prend en paramètre une chaîne `mdp` et renvoie `True` si `mdp` est un mot de passe fort, `False` sinon. Compléter les lignes 2, 10, 12 et 14. ▷ coup de pouce

```
1  def mdp_fort(mdp):
2      if ... :
3          return False
4      majuscules = 0
5      chiffres = 0
6      symboles = 0
7      for caractere in mdp:
8          if caractere.isupper():
9              majuscules += 1
10         if ... :
11             chiffres += 1
12         if ... :
13             symboles += 1
14     if ... :
15         return False
16     return True
```

Pour aider le personnel, le service informatique a écrit une fonction `creation_mdp` qui génère aléatoirement un mot de passe. Elle prend en paramètres quatre entiers : la longueur `n` du mot de passe, et les nombres minimaux `nbr_m` de majuscules, `nbr_c` de chiffres et `nbr_s` de symboles qu'il doit contenir. Elle renvoie une chaîne respectant ces conditions (la fonction `choice`, du module `random`, renvoie un élément choisi au hasard).

2. ▶ Compléter la ligne 9 et les lignes 13 à 19.

```
1  def creation_mdp(n, nbr_m, nbr_c, nbr_s):
2      mdp = ''
3      caracteres = 'abcdefghijklmnopqrstuvwxyz' + \
4                   'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789' + \
5                   '#@!?%<>=€$+-*/&'
6      majuscules = 0
7      chiffres = 0
8      symboles = 0
9      while ... :
10         # la variable c contient un caractere
11         # choisi aleatoirement dans la variable caracteres
12         c = choice(caracteres)
13         if ... :
14             ...
15         if ... :
16             ...
17         if ... :
18             ...
19         mdp = ...
20     return mdp
```

Sans le générateur, le personnel a tendance à insérer des mots de la langue dans son mot de passe : `@20!HôPiTaL&24#` est fort, mais on y trouve le mot « hôpital ». Un mot de passe fort qui ne contient aucun mot de 4 lettres ou plus de la langue française est dit « extra fort ». On dispose de la variable `dicoFR` (de type `list`), qui contient tous les mots de la langue française, et de la fonction `transforme`, qui renvoie une copie de la chaîne où toutes les majuscules sont devenues des minuscules (par exemple, `transforme('@20!HôPiTaL&24#')` renvoie `'@20!hôpital&24#'`).

On suppose que les mots présents dans un mot de passe sont entiers et séparés par des chiffres ou des symboles. La fonction `recherche_mot` prend en paramètre un mot de passe `mdp` et renvoie la liste des mots (en minuscules) qu'il contient :

```
>>> recherche_mot('@20!NeUrO&24#')
['neuro']
>>> recherche_mot('Chef!14Neuro@85!')
['chef', 'neuro']
```

3. ▶ Compléter les lignes 5, 6, 15, 16 et 18 de la fonction `recherche_mot`. ▷ coup de pouce

```
1  def recherche_mot(mdp):
2      mot = transforme(mdp)
3      trouve = []
4      i = 0
5      while ... :
6          if ... :   # si le caractere est un chiffre
7              i = i + 1
8          elif mot[i] in liste_symboles:
9              i = i + 1
10         else:
11             # si le caractere est une lettre, on prend les
12             # lettres qui la suivent jusqu'au moment ou
13             # on trouve un chiffre ou un symbole
14             chaine = ''
15             while ... :
16                 chaine = ...
17                 i = i + 1
18             ...
19     return trouve
```

4. ▶ Écrire une fonction `mdp_extra_fort` qui prend en paramètre une chaîne `mdp` et renvoie `True` si `mdp` est un mot de passe extra fort, `False` sinon. ▷ coup de pouce ▷ début de solution

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 16** IA IA IA *Type bac — Les casiers connectés du lycée* (sujet maison, façon bac) ▶ corrigé p. 945 ≈ 60 min

Cet exercice n'est pas un sujet officiel : il a été écrit au format du bac pour s'entraîner.

Le lycée installe des casiers connectés. Pour ouvrir son casier, un élève utilise une application sur son téléphone, qui envoie au casier, par radio, un court message texte, par exemple `"OUVRE07"` pour le casier numéro 07. Comme n'importe qui peut capter ces ondes, le message est chiffré.

Partie A — Un XOR avec une clé courte. Chaque caractère est remplacé par son code (`ord("O")` vaut 79, `ord("U")` vaut 85…). On combine ensuite chaque code avec un nombre de la clé par un **OU exclusif** (XOR, opérateur `^` en Python). Rappel : bit à bit, le XOR vaut 1 si les deux bits sont différents, 0 s'ils sont égaux. La clé, enregistrée une fois pour toutes dans l'application et dans le casier, est la liste `[23, 5, 42]` : elle est **répétée** autant que nécessaire.

1. Écrire 79 et 23 en binaire sur 8 bits, puis calculer 79 `^` 23 bit à bit. Calculer ensuite 88 `^` 23. Que constate-t-on ?

Un message est représenté par la liste de ses codes. On considère la fonction suivante, incomplète :

```
1 def chiffrer_xor(codes, cle):
2     resultat = []
3     for i in range(len(codes)):
4         k = cle[...]                  # (a) la cle est repetee
5         resultat.append(...)          # (b)
6     return resultat
```

2. ▶ Recopier et compléter les lignes `(a)` et `(b)`. Pour un message de 7 codes, donner l'indice de la clé utilisé pour chaque valeur de `i` de 0 à 6. ▷ coup de pouce

3. Expliquer pourquoi la *même* fonction `chiffrer_xor`, avec la même clé, permet au casier de **déchiffrer**. Ce chiffrement est-il symétrique ou asymétrique ? Justifier.

Ève, une élève curieuse, capte le message chiffré `[88, 80, 124, 69, 64, 26, 32]`. Elle ne connaît pas la clé, mais elle sait que tous les messages commencent par `"OUVRE"` et que la clé contient trois nombres.

4. Montrer que si $c = m \text{ ^ } k$, alors $k = c \text{ ^ } m$. En déduire, sachant que `"OUV"` a pour codes 79, 85, 86, les trois nombres de la clé (détailler les calculs en binaire).

5. ▶ Écrire une fonction `retrouver_cle(intercepte, debut_connu, n)` qui renvoie la liste des `n` premiers nombres de la clé, à partir de la liste `intercepte` des codes chiffrés et de la liste `debut_connu` des codes du début du message clair (on suppose qu'elles contiennent au moins `n` éléments). Quel appel permet ensuite à Ève de lire tout le message ? ▷ coup de pouce ▷ début de solution

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

6. Le cours présente le masque jetable comme un chiffrement **inviolable**. Citer les conditions qu'il impose et expliquer lesquelles ne sont pas respectées ici.

Partie B — Échanger une clé neuve avec RSA. Pour corriger le défaut, l'application tire au hasard une **nouvelle clé** à chaque ouverture et l'envoie d'abord au casier, chiffrée avec RSA. Le casier possède une paire de clés fabriquée avec $p = 17$, $q = 19$ et $e = 5$.

7. Calculer n et φ. Vérifier que e est premier avec φ, puis que $d = 173$ convient (calculer 5×173). Donner la clé publique et la clé privée du casier.

8. L'application doit transmettre le nombre $K = 42$ (un nombre de la nouvelle clé). Écrire l'instruction Python qui calcule le nombre chiffré C ; on obtient $C = 264$. Écrire l'instruction que le casier exécute pour retrouver K.

9. Avec ces petits nombres, Ève pourrait retrouver la clé privée. Expliquer comment, puis pourquoi c'est impossible en pratique avec un vrai RSA. ▷ coup de pouce

Partie C — Le faux casier.

10. Ève installe un émetteur qui se fait passer pour le casier et envoie à l'application *sa propre* clé publique. Expliquer ce qu'elle peut alors faire. Quel élément, utilisé par HTTPS lors de la poignée de main TLS, permettrait à l'application de déjouer cette attaque ?

Critiquer une réponse d'IA ↑ cours

★★ **Exercice 17** *Des clés RSA fabriquées par un assistant d'IA* ▶ corrigé p. 947

Un élève demande à un assistant d'IA : « Écris un programme Python qui fabrique une paire de clés RSA à partir de deux nombres premiers `p` et `q`, puis qui chiffre et déchiffre un entier. » Voici la réponse obtenue :

```
1  p, q = 5, 11
2  n = p * q                  # module public
3  e = 3                      # exposant public, premier avec (p-1)(q-1)
4  d = pow(e, -1, n)          # exposant prive : inverse modulaire de e
5
6  def chiffrer(M):
7      return pow(M, e, n)    # C = M^e mod n
8
9  def dechiffrer(C):
10     return pow(C, d, n)    # M = C^d mod n
```

« La clé publique est `(e, n)`, la clé privée `(d, n)` ; `pow(e, -1, n)` calcule l'inverse modulaire de `e`, exactement comme dans le cours. »

1. La réponse est-elle correcte ? La vérifier en console avec le message $M = 7$: calculer $C =$ `chiffrer(7)` (faisable à la main), puis `dechiffrer(C)`. Retrouve-t-on 7 ?

2. Localiser et corriger l'erreur (on donnera la valeur correcte de d). ▷ coup de pouce

3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

S'entraîner à l'oral ↑cours

★★ **Exercice 18** *Expliquer en deux minutes* ▶ corrigé p. 947

Au Grand Oral comme devant l'examinateur de l'épreuve pratique, il faut savoir **expliquer** une notion clairement, sans lire ses notes. Choisir l'un des sujets suivants (ou le tirer au sort) :

Sujet 1. Expliquer à un camarade de Première la différence entre chiffrement **symétrique** et chiffrement **asymétrique**.
Sujet 2. Expliquer comment **HTTPS** combine les deux types de chiffrement.
Sujet 3. Expliquer pourquoi **RSA** est considéré comme sûr alors que la clé publique est connue de tous.

Préparation (5 min, seul). Noter au brouillon **trois idées** et **un exemple**, rien de plus : comme au Grand Oral, ces notes servent d'aide-mémoire. **Exposé (2 min, chronométré).** Debout, face à un camarade, **sans lire ses notes** et sans rien écrire : tout passe par la parole. **Retour (2 min).** Le camarade pose une question de relance (on peut alors s'aider d'un schéma), remplit la grille ci-dessous et donne un conseil ; puis on échange les rôles. Cette grille reprend les critères d'évaluation du Grand Oral. ▷coup de pouce

Grille entre camarades	Oui	En partie	Pas encore
Clair — audible, posé ; chaque mot technique est expliqué	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	☐	☐	☐
Construit — un fil conducteur, tenu en deux minutes, sans lire	☐	☐	☐
Un conseil pour la prochaine fois :			

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Remplacer chaque lettre par son rang, ajouter la clé (la retrancher pour déchiffrer), revenir entre 0 et 25 si besoin, puis reconvertir en lettre. Pour la question 4 : sans la clé, combien d'essais faut-il au plus pour être sûr de la trouver ? ↑énoncé

Ex. 2 – piste Ligne `(a)` : le rang décalé doit rester entre 0 et 25 ; quel opérateur Python donne le reste d'une division ? Ligne `(b)` : `chr` fait l'inverse de `ord` ; à quel code faut-il ajouter le nouveau rang ? Pour déchiffrer, on décale dans l'autre sens. ↑énoncé

Ex. 3 q. 1 – piste Q1 : essayer $a = 2$: quelles lettres deviennent `A` ? Peut-on encore déchiffrer sans ambiguïté ? ↑énoncé

Ex. 3 q. 4 – piste Q4 : partir de `y = (a * x + b) % 26` et isoler x : on ne divise pas par a modulo 26, on multiplie par son inverse. ↑énoncé

Ex. 4 q. 3 – piste Avec une clé de 3 lettres, comment sont chiffrées la 1re, la 4e, la 7e… lettre du message ? Quel chiffrement déjà cassé retrouve-t-on ? ↑énoncé

Ex. 5 – piste Pour la question 1, écrire 3 et 66 en binaire sur 8 bits et appliquer la règle du XOR bit à bit (1 si les deux bits sont différents, 0 sinon). Pour les questions 3 et 4, relire la partie du cours sur le masque jetable. ↑énoncé

Ex. 6 q. 3 – piste Tout le monde doit pouvoir écrire à Bob, mais seul Bob doit pouvoir lire : laquelle de ses deux clés reste chez lui ? ↑énoncé

Ex. 7 q. 3 – piste Tester $d = 1,2,3,\ldots$: calculer $3d$ et regarder le reste de sa division par 20. ↑énoncé

Ex. 8 q. 4 – piste Calculer d'abord 2^7, puis le reste de sa division par n. ↑énoncé

Ex. 9 q. 2 – piste Q2 : imaginer M et e de plusieurs centaines de chiffres : combien de chiffres aurait `M ** e` ? ↑énoncé

Ex. 9 q. 3 – piste Q3 : reprendre les trois premières étapes de RSA du cours : une ligne par étape. ↑

Ex. 10 q. 1 – piste Q1 : calculer 5^4, puis lui retirer le plus grand multiple de 23 possible. ↑énoncé

Ex. 10 q. 4 – piste Q4 : pour calculer le secret, il faut connaître a ou b. Quelle opération Ève devrait-elle savoir « inverser » pour les retrouver à partir de A ou B ? ↑énoncé

Ex. 11 – piste Question 1 : lequel des deux chiffrements est lent, mais permet d'échanger une clé sans secret partagé au départ ? Question 2 : le client ne peut rien chiffrer tant qu'il n'a pas reçu, et vérifié, la clé publique du serveur. Question 3 : comment le client saurait-il, sans certificat, que la clé publique reçue appartient bien au serveur ? ↑énoncé

Ex. 12 q. 2 – piste Q2 : on veut l'*indice*, pas l'élément : quelle forme de boucle `for` donne accès aux indices ? ↑énoncé

Ex. 12 q. 3 – début de solution Q3 :

```
def lettres_vers_indices(chaine):
    resultat = []
    for lettre in chaine:

        ...
```

↑énoncé

Ex. 12 q. 5 – piste Q5 : comparer les longueurs du message et de la clé, puis relire la première ligne de la fonction. ↑énoncé

Ex. 12 q. 6 – piste Q6 : on fait l'opération inverse : soustraire au lieu d'additionner. Si le résultat est négatif, comment revenir entre 0 et 25 ? ↑énoncé

Ex. 13 q. 2 – piste Q2 : écrire d'abord l'ordre d'insertion complet, puis remplir la grille ligne par ligne, 6 caractères par ligne. ↑énoncé

Ex. 13 q. 6 – piste Q6 : la variable `indice` avance de 1 à chaque case : quel caractère de `ordre_insertion` placer en case `(i, j)` ? ↑énoncé

Ex. 13 q. 7 – début de solution Q7 :

```
def generer_dico(cle):
    grille = generer_grille(cle)
    dico = {}
    for i in range(6):
        for j in range(6):
```

↑énoncé

Ex. 14 – piste Relire dans le cours la différence entre les deux familles de chiffrement. Pour la question 3 : des deux clés de Bob, laquelle le serveur connaît-il, et laquelle Bob est-il seul à posséder ? ↑énoncé

Ex. 15 q. 1 – piste Q1, ligne 2 : la longueur. Ligne 14 : il suffit qu'*une seule* des trois conditions de comptage échoue pour renvoyer `False`. ↑énoncé

Ex. 15 q. 3 – piste Q3 : la boucle intérieure doit s'arrêter en fin de chaîne *ou* sur un chiffre ou un symbole : tester `i < len(mot)` en premier. ↑énoncé

Ex. 15 q. 4 – piste Q4 : réutiliser `mdp_fort` et `recherche_mot`, puis examiner chaque mot trouvé : sa longueur et sa présence dans `dicoFR`. ↑énoncé

Ex. 15 q. 4 – début de solution Q4 :

```
def mdp_extra_fort(mdp):
    if not mdp_fort(mdp):
        return False
    for mot in recherche_mot(mdp):
```

↑énoncé

Ex. 16 q. 2 – piste Q2 : pour faire « tourner » un indice sur une clé de longueur 3, utiliser le reste de la division de `i` par `len(cle)`. ↑énoncé

Ex. 16 q. 5 – piste Q5 : appliquer la question 4 à chaque position : le i-ième nombre de la clé s'obtient à partir de `intercepte[i]` et `debut_connu[i]`. ↑énoncé

Ex. 16 q. 5 – début de solution Q5 :

```
def retrouver_cle(intercepte, debut_connu, n):
    cle = []
    for i in range(n):
```

↑énoncé

Ex. 16 q. 9 – piste Q9 : avec $n = 323$, combien de divisions faut-il essayer pour trouver p et q ? Et avec un n de 600 chiffres ? ↑ énoncé

Ex. 17 q. 2 – piste Relire l'étape 3 de RSA dans le cours : modulo *quel* nombre calcule-t-on l'inverse de e ? ↑ énoncé

Ex. 18 – piste Structurer chaque explication en trois temps : une **définition** en une phrase, un **exemple** petit et concret déroulé à voix haute, puis le **piège** (l'erreur que l'auditeur risque de faire). Finir par une phrase qui répond à la question posée. Une image aide beaucoup à l'oral (un coffre et sa clé, un cadenas ouvert que l'on distribue) ; dire ensuite précisément ce que représente chaque objet. Sujet 3 : qu'est-ce qui est facile à calculer, et qu'est-ce qui est difficile ? ↑ énoncé

TP ET PROJETS

TP Chiffrer, casser, signer ▶ corrigé p. 948

de César à RSA, en Python

Durée : 3 h (deux séances) · Sur machine, par deux

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t10-tp-crypto` — archive .zip

Consignes

- Fichier à télécharger (lien ci-dessus) : `tp_crypto_depart.py`. Il contient un texte français `TEXTE`, un tableau `FREQ_FR` des fréquences des lettres en français, la fonction `normaliser` (majuscules, sans accents), des squelettes de fonctions et des tests. Le lancer affiche l'état des tests.
- Les messages sont en **majuscules sans accents** ; seules les lettres `A` à `Z` sont chiffrées, les autres caractères (espaces, ponctuation) sont recopiés tels quels.
- Niveau : ★ échauffement ★★ classique ★★★ défi (dont les extraits de bac les plus exigeants) ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme. Le symbole ▶ signale ce qui est à programmer et tester.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

But du TP

Programmer les chiffrements du cours **et les attaques** qui les cassent : César par force brute puis par analyse de fréquences, Vigenère colonne par colonne, le masque jetable réutilisé ; puis construire un **RSA complet** — génération de clés avec l'algorithme d'Euclide étendu, chiffrement d'un message, signature — et mesurer pourquoi sa sécurité tient : **factoriser** n devient vite hors de portée. *Produit final* : une petite bibliothèque de chiffrement et d'attaques, testée, et un tableau de mesures.

Partie A — César : chiffrer, puis casser sans la clé

★ Exercice 1 *César, version complète*

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1. ▶ Compléter `decaler(lettre, k)` : une majuscule de rang x devient la lettre de rang `(x + k) % 26` ; tout autre caractère est renvoyé inchangé. Puis compléter `cesar(texte, k)`. Exemple : `cesar("BONJOUR !", 3)` vaut `"ERQMRXU !"`.

2. ▶ Chiffrer `clair = normaliser(TEXTE)` avec la clé 11, afficher le début du chiffré, puis le déchiffrer.

★ **Exercice 2** *La force brute*

On a intercepté : `KXGWXS-OHNL WXOTGM EX FNLXX HVXTGHZKTIABJNX T FBWB.`

1. ▶ Afficher les 26 déchiffrements possibles (un par ligne, précédé de la clé). Lequel est le bon ? Quelle est la clé ?
2. Un programme doit faire ce tri tout seul. Proposer une idée pour reconnaître automatiquement « du français ».

★★ **Exercice 3** *L'analyse de fréquences, automatisée*

1. ▶ Compléter `frequences(texte)` (dictionnaire lettre → nombre d'apparitions, pour les lettres `A` à `Z` seulement) et `plus_frequente(texte)`. Quelles sont les trois lettres les plus fréquentes de `clair` ?

2. ▶ Compléter `casser_cesar(chiffre)`, qui suppose que la lettre la plus fréquente du chiffré est un `E` et en déduit la clé. Vérifier qu'elle retrouve la clé 11 sur le long texte.

3. ▶ Essayer `casser_cesar` sur `cesar("ATTAQUE A L AUBE", 7)`. Quelle clé trouve-t-on ? Pourquoi l'hypothèse « la plus fréquente est un `E` » échoue-t-elle ici ?

4. ▶ Méthode plus robuste : compléter `score(texte)`, la somme des valeurs `FREQ_FR[c]` pour toutes les lettres `c` du texte (un texte français contient beaucoup de lettres fréquentes, donc a un score élevé), puis `casser_cesar_score(chiffre)` qui renvoie la clé k pour laquelle `cesar(chiffre, -k)` a le **meilleur score**. Tester sur le message court, sur le message intercepté, et sur le long texte. ▷ coup de pouce

Partie B — Vigenère, et comment Babbage l'a cassé

★ **Exercice 4** *Chiffrer avec une clé-mot*

▶ Compléter `vigenere(texte, cle, sens=1)` : la i^{e} **lettre** du texte est décalée selon la lettre `cle[i % len(cle)]` (`A` → 0, `B` → 1…) ; avec `sens = -1`, on décale en arrière (déchiffrement). Les caractères qui ne sont pas des lettres sont recopiés et **ne font pas avancer** la clé. Vérifier l'exemple du cours : `vigenere("NSI ROCKS", "CLE")` vaut `"PDM TZGMD"`. ▷ coup de pouce

★★ **Exercice 5** *Casser Vigenère quand on connaît la longueur de la clé*

On a chiffré `clair` avec une clé de 6 lettres : `chiffre = vigenere(clair, "MONACO")`. Faisons comme si l'on ignorait la clé.

1. Les lettres d'indices $0, 6, 12, \ldots$ du texte (en ne comptant que les lettres) sont toutes décalées avec la *même* lettre de la clé. Par quel chiffrement, déjà cassé, sont-elles donc chiffrées ?

2. ▶ Compléter `colonnes(chiffre, L)`, qui renvoie la liste des L chaînes formées des lettres d'indices $0, L, 2L \ldots$, puis $1, L+1, \ldots$, etc. (les caractères qui ne sont pas des lettres sont ignorés). Exemple : `colonnes("AB CDE", 2)` vaut `["ACE", "BD"]`. ▷ coup de pouce

3. ▶ En déduire `casser_vigenere(chiffre, L)`, qui casse chaque colonne avec `casser_cesar_score` et renvoie la clé. Retrouve-t-on `MONACO` ? Déchiffrer.

★★★ **Exercice 6** *Défi : trouver la longueur de la clé*

[au-delà du programme] L'**indice de coïncidence** d'un texte est la probabilité que deux lettres tirées au hasard soient identiques : si $n_A, n_B, \ldots$ sont les nombres d'apparitions des lettres et N le nombre total de lettres,

$$IC = \frac{n_A(n_A-1) + n_B(n_B-1) + \cdots + n_Z(n_Z-1)}{N(N-1)}.$$

1. ▶ Écrire `indice_coincidence(texte)`. Calculer l'indice de `clair`, puis d'un texte de 2 000 lettres tirées au hasard. Qu'est-ce qui les distingue ?
2. ▶ Pour L de 1 à 8, calculer la **moyenne** des indices des colonnes de `chiffre`. Pour quelles valeurs de L retrouve-t-on l'indice d'un texte français ? Pourquoi ?
3. ▶ Écrire `longueur_cle(chiffre)`, qui renvoie la plus petite longueur L dont la moyenne dépasse 0,065, puis casser sans aucune indication `vigenere(clair, "SECRETS")`.

Partie C — Le masque jetable, et l'erreur fatale

Outils Python de cette partie

- Le type `bytes` représente une suite d'octets : `b"RDV"` est la suite `82, 68, 86`, et `bytes([3, 3])` la suite `3, 3`. On accède à l'octet `i` par `m[i]`, qui est un entier ; `len(m)` donne le nombre d'octets.
- `import secrets` puis `secrets.token_bytes(n)` renvoie `n` octets tirés au hasard d'une façon adaptée à la cryptographie (imprévisible, contrairement au module `random`).
- `m.hex()` écrit les octets de `m` en hexadécimal (deux chiffres par octet), pour afficher un chiffré illisible.

10

★★ **Exercice 7** *Un masque, deux messages*

1. ▶ Compléter `xor_octets(a, b)`, qui renvoie la suite des `a[i] ^ b[i]` (construire la liste de ces entiers, puis la convertir avec `bytes(liste)`).

2. ▶ Créer un masque aléatoire de la longueur du message (22 octets) : `import secrets` puis `masque = secrets.token_bytes(22)`. Chiffrer `m1 = b"RDV A 18H SOUS LE PONT"`, afficher le chiffré (`c1.hex()` l'affiche en hexadécimal) et le déchiffrer. Pourquoi utiliser le module `secrets` plutôt que `random` pour fabriquer une clé ?

3. ▶ L'expéditeur, paresseux, **réutilise** le même masque pour `m2 = b"LE CODE EST 4071 MERCI"`. Ève intercepte `c1` et `c2`. Calculer `xor_octets(c1, c2)` et le comparer à `xor_octets(m1, m2)`. Justifier l'égalité avec la propriété $(m \oplus k) \oplus k = m$.

4. ▶ Ève devine le premier message (un rendez-vous, elle a vu la scène). Montrer qu'elle obtient alors le second **sans connaître le masque**. Conclure sur la règle « jetable ».

Partie D — Un RSA complet

★ **Exercice 8** *Six jeux de clés*

Voici six jeux de clés RSA. Pour chacun : $n = pq$, $\varphi = (p-1)(q-1)$, d est l'inverse de e modulo φ, et $C = M^e \bmod n$.

p	q	$n = pq$	φ	e	d	M	$C = M^e \bmod n$
3	11	33	20	3	7	4	31
5	11			3		7	
5	7	35	24	5	5	10	5
7	11	77	60	7	43	2	51
11	13			7		9	
13	17	221	192	5	77	8	60

1. À la main, sur le cahier, compléter la deuxième ligne ($p = 5$, $q = 11$) : trouver d en testant $d = 1, 2, 3\ldots$, puis calculer C. Vérifier que $C^d \bmod n$ redonne M (calcul fait par Python : `pow(C, d, n)`).
2. ▶ Écrire une boucle qui, pour chaque ligne, calcule n, φ, d (avec `pow(e, -1, phi)`) et C, et vérifie par un `assert` que le déchiffrement redonne M. En déduire la cinquième ligne.
3. Dans la troisième ligne, $d = e$: clé publique et clé privée ont le même exposant. Pourquoi ce jeu de clés serait-il catastrophique en vrai ?

★★ **Exercice 9** *Calculer d soi-même : l'algorithme d'Euclide étendu*

[au-delà du programme] Tester $d = 1, 2, 3\ldots$ est impossible quand φ a des centaines de chiffres. On utilise l'**algorithme d'Euclide étendu**, qui calcule $g = \text{pgcd}(a, b)$ **et** deux entiers u, v tels que $au + bv = g$. Il est récursif :

- si $b = 0$: renvoyer $(a, 1, 0)$ (car $a \times 1 + 0 \times 0 = a$) ;
- sinon : calculer (g, u', v') pour le couple $(b, a \,\%\, b)$, puis renvoyer $(g,\ v',\ u' - (a \,//\, b) \times v')$.

1. ▶ Compléter `pgcd(a, b)` (version itérative : tant que $b \neq 0$, remplacer (a, b) par $(b, a \,\%\, b)$) et `euclide_etendu(a, b)`. Vérifier : `euclide_etendu(240, 46)` vaut `(2, -9, 47)`, et $240 \times (-9) + 46 \times 47 = 2$.
2. Si $\text{pgcd}(e, \varphi) = 1$, on obtient $eu + \varphi v = 1$. Expliquer pourquoi $u \,\%\, \varphi$ est alors l'inverse de e modulo φ.
3. ▶ En déduire `inverse_modulaire(e, phi)` (qui renvoie `None` si $\text{pgcd}(e, \varphi) \neq 1$), et la comparer à `pow(e, -1, phi)` sur les six jeux de clés.

★★ **Exercice 10** *Fabriquer ses propres clés*

1. ▶ Compléter `est_premier(n)` (on teste les diviseurs d tant que $d \times d \leqslant n$). Écrire `premier_aleatoire(mini, maxi)`, qui tire des entiers au hasard entre `mini` et `maxi` jusqu'à en trouver un premier.
2. ▶ Écrire `generer_cles()` : tirer deux premiers distincts p, q entre 1 000 et 9 999, prendre $e = 65\,537$ (l'exposant utilisé en pratique), recommencer si $\text{pgcd}(e, \varphi) \neq 1$, et renvoyer `((e, n), (d, n))`. Afficher vos clés.
3. Pourquoi faut-il que $p \neq q$? Pourquoi `pgcd(e, phi)` doit-il valoir 1 ?

★★ **Exercice 11** *Chiffrer un vrai message*

1. ▶ Écrire `chiffrer_message(message, cle_publique)`, qui renvoie la liste des `pow(ord(c), e, n)` pour chaque caractère `c`, et `dechiffrer_message(nombres, cle_privee)`, qui reconstruit la chaîne avec `chr`. Chiffrer puis déchiffrer `"Rendez-vous à 18 h"`. Pourquoi faut-il $n >$ `ord(c)` pour tout caractère ?

2. Observer la liste chiffrée : que remarque-t-on pour les lettres répétées ? Quelle attaque de la partie A redevient possible ? *(En pratique, RSA ajoute un « remplissage » aléatoire au message, et ne sert qu'à chiffrer une clé de session AES, comme dans HTTPS.)*

★★ **Exercice 12** *Signer*

Pour signer, on chiffre avec la clé **privée** ; tout le monde vérifie avec la clé **publique**. On ne signe pas le message lui-même mais son **empreinte** [au-delà du programme] (fonction fournie `empreinte(message, n)`, un nombre entre 0 et $n-1$ calculé par la fonction de hachage SHA-256 : changer une seule lettre du message change complètement l'empreinte).

1. ▶ Écrire `signer(message, cle_privee)`, qui renvoie $h^d \bmod n$ où h est l'empreinte, et `verifier(message, signature, cle_publique)`, qui renvoie `True` si $s^e \bmod n$, où s est la signature, est égal à l'empreinte du message.

2. ▶ Signer `"Je dois 10 euros a Bob"`, vérifier la signature, puis vérifier la même signature avec le message `"Je dois 1000 euros a Bob"`. Qu'est-ce qu'une signature garantit ?

Partie E — Factoriser n : l'expérience qui rassure

★★ **Exercice 13** *Quand la factorisation devient lente*

1. ▶ Compléter `factoriser(n)`, qui cherche le plus petit diviseur $d \geqslant 2$ de n (tant que $d \times d \leqslant n$) et renvoie `(d, n // d)`, ou `None` si n est premier. Vérifier : `factoriser(3233)` vaut `(53, 61)`.

2. ▶ Chacun des nombres suivants est le produit de deux nombres premiers. Mesurer le temps de `factoriser` (avec `time.perf_counter()`), puis recopier et compléter le tableau :

n	chiffres de n	facteurs trouvés	durée (s)
1373249	7		
133763569	9		
13334100011	11		
1333361000071	13		
133333863333859	15		

3. Par combien la durée est-elle multipliée quand n gagne deux chiffres ? Expliquer ce facteur à partir du nombre de tours de boucle.

4. Une clé RSA réelle de 2 048 bits a un n de 617 chiffres. En prolongeant vos mesures, estimer la durée de `factoriser` sur un tel nombre, et la comparer à l'âge de l'Univers (environ 4×10^{17} secondes). *(Il existe des algorithmes de factorisation bien meilleurs, mais aucun ne casse une telle clé.)*

5. ▶ **Attaque** : chaque binôme publie sa clé publique et chiffre avec elle un court message secret, qu'il donne à un autre binôme (comme si celui-ci l'avait intercepté). Factoriser le n de l'autre binôme, recalculer φ et d, puis déchiffrer son message. Combien de temps a-t-il fallu ? Qu'en conclure sur des nombres premiers à 4 chiffres ?

Bilan du TP

À rédiger (une quinzaine de lignes)

1. Pour César, Vigenère et le masque jetable réutilisé : quelle faiblesse a-t-on exploitée ? Qu'est-ce qu'un bon chiffrement ne doit pas conserver du message ?
2. Résumer en quatre étapes la fabrication d'une paire de clés RSA, en citant les fonctions écrites.
3. Sur quoi repose la sécurité de RSA ? Appuyer la réponse sur les mesures de la partie E.
4. Chiffrer et signer : quelle clé utilise-t-on dans chaque cas, et qu'est-ce que chacun garantit ?

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 q. 4 – piste Même structure qu'une recherche de maximum : une boucle `for k in range(26)`, deux variables `meilleure_cle` et `meilleur_score`. ↑énoncé

Ex. 4 – piste Garder un compteur `i` des lettres déjà chiffrées, distinct de la position dans le texte ; réutiliser `decaler` avec le décalage `sens * k`. ↑énoncé

Ex. 5 q. 2 – piste Partir d'une liste de `L` chaînes vides. Parcourir le chiffré en comptant les lettres avec un compteur `i` (comme dans `vigenere`) : la lettre d'indice `i` s'ajoute à la chaîne d'indice `i % L`. ↑énoncé

FICHE DE RÉVISION

L'essentiel

- **Chiffrer** (avec la clé) / **déchiffrer** (avec la clé) / **décrypter** (**sans** la clé : l'attaquant). Ne jamais dire « crypter ».
- **Principe de Kerckhoffs** : la sécurité repose **uniquement sur le secret de la clé**, pas sur celui de l'algorithme (qui peut être public).
- **Chiffrement symétrique** : **une** clé partagée chiffre et déchiffre. Rapide (idéal pour de gros volumes). Exemples : César, Vigenère, **masque jetable**, **AES**. *Défaut* : il faut **échanger la clé** sur un canal non sûr.
- Le **masque jetable** (clé aléatoire, aussi longue que le message, jamais réutilisée) est **inviolable** (Shannon).
- **Chiffrement asymétrique** : **deux** clés jumelles, **publique** (connue de tous, sert à chiffrer) et **privée** (secrète, sert à déchiffrer). On ne peut pas déduire l'une de l'autre.
  - **confidentialité** : on chiffre avec la clé **publique du destinataire** ;
  - **authentification (signature)** : on chiffre avec sa **propre clé privée**.
- **HTTPS** = le mariage des deux : l'**asymétrique** sert à **échanger** une clé de session, puis le **symétrique** (AES), rapide, chiffre **toutes les données**.

RSA, pas à pas (à savoir dérouler)

1. choisir deux nombres premiers p et q ; calculer $n = p \times q$;
2. calculer $\varphi = (p-1)(q-1)$; choisir e **premier avec** φ $\quad\rightarrow$ **clé publique** (e, n) ;
3. calculer d, l'**inverse modulaire** de e $(e \times d \equiv 1\ [\varphi])$ $\quad\rightarrow$ **clé privée** (d, n) ;
4. **chiffrer** : $C = M^e \bmod n$; $\quad$ **déchiffrer** : $M = C^d \bmod n$.

```python
n = p * q
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)      # inverse modulaire de e
C = pow(M, e, n)         # chiffrer   (= M**e % n, en rapide)
M = pow(C, d, n)         # dechiffrer
```

*Sécurité de RSA : il est **facile** de multiplier $p \times q$, mais **très difficile de factoriser** n pour retrouver la clé privée. Menace future : l'ordinateur quantique (algorithme de Shor).*

À retenir aussi

- **Diffie-Hellman** : fabriquer une clé secrète commune **en public** (exponentiation modulaire) ; Ève ne peut pas la retrouver (logarithme discret).
- Le **certificat** (validé par une **autorité de certification**) authentifie le serveur dans HTTPS et évite l'attaque de l'*homme du milieu*.

Erreurs à éviter

- dire « crypter » ; croire César ou Vigenère « sûrs » ;
- se tromper de clé : **publique du destinataire** pour chiffrer, **sa propre privée** pour signer ;
- calculer φ avec $n-1$ (c'est $(p-1)(q-1)$) ; oublier que e doit être premier avec φ ;
- croire que HTTPS n'utilise que RSA (les **données** sont chiffrées en symétrique).

Savoir-faire à maîtriser — je sais…

- ☐ **chiffrer** et **déchiffrer** par décalage (César), par le masque jetable / XOR ;
- ☐ **expliquer** la différence symétrique / asymétrique et le **problème de l'échange de clé** ;
- ☐ **dérouler RSA** : calculer n, φ, choisir e, trouver d, chiffrer et déchiffrer un nombre ;
- ☐ **expliquer** l'authentification (signer avec la clé privée) et le rôle d'un certificat ;
- ☐ **décrire** le fonctionnement de **HTTPS** (asymétrique pour la clé, symétrique pour les données).

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 954

1. Quelle est la différence entre *déchiffrer* et *décrypter* ?
2. Que donne `ZOO` chiffré par César avec $k = 3$?
3. RSA avec $p = 5$, $q = 7$: que valent n et φ ?
4. Alice veut *signer* un message pour Bob : avec quelle clé chiffre-t-elle ?
5. Pourquoi HTTPS ne chiffre-t-il pas toutes les données avec RSA ?

Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 954

1. Combien de machines au plus sur un réseau `/26` ? (ch. 9)
2. Quelle est la condition d'arrêt du tri fusion, et pourquoi convient-elle ? (ch. 7)
3. Quelle différence entre un programme et un processus ? (ch. 4)

Vers le bac

- Problème de synthèse n° 8 — Router des paquets avec un dictionnaire, p. 711
- Problème de synthèse n° 9 — RSA miniature et exponentiation rapide, p. 712
- Épreuve pratique : sujet 30 — Messages secrets, p. 739

DÉFIS ADVENT OF CODE

DÉFI 1 Combo Breaker

la carte d'hôtel — Diffie-Hellman et exponentiation modulaire

Jour 25

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t10-defi-aoc-2020-25` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/25`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Une carte de chambre et une porte se mettent d'accord sur une clé secrète en s'échangeant deux nombres en public. On a intercepté ces deux nombres et l'on veut retrouver la clé secrète. Pour cela, il faut retrouver un nombre de tours de boucle qui peut atteindre plusieurs millions : c'est une recherche par essais successifs, que seul un programme peut mener.
Ce qu'on réinvestit. L'échange de clés de **Diffie-Hellman** et l'**exponentiation modulaire** vus dans ce chapitre : la carte et la porte appliquent exactement ce protocole, et l'on joue ici le rôle de l'espion. Le défi fait toucher du doigt pourquoi ce protocole est sûr : calculer une puissance modulaire est rapide, mais retrouver l'exposant à partir du résultat (le logarithme discret) est très lent quand le modulo est grand.
La démarche de la fiche.

1. Comprendre la transformation et le rôle de chaque nombre, puis faire quelques calculs

à la main et à la calculatrice.
2. Programmer la transformation d'un nombre.
3. Retrouver le secret d'un des deux appareils par essais successifs, puis en déduire la clé de chiffrement.
4. Mesurer la vitesse de cette recherche et expliquer pourquoi, avec de vrais modulos, l'espion est battu.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`handshake`	poignée de main (échange initial)	`subject number`	nombre de départ (base)
`loop size`	nombre de tours de boucle (secret)	`remainder`	reste (de la division)
`public key`	clé publique	`encryption key`	clé de chiffrement (secrète)
`eavesdrop`	écouter en cachette	`reverse-engineer`	retrouver par analyse
`trial and error`	essais successifs	`card / door`	carte / porte

Résumé de l'énoncé (partie 1)

Contexte. Dernier jour : votre carte de chambre ne fonctionne pas, et l'accueil est fermé. La carte et la porte s'authentifient par un petit protocole cryptographique : chacune envoie un nombre à l'autre, puis toutes deux calculent la même clé secrète. En écoutant ce qu'elles s'échangent, vous allez retrouver cette clé.
Les données. Le fichier contient deux lignes, chacune un entier : les deux nombres que vous avez interceptés. Le premier est la clé publique de la carte, le second celle de la porte.
La règle.

- Transformer un nombre s (le « sujet ») avec n tours : partir de 1 puis, n fois, multiplier par s et prendre le reste de la division par 20201227. Autrement dit, calculer s^n mod 20201227. Par exemple, transformer 5 avec 2 tours donne 25.
- La carte et la porte ont chacune un nombre de tours secret, qu'elles ne communiquent jamais.
- Chacune publie la transformée de 7 avec son secret : c'est sa clé publique.
- La clé de chiffrement est la transformée de la clé publique de la porte avec le secret de la carte ; c'est aussi la transformée de la clé publique de la carte avec le secret de la porte.

La réponse attendue. La clé de chiffrement. Par exemple, si le secret de la carte était 2 et la clé publique de la porte 1000, la clé de chiffrement serait 1000^2 mod 20201227 = 1000000.

Exercice 1 *Vérifier sa compréhension*

But. Vérifier sa compréhension de la transformation et de ce qu'il faut retrouver.
À quoi ça sert ? La deuxième question évite un travail inutile (un seul secret suffit) ; la troisième garantit que la recherche par essais successifs du programme se termine.

1. Que vaut la transformée de 7 avec 0 tour ? avec 1 tour ?
2. Faut-il retrouver les deux secrets pour calculer la clé ?
3. Pourquoi est-on sûr qu'une recherche « essayer $n = 1, 2, 3 \ldots$ » finira par trouver le secret ?

Exercice 2 *À la main, sur un petit exemple*

But. Calculer quelques transformées, puis vérifier sur un exemple complet que la carte et la porte trouvent la même clé.
À quoi ça sert ? On voit à partir de quand le modulo intervient, et l'on dispose de valeurs sûres (`10` tours, `19859298`, `1455220`) pour tester chacune des fonctions suivantes.

Transformer à la main le nombre `7` avec 1, 2 puis 3 tours de boucle (le modulo ne sert pas encore). À partir de combien de tours le reste de la division modifie-t-il le résultat ?

Dans un exemple inventé, la carte utilise `10` tours et la porte `17` tours. Avec la calculatrice ou Python, vérifier que les clés publiques sont `19859298` (carte) et `11869933` (porte), puis que la clé de chiffrement vaut `1455220`, qu'on la calcule du côté de la carte ou de la porte.

2 Programmer la partie 1

Exercice 3 *Transformer*

But. Programmer la transformation de l'énoncé.
À quoi ça sert ? Cette fonction sert deux fois : pour vérifier l'exemple et, une fois le secret trouvé, pour calculer la clé de chiffrement à partir de la clé publique de l'autre appareil.

Écrire une fonction `transformer(sujet, nb_tours)` qui applique la transformation de l'énoncé. Vérifier les valeurs de l'exemple précédent. ▷ coup de pouce

Exercice 4 *Retrouver un secret*

But. Retrouver le nombre de tours secret à partir d'une clé publique, puis calculer la clé de chiffrement.
À quoi ça sert ? C'est le travail de l'espion et la réponse du défi. La façon d'écrire la boucle compte : relancer la transformation depuis le début à chaque essai rendrait la recherche beaucoup trop lente.

Écrire une fonction `trouver_nb_tours(cle_publique)` qui retrouve le nombre de tours secret à partir d'une clé publique. Tester : `trouver_nb_tours(19859298)` doit renvoyer `10`. Calculer ensuite la clé de chiffrement de vos données. ▷ coup de pouce ▷ début de solution

3 Partie 2

Il n'y a pas de deuxième énigme ce jour-là : la seconde étoile est offerte quand les 49 autres étoiles de l'année ont été obtenues.

Outil Python : mesurer un temps (`time.perf_counter`)

`perf_counter()` renvoie un temps en secondes, très précis ; la différence entre deux appels donne la durée d'un calcul.

```
1 from time import perf_counter
2 debut = perf_counter()
3 n = trouver_nb_tours(19859298)          # le calcul a chronometrer
4 duree = perf_counter() - debut
5 print(f"{n} tours en {duree:.6f} s")    # 6 chiffres apres la virgule
```

Intérêt. En divisant le nombre de tours par la durée, on obtient une vitesse (tours par seconde) qui permet d'extrapoler.

★★★ **Exercice 5** *Défi : pourquoi est-ce sûr ?*

But. Mesurer la vitesse de la recherche, puis estimer sa durée pour des modulos beaucoup plus grands.
À quoi ça sert ? Le défi n'a été résolu que parce que le modulo est petit ; ce calcul d'ordre de grandeur explique pourquoi le même protocole, avec des modulos de centaines de chiffres, protège réellement nos communications.

Les vrais protocoles utilisent un modulo de plus de 600 chiffres. Chronométrer `trouver_nb_tours` sur vos deux clés publiques (encadré ci-dessus) et en déduire le nombre de tours par seconde de votre programme. Pour un modulo d'environ 2×10^7, la recherche essaie au pire 2×10^7 valeurs ; estimer le temps qu'elle prendrait pour un modulo de l'ordre de 10^{20}, puis 10^{600}. Expliquer pourquoi l'espion est battu alors que la carte et la porte calculent vite.

▷ coup de pouce

4 Rappel : l'échange de clés de Diffie-Hellman

Se mettre d'accord sur un secret en parlant en public (chapitre 10)

La transformation de l'énoncé calcule sujet^n mod p avec $p = 20201227$. C'est le protocole de **Diffie et Hellman** (1976) vu en cours. Un autre exemple avec de petits nombres : $p = 29$ et $g = 2$, connus de tous.

- Alice choisit en secret $a = 5$ et envoie $A = 2^5 \bmod 29 = 3$.
- Bob choisit en secret $b = 11$ et envoie $B = 2^{11} \bmod 29 = 18$.
- Alice calcule $B^a \bmod 29 = 18^5 \bmod 29 = 15$; Bob calcule $A^b \bmod 29 = 3^{11} \bmod 29 = 15$.

Ils obtiennent le même secret car $(g^b)^a = (g^a)^b = g^{ab}$. Un espion qui a entendu 3 et 18 doit retrouver a ou b : c'est le problème du **logarithme discret**.
Pourquoi est-ce difficile ? Les puissances successives de 2 modulo 29 semblent tirées au hasard : $2, 4, 8, 16, 3, 6, 12, 24, 19, 9, 18, 7 \ldots$ Aucune régularité ne permet de « remonter » d'un résultat à l'exposant : la méthode naïve essaie les exposants un à un, et même les meilleures méthodes connues deviennent beaucoup trop lentes quand p est très grand. À l'inverse, calculer une puissance est rapide : l'exponentiation rapide (approfondissement 1) calcule $a^b \bmod m$ en peu de multiplications, en réduisant modulo m à chaque étape, sans jamais manipuler le nombre géant a^b.

5 Approfondissement

★★ **Exercice 6** *Approfondissement 1 : programmer l'exponentiation rapide*

But. Calculer $a^b \bmod m$ avec un nombre de multiplications proportionnel au nombre de chiffres de b.
À quoi ça sert ? C'est ce qui permet à la carte et à la porte de calculer vite même avec d'énormes exposants : c'est l'autre moitié de l'argument de sécurité, et un bel exemple de récursivité qui divise le problème par deux.

Pour calculer $a^b \bmod m$, il n'est pas nécessaire d'effectuer b multiplications : l'**exponentiation rapide** est fondée sur $a^b = \left(a^{b/2}\right)^2$ si b est pair et $a^b = \left(a^{(b-1)/2}\right)^2 \times a$ si b est impair.

1. À la main : calculer $2^{11} \bmod 29$ en n'utilisant que des carrés et des multiplications par 2 (partir de $2^{11} = (2^5)^2 \times 2$). Combien de multiplications a-t-on faites ? Combien en fallait-il avec la méthode « une multiplication par tour » ?

2. Écrire une fonction **récursive** `puissance_rapide(a, b, m)` qui renvoie $a^b \bmod m$. La comparer à `transformer(a, b)` (avec `m = 20201227`) sur de nombreuses valeurs. ▷ coup de pouce

▷ début de solution

3. Combien d'appels récursifs pour $b = 10^{600}$? Quelle est la complexité en fonction de b ?

★★★ **Exercice 7** *Approfondissement 2 : le logarithme discret plus vite (pas de bébé, pas de géant)*

But. Retrouver un nombre de tours en environ $\sqrt{p}$ opérations au lieu de p, grâce à un dictionnaire.

À quoi ça sert ? On échange du temps de calcul contre de la mémoire : la recherche devient bien plus rapide sur le défi, et l'on peut se demander si cela suffit à menacer le protocole avec de vrais modulos.

On cherche n tel que $g^n \bmod p = c$, avec $0 \leq n < p$. On pose m un entier tel que $m^2 > p$ et on écrit $n = i \times m + j$ avec $0 \leq i, j < m$.

- **Pas de bébé** : on range dans un dictionnaire les m valeurs $g^j \bmod p$ (clé) avec leur exposant j (valeur).
- **Pas de géant** : on calcule c, puis $c \times g^{-m}$, $c \times g^{-2m}$… (modulo p), jusqu'à tomber sur une clé du dictionnaire. Si $c \times g^{-im} = g^j$, alors $c = g^{im+j}$.

On admet que, p étant premier, $g^{p-1} \bmod p = 1$ (petit théorème de Fermat) : multiplier par g^{-m} revient donc à multiplier par $g^{p-1-m} \bmod p$.

1. À la main, avec $p = 29$, $g = 2$ et $m = 6$: écrire le dictionnaire des pas de bébé. Retrouver le secret de Bob à partir de sa clé publique 18, sachant que $2^{22} \bmod 29 = 5$.
2. Écrire `log_discret(g, cible, p)` et vérifier qu'elle retrouve les nombres de tours de l'exemple inventé de la fiche, puis de vos données. ▷ coup de pouce ▷ début de solution
3. Combien d'opérations pour $p \approx 2 \times 10^7$? Et pour $p \approx 10^{600}$? Cela remet-il en cause la sécurité du protocole ? Quelle quantité de mémoire faut-il ?

10

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Une boucle `for` de `nb_tours` tours, avec `valeur = valeur * sujet % 20201227`. Une méthode beaucoup plus rapide, l'exponentiation rapide, est programmée dans l'approfondissement 1. ↑ énoncé

Ex. 4 – piste Ne pas appeler `transformer(7, n)` pour `n = 1, 2, 3...` : chaque appel recommencerait tout depuis le début. Faire **une seule** boucle `while` qui multiplie la valeur par 7 à chaque tour et compte les tours, jusqu'à tomber sur la clé publique. ↑ énoncé

Ex. 4 – début de solution Il suffit de retrouver le secret d'**un seul** des deux appareils : la clé vaut `transformer(cle_publique_porte, tours_carte)`. Le nombre de tours peut dépasser plusieurs millions : c'est normal. ↑ énoncé

Ex. 5 – piste La recherche essaie, au pire, toutes les valeurs du nombre de tours, jusqu'au modulo. La carte, elle, n'a qu'à calculer une puissance, ce que l'exponentiation rapide (approfondissement 1) fait en un nombre d'étapes proportionnel au nombre de *chiffres* de l'exposant. ↑ énoncé

Ex. 6 q. 2 – piste Cas de base : $b = 0$, le résultat est 1. Sinon, calculer **une seule fois** `puissance_rapide(a, b // 2, m)`, l'élever au carré modulo `m`, puis multiplier par `a` si `b` est impair. ↑ énoncé

Ex. 6 q. 2 – début de solution Appeler deux fois la fonction sur `b // 2` (au lieu de stocker le résultat dans une variable) redonnerait une complexité linéaire : bien garder le résultat dans une variable `moitie`. ↑ énoncé

Ex. 7 q. 2 – piste `m = int(p ** 0.5) + 1` ; une boucle `for` remplit le dictionnaire `bebes` ; une boucle `while` fait les pas de géant tant que la valeur courante n'est pas une clé de `bebes` (et que `i < m`). ↑ énoncé

Ex. 7 q. 2 – début de solution Calculer une fois pour toutes `facteur = puissance_rapide(g, p - 1 - m, p)` (approfondissement 1) ; à chaque pas de géant, `geant = geant * facteur % p`. À la sortie, la réponse est `i * m + bebes[geant]`. ↑ énoncé

DÉFI 2 Shuttle Search

les navettes — modulo et efficacité

Jour 13

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t10-defi-aoc-2020-13` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/13`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Des navettes partent du port à intervalles réguliers, chacune avec sa propre période. La partie 1 demande laquelle prendre pour attendre le moins possible : un simple calcul par navette. La partie 2 demande le premier instant où les navettes partent les unes après les autres, minute après minute : la réponse dépasse 10^{14}, si bien que tester les instants un par un prendrait des années, même pour un ordinateur. Il faut une idée.
Ce qu'on réinvestit. L'**arithmétique modulaire** de ce chapitre. Le reste `%` répond à la question « depuis combien de minutes la dernière navette est-elle partie ? », ce qui donne l'attente sans aucune boucle. En partie 2, on raisonne sur des conditions de la forme « $t + i$ est un multiple de b », et l'approfondissement réutilise l'inverse modulaire rencontré avec le chiffrement affine et RSA.

La démarche de la fiche.

1. Vérifier sa compréhension de la règle, puis calculer à la main les attentes sur un petit exemple.
2. Lire le fichier, puis calculer l'attente de chaque navette avec un modulo : réponse de la partie 1.
3. Traduire la condition de la partie 2 avec `%`, puis la résoudre par une recherche à pas croissant (section dédiée).
4. En approfondissement, retrouver le même résultat par la formule des restes chinois.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`shuttle bus`	navette	`timestamp`	instant (en minutes)
`depart`	partir	`earliest`	le plus tôt possible
`out of service`	hors service	`ID`	identifiant (numéro)
`offset`	décalage	`subsequent`	suivant, consécutif

Résumé de l'énoncé (partie 1)

Contexte. Arrivé au port, vous devez rejoindre l'aéroport en navette. Chaque navette fait une boucle de durée fixe et repasse donc régulièrement au port ; son numéro est justement la durée de sa boucle, en minutes. Vous voulez savoir laquelle prendre pour attendre le moins.
Les données. Le fichier contient deux lignes. Ligne 1 : l'instant a (en minutes) à partir duquel vous pouvez partir. Ligne 2 : les numéros des navettes séparés par des virgules ; un `x` désigne une navette hors service, à ignorer dans la partie 1.
La règle.

- Toutes les navettes sont parties ensemble à l'instant 0 : la navette numéro b part donc aux instants 0, b, $2b$, $3b$… autrement dit aux multiples de b.
- Pour chaque navette, on cherche son premier départ à partir de l'instant a ; un départ à l'instant a lui-même convient (attente nulle).
- L'attente est la différence entre ce départ et a. On retient la navette dont l'attente est la plus courte.

La réponse attendue. Le numéro de cette navette multiplié par le nombre de minutes d'attente. Par exemple, avec $a = 10$ et les navettes 3 et 7, la navette 3 part à 12 (attente 2) et la navette 7 à 14 (attente 4) : la réponse serait $3 \times 2 = 6$.

Exercice 1  *Comprendre ce qui est demandé*

But. Vérifier sur trois questions courtes qu'on a compris comment se calcule l'attente.
À quoi ça sert ? Le cas où a est un multiple du numéro (attente nulle, et non égale au numéro) est le piège principal de la partie 1.

Questions de vérification, à traiter sur le cahier :

1. Si a est un multiple de b, combien de minutes attend-on la navette b ?
2. Avec $a = 30$, à quel instant part la navette 7, et combien attend-on ?
3. Les `x` jouent-ils un rôle dans la partie 1 ?

Exercice 2 *À la main, sur un petit exemple*

But. Calculer à la main le prochain départ et l'attente de chaque navette.
À quoi ça sert ? On obtient un exemple dont on **connaît la réponse** (14) ; le même fichier servira d'exemple pour la partie 2.

On considère le fichier (inventé) suivant :

```
96
5,x,7,x,x,11
```

Pour chaque navette, calculer le premier départ à partir de l'instant 96 et l'attente correspondante. Vérification : la réponse est `14`.

2 Programmer la partie 1

Exercice 3 *Lire le fichier*

But. Extraire du fichier l'instant d'arrivée et, pour chaque navette en service, son numéro et sa position dans la ligne.
À quoi ça sert ? La partie 1 n'utilise que les numéros ; la position comptera en partie 2, où les `x` occupent une place : en la gardant dès maintenant, on n'aura pas à relire le fichier.

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Écrire une fonction `lire_notes(nom_fichier)` qui renvoie l'instant d'arrivée (un entier) et la liste des couples `(position, numero)` des navettes en service (deux entiers), la position étant l'indice du numéro dans la ligne, `x` compris. Par exemple, pour la ligne `5,x,7`, on obtient `[(0, 5), (2, 7)]`.

▷ coup de pouce

Exercice 4 *L'attente, avec un modulo*

But. Calculer l'attente d'une navette par un seul calcul de reste, puis choisir la meilleure.
À quoi ça sert ? Le modulo évite de faire défiler les minutes une à une ; c'est le réflexe qui rendra possible la partie 2, où les instants sont gigantesques.

Écrire une fonction `attente(arrivee, numero)` qui renvoie le nombre de minutes à attendre la navette `numero`, **sans boucle**. En déduire la réponse. Tester sur l'exemple inventé de la fiche, puis sur vos données.

▷ coup de pouce

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. La compagnie propose un concours : trouver le moment où les navettes partent les unes après les autres, minute après minute, dans l'ordre de la liste. La première ligne du fichier ne sert plus.
La nouvelle règle.

- Les positions dans la liste sont numérotées à partir de 0, `x` compris : les `x` ne sont plus ignorés, ils occupent une position.
- Un `x` n'impose aucune condition.
- La navette en position i doit partir exactement à l'instant $t + i$, et cela pour toutes les navettes en service.

La réponse attendue. Le plus petit instant t qui convient. Par exemple, avec la liste `3,x,5`, il faut que 3 parte à t et 5 à $t + 2$: $t = 3$ convient (3 est multiple de 3, 5 de 5) et la réponse serait 3.

Exercice 5 *Comprendre la condition*

But. Écrire la condition de la partie 2 avec `%` et la vérifier sur l'exemple.
À quoi ça sert ? Une condition écrite avec `%` se programme directement ; c'est elle que testera la recherche de l'exercice suivant pour chaque navette.

Traduire la condition sur une navette de numéro b placée en position i par une égalité avec `%`. Sur l'exemple inventé de la fiche, on doit trouver $t = 215$: le vérifier pour chacune des trois navettes.

★★★ Exercice 6 *Défi : la partie 2 sur vos données*

But. Trouver le premier instant qui convient, en augmentant le pas à chaque navette satisfaite.
À quoi ça sert ? La force brute est impossible ici : cette méthode, expliquée dans la section suivante, ne fait que quelques centaines d'essais au lieu de 10^{14}. Elle donne la réponse de la partie 2.

L'énoncé prévient que la réponse dépasse 10^{14} : tester tous les instants un par un prendrait des années. Lire la section suivante, puis écrire une fonction `premier_instant(navettes)`, où `navettes` est la liste de couples renvoyée par `lire_notes`, qui suit cette méthode. Vérifier sur l'exemple inventé de la fiche ($t = 215$) et sur les exemples donnés sur le site, avant vos données.

▷ coup de pouce ▷ début de solution

4 Pour y arriver : synchroniser des périodes [au-delà du programme]

Recherche à pas croissant

On cherche t tel que $t \,\%\, 5 = 0$, $(t + 2) \,\%\, 7 = 0$ et $(t + 5) \,\%\, 11 = 0$ (exemple inventé de la fiche).
Idée. Si t convient pour la navette 5, alors $t + 5$, $t + 10\ldots$ conviennent aussi : la condition est *périodique*. Si t convient pour 5 *et* 7, alors $t + 35$, $t + 70\ldots$ conviennent encore pour les deux, puisque 35 est un multiple de 5 et de 7.
Méthode. On part de $t = 0$ avec un pas de 1. Pour chaque navette, on avance de *pas* en *pas* jusqu'à ce qu'elle soit satisfaite, puis on multiplie le pas par son numéro : le pas est toujours le produit des périodes déjà satisfaites.

- navette 5 : $t = 0$ convient tout de suite ; le pas devient 5 ;
- navette 7 : on teste 0, puis 5 ; $5 + 2 = 7$ est divisible par 7 ; le pas devient $5 \times 7 = 35$;
- navette 11 : on teste 5, 40, 75, 110, 145, 180, 215 ; $215 + 5 = 220 = 11 \times 20$. Donc $t = 215$.

Il a suffi de 10 essais en tout $(1 + 2 + 7)$ au lieu de 215. Pour chaque navette, on fait au plus

« numéro » essais : quelques centaines en tout, même si t dépasse 10^{14}.
Pourquoi ça marche. Les numéros des navettes sont premiers entre eux (ici ce sont même des nombres premiers) : en avançant de pas en pas, on finit toujours par satisfaire la nouvelle navette en moins de « numéro » essais. Si les numéros avaient un diviseur commun, il faudrait prendre leur PPCM.
Pour aller plus loin. En mathématiques, ce problème est résolu par le *théorème des restes chinois* : si $n_1, \dots, n_k$ sont premiers entre eux deux à deux, il existe une unique solution t entre 0 et $n_1 \times \cdots \times n_k - 1$ à un système de conditions $t \,\%\, n_j = r_j$. La méthode ci-dessus est une façon simple de la calculer.

5 Approfondissement

Méthode : l'inverse modulaire (par une boucle)

Comme dans le cours (chiffrement affine, RSA), l'**inverse de a modulo m** est l'entier y entre 0 et $m-1$ tel que $a \times y \bmod m = 1$. Il n'existe que si a et m sont premiers entre eux. Quand m est petit (ici, un numéro de navette), on peut le chercher en essayant $y = 1, 2, 3 \dots$

```python
def inverse_mod(a, m):
    """Renvoie l'inverse de a modulo m, ou None s'il n'existe pas."""
    a = a % m
    for y in range(1, m):
        if a * y % m == 1:
            return y
    return None

print(inverse_mod(6, 7))          # 6 : 6 * 6 = 36 = 5 * 7 + 1
print(6 * inverse_mod(6, 7) % 7)  # 1
print(inverse_mod(55, 7))         # 6 : 55 % 7 = 6, on se ramene au cas precedent
```

Intérêt. Au plus $m-1$ essais : c'est instantané pour les numéros de navettes (quelques centaines). Pour les très grands nombres de RSA, on utilise plutôt l'algorithme d'Euclide étendu (TP du chapitre 10, partie D), dont le nombre d'étapes est proportionnel au nombre de chiffres.

★★★ **Exercice 7** *Approfondissement 1 : la formule des restes chinois*

But. Calculer directement l'instant cherché par la formule des restes chinois, avec l'inverse modulaire.
À quoi ça sert ? On relie le défi au cours (inverse modulaire du chiffrement affine et de RSA) et l'on dispose d'une seconde méthode pour contrôler la réponse de la partie 2.

La section précédente cite le théorème des restes chinois. Il donne aussi une **formule** : pour le système $t \,\%\, n_j = r_j$ $(j = 1, \dots, k)$, avec des n_j premiers entre eux deux à deux, on pose $N = n_1 \times \cdots \times n_k$, $N_j = N/n_j$ et y_j l'inverse de N_j modulo n_j. Alors

$$t = \Big(\sum_j r_j \, N_j \, y_j \Big) \bmod N.$$

1. Recopier et exécuter le code de l'encadré. Que renvoie `inverse_mod(4, 6)` ? Pourquoi ?
2. Pour une navette de numéro b en position i, quel reste r doit avoir t modulo b ? Sur l'exemple inventé de la fiche (`5,x,7,x,x,11`), calculer pour chaque navette r_j, N_j et y_j, puis t. On doit retrouver `215`.
3. Vérifier sur l'exemple que le terme associé à la navette 7 est un multiple de 5 et de 11, et qu'il a le bon reste modulo 7. En déduire pourquoi la formule fonctionne.

4. Écrire une fonction `restes_chinois(navettes)` et la comparer à `premier_instant` : nombre d'opérations, lisibilité. ▷ coup de pouce ▷ début de solution

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Après `strip`, `ligne.split(",")` découpe la seconde ligne ; on parcourt les morceaux par leur indice et on ne garde que ceux qui sont différents de `"x"`, convertis avec `int`. ↑

Ex. 4 – piste `arrivee % numero` donne le temps écoulé depuis le dernier départ. Attention au cas où il vaut `0` : on part tout de suite. ↑énoncé

Ex. 6 – piste On traite les navettes une par une. Une fois une navette satisfaite, on ne regarde plus que des instants qui la satisfont encore. ↑énoncé

Ex. 6 – début de solution Deux variables suffisent : `t` (candidat actuel) et `pas`. Pour chaque navette, on ajoute `pas` à `t` tant que la condition n'est pas vérifiée, puis on met à jour `pas`. ↑énoncé

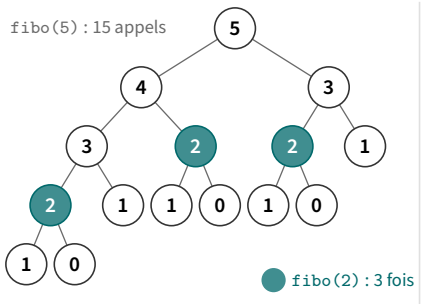
Ex. 7 q. 4 – piste `(-i) % b` donne un reste compris entre 0 et `b - 1`, même quand `-i` est négatif. ↑

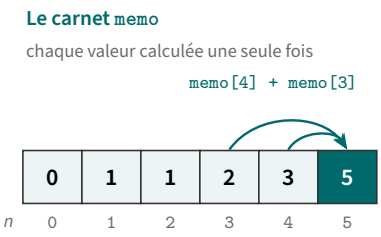
Ex. 7 q. 4 – début de solution Une première boucle calcule le produit N ; une seconde ajoute `r * (N // b) * inverse_mod(N // b, b)` pour chaque navette ; on renvoie le total modulo N. ↑énoncé

11

CHAPITRE 11

Programmation dynamique





DANS CE CHAPITRE

- Activités préparatoires 528
- Cours 531
- Exercices 543
- TP et projets 558
- Fiche de révision 570
- Défis Advent of Code 572
- Corrigés 955

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Le calcul qui n'en finit pas

▶ corrigé p. 955

chronométrer, dessiner, compter… puis prendre des notes

Durée : 30 min · Par deux

Consignes

- L'exercice 1 se fait sur machine (console ou éditeur en ligne), les suivants sur le cahier.
- On rappelle la suite de Fibonacci : $F_0 = 0$, $F_1 = 1$, et chaque terme est la somme des deux précédents : $F_n = F_{n-1} + F_{n-2}$.

Exercice 1 *Chronométrer*

Recopier et lancer le programme suivant, qui calcule F_n en suivant mot pour mot la définition, et affiche le temps de calcul (en secondes) pour $n = 20, 23, 26, \ldots, 35$.

```
import time

def fibo(n):
    if n <= 1:
        return n
    return fibo(n - 1) + fibo(n - 2)

for n in range(20, 36, 3):
    debut = time.time()
    fibo(n)
    print(n, time.time() - debut)
```

1. Recopier sur le cahier le tableau suivant et le compléter avec les temps obtenus.

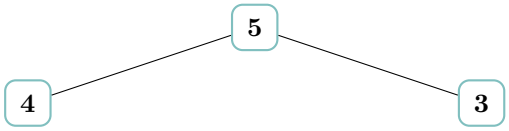
n	20	23	26	29	32	35
temps (s)						

2. Quand n augmente de 3, par combien le temps est-il à peu près multiplié ?

3. En déduire une estimation du temps pour $n = 38$, puis pour $n = 50$. Le programme est-il utilisable ?

Exercice 2 *Dessiner les appels*

4. Pour comprendre, on dessine l'**arbre des appels** de `fibo(5)` : sous chaque appel, on dessine les deux appels qu'il déclenche (un appel `fibo(0)` ou `fibo(1)` n'en déclenche aucun). Recopier et compléter sur le cahier l'arbre commencé ci-dessous.



5. Combien d'appels à `fibo` en tout (en comptant l'appel `fibo(5)`) ?
6. Combien de fois `fibo(3)` est-il calculé ? `fibo(2)` ? `fibo(1)` ?
7. Entourer, sur votre dessin, un **sous-arbre** qui apparaît plusieurs fois à l'identique. Que fait l'ordinateur à chaque fois qu'il le rencontre ?

Exercice 3 *Compter sans dessiner*

On note $A(n)$ le nombre total d'appels déclenchés par `fibo(n)`. Ainsi $A(0) = A(1) = 1$.

8. Expliquer pourquoi $A(n) = 1 + A(n-1) + A(n-2)$ pour $n \geqslant 2$.
9. Recopier et compléter le tableau (vérifier que $A(5)$ correspond à votre dessin).

n	0	1	2	3	4	5	6	7	8
$A(n)$	1	1							

10. Par combien $A(n)$ est-il à peu près multiplié quand n augmente de 1 ? Et de 3 ? Comparer avec l'exercice 1.

Exercice 4 *Imaginer un carnet*

On reprend le calcul de `fibo(6)`, mais on dispose d'un **carnet** où l'on note chaque résultat dès qu'on l'a calculé. Nouvelle règle : *avant* de calculer `fibo(k)`, on regarde dans le carnet ; si la valeur y est, on la recopie sans rien recalculer.

11. Dérouler le calcul de `fibo(6)` avec le carnet. Dans quel ordre les valeurs sont-elles écrites dans le carnet ?
12. Combien d'additions a-t-on faites ? Sans carnet, `fibo(6)` en ferait 12. Et pour `fibo(40)`, avec carnet ?
13. Autre idée, sans aucun appel : recopier puis remplir directement le tableau des valeurs, de gauche à droite, chaque case étant la somme des deux précédentes.

k	0	1	2	3	4	5	6	7	8	9	10
F_k	0	1									

Exercice 5  *Mettre des mots*

14. Recopier et compléter les phrases suivantes avec vos mots.

- La version de l'exercice 1 est lente parce que…
- Le carnet permet de…
- Avec le carnet, on part de… et on descend ; avec le tableau, on part de… et on remonte.

15. Proposer un nom pour l'idée « noter un résultat pour ne pas le recalculer ».

COURS

Programme (BO) *« Programmation dynamique. Utiliser la programmation dynamique pour écrire un algorithme. »* Commentaire du BO : *« Les exemples de l'alignement de séquences ou du rendu de monnaie peuvent être présentés. La discussion sur le coût en mémoire peut être développée. »*

Prérequis la **récursivité** (absolument indispensable), la notion de **coût** d'un algorithme (linéaire, quadratique, exponentiel), les **listes** et les **dictionnaires**, et le souvenir des **algorithmes gloutons** de Première. → fiches de révision : ch. 1 p. 58
→ rappels de 1re : listes p. 1057, dictionnaires p. 1058, parcours et coût p. 1059

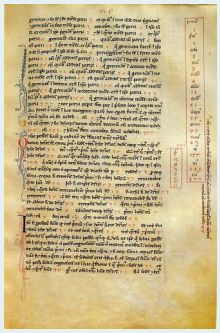
Objectifs *reconnaître* un problème qui se prête à la programmation dynamique (sous-problèmes qui *se recouvrent*) ; *mémoïser* une fonction récursive (*top-down*) ; *tabuler* un problème de bas en haut (*bottom-up*) ; *comprendre* que ce sont deux faces d'une même idée ; l'*appliquer* au rendu de monnaie, au sac à dos, à la comparaison de deux mots.

Situation de départ

1. Le calcul récursif de Fibonacci du chapitre 1 met environ une heure pour `fibo(50)`. Peut-on le rendre instantané, sans changer la formule ?
2. Pour rendre 6 euros avec des pièces de 1, 3 et 4 euros, prendre toujours la plus grosse pièce possible donne-t-il le moins de pièces ? Comment être sûr d'avoir le minimum ?
3. Un randonneur ne peut pas porter plus de 10 kg et hésite entre plusieurs objets, chacun avec son poids et son utilité. Faut-il essayer toutes les combinaisons pour trouver le meilleur chargement ?

I Le problème : quand la récursivité recalcule mille fois la même chose

Un peu d'histoire. Cette suite doit son nom à **Leonardo Fibonacci** (Léonard de Pise), qui la fait connaître en Europe en **1202** dans son *Liber abbaci* (« Livre du calcul »), l'ouvrage qui popularise aussi les chiffres indo-arabes. Elle y répond à un problème de **lapins** (combien de couples au bout d'un an ?), dont la page ci-contre, copie médiévale conservée à Florence, porte les réponses en marge : $1, 2, 3, 5, 8, \ldots, 377$.

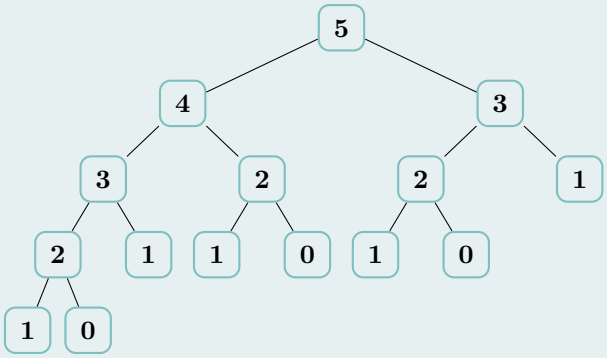


Le problème des lapins dans le Liber abbaci

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$ et $F_n = F_{n-1} + F_{n-2}$. Sa version récursive naïve, déjà rencontrée au chapitre 1, recopie la définition :

```python
def fibo(n):
    if n <= 1:                          # CAS DE BASE
        return n
    return fibo(n - 1) + fibo(n - 2)    # APPEL RECURSIF (deux fois)
```

Ce code est *juste*, mais son temps de calcul explose : il est multiplié par environ 1,6 chaque fois que n augmente de 1 (environ 2 secondes pour `fibo(35)`, une dizaine pour `fibo(38)`, et de l'ordre d'une heure pour `fibo(50)`). L'**arbre des appels** de `fibo(5)` montre pourquoi :



`fibo(2)` y est calculé **trois** fois, `fibo(3)` deux fois, chacun relançant tout son sous-arbre : 15 appels pour `fibo(5)`, plus de 300 millions pour `fibo(40)`. Le nombre d'appels, et donc le temps, croît comme $1{,}6^n$: le coût est **exponentiel**.

Règle 1 — Le constat qui lance tout le chapitre

La récursivité naïve est parfois catastrophique parce qu'elle **recalcule sans cesse les mêmes sous-problèmes**. L'idée de toute la programmation dynamique tient en une phrase : **ne jamais calculer deux fois la même chose**. On *retient* le résultat d'un sous-problème la première fois, et on le *réutilise* ensuite.

↻ **Retour sur l'activité.** Dans l'activité, le temps de `fibo` était multiplié par environ 4 chaque fois que n augmentait de 3 (car $1{,}6^3 \approx 4$), et l'arbre de `fibo(5)` comptait 15 appels, dont trois calculs de `fibo(2)` : ce recalcul rend le programme inutilisable pour $n = 50$.

Ce fil rouge — « garder pour ne pas refaire » — va traverser tout le chapitre. Il existe **deux façons** de l'appliquer, et le programme demande de les connaître toutes les deux.

II Première façon : la mémoïsation (*top-down*)

Mémoïser, c'est garder la fonction récursive telle qu'elle est — on part « du haut », du problème complet — mais lui adjoindre un **carnet de notes** où l'on inscrit chaque résultat déjà calculé. Avant de se lancer dans un calcul, la fonction consulte le carnet : si la réponse y est, elle la lit ; sinon, elle la calcule *et l'écrit*.

Définition 1 — Mémoïsation

La **mémoïsation** consiste à munir une fonction récursive d'un **dictionnaire** (le « carnet ») qui associe à chaque argument déjà rencontré le résultat correspondant. On dit que l'on résout le problème **de haut en bas** (*top-down*) : on garde la structure récursive descendante, mais chaque sous-problème n'est **résolu qu'une seule fois**.

↻ **Retour sur l'activité.** Le « carnet » imaginé dans l'activité était exactement ce dictionnaire : avec lui, `fibo(6)` ne demande plus que 5 additions au lieu de 12, et `fibo(40)` seulement 39.

```
1  def fibo(n, memo={}):
2      if n <= 1:                                  # CAS DE BASE
3          return n
4      if n in memo:                               # VERIFIER : deja calcule, on lit le
                                                   # carnet
5          return memo[n]
6      resultat = fibo(n - 1, memo) + fibo(n - 2, memo)   # APPEL RECURSIF
7      memo[n] = resultat                          # METTRE A JOUR : on ecrit dans le
                                                   # carnet
8      return resultat                             # RENVOYER
```

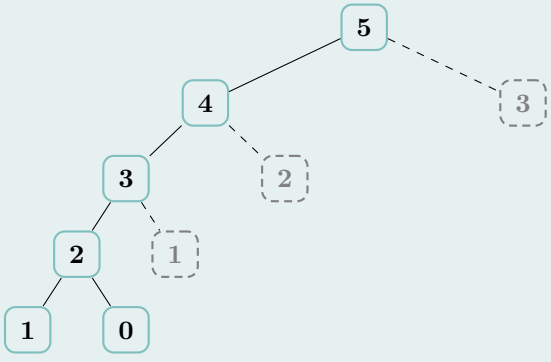
Les commentaires en majuscules nomment les temps de toute fonction mémoïsée, toujours les mêmes : `CAS DE BASE` ; `VERIFIER` si le résultat est déjà dans le carnet ; sinon `APPEL RECURSIF` ; puis `METTRE A JOUR` le carnet avant de `RENVOYER`.

▷ **Je prédis.** Dans une session neuve, on appelle `fibo(5)`. Que contient ensuite le carnet `memo` ?

▷ **Je vérifie.** `{2: 1, 3: 2, 4: 3, 5: 5}`. Les cas de base `0` et `1` n'y sont jamais écrits : ils sont renvoyés avant. Le premier résultat noté est celui de `fibo(2)`, le plus profond.

Avec cette seule modification, `fibo(50)` est **instantané**, et même `fibo(500)` passe sans effort. Chaque valeur F_k n'est calculée qu'une fois : il y a n sous-problèmes distincts, donc le coût devient **linéaire**, $O(n)$ — on passe d'un coût exponentiel (environ $1{,}6^n$, que l'on majore souvent par 2^n) à n, l'un des plus beaux gains de tout le cours.

Sur l'arbre des appels, la mémoïsation **coupe** toutes les branches redondantes : la deuxième fois qu'une valeur est demandée, on la *lit dans le carnet* au lieu de relancer tout son sous-arbre.



Arbre de `fibo(5)` *mémoïsé* : les nœuds en pointillés gris `3` et `2`, déjà calculés, sont **lus dans le carnet** — leurs sous-arbres ne sont jamais recalculés ; le `1` en pointillés est un cas de base, renvoyé directement (`n <= 1`). Comparez avec l'arbre complet du début !

Remarque

Le paramètre `memo={}` par défaut est pratique mais c'est un **piège classique** : en Python, un argument par défaut mutable est **partagé** entre tous les appels (voir le chapitre 1). Ici cela nous arrange (le carnet survit d'un appel à l'autre), mais dans un autre contexte cela cause des bugs sournois. Une version plus sûre passe le dictionnaire explicitement, ou l'initialise à `None`.

► **Exercices d'application :** exercices 1 à 3 (mémoïsation)

III Seconde façon : la tabulation (*bottom-up*)

L'autre stratégie renverse le point de vue. Plutôt que de partir du problème complet et de descendre, on **part des plus petits cas** et on **remonte** en remplissant un tableau, jusqu'à atteindre la valeur cherchée. Il n'y a alors *plus aucune récursivité* : juste une boucle et un tableau.

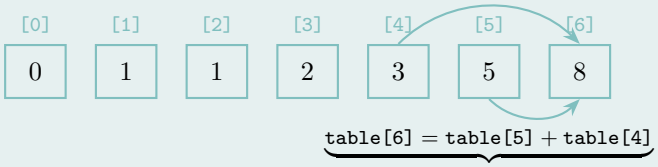
Définition 2 — Tabulation

La **tabulation** consiste à résoudre les sous-problèmes **du plus petit au plus grand** en rangeant leurs résultats dans un **tableau** (une « table »). On dit que l'on résout le problème **de bas en haut** (*bottom-up*) : chaque case du tableau se calcule à partir de cases déjà remplies.

↻ **Retour sur l'activité.** Le tableau rempli de gauche à droite à la fin de l'activité $(0, 1, 1, 2, 3, 5, 8, \ldots, 55)$ était une tabulation : on part des petits cas et l'on remonte, sans aucun appel récursif.

```python
def fibo(n):
    if n <= 1:                                  # CAS DE BASE
        return n
    table = [0] * (n + 1)                       # INITIALISER : table[k] contiendra F_k
    table[0] = 0                                # INITIALISER : les cas de base
    table[1] = 1                                # INITIALISER
    for k in range(2, n + 1):                   # PARCOURIR : du plus petit au plus grand
        table[k] = table[k - 1] + table[k - 2]  # METTRE A JOUR : cases deja calculees
    return table[n]                             # RENVOYER
```

Ici encore le coût est **linéaire**. On construit `table[2]`, puis `table[3]`, etc. : quand on calcule `table[k]`, les deux cases dont on a besoin sont *déjà là*. On avance en terrain conquis.



Tabulation : on remplit de gauche à droite. Chaque nouvelle case ne lit que des cases *déjà calculées* à sa gauche.

Exemple

Déroulons pour $n = 6$. On remplit la table de gauche à droite :

k	0	1	2	3	4	5
F_k	0	1	1	2	3	5

$F_6 = F_5 + F_4 = 5 + 3 = 8$. Chaque case n'a lu que ses deux voisines de gauche.

► **Exercices d'application :** exercices 4 à 8 (tabulation)

IV Deux faces d'une même pièce

Mémoïsation et tabulation résolvent **le même problème** avec **la même idée** (ne rien recalculer), mais dans deux sens opposés. Il faut savoir les distinguer et passer de l'une à l'autre.

	Mémoïsation (*top-down*)	**Tabulation (*bottom-up*)**
Sens	du problème complet vers les petits cas	des petits cas vers le problème complet
Forme du code	une fonction **récursive** + un dictionnaire	une **boucle** + un tableau
Ce qu'on calcule	**seulement** les sous-problèmes utiles	**tous** les sous-problèmes jusqu'à la cible
Atout	naturel si on part de la formule récursive	pas de récursivité, pas de pile d'appels qui déborde

Méthode — mémoïser ou tabuler ?

Aucune des deux n'est « meilleure » dans l'absolu. La mémoïsation est souvent la plus facile à écrire quand on tient déjà la formule récursive ; la tabulation évite le risque de « dépassement de la profondeur de récursion » et se prête bien quand on a besoin de *toutes* les valeurs intermédiaires. Un bon réflexe de contrôle : **écrire la relation de récurrence**, puis choisir l'un des deux emballages.

V Un cas d'école : le rendu de monnaie

Vous devez rendre une somme avec le **moins de pièces possible**, en piochant dans un système de valeurs, par exemple $\{1, 2, 5\}$. En Première, vous avez vu l'algorithme **glouton** : prendre à chaque fois la plus grosse pièce possible. C'est rapide, mais…

Propriété 1 — Le glouton peut se tromper

Avec le système $\{1, 3, 4\}$, pour rendre 6, le glouton prend 4, puis 1, puis 1 : **trois** pièces. Or $3 + 3$ suffit : **deux** pièces. Le glouton n'est **pas toujours optimal**. Pour être *sûr* d'avoir le minimum, il faut explorer toutes les possibilités… intelligemment.

Posons la **relation de récurrence**. Notons $R(s)$ le nombre minimal de pièces pour rendre la somme s. Pour rendre s, on choisit une première pièce p (avec $p \leq s$), et il reste alors à rendre $s - p$ de façon optimale. On essaie *toutes* les premières pièces et on garde la meilleure :

$$R(0) = 0, \qquad R(s) = 1 + \min_{p \leq s} R(s - p).$$

Les sous-problèmes $R(s - p)$ **se recouvrent** massivement (rendre 4 intervient pour rendre 5, 6, 8…) : c'est le signal de la programmation dynamique.

Version mémoïsée (top-down).

```
def rendu(somme, pieces, memo=None):
    if memo is None:                  # INITIALISER (premier appel)
        memo = {}
    if somme == 0:                    # CAS DE BASE
        return 0
    if somme in memo:                 # VERIFIER : deja calcule
        return memo[somme]
    meilleur = float("inf")           # INITIALISER : +infini, aucune solution
```

```
    for p in pieces:                    # PARCOURIR : chaque premiere piece
        if p <= somme:                  # TESTER
            meilleur = min(meilleur, 1 + rendu(somme - p, pieces, memo))    # APPEL RECURSIF
    memo[somme] = meilleur              # METTRE A JOUR : le carnet
    return meilleur                     # RENVOYER
```

`float("inf")` est l'infini de Python, plus grand que tout nombre : la première solution trouvée le remplace toujours (voir l'encadré « Outil Python : l'infini » du projet Dijkstra, chapitre 8).

▷ **Je prédis.** Que renvoient `rendu(7, [2, 5])` et `rendu(3, [2, 5])` ?

▷ **Je vérifie.** 2 ($5+2$), puis `inf` : 3 ne peut pas se rendre avec des pièces de 2 et de 5, `meilleur` reste l'infini (et `1 + inf` vaut encore `inf`).

Version tabulée (bottom-up). On remplit un tableau `table[s]` $= R(s)$ pour s allant de 0 à la somme voulue :

```
def rendu(somme, pieces):
    table = [0] + [float("inf")] * somme                # INITIALISER : table[0] = 0
    for s in range(1, somme + 1):                       # PARCOURIR : sommes croissantes
        for p in pieces:                                # PARCOURIR : premieres pieces
            if p <= s and table[s - p] + 1 < table[s]:  # TESTER : mieux avec p ?
                table[s] = table[s - p] + 1             # METTRE A JOUR
    return table[somme]                                 # RENVOYER
```

Exemple

Système $\{1, 3, 4\}$, rendre 6. La table se construit de $s = 0$ à $s = 6$:

s	0	1	2	3	4	5	6
$R(s)$	0	1	2	1	1	2	**2**

On lit $R(6) = 2$ (c'est $3+3$), là où le glouton donnait 3. La programmation dynamique a trouvé le *vrai* minimum.

► **Exercices d'application :** exercices 9 à 11 (le rendu de monnaie)

VI Un tableau à deux dimensions : le sac à dos

Certains problèmes ont besoin d'une table à **deux entrées**. Le plus célèbre est le **sac à dos** : un sac supporte un poids maximal C (sa *capacité*) ; chaque objet i a un poids w_i et une valeur v_i ; on veut **maximiser la valeur totale** emportée sans dépasser C.

L'idée : on considère les objets un par un. Notons $T[i][c]$ la meilleure valeur atteignable avec les i premiers objets et une capacité c. Pour l'objet i, deux choix :

- **le laisser** : on garde $T[i-1][c]$;
- **le prendre** (si $w_i \leq c$) : on gagne v_i et il reste $c - w_i$, soit $v_i + T[i-1][c-w_i]$.

On garde le meilleur des deux :

$$T[i][c] = \max\big(\ T[i-1][c],\ \ v_i + T[i-1][c-w_i]\ \big).$$

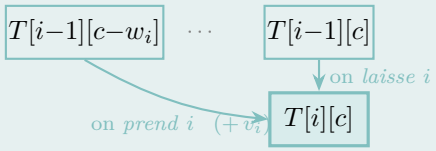


Table 2D : chaque case de la ligne i se calcule à partir de **deux cases de la ligne** $i-1$ (celle juste au-dessus, et celle décalée de w_i vers la gauche).

```python
def sac_a_dos(poids, valeurs, capacite):
    n = len(poids)
    # T[i][c] : meilleure valeur avec les i premiers objets, capacite c
    T = [[0] * (capacite + 1) for _ in range(n + 1)]   # INITIALISER : ligne 0 a 0
    for i in range(1, n + 1):                          # PARCOURIR : les objets
        for c in range(capacite + 1):                  # PARCOURIR : les capacites
            T[i][c] = T[i - 1][c]                      # METTRE A JOUR : on laisse l'objet i
            if poids[i - 1] <= c:                      # TESTER : peut-on le prendre ?
                avec = valeurs[i - 1] + T[i - 1][c - poids[i - 1]]
                if avec > T[i][c]:                     # TESTER : mieux en le prenant ?
                    T[i][c] = avec                     # METTRE A JOUR
    return T[n][capacite]                              # RENVOYER
```

▷ **Je prédis.** Que renvoie `sac_a_dos([2, 3, 4], [3, 4, 6], 5)` ?
▷ **Je vérifie.** 7 : les objets de poids 2 et 3 ($3+4$). Prendre d'abord l'objet le plus précieux (valeur 6, poids 4) ne laisserait de place pour rien d'autre.

Remarque
Le sac à dos « entier » (on prend un objet ou pas, jamais un morceau) n'a **pas** de bon algorithme glouton : c'est justement ce qui le rend intéressant. Sa version dynamique remplit une table de taille $n \times C$, donc en coût $O(n \times C)$ — bien mieux que d'essayer les 2^n sous-ensembles d'objets.

Pour les curieux — *Une nuance importante* (facultatif)
Ce coût est polynomial en la **valeur** de C, mais pas en sa **taille**. Écrite en binaire avec b bits, la capacité peut valoir jusqu'à 2^b : la table a alors de l'ordre de $n \times 2^b$ cases. Chaque bit ajouté à C double la table : le coût est **exponentiel** en la taille de C. On parle de coût **pseudo-polynomial** : l'algorithme est excellent pour une capacité de quelques milliers, mais inutilisable pour une capacité de 30 chiffres. On verra au chapitre *Calculabilité* pourquoi cette nuance compte.

Remarque — *Le coût en mémoire*
La programmation dynamique gagne du temps en **dépensant de la mémoire** : la table de `fibo` a $n+1$ cases, celle du sac à dos $(n+1) \times (C+1)$. On peut souvent réduire cette mémoire en ne gardant que ce qui sert encore. Pour `fibo`, chaque case ne lit que les deux précédentes : **deux variables** suffisent (mémoire constante, voir le corrigé de l'exercice 5). Pour le sac à dos, chaque ligne ne lit que la ligne du dessus : **deux lignes** de $C+1$ cases suffisent si l'on ne veut que la meilleure valeur — mais on ne peut plus alors « remonter » la table pour retrouver les objets choisis.

► **Exercices d'application :** exercices 12 **et** 13 (les chemins d'un robot, le sac à dos)

VII Comparer deux mots : la plus longue sous-séquence commune

Dernier grand classique, très présent au bac et utilisé en vrai (correcteurs orthographiques, `diff` de deux fichiers, comparaison d'ADN). Une **sous-séquence** d'un mot, c'est ce qu'il reste en *effaçant* des lettres sans changer l'ordre des autres. On cherche la plus longue sous-séquence commune (PLSC) à deux mots.

Notons $L[i][j]$ la longueur de la PLSC entre les i premières lettres de A et les j premières de B. La récurrence compare les dernières lettres :

$$L[i][j] = \begin{cases} L[i-1][j-1] + 1 & \text{si } A[i-1] = B[j-1] \quad \text{(on garde la lettre)} \\ \max\left(L[i-1][j],\ L[i][j-1]\right) & \text{sinon.} \end{cases}$$

```python
def plsc(a, b):
    n, m = len(a), len(b)
    L = [[0] * (m + 1) for _ in range(n + 1)]            # INITIALISER : ligne et colonne 0 a
                                                          # 0
    for i in range(1, n + 1):                             # PARCOURIR
        for j in range(1, m + 1):                         # PARCOURIR
            if a[i - 1] == b[j - 1]:                      # TESTER : dernieres lettres egales ?
                L[i][j] = L[i - 1][j - 1] + 1             # METTRE A JOUR : on garde la lettre
            else:
                L[i][j] = max(L[i - 1][j], L[i][j - 1])   # METTRE A JOUR
    return L[n][m]                                        # RENVOYER
```

▷ **Je prédis.** Que renvoie `plsc("ABC", "CBA")` ?

▷ **Je vérifie.** `1` : les trois lettres sont communes, mais dans l'ordre inverse ; on ne peut en garder qu'une en respectant l'ordre des deux mots.

mot `B` *(indice j)*

mot `A` *(indice i)*		`N`	`I`	`C`	`H`	`E`
	0	0	0	0	0	0
`C`	0	0	0	**1**	1	1
`H`	0	0	0	1	**2**	2
`I`	0	0	1	1	2	2
`E`	0	0	1	1	2	**3**
`N`	0	1	1	1	2	3

table `L` pour `plsc("CHIEN", "NICHE")`

Exemple

Pour `"CHIEN"` et `"NICHE"`, la plus longue sous-séquence commune est `"CHE"` (longueur 3) : on retrouve C, H, E dans le même ordre dans les deux mots. La table L a 6×6 cases ; on lit la réponse dans le coin en bas à droite, $L[5][5] = 3$. Dans la table ci-dessus, les cases **foncées** sont les lettres égales ($L[i-1][j-1]+1$) ; avec les cases **claires**, elles forment le chemin de remontée depuis $L[5][5]$, qui fait relire `C`, `H`, `E`.

11

► **Exercices d'application :** exercice 14 (comparer deux mots)

VIII Aller plus loin : la distance d'édition

La plus longue sous-séquence commune mesure une *ressemblance*. Sa cousine mesure une *différence*, et c'est peut-être l'algorithme dynamique le plus utilisé au monde : chaque fois que votre téléphone corrige « langaje » en « langage », chaque fois qu'un logiciel de bio-informatique aligne deux brins d'ADN, chaque fois qu'un `diff` compare deux versions d'un fichier, c'est elle qui travaille.

Définition 3 — Distance d'édition (ou de Levenshtein)

La **distance d'édition** entre deux mots est le **nombre minimal d'opérations élémentaires** pour transformer le premier en le second, les trois opérations autorisées étant : **insérer** une lettre, **supprimer** une lettre, ou **substituer** une lettre par une autre.

Le raisonnement est le même que pour la plus longue sous-séquence commune : on compare les *dernières* lettres. Notons $D[i][j]$ la distance d'édition entre les i premières lettres de A et les j premières de B.

- Transformer un mot en le mot vide coûte autant de **suppressions** que de lettres : $D[i][0] = i$. De même $D[0][j] = j$ (que des insertions).

- Si les dernières lettres sont **égales** ($A[i-1] = B[j-1]$), rien à payer sur ce caractère : $D[i][j] = D[i-1][j-1]$.
- Sinon, on choisit la **moins chère** des trois opérations, chacune ramenant à un sous-problème plus petit :

$$D[i][j] = 1 + \min\big(\underbrace{D[i-1][j]}_{\text{supprimer}}, \underbrace{D[i][j-1]}_{\text{insérer}}, \underbrace{D[i-1][j-1]}_{\text{substituer}}\big).$$

```
def distance(a, b):
    n, m = len(a), len(b)
    D = [[0] * (m + 1) for _ in range(n + 1)]   # INITIALISER
    for i in range(n + 1):
        D[i][0] = i                             # INITIALISER : a -> mot vide, i suppressions
    for j in range(m + 1):
        D[0][j] = j                             # INITIALISER : mot vide -> b, j insertions
    for i in range(1, n + 1):                   # PARCOURIR
        for j in range(1, m + 1):               # PARCOURIR
            if a[i - 1] == b[j - 1]:            # TESTER
                D[i][j] = D[i - 1][j - 1]       # METTRE A JOUR : lettres egales, rien a payer
            else:                               # METTRE A JOUR : la moins chere des trois
                                                # operations
                D[i][j] = 1 + min(min(D[i - 1][j],      # suppression
                                      D[i][j - 1]),     # insertion
                                  D[i - 1][j - 1])      # substitution
    return D[n][m]                              # RENVOYER
```

▷ **Je prédis.** Que renvoient `distance("ECRAN", "ECRIN")` et `distance("ABC", "CBA")` ?
▷ **Je vérifie.** `1` (une substitution, `A`→`I`), puis `2` : on substitue la première et la dernière lettre, le `B` du milieu reste en place.

Exemple

Passons de `"CHIEN"` à `"CHATS"`. On remplit la table D ligne par ligne (la première ligne et la première colonne comptent les lettres) :

	ø	C	H	A	T	S
ø	0	1	2	3	4	5
C	1	0	1	2	3	4
H	2	1	0	1	2	3
I	3	2	1	1	2	3
E	4	3	2	2	2	3
N	5	4	3	3	3	**3**

On lit la distance dans le coin en bas à droite : $D[5][5] = \mathbf{3}$. C'est logique : on garde `CH`, puis on **substitue** `I`→`A`, `E`→`T`, `N`→`S`, soit trois opérations.

En remontant la table (comme pour la plus longue sous-séquence commune), on peut même retrouver la *suite précise* des corrections : c'est exactement ce que fait un correcteur orthographique pour proposer les mots les plus « proches » de celui que vous avez tapé.

► **Exercices d'application :** exercice 15 (la distance d'édition)

IX La méthode générale : reconnaître un problème dynamique

Tous ces exemples partagent la même ossature. C'est elle qu'il faut savoir repérer.

Règle 2 — La recette de la programmation dynamique

Un problème se résout par programmation dynamique quand il réunit **deux ingrédients** :

1. **des sous-problèmes qui se recouvrent** : la solution fait appel plusieurs fois aux mêmes sous-calculs (sinon, mémoïser ne servirait à rien) ;
2. **une sous-structure optimale** : la solution optimale du problème se construit à partir des solutions optimales de ses sous-problèmes.

La démarche est alors toujours la même : **(a)** définir clairement ce que représente un sous-problème ; **(b)** écrire la **relation de récurrence** qui le relie à de plus petits ; **(c)** l'emballer soit en *mémoïsation*, soit en *tabulation*.

Remarque

Gloutons, diviser-pour-régner, programmation dynamique : ne pas confondre. Le *glouton* fait un choix définitif à chaque étape (rapide, mais parfois faux). *Diviser pour régner* découpe en sous-problèmes **indépendants** (comme le tri fusion : les deux moitiés n'ont rien en commun). La *programmation dynamique* s'attaque au cas où les sous-problèmes **se chevauchent** : c'est précisément ce chevauchement qui rend la mémoire payante.

X Ouverture

La programmation dynamique découpe toujours en sous-problèmes, mais quand ils se répètent, elle **mémorise** leurs résultats : elle dépense de la **mémoire** pour gagner du **temps**.

- Au **bac**, on vous demandera surtout d'écrire une **relation de récurrence**, de compléter une version **mémoïsée** ou une version **tabulée** (remplissage d'un tableau), et de comparer avec un algorithme **glouton**.

11

XI Un peu d'histoire : Bellman et un nom choisi pour tromper un ministre

La programmation dynamique est née dans les années 1950 sous la plume du mathématicien américain **Richard Bellman** (1920–1984), à la RAND Corporation. Le mot « programmation » n'a ici rien à voir avec le code : à l'époque, il signifie « planification », l'optimisation d'une suite de décisions (comme la *programmation linéaire*).

Et le mot « dynamique » ? Bellman a raconté, dans son autobiographie (1984), l'avoir choisi… par ruse. Selon son propre récit, le secrétaire à la Défense Charles Wilson, dont dépendaient les crédits de la recherche militaire, « avait une peur et une haine viscérales du mot *recherche* ». Pour faire accepter ses travaux de mathématiques, Bellman cherchait un nom qui sonne concret, impossible à critiquer : « dynamique » avait une allure décidée, positive, et « il était impossible d'employer ce mot dans un sens péjoratif ». La discipline la plus élégante de l'algorithmique devrait donc son nom à une opération de camouflage administratif ! (Les historiens relèvent que Wilson n'est en poste qu'à partir de 1953, alors que Bellman situe l'épisode vers 1950 : racontée trente ans plus tard, l'anecdote est sans doute un peu arrangée.)

Derrière l'anecdote, une idée profonde : le **principe d'optimalité** de Bellman — *une politique optimale est faite de sous-décisions elles-mêmes optimales* — est exactement la « sous-structure

optimale » de notre recette. On la retrouve aujourd'hui partout : dans les GPS, la reconnaissance vocale, l'alignement de séquences d'ADN, et jusque dans l'apprentissage par renforcement des IA qui jouent aux échecs ou au go.

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise :** la *Récursivité* (Fibonacci naïf, exponentiel) et *Diviser pour régner*, prolongé quand les sous-problèmes se recouvrent : la **mémoïsation** dépense de la mémoire pour gagner du temps.
- **Un écho :** le rendu de monnaie, résolu en glouton dans *Les algorithmes gloutons* (Première), trouve ici sa version toujours optimale.
- **Pour aller plus loin :** la *Recherche textuelle* reprendra l'idée « mémoire contre temps » pour chercher un motif exact.

Bilan — la carte mémoire

Notion	À retenir
Idée centrale	ne **jamais recalculer** deux fois le même sous-problème
Quand l'utiliser	sous-problèmes qui **se recouvrent** + sous-structure optimale
Mémoïsation	récursivité + **dictionnaire**, de haut en bas (*top-down*)
Tabulation	boucle + **tableau**, de bas en haut (*bottom-up*)
Étape clé	écrire la **relation de récurrence** avant de coder
Gain typique	de l'**exponentiel** au **linéaire** (Fibonacci : $2^n \to n$)
Rendu de monnaie	$R(s) = 1 + \min_{p \le s} R(s-p)$; le glouton peut se tromper
Sac à dos	table 2D $T[i][c]$: prendre ou laisser l'objet i
Mots	PLSC (ressemblance) et distance d'édition (différence), tables 2D
Ne pas confondre	glouton (choix définitif) / diviser-pour-régner (sous-pb **indépendants**)

Retour à la situation de départ

1. Oui : en **mémoïsant** la fonction (un dictionnaire note chaque résultat) ou en **tabulant** (une boucle remplit un tableau des petits cas vers le grand), chaque F_k n'est calculé qu'une fois : le coût passe d'exponentiel à **linéaire** (§ I à IV).
2. Non : le glouton prend $4 + 1 + 1$, soit trois pièces, alors que $3 + 3$ suffit. La relation de récurrence $R(s) = 1 + \min_{p \le s} R(s-p)$, mémoïsée ou tabulée, donne à coup sûr le minimum (§ V).
3. Non : c'est le problème du **sac à dos**. Une table à deux dimensions $T[i][c]$, remplie en choisissant pour chaque objet de le laisser ou de le prendre, donne la meilleure valeur en $n \times C$ étapes au lieu des 2^n combinaisons (§ VI).

Erreurs fréquentes

- **Croire qu'une fonction récursive suffit.** Sans mémo ni table, Fibonacci naïf est exponentiel : c'est *juste* mais inutilisable. *Le réflexe :* dès qu'on voit les mêmes appels revenir, on mémoïse.
- **Oublier le cas de base.** `R(0)=0`, `table[0]`, `L[0][j]=0`… sans lui, la récurrence tourne dans le vide.
- **Se tromper de sens en tabulant.** On ne peut remplir `table[s]` que si les cases dont il dépend (`table[s-p]`) sont **déjà** calculées : la boucle doit aller du petit vers le grand.
- **Confondre décalage d'indices.** Avec les i premiers objets, l'objet numéro i est `poids[i-1]` en Python (les listes commencent à 0). Source d'erreurs dans le sac à dos et la PLSC.
- **Prendre le glouton pour la programmation dynamique.** Le glouton fait *un* choix ; la programmation dynamique les **compare tous** via la récurrence.

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

☐ **expliquer** pourquoi une récursivité naïve peut être exponentielle (arbre des appels) → ex. 1 ;

☐ **mémoïser** une fonction récursive à l'aide d'un dictionnaire (*top-down*) → ex. 2, 3 ;

☐ **tabuler** un problème en remplissant un tableau du plus petit cas au plus grand (*bottom-up*) → ex. 5, 6, 7 ;

☐ **écrire une relation de récurrence** et l'implémenter des deux façons → ex. 3, 10, 12 ;

☐ **dérouler à la main** le remplissage d'une table (rendu de monnaie, pyramide, sac à dos) → ex. 4, 7, 9, 13 ;

☐ **reconnaître** les deux ingrédients (recouvrement + sous-structure optimale) et distinguer du glouton et de diviser-pour-régner → ex. 9, 19.

Vers le Grand Oral

- **Comment retenir ses calculs peut faire passer un algorithme de plusieurs siècles à une fraction de seconde ?** *(Fibonacci naïf vs mémoïsé ; l'arbre des appels ; l'exponentiel contre le linéaire.)*
- **Pourquoi rendre la monnaie n'est-il pas si simple qu'il y paraît ?** *(glouton non optimal sur* $\{1,3,4\}$ *; la récurrence du rendu ; l'optimalité garantie.)*
- **Comment un logiciel compare-t-il deux textes, ou deux brins d'ADN ?** *(plus longue sous-séquence commune ; la table* L *; applications au* `diff` *et à la bio-informatique.)*
- **Comment un téléphone devine-t-il le mot que vous vouliez écrire ?** *(distance d'édition de Levenshtein ; les trois opérations ; la table des corrections minimales.)*
- **Une même idée, deux mises en œuvre : top-down ou bottom-up, comment choisir ?** *(mémoïsation vs tabulation ; récursivité et pile d'appels ; le rôle de la relation de récurrence.)*

EXERCICES

Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

Mode d'emploi

Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 32.
Le symbole ▶ signale un exercice *à programmer et tester* en Python. Réflexe permanent : avant de coder, **écrire la relation de récurrence** et **repérer le cas de base**.

Le déclic : ne pas recalculer deux fois ↑ cours

★ Exercice 1 — *Compter les appels* ▶ corrigé p. 957 • essentiel

On reprend Fibonacci naïf :

```
1 def fibo(n):
2     if n <= 1:
3         return n
4     return fibo(n - 1) + fibo(n - 2)
```

1. Dessiner l'arbre des appels de `fibo(4)`.
2. Combien de fois `fibo(1)` est-il appelé lors du calcul de `fibo(4)` ?
3. On note $A(n)$ le nombre total d'appels à `fibo` déclenchés par `fibo(n)`. Justifier que $A(n) = 1 + A(n-1) + A(n-2)$ pour $n \geq 2$, et calculer $A(0), A(1), \ldots, A(6)$.
4. Expliquer en une phrase pourquoi ce coût rend `fibo(60)` inutilisable.

▷ coup de pouce

★ Exercice 2 — *Ajouter un carnet de notes* ▶ corrigé p. 957 • essentiel

▶ Recopier la fonction `fibo` et la transformer en une fonction **mémoïsée** `fibo_memo(n, memo=None)` qui utilise un dictionnaire `memo` (créé vide au premier appel, quand `memo` vaut `None`, comme dans la version sûre du cours) pour ne calculer chaque F_k qu'une seule fois. Vérifier que `fibo_memo(50)` vaut `12586269025` et s'obtient instantanément. ▷ coup de pouce

★★ Exercice 3 — *Monter un escalier* ▶ corrigé p. 957

Un escalier a n marches. On peut monter les marches **une par une** ou **deux par deux**. On note $M(n)$ le nombre de façons différentes d'arriver en haut.

1. Dénombrer à la main $M(1)$, $M(2)$, $M(3)$ et $M(4)$ (lister les façons pour $M(3)$).
2. Justifier que $M(n) = M(n-1) + M(n-2)$. Quel nombre célèbre reconnaît-on ? ▷ coup de pouce
3. ▶ Écrire une fonction **mémoïsée** `monter(n)` qui renvoie $M(n)$. Tester `monter(10)`.

▷ coup de pouce

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

De bas en haut : construire une table ↑cours

★ **Exercice 4** *Remplir une table à la main* ▶corrigé p. 958 • essentiel

On définit la suite u par $u_0 = 2$, $u_1 = 1$ et, pour $n \geq 2$, $u_n = u_{n-1} + 2\,u_{n-2}$.

1. Recopier et compléter la table :

n	0	1	2	3	4	5	6
u_n	2	1					

2. Dans quel ordre a-t-on rempli les cases ? Pour calculer la case 6, de quelles cases a-t-on eu besoin ?

3. Combien de cases faut-il remplir pour obtenir u_n ? Quel est l'ordre de grandeur du coût de cette méthode ?

▷ coup de pouce

★ **Exercice 5** *Fibonacci tabulé* ▶corrigé p. 958 • essentiel

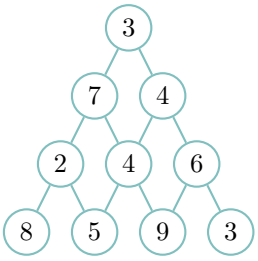
▶ Écrire `fibo_tab(n)` qui calcule F_n **sans récursivité**, en remplissant un tableau `table` de bas en haut. Comparer les valeurs obtenues avec celles de `fibo_memo` pour n de 0 à 20. ▷ coup de pouce

★ **Exercice 6** *Escalier tabulé* ▶corrigé p. 959

▶ Réécrire l'exercice de l'escalier en **tabulation** : une fonction `monter_tab(n)` avec une boucle et un tableau, sans récursivité. Vérifier qu'elle donne les mêmes résultats que la version mémoïsée. ▷ coup de pouce

★★ **Exercice 7** *Le plus grand chemin dans un triangle* ▶corrigé p. 959 • essentiel

On dispose un triangle de nombres, par exemple :



Partant du sommet, on descend à chaque étage vers le voisin de gauche ou de droite, jusqu'en bas. Le **score** d'un chemin est la somme des nombres traversés.

1. Donner un chemin de score maximal pour ce triangle, et son score.

2. On numérote les niveaux à partir de 0 (le sommet) et les cases de chaque niveau à partir de 0 (de gauche à droite). On note $S(i, j)$ le meilleur score d'un chemin qui part de la case j du niveau i (valeur de cette case comprise) et descend jusqu'en bas. Écrire la relation qui lie $S(i,j)$ à $S(i+1, j)$ et $S(i+1, j+1)$, et donner le cas de base (dernier niveau). ▷ coup de pouce

3. ▶ Le triangle est donné en liste de listes, ici `t = [[3],[7,4],[2,4,6],[8,5,9,3]]`. Écrire une fonction **tabulée** `meilleur_score(t)` qui remplit une table *du bas vers le haut* et renvoie $S(0,0)$. ▷ coup de pouce ▷ début de solution

   ▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★ **Exercice 8** *Reconnaître les deux approches* ▶ corrigé p. 959

On définit la suite de « Tribonacci » par $T_0 = 0$, $T_1 = 0$, $T_2 = 1$ et $T_n = T_{n-1} + T_{n-2} + T_{n-3}$ pour $n \geq 3$. Voici deux fonctions qui la calculent.

```python
def trib_a(n, memo=None):
    if memo is None:
        memo = {}
    if n <= 1:
        return 0
    if n == 2:
        return 1
    if n not in memo:
        memo[n] = trib_a(n - 1, memo) + trib_a(n - 2, memo) + trib_a(n - 3, memo)
    return memo[n]

def trib_b(n):
    table = [0, 0, 1] + [0] * n
    for k in range(3, n + 1):
        table[k] = table[k - 1] + table[k - 2] + table[k - 3]
    return table[n]
```

1. Laquelle procède par **mémoïsation**, laquelle par **tabulation** ? Justifier par un détail du code.
2. Que renvoient `trib_a(6)` et `trib_b(6)` ?
3. Dans `trib_a`, quelle ligne empêche de recalculer deux fois le même terme ?

▷ coup de pouce

Le rendu de monnaie ↑ cours

★ **Exercice 9** *Le glouton se trompe* ▶ corrigé p. 960 • essentiel

On dispose du système de pièces $\{1, 3, 4\}$.

1. Rendre la somme 6 avec l'algorithme glouton (toujours la plus grosse pièce possible). Combien de pièces ?
2. Trouver une meilleure solution à la main. Conclusion sur le glouton ?
3. Rappeler la relation de récurrence donnant $R(s)$, le nombre *minimal* de pièces pour rendre s, et le cas de base.
4. Remplir à la main la table des valeurs $R(0), R(1), \ldots, R(6)$ pour le système $\{1, 3, 4\}$, de gauche à droite.

▷ coup de pouce

★★ **Exercice 10** *Rendre au plus juste* ▶ corrigé p. 960 • essentiel

▶ Écrire, au choix en mémoïsation *ou* en tabulation, une fonction `rendu(somme, pieces)` qui renvoie le nombre minimal de pièces. Tester : `rendu(6, [1,3,4])` doit valoir `2`, et `rendu(11, [1,2,5])` doit valoir `3`. ▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 11** *Quelles pièces, exactement ?* ▶ corrigé p. 960 ≈ 20 min

▶ Modifier la version tabulée pour qu'elle renvoie non pas le *nombre* de pièces mais **la liste des pièces** d'une solution optimale. Tester avec `[1,3,4]` et la somme 6. ▷ coup de pouce ▷ début de solution

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Des tables à deux dimensions ↑cours

★★ **Exercice 12** *Les chemins d'un robot* ▶corrigé p. 961

Un robot part du coin supérieur gauche d'une grille de L lignes et C colonnes et veut atteindre le coin inférieur droit. À chaque pas, il ne peut aller que **vers la droite** ou **vers le bas**.

1. On note $N(i, j)$ le nombre de chemins menant du coin haut-gauche à la case (i, j). Justifier que $N(i, j) = N(i-1, j) + N(i, j-1)$, et donner les valeurs sur la première ligne et la première colonne. ▷coup de pouce

2. ▶ Écrire une fonction **tabulée** `nb_chemins(L, C)` qui renvoie le nombre de chemins jusqu'au coin bas-droit. Vérifier que `nb_chemins(3, 3)` vaut `6` et `nb_chemins(2, 2)` vaut `2`.
▷coup de pouce

   ▷Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★★ **Exercice 13** *Le sac à dos* ▶corrigé p. 961 ≈30 min

Un sac supporte une capacité `C`. On dispose d'objets de poids `poids[i]` et de valeurs `valeurs[i]` ; on veut maximiser la valeur emportée sans dépasser `C` (chaque objet est pris ou laissé, jamais coupé).

1. Rappeler la relation liant $T[i][c]$ (meilleure valeur avec les i premiers objets et une capacité c) à $T[i-1][\cdot]$, dans les deux cas « on laisse l'objet i » / « on le prend ». ▷coup de pouce

2. ▶ Écrire `sac_a_dos(poids, valeurs, capacite)`. Tester : `sac_a_dos([1,3,4,5], [1, 4,5,7], 7)` doit valoir `9`. ▷début de solution

   ▷Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

3. Quel serait le résultat d'un algorithme glouton qui prendrait d'abord les objets de plus grande valeur ? Est-il optimal ici ?

4. On ne garde que les trois premiers objets (poids $1, 3, 4$; valeurs $1, 4, 5$) et une capacité de 5. Remplir à la main la table T (4 lignes $i = 0$ à 3, 6 colonnes $c = 0$ à 5), ligne par ligne. Quelle est la meilleure valeur, et quels objets prend-on ?

★★★ **Exercice 14** *Comparer deux mots* ▶corrigé p. 962 ≈30 min

La **plus longue sous-séquence commune** (PLSC) à deux mots est utilisée par les correcteurs et par la commande `diff`. Une sous-séquence s'obtient en effaçant des lettres sans changer l'ordre des autres.

1. Donner à la main la PLSC de `"CHIEN"` et `"NICHE"`, et sa longueur. ▷coup de pouce

2. ▶ Écrire `plsc(a, b)` qui renvoie la longueur de la PLSC, en remplissant une table L où `L[i][j]` est la longueur de la PLSC des `i` premières lettres de `a` et des `j` premières de `b`. Tester avec `"CHIEN"`/`"NICHE"` (attendu 3) et `"ABCBDAB"`/`"BDCAB"` (attendu 4). ▷coup de pouce ▷début de solution

   ▷Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ **Exercice 15** *La distance d'édition* ▶ corrigé p. 962

La **distance d'édition** entre deux mots est le nombre minimal d'opérations (insérer, supprimer ou substituer une lettre) pour passer de l'un à l'autre. On note $D[i][j]$ la distance entre les `i` premières lettres de `a` et les `j` premières de `b`.

1. Donner à la main la distance d'édition entre `"NICHE"` et `"RICHE"`, puis entre `"CHAT"` et `"CHATON"`. ▷ coup de pouce

2. Justifier que $D[i][0] = i$ et $D[0][j] = j$ (transformer un mot en le mot vide).

3. ▶ Écrire `distance(a, b)` qui remplit la table D avec la règle : si les dernières lettres sont égales, $D[i][j] = D[i-1][j-1]$; sinon $D[i][j] = 1 + \min(D[i-1][j], D[i][j-1], D[i-1][j-1])$. Tester : `distance("CHIEN", "CHATS")` doit valoir 3, et `distance("CHATON", "MOUTON")` doit valoir 3. ▷ coup de pouce

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Exercices type bac ↑ cours

★★ **Exercice 16** *Vue sur la mer* (d'après Amérique du Nord 2026, jour 1) ▶ corrigé p. 963 ≈ 30 min

On considère la liste des nombres d'étages des immeubles d'une rue, en partant de la mer. On cherche une **sous-séquence strictement croissante de longueur maximale** : chaque élément retenu est strictement supérieur au précédent, l'ordre étant conservé. On prend `L2 = [3, 1, 8, 2, 5]`.

1. Donner toutes les sous-séquences strictement croissantes de *longueur 2* de `L2`.

2. Déterminer la plus longue sous-séquence strictement croissante de `L2` et sa longueur.

3. ▶ Écrire `est_strict_croissante(seq)` qui renvoie `True` si la liste `seq` est strictement croissante, `False` sinon.

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

4. **Version récursive.** On appelle `llsc_fin(tab, i)` la longueur maximale d'une sous-séquence strictement croissante se *terminant* à l'indice `i`. Recopier et compléter les lignes 2, 3 et 6 : ▷ coup de pouce

```
1  def llsc_fin(tab, i):
2      if ... :
3          return ...
4      max_len = 1
5      for j in range(i):
6          if tab[j] < ... :
7              max_len = max(max_len, llsc_fin(tab, j) + 1)
8      return max_len
9
10 def llsc_rec(tab):
11     n = len(tab)
12     return max([llsc_fin(tab, i) for i in range(n)])  # plus grande valeur de la liste
```

5. **Version dynamique.** On construit une liste `dyn` telle que `dyn[i]` contient la longueur d'une plus longue sous-séquence strictement croissante se terminant à l'indice `i`. Recopier et compléter les lignes 7 et 8 : ▷ coup de pouce

```python
def llsc_dyn(tab):
    n = len(tab)
    dyn = [1] * n
    for i in range(1, n):
        for j in range(i):
            if tab[j] < tab[i]:
                dyn[i] = max(..., ...)
    return ...
```

6. Citer un avantage de la version dynamique par rapport à la version récursive.

★★★ **Exercice 17** *Justifier un texte* (d'après Pays étrangers 2025, sujet PE2) ▶ corrigé p. 964 ≈ 45 min

On veut couper un texte en lignes de manière « esthétique », pour une largeur (justification) donnée. On modélise un découpage par une liste de couples : pour `['An','algorithm','must','be','seen','to','be','believed']`, le découpage `[(0,2),(2,5),(5,7),(7,8)]` signifie « ligne 1 = mots 0 à 1, ligne 2 = mots 2 à 4, etc. ».

Le **coût inesthétique** d'une ligne est le *carré* du nombre d'espaces supplémentaires nécessaires pour atteindre la justification ; le coût d'un découpage est la somme des coûts de ses lignes.

1. ▶ Écrire `cout(i, j, liste_mots, justification)` qui renvoie le coût inesthétique de la ligne formée des mots d'indices `i` à `j-1` si elle tient (en comptant une espace entre deux mots consécutifs), et `1000000` sinon. On doit avoir, pour `liste_mots = ['An','algorithm','must','be','seen','to','be','believed']` : `cout(0,2,liste_mots,15) == 9`, `cout(0,4,liste_mots,15) == 1000000`, `cout(5,8,liste_mots,15) == 1`. ▷ coup de pouce ▷ début de solution

   ▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

2. Pour une liste de n mots, expliquer pourquoi tester *tous* les découpages possibles n'est pas raisonnable. ▷ coup de pouce

3. On donne la fonction `justifie_dynamique` suivante. Elle remplit `cout_mini[i]` = coût minimal pour justifier les mots d'indice `i` jusqu'à la fin.

```python
def justifie_dynamique(liste_mots, justification):
    n = len(liste_mots)
    cout_mini = [0] * n
    retour_ligne = [0] * n
    for i in range(n - 1, -1, -1):
        cout_mini[i] = cout(i, n, liste_mots, justification)
        indice_mini = n
        for j in range(i + 1, n):
            best = cout_mini[j] + cout(i, j, liste_mots, justification)
            if best < cout_mini[i]:
                cout_mini[i] = best
                indice_mini = j
        retour_ligne[i] = indice_mini
    decoupage = []
    k = 0
    while k < n:
        decoupage.append((k, retour_ligne[k]))
        k = retour_ligne[k]
    return decoupage
```

Établir la relation qui lie `cout_mini[i]` aux `cout_mini[j]` pour $j > i$. En quoi ce parcours « à rebours » relève-t-il de la tabulation ? ▷ coup de pouce

4. Proposer une modification pour que la fonction renvoie **aussi** le coût inesthétique total du découpage retenu.

★★ **Exercice 18** *Charger un camion au meilleur prix* (d'après Métropole 2024, jour 2) ▶ corrigé p. 964 ≈ 60 min

Dans cet exercice, l'en-tête d'une fonction précise le type de ses paramètres et de la valeur renvoyée : ainsi `puissance(x: float, n: int) -> float` prend un flottant `x` et un entier `n` et renvoie le flottant `x**n`.

Une entreprise transporte des marchandises et souhaite maximiser son profit en optimisant le remplissage de ses moyens de transport, limités par leur **volume** (en litres). Chaque marchandise est caractérisée par son **prix** (en euros) et son **volume** indivisible (en litres). Par exemple, avec trois marchandises de couples (prix, volume) $m_1 = (100, 10)$, $m_2 = (100, 10)$ et $m_3 = (250, 20)$, s'il reste 25 litres, il vaut mieux charger m_3, qui rapporte 250 €, plutôt que m_1 et m_2, qui rapportent 200 € au total pour le même espace utilisé.

Partie A — Quelques outils

On définit une classe `Marchandise` dont chaque instance possède deux attributs entiers `prix` et `volume`.

1. Compléter le constructeur ci-dessous. Utiliser le mot-clé `assert` pour qu'une exception soit levée si le paramètre `v` n'est pas strictement positif. *(Rappel : `assert condition` déclenche une exception quand `condition` s'évalue à `False`.)*

```
1 class Marchandise:
2     def __init__(self, p: int, v: int) -> None:
3         ...
```

(Le sujet original annonçait `-> 'Marchandise'` : un constructeur ne renvoie rien, son annotation correcte est `-> None`.)

2. Donner une instruction qui crée une variable `m1` représentant une marchandise d'un volume de 7 litres coûtant 20 €.

3. Proposer une méthode `ratio(self) -> float` qui renvoie le ratio `prix/volume` d'une marchandise.

4. Proposer une fonction `prixListe(tab: list) -> int` qui renvoie le prix cumulé de l'ensemble des marchandises du tableau `tab`.

▷ coup de pouce

Partie B — Première approche de rangement

On considère les quatre marchandises de couples (prix, volume) : $m_1 = (40, 20)$, $m_2 = (210, 70)$, $m_3 = (160, 40)$ et $m_4 = (50, 50)$.

5. Préciser toutes les combinaisons de marchandises possibles sans dépasser un volume de 100 litres, et le prix associé. En déduire la combinaison qui maximise le prix.

Une première méthode, `ChargementGlouton`, trie les marchandises dans l'ordre **décroissant** de leur prix volumique (ratio prix/volume), puis transporte en priorité celles de plus grand prix volumique. Si une marchandise est trop volumineuse, on essaie la suivante, jusqu'à ce qu'aucune ne puisse plus rentrer. En notant `v_restant` le volume disponible et `m_i` la $(i+1)^{\text{e}}$ marchandise une fois les marchandises triées :

```
ChargementGlouton
n = nombre de marchandises
POUR i ALLANT de 0 a n-1 FAIRE
| SI volume de m_i <= v_restant ALORS
| | charger m_i
| | v_restant = v_restant - volume de m_i
TRANSPORTER le chargement prevu
```

Le tri dans l'ordre décroissant des prix volumiques donne m_3, m_2, m_1, m_4. Pour 100 litres, l'algorithme charge m_3 et m_1 pour un prix de 200 € (à comparer avec la combinaison trouvée à la question 5). Dans l'implémentation Python, les marchandises sont des instances de la classe `Marchandise`. On donne quelques qualificatifs : dichotomique, glouton, graphique, insertion, maximum, récursif, tri.

6. Indiquer, sans justification, le qualificatif qui s'applique le mieux à l'algorithme précédent.

7. Recopier et compléter la fonction `tri(tab: list) -> None` pour qu'elle trie **en place** un tableau d'objets `Marchandise` selon l'ordre décroissant des ratios : après l'appel `tri(tab)`, `tab[0]` contient la marchandise de plus haut ratio prix/volume. ▷ coup de pouce

```
def tri(tab: list) -> None:
    n = len(tab)
    for i in range(1, n):
        marchandise = tab[i]
        j = i - 1
        while ... and ... > ... :
            tab[j+1] = ...
            j = ...
        tab[j+1] = marchandise
```

8. Sans justifier, préciser le nom de ce tri et son coût temporel dans le pire des cas (constant, logarithmique, linéaire, quasi-linéaire ($n \log_2 n$), quadratique, cubique ou exponentiel).

9. Recopier et compléter la fonction `charge` qui applique l'algorithme `ChargementGlouton`.

```
def charge(tab: list, volume: int) -> list:
    tri(tab)
    chargement = []
    n = len(tab)
    for ...
        if ...
            ...
            ...
    return ...
```

11

Partie C — Rangement optimisé par récursivité

L'algorithme glouton ne renvoie pas toujours une solution optimale. On écrit donc une fonction récursive d'en-tête `chargeOptimale(tab: list, v_restant: int, i: int) -> list` qui calcule la charge optimale pour un volume `v_restant` en utilisant les marchandises à partir de l'indice `i` (n désigne le nombre de marchandises) :

- si `i >= n`, toutes les marchandises ont été essayées : l'appel renvoie la liste vide ;
- si `i < n` et la marchandise d'indice `i` a un volume strictement supérieur au volume restant, l'appel renvoie le résultat de `chargeOptimale(tab, v_restant, i+1)` ;
- si `i < n` et la marchandise d'indice `i` a un volume inférieur ou égal au volume restant, deux options sont possibles : soit on **n'utilise pas** la marchandise `i`, et le chargement est le résultat de l'appel récursif avec le même volume restant à partir de la marchandise suivante ; soit on **utilise** la marchandise `i`, et le chargement contient cette marchandise et celles du résultat de l'appel récursif à partir de la marchandise suivante, avec un volume restant diminué. On garde l'option qui maximise le prix transporté.

10. Recopier et compléter la fonction `chargeOptimale`. ▷ coup de pouce

```
def chargeOptimale(tab: list, v_restant: int, i: int) -> list:
    if i >= ...:
        return ...
    else:
```

```
        if tab[i].volume > v_restant:
            return chargeOptimale(tab, v_restant, i+1)
        else:
            option1 = chargeOptimale(tab, ..., ...)
            option2 = [tab[i]] + chargeOptimale(tab, ..., ...)
            if prixListe(option1) > prixListe(option2):
                return ...
            else:
                return ...
```

Prolongement (questions ajoutées au sujet d'origine).

11. Avec les marchandises $(100, 10)$, $(100, 10)$ et $(250, 20)$ de l'introduction, dans cet ordre, et un volume de 25 litres, montrer que l'appel `chargeOptimale(tab, 15, 2)` est effectué **deux fois**. Comment évolue ce phénomène quand le nombre de marchandises augmente ?

12. ▶ Écrire une fonction récursive **mémoïsée** `prix_max(tab, v_restant, i, memo)` qui renvoie seulement le **prix maximal** transportable (et non la liste des marchandises), en rangeant chaque résultat dans le dictionnaire `memo` avec la clé `(i, v_restant)`. Vérifier que `prix_max(tab, 100, 0, {})` renvoie `250` pour les quatre marchandises de la partie B.

▷ début de solution

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

★★ Exercice 19 — *Découper des planches au meilleur prix* (sujet maison, façon bac)

▶ corrigé p. 966 ≈ 60 min

Une scierie vend des morceaux de planche au détail. Le prix d'un morceau ne dépend que de sa longueur, un nombre entier de mètres, au plus égal à 5 :

Longueur du morceau (m)	1	2	3	4	5
Prix (€)	2	5	10	13	14

On représente ces tarifs par la liste `prix = [0, 2, 5, 10, 13, 14]`, où `prix[k]` est le prix d'un morceau de `k` mètres (et `prix[0] = 0`). La scierie reçoit des planches de n mètres (n entier) et les découpe en morceaux de longueurs entières afin de **maximiser la recette totale**. On note $V(n)$ cette recette maximale.

Partie A — À la main

1. Lister toutes les façons de découper une planche de 4 m (sans tenir compte de l'ordre des morceaux), avec la recette de chacune. En déduire $V(4)$. ▷ coup de pouce

2. Est-il intéressant de vendre entière une planche de 5 m ? Justifier.

3. On repère une découpe par les traits de scie, que l'on peut faire ou non à chacune des marques 1 m, 2 m, …, $(n-1)$ m. En oubliant la limite de 5 m par morceau, justifier qu'une planche de n mètres peut être découpée de 2^{n-1} façons. Pourquoi est-il déraisonnable de toutes les tester pour une planche de 60 m ? ▷ coup de pouce

Partie B — Une relation de récurrence

4. En considérant la longueur k du **premier morceau**, justifier que $V(0) = 0$ et, pour $n \geq 1$:

$$V(n) = \max_{1 \leq k \leq \min(n,\,5)} \big(\texttt{prix[k]} + V(n-k)\big).$$

5. Recopier et compléter la fonction récursive suivante, qui renvoie $V(n)$:

```
def valeur_rec(n, prix):
    if n == 0:
        return ...
    meilleur = 0
    for k in range(1, len(prix)):
        if k <= n:
            v = ... + valeur_rec(..., prix)
            if v > meilleur:
                meilleur = v
    return meilleur
```

6. Lors de l'appel `valeur_rec(8, prix)`, l'appel `valeur_rec(1, prix)` est exécuté 61 fois, pour 244 appels au total ; pour une planche de 20 m, on dépasse 800 000 appels. Expliquer ce phénomène.

Partie C — Programmation dynamique

7. ▶ Écrire une version **mémoïsée** `valeur_memo(n, prix, memo)` de la fonction précédente, où `memo` est un dictionnaire de clé `n`. ▷ début de solution

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

8. ▶ Écrire une version **tabulée** `valeur_tab(n, prix)` (une boucle et un tableau `V`, sans récursivité), puis recopier et compléter la table ci-dessous.

n	0	1	2	3	4	5	6	7	8
$V(n)$	0	2							

9. Pour connaître les morceaux à couper, on mémorise dans `choix[L]` la longueur du premier morceau d'une découpe optimale d'une planche de `L` mètres, puis on « remonte » la table. Recopier et compléter les lignes 8, 12 et 13, puis donner la valeur renvoyée par `decoupe_optimale(8, prix)` et par `decoupe_optimale(7, prix)`. ▷ coup de pouce

```
def decoupe_optimale(n, prix):
    V = [0] * (n + 1)
    choix = [0] * (n + 1)
    for L in range(1, n + 1):
        for k in range(1, len(prix)):
            if k <= L and prix[k] + V[L - k] > V[L]:
                V[L] = prix[k] + V[L - k]
                choix[L] = ...
    morceaux = []
    L = n
    while L > 0:
        morceaux.append(...)
        L = ...
    return morceaux
```

10. Un employé propose une méthode **gloutonne** : couper d'abord autant de morceaux que possible de la longueur qui a le meilleur prix au mètre, puis compléter avec la longueur de meilleur prix au mètre parmi celles qui rentrent encore, et ainsi de suite. Appliquer cette méthode à une planche de 8 m. Conclure.

★★ **Exercice 20** *Décoder un message secret* (sujet maison, façon bac) ▶ corrigé p. 968 ≈ 60 min

Dans un jeu d'énigmes, on code un mot en remplaçant chaque lettre par son rang dans l'alphabet (`A`→ 1, `B`→ 2, …, `Z`→ 26) et en écrivant les nombres à la suite, **sans séparateur**. Ainsi `NSI` est codé par le message `'14199'`. Mais le décodage est ambigu : `'14199'` peut aussi se lire `NAII`, `ADSI` ou `ADAII`. On veut **compter** les décodages possibles d'un message, représenté par une chaîne de chiffres.

Partie A — À la main

1. Coder le mot `BAC`. Montrer que le message obtenu admet exactement trois décodages.

▷ coup de pouce

2. Donner les cinq décodages du message `'1226'`.

3. Expliquer pourquoi le message `'106'` n'admet qu'un seul décodage, et le message `'100'` aucun.

▷ coup de pouce

Partie B — Une relation de récurrence

On note $D(i)$ le nombre de décodages des i premiers chiffres du message (de `message[0]` à `message[i-1]`), avec la convention $D(0) = 1$. Pour un message de n chiffres, le nombre cherché est $D(n)$. On dispose de la fonction suivante :

```
def lettre_seule(message, i):
    # le chiffre message[i-1] code-t-il a lui seul une lettre ?
    return message[i - 1] != '0'
```

4. La dernière lettre d'un décodage des i premiers chiffres provient soit du seul chiffre `message[i-1]`, soit des deux chiffres `message[i-2]` et `message[i-1]`. En déduire que $D(i)$ est la somme de deux termes :

   - $D(i-1)$ si `message[i-1]` est différent de `'0'`, et 0 sinon ;
   - $D(i-2)$ si $i \geq 2$ et si les deux chiffres `message[i-2]` et `message[i-1]` forment un nombre compris entre 10 et 26, et 0 sinon.

5. ▶ Écrire une fonction `deux_chiffres(message, i)` qui renvoie `True` si $i \geq 2$ et si les chiffres `message[i-2]` et `message[i-1]` forment un nombre compris entre 10 et 26, et `False` sinon. *(Rappel :* `int('7')` *vaut* 7*.)*

▷ coup de pouce

6. Recopier et compléter la fonction récursive suivante, qui renvoie $D(i)$:

```
def nb_rec(message, i):
    if i == 0:
        return ...
    total = 0
    if lettre_seule(message, i):
        total = total + ...
    if deux_chiffres(message, i):
        total = total + ...
    return total
```

7. Dessiner l'arbre des appels de `nb_rec('1111', 4)` et donner la valeur renvoyée. Quels appels sont effectués plusieurs fois ? Pour un message de 30 chiffres `1`, on compte plus de 3,5 millions d'appels : conclure.

Partie C — Programmation dynamique

8. ▶ Écrire une version **mémoïsée** `nb_memo(message, i, memo)` de la fonction `nb_rec`.

9. ▶ Écrire une fonction **tabulée** `nb_decodages(message)` qui remplit un tableau `D` de $D(0)$ à $D(n)$ et renvoie $D(n)$. Recopier et compléter la table pour le message `'123123'`.

▷ début de solution

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

i	0	1	2	3	4	5	6
chiffre `message[i-1]`	—	1	2	3	1	2	3
$D(i)$	1						

10. On considère un message formé de n chiffres `1`. Montrer que, pour $i \geq 2$, $D(i) = D(i-1) + D(i-2)$. Quelle suite célèbre reconnaît-on ? Combien de décodages admet le message `'111111'` ?

Critiquer une réponse d'IA ↑ cours

★★ **Exercice 21** *Un rendu de monnaie tabulé par un assistant d'IA* ▶ corrigé p. 970

Un élève demande à un assistant d'IA : « Écris en Python, par tabulation (*bottom-up*), une fonction `rendu(somme, pieces)` qui renvoie le nombre minimal de pièces pour rendre `somme` avec le système `pieces`. » Voici la réponse obtenue :

```
def rendu(somme, pieces):
    table = [0] * (somme + 1)          # table[s] = nb minimal pour s
    for s in range(1, somme + 1):
        for p in pieces:
            if p <= s and table[s - p] + 1 < table[s]:
                table[s] = table[s - p] + 1
    return table[somme]
```

« On remplit la table de 0 à `somme` : pour chaque montant `s`, on essaie chaque pièce `p` et on garde le meilleur `table[s - p] + 1`. C'est la relation de récurrence du cours, en version bottom-up. »

1. La réponse est-elle correcte ? ▶ Remplir la table à la main pour `rendu(6, [1, 3, 4])` (cases `table[0]` à `table[6]`), puis comparer avec la valeur attendue (le cours donne $2 : 3 + 3$).
2. Localiser et corriger l'erreur. ▷ coup de pouce
3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

S'entraîner à l'oral ↑cours

★★ **Exercice 22** IA IA IA *Expliquer en deux minutes* ▶ corrigé p. 970

Au Grand Oral comme devant l'examinateur de l'épreuve pratique, il faut savoir **expliquer** une notion clairement, sans lire ses notes. Choisir l'un des sujets suivants (ou le tirer au sort) :

Sujet 1. Expliquer pourquoi le calcul récursif naïf de la suite de **Fibonacci** est très lent et comment la **mémoïsation** le corrige.

Sujet 2. Expliquer la différence entre **mémoïsation** (de haut en bas) et **tabulation** (de bas en haut).

Sujet 3. Expliquer à un camarade de Première pourquoi la méthode gloutonne ne rend pas toujours la monnaie avec le moins de pièces, et ce que change la programmation dynamique.

Préparation (5 min, seul). Noter au brouillon **trois idées** et **un exemple**, rien de plus : comme au Grand Oral, ces notes servent d'aide-mémoire. **Exposé (2 min, chronométré).** Debout, face à un camarade, **sans lire ses notes** et sans rien écrire : tout passe par la parole. **Retour (2 min).** Le camarade pose une question de relance (on peut alors s'aider d'un schéma), remplit la grille ci-dessous et donne un conseil ; puis on échange les rôles. Cette grille reprend les critères d'évaluation du Grand Oral. ▷ coup de pouce

Grille entre camarades	**Oui**	**En partie**	**Pas encore**
Clair — audible, posé ; chaque mot technique est expliqué	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	☐	☐	☐
Construit — un fil conducteur, tenu en deux minutes, sans lire	☐	☐	☐
Un conseil pour la prochaine fois :			

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Chaque appel `fibo(n)` avec $n \geq 2$ a deux enfants, `fibo(n - 1)` et `fibo(n - 2)` ; `fibo(1)` et `fibo(0)` sont des feuilles. Pour la question 3 : l'appel compte pour 1 ; combien d'appels chacun de ses deux enfants déclenche-t-il ? ↑énoncé

Ex. 2 – piste Avant de calculer F_n, regarder si `n` est déjà une clé de `memo` ; sinon, le calculer comme dans `fibo`, le ranger dans `memo[n]`, puis le renvoyer. Le dictionnaire est transmis à chaque appel récursif. ↑énoncé

Ex. 3 q. 2 – piste Q2 : le dernier pas est de 1 ou de 2 marches : sur quelle marche était-on juste avant, dans chacun des deux cas ? ↑énoncé

Ex. 3 q. 3 – piste Q3 : reprendre le schéma de `fibo_memo` ; attention, les conditions d'arrêt ne sont pas celles de Fibonacci. ↑énoncé

Ex. 4 – piste Appliquer la relation case après case, de gauche à droite : chaque nouvelle case se calcule avec les deux cases qui la précèdent, déjà remplies. ↑énoncé

Ex. 5 – piste Créer un tableau de `n + 1` cases, y placer F_0 et F_1, puis une boucle qui remplit chaque case suivante à partir des deux précédentes. Que faire si `n` vaut 0 ? ↑énoncé

Ex. 6 – piste Même structure que `fibo_tab` ; seules les premières cases changent : relire les conditions d'arrêt de `monter`. ↑énoncé

Ex. 7 q. 2 – piste Q2 : depuis la case (i, j), quelles sont les deux cases atteignables à l'étage suivant ? Laquelle a-t-on intérêt à choisir ? ↑énoncé

Ex. 7 q. 3 – piste Q3 : la table a la même forme que le triangle, et son dernier niveau est déjà connu (condition d'arrêt) : on remonte niveau par niveau. ↑énoncé

Ex. 7 q. 3 – début de solution Q3 :

```
s = [[x for x in niveau] for niveau in t]
```

```
for i in range(len(t) - 2, -1, -1):
    for j in range(len(t[i])): ...
```
↑énoncé

Ex. 8 – piste Une fonction qui reste récursive et range ses résultats dans un dictionnaire mémoïse ; une fonction qui remplit un tableau avec une boucle, sans s'appeler elle-même, tabule. Pour la question 2, calculer à la main les termes T_0 à T_6. ↑énoncé

Ex. 9 – piste Question 1 : à chaque étape, prendre la plus grosse pièce qui ne dépasse pas ce qu'il reste à rendre. Question 2 : peut-on rendre 6 avec seulement deux pièces ? Question 3 : la dernière pièce rendue est l'une des pièces du système ; que reste-t-il alors à rendre ? ↑

Ex. 10 – piste Partir de la relation de récurrence de l'exercice précédent. En tabulation, on remplit `table[s]` pour `s` croissant, en essayant chaque pièce `p` $\leq$ `s` comme dernière pièce. ↑énoncé

Ex. 10 – début de solution
```
for s in range(1, somme + 1):
    for p in pieces:
        if p <= s: ...
```
Attention à la valeur initiale des cases : elle doit être « pire » que toute vraie solution. ↑énoncé

Ex. 11 – piste Mémoriser, pour chaque somme `s`, la dernière pièce utilisée pour l'atteindre au mieux, puis « remonter » depuis `somme`. ↑énoncé

Ex. 11 – début de solution `derniere = [None] * (somme + 1)` ; chaque fois que `table[s]` s'améliore grâce à `p`, noter `derniere[s] = p`. Puis partir de `s = somme` et, tant que `s > 0`, ajouter `derniere[s]` à la solution et le retirer de `s`. ↑énoncé

Ex. 12 q. 1 – piste Q1 : de quelles cases le robot peut-il arriver sur la case (i, j) ? Combien de chemins mènent à une case de la première ligne ? ↑énoncé

Ex. 12 q. 2 – piste Q2 : créer une table de `L` lignes et `C` colonnes remplie de 1, puis remplir les autres cases ligne par ligne. ↑énoncé

Ex. 13 q. 1 – piste Si l'on prend l'objet i, quelle capacité reste-t-il pour les $i-1$ premiers objets ? Et peut-on toujours le prendre ? ↑énoncé

Ex. 13 q. 2 – début de solution
```
T = [[0] * (capacite + 1) for _ in range(n + 1)]
for i in range(1, n + 1):
    for c in range(capacite + 1):
        T[i][c] = T[i - 1][c]   (on laisse l'objet) ...
```
↑énoncé

Ex. 14 q. 1 – piste Q1 : chercher les lettres communes aux deux mots, puis le plus grand nombre d'entre elles que l'on retrouve **dans le même ordre** dans les deux. ↑énoncé

Ex. 14 q. 2 – piste Q2 : comparer les dernières lettres `a[i-1]` et `b[j-1]` : si elles sont égales, elles peuvent terminer la sous-séquence commune ; sinon, on retire la dernière lettre de l'un **ou** de l'autre mot. Condition d'arrêt : un mot vide. ↑énoncé

Ex. 14 q. 2 – début de solution Q2 : `L = [[0] * (m + 1) for _ in range(n + 1)]` (ligne et colonne 0 : mot vide).
Si `a[i-1] == b[j-1]` : `L[i][j] = L[i-1][j-1] + 1` ; sinon, garder la meilleure de deux cases voisines. ↑énoncé

Ex. 15 q. 1 – piste Q1 : aligner les deux mots lettre à lettre : quelles lettres faut-il changer, ajouter ou retirer ? ↑énoncé

Ex. 15 q. 3 – piste Q3 : remplir d'abord la première colonne et la première ligne (question 2), puis les autres cases ligne par ligne avec la règle donnée ; les `i` premières lettres de `a` se terminent par `a[i-1]`. ↑énoncé

Ex. 16 q. 4 – piste Q4 : condition d'arrêt : que vaut la réponse pour l'indice 0 ? Ligne 6 : on ne peut prolonger par `tab[i]` qu'une sous-séquence qui se termine par une valeur... ↑énoncé

Ex. 16 q. 5 – piste Q5 : comparer avec la ligne 7 de `llsc_fin` : quel appel récursif est remplacé par une case de `dyn` ? La réponse est-elle forcément dans la dernière case ? ↑énoncé

Ex. 17 q. 1 – piste Q1 : la longueur d'une ligne est la somme des longueurs de ses mots, plus le nombre d'espaces entre eux : combien y en a-t-il entre les mots `i` à `j-1` ? ↑énoncé

Ex. 17 q. 1 – début de solution Q1 :

```
longueur = j - i - 1  # les espaces
for k in range(i, j):
    longueur += len(liste_mots[k])
if longueur > justification:
    return 1000000 ...
```

↑énoncé

Ex. 17 q. 2 – piste Q2 : après chaque mot sauf le dernier, on coupe ou non : combien de découpages cela fait-il en tout ? ↑énoncé

Ex. 17 q. 3 – piste Q3 : si la première ligne contient les mots `i` à `j-1`, que reste-t-il à justifier ? Dans quel ordre les cases de `cout_mini` sont-elles remplies ? ↑énoncé

Ex. 18 q. 4 – piste Q4 : un accumulateur initialisé à 0, auquel on ajoute l'attribut `prix` de chaque marchandise. ↑énoncé

Ex. 18 q. 7 – piste Q7 : tant qu'on n'est pas sorti du tableau, on décale d'une case vers la droite les marchandises dont le ratio est **plus petit** que celui de `marchandise`. ↑énoncé

Ex. 18 q. 10 – piste Q10 : `option1` : on n'utilise pas la marchandise `i` ; `option2` : on l'utilise, et le volume restant diminue de son volume. ↑énoncé

Ex. 18 q. 12 – début de solution Q12 :

```
if (i, v_restant) in memo:
    return memo[(i, v_restant)]
```

puis la même structure que `chargeOptimale`, en ne manipulant que des prix, et `memo[(i, v_restant)] = ...` avant de renvoyer. ↑énoncé

Ex. 19 q. 1 – piste Q1 : écrire 4 comme somme d'entiers de toutes les façons possibles, en rangeant les morceaux du plus long au plus court. ↑énoncé

Ex. 19 q. 3 – piste Q3 : à chacune des $n-1$ marques, combien de choix ? ↑énoncé

Ex. 19 q. 7 – début de solution Q7 :

```
if n in memo:
    return memo[n]
```

en tête de fonction (après la condition d'arrêt), et `memo[n] = meilleur` juste avant le `return`. ↑énoncé

Ex. 19 q. 9 – piste Q9 : quand `V[L]` s'améliore grâce à un premier morceau de longueur `k`, c'est `k` qu'il faut retenir. Pour remonter : couper ce premier morceau, puis recommencer avec ce qui reste. ↑énoncé

Ex. 20 q. 1 – piste Q1 : chercher où l'on peut regrouper deux chiffres consécutifs en un nombre compris entre 10 et 26. ↑énoncé

Ex. 20 q. 3 – piste Q3 : un `0` ne code aucune lettre à lui seul : avec quel chiffre doit-il être regroupé ? ↑

Ex. 20 q. 5 – piste Q5 : commencer par traiter le cas $i < 2$; `int` s'applique aussi à une chaîne de deux chiffres. ↑énoncé

Ex. 20 q. 9 – début de solution Q9 :

```
D = [0] * (n + 1)
D[0] = 1
for i in range(1, n + 1):
```

puis les deux tests de `nb_rec`, avec `D[i - 1]` et `D[i - 2]` à la place des appels récursifs. ↑énoncé

Ex. 21 q. 2 – piste Au départ, toutes les cases valent 0 : le test `table[s - p] + 1 < table[s]` peut-il être vrai une seule fois ? ↑énoncé

Ex. 22 – piste Structurer chaque explication en trois temps : une **définition** en une phrase, un **exemple** petit et concret déroulé à voix haute, puis le **piège** (l'erreur que l'auditeur risque de faire). Finir par une phrase qui répond à la question posée. Sujet 1 : dessiner mentalement l'arbre des appels de `fibo(5)` et repérer un appel qui revient plusieurs fois. Sujet 3 : préparer un petit système de pièces où le premier choix « évident » est mauvais. ↑énoncé

TP ET PROJETS

TP Le plus grand carré blanc ▶ corrigé p. 971

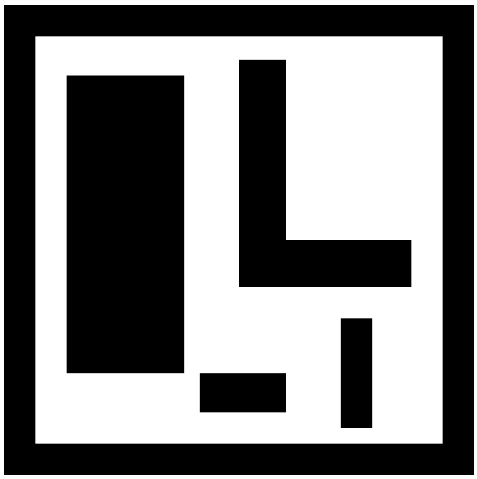
Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t11-tp-carre-blanc` — **archive .zip**

*Ce TP guidé réinvestit tout le chapitre : on découvre une **récurrence** en remplissant une grille à la main, puis on la programme en **récursif**, on l'accélère par **mémoïsation**, et enfin on la déroule en **tabulation** sur une vraie image. Fil rouge du chapitre : « ne jamais recalculer deux fois la même chose ».*

Fichier à télécharger (lien ci-dessus) : `carre_blanc.py` (chargement de l'image, **trame** des fonctions à écrire et **tests** — `python3 carre_blanc.py` affiche `OK` ou `A FAIRE ou ECHEC` pour chaque fonction). Il nécessite la bibliothèque `PIL` (Pillow) et les images fournies.

1 Le problème

On dispose d'une image en **noir et blanc**. On cherche la taille du **plus grand carré entièrement blanc** que l'on peut y dessiner (côtés parallèles aux bords).



Convention. Un pixel a pour coordonnées `(x, y)`, où `x` est la **colonne** et `y` la **ligne** (comme dans la bibliothèque `PIL`). L'idée centrale, qui va tout débloquer :

La bonne question à se poser

Pour chaque pixel, on calcule la taille du plus grand carré blanc **dont ce pixel est le coin inférieur droit**. Notons cette valeur `pgcb(x, y)`. La réponse au problème sera alors le **maximum** de toutes ces valeurs.

2 Découverte à la main

Travaillons d'abord sur une petite grille (1 = blanc, 0 = noir), en remplissant chaque case **de gauche à droite et de haut en bas**.

Exercice 1 *Le coin inférieur droit*

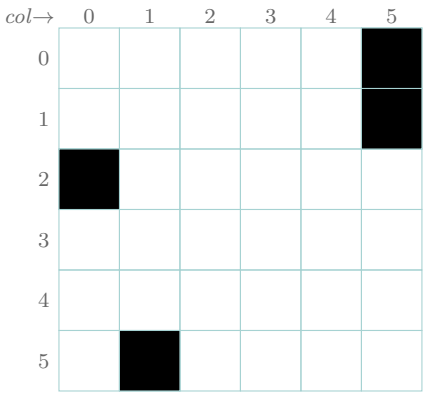
On regarde une case **blanche** dont les trois voisins *déjà calculés* portent les valeurs ci-dessous. Quelle valeur faut-il mettre dans la case « ? » ? (Un carré blanc s'appuyant sur ce coin est limité par le plus *petit* des trois carrés voisins.)

diagonale	**4**	**3** haut
gauche	**2**	**?**

Exercice 2 *Remplir toute la grille*

Recopier la grille ci-dessous et **écrire dans chaque case blanche** la valeur `pgcb`. Règles :

- case **noire** → on écrit **0** ;
- case blanche sur la **ligne du haut** ou la **colonne de gauche** → on écrit **1** ;
- autre case blanche → on écrit $1 + \min($**haut**, **gauche**, **diagonale**$)$.



En déduire sur le cahier la taille du plus grand carré blanc de cette grille (le maximum des valeurs écrites).

3 La formule

Définition 4 — Récurrence du plus grand carré blanc

$$\texttt{pgcb}(x,y) = \begin{cases} 0 & \text{si le pixel } (x,y) \text{ est } \textbf{noir}, \\ 1 & \text{si le pixel est } \textbf{blanc} \text{ et } x = 0 \text{ ou } y = 0, \\ 1 + \min\left(\texttt{pgcb}(x-1,y),\ \texttt{pgcb}(x,y-1),\ \texttt{pgcb}(x-1,y-1)\right) & \text{sinon.} \end{cases}$$

Les deux premières lignes sont les **cas de base** ; la troisième relie un sous-problème à **trois sous-problèmes plus petits** (à gauche, au-dessus, en diagonale). C'est exactement la structure d'un problème de **programmation dynamique**.

4 Version récursive

On utilise une fonction `est_noir(x, y)` renvoyant `True` si le pixel est noir. Dans `carre_blanc.py`, ce n'est encore qu'une trame qui renvoie toujours `False` : **commencer par l'écrire** (exercice « La fonction `est_noir` », section « Sur une vraie image »), car les tests de `pgcb` et de `pgcb_memo` en dépendent.

Exercice 3 *Écrire* `pgcb` *en récursif*

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

Recopier et compléter :

```
def pgcb(x, y):
    if est_noir(x, y):
        return ...              # (a) cas de base : pixel noir
    elif x == 0 or y == 0:
        return ...              # (b) cas de base : bord haut / gauche
    else:
        g = ...                 # (c) a gauche
        h = ...                 # (d) au-dessus
        d = ...                 # (e) en diagonale
        return 1 + min(min(g, h), d)
```

Exercice 4 *Le problème de la lenteur*

Si l'on teste `pgcb(250, 250)` sur l'image de la section « Sur une vraie image », le calcul ne se termine **jamais** en temps raisonnable.

1. En observant les trois appels récursifs, expliquer pourquoi un **même** sous-problème `pgcb(x, y)` est recalculé un très grand nombre de fois. *(Indication : pensez à l'arbre des appels de* `fibo` *du cours.)*

2. Quelle technique du cours permet d'y remédier ?

5 Mémoïsation (top-down)

Exercice 5 *Écrire* `pgcb_memo`

Écrire la fonction `pgcb_memo` qui fait le même calcul que `pgcb`, mais **mémorise** ses résultats dans un dictionnaire `memo` pour ne jamais recalculer deux fois la même case. Comme pour `fibo` dans le cours, on passe le dictionnaire en paramètre, dans la version sûre vue en remarque du cours : `def pgcb_memo(x, y, memo=None)`, puis, en tête de fonction, `if memo is None: memo = {}`. Vérifier que l'appel devient instantané :

```
>>> pgcb_memo(250, 250)
21
```

6 Sur une vraie image

Le fichier `carre_blanc.py` charge l'image fournie `carre_blanc.png` avec `PIL` (c'est déjà écrit en haut du fichier) :

```
from PIL import Image
NOM_IMAGE = "carre_blanc.png"
img = Image.open(NOM_IMAGE).convert("RGB")
pixels = img.load()
largeur, hauteur = img.size          # ici (600, 600)
```

Exercice 6 *La fonction* `est_noir`

Écrire `est_noir(x, y)` qui renvoie `True` si le pixel `(x, y)` est noir (couleur `(0, 0, 0)`), `False` sinon.

Exercice 7 *Chercher le plus grand carré de l'image*

Écrire `total_pgcb()` qui **parcourt tous les pixels** de l'image, ligne par ligne (`y` puis `x`), et renvoie la taille du plus grand carré blanc, avec les coordonnées `(x, y)` de son coin inférieur droit. Plusieurs carrés peuvent avoir la même taille maximale : en cas d'égalité, on garde le **premier** coin rencontré (comparaison stricte) :

```
>>> total_pgcb()
(200, (559, 239))
```

Attention : pour que ce soit efficace, il faut réutiliser **le même** dictionnaire de mémoïsation pour tous les pixels : créer `memo = {}` au début de `total_pgcb`, puis appeler `pgcb_memo(x, y, memo)`. (Un dictionnaire neuf évite aussi de réutiliser par erreur les résultats d'une autre image.)

Exercice 8 *Défi — la grande image*

Voici une image beaucoup plus grande (1200 × 1200 pixels), fournie sous le nom `carre_blanc_grand.png`. Cette fois, **impossible de deviner à l'œil** où se cache le plus grand carré blanc : plusieurs zones blanches se ressemblent ! C'est tout l'intérêt de l'algorithme.



Dans `carre_blanc.py`, remplacer `NOM_IMAGE = "carre_blanc.png"` par `NOM_IMAGE = "carre_blanc_grand.png"` (les tests sont alors remplacés par l'affichage des résultats), lancer `total_pgcb()` (ou `pgcb_bottom_up()`) et **noter sur le cahier la taille du plus grand carré blanc et son coin**. Sauriez-vous, *sans le programme*, désigner à coup sûr le bon carré ? (C'est bien là que la machine gagne.)

7 Bonus : tabulation (bottom-up)

Exercice 9 *Écrire* `pgcb_bottom_up`

En s'inspirant **du remplissage à la main** de la section « Découverte à la main », écrire `pgcb_bottom_up` qui construit une **table** `S` de la taille de l'image, la remplit **ligne par ligne** (sans aucune récursion), et renvoie la taille et le coin du plus grand carré. Comparer avec `total_pgcb` : on doit trouver le **même** résultat.

*Ce que ce TP illustre : un même problème résolu par **mémoïsation** (on part du haut et on descend en mémorisant) puis par **tabulation** (on part des cas de base et on remonte) — les deux visages de la programmation dynamique, à parité.*

PROJET

Seam carving

redimensionner une image sans la déformer

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t11-projet-seam-carving` — **archive .zip**

But du projet

Une photo de 300×200 pixels doit entrer dans un emplacement large de 200 pixels seulement, sans changer de hauteur. **Recadrer** coupe ce qui est près des bords ; **étirer** l'image (la mettre à l'échelle) écrase tout ce qu'elle montre. Le *seam carving* (« découpage par coutures », inventé en 2007 par Shai Avidan et Ariel Shamir) fait mieux : il retire, une par une, des **coutures** de pixels qui traversent l'image de haut en bas en passant là où il ne se passe rien. Trouver la meilleure couture parmi un nombre astronomique de possibilités est un problème de **programmation dynamique** : on y réinvestit tout le chapitre (relation de récurrence, tabulation, remontée de la table), comme dans le TP « Le plus grand carré blanc ». ▶ signale ce qui est à programmer et tester.

Consignes

- **Organisation.** Parties I à V **en commun**, seul ou par deux, sur machine : 2 heures environ. Puis la classe se répartit en **quatre groupes** ; chaque groupe traite **un** approfondissement de la partie VI (1 heure environ) et le **présente à l'oral, en 10 minutes, devant la classe**. L'oral est **évalué** (grille en fin de sujet) : c'est aussi un entraînement au Grand Oral.
- Les fichiers `projet_seam_carving_depart.py` et `seam_carving_ballons.png` sont **fournis** ; les placer dans le même dossier. Le programme utilise la bibliothèque `PIL` (Pillow), comme le TP du carré blanc. Relancer le fichier après chaque partie : chaque ligne `[A FAIRE]` doit devenir `[OK]`. On ne modifie pas les tests.
- **Convention.** Une image est une liste de lignes : `image[y][x]` est le pixel `(r, g, b)` de la **colonne** `x` et de la **ligne** `y`. Attention à l'ordre : la ligne d'abord. Les matrices de nombres (niveaux de gris, énergies, table de programmation dynamique) suivent la même convention. Au chargement, la photo est rangée dans la variable `photo`.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.



La photo du projet, 300 × 200 pixels (crédit en fin de sujet).

1 L'énergie d'un pixel

Retirer un pixel au milieu d'un ciel uniforme ne se voit pas ; retirer un pixel du bord d'un ballon se voit tout de suite. On mesure donc l'**énergie** d'un pixel : elle est grande là où l'image change brusquement, faible là où elle est uniforme. On travaille sur les **niveaux de gris** : le gris d'un pixel `(r, g, b)` est la moyenne entière `(r + g + b) // 3`.

Définition 5 — Énergie d'un pixel

Si `G` est la matrice des niveaux de gris, l'énergie du pixel de colonne x et de ligne y est

$$e(x,y) = \left|\, G[y][x+1] - G[y][x-1] \,\right| + \left|\, G[y+1][x] - G[y-1][x] \,\right|.$$

Au **bord** de l'image, un voisin qui n'existe pas est remplacé par le pixel lui-même (par exemple, en colonne 0, $G[y][x-1]$ est remplacé par $G[y][0]$).

★ **Exercice 1** *Une énergie à la main*

On considère la petite image en niveaux de gris ci-dessous (3 colonnes, 3 lignes).

	$x=0$	$x=1$	$x=2$
$y=0$	10	10	50
$y=1$	10	20	90
$y=2$	10	30	90

1. Calculer l'énergie du pixel central $(1,1)$, puis celle du coin $(2,0)$ en appliquant la règle des bords.
2. Quelle est l'énergie d'un pixel situé dans une zone où tous les gris sont égaux ?

★ **Exercice 2** *Programmer l'énergie*

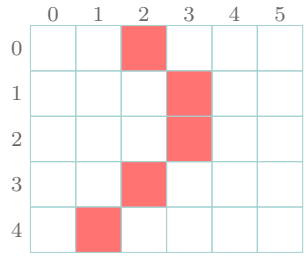
▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

1. ▶ Écrire `gris(image)`, qui renvoie la matrice des niveaux de gris, en **une ligne**, avec une liste en compréhension imbriquée.
2. ▶ Écrire `energie(G)`, qui renvoie la matrice des énergies (même taille que `G`). Vérifier que votre calcul de l'exercice 1 est retrouvé. ▷ coup de pouce
3. ▶ La fonction fournie `sauver_carte(matrice, nom)` enregistre une matrice de nombres comme une image en niveaux de gris (blanc pour les grandes valeurs). Enregistrer la carte des énergies de la photo et l'ouvrir. Quelles zones sont claires ? Lesquelles sont noires ?

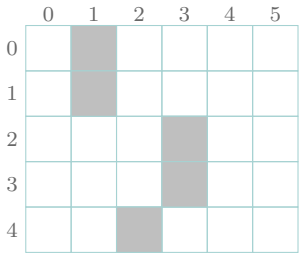
2 Les coutures et leur relation de récurrence (sur papier)

Définition 6 — Couture verticale

Une **couture verticale** est un chemin de pixels qui traverse l'image de haut en bas, avec **exactement un pixel par ligne** ; d'une ligne à la suivante, la colonne change d'**au plus 1** (les deux pixels se touchent, au moins par un coin). L'**énergie d'une couture** est la somme des énergies de ses pixels. On cherche une couture d'énergie **minimale** : c'est elle que l'on retirera.



une couture : colonnes $2, 3, 3, 2, 1$



pas une couture : de la ligne 1 à la ligne 2, on saute deux colonnes

★★ **Exercice 3** *Pourquoi pas toutes les essayer ?*

1. Dans une image de largeur au moins 2, justifier que, depuis n'importe quel pixel d'une ligne qui n'est pas la dernière, on peut toujours continuer la couture d'**au moins deux** façons. En déduire qu'une image de hauteur h contient au moins 2^{h-1} coutures qui partent d'un même pixel de la première ligne.
2. Pour la photo ($h = 200$), $2^{199} \approx 8 \times 10^{59}$. Combien d'années faudrait-il à un ordinateur qui examine un milliard de coutures par seconde ? (Une année dure environ 3×10^7 secondes.) Conclure.

11

★★ **Exercice 4** *La relation de récurrence*

On ne cherche pas directement la meilleure couture de l'image : on se pose une question plus petite pour **chaque** pixel. Notons $M(x, y)$ l'énergie minimale d'une couture qui part de la ligne 0 et **s'arrête** au pixel (x, y) (elle descend seulement jusqu'à la ligne y). On note $E(x, y)$ l'énergie du pixel (x, y).

1. Que vaut $M(x, 0)$? C'est le **cas de base**.
2. Une couture qui s'arrête en (x, y), avec $y \geq 1$, est passée par la ligne $y - 1$. Par quel(s) pixel(s) de cette ligne a-t-elle pu passer ? En déduire $M(x, y)$ en fonction de $E(x, y)$ et de valeurs de M sur la ligne $y - 1$. Que change le fait que x soit sur un bord ? ▷ coup de pouce
3. Où lit-on, une fois toutes les valeurs $M(x, y)$ calculées, l'énergie de la meilleure couture de toute l'image ?
4. Le cours donne une recette en deux ingrédients pour reconnaître un problème de programmation dynamique. Montrer que ce problème les réunit tous les deux.
5. Comparer votre relation avec celle du plus grand carré blanc (TP) et avec celle de la distance d'édition (cours) : qu'ont-elles en commun ? Dans quel ordre faudra-t-il remplir la table ?

Faire valider la relation de récurrence par le professeur avant de passer à la partie III.

★★ **Exercice 5** *Remplir la table à la main*

Voici les énergies d'une petite image de 5 colonnes et 4 lignes. Recopier le tableau et le compléter avec les valeurs $M(x, y)$, **ligne par ligne**, de haut en bas. Quelle est l'énergie de la meilleure couture ? Sur quel pixel de la dernière ligne s'arrête-t-elle ?

E	$x=0$	$x=1$	$x=2$	$x=3$	$x=4$
$y=0$	3	1	4	1	5
$y=1$	9	2	6	5	3
$y=2$	5	8	9	7	9
$y=3$	3	2	3	8	4

3 La table de programmation dynamique

★★ **Exercice 6** *Programmer la table*

1. ▶ Écrire `table_cumul(E)`, qui renvoie la table `M` (une matrice de même taille que `E`, avec `M[y][x]` $= M(x, y)$), remplie par **tabulation** en suivant votre relation de récurrence. Le test reprend le tableau de l'exercice 5 : votre calcul à la main doit être exactement celui du programme.
2. Combien de cases compte la table pour une image de largeur ℓ et de hauteur h ? Combien d'opérations, environ, pour la remplir ? Comparer avec l'exercice 3.
3. La remarque du cours sur le **coût en mémoire** dit que, pour le sac à dos, deux lignes de la table suffisent si l'on ne veut que la meilleure valeur. Est-ce aussi le cas ici ? Pourquoi va-t-on pourtant garder toute la table (voir la partie suivante) ?

4 Retrouver la couture : la remontée

La table donne l'énergie de la meilleure couture, mais pas la couture elle-même. Comme pour la plus longue sous-séquence commune du cours, on la retrouve en **remontant** la table depuis la case où se lit la réponse.

★ **Exercice 7** *Remonter à la main*

Sur la table de l'exercice 5, partir de la case de la dernière ligne où se lit l'énergie minimale. À chaque ligne, choisir, parmi les cases de la ligne du dessus d'où la couture a pu venir, celle qui a la plus petite valeur de M. Écrire la couture obtenue sous la forme de la liste de ses colonnes, de la ligne 0 à la ligne 3, et vérifier que la somme de ses énergies est bien le minimum trouvé.

★★ **Exercice 8** *Programmer la remontée*

1. ▶ Écrire `indice_du_min(ligne, debut, fin)`, qui renvoie l'indice du plus petit élément de `ligne` entre les indices `debut` et `fin` (inclus), avec une boucle. En cas d'égalité, on garde le **plus à gauche** : les tests en dépendent.
2. ▶ Écrire `couture_min(M)`, qui renvoie la liste `couture` telle que `couture[y]` soit la colonne du pixel de la couture sur la ligne `y`. On remplit cette liste **à partir de la fin**.▷ coup de pouce
3. ▶ La fonction fournie `marquer_couture(image, couture)` renvoie une copie de l'image où la couture est coloriée en rouge. Enregistrer la photo avec sa meilleure couture (`sauver_image`) et l'ouvrir. Par où passe-t-elle ? Pourquoi ?

5 Retirer des coutures, et comparer

★★ **Exercice 9** *Rétrécir l'image*

1. ▶ Écrire `supprimer_couture(matrice, couture)` en **une ligne** : une liste en compréhension qui, pour chaque ligne `y`, recolle la tranche située avant `couture[y]` et celle située après. Elle doit marcher aussi bien pour une image que pour une matrice de nombres.
2. ▶ Écrire `reduire_largeur(image, k)`, qui retire k coutures **une par une** : à chaque tour, on recalcule les gris, les énergies, la table, la couture, puis on la retire.
3. Pourquoi ne pas calculer une seule table, puis retirer d'un coup les k meilleures coutures qu'on y lit ? (Penser à ce qui arrive aux voisins des pixels retirés.)
4. ▶ Réduire la photo à 200 pixels de large et enregistrer le résultat. Mesurer la durée du calcul avec `time.time()` (le module `time` est importé). Combien de temps faut-il pour une couture ?

★ **Exercice 10** *Trois façons de rétrécir*

1. ▶ Écrire `recadrer(image, k)`, qui retire k colonnes sur les bords (`k // 2` à gauche, le reste à droite), et `mise_a_echelle(image, nouvelle_largeur)`, où la colonne `x` de la nouvelle image recopie la colonne `x * largeur // nouvelle_largeur` de l'ancienne. Une compréhension suffit pour chacune.
2. ▶ Enregistrer les trois versions de la photo en 200×200 et les comparer. Qu'est-ce que chaque méthode sacrifie ?
3. Imaginer une image sur laquelle le seam carving donnerait un mauvais résultat. Expliquer pourquoi.

6 Approfondissements : un sujet par groupe

Chaque groupe traite **son** sujet et prépare l'oral. Les fonctions à écrire sont déjà déclarées dans le fichier de départ `projet_seam_carving_depart.py`, avec un test par groupe. Les calculs sur la photo entière prennent quelques secondes : faire les essais avec une petite valeur de k avant de lancer le calcul complet.

Des coutures horizontales

★★ **Exercice 11** *Réduire la hauteur*

1. Définir une **couture horizontale**. Écrire, sur papier, la relation de récurrence qui donne l'énergie minimale d'une couture horizontale partant de la colonne 0 et s'arrêtant au pixel (x, y). Dans quel ordre remplir la table ?
2. ▶ Plutôt que de tout réécrire, on peut **transposer** l'image (échanger lignes et colonnes). Écrire en une compréhension la fonction `transposer(matrice)`, puis `reduire_hauteur(image, k)` en **une ligne**, à l'aide de `reduire_largeur`.
3. ▶ Retirer 60 lignes à la photo. Qu'est-ce qui a disparu ? Comparer avec une mise à l'échelle en hauteur.
4. ▶ *(Défi.)* Écrire `table_cumul_horizontale(E)` directement, sans transposer, et vérifier qu'elle donne la transposée de `table_cumul(transposer(E))`.
5. Pour passer de 300×200 à 200×150, faut-il retirer d'abord les coutures verticales ou les horizontales ? Proposer une expérience pour le savoir.

À l'oral : la relation de récurrence des coutures horizontales, l'astuce de la transposition, les images avant/après.

Élargir une image

★★★ **Exercice 12** *Ajouter des colonnes là où ça ne se voit pas*

1. ▶ Première idée : k fois de suite, trouver la couture d'énergie minimale et la **dédoubler** (chaque pixel de la couture est recopié une fois de plus, juste à côté de lui). Écrire `elargir_naif(image, k)`, l'essayer avec $k = 80$, et observer le résultat. Expliquer ce qui se passe.
2. Seconde idée : chercher d'abord les k coutures que l'on **retirerait** pour rétrécir l'image, puis dédoubler chacune d'elles dans l'image d'origine. Difficulté : après une suppression, les colonnes sont décalées. Pour s'y retrouver, on fabrique une matrice `origine` où `origine[y][x]` vaut d'abord `x`, et on lui fait subir **les mêmes suppressions** qu'à l'image. Pourquoi `supprimer_couture` peut-elle servir telle quelle ? ▶ Écrire `coutures_a_dupliquer(image, k)`, qui renvoie la liste des k coutures exprimées en colonnes de l'image d'origine.
3. ▶ Écrire `elargir(image, k)` : ligne par ligne, on recopie les pixels de l'image d'origine et, juste après chaque pixel d'une des k coutures, on insère la moyenne de ce pixel et de son voisin de droite (fonction `moyenne_pixels` fournie).
4. ▶ Élargir la photo de 80 pixels et comparer avec `mise_a_echelle(photo, 380)`. Pourquoi cette méthode ne permet-elle pas, en une fois, de doubler la largeur ?

À l'oral : pourquoi l'idée naïve échoue, le suivi des colonnes d'origine, les images avant/après.

Protéger ou effacer une zone

★★ **Exercice 13** *Guider les coutures*

On guide les coutures à l'aide d'un **masque** : une matrice de même taille que l'image, qui vaut 1 sur une zone à **protéger**, -1 sur une zone à **effacer**, 0 ailleurs. On ajoute à l'énergie de chaque pixel `masque[y][x] * PROTECTION`, où `PROTECTION = 10 ** 6`.

1. ▶ Écrire `masque_rectangle(hauteur, largeur, x1, y1, x2, y2, valeur)`, qui vaut `valeur` dans le rectangle des colonnes `x1` à `x2` et des lignes `y1` à `y2` (inclus), 0 ailleurs, puis `energie_modifiee(E, masque)`, chacune en une compréhension.
2. ▶ Écrire `reduire_largeur_masque(image, k, masque)`. Pourquoi faut-il retirer la couture du masque en même temps que de l'image ?
3. ▶ Retirer 180 colonnes à la photo, une fois sans protection, une fois en protégeant le groupe de ballons de gauche (colonnes 36 à 140, lignes 138 à 172). Comparer.
4. ▶ Écrire `effacer_zone(image, masque)`, qui retire des coutures tant qu'il reste dans le masque un pixel marqué -1, et renvoie l'image obtenue et le nombre de coutures retirées. Effacer le ballon de droite (colonnes 213 à 231, lignes 150 à 171). Combien de coutures a-t-il fallu ?
5. Une couture compte 200 pixels et l'énergie d'un pixel ne dépasse pas 2×255. Justifier que `PROTECTION = 10 ** 6` est assez grand pour qu'aucune couture ne traverse une zone protégée tant qu'il en existe une qui l'évite.

À l'oral : comment une simple modification des énergies guide l'algorithme, les images avant/après, l'objet effacé.

Glouton contre programmation dynamique

★★ **Exercice 14** *Trois algorithmes pour une même couture*

1. Un algorithme **glouton** part du pixel de plus faible énergie de la première ligne, puis descend, ligne après ligne, vers celui des voisins du dessous qui a la plus faible énergie. Sur la petite matrice ci-dessous, trouver à la main la couture gloutonne et la meilleure couture. Comparer leurs énergies.

0	5	5	5
9	9	5	5
9	9	5	5
9	9	5	5

2. ▶ Écrire `couture_gloutonne(E)` (avec `indice_du_min`) et `energie_couture(E, couture)`. Sur la photo, comparer l'énergie de la couture gloutonne à celle de la couture de la programmation dynamique, puis enregistrer la photo avec les deux coutures (deux couleurs différentes). Écrire `reduire_largeur_glouton(image, k)` et comparer les images réduites de 100 colonnes.
3. ▶ Écrire `cumul_rec(E, x, y)`, qui calcule $M(x, y)$ par une fonction récursive **sans mémoire**, puis `cumul_memo(E, x, y, memo=None)`, sa version **mémoïsée** (même construction que `fibo` dans le cours). Chronométrer les deux sur des bandes de la photo de plus en plus hautes : `bande = [ligne[100:140] for ligne in E[:h]]` pour $h = 8, 10, 12, 14, 16$, en calculant la valeur au pixel $(20, h-1)$. Comment évolue la durée quand h augmente de 2 ? Expliquer à l'aide de l'arbre des appels.
4. Avec `cumul_memo`, calculer la dernière ligne de la table de la photo entière et vérifier qu'on retrouve `table_cumul`. Que se passerait-il sur une photo de 3000 lignes ? Quel argument cela donne-t-il en faveur de la tabulation ?

À l'oral : pourquoi le glouton se trompe (contre-exemple), l'explosion de la version récursive naïve, mémoïsation contre tabulation.

11

7 L'oral : présenter son algorithme

Déroulement

Chaque groupe dispose de **10 minutes**, suivies de quelques minutes de questions de la classe et du professeur. Tous les membres du groupe prennent la parole. La présentation doit :

- rappeler en une minute le problème (la couture, son énergie) pour que la classe suive ;
- écrire **au tableau** la relation de récurrence (ou l'idée de l'algorithme) propre au groupe, et la justifier ;
- faire une **démonstration** : le programme qui tourne, les images avant/après, commentées ;
- dire une limite de la méthode.

Comme au Grand Oral : on parle sans lire ses notes, on regarde la classe, on s'appuie sur le tableau ; pour répondre à une question, on la reformule d'abord.

Grille d'évaluation de l'oral (une grille par élève ; note individuelle sur 20)

Critère	Insuffisant	Fragile	Satisfaisant	Très bien
Clarté de la récurrence (6 pts)	relation absente ou fausse 0–1	relation écrite mais non expliquée, cas de base oublié 2–3	sous-problème défini, cas de base et relation justifiés 4–5	idem, avec le sens de remplissage et le coût 6
Démonstration (4 pts)	rien ne tourne 0	images montrées sans commentaire 1–2	programme lancé, avant/après commentés 3	idem, avec une comparaison ou une limite montrée 4
Réponse aux questions (4 pts)	pas de réponse 0	réponse partielle ou hors sujet 1–2	réponse juste 3	réponse juste, précise et argumentée 4
Répartition de la parole (3 pts)	n'intervient pas 0	intervention très courte 1	part équitable du temps de parole 2	part équitable, enchaînements préparés avec le groupe 3
Expression orale (3 pts)	lit ses notes, inaudible 0	lit souvent, regarde peu la classe 1	parle librement, vocabulaire juste 2	parle librement, s'appuie sur le tableau, tient le temps 3

Crédit de l'image

Hot Air Balloon Silhouettes, Yaoqi LAI (Unsplash), 2015 — CC0 (domaine public), image réduite à 300 × 200 pixels : `https://commons.wikimedia.org/wiki/File:Hot_Air_Balloon_Silhouettes_(Unsplash).jpg`

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 2 q. 2 – piste Pour le pixel (x, y), calculer d'abord les colonnes des deux voisins, ramenées dans l'image : `xg = max(x - 1, 0)` et `xd = min(x + 1, largeur - 1)` ; de même pour les lignes du dessus et du dessous. ↑énoncé

Ex. 4 q. 2 – piste La partie de la couture qui va de la ligne 0 à la ligne $y - 1$ est elle-même une couture qui s'arrête sur la ligne $y - 1$: pour que le total soit le plus petit possible, elle doit être la meilleure de celles qui s'arrêtent à cet endroit. ↑énoncé

Ex. 8 q. 2 – piste Créer `couture = [0] * hauteur`, placer le bas de la couture avec `indice_du_min` sur la dernière ligne, puis parcourir les lignes en remontant avec `range(hauteur - 2, -1, -1)` ; sur chaque ligne, chercher le minimum entre les colonnes `x - 1` et `x + 1`, ramenées dans l'image. ↑énoncé

FICHE DE RÉVISION

L'essentiel

- **Idée maîtresse : ne jamais recalculer deux fois la même chose.** La programmation dynamique s'applique quand un problème se découpe en **sous-problèmes qui se recouvrent** (les mêmes reviennent) et a une **sous-structure optimale** (la solution se construit à partir de celles des sous-problèmes).
- **Fibonacci naïf** recalcule sans cesse les mêmes valeurs : le nombre d'appels croît comme $1{,}6^n$ (que l'on majore souvent par 2^n), coût *exponentiel*. La programmation dynamique le ramène à un coût **linéaire**.
- **Deux approches, deux faces d'une même idée :**
  - **mémoïsation** (*top-down*) : on garde la récursivité, mais on **stocke** chaque résultat dans un dictionnaire (« carnet ») pour le **relire** au lieu de le recalculer ;
  - **tabulation** (*bottom-up*) : on remplit un **tableau** des petits cas vers les grands, sans récursivité.
- **Exemples types** : suite de Fibonacci, montée d'escalier, rendu de monnaie, sac à dos, plus longue sous-séquence commune / distance d'édition.
- **Glouton ≠ dynamique** : le glouton fait le choix qui semble le meilleur *sur le moment* (parfois non optimal, ex. rendu de monnaie sur $\{1,3,4\}$) ; la programmation dynamique **explore tous les sous-problèmes** et garantit l'optimum.

Le code à savoir écrire

```python
def fibo_memo(n, memo=None):          # memoisation (top-down)
    if memo is None:
        memo = {}
    if n <= 1:
        return n
    if n in memo:                     # deja calcule : on lit le carnet
        return memo[n]
    resultat = fibo_memo(n - 1, memo) + fibo_memo(n - 2, memo)
    memo[n] = resultat                # on ecrit dans le carnet
    return resultat

def fibo_tab(n):                      # tabulation (bottom-up)
    if n <= 1:
        return n
    table = [0] * (n + 1)             # table[k] contiendra F_k
    table[1] = 1
    for k in range(2, n + 1):
        table[k] = table[k - 1] + table[k - 2]   # deja calcules
    return table[n]
```

À retenir aussi

- **Rendu de monnaie** : table à une dimension, on essaie chaque première pièce et on garde le minimum. **Sac à dos** : table à **deux dimensions**, chaque case compare « je prends l'objet » et « je le laisse ».
- La mémoïsation s'écrit **à la main** : un dictionnaire `memo`, le test `if n in memo`, puis `memo[n] = ...` avant de renvoyer le résultat.
- Bien identifier la **relation de récurrence** (comment un cas dépend des précédents) : c'est le cœur de tout exercice.

Erreurs à éviter

- oublier le(s) **cas de base** ou **conditions d'arrêt** (sinon récursion infinie ou table mal initialisée) ;
- **recalculer** au lieu de **relire** le résultat mémoïsé (le test `if n in memo` est essentiel) ;
- remplir la table dans le **mauvais ordre** (il faut connaître les petits cas avant les grands) ;
- confondre approche **gloutonne** (choix local) et **dynamique** (tous les sous-problèmes).

Savoir-faire à maîtriser — je sais…

☐ **expliquer** pourquoi une récursivité naïve peut être exponentielle (arbre des appels) ;

☐ **mémoïser** une fonction récursive à l'aide d'un dictionnaire (*top-down*) ;

☐ **tabuler** un problème en remplissant un tableau du plus petit cas au plus grand (*bottom-up*) ;

☐ **écrire une relation de récurrence** et l'implémenter des deux façons ;

☐ **dérouler à la main** le remplissage d'une table (rendu de monnaie, pyramide, sac à dos) ;

☐ **reconnaître** les deux ingrédients (recouvrement + sous-structure optimale) et distinguer du glouton et de diviser-pour-régner.

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 974

1. Quels sont les deux ingrédients d'un problème qui se prête à la programmation dynamique ?
2. Dans `fibo_tab(6)`, quelle valeur prend `table[6]` ?
3. Ordre de grandeur du coût de Fibonacci naïf ? de Fibonacci tabulé ?
4. Si l'on oublie d'écrire le résultat dans `memo`, que devient `fibo_memo` ?
5. Rendre 6 avec les pièces $\{1, 3, 4\}$: combien de pièces pour le glouton ? pour la programmation dynamique ?

Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 974

1. Bob envoie à Alice un message chiffré par RSA : avec quelle clé le chiffre-t-il ? (ch. 10)
2. Comment savoir, à l'aide d'un parcours, si un graphe non orienté est connexe ? (ch. 8)
3. On insère `1`, `2`, `3`, `4` dans cet ordre dans un ABR vide. Quelle est sa hauteur (convention du cours) ? (ch. 5)

Vers le bac

- Épreuve pratique : sujet 32 — Le grand escalier, p. 743

DÉFIS ADVENT OF CODE

DÉFI 1 Adapter Array

les adaptateurs — tri et programmation dynamique

Jour 10

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t11-defi-aoc-2020-10` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/10`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. On dispose d'une centaine d'adaptateurs électriques, chacun avec sa tension, qu'on peut brancher en chaîne entre une prise et un appareil. La partie 1 demande des statistiques sur la chaîne qui utilise tous les adaptateurs ; la partie 2 demande *combien* de chaînes différentes sont possibles. Ce nombre dépasse plusieurs milliers de milliards : impossible de les construire une à une, même avec un ordinateur.
Ce qu'on réinvestit. La **programmation dynamique** vue dans ce chapitre. Le nombre de façons d'atteindre un adaptateur se déduit du nombre de façons d'atteindre les adaptateurs qui peuvent se brancher juste avant lui : en résolvant ces sous-problèmes du plus petit au plus grand, et en mémorisant leurs résultats, on compte un nombre astronomique de possibilités sans en énumérer aucune. On utilise aussi un tri et un dictionnaire pour compter.
La démarche de la fiche.

1. Comprendre pourquoi la chaîne complète est la liste triée, puis calculer à la main la

réponse de la partie 1 sur un petit exemple.
2. Lire et trier les valeurs, puis compter les écarts avec un dictionnaire : réponse de la partie 1.
3. Constater qu'énumérer toutes les chaînes est impossible, puis les compter par programmation dynamique : réponse de la partie 2.
4. En défi et en approfondissement, écrire une version récursive mémoïsée, puis une méthode par blocs.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`adapter`	adaptateur	`rated for`	prévu pour (valeur nominale)
`joltage`	tension (unité inventée : le *jolt*)	`outlet`	prise de courant
`built-in`	intégré	`lower than`	inférieur à
`distribution`	répartition	`difference`	écart
`arrangement`	façon d'agencer, combinaison	`distinct`	différent

Résumé de l'énoncé (partie 1)

Contexte. En plein vol vers vos vacances, votre appareil n'a plus de batterie. La prise sous le siège ne fournit pas la bonne tension, mais votre sac déborde d'adaptateurs que l'on peut brancher les uns derrière les autres, comme des rallonges. Il s'agit de vérifier qu'on peut tous les enchaîner, et de décrire les écarts de tension le long de la chaîne.
Les données. Chaque ligne du fichier contient **un entier** : la tension de sortie d'un adaptateur (l'unité, le *jolt*, est inventée). Les lignes ne sont pas rangées, et dans les données toutes les valeurs sont différentes.
La règle.

- La prise vaut `0`. L'appareil possède son propre adaptateur intégré, qui vaut la plus grande valeur du sac plus `3`.
- Un adaptateur de valeur v accepte une source de valeur $v-1$, $v-2$ ou $v-3$: chaque maillon de la chaîne doit être plus grand que le précédent, d'au plus 3.
- Dans la partie 1, on utilise **tous** les adaptateurs pour former la chaîne prise → adaptateurs → appareil.
- On calcule les écarts entre éléments consécutifs de cette chaîne, y compris l'écart entre la prise et le premier adaptateur, et celui entre le dernier adaptateur et l'appareil. On compte les écarts égaux à 1 et les écarts égaux à 3.

La réponse attendue. Le produit (nombre d'écarts de 1) × (nombre d'écarts de 3). Par exemple, avec les adaptateurs 2, 3 et 6, la chaîne est 0, 2, 3, 6, 9 : les écarts sont 2, 1, 3, 3, soit un écart de 1 et deux écarts de 3, et la réponse serait $1 \times 2 = 2$.

Exercice 1 *Comprendre ce qui est demandé*

But. Vérifier sur trois questions courtes qu'on a compris comment se forme la chaîne complète.
À quoi ça sert ? Elles révèlent deux faits qui simplifient tout le programme : la chaîne est la liste triée, et il ne faut oublier ni la prise ni l'appareil.

Questions de vérification, à traiter sur le cahier :

1. Pourquoi la chaîne de la partie 1 est-elle forcément rangée dans l'ordre croissant ?
2. Que se passerait-il si deux adaptateurs voisins dans l'ordre croissant étaient séparés de 4 ?

3. L'écart entre la prise et le premier adaptateur compte-t-il ? Et celui entre le dernier adaptateur et l'appareil ?

Exercice 2 *À la main, sur un petit exemple*

But. Former la chaîne et calculer les écarts à la main sur dix adaptateurs.
À quoi ça sert ? On obtient un exemple dont on **connaît la réponse** (24), qui servira à tester chaque fonction. Cette même chaîne servira aussi en partie 2 et dans l'approfondissement.

On considère le fichier (inventé) suivant, écrit ici sur une seule ligne pour gagner de la place (dans le vrai fichier : une valeur par ligne) :

```
9  1  13  4  8  2  14  7  3  12
```

Ranger les valeurs dans l'ordre croissant, ajouter la prise et l'appareil, puis écrire la liste des écarts successifs. Vérification : on doit obtenir 24 pour la partie 1.

2 Programmer la partie 1

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Méthode : trier une liste par insertion (rappel de Première)

On parcourt la liste de gauche à droite ; chaque élément est ramené vers la gauche, en décalant d'un cran les éléments plus grands, jusqu'à sa place dans la partie déjà triée.

```
def tri_insertion(t):
    """Trie la liste t sur place, par ordre croissant."""
    for i in range(1, len(t)):
        x = t[i]
        j = i
        while j > 0 and t[j - 1] > x:
            t[j] = t[j - 1]          # decaler vers la droite
            j -= 1
        t[j] = x


t = [9, 1, 13, 4]
tri_insertion(t)
print(t)                          # [1, 4, 9, 13]
```

Intérêt. Le coût est quadratique dans le pire cas, ce qui reste négligeable pour une centaine d'adaptateurs. Le maximum d'une liste triée est son dernier élément.

Exercice 3 *Lire et ranger*

But. Construire la chaîne complète sous forme de liste triée, prise et appareil compris.
À quoi ça sert ? Toutes les questions suivantes, partie 2 comprise, partent de cette liste : une erreur ici (prise ou appareil oubliés) fausserait toutes les réponses.

Écrire une fonction `chaine(nom_fichier)` qui renvoie la liste triée des valeurs, complétée par `0` au début et par la valeur de l'appareil à la fin. ▷ coup de pouce

Méthode : compter avec un dictionnaire

Pour compter des apparitions, on associe à chaque valeur rencontrée son nombre d'apparitions. Avant d'augmenter un compteur, on teste si la clé existe déjà : sinon, on la crée avec la valeur 1.

```
compte = {"a": 2}
for c in ["a", "z", "z"]:
    if c in compte:
        compte[c] += 1      # cle deja presente
    else:
        compte[c] = 1       # premiere apparition
print(compte)               # {'a': 3, 'z': 2}
print("y" in compte)        # False : la cle n'existe pas
```

Intérêt. Le test `in` et l'accès `compte[c]` coûtent un temps constant en moyenne : on compte en un seul parcours.
À essayer. Recopier et exécuter ce code. Que se passe-t-il si l'on écrit `compte["y"]` ? Écrire une fonction `valeur(d, cle)` qui renvoie `d[cle]` si la clé existe, et `0` sinon.

Exercice 4 *Compter les écarts*

But. Compter, avec un dictionnaire, combien de fois apparaît chaque écart entre deux valeurs voisines.
À quoi ça sert ? Le dictionnaire obtenu donne en un seul parcours le nombre d'écarts de 1 et de 3, donc la réponse de la partie 1.

Écrire une fonction `ecarts(valeurs)` qui renvoie un dictionnaire associant à chaque écart son nombre d'apparitions. En déduire la réponse ; tester sur l'exemple inventé de la fiche, puis sur vos données. ▷ coup de pouce

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Vous voulez maintenant savoir de combien de manières différentes on aurait pu brancher l'appareil, sans être obligé d'utiliser tout le sac.
La nouvelle règle.

- On compte les chaînes qui relient la prise à l'appareil en respectant toujours la règle des écarts de 1 à 3, avec n'importe quel sous-ensemble des adaptateurs.
- Deux chaînes sont différentes si elles n'utilisent pas exactement les mêmes adaptateurs.
- Le nombre obtenu est gigantesque : il faut le compter, pas construire les chaînes une à une.

La réponse attendue. Le nombre total de chaînes possibles. Par exemple, avec les adaptateurs 1, 2 et 3 (appareil 6), l'adaptateur 3 est obligatoire, mais 1 et 2 sont facultatifs : il y a 4 chaînes.

Exercice 5 *Pourquoi pas tout énumérer ?*

But. Comprendre pourquoi construire toutes les chaînes une à une est hors de portée.
À quoi ça sert ? C'est ce constat qui justifie de changer de méthode : il faut *compter* les chaînes sans les fabriquer, ce que permet la programmation dynamique.

Sur l'exemple inventé de la fiche, on trouve `28` façons. Avec une centaine d'adaptateurs, l'énoncé annonce un nombre astronomique : expliquer pourquoi un programme qui construirait chaque façon une par une ne terminerait jamais.

Exercice 6 *Compter sans énumérer*

But. Écrire la relation qui relie $N(v)$ aux valeurs de N des adaptateurs précédents, puis la programmer.
À quoi ça sert ? C'est le cœur de la partie 2 : en remplissant N du plus petit au plus grand adaptateur, la valeur pour l'appareil est directement la réponse, obtenue en un seul parcours.

On note $N(v)$ le nombre de façons d'aller de la prise jusqu'à l'adaptateur de valeur v. Exprimer $N(v)$ à l'aide des valeurs de N pour les adaptateurs qui peuvent se brancher juste avant v. Écrire une fonction `nombre_facons(valeurs)` qui reçoit la liste renvoyée par `chaine` (prise et appareil compris), remplit un dictionnaire `N` en parcourant cette liste à partir du **deuxième** élément (la prise est le cas de base : `N[0] = 1`), et renvoie la valeur de `N` pour l'appareil. Vérifier : sur l'exemple, $N(4) = 7$, $N(9) = 14$ et la fonction renvoie `28`. ▷ coup de pouce ▷ début de solution

★★★ Exercice 7 *Défi : la version récursive*

But. Reformuler le même calcul de façon récursive, constater qu'il est trop lent, puis le corriger.
À quoi ça sert ? On voit concrètement pourquoi la mémoïsation est indispensable : c'est elle qui transforme une récursion exponentielle en un calcul aussi rapide que la version itérative.

Écrire une fonction récursive `depuis(valeurs, i)` qui renvoie le nombre de façons d'aller de `valeurs[i]` jusqu'à l'appareil (cas de base : `i` est l'indice de l'appareil, une seule façon) ; `depuis(valeurs, 0)` doit redonner la réponse de la partie 2. La tester sur vos données : que se passe-t-il ? Corriger avec la mémoïsation (ajouter un paramètre `memo`, dictionnaire partagé par tous les appels). ▷ coup de pouce

4 Approfondissement

★★ Exercice 8 *Approfondissement 1 : découper en blocs*

But. Compter les chaînes autrement, en découpant la liste en blocs indépendants.
À quoi ça sert ? Cette méthode exploite une particularité des données (pas d'écart de 2). La comparer à la programmation dynamique montre ce qu'on gagne et ce qu'on perd en s'appuyant sur une propriété particulière plutôt que sur une méthode générale.

Dans les données (et dans l'exemple inventé de la fiche), les écarts entre adaptateurs consécutifs valent 1 ou 3, jamais 2. On peut en profiter pour compter autrement.

1. Sur l'exemple, découper la chaîne $0, 1, 2, 3, 4, 7, 8, 9, 12, 13, 14, 17$ en **blocs** de valeurs consécutives (écarts de 1), séparés par les écarts de 3. Expliquer pourquoi la première et la dernière valeur de chaque bloc sont forcément utilisées.

2. À la main, compter les façons de traverser un bloc de 1, 2, 3, 4 puis 5 valeurs, la première et la dernière étant obligatoires. Vérifier que le produit sur les blocs de l'exemple redonne `28`.

3. Écrire `facons_bloc(m)` puis `nombre_facons_blocs(valeurs)`. ▷ coup de pouce ▷ début de solution

4. Pourquoi cette méthode tombe-t-elle en défaut s'il existe un écart de 2 ? Laquelle des deux méthodes est la plus générale ?

Coups de pouce *après avoir vraiment cherché : la* ***piste*** *d'abord, le* ***début de solution*** *si besoin.*

Ex. 3 – piste Lire les lignes non vides et les convertir avec `int`, puis trier avec le tri par insertion de l'encadré. L'appareil vaut le maximum plus 3, c'est-à-dire le dernier élément de la liste triée plus 3. ↑énoncé

Ex. 4 – piste On compare chaque valeur à la suivante : boucle sur les indices `i` de `0` à `len(valeurs) - 2`. ↑énoncé

Ex. 6 – piste C'est de la programmation dynamique : on résout le problème pour chaque adaptateur, du plus petit au plus grand, en réutilisant les résultats déjà calculés. ↑énoncé

Ex. 6 – début de solution Pour chaque valeur `v` après la prise : `N[v]` est la somme des `N[v - k]` pour `k` valant 1, 2 et 3, en comptant 0 si `v - k` n'est pas un adaptateur (tester d'abord `v - k in N`, comme dans l'encadré sur le comptage). ↑énoncé

Ex. 7 – piste Sans mémoire, les mêmes sous-problèmes sont recalculés un nombre exponentiel de fois. Un dictionnaire des résultats déjà connus règle le problème. ↑énoncé

Ex. 8 q. 3 – piste Un écart de 3 ne laisse aucun choix : les deux adaptateurs qui l'encadrent sont obligatoires. Les choix se font à l'intérieur de chaque bloc, indépendamment des autres blocs : on multiplie. ↑énoncé

Ex. 8 q. 3 – début de solution Parcourir la liste triée en tenant à jour la taille du bloc en cours ; à chaque écart de 3, multiplier le résultat par `facons_bloc(taille)` et repartir à 1. `facons_bloc(m)` peut réutiliser `nombre_facons` sur la liste `[i for i in range(m)]`. ↑énoncé

DÉFI 2 Rambunctious Recitation

le jeu de mémoire — dictionnaires et complexité

Jour 15

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t11-defi-aoc-2020-15` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/15`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Des elfes jouent à un jeu de mémoire : à chaque tour, le nombre annoncé dépend de la dernière fois où le nombre précédent avait été dit. Il faut trouver le nombre annoncé au tour 2020, puis au tour 30 000 000. La règle est simple, mais le jeu ne suit aucun motif prévisible : il faut le simuler tour par tour, et le faire assez vite pour que 30 millions de tours tiennent en quelques secondes.
Ce qu'on réinvestit. Ce défi prolonge ce chapitre : comme pour la **mémoïsation**, on gagne énormément de temps en conservant dans un dictionnaire des informations déjà calculées (ici, le dernier tour où chaque nombre a été dit) au lieu de les rechercher à nouveau dans toute la liste. On réinvestit aussi l'analyse de **complexité** et sa vérification par des mesures de temps.
La démarche de la fiche.

1. Vérifier sa compréhension de la règle, puis dérouler les dix premiers tours à la main.
2. Écrire une première version qui garde tous les nombres dans une liste et y cherche le précédent : réponse de la partie 1.
3. Estimer et mesurer le coût de cette version, pour constater qu'elle est trop lente pour la partie 2.

4. Remplacer la recherche par un dictionnaire, ce qui rend chaque tour de coût constant : réponse de la partie 2.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`memory game`	jeu de mémoire	`take turns`	parler à tour de rôle
`starting numbers`	nombres de départ	`spoken`	prononcé, dit
`most recently`	le plus récemment	`turn`	tour
`apart`	d'écart, séparés de	`age`	âge (ici : un écart de tours)

Résumé de l'énoncé (partie 1)

Contexte. En attendant votre avion, vous appelez les elfes du pôle Nord : ils jouent à un jeu de mémoire où chacun, à son tour, annonce un nombre qui dépend de ce qui a déjà été dit. Vous voulez prévoir ce qui sera annoncé bien plus tard dans la partie.
Les données. Le fichier contient **une seule ligne** : les nombres de départ, séparés par des virgules. Ils sont tous différents.
La règle.

- Les tours sont numérotés à partir de 1. Les premiers tours servent à annoncer les nombres de départ, dans l'ordre.
- Ensuite, à chaque tour, on regarde le nombre annoncé au tour précédent.
- S'il était annoncé pour la première fois, on annonce `0`.
- Sinon, on annonce l'écart entre le tour où il vient d'être annoncé et le tour où il l'avait été la fois d'avant : autrement dit, « il y a combien de tours qu'on l'avait déjà dit ? ». Seule compte l'avant-dernière fois, pas les plus anciennes.

La réponse attendue. Le nombre annoncé au tour 2020. Par exemple, avec les nombres de départ 1 et 3, la partie se poursuit par 0 (3 est nouveau), 0 (0 est nouveau), 1 (0 a été dit aux tours 4 et 3), puis 4 (1 a été dit aux tours 5 et 1).

Exercice 1 *Comprendre ce qui est demandé*

But. Vérifier sur trois questions courtes qu'on a compris la règle, en particulier les cas particuliers.
À quoi ça sert ? Ces cas (nombre nouveau, nombre répété deux fois de suite) sont ceux où un programme se trompe le plus souvent ; la dernière question servira à justifier l'optimisation du défi final.

Questions de vérification, à traiter sur le cahier :

1. Que dit-on juste après le dernier nombre de départ (les nombres de départ étant tous différents) ?
2. Si le même nombre est dit à deux tours consécutifs, que dit-on au tour suivant ?
3. Le nombre dit au tour k peut-il dépasser k (une fois les nombres de départ passés) ?

Exercice 2 *À la main, sur un petit exemple*

But. Dérouler à la main les dix premiers tours du jeu.
À quoi ça sert ? On obtient un exemple dont on **connaît le début** : le programme devra retrouver exactement ces dix nombres avant d'être lancé sur 2020 tours.

Avec les nombres de départ (inventés) `2,0,1`, écrire les dix premiers nombres dits, en numérotant les tours à partir de 1. Vérification : les cinq premiers sont `2, 0, 1, 0, 2` et le dixième vaut `2`.

2 Programmer la partie 1

Le fichier ne contient qu'une ligne : on la lit avec `read()`, on la nettoie avec `strip`, puis on la découpe avec `split(",")` avant de convertir chaque morceau avec `int`.

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Exercice 3 *Une première version avec une liste*

But. Simuler le jeu en gardant la liste de tous les nombres dits et en y cherchant la fois précédente.
À quoi ça sert ? C'est la traduction la plus directe de la règle : elle donne la réponse de la partie 1 et servira de référence pour vérifier la version rapide.

Écrire une fonction `jeu_liste(depart, n)` qui construit la liste de tous les nombres dits et renvoie le `n`-ième. Pour savoir quand le dernier nombre a été dit auparavant, on cherche dans la liste. Tester sur l'exemple inventé de la fiche : avec `n = 2020`, on doit trouver 779. ▷ coup de pouce

Outil Python : mesurer une durée (module `time`)

La fonction `time.perf_counter()` renvoie un instant en secondes (avec beaucoup de décimales). La différence entre deux appels donne la durée écoulée entre les deux.

```
import time

debut = time.perf_counter()      # instant de depart (en secondes)
total = 0
for i in range(1000000):
    total += i
duree = time.perf_counter() - debut
print(f"{duree:.3f} s")          # duree ecoulee, arrondie a 3 decimales
```

Intérêt. Vérifier *expérimentalement* une complexité : si doubler la taille des données double le temps, le programme est linéaire ; s'il le multiplie par 4, il est quadratique. Les durées varient d'une machine à l'autre : seuls les rapports comptent.
À essayer. Recopier et exécuter ce code, puis remplacer `1000000` par `2000000`. Comment évolue la durée ?

Exercice 4 *Combien d'opérations ?*

But. Évaluer le coût de la première version, par le calcul puis par des mesures.
À quoi ça sert ? On découvre que la recherche dans la liste rend la méthode quadratique : c'est ce qui la rend inutilisable pour la partie 2, et qui indique quoi améliorer.

Au tour numéro k, combien de cases la recherche peut-elle examiner au pire ? En déduire la complexité de `jeu_liste` en fonction de n. Mesurer le temps pour $n = 2020$, puis $n = 20\,000$ et $n = 40\,000$ avec le module `time` : le résultat est-il cohérent ?

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Les elfes sont infatigables : ils continuent la partie beaucoup plus longtemps.
La nouvelle règle.

- Même jeu, mêmes nombres de départ, même règle.
- Seul le nombre de tours change : il faut aller jusqu'au tour 30 000 000, soit près de quinze mille fois plus loin qu'en partie 1.

La réponse attendue. Le nombre annoncé au tour 30 000 000. La difficulté n'est pas la règle, mais le temps de calcul.

★★ **Exercice 5**  *30 millions de tours*

But. Remplacer la recherche dans la liste par un dictionnaire qui mémorise le dernier tour de chaque nombre.
À quoi ça sert ? Chaque tour coûte alors un temps constant : la simulation devient linéaire et 30 millions de tours prennent quelques secondes. C'est la réponse de la partie 2.

Combien de temps prendrait `jeu_liste` pour 30 millions de tours, d'après vos mesures ? Écrire une fonction `jeu_dict(depart, n)` qui ne garde **pas** la liste des nombres, mais un dictionnaire `dernier_tour` associant à chaque nombre déjà dit le dernier tour où il a été dit. Vérifier qu'elle donne `779` pour 2020 tours sur l'exemple, puis `5068275` pour 30 millions de tours (patience : quelques secondes). ▷ coup de pouce ▷ début de solution

★★★ **Exercice 6**  *Défi : encore plus vite*

But. Remplacer le dictionnaire par une liste indexée par le nombre lui-même, et comparer.
À quoi ça sert ? On gagne encore du temps en échange de mémoire : c'est un compromis classique entre temps de calcul et place occupée.

Hormis les nombres de départ (petits), les nombres dits sont toujours inférieurs au nombre de tours. Remplacer le dictionnaire par une liste de taille fixe `[0] * n` (une liste de `n` zéros) indexée par le nombre lui-même. Comparer les temps : pourquoi est-ce plus rapide, et qu'est-ce que cela coûte en mémoire ? ▷ coup de pouce

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Parcourir la liste à l'envers à partir de l'avant-dernier élément pour trouver l'occurrence précédente du dernier nombre. ↑ énoncé

Ex. 5 – piste Accéder à un dictionnaire ou y écrire coûte un temps à peu près constant, quelle que soit sa taille : chaque tour devient de coût constant, donc l'ensemble est linéaire en n. ↑

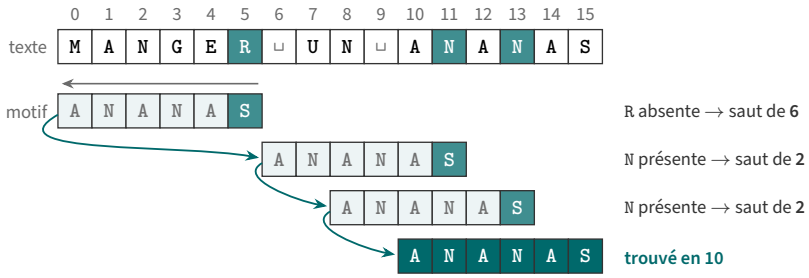
Ex. 5 – début de solution Garder le dernier nombre dit dans une variable à part, et ne le ranger dans le dictionnaire qu'**après** avoir calculé le suivant : sinon on écrase l'information dont on a besoin. ↑ énoncé

Ex. 6 – piste Un tour non encore joué peut être codé par `0`, puisque les tours sont numérotés à partir de 1. ↑ énoncé

12

CHAPITRE 12

Recherche textuelle



12

DANS CE CHAPITRE

- Activités préparatoires . . . 583
- Cours . . . 585
- Exercices . . . 594
- TP et projets . . . 605
- Fiche de révision . . . 610
- Défis Advent of Code . . . 612
- Corrigés . . . 975

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Chercher un mot, à la main

▶ corrigé p. 975

Durée : 15 à 20 min · Par deux

Consignes

- L'un déplace la bande, l'autre compte. Sans ordinateur.
- Découper la bande `SOLEIL` placée sous le texte (ou la recopier sur une bande de papier, avec des cases de même taille) : c'est la **fenêtre** que l'on fait glisser sur le texte, case par case.
- Les réponses s'écrivent sur le cahier. Une **comparaison**, c'est regarder une case de la bande et la case du texte qui lui fait face (pareil ou pas ?). Les espaces (␣) sont des caractères comme les autres. Travail **sans IA** : ni assistant, ni complétion de code ; vous devez pouvoir expliquer chaque réponse.

Le texte et le motif

On cherche le motif `SOLEIL` (6 lettres) dans le texte ci-dessous (43 caractères, positions numérotées à partir de 0).

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42
U	N	␣	P	O	T	A	G	E	R	␣	D	E	␣	R	A	D	I	S	␣	E	T	␣	D	E	␣	T	O	M	A	T	E	S	␣	A	U	␣	S	O	L	E	I	L

Bande à découper :

S	O	L	E	I	L

S	O	L	E	I	L

Première méthode : glisser d'une case à la fois

Poser la bande sous le texte, à la position 0. Comparer les cases **de gauche à droite** ; dès qu'une case diffère, arrêter et faire glisser la bande d'**une seule** case vers la droite. Recommencer jusqu'à trouver `SOLEIL`.

1. Faire la recherche en comptant les comparaisons (un bâton par comparaison, sur le cahier). Combien de positions a-t-on essayées ? Combien de comparaisons en tout ?
2. À quelle position trouve-t-on `SOLEIL` ? Pour la plupart des positions, combien de comparaisons a-t-il fallu ?
3. Si le texte était un livre d'un million de caractères, combien de positions faudrait-il essayer, à peu près ?

Deuxième méthode : regarder d'abord la fin

On recommence à la position 0, mais on compare maintenant en commençant par la **dernière** case de la bande.

4. À la position 0, quelle lettre du texte se trouve en face de la dernière case de la bande ? Apparaît-elle quelque part dans `SOLEIL` ?

5. Si l'on décale la bande de 1, 2, … ou 5 cases, cette lettre sera encore en face de la bande. Une de ces positions peut-elle donner `SOLEIL` ? De combien de cases peut-on donc faire sauter la bande d'un coup ?

6. Plus loin, la lettre en face de la dernière case est un `I`, qui *existe* dans `SOLEIL`. De combien faut-il avancer la bande pour que ce `I` se retrouve en face du `I` de la bande ?

7. Refaire toute la recherche avec cette règle : on regarde la lettre en face de la dernière case ; si elle n'est pas dans le motif, on saute toute la bande ; sinon, on avance pour l'aligner avec la même lettre de la bande (la plus à droite, sans compter la dernière case). Recopier sur le cahier le tableau ci-dessous et le compléter, une ligne par position essayée.

Position	**Lettre en face de la dernière case**	**Comparaisons**	**Saut**
0	…	…	…
…	…	…	…

Combien de positions a-t-on essayées ? Combien de comparaisons en tout ?

Ce qu'on a découvert

8. Comparer le nombre de comparaisons des deux méthodes. Pourquoi la deuxième a-t-elle pu ne **jamais lire** certaines lettres du texte ?

9. Pour aller vite, il est utile de préparer **avant** la recherche, pour chaque lettre, le saut à faire. Recopier et compléter sur le cahier ce « pense-bête » pour `SOLEIL` :

Lettre en face de la dernière case	`S`	`O`	`L`	`E`	`I`	toute autre lettre
Saut						

COURS

Programme (BO) *« Recherche textuelle. Étudier l'algorithme de Boyer-Moore pour la recherche d'un motif dans un texte. »* Commentaire du BO : *« L'intérêt du prétraitement du motif est mis en avant. L'étude du coût, difficile, ne peut être exigée. »*

Prérequis les **chaînes de caractères** et l'accès par indice (0 à $n-1$), la notion de **coût** (linéaire, quadratique), les **dictionnaires** (pour le prétraitement). → rappels de 1re : chaînes p. 1057, dictionnaires p. 1058, parcours et coût p. 1059

Objectifs *écrire* la recherche naïve ; *comprendre* les deux idées de **Boyer-Moore** (lire à l'envers, sauter loin) ; *construire* la table de prétraitement et *dérouler* **Boyer-Moore** et **Horspool**.

Situation de départ

1. Quand vous faites `Ctrl+F` dans un long document, l'ordinateur compare-t-il le mot cherché à chaque position du texte, lettre par lettre ?
2. Peut-on trouver un mot dans un texte sans même en lire toutes les lettres ?
3. Pourquoi la recherche d'un mot long peut-elle être plus rapide que celle d'un mot court ?

I Le problème : trouver une aiguille dans une botte de foin

Chaque fois que vous faites `Ctrl+F` dans un document, que `grep` fouille un fichier, qu'un antivirus reconnaît la « signature » d'un virus, ou qu'un biologiste cherche un gène dans un génome de **trois milliards** de bases, le même problème se pose : **trouver un motif dans un texte**.

Définition 1 — Recherche textuelle

On cherche les **occurrences** d'un **motif** (une petite chaîne, de longueur m) dans un **texte** (une grande chaîne, de longueur n, avec $m \leq n$). Une occurrence est une position i du texte à partir de laquelle le motif apparaît intégralement.

Exemple

Dans le vers de Verlaine

« Les sanglots longs des violons de l'automne… »

le motif `violons` apparaît à la position **23** (on compte à partir de 0, espaces compris).

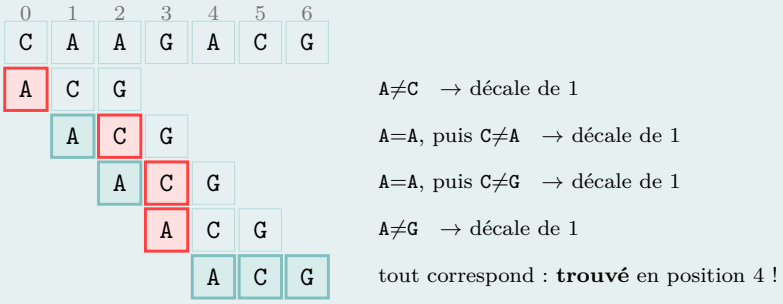
Pour les curieux — *Un terrain de jeu magnifique : l'ADN* (facultatif)

L'information génétique est une immense chaîne sur un alphabet de **quatre lettres** — `A`, `T`, `G`, `C` (les bases azotées). Chercher si le triplet `ACG` ou le gène `GATTACA` apparaît dans un brin, c'est *exactement* de la recherche textuelle — sur des textes de milliards de caractères. D'où l'importance d'algorithmes… rapides.

II La méthode naïve : la fenêtre glissante

L'idée la plus simple : on **superpose** le motif au début du texte, on compare lettre à lettre ; en cas d'échec, on décale le motif d'**un seul cran** vers la droite, et on recommence.

Cherchons `ACG` *dans* `CAAGACG`. *On compare de gauche à droite ; au premier désaccord, on glisse d'un cran.*



Cinq alignements, chacun décalé d'un seul cran : c'est lent. On « redécouvre » le texte à chaque pas.

```python
def recherche_naive(motif, texte):
    n, m = len(texte), len(motif)
    positions = []                                  # INITIALISER
    for i in range(n - m + 1):                      # PARCOURIR : chaque position de depart
        j = 0                                       # INITIALISER
        while j < m and texte[i + j] == motif[j]:   # TESTER
            j += 1                                  # METTRE A JOUR : on compare de gauche a droite
        if j == m:                                  # TESTER : tout le motif a correspondu
            positions.append(i)                     # METTRE A JOUR
    return positions                                # RENVOYER
```

▷ **Je prédis.** Que renvoie `recherche_naive("ACG", "CAAGACG")` ? Combien de comparaisons de caractères effectue-t-elle ?

▷ **Je vérifie.** `[4]`, après **9** comparaisons : 1 en $i = 0$, 2 en $i = 1$, 2 en $i = 2$, 1 en $i = 3$, 3 en $i = 4$ (les trois lettres correspondent).

Propriété 1 — Un coût qui peut exploser

Dans le pire des cas (par exemple motif `AAAA...AB` dans un texte `AAAA...A`), on compare presque tout le motif à chaque position : environ $n \times m$ comparaisons — un coût **quadratique**. Trop lent pour de très longs textes. Et surtout : la méthode naïve *oublie* tout ce qu'elle vient d'apprendre à chaque décalage.

↻ **Retour sur l'activité.** La première méthode de l'activité, la bande `SOLEIL` glissée d'une case à la fois, était cette recherche naïve : 38 positions essayées et 45 comparaisons pour trouver le motif.

► **Exercices d'application :** exercices 1 et 2 (méthode naïve)

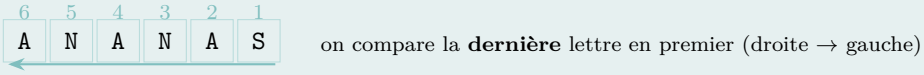
III Boyer-Moore : lire à l'envers, et sauter loin (1977)

En 1977, Robert **Boyer** et J Strother **Moore** ont une double idée, à la fois simple et géniale.

Règle 1 — Les deux idées de Boyer-Moore

(1) Comparer le motif de la *droite* vers la *gauche* (en commençant par sa dernière lettre).
(2) Prétraiter le motif pour, en cas d'échec, **sauter de plusieurs crans d'un coup** au lieu d'un seul — grâce à la **règle du mauvais caractère**.

↻ **Retour sur l'activité.** La deuxième méthode de l'activité appliquait ces deux idées : en regardant d'abord la dernière case, et en sautant toute la bande quand la lettre n'était pas dans `SOLEIL`, 9 positions et 14 comparaisons ont suffi ; les lettres survolées n'ont jamais été lues.



La règle du mauvais caractère. Supposons qu'en comparant de droite à gauche, on bute sur un caractère `c` du texte qui ne correspond pas. Deux cas :

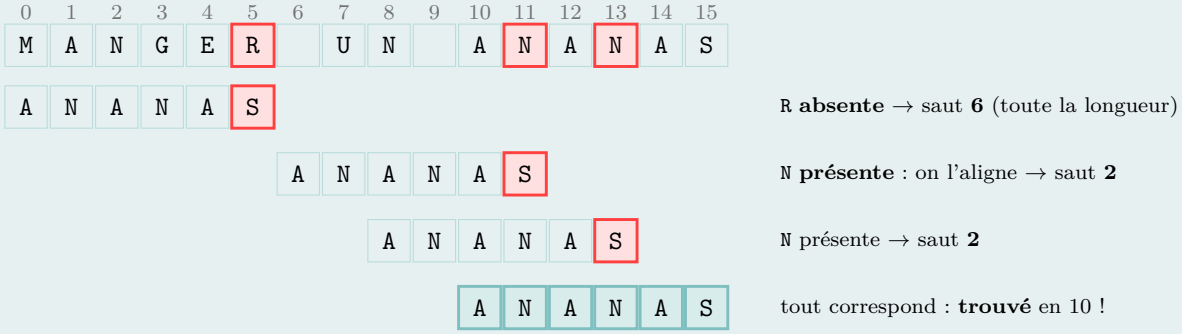
- `c` **n'existe pas** dans le motif : inutile d'essayer les alignements où `c` tomberait *dans* le motif — on place le motif juste **après** `c`. Si l'échec a lieu en position j du motif, le saut vaut $j+1$; c'est **toute la longueur** m du motif quand l'échec a lieu dès la dernière lettre ($j = m-1$).
- `c` **existe** dans le motif : on décale juste ce qu'il faut pour aligner `c` du texte avec *sa* dernière occurrence dans le motif. Si cette dernière occurrence est *à droite* de la position d'échec, ce décalage serait négatif ou nul : on avance alors d'un seul cran.

Le prétraitement consiste donc à connaître, pour chaque lettre, sa **dernière position** dans le motif.

Exemple — *Boyer-Moore sur le mot* `ANANAS`
Table des dernières positions du motif `ANANAS` (`A0 N1 A2 N3 A4 S5`) :

lettre	A	N	S
dernière position	4	3	5

Cherchons `ANANAS` dans `MANGER UN ANANAS`. On aligne le motif au début et on compare **par la droite**. Selon la lettre fautive, le saut change :



Le saut n'est pas toujours « toute la longueur » : quand la lettre fautive existe *dans le motif (ici le* `N`*, en dernière position* 3*), on décale juste ce qu'il faut pour l'aligner — soit* $5-3=\mathbf{2}$ *crans.*

```python
def table_dernier(motif):
    dernier = {}                # INITIALISER
    for k in range(len(motif)):   # PARCOURIR
        dernier[motif[k]] = k   # METTRE A JOUR : derniere position de chaque lettre
    return dernier              # RENVOYER

def boyer_moore(motif, texte):
    n, m = len(texte), len(motif)
    dernier = table_dernier(motif)
    positions = []                                  # INITIALISER
    i = 0                                           # INITIALISER
    while i <= n - m:                               # PARCOURIR : les fenetres
        j = m - 1                                   # INITIALISER : fin du motif
        while j >= 0 and texte[i + j] == motif[j]:   # TESTER
            j -= 1                                  # METTRE A JOUR : comparaison DROITE -> GAUCHE
        if j < 0:                                   # TESTER
            positions.append(i)                     # METTRE A JOUR : motif trouve
            i += 1
        else:
            c = texte[i + j]                        # le "mauvais caractere"
            if c in dernier:
                k = dernier[c]                      # derniere position de c
            else:
                k = -1                              # c absent du motif
            i += max(1, j - k)                      # METTRE A JOUR : saut (au moins 1)
    return positions                                # RENVOYER
```

▷ **Je prédis.** Que renvoie `table_dernier("ELLE")` ? Pour `boyer_moore("NAS", "ANANAS")`, quels sont les sauts successifs et le résultat ?

▷ **Je vérifie.** `{'E': 3, 'L': 2}` : chaque lettre garde sa *dernière* position. Puis : en $i = 0$, échec sur `A` (dernière position 1 dans `NAS`), saut $2 - 1 = 1$; en $i = 1$, échec sur `N` (position 0), saut 2 ; en $i = 3$, `NAS` est trouvé. Résultat `[3]`.

► **Exercices d'application :** exercices 3 **et** 4 (Boyer-Moore : les deux idées, dérouler)

IV Boyer-Moore-Horspool : la version simplifiée (1980)

Le vrai Boyer-Moore combine *deux* règles de saut (le mauvais caractère *et* le « bon suffixe » [au-delà du programme]). En 1980, Nigel **Horspool** propose une simplification élégante, tout aussi rapide en pratique et bien plus simple à écrire — au point qu'**elle suffit** amplement au lycée.

Règle 2 — L'astuce de Horspool

On garde la comparaison de droite à gauche. Mais, quel que soit l'endroit où l'échec se produit, on décide du saut en regardant **le caractère du texte aligné avec la *dernière* case de la fenêtre**. Une **seule** table de décalage suffit : pour chaque lettre du motif (sauf la dernière), « à quelle distance de la fin se trouve-t-elle ».

Horspool ou Boyer-Moore : quelle différence ? Les deux lisent le motif **de droite à gauche** et sautent grâce au mauvais caractère. Toute la différence tient au **caractère que l'on regarde** pour décider du saut.

	Boyer-Moore (mauvais caractère)	**Horspool**
Caractère observé	celui qui a **provoqué l'échec** (à la position j, *variable*)	**toujours** celui aligné avec la **fin** de la fenêtre
Le saut dépend-il de *où* l'échec a lieu ?	**oui** : décalage $= j -$ (dernière position de c)	**non** : une **seule** case observée, toujours la même
Table de prétraitement	**dernière position** de chaque lettre du motif	**distance à la fin** (lettres sauf la dernière), défaut m

Autrement dit : quand la comparaison échoue *avant* la fin (une partie du suffixe correspondait déjà), Boyer-Moore se sert de la lettre *à l'endroit précis* de l'échec, tandis que Horspool ignore cette position et se rabat *toujours* sur la lettre de fin de fenêtre. Horspool est donc **plus simple** — une seule règle, un seul point d'observation — et reste tout aussi rapide en pratique. *(Le « vrai » Boyer-Moore ajoute encore la règle du « bon suffixe » [au-delà du programme], qui réutilise la portion du motif déjà reconnue avant l'échec.)*

Construire la table de décalage. Pour un motif de longueur m, on parcourt ses lettres *sauf la dernière* : la lettre en position k reçoit le décalage $m - 1 - k$. Toute lettre absente reçoit le décalage m (saut maximal).

Exemple — la table de `MOTIF` *($m = 5$)* (la dernière lettre `F` n'est pas dans la table : elle prend le décalage par défaut 5) :

lettre	`M`	`O`	`T`	`I`	*autres*
décalage	4	3	2	1	5

↻ **Retour sur l'activité.** Le « pense-bête » de `SOLEIL` rempli à la fin de l'activité (`S`→ 5, `O`→ 4, `L`→ 3, `E`→ 2, `I`→ 1, toute autre lettre → 6) est exactement cette table : le `L` final ne compte pas, c'est celui de la position 2 qui donne le saut 3.

```
1  def table_decalage(motif):
2      m = len(motif)
3      dec = {}                    # INITIALISER
4      for k in range(m - 1):      # PARCOURIR : on ignore la DERNIERE lettre
5          dec[motif[k]] = m - 1 - k   # METTRE A JOUR
6      return dec                  # RENVOYER
```

```
7
8  def horspool(motif, texte):
9      n, m = len(texte), len(motif)
10     dec = table_decalage(motif)
11     positions = []                                  # INITIALISER
12     i = 0                                           # INITIALISER
13     while i <= n - m:                               # PARCOURIR : les fenetres
14         j = m - 1                                   # INITIALISER : fin du motif
15         while j >= 0 and texte[i + j] == motif[j]:   # TESTER
16             j -= 1                                  # METTRE A JOUR : comparaison DROITE -> GAUCHE
17         if j < 0:                                   # TESTER
18             positions.append(i)                     # METTRE A JOUR : motif trouve en i
19         c = texte[i + m - 1]                        # caractere aligne avec la FIN de la fenetre
20         if c in dec:                                # TESTER
21             i += dec[c]                             # METTRE A JOUR : saut lu dans la table
22         else:
23             i += m                                  # METTRE A JOUR : lettre absente, saut de m
24     return positions                                # RENVOYER
```

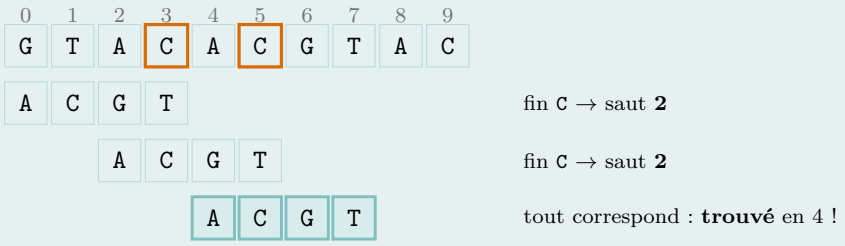
▷ **Je prédis.** Que renvoie `table_decalage("ELLE")` ? Puis, avant de regarder la figure ci-dessous : quels décalages successifs `horspool("ACGT", "GTACACGTAC")` effectue-t-elle, et que renvoie-t-elle ?

▷ **Je vérifie.** `{'E': 3, 'L': 1}` : le `E` final est ignoré, c'est le premier `E` qui donne 3 ; le second `L` écrase le premier (1 et non 2). Puis : fin `C` → saut 2, fin `C` → saut 2, motif trouvé en 4 (fin `T`, absente de la table → saut 4, et la recherche s'arrête). Résultat `[4]`.

Au bac — la table de Horspool peut être écrite autrement

Certains sujets rangent dans le dictionnaire la **position** k de chaque lettre du motif (sauf la dernière), et non le décalage : le saut $m-1-k$ est alors calculé *dans la boucle*, par une ligne du type `i += m - 1 - d[c]`. D'autres fonctions s'arrêtent à la **première** occurrence et renvoient sa position, ou -1 si le motif est absent. Les sauts sont les mêmes : il suffit de **lire le code de l'énoncé** et de suivre sa convention.

Visualisons Horspool sur `ACGT` *cherché dans* `GTACACGTAC` *(table* `A`→ 3, `C`→ 2, `G`→ 1, *autres* → 4*). La case orangée est la lettre alignée avec la* ***fin*** *de la fenêtre : c'est elle, et elle seule, qui décide du saut.*



Exemple — *Dérouler Horspool :* `MOTIF` *dans* `UN TEXTE AVEC UN MOTIF ICI`

Table : `M`→ 4, `O`→ 3, `T`→ 2, `I`→ 1, autres → 5.

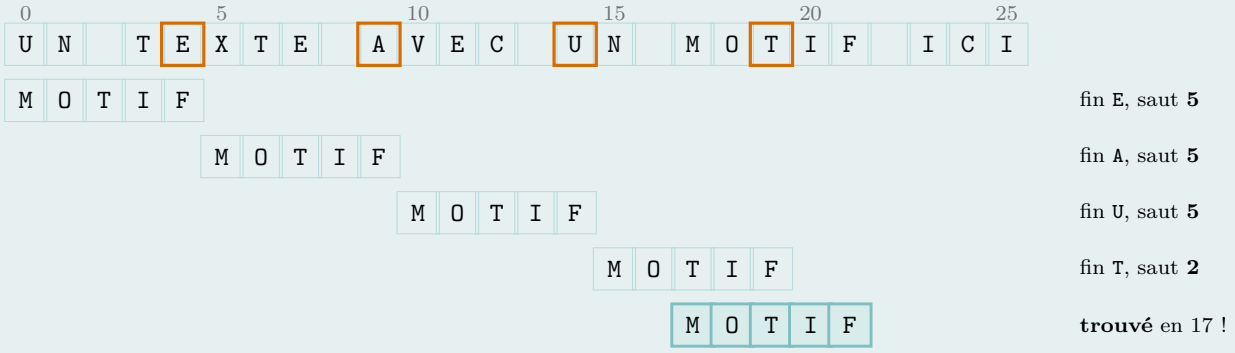
▷ **Je prédis.** Cacher le tableau : pour chaque fenêtre, prévoir la lettre de fin et le décalage.

fenêtre	texte aligné	lettre de fin	décalage
$i=0$	`UN TE`	`E` (absente)	5
$i=5$	`XTE A`	`A` (absente)	5
$i=10$	`VEC U`	`U` (absente)	5
$i=15$	`N MOT`	`T`	2
$i=17$	`MOTIF`	**trouvé !** (position 17)	

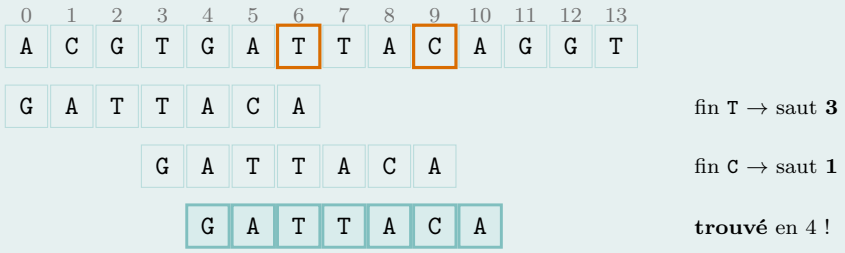
Cinq alignements ont suffi pour parcourir un texte de 26 caractères : presque à chaque échec,

on a bondi de 5 (la longueur du motif), *sans même lire* les lettres survolées.

Le même déroulé, en images (la case orangée = la lettre de fin de fenêtre qui décide du saut) :



Exemple — *Un clin d'œil à l'ADN :* `GATTACA`
Table de `GATTACA` : `G`→ 6, `A`→ 2, `T`→ 3, `C`→ 1, autres → 7. Dans le brin `ACGTGATTACAGGT`, Horspool teste $i = 0$ (fin `T` → saut 3), $i = 3$ (fin `C` → saut 1), puis $i = 4$: **`GATTACA` trouvé !** Trois alignements pour trouver le motif, au lieu de cinq pour la méthode naïve, qui essaie $i = 0, 1, 2, 3, 4$ avant de trouver. (La fonction cherche toutes les occurrences : elle teste encore $i = 6$, puis s'arrête.)



► **Exercices d'application :** exercices 5 **à** 7**,** 17 **et** 18 (Horspool : la table de décalage, dérouler, en Python ; tables et déroulés en série)

V Le coût, ou le miracle du sous-linéaire

Règle 3 — Complexité (non exigible au bac)

Prétraitement : construire la table coûte $O(m)$ (une seule passe sur le motif ; les lettres absentes du motif ne sont pas stockées : elles reçoivent le décalage par défaut m). Comme la mémoïsation du chapitre 11, on dépense un peu de mémoire pour gagner du temps.
Recherche : dans le *pire* cas, on reste en $O(n \times m)$ (comme le naïf). Mais en **pratique**, sur un texte « ordinaire », les sauts sont si grands qu'on est **sous-linéaire** : de l'ordre de $O(n/m)$ comparaisons.

C'est le **prodige** de Boyer-Moore : il peut trouver un mot **sans lire tout le texte**. Et — contre-intuition merveilleuse — *plus le motif est long, plus la recherche est rapide*, car les sauts possibles sont plus grands. C'est cet algorithme (ou ses cousins) qui fait tourner `Ctrl+F` et `grep`.

► **Exercices d'application :** exercices 8 **et** 9 (le coût sous-linéaire ; chercher un gène dans l'ADN)

VI Ouverture

- **La recherche textuelle est partout** : le `Ctrl+F` des éditeurs et `grep`, la bio-informatique (localiser un gène dans un génome de milliards de bases), les antivirus (repérer la « signature » d'un logiciel malveillant), la détection de plagiat, les moteurs de recherche, les filtres anti-spam…

> **Pour les curieux** — *Liens avec d'autres chapitres* (facultatif)
>
> - **Ce qu'il réutilise :** la table de prétraitement est un dictionnaire ; comme la mémoïsation (*Programmation dynamique*), on dépense de la mémoire pour gagner du temps.
> - **Déjà croisé :** `LIKE` cherche un motif en SQL (*Bases de données et SQL*) ; la plus longue sous-séquence commune (*Programmation dynamique*) compare deux textes.
> - **Pour aller plus loin :** vous verrez en *Calculabilité et décidabilité* qu'aucun antivirus ne peut détecter *tous* les virus.

Bilan — la carte mémoire

Notion	À retenir
Recherche textuelle	trouver un **motif** (m) dans un **texte** (n)
Méthode naïve	fenêtre glissante, gauche→droite, décalage 1 ; $O(n\,m)$
Boyer-Moore, idée 1	comparer le motif **de droite à gauche**
Boyer-Moore, idée 2	**prétraiter** le motif pour **sauter loin**
Mauvais caractère	lettre absente du motif ⇒ motif placé juste après elle (saut $j+1$, soit m si l'échec a lieu sur la dernière lettre)
Horspool	saut selon la lettre **alignée avec la fin** ; 1 table
Table (Horspool)	lettre en position k → décalage $m-1-k$; défaut m
Coût	prétraitement $O(m)$; recherche souvent **sous-linéaire** $O(n/m)$

Retour à la situation de départ

1. C'est ce que fait la méthode **naïve** : elle décale le motif d'un cran à chaque échec, jusqu'à $n \times m$ comparaisons au pire. Les éditeurs utilisent plutôt **Boyer-Moore** ou ses cousins, qui comparent de droite à gauche et sautent plusieurs crans d'un coup (§ II et III).
2. Oui : grâce au **prétraitement** du motif (une table de décalage), si la lettre observée n'apparaît pas dans le motif, on saute toute sa longueur sans lire les lettres survolées (§ III et IV).
3. Parce que les sauts peuvent atteindre la longueur m du motif : en pratique, la recherche demande de l'ordre de n/m comparaisons, elle est **sous-linéaire** (§ V).

Erreurs fréquentes

- **Comparer de gauche à droite** avec Boyer-Moore. Non : on lit le motif **à l'envers**.
- **Mettre la dernière lettre dans la table de Horspool.** On l'*exclut* (sinon le décalage serait 0 et l'algorithme bouclerait).
- **Oublier le décalage par défaut** m pour une lettre absente du motif.
- **Croire Boyer-Moore toujours plus rapide.** Au pire cas il reste en $O(n\,m)$; c'est *en pratique* qu'il brille.
- **Confondre indice du texte et indice du motif** dans `texte[i + j]` / `motif[j]`.

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

☐ **écrire** la recherche naïve et **évaluer** son coût → ex. 1, 2 ;

☐ **expliquer** les deux idées de Boyer-Moore (lire à l'envers, sauter) → ex. 3, 11 ;

☐ **construire** la table de prétraitement (Boyer-Moore *et* Horspool) → ex. 3, 5 ;

☐ **dérouler** Horspool sur un texte donné (suite des fenêtres et des sauts) → ex. 6, 13 ;

☐ **justifier** pourquoi la recherche peut être *sous-linéaire* → ex. 8.

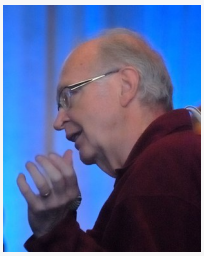
Vers le Grand Oral

- **Comment un ordinateur peut-il trouver un mot sans lire tout le texte ?** *(Boyer-Moore, sauts, sous-linéarité.)*
- **Pourquoi lire un mot « à l'envers » le fait-il trouver plus vite ?** *(comparaison droite→gauche, mauvais caractère.)*
- **Comment cherche-t-on un gène dans trois milliards de bases ?** *(recherche textuelle, alphabet ADN, algorithmes rapides.)*
- **Un antivirus « lit-il » vraiment tout mon disque ?** *(signatures, motifs, coût de la recherche.)*

Un peu d'histoire

Pour les curieux (facultatif)

En **1977**, Robert **Boyer** et J Strother **Moore** publient leur algorithme : le premier à trouver un motif en lisant *moins* de caractères qu'il n'y en a. La même année, une autre équipe (**Knuth**, **Morris** et **Pratt**) propose une approche différente, fondée sur les *préfixes* du motif. En **1980**, Nigel **Horspool** simplifie Boyer-Moore en ne gardant que la règle du mauvais caractère — une version si claire et si efficace qu'elle est aujourd'hui la plus enseignée. Un demi-siècle plus tard, ces idées font encore battre chaque `Ctrl+F` de la planète.



Donald Knuth en 2012

Parmi ces noms, **Donald Knuth** (né en 1938) est une légende : il écrit depuis 1962 *The Art of Computer Programming*, encyclopédie de l'algorithmique toujours inachevée, et il a créé **TEX**, le logiciel de composition sur lequel repose… la mise en page de ce cours.

EXERCICES

Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

Rappel : les positions sont comptées à partir de 0. Niveau : ★ échauffement ★★ classique ★★★ défi (dont les extraits de bac les plus exigeants) ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.

La méthode naïve ↑ cours

★ **Exercice 1** *Naïf, à la main* ▶ corrigé p. 976 • essentiel

On cherche le motif `AA` dans le texte `CAAAB`.

1. Dérouler la méthode naïve : à quelle(s) position(s) trouve-t-on le motif ?
2. Cette méthode compare le motif de la gauche vers la droite et décale d'un cran à chaque échec. Quel est son coût dans le **pire des cas**, en fonction de n (longueur du texte) et m (longueur du motif) ?

▷ coup de pouce

★ **Exercice 2** *Naïf, en Python* ▶ corrigé p. 976

Recopier et compléter la fonction de recherche naïve : ▷ coup de pouce

```
1  def recherche_naive(motif, texte):
2      n, m = len(texte), len(motif)
3      positions = []
4      for i in range(...):                # (a) toutes les positions de depart
5          j = 0
6          while j < m and texte[i + j] == motif[j]:
7              j += 1
8          if ... :                         # (b) tout le motif a correspondu ?
9              positions.append(i)
10     return positions
```

Boyer-Moore ↑ cours

★★ **Exercice 3** *Les deux idées, et un grand saut* ▶ corrigé p. 976 • essentiel

1. Rappeler les **deux idées** de l'algorithme de Boyer-Moore.
2. On considère le motif `CGGCAG`. Donner, pour chaque lettre du motif, sa **dernière position** (table du prétraitement de Boyer-Moore).
3. On compare (de droite à gauche) et l'échec se produit **dès la première comparaison**, sur la dernière lettre du motif : la lettre du texte qui provoque l'échec est un `T`. Sachant que `T` n'apparaît pas dans `CGGCAG`, de combien de crans peut-on décaler le motif d'un seul coup ? Justifier.

▷ coup de pouce

★★ **Exercice 4** *Dérouler Boyer-Moore* ▶ corrigé p. 977

On cherche le motif `GCAGC` dans le texte `ACGTGCAGCTGCAGC` (règle du mauvais caractère). La table des dernières positions est `G`→ 3, `C`→ 4… *(la compléter au besoin).*

position i	fenêtre du texte	échec / trouvé	décalage
$i = 0$	`ACGTG`		
… (compléter jusqu'à la fin du texte) …			

Donner la (les) position(s) d'occurrence. ▷ coup de pouce

Boyer-Moore-Horspool ↑ cours

★★ **Exercice 5** *Construire la table de décalage* ▶ corrigé p. 977

Pour un motif de longueur m, la table de Horspool donne à chaque lettre du motif, **sauf celle de la dernière position**, le décalage $m - 1 - k$, où k est sa position (à partir de 0) ; si une lettre apparaît plusieurs fois, on garde sa position la plus à droite (hors dernière position). Toute autre lettre reçoit m.

Construire la table de décalage de chacun de ces motifs (préciser le décalage par défaut) :

1. `ANANAS`
2. `GATTACA`
3. `ABRACADABRA`

▷ coup de pouce

★★ **Exercice 6** *Dérouler Horspool* ▶ corrigé p. 977 • essentiel

On cherche `ACGT` dans `GTACACGTAC`. Table : `A`→ 3, `C`→ 2, `G`→ 1, autres → 4.

Recopier et compléter le tableau (**la lettre de fin** est celle du texte alignée avec la dernière case de la fenêtre) :

position i	fenêtre	lettre de fin	décalage
$i = 0$	`GTAC`	`C`	…
…			

Conclure : le motif est-il présent, et où ? ▷ coup de pouce

★★ **Exercice 7** *Horspool en Python* ▶ corrigé p. 977

Recopier et compléter les deux fonctions : ▷ coup de pouce

```
1  def table_decalage(motif):
2      m = len(motif)
3      dec = {}
4      for k in range(...):              # (a) toutes les lettres SAUF la derniere
5          dec[motif[k]] = ...           # (b) le decalage
6      return dec
7
8  def horspool(motif, texte):
9      n, m = len(texte), len(motif)
10     dec = table_decalage(motif)
11     positions = []
12     i = 0
13     while i <= n - m:
14         j = m - 1
15         while j >= 0 and texte[i + j] == motif[j]:
16             j -= 1
17         if j < 0:
18             positions.append(i)
```

```
        c = texte[i + m - 1]
        if c in dec:
            i += dec[c]
        else:
            i += ...                    # (c) lettre absente de la table
    return positions
```

Coût ↑cours

★★ **Exercice 8** *Le miracle du sous-linéaire* ▶ corrigé p. 978 • essentiel

1. Combien coûte le **prétraitement** (construction de la table) en fonction de m ?
2. Expliquer pourquoi Horspool peut trouver un motif **sans lire tous les caractères** du texte. ▷ coup de pouce
3. Vrai ou faux (justifier) : « plus le motif est long, plus la recherche est *lente* ».
4. Boyer-Moore est-il *toujours* plus rapide que la méthode naïve ? ▷ coup de pouce

★★★ **Exercice 9** *Exercice d'entraînement — Chercher un gène dans l'ADN* ▶ corrigé p. 978 ≈ 30 min

On modélise un brin d'ADN par une chaîne sur l'alphabet `{A, T, G, C}`, et on cherche le motif `GATTACA` par l'algorithme de Horspool.

1. Construire la **table de décalage** de `GATTACA` (préciser le décalage par défaut).
2. Dérouler Horspool sur le brin `GTAGATTACAGT` : donner la suite des positions i testées, la lettre de fin de fenêtre, et le décalage à chaque étape. À quelle position trouve-t-on le motif ? ▷ coup de pouce ▷ début de solution
3. On donne la fonction ci-dessous, qui **compte** le nombre d'occurrences. Compléter les lignes `(a)` et `(b)`. ▷ coup de pouce

```
def compte_occurrences(motif, texte):
    dec = table_decalage(motif)
    n, m = len(texte), len(motif)
    nb = 0
    i = 0
    while i <= n - m:
        j = m - 1
        while j >= 0 and texte[i + j] == motif[j]:
            j -= 1
        if ... :                        # (a) motif entierement retrouve ?
            nb = nb + 1
        c = texte[i + m - 1]
        if c in dec:
            i += dec[c]
        else:
            i += ...                    # (b) decalage par defaut ?
    return nb
```

4. Un génome humain compte environ 3 milliards de bases. En quoi le choix de Horspool plutôt que la méthode naïve est-il crucial ici ?

Vers le bac ↑cours

★ **Exercice 10** *Type bac — Parcourir une chaîne de caractères* (d'après Centres étrangers 2023, jour 2) ▶ corrigé p. 978 ≈ 15 min

On rappelle quelques notions sur la manipulation des chaînes de caractères en Python. Une chaîne de caractères se comporte comme un tableau de caractères que l'on ne peut pas modifier. Par exemple :

```
>>> une_chaine = 'Bonjour'
>>> une_chaine[3]
'j'
>>> une_chaine[3] = 'z'
TypeError: 'str' object does not support item assignment
```

On peut aussi utiliser l'opérateur de concaténation `+` :

```
>>> une_chaine = 'a' + 'b'
>>> une_chaine
'ab'
>>> une_chaine = une_chaine + 'c'
>>> une_chaine
'abc'
```

On définit la fonction `bonjour` par le code suivant :

```
1 def bonjour(nom):
2     return 'Bonjour ' + nom + ' !'
```

1. Donner le résultat de l'exécution de `bonjour('Alan')`.
2. On exécute le programme suivant :

   ```
   1 une_chaine = 'Bonjour'
   2 x = (une_chaine[2] == une_chaine[3])
   3 y = (une_chaine[4] == une_chaine[1])
   ```

   Donner le type et les valeurs des variables `x` et `y`.
3. ▶ Écrire une fonction `occurrences_lettre(une_chaine, une_lettre)` prenant en paramètres une chaîne `une_chaine` et une lettre `une_lettre` et renvoyant le nombre d'occurrences de `une_lettre` dans `une_chaine`.

▷ coup de pouce

★ **Exercice 11** *Type bac — Le principe de Boyer-Moore* (d'après Asie 2025, jour 1) ▶ corrigé p. 979 ≈ 10 min

Dans ce sujet, on veut repérer un mot-clé (par exemple `"while"`) dans le texte d'un programme Python. On suppose disposer d'une fonction `recherche(mot, texte)` qui renvoie `True` si la chaîne de caractères `mot` est présente dans la chaîne de caractères `texte`, et `False` sinon.

1. Expliquer succinctement le principe de l'algorithme de Boyer-Moore, qui permet d'implémenter cette fonction `recherche`.

▷ coup de pouce

★★★ **Exercice 12** *Type bac — Rechercher et remplacer dans un éditeur* (sujet maison, façon bac) ▶ corrigé p. 979 ≈ 45 min

Un éditeur de texte propose deux commandes : **Rechercher**, qui surligne toutes les occurrences d'un motif, et **Remplacer tout**, qui remplace chaque occurrence d'un motif par un autre texte. On étudie leur programmation avec la méthode naïve. On rappelle la fonction du cours :

```
def recherche_naive(motif, texte):
    n, m = len(texte), len(motif)
    positions = []
    for i in range(n - m + 1):
        j = 0
        while j < m and texte[i + j] == motif[j]:
            j += 1
        if j == m:
            positions.append(i)
    return positions
```

Dans les questions 1 à 3 et 6, la variable `texte` contient la chaîne `'LE CHAT ET LE CHATON'`.

1. Donner la longueur n de `texte`, puis la valeur renvoyée par chacun des appels suivants :

   `recherche_naive('CHAT', texte)` et `recherche_naive('LE', texte)`

2. Combien de positions de départ `i` la boucle `for` examine-t-elle lors de la recherche du motif `'CHAT'` ?

3. Pour mesurer le coût, on compte les **comparaisons de caractères**, c'est-à-dire le nombre d'évaluations de `texte[i + j] == motif[j]`. Expliquer pourquoi, lors de la recherche de `'CHAT'`, toutes les positions de départ sauf deux ne coûtent qu'une seule comparaison. En déduire le nombre total de comparaisons.

4. Donner un motif de longueur 4 et un texte de longueur 12, écrits avec les lettres `A` et `B`, pour lesquels la méthode naïve effectue le plus grand nombre possible de comparaisons. Combien en effectue-t-elle ? Exprimer ce nombre en fonction de n et m. ▷ coup de pouce

5. ▶ Écrire une fonction `est_occurrence(motif, texte, i)` qui renvoie `True` si `motif` apparaît dans `texte` à partir de la position `i`, et `False` sinon. On suppose que `i + len(motif) <= len(texte)`. ▷ début de solution

   ▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

6. ▶ Pour la commande **Remplacer tout**, on parcourt le texte de gauche à droite en construisant une nouvelle chaîne. Recopier et compléter les lignes `(a)` à `(d)`, puis donner la valeur renvoyée par `remplacer_tout('CHAT', 'LOUP', texte)`. ▷ coup de pouce

```
def remplacer_tout(motif, nouveau, texte):
    n, m = len(texte), len(motif)
    resultat = ''
    i = 0
    while i < n:
        if i <= n - m and est_occurrence(motif, texte, i):
            resultat = ...          # (a)
            i = ...                 # (b)
        else:
            resultat = ...          # (c)
            i = ...                 # (d)
    return resultat
```

7. L'appel `recherche_naive('AA', 'AAA')` renvoie `[0, 1]`, alors que `remplacer_tout('AA', 'B', 'AAA')` renvoie `'BA'`. Expliquer cette différence, et pourquoi c'est bien le comportement attendu d'un éditeur.

8. Un fichier contient un million de caractères et l'on cherche un motif de 10 caractères. Donner l'ordre de grandeur du nombre de comparaisons de la méthode naïve dans le pire des cas. Citer un algorithme du cours plus efficace en pratique, et l'idée qui le rend plus rapide.

★★ **Exercice 13** *Type bac — Un antivirus à la recherche de signatures* (sujet maison, façon bac) ▶ corrigé p. 980 ≈ 45 min

Un antivirus repère un logiciel malveillant connu en cherchant sa **signature** — une courte suite d'octets qui le caractérise — dans le contenu des fichiers. Pour simplifier, le contenu d'un fichier et les signatures sont représentés par des chaînes de caractères écrites avec les chiffres hexadécimaux `0` à `9` et `A` à `F`, et l'on ne se préoccupe pas du découpage en octets.

L'antivirus utilise l'algorithme de **Boyer-Moore-Horspool**. On dispose des fonctions `table_decalage(motif)` et `horspool(motif, texte)` du cours ; `horspool` renvoie la liste des positions des occurrences de `motif` dans `texte`.

1. Rappeler comment est construite la table de décalage d'un motif de longueur m. Construire la table de décalage de la signature `'4D5A90'` (préciser le décalage par défaut).

2. Construire la table de décalage de la signature `'E8FFFF'`. La lettre `F` est la dernière lettre du motif : pourquoi figure-t-elle pourtant dans la table, et avec quel décalage ? ▷ coup de pouce

3. Le contenu d'un fichier, de 22 caractères, est :

   ```
   contenu = '00E84D5A0090FF4D5A90C3'
   ```

   Dérouler `horspool('4D5A90', contenu)` en recopiant et complétant le tableau ci-dessous. Quelle valeur la fonction renvoie-t-elle ?

position i	fenêtre	lettre de fin	décalage
$i = 0$	00E84D	…	…
…			

4. Combien d'alignements Horspool a-t-il testés ? Combien la méthode naïve en aurait-elle testés ? Combien de comparaisons de caractères Horspool a-t-il effectuées au total ?

5. ▶ Écrire une fonction `est_infecte(contenu, signatures)` qui prend en paramètres le contenu d'un fichier et une liste de signatures, et renvoie `True` si au moins une des signatures apparaît dans le contenu, `False` sinon. On utilisera la fonction `horspool`. ▷ coup de pouce

   ▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

6. L'antivirus analyse 100 000 fichiers avec la même liste de 1 000 signatures. Avec la fonction `est_infecte`, combien de fois, au plus, la table de décalage d'une même signature est-elle construite ? Proposer, sans écrire tout le code, une organisation qui évite ce travail inutile.

7. L'éditeur de l'antivirus affirme : « plus nos signatures sont longues, plus l'analyse est rapide ». Cette affirmation est-elle plausible ? Justifier.

★★★ **Exercice 14** *Type bac — Repérer des amorces dans un brin d'ADN* (sujet maison, façon bac) ▶ corrigé p. 981 ≈ 45 min

En biologie moléculaire, une **amorce** est une courte séquence d'ADN qui ne peut se fixer sur un brin qu'à l'endroit où elle apparaît telle quelle. Avant une expérience, un laboratoire vérifie par programme où ses amorces apparaissent dans un brin. Un brin et une amorce sont représentés par des chaînes de caractères sur l'alphabet `{A, C, G, T}`. Dans tout l'exercice, `brin = 'CATGGATGCATGCC'` (14 caractères) et l'on cherche l'amorce `'TGCATG'`.

Partie A — Boyer-Moore (règle du mauvais caractère)

On donne les fonctions suivantes :

```
def table_dernier(motif):
    dernier = {}
    for k in range(len(motif)):
        dernier[motif[k]] = k
    return dernier

def boyer_moore(motif, texte):
    n, m = len(texte), len(motif)
    dernier = table_dernier(motif)
    positions = []
    i = 0
    while i <= n - m:
        j = m - 1
        while j >= 0 and texte[i + j] == motif[j]:
            j -= 1
        if j < 0:
            positions.append(i)
            i += 1
        else:
            c = texte[i + j]
            if c in dernier:
                k = dernier[c]
            else:
                k = -1
            i += max(1, j - k)
    return positions
```

1. Donner le dictionnaire renvoyé par `table_dernier('TGCATG')`.
2. Dérouler `boyer_moore('TGCATG', brin)` : pour chaque position i testée, donner la fenêtre du brin, l'indice j et le caractère `c` qui provoquent l'échec (ou « trouvé »), et le décalage appliqué. Quelle valeur la fonction renvoie-t-elle ? ▷ coup de pouce
3. Lors de ce déroulé, la quantité `j - k` a été négative. À quelle position i ? Que se passerait-il si l'on remplaçait `max(1, j - k)` par `j - k` ? ▷ coup de pouce

Partie B — Boyer-Moore-Horspool

4. Construire la table de décalage de Horspool de l'amorce `'TGCATG'` (préciser le décalage par défaut).
5. Dérouler l'algorithme de Horspool sur le même brin (position i, fenêtre, lettre de fin de fenêtre, décalage). Comparer le nombre d'alignements testés par les deux algorithmes sur cet exemple.

Partie C — Programmer

6. ▶ Écrire une fonction `cherche_amorces(amorces, brin)` qui prend une liste d'amorces et un brin, et renvoie un **dictionnaire** associant à chaque amorce la liste de ses positions dans le brin (on utilisera la fonction `horspool` du cours). Par exemple, `cherche_amorces(['TGCATG', 'GGA', 'AAAA'], brin)` renvoie `{'TGCATG': [6], 'GGA': [3], 'AAAA': []}`.

▷ début de solution

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

7. ▶ En utilisant `cherche_amorces`, écrire une fonction `amorces_absentes(amorces, brin)` qui renvoie la liste des amorces qui n'apparaissent pas dans le brin.

8. Un génome humain compte environ 3 milliards de bases et une amorce environ 20 bases. Donner l'ordre de grandeur du nombre d'alignements testés par la méthode naïve, puis par Horspool dans un cas favorable où chaque saut vaut la longueur de l'amorce.

Critiquer une réponse d'IA ↑ cours

★★ **Exercice 15** *Une table de décalage construite par un assistant d'IA* ▶ corrigé p. 982

Un élève demande à un assistant d'IA : « Écris en Python la fonction `table_decalage(motif)` de l'algorithme de Horspool, puis la fonction `horspool(motif, texte)` qui renvoie la position de la première occurrence (ou `-1`). » Voici la réponse obtenue :

```
1  def table_decalage(motif):
2      m = len(motif)
3      dec = {}
4      for k in range(m):                # chaque lettre du motif
5          dec[motif[k]] = m - 1 - k     # distance a la fin du motif
6      return dec
7
8  def horspool(motif, texte):
9      n, m = len(texte), len(motif)
10     dec = table_decalage(motif)
11     i = 0
12     while i <= n - m:
13         j = m - 1
14         while j >= 0 and texte[i + j] == motif[j]:
15             j -= 1                    # comparaison droite -> gauche
16         if j < 0:
17             return i                  # motif trouve en i
18         c = texte[i + m - 1]          # lettre de fin de fenetre
19         if c in dec:
20             i += dec[c]
21         else:
22             i += m
23     return -1
```

« La table donne, pour chaque lettre du motif, sa distance à la fin ; une lettre absente vaut `m`. La recherche compare de droite à gauche et saute selon la lettre alignée avec la fin de la fenêtre. »

1. La réponse est-elle correcte ? Construire la table pour `ANANAS`, puis dérouler `horspool("ANANAS", "ANANES ANANAS")` : noter i, la lettre de fin de fenêtre et le décalage à chaque étape.

2. Localiser et corriger l'erreur. ▷ coup de pouce

3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

S'entraîner à l'oral ↑cours

★★ **Exercice 16** *Expliquer en deux minutes* ▶ corrigé p. 983

Au Grand Oral comme devant l'examinateur de l'épreuve pratique, il faut savoir **expliquer** une notion clairement, sans lire ses notes. Choisir l'un des sujets suivants (ou le tirer au sort) :

Sujet 1. Expliquer à un camarade de Première la recherche **naïve** d'un motif dans un texte et son coût.

Sujet 2. Expliquer l'idée de **Boyer-Moore-Horspool** : pourquoi comparer en partant de la fin du motif permet de sauter des positions.

Sujet 3. Expliquer pourquoi Boyer-Moore-Horspool est d'autant plus rapide que le motif est long, et dans quel cas il ne gagne rien.

Préparation (5 min, seul). Noter au brouillon **trois idées** et **un exemple**, rien de plus : comme au Grand Oral, ces notes servent d'aide-mémoire. **Exposé (2 min, chronométré).** Debout, face à un camarade, **sans lire ses notes** et sans rien écrire : tout passe par la parole. **Retour (2 min).** Le camarade pose une question de relance (on peut alors s'aider d'un schéma), remplit la grille ci-dessous et donne un conseil ; puis on échange les rôles. Cette grille reprend les critères d'évaluation du Grand Oral. ▷ coup de pouce

Grille entre camarades	Oui	En partie	Pas encore
Clair — audible, posé ; chaque mot technique est expliqué	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	☐	☐	☐
Construit — un fil conducteur, tenu en deux minutes, sans lire	☐	☐	☐
Un conseil pour la prochaine fois :			

Entraînement ↑cours

Des séries courtes pour rendre une méthode automatique : même consigne, données différentes. Les réponses sont brèves dans le corrigé.

★ **Exercice 17** *Tables de décalage en série* ▶ corrigé p. 983 • essentiel

Construire la table de décalage de Horspool de chacun des motifs suivants, en précisant le décalage par défaut. ▷ coup de pouce

a) `CHAT`
b) `MAMAN`
c) `RADAR`
d) `BONBON`

★ **Exercice 18** *Horspool à la main en série* ▶ corrigé p. 984 • essentiel

Pour chaque recherche, construire la table de décalage du motif, puis dérouler l'algorithme de Horspool : à chaque étape, donner la position i, la fenêtre, la lettre de fin de fenêtre et le décalage. Donner enfin les positions des occurrences. ▷ coup de pouce

a) motif `TAC`, texte `TICTACTOC`
b) motif `GAG`, texte `TGAGCGAGT`
c) motif `CODE`, texte `DECODERLECODE`

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Placer le motif sous le texte à chaque position possible, de 0 à $n - m$, et comparer lettre à lettre. Pour le coût, imaginer un texte et un motif où presque toutes les lettres coïncident à chaque position. ↑énoncé

Ex. 2 – piste Ligne `(a)` : la dernière position de départ possible est celle où le motif touche la fin du texte. Ligne `(b)` : à la sortie de la boucle `while`, que vaut `j` si toutes les lettres ont correspondu ? ↑énoncé

Ex. 3 q. 3 – piste Si `T` n'apparaît nulle part dans le motif, un alignement où le motif recouvre ce `T` peut-il donner une occurrence ? ↑énoncé

Ex. 4 – piste À chaque échec, repérer la lettre `c` du texte qui fait échouer et l'indice j du motif où cela se produit, puis appliquer la règle du mauvais caractère du cours (décalage d'au moins 1). ↑énoncé

Ex. 5 q. 3 – piste Parcourir les lettres de gauche à droite en ignorant la dernière : quand une lettre revient, sa nouvelle valeur remplace l'ancienne. ↑énoncé

Ex. 6 – piste À chaque étape, la lettre de fin est celle du texte en position $i + 3$; son décalage se lit dans la table. ↑énoncé

Ex. 7 – piste Pour `(c)`, la lettre `c` n'est pas dans la table : elle n'apparaît pas dans le motif (sauf peut-être en dernière position). De combien peut-on alors décaler la fenêtre ? ↑énoncé

Ex. 8 q. 2 – piste Q2 : quand on saute de m crans, quelles lettres du texte n'ont jamais été lues ? ↑énoncé

Ex. 8 q. 4 – piste Q4 : penser au pire cas, par exemple chercher `AAA` dans `AAAAAAAA` : les sauts sont-ils grands ? combien de comparaisons par alignement ? ↑énoncé

Ex. 9 q. 2 – piste Q2 : le motif a 7 lettres : la lettre de fin de la fenêtre i est `brin[i + 6]`. ↑énoncé

Ex. 9 q. 2 – début de solution Q2 : première étape : $i = 0$, fenêtre `GTAGATT`, lettre de fin `T`, décalage 3. ↑énoncé

Ex. 9 q. 3 – piste Q3 : comparer avec la fonction `horspool` du cours : quelle condition signalait une occurrence ? ↑énoncé

Ex. 10 – piste Question 2 : relever les lettres d'indices 1, 2, 3 et 4 de `'Bonjour'` (le premier indice est 0) ; une comparaison avec `==` donne un booléen. Question 3 : un compteur et une boucle sur les caractères de la chaîne. ↑énoncé

Ex. 11 – piste Relire les deux idées de Boyer-Moore dans le cours : dans quel sens compare-t-on le motif au texte, et que permet la table calculée à l'avance quand une comparaison échoue ? ↑énoncé

Ex. 12 q. 4 – piste Q4 : pour que chaque position de départ coûte m comparaisons, le motif doit « presque » correspondre partout : quel texte et quel motif choisir avec seulement deux lettres ? ↑énoncé

Ex. 12 q. 5 – début de solution Q5 :

```
def est_occurrence(motif, texte, i):
    for j in range(len(motif)):
        if texte[i + j] != motif[j]:
```

↑énoncé

Ex. 12 q. 6 – piste Q6 : deux cas : une occurrence commence en `i`, ou non. Dans chaque cas, que faut-il ajouter à `resultat`, et de combien faut-il avancer ? ↑énoncé

Ex. 13 q. 2 – piste Q2 : seule la *dernière position* du motif est ignorée : la lettre `F` apparaît-elle aussi ailleurs dans `E8FFFF` ? ↑énoncé

Ex. 13 q. 5 – piste Q5 : parcourir la liste des signatures : que renvoie `horspool` quand la signature est absente ? Peut-on conclure dès la première trouvée ? ↑énoncé

Ex. 14 q. 2 – piste Q2 : commencer à $i = 0$: comparer `brin[5]` et `motif[5]`, puis reculer tant que les lettres coïncident ; le décalage vient de la ligne `i += max(1, ...)`. ↑énoncé

Ex. 14 q. 3 – piste Q3 : chercher l'étape où la dernière occurrence de `c` dans le motif est *à droite* de l'indice j. ↑énoncé

Ex. 14 q. 6 – début de solution Q6 :

```
def cherche_amorces(amorces, brin):
    resultat = {}
    for amorce in amorces:
```

↑énoncé

Ex. 15 q. 2 – piste Comparer la boucle de `table_decalage` avec celle du cours : quelles lettres sont parcourues ? Quelle valeur reçoit alors la dernière lettre ? ↑énoncé

Ex. 16 – piste Structurer chaque explication en trois temps : une **définition** en une phrase, un **exemple** petit et concret déroulé à voix haute, puis le **piège** (l'erreur que l'auditeur risque de faire). Finir par une phrase qui répond à la question posée. Prévoir un motif court (quatre lettres) et un texte d'une ligne, que l'on fait « glisser » à voix haute. Sujet 2 : quel caractère du texte regarde-t-on pour décider du saut ? ↑énoncé

Ex. 17 – piste Écrire la position de chaque lettre sous le motif. Parcourir les lettres de gauche à droite **sauf la dernière** : la lettre en position k reçoit $m-1-k$, et une lettre qui revient voit sa valeur remplacée. Toute lettre absente de la table reçoit m. ↑énoncé

Ex. 18 – piste La lettre de fin de fenêtre est la lettre du **texte** en position $i+m-1$. On compare la fenêtre au motif de droite à gauche, puis, trouvé ou non, on décale de la valeur que la table donne pour la lettre de fin. On s'arrête quand la fenêtre dépasse la fin du texte. ↑

TP ET PROJETS

TP La course des algorithmes de recherche ▶ corrigé p. 984

compter, chronométrer et comparer naïf, Horspool et l'opérateur `in`

Durée : 2 h 30 à 3 h (deux séances) · Sur machine, par deux

Fichiers à télécharger : `maththibaud.github.io/informatique-en-autonomie/f/t12-tp-recherche-textuelle` — **archive .zip**

Consignes

- Fichier à télécharger (lien ci-dessus) : `tp_recherche_depart.py` (outils, squelettes et tests). Le lancer affiche l'état des tests.
- Texte d'étude : *Le Tour du monde en quatre-vingts jours* de Jules Verne (1872), **domaine public**. Le télécharger sur `gutenberg.org` (chercher le titre, choisir le format « Plain Text UTF-8 ») et l'enregistrer sous le nom `verne.txt`, dans le dossier du fichier Python. *Sans connexion : faire les parties C à E sur le génome seul.*
- Niveau : ★ échauffement ★★ classique ★★★ défi (dont les extraits de bac les plus exigeants) ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme. Le symbole ▶ signale ce qui est à programmer et tester.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

But du TP

Transformer les algorithmes du cours en **instruments de mesure** : chaque recherche renverra, en plus des positions trouvées, le **nombre de comparaisons de caractères** effectuées. On fera ensuite courir la méthode naïve et Boyer-Moore-Horspool sur un **vrai roman** et sur un **génome** d'un million de bases, on étudiera l'effet de la **longueur du motif** et de la **taille de l'alphabet**, on construira des **cas défavorables**, et on se mesurera à l'opérateur `in` de Python. *Produit final* : des tableaux de mesures et un bilan argumenté sur le coût de la recherche textuelle.

Convention de comptage. On compte **une comparaison** chaque fois qu'un caractère du texte est comparé à un caractère du motif, que la comparaison réussisse ou échoue.

Partie A — Compter le travail

★ Exercice 1 *La méthode naïve, avec compteur*

1. À la main : combien de comparaisons la méthode naïve fait-elle pour chercher `ACG` dans `CAAGACG` ? (Reprendre le schéma du cours : 5 alignements. Faire ce calcul *avant* de lancer les tests du fichier de départ.)

2. ▶ Compléter `naive_compte(motif, texte)`, qui renvoie le couple `(positions, comparaisons)`. Pour compter aussi la comparaison qui échoue, on peut écrire la boucle intérieure ainsi :

```
        j = 0
        while j < m and texte[i + j] == motif[j]:
            comp += 1                    # comparaison reussie
            j += 1
        if j < m:
            comp += 1                    # la comparaison qui a echoue
```

   Lancer les tests et comparer le nombre de comparaisons renvoyé par `naive_compte("ACG", "CAAGACG")` avec votre calcul de la question 1.

★ Exercice 2 *Horspool, avec compteur*

1. À la main : dérouler Horspool pour `ACG` dans `CAAGACG` (table : `A`→ 2, `C`→ 1, autres → 3). Combien d'alignements ? Combien de comparaisons ? *(Avant de lancer les tests.)*

2. ▶ Compléter `horspool_compte(motif, texte)` (même principe, comparaison de **droite à gauche** ; la fonction `table_decalage` est fournie). Vérifier les tests, dont l'exemple `MOTIF` du cours : combien de comparaisons pour Horspool ? et pour la méthode naïve ?

★★ Exercice 3 *Une référence, et mille tests*

1. ▶ Compléter `toutes_positions(motif, texte)`, une version « de référence », écrite autrement que les deux précédentes : pour chaque position `i` possible, on compare d'un coup la **tranche** `texte[i:i + m]` au motif. Attention : les occurrences peuvent se **chevaucher** (`AA` est trois fois dans `AAAA`). ▷ coup de pouce

2. ▶ Écrire une boucle de 1 000 **tests aléatoires** : texte de longueur 0 à 30 et motif de longueur 1 à 5, tous deux sur l'alphabet `"AB"` (avec `random.choice` et `random.randint`). À chaque tour, vérifier par un `assert` que les trois fonctions renvoient les mêmes positions. Pourquoi un alphabet de deux lettres est-il un bon choix pour ces tests ?

Partie B — Un vrai roman

★★ **Exercice 4** *À la recherche de Phileas Fogg*

▶ Charger le roman avec `texte = lire_texte("verne.txt")` et afficher sa longueur n.

1. ▶ Recopier et compléter le tableau (n est le nombre de caractères du texte) :

Motif	Occurrences	Comp. naïf	Comp. Horspool	Rapport naïf / Horspool
`Fix`				
`Fogg`				
`Aouda`				
`Passepartout`				
`quatre-vingts jours`				

2. Le nombre de comparaisons de la méthode naïve dépend-il beaucoup du motif ? Comparer à n. Et celui de Horspool ?

3. Pourquoi Horspool fait-il **moins de comparaisons qu'il n'y a de caractères** dans le texte ? Quels caractères n'a-t-il jamais lus ?

4. ▶ Chercher `fogg` (sans majuscule). Que se passe-t-il ? Comment chercher un mot « sans tenir compte des majuscules » ?

▷ coup de pouce

★★ **Exercice 5** *Un mini-*`grep`

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

La commande `grep` des systèmes Unix affiche les **lignes** d'un fichier qui contiennent un motif. On en écrit une version Python, fondée sur Horspool.

1. ▶ Écrire `grep(motif, texte)`, qui découpe le texte en lignes (`texte.split("\n")`) et renvoie la liste des couples `(numéro de ligne, ligne)` des lignes contenant le motif (la première ligne porte le numéro 1). Afficher les cinq premières lignes du roman qui contiennent `Passepartout`.

2. ▶ Écrire `surligner(motif, ligne)`, qui renvoie la ligne où chaque occurrence du motif est entourée de crochets. Exemple : `surligner("ab", "xxabyyab")` renvoie `"xx[ab]yy[ab]"`. Que choisir quand deux occurrences se chevauchent, comme pour `surligner("aa", "aaa")` ?

▷ coup de pouce

3. Le fichier de Gutenberg est coupé en lignes d'environ 70 caractères. Pour le motif `Phileas Fogg`, comparer : le nombre de lignes renvoyées par `grep`, le nombre d'occurrences trouvées par `horspool_compte` dans le texte entier, puis dans le texte où chaque fin de ligne `"\n"` a été remplacée par une espace (construire ce texte par un parcours des caractères). Expliquer les écarts.

Partie C — La longueur du motif, et la taille de l'alphabet

★★ **Exercice 6** *Plus le motif est long…*

1. ▶ Compléter `moyenne_comparaisons(algo, texte, m, essais=5, graine=1)` : elle choisit `essais` positions `p` au hasard (`random.randint(0, len(texte) - m)`), prend comme motif `texte[p:p + m]` (il est donc présent au moins une fois), et renvoie le nombre **moyen** de comparaisons de `algo`, divisé par la longueur du texte. Le paramètre `graine` est passé à `random.seed` (déjà écrit dans la trame) : les mêmes positions sont tirées à chaque exécution, ce qui rend les mesures reproductibles. Un résultat de 0,25 signifie : « en moyenne, une comparaison pour quatre caractères du texte ».

2. ▶ On crée un génome par `adn = genome(1_000_000)` (un million de bases `A`, `C`, `G`, `T`). Recopier et compléter le tableau (deux chiffres après la virgule) :

Longueur m du motif	**2**	**4**	**8**	**16**	**32**	**64**
Naïf, roman						
Horspool, roman						
Naïf, génome						
Horspool, génome						

3. Sur le roman, comment évolue le travail de Horspool quand m double ? Comparer avec $1/m$. Retrouve-t-on le « miracle du sous-linéaire » du cours ?

4. Sur le génome, Horspool cesse de s'améliorer au-delà de $m \approx 8$. Expliquer pourquoi : pour un long motif sur l'alphabet `ACGT`, que valent à peu près les décalages de la table ?▷ coup de pouce

5. Le naïf fait environ $1{,}33\,n$ comparaisons sur le génome. Justifier ce nombre : à une position donnée, quelle est la probabilité que la première comparaison réussisse ? les deux premières ?

6. *(Facultatif.)* ▶ Tracer les quatre courbes avec `matplotlib` (`plt.plot`, `plt.xscale("log")`).

Partie D — Les cas défavorables

★★ **Exercice 7** *Construire le pire*

On prend un texte de $n = 20\,000$ lettres `A` : `texte = "A" * 20000`.

1. ▶ Pour $m = 5$, 10, 20, 40, compter les comparaisons des deux algorithmes avec le motif `"A" * (m - 1) + "B"`, puis avec le motif `"B" + "A" * (m - 1)`. Présenter les résultats dans un tableau, avec une colonne $n \times m$.

2. Pour chaque motif, expliquer *quel* algorithme est piégé, et pourquoi (dans quel sens compare-t-il ? de combien décale-t-il ?).

3. Quel est donc le coût **dans le pire des cas** de chacun des deux algorithmes ? Ce pire cas est-il fréquent dans un texte en français ?

4. À l'inverse, construire un motif de 10 lettres pour lequel Horspool ne fait que $n/10$ comparaisons sur ce texte. Justifier.

Partie E — Face à Python

★★ **Exercice 8** *Contre la montre*

La fonction fournie `chrono(f, a, b)` exécute `f(a, b)` et renvoie le couple (résultat, durée en secondes).

1. ▶ On cherche deux motifs **absents** : chaque méthode doit donc parcourir tout le texte. Sur le génome, le motif est `genome(16, 1)` (16 bases tirées au hasard avec la graine 1, absentes du génome) ; sur le roman, c'est `"le gentleman dit"` (16 caractères, absent du roman). Pour chacun, mesurer la durée de `naive_compte`, de `horspool_compte` et de la fonction fournie `present(motif, texte)`, qui renvoie simplement `motif in texte`. Combien de fois l'opérateur `in` est-il plus rapide que votre Horspool, sur chacun des deux textes ?
2. L'opérateur `in` des chaînes est écrit en langage C et compilé, et son algorithme s'inspire justement de Boyer-Moore-Horspool. Expliquer l'écart de temps. Le nombre de comparaisons et la durée mesurent-ils la même chose ?

★★★ **Exercice 9** *Défi : Boyer-Moore réduit au mauvais caractère*

Dans le Boyer-Moore du cours, on regarde le caractère `c = texte[i + j]` qui a **provoqué l'échec** (position `j` du motif), et on décale de `max(1, j - k)`, où `k` est la **dernière** position de `c` dans le motif (lue dans un dictionnaire `dernier`), ou -1 si `c` n'apparaît pas dans le motif. Après une occurrence trouvée, on décale de 1.

1. ▶ Écrire `table_dernier(motif)` et `boyer_moore_compte(motif, texte)`, et les ajouter aux 1 000 tests de l'exercice 3.
2. ▶ Ajouter une ligne « Boyer-Moore » au tableau de l'exercice 6. Ce Boyer-Moore (réduit au mauvais caractère) fait-il mieux que Horspool ? Sur quel texte l'écart est-il le plus visible ?

Bilan du TP

À rédiger (une dizaine de lignes, en citant vos mesures)

1. Comparer le coût de la méthode naïve et de Horspool sur un texte « ordinaire », puis dans le pire des cas.
2. Expliquer pourquoi « plus le motif est long, plus la recherche est rapide », et pourquoi ce n'est plus vrai sur l'ADN.
3. À quoi sert le **prétraitement** du motif ? Combien coûte-t-il, comparé à la recherche elle-même ?
4. Pourquoi utiliser l'opérateur `in` en pratique, et à quoi bon avoir programmé Horspool ?

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 q. 1 – piste Les positions possibles vont de 0 à $n-m$; chaque position où la tranche est égale au motif est ajoutée à la liste, sans sauter de position (à cause des chevauchements). ↑

Ex. 4 q. 4 – piste Écrire une fonction `minuscules(texte)` qui recopie le texte caractère par caractère en remplaçant chaque majuscule `c` (de `"A"` à `"Z"`) par `chr(ord(c) + 32)`, puis chercher `fogg` dans le résultat. ↑énoncé

Ex. 5 q. 2 – piste Parcourir les positions renvoyées par `horspool_compte` en gardant un indice `i` : la partie de la ligne déjà recopiée. On ajoute `ligne[i:p]`, puis le motif entre crochets, puis `i` devient `p + len(motif)` ; une position `p` inférieure à `i` (chevauchement) est ignorée. ↑énoncé

Ex. 6 q. 4 – piste Afficher `table_decalage(adn[1000:1032])` : où se trouvent les dernières occurrences de `A`, `C`, `G` et `T` dans un motif de 32 bases ? Un saut ne peut pas dépasser ces distances. ↑énoncé

FICHE DE RÉVISION

L'essentiel

- **But** : trouver les **occurrences** d'un **motif** (longueur m) dans un **texte** (longueur n). Positions comptées à partir de 0.
- **Méthode naïve** : on superpose le motif au texte, on compare **de gauche à droite** ; au premier désaccord, on décale d'**un seul cran**. Coût $O(n \times m)$ (quadratique) au pire.
- **Boyer-Moore** repose sur **deux idées** : **(1)** comparer le motif **de droite à gauche** ; **(2) prétraiter** le motif pour **sauter de plusieurs crans** d'un coup.
- **Règle du mauvais caractère** : si la lettre `c` du texte fait échouer la comparaison,
  - `c` **absente** du motif $\Rightarrow$ on place le motif juste **après** `c` : saut de $j+1$ si l'échec a lieu en position j du motif, soit **toute la longueur** m si l'échec a lieu dès la dernière lettre ;
  - `c` **présente** $\Rightarrow$ on décale pour aligner `c` avec sa **dernière** occurrence dans le motif.
- **Horspool** (version simplifiée) : on décide du saut d'après **la lettre alignée avec la *fin* de la fenêtre**. **Table de décalage** : la lettre en position k (sauf la dernière) reçoit $m-1-k$; toute autre reçoit m.
- **Coût** : prétraitement $O(m)$; recherche souvent **sous-linéaire** $O(n/m)$ (on peut trouver un mot **sans lire tout le texte**). Plus le motif est **long**, plus c'est **rapide**.

Le code à savoir écrire

```python
def recherche_naive(motif, texte):
    n, m = len(texte), len(motif)
    positions = []
    for i in range(n - m + 1):
        j = 0
        while j < m and texte[i + j] == motif[j]:   # gauche -> droite
            j += 1
        if j == m:
            positions.append(i)
    return positions

def table_decalage(motif):              # pretraitement de Horspool
    m = len(motif)
    dec = {}
    for k in range(m - 1):              # on ignore la DERNIERE lettre
        dec[motif[k]] = m - 1 - k
    return dec

def horspool(motif, texte):
    n, m = len(texte), len(motif)
    dec = table_decalage(motif)
    positions = []
    i = 0
    while i <= n - m:
        j = m - 1
        while j >= 0 and texte[i + j] == motif[j]:   # DROITE -> GAUCHE
            j -= 1
        if j < 0:
            positions.append(i)
        c = texte[i + m - 1]           # lettre alignee avec la fin de la fenetre
        if c in dec:
            i += dec[c]                # saut lu dans la table
        else:
```

```
            i += m                      # lettre absente : saut de m
    return positions
```

À retenir aussi

- **Construire les tables** de `GATTACA` ($m = 7$) : Horspool `G`→ 6, `A`→ 2, `T`→ 3, `C`→ 1, défaut 7 ; Boyer-Moore (`table_dernier`, dernière position) `G`→ 0, `A`→ 6, `T`→ 3, `C`→ 5.
- **Dérouler Horspool** = donner, pour chaque fenêtre, la position i, la lettre de fin et le décalage.
- Applications : `Ctrl+F`, `grep`, antivirus (signatures), **bio-informatique** (chercher un gène dans un génome).

Erreurs à éviter

- comparer **de gauche à droite** avec Boyer-Moore (c'est **à l'envers**) ;
- mettre la **dernière lettre** dans la table de Horspool (on l'exclut) ;
- oublier le **décalage par défaut** m pour une lettre absente ;
- croire Boyer-Moore *toujours* plus rapide : au **pire cas** il reste en $O(n\,m)$;
- confondre l'indice du texte `i + j` et l'indice du motif `j`.

Savoir-faire à maîtriser — je sais…

☐ **écrire** la recherche naïve et **évaluer** son coût ;

☐ **expliquer** les deux idées de Boyer-Moore (lire à l'envers, sauter) ;

☐ **construire** la table de prétraitement (Boyer-Moore *et* Horspool) ;

☐ **dérouler** Horspool sur un texte donné (suite des fenêtres et des sauts) ;

☐ **justifier** pourquoi la recherche peut être *sous-linéaire*.

Se tester — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 990

1. Quel est le coût au pire de la recherche naïve d'un motif de longueur m dans un texte de longueur n ?
2. Donner la table de décalage de Horspool du motif `MAMAN`.
3. Horspool, motif `CHAT` : la fenêtre se termine sur la lettre `Z`. De combien décale-t-on ?
4. Pourquoi n'inscrit-on pas la dernière lettre du motif dans la table ?
5. Motif long, texte très long : méthode naïve ou Boyer-Moore ?

Et avant ? — *questions sur les chapitres précédents.* ▶ réponses p. 990

1. Dans une fonction mémoïsée, que contient le dictionnaire `memo` ? (ch. 11)
2. Une machine a l'adresse `192.168.1.20/24`. Quelle est l'adresse de son réseau ? (ch. 9)
3. Écrire la requête qui affiche, sans doublon, les identifiants des réalisateurs de la table `film`. (ch. 6)

Vers le bac

- Épreuve pratique : sujet 31 — Recherche d'un motif, p. 741

DÉFIS ADVENT OF CODE

DÉFI 1 Password Philosophy

les mots de passe — découpage de chaînes et ou exclusif

Jour 2

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t12-defi-aoc-2020-02` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/2`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Une base de mots de passe est en partie abîmée : chaque ligne contient un mot de passe et la règle qu'il devait respecter. Il faut compter les mots de passe conformes à leur règle. Le fichier compte un millier de lignes, toutes différentes : il faut un programme qui découpe chaque ligne, puis applique la règle.
Ce qu'on réinvestit. La **recherche dans un texte**, vue dans ce chapitre. Ici, le motif cherché est une simple lettre : on compte ses occurrences dans le mot de passe par une boucle de comptage, que l'on compare à la recherche naïve du cours (qui teste chaque position). L'approfondissement revient sur le **ou exclusif** du masque jetable, vu au chapitre 10, car c'est exactement l'opération logique dont la partie 2 a besoin.
La démarche de la fiche.

1. Comprendre la règle et ses pièges, puis l'appliquer à la main sur cinq lignes.
2. Écrire une fonction qui découpe une ligne en ses quatre morceaux.
3. Écrire une fonction de validation, puis compter les lignes valides : c'est la partie 1.

4. Partie 2 : garder le découpage, écrire une seconde fonction de validation.
5. En approfondissement, étudier le ou exclusif par sa table de vérité.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`password`	mot de passe	`policy`	règle, politique
`corrupted`	corrompu, abîmé	`valid`	valide, conforme
`lowest / highest`	plus petit / plus grand	`number of times`	nombre de fois
`log in`	se connecter	`instance`	occurrence (d'une lettre)

Résumé de l'énoncé (partie 1)

Contexte. Le héros part en vacances et doit descendre jusqu'à la côte en luge. Le loueur de luges ne parvient plus à se connecter : sa base de mots de passe est en partie abîmée. Il s'agit de repérer, dans cette base, les mots de passe qui respectent leur propre règle.
Les données. Chaque ligne du fichier décrit un mot de passe enregistré, accompagné de la règle de sécurité qui était en vigueur quand il a été choisi. Une ligne a la forme `a-b l: motdepasse` : deux entiers $a \leq b$ séparés par un tiret, un espace, une lettre `l` suivie de deux-points, un espace, puis le mot de passe, écrit en lettres minuscules. Les entiers peuvent avoir plusieurs chiffres.
La règle.

- On compte combien de fois la lettre `l` apparaît dans le mot de passe.
- Le mot de passe est valide si ce nombre est au moins a et au plus b, bornes comprises. Autrement dit, le nombre d'occurrences de `l` doit appartenir à l'intervalle $[a, b]$.

La réponse attendue. Le nombre de lignes dont le mot de passe respecte sa règle. Par exemple, `1-3 b: abbc` est valide (deux `b`) mais `2-3 x: xab` ne l'est pas (un seul `x`) : ces deux lignes seules donneraient la réponse 1.

Exercice 1 *Comprendre ce qui est demandé*

But. Vérifier qu'on a compris la règle et repérer les pièges du format des lignes.
À quoi ça sert ? Chaque question annonce une difficulté du programme : les bornes comprises dans le test, le `:` collé à la lettre lors du découpage, et les nombres à plusieurs chiffres qui interdisent de lire la ligne à des positions fixes.

Après lecture de l'énoncé et du résumé, vérifier sa compréhension :

1. Le mot de passe `kkk` respecte-t-il la règle `1-3 k` ? Et la règle `4-5 k` ?
2. Après `"2-3 z: zaz".split()`, que contient le deuxième morceau ? Quel piège pour comparer la lettre ?
3. Les nombres peuvent avoir deux chiffres (`10-12`). Pourquoi est-il alors dangereux de lire la ligne à des positions fixes ?

Exercice 2 *À la main, sur un petit exemple*

But. Appliquer la règle à la main sur cinq lignes.
À quoi ça sert ? On obtient un exemple dont on **connaît la réponse** (3) : le programme devra le retrouver avant d'être lancé sur les vraies données.

On considère le fichier (inventé) suivant :

```
2-4 m: mmxmz
1-2 k: akkkb
3-5 t: tttxt
1-1 q: rst
2-6 e: eeabcf
```

Pour chaque ligne, dire si le mot de passe respecte la règle de la partie 1. Vérification : on doit compter `3` mots de passe valides.

2 Programmer la partie 1

Exercice 3 *Découper une ligne*

But. Écrire une fonction qui transforme une ligne de texte en ses quatre morceaux utiles : deux entiers, la lettre, le mot de passe.
À quoi ça sert ? Séparer la lecture de la règle de son application rend chaque fonction plus simple à tester. Ce découpage sert tel quel dans les deux parties : seule la validation changera.

Écrire une fonction `decouper(ligne)` qui renvoie le tuple `(a, b, lettre, mdp)` formé de deux entiers et de deux chaînes. Par exemple `decouper("2-4 m: mmxmz")` doit renvoyer `(2, 4, "m", "mmxmz")`. Tester avec `assert`.

▷ coup de pouce

Méthode : compter les apparitions d'une lettre, et comparaisons enchaînées

Pour compter combien de fois une lettre apparaît dans une chaîne, on parcourt la chaîne caractère par caractère avec un **compteur** initialisé à 0. Python accepte aussi d'enchaîner les comparaisons, comme en mathématiques.

```
1  def compter(lettre, mot):
2      """Renvoie le nombre d'apparitions de lettre dans mot."""
3      nb = 0
4      for c in mot:
5          if c == lettre:
6              nb += 1
7      return nb
8
9  print(compter("a", "banane"))    # 2
10 print(compter("z", "banane"))    # 0 si absente
11 n = 3
12 print(1 <= n <= 5)               # True : equivaut a 1 <= n and n <= 5
```

Intérêt. Un seul parcours de la chaîne : le coût est proportionnel à sa longueur.
À essayer. Adapter `compter` pour compter un **morceau** de chaîne (`motif`) en testant chaque position `i` avec la tranche `mot[i:i + len(motif)]`. Combien d'apparitions de `"aa"` trouve-t-on dans `"aaaa"` ? Comparer avec la fonction `recherche_naive("aa", "aaaa")` du cours.

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Exercice 4 *Valider et compter*

But. Écrire la fonction qui dit si un mot de passe respecte sa règle, puis compter les lignes valides du fichier.
À quoi ça sert ? C'est le cœur de la partie 1 : la boucle de comptage de la lettre est une recherche dans un texte, et le nombre de lignes valides est la réponse à donner.

Écrire une fonction `valide1(a, b, lettre, mdp)` qui renvoie un booléen, puis le programme qui lit `input.txt` et compte les mots de passe valides. Tester sur l'exemple inventé de la fiche, puis sur vos données.

▷ coup de pouce

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Le loueur s'aperçoit qu'il s'est trompé de règlement : les deux nombres n'ont pas le sens qu'on leur a donné. Le fichier et le format des lignes ne changent pas ; seule la façon de lire la règle change.
La nouvelle règle.

- Les deux nombres désignent maintenant deux **positions** dans le mot de passe, numérotées **à partir de 1** (il n'y a pas de position 0).
- Le mot de passe est valide si la lettre se trouve à exactement une de ces deux positions : ni à aucune, ni aux deux.
- Les occurrences de la lettre ailleurs dans le mot ne comptent plus.

La réponse attendue. Le nombre de mots de passe valides selon cette nouvelle règle. Par exemple, `1-3 c: cbd` est valide (un `c` en position 1 seulement), mais `1-3 c: cbc` ne l'est pas (un `c` aux deux positions).

Exercice 5 *Une nouvelle règle*

But. Appliquer la règle de la partie 2 à la main, puis la programmer dans une seconde fonction de validation.
À quoi ça sert ? On réutilise `decouper` et la boucle qui compte les lignes valides ; seule la fonction de validation change. Les deux pièges sont la numérotation à partir de 1 et le « exactement une des deux positions ».

1. Sans programme : la ligne `1-3 a: aba` est-elle valide selon la nouvelle règle ? Et `2-3 b: abc` ?
2. Écrire `valide2(a, b, lettre, mdp)` et compter. Sur l'exemple inventé de la fiche, on doit trouver 2 mots de passe valides.

▷ coup de pouce ▷ début de solution

4 Approfondissement

Outil Python : les opérateurs bit à bit (`&`, `|`, `^`)

Sur des entiers, ces opérateurs travaillent chiffre binaire par chiffre binaire : `&` (et), `|` (ou), `^` (ou exclusif). Sur des booléens, ils donnent le même résultat que `and`, `or` et le ou exclusif.

```
print(6 & 3)          # 110 et 011 -> 010, soit 2
print(6 | 3)          # 110 ou 011 -> 111, soit 7
print(6 ^ 3)          # 110 xor 011 -> 101, soit 5
print(True ^ True)    # False
print(5 ^ 5)          # 0 : x ^ x vaut toujours 0
```

Intérêt. Un seul calcul traite tous les bits à la fois ; le ou exclusif est très utilisé en cryptographie (masque jetable de Vernam, chapitre 10) et pour coder des ensembles de petits entiers.

À essayer. Calculer à la main `12 ^ 10`, puis vérifier avec Python.

★★★ **Exercice 6** IA IA IA *Défi : une table de vérité*

But. Vérifier par un programme que trois écritures du ou exclusif sont équivalentes.

À quoi ça sert ? La partie 2 se résume à un ou exclusif entre deux booléens. Savoir l'écrire de plusieurs façons, et le vérifier sur tous les cas, prépare son usage en cryptographie (masque jetable, chapitre 10).

Dresser la table de vérité du ou exclusif. Vérifier que `p != q`, `p ^ q` et `(p or q) and not (p and q)` donnent le même résultat pour les quatre cas, en le faisant afficher par un programme.

▷ coup de pouce

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste `ligne.split()` coupe aux espaces et donne trois morceaux ; le premier se recoupe avec `split("-")`, le deuxième se débarrasse du `:` en ne gardant que son premier caractère. ↑

Ex. 4 – piste Compter les occurrences de `lettre` dans `mdp` avec une boucle et un compteur (voir l'encadré « Méthode »). Python accepte les comparaisons enchaînées : `a <= n <= b`. ↑

Ex. 5 – piste Attention au décalage : la position `1` de l'énoncé correspond à l'indice `0` en Python. ↑

Ex. 5 – début de solution Calculer deux booléens `p` et `q` (la lettre est-elle à chacune des deux positions ?). « L'un ou l'autre mais pas les deux » s'écrit `p != q` : c'est le *ou exclusif*. ↑énoncé

Ex. 6 – piste Deux boucles `for p in (False, True)` imbriquées suffisent à parcourir les quatre cas. ↑énoncé

DÉFI 2 Monster Messages

les messages du satellite — grammaires et récursivité

Jour 19

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t12-defi-aoc-2020-19` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/19`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. On reçoit quelques centaines de messages formés des lettres `a` et `b`, et une liste de règles numérotées qui décrivent ce qu'est un message correct. Les règles s'appuient les unes sur les autres et proposent des alternatives : un même message peut se découper de nombreuses façons. On veut compter les messages corrects ; à la main, explorer tous les découpages est impossible.

Ce qu'on réinvestit. Ce défi prolonge la **recherche textuelle** de ce chapitre : on ne cherche plus un motif fixe dans un texte, mais on vérifie si des messages entiers respectent des règles. La **récursivité** est l'outil naturel, puisque chaque règle est définie à partir d'autres règles ; l'approfondissement 2 réinvestit la **mémoïsation** du chapitre 11 pour éviter de refaire les mêmes calculs.

La démarche de la fiche.

1. Comprendre comment les règles s'emboîtent, puis trouver à la main les mots reconnus par de petites règles.
2. Lire le fichier : les règles dans un dictionnaire, les messages dans une liste.
3. Écrire une fonction récursive qui renvoie toutes les positions où une règle peut s'arrêter : elle décide si un message est valide (partie 1).

4. Vérifier que cette fonction résiste aux règles qui s'appellent elles-mêmes (partie 2).

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`rule / sub-rule`	règle / sous-règle	`to match`	correspondre à, être reconnu par
`pipe (|)`	barre verticale (« ou »)	`corrupted`	corrompu, abîmé
`build upon`	s'appuyer sur	`completely`	entièrement
`loop`	boucle	`unmatched`	non reconnu, en trop

Résumé de l'énoncé (partie 1)

Contexte. Arrivé dans une forêt, vous êtes contacté par les elfes : un satellite leur envoie des messages, mais la liaison est mauvaise et beaucoup arrivent abîmés. Pour faire le tri, ils disposent de règles qui décrivent exactement à quoi ressemble un message correct. Il faut leur dire combien de messages sont utilisables.
Les données. Le fichier a deux parties séparées par une ligne vide. D'abord les règles, une par ligne, sous la forme `numéro: définition` (par exemple `2: 4 3 | 3 1`), pas forcément rangées dans l'ordre des numéros. Ensuite les messages reçus, un par ligne, formés uniquement des lettres `a` et `b`.
La règle.

- Une règle « lettre », écrite entre guillemets comme `"a"`, reconnaît exactement ce caractère.
- Une règle composée est une suite de numéros : le texte doit se découper en morceaux consécutifs reconnus, dans l'ordre, par ces sous-règles.
- Le symbole `|` sépare des alternatives : il suffit que l'une d'elles convienne. Les règles s'emboîtent ainsi comme des briques, de la lettre jusqu'à la règle `0`.
- En partie 1, aucune règle ne fait appel à elle-même, directement ou par l'intermédiaire d'autres règles.
- Un message est valide si la règle `0` le reconnaît **en entier**, sans caractère en trop.

La réponse attendue. Le nombre de messages valides. Par exemple, avec les règles `0: 1 2 | 2 1`, `1: "a"` et `2: "b"`, la règle `0` reconnaît `ab` et `ba` seulement : parmi les messages `ab`, `ba`, `aba` et `bb`, la réponse serait 2 (`aba` a une lettre en trop).

Exercice 1 *Vérifier sa compréhension*

But. Repérer les pièges du découpage : choix d'alternative indépendants, alternatives à garder toutes, règles dans le désordre.
À quoi ça sert ? Chaque question correspond à une décision de programmation : la deuxième, en particulier, explique pourquoi la fonction de la partie 1 doit renvoyer **toutes** les positions possibles et pas une seule.

1. Une même règle peut-elle apparaître deux fois dans une définition ? Le choix d'alternative est-il alors forcément le même ?

2. Devant une règle à deux alternatives, peut-on garder la première qui reconnaît un début du texte et oublier l'autre ?

3. Les règles sont-elles forcément rangées dans l'ordre des numéros ? Quelle structure de données évite de s'en soucier ?

Exercice 2 *À la main, sur un petit exemple*

But. Énumérer les mots reconnus par de petites règles et tester quelques messages.
À quoi ça sert ? On construit à la main ce que le programme fera automatiquement : remonter des lettres jusqu'à la règle `0`. Les mots trouvés servent ensuite de tests, et le cas `bbaa` montre le piège du message reconnu seulement en partie.

On considère les règles (inventées) suivantes :

```
0: 1 2
1: 3 3 | 4
2: 4 3 | 3 1
3: "a"
4: "b"
```

1. Écrire tous les mots reconnus par la règle `1`, puis par la règle `2`.
2. En déduire tous les mots reconnus par la règle `0`. Vérification : il y en a `6`.
3. Parmi `aaab`, `bba`, `baab`, `aba` et `bbaa`, lesquels sont acceptés ? Que se passe-t-il pour `bbaa` ?

2 Programmer la partie 1

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Exercice 3 *Lire le fichier*

But. Ranger les règles dans un dictionnaire indexé par leur numéro, et les messages dans une liste.
À quoi ça sert ? Le dictionnaire donne en une instruction la définition de n'importe quelle règle, quel que soit l'ordre des lignes du fichier : la fonction récursive en aura besoin à chaque appel.

Lire `input.txt` et construire un dictionnaire `regles` : à chaque numéro (une chaîne, par exemple `'2'`), on associe soit la lettre (`'a'`), soit la liste des alternatives, chacune étant une liste de numéros (`[['4', '3'], ['3', '1']]`). Construire aussi la liste `messages`. ▷ coup de pouce

Méthode : reconnaître une règle « lettre »

Dans le dictionnaire `regles`, une valeur est soit une chaîne (une lettre), soit une liste d'alternatives. Dans ce défi, les seules lettres sont `a` et `b` : une règle est une règle « lettre » exactement quand elle vaut `"a"` ou `"b"`. Comparer une liste à une chaîne avec `==` ne provoque pas d'erreur : le résultat est simplement `False`.

```
regles = {"3": "a", "2": [["4", "3"], ["3", "1"]]}
print(regles["3"] == "a" or regles["3"] == "b")   # True : regle "lettre"
print(regles["2"] == "a" or regles["2"] == "b")   # False : alternatives
```

Intérêt. Cela évite de stocker une information supplémentaire (« type de règle ») : la valeur elle-même dit de quoi il s'agit. Question : que vaut `["a"] == "a"` ? Pourquoi ?

Exercice 4 *Reconnaître un message*

But. Écrire la fonction récursive `fins`, qui renvoie toutes les positions où peut s'arrêter un morceau reconnu par une règle.
À quoi ça sert ? C'est le cœur du défi : un message est valide quand l'une de ces positions est sa longueur, et compter les messages valides donne la réponse de la partie 1. Renvoyer toutes les fins possibles évite d'avoir à deviner la bonne alternative.

Écrire une fonction récursive `fins(regles, num, message, i)` qui renvoie la **liste des positions** où peut se terminer un morceau du message commençant à l'indice `i` et reconnu par la règle `num` (lire l'encadré « Grammaire, expression régulière et positions de fin », dans la section « Pour y arriver »). Un message est accepté si `len(message)` figure dans `fins(regles, '0', message, 0)`. Tester sur l'exemple, puis compter les messages acceptés. ▷ coup de pouce ▷ début de solution

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Les elfes s'aperçoivent que deux de leurs règles étaient fausses et vous envoient les bonnes : `8: 42 | 42 8` et `11: 42 31 | 42 11 31`. Ces deux règles remplacent les anciennes règles `8` et `11` ; toutes les autres sont inchangées.
La nouvelle règle.

- Ces règles font appel à elles-mêmes : le nombre de mots qu'elles reconnaissent devient infini.
- La règle `8` reconnaît une ou plusieurs répétitions de la règle `42`.
- La règle `11` reconnaît k fois la règle `42` suivie de k fois la règle `31`, avec le même $k \geq 1$ des deux côtés. Par exemple, si `42` reconnaissait `a` et `31` reconnaissait `b`, la règle `11` reconnaîtrait `ab`, `aabb`, `aaabbb`… mais pas `aab`.
- Un message est toujours valide si la règle `0` le reconnaît en entier.

La réponse attendue. Le nouveau nombre de messages valides.

Exercice 5 *Des règles récursives*

But. Remplacer les deux règles et vérifier que la fonction `fins` donne encore la bonne réponse.
À quoi ça sert ? Une fonction récursive bien écrite n'a pas besoin de savoir à l'avance combien de fois une règle se répète : comprendre pourquoi elle s'arrête toujours, c'est s'assurer que la réponse de la partie 2 est juste.

Deux règles sont remplacées par des règles qui **s'appellent elles-mêmes**. Modifier le dictionnaire comme indiqué et relancer le comptage. Votre fonction `fins` fonctionne-t-elle encore ? Pourquoi ne boucle-t-elle pas indéfiniment ? ▷ coup de pouce

Exemple de test (inventé ; en partie 2, on remplace les règles `8` et `11` comme dans le résumé) :

```
0: 8 11
8: 42
11: 42 31
42: 1 1 | 2 1
31: 1 2 | 2 2
1: "a"
2: "b"

aabaab
aaaaab
aaaabaabbb
aaabab
```

```
baaaaaab
abaaab
babaabab
```

Avec les règles d'origine, on doit compter `2` messages ; avec les règles modifiées, `4`.

4 Pour y arriver : grammaires et récursivité [au-delà du programme]

Grammaire, expression régulière et positions de fin

Grammaire. Les règles de l'énoncé forment une *grammaire* : chaque règle décrit un ensemble de mots à partir d'autres règles, comme « une phrase est un groupe nominal suivi d'un verbe ». On dit qu'un mot est *reconnu* s'il peut être obtenu en partant de la règle `0`.

Pourquoi pas une expression régulière ? En partie 1, on peut remplacer chaque numéro par sa définition jusqu'à n'avoir que des lettres et obtenir une expression régulière : sur l'exemple inventé de la fiche, la règle `0` devient `(aa|b)(ba|a(aa|b))`. Mais si une règle contient son propre numéro, ce remplacement ne s'arrête jamais. Pire : une règle du type « `X` suivi de `Y` autant de fois que de `X` » (même nombre des deux côtés) ne peut *pas du tout* s'écrire comme une expression régulière, car celles-ci ne savent pas compter.

L'idée de la fonction `fins`. Plutôt que de deviner *où* s'arrête la partie reconnue par une règle, on renvoie *toutes* les possibilités. Sur l'exemple inventé de la fiche, avec le mot `bbaa` : la règle `1` depuis l'indice `0` renvoie `[1]` (la lettre `b`) ; la règle `2` depuis l'indice `1` renvoie `[3]` (le morceau `ba`). Donc `fins(regles, '0', "bbaa", 0)` vaut `[3]` : le mot n'est pas reconnu en entier car $3 \neq 4$. Quand une règle a plusieurs alternatives, la liste peut contenir plusieurs positions ; une liste vide signifie « échec ».

5 Approfondissement

Outil Python : les ensembles (`set`)

Un **ensemble** est une collection sans doublon et sans ordre. On l'écrit entre accolades (`set()` pour l'ensemble vide), on peut le construire par compréhension comme une liste, et lui ajouter un élément avec `add`. La **réunion** `a | b` et l'**intersection** `a & b` construisent un nouvel ensemble (voir aussi l'encadré du défi *Crab Combat*, chapitre 4).

```python
debuts = {"a", "b"}
suites = {"a", "ab"}
mots = {d + s for d in debuts for s in suites}   # tous les "collages"
print(mots)                    # {'aa', 'aab', 'ba', 'bab'} (ordre quelconque)
print("ba" in mots)            # True : test d'appartenance tres rapide
mots = mots | {"bb", "aa"}     # reunion : "aa" deja present, pas de doublon
print(len(mots))               # 5
print(mots & {"bb", "ab"})     # {'bb'} : intersection
```

Intérêt. Le test `x in ensemble` prend un temps qui ne dépend pas de la taille de l'ensemble (contrairement à une liste, qu'il faut parcourir), et les doublons disparaissent d'eux-mêmes. Question : combien d'éléments a `{d + f for d in "ab" for f in "ab"}` ?

★★ **Exercice 6** *Approfondissement 1 : une méthode par blocs, propre aux données*

But. Résoudre la partie 2 autrement, en exploitant une particularité des vraies données.
À quoi ça sert ? Observer les données avant de programmer peut ramener un problème difficile (une grammaire récursive) à un simple découpage en blocs de même longueur ; on compare ensuite cette méthode, rapide mais spécialisée, à la méthode générale.

Dans les vraies données, la règle `0` est `8 11`, et les règles `42` et `31` ne contiennent pas de cycle : chacune reconnaît un ensemble **fini** de mots, et ces mots ont tous la même longueur.

1. À la main, sur l'exemple de test de la partie 2 : écrire l'ensemble des mots reconnus par `42` et par `31`. Découper `aaaabaabbb` et `baaaaaab` en blocs de deux lettres et dire, pour chaque bloc, à quelle règle il appartient.

2. Avec la règle `0: 8 11` modifiée, montrer qu'un message est valide si et seulement s'il s'écrit « p blocs de `42` suivis de q blocs de `31` » avec $p > q \geq 1$. Et en partie 1 ?

3. Écrire une fonction récursive `mots(regles, num)` qui renvoie l'ensemble des mots reconnus par une règle sans cycle. ▷ coup de pouce ▷ début de solution

4. Écrire `valide_par_blocs(message, m42, m31)` et retrouver le résultat de la partie 2. Pourquoi faut-il que les deux ensembles n'aient aucun mot commun ? Le vérifier sur vos données (que doit valoir `m42 & m31` ?). ▷ coup de pouce

5. Comparer avec la fonction `fins` : vitesse, et généralité (que se passe-t-il si l'on change la règle `0` ?).

★★ **Exercice 7** *Approfondissement 2 : mémoïser la fonction* `fins`

But. Éviter de recalculer plusieurs fois `fins` pour une même règle et une même position.
À quoi ça sert ? C'est la mémoïsation du chapitre 11 appliquée à un nouveau problème : elle borne le nombre de calculs différents et rend la méthode générale sûre même sur des messages longs.

Pour un message donné, la fonction `fins` peut être appelée plusieurs fois avec la même règle et la même position `i`.

1. Sur l'exemple de test de la partie 2, avec les règles modifiées, expliquer pourquoi le calcul de `fins(regles, '8', message, 0)` appelle deux fois `fins(regles, '42', message, 0)`. Que se passe-t-il ensuite à la position 2, à la position 4… ?

2. Écrire `fins_memo(regles, num, message, i, memo)` qui range chaque résultat dans le dictionnaire `memo`, de clé `(num, i)`, et le réutilise s'il est déjà calculé. Vérifier qu'on obtient les mêmes réponses. ▷ coup de pouce

3. Combien de couples `(num, i)` différents au plus pour un message de longueur n et R règles ? Que peut-on en dire sur la complexité ?

Un peu d'histoire : Chomsky, Backus, Naur et les grammaires

Un linguiste. En 1956, le linguiste américain **Noam Chomsky** (né en 1928) publie *Three Models for the Description of Language*. Il y compare plusieurs façons de décrire une langue par des règles, du plus simple au plus puissant. Il montre que les modèles les plus simples, qui lisent une phrase de gauche à droite sans rien retenir d'autre qu'un « état » parmi un nombre fini, ne suffisent pas pour l'anglais : les constructions emboîtées comme « *if* … *then* … », qui peuvent contenir d'autres « *if* … *then* … », demandent de **compter**. C'est exactement la règle `11` de la partie 2 (k fois `42` puis k fois `31`), qu'aucune expression régulière ne sait décrire.
Deux informaticiens. En 1959, à la première conférence internationale sur le traitement

de l'information (Unesco, Paris), **John Backus**, déjà l'inventeur du langage Fortran chez IBM, présente une notation pour décrire la syntaxe du nouveau langage ALGOL : chaque construction est définie par des règles qui peuvent se citer elles-mêmes, avec des alternatives séparées par une barre. Le Danois **Peter Naur** l'améliore et l'utilise pour rédiger le rapport ALGOL 60 (1960). En 1964, Donald Knuth propose d'appeler cette notation **BNF** (*Backus–Naur Form*). Les règles de l'énoncé, avec leurs `|` et leurs appels récursifs, en sont une version miniature.

Et aujourd'hui. La syntaxe de presque tous les langages de programmation est décrite par une grammaire de ce type ; celle de Python figure dans sa documentation officielle, et c'est à partir d'elle que l'interpréteur vérifie vos programmes avant de les exécuter (le message `SyntaxError` vient de là). Backus a reçu le prix Turing en 1977, Naur en 2005.

Sources : N. Chomsky, « Three Models for the Description of Language », *IRE Transactions on Information Theory*, 2 (1956), p. 113–124 ; P. Naur (éd.), « Report on the Algorithmic Language ALGOL 60 », *Communications of the ACM*, 3 (1960) ; D. Knuth, « Backus Normal Form vs. Backus Naur Form », *Communications of the ACM*, 7 (1964) ; site de l'ACM (lauréats du prix Turing).

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Le fichier se coupe en deux avec `strip()` puis `split("\n\n")`. Pour une règle, `split(": ")` sépare le numéro du reste, puis `split(" | ")` sépare les alternatives. Une règle « lettre » s'écrit `"a"` avec ses guillemets : la lettre est le caractère d'indice 1. ↑énoncé

Ex. 4 – piste Cas de base : une règle « lettre » renvoie `[i + 1]` si `message[i]` est cette lettre (et `i` dans les bornes), et `[]` sinon. ↑énoncé

Ex. 4 – début de solution Pour une alternative `['4', '3']` : partir de `positions = [i]` ; pour chaque sous-règle, remplacer `positions` par la liste de toutes les fins obtenues depuis chacune des positions. Les résultats des différentes alternatives sont mis bout à bout. ↑énoncé

Ex. 5 – piste Observer où apparaît le numéro de la règle dans sa propre définition : jamais en première position. Avant chaque appel récursif, au moins une lettre a donc été consommée. ↑énoncé

Ex. 6 q. 3 – piste Pour une alternative, partir de l'ensemble `{""}` (le mot vide) et, pour chaque sous-règle, coller à chaque début chacun des mots de la sous-règle. ↑énoncé

Ex. 6 q. 3 – début de solution `possibles = {debut + fin for debut in possibles for fin in mots(regles, sous_regle)}` ; les mots obtenus pour les différentes alternatives sont réunis dans l'ensemble résultat : `resultat = resultat | possibles`. ↑énoncé

Ex. 6 q. 4 – piste Compter le nombre `p` de blocs de tête qui sont dans `m42` ; tous les blocs restants doivent être dans `m31` (un booléen drapeau, comme dans le défi *Handy Haversacks*, chapitre 1). ↑énoncé

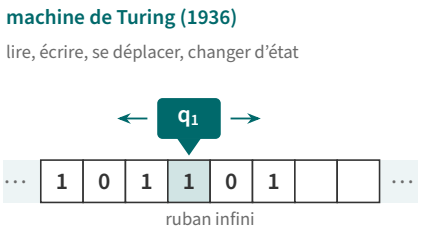
Ex. 7 q. 2 – piste Le dictionnaire doit être **vide au début de chaque message** : un même couple `(num, i)` ne donne pas le même résultat pour deux messages différents. ↑énoncé

13

CHAPITRE 13

Calculabilité et décidabilité

```
def paradoxe(prog):
    if halt(prog, prog):
        while True:
            pass
    else:
        return "je m'arrete"
```



13

DANS CE CHAPITRE

- Activités préparatoires . . . 625
- Cours . . . 627
- Exercices . . . 639
- TP et projets . . . 651
- Fiche de révision . . . 656
- Défis Advent of Code . . . 658
- Corrigés . . . 991

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Un programme peut en manger un autre

▶ corrigé p. 991

recettes, programmes qui lisent des programmes, et un devin malheureux

Durée : 25 min · Sans ordinateur, par deux

Consignes

- On lit, on prédit, on joue ; on répond sur le cahier. Aucun mot de vocabulaire n'est attendu avant l'exercice 4.

Exercice 1 *La recette de crêpes*

Une fiche porte la recette des crêpes. Recopier et compléter sur le cahier le tableau suivant : pour chaque situation, dire **qui travaille** (la « machine »), **ce qu'il reçoit** (la « donnée ») et si la recette est **exécutée** (on obtient des crêpes) ou seulement **lue comme un texte**.

Situation	Qui travaille ?	Ce qu'il reçoit	Exécutée ou lue ?
Un cuisinier suit la recette.			
Une photocopieuse en tire dix exemplaires.			
Un traducteur la met en anglais.			
Un correcteur orthographique la relit.			
Un éditeur compte le nombre de lignes de la recette pour la mise en page.			

1. Un même texte peut-il être tantôt « des instructions à suivre », tantôt « une donnée à traiter » ?

Exercice 2 *Python lit du Python*

On considère le fichier `test.py` suivant, qui définit une fonction puis l'appelle :

```
1 def accueil(n):
2     for k in range(n):
3         print("bonjour")
4
5 accueil(3)
```

2. Dans un terminal, on tape `python3 test.py`. Qu'est-ce qui s'affiche ? Dans cette commande, quel est le programme qui travaille, et quelle est sa donnée ?
3. On range maintenant le **texte** de la fonction dans une chaîne de caractères, et on écrit une autre fonction :

```
1 source = 'def accueil(n):\n    for k in range(n):\n        print("bonjour")\n'
2
3 def compter_lignes(code):
4     nb = 0
5     for c in code:
6         if c == "\n":          # un passage a la ligne
7             nb += 1
8     return nb
```

Que renvoie `compter_lignes(source)` ? Un « bonjour » est-il affiché ?

4. En Python, `exec(chaine)` exécute le programme écrit dans la chaîne. Qu'affiche l'instruction `exec(source + "accueil(2)\n")` ?
5. La fonction `compter_lignes` a elle aussi un texte (six lignes). Peut-on lui donner **son propre texte** en entrée ? Que renverrait-elle ?

Exercice 3 *Le devin et le contrariant*

Le jeu (à deux). Le **devin** écrit en secret sur un papier une prédiction : « tu vas lever la main » ou « tu ne vas pas lever la main », puis tend le papier au **contrariant**. Le contrariant lit la prédiction et fait **exactement le contraire**. Jouer trois manches en échangeant les rôles.

6. Combien de fois le devin a-t-il eu raison ? Existe-t-il une prédiction gagnante pour le devin ? Pourquoi ?
7. Et si le contrariant n'avait **pas le droit de lire** la prédiction : le devin pourrait-il gagner ?
8. **Le menteur.** La phrase « *Cette phrase est fausse.* » est-elle vraie ? fausse ?
9. **Le barbier.** Dans un village, le barbier rase tous les hommes qui ne se rasent pas eux-mêmes, et seulement ceux-là. Le barbier se rase-t-il lui-même ?
10. Qu'ont en commun le contrariant, la phrase du menteur et le barbier ?

Exercice 4 *Mettre des mots*

11. Recopier et compléter sur le cahier, avec vos mots.
    - Pour l'ordinateur, le texte d'un programme est avant tout …
    - Un programme qui lit le texte d'un autre programme et l'exécute s'appelle …
    - Un programme peut même recevoir en entrée …
    - Le contrariant est imbattable parce qu'il …
12. **Une question pour la suite.** On rêve d'un programme « devin » qui lirait le texte de **n'importe quel** programme et prédirait, sans l'exécuter, s'il finira par s'arrêter ou s'il tournera pour toujours. Un tel devin peut-il exister ? Donner votre intuition, en pensant au contrariant.

COURS

Programme (BO) *« Notion de programme en tant que donnée. Calculabilité, décidabilité. Comprendre que tout programme est aussi une donnée. Comprendre que la calculabilité ne dépend pas du langage de programmation utilisé. Montrer, sans formalisme théorique, que le problème de l'arrêt est indécidable. »*

Prérequis la notion de **fonction** et de **paramètre** ; les **boucles** et la **terminaison** (variant de boucle) ; la notion de **coût** d'un algorithme (linéaire et quadratique, vus en Première ; exponentiel, vu au chapitre 1 avec la suite de Fibonacci) ; les **paradigmes** de programmation (voir le chapitre *Programmation objet et paradigmes*). → fiches de révision : ch. 2 p. 113 → rappels de 1re : bases de Python p. 1056, spécification et tests p. 1063, parcours et coût p. 1059

Objectifs *comprendre* qu'un programme est une **donnée** comme une autre ; *connaître* la distinction **décidable / indécidable** ; *savoir refaire* le raisonnement du **problème de l'arrêt** ; *situer* la **thèse de Church-Turing** et la **Turing-complétude** ; *découvrir* la question **P versus NP** [au-delà du programme].

*Ce chapitre ne demande presque pas de code : c'est un chapitre d'**idées**. Nous allons prouver, avec quelques lignes de Python et un raisonnement d'une élégance redoutable, qu'il existe des questions parfaitement claires… auxquelles **aucun** programme ne pourra **jamais** répondre. Non pas parce que nos ordinateurs sont trop lents ou nos programmeurs trop paresseux : parce que c'est **impossible**. Bienvenue à la frontière de l'informatique.*

Situation de départ

1. Pour l'ordinateur, le texte d'un programme est-il seulement une suite d'instructions à exécuter, ou peut-il aussi être lu et analysé comme n'importe quelle donnée ?
2. Pourrait-on écrire un logiciel qui lit n'importe quel programme et annonce, sans le lancer, s'il finira par s'arrêter ?
3. Un antivirus peut-il garantir qu'il détecte tous les virus, même ceux qu'il ne connaît pas encore ?

I Un programme peut en manger un autre

Tout part d'une remarque en apparence anodine. Une fonction comme `accueil(n)`, qui affiche `n` fois « bonjour », est une **machine** : elle reçoit une **donnée d'entrée** (l'entier `n`) et produit une **sortie**. Enregistrons-la dans un fichier `test.py` et lançons, dans un terminal, la commande `python3 test.py`. **Qui est la machine, ici ? Qui est la donnée ?** La machine, c'est `python3`. Et sa donnée d'entrée, c'est `test.py` — un **programme**, c'est-à-dire une simple suite de caractères. Le programme `accueil` est devenu une **donnée** avalée par un autre programme :



Et l'on peut poursuivre la poupée russe : `python3` est lui-même lancé par le programme `Terminal`, qui tourne sur le **système d'exploitation**…

Règle 1 — Un programme est une donnée

Le **code source** d'un programme n'est qu'une **chaîne de caractères**. Rien n'interdit donc de le donner en **paramètre** à un autre programme — ni même de le donner **à lui-même**. Cette idée, qui semble un jeu, est la clé de tout le chapitre.

↻ **Retour sur l'activité.** La recette de crêpes de l'activité était tantôt exécutée (par le cuisinier), tantôt traitée comme un simple texte (par la photocopieuse, le traducteur, le correcteur) ; de même, `compter_lignes(source)` lisait le texte d'`accueil` sans afficher un seul « bonjour », alors qu'`exec` l'exécutait. Et `compter_lignes` pouvait recevoir son propre texte : elle renverrait 6.

À retenir — un programme est une donnée comme une autre

Vous connaissez déjà des programmes qui prennent des programmes en entrée : un **interpréteur** (comme `python3`), un **compilateur** (qui traduit du C en langage machine), un **éditeur** qui colore la syntaxe, un **antivirus** qui inspecte un exécutable (recherche de signatures, chapitre 12)… Traiter « un programme comme une donnée » n'a rien d'exotique.

► **Exercices d'application :** exercices 1 **et** 2 (un programme vu comme une donnée)

II Décidable ou indécidable ?

Avant d'aller plus loin, précisons le genre de questions qui nous intéresse : celles auxquelles on répond par **oui** ou par **non**.

Définition 1 — Problème de décision

Un **problème de décision** est une question dont la réponse est **« oui »** ou **« non »**, posée sur une donnée d'entrée.

Exemples : « l'entier n est-il premier ? » ; « le mot `m` est-il dans le dictionnaire ? » ; « la liste `L` est-elle triée ? » ; « ce graphe est-il connexe ? ». Pour chacun, on sait écrire un programme qui **s'arrête toujours** et renvoie la bonne réponse.

Définition 2 — Décidable / indécidable

Un problème de décision est **décidable** s'il existe un algorithme qui, **pour toute entrée**, **s'arrête** en un temps fini et renvoie la bonne réponse (*oui* ou *non*). Sinon, il est **indécidable**.

Les quatre exemples ci-dessus sont **décidables**. On serait tenté de croire que *tout* problème clairement posé est décidable, quitte à attendre longtemps. **C'est faux**, et c'est le grand résultat du chapitre. Il existe un problème d'apparence toute simple… qui ne l'est pas.

► **Exercices d'application :** exercices 3 **et** 4 (tri sélectif ; un décideur qui s'arrête)

III Le problème de l'arrêt : la plus belle preuve de l'informatique

A. Un programme s'arrête-t-il ?

Certains programmes s'arrêtent, d'autres tournent pour toujours. Comparons :

```
def compte(n):                  def bavard():
    while n != 0:                   while True:
        n = n - 1                       print("encore !")
    return "fini"               # ne s'arrete JAMAIS
```

▷ **Je prédis.** Les appels `compte(10)`, `compte(10.5)` et `compte(-3)` s'arrêtent-ils ?

▷ **Je vérifie.** `compte(10)` s'arrête (elle renvoie `"fini"`). Mais `compte(10.5)` ne s'arrête **jamais** : `n` vaut 10.5, 9.5, 8.5, ..., 0.5, $-0.5, \ldots$ et **ne tombe jamais sur** 0. `compte(-3)` non plus : `n` vaut -3, -4, $-5, \ldots$ et s'éloigne de 0.

Un œil exercé le « voit ». La question naturelle :

*Pourrait-on écrire un programme qui **analyse un autre programme** et prédit, à notre place, s'il va s'arrêter ?*

B. La machine rêvée : `halt`

Rêvons donc d'un programme `halt` qui prendrait en entrée :

- `prog` : le code source d'un programme ;
- `x` : une entrée pour ce programme ;

et qui renverrait `True` si `prog(x)` s'arrête, `False` sinon — en **s'arrêtant toujours**, en un temps fini, quel que soit `prog` (qu'il l'*analyse*, comme nous l'avons fait à l'œil, ou qu'il le simule en partie, peu importe : il doit finir par répondre, et répondre juste).



Par exemple, `halt(compte, 10)` renverrait `True`, et `halt(compte, 10.5)` renverrait `False`. Un tel outil serait **prodigieux** : fini les bugs de boucles infinies, on pourrait vérifier automatiquement que n'importe quel logiciel termine ! Nous n'écrivons pas le *contenu* de `halt` (on le suppose seulement possible) ; contentons-nous de nous en **servir**.

C. Le piège se referme : le programme qui se contredit

Construisons, à l'aide de `halt`, un petit programme un peu diabolique :

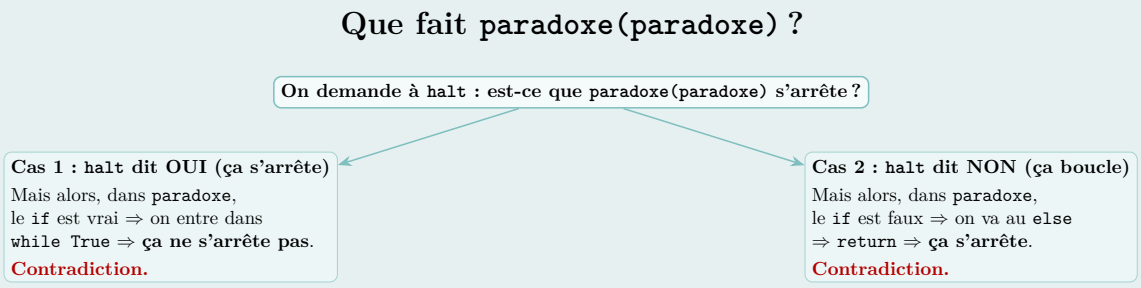
```
def paradoxe(prog):
    if halt(prog, prog):      # SI prog(prog) s'arrete...
        while True:           #     ... alors JE BOUCLE pour toujours
            pass
    else:                     # SINON (prog(prog) boucle)...
        return "je m'arrete" #     ... alors JE M'ARRETE
```

Lisons-le lentement. `paradoxe` prend un programme `prog` et fait **le contraire** de ce que fait `prog` lancé sur lui-même : si `prog(prog)` s'arrête, alors `paradoxe` boucle ; si `prog(prog)` boucle, alors `paradoxe` s'arrête. (On a le droit d'écrire `halt(prog, prog)` : un programme peut se prendre lui-même en donnée, on l'a vu au I.)

▷ **Je prédis.** Soit `zero` la fonction définie par `def zero(x): return 0`. Si `halt` existait, l'appel `paradoxe(zero)` s'arrêterait-il ?

▷ **Je vérifie.** Non : `zero(zero)` s'arrête (il renvoie 0), donc `halt(zero, zero)` vaut `True` et `paradoxe` entre dans le `while True`. Il fait toujours le *contraire* de `prog(prog)`.

Maintenant, la question fatale. Puisque `paradoxe` est un programme, donnons-le **à lui-même** :

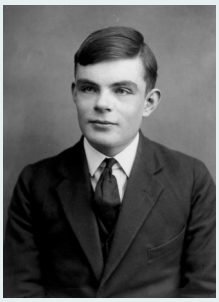


Dans les deux cas, la réponse de `halt` est **démentie** par `paradoxe` lui-même. Il n'y a pas d'échappatoire : `halt` ne peut donner *aucune* réponse correcte sur cette entrée. La seule hypothèse qu'on avait faite — `halt` *existe, s'arrête toujours et répond juste* — est donc **intenable**.

Théorème 1 — Indécidabilité du problème de l'arrêt — Turing, 1936

Il ne peut exister **aucun** programme qui, recevant en entrée un programme `P` et une entrée `E` quelconques, déciderait à coup sûr si `P` lancé sur `E` **s'arrête ou non**. Le **problème de l'arrêt** est **indécidable**.

↻ **Retour sur l'activité.** Le devin de l'activité ne pouvait jamais gagner contre le contrariant, qui lisait la prédiction puis faisait l'inverse : `halt` est ce devin, `paradoxe` ce contrariant. La question laissée ouverte à la fin de l'activité a donc sa réponse : **non**, un programme « devin » qui prédirait l'arrêt de n'importe quel programme ne peut pas exister.



Alan Turing (1912–1954), vers 16 ans

D. Pourquoi ça marche : l'auto-référence

Ce raisonnement porte un nom : la **diagonale**. Son moteur secret est l'**auto-référence** — une chose qui parle d'elle-même et se contredit. Vous la connaissez déjà :

Exemple — *Le paradoxe du menteur*
« *Cette phrase est fausse.* » Si elle est vraie, alors ce qu'elle dit est vrai, donc elle est fausse. Si elle est fausse, alors ce qu'elle dit est faux, donc elle est vraie. Aucune valeur de vérité ne tient. `paradoxe(paradoxe)`, c'est le menteur… écrit en Python.

Exemple — *Le barbier du village*
« Le barbier rase tous ceux qui ne se rasent pas eux-mêmes, et *seulement* ceux-là. » Le barbier se rase-t-il lui-même ? S'il se rase, il ne devrait pas ; s'il ne se rase pas, il devrait. Même impasse.

↻ **Retour sur l'activité.** Le point commun du contrariant, du menteur et du barbier, demandé dans l'activité, est cette auto-référence : chacun lit ce qui le concerne et fait le contraire, si bien qu'aucune réponse ne tient.

À retenir — impossible en général, possible au cas par cas

Rassurez-vous : on peut très bien, **au cas par cas**, prouver qu'*un* programme donné s'arrête (avec un **variant de boucle**, vu en Première). Ce qui est impossible, c'est une **unique méthode automatique et universelle** qui marcherait sur **tous** les programmes. La différence est capitale : on n'a pas prouvé que « c'est trop dur », on a prouvé que « ça ne peut pas exister ».

E. Une contagion : le théorème de Rice [au-delà du programme]

Le problème de l'arrêt n'est pas un cas isolé. En le « ramenant » (on dit **réduisant**) à d'autres questions, on montre qu'une foule de questions sur le *comportement* d'un programme sont, elles aussi, indécidables :

- « ce programme renverra-t-il un jour la valeur 42 ? »
- « ce programme affichera-t-il un jour un message d'erreur ? »
- « ces deux programmes calculent-ils exactement la même chose ? »
- « ce programme contient-il un virus (a-t-il tel comportement) ? »

Théorème 2 — Théorème de Rice, 1953 — *version imagée*

Toute question **non triviale** portant sur ce qu'un programme **fait** (son comportement, et non son texte) est **indécidable**.

C'est pourquoi un antivirus ne peut pas garantir qu'il détecte *tous* les virus, et pourquoi aucun outil ne pourra jamais dire à coup sûr « ce logiciel n'a aucun bug de comportement ». L'indécidabilité n'est pas une curiosité de laboratoire : elle borne, pour de bon, ce que l'informatique peut promettre.

► **Exercices d'application :** exercices 5 à 8 (le problème de l'arrêt, ses cousins, la contagion)

IV Calculabilité : ce qu'une machine peut, en principe, calculer

Le problème de l'arrêt est indécidable parce que la fonction qui le résoudrait n'est pas **calculable**. Éclaircissons ce mot.

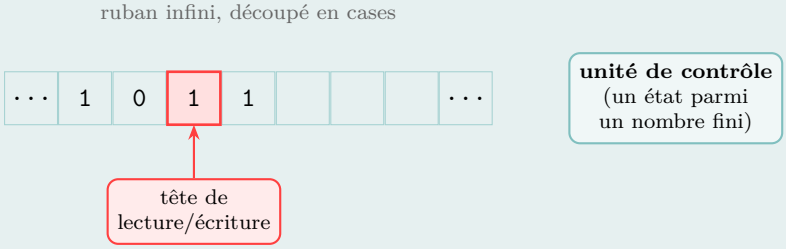
Définition 3 — Fonction calculable

Une fonction est **calculable** s'il existe un **algorithme** (une suite **finie** d'opérations élémentaires) qui, pour toute entrée x, produit sa sortie $f(x)$ en un temps **fini**.

Un problème de décision est alors **décidable** exactement quand sa fonction réponse ($oui = 1$ / $non = 0$) est **calculable**. « Indécidable » et « non calculable » sont les deux faces d'une même pièce.

A. La machine de Turing

Mais qu'est-ce, au juste, qu'une « opération élémentaire » ? En 1936, pour donner un sens **mathématique** précis au mot « calculer », Turing imagine une machine idéale, d'une simplicité extrême :



Un **ruban infini** découpé en cases ; une **tête** qui lit et écrit un symbole sur la case courante ; un **état** interne (parmi un nombre fini) ; et une **table de règles** du type *« si je suis dans l'état q et que je lis* `1`*, j'écris* `0`*, je me déplace à droite et je passe dans l'état q' »*. C'est tout. Et pourtant :

Règle 2 — Thèse de Church-Turing

Tout ce qui est « calculable » au sens intuitif peut être calculé par une machine de Turing. Autrement dit, la machine de Turing capture **exactement** la notion de calcul.

Cette affirmation n'est pas un théorème (on ne peut pas *prouver* une définition de l'intuition), mais **aucun** modèle de calcul proposé depuis 90 ans n'a jamais su calculer davantage. Fait remarquable : à la même époque, Alonzo **Church** définissait le calcul d'une tout autre façon (le **λ-calcul**, purement mathématique)… et l'on a démontré que les deux modèles calculent **exactement les mêmes fonctions**. Deux chemins, un même sommet.

La machine universelle (Turing, 1936)

Dans le même article, Turing décrit une machine **universelle** : on écrit sur son ruban la **table de règles** d'une autre machine, suivie de l'entrée de celle-ci, et la machine universelle **simule** pas à pas cette autre machine. Une seule machine suffit donc à exécuter **tous** les algorithmes : il suffit de lui fournir le bon programme *comme donnée*. C'est l'idée de l'ordinateur à **programme enregistré** (modèle de von Neumann, vu en Première), où programme et données partagent la même mémoire ; c'est aussi ce que fait l'interpréteur `python3` avec votre code. Le simulateur du TP proposé en fin de chapitre, qui lit une table de règles comme une donnée, est lui aussi une machine universelle.

B. Turing-complétude : tous les langages se valent

Notre preuve du problème de l'arrêt était écrite en Python, mais elle n'utilisait **rien de spécifique** à Python : elle vaut pour n'importe quel langage assez expressif.

Définition 4 — Turing-complet

Un langage (ou un système) est **Turing-complet** s'il peut simuler une machine de Turing — donc calculer **tout** ce qui est calculable.

Règle 3 — La calculabilité ne dépend pas du langage

Python, C, Java, OCaml, Scratch, … sont **tous** Turing-complets : ils calculent tous *le même* ensemble de fonctions. Ils diffèrent par la **commodité** et la **vitesse**, jamais par la **puissance** théorique.

Pour les curieux — *La Turing-complétude est partout, parfois par accident* (facultatif)
Sont Turing-complets, entre autres : le langage volontairement minimaliste **Brainfuck** (8 instructions !), le **jeu de la vie** de Conway, les circuits de ***redstone*** de *Minecraft*, le système de règles de **Magic : The Gathering**, et même… les **tableurs** avec leurs formules. Dès qu'on a « mémoire + test + boucle », la pleine puissance du calcul surgit — et, avec elle, le problème de l'arrêt.

► **Exercices d'application :** exercice 9 (vrai ou faux : calculabilité et Turing-complétude)

V Gödel : quand les mathématiques rencontrent leur propre limite [au-delà du programme]

Le problème de l'arrêt a un **grand frère**, né cinq ans plus tôt en mathématiques pures. Au début du XXe siècle, David **Hilbert** rêve d'achever les mathématiques : trouver un **jeu d'axiomes** à partir duquel on pourrait, **mécaniquement**, démontrer *tout* énoncé vrai — et *seulement* les vrais. Un rêve de **complétude** et de **certitude**.
En **1931**, un logicien viennois de 25 ans, Kurt **Gödel**, le fait voler en éclats. Entré à l'université de Vienne en 1924 pour étudier la physique, il bascule vers les mathématiques et la logique ; fuyant l'Autriche annexée par l'Allemagne nazie, il s'installera en 1940 à Princeton, où il deviendra l'ami inséparable d'Albert Einstein.



Kurt Gödel étudiant, en 1925

Théorème 3 — Théorèmes d'incomplétude de Gödel — *version imagée*

Dans tout système d'axiomes **cohérent** et assez riche pour parler d'arithmétique :

1. il existe des énoncés **vrais** que le système ne pourra **jamais démontrer** (incomplétude) ;
2. le système ne peut pas démontrer sa **propre cohérence**.

L'astuce de Gödel ? Fabriquer un énoncé mathématique qui, décodé, affirme : « *Cet énoncé n'est pas démontrable.* » S'il était démontrable, il serait faux (le système prouverait un faux : incohérent). Il est donc **vrai mais indémontrable**. On reconnaît, mot pour mot, le **menteur** et `paradoxe(paradoxe)` : la **même auto-référence**, le même vertige.

Pour les curieux (facultatif)
Gödel (1931, en *logique*) et Turing (1936, en *informatique*) racontent la même histoire vue de deux fenêtres. Turing a d'ailleurs *« informatisé »* Gödel : là où Gödel exhibe une **vérité** indémontrable, Turing exhibe un **programme** dont on ne peut pas décider l'arrêt. La toute-puissance mécanique — des maths comme des machines — a une **frontière**, et cette frontière est **la même**.

VI Calculable, oui — mais en un temps raisonnable ? P versus NP [au-delà du programme]

Sortons de l'*impossible* pour entrer dans le *difficile*. Parmi les problèmes **décidables** (donc résolubles), certains se résolvent **vite**, d'autres semblent réclamer un temps… astronomique. C'est la question la plus célèbre de l'informatique, et l'un des **sept problèmes du millénaire** dotés d'un prix d'**un million de dollars**.

A. La classe P : les problèmes qu'on sait résoudre vite

Définition 5 — Classe P

Un problème est dans la classe **P** s'il existe un algorithme qui le résout en temps **polynomial** (coût majoré par n^2, n^3, … où n est la taille de l'entrée).

En pratique, « classe P » ≈ « résoluble en temps **raisonnable** », même pour de grandes données. On y trouve : **trier** une liste (le tri fusion du chapitre 7), une **recherche dichotomique**, le **plus court chemin** (algorithme de Dijkstra [au-delà du programme], chapitres 8 et 9), tester si un nombre est **premier** (grâce à des algorithmes bien plus fins que la division par essais, dont le coût est exponentiel en le nombre de chiffres)… Tout, sauf les coûts **exponentiels** (2^n), qui explosent dès que n dépasse quelques dizaines.

B. L'idée reine : trouver, ou seulement vérifier ?

Voici la distinction sur laquelle tout repose. Il y a un monde entre **trouver** une solution et **vérifier** une solution qu'on vous propose.

Exemple — *Le Sudoku*
Compléter une grille de Sudoku partiellement remplie peut être **très long**, et de plus en plus long quand la grille grandit : on parle du Sudoku **généralisé**, sur une grille $n^2 \times n^2$ (4×4, 9×9, 16×16…). Mais si je vous **tends** une grille remplie, vérifier qu'elle est correcte (chaque ligne, colonne et bloc contient 1 à 9 ; 1 à 4 dans la grille réduite ci-dessous) est **immédiat**. **Vérifier** est facile ; **trouver** paraît dur.

1	2		4
3		1	
	1		3
2		4	1

Trouver le remplissage : potentiellement très long.

Vérifier une grille pleine : instantané.

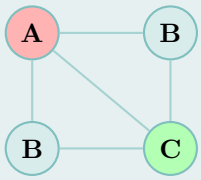
Définition 6 — Classe NP

Un problème de décision est dans la classe **NP** si, lorsque la réponse est « oui », on peut **vérifier** une solution proposée en temps **polynomial**. (**NP** signifie « **n**on déterministe **p**olynomial », *pas* « non polynomial » !)

Beaucoup de problèmes « cauchemardesques » de l'informatique sont dans NP :

- **Voyageur de commerce** (version décision) : « existe-t-il une tournée passant par ces n villes de longueur $\leq L$? » Une tournée proposée se vérifie en additionnant ses étapes.
- **Sac à dos** : « peut-on atteindre une valeur $\geq V$ sans dépasser le poids P ? »

- **Coloration de graphe** : « peut-on colorier ce graphe avec 3 couleurs sans que deux voisins partagent la même ? »
- **SAT** : une formule logique (un grand « ET » de « OU ») peut-elle être rendue **vraie** ? (*comme composer un club où chaque membre pose ses conditions, et où il faut contenter tout le monde à la fois.*)



Coloration à 3 couleurs.
Vérifier : parcourir les arêtes, aucun voisin de même couleur.
Trouver : essayer… 3^n possibilités.

C. La question à un million de dollars

D'abord, une évidence : si on sait **résoudre** vite (P), alors on sait **vérifier** vite (il suffit de résoudre et de comparer). Donc :

$$\mathbf{P} \subseteq \mathbf{NP}.$$

La vraie question, ouverte depuis 1971 et toujours **sans réponse**, est la réciproque :

$$\boxed{\mathbf{P} = \mathbf{NP} \,?} \qquad \textit{Vérifier vite implique-t-il pouvoir trouver vite ?}$$

Définition 7 — Problème NP-complet

Un problème est **NP-complet** s'il est « parmi les plus durs » de NP : **tout** problème de NP peut s'y **ramener** (réduction en temps polynomial). **SAT** (Cook, 1971), le Sudoku généralisé ($n^2 \times n^2$), le voyageur de commerce, la coloration à 3 couleurs sont NP-complets.

Pour les curieux — *Le sac à dos : NP-complet, et pourtant résolu au chapitre 11 ?* (facultatif)
Le sac à dos est lui aussi NP-complet. Pourtant, au chapitre *Programmation dynamique*, on l'a résolu en $O(n \times C)$, où C est la capacité. Ce n'est **pas** une contradiction : la taille d'une entrée se mesure en **nombre de bits**. La capacité C s'écrit avec environ $\log_2 C$ bits, donc C vaut environ $2^{\text{nombre de bits}}$: le coût $n \times C$ est **exponentiel** en la taille de l'entrée. C'est un coût **pseudo-polynomial** : rapide quand les nombres sont petits, impraticable quand ils ont des dizaines de chiffres. Un algorithme vraiment polynomial (en n et en le nombre de bits de C) prouverait que P = NP.

Conséquence spectaculaire : si l'on trouvait un jour un algorithme polynomial pour **un seul** problème NP-complet, on en aurait **d'un coup** pour **tous** les problèmes de NP — et alors **P = NP**. À l'inverse, une seule preuve d'impossibilité fermerait la question dans l'autre sens. La quasi-totalité des chercheurs **parient** que **P ≠ NP**… sans savoir le démontrer.

Pour les curieux — *Pourquoi ça compte, vraiment* (facultatif)
Si **P = NP** :

- la **cryptographie** s'effondre — on sait casser RSA si l'on sait **factoriser** un grand nombre, ce qui deviendrait rapide ;
- d'innombrables problèmes de **logistique**, d'**emploi du temps**, de **repliement de protéines** se résoudraient d'un coup ;
- plus troublant : **trouver** deviendrait aussi facile que **vérifier**. Or reconnaître une belle

preuve, une belle mélodie, est facile ; les *inventer* est dur. P = NP signifierait, en un sens, que la **créativité** est **mécanisable**. C'est pourquoi presque tout le monde espère que **P ≠ NP**.

Remarque — *Ne pas confondre !*
Indécidable (problème de l'arrêt) = « aucun algorithme, **jamais** ». **NP-complet** (Sudoku généralisé $n^2 \times n^2$) = « un algorithme existe, mais peut-être **lent** ». Le premier parle du **possible**, le second du **rapide**. Et « non déterministe » (une machine imaginaire qui essaierait toutes les pistes *à la fois*) n'a **rien** à voir avec l'ordinateur **quantique**.

► **Exercices d'application :** exercices 10 **à** 12 (P versus NP)

VII Ouverture

Ce chapitre prend du recul sur toute l'année : chaque programme écrit depuis le chapitre 1 est une donnée qu'un autre programme peut lire, et certaines questions sur ces programmes n'auront jamais de réponse automatique.

- Au **bac**, on vous demandera surtout d'expliquer qu'un programme est une **donnée**, de **dérouler** le raisonnement par l'absurde du problème de l'arrêt et de dire si une question sur des programmes est **décidable**.

Un peu d'histoire — les explorateurs de l'impossible

- **1928** — David **Hilbert** pose l'*Entscheidungsproblem* : existe-t-il une procédure mécanique décidant si un énoncé mathématique est vrai ? Il croit la réponse « oui ».
- **1931** — Kurt **Gödel** (25 ans) démontre ses **théorèmes d'incomplétude** : non, les mathématiques ne peuvent être à la fois complètes et cohérentes.
- **1936** — Alan **Turing** (24 ans) invente sa **machine**, et la machine **universelle** capable d'exécuter tous les algorithmes, et tranche l'*Entscheidungsproblem* par le **problème de l'arrêt**. La même année, Alonzo **Church** obtient le même résultat par le **λ-calcul**. L'**informatique théorique** est née… avant le premier ordinateur.
- **1939–45** — Turing casse le code **Enigma** à Bletchley Park : la théorie sert la guerre.
- **1953** — Henry **Rice** généralise : toute propriété non triviale du *comportement* d'un programme est indécidable.
- **1971** — Stephen **Cook** (et, indépendamment, Leonid **Levin**) fondent la théorie de la **NP-complétude** : la question **P versus NP** est posée. Elle est toujours ouverte.

Épilogue : Turing, persécuté pour son homosexualité, meurt en 1954. Dès 1966, son nom est donné à la plus haute récompense de l'informatique, le ***prix Turing*** *— le « Nobel » de la discipline ; il n'est réhabilité que bien plus tard (excuses officielles du gouvernement britannique en 2009, grâce posthume en 2013).*

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise :** la condition d'arrêt et le variant de la *Récursivité* ; le système qui charge et lance les programmes (*Processus et ordonnancement*).
- **Déjà croisé :** l'antivirus de la *Recherche textuelle* : le théorème de Rice explique pourquoi il ne peut tout détecter.
- **Pour aller plus loin :** P versus NP : le plus court chemin (*Graphes*) se calcule vite, factoriser semble difficile, et la sécurité de RSA (*Cryptographie*) en dépend.

Bilan — la carte mémoire

Notion	À retenir
Programme = donnée	un code source est une chaîne ; on peut le passer en argument, même à lui-même
Problème de décision	question à réponse *oui* / *non*
Décidable	un algorithme s'arrête toujours et répond juste
Indécidable	*aucun* algorithme ne peut répondre à coup sûr, pour toute entrée
Problème de l'arrêt	**indécidable** (Turing, 1936) : pas de `halt` universel
Preuve	la machine `paradoxe` se contredit dans les deux cas (auto-référence)
Théorème de Rice [au-delà du programme]	toute question non triviale sur le *comportement* est indécidable
Calculable	calculable par une machine de Turing (thèse de Church-Turing)
Turing-complet	un langage qui calcule tout le calculable ; tous équivalents
Classe P [au-delà du programme]	résoluble en temps polynomial (« raisonnable »)
Classe NP [au-delà du programme]	solution *vérifiable* en temps polynomial
P versus NP [au-delà du programme]	*trouver* = *vérifier* ? (1 M$, ouvert, on croit P $\neq$ NP)

Retour à la situation de départ

1. Les deux : le code source n'est qu'une **chaîne de caractères**, qu'un autre programme peut lire, analyser ou exécuter (comme l'interpréteur `python3`), et que l'on peut même donner en entrée au programme lui-même (§ I).
2. Non : le **problème de l'arrêt** est **indécidable** (Turing, 1936). Un tel programme `halt` permettrait d'écrire `paradoxe`, qui le contredit dans les deux cas. On peut prouver l'arrêt au cas par cas (variant), jamais par une méthode universelle (§ II et III).
3. Non : « ce programme a-t-il un comportement de virus ? » porte sur ce que le programme *fait* ; d'après le **théorème de Rice**, c'est indécidable. Un antivirus repère des signatures connues, sans garantie de tout détecter (§ III.E).

Erreurs fréquentes

- « Le problème de l'arrêt est indécidable parce que c'est trop long à calculer. » **Non** : c'est **impossible**, à tout jamais, quel que soit le temps ou l'ordinateur.
- « On ne peut jamais savoir si un programme s'arrête. » **Faux** : on peut le prouver **au cas par cas**. Ce qui n'existe pas, c'est une **méthode universelle** valable pour **tous** les programmes.
- Confondre **indécidable** (aucun algorithme) et **NP-complet** (un algorithme lent existe).
- Croire que « NP » veut dire « non polynomial ». Non : « **n**on déterministe **p**olynomial », et P $\subseteq$ NP.
- Croire un langage « plus puissant » qu'un autre : tous les langages Turing-complets calculent **les mêmes** fonctions.

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

- ☐ donner un exemple montrant qu'un programme peut prendre un programme (voire lui-même) en argument → ex. 1, 2 ;
- ☐ définir **décidable / indécidable** et donner des exemples décidables → ex. 3, 4 ;
- ☐ **refaire le raisonnement** du problème de l'arrêt (machine `halt`, programme `paradoxe`, les deux cas contradictoires) → ex. 6, 7 ;
- ☐ énoncer la **thèse de Church-Turing** et la **Turing-complétude** → ex. 9, 17 ;
- ☐ [au-delà du programme] expliquer la différence **vérifier / trouver**, et situer **P**, **NP**, **P versus NP** → ex. 10, 11, 12.

Vers le Grand Oral

- **Existe-t-il des questions qu'aucun ordinateur ne pourra jamais résoudre ?** (*le problème de l'arrêt, la preuve par l'auto-référence.*)
- **Vérifier une solution est-il plus facile que la trouver ?** (*P versus NP, Sudoku, cryptographie.*)
- **Gödel et Turing : les mathématiques et l'informatique ont-elles les mêmes limites ?**
- **Pourquoi tous les langages de programmation se valent-ils en théorie ?** (*Turing-complétude.*)

EXERCICES

Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

Chapitre d'idées : la plupart des exercices se rédigent en français. Soignez les raisonnements. Niveau : ★ échauffement ★★ classique ★★★ défi (dont les extraits de bac les plus exigeants) ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.

Un programme est une donnée ↑ cours

★ **Exercice 1** *Qui est la machine, qui est la donnée ?* ▶ corrigé p. 992 • essentiel

Pour chacune des situations, préciser quel est le **programme** (la machine) et quelle est sa **donnée d'entrée**.

1. Dans un terminal, on tape `python3 tri.py`.
2. Le compilateur `gcc` traduit un fichier `jeu.c` en exécutable.
3. Un antivirus analyse le fichier `photo.exe`.
4. Un éditeur de texte colore la syntaxe d'un fichier `code.py`.

▷ coup de pouce

★ **Exercice 2** *Un programme n'est qu'une chaîne de caractères* ▶ corrigé p. 993

En Python, on peut écrire un programme dans une chaîne à l'aide des triples guillemets :

```
programme = """
x = 10
while x > 0:
    x = x - 1
"""
```

1. Expliquer, en une ou deux phrases, pourquoi **tout** programme Python peut être vu comme une chaîne de caractères.
2. En déduire pourquoi il n'y a rien d'absurde à donner un programme **en argument** à un autre programme — ni même à un programme de se prendre **lui-même** en argument.

▷ coup de pouce

Décidable ou indécidable ? ↑ cours

★ **Exercice 3** *Tri sélectif* ▶ corrigé p. 993 • essentiel

Pour chaque problème de décision, dire s'il est **décidable** ou **indécidable**. Justifier d'un mot.

1. « L'entier n est-il premier ? »
2. « La liste `L` est-elle triée dans l'ordre croissant ? »
3. « Le programme `P`, lancé sur l'entrée `x`, va-t-il s'arrêter ? »
4. « Le mot `m` figure-t-il dans ce dictionnaire de 100 000 mots ? »
5. « Les deux programmes `P` et `Q` calculent-ils exactement la même fonction ? »

▷ coup de pouce

★★ **Exercice 4** *Un décideur qui s'arrête toujours* ▶ corrigé p. 993

On veut décider le problème « n est-il premier ? » pour un entier $n \geq 2$.

1. Recopier et compléter cette fonction : ▷ coup de pouce

```
def est_premier(n):
    for d in range(2, ...):        # (a) diviseurs a tester
        if n % d == 0:
            return ...             # (b)
    return ...                     # (c)
```

2. Expliquer pourquoi cette fonction **s'arrête toujours**, quelle que soit l'entrée. Que peut-on en conclure sur la décidabilité du problème ? ▷ coup de pouce

Le problème de l'arrêt ↑ cours

★★ **Exercice 5** *Va-t-il s'arrêter ?* ▶ corrigé p. 994 • essentiel

Pour chaque appel, dire s'il **s'arrête** ou non, et le justifier brièvement.

```
def compte(n):
    while n != 0:
        n = n - 1
    return "fini"
```

```
def mystere(n):
    while n != 1:
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
    return "fini"
```

1. `compte(5)` puis `compte(-3)`. ▷ coup de pouce
2. `mystere(6)`. Dérouler les valeurs successives de `n`.
3. **Culture :** personne au monde ne sait aujourd'hui si `mystere(n)` s'arrête pour *tout* entier $n \geq 1$ (*conjecture de Syracuse*). Que cela illustre-t-il sur la difficulté de prédire l'arrêt d'un programme ?

★★ **Exercice 6** *Refaire la preuve du problème de l'arrêt* ▶ corrigé p. 994 • essentiel

On **suppose** disposer d'une fonction `halt(prog, x)` qui **s'arrête toujours** et renvoie `True` si `prog(x)` s'arrête, `False` sinon. On écrit alors :

```
def paradoxe(prog):
    if halt(prog, prog):
        while True:
            pass
    else:
        return "je m'arrete"
```

1. On lance `paradoxe(paradoxe)`. **Cas 1 :** on suppose que `halt(paradoxe, paradoxe)` renvoie `True`. Quelle branche du `if` est exécutée ? L'appel s'arrête-t-il ? En quoi est-ce contradictoire ? ▷ coup de pouce
2. **Cas 2 :** on suppose que `halt(paradoxe, paradoxe)` renvoie `False`. Reprendre le même raisonnement.
3. **Conclure** : que vient-on de démontrer au sujet de `halt` ?

★★ **Exercice 7** *Cousins du paradoxe* ▶ corrigé p. 994

Expliquer en quoi les deux énoncés suivants « tournent en rond », et quel lien ils ont avec le programme `paradoxe` de l'exercice précédent.

1. « Cette phrase est fausse. »
2. « Le barbier du village rase exactement ceux qui ne se rasent pas eux-mêmes. Rase-t-il le barbier ? »

▷ coup de pouce

★★★ **Exercice 8** *La contagion de l'indécidable* ▶ corrigé p. 994 ≈ 15 min

On admet que le **problème de l'arrêt** est indécidable. On voudrait maintenant un programme `renvoie42(prog, x)` qui dirait à coup sûr si `prog(x)` finit par renvoyer la valeur 42.

1. Montrer que, si `renvoie42` existait, on pourrait s'en servir pour décider le problème de l'arrêt. En déduire que `renvoie42` ne peut pas exister. ▷ coup de pouce ▷ début de solution
2. Comment appelle-t-on le résultat général affirmant que « toute question non triviale sur le comportement d'un programme est indécidable » ?

Calculabilité et Turing-complétude ↑ cours

★ **Exercice 9** *Vrai ou faux* ▶ corrigé p. 995 • essentiel

Répondre **vrai** ou **faux** et justifier d'un mot.

1. Le langage C est « plus puissant » que Python : il peut calculer des fonctions que Python ne peut pas.
2. Une fonction calculable est une fonction pour laquelle il existe un algorithme qui la calcule en un temps fini.
3. La thèse de Church-Turing est un théorème que l'on peut démontrer.
4. Puisque le langage Brainfuck n'a que 8 instructions, il est moins puissant que Python.
5. Le problème de l'arrêt serait décidable si on l'écrivait en C plutôt qu'en Python.

▷ coup de pouce

P, NP et le million de dollars [au-delà du programme] ↑ cours

★ **Exercice 10** *Vérifier, est-ce facile ?* ▶ corrigé p. 995

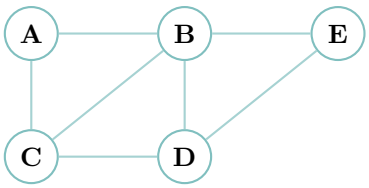
Pour chaque problème, dire si **vérifier** une solution qu'on vous propose est **facile** (rapide), et si **trouver** une solution semble facile ou difficile.

1. Trier une liste de 1000 nombres (la solution proposée est une liste : est-elle triée et contient-elle les mêmes nombres ?).
2. Compléter une grille de Sudoku 9×9.
3. « Existe-t-il une tournée passant par ces 30 villes, de longueur ≤ 500 km ? »

▷ coup de pouce

★★ **Exercice 11** *Colorier un graphe* ▶ corrigé p. 995

On veut colorier les sommets du graphe ci-dessous avec 3 **couleurs**, sans que deux sommets reliés aient la même couleur.



1. On vous **propose** le coloriage : A rouge, B vert, C bleu, D rouge, E rouge. Vérifier s'il est correct (parcourir les arêtes). Ce travail de vérification est-il long ?
2. Combien y a-t-il de façons d'attribuer une couleur (parmi 3) aux 5 sommets, si on les essaie « bêtement » toutes ? Et pour un graphe à n sommets ? ▷ coup de pouce
3. On considère le problème général : « un graphe quelconque peut-il être colorié avec 3 couleurs ? » Est-il dans **NP** ? Sait-on s'il est dans **P** ? Justifier.

★★ **Exercice 12** *Et si $P = NP$?* ▶ corrigé p. 996

Un mathématicien annonce un algorithme **polynomial** qui résout le problème SAT (donc P = NP).

1. Rappeler ce que signifient **P**, **NP**, et pourquoi on a toujours $\mathrm{P} \subseteq \mathrm{NP}$.
2. Citer **deux** conséquences majeures de P = NP (une pour la sécurité informatique, une pour l'idée de « créativité »). ▷ coup de pouce
3. La plupart des chercheurs pensent-ils que P = NP ou que $\mathrm{P} \neq \mathrm{NP}$? Ce point est-il *démontré* ?

Vers le bac ↑ cours

★★ **Exercice 13** *Terminaison et problème de l'arrêt (d'après Métropole septembre 2024, jour 2)* ▶ corrigé p. 996 • essentiel ≈ 45 min

On dit qu'un appel `f(x)` *termine* lorsque son évaluation renvoie une valeur au bout d'un nombre fini d'étapes.

Partie A — boucle `while`.

```
1  def f1(n):
2      i = n
3      while i != 10:
4          i = i + 1
5      return i
```

1. Donner les valeurs successives de `i` lors de `f1(7)`, et dire si `f1(7)` termine.
2. L'appel `f1(-2)` termine-t-il ? Si oui, quelle valeur renvoie-t-il ?
3. Donner les 5 premières valeurs de `i` lors de `f1(12)`, et dire si l'appel termine.
4. Préciser pour quels entiers `n` l'appel `f1(n)` termine. ▷ coup de pouce

Partie B — fonction récursive.

```python
def f2(n):
    if n == 0:
        return 0
    else:
        return n + f2(n - 2)
```

5. `f2(4)` termine-t-il ? Si oui, quelle valeur renvoie-t-il ?
6. `f2(5)` termine-t-il ? Justifier.
7. Déterminer l'ensemble des entiers naturels `n` pour lesquels `f2(n)` termine. ▷ coup de pouce
8. Écrire une fonction récursive `infini` telle que `infini(n)` ne termine pour *aucun* entier `n`. ▷ coup de pouce

Partie C — le problème de l'arrêt. On suppose disposer d'une fonction `arret(code_f, x)` qui renvoie `True` si `f(x)` termine et `False` sinon. On écrit :

```python
def paradoxe(x):
    if arret(x, x):
        infini(42)
    else:
        return 0
```

On dispose d'une variable `code_paradoxe` contenant le code de `paradoxe`, et on s'intéresse à `paradoxe(code_paradoxe)`.

9. Si `arret(code_paradoxe, code_paradoxe)` renvoie `True` : quelle instruction s'exécute ensuite ? L'appel `paradoxe(code_paradoxe)` termine-t-il ?
10. Même question si `arret(...)` renvoie `False`.
11. En déduire qu'une telle fonction `arret` ne peut pas exister.

★★★ Exercice 14 — *Un programme comme chaîne de caractères (d'après Asie 2025, jour 1)*

▶ corrigé p. 997 ≈ 45 min

En Python, `exec(chaine)` exécute le programme contenu dans la chaîne. On définit :

```python
programme1 = """
x = 10
y = 10
while x > 0:
    x = x - 1
    y = y + 1
"""
```

1. Donner les valeurs de `x` et `y` après `exec(programme1)`.
2. Expliquer pourquoi tout programme Python peut être vu comme une chaîne de caractères.
3. On considère :

```python
programme3 = "x = 10\nwhile x != 0:\n    x = x - 2"
programme4 = "x = 10\nwhile x > 0:\n    x = x + 2"
programme5 = "x = 10\nwhile x < 0:\n    x = x + 4"
programme6 = "x = 10\nwhile x != 0:\n    x = x - 4"
```

Pour chacun, dire si `exec` termine ou non, en justifiant.

On cherche une fonction `arret` telle que `arret(programme)` renvoie `True` si `exec(programme)` termine et `False` sinon. Cette fonction `arret` doit donc s'arrêter dans tous les cas.

4. On propose `def arret_essai1(programme): exec(programme) ; return True`. Que réalise ce code, et permet-il de décider si un programme s'arrête ?

▷ coup de pouce

5. Une autre idée : décréter qu'un programme s'arrête s'il ne contient pas le mot `"while"`. Écrire `arret_essai2(programme)` qui renvoie `True` si `"while"` **n'apparaît pas** dans `programme`, `False` sinon. Montrer, par un contre-exemple de chaque type, que cette fonction se trompe.

▷ coup de pouce

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

6. On suppose finalement disposer d'une vraie fonction `arret(programme)` (renvoyant `True` si `exec(programme)` termine, `False` sinon). Écrire `terminaison_inverse(programme)` qui **termine** si `programme` ne termine pas, et **ne termine pas** si `programme` termine.

▷ début de solution

7. On pose :
   `programme_paradoxal = "terminaison_inverse(programme_paradoxal)"`
   Étudier si `exec(programme_paradoxal)` termine, et conclure sur l'existence de `arret`.

8. Cette impossibilité est-elle due à une limite du langage Python ?

★★ Exercice 15 *Un correcteur automatique d'exercices (sujet maison, façon bac)*

▶ corrigé p. 998 ≈ 45 min

Une plateforme d'entraînement en ligne corrige automatiquement les fonctions Python écrites par les élèves. Pour l'exercice « écrire une fonction qui renvoie le carré d'un entier `n` », elle dispose d'une liste de tests, chaque test étant un couple `(entree, resultat_attendu)`, et de la fonction suivante :

```
def verifier(fonction, tests):
    reussis = 0
    for (entree, attendu) in tests:
        if fonction(entree) == attendu:
            reussis = reussis + 1
    return reussis

tests = [(0, 0), (2, 4), (3, 9)]
```

Partie A — Un programme qui en examine un autre.

1. Quelle est la particularité du premier paramètre de la fonction `verifier` ? Quelle notion du cours cela illustre-t-il ?

2. Quatre élèves ont rendu les fonctions suivantes :

```
def carre_a(n):
    return n * n

def carre_b(n):
    return 2 * n

def carre_d(n):
    if n == 0:
        return 0
    elif n == 2:
        return 4
    else:
        return 9
```

```
def carre_c(n):
    resultat = 0
    i = 0
    while i != n:
        resultat = resultat + n
        i = i + 1
    return resultat
```

Donner la valeur renvoyée par `verifier(carre_a, tests)`, `verifier(carre_b, tests)`, `verifier(carre_c, tests)` et `verifier(carre_d, tests)`.

3. Parmi les fonctions qui réussissent les trois tests, l'une est pourtant fausse pour la plupart des entiers positifs. Laquelle ? Donner une entrée qui le montre. Que peut-on en conclure sur ce que prouve une série de tests réussis ? ▷ coup de pouce

Partie B — Le piège de la terminaison.

4. Le professeur ajoute le test `(-3, 9)` à la liste. Donner les valeurs successives de `i` lors de l'appel `carre_c(-3)`. Cet appel termine-t-il ? Que se passe-t-il alors pour l'appel `verifier(carre_c, tests)` ?

5. Modifier la condition de la boucle `while` (et, si besoin, la première instruction du corps de la boucle) pour que `carre_c` termine et renvoie le bon résultat pour **tout** entier `n`, y compris négatif. Justifier la terminaison à l'aide d'un **variant**. ▷ coup de pouce *On pourra utiliser la fonction* `abs` *de Python, qui renvoie la valeur absolue d'un nombre.*

6. Pour ne plus jamais rester bloquée, la plateforme interrompt toute fonction qui calcule pendant plus de 2 secondes et la déclare « non terminante ». Cette règle permet-elle de **décider** si une fonction termine ? Donner un cas où elle se trompe.

7. Le développeur de la plateforme rêve d'une fonction `arret(fonction, x)` qui renverrait, toujours en un temps fini, `True` si `fonction(x)` termine et `False` sinon. Pourquoi est-ce impossible ? Rappeler le nom de ce résultat et l'idée de sa démonstration (on ne demande pas de code).

8. Il voudrait aussi une fonction `equivalentes(f, g)` qui dirait, à coup sûr, si deux fonctions renvoient le même résultat pour **toutes** les entrées. Est-ce possible ? Citer le résultat du cours qui permet de répondre.

★★★ **Exercice 16** *Un antivirus infaillible ? (sujet maison, façon bac)* ▶ corrigé p. 999 ≈ 45 min

Une jeune entreprise affirme avoir mis au point un antivirus « infaillible ». Son cœur est une fonction `dangereux(code)` qui prend en paramètre le code source d'un programme Python (une chaîne de caractères) et qui renverrait `True` si l'exécution de ce programme **appelle un jour** la fonction `formater_disque()`, et `False` sinon.

Partie A — Une première version, purement textuelle.

1. La question « le programme `code` appelle-t-il un jour `formater_disque()` ? » est-elle un **problème de décision** ? Préciser la donnée d'entrée et les réponses possibles.

2. Un stagiaire propose de chercher simplement le texte `"formater_disque"` dans le code. Écrire une fonction `contient(texte, motif)` qui renvoie `True` si la chaîne `motif` apparaît dans la chaîne `texte`, et `False` sinon, **sans** utiliser l'opérateur `in` sur les chaînes. ▷ coup de pouce ▷ début de solution

   ▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

3. On pose :

```
1 def analyse_textuelle(code):
2     return contient(code, "formater_disque")
```

   Cette fonction s'arrête-t-elle toujours ? Quel problème **décide**-t-elle exactement ?

4. On considère les trois programmes suivants :

```
code1 = "if 1 > 2:\n    formater_disque()\n"
code2 = "# surtout ne jamais appeler formater_disque\nx = 0\n"
code3 = "nom = 'formater_' + 'disque'\nexec(nom + '()')\n"
```

Pour chacun, donner la réponse de `analyse_textuelle`, puis dire si son exécution formate réellement le disque. Lesquels sont des « fausses alertes » ? Lequel est un virus « non détecté » ?

Partie B — L'antivirus parfait n'existe pas.

On suppose maintenant qu'une fonction `dangereux(code)` parfaite existe : elle s'arrête toujours et ne se trompe jamais. Soit `programme` le code source d'un programme quelconque qui, lui, n'appelle jamais `formater_disque()`.

5. On fabrique la chaîne `piege = programme + "\nformater_disque()\n"`, c'est-à-dire le même programme suivi d'un appel à `formater_disque()`. On suppose que `programme` ne provoque jamais d'erreur. Expliquer pourquoi l'exécution de `piege` appelle `formater_disque()` **si et seulement si** l'exécution de `programme` termine.

6. En déduire une fonction `arret(programme)`, de deux lignes, qui utilise `dangereux` et renvoie `True` si l'exécution de `programme` termine, `False` sinon. ▷ coup de pouce

7. Quelle contradiction obtient-on ? Que peut-on en conclure sur la fonction `dangereux` ?

8. Quel théorème du cours généralise ce raisonnement ? Les antivirus réels sont-ils pour autant inutiles ? Que doit-on accepter en les utilisant ?

★★★ **Exercice 17** IA IA IA *Un interpréteur miniature (sujet maison, façon bac)* ▶ corrigé p. 1000 ≈ 60 min

On invente un langage de programmation minuscule. Un programme agit sur une **seule** variable `x`, un entier positif ou nul ; c'est une liste d'instructions, numérotées à partir de 0, parmi :

- `"inc"` : augmenter `x` de 1 et passer à l'instruction suivante ;
- `"dec"` : diminuer `x` de 1 (sauf s'il vaut déjà 0) et passer à l'instruction suivante ;
- `"sinul k"` : si `x` vaut 0, aller à l'instruction numéro `k`, sinon passer à la suivante ;
- `"aller k"` : aller à l'instruction numéro `k`.

Le programme s'arrête lorsque le numéro de l'instruction à exécuter dépasse la dernière instruction. On donne trois programmes :

```
P1 = ["sinul 3", "dec", "aller 0"]
P2 = ["inc", "aller 0"]
P3 = ["sinul 6", "dec", "sinul 5", "dec", "aller 0", "aller 5"]
```

Un interpréteur Python de ce langage exécute au plus `limite` instructions ; il renvoie la valeur finale de `x` si le programme s'est arrêté, et `None` si la limite a été atteinte avant :

```
def executer(programme, x, limite):
    i = 0              # numero de l'instruction courante
    etapes = 0         # nombre d'instructions deja executees
    while i < len(programme):
        if etapes == limite:
            return None
        mots = programme[i].split()   # "sinul 3" -> ["sinul", "3"]
        if mots[0] == "inc":
            x = x + 1
            i = i + 1
        elif mots[0] == "dec":
            ...
        elif mots[0] == "sinul":
```

```
            ...
        elif mots[0] == "aller":
            i = int(mots[1])
        etapes = etapes + 1
    return x
```

1. La fonction `executer` reçoit un programme en paramètre. Sous quelle forme ? Citer un programme d'usage courant qui fonctionne sur le même principe.
2. Recopier et compléter les deux blocs `...` (instructions `"dec"` et `"sinul"`). ▷ coup de pouce ▷ début de solution
3. Dérouler l'exécution de `P1` avec `x` valant 2 au départ : donner, à chaque étape, le numéro de l'instruction exécutée et la valeur de `x`. Que renvoie `executer(P1, 2, 100)` ? Combien d'instructions ont été exécutées ? Plus généralement, que fait `P1` ?
4. Que renvoie `executer(P2, 0, 1000)` ? Le programme `P2` s'arrête-t-il ? Justifier.
5. Pour quelles valeurs de départ de `x` le programme `P3` s'arrête-t-il ? Justifier en décrivant ce que fait chaque passage dans la boucle des instructions 0 à 4, et le rôle de l'instruction numéro 5. ▷ coup de pouce
6. On propose la fonction suivante :

```
def arret_limite(programme, x):
    return executer(programme, x, 10**6) is not None
```

   Sachant que `P1` exécute $3x + 1$ instructions lorsqu'il démarre avec la valeur `x`, que renvoie `arret_limite(P1, 10**6)` ? Est-ce la bonne réponse ? Augmenter la limite résoudrait-il le problème *en général* ?
7. En revanche, lorsque `arret_limite` renvoie `True`, peut-on être certain que le programme s'arrête ? Expliquer la dissymétrie entre les deux réponses.
8. On enrichit ce langage (plusieurs variables, nouvelles instructions) jusqu'à le rendre **Turing-complet**. Existe-t-il alors un programme qui décide, pour tout programme de ce langage et toute valeur de départ, s'il s'arrête ? Justifier à l'aide du cours.

Critiquer une réponse d'IA ↑ cours

★★ **Exercice 18** *Le problème de l'arrêt expliqué par un assistant d'IA* ▶ corrigé p. 1001

Un élève demande à un assistant d'IA : « Le problème de l'arrêt est indécidable : cela veut-il dire qu'on ne peut jamais savoir si un programme s'arrête ? » Voici la réponse obtenue :

Réponse de l'assistant

« Le problème de l'arrêt consiste à décider, pour un programme `P` et une entrée `E` quelconques, si `P` lancé sur `E` s'arrête. Alan Turing a démontré en 1936 qu'aucun algorithme ne peut le résoudre dans tous les cas : on suppose qu'un tel programme `halt` existe, on construit un programme `paradoxe` qui fait le contraire de ce que `halt` prédit sur lui-même, et l'on aboutit à une contradiction. Ce résultat ne dépend pas du langage : il vaut pour tout langage Turing-complet, Python compris. Conséquence directe : il est impossible de prouver qu'un programme donné s'arrête ; la seule chose que l'on puisse faire est de l'exécuter et d'attendre. C'est aussi pourquoi de nombreuses questions sur le comportement d'un programme (renvoie-t-il toujours 0 ? affiche-t-il un jour quelque chose ?) sont elles aussi indécidables. »

1. La réponse est-elle correcte ? Pour le vérifier, considérer le programme ci-dessous : peut-on affirmer, *sans l'exécuter*, qu'il s'arrête ? Comment ?

```
1 n = 10
2 while n > 0:
3     n = n - 1
```

2. Localiser et corriger la phrase erronée. ▷ coup de pouce

3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

S'entraîner à l'oral ↑ cours

★★ **Exercice 19** *Expliquer en deux minutes* ▶ corrigé p. 1001

Au Grand Oral comme devant l'examinateur de l'épreuve pratique, il faut savoir **expliquer** une notion clairement, sans lire ses notes. Choisir l'un des sujets suivants (ou le tirer au sort) :

Sujet 1. Expliquer à un camarade de Première en quoi « un **programme** est aussi une **donnée** », avec des exemples.

Sujet 2. Expliquer, sans formalisme, l'idée de la preuve que le **problème de l'arrêt** est indécidable.

Sujet 3. Expliquer pourquoi « **indécidable** » ne veut pas dire « on ne sait jamais répondre ».

Préparation (5 min, seul). Noter au brouillon **trois idées** et **un exemple**, rien de plus : comme au Grand Oral, ces notes servent d'aide-mémoire. **Exposé (2 min, chronométré).** Debout, face à un camarade, **sans lire ses notes** et sans rien écrire : tout passe par la parole. **Retour (2 min).** Le camarade pose une question de relance (on peut alors s'aider d'un schéma), remplit la grille ci-dessous et donne un conseil ; puis on échange les rôles. Cette grille reprend les critères d'évaluation du Grand Oral. ▷ coup de pouce

Grille entre camarades	Oui	En partie	Pas encore
Clair — audible, posé ; chaque mot technique est expliqué	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	☐	☐	☐
Construit — un fil conducteur, tenu en deux minutes, sans lire	☐	☐	☐
Un conseil pour la prochaine fois :			

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Le programme est celui que l'on lance, celui qui « travaille » ; la donnée est ce qu'il reçoit, puis lit ou transforme. Une fois les quatre réponses écrites, que remarque-t-on sur la nature des données ? ↑ énoncé

Ex. 2 – piste De quoi est fait le code source d'un programme quand on l'ouvre dans un éditeur ? Et que peut faire un programme d'une chaîne de caractères qu'il reçoit ? ↑ énoncé

Ex. 3 – piste Un problème est décidable s'il existe un algorithme qui répond, en un temps fini, pour **toute** entrée. Chercher pour chacun un tel algorithme (un parcours fini, des divisions à tester…) ; quand la question porte sur le comportement d'un programme quelconque, se rappeler les résultats du cours. ↑ énoncé

Ex. 4 q. 1 – piste Q1 : un diviseur d de n, autre que 1 et n, est compris entre 2 et $n-1$. Dès qu'on en trouve un, peut-on conclure ? ↑ énoncé

Ex. 4 q. 2 – piste Q2 : combien de tours la boucle `for` fait-elle au plus ? Peut-elle tourner indéfiniment ? ↑énoncé

Ex. 5 q. 1 – piste Pour `compte(-3)`, écrire les premières valeurs de `n` : se rapproche-t-on de 0 ? ↑énoncé

Ex. 6 q. 1 – piste Suivre le code ligne à ligne : quelle instruction est exécutée quand le test du `if` est vrai ? Comparer ensuite avec ce que `halt` avait prédit. ↑énoncé

Ex. 7 q. 2 – piste Supposer la phrase vraie, puis fausse : que se passe-t-il dans chaque cas ? Même démarche pour le barbier : il se rase, puis il ne se rase pas. ↑énoncé

Ex. 8 q. 1 – piste Penser à un programme qui ferait un très long calcul avant, *peut-être*, de renvoyer 42. ↑énoncé

Ex. 8 q. 1 – début de solution Partir d'un programme `prog` quelconque et fabriquer `prog2(x)` qui exécute d'abord `prog(x)`, puis renvoie 42. Quand `prog2(x)` renvoie-t-il 42 ? ↑énoncé

Ex. 9 – piste Relire dans le cours les définitions de fonction calculable, de la thèse de Church-Turing et de la Turing-complétude. Pour les affirmations 1, 4 et 5, se demander si elles comparent des langages Turing-complets. ↑énoncé

Ex. 10 – piste Pour chaque problème, distinguer deux tâches : contrôler une solution qu'on vous donne, ou la fabriquer soi-même. Pour fabriquer, combien de possibilités faudrait-il essayer ? ↑énoncé

Ex. 11 q. 2 – piste Chaque sommet a 3 choix, indépendamment des autres : combien de choix pour 2 sommets ? pour 3 ? ↑énoncé

Ex. 12 q. 2 – piste Casser une clé ou trouver une preuve mathématique : est-ce difficile à *trouver*, facile à *vérifier* ? Que changerait P = NP ? ↑énoncé

Ex. 13 q. 4 – piste Q4 : distinguer $n \leq 10$ et $n > 10$: dans quel cas `i` s'éloigne-t-il de 10 ? ↑énoncé

Ex. 13 q. 7 – piste Q7 : `n` diminue de 2 à chaque appel : pour quels `n` atteint-on la condition d'arrêt `n == 0` ? ↑énoncé

Ex. 13 q. 8 – piste Q8 : que se passe-t-il pour une fonction récursive sans aucun cas de base ? ↑énoncé

Ex. 14 q. 4 – piste Q4 : que se passe-t-il si le programme passé ne s'arrête pas ? ↑énoncé

Ex. 14 q. 5 – piste Q5 : il faut un cas où elle répond `False` à tort et un cas où elle répond `True` à tort. Une boucle `while` est-elle le seul moyen de ne pas s'arrêter ? ↑énoncé

Ex. 14 q. 6 – début de solution Q6 :

```
def terminaison_inverse(programme):
    if arret(programme):
        ...
```

↑énoncé

Ex. 15 q. 3 – piste Q3 : essayer chaque fonction sur une entrée qui ne figure pas dans la liste de tests. ↑

Ex. 15 q. 5 – piste Q5 : la boucle doit faire autant de tours que la valeur absolue de `n`. Un variant est un entier positif qui diminue strictement à chaque tour. ↑énoncé

Ex. 16 q. 2 – piste Q2 : comparer `motif` à chaque tranche `texte[i:i + len(motif)]` : quelles valeurs de `i` faut-il essayer ? ↑énoncé

Ex. 16 q. 2 – début de solution Q2 :

```
def contient(texte, motif):
    n = len(motif)
    for i in range(len(texte) - n + 1):
```

↑énoncé

Ex. 16 q. 6 – piste Q6 : d'après la question 5, demander si `piege` est dangereux revient à poser une autre question sur `programme` : laquelle ? ↑énoncé

Ex. 17 q. 2 – piste Q2 : pour `"dec"`, penser au cas où `x` vaut déjà 0 ; pour `"sinul"`, `mots[1]` est une chaîne, et il y a deux cas selon la valeur de `x`. ↑énoncé

Ex. 17 q. 2 – début de solution Q2 :

```
elif mots[0] == "dec":
```

```
if x > 0:
   x = x - 1
i = i + 1
```

↑ énoncé

Ex. 17 q. 5 – piste Q5 : dérouler `P3` avec `x` valant 2, puis 3, et comparer les deux exécutions. ↑ énoncé

Ex. 18 q. 2 – piste La question 1 montre qu'on peut prouver l'arrêt d'un programme sans l'exécuter : quelle phrase de la réponse affirme le contraire ? ↑ énoncé

Ex. 19 – piste Structurer chaque explication en trois temps : une **définition** en une phrase, un **exemple** petit et concret déroulé à voix haute, puis le **piège** (l'erreur que l'auditeur risque de faire). Finir par une phrase qui répond à la question posée. Sujet 2 : c'est un raisonnement par l'absurde ; annoncer ce que l'on suppose, construire le programme « contradictoire », puis poser la question qui fâche. S'entraîner à le dire en moins d'une minute. ↑ énoncé

TP ET PROJETS

TP La machine de Turing et le prix de la force brute ▶ corrigé p. 1002

simuler, programmer, mesurer

Durée : 2 séances (environ 3 h) · Par deux, sur machine

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t13-tp-calculabilite` — **archive .zip**

Consignes

- Outils : Python (Thonny, IDLE ou VS Code) et le fichier à télécharger `tp_calculabilite_depart.py`.
- ▶ signale ce qui se fait sur machine. Lancer le fichier après chaque fonction complétée : les tests de la fin affichent `OK` ou `ECHEC`.
- Noter sur le cahier (ou dans un fichier) les résultats et les mesures demandés : le bilan final s'appuie dessus.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

But du TP

Le cours affirme deux choses étonnantes : une machine **minuscule** (un ruban, une tête, quelques règles) sait calculer tout ce qui est calculable ; et, parmi les problèmes calculables, certains sont **faciles à vérifier** mais semblent **très longs à résoudre**. On va le voir de ses propres yeux : écrire un **simulateur de machine de Turing** et le programmer (ajouter 1 en binaire, reconnaître un palindrome), puis **mesurer** l'explosion d'une recherche par force brute, et enfin expérimenter les limites d'un « détecteur d'arrêt » à chronomètre.

Partie 1 — Un simulateur de machine de Turing

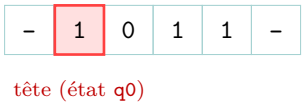
Une machine de Turing lit et écrit sur un **ruban** infini de cases. Dans le fichier, le ruban est un **dictionnaire** `position -> symbole` (une case absente contient le blanc `"_"`) et la machine est un dictionnaire de **règles** :

```
(etat, symbole_lu) -> (symbole_ecrit, deplacement, nouvel_etat)
```

avec `deplacement` $\in \{$`"G"`, `"D"`$\}$.

La machine démarre dans l'état `"q0"`, la tête sur la case 0 (premier symbole du mot) ; elle s'arrête dès qu'elle entre dans un **état final** (`"fin"`, `"oui"` ou `"non"`).

La fonction fournie `executer(regles, mot, limite=10000, trace=False)` fait tourner la machine. Elle renvoie le triplet `(etat_final, ruban_final, nombre_d_etapes)`, ou `None` si la machine ne s'est pas arrêtée après `limite` étapes (et `("bloquee", ruban, etapes)` si aucune règle ne s'applique).




```
INVERSER = {("q0", "0"): ("1", "D", "q0"),
            ("q0", "1"): ("0", "D", "q0"),
            ("q0", "_"): ("_", "G", "fin")}
```

★ Exercice 1 — *Faire tourner une machine à la main*

1. Dérouler la machine `INVERSER` sur le mot `1011` : à chaque étape, noter l'état, le contenu du ruban et la position de la tête. Combien d'étapes avant l'arrêt ? Que contient le ruban à la fin ?
2. Que fait cette machine sur un mot binaire quelconque ?

★ Exercice 2 — *Le cœur du simulateur*

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

▶ Compléter `une_etape(regles, ruban, tete, etat)` : lire le symbole sous la tête, trouver la règle, écrire le nouveau symbole, déplacer la tête, et renvoyer `(nouvelle_position, nouvel_etat)`. La première ligne, fournie, lit le symbole avec `lire_case` (une case absente du ruban contient un blanc). La fonction `executer`, fournie, l'appelle en boucle. Lancer le fichier (`test_une_etape` doit afficher `OK`), puis taper dans la console `executer(INVERSER, "1011", trace=True)` et comparer avec la question 1. ▷ coup de pouce

★★ Exercice 3 — *Une machine qui ne s'arrête pas*

▶ La machine `FUITE` du fichier part vers la droite et ne revient jamais. Que renvoie `executer(FUITE, "1")` ? Et avec `limite=10**6` ?

1. Pourquoi `executer` a-t-il besoin d'un paramètre `limite` ?
2. Quand `executer` renvoie `None`, peut-on affirmer que la machine ne s'arrêtera **jamais** ? Pourquoi augmenter la limite ne règle-t-il pas la question ? Quel résultat du cours est en jeu ?

Partie 2 — Programmer des machines

Écrire une machine, c'est remplir un dictionnaire de règles. **Conseil** : dessiner d'abord la table des règles sur papier, puis la dérouler sur un petit mot, *avant* de la taper.

★★ Exercice 4 — *Ajouter 1 en binaire*

▶ Recopier et compléter sur le cahier la table des règles ci-dessous (les règles de l'état `q0`, puis celles de l'état `retenue`), puis compléter `INCREMENT` pour qu'elle transforme un nombre binaire en son successeur : `1011` → `1100`, `111` → `1000`, `0` → `1`. Elle s'arrête dans l'état `"fin"`. ▷ coup de pouce

État	Lu	Écrit	Déplacement	Nouvel état
q0	0			
q0	1			
q0	_			
retenue	0			
retenue	1			
retenue	_			

Combien d'étapes pour `1011` ? pour `111` ?

★★ **Exercice 5** *Un nombre pair de 1 ?*

▶ Compléter `PARITE` : la machine lit un mot binaire et s'arrête dans l'état `"oui"` s'il contient un nombre **pair** de `1`, dans l'état `"non"` sinon. Elle ne doit **rien écrire** de nouveau sur le ruban.

1. Combien d'états non finaux suffisent ? Que « retient » chacun d'eux ?
2. Une machine de Turing n'a pas de variables : où mémorise-t-elle l'information ?

★★★ **Exercice 6** *Reconnaître un palindrome*

Un **palindrome** (ici, un mot écrit avec les lettres `a` et `b`) se lit de la même façon dans les deux sens (`abba`, `aba`, `b`, le mot vide). La machine `PALINDROME` du fichier procède ainsi : elle **efface** le premier symbole en le mémorisant dans son état (`cherche_a` ou `cherche_b`), va jusqu'au bout du mot, **compare** le dernier symbole avec celui qu'elle a mémorisé, l'efface s'il est égal, **revient** au début et recommence.

▶ Les règles des états `q0`, `cherche_a` et `cherche_b` sont données. Compléter celles des états `compare_a`, `compare_b` et `retour`. Le test essaie les 511 mots sur $\{a, b\}$ de longueur 0 à 8.

▷ coup de pouce

★★ **Exercice 7** *Combien d'étapes ?*

▶ Pour chaque longueur n, fabriquer un palindrome (par exemple `"ab" * k + "ba" * k`, de longueur $n = 4k$) et relever le nombre d'étapes (troisième élément du résultat de `executer`, avec `limite=10**7`) dans le tableau suivant, recopié sur le cahier.

Longueur n	20	40	80	160	320
Étapes					

1. Quand la longueur double, par combien le nombre d'étapes est-il (à peu près) multiplié ? Quel est donc l'ordre de grandeur du coût de cette machine ?
2. En Python, `mot == mot[::-1]` répond instantanément (`mot[::-1]` est le mot lu à l'envers : tranche parcourue avec un pas de -1). La machine de Turing calcule-t-elle « moins de choses » que Python ? Relier à la thèse de Church-Turing : qu'est-ce qui diffère d'un modèle de calcul à l'autre, et qu'est-ce qui ne diffère pas ?

Partie 3 — Trouver ou vérifier ? [au-delà du programme]

Le problème du sous-ensemble de somme. On donne une liste de nombres et une `cible`. Peut-on choisir certains de ces nombres (chacun au plus une fois) dont la somme vaut exactement la cible ? Avec `[8, 3, 12, 5, 7]` et la cible 15 : oui, $3 + 12$ (ou $8 + 7$, ou $3 + 5 + 7$). Avec la cible 100 : non. Ce problème est **NP-complet**, comme le Sudoku généralisé du cours. Un choix est représenté par la liste de ses **indices** : `[1, 2]` désigne 3 et 12.

★ **Exercice 8** *Vérifier une solution proposée*

▶ Compléter `verifier(nombres, cible, choix)` : `True` si les indices de `choix` sont tous différents et si les nombres correspondants ont pour somme `cible`.

1. Combien d'additions fait `verifier` pour un choix de k indices ? Son coût est-il polynomial ?
2. ▶ Mesurer, avec la fonction fournie `chronometrer(fonction, arg1, arg2, ...)`, qui renvoie la durée en secondes de l'appel `fonction(arg1, arg2, ...)`, la vérification d'un choix de 500 000 indices dans une liste d'un million de nombres.

★★ **Exercice 9** *Chercher par force brute*

La fonction fournie `sous_ensemble(k, n)` fabrique le k-ième choix possible parmi les indices 0 à $n-1$: l'indice i est choisi si le bit numéro i de l'écriture binaire de k vaut 1, les bits étant numérotés à partir de 0 **depuis la droite** (le bit numéro i a le poids 2^i).

1. Que renvoient `sous_ensemble(13, 4)` et `sous_ensemble(0, 4)` ? Pour n nombres, combien y a-t-il de choix possibles ? Pourquoi `k` doit-il aller de 0 à $2^n - 1$?
2. ▶ Compléter `chercher(nombres, cible)`, qui essaie les choix dans l'ordre ($k = 0, 1, 2, \dots$) et renvoie le couple `(choix, nombre_d_essais)`, où `choix` est le premier choix qui convient, ou `None` si aucun choix ne convient. Que renvoie `chercher([8, 3, 12, 5, 7], 15)` ? Et avec la cible 100 ?

★★ **Exercice 10** *Mesurer l'explosion*

▶ La fonction fournie `mesures(tailles)` tire n nombres **pairs** et une cible **impaire** : il n'y a aucune solution, donc `chercher` essaie **tous** les choix. Lancer `mesures([10, 12, 14, 16, 18, 20])` (patience pour la dernière !) et recopier les durées sur le cahier, dans un tableau comme celui-ci.

n	10	12	14	16	18	20
Durée (s)						

1. Par combien la durée est-elle multipliée quand n augmente de 2 ? de 1 ?
2. Estimer la durée pour $n = 40$, puis pour $n = 60$ (en jours, en années).
3. Un ordinateur 1 000 fois plus rapide permettrait de traiter, dans le même temps, des listes plus longues de combien d'éléments seulement ? (On rappelle que $2^{10} = 1\,024$.)
4. Résumer en une phrase la différence entre **vérifier** et **trouver** pour ce problème, et dire ce qu'affirmerait « P = NP » à son sujet.

Partie 4 — Une expérience sur l'arrêt

★★★ **Exercice 11** *La suite de Syracuse*

On part d'un entier $n \geq 1$; s'il est pair on le divise par 2, sinon on le remplace par $3n + 1$; on recommence jusqu'à obtenir 1. Par exemple, $6 \to 3 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$: 8 étapes.

1. ▶ Compléter `syracuse_etapes(n, limite)`, qui renvoie le nombre d'étapes pour atteindre 1, ou `None` si 1 n'est pas atteint en `limite` étapes.
2. ▶ Vérifier que, pour tous les entiers de 1 à 100 000, la suite atteint 1 en moins de 1 000 étapes. Quel entier de cet intervalle demande le plus d'étapes, et combien ?
3. Que renvoie `syracuse_etapes(27, 100)` ? Et `syracuse_etapes(27, 1000)` ? Qu'en conclure sur la réponse `None` ?
4. Personne ne sait démontrer que la suite atteint 1 pour **tout** entier (c'est la **conjecture de Syracuse**) : une réponse `None` ne prouve rien, et aucune limite ne suffit. D'autres conjectures célèbres sont dans le même cas, par exemple celle de **Goldbach** (« tout entier pair ≥ 4 est la somme de deux nombres premiers »). On considère le programme suivant, qu'on ne lancera pas (la fonction `est_somme_de_deux_premiers` est supposée écrite, elle s'arrête toujours) :

```
1 def chercher_contre_exemple():
2     n = 4
3     while est_somme_de_deux_premiers(n):
4         n = n + 2
5     return n
```

Dans quel cas ce programme s'arrête-t-il ? Si l'on disposait de la fonction `halt` du cours, comment trancherait-on la conjecture de Goldbach ? Que montre cet exemple sur la puissance qu'aurait un « détecteur d'arrêt » universel ?

Bilan à rédiger

En une dizaine de lignes, sur le cahier ou dans un fichier, en vous appuyant sur **vos** mesures :

- expliquer pourquoi un simulateur de machine de Turing en quelques lignes de Python illustre la **Turing-complétude** de Python, et ce que la machine `PALINDROME` montre sur la différence entre *pouvoir* calculer et calculer *vite* ;
- citer vos durées pour la vérification et pour la recherche, et en tirer la distinction **vérifier / trouver** ;
- expliquer pourquoi une `limite` (ou un chronomètre) ne fournit pas un détecteur d'arrêt, et ce que cela a à voir avec le problème de l'arrêt.

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 2 – piste `ecrit, deplacement, nouvel_etat = regles[(etat, lu)]`, puis `ruban[tete] = ecrit`. Déplacer à droite, c'est ajouter 1 à `tete`. ↑énoncé

Ex. 4 – piste Deux phases, donc deux états : `q0` va à droite jusqu'au premier blanc (le chiffre des unités est juste avant) ; un état `retenue` revient vers la gauche, transforme les `1` en `0` tant que la retenue se propage, et s'arrête dès qu'il peut poser un `1` — sur un `0`, ou sur le blanc à gauche du nombre. ↑énoncé

Ex. 6 – piste Dans `compare_a`, trois cas : on lit `a` (égal : effacer et partir à gauche dans l'état `retour`), on lit `b` (différent : `"non"`), on lit un blanc (il ne restait qu'un symbole, le mot était de longueur impaire : `"oui"`). L'état `retour` va à gauche jusqu'au blanc, puis repasse à droite dans l'état `q0`. ↑énoncé

FICHE DE RÉVISION

L'essentiel

- **Un programme est une donnée** : son code source est une **chaîne de caractères**. On peut le passer en argument à un autre programme, voire **à lui-même**.
- **Problème de décision** : question à réponse *oui* / *non*. Il est **décidable** s'il existe un algorithme qui **s'arrête toujours** et répond juste ; sinon il est **indécidable**.
- **Problème de l'arrêt** (Turing, 1936) : il n'existe **aucun** programme `halt(P, E)` qui, en s'arrêtant toujours, déciderait à coup sûr si `P(E)` s'arrête. C'est le problème **indécidable** le plus célèbre, et le résultat central du chapitre.
- **Preuve** : on suppose `halt` et on construit `paradoxe` qui fait **le contraire** de ce que `halt` prédit de lui-même. `paradoxe(paradoxe)` se **contredit** dans les deux cas ⇒ `halt` ne peut pas exister (raisonnement d'**auto-référence**, comme « cette phrase est fausse »).
- **Théorème de Rice** : toute question **non triviale** sur le *comportement* d'un programme est indécidable (« renvoie-t-il 42 ? », « a-t-il un bug ? »…).
- **Calculable** = calculable par une **machine de Turing** (**thèse de Church-Turing**). Un problème est décidable exactement quand sa fonction réponse est calculable.
- **Turing-complet** : tous les langages usuels (Python, C, Scratch…) calculent **le même** ensemble de fonctions ; ils diffèrent par la commodité et la vitesse, jamais par la puissance.
- [au-delà du programme] **P** : résoluble en temps **polynomial** (« raisonnable »). **NP** : solution *oui* **vérifiable** en temps polynomial. On a **P** ⊆ **NP**.
- **P versus NP** : *trouver* est-il aussi facile que *vérifier* ? Problème **ouvert** (1 M$). On croit **P** ≠ **NP**.

Le raisonnement à savoir refaire

```python
def paradoxe(prog):
    if halt(prog, prog):      # halt : True si prog(prog) s'arrete
        while True:           # ... alors JE boucle
            pass
    else:
        return "je m'arrete" # ... sinon JE m'arrete
# paradoxe(paradoxe) :
#  - si halt dit "s'arrete" -> on boucle  -> NE s'arrete pas  (contradiction)
#  - si halt dit "boucle"   -> on return  -> s'arrete         (contradiction)
#  => halt ne peut pas exister : le probleme de l'arret est indecidable.
```

À bien distinguer

- **Indécidable** = « aucun algorithme, jamais » (problème de l'arrêt). **NP-complet** = « un algorithme existe, mais peut-être très lent » (Sudoku généralisé $n^2 \times n^2$, voyageur de commerce).
- **Vérifier ≠ trouver** : vérifier une grille de Sudoku pleine est facile ; la remplir semble dur. C'est toute l'idée de NP.

Erreurs à éviter

- « C'est indécidable car trop long » : **non**, c'est **impossible** par principe, quel que soit le temps.
- « On ne peut jamais savoir si un programme s'arrête » : **faux**, on peut au **cas par cas** ; ce qui n'existe pas, c'est une méthode **universelle**.
- « NP = non polynomial » : **non**, « non déterministe polynomial », et P $\subseteq$ NP.
- « Tel langage est plus puissant » : tous les langages Turing-complets calculent les mêmes fonctions.

Savoir-faire à maîtriser — je sais…

- ☐ donner un exemple montrant qu'un programme peut prendre un programme (voire lui-même) en argument ;
- ☐ définir **décidable** / **indécidable** et donner des exemples décidables ;
- ☐ **refaire le raisonnement** du problème de l'arrêt (machine `halt`, programme `paradoxe`, les deux cas contradictoires) ;
- ☐ énoncer la **thèse de Church-Turing** et la **Turing-complétude** ;
- ☐ [au-delà du programme] expliquer la différence **vérifier / trouver**, et situer **P**, **NP**, **P versus NP**.

Se tester — *répondre sans regarder la fiche, puis vérifier.* ▶ réponses p. 1005

1. Définir un problème **décidable**.
2. Une fonction `g(n)` répète `n = n - 2` tant que `n != 0`. L'appel `g(7)` s'arrête-t-il ?
3. Si `halt(paradoxe, paradoxe)` renvoie `False`, que fait `paradoxe(paradoxe)` ?
4. Citer deux programmes d'usage courant qui prennent un autre programme comme donnée.
5. Vérifier une grille de Sudoku remplie est rapide ; la remplir semble dur. Quelle question ouverte cela illustre-t-il ?

Et avant ? — *questions sur les chapitres précédents.* ▶ réponses p. 1005

1. Donner la table de décalage de Horspool du motif `NSI`. (ch. 12)
2. Quelle différence entre chiffrement symétrique et asymétrique ? (ch. 10)
3. Que renvoie `fusion([1, 4, 9], [2, 3])` ? (ch. 7)

DÉFIS ADVENT OF CODE

DÉFI 1 Handheld Halting

la console bloquée — interpréteur et problème de l'arrêt

Jour 8

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t13-defi-aoc-2020-08` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/8`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. Une console de jeu exécute un petit programme de plusieurs centaines d'instructions, qui tourne en rond. Il faut d'abord détecter la boucle infinie et dire où en est la machine à ce moment, puis trouver l'unique instruction à corriger pour que le programme se termine. Suivre des centaines de sauts à la main est impossible : on va écrire un **interpréteur**, c'est-à-dire un programme Python qui exécute ce programme.

Ce qu'on réinvestit. Le **problème de l'arrêt** vu dans ce chapitre. On sait qu'aucun programme ne peut décider, pour tout programme, s'il s'arrête. Pourtant, on va ici détecter une boucle infinie à coup sûr : c'est possible seulement parce que cette machine a un nombre fini d'états. Le défi permet de comprendre précisément où passe la frontière entre ce qui est décidable et ce qui ne l'est pas.

La démarche de la fiche.

1. Comprendre les trois instructions, puis exécuter un petit programme à la main.
2. Lire le programme du fichier dans une liste de tuples.

3. Écrire un interpréteur qui s'arrête dès qu'une instruction revient : c'est la partie 1.
4. Partie 2 : essayer chaque réparation possible avec ce même interpréteur.
5. En approfondissement, réduire le nombre d'essais, jusqu'à une réparation en temps linéaire.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`boot code`	programme de démarrage	`instruction`	instruction
`operation / argument`	opération / argument	`accumulator`	accumulateur
`jump`	saut	`offset`	décalage
`infinite loop`	boucle infinie	`terminate`	se terminer

Résumé de l'énoncé (partie 1)

Contexte. En plein vol, un enfant assis à côté du héros n'arrive plus à démarrer sa console de jeu portable : son programme de démarrage tourne en rond. On veut comprendre où il se bloque.
Les données. Le fichier contient ce programme, écrit dans un langage très simple, avec une instruction par ligne. Une instruction est formée d'une opération (`acc`, `jmp` ou `nop`), d'un espace, puis d'un entier toujours écrit avec son signe (`+4`, `-20`). La machine ne possède qu'une seule mémoire, l'*accumulateur*, qui vaut 0 au départ.
La règle.

- `acc n` ajoute `n` à l'accumulateur, puis passe à la ligne suivante.
- `jmp n` est un saut de `n` lignes compté à partir de l'instruction courante : `jmp +1` va à la ligne suivante, `jmp -2` remonte de deux lignes.
- `nop n` ne fait rien (l'argument est ignoré) et passe à la ligne suivante.
- On commence à la première ligne, et l'on exécute les instructions une à une.
- Le programme finit par boucler : dès qu'une instruction est sur le point d'être exécutée une deuxième fois, on s'arrête **avant** de l'exécuter.

La réponse attendue. La valeur de l'accumulateur à ce moment. Par exemple, pour le programme de trois lignes `acc +3`, `acc -1`, `jmp -2`, l'accumulateur vaut 3 puis 2, puis le saut ramène à la première ligne, déjà exécutée : la réponse serait 2.

Exercice 1 *Comprendre ce qui est demandé*

But. Vérifier qu'on sait exécuter les instructions dans les cas particuliers.
À quoi ça sert ? La première question montre qu'un saut nul boucle sur place et qu'un `nop` ignore son argument ; la seconde précise le moment exact de l'arrêt, donc la valeur de l'accumulateur à renvoyer.

Après lecture de l'énoncé et du résumé, vérifier sa compréhension :

1. Que fait l'instruction `jmp +0` ? Et `nop -5` ?
2. Si l'instruction qui revient est un `acc`, son ajout est-il compté une seconde fois ?

Exercice 2 *À la main, sur un petit exemple*

But. Exécuter un programme de huit lignes à la main, en notant l'accumulateur après chaque instruction.
À quoi ça sert ? Le tableau obtenu est exactement ce que l'interpréteur devra calculer ; il donne un exemple dont on **connaît la réponse** (`8`), et il servira à retrouver une erreur pas à pas en cas de résultat faux.

```
acc +2
jmp +3
acc +10
jmp +3
nop -2
acc -4
jmp -4
acc +7
```

On considère le programme (inventé) suivant, dont les lignes sont numérotées à partir de 0.
Exécuter le programme à la main dans un tableau à trois colonnes : numéro de ligne, instruction, accumulateur après l'instruction. Vérification : on doit trouver `8`.

2 Programmer la partie 1

Exercice 3 *Lire le programme*

But. Transformer chaque ligne du fichier en un tuple (opération, argument entier).
À quoi ça sert ? L'interpréteur travaille sur cette liste : la ligne numéro `ip` du programme est simplement `programme[ip]`. Les arguments doivent être des entiers pour pouvoir être ajoutés à l'accumulateur ou au numéro de ligne.

Construire la liste `programme` de tuples `(operation, argument)`, l'argument étant un entier. Par exemple la ligne `jmp -4` donne `("jmp", -4)`. ▷ coup de pouce

Outil Python : les ensembles (`set`) — rappel

Tout est présenté dans l'encadré « Outil Python : les ensembles » du défi *Crab Combat* (chapitre 4) : ensemble vide `set()`, ajout avec `add`, retrait avec `remove`, test `x in ensemble` en temps constant, éléments non modifiables (tuples). Un ensemble ne contient jamais deux fois le même élément.
À essayer. Que vaut `len(vus)` après `vus = {1, 2}` puis `vus.add(2)` ?

Exercice 4 *Un interpréteur*

But. Simuler la machine, une instruction par tour de boucle, et s'arrêter dès qu'une instruction revient.
À quoi ça sert ? C'est la réponse de la partie 1. Cet interpréteur est l'outil central du défi : la partie 2 le réutilise pour tester chaque réparation possible.

Écrire une fonction `executer(programme)` qui simule la machine : une variable `acc`, une variable `ip` (numéro de l'instruction courante), et une boucle qui exécute une instruction par tour. La fonction s'arrête dès qu'une instruction s'apprête à être exécutée pour la deuxième fois et renvoie `acc`. Tester sur l'exemple inventé de la fiche (réponse `8`), puis sur vos données. ▷ coup de pouce

Le problème de l'arrêt : rappel du cours

Peut-on écrire un programme `arret(P, x)` qui dirait, pour **tout** programme `P` et toute entrée `x`, si `P` finit par s'arrêter sur `x` ? Turing a démontré en 1936 que non : ce problème est *indécidable*. On le prouve par l'absurde, en construisant un programme qui fait le contraire de ce que `arret` prédit pour lui-même.
Pourquoi la détection réussit-elle ici ? Parce que la machine de l'énigme a un **nombre fini** d'états : l'état ne dépend que de `ip`, et l'accumulateur n'influence jamais la suite des instructions. Si une instruction revient, tout se répète à l'identique. Un programme quelconque peut, lui, utiliser une mémoire non bornée sans jamais repasser par le même état.

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Il faut maintenant réparer la console pour que le programme aille jusqu'au bout. Les instructions s'exécutent toujours de la même façon ; on cherche quelle ligne modifier.
La nouvelle règle.

- Exactement une instruction est fautive : un `jmp` devrait être un `nop`, ou un `nop` un `jmp`. L'argument ne change pas, et les `acc` sont corrects.
- Le programme réparé se termine, c'est-à-dire qu'il cherche à exécuter la ligne qui suit immédiatement la dernière.

La réponse attendue. La valeur de l'accumulateur quand le programme réparé se termine. Par exemple, le programme `acc +4`, `jmp -1` boucle ; en remplaçant `jmp -1` par `nop -1`, il se termine avec un accumulateur égal à 4.

Méthode : copier une liste avec une tranche (`a[:]`)

Écrire `b = a` ne copie pas la liste : `a` et `b` désignent la **même** liste, et modifier l'une modifie « l'autre ». La tranche complète `a[:]` (du début à la fin) fabrique une nouvelle liste contenant les mêmes éléments.

```
a = [1, 2, 3]
b = a               # meme liste, deux noms
b[0] = 9
print(a)            # [9, 2, 3] : a a change aussi !
c = a[:]            # vraie copie : une nouvelle liste
c[0] = 0
print(a, c)         # [9, 2, 3] [0, 2, 3]
```

Intérêt. Chaque essai de la partie 2 part d'un programme intact. Une copie coûte un temps proportionnel à la longueur de la liste.
À essayer. Avec `a = [[1], [2]]` et `c = a[:]`, que devient `a` après `c[0].append(5)` ? Pourquoi n'est-ce pas un problème pour une liste de tuples ?

Exercice 5 — *Réparer le programme*

But. Essayer une à une les réparations autorisées et repérer celle qui fait terminer le programme.
À quoi ça sert ? C'est la réponse de la partie 2. Il faut pour cela que l'interpréteur distingue une fin normale d'une boucle, et que chaque essai parte du programme d'origine.

1. Modifier `executer` pour qu'elle renvoie aussi un booléen indiquant si le programme s'est terminé normalement. Quelle condition sur `ip` signale une fin normale ?

2. Essayer toutes les modifications autorisées par l'énoncé, une à une, jusqu'à trouver celle qui fait terminer le programme. Sur l'exemple inventé de la fiche, on doit obtenir `5`.

▷ coup de pouce ▷ début de solution

4 Approfondissement

★★★ **Exercice 6** *Défi : combien d'exécutions ?*

But. Évaluer le coût de la méthode par essais successifs, puis réduire le nombre d'essais.
À quoi ça sert ? La méthode précédente relance tout le programme à chaque essai : estimer son coût au pire montre ce qu'on peut gagner en éliminant les essais inutiles.

Pour un programme de n instructions, combien d'instructions la méthode précédente exécute-t-elle au pire ? Proposer et tester une idée pour réduire le nombre d'essais.

▷ coup de pouce

★★ **Exercice 7** *Approfondissement 1 : une réparation en temps linéaire*

But. Trouver la ligne à réparer sans essais, en voyant le programme comme un graphe de lignes.
À quoi ça sert ? En remontant les flèches depuis la fin du programme, on sait d'avance quelles lignes y mènent : une seule exécution réparée suffit. Le problème de réparation, qui semblait demander de nombreux essais, se résout en temps linéaire.

On note n le nombre de lignes et succ(k) la ligne exécutée juste après la ligne k ; la valeur n représente la fin du programme.

1. Sur l'exemple inventé de la fiche, donner succ(k) pour k de 0 à 7.
2. Quelles lignes mènent, sans modification, jusqu'à la fin 8 ? On les trouve en **remontant** les flèches depuis 8.
3. Parmi les lignes atteintes lors de l'exécution de la partie 1, laquelle, une fois échangée (`jmp` et `nop`), envoie vers une ligne trouvée à la question 2 ?
4. Écrire `reparer_lineaire(programme)` qui applique cette méthode et renvoie l'accumulateur du programme réparé. Quelle est sa complexité ?

▷ coup de pouce ▷ début de solution

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste `int("+3")` et `int("-4")` fonctionnent directement : pas besoin de traiter le signe à part. ↑énoncé

Ex. 4 – piste Garder dans un ensemble `deja_vues` les numéros des instructions exécutées. La condition de la boucle `while` porte sur `ip`. ↑énoncé

Ex. 5 – piste Chaque essai doit partir du programme **d'origine** : si l'on modifie la liste sans la recopier, les modifications s'accumulent d'un essai à l'autre. ↑énoncé

Ex. 5 – début de solution `copie = programme[:]`, puis on remplace un seul tuple de `copie`. Inutile d'essayer les lignes `acc`. ↑énoncé

Ex. 6 – piste Modifier une instruction jamais atteinte par l'exécution de la partie 1 ne change rien. ↑

Ex. 7 – piste Construire le graphe inverse : un dictionnaire qui associe à chaque ligne s la liste des lignes k telles que succ(k) = s. Un parcours (avec une pile) depuis n donne l'ensemble `vers_fin`. ↑énoncé

Ex. 7 – début de solution Une seule exécution réparée suffit à la fin. Pourquoi est-on sûr qu'elle se termine ? Si la ligne k modifiée était sur le chemin de la nouvelle ligne vers la fin, k serait dans `vers_fin`, et le programme d'origine se terminerait. ↑

DÉFI 2 Conway Cubes

les cubes de Conway — ensembles et simulation en 3D puis 4D

Jour 17

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t13-defi-aoc-2020-17` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/17`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. On part d'une petite grille plate de cases allumées ou éteintes, plongée dans un espace infini. À chaque étape, chaque case s'allume ou s'éteint selon le nombre de ses voisines allumées. On veut savoir combien de cases sont allumées après six étapes. La zone allumée grandit dans toutes les directions et chaque case a 26 voisines (80 en dimension 4) : impossible de suivre cela à la main.

Ce qu'on réinvestit. Ce défi est une variante en trois dimensions du *jeu de la vie* de Conway, dont la version plane est un exemple célèbre de système Turing-complet, notion vue dans ce chapitre : des règles locales très simples suffisent à produire des comportements aussi riches qu'un programme. Côté programmation, on représente l'espace infini par l'**ensemble** des seules cases allumées, et on construit les voisins d'un point de façon à ce que la dimension soit un simple paramètre.

La démarche de la fiche.

1. Comprendre les règles (voisinage, allumage, extinction) et les appliquer à la main sur un petit exemple.
2. Lire le fichier dans un ensemble de coordonnées.

3. Écrire la fonction des voisins, puis celle qui calcule un cycle : six cycles donnent la réponse de la partie 1.
4. Passer en dimension 4 en modifiant le moins de code possible, puis écrire un code valable pour toute dimension.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`cube`	cube (une case de l'espace)	`active / inactive`	actif / inactif
`neighbor`	voisin	`coordinate`	coordonnée
`cycle`	cycle, étape	`simultaneously`	en même temps
`slice`	tranche, coupe	`boot up`	démarrer
`infinite`	infini	`dimension`	dimension

Résumé de l'énoncé (partie 1)

Contexte. Les elfes vous demandent de l'aide pour une source d'énergie expérimentale : une grille infinie de cubes, chacun actif ou inactif, qui évolue par cycles selon l'état de ses voisins. C'est une version en trois dimensions du célèbre *jeu de la vie* de Conway. Avant de démarrer la source, il faut prévoir son état après la phase de démarrage, qui dure six cycles.
Les données. Le fichier est une petite grille de `#` (actif) et de `.` (inactif), toutes les lignes ayant la même longueur. C'est la tranche $z = 0$ de l'espace : le caractère de la ligne y, colonne x (en numérotant à partir de 0), donne l'état du cube $(x, y, 0)$. Tous les autres cubes de l'espace, en nombre infini, sont inactifs au départ.
La règle.

- Les voisins d'un cube sont les 26 cubes dont chaque coordonnée diffère d'au plus 1 de la sienne : les cubes qui le touchent par une face, une arête ou un coin. Un cube n'est pas son propre voisin.
- À chaque cycle, on compte les voisins actifs de chaque cube **dans l'état d'avant le cycle**, puis tous les cubes changent **en même temps**.
- Un cube actif reste actif s'il a exactement 2 ou 3 voisins actifs ; sinon, il devient inactif.
- Un cube inactif devient actif s'il a exactement 3 voisins actifs ; sinon, il reste inactif.
- Des cubes situés hors de la grille de départ (y compris hors du plan $z = 0$) peuvent donc s'allumer : la zone active grandit.

La réponse attendue. Le nombre de cubes actifs après 6 cycles. Par exemple, avec trois cubes actifs alignés au départ, le cube du milieu reste seul actif parmi eux (2 voisins actifs), les deux bouts s'éteignent (1 seul voisin actif) et 8 cubes qui entourent le milieu s'allument : après un cycle, il y a 9 cubes actifs.

Exercice 1 *Comprendre ce qui est demandé*

But. Vérifier sa compréhension du voisinage et des deux règles d'évolution.
À quoi ça sert ? Ce sont exactement les tests que fera la fonction `cycle` ; la dernière question explique pourquoi on ne pourra pas se contenter de parcourir la grille de départ.

Questions de vérification, à traiter sur le cahier :

1. Le cube $(0, 0, 0)$ est-il voisin de $(1, 1, -1)$? De $(2, 0, 0)$?
2. Un cube actif qui a 4 voisins actifs reste-t-il actif ? Un cube inactif qui en a 2 devient-il actif ?

3. Un cube situé en dehors de la grille de départ peut-il devenir actif ?

Exercice 2 *À la main, sur un petit exemple*

But. Appliquer les règles à la main sur une grille de 3 lignes.
À quoi ça sert ? On obtient des valeurs de référence (5 cubes au départ, 12 après un cycle) qui serviront à tester chaque fonction avant de lancer la simulation sur les vraies données.

On considère l'état initial (inventé) suivant, dans le plan $z = 0$:

```
#.#
.##
#..
```

Combien de cubes actifs au départ ? Compter les voisins actifs du cube central $(1, 1, 0)$ et dire s'il reste actif. Vérification : après un cycle, il y a `12` cubes actifs ; après six cycles, `138`.

2 Programmer la partie 1

Outil Python : les ensembles (`set`) — rappel

Tout est présenté dans l'encadré « Outil Python : les ensembles » du défi *Crab Combat* (chapitre 4) : `set()`, `add`, `remove`, test `x in ensemble` en temps constant, éléments non modifiables (tuples).
À essayer. Essayer `actifs = set()` puis `actifs.add([1, 2, 3])` : que se passe-t-il ? Pourquoi range-t-on les cubes sous forme de tuples `(x, y, z)` ? L'ensemble en compréhension `{(x, 0, 0) for x in range(3)}` contient les trois cubes d'une ligne.

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Exercice 3 *Un ensemble de tuples*

But. Lire le fichier et ne garder que les coordonnées des cubes actifs, dans un ensemble.
À quoi ça sert ? Un tableau ne peut pas représenter un espace infini qui grandit à chaque cycle ; l'ensemble des cubes actifs, lui, ne stocke que l'utile et permet de savoir en temps constant si un cube est actif. Toute la simulation travaillera sur cet ensemble.

L'espace est infini : pas de liste de listes. Écrire une fonction `lire_actifs(nom_fichier)` qui renvoie l'**ensemble** des coordonnées `(x, y, 0)` des cubes actifs. Vérifier qu'on obtient 5 éléments sur l'exemple.

▷ coup de pouce

Méthode : construire les décalages coordonnée par coordonnée

Pour obtenir tous les tuples de longueur `d` formés de -1, 0 et 1, on part de la liste `[()]` (le tuple vide) et, à chaque tour, on prolonge chaque tuple par -1, par 0 et par 1 : la longueur des tuples augmente de 1 et leur nombre est multiplié par 3.

```
tuples = [()]
for _ in range(2):                          # deux coordonnees
    tuples = [t + (c,) for t in tuples for c in (-1, 0, 1)]
print(tuples)       # [(-1, -1), (-1, 0), (-1, 1), (0, -1), ..., (1, 1)]
print(len(tuples))  # 9 : 3 ** 2 couples
```

Intérêt. Le nombre de boucles imbriquées devient un **paramètre** : le même code sert en dimension 3, 4 ou plus.
À essayer. Combien de tuples obtient-on avec 4 tours de boucle ? Lequel faut-il retirer pour obtenir les décalages vers les voisins ?

Exercice 4 *Les voisins*

But. Obtenir la liste des 26 voisins d'un cube.
À quoi ça sert ? Compter les voisins actifs d'un cube, c'est parcourir cette liste et tester l'appartenance à l'ensemble des actifs : cette fonction est appelée des milliers de fois par cycle, et c'est la seule qui changera vraiment en dimension 4.

Écrire une fonction `voisins(cube)` qui renvoie la liste des 26 voisins d'un cube. La tester sur `(0, 0, 0)`. ▷ coup de pouce

Exercice 5 *Un cycle*

But. Calculer l'ensemble des cubes actifs après un cycle, puis répéter six fois.
À quoi ça sert ? C'est la simulation elle-même : six appels successifs donnent la réponse de la partie 1. La difficulté est de choisir les cubes à examiner, puisqu'on ne peut pas parcourir un espace infini.

Écrire une fonction `cycle(actifs)` qui renvoie le **nouvel** ensemble des cubes actifs. Quels cubes faut-il examiner ? Pas seulement les cubes actifs : la zone active grandit à chaque cycle. Simuler six cycles sur l'exemple inventé de la fiche (on doit trouver `12` cubes actifs après un cycle et `138` après six cycles), puis sur vos données. ▷ coup de pouce ▷ début de solution

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Mauvaise surprise : la source d'énergie fonctionne en réalité dans un espace à quatre dimensions. Chaque cube a donc une quatrième coordonnée w, et beaucoup plus de voisins.
La nouvelle règle.

- Les points sont des quadruplets (x, y, z, w) ; la grille du fichier donne l'état des points $(x, y, 0, 0)$, tous les autres sont inactifs.
- Les voisins sont définis de la même façon : chaque coordonnée diffère d'au plus 1, le point lui-même exclu.
- Les règles d'allumage et d'extinction sont exactement les mêmes qu'en partie 1.

La réponse attendue. Le nombre de cubes actifs après 6 cycles, en dimension 4.

★★ **Exercice 6** *Une dimension de plus*

But. Adapter la lecture et les voisins à la dimension 4, sans toucher à la logique du cycle.
À quoi ça sert ? Si les fonctions ont été bien découpées, la règle d'évolution ne dépend pas de la dimension : seules les fonctions qui fabriquent les points changent, et la réponse de la partie 2 s'obtient presque gratuitement.

Combien de voisins un point a-t-il en dimension 4 ? Si votre code est bien écrit, peu de choses changent : adapter `lire_actifs` et `voisins`. Sur l'exemple inventé de la fiche, on doit trouver `36` cubes actifs après un cycle et `680` après six cycles. ▷ coup de pouce

★★★ **Exercice 7** *Défi : un code pour toutes les dimensions*

But. Écrire une seule fonction de simulation où la dimension est un paramètre.
À quoi ça sert ? Plutôt que deux programmes presque identiques, un seul code résout les deux parties : c'est la généralisation, et elle permet aussi de mesurer comment le temps de calcul explose quand la dimension augmente.

Écrire `voisins(point)` pour un tuple de longueur quelconque, sans écrire une boucle par dimension, et une fonction `simuler(nom_fichier, dimension, nb_cycles)`, où `nom_fichier` est le nom du fichier de données, qui renvoie le nombre de cubes actifs après `nb_cycles` cycles et sert aux deux parties. Que donne la dimension 2 ? Et la dimension 5 sur l'exemple inventé de la fiche (mesurer le temps) ? ▷ coup de pouce

4 Approfondissement

★★ **Exercice 8** *Approfondissement 1 : les décalages par récursivité*

But. Construire les décalages vers les voisins avec une fonction récursive.
À quoi ça sert ? La liste des décalages en dimension d se déduit de celle en dimension $d-1$: c'est une définition récursive naturelle, qui donne un code court valable pour toute dimension, et une autre manière de répondre au défi précédent.

On veut construire la liste des décalages par une fonction récursive, plutôt qu'avec la boucle de l'encadré « Méthode ».

1. Lister à la main les tuples de longueur 1 formés de -1, 0, 1, puis ceux de longueur 2, obtenus en plaçant -1, 0 ou 1 devant chacun des précédents. Combien y en a-t-il en longueur d ?

2. Écrire une fonction récursive `decalages_rec(dimension)` qui renvoie les 3^d tuples, le tuple nul compris. Le cas de base est la dimension 0, qui renvoie `[()]` (une liste contenant le tuple vide). ▷ coup de pouce ▷ début de solution

3. Vérifier qu'une fois le tuple nul retiré, on obtient les mêmes décalages qu'avec la méthode coordonnée par coordonnée en dimension 3 et 4, puis l'utiliser dans `simuler`.

4. Combien d'appels récursifs fait `decalages_rec(d)` si l'appel `decalages_rec(dimension - 1)` est placé dans la boucle, comme dans le coup de pouce ? Et si l'on range son résultat dans une variable avant la boucle ? Combien de voisins un point a-t-il en dimension 5 ?

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste Un `set` de tuples : l'appartenance `(x, y, z) in actifs` se teste en temps quasi constant, et un cube inactif n'est tout simplement pas dans l'ensemble. ↑énoncé

Ex. 4 – piste Trois boucles imbriquées sur les décalages dans `[-1, 0, 1]`, en excluant le décalage nul. ↑énoncé

Ex. 5 – piste Seul un cube voisin d'au moins un cube actif peut être actif après le cycle. Les cubes à examiner sont donc les cubes actifs et leurs voisins. ↑énoncé

Ex. 5 – début de solution Plus efficace : parcourir les cubes actifs et, pour chacun de leurs voisins, ajouter 1 dans un dictionnaire `compte`. On applique ensuite les règles à chaque clé de `compte` ; un cube actif absent de `compte` n'a aucun voisin actif. ↑énoncé

Ex. 6 – piste En dimension d, un point a $3^d - 1$ voisins. ↑énoncé

Ex. 7 – piste Construire les décalages coordonnée par coordonnée (voir l'encadré « Méthode »), ou utiliser la fonction récursive de l'approfondissement 1. ↑énoncé

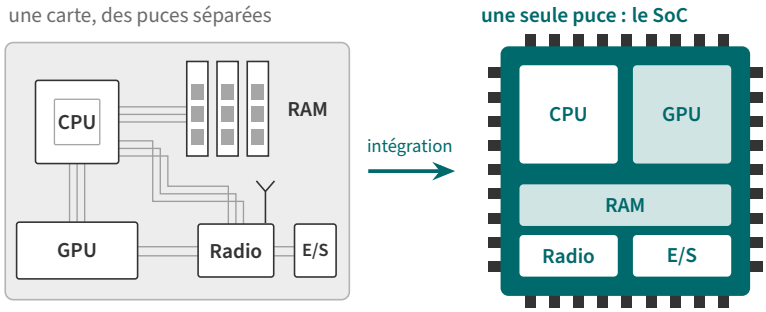
Ex. 8 q. 2 – piste `(-1,)` est un tuple à un seul élément (la virgule est obligatoire), et `(-1,) + (0, 1)` vaut `(-1, 0, 1)`. ↑énoncé

Ex. 8 q. 2 – début de solution Pour chaque `debut` dans `(-1, 0, 1)` et chaque `fin` de `decalages_rec(dimension - 1)`, ajouter `(debut,) + fin` au résultat. ↑énoncé

14

CHAPITRE 14

Systèmes sur puce et informatique embarquée



DANS CE CHAPITRE

- Activités préparatoires 670
- Cours 672
- Exercices 680
- TP et projets 692
- Fiche de révision 698
- Défi Advent of Code 700
- Corrigés 1007

ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Un ordinateur dans une carte de 5 cm

▶ corrigé p. 1007

percevoir, décider, agir avec un micro:bit simulé

Durée : 20 min · Par deux, sur un ordinateur

Consignes

- Ouvrir l'éditeur Python officiel du micro:bit : `python.microbit.org` (gratuit, sans compte). À droite de l'écran se trouve un **simulateur** : la carte, ses deux boutons A et B, son écran de 5 × 5 LED et, en dessous, des curseurs qui simulent ses capteurs (dont la **température**).
- Effacer le programme d'exemple, taper le programme demandé, puis lancer le simulateur (bouton de lecture sous la carte). Après chaque modification, relancer.
- Le micro:bit est une vraie carte, utilisée dans les collèges du monde entier ; tout ce que vous écrivez ici fonctionnerait tel quel sur la carte réelle.

Exercice 1 *Un bouton, une lumière*

Programme 1 :

```
from microbit import *

while True:
    if button_a.is_pressed():
        display.show(Image.HEART)
    else:
        display.clear()
    sleep(100)
```

1. Lancer le programme, puis appuyer sur le bouton A du simulateur (et le relâcher). Décrire ce qui se passe.
2. Dans ce programme, quelle instruction **perçoit** le monde extérieur ? laquelle **décide** ? lesquelles **agissent** ? (Une même ligne peut contenir deux de ces rôles.)
3. Supprimer la ligne `while True:` (et décaler le reste vers la gauche), relancer, appuyer sur A. Que se passe-t-il ? Pourquoi la boucle sans fin est-elle indispensable ?
4. Remettre la boucle. Modifier le programme pour que le cœur ne s'affiche que si l'on appuie **en même temps** sur A et sur B (le bouton B s'appelle `button_b`). Recopier sur le cahier la ligne modifiée.

Exercice 2 *Un thermomètre*

Programme 2 :

```
from microbit import *

while True:
    t = temperature()
    display.scroll(str(t))
    sleep(500)
```

5. Lancer le programme et faire glisser le curseur de température du simulateur. Qu'affiche la carte ? Quel est le type de la valeur renvoyée par `temperature()` ?

6. Ici, quel composant joue le rôle de **capteur** ? Lequel joue le rôle d'**actionneur** ?

Exercice 3 *Un thermostat*

On veut chauffer une pièce à 19 degrés : la LED du centre de l'écran (colonne 2, ligne 2) représente le **radiateur**. L'instruction `display.set_pixel(2, 2, 9)` l'allume au maximum ; `display.clear()` éteint tout l'écran.

Programme 3, à compléter :

```
from microbit import *

consigne = 19
while True:
    t = temperature()
    if ...:
        display.set_pixel(2, 2, 9)    # radiateur allume
    else:
        display.clear()               # radiateur eteint
    sleep(1000)
```

7. Compléter la condition, puis vérifier avec le curseur à 15 degrés et à 25 degrés. Recopier la condition sur le cahier.

8. Le programme ne regarde la température qu'une fois par seconde. Que pourrait-il se passer si l'on remplaçait `sleep(1000)` par `sleep(600000)` (dix minutes) ? Et si ce programme pilotait un **airbag** au lieu d'un radiateur ?

Exercice 4 *Ce qu'on a découvert*

9. Recopier et compléter sur le cahier, avec vos mots.

- Les trois programmes ont la même forme : une boucle qui, sans fin,...
- Un **capteur** sert à... ; un **actionneur** sert à...
- Sur cette carte, on n'a lancé ni Windows, ni Android : dès qu'elle est alimentée, elle... Pourtant, il y a bien un processeur et de la mémoire. Où sont-ils, d'après vous ?

COURS

Programme (BO) *« Composants intégrés d'un système sur puce. Identifier les principaux composants sur un schéma de circuit et les avantages de leur intégration en termes de vitesse et de consommation. »* Commentaire du BO : *« Le circuit d'un téléphone peut être pris comme exemple : microprocesseurs, mémoires locales, interfaces radio et filaires, gestion d'énergie, contrôleurs vidéo, accélérateur graphique, réseaux sur puce, etc. »*

Prérequis le modèle de **von Neumann** (unité de calcul, mémoire, entrées-sorties, bus — vu en Première), la notion de **processus**. → fiches de révision : ch. 4 p. 213
→ rappels de 1re : architecture et OS p. 1065

Objectifs *comprendre* pourquoi un ordinateur entier tient sur une puce ; *distinguer* microcontrôleur et SoC ; *dérouler* le fonctionnement d'un système embarqué ; *peser* les intérêts et les limites de l'intégration.

Situation de départ

1. Un téléphone qui tient dans la poche est aussi puissant qu'un ordinateur de bureau. Où sont donc son processeur, sa mémoire, sa carte graphique ?
2. Un thermostat, une ampoule connectée ou un airbag contiennent eux aussi un petit ordinateur. Que fait-il, sans écran ni système comme Windows ou Android ?
3. Pourquoi ne peut-on presque jamais réparer ou améliorer un téléphone en changeant une seule pièce ?

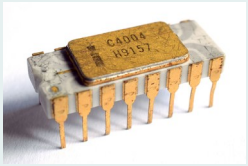
I L'ordinateur qui a rétréci

En 1945, le premier grand ordinateur électronique, l'**ENIAC**, occupait une **salle entière**, pesait **30 tonnes**, contenait près de 18 000 tubes à vide et consommait de quoi alimenter un quartier. Aujourd'hui, la puce qui fait battre votre téléphone est plus petite qu'un ongle, contient **des milliards** de composants, et calcule des *millions* de fois plus vite. En moins d'un siècle, l'ordinateur a rétréci au point de **disparaître** : il est partout, et on ne le voit plus.

Comment un ordinateur complet — processeur, mémoire, périphériques — a-t-il pu tenir sur un seul morceau de silicium grand comme une pièce de monnaie ? C'est l'histoire du **système sur puce**, et le sujet de ce chapitre.

Règle 1 — La loi de Moore (1965, révisée en 1975)

Gordon Moore, cofondateur d'Intel, prévoit en 1965 que le nombre de transistors sur une puce va doubler chaque année ; en 1975, il révise sa prévision : **le nombre de transistors sur une puce double environ tous les deux ans**. Ce n'est pas une loi physique, mais une prédiction économique — restée étonnamment juste pendant un demi-siècle. Le premier microprocesseur, l'**Intel 4004** (1971), comptait 2 300 transistors ; une puce d'aujourd'hui en compte **des dizaines de milliards**, gravés « en 3 **nanomètres** », voire « en 2 nm ». Ces noms sont devenus commerciaux et ne mesurent plus exactement un transistor, mais l'ordre de grandeur est là : les motifs les plus fins ne font que quelques dizaines d'atomes de large.



L'Intel 4004 (1971), l'un des tout premiers microprocesseurs : 2 300 transistors.

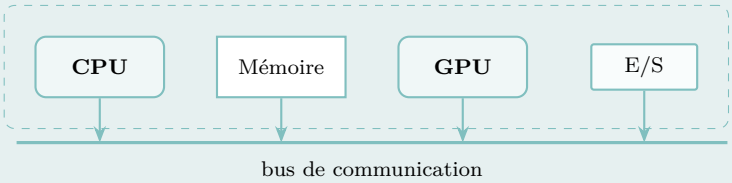
Cette miniaturisation a tout changé : en rapprochant les composants, on a pu **augmenter la fréquence** (sur de courtes distances, les signaux arrivent plus tôt, et des transistors plus petits commutent plus vite), **baisser le coût** (tout est fabriqué d'un coup) et **réduire la consommation** (des transistors plus petits fonctionnent sous une tension plus faible).

II Rappel : ce qu'il y a dans un ordinateur

Depuis la Première, vous connaissez le **modèle de von Neumann** : un ordinateur, c'est une **unité de calcul** (le processeur, CPU), une **mémoire** (qui contient à la fois les programmes et les données), des **entrées-sorties** (clavier, écran, réseau…), le tout relié par des **bus** de communication.

Dans un ordinateur classique, ces éléments sont des **composants séparés**, enfichés sur une grande **carte mère** : le processeur ici, les barrettes de mémoire là, la carte graphique dans son connecteur… On peut les **choisir**, les **remplacer**, les **faire évoluer**. C'est modulaire — mais encombrant, coûteux et gourmand en énergie (les longs fils de cuivre consomment et ralentissent).

la **carte mère** : des composants séparés, remplaçables

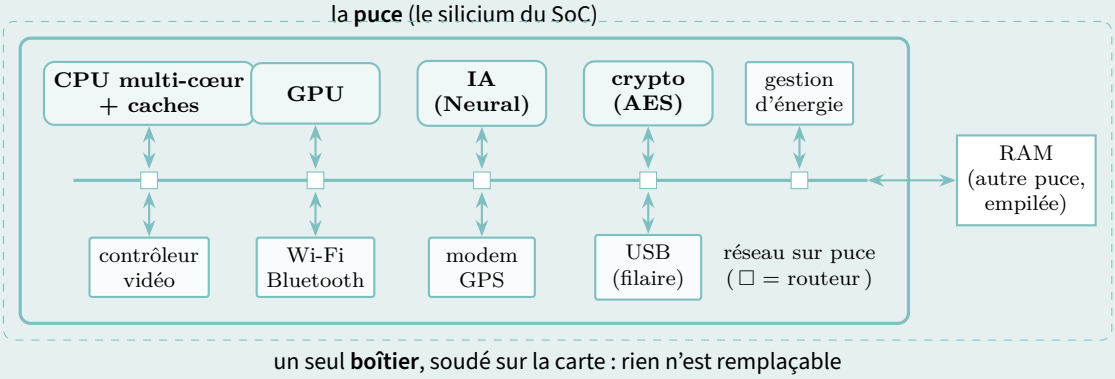


► **Exercices d'application :** exercices 1, 4 **et** 18 (le bon vocabulaire ; la loi de Moore chiffrée ; la loi de Moore en série)

III Le système sur puce (SoC)

Définition 1 — Système sur puce (*System on Chip*, SoC)

Un **système sur puce** rassemble sur un **unique circuit intégré** la plupart des composants qui étaient dispersés sur la carte mère : le **processeur** (souvent multi-cœur, comme au chapitre 4) et ses **mémoires locales** (les caches), un **processeur graphique** (GPU), un **contrôleur vidéo**, des circuits de **gestion d'énergie**, et une foule de **périphériques** : interfaces **radio** (Wi-Fi, Bluetooth, GPS, modem) et **filaires** (USB, liaisons vers l'écran, l'appareil photo, les capteurs…). Ces blocs communiquent par un **réseau sur puce**. Dans un téléphone, la **mémoire vive** principale est une autre puce, empilée sur le SoC dans le **même boîtier**. Un ordinateur complet, dans un boîtier grand comme un ongle.



Mémoires locales et mémoire vive. Au plus près des cœurs, la puce contient des **mémoires locales** très rapides, les **caches** (quelques Mo) : le processeur y garde les instructions et les données qu'il vient d'utiliser, à quelques dixièmes de millimètre de ses circuits de calcul. La **mémoire vive** principale (plusieurs Go) est une puce *séparée* : dans un téléphone, elle est **empilée** sur le SoC, dans le même boîtier. Elle n'est donc pas gravée sur le silicium du SoC, mais elle est soudée tout contre lui : quelques millimètres de trajet… et aucune possibilité de la changer.

Un réseau à l'intérieur de la puce. Sur une puce de plusieurs milliards de transistors, un **bus** unique que tous les blocs se partagent deviendrait un goulot d'étranglement : un seul échange à la fois. Les SoC récents relient donc leurs blocs par un **réseau sur puce** (*Network on Chip*) : de petits **routeurs** (les carrés du schéma) acheminent les données d'un bloc à l'autre, par paquets, et plusieurs échanges ont lieu en même temps — exactement comme les routeurs d'Internet du chapitre 9.

Énergie, affichage, connexions. Les circuits de **gestion d'énergie** règlent en permanence la tension et la fréquence de chaque bloc, et éteignent ceux qui ne servent pas (le GPU quand on écoute de la musique écran éteint, par exemple) : c'est une des clés de l'autonomie. Le **contrôleur vidéo** envoie à l'écran, des dizaines de fois par seconde, l'image calculée par le GPU. Les **interfaces filaires** (USB, liaisons vers l'écran, l'appareil photo, les capteurs) complètent les **interfaces radio**.

Un SoC est un **vrai ordinateur**, aussi puissant : plusieurs cœurs cadencés à plusieurs gigahertz, des gigaoctets de mémoire dans le boîtier, des **processeurs spécialisés** (graphisme, intelligence artificielle, cryptographie)… le tout sur une centaine de millimètres carrés.

Exemple — *La puce Apple A13 (iPhone 11, 2019)*
8,5 **milliards** de transistors gravés à 7 nm, sur une puce grande comme un ongle :

- un CPU à **6 cœurs** (2 rapides à 2,65 GHz, 4 économes à 1,8 GHz) ;
- un **GPU** à 4 cœurs pour la 3D ;
- un **Neural Engine** à 8 cœurs, dédié à l'intelligence artificielle (reconnaissance faciale…) ;
- un module **cryptographique** (il chiffre et déchiffre les données très vite, par exemple en AES, vu au chapitre 10) ;
- des **mémoires caches** sur la puce elle-même, et 4 Go de **RAM empilés sur la puce**, dans le même boîtier (ces 4 Go sont une autre puce : ils ne font pas partie des 8,5 milliards de transistors).

Plus de puissance, dans votre poche, que tous les ordinateurs de la mission Apollo réunis.



La puce Apple A13 Bionic.

Repères — logiciel et matériel : des rôles qui changent

- **1945 : le matériel fait tout.** Pour changer de calcul, on **recâble** l'ENIAC à la main : des jours de travail pour chaque nouveau problème.
- **1948 : le logiciel prend le relais.** Les premiers ordinateurs à **programme enregistré** (la petite machine de Manchester, en juin 1948, puis l'EDSAC de Cambridge en 1949) rangent le programme *en mémoire*, comme une donnée : une même machine fait tout, il suffit de changer de programme. C'est le modèle de von Neumann (vu en Première) et l'idée de la machine universelle de Turing (chapitre 13).
- **Années 1960–1970 : le logiciel s'étoffe.** Systèmes d'exploitation, langages, bibliothèques : le matériel devient une plateforme générique, et c'est le logiciel qui porte l'essentiel des fonctions.
- **Aujourd'hui : le retour du matériel spécialisé.** Un SoC regroupe des blocs dédiés (GPU, Neural Engine, module AES) qui font *en matériel*, plus vite et avec moins d'énergie, ce que le CPU ferait en logiciel. Et la frontière se brouille : une puce est d'abord **décrite dans un langage**, comme un programme, puis simulée et testée avant d'être gravée ; et ses blocs se pilotent par des **API**, comme une bibliothèque (`display.scroll(...)` ou `accelerometer.get_values()` sur le micro:bit).

► **Exercices d'application :** exercices 2, 3, 19 **et** 20 (SoC ou pas ; microcontrôleur contre SoC ; lire le schéma d'un SoC ; logiciel ou matériel)

IV Le microcontrôleur : le petit cerveau de l'embarqué

Tous les systèmes sur puce ne sont pas des monstres de puissance. À l'autre bout du spectre, le **microcontrôleur** est un mini-ordinateur **minuscule, bon marché et sobre**, conçu pour *une* tâche précise dans un **système embarqué** : la carte à puce de votre carte bancaire, le thermostat, le drone, la manette de jeu, le régulateur de vitesse d'une voiture, l'ampoule connectée…

	Microcontrôleur	**SoC (téléphone, tablette)**
Puissance	quelques dizaines de MHz	plusieurs cœurs à plusieurs GHz
Mémoire	quelques **Ko** à quelques centaines de **Ko**	plusieurs **Go**
Consommation	minuscule (des années sur pile)	modérée
Prix	quelques euros	quelques dizaines d'euros
Système d'exploitation	**aucun** (le programme, seul)	un OS complet (iOS, Android…)
Rôle	**une** tâche précise, embarquée	un ordinateur polyvalent

Remarque
Un microcontrôleur, en général, ne fait pas tourner de système d'exploitation complet (il existe seulement de petits systèmes dits « temps réel », très légers) : dès qu'il est sous tension, il exécute *directement* le programme gravé dans sa mémoire. C'est justement ce qu'on veut pour du **temps réel** : un airbag ne peut pas attendre que l'OS ait fini de basculer entre ses processus.

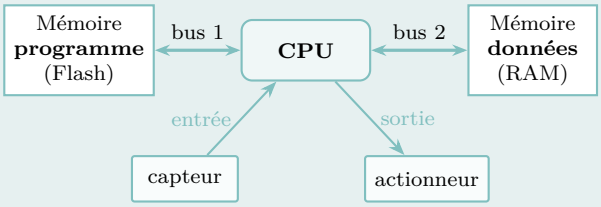
↻ **Retour sur l'activité.** Le micro:bit de l'activité est un microcontrôleur : dès qu'elle est alimentée, la carte exécute directement le programme, sans Windows ni Android, et son processeur comme sa mémoire se trouvent dans la puce elle-même.

A. Capteurs, actionneurs, et l'architecture de Harvard

Un système embarqué dialogue avec le **monde physique** par ses ports d'entrée-sortie : il **lit** des grandeurs physiques grâce à des **capteurs** (température, vitesse, lumière…) et **agit** grâce à des **actionneurs** (moteur, chauffage, LED, frein…). Un **convertisseur analogique-numérique** transforme la tension d'un capteur en nombre binaire exploitable.

↻ **Retour sur l'activité.** Sur le micro:bit, les boutons A et B et le capteur de température étaient des capteurs ; l'écran de LED, qui jouait le « radiateur » du thermostat, était un actionneur.

[au-delà du programme] Beaucoup de microcontrôleurs adoptent l'**architecture de Harvard** : contrairement à von Neumann (une seule mémoire pour tout), le **programme** (en **mémoire morte**, la Flash, qui garde son contenu hors tension) et les **données** (en **mémoire vive**, la RAM, effacée à l'extinction) sont dans *deux mémoires séparées*, reliées par *deux bus distincts*. Avantage : on peut lire une instruction *et* une donnée **en même temps** — un vrai gain de temps.



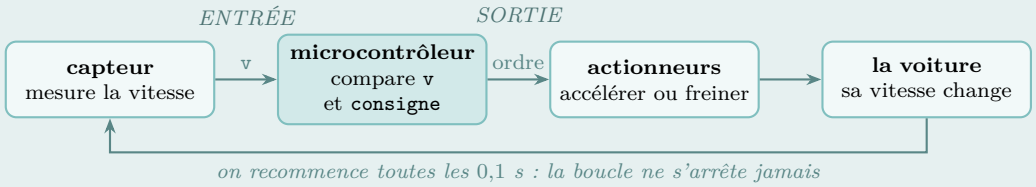
architecture de Harvard : deux mémoires, deux bus

B. Dérouler le fonctionnement d'un système embarqué

Presque tous les systèmes embarqués répètent sans fin la même **boucle** : *lire les capteurs* → *décider* → *commander les actionneurs*. Exemple d'un **régulateur de vitesse** qui maintient une voiture à la vitesse `consigne` :

```
while True:                          # boucle sans fin
    v = lire_capteur_vitesse()       # ENTREE : capteur -> nombre
    if v < consigne:
        accelerer()                  # SORTIE : commande un actionneur
    elif v > consigne:
        freiner()
    attendre(0.1)                    # contrainte TEMPS REEL : reagir vite
```

▷ **Je prédis.** Que fait la voiture pendant un tour de boucle où la vitesse lue vaut exactement `consigne` ?
▷ **Je vérifie.** Rien : ni le `if` ni le `elif` ne s'applique, aucun actionneur n'est commandé ; le programme attend 0,1 s, puis relit le capteur.



Règle 2 — Les trois caractéristiques d'un système embarqué

(1) Acquisition de grandeurs physiques par des capteurs ; **(2) contrôle** d'un procédé par des actionneurs ; **(3) contraintes de temps** fortes (la réponse doit arriver *à temps* — pensez à un airbag ou à un frein ABS).

↻ **Retour sur l'activité.** Les trois programmes de l'activité avaient exactement cette forme : une boucle sans fin qui perçoit, décide (`if`) et agit. Sans le `while True`, la carte ne lisait le bouton qu'une fois ; et un airbag qui ne consulterait ses capteurs que toutes les dix minutes ne servirait à rien : c'est la contrainte de temps.

► **Exercices d'application :** exercices 5, 6 et 17 (dérouler un thermostat ; von Neumann ou Harvard ; systèmes embarqués en série)

V Le prix de l'intégration : intérêts et limites

Pourquoi tout intégrer sur une puce plutôt que garder une carte mère modulaire ? Et qu'y perd-on ?

Intérêts (pourquoi c'est génial)	Limites (ce qu'on y perd)
Énergie : composants très proches, sans câblage énergivore ⇒ faible consommation, peu de chaleur, souvent *sans ventilateur* (silencieux).	**Non réparable** : si un seul transistor lâche, toute la puce est perdue. On ne change pas *une* pièce.
Taille : tout tient dans un objet minuscule (montre, écouteur, capteur).	**Non évolutif** : impossible d'ajouter de la mémoire ou de changer le processeur.
Coût : fabriqué en masse, d'un seul tenant.	**Obsolescence** : la puce vieillit d'un bloc ; on remplace tout l'objet.
Vitesse : les signaux parcourent de très courtes distances.	**Déchets électroniques** : des milliards d'objets non réparables → montagne de déchets (enjeu du *droit à la réparation*).

Pour les curieux (facultatif)
Ce compromis est au cœur d'un débat de société : l'intégration rend les objets *plus petits, moins chers, moins gourmands…* mais *jetables*. Face à la montagne de déchets électroniques, des lois sur le **droit à la réparation** et l'**indice de réparabilité** cherchent à rééquilibrer la balance.

▶ **Exercices d'application :** exercices 7 **et** 8 (le prix de l'intégration ; un microcontrôleur dans la voiture)

VI Ouverture

- **L'informatique invisible.** SoC et microcontrôleurs ont fait naître l'**Internet des objets** (*Internet of Things*, IoT) : montres, ampoules, thermostats, voitures, capteurs agricoles… **des dizaines de milliards** d'objets à puce, souvent connectés. Cette révolution silencieuse pose des questions de **sécurité** (chaque objet est une porte d'entrée) et de **vie privée** (chaque objet est un capteur).

▶ **Exercices d'application :** exercice 14 (l'informatique invisible)

Pour les curieux — *Liens avec d'autres chapitres* (facultatif)

- **Ce qu'il réutilise :** le modèle de von Neumann, vu dans *Architecture des ordinateurs et systèmes d'exploitation* (Première), et les processus (*Processus et ordonnancement*), ordonnancés ici sur plusieurs cœurs.
- **Déjà croisé :** les modules Wi-Fi, Bluetooth et modem relient chaque objet aux *Réseaux*.
- **Pour aller plus loin :** chaque objet connecté est une porte d'entrée, d'où le module AES de la puce (*Cryptographie*).

Bilan — la carte mémoire

Notion	À retenir
Loi de Moore	nb de transistors ×2 tous les 2 ans (empirique)
Carte mère	composants **séparés**, remplaçables, modulaires
SoC	**tout sur une puce** (CPU et caches, GPU, vidéo, énergie, périphériques) ; RAM dans le boîtier
Réseau sur puce	des routeurs relient les blocs (plutôt qu'un bus unique)
Logiciel / matériel	recâblage (1945) → programme enregistré (1948) → blocs spécialisés
Microcontrôleur	mini-SoC : peu puissant, sobre, **sans OS**, embarqué
von Neumann / Harvard	1 mémoire (données+prog.) / 2 mémoires + 2 bus [au-delà du programme]
Système embarqué	capteur → CPU → actionneur, **temps réel**
Intérêts SoC	énergie, taille, coût, vitesse
Limites SoC	non réparable, non évolutif, déchets
IoT	des milliards d'objets connectés à puce

Retour à la situation de départ

1. Ils sont réunis sur une seule puce, un **système sur puce** (SoC) : processeur multi-cœur et ses caches, GPU, contrôleur vidéo et périphériques (Wi-Fi, modem…) sur un unique circuit intégré, la mémoire vive étant empilée dans le même boîtier, grâce à la miniaturisation décrite par la **loi de Moore** (§ I et III).
2. C'est un **microcontrôleur** : sans système d'exploitation complet, il exécute directement son programme, une boucle sans fin « lire les **capteurs**, décider, commander les **actionneurs** », sous des contraintes de temps fortes (§ IV).
3. Parce que tout est intégré sur la même puce : c'est la contrepartie des gains de l'intégration (énergie, taille, coût, vitesse), qui rend un SoC **non réparable** et **non évolutif** (§ V).

Erreurs fréquentes

- **Confondre microcontrôleur et SoC.** Les deux intègrent tout sur une puce, mais le microcontrôleur est *minuscule, sobre, sans OS* (une tâche embarquée) ; le SoC est un *ordinateur complet* (téléphone).
- **Croire qu'un SoC est réparable** comme un PC. Non : rien n'est remplaçable, c'est tout ou rien.
- **Confondre capteur et actionneur.** Le capteur *lit* (entrée) ; l'actionneur *agit* (sortie).
- **Mélanger von Neumann et Harvard** [au-delà du programme]. Von Neumann : *une* mémoire pour programme *et* données. Harvard : deux mémoires, deux bus.

Savoir-faire à maîtriser

À la fin du chapitre, je sais…

☐ **définir** un système sur puce et **identifier** ses constituants sur un schéma → ex. 1, 2, 13, 19 ;

☐ **distinguer** microcontrôleur et SoC, carte mère et puce → ex. 2, 3 ;

☐ **dérouler** le fonctionnement d'un système embarqué (capteur → décision → actionneur) → ex. 5, 11 ;

☐ **énoncer** la loi de Moore et **peser** les intérêts et les limites de l'intégration → ex. 4, 7 ;

☐ **situer** dans le temps l'évolution des rôles du logiciel et du matériel → ex. 20.

Vers le Grand Oral

- **Comment un ordinateur entier a-t-il pu tenir sur une puce ?** *(miniaturisation, loi de Moore, du 4004 au SoC.)*
- **Pourquoi votre téléphone chauffe-t-il moins qu'un PC ?** *(intégration, courtes distances, faible consommation.)*
- **Faut-il regretter les objets qu'on ne peut plus réparer ?** *(le prix de l'intégration ; déchets électroniques ; droit à la réparation.)*
- **Comment un objet connecté « sent-il » le monde et agit-il ?** *(capteurs, actionneurs, boucle temps réel, IoT.)*

Un peu d'histoire

Pour les curieux (facultatif)

Tout commence en **1958–1959** : **Jack Kilby** (Texas Instruments, 1958, sur un morceau de germanium) et **Robert Noyce** (Fairchild, 1959, sur du silicium ; futur cofondateur d'Intel) inventent, chacun de leur côté, le **circuit intégré** — l'idée de graver plusieurs composants sur un seul morceau de semi-conducteur. Kilby recevra le prix Nobel de physique en 2000. En **1971**, une petite équipe d'Intel menée par **Federico Faggin** grave le premier **microprocesseur**, l'**Intel 4004** : 2 300 transistors, aussi puissant que l'ENIAC de 30 tonnes… sur une puce de la taille d'un ongle. Un demi-siècle plus tard, ses descendants en comptent des dizaines de milliards. L'ordinateur n'a jamais cessé de rétrécir — jusqu'à se glisser dans les objets du quotidien.

EXERCICES

Usage de l'IA, indiqué sur chaque exercice : IA aucune ; IA en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; IA l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

Niveau : ★ échauffement ★★ classique ★★★ défi (dont les extraits de bac les plus exigeants) ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme.

★ Exercice 1 — *Le bon vocabulaire*

► corrigé p. 1008 • essentiel

Associer chaque terme à sa définition. ▷ coup de pouce

(1) Système sur puce (SoC)	**(a)** composant qui *agit* sur le monde physique (moteur, LED…)
(2) Microcontrôleur	**(b)** composant qui *mesure* une grandeur physique (température…)
(3) Capteur	**(c)** ordinateur complet et puissant sur une seule puce, avec OS (smartphone)
(4) Actionneur	**(d)** plaque où sont enfichés des composants séparés et remplaçables
(5) Carte mère	**(e)** mini-ordinateur sobre et bon marché, sans OS, pour l'embarqué

★ Exercice 2 — *SoC ou pas SoC ?*

► corrigé p. 1008 • essentiel

Pour chacun de ces objets, dire s'il repose plutôt sur un **microcontrôleur**, un **SoC de smartphone/tablette**, ou une **carte mère classique**, et justifier en une ligne : *(a)* une carte bancaire ; *(b)* un smartphone ; *(c)* un PC de jeu que l'on fait évoluer ; *(d)* un thermostat connecté ; *(e)* une montre connectée. ▷ coup de pouce

★★ Exercice 3 — *Microcontrôleur contre SoC*

► corrigé p. 1008

1. Citer **trois** différences importantes entre un microcontrôleur et le SoC d'un smartphone. ▷ coup de pouce

2. Pourquoi n'installe-t-on en général **pas** de système d'exploitation complet sur un microcontrôleur ? Quel avantage cela donne-t-il pour une application à **temps réel** (par exemple un airbag) ?

3. Le SoC et le microcontrôleur ont pourtant un point commun essentiel. Lequel ?

★★ Exercice 4 — *La loi de Moore, chiffres en main*

► corrigé p. 1008

En 1971, l'Intel 4004 comptait 2 300 transistors. La loi de Moore affirme que ce nombre **double tous les deux ans**.

1. Combien de « doublements » la loi prévoit-elle entre 1971 et 2021 (soit 50 ans) ? ▷ coup de pouce

2. En déduire une estimation du nombre de transistors d'une puce en 2021. (On rappelle $2^{25} \approx 33{,}5$ millions ; on donnera un ordre de grandeur.)

3. La puce Apple A15 (2021) compte environ 15 **milliards** de transistors. La loi de Moore est-elle une prédiction *parfaite* ? Commenter.

4. Les puces récentes sont gravées « en 3 nm » (une appellation commerciale, qu'on prendra ici comme ordre de grandeur de la largeur d'un transistor). Un atome de silicium mesure environ 0,2 nm. Combien d'atomes, environ, dans la largeur d'un transistor ? Que suggère ce résultat sur l'**avenir** de la loi de Moore ?

★★ **Exercice 5** *Dérouler un thermostat* ▶ corrigé p. 1009 • essentiel

Un thermostat connecté maintient une pièce à la température `consigne`. Il dispose d'un capteur de température et peut commander un radiateur (l'allumer ou l'éteindre).

1. Identifier le **capteur** et l'**actionneur** de ce système.
2. Recopier et compléter la boucle de fonctionnement :

```
while True:
    t = ...                    # (a) lire la temperature
    if t < consigne:
        ...                    # (b) que faire ?
    else:
        ...                    # (c) que faire ?
    attendre(30)               # on re-teste toutes les 30 s
```

3. Ce système présente-t-il une **contrainte de temps réel** aussi forte qu'un système de freinage ABS ? Justifier. ▷ coup de pouce

★★ **Exercice 6** *Von Neumann ou Harvard ?* *[au-delà du programme]* ▶ corrigé p. 1009

1. Dans le modèle de **von Neumann**, où sont rangés le programme et les données ? Et dans l'architecture de **Harvard** ?
2. L'architecture de Harvard utilise **deux bus** distincts. Quel avantage concret cela procure-t-il au processeur à chaque cycle ?
3. Un SoC de smartphone suit plutôt le modèle de von Neumann, un petit microcontrôleur plutôt Harvard. Proposer une explication. ▷ coup de pouce

★★ **Exercice 7** *Le prix de l'intégration* ▶ corrigé p. 1009 • essentiel

1. Citer **deux intérêts** et **deux limites** d'un système sur puce par rapport à une carte mère classique.
2. La batterie du téléphone de Léa est usée, mais elle est *soudée* à la carte, tout comme le reste. Expliquer, avec le vocabulaire du cours, pourquoi c'est un problème — et à quel enjeu de société cela renvoie.
3. *Pour aller plus loin.* Pourquoi la faible consommation d'un SoC entraîne-t-elle aussi **moins de chaleur** et donc souvent **l'absence de ventilateur** ? ▷ coup de pouce

★★ **Exercice 8** *Exercice d'entraînement — un microcontrôleur dans la voiture* ▶ corrigé p. 1010

Une voiture moderne embarque des dizaines de calculateurs. On s'intéresse au **régulateur de vitesse**, piloté par un microcontrôleur.

1. Le microcontrôleur reçoit la vitesse du véhicule et commande l'accélération. Nommer le composant **d'entrée** et le composant **de sortie**, et préciser lequel est un capteur, lequel un actionneur.
2. Citer **deux avantages** d'utiliser un microcontrôleur (plutôt qu'un ordinateur classique avec carte mère) dans une voiture.
3. Expliquer pourquoi la **contrainte de temps réel** est ici une question de **sécurité**.

4. Voici le cœur du programme embarqué. Que se passe-t-il, concrètement, quand `v` vaut 120 et `consigne` vaut 130 ? ▷ coup de pouce

```
while True:
    v = lire_capteur_vitesse()
    if v < consigne:
        accelerer()
    elif v > consigne:
        freiner()
    attendre(0.1)
```

★ **Exercice 9** *Type bac — Plusieurs processeurs sur une puce* (d'après Amérique du Nord 2026, jour 2) ▶ corrigé p. 1010 ≈ 15 min

Un système d'exploitation permet d'exécuter plusieurs applications à la fois en donnant l'impression qu'elles fonctionnent simultanément. En réalité, le système répartit le temps de calcul du processeur entre les différents processus, de sorte qu'ils s'exécutent chacun à leur tour très rapidement. Même si un seul processus utilise réellement le processeur à un instant donné, cette alternance rapide donne l'impression que tout s'exécute en même temps.

1. Expliquer l'intérêt d'utiliser une machine équipée de plusieurs processeurs plutôt qu'une machine équipée d'un seul processeur.

2. Décrire un avantage et un inconvénient des systèmes sur puces, tels que ceux utilisés dans les smartphones.

▷ coup de pouce

★★★ **Exercice 10** *Type bac — Choisir la puce d'une montre connectée* (sujet maison, façon bac) ▶ corrigé p. 1010 ≈ 45 min

Une entreprise conçoit une montre connectée destinée aux coureurs. La montre doit mesurer le rythme cardiaque en continu, afficher l'heure, et, de temps en temps, faire tourner de petites applications (carte, messages) sous un système d'exploitation. Sa batterie a une capacité de 300 mAh. Les ingénieurs hésitent entre trois puces :

	Puce A	**Puce B**	**Puce C**
Fréquence	64 MHz	4 cœurs à 1,7 GHz	120 MHz
Mémoire vive	256 Ko	2 Go	1 Mo
Système d'exploitation	aucun	oui	aucun
Consommation moyenne	2 mA	20 mA	8 mA
Prix	2 €	35 €	5 €

On admet que l'autonomie d'un appareil, en heures, est égale à la capacité de la batterie (en mAh) divisée par la consommation moyenne (en mA).

1. Parmi ces trois puces, lesquelles sont des **microcontrôleurs** ? Laquelle est un **SoC** de type smartphone ? Justifier à l'aide de deux lignes du tableau.

2. Rappeler pourquoi on dit d'un microcontrôleur comme d'un SoC qu'il est un « système sur puce ».

3. Calculer l'autonomie de la montre avec chacune des trois puces.

4. Le cahier des charges impose une autonomie d'au moins 24 heures. Quelle(s) puce(s) conviennent ? Laquelle, pourtant, est la seule capable de faire tourner les applications prévues ? Pourquoi ?

5. Les puces sont décrites en Python par une liste de dictionnaires :

```
puces = [
    {"nom": "A", "frequence_MHz": 64, "os": False, "conso_mA": 2},
    {"nom": "B", "frequence_MHz": 1700, "os": True, "conso_mA": 20},
    {"nom": "C", "frequence_MHz": 120, "os": False, "conso_mA": 8},
]
```

Écrire une fonction `autonomie(capacite_mAh, conso_mA)` qui renvoie l'autonomie en heures, puis une fonction `puces_compatibles(puces, capacite_mAh, heures_min)` qui renvoie la liste des **noms** des puces offrant une autonomie d'au moins `heures_min` heures. Que renvoie `puces_compatibles(puces, 300, 72)` ?

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

6. Pour concilier les deux exigences, les ingénieurs retiennent **deux** puces : la puce A fonctionne en permanence (24 h sur 24) pour lire le capteur cardiaque, et la puce B n'est réveillée que 2 heures par jour au total. Calculer la charge consommée en une journée (en mAh), puis l'autonomie de la montre, en jours.

7. Le capteur cardiaque doit être lu toutes les 20 millisecondes, sans jamais prendre de retard. Expliquer pourquoi cette tâche est confiée à la puce A plutôt qu'à la puce B. ▷ coup de pouce ▷ début de solution

★★★ **Exercice 11** *Type bac — Une station météo au lycée* (sujet maison, façon bac) ▶ corrigé p. 1011 ≈ 45 min

Le club de sciences du lycée installe sur le toit une station météo. Elle est pilotée par un **microcontrôleur** d'architecture de Harvard, qui dispose de 32 Ko de mémoire Flash et de 2 Ko (2 048 octets) de mémoire vive. Toutes les 10 minutes, il lit trois capteurs (température, humidité, vitesse du vent) ; si la température passe sous 0 °C, il allume une LED rouge d'alerte « gel » sur le boîtier.

1. Citer les **capteurs** et l'**actionneur** de ce système embarqué.

2. Dans quelle mémoire est stocké le programme du microcontrôleur ? Dans laquelle sont stockées les mesures ? Pourquoi le programme n'est-il pas perdu lorsqu'on coupe l'alimentation ?

3. Le capteur de température délivre une tension, transformée par un **convertisseur analogique-numérique** en un entier codé sur 10 bits. Combien de valeurs différentes ce convertisseur peut-il produire ? Quelles sont la plus petite et la plus grande ?

4. La documentation du capteur indique la formule de conversion suivante :

$$\text{température (en °C)} = \text{valeur} \times 0{,}1 - 40.$$

Écrire une fonction `convertir(valeur)` qui renvoie la température correspondant à une valeur lue. Que renvoie `convertir(512)` (à l'arrondi près) ? Quelle est la plus haute température mesurable ?

5. Écrire une fonction `alerte_gel(mesures)` qui prend en paramètre une liste (non vide) de valeurs brutes lues sur le convertisseur et qui renvoie `True` si l'une d'elles correspond à une température strictement négative, `False` sinon. Que renvoie `alerte_gel([412, 405, 398, 401, 420])` ?

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

6. On dispose des fonctions `lire_capteur_temperature()`, qui renvoie la valeur brute lue sur le convertisseur, `allumer_led()` et `eteindre_led()`. L'alerte porte sur la **dernière** mesure lue. Recopier et compléter la boucle principale du programme :

```
mesures = []
while True:
    valeur = ...                    # (a) lire le capteur de temperature
    mesures.append(valeur)
    if ...:                         # (b) faut-il donner l'alerte ?
        allumer_led()
    else:
        ...                         # (c)
    attendre(600)                   # 600 s = 10 minutes
```

7. Dans le programme complet, les **trois** capteurs sont lus toutes les 10 minutes et chaque mesure est rangée en mémoire vive sur 2 octets. Combien de mesures la station enregistre-t-elle par jour ? Combien d'octets cela représente-t-il ? Combien de jours complets de mesures la mémoire vive peut-elle conserver au maximum ? Quel défaut de la boucle précédente cela révèle-t-il ?

8. [au-delà du programme] Quel avantage l'architecture de Harvard apporte-t-elle par rapport au modèle de von Neumann ? Pourquoi ce type de système n'a-t-il pas besoin de système d'exploitation ? ▷ coup de pouce ▷ début de solution

★★ **Exercice 12** *Type bac — Une salle informatique à renouveler* (sujet maison, façon bac) ▶ corrigé p. 1012 ≈ 45 min

Le lycée doit renouveler les 30 ordinateurs d'une salle. Deux solutions sont étudiées :

- **solution 1** : des ordinateurs « tour » classiques, construits autour d'une **carte mère** (processeur, barrettes de mémoire, carte graphique, carte Wi-Fi enfichés séparément), avec ventilateurs ; puissance moyenne 150 W ;
- **solution 2** : des mini-PC de la taille d'un livre, construits autour d'un **système sur puce**, sans ventilateur, avec une mémoire vive **soudée** contre la puce ; puissance moyenne 15 W.

La salle est utilisée 8 heures par jour, 160 jours par an. On rappelle que l'énergie (en Wh) est égale à la puissance (en W) multipliée par la durée (en h), et que 1 kWh = 1 000 Wh.

1. Parmi les éléments suivants, indiquer ceux qui sont **intégrés sur la puce** d'un système sur puce : processeur (CPU), mémoires caches, processeur graphique (GPU), contrôleur Wi-Fi, alimentation électrique, ventilateur.

2. Expliquer pourquoi le fait de rapprocher tous les composants sur une même puce permet à la fois de **réduire la consommation** et de **se passer de ventilateur**. Quel autre avantage cela présente-t-il dans une salle de classe ?

3. Calculer, en kWh, l'énergie consommée en un an par la salle avec chacune des deux solutions, puis l'économie réalisée par la solution 2. Le prix de l'électricité est de 0,20 € par kWh : quelle économie annuelle, en euros, cela représente-t-il ?

4. Pendant la transition, la salle mélange des tours et des mini-PC. On la représente par une liste, et les puissances par un dictionnaire :

```
puissances = {"tour": 150, "mini": 15}
salle = ["tour", "mini", "mini", "tour", "mini"]
```

Écrire une fonction `energie_salle(postes, puissances, heures)` qui renvoie l'énergie, en kWh, consommée par tous les postes de la liste `postes` pendant `heures` heures. Que renvoie `energie_salle(["mini", "tour", "mini"], puissances, 10)` ?

▷ **Avant de coder.** Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

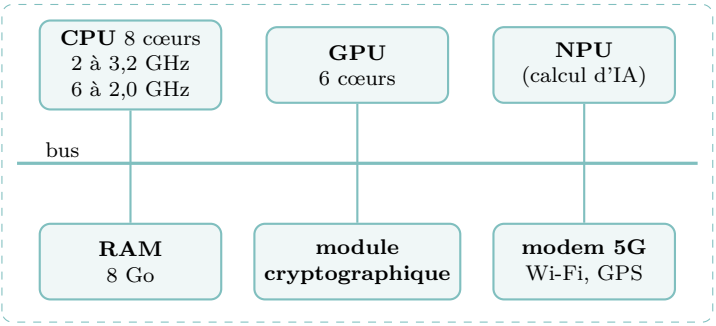
5. Dans trois ans, les logiciels utilisés demanderont deux fois plus de mémoire vive. Expliquer ce que l'on pourra faire avec la solution 1, et ce qu'il faudra faire avec la solution 2.

6. Un mini-PC tombe en panne à cause d'un seul transistor défectueux de sa puce. Peut-on le réparer ? Quelle conséquence environnementale cela a-t-il, et quelle réponse la société tente-t-elle d'y apporter ?

7. Rédiger, en quelques lignes, une recommandation argumentée pour le proviseur (deux arguments au moins pour chaque solution). ▷ coup de pouce ▷ début de solution

★★★ **Exercice 13** *Type bac — Au cœur d'un smartphone* (sujet maison, façon bac) ▶ corrigé p. 1013 ≈ 45 min

Le schéma ci-dessous décrit la puce **Azur X1** (fictive) d'un smartphone sorti en 2025, avec sa mémoire vive : les 8 Go de RAM sont une autre puce, empilée sur l'Azur X1 dans le même boîtier (cadre en pointillés). L'Azur X1 compte 16 milliards de transistors.



Partie A — Architecture.

1. Justifier que l'Azur X1 est un **système sur puce** et non un simple microcontrôleur (deux arguments).

2. Classer les six blocs du schéma selon les trois éléments du modèle de von Neumann : unité de calcul, mémoire, entrées-sorties.

3. Le processeur comporte deux sortes de cœurs : 2 cœurs rapides et 6 cœurs économes. Proposer, pour chacune des deux situations suivantes, le type de cœur le plus adapté, en justifiant : *(a)* lire de la musique, écran éteint ; *(b)* faire tourner un jeu en 3D.

4. *(Rappel du chapitre sur les processus.)* Quel logiciel décide, à chaque instant, quel processus s'exécute sur quel cœur ? Quel est l'intérêt d'avoir 8 cœurs plutôt qu'un seul ?

5. Pourquoi intégrer des processeurs **spécialisés** (GPU, NPU, module cryptographique) plutôt que de tout confier au CPU ?

Partie B — Loi de Moore.

6. L'ancêtre de l'Azur X1, sorti en 2015, comptait 1 milliard de transistors. Selon la loi de Moore, combien de transistors l'Azur X1 aurait-il dû compter en 2025 ? Comparer à la valeur réelle et commenter.

7. On souhaite écrire une fonction `annee_prevue(transistors, annee, objectif)` qui, partant d'une puce de `transistors` transistors sortie l'année `annee`, renvoie l'année à partir de laquelle la loi de Moore prévoit au moins `objectif` transistors. Recopier et compléter :

```
def annee_prevue(transistors, annee, objectif):
    while ...:
        transistors = ...
        annee = ...
    return annee
```

8. Que renvoie `annee_prevue(10**9, 2015, 16 * 10**9)` ? Avec combien d'années de retard sur la prévision l'Azur X1 est-il arrivé ?

9. Que renvoie `annee_prevue(2300, 1971, 10**9)` (on part de l'Intel 4004) ? Justifier par un calcul, sachant que $2^{18} = 262\,144$ et $2^{19} = 524\,288$. ▷ coup de pouce ▷ début de solution

★ Exercice 14 *L'informatique invisible* ▶ corrigé p. 1014

Des dizaines de milliards d'objets connectés (*IoT*) nous entourent, chacun avec sa puce.

1. Donner trois exemples d'objets connectés de la vie courante.

2. Chaque objet connecté est à la fois une **porte d'entrée** sur un réseau et un **capteur** du monde. En quoi cela pose-t-il, respectivement, un problème de **sécurité** et de **vie privée** ?

▷ coup de pouce

★★ Exercice 15 *Le système sur puce expliqué par un assistant d'IA* ▶ corrigé p. 1014

Un élève demande à un assistant d'IA : « Qu'est-ce qu'un système sur puce, et qu'est-ce qui l'a rendu possible ? » Voici la réponse obtenue :

Réponse de l'assistant

« Un système sur puce (*SoC*, *System on a Chip*) regroupe sur un seul circuit intégré presque tous les composants d'un ordinateur : processeur et mémoires caches, processeur graphique, contrôleurs d'entrées-sorties et modules de communication (Wi-Fi, Bluetooth…). Ses avantages : composants très proches donc communications rapides, faible consommation et peu de chaleur (souvent sans ventilateur), encombrement et coût réduits ; son inconvénient : il n'est ni réparable ni évolutif. Cette intégration a été rendue possible par la miniaturisation des transistors, décrite par la loi de Moore : selon cette loi, la fréquence d'horloge des processeurs double environ tous les deux ans, ce qui a permis de passer des 2 300 transistors de l'Intel 4004 (1971) aux dizaines de milliards d'aujourd'hui. Les SoC équipent les smartphones, les tablettes, les objets connectés et de plus en plus d'ordinateurs portables. »

1. La réponse est-elle correcte ? Pour le vérifier, faire un calcul d'ordre de grandeur : si la fréquence d'horloge avait doublé tous les deux ans depuis 1971 (le 4004 tournait à environ 0,7 MHz), quelle fréquence aurait un processeur de 2021 ? Est-ce réaliste (un processeur actuel tourne entre 2 et 5 GHz) ?

2. Localiser et corriger la phrase erronée.

3. Quelle vérification simple, faite avant de faire confiance à la réponse, aurait suffi à détecter l'erreur ?

▷ coup de pouce

★★ **Exercice 16** IA IA IA *Expliquer en deux minutes* ▶ corrigé p. 1014

Au Grand Oral comme devant l'examinateur de l'épreuve pratique, il faut savoir **expliquer** une notion clairement, sans lire ses notes. Choisir l'un des sujets suivants (ou le tirer au sort) :

Sujet 1. Expliquer à un parent ce qu'est un **système sur puce** et pourquoi son smartphone en contient un.

Sujet 2. Expliquer la différence entre un **microcontrôleur** (dans un thermostat, un airbag) et le SoC d'un smartphone.

Sujet 3. Expliquer ce que dit la **loi de Moore**, ce qu'elle ne dit pas, et ses limites.

Préparation (5 min, seul). Noter au brouillon **trois idées** et **un exemple**, rien de plus : comme au Grand Oral, ces notes servent d'aide-mémoire. **Exposé (2 min, chronométré).** Debout, face à un camarade, **sans lire ses notes** et sans rien écrire : tout passe par la parole. **Retour (2 min).** Le camarade pose une question de relance (on peut alors s'aider d'un schéma), remplit la grille ci-dessous et donne un conseil ; puis on échange les rôles. Cette grille reprend les critères d'évaluation du Grand Oral. ▷ coup de pouce

Grille entre camarades	Oui	En partie	Pas encore
Clair — audible, posé ; chaque mot technique est expliqué	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	☐	☐	☐
Construit — un fil conducteur, tenu en deux minutes, sans lire	☐	☐	☐
Un conseil pour la prochaine fois :			

Entraînement ↑ cours

Des séries courtes pour rendre une méthode automatique : même consigne, données différentes. Les réponses sont brèves dans le corrigé.

★ **Exercice 17** IA IA IA *Systèmes embarqués en série* ▶ corrigé p. 1015 • essentiel

Pour chacun des trois systèmes embarqués ci-dessous, nommer le **capteur** et l'**actionneur**, puis dérouler la boucle sur la suite de mesures donnée : pour chaque tour, indiquer la commande envoyée (ou l'absence de commande) et l'état de l'actionneur à la fin du tour. Au départ, tous les actionneurs sont à l'arrêt. ▷ coup de pouce

a) Éclairage d'un couloir. Mesures successives (en lux) : 120, 60, 40, 35, 80.

```
while True:
    lum = lire_luminosite()        # en lux
    if lum < 50:
        allumer_lampe()
    else:
        eteindre_lampe()
    attendre(1)
```

b) Arrosage d'un jardin. Mesures successives (humidité du sol, en %) : 45, 28, 35, 62, 50.

```
while True:
    h = lire_humidite()            # en pourcentage
    if h < 30:
        demarrer_pompe()
    elif h > 60:
        arreter_pompe()
    attendre(600)
```

c) Ventilation d'un boîtier. Mesures successives (en °C) : 45, 55, 72, 68, 49.

```python
while True:
    t = lire_temperature()          # en degres C
    if t >= 70:
        ventiler(2)                 # vitesse rapide
    elif t >= 50:
        ventiler(1)                 # vitesse lente
    else:
        ventiler(0)                 # arret
    attendre(5)
```

★ **Exercice 18** *La loi de Moore en série* ▶ corrigé p. 1015 • essentiel

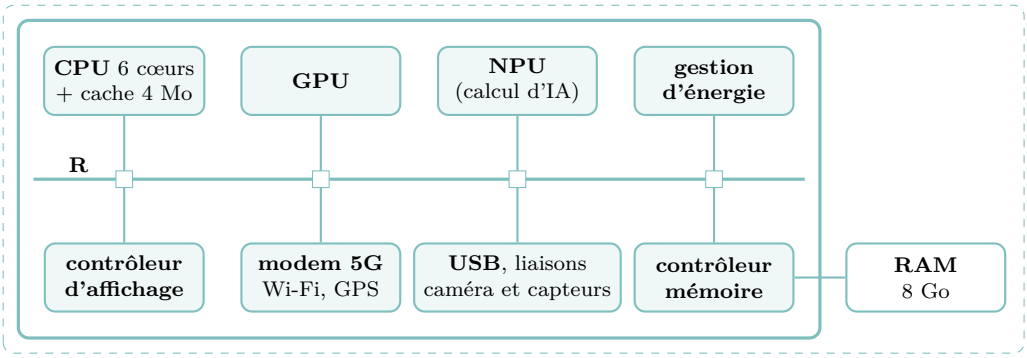
On admet la loi de Moore : le nombre de transistors d'une puce **double tous les deux ans**. Estimer le nombre de transistors prévu : ▷ coup de pouce

a) pour une puce de 5 000 transistors, 10 ans plus tard ;
b) pour une puce d'un million de transistors, 20 ans plus tard ;
c) pour une puce de 3 milliards de transistors, 6 ans plus tard.

Compléments ↑ cours

★★ **Exercice 19** *Lire le schéma d'un SoC de téléphone* ▶ corrigé p. 1015

Voici le schéma simplifié de la puce **Corail C2** (fictive), qui équipe un smartphone. Elle compte 15 milliards de transistors. Ses 8 Go de mémoire vive forment une autre puce, empilée sur elle dans le même boîtier.



1. Pour chacun des huit blocs de la puce, dire s'il sert surtout à **calculer**, à **mémoriser**, à **communiquer** avec l'extérieur (préciser : radio ou filaire), ou à **faire fonctionner** la puce elle-même.

2. Les blocs sont reliés par la ligne **R**, munie de petits carrés. Comment appelle-t-on ce dispositif ? Pourquoi est-il préférable, sur une puce de 15 milliards de transistors, à un bus unique partagé par tous les blocs ?

3. Le cache de 4 Mo est gravé à côté des cœurs. Pourquoi le processeur cherche-t-il d'abord ses données dans ce cache plutôt que dans la mémoire vive ?

4. Il faut au moins un transistor pour mémoriser un bit. Combien de transistors faudrait-il, au minimum, pour graver les 8 Go de mémoire vive sur la puce (1 Go $= 2^{30}$ octets, $2^{30} \approx 1{,}07 \times 10^9$) ? Comparer aux 15 milliards de transistors de la Corail C2 et conclure.

5. On écoute de la musique, écran éteint. Citer deux blocs que la gestion d'énergie peut mettre en veille. Quel est l'intérêt pour l'utilisateur ?

6. Expliquer en quoi le regroupement de tous ces blocs sur une même puce, et de la mémoire vive dans le même boîtier, est un avantage en termes de **vitesse** et de **consommation**, puis donner un inconvénient pour l'utilisateur.

▷ coup de pouce

★ Exercice 20 *Logiciel ou matériel : situer dans le temps*

► corrigé p. 1016

1. Ranger ces événements dans l'ordre chronologique et donner la date de chacun : *(a)* Intel commercialise le premier microprocesseur, le 4004 ; *(b)* pour chaque nouveau calcul, on recâble l'ENIAC à la main ; *(c)* un premier programme enregistré en mémoire s'exécute à Manchester ; *(d)* la puce A13 de l'iPhone 11 intègre un processeur dédié à l'intelligence artificielle ; *(e)* Kilby et Noyce inventent le circuit intégré ; *(f)* Turing décrit une machine universelle.

2. Pour chacune des situations suivantes, dire si la fonction est assurée par du **matériel dédié** ou par du **logiciel** exécuté par un processeur généraliste : *(a)* changer le calcul que fait l'ENIAC ; *(b)* trier une liste avec un programme Python sur un PC ; *(c)* chiffrer en AES les données stockées dans un iPhone 11 ; *(d)* jouer à un autre jeu sur une console en lançant un autre programme.

3. Expliquer pourquoi l'arrivée du programme enregistré (1948) a donné l'avantage au logiciel, puis pourquoi les SoC actuels redonnent de l'importance au matériel spécialisé.

4. Sur le micro:bit, l'instruction `display.scroll("Bonjour")` fait défiler un texte sur l'écran de LED. En quoi ressemble-t-elle à l'appel d'une fonction de bibliothèque ? Qu'est-ce qui, derrière cette instruction, est du matériel ?

▷ coup de pouce

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 – piste Commencer par les termes les plus simples : un capteur mesure, un actionneur agit. Pour les trois autres, se demander : tout est-il sur une seule puce ? Y a-t-il un système d'exploitation ? Les composants sont-ils séparés et remplaçables ? ↑ énoncé

Ex. 2 – piste Pour chaque objet, se demander s'il fait une seule tâche précise avec peu de ressources, s'il s'agit d'un ordinateur complet et polyvalent mais compact, ou si l'on veut pouvoir changer ses composants un par un. ↑ énoncé

Ex. 3 q. 1 – piste Comparer point par point : puissance (fréquence), quantité de mémoire, consommation, prix, présence ou non d'un système d'exploitation. ↑ énoncé

Ex. 4 q. 1 – piste Combien de périodes de deux ans y a-t-il en 50 ans ? Chaque doublement multiplie le nombre de transistors par 2 : par combien est-il multiplié après tous ces doublements ? ↑ énoncé

Ex. 5 q. 3 – piste Question 2 : que doit faire le radiateur quand il fait trop froid ? quand la consigne est atteinte ? Question 3 : comparer les conséquences d'un retard de quelques secondes dans les deux systèmes. ↑ énoncé

Ex. 6 q. 3 – piste Relire le schéma des deux architectures dans le cours : combien de mémoires, combien de bus ? Pour la question 3, se demander lequel des deux exécute toujours le même programme, et lequel doit pouvoir en charger de nouveaux. ↑ énoncé

Ex. 7 q. 3 – piste Reprendre les critères du cours : consommation, taille, coût, vitesse d'un côté ; réparation, évolution, déchets de l'autre. Pour la question 3, se demander ce que devient l'énergie électrique consommée par un circuit. ↑ énoncé

Ex. 8 q. 4 – piste Entrée : ce qui *mesure* ; sortie : ce qui *agit*. Pour la question 4, dérouler un tour de boucle à la main avec les valeurs données. ↑ énoncé

Ex. 9 – piste Question 1 : avec un seul processeur, deux processus peuvent-ils s'exécuter réellement au même instant ? Question 2 : relire dans le cours les intérêts et les limites de l'intégration sur une seule puce. ↑énoncé

Ex. 10 q. 7 – piste Questions 1 et 4 : regarder les lignes « Fréquence », « Mémoire vive » et « Système d'exploitation » du tableau. Question 6 : calculer séparément la charge consommée par chaque puce en une journée (consommation multipliée par la durée de fonctionnement), puis additionner. Question 7 : penser au partage du processeur par un système d'exploitation. ↑énoncé

Ex. 10 q. 7 – début de solution Question 5 : `autonomie` tient en une ligne (`return` d'une division). Pour `puces_compatibles` : partir de `resultat = []`, parcourir `for puce in puces:` et ajouter le nom de la puce quand `autonomie(capacite_mAh, puce["conso_mA"])` est suffisante. ↑énoncé

Ex. 11 q. 8 – piste Question 3 : combien de nombres différents peut-on écrire avec 10 bits ? Question 5 : réutiliser `convertir`. Question 7 : compter les relevés par heure, puis par jour, avant de multiplier par le nombre de capteurs et d'octets ; comparer ensuite à la taille de la mémoire vive. ↑énoncé

Ex. 11 q. 8 – début de solution Question 5 :

```
def alerte_gel(mesures):
    for valeur in mesures:
        if convertir(valeur) < 0:
```

Que renvoyer dans ce cas ? Et que renvoyer après la boucle ? ↑énoncé

Ex. 12 q. 7 – piste Question 3 : énergie d'un poste en un an = puissance × heures par jour × jours par an ; multiplier ensuite par le nombre de postes. Question 4 : accumuler, poste par poste, l'énergie consommée. Questions 5 et 6 : se demander ce qui est *enfiché* et ce qui est *gravé* sur la puce. ↑énoncé

Ex. 12 q. 7 – début de solution Question 4 : commencer par `total = 0`, puis `for poste in postes:` ; la puissance d'un poste se lit avec `puissances[poste]`. Ne pas oublier de convertir le total en kWh avant de le renvoyer. ↑énoncé

Ex. 13 q. 9 – piste Question 2 : pour chaque bloc, se demander s'il calcule, s'il retient des données ou s'il échange avec l'extérieur. Question 6 : combien de périodes de deux ans entre 2015 et 2025 ? Question 9 : chercher le plus petit nombre de doublements qui fait dépasser $10^9 \div 2\,300$. ↑énoncé

Ex. 13 q. 9 – début de solution Question 7 : la boucle tourne tant que l'objectif n'est pas atteint, `while transistors < objectif:` ; que deviennent `transistors` et `annee` à chaque période de deux ans ? ↑énoncé

Ex. 14 – piste Question 1 : penser aux objets de la maison, du poignet, de la voiture. Question 2 : un objet relié à un réseau peut être joint à distance, par n'importe qui ; un capteur enregistre des informations sur son utilisateur : qui peut y accéder ? ↑énoncé

Ex. 15 q. 3 – piste Question 1 : compter les doublements en 50 ans, multiplier la fréquence de départ par 2 autant de fois, puis convertir en GHz (1 GHz = 1 000 MHz) et comparer. ↑

Ex. 16 – piste Structurer chaque explication en trois temps : une **définition** en une phrase, un **exemple** petit et concret déroulé à voix haute, puis le **piège** (l'erreur que l'auditeur risque de faire). Finir par une phrase qui répond à la question posée. Partir d'un objet que l'auditeur a en main (son téléphone, sa carte bancaire) et dire ce qu'il y a dedans avant de nommer la notion. Sujet 3 : qu'est-ce qui double, exactement ? ↑énoncé

Ex. 17 – piste Un tour de boucle suit toujours le même schéma : lire le capteur, décider avec les tests, commander l'actionneur. Faire un tableau avec une ligne par mesure : valeur lue, test vrai, commande envoyée, état de l'actionneur. Si aucun test n'est vrai, aucune commande n'est envoyée : l'actionneur reste dans son état. ↑énoncé

Ex. 18 – piste Commencer par compter les périodes de deux ans, c'est-à-dire le nombre de doublements d. Le nombre de transistors est alors multiplié par 2^d. On rappelle $2^{10} = 1\,024 \approx 1\,000$. ↑énoncé

Ex. 19 – piste Question 1 : un contrôleur d'affichage envoie l'image à l'écran ; un contrôleur mémoire organise les échanges avec la RAM. Question 2 : relire le paragraphe « Un réseau à l'intérieur de la puce » du cours et penser aux routeurs du chapitre 9. Question 4 : un octet vaut 8 bits. ↑énoncé

Ex. 20 – piste Question 1 : relire l'encadré « Repères » du cours (§ III) et l'histoire de la fin du cours ; la machine universelle est vue au chapitre 13. Question 3 : avec un programme en mémoire, que faut-il changer pour faire faire autre chose à la machine ? Que gagne-t-on à confier une tâche fréquente à un circuit spécialisé ? ↑énoncé

TP ET PROJETS

TP Programmer un système embarqué ▶ corrigé p. 1017

capteurs, boucle temps réel et système sur puce avec le micro:bit

Durée : 2 h 30 à 3 h (deux séances) · Sur machine, par deux

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t14-tp-soc` — archive .zip

Consignes

- Parties A et B : éditeur Python officiel du micro:bit, `python.microbit.org`, avec son **simulateur** (curseurs des capteurs sous la carte ; la sortie de `print` s'affiche dans la console série, sous l'éditeur). **Aucune carte n'est nécessaire** : tout le TP se fait dans le simulateur (parties A et B) et en Python sur l'ordinateur (parties C et D).
- Parties C et D : Python habituel sur l'ordinateur, avec le fichier à télécharger `tp_soc_depart.py`.
- Niveau : ★ échauffement ★★ classique ★★★ défi (dont les extraits de bac les plus exigeants) ; • essentiel : parcours minimum ; ▷ coup de pouce : indice en fin de rubrique. L'icône en haut à droite d'un bloc de code ouvre une page d'où copier le programme. Le symbole ▶ signale ce qui est à programmer et tester.
- Usage de l'IA, indiqué sur chaque exercice : aucune ; en appui (déboguer, reformuler, vérifier ; la réponse finale est écrite et comprise par vous) ; l'exercice s'appuie sur une réponse d'IA. Voir la charte, en début de livre.

But du TP

Construire de vrais **systèmes embarqués** — une veilleuse, une alarme, un podomètre — sur le modèle **capteur** → **traitement** → **actionneur**, mesurer le **rythme** de leur boucle et leur **consommation**, **tester leur logique** sur ordinateur avec des capteurs simulés, puis **analyser la puce** qui les fait tourner et la comparer à celle d'un PC. *Produit final* : trois programmes embarqués qui fonctionnent, et une fiche d'analyse du système sur puce.

Aide-mémoire micro:bit (MicroPython). Toujours commencer par `from microbit import *`.

Instruction	Rôle
`temperature()`	température en degrés (entier)
`display.read_light_level()`	lumière ambiante, de 0 (noir) à 255 (plein jour)
`accelerometer.get_values()`	triplet `(x, y, z)` d'accélérations en milli-g
`button_a.was_pressed()`	`True` si A a été pressé depuis le dernier appel
`display.set_pixel(x, y, v)`	allume la LED de colonne `x`, ligne `y` (de 0 à 4), intensité `v` de 0 à 9
`display.show(Image.HEART)`, `display.clear()`	affiche une image ; éteint l'écran
`display.scroll("texte")`	fait défiler un texte (et **attend** la fin du défilement)
`sleep(n)`, `running_time()`	attend `n` millisecondes ; temps écoulé depuis le démarrage, en ms
`import music` puis `music.pitch(880, 100)`	joue un son de 880 Hz pendant 100 ms

Partie A — La boucle et son rythme

★ **Exercice 1** *Combien de tours par seconde ?*

▶ Taper et lancer ce programme, puis ouvrir la console série.

```
1  from microbit import *
2
3  n = 0
4  debut = running_time()
5  while True:
6      n = n + 1
7      if running_time() - debut >= 1000:     # une seconde s'est ecoulee
8          print(n)
9          n = 0
10         debut = running_time()
```

1. Que représente le nombre affiché chaque seconde ? Noter sa valeur (approximative).
2. Ce programme a-t-il un capteur ? un actionneur ? Que fait-il de « utile » ?
3. Le simulateur tourne dans le navigateur, sur le processeur de l'ordinateur. Une vraie carte donnerait-elle la même valeur ? Pourquoi ?

★★ **Exercice 2** *Ce qui ralentit la boucle*

▶ Ajouter, **une à la fois**, dans la boucle (juste après `n = n + 1`), chacune des lignes suivantes : relancer, relever le nombre de tours par seconde dans le tableau (recopié sur le cahier), puis **retirer la ligne** avant d'essayer la suivante.

Ligne ajoutée	Tours par seconde
(rien)	
`x, y, z = accelerometer.get_values()`	
`sleep(20)`	
`display.scroll("Hi")`	
`display.scroll("Hi", wait=False)`	

1. Avec `sleep(20)`, quel est le nombre **maximal** de tours par seconde ? Le comparer à la mesure.
2. Avec `display.scroll("Hi")`, la carte lit-elle encore ses capteurs pendant le défilement ? Imaginer les conséquences pour un **airbag** ou un **podomètre**.
3. Quel est l'effet de `wait=False` ? Énoncer une règle de programmation d'un système **temps réel**.

Partie B — Trois systèmes embarqués

Pour chaque système, on identifie d'abord **capteur**, **traitement** et **actionneur**, puis on programme la boucle.

★ Exercice 3 — *Une veilleuse qui s'allume avec la nuit*

La veilleuse allume d'autant plus de LED qu'il fait sombre : 0 LED en plein jour (lumière 255), les 25 dans le noir complet (lumière 0). On calcule ce nombre avec `(255 - lumiere) * 25 // 255`.

1. Combien de LED pour une lumière de 128 ? de 200 ?
2. ▶ Compléter les pointillés dans l'éditeur et tester le programme (faire varier le curseur de lumière du simulateur). Les LED se remplissent ligne par ligne : la LED numéro `k` (de 0 à 24) est en colonne `k % 5` et en ligne `k // 5`.

```
from microbit import *

while True:
    lumiere = ..............................
    nb = ...................................
    display.clear()
    for k in range(nb):
        display.set_pixel(.........., .........., 9)
    sleep(200)
```

3. Identifier le capteur, le traitement et l'actionneur. Pourquoi `sleep(200)` est-il raisonnable ici, et ne le serait-il pas pour un frein ABS ?

★★ Exercice 4 — *Une alarme de température qui ne panique pas*

Un local technique ne doit pas dépasser 30 degrés. Cahier des charges :

- l'alarme **se déclenche** dès que la température atteint 30 degrés : image `Image.NO` et un bip (`music.pitch(880, 100)`) à chaque tour ;
- elle ne **s'arrête** que lorsque la température est redescendue à 28 degrés ou moins (entre 28 et 30, elle garde son état précédent) : c'est une **hystérésis** ;
- le bouton A **coupe le son** de l'alarme en cours (l'image reste) ; le son reviendra à la prochaine alarme ;
- le bouton B fait défiler la température **minimale** et **maximale** relevées depuis le démarrage, sous la forme `"22/31"` ;
- hors alarme, l'écran est éteint (`display.clear()`).

1. Dresser sur le cahier un tableau donnant l'état de l'alarme (déclenchée ou non) pour la suite de températures : 26, 29, 30, 31, 29, 30, 29, 28, 27.
2. ▶ Écrire le programme complet. On utilisera les variables `en_alarme`, `son_coupe`, `mini` et `maxi`.
3. Pourquoi ne pas ranger *toutes* les températures dans une liste pour calculer le minimum et le maximum à la demande ? (Penser à la mémoire vive d'un microcontrôleur, et à une alarme qui fonctionne des mois.)

★★★ **Exercice 5** *Un podomètre*

Quand on marche, la carte (à la ceinture) est secouée à chaque pas. On mesure la **norme** de l'accélération $\sqrt{x^2+y^2+z^2}$: au repos, elle vaut environ 1000 milli-g (la pesanteur), et elle oscille au-dessus et au-dessous à chaque pas.

1. Calculer la norme pour `(0, 0, -1024)` et pour `(300, -400, 1000)`.
2. ▶ Écrire un programme qui, environ 50 fois par seconde, calcule la norme (`import math`, puis `math.sqrt`) et compte un pas **quand la norme passe au-dessus de** 1250 ; pour compter le pas suivant, il faut d'abord que la norme soit **redescendue sous** 1050. Une variable booléenne `en_haut` mémorise si l'on est déjà au-dessus. Le bouton A affiche le nombre de pas (sans bloquer la boucle), le bouton B le remet à zéro. ▷ coup de pouce
3. ▶ Tester dans le simulateur en faisant varier le curseur de l'axe `z` (ou avec le geste « secouer »).

Partie C — Tester la logique sans la carte

Tester un programme embarqué sur la carte est lent et peu reproductible (on ne marche pas deux fois de la même façon !). Une bonne pratique consiste à isoler la partie **traitement** dans des fonctions *sans capteur ni actionneur*, que l'on teste sur ordinateur avec des **capteurs simulés** : des listes de mesures. Ouvrir `tp_soc_depart.py` et le lancer : il affiche l'état des tests.

★ **Exercice 6** *Les fonctions de traitement*

▷ Avant de coder. Écrire trois appels attendus sous forme d'`assert` : un cas **normal**, un cas **limite**, un cas **piège**.

▶ Compléter, dans l'ordre, `veilleuse`, `nb_basculements`, `nb_led`, `alarme`, `mise_a_jour`, `norme`, `compter_pas_naif` et `compter_pas` (leur rôle est décrit dans le fichier), jusqu'à ce que tous les tests affichent `OK`. On y retrouve les décisions de la partie B (nombre de LED, alarme, podomètre à deux seuils), plus deux variantes : une veilleuse **tout ou rien** qui s'allume sous 40 et ne s'éteint qu'au-dessus de 80 (hystérésis, comme l'alarme), et un compteur de pas **naïf** à un seul seuil (1200).

★★ **Exercice 7** *Pourquoi deux seuils ? Les mesures parlent*

1. ▶ La fonction fournie `lumiere_simulee()` renvoie 70 niveaux de lumière relevés en fin de journée. Afficher le nombre de basculements de la veilleuse **naïve** (allumée sous 60, éteinte au-dessus) et de la veilleuse **à hystérésis** (seuils 40 et 80) : `nb_basculements(simuler_veilleuse(niveaux, False))`, puis avec `True`. Pourquoi la veilleuse naïve « clignote »-t-elle ?
2. ▶ `marche_simulee()` simule 10 secondes de marche à 2 pas par seconde. Comparer les nombres de pas trouvés par `compter_pas_naif` et par `compter_pas`. Lequel a raison ?
3. ▶ Recommencer avec une marche lente, d'un pas par seconde, obtenue par `marche_simulee(cadence=1, graine=3)`. Combien de pas devrait-on trouver ? Qu'est-ce qui, dans le signal, trompe le compteur naïf ?
4. ▶ Fabriquer une liste `repos` de 500 normes mesurées au repos, tirées au hasard entre 940 et 1060, et vérifier qu'aucun des deux compteurs n'y compte de pas. ▷ coup de pouce

Partie D — Consommation : combien de temps sur deux piles ?

On travaille avec des **ordres de grandeur** (hypothèses de l'exercice) : la carte est alimentée par deux piles AAA de capacité 1 000 mAh (elles peuvent fournir 1 000 mA pendant 1 h, ou 10 mA pendant 100 h…) ; en fonctionnement, la carte consomme environ 30 mA ; en **veille**, un microcontrôleur peut descendre vers 0,01 mA.

★★ **Exercice 8** *Faire durer les piles*

1. Combien de temps (en heures, puis en jours) la carte tourne-t-elle si elle reste toujours active ?

2. La veilleuse n'a pas besoin de regarder la lumière 5 fois par seconde. On suppose qu'elle est active 1 % du temps et en veille le reste. Calculer le courant moyen $I = 0{,}01 \times 30 + 0{,}99 \times 0{,}01$, puis l'autonomie. Quel facteur a-t-on gagné ?

3. Un smartphone a une batterie d'environ 4 000 mAh qui tient à peu près une journée d'utilisation. Quel est son courant moyen ? Combien de fois plus que la carte active ?

4. ▶ Sur l'ordinateur, mesurer le nombre de tours par seconde d'une boucle vide semblable à celle de l'exercice 1 :

```
import time
n = 0
debut = time.time()
while time.time() - debut < 1:
    n = n + 1
print(n)
```

Comparer avec la mesure de l'exercice 1. Qu'est-ce qui explique l'écart ? Qu'est-ce que l'on paie, en échange de cette vitesse ?

Partie E — Analyse : la puce du micro:bit face à un PC

La carte micro:bit (version 2) est construite autour d'une seule puce, le **nRF52833** (Nordic Semiconductor), qui contient : un cœur de processeur ARM Cortex-M4 à 64 MHz, 512 Ko de mémoire Flash, 128 Ko de mémoire vive, une radio Bluetooth, un convertisseur analogique-numérique et des minuteries. Il n'y a **aucun système d'exploitation** : au démarrage, la puce exécute directement l'interpréteur MicroPython rangé en Flash, qui exécute à son tour votre programme.

★★ **Exercice 9** *Fiche d'analyse*

1. Recopier et compléter le tableau (pour le PC, prendre un ordinateur de bureau typique : processeur à 8 cœurs à 3,5 GHz, 16 Go de mémoire vive, consommation de l'ordre de 100 W).

	nRF52833 (micro:bit)	**PC de bureau**	**Rapport PC / micro:bit**
Fréquence	64 MHz		
Nombre de cœurs			
Mémoire vive	128 Ko		
Puissance électrique	≈ 0,1 W (3 V × 30 mA)		
Système d'exploitation			—
Composants	une seule puce		—

2. Parmi les composants du nRF52833, lesquels jouent le rôle de **processeur**, de **mémoire**, d'**entrées-sorties** ? En quoi est-ce un **système sur puce** ? Est-ce plutôt un microcontrôleur ou un SoC de téléphone ? Justifier avec deux critères du cours.

3. Le programme est en Flash, les variables en mémoire vive, et le processeur lit instruction et donnée par deux bus distincts. Quel nom porte cette architecture ? Quel avantage apporte-t-elle ?

4. Pourquoi un système d'exploitation n'est-il ni nécessaire, ni même souhaitable pour l'alarme de la partie B ? Quel est l'inconvénient de MicroPython (un **interpréteur**) par rapport à un programme compilé en langage machine ?

5. Si la radio Bluetooth de la puce tombe en panne, que faut-il changer ? Et sur un PC, si la carte Wi-Fi tombe en panne ? Relier à un intérêt et une limite de l'intégration.

Bilan du TP

À rédiger (une dizaine de lignes)

1. Décrire, sur l'exemple du podomètre, les trois caractéristiques d'un système embarqué : acquisition, contrôle, contraintes de temps.
2. Expliquer à quoi sert une hystérésis (deux seuils), en citant vos mesures de la partie C.
3. Pourquoi une puce bien moins puissante qu'un PC est-elle le bon choix pour une veilleuse ou un podomètre ? Donner deux arguments chiffrés.

Coups de pouce

*après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 5 q. 2 – piste La boucle contient deux tests sur la norme : `if not en_haut and a > 1250:` (on vient de franchir le seuil haut : on compte un pas et on passe `en_haut` à `True`), et `elif en_haut and a < 1050:` (on est redescendu : `en_haut` repasse à `False`). ↑énoncé

Ex. 7 q. 4 – piste Le module `random` est déjà importé dans le fichier ; une liste en compréhension suffit : `[random.randint(940, 1060) for i in range(500)]`. ↑énoncé

FICHE DE RÉVISION

L'essentiel

- **Système sur puce (SoC, *System on Chip*)** : un **ordinateur complet sur une seule puce** — processeur (souvent multi-cœur) et ses **mémoires locales** (caches), GPU, **contrôleur vidéo**, **gestion d'énergie**, interfaces **radio** (Wi-Fi, GPS, modem…) et **filaires** (USB…), reliés par un **réseau sur puce**. Dans un téléphone, la **mémoire vive** est une autre puce, empilée dans le **même boîtier**. C'est le cœur des smartphones et tablettes.
- À l'opposé, une **carte mère** classique porte des **composants séparés**, que l'on peut **choisir, remplacer, faire évoluer**.
- Un **microcontrôleur** est un *mini*-système sur puce : **peu puissant** (MHz), **peu de mémoire** (Ko), très **sobre** et **bon marché**, en général **sans système d'exploitation** complet. Il pilote **une** tâche précise dans un **système embarqué** (carte bancaire, thermostat, drone, régulateur de vitesse…).
- **Système embarqué** : il **lit** le monde par des **capteurs** (entrée) et **agit** par des **actionneurs** (sortie), avec de fortes **contraintes de temps réel** (une réponse *à temps*). Boucle typique : *lire les capteurs* → *décider* → *commander les actionneurs*.
- **Loi de Moore** : le nombre de transistors d'une puce **double environ tous les deux ans** (observation empirique, pas une loi physique) — du **2 300** de l'Intel 4004 (1971) aux dizaines de **milliards** aujourd'hui.

Le prix de l'intégration

Intérêts	Limites
faible **consommation** (courtes distances, peu de chaleur, souvent sans ventilateur)	**non réparable** : un composant en panne = toute la puce perdue
petite **taille**, faible **coût**, grande **vitesse**	**non évolutif** ; **obsolescence** ; **déchets électroniques** (droit à la réparation)

À retenir aussi

- **von Neumann** : une *seule* mémoire (programme + données). **Harvard** (fréquent sur microcontrôleurs) [au-delà du programme] : *deux* mémoires séparées et *deux* bus ⇒ on lit une instruction **et** une donnée en même temps.
- **IoT** (objets connectés) : des milliards d'objets à puce ; chacun est une **porte d'entrée** (sécurité) et un **capteur** (vie privée).
- **Logiciel et matériel** : en 1945, on recâble l'ENIAC pour chaque calcul (tout est matériel) ; en 1948, le **programme enregistré** donne la main au logiciel ; aujourd'hui, les SoC regroupent des **blocs spécialisés** (GPU, IA, AES), pilotés par des API.

Erreurs à éviter

- confondre **microcontrôleur** (minuscule, sobre, sans OS, une tâche) et **SoC** (ordinateur complet) ;
- croire un SoC **réparable** comme un PC (rien n'est remplaçable) ;
- confondre **capteur** (lit, entrée) et **actionneur** (agit, sortie) ;
- mélanger **von Neumann** (une mémoire) et **Harvard** (deux mémoires, deux bus).

Savoir-faire à maîtriser — je sais…

- ☐ **définir** un système sur puce et **identifier** ses constituants sur un schéma ;
- ☐ **distinguer** microcontrôleur et SoC, carte mère et puce ;
- ☐ **dérouler** le fonctionnement d'un système embarqué (capteur → décision → actionneur) ;
- ☐ **énoncer** la loi de Moore et **peser** les intérêts et les limites de l'intégration ;
- ☐ **situer** dans le temps l'évolution des rôles du logiciel et du matériel.

Se tester — *répondre sans regarder la fiche, puis vérifier.* ► réponses p. 1022

1. Qu'est-ce qu'un système sur puce ?
2. Dans un thermostat, quel est le capteur et quel est l'actionneur ?
3. Une puce de 1 milliard de transistors en 2020 : combien la loi de Moore en prévoit-elle en 2026 ?
4. La mémoire d'un smartphone devient insuffisante : peut-on en ajouter ? Pourquoi ?
5. Pour piloter un airbag, microcontrôleur ou SoC avec système d'exploitation ? Pourquoi ?

Et avant ? — *questions sur les chapitres précédents.* ► réponses p. 1022

1. Que dit le résultat du problème de l'arrêt ? (ch. 13)
2. Écrire la relation de récurrence du rendu de monnaie : $R(s)$, nombre minimal de pièces pour rendre s. (ch. 11)
3. Écrire le dictionnaire d'adjacence du graphe non orienté d'arêtes `A-B`, `A-C`, `B-C`. (ch. 8)

Vers le bac

- Problème de synthèse n° 10 — Détecter un plagiat : la plus longue sous-chaîne commune, p. 712
- Problème de synthèse n° 11 — Une machine minuscule, p. 713

DÉFI ADVENT OF CODE

DÉFI Docking Data

l'amarrage — masques et opérations bit à bit

Jour 14

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/t14-defi-aoc-2020-14` — **archive .zip**

Consignes

- L'énoncé se lit sur le site (voir le QR code) : `https://adventofcode.com/2020/day/14`. On peut le lire en anglais ou le faire traduire par le navigateur (clic droit, *Traduire en français*).
- Avec un compte (GitHub, Google…), le bouton *get your puzzle input* donne **votre** fichier de données, à enregistrer sous `input.txt` à côté de votre programme. Sans compte : le dossier du défi (lien ci-dessus) contient des **données maison** (`input.txt`) et le programme `verifier.py`, qui dit si vos réponses sont justes.
- **Attention aux exemples.** L'énoncé en ligne a son propre exemple, avec d'autres réponses. Les valeurs de test de cette fiche portent sur **l'exemple inventé de la fiche** (exercice « À la main ») : l'enregistrer dans un fichier `exemple.txt` à côté du programme, et ouvrir `"exemple.txt"` au lieu de `"input.txt"` pour tester.
- La partie 2 n'apparaît qu'une fois la partie 1 validée sur le site.
- L'IA peut aider à comprendre un message d'erreur, jamais à écrire la solution.

Le défi en bref

Le problème. L'ordinateur d'un ferry exécute un petit programme de près de cinq cents lignes qui écrit des valeurs en mémoire, en les retouchant au passage avec un masque de 36 bits. Il faut savoir ce que contient la mémoire à la fin. Les calculs en binaire sur 36 bits sont trop longs à la main, et en partie 2 une seule ligne peut écrire dans des centaines d'adresses.
Ce qu'on réinvestit. Le défi met en scène l'ordinateur embarqué d'un ferry, un système comme ceux de ce chapitre : il initialise sa mémoire adresse par adresse. On y manipule l'**écriture binaire** des entiers et des opérations bit à bit ; le masque, appliqué bit par bit, rappelle le masque de sous-réseau vu au chapitre 9. Un **dictionnaire** représente une mémoire immense dont seules quelques cases sont utilisées, et l'approfondissement 1 réinvestit la récursivité du chapitre 1.
La démarche de la fiche.

1. Vérifier sa compréhension de la règle, puis appliquer un masque à la main sur un petit

programme.
2. Analyser une ligne du fichier, puis appliquer le masque à une valeur avec deux opérations bit à bit.
3. Simuler la mémoire avec un dictionnaire : réponse de la partie 1.
4. Pour la partie 2, énumérer toutes les adresses obtenues en remplaçant les bits flottants, puis le faire par récursivité en approfondissement.

1 Lire l'énoncé

Vocabulaire utile pour lire le texte original :

`bitmask`	masque de bits	`overwrite`	écraser, remplacer
`unchanged`	inchangé	`memory address`	adresse mémoire
`36-bit unsigned`	entier positif sur 36 bits	`most significant bit`	bit de poids fort
`initialization`	initialisation	`sum`	somme
`floating`	flottant (bit qui prend les deux valeurs)	`decoder`	décodeur

Résumé de l'énoncé (partie 1)

Contexte. À l'approche du port, l'ordinateur du ferry doit initialiser sa mémoire, mais il ne comprend pas le système de masques de bits utilisé par le port. Vous allez émuler ce système, c'est-à-dire le reproduire par un programme.
Les données. Le fichier est un petit programme ; chaque ligne est de l'une des deux formes suivantes.

- `mask =` suivi de 36 caractères parmi `0`, `1`, `X` : elle change le masque courant. Le masque s'écrit bit de poids fort à gauche (2^{35}) et reste en vigueur jusqu'au `mask` suivant.
- `mem[a] = v` : elle écrit la valeur v à l'adresse a. Adresses et valeurs sont des entiers positifs sur 36 bits, écrits en décimal.

La règle.

- Juste avant chaque écriture, on applique le masque à la **valeur** : un `0` ou un `1` du masque impose ce bit, un `X` laisse le bit de v tel quel. Autrement dit, le masque ne retouche que certains bits.
- Toutes les cases de la mémoire valent 0 au départ.
- Écrire à nouveau à la même adresse remplace l'ancienne valeur.

La réponse attendue. La somme de toutes les valeurs présentes en mémoire à la fin du programme. Par exemple, si le masque se termine par `10XX` (et ne contient que des `X` avant), écrire 7 (`0111`) donne `1011`, soit 11, et écrire 1 (`0001`) donne `1001`, soit 9 ; à deux adresses différentes, la réponse serait $11 + 9 = 20$.

Exercice 1 *Comprendre ce qui est demandé*

But. Vérifier sur trois questions courtes qu'on a compris le rôle du masque et de la mémoire.
À quoi ça sert ? La dernière question prépare un choix essentiel du programme : la mémoire doit être un dictionnaire, pas une liste.

Questions de vérification, à traiter sur le cahier :

1. Si le masque ne contient que des `X`, quelle valeur est écrite ?

2. Deux lignes `mem` écrivent à la même adresse : que contient la case à la fin ?

3. Pourquoi ne peut-on pas représenter la mémoire par une liste de 2^{36} cases ?

Exercice 2 *À la main, sur un petit exemple*

But. Appliquer à la main le masque à trois valeurs écrites en binaire.
À quoi ça sert ? On obtient un exemple dont on **connaît la réponse** (40) et des valeurs intermédiaires pour tester `appliquer` ; on voit aussi l'effet d'une écriture qui en écrase une autre.

On considère le programme (inventé) suivant :

```
mask = XXXXXXXXXXXXXXXXXXXXXXXXXXXXX1X0X0
mem[3] = 6
mem[10] = 21
mem[3] = 13
```

Écrire 6, 21 et 13 en binaire sur 6 bits, appliquer le masque et donner la valeur écrite à chaque fois. Vérification : la réponse est `40`.

2 Programmer la partie 1

Exercice 3 *Lire une ligne*

But. Transformer une ligne du fichier en une donnée facile à utiliser : le nouveau masque, ou le couple adresse–valeur.
À quoi ça sert ? Le reste du programme n'a plus à se soucier du texte : il distingue les deux sortes de lignes par le premier élément renvoyé. Cette fonction servira telle quelle en partie 2.

Outil Python : `strip` (nettoyer une ligne lue)

Chaque ligne lue dans un fichier (avec `for ligne in fichier` ou `readlines()`) se termine par le caractère de fin de ligne `"\n"` (sauf parfois la dernière). `ligne.strip()` renvoie une copie de la ligne sans les espaces ni les fins de ligne placés au début et à la fin.

```
ligne = "1721\n"
print(ligne.strip())                  # 1721
print(len(ligne), len(ligne.strip())) # 5 4
```

Intérêt. Comparer ou découper une ligne sans que la fin de ligne ne gêne : `"abc\n" == "abc"` vaut `False`.

Écrire une fonction `analyser(ligne)` qui renvoie `("mask", chaine)` ou `("mem", adresse, valeur)` selon la ligne. ▷ coup de pouce

Outil Python : opérations bit à bit et conversions binaires (`&`, `|`, `int(s, 2)`, `bin`)

Les opérateurs `&` (et) et `|` (ou) travaillent sur l'écriture binaire des entiers, bit par bit, tous les bits en même temps. `int(chaine, 2)` lit une chaîne binaire ; `bin(n)` écrit `n` en binaire avec le préfixe `0b`. Une tranche retire le préfixe, et une boucle complète par des zéros à gauche pour obtenir 36 chiffres.

```
a = int("1100", 2)          # 12 : chaine binaire -> entier
b = int("1010", 2)          # 10
print(a & b)                # 8  (1000) : 1 la ou les deux bits valent 1
print(a | b)                # 14 (1110) : 1 la ou au moins un des bits vaut 1
print(bin(a & b))           # 0b1000 : entier -> chaine binaire, prefixe 0b
print(bin(5)[2:])           # 101 : la tranche retire le prefixe
```

```python
def en_binaire(n, nb_bits):
    """Renvoie l'ecriture binaire de n sur nb_bits chiffres (0 a gauche)."""
    chaine = bin(n)[2:]
    while len(chaine) < nb_bits:
        chaine = "0" + chaine
    return chaine


print(en_binaire(5, 36))       # 36 caracteres, completes par des 0 a gauche
```

Intérêt. `x & m` met à 0 tous les bits de `x` là où `m` a un 0 ; `x | m` met à 1 tous les bits de `x` là où `m` a un 1. Une seule opération, très rapide, remplace une boucle sur les 36 bits.
À essayer. Calculer à la main `13 & 6` et `13 | 6` (écrire 13 et 6 sur 4 bits), puis vérifier avec Python.

Méthode : parcourir une chaîne caractère par caractère

Pour remplacer, compter ou repérer un caractère dans une chaîne, on la parcourt avec une boucle. Une chaîne n'est pas modifiable : on en construit une **nouvelle** par concaténations successives.

```python
def remplacer(s, a, b):
    """Renvoie une copie de s ou chaque caractere a est remplace par b."""
    resultat = ""
    for c in s:
        if c == a:
            resultat += b
        else:
            resultat += c
    return resultat


def premier_indice(s, a):
    """Renvoie l'indice de la premiere occurrence de a dans s, ou -1."""
    i = 0
    while i < len(s) and s[i] != a:
        i += 1
    if i == len(s):
        return -1
    return i


m = "X1X0"
print(remplacer(m, "X", "0"))      # 0100 : une NOUVELLE chaine
print(premier_indice(m, "X"))      # 0 : indice de la premiere occurrence
print(premier_indice(m, "Z"))      # -1 : absent
```

Intérêt. Pour compter les `X`, on reprend la même boucle avec un compteur. Le coût de ces fonctions est proportionnel à la longueur de la chaîne ; `premier_indice` s'arrête dès qu'il a trouvé.
À essayer. Après `remplacer(m, "X", "0")`, que vaut `m` ? Pourquoi ?

Exercice 4 *Appliquer le masque avec deux opérations bit à bit*

But. Calculer la valeur réellement écrite, avec deux opérations bit à bit au lieu d'une boucle sur les 36 bits.
À quoi ça sert ? C'est le cœur de la partie 1 : chaque écriture du programme passe par cette fonction.

Écrire une fonction `appliquer(masque, valeur)`. On construira deux entiers à partir du masque : l'un pour forcer des bits à `1`, l'autre pour forcer des bits à `0`. Vérifier : avec `m` le masque de l'exemple inventé de la fiche, `appliquer(m, 6)` vaut `18`. ▷ coup de pouce ▷ début de solution

Exercice 5 *La mémoire*

But. Exécuter tout le programme en stockant la mémoire dans un dictionnaire.
À quoi ça sert ? On obtient la réponse de la partie 1. Le dictionnaire ne contient que les adresses réellement écrites : quelques centaines au lieu de 2^{36} cases.

La mémoire a 2^{36} cases : une liste est impossible. Utiliser un dictionnaire `memoire` (adresse → valeur), exécuter le programme et afficher la somme des valeurs. Tester sur l'exemple inventé de la fiche (réponse `40`), puis sur vos données.

3 Partie 2

Une fois la partie 1 validée, lire la suite de l'énoncé sur le site.

Résumé de la partie 2 (à lire après avoir validé la partie 1)

Ce qui change. Le programme ne fonctionne toujours pas : la version du système utilisée par le port est en fait un « décodeur d'adresses ». Le masque agit maintenant sur l'**adresse**, et la valeur est écrite telle quelle.
La nouvelle règle.

- Un `0` du masque laisse le bit de l'adresse inchangé (attention : ce n'est plus comme en partie 1).
- Un `1` force le bit de l'adresse à 1.
- Un `X` est *flottant* : il prend les deux valeurs 0 et 1. Avec k bits flottants, la valeur est donc écrite dans 2^k adresses.

La réponse attendue. La même qu'en partie 1 : la somme des valeurs en mémoire à la fin. Par exemple, si le masque se termine par `0X1` (que des `0` avant), écrire 7 à l'adresse 4 (`100`) donne le modèle `1X1` : la valeur 7 est écrite aux adresses 5 et 7, ce qui apporte 14 à la somme.

★★ Exercice 6 *Un masque sur les adresses*

But. Calculer la liste de toutes les adresses désignées par une adresse et un masque contenant des bits flottants.
À quoi ça sert ? C'est la seule vraie nouveauté de la partie 2 : avec cette fonction, il suffit d'écrire la valeur à chacune de ces adresses dans le dictionnaire pour obtenir la réponse.

Écrire une fonction `adresses(masque, adresse)` qui renvoie la liste de toutes ces adresses.
Pour tester, on utilise un autre programme inventé :

```
mask = 000000000000000000000000000000X10X0X
mem[5] = 4
mask = 000000000000000000000000000000X0000
mem[1] = 10
```

Vérification : `mem[5] = 4` écrit dans 8 adresses, de `16` à `53` ; la réponse finale est `48` (une adresse est écrite deux fois). ▷ coup de pouce ▷ début de solution

★★★ Exercice 7 *Défi : combien de `X` au maximum ?*

But. Mesurer, sur vos données, combien d'adresses la partie 2 écrit réellement.
À quoi ça sert ? On vérifie par l'expérience que le nombre de bits flottants reste petit, ce qui explique pourquoi la méthode termine rapidement alors que la mémoire est immense.

Afficher le plus grand nombre de `X` parmi les masques de vos données, et le nombre total d'adresses écrites. Pourquoi le dictionnaire reste-t-il de taille raisonnable alors que la mémoire a 2^{36} cases ?

4 Approfondissement

★★ **Exercice 8** *Approfondissement 1 : énumérer les adresses par récursivité*

But. Énumérer les 2^k adresses par une fonction récursive qui remplace un `X` à la fois.
À quoi ça sert ? La récursivité donne une solution courte et naturelle à ce problème d'énumération ; compter les appels relie cette solution à sa complexité.

1. À la main, pour le modèle `X1X` : remplacer le premier `X` par `0`, puis par `1`, et recommencer sur chacune des deux chaînes obtenues. Dessiner l'arbre de ces remplacements et lister les chaînes finales dans l'ordre.

2. Écrire une fonction récursive `remplacer_x(modele)` qui renvoie la liste de toutes les chaînes obtenues en remplaçant les `X` par des `0` et des `1`. Quel est le cas de base ? ▷ coup de pouce ▷ début de solution

3. Utiliser `remplacer_x` pour réécrire `adresses` ; vérifier les 8 adresses de l'exemple de la partie 2 et la réponse `48`.

4. Combien d'appels récursifs pour un modèle contenant k fois `X` ? Comparer avec la version qui énumère les entiers de 0 à $2^k - 1$.

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 3 – piste `ligne.split(" = ")` sépare la partie gauche de la valeur. Pour l'adresse, on peut découper `"mem[3]"` avec une tranche qui enlève les 4 premiers caractères et le dernier. ↑

Ex. 4 – piste En Python, `a | b` (ou bit à bit) et `a & b` (et bit à bit) agissent sur chaque bit ; `int("101", 2)` convertit une chaîne binaire en entier. ↑énoncé

Ex. 4 – début de solution Remplacer (par une boucle sur les caractères) les `X` du masque par des `0` donne un entier `ou_masque` à combiner avec `|` ; les remplacer par des `1` donne un entier `et_masque` à combiner avec `&`. ↑énoncé

Ex. 6 – piste Travailler sur des chaînes de 36 caractères : `en_binaire(adresse, 36)` (encadré de la partie 1) donne l'écriture binaire complétée par des zéros. On construit d'abord la chaîne résultat avec ses `X`. ↑énoncé

Ex. 6 – début de solution Pour remplacer les k bits `X` : énumérer les entiers de 0 à $2^k - 1$ et utiliser leurs k bits ; ou bien écrire une fonction récursive qui remplace le premier `X` par `0`, puis par `1`. ↑énoncé

Ex. 8 q. 2 – piste Cas de base : le modèle ne contient aucun `X` (`premier_indice` renvoie -1) ; la liste ne contient alors que le modèle lui-même. ↑énoncé

Ex. 8 q. 2 – début de solution Sinon, avec `i = premier_indice(modele, "X")`, construire par tranches `modele[:i] + "0" + modele[i + 1:]` et la même chaîne avec `"1"`, puis concaténer les deux listes renvoyées par les appels récursifs. ↑énoncé

Problèmes de synthèse et exercices type bac

Mode d'emploi

- Étoiles, • essentiel, coup de pouce, ▶ et pastilles IA : voir le mode d'emploi du chapitre 1 p. 32.
- Chaque problème **croise** des notions de plusieurs chapitres, comme les exercices du bac, qui ne disent jamais « ceci est un exercice sur les piles ».
- Sous chaque titre, la ligne Chapitres croisés indique ce qu'il faut avoir vu. Un problème peut être repris plus tard dans l'année, pour réviser.
- ▶ *À programmer et tester* en Python. Les classes `Pile`, `File`, `Cellule` et `Noeud` sont celles du cours.
- Dans cette feuille :  pour les exercices qui entraînent des méthodes du cours,  pour les problèmes plus ouverts.

Après les chapitres 1 à 3 (récursivité, programmation objet, structures linéaires)

★★ **Exercice 1**  *Dérécursiver avec une pile* ▶ corrigé p. 1023 ≈ 30 min Chapitres croisés : *Récursivité ; Piles*

On considère la fonction récursive suivante, où `n` est un entier positif ou nul.

```
1 def afficher_binaire(n):
2     if n >= 2:
3         afficher_binaire(n // 2)
4     print(n % 2, end="")
```

1. Dérouler l'appel `afficher_binaire(13)` : donner la liste des appels successifs, puis ce qui est affiché. Que représente l'affichage obtenu ?

2. Dessiner la **pile d'appels** au moment où l'appel le plus profond s'exécute (sommet en haut). Expliquer pourquoi le chiffre affiché en premier est celui du dernier appel lancé.

3. ▶ Écrire une fonction **non récursive** `binaire(n)` qui renvoie l'écriture binaire de `n` sous forme de chaîne (par exemple `binaire(13)` renvoie `"1101"` et `binaire(0)` renvoie `"0"`). Elle utilisera une `Pile` pour jouer le rôle de la pile d'appels. ▷ coup de pouce

4. On remplace la `Pile` par une `File` (`enfiler` et `defiler` à la place de `empiler` et `depiler`). Que renvoie alors l'appel pour `n = 13` ? Expliquer.

★★ **Exercice 2** IA *Annuler et rétablir* ▶ corrigé p. 1024 ≈ 45 min Chapitres croisés : *Programmation objet ; Piles*

Un éditeur de texte très simple garde le texte saisi dans un attribut `texte` (une chaîne). Il propose deux boutons : **Annuler**, qui revient au texte d'avant la dernière modification, et **Rétablir**, qui refait une modification qu'on vient d'annuler. Pour cela, l'objet possède deux piles : `annulations` contient les textes précédents, `retablissements` les textes annulés. Comme dans tous les éditeurs, une nouvelle saisie **efface l'historique des rétablissements** : après `ecrire`, on ne peut plus rétablir ce qui avait été annulé.

```
class Editeur:
    def __init__(self):
        ...

    def ecrire(self, mot):
        """ajoute mot a la fin du texte (et vide la pile des retablissements)"""
        ...

    def annuler(self):
        """revient au texte precedent (sans effet s'il n'y en a pas)"""
        ...

    def retablir(self):
        """refait la derniere modification annulee (sans effet sinon)"""
        ...
```

1. ▶ Écrire la méthode `__init__` : le texte est vide au départ et les deux piles aussi.
2. On exécute les instructions ci-dessous. Donner, après chacune, la valeur de `e.texte` et le contenu des deux piles (dessinées verticalement, sommet en haut).

```
e = Editeur()
e.ecrire("Bon")
e.ecrire("jour")
e.annuler()
e.retablir()
e.annuler()
e.ecrire("soir")
```

3. ▶ Écrire les méthodes `ecrire`, `annuler` et `retablir`, en accord avec le déroulé de la question 2. ▷ coup de pouce
4. Après la dernière instruction de la question 2, un appel à `e.retablir()` ne doit rien changer. Pourquoi ce choix est-il raisonnable, et quelle ligne de votre méthode `ecrire` le met-elle en œuvre ?

★★★ **Exercice 3** IA *Listes chaînées récursives* ▶ corrigé p. 1024 ≈ 45 min Chapitres croisés : *Programmation objet ; Listes chaînées ; Récursivité*

On rappelle la classe `Cellule` du cours : une liste chaînée est soit `None` (liste vide), soit une cellule dont l'attribut `suivante` est elle-même une liste chaînée. C'est une **définition récursive**, qui appelle naturellement des fonctions récursives.

```
class Cellule:
    def __init__(self, v, s):
        self.valeur = v
        self.suivante = s

lst = Cellule(3, Cellule(1, Cellule(4, None)))
```

1. Représenter `lst` par un schéma de cellules reliées par des flèches.
2. ▶ Écrire les fonctions récursives `longueur(lst)` et `somme(lst)`, qui renvoient le nombre d'éléments et leur somme (0 pour une liste vide).
3. ▶ Écrire la fonction récursive `contient(lst, x)`, qui renvoie `True` si `x` est dans la liste et `False` sinon. Pourquoi peut-elle s'arrêter avant la fin de la liste ?
4. ▶ On veut une fonction `renverser(lst)` qui renvoie une **nouvelle** liste chaînée contenant les mêmes valeurs dans l'ordre inverse, sans modifier `lst`. Écrire une fonction récursive `renverser_dans(lst, acc)`, où `acc` est la liste déjà renversée, puis en déduire `renverser`.

▷ coup de pouce

5. Ajouter une cellule **en tête** coûte une seule opération. Combien d'opérations faut-il pour ajouter une cellule **en queue** d'une liste de n cellules ? Comparer avec les opérations d'une pile.

Après le chapitre 5 (arbres)

★★★ **Exercice 4** IA IA IA *Trier avec un arbre binaire de recherche* ▶ corrigé p. 1025 ≈ 45 min Chapitres croisés : *Arbres ; Récursivité ; coût des algorithmes*

On dispose de la fonction `inserer` du cours, qui insère `x` dans l'ABR `a` et renvoie l'arbre obtenu (une valeur égale à celle d'un nœud va à droite).

```
def inserer(a, x):
    """Insere x dans l'ABR a et renvoie l'arbre obtenu."""
    if a is None:
        return Noeud(x)
    if x < a.valeur:
        a.gauche = inserer(a.gauche, x)
    else:
        a.droite = inserer(a.droite, x)
    return a
```

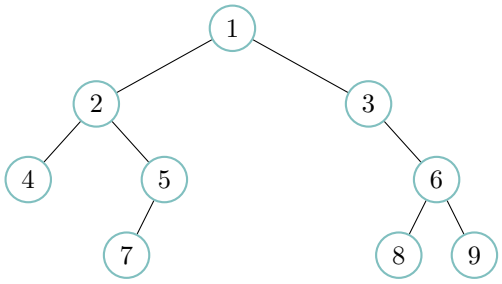
1. Dessiner l'ABR obtenu en insérant successivement $5, 3, 8, 1, 4$ dans un arbre vide.
2. ▶ Écrire une fonction récursive `infixe_dans(a, resultat)` qui ajoute à la fin de la liste Python `resultat` les valeurs de `a` dans l'ordre du parcours **infixe**. Que contient `resultat` pour l'arbre de la question 1 ?
3. ▶ En déduire une fonction `tri_abr(tab)` qui renvoie une liste contenant les valeurs de `tab` triées dans l'ordre croissant.
4. Que renvoie `tri_abr([3, 1, 3])` ? Où le second `3` est-il placé dans l'arbre ? Quelle modification de `inserer` donnerait au contraire un tri **sans doublon** (chaque valeur une seule fois) ?
5. On trie le tableau `[1, 2, 3, ..., 100]`, déjà trié. Quelle est la hauteur de l'arbre construit ? Combien de comparaisons coûte l'insertion du dernier élément ? En déduire l'ordre de grandeur du coût du tri dans ce cas, et le comparer au cas d'un tableau mélangé, pour lequel la hauteur reste de l'ordre de $\log_2(n)$ (dans un essai avec 100 valeurs mélangées, l'arbre avait une hauteur de 12).

▷ coup de pouce

6. Avec un tableau trié de 10 000 valeurs, Python déclenche une `RecursionError`. Expliquer pourquoi, en lien avec la question précédente.

★★ **Exercice 5** *La largeur d'un arbre* ▶ corrigé p. 1026 ≈ 30 min Chapitres croisés : *Arbres ; Files ; dictionnaires*

On appelle **niveau** d'un nœud le nombre de nœuds sur le chemin qui va de la racine jusqu'à lui (la racine est au niveau 1 ; attention, ce n'est pas la convention du cours, où l'on parle de **profondeur** et où la racine est à la profondeur 0 : suivez l'énoncé). La **largeur** d'un arbre est le plus grand nombre de nœuds situés sur un même niveau.



1. Pour l'arbre ci-dessus, donner le nombre de nœuds de chaque niveau, puis sa largeur.
2. ▶ Écrire une fonction `noeuds_par_niveau(a)` qui renvoie un dictionnaire associant à chaque niveau son nombre de nœuds (pour l'arbre ci-dessus : `{1: 1, 2: 2, 3: 3, 4: 3}`). Elle s'inspirera du parcours en largeur du cours, en enfilant des couples `(noeud, niveau)`.

▷ coup de pouce

3. ▶ En déduire une fonction `largeur_max(a)`.
4. On remplace la `File` par une `Pile`. Le dictionnaire obtenu est-il différent ? L'ordre dans lequel les nœuds sont visités l'est-il ? Justifier.

Après les chapitres 6 à 8 (bases de données, diviser pour régner, graphes)

★★ **Exercice 6** *Une médiathèque, de la table à l'objet* ▶ corrigé p. 1027 ≈ 45 min Chapitres croisés : *Bases de données ; Programmation objet*

Une médiathèque gère ses prêts avec trois relations : `adherent(id_adherent, nom, prenom)`, `livre(id_livre, titre, auteur)` et `emprunt(id_emprunt, #id_adherent, #id_livre, jour, rendu)`. L'attribut `jour` est le numéro, dans l'année, du jour de l'emprunt ; `rendu` vaut `1` si le livre a été rendu et `0` sinon.

id_adherent	nom	prenom
1	Durand	Lina
2	Rossi	Marco
3	Durand	Paul

id_livre	titre	auteur
10	Le Petit Prince	Saint-Exupéry
11	Vingt mille lieues sous les mers	Verne
12	Les Misérables	Hugo
13	Le Tour du monde en 80 jours	Verne

id_emprunt	id_adherent	id_livre	jour	rendu
1	1	10	3	1
2	2	11	12	0
3	1	12	20	0
4	3	10	25	0
5	2	13	40	0

1. Pourquoi `id_livre` ne peut-il pas servir de clé primaire à la relation `emprunt` ? Citer les clés étrangères de cette relation et la relation à laquelle chacune fait référence.
2. Écrire une requête qui donne les titres des livres empruntés par Lina Durand. Pourquoi ne suffit-il pas de tester `nom = 'Durand'` ?

3. Donner le résultat de la requête suivante :

```sql
SELECT COUNT(*)
FROM emprunt
WHERE rendu = 0;
```

4. Écrire une requête qui donne, pour chaque auteur, le nombre d'emprunts de ses livres.

5. Marco Rossi rapporte *Vingt mille lieues sous les mers*. Écrire la requête qui met à jour la base.

6. Après cette mise à jour, un programme Python interroge la base et obtient la liste des emprunts non rendus, sous forme de triplets `(id_emprunt, titre, jour)` :

```python
lignes = [(3, "Les Misérables", 20), (4, "Le Petit Prince", 25),
          (5, "Le Tour du monde en 80 jours", 40)]
```

▶ Écrire une classe `Emprunt` dont le constructeur reçoit `numero`, `titre` et `jour`, avec une méthode `en_retard(self, aujourdhui)` qui renvoie `True` si, le jour numéro `aujourdhui`, **strictement plus de 21 jours** se sont écoulés depuis le jour de l'emprunt.

7. ▶ Écrire une fonction `titres_en_retard(lignes, aujourdhui)` qui construit la liste des objets `Emprunt` puis renvoie la liste des titres en retard. Que renvoie-t-elle pour `aujourdhui = 47` ?

▷ coup de pouce

★★★ Exercice 7 — *Ordonnancer des tâches qui dépendent les unes des autres*

▶ corrigé p. 1028 ≈ 60 min Chapitres croisés : *Graphes ; Files ; Processus*

Pour construire un logiciel, un système doit exécuter des tâches dont certaines ne peuvent commencer qu'une fois d'autres terminées. On représente ces dépendances par un dictionnaire qui associe à chaque tâche la liste de ses **prérequis** :

```python
taches = {"A": [], "B": ["A"], "C": ["A"], "D": ["B", "C"],
          "E": ["C"], "F": ["D", "E"]}
```

1. Dessiner le **graphe orienté** des dépendances : un arc va de `X` vers `Y` quand `X` est un prérequis de `Y`.

2. ▶ Écrire une fonction `successeurs(prerequis)` qui renvoie le dictionnaire associant à chaque tâche la liste des tâches qui en dépendent (pour `taches` : `"A"` est associée à `["B", "C"]`, `"F"` à `[]`).

3. Comme l'ordonnanceur d'un système d'exploitation, on range dans une **file** les tâches *prêtes*, c'est-à-dire celles dont tous les prérequis sont faits. Au départ, on enfile les tâches sans prérequis. Tant que la file n'est pas vide, on défile une tâche, on l'exécute, puis on retire 1 au nombre de prérequis restants de chacun de ses successeurs ; un successeur dont ce nombre tombe à 0 est enfilé. Dérouler cet algorithme sur `taches` en donnant le contenu de la file après chaque tâche exécutée, puis l'ordre d'exécution.

4. ▶ Écrire une fonction `ordre_execution(prerequis)` qui renvoie la liste des tâches dans l'ordre d'exécution, ou `None` si certaines tâches ne peuvent jamais être exécutées. On utilisera la classe `File` du cours et un dictionnaire `attente` (tâche → nombre de prérequis non encore faits).

▷ coup de pouce

5. On ajoute une dépendance : `A` doit maintenant attendre `F`.

```python
taches["A"] = ["F"]
```

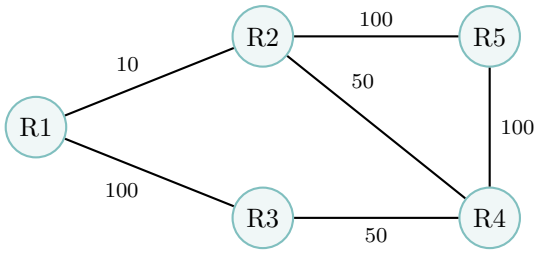
Que renvoie `ordre_execution(taches)` ? Que contient le graphe ? Quelle situation du chapitre *Processus* reconnaît-on ?

6. Combien de fois, au plus, chaque tâche est-elle enfilée ? chaque dépendance examinée ? En déduire le coût de l'algorithme en fonction du nombre de tâches et de dépendances.

Après les chapitres 9 et 10 (réseaux, cryptographie)

★★ **Exercice 8** *Router des paquets avec un dictionnaire* ▶ corrigé p. 1029 ≈ 60 min Chapitres croisés : *Réseaux ; Graphes ; Files*

Un réseau de cinq routeurs est représenté par un dictionnaire : à chaque routeur est associé le dictionnaire de ses voisins, avec le **débit** de la liaison en Mbit/s.




```
reseau = {"R1": {"R2": 10, "R3": 100},
          "R2": {"R1": 10, "R4": 50, "R5": 100},
          "R3": {"R1": 100, "R4": 50},
          "R4": {"R2": 50, "R3": 50, "R5": 100},
          "R5": {"R2": 100, "R4": 100}}
```

1. En OSPF, le coût d'une liaison vaut 10^8/débit, le débit étant en bit/s. ▶ Écrire une fonction `cout(debit)` qui prend un débit en Mbit/s et renvoie ce coût. Donner les coûts des liaisons à 10, 50 et 100 Mbit/s.

2. Le protocole RIP compte les **sauts**. ▶ Écrire une fonction `sauts(reseau, depart)` qui renvoie le dictionnaire associant à chaque routeur son nombre de sauts depuis `depart`, par un **parcours en largeur** avec une `File`. Quelle route RIP suit un paquet de R1 vers R5 ?

▷ coup de pouce

3. ▶ Écrire une fonction `est_chemin(reseau, chemin)`, où `chemin` est une liste de routeurs, qui vérifie que deux routeurs consécutifs sont toujours reliés, puis une fonction `cout_chemin(reseau, chemin)` qui renvoie la somme des coûts OSPF de ses liaisons.

4. ▶ Écrire une fonction `meilleur_chemin(reseau, chemins)` qui renvoie, parmi une liste de chemins candidats, un chemin valide de coût minimal. L'appliquer aux candidats `R1-R2-R5`, `R1-R3-R4-R5`, `R1-R2-R4-R5`, `R1-R3-R4-R2-R5` et `R1-R4-R5`. Quelle route OSPF suit le paquet ? Comparer avec RIP et expliquer.

5. La liaison R3–R4 tombe en panne. Écrire les deux instructions qui la retirent du dictionnaire, puis donner les nouvelles routes RIP et OSPF de R1 vers R5.

★★ **Exercice 9** *RSA miniature et exponentiation rapide* ▶ corrigé p. 1030 ≈ 45 min Chapitres croisés : *Cryptographie ; Diviser pour régner ; Récursivité*

Alice choisit $p = 11$ et $q = 17$; sa clé publique est (e, n) avec $e = 7$, et sa clé privée est $d = 23$. Un nombre m (avec $0 \leq m < n$) se chiffre en $c = m^e \bmod n$ et se déchiffre en $m = c^d \bmod n$.

En Python, on peut écrire naïvement `c = m ** e % n`. Mais ce calcul forme d'abord le nombre m^e, gigantesque dans un vrai RSA : la question 4 fait mieux, comme `pow(m, e, n)` vu au chapitre *Cryptographie*.

1. Calculer n et $\varphi = (p-1)(q-1)$, puis vérifier que $e \times d \bmod \varphi = 1$.
2. Chiffrer **à la main** le nombre $m = 42$, en calculant successivement 42^2, 42^3, 42^6 et 42^7 modulo n (on réduit modulo n après chaque multiplication).
3. Combien de multiplications faut-il pour calculer a^k en multipliant a par lui-même, une fois après l'autre ? Dans un vrai RSA, d a environ 600 chiffres : est-ce envisageable ?
4. On utilise plutôt la méthode **diviser pour régner** : $a^k = \left(a^{k/2}\right)^2$ si k est pair, et $a^k = \left(a^{(k-1)/2}\right)^2 \times a$ si k est impair. ▶ Écrire une fonction **récursive** `expo_mod(a, k, n)` qui renvoie $a^k \bmod n$ avec **un seul** appel récursif, en réduisant modulo n après chaque multiplication. ▷ coup de pouce
5. Compter les multiplications effectuées par `expo_mod(c, 23, 187)` (élever au carré compte pour une). Quel est l'ordre de grandeur de ce nombre pour un exposant k quelconque ?
6. ▶ Déchiffrer le résultat de la question 2 avec `expo_mod`. Chiffrer le message `[72, 105]`.

Après les chapitres 11 à 14 (programmation dynamique, recherche textuelle, calculabilité, systèmes sur puce)

★★★ **Exercice 10** *Détecter un plagiat : la plus longue sous-chaîne commune* ▶ corrigé p. 1030 ≈ 45 min Chapitres croisés : *Recherche textuelle ; Programmation dynamique*

Pour repérer un passage recopié, on cherche la plus longue **sous-chaîne** commune à deux textes. Attention : une sous-chaîne est faite de caractères **consécutifs** (`a[i:j]`), contrairement à la sous-séquence du chapitre *Programmation dynamique*.

1. Donner la plus longue sous-chaîne commune à `"ALGORITHME"` et `"LOGARITHME"`.
2. On dispose de la fonction `recherche_naive(motif, texte)` du cours, qui renvoie la liste des positions de `motif` dans `texte`. ▶ Écrire une fonction `plus_longue_naive(a, b)` qui teste toutes les sous-chaînes `a[i:j]` de `a` et renvoie la plus longue qui apparaît dans `b`. Pour deux textes de n caractères, combien de sous-chaînes teste-t-elle environ ? Que penser de son coût ?
3. On note $L[i][j]$ la longueur de la plus longue sous-chaîne commune qui **se termine** par le caractère `a[i-1]` dans `a` et par `b[j-1]` dans `b` (avec $L[0][j] = L[i][0] = 0$). Justifier que $L[i][j] = L[i-1][j-1] + 1$ si `a[i-1] == b[j-1]`, et $L[i][j] = 0$ sinon.
4. Remplir la table L pour `a = "ABAB"` et `b = "BABA"`. Où lit-on la longueur cherchée ? Donner une plus longue sous-chaîne commune. ▷ coup de pouce
5. ▶ Écrire une fonction `plus_longue_dyn(a, b)` qui remplit la table ligne par ligne, des petits indices vers les grands (méthode ascendante), et renvoie une plus longue sous-chaîne commune (et non seulement sa longueur). Quel est son coût pour deux textes de n et m caractères ?

★★ **Exercice 11** *Une machine minuscule* ▶ corrigé p. 1031 ≈ 45 min Chapitres croisés : *Calculabilité ; Systèmes sur puce*

Une machine très simple n'a qu'un registre `x` (un entier) et un compteur `position` qui indique l'instruction à exécuter, en partant de `0`. Un programme est une **liste** d'instructions :

- `("ajoute", k)` : ajoute `k` à `x`, puis passe à l'instruction suivante ;
- `("saute_si_nul", c)` : si `x` vaut `0`, va à l'instruction `c` ; sinon passe à la suivante ;
- `("saute", c)` : va à l'instruction `c` ;
- `("stop",)` : arrête la machine.

```
prog_a = [("saute_si_nul", 3), ("ajoute", -1), ("saute", 0), ("stop",)]
prog_b = [("saute_si_nul", 3), ("ajoute", -2), ("saute", 0), ("stop",)]
```

1. Dérouler `prog_a` avec `x = 2` au départ : noter `position` et `x` après chaque instruction. Combien d'instructions sont exécutées avant `stop` ? Et pour une valeur de départ `x` positive quelconque ?

2. Que fait `prog_b` avec `x = 4` ? avec `x = 3` ? Justifier.

3. ▶ Écrire une fonction `executer(programme, x, max_pas)` qui exécute le programme et renvoie la valeur finale de `x` s'il atteint `stop` après avoir exécuté au plus `max_pas` instructions (`stop` non comprise), et `None` sinon.

4. `executer(prog_a, 50, 100)` renvoie `None`, alors que `prog_a` s'arrête. Pourquoi ? Peut-on choisir `max_pas` assez grand pour que `executer` réponde juste pour *tout* programme ? Relier la réponse au problème de l'arrêt. En quoi la fonction `executer` illustre-t-elle qu'un programme est une **donnée** ?

5. Le programme d'un microcontrôleur répète sans fin « lire les capteurs, décider, commander ». Écrire un programme de cette machine qui ne s'arrête jamais, quelle que soit la valeur de départ de `x`. Un programme qui ne s'arrête pas est-il forcément faux ?

Coups de pouce *après avoir vraiment cherché : la **piste** d'abord, le **début de solution** si besoin.*

Ex. 1 q. 3 – piste Deux boucles : la première calcule les restes `n % 2` et les range dans la pile ; la seconde les ressort pour construire la chaîne. Attention au cas `n = 0`, qui doit produire un chiffre. ↑énoncé

Ex. 2 q. 3 – piste Avant de modifier le texte, on sauvegarde l'ancien dans la pile qui permettra d'y revenir. Annuler et rétablir sont symétriques : chacune sauvegarde le texte courant dans l'autre pile. ↑énoncé

Ex. 3 q. 4 – piste Penser à une pile d'assiettes : on prend la première cellule de `lst` et on la pose *en tête* de `acc` (une nouvelle cellule), puis on continue avec le reste. Que vaut `acc` au tout premier appel ? ↑énoncé

Ex. 4 q. 5 – piste Insérer une valeur parcourt un seul chemin de la racine vers le bas : son coût est au plus la hauteur. Ici, chaque nouvelle valeur est plus grande que toutes les autres : où va-t-elle ? ↑énoncé

Ex. 5 q. 2 – piste Quand on défile un couple `(n, niveau)`, ses enfants sont au niveau `niveau + 1`. Le dictionnaire se remplit comme un compteur d'occurrences. ↑énoncé

Ex. 6 q. 7 – piste Deux listes par compréhension : la première crée un objet par triplet (`for (numero, titre, jour) in lignes`), la seconde ne garde que le titre des objets en retard. ↑

Ex. 7 q. 4 – piste À la fin, comparer la longueur de la liste obtenue au nombre de tâches. ↑énoncé

Ex. 8 q. 2 – piste C'est le calcul des distances du parcours en largeur du chapitre *Graphes* : un voisin découvert depuis `r` est à la distance de `r` plus 1. ↑énoncé

Ex. 9 q. 4 – piste Condition d'arrêt : $a^0 = 1$. Ranger le résultat de l'appel sur `k // 2` dans une variable avant de l'élever au carré. ↑énoncé

Ex. 10 q. 4 – piste Contrairement à la sous-séquence, la réponse n'est pas dans le coin en bas à droite : c'est le **maximum** de toute la table. ↑énoncé

II

DEUXIÈME PARTIE

Préparer les épreuves du bac

Réussir la partie écrite

Le cadre

En **juin**, **3 h 30**, notée sur **12**. Avec la partie pratique (1 h sur machine, notée sur 8), elle forme l'épreuve de spécialité, de coefficient **16**. Le sujet comporte **trois exercices indépendants**, chacun annoncé par les notions qu'il mobilise ; le barème de chaque exercice est indiqué sur le sujet. Calculatrice interdite : on calcule à la main, on raisonne en ordres de grandeur. Tout se fait **sans machine** : le code s'écrit et se teste sur papier.

Lire le sujet et choisir l'ordre (10 minutes)

1. **Survoler les trois exercices** : lire leurs titres, les notions annoncées, et souligner les questions que l'on sait traiter tout de suite.
2. **Repérer les annexes** (à rendre avec la copie) et les **fonctions fournies** : l'énoncé les donne pour qu'on s'en serve.
3. **Choisir l'ordre** : commencer par l'exercice où l'on est le plus à l'aise, pour sécuriser des points et prendre confiance. On peut traiter les exercices dans n'importe quel ordre, à condition de les **numéroter clairement**.
4. **Ne jamais abandonner un exercice** : les questions de la fin sont souvent indépendantes, et une question non résolue peut être **admise** pour traiter la suivante.

Gérer 3 h 30

	Lecture	**1er exercice**	**2e exercice**	**3e exercice**	**Relecture**
Durée indicative	10 min	$\approx$ 55 min	$\approx$ 55 min	$\approx$ 75 min	15 min

Repère : le sujet est barémé sur **20** (en général 6, 6 et 8 points), puis la note est ramenée sur 12. Une fois la lecture et la relecture mises de côté, il reste un peu plus de 3 h, soit environ **9 minutes par point** de ce barème : l'exercice sur 8 points mérite un peu plus de temps que les deux autres. Bloqué plus de **5 minutes** sur une question ? La marquer d'une croix dans la marge, passer à la suivante, y revenir à la fin. Dès qu'un exercice dépasse son temps de plus de 15 minutes, passer au suivant.

Écrire du code sur papier

- **Indentation visible** : décaler nettement chaque bloc (au besoin, tracer au crayon un trait vertical sous chaque `for`, `if`, `def`).
- **Respecter la signature** imposée (nom de la fonction, ordre des paramètres) et choisir des **noms explicites** pour ses variables.
- **« Compléter »** : ne recopier que les lignes à trous, avec leur numéro.
- **Tester mentalement** sur un petit exemple *et* sur un cas limite (liste vide, un seul élément, valeurs négatives), puis **tracer** les variables dans un tableau.

Exemple corrigé. « Écrire une fonction `maximum(tab)` qui renvoie le plus grand élément d'une liste non vide d'entiers. »

Copie à corriger :

```
def maximum(tab):
    m = 0
    for x in tab:
        if x > m:
            m = x
    print(m)
```

Version juste :

```
def maximum(tab):
    m = tab[0]        # la liste est non vide
    for x in tab:
        if x > m:
            m = x
    return m
```

Deux erreurs : `print` n'est pas `return` (la copie renvoie `None`) ; `m = 0` donne un résultat faux pour `[-3, -1, -7]` (la copie affiche `0`, la version juste renvoie `-1`). **Trace** de `maximum([4, 9, 2])` :

x	—	4	9	2
m	4	4	9	9

renvoie `9`.

Rédiger une justification

Une justification est une **phrase complète** qui cite le code ou le cours. Trois modèles, sur la boucle ci-contre.

```
def nb_tours(n):
    tours = 0
    while n > 0:
        n = n // 2
        tours = tours + 1
    return tours
```

- **Terminaison** (un *variant*) : « `n` est un entier positif ou nul qui **diminue strictement** à chaque tour, car `n // 2 < n` dès que $n \geqslant 1$; il finit par valoir 0 et la condition `n > 0` devient fausse. »
- **Coût** : « `n` est divisé par 2 à chaque tour : `nb_tours(1000)` fait 10 tours ($1000 \to 500 \to \cdots \to 1 \to 0$). Le nombre de tours est de l'ordre de $\log_2 n$: le coût est **logarithmique**. »
- **Correction** (un *invariant*, pour `maximum`) : « à la fin de chaque tour, `m` est le plus grand des éléments déjà parcourus ; après le dernier tour, c'est donc le plus grand de toute la liste. »

Les verbes de consigne

Verbe	Ce qui est attendu	Exemple de réponse
Donner, indiquer	La réponse seule suffit.	« La clé primaire est `id`. »
Justifier	La réponse *et* sa raison, tirée du code, de l'énoncé ou du cours.	« Non : deux lignes ont la valeur `3` pour `num`, qui ne peut donc pas être une clé primaire. »
Expliquer	Le mécanisme ou le pourquoi, en phrases.	« La pile rend d'abord le dernier élément empilé, donc… »
Écrire	Le code complet (fonction, requête, classe), qui respecte la signature imposée.	Une fonction entière, indentée, avec son `return`.
Compléter	Uniquement les parties manquantes, repérées par leur numéro de ligne.	« Ligne 4 : `return 0` ».
Déterminer, calculer	Le résultat *et* la démarche qui y mène.	Un calcul posé, une trace.

Les erreurs qui coûtent des points

- **`print` au lieu de `return`** : la fonction doit *renvoyer* sa valeur (voir l'exemple de `maximum`).
- **Oublier la condition d'arrêt** d'une fonction récursive ou un **cas limite** (liste vide, arbre vide `None`).
- **`=` au lieu de `==`** dans un test ; **indices hors limites** : le dernier indice de `t` est `len(t) - 1`.
- **Ignorer la convention de l'énoncé** (hauteur d'un arbre, sens d'une pile, coût OSPF) : on applique *celle du sujet*.
- **SQL** : oublier la condition `ON` d'une jointure ou les apostrophes autour d'une chaîne (`WHERE nom = 'Durand'`).
- **Répondre sans justifier** quand le verbe est « justifier » ou « expliquer » : la réponse seule rapporte peu, voire rien.
- **Laisser une question blanche** : une démarche partielle juste, une idée bien exprimée rapportent des points.
- **Une copie illisible** : code mal indenté, numéros de questions absents, ratures partout. Le correcteur ne devine pas.

Un calendrier de révision, d'avril à juin

Les dix semaines qui précèdent l'écrit se préparent. Le principe : revoir chaque chapitre **plusieurs fois, à quelques semaines d'intervalle**, plutôt qu'une seule fois longuement. C'est cette **révision espacée** qui fait tenir les connaissances jusqu'au jour de l'épreuve. Comptez **3 à 4 heures par semaine**, en plusieurs séances courtes (un peu plus pendant les deux dernières). Dans le tableau, S-10 désigne la dixième semaine avant l'écrit, S-1 la dernière ; les autres mots employés sont expliqués juste en dessous.

Semaine	Chapitres	Écrit	Pratique et Grand Oral
S-10	**À fond** : 1 Récursivité ; 2 POO	Type bac ch. 1	GO : choisir vos deux questions, formuler chacune en une phrase
S-9	**À fond** : 3 Piles, files, listes chaînées ; 4 Processus **Rappel** : 1	Type bac ch. 3 ; synthèse 1 à 3	Pratique : sujet 27 GO : problématique et plan de la question 1
S-8	**À fond** : 5 Arbres **Rappel** : 2, 3	Type bac ch. 5 ; synthèse 5	Pratique : sujets 24 et 25 GO : exemple déroulé et schéma de la question 1
S-7	**À fond** : 6 Bases de données ; 7 Diviser pour régner **Rappel** : 1, 4	Type bac ch. 6 (SQL)	Pratique : sujet 29 GO : problématique et plan de la question 2
S-6	**À fond** : 8 Graphes **Rappel** : 3, 5	Type bac ch. 8 ; synthèse 6 à 8	Pratique : sujets 26 et 28 GO : exemple déroulé et schéma de la question 2
S-5	**À fond** : 9 Réseaux ; 10 Cryptographie **Rappel** : 2, 6	Type bac ch. 9 ou 10 ; synthèse 9 et 10	Pratique : sujet 30 GO : chaque exposé à voix haute, chronométré (10 min), avec vos seules notes
S-4	**À fond** : 11 Programmation dynamique ; 12 Recherche textuelle **Rappel** : 7, 8	Type bac ch. 11 ou 12	Pratique : sujet 32 GO : trois relances par question, avec vos réponses
S-3	**À fond** : 13 Calculabilité ; 14 Systèmes sur puce **Rappel** : 4, 9, 10	Type bac ch. 13 ou 14 ; synthèse 11 à 14	Pratique : sujet 31 GO : exposés devant un camarade, avec la grille entre camarades
S-2	**Rappel** : 1 à 7	Un **sujet complet** (3 exercices, 3 h 30) en conditions réelles, puis sa correction	GO : un oral blanc complet (20 min de préparation, 10 min d'exposé, 10 min d'échange)
S-1	**Rappel** : 8 à 14	Deux exercices type bac sur vos chapitres les moins sûrs ; relire votre liste d'erreurs. Rien de nouveau les deux derniers jours	GO : relire vos deux plans, sans plus
Épreuve pratique	Notions de Première et POO (ch. 2), les plus fréquentes à l'épreuve ; puis vos chapitres les moins sûrs	Les jours qui la précèdent (avant ou après l'écrit, selon le calendrier de l'année) : deux ou trois sujets en 1 h chrono, dont les sujets 24 à 32 pas encore faits ; relire la fiche « Réussir l'épreuve pratique ».	
Grand Oral	Les chapitres de vos deux questions	Après l'écrit, chaque jour : un exposé de 10 minutes par question, chronométré, en partant de notes préparées en 20 minutes ; revoir les relances.	

Les mots du tableau

- **À fond** : le chapitre est revu avec sa **fiche de révision** (liste « Savoir-faire », puis « Se tester » de mémoire) ; pour chaque savoir-faire mal maîtrisé, refaire un exercice • essentiel.

- **Rappel** : 15 minutes par chapitre, livre fermé, avec les questions « Se tester » et « Et avant ? » de sa fiche.
- **Type bac** : un exercice type bac du chapitre, **chronométré** à la durée indiquée à côté de son titre (de 15 à 60 minutes), puis corrigé.
- **Synthèse** : un problème de synthèse (fin de la première partie du livre), choisi dans la rubrique indiquée (« 1 à 3 » renvoie à « Après les chapitres 1 à 3 »).
- **Pratique** : un sujet d'épreuve pratique de la deuxième partie du livre (sujets 24 à 32), en 1 h sur machine ; **GO** : Grand Oral.

Cinq conseils de méthode

- **Un planning réaliste.** Reportez le tableau dans votre agenda en tenant compte de vos autres matières et des vacances : quatre séances courtes valent mieux qu'une longue, toujours repoussée. Une semaine a sauté ? Ne rattrapez pas tout : décalez la suite, mais gardez les rappels.
- **Des sujets complets, en conditions réelles.** Au moins un sujet entier (deux, c'est mieux), d'une traite, en 3 h 30, sans cours, sans machine ni IA, en suivant la gestion du temps présentée plus haut. Les vacances de printemps s'y prêtent bien.
- **Corriger avec le corrigé du livre**, d'une autre couleur : comparer point par point, noter ce qui manque (une justification, un cas limite, un `return`), lire les *erreurs fréquentes* signalées. Le corrigé ne se lit qu'après avoir vraiment cherché.
- **Refaire ses erreurs.** Tenez une liste « mes erreurs » (une ligne par erreur : chapitre, exercice, ce qu'il fallait faire) et refaites chaque exercice raté une semaine plus tard, sans regarder le corrigé. C'est cette liste que l'on relit la veille de l'épreuve.
- **Se tester plutôt que relire.** Livre fermé : répondre aux questions « Se tester », écrire le code sur papier avant de le taper, dérouler un algorithme à la main. Relire donne l'impression de savoir ; se tester montre ce que l'on sait vraiment.

Index des exercices type bac

Les extraits de sujets de bac (et les sujets « façon bac ») des feuilles d'exercices, chapitre par chapitre, avec la durée indicative à l'écrit. Un clic mène à l'énoncé ; le corrigé, avec un barème indicatif, est en fin de livre. Pour s'entraîner en conditions : un exercice par séance, chronométré.

Chapitre 1 — Récursivité

- Ex. 29 p. 38 — Analyser et écrire des fonctions récursives (d'après Sujet zéro 2023, sujet B) ≈ 20 min
- Ex. 30 p. 38 — Les scores au rugby (d'après Polynésie 2024, jour 1) ≈ 30 min
- Ex. 31 p. 39 — Itinéraires dans une grille (d'après Centres étrangers 2024, groupe 1, jour 2) ≈ 30 min
- Ex. 32 p. 41 — Les instructions d'un robot (d'après Asie 2026, jour 2) ≈ 45 min
- Ex. 33 p. 43 — Le cadenas de l'escape game (sujet maison, façon bac) ≈ 60 min

Chapitre 2 — Programmation orientée objet et paradigmes

- Ex. 16 p. 90 — Le domaine skiable (d'après Métropole 2024, jour 2) ≈ 20 min
- Ex. 17 p. 91 — Valeurs nutritionnelles (d'après La Réunion 2023, jour 1) ≈ 20 min
- Ex. 18 p. 92 — La classe Date (d'après Asie 2025, jour 2) ≈ 30 min
- Ex. 19 p. 92 — Colorier une carte (d'après Métropole 2023, jour 1) ≈ 15 min
- Ex. 20 p. 93 — Des colis à expédier (d'après Amérique du Nord 2025, jour 1) ≈ 60 min
- Ex. 21 p. 95 — Puissance 4 : la grille de jeu (d'après Amérique du Nord 2026, jour 1, partie A) ≈ 30 min

Chapitre 3 — Structures linéaires : piles, files, listes chaînées

- Ex. 6 p. 143 — La structure de pile (d'après Asie 2024, jour 1) ≈ 15 min
- Ex. 13 p. 145 — Ordonnancer des processus avec une file (d'après Amérique du Nord 2024, jour 1) ≈ 30 min
- Ex. 14 p. 146 — Une file de priorité de tâches (d'après Métropole 2025, jour 1) ≈ 30 min
- Ex. 19 p. 148 — Piles et files pour parcourir un graphe (d'après Polynésie 2024, jour 2) ≈ 20 min
- Ex. 24 p. 150 — Le collier de bonbons : maillons chaînés (d'après Amérique du Nord 2025, jour 2) ≈ 45 min

Chapitre 4 — Processus et ordonnancement

- Ex. 9 p. 193 — Le tourniquet en Python *(d'après Polynésie 2023, jour 1)* ≈ 45 min
- Ex. 12 p. 195 — Observer puis ordonnancer des processus *(d'après Amérique du Nord 2023, jour 1)* ≈ 30 min
- Ex. 13 p. 196 — Tourniquet cycle par cycle et interblocage *(d'après Amérique du Nord 2024, jour 1)* ≈ 45 min
- Ex. 14 p. 197 — Une application de streaming *(d'après Amérique du Nord 2026, jour 2)* ≈ 45 min
- Ex. 15 p. 200 — Des files de priorité pour l'ordonnanceur *(d'après Asie 2025, jour 2)* ≈ 60 min

Chapitre 5 — Arbres

- Ex. 16 p. 245 — ABR : mesures, insertion, recherche (d'après Métropole 2023, jour 2) ≈ 20 min
- Ex. 17 p. 245 — Un arbre binaire par une classe (d'après Sujet zéro 2023, sujet B) ≈ 20 min
- Ex. 18 p. 246 — Un arbre de décision (d'après Amérique du Nord 2025, jour 1) ≈ 30 min
- Ex. 19 p. 247 — Un arbre représenté par des listes (d'après Amérique du Sud 2023, jour 1) ≈ 30 min
- Ex. 20 p. 248 — La plus grande somme racine-feuille (d'après Amérique du Sud 2022, jour 2) ≈ 20 min
- Ex. 21 p. 248 — Puissance 4 : l'arbre de coups et l'algorithme min-max (d'après Amérique du Nord 2026, jour 1, parties B et C) ≈ 45 min

Chapitre 6 — Bases de données et SQL

- Ex. 17 p. 301 — Location entre particuliers (d'après Polynésie 2023, jour 2) ≈ 30 min
- Ex. 18 p. 302 — Location de véhicules (d'après La Réunion 2023, jour 2) ≈ 30 min
- Ex. 19 p. 302 — Podcasts de radio (d'après Centres étrangers 2023, jour 1) ≈ 30 min
- Ex. 20 p. 303 — Vols spatiaux (d'après Métropole 2023, jour 2) ≈ 30 min
- Ex. 21 p. 304 — Ludothèque (d'après Métropole 2025, jour 2) ≈ 30 min

Chapitre 7 — Diviser pour régner

- Ex. 6 p. 347 — Coder un texte avec « diviser pour régner » *(d'après Métropole 2025, jour 2)* ≈ 45 min
- Ex. 7 p. 348 — Le tri de Stooge *(d'après Amérique du Nord 2024, jour 2)* ≈ 45 min
- Ex. 8 p. 350 — L'élément absolument majoritaire *(d'après Asie 2024, jour 2)* ≈ 60 min
- Ex. 9 p. 351 — Les deux points les plus proches *(d'après Polynésie 2023, jour 1)* ≈ 60 min
- Ex. 10 p. 353 — Comparer deux classements *(sujet maison, façon bac)* ≈ 60 min

Chapitre 8 — Graphes

- Ex. 15 p. 409 — Un cercle d'amis *(d'après Amérique du Nord 2024, jour 1, ex. 2)* ≈ 45 min
- Ex. 16 p. 410 — La recette du pain : un graphe de tâches *(d'après Asie 2024, jour 1, ex. 2)* ≈ 30 min
- Ex. 17 p. 411 — Un parc d'attractions *(d'après Pays étrangers 2025, sujet PE2, ex. 3 (partie A))* ≈ 30 min
- Ex. 18 p. 412 — Un réseau d'ordinateurs *(d'après Centres étrangers 2024, groupe 1, jour 1, ex. 1)* ≈ 30 min
- Ex. 19 p. 412 — Une agence de voyages *(d'après Polynésie 2024, jour 2, ex. 1)* ≈ 45 min
- Ex. 23 p. 415 — Sortir d'un labyrinthe *(type bac)* ≈ 40 min

Chapitre 9 — Réseaux

- Ex. 11 p. 458 — Réseau d'un établissement *(d'après La Réunion 2023, jour 2)* ≈ 60 min
- Ex. 12 p. 460 — Des adresses, RIP puis OSPF *(d'après Centres étrangers 2023, jour 1)* ≈ 30 min
- Ex. 13 p. 461 — Bob écrit à Alice *(d'après Sujet zéro 2023, sujet A)* ≈ 30 min
- Ex. 14 p. 462 — Du réseau local L1 au réseau local L2 *(d'après Centres étrangers 2024, groupe 1, jour 1)* ≈ 45 min
- Ex. 15 p. 464 — Le réseau d'une société *(d'après Métropole septembre 2024, jour 1)* ≈ 30 min

Chapitre 10 — Cryptographie

- Ex. 12 p. 498 — La méthode du masque jetable *(d'après Métropole 2025, jour 2)* ≈ 60 min
- Ex. 13 p. 499 — Le chiffrement de Polybe *(d'après Centres étrangers 2026, groupe 1, jour 1)* ≈ 60 min
- Ex. 14 p. 500 — Transmettre une clé de session *(d'après Métropole 2024, jour 2)* ≈ 15 min
- Ex. 15 p. 501 — Les mots de passe d'un hôpital *(d'après Polynésie 2024, jour 2)* ≈ 45 min
- Ex. 16 p. 503 — Les casiers connectés du lycée *(sujet maison, façon bac)* ≈ 60 min

Chapitre 11 — Programmation dynamique

- Ex. 16 p. 547 — Vue sur la mer *(d'après Amérique du Nord 2026, jour 1)* ≈ 30 min
- Ex. 17 p. 548 — Justifier un texte *(d'après Pays étrangers 2025, sujet PE2)* ≈ 45 min
- Ex. 18 p. 549 — Charger un camion au meilleur prix *(d'après Métropole 2024, jour 2)* ≈ 60 min
- Ex. 19 p. 551 — Découper des planches au meilleur prix *(sujet maison, façon bac)* ≈ 60 min
- Ex. 20 p. 553 — Décoder un message secret *(sujet maison, façon bac)* ≈ 60 min

Chapitre 12 — Recherche textuelle

- Ex. 10 p. 597 — Parcourir une chaîne de caractères *(d'après Centres étrangers 2023, jour 2)* ≈ 15 min
- Ex. 11 p. 597 — Le principe de Boyer-Moore *(d'après Asie 2025, jour 1)* ≈ 10 min
- Ex. 12 p. 598 — Rechercher et remplacer dans un éditeur *(sujet maison, façon bac)* ≈ 45 min
- Ex. 13 p. 599 — Un antivirus à la recherche de signatures *(sujet maison, façon bac)* ≈ 45 min
- Ex. 14 p. 600 — Repérer des amorces dans un brin d'ADN *(sujet maison, façon bac)* ≈ 45 min

Chapitre 13 — Calculabilité et décidabilité

- Ex. 13 p. 642 — Terminaison et problème de l'arrêt (d'après Métropole septembre 2024, jour 2) ≈ 45 min
- Ex. 14 p. 643 — Un programme comme chaîne de caractères (d'après Asie 2025, jour 1) ≈ 45 min
- Ex. 15 p. 644 — Un correcteur automatique d'exercices (sujet maison, façon bac) ≈ 45 min
- Ex. 16 p. 645 — Un antivirus infaillible ? (sujet maison, façon bac) ≈ 45 min
- Ex. 17 p. 646 — Un interpréteur miniature (sujet maison, façon bac) ≈ 60 min

Chapitre 14 — Systèmes sur puce et informatique embarquée

- Ex. 9 p. 682 — Plusieurs processeurs sur une puce *(d'après Amérique du Nord 2026, jour 2)* ≈ 15 min
- Ex. 10 p. 682 — Choisir la puce d'une montre connectée *(sujet maison, façon bac)* ≈ 45 min
- Ex. 11 p. 683 — Une station météo au lycée *(sujet maison, façon bac)* ≈ 45 min
- Ex. 12 p. 684 — Une salle informatique à renouveler *(sujet maison, façon bac)* ≈ 45 min
- Ex. 13 p. 685 — Au cœur d'un smartphone *(sujet maison, façon bac)* ≈ 45 min

Réussir l'épreuve pratique

Le cadre

1 heure, **sur machine**, notée sur **8**. Avec la partie écrite (3 h 30, notée sur 12), elle forme l'épreuve de spécialité, de coefficient **16**. Le sujet est accompagné de fichiers Python (code de départ, données) déjà présents sur l'ordinateur. Il demande d'**écrire** du code, de le **tester** et de **corriger** un code fourni. Un examinateur est présent : il vient vous voir aux **appels professeur** indiqués dans le sujet, et peut intervenir à tout moment.

S'entraîner pendant l'année

- Les **sujets 1 à 23** reprennent le format de la banque nationale ; ils sont disponibles sur la **plateforme de la classe**.
- Les **sujets 24 à 32**, écrits pour les chapitres de Terminale que les premiers travaillent peu, sont dans la **deuxième partie de ce livre** ; leurs corrigés sont en fin de livre.
- Traitez chaque sujet **en 1 heure chronométrée**, sans aide d'IA, puis seulement ensuite lisez le corrigé.

Gérer l'heure

	Lecture	Écrire et tester	Corriger un code fourni	Relecture
Durée indicative	5 min	≈ 25 min	≈ 25 min	5 min

1. **Lire tout le sujet**, puis **ouvrir les fichiers** fournis et les **exécuter** tels quels : on voit ce qui existe déjà et ce qui manque.
2. **Repérer les appels professeur** : préparer, pour chacun, ce que l'on va montrer et dire.
3. **Enregistrer souvent**, sous le nom de fichier demandé.
4. **Bloqué plus de 5 minutes** ? Le dire à l'examinateur : une aide ponctuelle vaut mieux qu'une demi-heure perdue, et la suite du sujet reste accessible.

Écrire, tester, corriger

- **Respecter la signature** imposée (nom de la fonction, paramètres, valeur renvoyée) : les tests de l'examinateur l'appellent telle quelle.
- **`return`, pas `print`** : une fonction renvoie son résultat ; un `print` sert seulement à tracer, et on le retire ensuite.
- **Tester** avec `assert` sur un cas courant *et* sur les cas limites (liste vide, un seul élément, élément absent, arbre vide `None`), en calculant le résultat attendu **à la main**.
- **Corriger un code fourni** : exécuter, lire le message d'erreur de bas en haut (type, puis ligne), trouver le plus petit exemple qui échoue, tracer les variables jusqu'à la première valeur inattendue, corriger **une seule chose**, puis relancer **tous** les tests.

> **Remarque** — *À l'appel professeur*
> On montre le code qui s'exécute et on **explique** sa démarche : « j'attendais …, j'ai obtenu … ; sur l'exemple …, la variable … vaut … au lieu de … ; j'ai donc corrigé … ». Une erreur bien localisée et bien expliquée est valorisée, même si la correction n'est pas terminée.

Les erreurs qui coûtent des points

- Modifier le code fourni au-delà de ce qui est demandé, ou changer le nom d'une fonction.
- Rendre un fichier qui ne s'exécute plus (erreur de syntaxe laissée en fin d'épreuve) : mieux vaut une fonction incomplète mais un fichier qui tourne.
- Écrire des tests qui recopient le résultat obtenu au lieu du résultat attendu.
- Ne rien dire à l'appel professeur : l'explication orale fait partie de l'évaluation.

Présentation des sujets

Ces sujets d'entraînement reprennent le format de l'épreuve pratique du baccalauréat (durée 1 heure, fichier Python à compléter, appels professeur), mais ne font pas partie de la banque nationale. Ils couvrent des notions de Terminale : arbres (expression arithmétique, arbre binaire de recherche), graphes (labyrinthe), piles (notation polonaise inverse), ordonnancement, recherche dichotomique, cryptographie, recherche textuelle et programmation dynamique. Ils sont numérotés à partir de 24 car ils font suite à une première série de 23 sujets d'entraînement, disponible sur la plateforme de la classe (rubrique Entraînement pratique) et qui ne figure pas dans ce livre. Les fichiers fournis (codes de départ et données) sont à télécharger : le lien (et le QR code) vers le dossier du sujet et vers l'archive .zip est indiqué en tête de chaque sujet (aucun compte n'est nécessaire). Les corrigés sont regroupés en fin de livre.

Sujet n° 24 — Arbre d'une expression arithmétique

▶ corrigé p. 1033

Fichiers à télécharger : `maththibaud.github.io/informatique-en-autonomie/f/tpratique-24-arbre-expression` — **archive .zip**

Durée de l'épreuve : 1 heure.

Déroulement de l'épreuve

- Cette situation d'évaluation comporte ce document ainsi que des fichiers de codes et de données présents sur l'ordinateur. Le candidat doit **agir en autonomie** et faire preuve d'initiative tout au long de l'épreuve.
- En cas de difficulté, le candidat peut solliciter l'examinateur. Des moments privilégiés sont indiqués sous la forme d'**appels professeur**. L'examinateur peut intervenir à tout moment s'il le juge utile.

Une expression arithmétique comme $(3 + 5) \times (7 - 2)$ peut être représentée par un **arbre binaire** : chaque **feuille** porte un nombre entier et chaque autre nœud porte un **opérateur** (`'+'`, `'-'` ou `'*'`) qui s'applique à la valeur de son sous-arbre gauche et à celle de son sous-arbre droit, **dans cet ordre**.

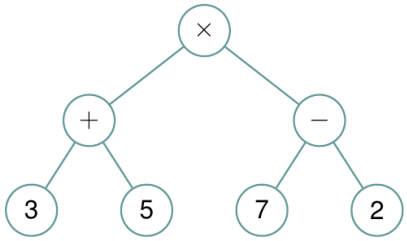


Figure 1. L'arbre `e1` de l'expression $(3 + 5) \times (7 - 2)$.

Dans le fichier `expression.py`, un nœud est un objet de la classe `Noeud` (attributs `valeur`, `gauche` et `droite`) et la fonction `est_feuille` indique si un nœud n'a aucun fils. L'arbre de la figure 1 s'écrit :

```
e1 = Noeud('*',
           Noeud('+', Noeud(3), Noeud(5)),
           Noeud('-', Noeud(7), Noeud(2)))
```

Le fichier définit aussi les arbres `e2`, `e3`, `e4` et `e5`. Les fonctions de ce sujet sont **récursives** : on traite séparément le cas d'une feuille, puis celui d'un opérateur, en s'appuyant sur les deux sous-arbres.

Question 1

Dessiner sur papier les arbres `e4` et `e5` définis dans le fichier, puis donner, pour chacun, l'expression qu'il représente et sa valeur. Expliquer pourquoi une expression représentée par un arbre n'a pas besoin de parenthèses.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 2

La fonction `en_chaine` doit renvoyer l'écriture de l'expression où chaque opération est entourée de parenthèses : `en_chaine(e1)` doit valoir `"((3 + 5) * (7 - 2))"`. Exécuter `en_chaine(e1)` et expliquer pourquoi le résultat obtenu est ambigu, puis corriger la fonction. La fonction `test_en_chaine` permet de vérifier la correction.

Question 3

Écrire le corps de la fonction récursive `evaluer` qui renvoie la valeur de l'expression représentée par un arbre. Par exemple `evaluer(e1)` doit renvoyer `40`. Des tests sont fournis dans la fonction `test_evaluer`, on pourra les compléter avec les arbres `e2`, `e4` et `e5`.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 4

Écrire le corps de la fonction récursive `nb_operateurs` qui renvoie le nombre d'opérateurs d'une expression (c'est-à-dire le nombre de nœuds qui ne sont pas des feuilles). Tester avec la fonction `test_nb_operateurs`.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Description du dossier

Le dossier fourni au candidat comporte les éléments suivants :

- une version PDF de l'énoncé ;
- un code source de départ `expression.py` ;
- l'image de la figure 1 (`arbre_e1.png`).

Sujet n° 25 — Arbre binaire de recherche

► corrigé p. 1035

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/tpratique-25-abr` — **archive .zip**

Durée de l'épreuve : 1 heure.

Déroulement de l'épreuve

- Cette situation d'évaluation comporte ce document ainsi que des fichiers de codes et de données présents sur l'ordinateur. Le candidat doit **agir en autonomie** et faire preuve d'initiative tout au long de l'épreuve.
- En cas de difficulté, le candidat peut solliciter l'examinateur. Des moments privilégiés sont indiqués sous la forme d'**appels professeur**. L'examinateur peut intervenir à tout moment s'il le juge utile.

Un **arbre binaire de recherche** (ABR) est un arbre binaire dont les nœuds portent des valeurs comparables et tel que, pour **chaque** nœud, toutes les valeurs de son sous-arbre gauche sont **strictement plus petites** que sa valeur, et toutes celles de son sous-arbre droit **strictement plus grandes**. Un ABR ne contient donc pas deux fois la même valeur.

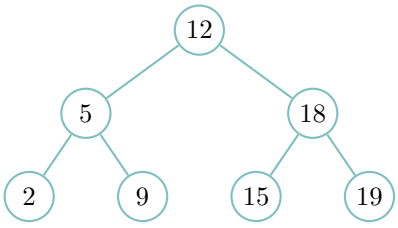


Figure 1. Un ABR : à gauche de `12`, les valeurs `5`, `2` et `9`, toutes plus petites ; à droite, `18`, `15` et `19`, toutes plus grandes. La règle vaut aussi pour `5` et pour `18`.

Dans le fichier `abr.py`, un nœud est un objet de la classe `Noeud` (attributs `valeur`, `gauche` et `droite`) et l'arbre vide est représenté par `None`. Le fichier fournit :

- la fonction récursive `inserer(a, x)` qui insère la valeur `x` à sa place dans l'ABR `a` et renvoie l'arbre obtenu (si `x` est déjà présente, rien ne change) ;
- la fonction `construire_abr(valeurs)` qui insère une à une, dans l'ordre, les valeurs d'une liste à partir d'un arbre vide ;
- la fonction `hauteur(a)` (la hauteur de l'arbre vide vaut `0`, celle d'un arbre réduit à un nœud vaut `1`).

Question 1

Dessiner sur papier l'ABR obtenu par `construire_abr([8, 3, 10, 1, 6, 14, 4, 7])`, et donner sa hauteur. Vérifier la hauteur à l'aide de la fonction `hauteur`.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 2

Écrire le corps de la fonction récursive `recherche(a, x)` qui renvoie `True` si `x` est une valeur de l'ABR `a` et `False` sinon. La fonction doit utiliser la propriété des ABR pour ne descendre que dans **un seul** des deux sous-arbres. Des tests sont fournis dans la fonction `test_recherche`.

Question 3

Écrire le corps de la fonction récursive `infixe(a)` qui renvoie la liste des valeurs de l'arbre `a` dans l'ordre du parcours infixe (sous-arbre gauche, puis racine, puis sous-arbre droit). En déduire le corps de la fonction `tri_abr(valeurs)` qui renvoie la liste des valeurs triée dans l'ordre croissant, sans doublon. Tester avec la fonction `test_tri`.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 4

La fonction fournie `experience(n)` construit deux ABR contenant les entiers de `1` à `n` : l'un en les insérant dans l'ordre croissant, l'autre dans un ordre aléatoire, et affiche leurs hauteurs. Exécuter `experience(200)` plusieurs fois, décrire la forme de chacun des deux arbres et expliquer la conséquence sur le coût de la fonction `recherche`.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Description du dossier

Le dossier fourni au candidat comporte les éléments suivants :

- une version PDF de l'énoncé ;
- un code source de départ `abr.py`.

Sujet n° 26 — Sortir d'un labyrinthe

▶ corrigé p. 1038

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/tpratique-26-labyrinthe` — **archive .zip**

Durée de l'épreuve : 1 heure.

Déroulement de l'épreuve

- Cette situation d'évaluation comporte ce document ainsi que des fichiers de codes et de données présents sur l'ordinateur. Le candidat doit **agir en autonomie** et faire preuve d'initiative tout au long de l'épreuve.
- En cas de difficulté, le candidat peut solliciter l'examinateur. Des moments privilégiés sont indiqués sous la forme d'**appels professeur**. L'examinateur peut intervenir à tout moment s'il le juge utile.

Un labyrinthe est décrit dans un fichier texte : chaque ligne du fichier est une ligne du labyrinthe, le caractère `#` est un **mur**, une espace est une case libre, `E` est l'entrée et `S` la sortie. On se déplace d'une case à une case voisine (en haut, en bas, à gauche ou à droite, jamais en diagonale), sans traverser les murs.

```
#######
#E    #
# ### #
#   #S#
#######
```

Figure 1. Le labyrinthe du fichier `petit.txt`.

Une case est repérée par le couple `(ligne, colonne)`, les numéros commençant à `0` en haut à gauche : dans `petit.txt`, l'entrée est la case `(1, 1)` et la sortie la case `(3, 5)`.

Le labyrinthe est vu comme un **graphe** : les sommets sont les cases qui ne sont pas des murs, et deux sommets sont reliés s'ils sont voisins. Dans le fichier `labyrinthe.py`, la fonction `lire_labyrinthe` renvoie la grille (liste de chaînes de caractères, une par ligne) et la fonction `construire_graphe` renvoie le graphe sous forme d'un **dictionnaire** qui associe à chaque case la liste de ses cases voisines.

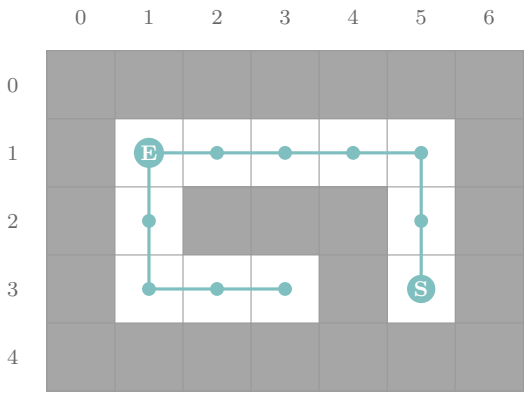


Figure 2. Le graphe associé à `petit.txt` : un sommet par case libre, une arête entre deux cases voisines. Les numéros de ligne sont à gauche, ceux de colonne en haut.

Question 1

Écrire le corps de la fonction `cases_voisines(grille, case)` qui renvoie la liste des cases qui ne sont pas des murs parmi les quatre cases voisines de `case`, dans l'ordre : haut, bas, gauche, droite. Par exemple, avec la grille de `petit.txt`, `cases_voisines(grille, (1, 1))` renvoie `[(2, 1), (1, 2)]`. Tester avec la fonction `test_voisines`.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 2

La fonction `plus_court_chemin` réalise un **parcours en largeur** du graphe à partir de la case de départ, en mémorisant pour chaque case découverte la case qui l'a fait découvrir (dictionnaire `parent`), puis reconstruit le chemin en remontant les parents depuis l'arrivée. Compléter les lignes manquantes de cette fonction, repérées par `A COMPLETER`. Tester avec la fonction `test_chemin`.

Question 3

Utiliser la fonction fournie `sortir` sur le fichier `labyrinthe.txt`. Relever le nombre de déplacements nécessaires pour sortir et observer le chemin affiché.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 4

Dans la fonction `plus_court_chemin`, on remplace `en_attente.pop(0)` par `en_attente.pop()`, puis on appelle de nouveau `sortir("labyrinthe.txt")`. Obtient-on encore un chemin de l'entrée à la sortie ? Est-il le plus court ? Expliquer ce qui a changé dans le parcours.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

II

Description du dossier

Le dossier fourni au candidat comporte les éléments suivants :

- une version PDF de l'énoncé ;
- un code source de départ `labyrinthe.py` ;
- deux labyrinthes au format texte : `petit.txt` et `labyrinthe.txt`.

Sujet n° 27 — Calculatrice en notation polonaise inverse

► corrigé p. 1040

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/tpratique-27-npi` — **archive .zip**

Durée de l'épreuve : 1 heure.

Déroulement de l'épreuve

- Cette situation d'évaluation comporte ce document ainsi que des fichiers de codes et de données présents sur l'ordinateur. Le candidat doit **agir en autonomie** et faire preuve d'initiative tout au long de l'épreuve.
- En cas de difficulté, le candidat peut solliciter l'examinateur. Des moments privilégiés sont indiqués sous la forme d'**appels professeur**. L'examinateur peut intervenir à tout moment s'il le juge utile.

Dans la **notation polonaise inverse** (NPI), utilisée par certaines calculatrices, on écrit un opérateur **après** ses deux opérandes : l'expression $3+4$ s'écrit `3 4 +` et l'expression $(3+4)\times 2$ s'écrit `3 4 + 2 *`. Cette notation n'utilise jamais de parenthèses.

Une expression en NPI s'évalue avec une **pile**, en lisant ses éléments de gauche à droite :

- un nombre est **empilé** ;
- un opérateur **dépile** deux nombres, leur applique l'opération et **empile** le résultat. Le premier nombre dépilé est l'opérande de **droite**, le second celui de **gauche** : `7 2 -` vaut $7-2=5$.

À la fin, la pile contient un seul nombre : la valeur de l'expression.

Dans ce sujet, les expressions sont des chaînes de caractères dont les éléments, séparés par des espaces, sont des **entiers positifs** ou l'un des opérateurs `+`, `-` et `*`. On rappelle que `"3 4 +".split()` renvoie la liste `['3', '4', '+']` et que `int('3')` renvoie l'entier `3`. Le fichier `npi.py` fournit une classe `Pile` proche de celle du cours, **complétée d'une méthode `taille`** (méthodes `est_vide`, `empiler`, `depiler`, `sommet` et `taille`) ; attention, sa méthode `depiler` provoque une erreur si la pile est vide.

Question 1

Évaluer à la main l'expression `5 1 2 + 4 * + 3 -` en dessinant l'état de la pile après la lecture de chaque élément.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

II

Question 2

Écrire le corps de la fonction `evaluer_npi(expression)` qui renvoie la valeur d'une expression en NPI, en utilisant un objet de la classe `Pile`. Des tests sont fournis dans la fonction `test_evaluer`, on pourra les compléter.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 3

Une expression peut être mal formée : dans `"3 +"` il manque un opérande, et dans `"3 4"` il manque un opérateur. Modifier la fonction `evaluer_npi` pour qu'elle renvoie `None` lorsque l'expression est mal formée (sans provoquer d'erreur). Tester avec la fonction `test_erreurs`, que l'on complétera avec d'autres cas.

Question 4

Traduire en NPI l'expression $(2 + 3) \times (7 - 4)$, puis vérifier le résultat à l'aide de la fonction `evaluer_npi`.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Description du dossier

Le dossier fourni au candidat comporte les éléments suivants :

- une version PDF de l'énoncé ;
- un code source de départ `npi.py`.

Sujet n° 28 — Ordonnancement de tâches : le tourniquet

► corrigé p. 1043

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/tpratique-28-tourniquet` — **archive .zip**

Durée de l'épreuve : 1 heure.

Déroulement de l'épreuve

- Cette situation d'évaluation comporte ce document ainsi que des fichiers de codes et de données présents sur l'ordinateur. Le candidat doit **agir en autonomie** et faire preuve d'initiative tout au long de l'épreuve.
- En cas de difficulté, le candidat peut solliciter l'examinateur. Des moments privilégiés sont indiqués sous la forme d'**appels professeur**. L'examinateur peut intervenir à tout moment s'il le juge utile.

Un système d'exploitation doit partager le processeur entre plusieurs tâches. On suppose ici que toutes les tâches sont prêtes à l'instant `0`, dans un ordre donné, et qu'une tâche est décrite par un couple `(nom, duree)`, la durée étant un nombre entier d'unités de temps.

On compare deux façons de choisir la tâche à exécuter, toutes deux fondées sur une **file** de tâches en attente :

- **premier arrivé, premier servi** : on défile une tâche et on l'exécute **jusqu'au bout**, puis on passe à la suivante ;
- **tourniquet** de quantum `q` : on défile une tâche et on l'exécute pendant **au plus** `q` unités de temps. Si elle n'est pas terminée, elle est **enfilée de nouveau**, en fin de file, avec sa durée restante.

Chaque ordonnancement est résumé par la liste des couples `(nom, instant de fin)`, dans l'ordre où les tâches se terminent. Par exemple, pour les tâches `[("A", 3), ("B", 1), ("C", 4)]`, l'ordonnancement premier arrivé, premier servi donne `[("A", 3), ("B", 4), ("C", 8)]`.

Le fichier `ordonnanceur.py` fournit une classe `File` proche de celle du cours, **complétée d'une méthode `taille`** (méthodes `est_vide`, `enfiler`, `defiler`, `tete` et `taille`), la fonction `lire_taches` qui lit les tâches d'un fichier CSV et la fonction `premier_arrive`.

Question 1

Pour les tâches `[("A", 3), ("B", 1), ("C", 4)]` et un tourniquet de quantum 2, représenter sur une frise l'exécution des tâches au cours du temps ainsi que le contenu de la file, puis donner la liste des couples `(nom, instant de fin)`.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

II

Question 2

Écrire le corps de la fonction `tourniquet(taches, quantum)` qui renvoie la liste des couples `(nom, instant de fin)` obtenue avec un tourniquet de quantum `quantum`, en utilisant un objet de la classe `File`. On pourra s'inspirer de la fonction `premier_arrive`. Tester avec la fonction `test_tourniquet`.

Question 3

Écrire le corps de la fonction `temps_moyen(fins)` qui renvoie la moyenne des instants de fin d'une liste de couples `(nom, instant de fin)`. Tester avec la fonction `test_temps_moyen`.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 4

Exécuter `comparer("taches.csv")`, qui affiche le temps moyen de fin des tâches du fichier pour l'ordonnancement premier arrivé, premier servi et pour plusieurs quantums. Interpréter les résultats : quelles tâches profitent du tourniquet ? Que se passe-t-il quand le quantum devient grand ? Pourquoi, en pratique, ne choisit-on pas un quantum aussi petit que possible ?

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Description du dossier

Le dossier fourni au candidat comporte les éléments suivants :

- une version PDF de l'énoncé ;
- un code source de départ `ordonnanceur.py` ;
- un fichier de données `taches.csv` (colonnes `nom` et `duree`).

Sujet n° 29 — Recherche dichotomique

► corrigé p. 1045

Fichiers à télécharger :
`maththibaud.github.io/informatique-en-autonomie/f/tpratique-29-dichotomie` — **archive .zip**

Durée de l'épreuve : 1 heure.

Déroulement de l'épreuve

- Cette situation d'évaluation comporte ce document ainsi que des fichiers de codes et de données présents sur l'ordinateur. Le candidat doit **agir en autonomie** et faire preuve d'initiative tout au long de l'épreuve.
- En cas de difficulté, le candidat peut solliciter l'examinateur. Des moments privilégiés sont indiqués sous la forme d'**appels professeur**. L'examinateur peut intervenir à tout moment s'il le juge utile.

On cherche une valeur `x` dans un tableau `tab` (une liste Python). La **recherche séquentielle**, fournie dans le fichier `dichotomie.py`, compare `x` à tous les éléments, l'un après l'autre.

Lorsque le tableau est **trié dans l'ordre croissant**, on peut faire beaucoup mieux avec la **recherche dichotomique** : on maintient deux indices `debut` et `fin` qui délimitent la zone où `x` peut encore se trouver (au départ, tout le tableau). Tant que cette zone n'est pas vide, on compare `x` à l'élément du milieu, d'indice `milieu = (debut + fin) // 2` :

- s'il est égal à `x`, on a trouvé ;
- s'il est plus petit que `x`, on continue dans la moitié droite (`debut = milieu + 1`) ;
- sinon, on continue dans la moitié gauche (`fin = milieu - 1`).

Si la zone devient vide (`debut > fin`), `x` n'est pas dans le tableau.

Question 1

Écrire le corps de la fonction `recherche_dichotomique(tab, x)` qui renvoie un indice où se trouve `x` dans le tableau trié `tab`, ou `-1` si `x` n'y est pas. La fonction utilisera une boucle `while`. Des tests sont fournis dans la fonction `test_recherche`.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 2

La fonction `comparaisons_sequentielle` renvoie le nombre d'éléments du tableau comparés à `x` par la recherche séquentielle. Compléter de même la fonction `comparaisons_dichotomique` (on compte un passage dans la boucle pour chaque élément du milieu examiné), puis exécuter `comparer(1000)` et `comparer(1000000)`. Comparer et expliquer les nombres obtenus.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 3

Le tableau peut contenir plusieurs fois la même valeur. La fonction `premiere_occurrence(tab, x)` doit renvoyer le **plus petit** indice où se trouve `x`, en effectuant elle aussi une recherche dichotomique, mais le test `test_premiere_occurrence` échoue. Trouver l'erreur et corriger la fonction, sans en faire une recherche séquentielle.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 4

Écrire le corps de la fonction **récursive** `recherche_rec(tab, x, debut, fin)` qui cherche `x` entre les indices `debut` et `fin` (inclus) du tableau trié `tab` et renvoie un indice où il se trouve, ou `-1`. Pour chercher dans tout le tableau, on appelle `recherche_rec(tab, x, 0, len(tab) - 1)`. Tester avec la fonction `test_recherche_rec`.

Description du dossier

Le dossier fourni au candidat comporte les éléments suivants :

- une version PDF de l'énoncé ;
- un code source de départ `dichotomie.py`.

Sujet n° 30 — Messages secrets

► corrigé p. 1048

Fichiers à télécharger : `maththibaud.github.io/informatique-en-autonomie/f/tpratique-30-messages-secrets` — **archive .zip**

Durée de l'épreuve : 1 heure.

Déroulement de l'épreuve

- Cette situation d'évaluation comporte ce document ainsi que des fichiers de codes et de données présents sur l'ordinateur. Le candidat doit **agir en autonomie** et faire preuve d'initiative tout au long de l'épreuve.
- En cas de difficulté, le candidat peut solliciter l'examinateur. Des moments privilégiés sont indiqués sous la forme d'**appels professeur**. L'examinateur peut intervenir à tout moment s'il le juge utile.

On numérote les lettres majuscules de `A` à `Z` par leur **rang** : `A` a le rang 0, `B` le rang 1, ..., `Z` le rang 25. En Python, le rang de la lettre `lettre` est `ord(lettre) - ord("A")`, et la lettre de rang `r` est `chr(r + ord("A"))`. Tous les textes de ce sujet sont en majuscules, sans accents.

Dans le **chiffre de César** de clé `k`, chaque lettre de rang `r` est remplacée par la lettre de rang `(r + k) % 26` : le décalage est circulaire (avec `k = 3`, `Z` devient `C`). Pour déchiffrer, on décale de `-k`.

Question 1

Écrire le corps de la fonction `cesar(message, k)` du fichier `crypto.py`. Les caractères qui ne sont pas des lettres de `A` à `Z` (espaces, ponctuation) restent inchangés. Par exemple, `cesar("BONJOUR", 3)` renvoie `"ERQMRXU"`. Des tests sont fournis dans la fonction `test_cesar`.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 2

Le fichier `message_secret.txt` contient un texte français chiffré par César, avec une clé inconnue. En français, la lettre la plus fréquente est presque toujours le `E`.

a. Écrire le corps de la fonction `occurrences(texte)`, qui renvoie un dictionnaire associant à chaque lettre présente dans `texte` son nombre d'apparitions (les autres caractères sont ignorés). Tester avec `test_occurrences`.

b. À l'aide de la fonction `lire_fichier` et de la fonction `lettre_plus_frequente` fournies, déterminer la clé utilisée, puis afficher le message en clair.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 3

Dans le **chiffre de Vigenère**, la clé est un mot, répété autant que nécessaire : la lettre d'indice `i` du message est décalée du **rang** de la lettre d'indice `i % len(cle)` de la clé. Par exemple, avec la clé `"CLE"`, la première lettre est décalée de 2, la deuxième de 11, la troisième de 4, la quatrième à nouveau de 2, etc. Ainsi, `NSIROCKS` chiffré avec la clé `CLE` donne `PDMTZGMD`. Dans les questions 3 et 4, les messages et les clés ne contiennent que des lettres majuscules, sans espace.

La fonction `vigenere(message, cle)` fournie ne donne pas ce résultat : le test `test_vigenere` échoue. Trouver l'erreur et corriger la fonction.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 4

Écrire le corps de la fonction `dechiffre_vigenere(message, cle)`, qui déchiffre un message chiffré par Vigenère avec la clé `cle`. Tester avec `test_dechiffre_vigenere`.

Expliquer ensuite pourquoi la méthode de la question 2 ne permet pas, telle quelle, de retrouver la clé d'un message chiffré par Vigenère.

Description du dossier

Le dossier fourni au candidat comporte les éléments suivants :

- une version PDF de l'énoncé ;
- un code source de départ `crypto.py` ;
- un fichier de données `message_secret.txt`.

Sujet n° 31 — Recherche d'un motif

► corrigé p. 1050

Fichiers à télécharger : `maththibaud.github.io/informatique-en-autonomie/f/tpratique-31-recherche-motif` — **archive .zip**

Durée de l'épreuve : 1 heure.

Déroulement de l'épreuve

- Cette situation d'évaluation comporte ce document ainsi que des fichiers de codes et de données présents sur l'ordinateur. Le candidat doit **agir en autonomie** et faire preuve d'initiative tout au long de l'épreuve.
- En cas de difficulté, le candidat peut solliciter l'examinateur. Des moments privilégiés sont indiqués sous la forme d'**appels professeur**. L'examinateur peut intervenir à tout moment s'il le juge utile.

On cherche toutes les positions où un **motif** (une chaîne de longueur `m`) apparaît dans un **texte** (une chaîne de longueur `n`). Une occurrence commence à la position `i` si `texte[i + j] == motif[j]` pour tout `j` de `0` à `m - 1`. Par exemple, `"AA"` apparaît aux positions `0`, `1` et `2` dans `"AAAA"` : les occurrences peuvent se chevaucher.

Le fichier `adn.txt` contient un brin d'ADN de 100 000 bases (lettres `A`, `C`, `G`, `T`). La fonction `lire_fichier` fournie dans `recherche.py` renvoie son contenu sous forme de chaîne.

Question 1

Dans la **méthode naïve**, on place le motif sous le texte à chaque position `i` possible, et on compare les caractères de gauche à droite jusqu'au premier désaccord. Écrire le corps de la fonction `recherche_naive(motif, texte)`, qui renvoie la liste de toutes les positions où commence une occurrence. Tester avec `test_recherche_naive`, puis déterminer combien de fois le motif `"GATTACA"` apparaît dans le brin d'ADN.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 2

L'algorithme de **Boyer-Moore-Horspool** compare le motif au texte de **droite à gauche**. Après chaque tentative, **réussie ou non**, il regarde le caractère `c` du texte aligné avec la **dernière** case du motif et décale le motif d'autant de cases que l'indique une **table de décalage** :

- chaque lettre du motif, **sauf la dernière**, est associée à sa distance à la fin du motif : la lettre d'indice `k` reçoit `m - 1 - k` (pour une lettre répétée, on garde la valeur la plus petite) ;
- une lettre absente de la table provoque un décalage de `m`.

Par exemple, la table de `"MOTIF"` est :

```
{"M": 4, "O": 3, "T": 2, "I": 1}
```

II

Écrire le corps de la fonction `table_decalage(motif)`, qui renvoie cette table sous forme de dictionnaire. Tester avec `test_table_decalage`.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 3

La fonction `horspool(motif, texte)` fournie programme cet algorithme, mais le test `test_horspool` échoue : certaines occurrences ne sont pas trouvées. Trouver l'erreur et corriger la fonction.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 4

Compléter les fonctions `comparaisons_naive` et `comparaisons_horspool`, qui renvoient le nombre de comparaisons de caractères effectuées par chacune des deux méthodes (on compte chaque comparaison entre un caractère du texte et un caractère du motif, y compris celle qui échoue). Tester avec `test_comparaisons`, puis exécuter `comparer()`. Commenter les résultats obtenus pour chacun des trois cas.

Description du dossier

Le dossier fourni au candidat comporte les éléments suivants :

- une version PDF de l'énoncé ;
- un code source de départ `recherche.py` ;
- un fichier de données `adn.txt`.

Sujet n° 32 — Le grand escalier

► corrigé p. 1053

Fichiers à télécharger : `maththibaud.github.io/informatique-en-autonomie/f/tpratique-32-grand-escalier` — **archive .zip**

Durée de l'épreuve : 1 heure.

Déroulement de l'épreuve

- Cette situation d'évaluation comporte ce document ainsi que des fichiers de codes et de données présents sur l'ordinateur. Le candidat doit **agir en autonomie** et faire preuve d'initiative tout au long de l'épreuve.
- En cas de difficulté, le candidat peut solliciter l'examinateur. Des moments privilégiés sont indiqués sous la forme d'**appels professeur**. L'examinateur peut intervenir à tout moment s'il le juge utile.

Pour monter au Rocher, on emprunte un long escalier. Chaque marche demande un certain **effort** (elle est plus ou moins haute, glissante…), donné par un entier. On part du sol, au pied de l'escalier, et l'on monte à chaque pas **d'une ou de deux marches**. On paie l'effort de chaque marche sur laquelle on pose le pied. Le sommet est situé juste au-dessus de la dernière marche : on l'atteint depuis la dernière ou l'avant-dernière marche, sans effort supplémentaire.

Les efforts sont rangés dans une liste `couts` : `couts[i]` est l'effort de la marche `i`, numérotée à partir de 0 (la plus basse). Par exemple, pour `couts = [10, 15, 20]`, le meilleur choix consiste à monter directement sur la marche 1 (deux marches d'un coup), puis à sauter au sommet : l'effort total vaut 15.

On note $E(i)$ l'effort minimal pour arriver sur la marche `i` (effort de la marche `i` compris). On a `E(0) = couts[0]`, `E(1) = couts[1]` et, pour $i \geqslant 2$:

```
E(i) = couts[i] + min(E(i - 1), E(i - 2))
```

L'effort minimal pour atteindre le sommet d'un escalier de n marches vaut alors `min(E(n - 1), E(n - 2))`.

Question 1

Justifier la relation donnant $E(i)$ en considérant la marche d'où l'on arrive sur la marche `i`. Écrire ensuite le corps de la fonction récursive `effort_rec(couts, i)` du fichier `escalier.py`, qui renvoie $E(i)$. Des tests sont fournis dans la fonction `test_effort_rec`.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

II

Question 2

Ajouter, en **première ligne** du corps de `effort_rec`, l'instruction `APPELS[0] = APPELS[0] + 1`, qui compte les appels de la fonction. La fonction fournie `compter_appels(n)` renvoie alors le nombre d'appels effectués par la version récursive pour un escalier de `n` marches. L'exécuter pour `n` valant 10, 20 et 25. Dessiner le début de l'arbre des appels de $E(4)$ et expliquer pourquoi le nombre d'appels augmente si vite. Peut-on utiliser cette version pour l'escalier du fichier `escalier.txt`, qui compte 100 marches ?

Attention : ne pas appeler `effort_rec` *sur un escalier de plus de* 30 *marches, le calcul ne se terminerait pas en un temps raisonnable.*

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 3

Écrire le corps de la fonction `effort_memo(couts, i, memo)`, qui renvoie elle aussi $E(i)$, mais mémorise dans le dictionnaire `memo` chaque valeur calculée, pour ne jamais la recalculer. Tester avec `test_effort_memo`, puis déterminer l'effort minimal pour monter l'escalier du fichier `escalier.txt` à l'aide des fonctions `lire_escalier` et `effort_total_memo`.

► **Appel professeur.** Appeler le professeur pour lui présenter votre réponse ou en cas de difficulté.

Question 4

Écrire le corps de la fonction `effort_tab(couts)`, qui calcule l'effort minimal pour atteindre le sommet **sans récursivité**, en remplissant une liste `e` du bas vers le haut de l'escalier (`e[i]` contient $E(i)$). Tester avec `test_effort_tab`.

La fonction `effort_glouton(couts)` fournie applique une stratégie gloutonne : poser toujours le pied sur la moins pénible des deux marches suivantes. Comparer son résultat à celui de `effort_tab` sur `[10, 15, 20]` et sur l'escalier du fichier, puis expliquer la différence.

Description du dossier

Le dossier fourni au candidat comporte les éléments suivants :

- une version PDF de l'énoncé ;
- un code source de départ `escalier.py` ;
- un fichier de données `escalier.txt`.

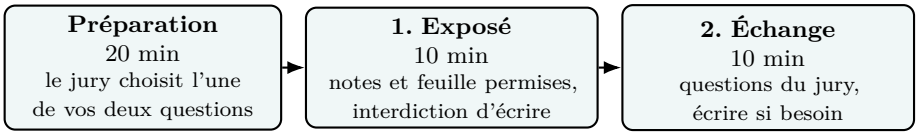
Préparer le Grand Oral

Le Grand Oral se passe **fin juin**, **après les épreuves écrites**. Vous y préparez **deux questions** adossées à vos enseignements de spécialité ; **au moins l'une des deux** doit mobiliser la **NSI**. Le jury en choisit une, vous exposez votre réponse, puis vous échangez avec lui. Ce n'est pas une récitation de cours : c'est **vous**, expliquant **pourquoi une question vous intéresse** et **comment la NSI y répond**.

Ce qu'est — et n'est pas — une bonne question

- Une bonne question est une **vraie question**, pas un titre de chapitre.
  Pas : « Les graphes ». *Plutôt :* « Comment un GPS trouve-t-il le chemin le plus court ? »
- Elle a un **enjeu** qu'on peut discuter (technique, mais aussi de société, d'éthique, d'histoire).
- Elle est **à votre portée** : vous savez y répondre *et* improviser dans l'échange.
- Elle vous **tient à cœur** : reliez-la à un projet, un stage, une envie d'orientation.

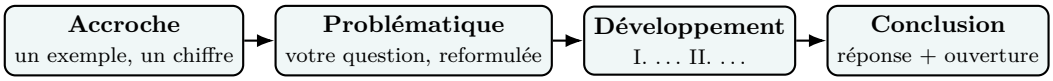
Le déroulement de l'épreuve



- **Préparation (20 min).** Vous remettez au jury vos deux questions ; il en choisit **une**. Vous mettez vos idées en ordre et écrivez vos **notes** ; vous pouvez aussi préparer une **feuille** (un schéma, quelques mots-clés) à montrer au jury : elle n'est **pas évaluée**.
- **Exposé (10 min).** Vous expliquez **pourquoi vous avez choisi** cette question, puis vous la développez et vous **y répondez**. Vous pouvez vous aider de vos notes et montrer votre feuille, mais vous n'avez **pas le droit d'écrire** pendant ces dix minutes.
- **Échange avec le jury (10 min).** Le jury approfondit, vous fait préciser, peut interroger sur d'autres points du programme de spécialité. Vous avez alors **le droit d'écrire** si besoin (compléter un schéma, refaire un calcul).

Vous passez donc **20 minutes** devant le jury, après 20 minutes de préparation (coefficient **8**). Il n'y a pas de temps à part pour parler de votre projet d'orientation : c'est dans l'exposé, en accroche ou en conclusion, que vous dites pourquoi la question compte pour vous. Un jury de deux professeurs vous évalue avec la grille présentée plus bas.

La structure de l'exposé (10 minutes)



- **Accroche** (1 min) : une situation concrète (« quand j'envoie un mot de passe sur un site... »), un chiffre qui étonne ; puis **pourquoi vous avez choisi** cette question (un projet, une lecture, une envie d'orientation).
- **Problématique** (1 min) : votre question, clairement énoncée, et son **enjeu**. Annoncez votre **plan en deux temps**.
- **Développement** (7 min) : deux parties d'environ 3 min 30. **Montrez un mécanisme**, pas seulement un vocabulaire : un exemple chiffré, un mini-algorithme déroulé, une idée expliquée pas à pas valent mieux que dix définitions. Puis regardez-le sous d'autres angles : ses défauts, une limite, un cas réel.
- **Conclusion** (1 min) : **répondez** vraiment à la question, puis **ouvrez** (une limite, un lien avec votre projet d'orientation).

Dix minutes, c'est long si l'on survole et court si l'on veut tout dire : **un seul** mécanisme bien expliqué, puis examiné sous plusieurs angles, vaut mieux que cinq notions énumérées. Le reste (détails, autres exemples) se garde pour l'échange. *Chronométrez-vous à voix haute : au premier essai, presque personne ne tombe juste.*

> **Remarque** — *Notes, feuille et écriture*
> Pendant la **préparation**, vous écrivez vos notes et, si vous le souhaitez, une feuille à montrer au jury (un schéma, des mots-clés) ; cette feuille n'est pas évaluée. Pendant l'**exposé**, vous pouvez consulter vos notes et montrer votre feuille, mais vous **n'écrivez plus rien**. Des notes réduites au plan, aux chiffres et aux valeurs de l'exemple vous laissent regarder le jury ; un texte entièrement rédigé pousse à le lire, et la qualité orale s'en ressent. Pendant l'**échange**, vous avez de nouveau **le droit d'écrire** si besoin : compléter un schéma (graphe, arbre, chronogramme), refaire un calcul pour répondre au jury.

> **Remarque**
> Le jury vous notera surtout sur votre capacité à **expliquer clairement**, à **réagir** à ses questions et à **prendre du recul**. Préparez donc, pour chaque question, **trois relances probables** du jury et votre réponse.

Construire ses deux questions : une méthode

Les deux questions portent sur vos deux spécialités, **chacune séparément** ou **les deux à la fois** (une question transversale, par exemple NSI et mathématiques). Pour chacune, suivez ces cinq étapes.

1. **Partir de vous.** Un objet que vous utilisez, une actualité, un stage, un métier visé : notez trois sujets qui vous intriguent vraiment.
2. **Trouver la NSI dedans.** Pour chaque sujet, listez les **notions du programme** qui l'expliquent (au moins un chapitre de Terminale). Si vous n'en trouvez pas, changez de sujet.
3. **Formuler une vraie question.** Commencez par « Comment... », « Pourquoi... » ou « Peut-on... ». Elle doit avoir un **enjeu** et appeler une **réponse** (pas une définition).
4. **La passer au crible.** Puis-je la traiter en 10 minutes, sans survoler ni délayer ? Ai-je **un mécanisme** précis à expliquer (un algorithme, une structure, un protocole) ? Puis-je imaginer **trois relances** du jury et y répondre ?
5. **Bâtir le plan et le lien.** Deux parties, un exemple déroulé, une conclusion qui répond ; puis une phrase qui dit pourquoi la question compte pour vous (votre projet, votre **orientation**).

Une question par grand thème du programme

Thème	Question possible	Notions à mobiliser
Histoire de l'informatique	« Pourquoi une machine imaginée sur le papier en 1936 est-elle encore le modèle de nos ordinateurs ? »	machine de Turing, programme-donnée, calculabilité
Structures de données	« Comment retrouver un nom parmi un million en une vingtaine d'étapes ? »	arbres binaires de recherche, hauteur, $\log_2 n$
Bases de données	« Comment une billetterie en ligne évite-t-elle de vendre deux fois la même place ? »	modèle relationnel, contraintes d'intégrité, SGBD et accès simultanés
Architectures, systèmes et réseaux	« Pourquoi un smartphone ne chauffe-t-il pas comme un ordinateur de bureau ? »	système sur puce, consommation, loi de Moore et ses limites
Langages et programmation	« Pourquoi existe-t-il des centaines de langages de programmation ? »	paradigmes (impératif, objet, fonctionnel), récursivité
Algorithmique	« Comment un correcteur orthographique devine-t-il le mot que je voulais écrire ? »	distance d'édition, programmation dynamique, coût

Exemple transversal (NSI et mathématiques) : « Pourquoi les nombres premiers protègent-ils nos paiements en ligne ? » (RSA, arithmétique modulaire, factorisation).

Banque de questions NSI (avec angle et notions)

Pour chaque question : la *problématique*, les *notions du programme* à mobiliser (chapitre entre parenthèses) et un *angle* pour ne pas réciter le cours.

Sécurité & société

- **« Comment deux machines qui ne se sont jamais parlé peuvent-elles partager un secret ? »**
  Notions : chiffrement symétrique vs asymétrique, échange de clé, HTTPS (*ch. 10 Cryptographie*, *ch. 9 Réseaux*).
  Angle : le paradoxe du cadenas (clé publique/privée) ; pourquoi je peux payer en ligne en confiance.
- **« Un mot de passe est-il vraiment secret dans une base de données ? »**
  Notions : SGBD, fonctions de hachage [au-delà du programme], sécurisation des accès (*ch. 6 Bases de données*, *ch. 10 Cryptographie*).
  Angle : pourquoi un site sérieux ne *connaît* pas votre mot de passe (il n'en stocke que l'empreinte).

Intelligence artificielle & données

- **« Une machine peut-elle deviner à quoi ressemble un chiffre manuscrit ? »**
  Notions : k plus proches voisins, distance, jeu d'entraînement (*Algorithmique 1re*, réinvesti).
  Angle : montrer l'algorithme sur 3 points, puis parler des **biais** (données d'entraînement).
- **« À qui appartiennent les données que je laisse en ligne ? »**
  Notions : bases de données, modèle relationnel, usage responsable des données, RGPD (*ch. 6 Bases de données*).
  Angle : croiser deux tables (jointure) suffit à ré-identifier une personne « anonyme ».

Algorithmique & efficacité

- **« Comment un GPS calcule-t-il le plus court chemin en une seconde ? »**
  Notions : graphes, parcours, plus court chemin, coût d'un algorithme (*ch. 8 Graphes*).
  Angle : la carte = un graphe ; dérouler une étape ; pourquoi on ne teste pas « tous les chemins ».
- **« Rendre la monnaie : la solution évidente est-elle toujours la meilleure ? »**
  Notions : algorithmes gloutons, optimalité, systèmes de pièces (*Gloutons 1re*).
  Angle : un contre-exemple ($\{1,6,10\}$ pour rendre 12) où la méthode « gloutonne » échoue.
- **« Pourquoi couper un problème en deux le rend-il plus rapide ? »**
  Notions : diviser pour régner, tri fusion, coût $n \log_2 n$, récursivité (*ch. 7 Diviser pour régner*).
  Angle : comparer, chiffres à l'appui, n^2 et $n \log_2 n$ quand n devient grand.
- **« Comment éviter de recalculer mille fois la même chose ? »**
  Notions : programmation dynamique, mémoïsation, coût en mémoire (*ch. 11 Programmation dynamique*).
  Angle : Fibonacci naïf (explosion) vs mémoïsé ; « la mémoire contre le temps ».

Les limites & l'histoire

- **« Existe-t-il des problèmes qu'aucun ordinateur ne pourra jamais résoudre ? »**
  Notions : calculabilité, décidabilité, problème de l'arrêt, programme-donnée (*ch. 13 Calculabilité*).
  Angle : l'idée de Turing (1936) ; sans formalisme, l'argument « si un tel programme existait… ».
- **« Y a-t-il des problèmes que l'on sait résoudre mais pas assez vite ? »**
  Notions : coût exponentiel, explosion combinatoire, P vs NP [au-delà du programme] (*Gloutons 1re*, *ch. 8 Graphes*, *ch. 13 Calculabilité*).
  Angle : le voyageur de commerce ; pourquoi « vérifier » est facile mais « trouver » est dur.

Machines & réseaux

- **« Comment un message arrive-t-il intact malgré des paquets perdus ? »**
  Notions : paquets, routage, accusés de réception, TCP (*ch. 9 Réseaux*, *Le Web 1re*).
  Angle : dérouler le bit alterné ; le lien avec les graphes (routage RIP/OSPF).
- **« Comment un ordinateur fait-il semblant de tout faire en même temps ? »**
  Notions : processus, ordonnancement, interblocage (*ch. 4 Processus*).
  Angle : le « chef d'orchestre » ; un chronogramme ; le risque d'interblocage.

La grille d'évaluation du jury

Le jury s'appuie sur une **grille indicative nationale** à cinq critères, chacun décrit sur quatre niveaux. Voici sa substance, reformulée.

Critère	Très insuffisant	Insuffisant	Satisfaisant	Très satisfaisant
Qualité orale (voix, présence)	On entend mal du début à la fin ; l'attention du jury n'est pas captée.	Plus audible au fil de l'épreuve, mais voix monotone ; vocabulaire pauvre ou approximatif.	Voix qui varie, parole assurée, vocabulaire adapté ; le candidat intéresse.	La voix porte le propos (débit, rythme, nuances) ; engagement net ; vocabulaire riche et précis.
Prise de parole en continu (l'exposé)	Phrases hachées, pauses et reprises ; ou phrases longues et mal construites.	Propos assez clair, mais schématique et répétitif.	Propos articulé et pertinent, phrases bien construites.	Propos fluide et efficace, qui exploite tout le temps imparti pour développer ses idées.
Qualité des connaissances	Connaissances floues ; pas de réponse aux questions, même avec de l'aide.	Des connaissances réelles, mais difficiles à mobiliser face aux questions.	Connaissances précises, mobilisées pour répondre, parfois après une relance.	Connaissances maîtrisées, utilisées à bon escient et exposées clairement.
Qualité de l'interaction (l'échange)	Réponses rares ou très brèves ; le jury porte seul l'échange.	Un début d'échange, qui reste limité.	Le candidat répond, réagit, se reprend et reformule en s'appuyant sur le jury.	Il s'empare des questions, prend des initiatives dans l'échange, exprime une pensée personnelle.
Argumentation (construction)	Sujet mal compris ; propos décousu, sans arguments.	Un début de démonstration, mais des manques ; structure fragile.	Démonstration construite, appuyée sur des arguments précis et pertinents.	Enjeux maîtrisés ; argumentation personnelle, construite et raisonnée.

Remarque — *Ce que cela change pour un exposé de NSI*

- *Connaissances* : le jury attend un vocabulaire **exact** (une file n'est pas une pile, un processus n'est pas un programme) et un mécanisme que vous savez **redérouler** pendant l'échange, en écrivant si besoin.
- *Argumentation* : un coût chiffré, un contre-exemple, une limite sont de vrais arguments ; « c'est plus rapide » sans preuve n'en est pas un.
- *Interaction* : si vous ne savez pas, dites-le, puis **raisonnez à voix haute** à partir de ce que vous savez : c'est valorisé.

La grille entre camarades

Pour s'entraîner toute l'année (en particulier avec l'exercice **« Expliquer en deux minutes »**, à la fin des exercices de chaque chapitre), on utilise une version simplifiée : **trois critères**, chacun relié à la grille du jury.

Grille entre camarades	Critère du jury	Oui	En partie	Pas encore
Clair — audible, posé ; chaque mot technique est expliqué	qualité orale, parole en continu	☐	☐	☐
Juste — c'est exact, et l'exemple montre vraiment l'idée	connaissances	☐	☐	☐
Construit — un fil conducteur, tenu dans le temps, sans lire	argumentation	☐	☐	☐
Un conseil pour la prochaine fois :				

Le cinquième critère, **l'interaction**, s'entraîne par la **question de relance** : après l'exposé, le camarade pose une question, et l'on y répond sans revenir à ses notes.

- **Évaluer, c'est observer** : cochez ce que vous avez entendu, pas ce que vous pensez du camarade.
- **Un seul conseil**, concret et réalisable (« donne ton exemple plus tôt », « définis *variant* avant de l'utiliser »).
- **Changez de rôle** à chaque fois : on apprend autant en écoutant qu'en parlant.

Les pièges à éviter

- **Réciter le cours** sans répondre à la question : le jury attend une *démonstration de compréhension*, pas un exposé.
- **Choisir une question trop large** (« l'intelligence artificielle ») : impossible à traiter en 10 minutes. Restreignez.
- **Choisir une question trop technique** que vous ne maîtrisez pas dans l'échange.
- **Oublier le « pourquoi vous »** : reliez la question à un projet, une lecture, une envie d'orientation.
- **Lire ses notes** : le plan, les chiffres et les valeurs de l'exemple suffisent. Vouloir tout rédiger pendant les 20 minutes de préparation, c'est manquer de temps, puis lire au lieu de parler.
- **Oublier de rendre vos idées visuelles** : pendant la préparation, tracez un schéma clair (graphe, arbre, chronogramme) sur la feuille que vous montrerez pendant l'exposé ; rien ne pourra y être ajouté avant l'échange, où vous pourrez le compléter.

Un exposé modèle

Voici l'exposé complet d'un élève sur la question **« Comment génère-t-on de l'aléatoire en informatique ? »**. Elle s'appuie sur le chapitre 10 (*Cryptographie* : clés secrètes, module `secrets`), sur la graine de `random.seed` rencontrée au chapitre 2 et sur l'opérateur `%` (reste de la division euclidienne). L'exposé suit la structure présentée plus haut : un seul mécanisme, déroulé avec de vraies valeurs, et une conclusion qui répond ; dit d'une voix posée, il dure dix minutes. Ne l'apprenez pas par cœur : servez-vous-en comme d'un **patron** pour vos propres questions. Les passages marqués [au-delà du programme] dépassent le programme : on peut les dire, à condition de savoir les expliquer si le jury creuse.

Le script minuté (10 minutes)

0:00 — Accroche (1 min)

« Dans le hall de l'entreprise Cloudflare, à San Francisco, un mur d'une centaine de lampes à lave est filmé en permanence. Ce n'est pas de la décoration : les images de ces bulles qui montent et descendent servent, avec d'autres sources, à fabriquer du hasard pour sécuriser des connexions sur Internet. Pourquoi une entreprise d'informatique aurait-elle besoin de lampes à lave ? Moi, je me suis posé la question en programmant un jeu de dés en Python. Quand j'écris `randint(1, 6)`, comment une machine qui ne fait qu'exécuter des instructions peut-elle « choisir » ? Et j'ai remarqué une chose étrange : quand je fixais la graine avec `random.seed(42)`, comme on l'avait fait en cours pour tester un programme, mes dés donnaient exactement les mêmes résultats à chaque lancement. Ce hasard-là avait de la mémoire. »

1:00 — Problématique et plan (1 min)

« D'où ma question : comment génère-t-on de l'aléatoire en informatique ? C'est presque un paradoxe. Un ordinateur est déterministe : avec les mêmes données, un programme donne toujours le même résultat. Produire de l'imprévu avec une machine prévisible semble donc impossible. Je vais vous montrer qu'il y a en fait deux réponses, selon l'usage. D'abord, comment on fabrique, par un simple calcul, des nombres qui ont l'air aléatoires, et pourquoi cela suffit pour un jeu ou une simulation. Ensuite, pourquoi ce même calcul devient dangereux dès qu'il s'agit de sécurité, et où l'ordinateur va alors chercher un vrai hasard. »

2:00 — I. Un calcul qui imite le hasard (3 min 30)

« La méthode la plus simple s'appelle un *générateur congruentiel linéaire*. On part d'un nombre, la **graine**, et on calcule chaque nombre à partir du précédent, toujours avec la même formule : a fois le nombre, plus c, modulo m, c'est-à-dire le reste de la division par m. Prenons $a = 5$, $c = 3$, $m = 16$ et la graine 7. Cinq fois sept plus trois, trente-huit ; divisé par seize, il reste six. Cinq fois six plus trois, trente-trois : il reste un. Puis viennent huit, onze, dix, cinq, douze, quinze… Sans la formule, cette suite a l'air désordonnée. Pour obtenir un nombre entre zéro et un, comme la fonction `random` de Python, il suffit de diviser par seize : six seizièmes, c'est 0,375.

Regardons maintenant ses défauts, car ils expliquent tout le reste. Premier défaut : la suite est entièrement fixée par la graine. Si je repars de sept, j'obtiens exactement les mêmes nombres : c'est ce qui arrivait à mes dés. Beaucoup de grands frères et de grandes sœurs l'ont vécu sans le savoir : sur les calculatrices TI-83, une réinitialisation remet la graine à zéro, et `rand` redonne alors toujours la même suite, 0,943, puis 0,908, puis 0,146… Toutes les calculatrices réinitialisées « tiraient au sort » les mêmes nombres. Mais c'est aussi une qualité : un scientifique peut rejouer une simulation à l'identique, un programmeur peut retrouver un bug qui n'apparaît qu'une fois sur mille.

Deuxième défaut : il n'y a que seize restes possibles, et chaque nombre ne dépend que du précédent. Dès qu'un nombre revient, toute la suite recommence. Ici, on retombe sur sept après seize nombres : la **période** vaut seize. Imaginez une ronde de seize cases, toujours parcourue dans le même ordre : c'est le schéma de ma feuille.

Troisième défaut, plus caché : la parité. Sept, impair ; six, pair ; un, impair ; huit, pair… Les nombres sont tour à tour impairs et pairs, sans aucune exception. Si je jouais à pile ou face avec la parité, j'obtiendrais pile, face, pile, face : moitié-moitié, mais parfaitement prévisible. La raison est simple : multiplier par cinq garde la parité, ajouter trois la change, et le reste de la division par seize, un nombre pair, la conserve.

Les vrais générateurs corrigent ces défauts avec de très grands nombres et des calculs plus élaborés. Celui de Python a une période qui s'écrit avec plus de six mille chiffres [au-delà du programme] : personne ne verra jamais la ronde recommencer. Pour mélanger une liste de lecture ou simuler un jeu de dés, c'est parfait. On parle de nombres *pseudo-aléatoires* : ils ont l'air aléatoires, mais ils sont calculés. »

5:30 — II. Quand le hasard doit être imprévisible (3 min 30)

« Le problème, c'est la sécurité. Dans le chapitre de cryptographie, on a vu que la solidité d'un chiffrement repose sur le secret de la clé : l'algorithme, lui, est public. Or une clé, c'est un grand nombre tiré au hasard. Si je la fabrique avec mon générateur, un attaquant qui connaît la formule et voit passer un seul nombre, par exemple six, calcule tous les suivants : un, huit, onze… Il n'a même pas besoin de casser le chiffrement. Et si la graine est l'heure de l'ordinateur, à la seconde près, il lui suffit d'essayer les 86 400 secondes d'une journée : c'est instantané pour une machine.

Ce n'est pas un cas d'école. En 2008, on a découvert qu'une version de la bibliothèque OpenSSL distribuée par Debian, une distribution Linux très utilisée, ne produisait, pour un type de clé donné, que quelques dizaines de milliers de clés différentes [au-delà du programme]. Un attaquant pouvait toutes les essayer. Le chiffrement était solide : c'est le hasard qui était faible.

Il faut donc une graine que personne ne peut deviner, et pour cela l'ordinateur sort du calcul : il va chercher du hasard dans le monde physique. Par exemple l'instant exact, à la microseconde près, où je frappe une touche ou bouge la souris, l'arrivée des données du disque ou du réseau, ou le bruit électrique d'un circuit du processeur prévu pour cela [au-delà du programme]. Aucune de ces mesures n'est parfaite, mais personne ne peut toutes les prévoir. Le système d'exploitation les accumule dans une réserve qu'on appelle l'**entropie**. Il en tire des graines, qu'un générateur conçu pour la cryptographie étire en autant de nombres qu'il faut, de telle sorte qu'en voyant ces nombres on ne puisse ni remonter à la graine, ni prévoir les suivants. Voilà le rôle des lampes à lave du début : une source de plus, très difficile à prévoir, pour alimenter cette réserve.

En Python, ces deux mondes correspondent à deux modules : `random`, pour simuler, rapide

et reproductible avec une graine ; `secrets`, pour les mots de passe et les clés, qui puise dans le hasard du système. La documentation de Python le dit elle-même : `random` ne doit pas servir pour la sécurité. »

9:00 — Conclusion et ouverture (1 min)

« Pour répondre à ma question : un ordinateur ne crée pas le hasard, il l'imite ou il va le chercher. Pour simuler, mélanger ou tirer au sort, un calcul déterministe qui a l'air aléatoire suffit, et sa prévisibilité devient même un avantage, puisqu'on peut rejouer une expérience. Pour protéger un secret, il faut au contraire de l'imprévisible : on le puise dans le monde physique, puis le calcul le fait fructifier. C'est ce mélange de physique, de mathématiques et de programmation qui m'attire : je voudrais poursuivre mes études en informatique et me spécialiser en cybersécurité, un domaine où, comme chez Debian, une seule graine trop facile à deviner suffit à ruiner le meilleur des chiffrements. »

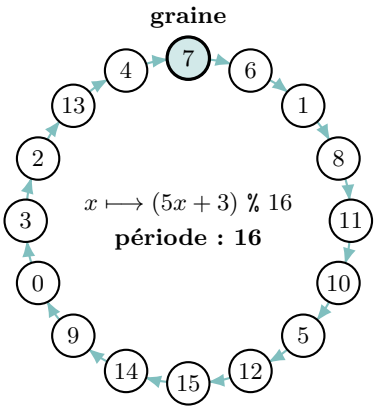
Les notes et la feuille préparées en 20 minutes

L'élève écrit des **notes** qui tiennent en une demi-page : le plan minuté, les valeurs de l'exemple et les chiffres à ne pas oublier, rien de rédigé.

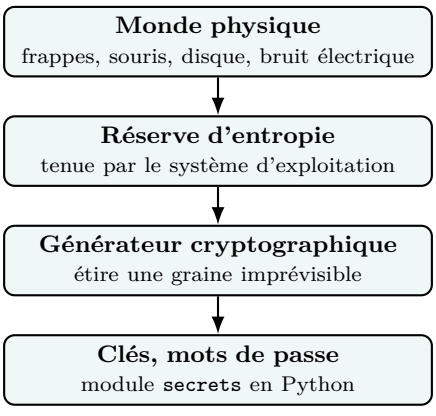
Ses notes

- **0:00 Accroche** : lampes à lave (Cloudflare) → mon jeu de dés, `random.seed(42)` = mêmes dés.
- **1:00 Question** : machine déterministe ⇒ hasard ? Plan : I. imiter ; II. aller chercher.
- **2:00 I.** $x \mapsto (5x+3)$ % 16, graine 7 : 6, 1, 8, 11, 10, 5, 12… ÷16 → 0,375. Trois défauts : graine (TI-83 réinitialisée : 0,943 ; 0,908 ; 0,146… ; rejouer : qualité) ; période 16 (ronde, *montrer la feuille*) ; parité qui alterne. Python : période > 6 000 chiffres. Pseudo-aléatoire.
- **5:30 II.** Clé = nombre au hasard. Un nombre vu ⇒ toute la suite ; graine = heure ⇒ 86 400 essais. Debian 2008. Entropie : touches, souris, disque, réseau, bruit → générateur crypto. `random` / `secrets`.
- **9:00 Conclusion** : imiter ou aller chercher ; cybersécurité.

Sur la **feuille** qu'il montrera au jury, il trace le schéma ci-dessous. Pendant l'exposé, il la montre quand il en parle (« voici la ronde de seize cases »), sans rien y ajouter ; pendant l'échange, il peut la compléter ou y refaire un calcul pour appuyer ses réponses.



I. Un calcul qui imite le hasard



II. Un hasard imprévisible

Remarque — *Pourquoi cet exposé fonctionne*

- Le mécanisme est **déroulé avec de vraies valeurs** ($5 \times 7 + 3 = 38$, reste 6) au lieu d'une suite de définitions.
- Chaque mot technique (graine, période, entropie) est **expliqué** au moment où il apparaît.
- Il **tient les dix minutes sans délayer** : un seul exemple, regardé sous trois angles (ses trois défauts), avec deux cas réels : les calculatrices TI-83, familières à beaucoup, et la faille de 2008, qui montre l'enjeu.
- La conclusion **répond** à la question ; le jeu de dés de l'accroche et le projet de cybersécurité de la fin donnent le « pourquoi moi ». Pour un projet en sciences, en économie ou en finance, le lien naturel serait la **simulation** : on y tire des millions de nombres pseudo-aléatoires pour estimer ce qu'on ne sait pas calculer directement.

Les relances probables du jury, et des réponses modèles

1. **« Votre ordinateur peut-il produire un vrai hasard ? »**
   « Par le calcul seul, non : un programme est déterministe, avec la même entrée il donne la même sortie. Mais l'ordinateur est relié au monde physique : il mesure des événements imprévisibles (instants des frappes, mouvements de la souris, bruit électrique) et les accumule dans sa réserve d'entropie. C'est ce hasard physique qui sert de graine aux générateurs de la cryptographie. »

2. **« Pourquoi un mot de passe généré avec `random` est-il dangereux ? »**
   « Parce que `random` est fait pour simuler, pas pour protéger. Ses nombres sont calculés à partir d'un état interne, et en observant assez de nombres produits, on peut reconstituer cet état et prévoir tous les suivants (pour le générateur de Python, 624 nombres consécutifs de 32 bits suffisent [au-delà du programme]). Si la graine a été fixée avec l'heure, il suffit même d'essayer 86 400 graines par jour. Pour un mot de passe, j'utiliserais le module `secrets`. »

3. **« Quelle est la période de votre générateur ? Peut-elle être plus courte ? »**
   « Elle vaut 16, le maximum possible : il n'y a que seize restes, de 0 à 15, et ils sortent tous une fois avant que 7 revienne. Mais tout dépend des paramètres : avec $c = 2$ au lieu de 3, la suite issue de 7 est 7, 5, 11, 9, 15, 13, 3, 1, puis de nouveau 7 : période 8, et jamais un nombre pair. On connaît des conditions sur a, c et m qui garantissent la période maximale [au-delà du programme]. »

4. **« Comment trouveriez-vous cette période avec un programme ? »**
   « Je note dans un dictionnaire le rang où chaque nombre apparaît pour la première fois ; au premier nombre déjà vu, l'écart entre les deux rangs donne la longueur du cycle. Attendre le retour de la graine ne suffit pas : avec $a = 4$, la suite 7, 15, 15, 15… n'y revient jamais. »

```python
def suivant(x, a, c, m):
    """Renvoie le nombre qui suit x."""
    return (a * x + c) % m

def periode(graine, a, c, m):
    """Renvoie la longueur du cycle sur lequel la suite finit par tourner."""
    rang = {}        # nombre -> rang de sa premiere apparition
    x = graine
    k = 0
    while x not in rang:
        rang[x] = k
        x = suivant(x, a, c, m)
        k = k + 1
    return k - rang[x]
```

`periode(7, 5, 3, 16)` renvoie `16`, `periode(7, 5, 2, 16)` renvoie `8` et `periode(7, 4, 3, 16)` renvoie `1`.

5. « Comment vérifier qu'une suite a l'air aléatoire ? »
« Par des tests statistiques. Le premier consiste à compter les fréquences : chaque valeur doit sortir à peu près aussi souvent que les autres. Mais cela ne suffit pas : avec mon générateur, la parité donne exactement moitié-moitié, et pourtant elle alterne sans exception. Il faut donc aussi tester les enchaînements. Et réussir ces tests ne prouve pas qu'une suite est imprévisible. »

6. « Avec votre générateur, comment simuler un dé à six faces ? »
« On pourrait prendre le reste de la division par six, plus un. Mais seize n'est pas un multiple de six : parmi les nombres de 0 à 15, ceux qui donnent les faces 1 à 4 sont trois chacun, ceux qui donnent 5 et 6 seulement deux. Le dé serait truqué : 3 chances sur 16 pour chacune des faces 1 à 4, 2 sur 16 pour 5 et pour 6. La solution : rejeter les nombres de 12 à 15 et passer au suivant ; il reste douze nombres, deux par face. »

Pendant l'échange : trois attitudes

- **Répondre d'abord en une phrase** (« Non, pas par le calcul seul »), puis justifier avec un exemple : le jury entend tout de suite que vous avez compris la question.
- **Écrire quand cela aide** : compléter la ronde sur votre feuille, recalculer un terme devant le jury. Un calcul fait en direct convainc plus qu'une affirmation.
- **Si vous ne savez pas, le dire, puis repartir de ce que vous savez** : « Je ne connais pas le détail du générateur de Python, mais il a forcément une graine et une période, puisque c'est un calcul sur une mémoire finie. »

À faire dès maintenant

☐ choisir **une** question NSI dans la banque (ou en inventer une proche) qui vous **motive** ;

☐ la reformuler en *une phrase*, avec un **enjeu** clair ;

☐ préparer **deux parties** et **un schéma** à montrer pendant l'exposé (et à compléter pendant l'échange) ;

☐ réduire ses **notes** à une demi-page (plan minuté, valeurs de l'exemple, chiffres) et **chronométrer** l'exposé : dix minutes ;

☐ lister **trois relances** probables du jury et vos réponses ;

☐ dire, en accroche ou en conclusion, pourquoi la question compte pour vous (votre **projet d'orientation**).

III

TROISIÈME PARTIE

Corrigés

CORRIGÉS DU CHAPITRE 1

1 Récursivité

CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Des appels qui s'empilent

↑énoncé p. 18

Exercice 1 IA IA IA *Une fonction en appelle une autre*

1. Affichage :

```
Debut fctB
fctB 0
fctB 1
Debut fctA
fctA 0
fctA 1
fctA 2
Fin fctA
Retour dans fctB
fctB 2
Fin fctB
```

2. Deux post-it : `fctA` (reprendre ligne 4, `i = 1`) au-dessus, `fctB` (reprendre ligne 12, `i = 2`) en dessous :

`fctA` — ligne 4, `i = 1`	*dessus du tas : s'exécute*
`fctB` — ligne 12, `i = 2`	*en pause*

3. `fctB` est **en pause** : elle attend, sans rien faire, que `fctA` se termine. Le post-it de `fctB` indique la ligne où reprendre (ligne 12 : « Retour dans fctB ») : l'exécution reprend **exactement** là où elle s'était arrêtée.

4. Oui : pour reprendre correctement, `fctB` doit retrouver *son* `i` (ici 2), puis afficher `fctB 2` et sortir de sa boucle. Les deux `i` sont des variables **distinctes** : chaque appel possède **ses propres variables locales**, que la boucle de `fctA` ne touche pas. C'est pourquoi on note les variables de chaque fonction sur son propre post-it.

Exercice 2 *Les poupées russes*

5. Affichage :

```
J'ouvre la poupee 3
J'ouvre la poupee 2
J'ouvre la poupee 1
Je referme la poupee 1
Je referme la poupee 2
Je referme la poupee 3
```

6. Trois post-it au plus (`ouvrir(3)`, `ouvrir(2)`, `ouvrir(1)`), au moment où l'on ouvre la poupée 1 : c'est le fond de la descente.

7. La ligne 2 (« J'ouvre ») s'exécute **à la descente**, dans l'ordre des appels $(3, 2, 1)$. La ligne 5 (« Je referme ») s'exécute **à la remontée**, quand on retire les post-it, donc dans l'ordre inverse $(1, 2, 3)$.

8. `ouvrir(1)` affiche « J'ouvre la poupee 1 » puis « Je referme la poupee 1 » (pas d'appel, car $1 > 1$ est faux). `ouvrir(10)` affiche 20 lignes : 10 ouvertures puis 10 fermetures.

Exercice 3 *Le miroir sans fin*

9. Le tas grandit d'un post-it à chaque étape et on n'en retire jamais : le jeu ne s'arrête pas.

10. L'ordinateur finit par manquer de place : Python s'arrête sur une erreur

```
RecursionError: maximum recursion depth exceeded
```

11. Le test `if n > 1` : quand `n` vaut `1`, on n'appelle plus `ouvrir`. Comme `n` diminue de 1 à chaque appel, on finit forcément par atteindre 1.

Exercice 4 *Mettre des mots*

12. Propositions possibles, puis mots du cours :

Notion	**Propositions fréquentes**	**Mot du cours**
le tas de post-it	tas, file, liste d'attente	**pile d'exécution** (ou pile d'appels)
poser / retirer le post-it du dessus	ajouter, enlever	**empiler** / **dépiler**
fonction qui s'appelle elle-même	fonction miroir, boucle	**fonction récursive**
cas où l'on ne s'appelle plus	la fin, le dernier	**condition d'arrêt** (**cas de base**)

On retire toujours le *dernier* posé : c'est une pile, et non une file (les piles seront étudiées au chapitre « structures linéaires »).

13. Quand une fonction en appelle une autre, la première est **mise en pause** et reprend là où elle s'était arrêtée. Les instructions écrites après l'appel s'exécutent **au retour**, dans l'ordre inverse des appels. Pour qu'une fonction qui s'appelle elle-même s'arrête, il faut un **cas où elle ne s'appelle plus**, et que chaque appel s'en **rapproche**.

1

CORRIGÉS DES EXERCICES

Remarque Plusieurs écritures sont souvent possibles : on donne **une** version simple.

Comprendre et analyser

Exercice 1 *Avant ou après l'appel ?* ↑énoncé p. 32

▶ **Solution.** `a(3)` affiche `3 2 1` ; `b(3)` affiche `1 2 3`.

Dans `a`, le `print` est *avant* l'appel récursif : il s'exécute « à la descente », dans l'ordre des appels $(3, 2, 1)$. Dans `b`, le `print` est *après* : il s'exécute « à la remontée », donc dans l'ordre inverse $(1, 2, 3)$.

Exercice 2 *Dérouler une trace* ↑énoncé p. 32

▶ **Solution.** Le `print(n)` étant *après* l'appel, il s'exécute à la remontée. L'appel `mystere(20)` affiche :

```
1 2 5 10 20
```

(descente : $20 \to 10 \to 5 \to 2 \to 1$ *; puis on affiche en remontant.)*

▷ **Erreur fréquente.** Répondre `20 10 5 2 1` : c'est l'ordre de la descente. Le `print` étant placé *après* l'appel récursif, chaque appel n'affiche qu'une fois l'appel suivant terminé.

1. voir ci-dessus.
2. elle affiche la suite des divisions entières par 2, de n jusqu'à 1, dans l'ordre **croissant**.
3. **variant :** l'entier `n`. Il est positif, et à chaque appel on passe à `n // 2`, strictement plus petit tant que `n > 1`. Il atteint donc 1 (condition d'arrêt) : la fonction **se termine**.

Exercice 3 *Cherchez l'erreur* ↑énoncé p. 33

▶ **Solution.** **(A)** il **manque la condition d'arrêt** : `somme` s'appelle indéfiniment (n devient négatif sans jamais arrêter) ⇒ `RecursionError`. Correction :

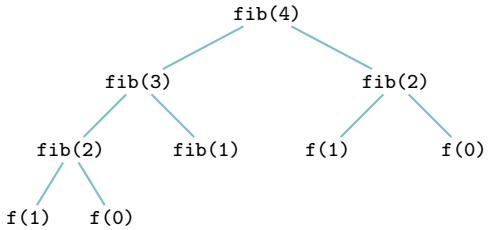
```
1 def somme(n):
2     if n == 0:                  # CAS DE BASE : enfin present
3         return 0
4     return n + somme(n - 1)     # APPEL RECURSIF, COMBINER
```

(B) la condition d'arrêt `n == 1` n'est **jamais atteinte** si on appelle `fact(0)` : on enchaîne `fact(-1)`, `fact(-2)`... ⇒ `RecursionError`. Correction : mettre `if n == 0` (ou `if n <= 1`).

▷ **Erreur fréquente.** Juger (B) correct parce qu'il contient une condition d'arrêt : il faut vérifier qu'elle est **atteinte** pour toutes les valeurs autorisées, ici aussi pour `n = 0`.

Exercice 4 *Compter les appels* ↑énoncé p. 33

▶ **Solution.** Arbre des appels de `fib(4)` :



En notant $C(n)$ le nombre d'appels, $C(0) = C(1) = 1$ et $C(n) = 1 + C(n-1) + C(n-2)$, d'où $C(2) = 3$, $C(3) = 5$, $C(4) = \mathbf{9}$.

▷ **Erreur fréquente.** Oublier l'appel initial, ou ne compter que les feuilles de l'arbre : on trouve 8 ou 5 au lieu de 9. Chaque nœud de l'arbre est un appel.

3. le nombre d'appels est multiplié par environ 1,6 quand n augmente de 1 (il fait plus que doubler quand n augmente de 2) : le coût est **exponentiel**, vite ingérable.
4. la **programmation dynamique** (mémoïsation) : on mémorise les valeurs déjà calculées (méthode étudiée au chapitre 11).

Écrire des fonctions récursives — les nombres

Exercice 5 *Le marchand de sable* ↑énoncé p. 33

▶ **Solution.** **Démarche :** condition d'arrêt $n = 0$: on affiche `"Je dors !"`. Sinon, on affiche *un* mouton et l'appel récursif sur `n - 1` affiche les autres ; `n` diminue de 1 à chaque appel, on se rapproche donc de la condition d'arrêt.

```
def moutons(n):
    if n == 0:                  # CAS DE BASE
        print("Je dors !")
    else:
        print("Encore un mouton")
        moutons(n - 1)          # APPEL RECURSIF : les autres moutons
```

Exercice 6 *Somme des nombres impairs* ↑énoncé p. 33

▶ **Solution.** **Démarche :** le n-ième nombre impair est $2n - 1$ (1, 3, 5 pour $n = 1, 2, 3$). Condition d'arrêt : `somme_impairs(0)` $= 0$ (aucun terme) ; on se rapproche de 0 en appelant `somme_impairs(n - 1)` ; on combine en ajoutant le dernier impair, car $1 + 3 + \cdots + (2n-1) = (1 + 3 + \cdots + (2n-3)) + (2n-1)$.

```
def somme_impairs(n):
    if n == 0:                                  # CAS DE BASE : aucun terme
        return 0
    return (2 * n - 1) + somme_impairs(n - 1)   # APPEL RECURSIF, COMBINER

for n in range(1, 7):
    print(somme_impairs(n))
```

On obtient `1, 4, 9, 16, 25, 36` : ce sont les **carrés**. La somme des n premiers impairs vaut n^2 (on passe d'un carré de côté $n-1$ au carré de côté n en ajoutant une « équerre » de $2n-1$ cases).

▷ **Erreur fréquente.** Écrire `return n + somme_impairs(n - 1)` en recopiant la somme des entiers du cours : on additionne alors $1 + 2 + \cdots + n$ (`somme_impairs(3)` renverrait `6`). Le terme ajouté doit être le n-ième impair, `2 * n - 1`.

Exercice 7 *Multiplier avec des additions* ↑énoncé p. 33

▶ **Solution.** **Démarche :** condition d'arrêt $a \times 0 = 0$; appel récursif sur `n - 1` ; on combine en ajoutant `a`, car $a \times n = a + a \times (n-1)$. C'est la structure de `puissance` du cours, avec une addition à la place de la multiplication.

```
def produit(a, n):
    if n == 0:                  # CAS DE BASE : a x 0 = 0
        return 0
    return a + produit(a, n - 1)  # APPEL RECURSIF, COMBINER : + a

print(produit(7, 6))      # 42
print(produit(1000, 3))   # 3000
```

`produit(7, 6)` appelle `produit(7, 5)`, puis `produit(7, 4)`, …, jusqu'à `produit(7, 0)` : 7 appels en tout, et plus généralement $n+1$ appels. Pour 3×1000, `produit(1000, 3)` ne fait que 4 appels, contre 1 001 pour `produit(3, 1000)` ; ce dernier dépasse même la profondeur maximale réglée par défaut dans Python (environ 1 000 appels imbriqués) et provoque un `RecursionError`. Il vaut donc mieux faire porter la récursion sur le plus petit des deux nombres.

▷ **Erreur fréquente.** Prendre pour condition d'arrêt `if n == 1: return a` : le résultat est juste pour $n \geqslant 1$, mais l'appel `produit(7, 0)` ne rencontre jamais la condition d'arrêt et provoque un `RecursionError`.

Exercice 8 *Compter les chiffres* ↑énoncé p. 33

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert nombre_de_chiffres(34126) == 5    # cas normal
assert nombre_de_chiffres(7) == 1        # cas limite : un seul chiffre (cas de base)
assert nombre_de_chiffres(1000) == 4     # cas piege : des zeros a la fin
```

Démarche : condition d'arrêt : un nombre à un seul chiffre (`n < 10`) ; `n // 10` supprime le dernier chiffre, ce qui rapproche de la condition d'arrêt ; on combine en ajoutant 1 pour le chiffre supprimé.

```
def nombre_de_chiffres(n):
    if n < 10:                              # CAS DE BASE : un seul chiffre
        return 1
    return 1 + nombre_de_chiffres(n // 10)   # APPEL RECURSIF, COMBINER : + 1
```

▷ **Erreur fréquente.** Écrire `return nombre_de_chiffres(n // 10)` sans le `1 +` : on ne combine pas le résultat de l'appel, et la fonction renvoie toujours 1.

Exercice 9 *Compter les bits à 1* ↑énoncé p. 34

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert nombre_de_bits(10) == 2       # cas normal
assert nombre_de_bits(0) == 0        # cas limite : le cas de base
assert nombre_de_bits(256) == 1      # cas piege : une puissance de 2, un seul 1
```

Démarche : même idée qu'à l'exercice précédent, mais en base 2. Condition d'arrêt `n == 0` (aucun bit à 1) ; `n // 2` enlève le dernier bit ; on combine en ajoutant ce dernier bit `n % 2`, qui vaut 1 ou 0.

```
def nombre_de_bits(n):
    if n == 0:                              # CAS DE BASE : aucun bit a 1
        return 0
    return (n % 2) + nombre_de_bits(n // 2)   # APPEL RECURSIF, COMBINER
```

Exercice 10 *PGCD (algorithme d'Euclide)* ↑énoncé p. 34

▶ **Solution.** **Démarche :** on traduit les deux égalités de l'énoncé : $\text{pgcd}(a, 0) = a$ donne la condition d'arrêt, $\text{pgcd}(a, b) = \text{pgcd}(b,\ a \,\%\, b)$ l'appel récursif. Rien à combiner : on renvoie directement le résultat de l'appel.

```
def pgcd(a, b):
    if b == 0:                  # CAS DE BASE : pgcd(a, 0) = a
        return a
    return pgcd(b, a % b)       # APPEL RECURSIF (rien a combiner)
```

Variant : `b`, *qui décroît strictement (*`a % b < b`*) jusqu'à* 0.

Exercice 11 *Une suite récurrente double* ↑énoncé p. 34

▶ **Solution.** **Démarche :** deux conditions d'arrêt ($n = 0$ et $n = 1$) ; deux appels récursifs, sur $n - 1$ et $n - 2$ (les paramètres `a` et `b` sont transmis tels quels) ; on combine leurs résultats avec la formule de l'énoncé.

```
def suite(n, a, b):
    if n == 0:                  # CAS DE BASE (1er)
        return a
    if n == 1:                  # CAS DE BASE (2e)
        return b
    return 3 * suite(n - 1, a, b) + 2 * suite(n - 2, a, b) + 5   # APPEL RECURSIF x 2, COMBINER
```

Exercice 12 *Suite de Syracuse* ↑énoncé p. 34

▶ **Solution.** **Démarche :** on affiche d'abord le terme reçu. Condition d'arrêt : $u = 1$, on n'appelle plus. Sinon, un seul appel récursif, sur le terme suivant ($u/2$ ou $3u+1$). Rien à combiner : la fonction affiche, elle ne renvoie rien. *(Ici, on ne connaît pas de variant : c'est justement la conjecture.)*

```
def syracuse(u):
    print(u)
    if u > 1:                   # sinon CAS DE BASE : u == 1, on s'arrete
        if u % 2 == 0:          # TESTER : u pair ?
            syracuse(u // 2)    # APPEL RECURSIF
        else:
            syracuse(3 * u + 1)  # APPEL RECURSIF
```

Exercice 13 *Exponentiation rapide* ↑énoncé p. 34

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert puissance_rapide(2, 10) == 1024   # cas normal
assert puissance_rapide(5, 0) == 1       # cas limite : le cas de base
assert puissance_rapide(-2, 3) == -8     # cas piege : x negatif et n impair
```

Si `n` est pair, un seul appel sur `n // 2` suffit (on élève au carré).

```
def puissance_rapide(x, n):
    if n == 0:                          # CAS DE BASE
        return 1
    if n % 2 == 0:                      # TESTER : n pair ?
        p = puissance_rapide(x, n // 2)  # APPEL RECURSIF : un seul
        return p * p                    # COMBINER : au carre
    return x * puissance_rapide(x, n - 1)   # APPEL RECURSIF, COMBINER : fois x
```

Chaînes de caractères et listes

Exercice 14 *Compter une lettre* ↑énoncé p. 34

▶ **Solution.** **Démarche :** condition d'arrêt : la chaîne vide (aucune occurrence) ; on se rapproche en appelant sur le reste `s[1:]` ; on combine en ajoutant 1 si le premier caractère est `c`, rien sinon.

```
def compter(s, c):
    if s == "":                     # CAS DE BASE : chaine vide
        return 0
    if s[0] == c:                   # TESTER : le premier caractere
        return 1 + compter(s[1:], c)  # APPEL RECURSIF, COMBINER : + 1
    return compter(s[1:], c)        # APPEL RECURSIF
```

Autre méthode (itérative) : on parcourt la chaîne et on incrémente un compteur.

```
def compter(s, c):
    nb = 0                      # INITIALISER
    for lettre in s:            # PARCOURIR
        if lettre == c:         # TESTER
```

```
            nb += 1                 # METTRE A JOUR
    return nb                       # RENVOYER
```

Exercice 15 *Inverser une chaîne* ↑énoncé p. 34

▶ **Solution.** **Démarche :** condition d'arrêt : la chaîne vide, qui est sa propre inverse ; appel récursif sur le reste `s[1:]` ; on combine en plaçant le premier caractère *à la fin* de l'inverse du reste.

```
def inverser(s):
    if s == "":                     # CAS DE BASE : chaine vide
        return ""
    return inverser(s[1:]) + s[0]   # APPEL RECURSIF, COMBINER : s[0] a la fin
```

Autre méthode (itérative) : on parcourt la chaîne et on place chaque caractère *devant* ceux déjà lus.

```
def inverser(s):
    resultat = ""                   # INITIALISER
    for lettre in s:                # PARCOURIR
        resultat = lettre + resultat  # METTRE A JOUR : lettre devant
    return resultat                 # RENVOYER
```

Exercice 16 *Palindrome (contexte ADN)* ↑énoncé p. 35

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert palindrome("GATTACATTAG")    # cas normal
assert palindrome("")               # cas limite : chaine vide (cas de base)
assert not palindrome("GATCAG")     # cas piege : bords egaux, mais pas le milieu
```

Démarche : condition d'arrêt : une chaîne de moins de deux caractères est un palindrome ; si les deux bords diffèrent, ce n'en est pas un ; sinon, on recommence sur le milieu `s[1:-1]`, plus court de deux caractères (c'est la fonction du cours).

```
def palindrome(s):
    if len(s) < 2:              # CAS DE BASE : 0 ou 1 caractere
        return True
    if s[0] != s[-1]:           # TESTER : les deux bords
        return False
    return palindrome(s[1:-1])  # APPEL RECURSIF : sans les bords
```

Remarque : `palindrome("GATTACATTAG")` *renvoie bien* `True` *(la chaîne se lit pareil dans les deux sens).*

Exercice 17 *Somme et maximum d'une liste* ↑énoncé p. 35

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert maximum([3, 9, 4]) == 9          # cas normal
assert maximum([5]) == 5                # cas limite : un seul element (cas de base)
assert maximum([-7, -2, -5]) == -2      # cas piege : que des negatifs
```

Démarche : condition d'arrêt : la liste à **un seul** élément (la liste est non vide) ; appel récursif sur le reste `tab[1:]` ; on combine `tab[0]` avec le résultat obtenu pour le reste (addition, ou comparaison pour le maximum).

```
def somme_liste(tab):
    if len(tab) == 1:                   # CAS DE BASE : un seul element
        return tab[0]
    return tab[0] + somme_liste(tab[1:])  # APPEL RECURSIF, COMBINER

def maximum(tab):
    if len(tab) == 1:                   # CAS DE BASE : un seul element
        return tab[0]
    m = maximum(tab[1:])                # APPEL RECURSIF : maximum du reste
    if tab[0] > m:                      # COMBINER : comparer tab[0] a m
        return tab[0]
    return m
```

▷ **Erreur fréquente.** Écrire `if tab[0] > maximum(tab[1:])`, puis `return maximum(tab[1:])` : le résultat est juste, mais chaque niveau lance deux fois le même appel, et le coût devient exponentiel (plus de 1 000 appels pour une liste croissante de 10 éléments). On range le résultat de l'appel dans une variable.

Exercice 18 *Recherche dans un tableau* ↑énoncé p. 35

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert appartient(4, [7, 4, 1], 0)          # cas normal
assert not appartient(7, [7, 4, 1], 3)      # cas limite : i == len(t), rien a examiner
assert not appartient(7, [7, 4, 1], 1)      # cas piege : present, mais avant l'indice i
```

Démarche : deux conditions d'arrêt : on a dépassé la fin du tableau (`i == len(t)` : la valeur est absente) ou `t[i]` est la valeur cherchée. Sinon, on se rapproche de la fin du tableau en appelant avec `i + 1`.

```
def appartient(v, t, i):
    if i == len(t):                 # CAS DE BASE : fin du tableau, absent
        return False
    if t[i] == v:                   # CAS DE BASE : trouve
        return True
    return appartient(v, t, i + 1)  # APPEL RECURSIF : a partir de i + 1
```

Exercice 19 *Recherche dichotomique récursive* ↑énoncé p. 35

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
t = [2, 5, 8, 12, 16]
assert dicho(t, 12, 0, 4) == 3          # cas normal
assert dicho([], 5, 0, -1) == -1        # cas limite : tableau vide
assert dicho(t, 9, 0, 4) == -1          # cas piege : absent, entre deux valeurs
```

On compare au milieu, et on ne garde qu'une moitié.

```
def dicho(t, v, gauche, droite):
    if gauche > droite:                         # CAS DE BASE : zone vide, absent
        return -1
    milieu = (gauche + droite) // 2             # DIVISER
    if t[milieu] == v:                          # TESTER
        return milieu
    if t[milieu] < v:                           # TESTER : quelle moitie garder ?
        return dicho(t, v, milieu + 1, droite)  # APPEL RECURSIF : moitie droite
    return dicho(t, v, gauche, milieu - 1)      # APPEL RECURSIF : moitie gauche
```

Dénombrement et combinatoire

Exercice 20 *Mots de A et B* ↑énoncé p. 35

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert frises(2) == ['AA', 'AB', 'BA', 'BB']    # cas normal
assert frises(0) == ['']                        # cas limite : un seul mot, le mot vide
assert len(frises(3)) == 8                      # cas piege : 2 ** 3 mots, et non 2 * 3
```

Démarche : condition d'arrêt : il existe un seul mot de longueur 0, le mot vide (d'où la liste `[""]`) ; appel récursif sur `n - 1` ; on combine en prolongeant chaque mot obtenu par un `A`, puis par un `B`.

```
def frises(n):
    if n == 0:                      # CAS DE BASE : le mot vide
        return [""]
    resultat = []                   # INITIALISER
    for mot in frises(n - 1):       # APPEL RECURSIF, puis PARCOURIR ses mots
        resultat.append(mot + "A")  # COMBINER
        resultat.append(mot + "B")  # COMBINER
    return resultat                 # RENVOYER
```

Test : `frises(2)` *renvoie* `['AA', 'AB', 'BA', 'BB']`.

Exercice 21 *Décompositions en 1 et 2* ↑énoncé p. 35

▶ Solution. **Démarche :** on construit la décomposition au fur et à mesure dans `debut` : elle continue soit par `1` (il reste `n - 1`), soit par `2` (il reste `n - 2`, si `n >= 2`). Condition d'arrêt : il ne reste plus rien à décomposer (`n == 0`), on affiche `debut`.

```
def decoupe(n, debut=""):
    if n == 0:                          # CAS DE BASE : plus rien a decomposer
        print(debut)
    else:
        if debut != "":
            debut = debut + " + "
        decoupe(n - 1, debut + "1")     # APPEL RECURSIF : on continue par 1
        if n >= 2:
            decoupe(n - 2, debut + "2") # APPEL RECURSIF : ou par 2
```

Le nombre de décompositions de `n` *est* u_{n+1}*, où* (u_n) *est la suite de* ***Fibonacci*** ($u_0 = 0$, $u_1 = 1$) *: on trouve* $1, 2, 3, 5, 8\ldots$ *pour* $n = 1, 2, 3, 4, 5$.

La valeur par défaut `""` est sans danger : une chaîne ne se modifie pas, `debut + "1"` fabrique une **nouvelle** chaîne. Avec une liste par défaut (`debut=[]`) remplie par `append`, la même liste serait partagée par tous les appels `decoupe(...)` lancés par l'utilisateur : le deuxième appel repartirait du contenu laissé par le premier. On écrirait alors `debut=None`, puis `if debut is None: debut = []` (voir le cours).

Exercice 22 *Triangle de Pascal* ↑énoncé p. 36

▶ Solution. *Appels attendus, écrits avant de coder :*

```
assert pascal(4, 2) == 6     # cas normal
assert pascal(0, 0) == 1     # cas limite : la premiere ligne
assert pascal(5, 5) == 1     # cas piege : p == n, le bord droit du triangle
```

Démarche : on traduit la définition ligne à ligne : $p = 0$ ou $n = p$ donnent les conditions d'arrêt ; la somme des deux coefficients de la ligne $n - 1$ donne les deux appels récursifs (n diminue de 1 à chaque appel).

```
def pascal(n, p):
    if p == 0 or n == p:                          # CAS DE BASE : les bords
        return 1
    return pascal(n - 1, p - 1) + pascal(n - 1, p)  # APPEL RECURSIF x 2, COMBINER
```

Défi. Pour chaque ligne `n`, on construit une chaîne avec les valeurs `pascal(n, p)`, `p` allant de `0` à `n`, puis on l'affiche :

```
for n in range(6):
    ligne = ""
    for p in range(n + 1):
        ligne += str(pascal(n, p)) + " "
    print(ligne)
```

On obtient les lignes `1`, `1 1`, `1 2 1`, `1 3 3 1`, `1 4 6 4 1`, `1 5 10 10 5 1`.

Exercice 23 *Les tours de Hanoï* ↑énoncé p. 36

▶ Solution. Déplacer `n` disques = déplacer `n-1` disques de côté, bouger le grand, puis ramener les `n-1`.

```
def hanoi(n, depart, inter, arrivee):
    if n > 0:                                  # sinon CAS DE BASE : rien a faire
        hanoi(n - 1, depart, arrivee, inter)   # APPEL RECURSIF : n - 1 disques de cote
        print(depart, "->", arrivee)           # le grand disque
        hanoi(n - 1, inter, depart, arrivee)   # APPEL RECURSIF : n - 1 disques dessus
```

Nombre de déplacements : $H(n) = 2\,H(n-1) + 1$ *avec* $H(0) = 0$*, soit* $H(n) = 2^n - 1$.

Récursif ou itératif ?

Exercice 24  *Traduire dans les deux sens* ↑énoncé p. 36

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert factorielle(5) == 120      # cas normal
assert factorielle(0) == 1        # cas limite : la boucle ne tourne pas
assert factorielle(1) == 1        # cas piege : range(2, 2) est vide, lui aussi
```

1. Versions itératives. Démarche : un accumulateur initialisé avec la valeur de la condition d'arrêt (`1` pour un produit, `0` pour une somme), mis à jour à chaque tour d'une boucle `for`.

```
def factorielle(n):
    p = 1                         # INITIALISER : la valeur du cas de base
    for k in range(2, n + 1):     # PARCOURIR
        p = p * k                 # METTRE A JOUR
    return p                      # RENVOYER

def somme(n):
    s = 0                         # INITIALISER
    for k in range(n + 1):        # PARCOURIR
        s = s + k                 # METTRE A JOUR
    return s                      # RENVOYER

def puissance(x, n):
    p = 1                         # INITIALISER
    for _ in range(n):            # PARCOURIR
        p = p * x                 # METTRE A JOUR
    return p                      # RENVOYER
```

2. Version récursive de `f` (qui calcule la **somme des éléments** d'une liste). **Démarche :** condition d'arrêt : la liste vide, de somme 0 ; on combine le premier élément avec la somme du reste `t[1:]` :

```
def f(t):
    if t == []:                   # CAS DE BASE : liste vide
        return 0
    return t[0] + f(t[1:])        # APPEL RECURSIF, COMBINER
```

3. Palindrome itératif (deux indices qui se rapprochent) :

```
def est_palindrome(s):
    i = 0                         # INITIALISER
    j = len(s) - 1                # INITIALISER
    while i < j:                  # PARCOURIR
        if s[i] != s[j]:          # TESTER
            return False          # RENVOYER
        i = i + 1                 # METTRE A JOUR
        j = j - 1                 # METTRE A JOUR
    return True                   # RENVOYER
```

La version récursive colle mieux à la définition ; l'itérative évite d'empiler un appel par caractère.

Dessins récursifs (module Turtle)

Exercice 25 *Carrés imbriqués* ↑énoncé p. 37

▶ Solution. Un carré, un petit décalage vers l'intérieur, puis on recommence en plus petit.

```
import turtle as t

def carres(cote):
    if cote < 10:                # CAS DE BASE : trop petit, on s'arrete
        return
    for _ in range(4):
        t.forward(cote)
        t.left(90)
    d = cote * 0.15              # se decaler vers l'interieur
    t.penup()
    t.forward(d)
    t.left(90)
    t.forward(d)
    t.right(90)
    t.pendown()
    carres(cote * 0.7)           # APPEL RECURSIF : un carre plus petit
```

Exercice 26 *Le flocon de Koch* ↑énoncé p. 37

▶ Solution. On réutilise `koch` du cours et on assemble trois côtés.

```
def flocon(taille, n):
    for _ in range(3):
        koch(taille, n)
        t.right(120)
```

Exercice 27 *Le triangle de Sierpiński* ↑énoncé p. 37

▶ Solution. Condition d'arrêt `n == 0` : on trace le triangle plein. Sinon, on recommence sur les trois « coins » (les triangles formés avec les milieux des côtés).

```
def milieu(p, q):
    return ((p[0] + q[0]) / 2, (p[1] + q[1]) / 2)

def tracer_triangle(a, b, c):
    t.penup()
    t.goto(a)
    t.pendown()
    t.begin_fill()      # triangle plein
    t.goto(b)
    t.goto(c)
    t.goto(a)
    t.end_fill()

def sierpinski(a, b, c, n):
    if n == 0:                                          # CAS DE BASE
        tracer_triangle(a, b, c)
    else:
        sierpinski(a, milieu(a, b), milieu(a, c), n - 1)  # APPEL RECURSIF : coin a
        sierpinski(milieu(a, b), b, milieu(b, c), n - 1)  # APPEL RECURSIF : coin b
        sierpinski(milieu(a, c), milieu(b, c), c, n - 1)  # APPEL RECURSIF : coin c
```

Exercice 28 *La courbe de Hilbert* ↑énoncé p. 37

▶ **Solution.** Version classique (`angle` vaut 90 ou −90 selon le sens) :

```
def hilbert(n, angle, longueur):
    if n == 0:                          # CAS DE BASE : rien a tracer
        return
    t.left(angle)
    hilbert(n - 1, -angle, longueur)    # APPEL RECURSIF : copie pivotee
    t.forward(longueur)
    t.right(angle)
    hilbert(n - 1, angle, longueur)     # APPEL RECURSIF
    t.forward(longueur)
    hilbert(n - 1, angle, longueur)     # APPEL RECURSIF
    t.right(angle)
    t.forward(longueur)
    hilbert(n - 1, -angle, longueur)    # APPEL RECURSIF : copie pivotee
    t.left(angle)

# appel : hilbert(4, 90, 10)
```

Vers le bac

Exercice 29 *Analyser et écrire des fonctions récursives (d'après Sujet zéro 2023, sujet B)* ↑énoncé p. 38

Barème indicatif (sur 3) : Q1.a : 0,5 ; Q1.b : 0,5 ; Q2 : 0,5 ; Q3.a : 0,5 ; Q3.b : 0,25 ; Q4 : 0,75

1. **a.** ▶ **Solution.** Une fonction **récursive** est une fonction qui **s'appelle elle-même** dans son propre corps ; elle doit posséder une **condition d'arrêt** qui garantit qu'elle se termine.

b. ▶ **Solution.** Chaque appel se fait avec `n - 1`. Après l'affichage de `0`, l'appel suivant est `compte_rebours(-1)` : la condition `n >= 0` est alors **fausse**, donc aucun nouvel appel n'est lancé, et la fonction s'arrête.

2. ▶ **Solution.** **Démarche :** on traduit la définition : $0! = 1$ donne la condition d'arrêt, et $n! = n \times (n-1)!$ l'appel récursif.

```
def fact(n):
    if n == 0:                  # CAS DE BASE
        return 1
    else:
        return n * fact(n - 1)  # APPEL RECURSIF, COMBINER : x n
```

3. **a.** ▶ **Solution.** Le `print(n)` est exécuté *avant* l'appel récursif, pour `n = 3, 2, 1` (pas pour `0`). La console affiche :

```
3
2
1
```

▷ **Erreur fréquente.** Ajouter `0` à la fin de l'affichage : le `print` est dans le `else`, il n'est pas exécuté pour `n == 0`.

b. ▶ **Solution.** `res` reçoit $0 + 1 + 2 + 3 = \mathbf{6}$.

4. ▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert somme_entiers(4) == 10    # cas normal
assert somme_entiers(0) == 0     # cas limite
assert somme_entiers(1) == 1     # cas piege : n est compris, range(n + 1)
```

Démarche : un accumulateur initialisé à 0, auquel une boucle `for` ajoute chaque entier de 0 à `n`.

```
def somme_entiers(n):
    s = 0                       # INITIALISER
    for k in range(n + 1):      # PARCOURIR : de 0 a n inclus
        s = s + k               # METTRE A JOUR
    return s                    # RENVOYER
```

Exercice 30 IA IA IA *Les scores au rugby (d'après Polynésie 2024, jour 1)* ↑énoncé p. 38

Barème indicatif (sur 4) : Q1 : 0,5 ; Q2 : 1 ; Q3 : 1 ; Q4 : 0,5 ; Q5 : 1

1. ▶ **Solution.** On retire des pénalités (des 3) tant qu'on peut.

```
def possible_avec_penalites_seules(score):
    if score == 0:                                  # CAS DE BASE : atteint
        return True
    if score < 0:                                   # CAS DE BASE : depasse
        return False
    return possible_avec_penalites_seules(score - 3)  # APPEL RECURSIF : une penalite de
                                                    # moins
```

2. ▶ **Solution.** Cas de base (et $f(n) = 0$ pour $n < 0$) :

$$f(0) = 1, \quad f(1) = 0, \quad f(2) = 0, \quad f(3) = 1, \quad f(4) = 0, \quad f(5) = 1, \quad f(6) = 1.$$

▷ **Erreur fréquente.** Remplacer la convention $f(0) = 1$ par $f(0) = 0$: il y a bien *une* façon d'obtenir 0 point, ne rien marquer. Avec $f(0) = 0$, tous les appels renverraient 0.

3. ▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert nb_solutions(10) == 3     # cas normal
assert nb_solutions(0) == 1      # cas limite : une seule facon, ne rien marquer
assert nb_solutions(4) == 0      # cas piege : un score impossible
```

Avec ces cas de base, la relation $f(n) = f(n-3) + f(n-5) + f(n-7)$ est en fait valable pour tout $n \geqslant 1$:

```
def nb_solutions(score):
    if score < 0:                   # CAS DE BASE : score depasse
        return 0
    if score == 0:                  # CAS DE BASE : une facon
        return 1
    return (nb_solutions(score - 3)    # APPEL RECURSIF x 3,
            + nb_solutions(score - 5)  # puis COMBINER : la somme
            + nb_solutions(score - 7))
```

4. ▶ **Solution.** La **programmation dynamique** (mémoïsation) : on garde en mémoire les valeurs déjà calculées pour ne pas les recalculer (méthode étudiée au chapitre 11).

5. ▶ **Solution.** **Démarche :** pour un score nul, une seule solution, la liste `[0]` ; sinon, chaque solution obtenue pour `score - coup` (avec `coup` parmi 3, 5 et 7) est prolongée par `score`.

```
def solutions_possibles(score):
    if score < 0:                                # CAS DE BASE : aucune solution
        resultat = []
    elif score == 0:                             # CAS DE BASE : une solution, [0]
        resultat = [[0]]
    else:
        resultat = []                            # INITIALISER
```

```
        for coup in [3, 5, 7]:                          # PARCOURIR les coups
            liste = solutions_possibles(score - coup)   # APPEL RECURSIF
            for solution in liste:                      # PARCOURIR ses solutions
                solution.append(score)                  # COMBINER : prolonger par score
                resultat.append(solution)               # METTRE A JOUR
    return resultat                                     # RENVOYER
```

Test : `solutions_possibles(8)` *renvoie* `[[0, 5, 8], [0, 3, 8]]`.

Pourquoi l'indentation du sujet officiel (`resultat.append(...)` *au niveau de la boucle* `for coup`*) ne peut pas fonctionner : la ligne ne s'exécute qu'une fois par coup, alors que* `liste` *peut contenir plusieurs solutions.* `resultat.append(solution)` *y lève une erreur si* `liste` *est vide (*`solution` *n'existe pas) ;* `resultat.append(liste)` *imbrique les listes d'un niveau de trop. Sans indenter, il faudrait écrire* `resultat = resultat + liste`.

Exercice 31 IA IA IA *Itinéraires dans une grille (d'après Centres étrangers 2024, groupe 1, jour 2)*

↑énoncé p. 39

Barème indicatif (sur 4) : Q1 : 1,5 ; Q2 : 0,5 ; Q3 : 1 ; Q4 : 1

1. ▶ **Solution.** `i` compte les déplacements vers le bas, `j` ceux vers la droite :

```
from random import choice

def itineraire_aleatoire(m, n):
    itineraire = ''                             # INITIALISER
    i, j = 0, 0                                 # INITIALISER
    while i != m and j != n:                    # PARCOURIR : jusqu'a un bord
        deplacement = choice(['D', 'B'])
        itineraire = itineraire + deplacement   # METTRE A JOUR
        if deplacement == 'D':                  # TESTER
            j = j + 1                           # METTRE A JOUR
        else:
            i = i + 1                           # METTRE A JOUR
    if i == m:
        itineraire = itineraire + 'D' * (n - j)
    if j == n:
        itineraire = itineraire + 'B' * (m - i)
    return itineraire
```

La boucle s'arrête dès que `i` vaut `m` ou que `j` vaut `n` ; les deux `if` complètent alors avec les `n - j` lettres `D` ou les `m - i` lettres `B` qui manquent. Le résultat contient donc toujours exactement `m` lettres `B` et `n` lettres `D`. La grille parcourue a donc $m+1$ lignes et $n+1$ colonnes (indices de 0 à m et de 0 à n) : ce n'est pas la même convention que dans la suite, où $m \times n$ désigne le nombre de **cases**.

2. ▶ **Solution.** Dans une grille d'une seule ligne, on ne peut jamais descendre : le seul chemin consiste à aller toujours à droite (`'DDD'` pour $n = 4$ colonnes). Donc $N(1, n) = 1$.

3. ▶ **Solution.** Le premier déplacement est soit vers la droite, soit vers le bas, et ces deux familles de chemins sont disjointes. Après un pas vers le bas, il reste à traverser une grille de $(m-1) \times n$ cases : $N(m-1, n)$ chemins ; après un pas vers la droite, une grille de $m \times (n-1)$ cases : $N(m, n-1)$ chemins. D'où $N(m, n) = N(m-1, n) + N(m, n-1)$.

4. ▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert nombre_chemins(3, 3) == 6     # cas normal
assert nombre_chemins(1, 5) == 1     # cas limite : une seule ligne
assert nombre_chemins(5, 1) == 1     # cas piege : une seule colonne, l'autre cas de base
```

Les cas de base (conditions d'arrêt) sont $m = 1$ ou $n = 1$:

```python
def nombre_chemins(m, n):
    if m == 1 or n == 1:                                    # CAS DE BASE
        return 1
    return nombre_chemins(m - 1, n) + nombre_chemins(m, n - 1)  # APPEL RECURSIF x 2,
                                                            # COMBINER
```

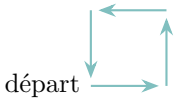
▷ **Erreur fréquente.** Ne tester que `m == 1` : l'appel `nombre_chemins(m, n - 1)` fait alors descendre `n` à 0, puis en dessous, sans jamais s'arrêter (`RecursionError`). Il faut les deux conditions d'arrêt.

Par exemple `nombre_chemins(3, 3)` renvoie `6`. *(Cette fonction recalcule de nombreuses fois les mêmes valeurs : on l'améliorera dans le chapitre sur la programmation dynamique.)*

Exercice 32 IA IA IA *Les instructions d'un robot (d'après Asie 2026, jour 2)* ↑énoncé p. 41

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 1 ; Q4 : 1 ; Q5 : 1 ; Q6 : 1 ; Q7 : 1

1. ▶ **Solution.** `'4(AG)'` équivaut à `'AGAGAGAG'` : le robot avance d'un pas puis tourne à gauche, quatre fois. Il parcourt un **carré** de côté un pas, dans le sens inverse des aiguilles d'une montre, et revient à son point de départ (orienté comme au début).



2. ▶ **Solution.** Ligne 4 : `return len(intrus) == 0` (ou `return intrus == []`).

3. ▶ **Solution.** Ligne 4 : `return False` (entier en fin de chaîne) ; ligne 8 : `return False` (entier juste avant une parenthèse fermante) ; ligne 9 : `return True`. *Le test de la ligne 6 porte sur la parenthèse* ***fermante*** *: un entier juste avant une parenthèse ouvrante, comme dans* `'3(AD)'`*, est au contraire la répétition normale d'un bloc.*

4. ▶ **Solution.** *Appels attendus, écrits avant de coder :*

```python
assert parenthesage_correct('2(3(AD)G)2A')    # cas normal
assert parenthesage_correct('3ADA')           # cas limite : aucune parenthese
assert not parenthesage_correct(')A(')        # cas piege : autant de ( que de ), mal
                                              # ordonnees
```

Démarche : on traduit l'algorithme de l'énoncé : un compteur qui augmente sur `'('` et diminue sur `')'`, un échec dès qu'il devient négatif, un succès s'il vaut 0 à la fin.

```python
def parenthesage_correct(chaine):
    parenthese = 0                      # INITIALISER
    for c in chaine:                    # PARCOURIR
        if c == '(':                    # TESTER
            parenthese = parenthese + 1 # METTRE A JOUR
        elif c == ')':                  # TESTER
            parenthese = parenthese - 1 # METTRE A JOUR
        if parenthese < 0:              # TESTER : fermante en trop
            return False
    return parenthese == 0              # RENVOYER
```

5. ▶ **Solution.**

Fin de l'itération	`nombre`	`indice`
1	`'1'`	3
2	`'17'`	4
3	`'179'`	5

La boucle s'arrête car `chaine[5]` vaut `'A'` ; la fonction renvoie `(179, 4)`.

6. ▶ **Solution.** Ligne 7 : `bloc = bloc + caractere` ; ligne 8 : `indice = indice + 1` ; ligne 9 : `caractere = chaine[indice]`.

7. ▶ **Solution.** Ligne 12 : `t = lire_nombre(chaine, indice)` ; ligne 16 : `t = lire_bloc(chaine, indice)` ; ligne 20 : `lire_parcours(bloc)` — c'est l'**appel récursif** : le bloc est lui-même une chaîne d'instructions, éventuellement avec des blocs imbriqués. La récursivité s'arrête sur un bloc qui ne contient plus de parenthèse (condition d'arrêt : aucun nouveau bloc à lire). Par exemple, `lire_parcours('2(3(AD)G)2A')` exécute `ADADADG ADADADG AA`.

Exercice 33 *Le cadenas de l'escape game (sujet maison, façon bac)* ↑énoncé p. 43

Barème indicatif (sur 8) : Q1 : 0,5 ; Q2 : 1 ; Q3 : 0,5 ; Q4 : 0,5 ; Q5 : 1 ; Q6 : 1 ; Q7 : 1 ; Q8 : 0,5 ; Q9 : 1 ; Q10 : 1

1. ▶ **Solution.** `ABC`, `ACB`, `BAC`, `BCA`, `CAB`, `CBA` : **6 anagrammes**.

2. ▶ **Solution.** Pour écrire une anagramme d'un mot de n lettres différentes, on choisit d'abord la première lettre : n choix. Il reste alors à écrire une anagramme des $n-1$ lettres restantes, toutes différentes : $P(n-1)$ façons, quel que soit le premier choix. D'où $P(n) = n \times P(n-1)$. (On reconnaît la factorielle : $P(n) = n!$.)

```
1  def nb_anagrammes(n):
2      if n <= 1:                          # CAS DE BASE
3          return 1
4      return n * nb_anagrammes(n - 1)     # APPEL RECURSIF, COMBINER : x n
```

`nb_anagrammes(3)` renvoie 6, `nb_anagrammes(4)` renvoie 24.

3. ▶ **Solution.** $40\,320 \times 2 = 80\,640$ secondes, soit $80\,640/3\,600 = 22{,}4$ heures : près d'une journée entière. Le nombre d'anagrammes est multiplié par n à chaque lettre ajoutée (par 9 pour 9 lettres, puis par 10…) : la méthode devient très vite inutilisable.

4. ▶ **Solution.**

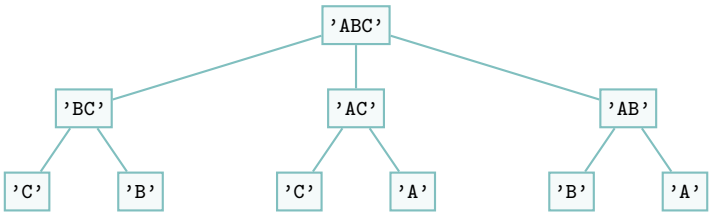
```
1  def sans_lettre(mot, i):
2      return mot[:i] + mot[i + 1:]
```

5. ▶ **Solution.** Ligne 3 : `return [mot]` (un mot d'au plus une lettre n'a qu'une anagramme : lui-même) ; ligne 6 : `for fin in anagrammes(sans_lettre(mot, i)):` ; ligne 7 : `resultat.append(mot[i] + fin)`.

```
1  def anagrammes(mot):
2      if len(mot) <= 1:                              # CAS DE BASE
3          return [mot]
4      resultat = []                                  # INITIALISER
5      for i in range(len(mot)):                      # PARCOURIR : la lettre de tete
6          for fin in anagrammes(sans_lettre(mot, i)):  # APPEL RECURSIF
7              resultat.append(mot[i] + fin)          # COMBINER
8      return resultat                                # RENVOYER
```

`anagrammes('ABC')` renvoie `['ABC', 'ACB', 'BAC', 'BCA', 'CAB', 'CBA']` (même ordre qu'à la question 1).

6. ▶ **Solution.** L'appel `anagrammes('ABC')` lance un appel par lettre placée en tête, sur les lettres restantes : `anagrammes('BC')`, puis `anagrammes('AC')`, puis `anagrammes('AB')`. Chacun lance à son tour deux appels sur un mot d'une lettre (condition d'arrêt) :



Il y a $1 + 3 + 6 = \mathbf{10}$ appels. Pour `'ABCD'`, l'appel initial lance 4 appels sur des mots de 3 lettres, et chacun d'eux en provoque 10 (arbre ci-dessus) : $1 + 4 \times 10 = \mathbf{41}$ appels.

7. ▶ **Solution.** Le **variant** est la longueur du mot. Chaque appel récursif porte sur `sans_lettre(mot, i)`, qui a une lettre de moins : cet entier positif décroît strictement à chaque appel, donc finit par valoir 1 (ou 0 pour le mot vide) : c'est la condition d'arrêt `len(mot) <= 1`, qui renvoie sans appel récursif. De plus, chaque appel ne lance qu'un nombre fini d'appels (un par lettre) : l'exécution se termine.

8. ▶ **Solution.** La fonction ne distingue pas les deux `L` : elle les traite comme deux lettres différentes. La liste contient $4 \times 3 \times 2 \times 1 = \mathbf{24}$ mots, mais chaque mot y figure deux fois (une fois pour chaque façon de placer les deux `L`, qui donnent le même mot) : il n'y a que $24/2 = \mathbf{12}$ mots différents. La formule $P(n) = n!$ ne vaut que pour des lettres toutes différentes.

9. ▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert retirer('ALLO', 'L') == 'ALO'      # cas normal
assert retirer('', 'L') == ''             # cas limite : mot vide (cas de base)
assert retirer('ALLO', 'Z') == 'ALLO'     # cas piege : lettre absente
```

```
def retirer(mot, lettre):
    if mot == '':                         # CAS DE BASE : mot vide
        return ''
    if mot[0] == lettre:                  # TESTER : premiere apparition ?
        return mot[1:]
    return mot[0] + retirer(mot[1:], lettre)  # APPEL RECURSIF, COMBINER
```

`retirer('ALLO', 'L')` renvoie `'ALO'` et `retirer('ALLO', 'Z')` renvoie `'ALLO'`.

10. ▶ **Solution.** Si `mot1` est vide, `mot2` doit l'être aussi. Sinon, la première lettre de `mot1` doit figurer dans `mot2` ; on la retire des deux mots et on recommence :

```
def est_anagramme(mot1, mot2):
    if mot1 == '':                                  # CAS DE BASE
        return mot2 == ''
    if mot1[0] not in mot2:                         # TESTER
        return False
    return est_anagramme(mot1[1:], retirer(mot2, mot1[0]))  # APPEL RECURSIF
```

Déroulement de `est_anagramme('ALLO', 'LOLA')` :

- `'A'` est dans `'LOLA'` ; `retirer('LOLA', 'A')` vaut `'LOL'` → appel `est_anagramme('LLO', 'LOL')` ;
- `'L'` est dans `'LOL'` ; on retire le premier `L` → appel `est_anagramme('LO', 'OL')` ;
- `'L'` est dans `'OL'` → appel `est_anagramme('O', 'O')` ;
- `'O'` est dans `'O'` → appel `est_anagramme("", "")` : condition d'arrêt, renvoie `True`.

La valeur `True` remonte jusqu'à l'appel initial. À l'inverse, `est_anagramme('ALLO', 'ALO')` renvoie `False` (après avoir retiré `A` et `L`, il ne reste plus de `L` dans `'O'`).

Critiquer une réponse d'IA

Exercice 34 *Un test de tri écrit par un assistant d'IA* ↑énoncé p. 44

▶ **Solution.** **1.** `est_triee([1, 5, 2, 8])` : `1 > 5` est faux, appel sur `[2, 8]` ; `2 > 8` est faux, appel sur `[]` → `True`. La fonction renvoie donc `True`, alors que la liste **n'est pas triée** ($5 > 2$). Pour `[1, 2, 5, 8]` : `True` (correct). La réponse est donc **fausse**.

2. L'erreur : l'appel récursif `tab[2:]` saute deux éléments à la fois, si bien que la paire `(tab[1], tab[2])` n'est **jamais comparée**. Il faut avancer d'un seul élément :

```
1     return est_triee(tab[1:])          # APPEL RECURSIF : on garde tab[1] pour la suite
```

Avec cette ligne, `est_triee([1, 5, 2, 8])` renvoie `False` et `est_triee([1, 2, 5, 8])` renvoie `True`.

3. Tester sur une liste **non triée** dont le désordre n'est pas entre les deux premiers éléments (comme `[1, 5, 2, 8]`) : une fonction booléenne se teste sur des cas `True` *et* sur des cas `False`. Vérifier aussi que *chaque* paire consécutive est bien examinée par la récurrence.

S'entraîner à l'oral

Exercice 35 *Expliquer en deux minutes* ↑énoncé p. 45

▶ **Solution.** Pas de texte à apprendre par cœur : voici les **éléments attendus** pour chaque sujet. L'ordre, les mots et l'exemple peuvent être différents ; l'explication est réussie si ces idées y sont, justes et reliées entre elles.

Sujet 1.

- Une fonction récursive s'appelle elle-même sur un problème **plus petit** ; elle contient une **condition d'arrêt** (cas de base) qui renvoie un résultat **sans** appel récursif.
- Le **variant** : un entier positif (`n`, la longueur d'une liste…) qui **décroît strictement** à chaque appel ; il finit donc par atteindre la condition d'arrêt.
- Exemple : `somme(n)` renvoie `n + somme(n - 1)`, avec arrêt pour `n == 0` : `somme(3)` → `somme(2)` → `somme(1)` → `somme(0)`, qui s'arrête.
- Piège : oublier la condition d'arrêt, ou un appel qui ne s'en rapproche pas (`somme(n + 1)`, ou `somme(-1)` avec le seul test `n == 0`).

Sujet 2.

- Chaque appel en cours est mémorisé dans la **pile d'exécution** (paramètres, variables locales, endroit où reprendre) : le dernier appel lancé est le premier terminé.
- Exemple : `fact(3)` empile `fact(3)`, `fact(2)`, `fact(1)`, puis les appels se terminent dans l'ordre inverse.
- Python limite la profondeur de la pile (environ 1 000 appels par défaut) : au-delà, il s'arrête avec `RecursionError`.
- Deux causes : une récursion **infinie** (condition d'arrêt jamais atteinte) ou une entrée trop grande (une version itérative s'impose alors).

Sujet 3.

- L'appelant ne peut finir son calcul qu'après avoir reçu la valeur renvoyée par l'appel récursif.
- Exemple : `fact(3) = 3 * fact(2)` ; descente 3, 2, 1 jusqu'à la condition d'arrêt, puis remontée : $1 \to 2 \to 6$.
- Ce qui est écrit **avant** l'appel s'exécute à la descente, ce qui est **après** à la remontée (`print` avant l'appel : `3 2 1` ; après : `1 2 3`).
- Piège : croire que la valeur de la condition d'arrêt est directement le résultat final.

Réponses des questions de la fiche de révision

↑ questions p. 59

Se tester

1. une **condition d'arrêt** (cas de base, sans appel) et un **cas récursif** sur une donnée strictement plus petite (§ II).
2. `1 2 3` : le `print` est après l'appel, il s'exécute à la remontée (§ II).
3. 6 appels (`n` = 5, 4, 3, 2, 1, 0) : coût linéaire (§ III.A et IV.B).
4. le `return` oublié devant l'appel récursif (ou résultat de l'appel non combiné) (§ III et Erreurs fréquentes).
5. les mêmes valeurs sont recalculées sans cesse : coût exponentiel. On verra au chapitre *Programmation dynamique* comment éviter ces appels répétés (§ IV.B).

Et avant ?

1. `9`, `[7, 2, 9]` et `3` : indice négatif depuis la fin, tranche à partir de l'indice 1 (Première).
2. `None` : `print` affiche seulement, seul `return` renvoie une valeur réutilisable (Première).
3. $n \times n = n^2$ tours : coût quadratique, comme les tris par sélection et par insertion (Première).

CORRIGÉS DU CHAPITRE 2

2 Programmation orientée objet et paradigmes

CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 La fiche d'identité d'un objet

↑ énoncé p. 67

Exercice 1 *Deux lampes*

1. **Caractéristiques** : couleur : bleue ; puissance de l'ampoule : 40 W ; allumée : non (on accepte aussi : marque, hauteur, position du bras…). **Actions** : appuyer sur l'interrupteur (allumer / éteindre), changer l'ampoule, orienter le bras, dire si elle éclaire.

2. **Identique** : la liste des caractéristiques (leurs *noms*) et la liste des actions. **Change** : les *valeurs* (bleue / rouge, 40 / 60 W, éteinte / allumée). Deux lampes du même modèle ont la même *forme* de fiche, mais pas le même contenu.

Exercice 2 *Le modèle commun*

3. On garde le titre « Lampe », les noms des caractéristiques et toutes les actions ; on enlève les valeurs, propres à chaque lampe. Cette fiche modèle ne décrit aucune lampe en particulier : elle sert à en fabriquer.

4. Il faut fournir la couleur et la puissance de l'ampoule (elles varient d'une lampe à l'autre). La caractéristique « allumée » vaut **toujours** « non » à la sortie de l'usine : inutile de la demander.

Exercice 3 *Les actions changent l'état*

5. Seule « allumée » a changé : éteinte → allumée → éteinte → **allumée**. Couleur et puissance n'ont pas bougé. L'état d'un objet est l'ensemble des valeurs de ses caractéristiques *à un instant donné*.

6. La **nouvelle puissance** (la nouvelle ampoule). Une action peut avoir besoin d'informations en plus de l'objet lui-même.

7. **Modifient** : appuyer sur l'interrupteur, changer l'ampoule, orienter le bras. **Informent** sans rien changer : dire si elle éclaire, dire sa puissance.

8. N'importe qui peut mettre l'objet dans un état **absurde** (un solde de -50 € alors qu'on ne peut pas dépenser ce qu'on n'a pas). Si le solde ne se modifie que par des actions (« recharger », « payer un repas »), ces actions peuvent **vérifier** la demande et refuser un paiement sans provision.

Exercice 4 *À vous*

9. Exemple (réveil) : heure affichée : 7 h 12 ; heure de l'alarme : 7 h 30 ; alarme activée : oui. Actions : avancer d'une minute, sonner, régler l'alarme *(information : la nouvelle heure)*, dire si l'alarme est activée. Exemple (ascenseur) : étage actuel, portes ouvertes, nombre de personnes ; actions : aller à un étage *(information : le numéro de l'étage)*, ouvrir les portes, dire l'étage.

Exercice 5 *Ce qu'on a découvert*

10. Un objet est décrit à la fois par **ce qu'il sait** (ses caractéristiques et leurs valeurs : son état) et par **ce qu'il sait faire** (ses actions : son comportement). La fiche modèle sert à **fabriquer** autant de lampes qu'on veut, toutes construites sur le même plan. Les lampes de Léa et de Tom sont deux **exemplaires** du même modèle. Une action peut **modifier** une caractéristique, ou seulement **donner une information** sur l'objet.

CORRIGÉS DES EXERCICES

Remarque Plusieurs écritures sont souvent possibles : on donne **une** version simple.

Le paradigme fonctionnel

Exercice 1 *Pure ou pas pure ?* ↑énoncé p. 85

▶ Solution.

- `f1` : **pure** — son résultat ne dépend que de `x`, aucun effet de bord.
- `f2` : **non pure** — elle modifie la variable globale `compteur` (effet de bord).
- `f3` : **non pure** — elle modifie la liste reçue avec `append` (effet de bord).
- `f4` : **pure** — résultat fonction des seuls paramètres, aucun effet de bord.

Exercice 2 *Rendre une fonction pure* ↑énoncé p. 85

▶ Solution. *Appels attendus, écrits avant de coder :*

```
assert double_tout_pur([1, 2, 3]) == [2, 4, 6]          # cas normal
assert double_tout_pur([]) == []                        # cas limite : liste vide
t = [5, -1]
assert double_tout_pur(t) == [10, -2] and t == [5, -1]  # cas piege : t intact
```

On **construit une nouvelle liste** au lieu de modifier l'ancienne : on parcourt `tab` et on garde le double de chaque élément.

```
def double_tout_pur(tab):
    return [x * 2 for x in tab]
```

Autre méthode (sans compréhension, comme le suggère le coup de pouce) : on part d'une liste vide que l'on remplit dans une boucle.

```
def double_tout_pur(tab):
    resultat = []                # INITIALISER : une nouvelle liste
    for x in tab:                # PARCOURIR
        resultat.append(x * 2)   # METTRE A JOUR
    return resultat              # RENVOYER
```

La liste d'origine `t` reste bien `[1, 2, 3]`.

Exercice 3 *Défi — des fonctions qui manipulent des fonctions* ↑énoncé p. 86

▶ Solution. *Appels attendus, écrits avant de coder :*

```
assert appliquer(abs, [-2, 3, -5]) == [2, 3, 5]          # cas normal
assert appliquer(double, []) == []                       # cas limite : liste vide
t = [1, 2]
assert appliquer(plus_un, t) == [2, 3] and t == [1, 2]   # cas piege : tab intact
```

Les paramètres `f` et `test` sont des **fonctions** : on les appelle comme les autres. On construit toujours une **nouvelle** liste, donc `tab` n'est jamais modifié : les fonctions sont pures. Une création de liste par compréhension convient parfaitement : on transforme chaque élément (`appliquer`), ou on ne garde que ceux qui passent le test (`garder`).

```
def appliquer(f, tab):
    return [f(x) for x in tab]

def garder(test, tab):
    return [x for x in tab if test(x)]

def composer(f, g):
    def h(x):                # fonction definie a l'interieur
        return f(g(x))
    return h                 # on renvoie la fonction elle-meme
```

`composer(double, plus_un)(3)` vaut `double(plus_un(3))` $= 2 \times 4 = 8$, alors que `composer(plus_un, double)(3)` vaut `plus_un(double(3))` $= 6 + 1 = 7$: l'ordre de composition compte.

4. La **compréhension** : `[abs(x) for x in [-2, 3, -5]]` vaut `[2, 3, 5]` et `[x for x in [1, 2, 3, 4] if est_pair(x)]` vaut `[2, 4]`. C'est d'ailleurs exactement le corps des fonctions `appliquer` et `garder`.

Lire et comprendre une classe

Exercice 4 *Lister attributs et méthodes* ↑énoncé p. 86

▶ Solution.

1. Attributs : `titre` (`str`), `auteur` (`str`), `pages` (`int`), `emprunte` (`bool`).
2. Méthodes : `get_titre` *renvoie* une information (l'attribut `titre`) ; `emprunter` *modifie* l'état (passe `emprunte` à `True`). *(Le constructeur `__init__` est aussi une méthode.)*
3. `self` désigne l'**instance en cours** : c'est par lui qu'une méthode accède aux attributs *de cet objet précis.*

Exercice 5 *Prévoir l'affichage* ↑énoncé p. 87

▶ Solution. L'affichage est :

```
1984
False
True
```

À la création, `emprunte` vaut `False` ; après l'appel `b.emprunter()`, il vaut `True`.

Exercice 6 *Retrouver l'erreur* ↑énoncé p. 87

▶ Solution.

(a) Il manque `self` en premier paramètre : `def __init__(self, nom):`.

(b) La ligne `nom = nom` ne crée aucun attribut (elle réaffecte le paramètre à lui-même) : il faut `self.nom = nom`.

(c) `nom` n'existe pas en dehors de la classe ; l'attribut se lit `medor.nom` (notation pointée).

Créer et manipuler des instances

Exercice 7 *Créer des instances* ↑énoncé p. 87

▶ **Solution.** **Démarche :** on appelle la classe comme une fonction, avec les valeurs de `x` et `y` (`self` ne se donne pas : c'est l'objet créé) ; un attribut se lit avec la notation pointée.

```
origine = Point(0, 0)
p = Point(3, 5)
print(p.x)      # affiche 3
```

Exercice 8 *Compléter un constructeur* ↑énoncé p. 87

▶ **Solution.** **Démarche :** dans le constructeur, chaque attribut `self....` reçoit le paramètre correspondant ; `notes` n'a pas de paramètre : on lui affecte une liste vide, propre à chaque élève.

```
class Eleve:
    def __init__(self, nom, classe):
        self.nom = nom
        self.classe = classe
        self.notes = []
```

Exercice 9 *Ajouter des méthodes* ↑énoncé p. 87

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
e = Eleve("Zoe", "TG1")
e.ajouter_note(12)
e.ajouter_note(16)
assert e.moyenne() == 14.0      # cas normal
e2 = Eleve("Lou", "TG2")
e2.ajouter_note(9)
assert e2.moyenne() == 9        # cas limite : une seule note
e2.ajouter_note(10)
assert e2.moyenne() == 9.5      # cas piege : division decimale, pas //
```

Démarche : chaque méthode reçoit `self` en premier paramètre et agit sur l'attribut `self.notes` : `ajouter_note` le modifie (`append`), `moyenne` le parcourt avec un accumulateur, puis divise par le nombre de notes.

```
class Eleve:
    def __init__(self, nom, classe):
        self.nom = nom
        self.classe = classe
        self.notes = []

    def ajouter_note(self, note):
        self.notes.append(note)

    def moyenne(self):
        total = 0                       # INITIALISER
        for note in self.notes:         # PARCOURIR
            total += note               # METTRE A JOUR
        return total / len(self.notes)  # RENVOYER
```

▷ **Erreur fréquente.** Écrire `notes.append(note)` sans `self.` : dans la méthode, `notes` n'est pas une variable, Python lève `NameError`. L'attribut de l'objet s'écrit toujours `self.notes`.

Écrire une classe complète

Exercice 10 *La classe Rectangle* ↑énoncé p. 88

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
assert Rectangle(4, 3).aire() == 12        # cas normal
assert Rectangle(5, 5).est_carre()         # cas limite : largeur egale a hauteur
assert Rectangle(4, 3).perimetre() == 14   # cas piege : 2 * (4 + 3), pas 4 + 3
```

Démarche : le constructeur range les deux dimensions dans des attributs ; chaque méthode ne fait qu'un calcul sur `self.largeur` et `self.hauteur` ; `est_carre` renvoie directement le résultat de la comparaison.

```
class Rectangle:
    def __init__(self, largeur, hauteur):
        self.largeur = largeur
        self.hauteur = hauteur

    def aire(self):
        return self.largeur * self.hauteur

    def perimetre(self):
        return 2 * (self.largeur + self.hauteur)

    def est_carre(self):
        return self.largeur == self.hauteur
```

▷ **Erreur fréquente.** Appeler une méthode sans parenthèses (`r.aire`) : on obtient la méthode elle-même, pas son résultat. On écrit `r.aire()`.

Exercice 11 *La classe Chronometre* ↑énoncé p. 88

▶ **Solution.** *Appels attendus, écrits avant de coder :*

```
c = Chronometre()
for i in range(75):
    c.tic()
assert c.affichage() == "01:15"     # cas normal
c.reset()
assert c.affichage() == "00:00"     # cas limite : compteur a zero
for i in range(60):
    c.tic()
assert c.affichage() == "01:00"     # cas piege : 60 s font 1 min, pas "00:60"
```

Le format `"mm:ss"` s'obtient avec la division entière (`//` pour les minutes) et le reste (`%` pour les secondes).

```
class Chronometre:
    def __init__(self):
        self.secondes = 0

    def tic(self):
        self.secondes = self.secondes + 1

    def reset(self):
        self.secondes = 0

    def affichage(self):
        m = self.secondes // 60
        s = self.secondes % 60
        return f"{m:02d}:{s:02d}"
```

▷ **Erreur fréquente.** Écrire `f"{m}:{s}"` : après 75 secondes, on obtient `"1:15"` et non `"01:15"`. Le format `02d` complète par un zéro à gauche.

Encapsulation

Exercice 12 IA IA IA *Corriger une mauvaise pratique* ↑énoncé p. 88

▶ Solution. 1. En écrivant directement `c.solde = c.solde - 500`, on *court-circuite* la classe : aucun contrôle n'est fait, et le solde peut devenir absurde (négatif). L'objet n'a plus la garantie de rester dans un état correct.

2. *Appels attendus, écrits avant de coder :*

```
c = Compte(100)
assert c.retirer(30) and c.solde() == 70        # cas normal
assert c.retirer(70) and c.solde() == 0         # cas limite : tout retirer
assert not c.retirer(-50) and c.solde() == 0    # cas piege : montant negatif refuse
```

Démarche : le solde devient un attribut « interne » (`_solde`) ; on n'y accède plus que par les méthodes de l'interface. `retirer` **teste d'abord** le montant, et ne modifie l'état que si le retrait est accepté.

```
class Compte:
    def __init__(self, solde):
        self._solde = solde

    def solde(self):
        return self._solde

    def retirer(self, montant):
        if 0 < montant <= self._solde:
            self._solde = self._solde - montant
            return True
        return False
```

Désormais `c.retirer(500)` sur un solde de 100 renvoie `False` et laisse le solde inchangé.

▷ **Erreur fréquente.** Ne tester que `montant <= self._solde` : `c.retirer(-50)` serait accepté et *augmenterait* le solde. La condition `0 < montant` ferme cette porte.

Méthodes spéciales et objets qui interagissent

Exercice 13 IA IA IA *Défi — la classe Fraction* ↑énoncé p. 89

▶ Solution. *Appels attendus, écrits avant de coder :*

```
assert repr(Fraction(6, 8)) == "3/4"      # cas normal
assert repr(Fraction(0, 5)) == "0/1"      # cas limite : numerateur nul
f = Fraction(3, 4)
assert repr(f.fois(Fraction(2, 3))) == "1/2" and repr(f) == "3/4"   # cas piege : f intacte
```

On simplifie **dans le constructeur** en divisant par le PGCD. On définit `__repr__` (et non `__str__`) : ainsi la fraction s'affiche proprement en tapant simplement son nom dans la console — et `print` en bénéficie aussi par ricochet. La méthode `fois` construit et **renvoie** une nouvelle `Fraction` (elle ne modifie rien : c'est l'esprit fonctionnel).

```
def pgcd(a, b):
    while b != 0:
        r = a % b       # METTRE A JOUR : on garde le reste
        a = b
        b = r
    return a            # RENVOYER

class Fraction:
    def __init__(self, num, den):
        d = pgcd(num, den)
        self.numerateur = num // d
        self.denominateur = den // d

```

```
    def __repr__(self):
        return f"{self.numerateur}/{self.denominateur}"

    def fois(self, autre):
        return Fraction(self.numerateur * autre.numerateur,
                        self.denominateur * autre.denominateur)
```

Exercice 14 *Défi — un réseau d'amis* ↑énoncé p. 89

▶ Solution. *Appels attendus, écrits avant de coder :*

```
a, b = Personne("Ada"), Personne("Bob")
a.ajouter_ami(b)
assert a.noms_amis() == ["Bob"] and b.noms_amis() == ["Ada"]   # cas normal
a.ajouter_ami(a)
assert a.noms_amis() == ["Bob"]       # cas limite : pas son propre ami
b.ajouter_ami(a)
assert b.noms_amis() == ["Ada"]       # cas piege : deja amis, pas de doublon
```

L'attribut `amis` contient des **objets** : un objet en connaît d'autres, et ils **interagissent** (une méthode de `a` modifie aussi l'état de `autre`).

```
class Personne:
    def __init__(self, nom):
        self.nom = nom
        self.amis = []          # liste d'objets Personne

    def ajouter_ami(self, autre):
        if autre is self or autre in self.amis:
            return              # cas interdits : rien a faire
        self.amis.append(autre)
        autre.amis.append(self) # amitie reciproque

    def noms_amis(self):
        return [p.nom for p in self.amis]

    def amis_communs(self, autre):
        return [p.nom for p in self.amis if p in autre.amis]

    def amis_d_amis(self):
        resultat = []                                   # INITIALISER
        for ami in self.amis:                           # PARCOURIR
            for p in ami.amis:                          # PARCOURIR
                if p is not self and p not in self.amis:  # TESTER
                    if p.nom not in resultat:           # TESTER : pas de doublon
                        resultat.append(p.nom)          # METTRE A JOUR
        return resultat                                 # RENVOYER
```

Sur l'exemple : `a.amis_d_amis()` renvoie `['Dan']` (ami de Chloé) ; Bob, déjà ami d'Ada, n'est pas proposé.

Pourquoi des objets ? En stockant les objets eux-mêmes, on accède directement à *toutes* leurs informations, y compris à *leurs* amis (indispensable pour `amis_d_amis`) ; deux personnes peuvent aussi porter le même nom sans être confondues. Avec de simples noms, il faudrait une structure supplémentaire pour retrouver l'objet correspondant.

Choisir un paradigme

Exercice 15 *Le bon style au bon endroit* ↑énoncé p. 90

▶ **Solution.**

a) objet : chaque entité (personnage, monstre, objet) a un état et des comportements propres ;
b) fonctionnel : une transformation sans effet de bord évite de corrompre la liste d'origine ;
c) impératif : un tri se décrit naturellement comme une suite d'étapes qui modifient un tableau.

Vers le bac

Exercice 16 *Le domaine skiable (d'après Métropole 2024, jour 2)* ↑énoncé p. 90

Barème indicatif (sur 3) : Q1 : 0,5 ; Q2 : 0,75 ; Q3 : 0,25 ; Q4 : 0,75 ; Q5 : 0,75

▶ **Solution.**

1. Attributs de `Piste` : `nom` (`str`), `denivele` (`int`), `longueur` (`float`), `couleur` (`str`), `ouverte` (`bool`).

2. *Appels attendus, écrits avant de coder :*

```
p, q, r = Piste('A', 55, 1.0), Piste('B', 100, 1.0), Piste('C', 120, 1.0)
for piste in [p, q, r]:
    piste.set_couleur()
assert p.couleur == 'bleue'     # cas normal
assert q.couleur == 'noire'     # cas limite : 100 pile, le seuil est inclus
assert r.couleur == 'noire'     # cas piege : 120 >= 40 aussi, l'ordre des tests compte
```

Démarche : une cascade `if` / `elif` / `else`, en partant du seuil le plus élevé : le premier test vrai l'emporte.

```
def set_couleur(self):
    if self.denivele >= 100:
        self.couleur = 'noire'
    elif self.denivele >= 70:
        self.couleur = 'rouge'
    elif self.denivele >= 40:
        self.couleur = 'bleue'
    else:
        self.couleur = 'verte'
```

▷ **Erreur fréquente.** Tester les seuils dans l'ordre croissant (`if self.denivele >= 40` en premier) : une piste noire de 120 m serait classée bleue, car le premier test vrai l'emporte.

3. Proposition **(D)** : une liste d'objets de type `Piste`.

4. **Démarche :** on parcourt la liste d'objets `Piste` renvoyée par `get_pistes()` et on modifie l'attribut `ouverte` de chaque piste verte.

```
for piste in lievre_blanc.get_pistes():     # PARCOURIR
    if piste.get_couleur() == 'verte':      # TESTER
        piste.ouverte = False               # METTRE A JOUR
```

(`set_couleur` *a été appelée sur chaque piste à la fin de l'annexe : ici,* `Les Marmottes` *et* `Le Jardin` *sont fermées.)*

▷ **Erreur fréquente.** Écrire `piste.ouverte == False` : c'est une comparaison, qui ne modifie rien. L'affectation s'écrit avec un seul `=`.

5. On parcourt les pistes et on garde le nom de celles qui ont la bonne couleur :

```
def pistes_de_couleur(lst, couleur):
    return [piste.get_nom() for piste in lst if piste.get_couleur() == couleur]
```

Autre méthode (boucle et `append`) :

```
def pistes_de_couleur(lst, couleur):
    noms = []                                # INITIALISER
    for piste in lst:                        # PARCOURIR
        if piste.get_couleur() == couleur:   # TESTER
            noms.append(piste.get_nom())     # METTRE A JOUR
    return noms                              # RENVOYER
```

Par exemple `pistes_de_couleur(lievre_blanc.get_pistes(), 'noire')` renvoie `['Petit Bonheur', 'Foret', 'Duvallon']`.

Exercice 17 *Valeurs nutritionnelles (d'après La Réunion 2023, jour 1)* ↑énoncé p. 91

Barème indicatif (sur 3) : Q1 : 0,25 ; Q2 : 0,25 ; Q3 : 0,25 ; Q4 : 0,75 ; Q5 : 0,5 ; Q6 : 1

▶ Solution.

1. `lait = Aliment(65.1, 3.32, 4.85, 3.63)`.
2. `lait.energie` (renvoie `65.1`).
3. `lait.proteines = 3.4`.
4.
```
def energie_reelle(self, masse):
    return self.energie * masse / 100
```
(`lait.energie_reelle(245)` *renvoie* `159.49499999999998`, *soit* 159,495 *à l'erreur d'arrondi des flottants près.)*
5. `aliments['pain'].energie_reelle(220)`.
6. *Appels attendus, écrits avant de coder :*

```
lait, pain = Aliment(65.1, 3.32, 4.85, 3.63), Aliment(285, 9, 55, 1.4)
assert plus_calorique({'lait': lait, 'pain': pain}) == 'pain'   # cas normal
assert plus_calorique({'lait': lait}) == 'lait'                 # cas limite : un seul
                                                                # aliment
assert plus_calorique({'pain': pain, 'lait': lait}) == 'pain'   # cas piege : champion
                                                                # en tete
```

Démarche : une recherche de maximum : on parcourt les clés du dictionnaire en mémorisant le nom du « champion », et l'on compare les attributs `energie` des objets associés.

```
def plus_calorique(aliments):
    nom_max = None                           # INITIALISER : pas encore de champion
    for nom in aliments:                     # PARCOURIR
        if nom_max is None or aliments[nom].energie > aliments[nom_max].energie:
            nom_max = nom                    # METTRE A JOUR
    return nom_max                           # RENVOYER
```

Exercice 18 *La classe Date (d'après Asie 2025, jour 2)* ↑énoncé p. 92

Barème indicatif (sur 4) : Q1 : 0,5 ; Q2 : 0,25 ; Q3 : 0,25 ; Q4 : 0,75 ; Q5 : 1,25 ; Q6 : 1

▶ Solution.

1. `self.jour = jour` ; `self.mois = mois` ; `self.annee = annee`.
2. `d = Date(19, 6, 2024)`.
3. `return self.annee`.
4. **Démarche :** on traduit la règle de l'énoncé avec `and` et `or`, en testant la divisibilité par le reste `%`.

```
def est_bissextile(self):
    a = self.annee
    return (a % 4 == 0 and a % 100 != 0) or (a % 400 == 0)
```

Ainsi 2024 et 2000 sont bissextiles, mais 1900 ne l'est pas.

5. *Appels attendus, écrits avant de coder :*

```
assert Date(19, 6, 2023).nb_jours_passes() == 170   # cas normal
assert Date(1, 1, 2024).nb_jours_passes() == 1      # cas limite : 1er janvier
assert Date(1, 3, 2024).nb_jours_passes() == 61     # cas piege : le 29 fevrier compte
```

Démarche : un accumulateur initialisé au jour de la date, auquel on ajoute les jours des mois **précédents** (indices `0` à `self.mois - 2`), puis 1 si l'année est bissextile et que la date est après février.

```
def nb_jours_passes(self):
    total = self.jour                              # INITIALISER
    for m in range(self.mois - 1):                 # PARCOURIR : mois precedents
        total = total + self.nb_jours_par_mois[m]  # METTRE A JOUR
    if self.mois > 2 and self.est_bissextile():    # TESTER
        total = total + 1                          # METTRE A JOUR : 29 fevrier
    return total                                   # RENVOYER
```

6. **Démarche :** on réutilise `est_bissextile` pour connaître le nombre de jours de l'année, puis on retranche `nb_jours_passes()`.

```
def nb_jours_restants(self):
    if self.est_bissextile():
        return 366 - self.nb_jours_passes()
    return 365 - self.nb_jours_passes()
```

Exercice 19 — *Colorier une carte (d'après Métropole 2023, jour 1)*

↑énoncé p. 92

Barème indicatif (sur 2) : Q1 : 0,5 ; Q2 : 0,25 ; Q3 : 1,25

▶ Solution.

1. `nom`, `tab_voisines` et `couleur_attribuee` sont des **attributs** ; le paramètre `nom_region` est de type `str` (chaîne de caractères).
2. `ge = Region("Grand Est")`.
3. *Appels attendus, écrits avant de coder :*

```
ge = Region("Grand Est")
assert ge.renvoie_premiere_couleur_disponible() == 'rouge'   # cas normal
assert ge.renvoie_nb_voisines() == 0          # cas limite : aucune voisine
assert not ge.est_coloriee()                  # cas piege : None, pas encore de couleur
```

Démarche : chaque méthode lit un attribut de `self` : le premier élément de la liste, sa longueur, ou le test « une couleur a-t-elle été attribuée ? ».

```
def renvoie_premiere_couleur_disponible(self):
    return self.tab_couleurs_disponibles[0]

def renvoie_nb_voisines(self):
    return len(self.tab_voisines)

def est_coloriee(self):
    return self.couleur_attribuee is not None
```

Exercice 20 — *Des colis à expédier (d'après Amérique du Nord 2025, jour 1)*

↑énoncé p. 93

Barème indicatif (sur 8) : Q1 : 0,5 ; Q2 : 0,75 ; Q3 : 0,5 ; Q4 : 0,5 ; Q5 : 1 ; Q6 : 0,75 ; Q7 : 0,5 ; Q8 : 1,25 ; Q9 : 0,75 ; Q10 : 1,5

▶ Solution.

1. Méthode à placer dans la classe `Colis` :

```
    def passer_transit(self):
        self.etat = 'transit'
```

2. **Démarche :** un test sur l'attribut `poids` du colis décide entre l'ajout et le message.

```
1 def ajouter_colis(liste, colis):
2     if colis.poids <= 25:
3         liste.append(colis)
4     else:
5         print("Dépassement du poids maximal autorisé")
```

3.
```
1 def nb_colis(liste):
2     return len(liste)
```

4. Ligne 2 : `total = 0` ; ligne 4 : `total = total + c.poids`. Avec `liste_colis`, on obtient `15.25`.

5. On ne garde que les colis dont l'état est `statut` (une nouvelle liste est créée) :

```
1 def liste_colis_etat(liste, statut):
2     return [c for c in liste if c.etat == statut]
```

6. C'est un **tri par sélection** (on sélectionne le plus lourd des colis restants et on le place en position `i`), de coût **quadratique** (n^2) dans le pire des cas.

7. Par exemple le **tri par insertion** (vu en Première), lui aussi de coût quadratique (n^2) dans le pire des cas. *Le tri fusion, que l'on étudiera au chapitre Diviser pour régner, fera mieux :* $n \log_2 n$.

8. **Démarche :** condition d'arrêt : plus de colis à examiner (on renvoie une liste vide) ; si le colis de rang `rang` tient, on le charge et la capacité diminue de son poids ; sinon on passe au suivant sans le charger.

```
1 def chargement_glouton(liste, rang, capacite):
2     if rang == len(liste):                                  # CAS DE BASE : plus
                                                              # de colis
3         return []
4     elif liste[rang].poids <= capacite:                     # TESTER : le colis
                                                              # tient
5         return [liste[rang]] + chargement_glouton(liste, rang + 1,  # COMBINER
6                                                   capacite - liste[rang].poids)
7     else:
8         return chargement_glouton(liste, rang + 1, capacite)        # APPEL RECURSIF
```

9. Chaque appel récursif augmente `rang` de 1 : pour une liste de n colis, il y a $n + 1$ appels imbriqués. Python limite la profondeur de la pile d'appels (environ 1000 par défaut) : avec une liste de plusieurs milliers de colis, la limite est dépassée et Python lève `RecursionError`.

10. *Appels attendus, écrits avant de coder :*

```
1 c1, c2, c3 = Colis('A', 10, ''), Colis('B', 6, ''), Colis('C', 3, '')
2 assert chargement_glouton2([c1, c2, c3], 14) == [c1, c3]       # cas normal : B ne tient
                                                                 # plus
3 assert chargement_glouton2([], 20) == []                       # cas limite : aucun colis
4 assert chargement_glouton2([c1, c2, c3], 19) == [c1, c2, c3]   # cas piege : charge =
                                                                 # capacite
```

Démarche : on parcourt les colis dans l'ordre en tenant à jour la charge du camion ; on garde un colis seulement s'il tient encore.

```
1 def chargement_glouton2(liste, capacite):
2     colis_a_charger = []                    # INITIALISER
3     charge = 0                              # INITIALISER
4     for c in liste:                         # PARCOURIR
```

```
        if charge + c.poids <= capacite:    # TESTER
            colis_a_charger.append(c)       # METTRE A JOUR
            charge = charge + c.poids       # METTRE A JOUR
    return colis_a_charger                  # RENVOYER
```

Elle applique la même règle que `chargement_glouton` (on garde un colis s'il tient dans la charge restante, en parcourant la liste dans le même ordre) : elle renvoie donc la même liste que `chargement_glouton(liste, 0, capacite)`.

Exercice 21 *Puissance 4 : la grille de jeu (d'après Amérique du Nord 2026, jour 1, partie A)*

↑ énoncé p. 95

Barème indicatif (sur 4) : Q1 : 0,75 ; Q2 : 1 ; Q3 : 0,5 ; Q4 : 0,5 ; Q5 : 1,25

▶ Solution.

1.
```
def __init__(self):
    self.grille = [[0 for j in range(7)] for i in range(6)]
```

▷ **Erreur fréquente.** Écrire `[[0] * 7] * 6` : on crée six références vers la **même** ligne, et un pion posé dans une ligne apparaîtrait dans les six.

2. Ligne 4 : `ligne = ligne - 1` (on remonte d'une rangée) ; ligne 6 : `self.grille[ligne][colonne] = joueur` ; ligne 7 : `return True` ; ligne 9 : `return False` (colonne pleine).

3. Le joueur 1 commence : il joue en colonne 2, le joueur 2 en colonne 3, puis le joueur 1 à nouveau en colonne 3 (son pion se pose sur celui du joueur 2).

```
jeu1 = Grille()
jeu1.joue(2, 1)
jeu1.joue(3, 2)
jeu1.joue(3, 1)
```

4. Case `(5, 3)` (joueur 2) : 7 alignements (4 horizontaux, 1 vertical, 1 dans chaque diagonale). Case `(4, 3)` (joueur 1) : 10 alignements (4 horizontaux, 2 verticaux, 2 dans chaque diagonale). Case `(5, 2)` (joueur 1) : 5 alignements (3 horizontaux, 1 vertical, 1 diagonal). Score : $7 - (10 + 5) = -8$.

5. *Appels attendus, écrits avant de coder :*

```
assert jeu1.score() == -8       # cas normal : la grille jeu1 de la question 3
assert Grille().score() == 0    # cas limite : grille vide
g = Grille()
g.joue(3, 2)
assert g.score() == 7           # cas piege : un pion du joueur 2 compte en plus
```

Démarche : deux boucles imbriquées parcourent les cases ; on ajoute la valeur des cases du joueur 2 et l'on retranche celle des cases du joueur 1.

```
def score(self):
    s = 0                                           # INITIALISER
    for ligne in range(6):                          # PARCOURIR
        for colonne in range(7):                    # PARCOURIR
            if self.grille[ligne][colonne] == 2:    # TESTER
                s = s + valeur_case(ligne, colonne) # METTRE A JOUR
            elif self.grille[ligne][colonne] == 1:  # TESTER
                s = s - valeur_case(ligne, colonne) # METTRE A JOUR
    return s                                        # RENVOYER
```

Critiquer une réponse d'IA

Exercice 22 *Une classe écrite par un assistant d'IA* ↑énoncé p. 96

1. Non. On attend `1 0` (Bob n'a rien ajouté) ; l'exécution affiche `1 1`, et `p2.articles` vaut `['stylo']` : le stylo d'Alice est *aussi* dans le panier de Bob.

2. L'erreur : la valeur par défaut `articles=[]` est créée **une seule fois**, à la définition de la classe, et **partagée** par toutes les instances qui ne fournissent pas de liste (une liste est un objet **mutable** : `append` modifie cette liste commune). Correction : créer la liste *dans* le constructeur, pour chaque instance :

```
class Panier:
    def __init__(self, client):
        self.client = client
        self.articles = []            # une liste NEUVE par panier
```

(le reste de la classe est inchangé). Le scénario affiche alors `1 0`.

3. Créer **deux instances**, modifier l'une, vérifier que l'autre n'a pas bougé : un test avec un seul objet ne peut pas révéler un état partagé. Réflexe : jamais de liste (ni de dictionnaire) comme valeur par défaut d'un paramètre.

S'entraîner à l'oral

Exercice 23 *Expliquer en deux minutes* ↑énoncé p. 97

▶ Solution. Pas de texte à apprendre par cœur : voici les **éléments attendus** pour chaque sujet. L'ordre, les mots et l'exemple peuvent être différents ; l'explication est réussie si ces idées y sont, justes et reliées entre elles.

Sujet 1.

- Une **classe** est un modèle (un plan, un moule) qui décrit les attributs et les méthodes communs ; une **instance** est un objet concret fabriqué à partir de ce modèle, avec ses propres valeurs d'attributs.
- Exemple : la classe `Chien` ; `rex = Chien('Rex', 4)` et `medor = Chien('Médor', 2)` sont deux instances de la même classe.
- Modifier `rex.age` ne change pas `medor.age` : chaque instance a son propre **état**.
- Piège : confondre attribut (une donnée) et méthode (une action), ou dire que la classe « contient » les objets.

Sujet 2.

- **Encapsulation** : l'état d'un objet (ses attributs) n'est modifié qu'à travers les méthodes de son **interface** ; l'utilisateur n'a pas à connaître l'implémentation.
- Exemple : un compte bancaire dont la méthode `retirer(montant)` vérifie que le solde suffit ; écrire directement `compte._solde = -500` contourne la règle.
- Intérêt : on peut changer l'implémentation interne sans casser le code qui utilise la classe, et les règles de cohérence sont vérifiées à un seul endroit.
- En Python, le tiret bas (`_solde`) est une **convention** : rien n'est techniquement interdit, mais on s'engage à passer par les accesseurs et mutateurs.

Sujet 3.

- `__init__` est le **constructeur** : il est appelé automatiquement à la création de l'objet (`Chien('Rex', 4)`) et initialise ses attributs.
- `self` désigne l'instance sur laquelle la méthode agit : `rex.aboyer()` revient à `Chien.aboyer(rex)`.
- Exemple : dans `self.nom = nom`, `self.nom` est l'attribut de l'objet, `nom` le paramètre reçu.
- Piège : oublier `self` dans la liste des paramètres, ou écrire `nom = nom` (une variable locale, perdue à la fin de la méthode).

Modularité

Exercice 24 *Créer et documenter un module* ↑énoncé p. 97

1. `choice(["pile", "face"])` renvoie l'un des deux éléments, `"pile"` ou `"face"`, au hasard. La documentation exige une séquence **non vide** (« *non-empty sequence* ») : `choice([])` ne respecte pas la spécification. Dans la console, l'appel lève une erreur : `IndexError: Cannot choose from an empty sequence`.

2. *Appels attendus, écrits avant de coder :*

```
assert en_hms(3723) == (1, 2, 3)      # cas normal
assert en_hms(0) == (0, 0, 0)         # cas limite : duree nulle
assert en_hms(3600) == (1, 0, 0)      # cas piege : 60 min font 1 h, pas (0, 60, 0)
```

Fichier `temps.py` :

```
"""Module temps : conversions entre secondes et heures-minutes-secondes."""

def en_secondes(h, m, s):
    """Renvoie la duree h heures, m minutes, s secondes, en secondes."""
    return h * 3600 + m * 60 + s

def en_hms(duree):
    """Renvoie le triplet (h, m, s) d'une duree en secondes (entier >= 0),
    avec 0 <= m < 60 et 0 <= s < 60."""
    h = duree // 3600
    reste = duree % 3600
    m = reste // 60
    s = reste % 60
    return (h, m, s)

if __name__ == "__main__":
    assert en_secondes(1, 2, 3) == 3723
    assert en_secondes(0, 0, 0) == 0
    assert en_hms(3723) == (1, 2, 3)
    assert en_hms(59) == (0, 0, 59)
    print("Tests du module temps : OK")
```

3. Fichier `trajet.py` :

```
from temps import en_secondes, en_hms

etape1 = en_secondes(1, 47, 30)
etape2 = en_secondes(2, 25, 45)
print(en_hms(etape1 + etape2))     # affiche (4, 13, 15)
```

$6\,450 + 8\,745 = 15\,195$ secondes, soit 4 h 13 min 15 s. (`import temps` puis `temps.en_secondes(...)` convient aussi.)

4. `help(temps)` affiche la **documentation** du module : la docstring de tête, puis la liste des fonctions avec leurs paramètres et leurs docstrings. C'est pour cela qu'on les écrit : l'utilisateur sait se servir du module sans lire son code. Les tests ne s'exécutent **pas** : quand `temps.py` est importé, la variable `__name__` vaut `"temps"` et non `"__main__"`, donc le bloc protégé est sauté. Il ne s'exécute que si l'on lance `temps.py` lui-même.

Réponses des questions de la fiche de révision

↑ questions p. 114

Se tester

1. `Voiture` est la classe (le moule) ; `v` est l'instance (l'objet) (§ III.B).
2. `15` : le retrait de 20 est refusé (il ferait passer le solde en négatif), puis on dépose 5 (§ VI).
3. aucun attribut n'est créé : `AttributeError` dès qu'on lit `obj.nom` (il fallait `self.nom = nom`) (§ IV.A).
4. non : elle modifie la liste reçue (effet de bord) (§ II).
5. l'objet : chaque entité porte ses données et son comportement (§ VIII).

Et avant ?

1. si `n == 0` : renvoyer `1` ; sinon renvoyer `x * puissance(x, n - 1)` (ch. 1, § III.B).
2. la longueur de `tab` : entier positif qui diminue de 1 à chaque appel, jusqu'à la liste vide (ch. 1, § IV.A).
3. `d["Alan"]` (qui vaut `12`) ; `d["Grace"] = 18` (Première).

CORRIGÉS DU CHAPITRE 3

3 Structures linéaires : piles, files, listes chaînées

CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Par où entre-t-on, par où sort-on ?

↑ énoncé p. 127

Exercice 1 *Règle du tas*

1. `4`, `3`, `2`, `1` : l'ordre est **inversé**.

2. Cartes prises : `2`, puis `4`, puis `7`. Tas final : la seule carte `5`, qui est à la fois au fond et au dessus.

3. Il faut retirer les **sept** cartes posées par-dessus : seul le dessus est accessible.

Exercice 2 *Règle de la rangée*

4. `1`, `2`, `3`, `4` : l'ordre est **conservé**.

5. Cartes prises : `5`, puis `2`, puis `7`. Rangée finale : la seule carte `4`. Même séquence qu'à la question 2, mais pas les mêmes cartes prises : seule la règle a changé.

6. Le tas renverse l'ordre (pile d'assiettes, pile de copies à corriger, bouton « page précédente ») ; la rangée le conserve (file d'attente à la cantine, documents envoyés à une imprimante, caisse d'un magasin).

Exercice 3 *Le bouton « Annuler »*

7. Il reste `Le chat`. Ont été annulés `bien`, puis `dort` : la **dernière** action est annulée la première.

8. `dort` revient : c'est le dernier mot annulé. Les mots annulés se rangent donc en **tas** (le dernier annulé est le premier refait). L'éditeur tient **deux tas** : celui des actions faites (pour annuler) et celui des actions annulées (pour refaire). Annuler fait passer une carte du premier tas au second ; refaire fait l'inverse.

9. Non : `bien` suivait `dort`, qui n'est plus dans le texte ; le refaire après `mal` donnerait un texte qui n'a jamais existé. La plupart des éditeurs **vident** le tas des actions annulées dès qu'une nouvelle action est faite.

Exercice 4 *Ce qu'on a découvert*

10. Avec la règle du tas, la carte qui sort est toujours **la dernière entrée**. Avec la règle de la rangée, c'est toujours **la première entrée** (la plus ancienne). Ce qui distingue les deux paquets, ce n'est pas leur contenu mais **la façon d'ajouter et de retirer** : par où on entre, par où on sort.

CORRIGÉS DES EXERCICES

Piles

Exercice 1 *La bonne opération sur une pile* ↑énoncé p. 142

1. `p.empiler(7)` **2.** `p.depiler()` **3.** `p.est_vide()` **4.** `p.sommet()`

Exercice 2 *Trace d'une pile* ↑énoncé p. 142

Valeurs renvoyées : `p.depiler()` → `2` ; `p.sommet()` → `9` ; `p.depiler()` → `9`.

▷ **Erreur fréquente.** Traiter `sommet()` comme `depiler()` : `sommet` renvoie l'élément du sommet *sans le retirer*. Sinon, le dernier `depiler()` renverrait `7` et il ne resterait que `4`.

Contenu final (sommet en haut) : $\boxed{7}$ au sommet, $\boxed{4}$ en dessous. Détail :

Instruction	Renvoie	Pile écrite en ligne (sommet à droite)
`empiler(4)`	—	4
`empiler(7)`	—	4 ; 7
`empiler(2)`	—	4 ; 7 ; 2
`depiler()`	2	4 ; 7
`empiler(9)`	—	4 ; 7 ; 9
`sommet()`	9	4 ; 7 ; 9
`depiler()`	9	4 ; 7

Exercice 3 *Parenthésage à la main* ↑énoncé p. 142

a) `{[()]}` : on empile `{`, `[`, `(` ; `)` dépile `(` (ok), `]` dépile `[` (ok), `}` dépile `{` (ok). Pile vide en fin ⇒ **correcte**.

b) `(]` : on empile `(` ; `]` dépile `(`, or `]` attend `[` ⇒ mauvais appariement ⇒ **incorrecte**.

c) `([` : on empile `(`, `[` ; `]` dépile `[` (ok). En fin, la pile n'est **pas vide** (il reste `(`) ⇒ **incorrecte** (une ouvrante jamais refermée).

d) `())(` : on empile `(` ; `)` dépile `(` (ok, pile vide) ; `)` suivante : la pile est **vide** ⇒ fermante sans ouvrante ⇒ **incorrecte**.

▷ **Erreur fréquente.** Conclure « correcte » pour `([` parce qu'aucune erreur n'est apparue pendant la lecture : il faut encore vérifier qu'à la fin la pile est **vide**.

Exercice 4 *Programmer le parenthésage* ↑énoncé p. 143

Démarche : on reprend l'algorithme du cours : chaque ouvrante est empilée ; une fermante doit trouver au sommet l'ouvrante qui lui correspond (un dictionnaire donne cette correspondance) ; à la fin, la pile doit être vide. *Appels attendus, écrits avant de coder :*

```
assert parenthesage_correct("{[()]}")       # cas normal
assert parenthesage_correct("")             # cas limite : chaine vide
assert not parenthesage_correct("([)]")     # cas piege : bons nombres, mauvais ordre
```

```
def parenthesage_correct(expr):
    p = Pile()                              # INITIALISER
    correspond = {')': '(', ']': '[', '}': '{'}
    for c in expr:                          # PARCOURIR
        if c in "([{":                      # TESTER : ouvrante, en attente
            p.empiler(c)                    # METTRE A JOUR
        elif c in ")]}":                    # TESTER : fermante
            if p.est_vide():                # TESTER : rien a refermer
                return False                # RENVOYER
            if p.depiler() != correspond[c]:   # TESTER : mauvais appariement
                return False                # RENVOYER
    return p.est_vide()                     # RENVOYER : tout est-il referme ?
```

Tests : `{[()]}` → `True` ; `(]` → `False` ; `([)` → `False` ; `())(` → `False`.

▷ **Erreur fréquente.** Appeler `p.depiler()` sans tester d'abord `p.est_vide()` : sur `())(`, la deuxième fermante ferait dépiler une pile vide, ce qui provoque une erreur au lieu de renvoyer `False`.

Exercice 5 *Renverser avec une pile* ↑énoncé p. 143

Démarche : une pile inverse l'ordre : on empile tous les éléments de `tab`, puis on dépile tant que la pile n'est pas vide, en ajoutant chaque élément dépilé à la nouvelle liste. *Appels attendus, écrits avant de coder :*

```
t = [4, 7, 2]
assert renverser(t) == [2, 7, 4]        # cas normal
assert renverser([]) == []              # cas limite : liste vide
assert t == [4, 7, 2]                   # cas piege : tab n'est pas modifie
```

```
def renverser(tab):
    p = Pile()                          # INITIALISER
    for x in tab:                       # PARCOURIR : on empile tout
        p.empiler(x)                    # METTRE A JOUR
    resultat = []                       # INITIALISER
    while not p.est_vide():             # PARCOURIR : on depile tout
        resultat.append(p.depiler())    # METTRE A JOUR : ordre inverse
    return resultat                     # RENVOYER
```

Exercice 6 *La structure de pile (d'après Asie 2024, jour 1)* ↑énoncé p. 143

Barème indicatif (sur 2) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 1

1. On empile 3, 2, 10 (sommet 10). Premier `depiler` → 10 ; second `depiler` → 2. Donc `elt` vaut **2**. La pile `essai` ne contient plus que 3 (au sommet et au fond).

2. Politique **LIFO** : le dernier empilé (10) sort en premier, puis l'avant-dernier (2). C'est donc 2 qui est la dernière valeur affectée à `elt` ; 3, empilé en premier, reste au fond.

3. Démarche : une boucle `while` qui tourne tant que la pile n'est pas vide ; chaque tour dépile un élément et l'affiche.

```
def vider(p):
    while not p.estVide():       # PARCOURIR : jusqu'a la pile vide
        print(p.depiler())
```

Pour `3, 2, 10` empilés dans cet ordre, l'affichage est 10, puis 2, puis 3 (du sommet vers le fond).

Exercice 7 *La taille d'une pile* ↑énoncé p. 143

1. Le programme affiche `3`, puis `0`. Pour compter, la fonction **dépile tout** : elle modifie la pile reçue en paramètre (**effet de bord**), qui est vide après le premier appel. Connaître la taille d'une pile ne doit pas la détruire.

▷ **Erreur fréquente.** Répondre que le programme affiche deux fois `3` : la fonction a vidé la pile lors du premier appel, le second trouve une pile vide.

2. On range chaque élément dépilé dans une pile auxiliaire, puis on le remet dans `p` : *Appels attendus, écrits avant de coder :*

```
p = Pile()
for x in [4, 7, 2]:
    p.empiler(x)
assert taille(p) == 3                    # cas normal
assert taille(Pile()) == 0               # cas limite : pile vide
assert taille(p) == 3 and p.sommet() == 2   # cas piege : p n'a pas ete videe
```

```
def taille(p):
    aux = Pile()                    # INITIALISER
    n = 0                           # INITIALISER
    while not p.est_vide():         # PARCOURIR : on vide p dans aux
        aux.empiler(p.depiler())    # METTRE A JOUR
        n = n + 1                   # METTRE A JOUR : on compte
    while not aux.est_vide():       # PARCOURIR : on remet tout dans p
        p.empiler(aux.depiler())    # METTRE A JOUR
    return n                        # RENVOYER
```

3. Fin du comptage : `p` est vide et `aux` contient, du fond vers le sommet, `2`, `7`, `4` (l'ordre est **inversé**). Fin de l'appel : `aux` est vide et `p` contient de nouveau `4`, `7`, `2` (`2` au sommet) : le second transvasement inverse une seconde fois l'ordre, ce qui le rétablit.

Exercice 8 *Copier une pile* ↑énoncé p. 144

1. `q = p` ne crée aucune pile : `q` et `p` sont **deux noms pour le même objet**. `q.depiler()` retire donc aussi le sommet `2` de `p`.

2. À la fin, `p` est vide et `q` contient `2`, `7`, `4` (`4` au sommet). Deux défauts : l'ordre est **inversé** (le sommet de `q` est le fond de `p`) et `p` a été **détruite**.

3. Deux transvasements ; pendant le second, chaque élément est empilé à la fois dans `p` (restauration) et dans la copie : *Appels attendus, écrits avant de coder :*

```
p = Pile()
for x in [4, 7, 2]:
    p.empiler(x)
q = copier_pile(p)
assert q.depiler() == 2 and q.depiler() == 7   # cas normal : meme ordre
assert copier_pile(Pile()).est_vide()          # cas limite : pile vide
assert p.sommet() == 2                         # cas piege : p intacte
```

```
def copier_pile(p):
    aux = Pile()                    # INITIALISER
    while not p.est_vide():         # PARCOURIR : p renversee dans aux
        aux.empiler(p.depiler())    # METTRE A JOUR
    q = Pile()                      # INITIALISER
    while not aux.est_vide():       # PARCOURIR : retour a l'ordre d'origine
        x = aux.depiler()
        p.empiler(x)                # METTRE A JOUR : on restaure p
        q.empiler(x)                # METTRE A JOUR : on construit la copie
    return q                        # RENVOYER
```

Test : avec `p` contenant `4`, `7`, `2`, `q = copier_pile(p)` puis `q.depiler()` renvoie `2`, et `p.sommet()` vaut toujours `2` : les deux piles sont bien indépendantes.

Exercice 9 *Défi — des balises bien imbriquées* ↑énoncé p. 144

Appels attendus, écrits avant de coder :

```
assert balises_ok(["<b>", "<i>", "</i>", "</b>"])       # cas normal
assert balises_ok([])                                    # cas limite : aucune balise
assert not balises_ok(["<b>", "<i>", "</b>", "</i>"])   # cas piege : mauvais ordre
```

```
def balises_ok(liste):
    p = Pile()                          # INITIALISER
    for b in liste:                     # PARCOURIR
        if b[:2] != "</":               # TESTER : ouvrante <x>
            p.empiler(b)                # METTRE A JOUR
        else:                           # fermante </x>
            if p.est_vide():            # TESTER : rien a fermer
                return False            # RENVOYER
            ouvrante = p.depiler()
            if ouvrante != "<" + b[2:]:     # TESTER : "</b>" attend "<b>"
                return False            # RENVOYER
    return p.est_vide()                 # RENVOYER
```

`["<b>", "<i>", "</i>", "</b>"]` → `True` ; `["<b>", "<i>", "</b>", "</i>"]` → `False` (à `</b>`, on dépile `<i>` qui ne correspond pas).

Exercice 10 *Défi — notation polonaise inverse* ↑énoncé p. 144

Appels attendus, écrits avant de coder :

```
assert evaluer_npi("3 4 +") == 7      # cas normal
assert evaluer_npi("42") == 42        # cas limite : un seul nombre, aucun operateur
assert evaluer_npi("10 4 -") == 6     # cas piege : l'ordre des operandes (pas -6)
```

```
def evaluer_npi(expr):
    p = Pile()                          # INITIALISER
    for jeton in expr.split():          # PARCOURIR
        if jeton in "+-*":              # TESTER : un operateur
            b = p.depiler()             # 2e operande (depile en premier)
            a = p.depiler()             # 1er operande
            if jeton == "+":
                p.empiler(a + b)        # METTRE A JOUR
            elif jeton == "-":
                p.empiler(a - b)        # METTRE A JOUR : attention, a - b
            else:
                p.empiler(a * b)        # METTRE A JOUR
        else:
            p.empiler(int(jeton))       # METTRE A JOUR : un nombre
    return p.depiler()                  # RENVOYER : le nombre restant
```

`"3 4 +"` → `7` ; `"5 1 2 + 4 * +"` → `17` (on empile 5, puis 1+2=3, puis 3*4=12, puis 5+12=17).

Files

Exercice 11 *La bonne opération sur une file* ↑énoncé p. 145

1. `f.enfiler(7)` **2.** `f.defiler()` **3.** `f.est_vide()` **4.** `f.tete()`

Exercice 12 *Trace d'une file* ↑énoncé p. 145

Valeurs renvoyées : `f.defiler()` → 4 ; `f.tete()` → 7 ; `f.defiler()` → 7.

Contenu final (tête à droite) : 9 ; 2, soit 2 en tête. Détail :

Instruction	Renvoie	File (tête à droite)
`enfiler(4)`	—	4
`enfiler(7)`	—	7 ; 4
`enfiler(2)`	—	2 ; 7 ; 4
`defiler()`	4	2 ; 7
`enfiler(9)`	—	9 ; 2 ; 7
`tete()`	7	9 ; 2 ; 7
`defiler()`	7	9 ; 2

Exercice 13 *Ordonnancer des processus avec une file (d'après Amérique du Nord 2024, jour 1)* ↑énoncé p. 145

Barème indicatif (sur 4) : Q1 : 0,75 ; Q2 : 1 ; Q3 : 1,5 ; Q4 : 0,75

1. L'unique attribut est `contenu`, de type `list` (une liste Python). `enfile` ajoute en queue avec `append` (immédiat), tandis que `defile` retire la tête avec `pop(0)`, ce qui oblige à **décaler tous les éléments restants** : le coût est proportionnel au nombre d'éléments (cf. cours).

2. Quand la file est vide, `self.contenu` vaut `[]` et `pop(0)` sur une liste vide lève une `IndexError`. On protège la méthode :

```
def defile(self):
    if self.contenu == []:
        return None
    return self.contenu.pop(0)
```

3. Démarche : chaque ligne appelle une méthode de l'interface : on défile le processus élu, on l'exécute un cycle, puis on le réenfile (en fin de file) s'il n'est pas fini.

```
def ajoute_nouveau_processus(self, proc):
    self.file.enfile(proc)

def tourniquet(self):
    self.temps = self.temps + 1
    if not self.file.est_vide():
        proc = self.file.defile()      # (a) le processus elu
        proc.execute()                 # (b) un cycle consomme
        if not proc.est_fini():
            self.file.enfile(proc)     # (c) remis en fin de file
        return proc.nom
    else:
        return None
```

4. On donne ici le contenu de l'attribut `contenu` (liste Python : la tête est l'indice `0`, donc **à gauche**, à l'inverse de la convention de dessin « tête à droite » de la feuille). File de départ : `[p1(2), p2(1)]`.

- `tourniquet()` : on défile `p1`, il s'exécute (reste `1`), pas fini ⇒ réenfilé. File : `[p2(1), p1(1)]`. Renvoie `"p1"`.
- `tourniquet()` : on défile `p2`, il s'exécute (reste `0`), fini ⇒ non réenfilé. File : `[p1(1)]`. Renvoie `"p2"`.
- `tourniquet()` : on défile `p1`, il s'exécute (reste `0`), fini. File : `[]`. Renvoie `"p1"`.

Noms renvoyés : `"p1"`, `"p2"`, `"p1"` ; la file finale est **vide**.

▷ **Erreur fréquente.** Faire exécuter `p1` deux fois de suite : après son premier cycle, il est remis en **fin** de file, derrière `p2`, qui passe donc avant lui.

Exercice 14 *Une file de priorité de tâches (d'après Métropole 2025, jour 1)*

↑énoncé p. 146

Barème indicatif (sur 4) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 1 ; Q4 : 0,5 ; Q5 : 0,5 ; Q6 : 1

1. `defiler` retire la tête, ici `(<t3>, 4)` ; `f.defiler()[0]` vaut donc `<t3>` (la tâche). Après : `(<t5>, 1)` ; `(<t4>, 1)` ; `(<t2>, 3)` ; `(<t1>, 3)` (tête = `(<t1>, 3)`).

▷ **Erreur fréquente.** Défiler `(<t5>, 1)`, l'élément écrit en premier : la tête est à **droite**, c'est `(<t3>, 4)` qui sort.

2. `examiner` renvoie la tête *sans* la retirer, ici `(<t3>, 4)` ; `f.examiner()[1]` vaut donc `4` (la priorité). La file `f` est **inchangée**.

3. Démarche : on défile vers `f_aux` les tâches plus prioritaires que `p`, on enfile `(t, p)`, puis on transvase le reste.

```
def ajouter_file_prio(f, t, p):
    f_aux = File()                      # INITIALISER
    while (not f.est_vide()) and f.examiner()[1] > p:   # PARCOURIR
        f_aux.enfiler(f.defiler())      # METTRE A JOUR : les plus prioritaires
    f_aux.enfiler((t, p))               # METTRE A JOUR : (t, p) a sa place
    while not f.est_vide():             # PARCOURIR : le reste
        f_aux.enfiler(f.defiler())      # METTRE A JOUR : on transvase
    return f_aux                        # RENVOYER
```

(Attention : avec la condition stricte `> p`*, la boucle s'arrête dès qu'elle rencontre une tâche de* même *priorité ;* `(t, p)` *est donc placé* devant *les tâches de même priorité déjà présentes et passe en premier. Pour respecter l'ordre d'arrivée à priorité égale, il faudrait écrire* `>= p`*.)*

4. En insérant `(t6, 2)` (entre les priorités 3 et 1) puis `(t7, 4)` : la tête `(t3, 4)` n'est pas *strictement* plus prioritaire, donc la boucle ne tourne pas et `t7` est enfilé en premier — il passe *devant* `t3`. On obtient (tête à droite) : `(t5, 1)` ; `(t4, 1)` ; `(t6, 2)` ; `(t2, 3)` ; `(t1, 3)` ; `(t3, 4)` ; `(t7, 4)`.

5. Au pire, on défile puis ré-enfile les `m` éléments de `f` : le coût est **proportionnel à `m`** (linéaire).

6. Démarche : tant que la file n'est pas vide : défiler, avancer la tâche, la noter, et la réinsérer à sa place (avec sa priorité) si elle n'est pas terminée.

```
def planning(f):
    resultat = []                       # INITIALISER
    while not f.est_vide():             # PARCOURIR
        tache, prio = f.defiler()
        tache.avancer(25)
        resultat.append(tache)          # METTRE A JOUR
        if not tache.est_terminee():    # TESTER
            f = ajouter_file_prio(f, tache, prio)  # METTRE A JOUR : reinseree
    return resultat                     # RENVOYER
```

Exercice 15 *Une file écrite par un assistant d'IA*

↑énoncé p. 147

1. Non. Après les trois `enfiler`, `contenu` vaut `[7, 19, 22]` ; `f.tete()` renvoie bien `7`, mais `f.defiler()` renvoie `22`, le *dernier* entré, et laisse `[7, 19]`. Le premier entré, `7`, n'est pas le premier sorti : la structure se comporte comme une **pile**, et `tete` et `defiler` ne désignent même pas le même élément.

2. L'erreur : `pop()` sans argument retire le **dernier** élément de la liste (la queue). Pour retirer la tête, placée au début :

```
    def defiler(self):
        return self.contenu.pop(0)   # retire et renvoie le premier = tete
```

La séquence donne alors `f.defiler()` → `7`, et il reste `[19, 22]`.

3. Enfiler **deux ou trois valeurs distinctes** puis défiler une fois : on doit ressortir la *première* entrée. Vérifier aussi la cohérence interne de la classe : `tete()` et `defiler()` doivent parler du **même** élément.

Exercice 16 *Copier une file* ↑énoncé p. 147

1. Oui : la file est FIFO, le premier défilé est le premier enfilé dans `copie`, qui reçoit donc les éléments dans le même ordre. Avec une pile, la même méthode *inverse* l'ordre (exercice 8). Il manque la restauration : `f` a été vidée.

2. Chaque tour de boucle défile un élément puis le réenfile : `f` garde toujours le même nombre d'éléments et n'est **jamais vide**. La condition `not f.est_vide()` reste vraie : boucle infinie (et `copie` grossit sans fin).

3. Une file auxiliaire garde les éléments, puis on les remet dans `f` : *Appels attendus, écrits avant de coder :*

```
f = File()
for x in [3, 8, 5]:
    f.enfiler(x)
c = copier_file(f)
assert c.defiler() == 3 and c.defiler() == 8   # cas normal : meme ordre
assert copier_file(File()).est_vide()          # cas limite : file vide
assert f.tete() == 3                           # cas piege : f n'a pas ete videe
```

```
def copier_file(f):
    aux = File()                    # INITIALISER
    copie = File()                  # INITIALISER
    while not f.est_vide():         # PARCOURIR
        x = f.defiler()
        copie.enfiler(x)            # METTRE A JOUR : la copie, dans l'ordre
        aux.enfiler(x)              # METTRE A JOUR : de quoi restaurer f
    while not aux.est_vide():       # PARCOURIR : f retrouve son etat
        f.enfiler(aux.defiler())    # METTRE A JOUR
    return copie                    # RENVOYER
```

Le transvasement de `aux` vers `f` conserve l'ordre : `f` retrouve les mêmes éléments, avec la même tête.

Piles et files ensemble

Exercice 17 *LIFO ou FIFO ?* ↑énoncé p. 148

a) **Pile** (LIFO) : « annuler » retire la *dernière* action effectuée.
b) **File** (FIFO) : les documents s'impriment dans l'*ordre d'arrivée*.
c) **Pile** (LIFO) : la dernière fonction appelée est la première dont on revient (pile d'appels).
d) **File** (FIFO) : les caractères sont traités dans l'ordre où ils ont été tapés.
e) **Pile** (LIFO) : « précédent » revient à la *dernière* page visitée.

Exercice 18 *Même séquence, deux structures* ↑énoncé p. 148

1. De la pile : `4, 3, 2, 1` (le dernier entré sort en premier). **2.** De la file : `1, 2, 3, 4` (le premier entré sort en premier). **3.** La **pile** inverse l'ordre ; la **file** le conserve.

Exercice 19 *Piles et files pour parcourir un graphe (d'après Polynésie 2024, jour 2)* ↑énoncé p. 148

Barème indicatif (sur 3) : Q1 : 0,5 ; Q2 : 0,75 ; Q3 : 1,25 ; Q4 : 0,5

1. LIFO = *Last In, First Out* (dernier entré, premier sorti) : c'est la **pile**. **FIFO** = *First In, First Out* (premier entré, premier sorti) : c'est la **file**.

2. On défile `A` et on ajoute `B`, `C` ; on défile `B` et on ajoute `D` ; on défile `C` puis `D` (voisins déjà vus). Ordre d'entrée dans `visite` : `A, B, C, D`.

3. Démarche : on traduit l'algorithme ligne à ligne ; c'est la fonction `parcours` où la file est remplacée par une pile, et où un sommet n'est marqué visité qu'au dépilement.

```python
def parcours2(graphe, depart):
    p = Pile()                    # INITIALISER
    p.empiler(depart)             # INITIALISER
    visite = []                   # INITIALISER
    while not p.est_vide():       # PARCOURIR
        x = p.depiler()
        if x not in visite:       # TESTER
            visite.append(x)      # METTRE A JOUR
            for v in graphe[x]:   # PARCOURIR : les voisins
                p.empiler(v)      # METTRE A JOUR
    return visite                 # RENVOYER
```

4. Un résultat possible : `['A', 'C', 'D', 'B']` (l'ordre dépend de l'ordre d'empilement des voisins). Avec la pile, on repart toujours du dernier sommet ajouté : l'ordre de visite diffère de celui de la question 2.

Exercice 20 *Défi — une file avec deux piles* ↑énoncé p. 149

1. `enfiler(1)`, `enfiler(2)`, `enfiler(3)` ⇒ `entree` contient `1, 2, 3` (sommet 3), `sortie` vide. Premier `defiler` : `sortie` étant vide, on transvase `entree` dans `sortie` (on dépile `3, 2, 1` et on les empile) ⇒ `sortie` contient `3, 2, 1` avec 1 au sommet ; on dépile ⇒ **1**. Second `defiler` : `sortie` n'est pas vide, on dépile ⇒ **2**. On obtient bien 1 puis 2 : comportement **FIFO**.

2. Une pile *inverse* l'ordre. En transvasant `entree` dans `sortie`, on inverse une fois : l'élément le plus *ancien* (entré en premier) se retrouve au *sommet* de `sortie`, donc prêt à sortir en premier. C'est cette double inversion (empilement puis transvasement) qui rétablit l'ordre d'arrivée.

Listes chaînées

Exercice 21 *Longueur d'une liste (avec une boucle)* ↑énoncé p. 149

Démarche : une variable `c` sert de curseur : elle part de la première cellule et avance en suivant le lien `suivante` ; on compte une cellule à chaque tour, et l'on s'arrête quand le curseur vaut `None`. *Appels attendus, écrits avant de coder :*

```python
lst = Cellule(1, Cellule(2, Cellule(3, None)))
assert longueur(lst) == 3                 # cas normal
assert longueur(None) == 0                # cas limite : liste vide
assert longueur(Cellule(5, None)) == 1    # cas piege : la derniere cellule compte
```

```python
def longueur(lst):
    n = 0                     # INITIALISER
    c = lst                   # INITIALISER : le curseur
    while c is not None:      # PARCOURIR
        n = n + 1             # METTRE A JOUR
        c = c.suivante        # METTRE A JOUR : on avance d'une cellule
    return n                  # RENVOYER
```

▷ **Erreur fréquente.** Écrire `while c.suivante is not None` : la dernière cellule n'est pas comptée, et la liste vide provoque une erreur (`None` n'a pas d'attribut `suivante`).

Exercice 22 *Nombre d'occurrences* ↑énoncé p. 149

Démarche : même parcours que pour la longueur, mais le compteur n'augmente que si la valeur de la cellule est `x`. *Appels attendus, écrits avant de coder :*

```python
lst = Cellule(2, Cellule(5, Cellule(2, None)))
assert occurrences(5, lst) == 1       # cas normal
assert occurrences(2, None) == 0      # cas limite : liste vide
assert occurrences(2, lst) == 2       # cas piege : en tete et en derniere cellule
```

```python
def occurrences(x, lst):
    n = 0                     # INITIALISER
    c = lst                   # INITIALISER
    while c is not None:      # PARCOURIR
```

```
        if c.valeur == x:      # TESTER
            n = n + 1          # METTRE A JOUR
        c = c.suivante         # METTRE A JOUR : on avance
    return n                   # RENVOYER
```

Autre méthode : récursivement, la liste vide contient `0` fois `x` ; sinon, on compte la première cellule (`1` ou `0`) et on ajoute les occurrences dans le reste.

```
def occurrences(x, lst):
    if lst is None:                          # CAS DE BASE : liste vide
        return 0
    if lst.valeur == x:                      # TESTER
        return 1 + occurrences(x, lst.suivante)  # APPEL RECURSIF, COMBINER
    return occurrences(x, lst.suivante)      # APPEL RECURSIF
```

Exercice 23 *Le n-ième élément* ↑énoncé p. 149

Démarche : en récursif, deux conditions d'arrêt : la liste est vide (l'indice est trop grand : on lève `IndexError`) ou `n == 0` (on renvoie la valeur de la première cellule). Sinon, l'élément d'indice `n` de la liste est l'élément d'indice `n - 1` du reste. *Appels attendus, écrits avant de coder :*

```
lst = Cellule(4, Cellule(7, Cellule(2, None)))
assert nieme(1, lst) == 7      # cas normal
assert nieme(0, lst) == 4      # cas limite : l'indice 0, la tete
assert nieme(2, lst) == 2      # cas piege : le dernier indice est longueur - 1
```

```
def nieme(n, lst):
    if lst is None:                          # CAS DE BASE : liste trop courte
        raise IndexError("indice invalide")
    if n == 0:                               # CAS DE BASE : on y est
        return lst.valeur                    # RENVOYER
    return nieme(n - 1, lst.suivante)        # APPEL RECURSIF
```

Autre méthode : avec une boucle, on suit `n` fois le lien `suivante` ; si l'on tombe sur `None` en chemin, la liste est trop courte.

```
def nieme(n, lst):
    c = lst                                  # INITIALISER
    for i in range(n):                       # PARCOURIR : n liens a suivre
        if c is None:                        # TESTER : liste trop courte
            raise IndexError("indice invalide")
        c = c.suivante                       # METTRE A JOUR
    if c is None:                            # TESTER
        raise IndexError("indice invalide")
    return c.valeur                          # RENVOYER
```

Exercice 24 *Le collier de bonbons : maillons chaînés (d'après Amérique du Nord 2025, jour 2)* ↑énoncé p. 150

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 0,75 ; Q3 : 1,5 ; Q4 : 0,75 ; Q5 : 0,5 ; Q6 : 2

1. Ce sont des **attributs** de la classe `Bonbon`. `pred` est le **prédécesseur**, `valeur` la **valeur** (le contenu) et `succ` le **successeur** du maillon. `pred` et `succ` sont eux-mêmes des maillons : la structure est une **liste doublement chaînée** (et ici circulaire).

2. `a = zero.succ.valeur` : `zero.succ` est `un`, donc `a = 1`.
`b = un.succ.succ.pred.valeur` : `un.succ` = `deux`, `deux.succ` = `zero`, `zero.pred` = `deux`, donc `b = 2`.

3. Démarche : une variable `precedent` retient le dernier bonbon créé ; chaque nouveau bonbon lui est relié dans les deux sens ; après la boucle, on referme le collier entre le dernier et le premier. *Appels attendus, écrits avant de coder :*

```
b = creer_collier(3)
u = creer_collier(1)
assert b.succ.valeur == 1 and b.succ.succ.succ == b   # cas normal : referme
```

```
assert u.succ == u and u.pred == u                    # cas limite : un seul bonbon
assert b.pred.valeur == 2                             # cas piege : chainage arriere
```

```
def creer_collier(n):
    premier = Bonbon(0)                # INITIALISER
    precedent = premier                # INITIALISER
    for i in range(1, n):              # PARCOURIR
        courant = Bonbon(i)
        precedent.succ = courant       # METTRE A JOUR : chainage avant
        courant.pred = precedent       # METTRE A JOUR : chainage arriere
        precedent = courant            # METTRE A JOUR
    precedent.succ = premier           # on referme le collier
    premier.pred = precedent
    return premier                     # RENVOYER
```

(Pour `n = 1`*, la boucle ne s'exécute pas :* `premier.succ` *et* `premier.pred` *pointent sur* `premier` *lui-même.)*

4. On relie le prédécesseur et le successeur de `deux` :

```
deux.pred.succ = deux.succ
deux.succ.pred = deux.pred
```

5. Un collier réduit à un seul bonbon `b` vérifie `b.succ == b` **et** `b.pred == b` (il est son propre successeur et prédécesseur).

6.

```
def dernier_chaine(n):
    b = creer_collier(n)           # INITIALISER
    while b.succ != b:             # PARCOURIR : plus d'un bonbon
        mange = b.succ             # b mange son successeur
        b.succ = mange.succ        # METTRE A JOUR : "mange" retire
        mange.succ.pred = b        # METTRE A JOUR
        b = b.succ                 # METTRE A JOUR : bonbon suivant
    return b.valeur                # RENVOYER
```

Exercice 25 *Copier une liste chaînée* ↑énoncé p. 150

1. `lst.valeur` vaut 10. L'instruction `autre = lst` ne crée aucune cellule : `autre` et `lst` désignent la **même** première cellule, donc la modifier par l'un des noms la modifie pour l'autre.

2. Démarche : condition d'arrêt : la liste vide, dont la copie est vide ; sinon, une **nouvelle** cellule de même valeur, suivie de la copie du reste. *Appels attendus, écrits avant de coder :*

```
lst = Cellule(1, Cellule(2, None))
c = copier_liste(lst)
assert c.valeur == 1 and c.suivante.valeur == 2   # cas normal
assert copier_liste(None) is None                 # cas limite : liste vide
assert c.suivante is not lst.suivante             # cas piege : le reste aussi est neuf
```

```
def copier_liste(lst):
    if lst is None:                    # CAS DE BASE : liste vide
        return None
    reste = copier_liste(lst.suivante)    # APPEL RECURSIF : copie du reste
    return Cellule(lst.valeur, reste)     # COMBINER : une cellule neuve devant
```

Chaque appel crée une **nouvelle** cellule (`Cellule(...)`) avec la même valeur, suivie de la copie du reste.

▷ **Erreur fréquente.** Renvoyer `Cellule(lst.valeur, lst.suivante)` : seule la première cellule est neuve, le reste est partagé avec `lst`. Il faut copier aussi le reste, par l'appel récursif.

3. `c` est une nouvelle cellule : `c.valeur = 10` ne change qu'elle : les cellules de `lst` ne sont pas touchées.

Exercice 26 *Défi — insertion dans une liste triée* ↑énoncé p. 151

Appels attendus, écrits avant de coder :

```
lst = Cellule(1, Cellule(2, Cellule(5, None)))
assert nieme(2, inserer(3, lst)) == 3     # cas normal : au milieu
assert inserer(3, None).valeur == 3       # cas limite : liste vide
assert nieme(3, inserer(9, lst)) == 9     # cas piege : tout a la fin
```

```
def inserer(x, lst):
    if lst is None or x <= lst.valeur:    # CAS DE BASE : x se place ici
        return Cellule(x, lst)            # devant le reste
    reste = inserer(x, lst.suivante)      # APPEL RECURSIF
    return Cellule(lst.valeur, reste)     # COMBINER : la premiere valeur devant
```

Insérer `3` dans `1, 2, 5, 8` : `3 > 1` et `3 > 2`, puis `3 <= 5` ⇒ on obtient `1, 2, 3, 5, 8`.

S'entraîner à l'oral

Exercice 27 *Expliquer en deux minutes* ↑énoncé p. 151

▶ Solution. Pas de texte à apprendre par cœur : voici les **éléments attendus** pour chaque sujet. L'ordre, les mots et l'exemple peuvent être différents ; l'explication est réussie si ces idées y sont, justes et reliées entre elles.

Sujet 1.

- **Pile** : dernier entré, premier sorti (LIFO) ; opérations empiler, dépiler, sommet ; une pile d'assiettes ; usages : le bouton « Annuler », la pile d'exécution, le retour arrière du navigateur.
- **File** : premier entré, premier sorti (FIFO) ; opérations enfiler, défiler ; une file d'attente ; usages : la file d'impression, l'ordonnancement des processus, le parcours en largeur.
- Exemple : on fait entrer $1, 2, 3$; la pile rend 3 d'abord, la file rend 1.
- Piège : se tromper de bout. La pile se dessine verticalement, sommet en haut ; la file se dessine horizontalement, mais le côté de la tête n'est pas fixé : suivre celui de l'énoncé.

Sujet 2.

- Le **type abstrait** (l'interface) dit *quelles* opérations existent et ce qu'elles font (`empiler`, `depiler`, `est_vide`), pas *comment*.
- L'**implémentation** est un choix concret : une liste Python, une liste chaînée…
- Exemple : un programme qui évalue une expression avec une pile fonctionne sans changement quelle que soit l'implémentation de la classe `Pile`.
- Intérêt : on peut changer d'implémentation (pour gagner en efficacité) sans toucher au code qui l'utilise ; seul le coût des opérations change.

Sujet 3.

- Une liste chaînée est une suite de **maillons** : chacun contient une valeur et un lien vers le maillon suivant ; on ne connaît directement que la **tête**.
- Ajout en tête : on crée un maillon qui pointe vers l'ancienne tête ; quelques opérations, quelle que soit la longueur (coût constant).
- Accès au k-ième élément : il faut suivre k liens depuis la tête ; le coût est proportionnel à k.
- Comparaison avec un tableau : accès direct par l'indice, mais insertion au début coûteuse (il faut décaler tous les éléments).

Interface d'une liste, liste ou dictionnaire

Exercice 28 *L'interface d'une liste* ↑énoncé p. 151

1. On lit de l'intérieur vers l'extérieur : `1`, puis `8` devant, puis `3` devant. `L` est la liste `3, 8, 1`. `tete(L)` → `3` ; `tete(queue(L))` → `8` ; après trois `queue`, il ne reste rien : `est_vide(...)` → `True`.

2. *Appels attendus, écrits avant de coder :*

```
L = ajouter_tete(3, ajouter_tete(8, ajouter_tete(1, liste_vide())))
assert appartient(8, L)                  # cas normal
assert not appartient(8, liste_vide())   # cas limite : liste vide
assert appartient(1, L)                  # cas piege : la derniere valeur
```

```
def appartient(x, L):
    """Renvoie True si x est un element de la liste L."""
    if est_vide(L):                  # CAS DE BASE : liste vide
        return False
    if tete(L) == x:                 # CAS DE BASE : x trouve en tete
        return True
    return appartient(x, queue(L))   # APPEL RECURSIF : dans la queue
```

`appartient(8, L)` → `True` ; `appartient(5, L)` → `False`. Variant : la longueur de `L`, qui diminue de 1 à chaque appel.

3.

```
def renverser(L):
    """Renvoie une nouvelle liste : les elements de L dans l'ordre inverse."""
    r = liste_vide()                   # INITIALISER
    while not est_vide(L):             # PARCOURIR
        r = ajouter_tete(tete(L), r)   # METTRE A JOUR : passe devant r
        L = queue(L)                   # METTRE A JOUR : on avance
    return r                           # RENVOYER
```

Sur `3, 8, 1`, `r` vaut successivement `3`, puis `8, 3`, puis `1, 8, 3`. La variable locale `L` avance dans la liste, mais la liste reçue n'est pas modifiée.

4.

```
def liste_vide():
    return None

def est_vide(L):
    return L is None

def ajouter_tete(x, L):
    return Cellule(x, L)

def tete(L):
    return L.valeur

def queue(L):
    return L.suivante
```

Les fonctions des questions 2 et 3 marchent sans changement avec cette implémentation : elles n'utilisent que l'interface.

5. Non : `ajouter_tete` **renvoie** une nouvelle liste `9, 3, 8, 1`, dont la suite est `L` ; `L` vaut toujours `3, 8, 1`. Au contraire, `p.empiler(9)` ne renvoie rien et **modifie** la pile `p`. Les deux structures ne se distinguent pas par leur contenu, mais par leur **jeu de méthodes**.

Exercice 29 *Liste ou dictionnaire ?* ↑énoncé p. 152

1. `b in autorises` compare `b` aux éléments un par un : au pire n comparaisons, quand `b` est absent (ou est le dernier élément). Coût linéaire, en $O(n)$.

2. Python ne parcourt pas les clés : il calcule, à partir de `b`, l'endroit où cette clé serait rangée. Le coût est à peu près constant, en $O(1)$, quel que soit n.

3.

```
    debut = time.perf_counter()
    trouve = -1 in badges
    t_dico = time.perf_counter() - debut
```

Ordres de grandeur obtenus (ils dépendent de la machine) :

n	**liste**	**dictionnaire**
10 000	environ 0,1 ms	moins d'une microseconde
100 000	environ 1 ms	moins d'une microseconde
1 000 000	environ 10 ms	de l'ordre d'une microseconde

Quand n est multiplié par 10, le temps de la liste est multiplié par 10 environ (coût linéaire) ; celui du dictionnaire ne change presque pas (les petites variations sont dues à la mesure elle-même).

4. Non. Ce sont les **clés** qui se trouvent en temps constant. `badges.values()` est parcouru valeur par valeur : $O(n)$, comme une liste. Si l'on cherche souvent par nom, il faut un second dictionnaire dont les clés sont les noms.

5. Pour les passages, une **liste** (ou une **file**) : on ajoute chaque passage à la fin, et l'ordre est conservé. Pour les badges autorisés, le **dictionnaire**, puisque l'opération la plus fréquente est « ce badge est-il autorisé ? ». Un même programme peut utiliser les deux structures, chacune pour ce qu'elle fait bien.

CORRIGÉS DES TP

TP Piles et files au travail ↑énoncé p. 155

Avec ces solutions, les huit tests du fichier `tp_structures_lineaires_depart.py` affichent `[OK]`.

1 Deux implémentations pour une même interface

Exercice 1 *Une pile chaînée*

1. `empiler(7)` crée une cellule `7` dont le lien pointe vers l'ancienne tête `3`, et `_tete` désigne désormais cette cellule : `7 → 3 → 8 → 5 → None`. `depiler()` renvoie `7` et ramène `_tete` sur `3`. Chaque opération modifie **un** lien (plus la création d'une cellule), quelle que soit la taille de la pile : coût constant.

2. et 4.

```
class PileChainee:
    def __init__(self):
        self._tete = None
        self._taille = 0
    def est_vide(self):
        return self._tete is None
```

```
    def empiler(self, x):
        self._tete = Cellule(x, self._tete)      # nouvelle cellule devant
        self._taille = self._taille + 1
    def depiler(self):
        if self.est_vide():
            raise IndexError("pile vide")
        v = self._tete.valeur
        self._tete = self._tete.suivante         # la tete avance d'un cran
        self._taille = self._taille - 1
        return v
    def sommet(self):
        if self.est_vide():
            raise IndexError("pile vide")
        return self._tete.valeur
    def taille(self):
        return self._taille
    def __repr__(self):
        texte = ""
        c = self._tete
        while c is not None:                     # on parcourt depuis le sommet
            texte = texte + str(c.valeur)
            if c.suivante is not None:           # pas de virgule apres le dernier
                texte = texte + ", "
            c = c.suivante
        return "sommet -> [" + texte + "]"
```

3. Les deux appels renvoient `"OK"`. La fonction de test n'utilise que les méthodes de l'**interface** : toute classe qui les fournit avec le bon comportement passe le test, quelle que soit son implémentation.

Exercice 2 *Une file chaînée à deux entrées*

1. Après `enfiler(1)` : `5 → 8 → 3 → 1 → None`, `_queue` sur `1`. Après `defiler()` (qui renvoie `5`) : `_tete` sur `8`. Sans `_queue`, il faudrait parcourir toute la chaîne depuis la tête pour trouver la dernière cellule : `enfiler` coûterait n opérations.

2. Enfiler dans une file vide : la nouvelle cellule devient **à la fois** tête et queue. Défiler le dernier élément : `_tete` devient `None`, et `_queue` doit redevenir `None` elle aussi (sinon elle désigne une cellule qui n'est plus dans la file).

3.

```
class FileChainee:
    def __init__(self):
        self._tete = None
        self._queue = None
        self._taille = 0
    def est_vide(self):
        return self._tete is None
    def enfiler(self, x):
        c = Cellule(x, None)
        if self.est_vide():
            self._tete = c
        else:
            self._queue.suivante = c      # on accroche derriere la queue
        self._queue = c
        self._taille = self._taille + 1
    def defiler(self):
        if self.est_vide():
            raise IndexError("file vide")
        v = self._tete.valeur
        self._tete = self._tete.suivante
        if self._tete is None:            # la file vient d'etre videe
            self._queue = None
        self._taille = self._taille - 1
```

```
        return v
    def tete(self):
        if self.est_vide():
            raise IndexError("file vide")
        return self._tete.valeur
    def taille(self):
        return self._taille
```

Le test qui ré-enfile après avoir vidé la file vise une erreur fréquente : oublier de remettre `_queue` à `None` **et** tester `_queue` (au lieu de `_tete`) pour savoir si la file est vide dans `enfiler`. Le `9` est alors accroché derrière une cellule fantôme qui n'est plus dans la file, `_tete` reste `None`, et le `defiler` suivant lève `IndexError` : le test échoue.

Exercice 3 *Mesurer le gain*

1. Valeurs obtenues sur un ordinateur portable récent (elles varient d'une machine à l'autre, seul l'ordre de grandeur compte) :

`n`	10 000	50 000	100 000	200 000
`FileListe` (s)	0,013	0,29	1,1	4,5
`FileChainee` (s)	0,011	0,057	0,12	0,24

2. Quand `n` double, le temps de `FileChainee` **double** (coût linéaire : chaque opération est en temps constant) alors que celui de `FileListe` est multiplié par environ **4** (coût quadratique) : chacun des n `defiler` fait un `pop(0)` qui décale tous les éléments restants.

3. On peut changer d'implémentation (pour gagner en vitesse, par exemple) sans réécrire ni même relire les programmes qui utilisent la structure : il suffit que l'interface soit respectée.

2 Un vérificateur pour de vrais programmes

Exercice 4 *Localiser l'erreur dans un programme Python*

1. On empile le triplet `(symbole, ligne, colonne)` de chaque ouvrant.

2. `dans_chaine` redevient `None` au guillemet **identique** à celui qui a ouvert la chaîne ; `dans_commentaire` redevient `False` au **retour à la ligne**.

3.

```
def verifier_code(code):
    correspond = {')': '(', ']': '[', '}': '{'}
    p = PileChainee()
    ligne = 1
    colonne = 0
    dans_chaine = None
    dans_commentaire = False
    for c in code:
        colonne = colonne + 1
        if c == "\n":
            ligne = ligne + 1
            colonne = 0
            dans_commentaire = False
        elif dans_commentaire:
            pass
        elif dans_chaine is not None:
            if c == dans_chaine:
                dans_chaine = None
        elif c == "#":
            dans_commentaire = True
        elif c in "\"'":
            dans_chaine = c
        elif c in "([{":
            p.empiler((c, ligne, colonne))
```

```
        elif c in correspond:
            if p.est_vide():
                return f"ligne {ligne}, colonne {colonne} : '{c}' ne ferme rien"
            o, lo, co = p.depiler()
            if o != correspond[c]:
                return (f"ligne {ligne}, colonne {colonne} : '{c}' ferme '{o}'"
                        f" ouvert ligne {lo}, colonne {co}")
    if not p.est_vide():
        o, lo, co = p.depiler()
        return f"ligne {lo}, colonne {co} : '{o}' jamais ferme"
    return None
```

Sur `'d = {"a": [1, 2],\n     "b": (3, 4]}'` (la ligne 2 commence par **cinq espaces**), la fonction renvoie `ligne 2, colonne 16 : ']' ferme '(' ouvert ligne 2, colonne 11`. *Limite assumée :* les chaînes triples `"""..."""` et les guillemets échappés `\"` ne sont pas gérés.

Exercice 5 *Défi — une vraie page HTML*

```
def verifier_html(html):
    p = PileChainee()
    for b in extraire_balises(html):
        if b[0] == "/":                          # balise fermante
            if p.est_vide() or p.depiler() != b[1:]:
                return False
        else:
            p.empiler(b)
    return p.est_vide()
```

La page `'<p>Un <b>gras <i>italique</b></i></p>'` donne `False` (`</b>` arrive alors que `i` est au sommet) ; la page `'<ul><li>un</li><li>deux</ul>'` aussi (`li` jamais fermé). Les navigateurs, eux, *tolèrent* ces erreurs et réparent la page comme ils peuvent.

3 Une calculatrice qui respecte les priorités

Exercice 6 *Une évaluation qui ne plante pas*

1. Pile après chaque jeton de `"7 2 - 3 *"` : `7` ; `7 2` ; `5` (on dépile 2 puis 7, et l'on calcule $7-2$) ; `5 3` ; `15`. Le résultat est 15.

2. Pour `"3 +"`, la pile ne contient que `3` quand arrive `+` : il manque un opérande, le second `depiler` échouerait. Pour `"3 4 5 +"`, tout se passe bien jusqu'au bout, mais la pile finale contient `3 9` : deux nombres, on ne sait pas lequel est « le » résultat. D'où deux vérifications : `p.taille() < 2` **dans la boucle**, juste avant de dépiler pour un opérateur ; `p.taille() != 1` **après la boucle**, avant de renvoyer le résultat.

3.

```
def evaluer_npi(expr):
    p = PileChainee()
    for jeton in expr.split():
        if jeton in PRIORITE:                        # un operateur
            if p.taille() < 2:
                raise ValueError(f"il manque un operande pour '{jeton}'")
            b = p.depiler()                          # operande de droite
            a = p.depiler()                          # operande de gauche
            p.empiler(calculer(a, jeton, b))
        else:
            p.empiler(float(jeton))
    if p.taille() != 1:
        raise ValueError("expression incomplete : il reste "
                         + str(p.taille()) + " nombres")
    return p.depiler()
```

`"10 4 /"` donne `2.5` ; `"1 0 /"` lève `ZeroDivisionError` (levée par `calculer`).

4. La boucle ne fait rien et la pile reste vide : la seconde vérification lève `ValueError("expression incomplete : il reste 0 nombres")`. L'erreur est bien détectée, mais le message est maladroit ; on peut tester `p.est_vide()` à part et écrire « expression vide ».

Exercice 7 *De l'écriture usuelle à la NPI*

1.

Jeton	Pile écrite en ligne (sommet à droite)	Sortie
2	(vide)	2
*	*	2
(	* (	2
3	* (	2 3
+	* (+	2 3
4	* (+	2 3 4
)	*	2 3 4 +
-	- (* dépilé : priorité $2 \geq 1$)	2 3 4 + *
5	-	2 3 4 + * 5
fin	(vide)	2 3 4 + * 5 -

L'évaluation de `2 3 4 + * 5 -` donne 9, comme $2 \times (3+4) - 5$.

2. Avec « supérieure ou égale », `10 - 4 - 3` devient `10 4 - 3 -`, qui vaut $(10-4)-3 = 3$: c'est le bon résultat, car les opérations de même priorité se font de **gauche à droite**. Avec « strictement supérieure », le premier `-` resterait dans la pile : on obtiendrait `10 4 3 - -`, soit $10-(4-3) = 9$, ce qui est faux.

3.

```
1  def infixe_vers_npi(expr):
2      sortie = []
3      ops = PileChainee()
4      for j in decouper(expr):
5          if j in PRIORITE:
6              while (not ops.est_vide() and ops.sommet() != '('
7                     and PRIORITE[ops.sommet()] >= PRIORITE[j]):
8                  sortie.append(ops.depiler())
9              ops.empiler(j)
10         elif j == '(':
11             ops.empiler(j)
12         elif j == ')':
13             while ops.sommet() != '(':
14                 sortie.append(ops.depiler())
15             ops.depiler()                  # on jette la '('
16         else:
17             sortie.append(j)               # un nombre
18     while not ops.est_vide():
19         sortie.append(ops.depiler())
20     texte = ""                             # on recolle les jetons
21     for i in range(len(sortie)):
22         if i > 0:
23             texte = texte + " "            # un espace entre deux jetons
24         texte = texte + sortie[i]
25     return texte
```

`"12*(3+4)-(5+5)*2"` se traduit en `12 3 4 + * 5 5 + 2 * -` et vaut `64.0`.

4 Les boutons « Annuler » et « Rétablir »

Exercice 8 *Un mini-éditeur*

1. `annuler()` range l'état courant dans `_retablir` (pour pouvoir le refaire) ; `retablir()` le range dans `_annuler` (pour pouvoir l'annuler de nouveau).

2.

```
class Editeur:
    def __init__(self):
        self.texte = ""
        self._annuler = PileChainee()
        self._retablir = PileChainee()

    def _sauver(self):
        self._annuler.empiler(self.texte)
        self._retablir = PileChainee()      # une nouvelle action efface l'avenir

    def ecrire(self, s):
        self._sauver()
        self.texte = self.texte + s

    def effacer(self, k):
        self._sauver()
        self.texte = self.texte[:max(0, len(self.texte) - k)]

    def annuler(self):
        if not self._annuler.est_vide():
            self._retablir.empiler(self.texte)
            self.texte = self._annuler.depiler()

    def retablir(self):
        if not self._retablir.est_vide():
            self._annuler.empiler(self.texte)
            self.texte = self._retablir.depiler()
```

(Écrire `_sauver` une fois évite de dupliquer le code dans `ecrire` et `effacer` ; la méthode commence par `_` car elle ne fait pas partie de l'interface.)

3. Sans vidage : on tape `Le chat dort bien`, on annule deux fois (`Le chat`), on tape `mal` (`Le chat mal`), puis `retablir()` ressort l'état `Le chat dort`, qui n'a jamais suivi `Le chat mal` : l'« avenir » annulé n'a plus de sens après une nouvelle action.

4. Environ $1\,000 \times 100\,000 = 10^8$ caractères, soit une centaine de mégaoctets. On peut empiler plutôt l'**action** et ce qu'il faut pour la défaire (par exemple `("ecrire", " mal")` ou `("effacer", "bien")`) : quelques caractères par action. C'est ce que font les vrais éditeurs.

5 Simuler la file d'attente d'un guichet

Exercice 9 *À la main*

Client	Arrivée	Début	Attente	Fin
1	0	0	0	4
2	1	4	3	7
3	2	7	5	9
4	3	9	6	14
5	5	14	9	15

Pour les dix clients, les attentes sont $0, 3, 5, 6, 9, 9, 12, 11, 12, 12$: moyenne $79/10 = 7{,}9$ minutes.

Exercice 10 *Programmer la simulation*

Version directement écrite pour plusieurs guichets (avec `nb_guichets = 1`, c'est la réponse à la question 1) :

```
def simuler(clients, nb_guichets=1):
    f = FileChainee()
    libre_a = [0] * nb_guichets        # minute ou chaque guichet se libere
    attentes = []
    i = 0                              # indice du prochain client a arriver
    t = 0
    file_max = 0
    while i < len(clients) or not f.est_vide():
        while i < len(clients) and clients[i][0] == t:     # a) arrivees
            f.enfiler(clients[i])
            i = i + 1
        for g in range(nb_guichets):                       # b) services
            if libre_a[g] <= t and not f.est_vide():
                arrivee, duree = f.defiler()
                attentes.append(t - arrivee)
                libre_a[g] = t + duree
        file_max = max(file_max, f.taille())
        t = t + 1
    total = 0
    attente_max = 0
    for a in attentes:
        total += a
        if a > attente_max:
            attente_max = a
    return total / len(attentes), attente_max, file_max
```

Résultats : `(7.9, 12, 6)` avec un guichet, `(0.9, 3, 2)` avec deux.

Exercice 11 *Quand la file explose*

1. Valeurs obtenues (graine 42, identiques sur toutes les machines) :

proba	Charge	1 guichet	2 guichets
0,20	0,70	3,9	0,1
0,25	0,875	8,6	0,2
0,30	1,05	48,8	0,3
0,35	1,225	172,6	0,7

(Les attentes maximales correspondantes, avec un guichet, sont 22, 40, 85 et 325 minutes.)

2. Au-delà d'une charge de 1, il arrive en moyenne **plus de travail** par minute que le guichet ne peut en traiter : la file ne se vide jamais et s'allonge sans cesse, donc l'attente croît avec le nombre de clients au lieu de se stabiliser. Même en dessous de 1, l'attente grimpe vite à l'approche de 1 (le hasard crée des « paquets » d'arrivées que le guichet n'absorbe pas). Avec deux guichets, la charge par guichet est divisée par deux : l'attente reste minime.

3. On construit la liste des valeurs de `proba`, puis, par compréhension, la liste des attentes moyennes correspondantes pour un et deux guichets.

```
import matplotlib.pyplot as plt
probas = [k / 100 for k in range(5, 41)]
un = [simuler(generer_clients(500, p, 6, 42), 1)[0] for p in probas]
deux = [simuler(generer_clients(500, p, 6, 42), 2)[0] for p in probas]
plt.plot(probas, un, label="1 guichet")
plt.plot(probas, deux, label="2 guichets")
plt.xlabel("probabilite d'arrivee par minute")
plt.ylabel("attente moyenne (min)")
plt.legend()
plt.show()
```

Autre méthode : une seule boucle qui remplit les trois listes au fur et à mesure (les clients ne sont alors générés qu'une fois par valeur de `proba`).

```
probas = []
un = []
deux = []
for k in range(5, 41):
    p = k / 100
    probas.append(p)
    cl = generer_clients(500, p, 6, 42)
    un.append(simuler(cl, 1)[0])
    deux.append(simuler(cl, 2)[0])
```

Avec un guichet, l'attente moyenne reste sous 5 minutes jusqu'à `proba` = 0,23 (4,9 min) et passe à 7,3 min pour 0,24 : environ un client toutes les 4,3 minutes au maximum.

6 Bilan à rédiger

Exercice 12 *Ce que j'ai construit*

Éléments attendus :

- **Pile (LIFO)** pour le vérificateur (la dernière parenthèse ou balise ouverte est la première à fermer), pour la calculatrice (les deux derniers nombres calculés sont les opérandes ; les opérateurs en attente ressortent dans l'ordre inverse) et pour Annuler / Rétablir (on annule d'abord la dernière action). **File (FIFO)** pour le guichet (premier arrivé, premier servi).
- Partie 1 : `FileListe` remplacée par `FileChainee` ; la vitesse change (coût linéaire au lieu de quadratique), les résultats et le code utilisateur ne changent pas. De même, toutes les applications fonctionnent avec `PileListe` à la place de `PileChainee`.
- Réponse personnelle ; par exemple un test précis ou une trace (tableau de la gare de triage, tableau du guichet).

Réponses des questions de la fiche de révision

↑ questions p. 163

Se tester

1. dernier entré, premier sorti / premier entré, premier sorti ; la file est FIFO (§ II et III).
2. `2` (le `3` a été dépilé) (§ II.A).
3. linéaire, O(n) : tous les éléments sont décalés (§ IV).
4. `c = c.suivante` (on n'avance pas dans la chaîne) (§ V.B).
5. une pile : on revient d'abord à la dernière page visitée (§ II.B).

Et avant ?

1. convention d'encapsulation : attribut interne, à ne manipuler que par les méthodes (ch. 2, § VI).
2. 4 : `compte(3)`, `compte(2)`, `compte(1)` et `compte(0)` (ch. 1, § II).
3. linéaire : on peut devoir examiner les n cases (Première).

4 Processus et ordonnancement

CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 L'ordonnanceur, c'est vous

↑énoncé p. 177

Exercice 1 *Première règle : chacun son tour, jusqu'au bout*

▶ **Solution. 1.**



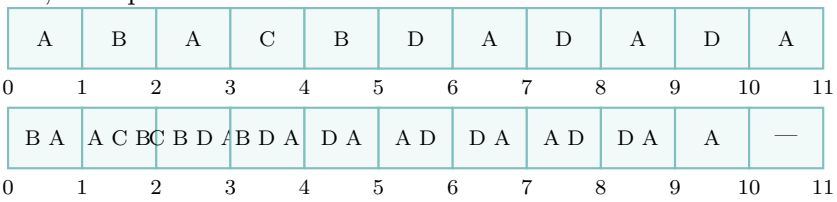
2. A : 5 ; B : 7 ; C : 8 ; D : 11.

3. C arrive à l'instant 2 et ne démarre qu'à l'instant 7 : il attend **5 tics** pour un travail de 1 tic. (Les autres : A 0, B 4, D 5.)

4. Rien ne se passe pendant longtemps : la souris semble « gelée » tant que la vidéo n'est pas encodée. Une tâche longue arrivée la première bloque toutes les petites.

Exercice 2 *Deuxième règle : un tic chacun, puis au bout de la file*

▶ **Solution. 5.** À l'instant 1, B arrive et se place devant A qui revient ; à l'instant 2, C se place devant B ; à l'instant 3, D se place devant A.



6. C démarre à l'instant 3 : il n'attend plus que **1 tic**. B finit à l'instant 5 : **2 tics** d'attente (au lieu de 4).

7. La vidéo A finit plus tard (11 au lieu de 5 ; 6 tics d'attente). C'est acceptable : personne ne regarde la vidéo s'encoder, alors que l'utilisateur attend la réaction à son clic. (D, l'antivirus, finit aussi un peu plus tôt : 10 au lieu de 11.)

8. Chaque changement de tâche coûte du temps (il faut ranger l'avancement de l'une et reprendre celui de l'autre) : trop de changements, et le processeur passe une part de son temps à changer au lieu de travailler. (C'est la *commutation de contexte* du cours ; le tic s'appellera *quantum*.)

Exercice 3 *Deux tâches, deux ressources*

▶ **Solution. 9.** P prend l'imprimante, Q prend le scanner. À sa ligne 2, P demande le scanner, tenu par Q : il doit **attendre**. À sa ligne 2, Q demande l'imprimante, tenue par P : il doit attendre aussi.

10. P tient l'imprimante et attend le scanner ; Q tient le scanner et attend l'imprimante. Les flèches forment une **boucle** : P → scanner → Q → imprimante → P.

11. Non : chacun rendra son objet quand il aura *fini*, et aucun ne peut finir sans l'objet de l'autre. Personne ne bouge plus jamais.

12. Que Q prenne, lui aussi, l'**imprimante en premier** (lignes 1 et 2 échangées). Celui qui obtient l'imprimante obtiendra ensuite le scanner, l'autre attend son tour : plus de boucle possible. (Autres réponses acceptables : prendre les deux objets d'un coup, ou rendre son objet si l'autre n'est pas libre.)

Exercice 4 *Ce qu'on a découvert*

▶ **Solution. 13.**

- … **choisit à chaque instant quelle tâche obtient le processeur** (et pour combien de temps) : c'est l'**ordonnanceur** du système d'exploitation, et la règle choisie est un **compromis**.
- … **un objet** (une ressource) tenu par une autre tâche. Différence : une tâche qui attend le processeur pourrait avancer tout de suite si on le lui donnait ; une tâche qui attend une ressource ne peut pas avancer, même si le processeur est libre.
- … plus personne n'avance, pour toujours : c'est un **interblocage**.

CORRIGÉS DES EXERCICES

Programme, processus, états

Exercice 1 ↑ énoncé p. 191 **1.** Programme (fichier inerte sur le disque). **2.** Deux *processus* (deux exécutions du même programme). **3.** Programme (le texte de la recette). **4.** Processus (seul un programme *en exécution* a un PID et un état).

Exercice 2 ↑ énoncé p. 191 **1. Élu** (utilise le processeur, s'exécute) ; **prêt** (pourrait s'exécuter, attend son tour) ; **bloqué** (attend une ressource).
2. (a) élection ; (b) blocage ; (c) fin du quantum (réquisition) ; (d) déblocage.
3. Par exemple « bloqué → élu » est impossible : un processus débloqué redevient d'abord *prêt* et attend son tour (le processeur peut être occupé).

Exercice 3 ↑ énoncé p. 191 Prêt (création) $\xrightarrow{\text{élection}}$ Élu $\xrightarrow{\text{blocage}}$ Bloqué (attente disque) $\xrightarrow{\text{déblocage}}$ Prêt $\xrightarrow{\text{élection}}$ Élu $\xrightarrow{\text{terminaison}}$ fin. On passe bien par « prêt » après le déblocage, et la terminaison a lieu depuis « élu ».

▷ **Erreur fréquente.** Faire passer P directement de « bloqué » à « élu » quand sa donnée arrive : il redevient d'abord **prêt**, et c'est l'ordonnanceur qui l'élira, quand le processeur sera libre.

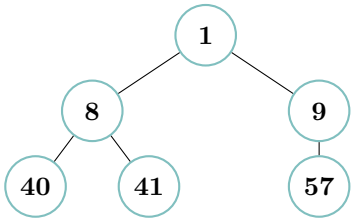
Exercice 4 ↑ énoncé p. 191 États : ① = **Prêt**, ② = **Élu**, ③ = **Bloqué**. Transitions : **(a)** = *élection* (prêt → élu), **(b)** = *fin du quantum* (élu → prêt), **(c)** = *blocage* (élu → bloqué), **(d)** = *déblocage* (bloqué → prêt). Le processus est créé dans l'état *Prêt* et se termine depuis l'état *Élu*.

Arbre de processus, PID et PPID

Exercice 5 ↑énoncé p. 192 **1.** PPID de 42 : c'est 17. PPID de 23 : c'est 1. **2.** 17 a **deux** fils : 42 et 43. **3.** 1 (`init`) n'a pas de père utile ici ; son PPID vaut 0 (le processus de démarrage). **4.** Tuer 17 ne tue pas ses fils : 42 et 43 deviennent **orphelins** et sont **adoptés par `init`** ; leur PPID devient 1.

▷ **Erreur fréquente.** Confondre PID et PPID : le PPID de 42 n'est pas 42, c'est le PID de son **père**, le nœud juste au-dessus dans l'arbre.

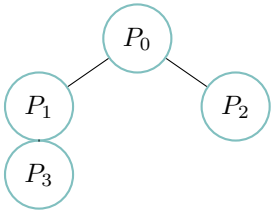
Exercice 6 ↑énoncé p. 192



(`init` 1 a pour fils `bash` 8 et `sshd` 9 ; 8 a pour fils `python` 40 et `vim` 41 ; 9 a pour fils `bash` 57.)

Exercice 7 ↑énoncé p. 192 **1.** Après la ligne 2 (un `fork`) : **2** processus. Après la ligne 3 (un second `fork`) : $2 \times 2 = \mathbf{4}$ processus. « A » est donc affiché **4 fois**.

2. Arbre des processus (chaque `fork` fait naître un fils, qui peut à son tour engendrer) :



(P_0 crée P_1 au 1er `fork` ; au 2e, P_0 crée P_2 et P_1 crée P_3 : quatre processus en tout.)

3. Avec un troisième `fork`, on double encore : $2^3 = \mathbf{8}$ processus. En général, n appels `fork` successifs donnent $\mathbf{2^n}$ processus.

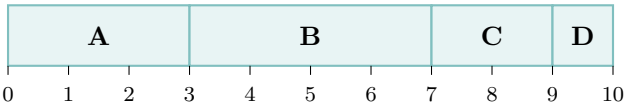
▷ **Erreur fréquente.** Compter un processus de plus par `fork` ($1 + 2 = 3$) : le second `fork` est exécuté par *chacun* des deux processus déjà présents, et le nombre double.

Ordonnancer à la main

Exercice 8 ↑énoncé p. 193 Rappel des données : A(0,3), B(1,4), C(2,2), D(4,1).

Démarche : pour chaque stratégie, on avance dans le temps ; à chaque libération du processeur (ou fin de quantum), on applique la règle d'élection aux seuls processus **déjà arrivés** ; on remplit ensuite, pour chaque processus, l'instant de fin et le temps d'attente.

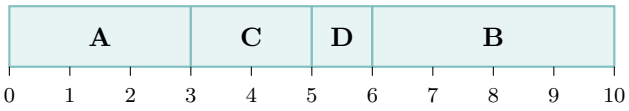
1. Premier arrivé, premier servi.



Attente : A= 0, B= 2, C= 5, D= 5. **Moyenne** = 3,00.

▷ **Erreur fréquente.** Compter l'attente depuis $t = 0$ au lieu de l'instant d'arrivée : pour B, on trouverait $7 - 4 = 3$ au lieu de $7 - 1 - 4 = 2$.

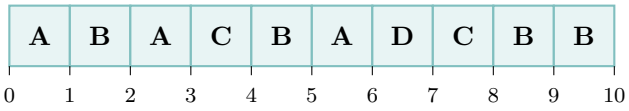
2. Plus court d'abord.



Attente : A= 0, C= 1, D= 1, B= 5. **Moyenne** = 1,75.

▷ **Erreur fréquente.** Choisir à $t = 3$ parmi *tous* les processus : D, le plus court, n'arrive qu'à $t = 4$. À $t = 3$, seuls B et C sont prêts, et l'on élit C.

3. Tourniquet (quantum 1).



Fins : A= 6, B= 10, C= 8, D= 7. Attente : A= 3, B= 5, C= 4, D= 2. **Moyenne** = 3,50.

4. Le **plus court d'abord** minimise l'attente moyenne (1,75). Le **tourniquet** est le plus *réactif* : chaque processus reçoit du processeur très tôt (aucun n'attend plus d'un tour avant de commencer), au prix de nombreuses commutations et d'une attente moyenne plus élevée.

Programmer un ordonnanceur

Exercice 9 ↑énoncé p. 193

Barème indicatif (sur 6) : Q1 : 1 ; Q2 : 1 ; Q3 : 1,5 ; Q4 : 1,5 ; Q5 : 1

1. Quantum 1 : `11, 20, 32, 11, 20, 32, 11, 32, 11`.

2. Quantum 2 : `11, 11, 20, 20, 32, 32, 11, 11, 32`.

3. Méthodes complétées :

```
    def execute_un_cycle(self):
        self.reste_a_faire = self.reste_a_faire - 1

    def change_etat(self, nouvel_etat):
        self.etat = nouvel_etat

    def est_termine(self):
        return self.reste_a_faire == 0
```

4. Fonction complétée :

```
def tourniquet(liste_attente, quantum):
    ordre_execution = []                          # INITIALISER
    while liste_attente != []:                    # PARCOURIR la file
        processus = liste_attente.pop(0)
        processus.change_etat("En cours d'exécution")
        compteur = 0                              # INITIALISER
        while compteur < quantum and not processus.est_termine():
            ordre_execution.append(processus.pid)  # METTRE A JOUR
            processus.execute_un_cycle()
            compteur = compteur + 1
        if not processus.est_termine():           # TESTER
            processus.change_etat("Suspendu")
            liste_attente.append(processus)
        else:
            processus.change_etat("Terminé")
    return ordre_execution                        # RENVOYER
```

5.

```
liste_attente = [Processus(11, 4), Processus(20, 2), Processus(32, 3)]
print(tourniquet(liste_attente, 1))   # [11, 20, 32, 11, 20, 32, 11, 32, 11]
# nouveaux processus : les anciens sont termines et la liste est vide
liste_attente = [Processus(11, 4), Processus(20, 2), Processus(32, 3)]
print(tourniquet(liste_attente, 2))   # [11, 11, 20, 20, 32, 32, 11, 11, 32]
```

Interblocage

Exercice 10 ↑énoncé p. 194 **1.** P_1 **détient** R_1 (flèche $R_1 \to P_1$) et **attend** R_2 (flèche $P_1 \to R_2$).

2. Oui : le graphe contient un **cycle** $P_1 \to R_2 \to P_2 \to R_1 \to P_1$. Chacun attend une ressource tenue par l'autre : c'est un interblocage.

3. On peut **tuer** l'un des deux processus (par exemple `kill` sur P_2) : il libère alors sa ressource, ce qui débloque l'autre.

▷ **Erreur fréquente.** Lire les flèches à l'envers : $R_1 \to P_1$ signifie que R_1 est *détenue* par P_1 ; c'est $P_1 \to R_2$ qui signifie que P_1 attend R_2.

Exercice 11 ↑énoncé p. 194 **1.** P_1 obtient I, P_2 obtient S. Puis P_1 demande S (tenu par P_2) : P_1 se bloque. Puis P_2 demande I (tenu par P_1) : P_2 se bloque. Chacun tient ce que l'autre attend ⇒ interblocage.

2. On casse l'**attente circulaire**. Si tout le monde demande *toujours* I avant S, alors celui qui obtient I le premier obtiendra ensuite S sans rival (personne ne peut détenir S sans détenir déjà I) : aucun cycle d'attente ne peut se former, donc plus d'interblocage possible.

Exercices type bac

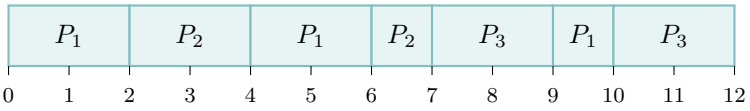
Exercice 12 *Type bac — Observer puis ordonnancer des processus* ↑énoncé p. 195

Barème indicatif (sur 4) : Q1 : 0,25 ; Q2 : 0,25 ; Q3 : 0,5 ; Q4 : 0,5 ; Q5 : 0,75 ; Q6 : 0,75 ; Q7 : 1

Partie A. 1. Le processus 5468 est issu de `/snap/gimp/393/usr/bin/gimp` : c'est l'application **GIMP**. **2.** C'est le processus de PID **1770** (`gnome-shell`), avec `00:00:51` de temps processeur. **3.** On compte les apparitions de chaque PPID : 1611 apparaît 5 fois (fils 1752, 1770, 5564, 5571, 5615), aucun autre plus de 2 fois : c'est le processus **1611**. **4.** Le PPID de `ps` (5629) est 5622, qui est un `bash` : la commande a été lancée depuis ce shell. **5.** On remonte les PPID : 5629 → 5622 → 5571 → 1611 → 1, d'où la succession **1** → **1611** → **5571** → **5622** → **5629**.

Partie B. 6. Élu : le processus s'exécute sur le processeur. **Prêt** : il pourrait s'exécuter et attend que l'ordonnanceur lui attribue le processeur. **Bloqué** : il attend une ressource (lecture disque, saisie…) et ne peut pas avancer même si le processeur est libre.

7. À $t = 0$, seul P_1 est là : il s'exécute un quantum (0–2), puis passe derrière P_2 (arrivé à $t = 1$). P_2 s'exécute de 2 à 4, puis P_1 de 4 à 6 ; entre-temps P_3 est arrivé ($t = 5$) et se trouve *devant* P_1 qui revient en fin de file à $t = 6$. D'où :



P_2 se termine à $t = 7$, P_1 à $t = 10$ et P_3 à $t = 12$.

▷ **Erreur fréquente.** Remettre P_1 dans la file devant P_3 à $t = 6$: P_3, arrivé à $t = 5$ pendant le quantum de P_1, est déjà dans la file ; P_1 se place derrière lui.

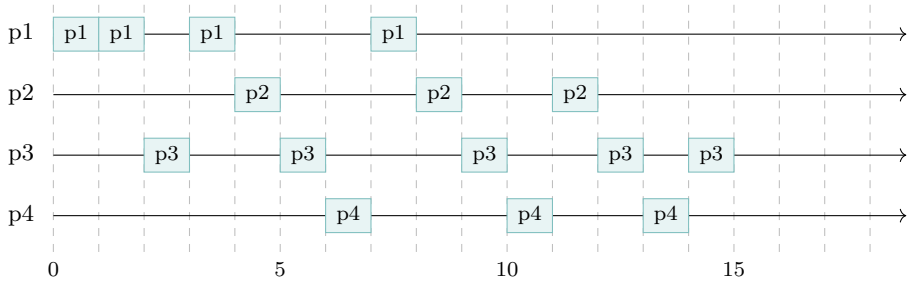
Exercice 13 *Type bac — Tourniquet cycle par cycle et interblocage*

↑énoncé p. 196

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 1,5 ; Q4 : 2 ; Q5 : 1,5

1. Élu, prêt, bloqué. **2.** Sans ressource à attendre, un processus ne peut jamais être bloqué : il est soit **élu**, soit **prêt**.

3. Au cycle c, on enfile d'abord le processus créé à ce cycle, puis on exécute la tête de file. On obtient la suite `p1 p1 p3 p1 p2 p3 p4 p1 p2 p3 p4 p2 p3 p4 p3` (15 cycles $= 4+3+5+3$) :



4. On boucle tant qu'il reste un processus à créer ou un processus dans la file. `ordo.temps` est le cycle en cours (il est incrémenté par `tourniquet`) ; on calcule d'abord `dernier`, la plus grande *clé* du dictionnaire, c'est-à-dire la date de création du dernier processus (ici 3), par une recherche de maximum :

```
dernier = 0             # INITIALISER
for t in depart_proc:   # PARCOURIR les cles
    if t > dernier:     # TESTER
        dernier = t     # METTRE A JOUR
ordo = Ordonnanceur()
while ordo.temps <= dernier or not ordo.file.est_vide():
    if ordo.temps in depart_proc:
        ordo.ajoute_nouveau_processus(depart_proc[ordo.temps])
    print(ordo.tourniquet())
```

Ce programme affiche successivement `p1`, `p1`, `p3`, `p1`, `p2`, …, `p3` (la suite de la question 3).

5. On suit le chronogramme de l'exemple, une instruction par cycle d'exécution : cycle 0, D acquiert le **fichier** ; cycle 1, D calcule ; cycle 2, B acquiert le **clavier** ; cycle 3, D demande le clavier, détenu par B : **D se bloque** ; cycle 4, A acquiert le GPU ; cycle 5, B demande le fichier, détenu par D : **B se bloque**. B attend le fichier tenu par D, et D attend le clavier tenu par B : c'est une attente circulaire, donc un **interblocage** (aucun des deux ne libérera jamais sa ressource).

Exercice 14 *Type bac — Une application de streaming*

↑énoncé p. 197

Barème indicatif (sur 6) : Q1 : 0,75 ; Q2 : 0,5 ; Q3 : 0,5 ; Q4 : 0,75 ; Q5 : 1 ; Q6 : 0,5 ; Q7 : 1 ; Q8 : 1

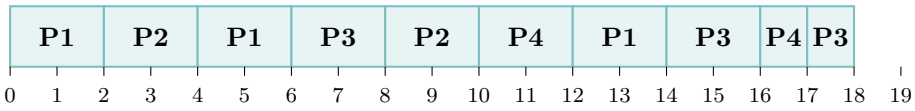
Partie A. 1. Les trois états : à gauche **prêt**, à droite **élu**, en bas **bloqué**. Flèche prêt → élu : **élection** ; élu → bloqué : **blocage** ; bloqué → prêt : **déblocage**.

2. P1 (interface cachée, en attente d'une action de l'utilisateur) : **bloqué**. P2 (attend la carte Wi-Fi) : **bloqué**. P3 (décode et envoie la musique) : **élu** — il s'exécute (sur un monoprocesseur, il alterne en réalité entre élu et prêt).

3. Un interblocage survient lorsque plusieurs processus attendent chacun une ressource **détenue par un autre** processus du groupe, de façon **circulaire** : aucun ne peut avancer, donc aucun ne libère ce qu'il tient.

4. Il suffit d'un scénario : P1 obtient le MIC (« demander MIC »), puis P2 obtient le CAL (« demander CAL »). Ensuite P1 demande le CAL (tenu par P2) et se bloque, et P2 demande le MIC (tenu par P1) et se bloque. Chacun attend la ressource de l'autre : interblocage. (On en trouve d'autres, par exemple P3 qui tient CAL et demande MIC pendant que P1 tient MIC et demande CAL.)

Partie B. 5. Les arrivées se placent en fin de file ; un processus préempté y revient après les processus arrivés pendant son quantum :



P2 se termine à 10 ms, P1 à 14 ms, P4 à 17 ms et P3 à 18 ms.

6.

```
fp = creer_file_vide()
enfiler(fp, P1)
enfiler(fp, P2)
enfiler(fp, P3)
enfiler(fp, P4)
```

7. Si le temps restant dépasse le quantum, le processus consomme un quantum et repart en queue de file ; sinon il se termine.

```
def execute_un_processus(file_d_attente, t):
    processus = defiler(file_d_attente)
    if processus['temps'] > quantum:
        processus['temps'] = processus['temps'] - quantum
        enfiler(file_d_attente, processus)
        return t + quantum
    else:
        return t + processus['temps']
```

8.

```
def execute_tous_processus(file_d_attente):
    t = 0                                          # INITIALISER
    while not est_vide(file_d_attente):
        t = execute_un_processus(file_d_attente, t)  # METTRE A JOUR
    return t                                       # RENVOYER
```

Avec la file `fp`, l'appel renvoie 18 : la somme des temps d'exécution $(6 + 4 + 5 + 3)$, puisque le processeur n'est jamais inoccupé.

Exercice 15 *Type bac — Des files de priorité pour l'ordonnanceur*

↑énoncé p. 200

Barème indicatif (sur 8) : Q1 : 0,25 ; Q2 : 0,5 ; Q3 : 0,5 ; Q4 : 0,5 ; Q5 : 1 ; Q6 : 0,75 ; Q7 : 0,5 ; Q8 : 1,25 ; Q9 : 1,25 ; Q10 : 1,5

1. Un **ordonnanceur**. **2.** Élu, prêt, bloqué. **3.** Une **file** : parmi les processus de même priorité, le premier arrivé doit être le premier servi.

4.

```
class Processus:
    def __init__(self, PID, priorite, temps_CPU):
        self.priorite = priorite
        self.PID = PID
        self.temps_utilisation = 0
        self.temps_CPU = temps_CPU
```

5. Au cycle 3, P2 (préempté par P3, de même valeur de priorité 0) passe à la valeur de priorité 1, derrière P1. Au cycle 4, aucun processus en attente n'est au moins aussi prioritaire que P3 (valeur de priorité inférieure ou égale à 0) : P3 continue, et sa valeur de priorité passe à 1. Au cycle 5, P1 et P2 (valeur 1) attendent : P3 est retiré, sa valeur de priorité passe à 2, et P1 (tête de la file 1) est élu.

```
Cycle 3: CPU=P3     liste_files=[[], [P2, P1], []]
Cycle 4: CPU=P3     liste_files=[[], [P2, P1], []]
Cycle 5: CPU=P1     liste_files=[[], [P2], [P3]]
```

6. À chaque cycle passé sur le processeur, la valeur de priorité du processus long augmente de 1 : au bout de quelque temps, elle est très grande (il est très peu prioritaire). Or les processus courts arrivent sans cesse avec la priorité 0 et, avec une durée de 4, n'atteignent jamais une valeur

de priorité élevée. Il y a toujours un processus plus prioritaire en attente : le processus long n'est plus jamais élu. C'est une **famine**.

7. Pendant qu'il attend, le `temps_d_attente` du processus long augmente ; dès qu'il dépasse `Max_Temps`, sa valeur de priorité diminue (il redevient plus prioritaire), et ainsi de suite jusqu'à ce qu'elle rejoigne celle des processus courts. Il finit donc par être élu régulièrement, et son temps d'utilisation atteint 1000 en un temps fini.

8. *Appels attendus, écrits avant de coder :*

```
assert meilleur_priorite([[], [p2], [p3, p1]]) == 1   # cas normal : exemple de l'enonce
assert meilleur_priorite([[], [], []]) is None         # cas limite : plus aucun processus
assert meilleur_priorite([[p3], [p2], [p1]]) == 0      # cas piege : 0 est une vraie reponse
```

La valeur de priorité d'un processus est l'indice de sa file : on renvoie l'indice de la première file non vide.

```
def meilleur_priorite(liste_files):
    for i in range(len(liste_files)):  # PARCOURIR les files
        if liste_files[i] != []:       # TESTER
            return i                   # RENVOYER la premiere non vide
    return None                        # RENVOYER : toutes sont vides
```

9. La tête de chaque file est à droite (dernier élément de la liste Python), comme dans la simulation de la question 5.

```
def prioritaire(liste_files):
    i = meilleur_priorite(liste_files)
    if i is None:
        return None
    return liste_files[i].pop(len(liste_files[i]) - 1)
```

10. Une version possible, qui renvoie le processus en cours à l'issue du cycle. On enfile à gauche (`[p] + file`, la tête étant à droite) et on crée une nouvelle file si la priorité dépasse le nombre de files :

```
def placer(p, liste_files):
    while len(liste_files) <= p.priorite:
        liste_files.append([])
    liste_files[p.priorite] = [p] + liste_files[p.priorite]

def gerer(p, liste_files):
    if p is None:                                # processeur libre
        return prioritaire(liste_files)
    if p.temps_utilisation == p.temps_CPU:       # p a termine
        return None
    p.temps_utilisation = p.temps_utilisation + 1
    m = meilleur_priorite(liste_files)
    if m is not None and m <= p.priorite:        # un processus au moins
        p.priorite = p.priorite + 1              # aussi prioritaire attend
        placer(p, liste_files)
        return prioritaire(liste_files)
    else:
        p.priorite = p.priorite + 1
        return p
```

En partant de `p = None` et de `liste_files = [[P3, P2, P1], [], []]`, cinq appels successifs `p = gerer(p, liste_files)` reproduisent exactement la simulation de la question 5.

Critiquer une réponse d'IA

Exercice 16 *Les états d'un processus expliqués par un assistant d'IA* ↑énoncé p. 201

1. Non. Si P_2 passait directement de bloqué à élu à la fin de sa lecture, il y aurait **deux** processus élus sur le même cœur (P_1 et P_2), ce qui contredit « un seul élu à la fois » — ou alors P_2 « volerait » le processeur à P_1 sans décision de l'ordonnanceur. En réalité, P_2 redevient **prêt** et attend d'être élu à son tour.

2. La phrase fautive est « *dès que la ressource attendue arrive, le processus bloqué reprend immédiatement le processeur (bloqué → élu)* ». Le cours dit l'inverse : « *Déblocage (bloqué → prêt) : la ressource arrive ; le processus ne repart pas directement en exécution, il redevient prêt et attend son tour* », et signale ce piège : « *on ne passe jamais de bloqué directement à élu* ». Les quatre transitions correctes : prêt → élu (élection), élu → prêt (fin de quantum), élu → bloqué (blocage), bloqué → prêt (déblocage). Le reste de la réponse (définition des trois états, unicité de l'élu, terminaison depuis l'état élu) est exact.

3. Compter les **flèches** de l'automate décrit et les comparer à celles du cours : toute flèche qui *arrive* sur « élu » doit partir de « prêt », puisque seul l'ordonnanceur donne le processeur. Une transition qui contourne l'ordonnanceur est suspecte par construction.

S'entraîner à l'oral

Exercice 17 *Expliquer en deux minutes* ↑énoncé p. 202

▶ Solution. Pas de texte à apprendre par cœur : voici les **éléments attendus** pour chaque sujet. L'ordre, les mots et l'exemple peuvent être différents ; l'explication est réussie si ces idées y sont, justes et reliées entre elles.

Sujet 1.

- **Programme** : un fichier d'instructions, inerte, stocké sur le disque.
- **Processus** : un programme **en cours d'exécution**, avec son état (mémoire, registres, fichiers ouverts), identifié par un **PID**.
- Exemple : lancer deux fois le même éditeur de texte donne un seul programme mais deux processus.
- Piège : un processus peut en créer d'autres (relation père–fils) ; un même programme peut donc correspondre à plusieurs processus.

Sujet 2.

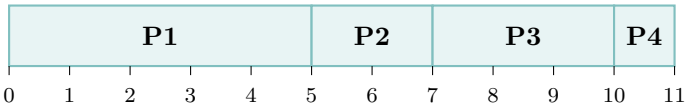
- Trois états : **prêt** (attend le processeur), **élu** (s'exécute), **bloqué** (attend une ressource, par exemple une lecture sur le disque).
- L'**ordonnanceur** du système d'exploitation choisit à chaque instant le processus élu parmi les processus prêts ; avec le **tourniquet**, chacun reçoit un quantum de temps puis retourne en fin de file.
- Exemple : trois processus et un quantum de quelques millisecondes : ils alternent si vite qu'ils semblent simultanés.
- Piège : un processus bloqué ne redevient pas directement élu ; il repasse d'abord par l'état prêt.

Sujet 3.

- **Interblocage** : plusieurs processus s'attendent mutuellement, chacun détenant une ressource que l'autre demande ; aucun ne peut plus avancer.
- Exemple : P1 détient l'imprimante et demande le scanner ; P2 détient le scanner et demande l'imprimante.
- On le repère par un **cycle** dans le graphe d'attente.
- Pour l'éviter : demander les ressources toujours dans le même ordre ; pour en sortir : arrêter l'un des processus. Piège : ce n'est pas le « bug » d'un seul programme, mais une interaction.

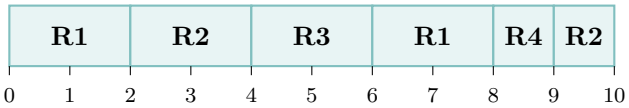
Entraînement

Exercice 18 IA IA IA *Chronogrammes en série* ↑énoncé p. 202 **a) Premier arrivé, premier servi.**



Attente : P1= 0, P2= 7 − 1 − 2 = 4, P3= 10 − 2 − 3 = 5, P4= 11 − 4 − 1 = 6. **Moyenne** = 3,75.

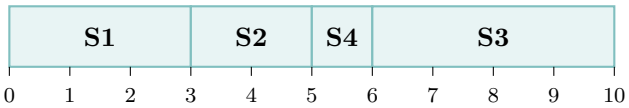
b) Tourniquet de quantum 2. File (dans l'ordre de passage) à $t = 2$: R2, R3, R1 (R3, arrivé à $t = 2$, passe devant R1) ; à $t = 4$: R3, R1, R4, R2 (R4 passe devant R2).



Fins : R1= 8, R2= 10, R3= 6, R4= 9. Attente : R1= 4, R2= 6, R3= 2, R4= 4. **Moyenne** = 4,00.

▷ **Erreur fréquente.** Remettre R1 dans la file avant R3 à $t = 2$: d'après la convention, le processus qui arrive à la fin d'un quantum passe devant celui qui vient d'être interrompu.

c) Par priorités. À $t = 3$, S2, S3 et S4 sont arrivés ; S2 et S4 ont la valeur 1, et S2 est arrivé le premier.



Attente : S1= 0, S2= 5 − 1 − 2 = 2, S3= 10 − 2 − 4 = 4, S4= 6 − 3 − 1 = 2. **Moyenne** = 2,00.

CORRIGÉS DES TP

TP Observer, ordonnancer, bloquer ↑énoncé p. 206

Partie A — Observer et gérer les processus dans un terminal

Exercice 1 IA IA IA *Premiers pas : qui suis-je ?*

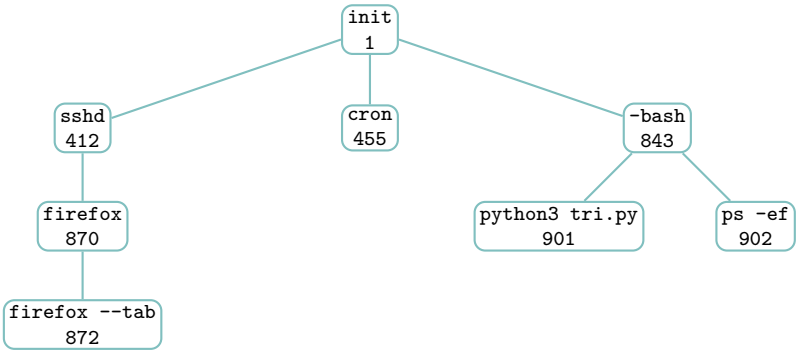
1. Le PID affiché par `echo $$` est celui de la ligne `sh` (ou `bash`) dans la sortie de `ps`.

2. Le PPID de `ps` est le PID du shell : c'est le shell qui a lancé la commande. `ps` se voit lui-même car, au moment où il lit la liste des processus, il est *lui-même* un processus en cours d'exécution.

3. Le PID change : chaque commande tapée crée un **nouveau** processus (un nouveau numéro, en général le suivant libre), qui disparaît une fois la commande terminée. Un programme (`ps`) peut donc donner de nombreux processus successifs.

Exercice 2 *Lire une sortie de* `ps -ef`

1. Arbre des processus :



2. Le père de `python3 tri.py` (901) est `-bash` (843) : il a été lancé depuis le terminal.

3. Les frères de `ps -ef` sont les autres fils de 843 : `python3 tri.py`. Le processus qui a le plus de descendants est `init` (tous les autres) ; parmi ses fils, `sshd` et `-bash` en ont deux chacun.

4. Le 0 désigne le processus de démarrage du système, qui n'est pas un vrai processus utilisateur : `init` est la racine de l'arbre.

Exercice 3 *Premier plan, arrière-plan,* `kill`

1. Non : le shell attend la fin de `sleep 5` avant de rendre l'invite.

2. Le `&` lance la commande en **arrière-plan** : le shell rend la main aussitôt. `jobs` affiche `[1] sleep 300` et `[2] sleep 400` ; les deux `sleep` ont pour PPID le PID du shell (celui donné par `echo $$`).

3. Après les deux `kill`, `jobs` signale les tâches « `Terminated` » puis ne montre plus rien ; elles ont disparu de `ps`.

4. Tableau-mémo :

Commande	Rôle	Exemple
`ps -o pid,ppid,comm`	liste les processus avec PID, PPID et nom de commande	`ps -o pid,ppid,comm`
`<commande> &`	lance en arrière-plan, le shell rend la main	`sleep 300 &`
`jobs`	tâches lancées depuis ce shell, numérotées `[1]`, `[2]`…	`jobs`
`kill <PID>`	envoie au processus un signal d'arrêt (`kill %n` : tâche n)	`kill 1234`, `kill %2`
`top`	liste rafraîchie en temps réel (CPU, mémoire, état) ; `q` pour quitter	`top`

Exercice 4 *Une famille de processus, et un glouton*

1. Chaîne attendue : premier shell (PID a) → `sh` (PID b, PPID a) → `sleep 500` (PPID b). *Remarque* : après `exit`, le `sleep 500` continue de tourner ; son père a disparu, il est « adopté » (son PPID devient en général 1). Le faire constater, puis le tuer.

2. `yes` est en tête de `top`, avec près de 100 % d'un cœur (le pourcentage exact dépend de la machine ; sous JSLinux, l'émulateur est lent mais `yes` prend tout ce qu'il peut).

3. `R` (*running*) ↔ **élu ou prêt** (le processus veut le processeur) ; `S` (*sleeping*) ↔ **bloqué** (il attend un événement). `sleep 500` est `S` (il attend l'expiration d'un délai), `yes` est `R`.

4. Après `kill`, l'occupation du processeur retombe près de 0 %.

Partie B — Programmer un simulateur d'ordonnancement

Exercice 5 *Premier arrivé, premier servi*

```python
def trier_par_arrivee(procs):
    # tri par insertion selon l'instant d'arrivee
    t = [p for p in procs]
    for i in range(1, len(t)):
        j = i
        while j > 0 and t[j - 1][1] > t[j][1]:
            temp = t[j - 1]                # echange des deux voisins
            t[j - 1] = t[j]
            t[j] = temp
            j = j - 1
    return t

def fcfs(procs):
    chrono = []
    t = 0
    for nom, arrivee, duree in trier_par_arrivee(procs):
        while t < arrivee:          # processeur inoccupe
            chrono.append(".")
            t = t + 1
        for k in range(duree):
            chrono.append(nom)
            t = t + 1
    return chrono

def gantt_ligne(chrono):
    s = ""
    for c in chrono:
        s = s + c
    return s
```

`gantt_ligne(fcfs(JEU_COURS))` vaut `"AAAABBBCDD"` : c'est le chronogramme du cours.

Exercice 6 *Mesurer : temps de séjour et d'attente*

```python
def statistiques(chrono, procs):
    stats = {}
    for nom, arrivee, duree in procs:
        fin = 0
        for i in range(len(chrono)):
            if chrono[i] == nom:
                fin = i + 1
        sejour = fin - arrivee
        stats[nom] = (sejour, sejour - duree)
    return stats

def moyennes(stats):
    total_sejour = 0
    total_attente = 0
    for nom in stats:
        sejour, attente = stats[nom]
        total_sejour = total_sejour + sejour
        total_attente = total_attente + attente
    n = len(stats)
    return (total_sejour / n, total_attente / n)
```

FCFS sur `JEU_COURS` : `{'A': (4, 0), 'B': (6, 3), 'C': (6, 5), 'D': (7, 5)}`, moyennes `(5.75, 3.25)`.

Exercice 7 *Le plus court d'abord*

```python
def sjf(procs):
    chrono = []
    t = 0
    restants = [p for p in procs]
    while restants != []:
        elu = None
        for p in restants:          # le plus court parmi les arrives
            if p[1] <= t and (elu is None or p[2] < elu[2]):
                elu = p
        if elu is None:             # aucun processus pret
            chrono.append(".")
            t = t + 1
        else:
            restants = [p for p in restants if p != elu]    # on retire l'elu
            for k in range(elu[2]):
                chrono.append(elu[0])
                t = t + 1
    return chrono
```

`"AAAACDDBBB"`, attente moyenne 2,5 (séjour moyen 5).

Exercice 8 *Le tourniquet*

1.

```python
def tourniquet(procs, quantum):
    a_venir = trier_par_arrivee(procs)       # fonction de l'exercice 5
    reste = {}
    for nom, arrivee, duree in procs:
        reste[nom] = duree
    file = []
    chrono = []
    t = 0
    while a_venir != [] or file != []:
        while a_venir != [] and a_venir[0][1] <= t:    # arrivees
            file.append(a_venir.pop(0)[0])
        if file == []:
            chrono.append(".")
            t = t + 1
        else:
            elu = file.pop(0)
            n = min(quantum, reste[elu])
            for k in range(n):
                chrono.append(elu)
                t = t + 1
            reste[elu] = reste[elu] - n
            while a_venir != [] and a_venir[0][1] <= t:  # arrivees pendant le quantum
                file.append(a_venir.pop(0)[0])
            if reste[elu] > 0:
                file.append(elu)                        # en queue, APRES les arrivees
    return chrono
```

2. On obtient bien `"ABACBDABDA"` (quantum 1, attente moyenne 3,75) et `"AABBCAADDB"` (quantum 2, attente moyenne 3,75 aussi).

3.

```python
def commutations(chrono):
    c = 0
    for i in range(1, len(chrono)):
        if chrono[i] != chrono[i - 1] and chrono[i] != "." and chrono[i - 1] != ".":
            c = c + 1
    return c
```

Sur `JEU_COURS` : 9 commutations pour le tourniquet de quantum 1, 3 pour FCFS.

Exercice 9 *Un vrai diagramme de Gantt*

```python
def completer(texte, n):
    """Ajoute des espaces a droite de texte jusqu'a n caracteres."""
    while len(texte) < n:
        texte = texte + " "
    return texte

def gantt(chrono, procs):
    lignes = ""
    for nom, arrivee, duree in procs:
        fin = 0
        for i in range(len(chrono)):
            if chrono[i] == nom:
                fin = i + 1
        ligne = ""
        for t in range(len(chrono)):
            if chrono[t] == nom:
                ligne = ligne + "#"
            elif arrivee <= t and t < fin:      # arrive mais pas fini
                ligne = ligne + "."
            else:
                ligne = ligne + " "
        lignes = lignes + completer(nom, 3) + "|" + ligne + "|\n"
    axe = ""
    for t in range(0, len(chrono) + 1, 5):
        axe = axe + completer(str(t), 5)
    return lignes + "    " + axe
```

Le nombre de `.` d'une ligne est exactement le **temps d'attente** du processus (arrivé, pas fini, pas élu).

Exercice 10 *Campagne de mesures*

1. Diagramme FCFS sur `JEU_TP` : entre 16 et 18, aucun processus n'est prêt (`E` arrive à 18) : le processeur est **inoccupé** (deux `"."` dans le chronogramme).

```
A  |########             |
B  | .......##           |
C  |  ........#####      |
D  |    ...........#     |
E  |                 ### |
F  |                  ..#|
    0    5    10   15   20
```

2. Valeurs obtenues :

Stratégie	Séjour moyen	Attente moyenne	Commutations
Premier arrivé (FCFS)	8,00	4,67	4
Plus court d'abord (SJF)	7,17	3,83	4
Tourniquet, quantum 1	6,67	3,33	16
Tourniquet, quantum 2	7,00	3,67	10
Tourniquet, quantum 4	7,83	4,50	6

Remarque : ici, le tourniquet de quantum 1 bat le plus court d'abord *non préemptif* en attente moyenne, car le long processus `A` (arrivé le premier) monopolise le processeur 8 unités en SJF. Le résultat « SJF est optimal » du cours vaut quand tous les processus sont présents au départ.

3.

```python
for q in range(1, 9):
    c = tourniquet(JEU_TP, q)
    print(q, moyennes(statistiques(c, JEU_TP))[1], commutations(c))
```

quantum	1	2	3	4	5	6	7	8
attente moyenne	3,33	3,67	4,33	4,5	4,5	5,0	5,5	4,67
commutations	16	10	7	6	5	5	5	4

Quand le quantum augmente, les commutations diminuent fortement et l'attente moyenne augmente (pas tout à fait régulièrement : elle redescend à 8). Avec un quantum de 8 ($\geqslant$ la plus longue durée), aucun processus n'est jamais interrompu : le tourniquet **devient FCFS** (même chronogramme, 4,67).

4. Quantum minuscule : le processeur passe une part importante de son temps à commuter (travail inutile pour l'utilisateur). Quantum énorme : on retombe sur FCFS, un gros calcul bloque tout le monde et la machine n'est plus réactive. On choisit un quantum de quelques millisecondes : grand devant le coût d'une commutation, petit devant le temps de réaction humain.

Exercice 11 *Défi : le plus court temps restant d'abord*

```python
def srtf(procs):
    reste = {}
    arrivee = {}
    a_faire = 0                     # unites de travail restant en tout
    for nom, a, d in procs:
        reste[nom] = d
        arrivee[nom] = a
        a_faire = a_faire + d
    chrono = []
    t = 0
    while a_faire > 0:
        elu = None
        for nom in reste:           # le plus petit temps restant parmi les arrives
            if reste[nom] > 0 and arrivee[nom] <= t:
                if elu is None or reste[nom] < reste[elu]:   # a egalite : premier arrive
                    elu = nom
        if elu is None:
            chrono.append(".")
        else:
            chrono.append(elu)
            reste[elu] = reste[elu] - 1
            a_faire = a_faire - 1
        t = t + 1
    return chrono
```

1. `"AACAADDBBB"` sur `JEU_COURS` (attente moyenne 2,25).

2. Sur `JEU_TP` : `"ABBCDCCCCAAAAAAA..EFEE"`, attente moyenne 1,83, la meilleure de toutes. Mais `A` attend 8 unités et finit en dernier parmi les quatre premiers : c'est le risque de **famine** des longs processus (si de petits processus arrivaient sans cesse, `A` ne finirait jamais).

Partie C — Provoquer, guérir et éviter un interblocage

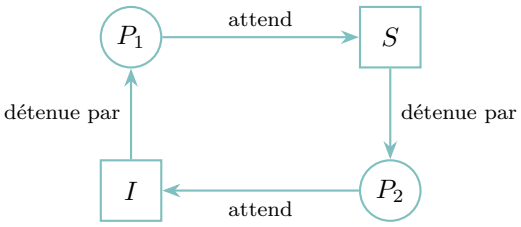
Exercice 12 *Provoquer un interblocage (sans rester bloqué !)*

1. Sortie obtenue (en 4 s environ) :

```
P1 tient l'imprimante, demande le scanner
P2 tient le scanner, demande l'imprimante
Au bout de 4 s, P1 et P2 sont toujours bloqués : interblocage !
```

Les lignes `"... imprime et scanne"` n'apparaissent jamais.

2. Graphe d'attente (cycle $P_1 \to S \to P_2 \to I \to P_1$) :



3. Le `sleep(0.1)` laisse à l'autre fil le temps de prendre sa première ressource : il **force** le mauvais entrelacement. Sans lui, P1 a presque toujours le temps de prendre les deux verrous avant que P2 démarre : 5 essais sur 5 sans blocage. Un interblocage est donc une panne **intermittente**, qui dépend du hasard de l'ordonnancement : un programme peut passer tous ses tests et se figer un jour en production.

4. `join(timeout=2)` : le programme principal n'attend chaque fil que 2 s au plus, au lieu d'attendre indéfiniment. `daemon=True` : à la fin du programme principal, Python n'attend pas ces fils (encore bloqués) pour s'arrêter.

Exercice 13 *Guérir : abandonner au bout d'un délai*

```python
def tache(nom, premier, second, journal):
    with premier:
        time.sleep(0.1)
        if second.acquire(timeout=1):
            journal.append(nom + " a les deux ressources")
            second.release()
        else:
            journal.append(nom + " abandonne")

imprimante = threading.Lock()
scanner = threading.Lock()
journal = []
t1 = threading.Thread(target=tache, args=("P1", imprimante, scanner, journal))
t2 = threading.Thread(target=tache, args=("P2", scanner, imprimante, journal))
t1.start()
t2.start()
t1.join()
t2.join()
print(journal)
```

2. Le résultat **varie** d'une exécution à l'autre (environ 1,1 s à chaque fois). Sur six essais : quatre fois `['P1 abandonne', 'P2 abandonne']` (dans un ordre ou l'autre), deux fois `['P2 abandonne', 'P1 a les deux ressources']` : celui qui abandonne le premier libère sa ressource, que l'autre obtient juste avant la fin de son délai.

3. En abandonnant, le fil **libère** ce qu'il détient : on casse la condition « **détention et attente** » (on peut aussi dire qu'il renonce de lui-même, ce qui revient à une préemption). Le coût : du travail perdu, à recommencer.

Exercice 14 *Éviter : toujours le même ordre*

Avec `args=("P2", imprimante, scanner, journal)`, le journal vaut toujours `['P1 a les deux ressources', 'P2 a les deux ressources']` et l'exécution dure environ 0,2 s (au lieu de 1,1 s). Celui qui obtient l'imprimante en premier obtient ensuite le scanner, que personne d'autre ne peut détenir sans avoir d'abord l'imprimante : aucun fil ne peut détenir le scanner en attendant l'imprimante, donc le cycle $I \to \cdots \to S \to \cdots \to I$ ne peut pas se former. On casse l'**attente circulaire**.

Exercice 15 *Détecter : chercher un cycle dans le graphe d'attente*

1. $P_1 \to R_2 \to P_2 \to R_3 \to P_3 \to R_1 \to P_1$: cycle, donc interblocage (à trois).

2.

```
def detecte_interblocage(detient, attend, depart):
    vus = []
    p = depart
    while p in attend:            # p attend une ressource
        if p in vus:
            return True           # on est revenu sur nos pas : cycle
        vus.append(p)
        r = attend[p]
        if r not in detient:      # ressource libre : pas de blocage
            return False
        p = detient[r]
    return False                  # p n'attend rien
```

3. `True` sur l'exemple ; `False` après `del attend["P3"]` (la chaîne s'arrête sur `P3`, qui finira et libérera `R3`). C'est une recherche de **cycle** dans un graphe **orienté** (qu'on étudiera au chapitre *Graphes*) ; ici chaque processus n'attend qu'une ressource, donc on suit un seul chemin. Tous les processus rencontrés sont alors sur ce chemin, encore « en cours » : revenir sur l'un d'eux, c'est exactement le critère des graphes orientés qu'on étudiera au chapitre *Graphes* (retomber sur un sommet **en cours**). Si un processus pouvait attendre plusieurs ressources, il faudrait un vrai parcours en profondeur qui distingue sommets « en cours » et « terminés » : retomber sur un sommet simplement déjà vu ne suffirait plus à prouver un cycle.

Exercice 16 *Défi : le dîner des philosophes*

```
def vie(i, fourchettes, repas, abandons, ordonne):
    n = len(fourchettes)
    g = i
    d = (i + 1) % n
    if ordonne and d < g:        # toujours le plus petit numero d'abord
        g = d
        d = i
    for k in range(3):
        with fourchettes[g]:
            time.sleep(0.05)
            if fourchettes[d].acquire(timeout=0.5):
                repas[i] = repas[i] + 1
                fourchettes[d].release()
            else:
                abandons.append(i)
        time.sleep(0.01)

def philosophes(n, ordonne):
    fourchettes = [threading.Lock() for i in range(n)]   # un verrou par fourchette
    repas = [0] * n
    abandons = []
    fils = []
    for i in range(n):
        f = threading.Thread(target=vie, args=(i, fourchettes, repas, abandons, ordonne))
        fils.append(f)
```

```
        f.start()
    for f in fils:
        f.join()
    return repas, abandons
```

Mesures : sans ordre, `([0, 2, 0, 1, 0], [0, 4, 3, 2, 1, 0, 2, 4, 0, 3, 4, 2])` en 1,7 s environ (chacun tient sa fourchette gauche et attend la droite : cycle à cinq, beaucoup d'abandons ; les valeurs changent à chaque essai). Avec l'ordre : `([3, 3, 3, 3, 3], [])` en 0,4 s environ. Seul le philosophe 4 change de comportement : il prend la fourchette 0 avant la 4, ce qui casse le cycle.

Bilan — éléments attendus

- Sur `JEU_TP` : SRTF (1,83) < tourniquet q1 (3,33) < q2 (3,67) < SJF (3,83) < q4 (4,5) < FCFS (4,67). Serveur de calcul (lots de tâches, durées connues) : plus court d'abord, peu de commutations. Bureau : tourniquet, pour la réactivité.
- Quantum : petit = réactif mais beaucoup de commutations coûteuses ; grand = efficace mais on retombe sur FCFS.
- L'interblocage naît de l'**interaction** de programmes corrects séparément. Le subir (tout se fige : on tue un processus) ; le guérir (délai, abandon : travail perdu) ; l'éviter (ordre fixe : contrainte de conception, mais aucun coût à l'exécution).
- Un ordonnanceur réel vit dans le noyau, s'exécute des milliers de fois par seconde et manipule les registres : il est écrit en C et en assembleur ; Python sert à *simuler* et comprendre.

Réponses des questions de la fiche de révision

↑ questions p. 214

Se tester

1. Prêt : attend seulement le processeur ; bloqué : attend une ressource (entrée-sortie), même si le processeur est libre (§ III).
2. Prêt (jamais directement élu : il attend d'être élu par l'ordonnanceur) (§ III).
3. A, B, A (§ VI.C).
4. $2^3 = 8$ (§ IV).
5. Il y a un cycle ; on casse le cycle (par exemple en tuant un processus, qui libère sa ressource) (§ VII).

Et avant ?

1. `9` : le 4, premier entré, est sorti le premier (ch. 3, § III.A).
2. `def __init__(self, pid):` puis `self.pid = pid` (ch. 2, § IV.B).
3. elle s'appelle sans fin, la pile d'exécution déborde : `RecursionError` (ch. 1, § II).

Arbres

5

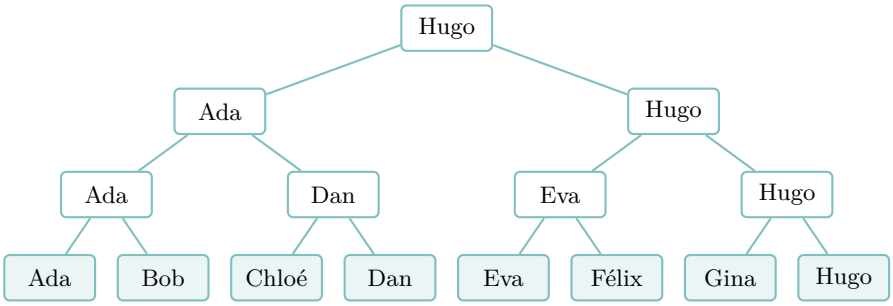
CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Un tournoi, des dossiers, une famille

↑ énoncé p. 227

Exercice 1 *Le tournoi*

1. Tableau complété :



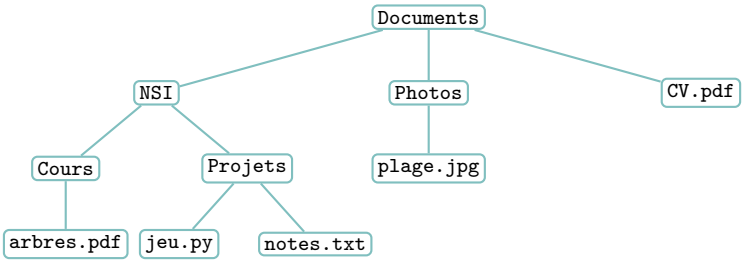
2. 7 matchs $(4 + 2 + 1)$ en 3 tours.

3. 16 joueurs : 15 matchs en 4 tours ; 64 joueurs : 63 matchs en 6 tours. Chaque match élimine **exactement un** joueur et il faut en éliminer tous sauf un : avec n joueurs, il y a $n - 1$ matchs. Le nombre de tours est l'exposant de 2 : $2^3 = 8$, $2^4 = 16$, $2^6 = 64$ (c'est $\log_2 n$) ; chaque tour divise par deux le nombre de joueurs restants.

4. Hugo a joué 3 matchs (un par tour) ; un joueur éliminé au premier tour n'en a joué qu'un. Le vainqueur est relié aux joueurs du bas par un chemin de 3 traits.

Exercice 2 *Les dossiers*

5. Dessin attendu :



6. On perd **qui est dans qui** : la liste dit quels éléments existent, mais plus que `jeu.py` est dans `Projets`, lui-même dans `NSI`. Une structure « en ligne » ne sait pas représenter une hiérarchie.

7. 3 marches : `Documents` → `NSI` → `Projets` → `jeu.py`. Les plus éloignés sont `arbres.pdf`, `jeu.py` et `notes.txt` (à 3 marches).

8. Dans le tournoi, chaque case a **exactement deux** traits vers le bas (ou aucun, pour un joueur du premier tour). Dans les dossiers, un élément peut en avoir **trois** (`Documents`), un seul (`Cours`) ou aucun (un fichier).

Exercice 3 *La famille*

9. 4 grands-parents, 8 arrière-grands-parents ; à la 10e génération, $2^{10} = 1\,024$ ancêtres (en réalité moins, car certains ancêtres se répètent : la « famille » cesse alors d'être un arbre).

10. Comme au tournoi, chaque personne est reliée vers le bas à exactement **deux** éléments (ses deux parents) : le nombre d'éléments double à chaque étage. On retrouve la même forme, sans le « vainqueur ».

Exercice 4 *Inventer le vocabulaire*

11. Propositions possibles, puis mots du cours :

Notion	Propositions fréquentes	Mot du cours
élément tout en haut	sommet, chef, tronc	**racine**
élément tout en bas	bout, extrémité	**feuille**
juste au-dessus / juste en dessous	parent / enfant	**père** / **fils**
un élément et tout ce qui est en dessous	branche, sous-dossier	**sous-arbre**
nombre total d'éléments	nombre, effectif	**taille**
nombre d'étages	niveaux, profondeur	**hauteur**

« Enfant » se dit aussi en informatique ; le trait entre deux éléments s'appellera une **arête**.

Exercice 5 *Ce qu'on a découvert*

12. Chaque élément, sauf celui du haut, est relié à **un seul** élément au-dessus de lui. Une liste ne suffit pas, car elle range les éléments **à la suite** et perd la relation « est dans » / « descend de ». Le tournoi et la famille ont **au plus deux** éléments sous chaque élément.

CORRIGÉS DES EXERCICES

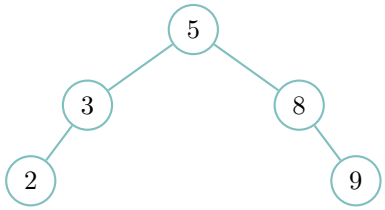
Lire un arbre

Exercice 1 *Vocabulaire* ↑énoncé p. 241

1. Racine : `A`. Feuilles : `D`, `G`, `F`. **2.** Taille = 7 ; hauteur = 4 (chemin `A--C--E--G`). **3.** Père de `G` : `E`. Fils de `C` : `E` et `F`. **4.** Profondeur de `E` : 2 (deux arêtes : `A--C--E` ; la racine est à la profondeur 0). **5.** Sous-arbre de racine `C` : les nœuds `C`, `E`, `F`, `G`.

▷ **Erreur fréquente.** Donner 3 pour la hauteur en comptant les arêtes du chemin `A--C--E--G` : la convention de la feuille compte les **nœuds**, d'où 4. Toujours appliquer la convention de l'énoncé.

Exercice 2 *Construire à partir d'une description* ↑énoncé p. 241



Taille = 5, hauteur = 3.

Parcourir un arbre

Exercice 3 *Les quatre parcours* ↑énoncé p. 242

- préfixe : `4, 2, 1, 3, 6, 5, 7`
- infixe : `1, 2, 3, 4, 5, 6, 7`
- suffixe : `1, 3, 2, 5, 7, 6, 4`
- largeur : `4, 2, 6, 1, 3, 5, 7`

Le parcours **infixe** donne les valeurs **triées par ordre croissant** : cet arbre est en fait un **ABR**.

▷ **Erreur fréquente.** Écrire en préfixe `4, 2, 6, ...`, niveau par niveau : c'est le parcours en largeur. En profondeur, on termine tout le sous-arbre gauche (`2, 1, 3`) avant de passer au droit.

Exercice 4 *Parcours d'un arbre irrégulier* ↑énoncé p. 242

1.

- préfixe : `M, K, R, E, P, T, S, A, N`
- infixe : `K, E, R, P, M, S, N, A, T`
- suffixe : `E, P, R, K, N, A, S, T, M`
- largeur : `M, K, T, R, S, E, P, A, N`

▷ **Erreur fréquente.** Commencer le parcours infixe par `E`, la première feuille rencontrée : `K` n'a pas de fils gauche, son sous-arbre gauche est vide, on le visite donc **tout de suite**, avant son sous-arbre droit. De même, `T` n'a pas de fils droit : il est visité juste après son sous-arbre gauche, en dernier.

2. Ni l'un ni l'autre. Le premier nœud visité en infixe est `K`, qui a un fils droit ; le dernier est `T`, qui a un fils gauche. En infixe, le premier nœud visité est celui que l'on atteint en descendant à gauche tant que c'est possible : il n'a pas de fils gauche, mais il peut avoir un fils droit.

3.

- **Préfixe.** Le premier nœud visité est la racine : ce n'est une feuille que si l'arbre est réduit à sa racine. Le dernier est **toujours** une feuille : un nœud qui a un fils est visité *avant* ce fils, il ne peut donc pas être le dernier.
- **Infixe.** Ni le premier ni le dernier ne sont forcément des feuilles (question 2).
- **Suffixe.** Le premier nœud visité est **toujours** une feuille : un nœud est visité *après* ses fils, donc le premier visité n'en a pas. Le dernier est la racine.

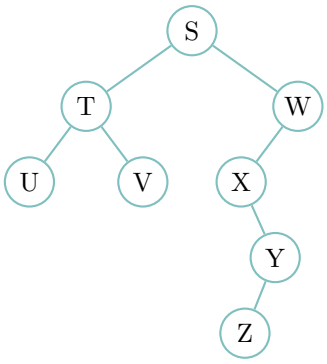
Exercice 5 *Retrouver le parcours, retrouver l'arbre* ↑énoncé p. 242

1. On repère d'abord la place de la racine `5`. Elle est en **dernier** dans la suite a) : c'est le **suffixe**. Elle est **au milieu** dans la suite d), après les quatre nœuds du sous-arbre gauche (`4 9 7 1`) : c'est l'**infixe**. Elle est en **premier** dans b), c) et e) : il reste à départager le préfixe et la largeur.

- En largeur, les deux fils de la racine, `9` et `2`, viennent juste après elle : c'est b).
- En préfixe, on entre dans le sous-arbre gauche : `9`, puis **son** fils gauche `4` : c'est e).
- La suite c) donne `1` juste après `9` : ce n'est pas le préfixe (le fils gauche `4` de `9` doit passer avant `1`), ni la largeur (`2` doit suivre `9`). Elle n'est **aucun** des quatre parcours.

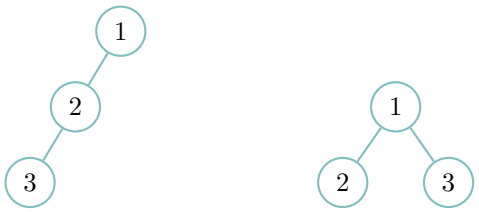
2. La racine est `S` (première lettre du préfixe). Dans l'infixe, `U T V` est à sa gauche et `X Z Y W` à sa droite.

- À gauche : préfixe `T U V`, infixe `U T V` : racine `T`, fils gauche `U`, fils droit `V`.
- À droite : préfixe `W X Y Z`, infixe `X Z Y W` : racine `W`, tout le reste à sa gauche (rien à droite). Puis préfixe `X Y Z`, infixe `X Z Y` : racine `X`, rien à gauche ; enfin préfixe `Y Z`, infixe `Z Y` : racine `Y`, fils gauche `Z`.



Parcours suffixe : `U, V, T, Z, Y, X, W, S`.

3. Les deux arbres ci-dessous ont le même parcours préfixe `1, 2, 3` (à gauche, `3` est le fils gauche de `2` ; à droite, `2` et `3` sont les deux fils de `1`). Leurs parcours infixes, `3, 2, 1` et `2, 1, 3`, sont différents : c'est le **couple** préfixe et infixe (avec des valeurs toutes différentes) qui permet de reconstruire l'arbre.



Exercice 6 *Programmer les parcours en profondeur* ↑énoncé p. 243 **Démarche :** condition d'arrêt : l'arbre vide, on n'affiche rien. Sinon, trois instructions : afficher la racine, parcourir le sous-arbre gauche, parcourir le sous-arbre droit. Seule la place de l'affichage distingue les trois parcours.

```
def prefixe(a):
    if a is not None:          # CAS DE BASE : arbre vide, rien a afficher
        print(a.valeur)        # visiter : la ligne a deplacer
        prefixe(a.gauche)      # APPEL RECURSIF
        prefixe(a.droite)      # APPEL RECURSIF
```

Infixe : placer `print(a.valeur)` *entre* `prefixe(a.gauche)` et `prefixe(a.droite)`. **Suffixe** : placer `print(a.valeur)` *après* les deux appels.

▷ **Erreur fréquente.** Écrire `print(a.valeur)` sans avoir testé `a is not None` : sur un sous-arbre vide, Python lève `AttributeError` (`None` n'a pas d'attribut `valeur`).

Exercice 7 *Défi — parcours en largeur* ↑énoncé p. 243

```
def largeur(a):
    if a is None:                # TESTER : arbre vide, rien a afficher
        return
    f = File()                   # INITIALISER
    f.enfiler(a)                 # INITIALISER : la racine attend son tour
    while not f.est_vide():      # PARCOURIR
        n = f.defiler()
        print(n.valeur)
        if n.gauche is not None: # TESTER
            f.enfiler(n.gauche)  # METTRE A JOUR : le fils passe en fin de file
        if n.droite is not None: # TESTER
            f.enfiler(n.droite)  # METTRE A JOUR
```

Mesurer un arbre

Exercice 8 *Taille* ↑énoncé p. 243 **Démarche :** condition d'arrêt : l'arbre vide, de taille 0. Sinon, on compte la racine (1) et l'on ajoute les tailles des deux sous-arbres, obtenues par deux appels récursifs.

```
def taille(a):
    if a is None:                                  # CAS DE BASE : arbre vide
        return 0
    return 1 + taille(a.gauche) + taille(a.droite)     # APPEL RECURSIF x 2, COMBINER
```

Exercice 9 *Hauteur* ↑énoncé p. 243 *Appels attendus, écrits avant de coder :*

```
a = Noeud(1, Noeud(2), Noeud(3, None, Noeud(4)))
assert hauteur(a) == 3            # cas normal : chemin 1, 3, 4
assert hauteur(None) == 0         # cas limite : arbre vide
assert hauteur(Noeud(5)) == 1     # cas piege : une feuille compte 1 noeud, pas 0
```

Démarche : condition d'arrêt : l'arbre vide, de hauteur 0. Sinon, la racine ajoute un niveau au plus haut des deux sous-arbres.

```
def hauteur(a):
    if a is None:                                    # CAS DE BASE : arbre vide
        return 0
    return 1 + max(hauteur(a.gauche), hauteur(a.droite))   # APPEL RECURSIF x 2, COMBINER
```

▷ **Erreur fréquente.** Écrire `1 + hauteur(a.gauche) + hauteur(a.droite)` : on additionne les deux sous-arbres (on obtient presque la taille) au lieu de garder le plus haut des deux.

Avec la convention des arêtes, seul le cas de base change : l'arbre vide vaut -1, pour qu'une feuille vaille $1 + \max(-1, -1) = 0$.

```
def hauteur_aretes(a):
    if a is None:                                          # CAS DE BASE
        return -1
    return 1 + max(hauteur_aretes(a.gauche), hauteur_aretes(a.droite))   # COMBINER
```

Exercice 10 *Nombre de feuilles* ↑énoncé p. 243 *Appels attendus, écrits avant de coder :*

```
a = Noeud(1, Noeud(2), Noeud(3, None, Noeud(4)))
assert nb_feuilles(a) == 2                      # cas normal : 2 et 4
assert nb_feuilles(None) == 0                   # cas limite : arbre vide
assert nb_feuilles(Noeud(3, None, Noeud(4))) == 1   # cas piege : 3 a un fils, pas une feuille
```

Démarche : deux conditions d'arrêt : l'arbre vide (0 feuille) et le nœud sans fils (1 feuille). Sinon, on additionne les nombres de feuilles des deux sous-arbres.

```
def nb_feuilles(a):
    if a is None:                                  # CAS DE BASE : arbre vide
        return 0
    if a.gauche is None and a.droite is None:      # CAS DE BASE : une feuille
        return 1
    return nb_feuilles(a.gauche) + nb_feuilles(a.droite)   # APPEL RECURSIF x 2, COMBINER
```

Arbres binaires de recherche

Exercice 11 *Rechercher dans un ABR* ↑énoncé p. 243 *Appels attendus, écrits avant de coder :*

```
a = Noeud(8, Noeud(3, Noeud(1), Noeud(6)), Noeud(10, None, Noeud(14)))   # l'ABR du cours
assert recherche(a, 6)            # cas normal
assert not recherche(None, 5)     # cas limite : arbre vide
assert not recherche(a, 7)        # cas piege : absent, mais encadre par 6 et 8
```

Démarche : on compare `x` à la racine : égal, il est trouvé ; plus petit, on cherche à gauche ; plus grand, à droite. Un seul appel récursif ; condition d'arrêt : l'arbre vide (`x` est absent).

```
def recherche(a, x):
    if a is None:                    # CAS DE BASE : x est absent
        return False
    if x == a.valeur:                # TESTER : trouve
        return True
    if x < a.valeur:                 # TESTER : de quel cote descendre ?
        return recherche(a.gauche, x)   # APPEL RECURSIF : les plus petits sont a gauche
    else:
        return recherche(a.droite, x)   # APPEL RECURSIF : les plus grands sont a droite
```

Autre méthode : comme il n'y a qu'**un seul** appel récursif, on peut aussi descendre avec une boucle : tant qu'on n'est pas sorti de l'arbre, on compare et on part à gauche ou à droite.

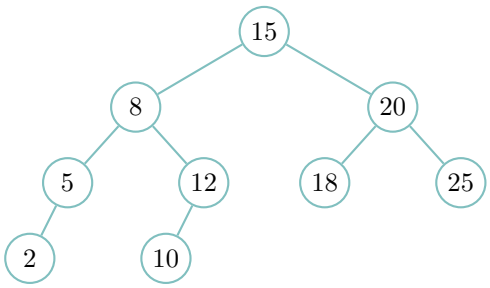
```
def recherche(a, x):
    while a is not None:          # PARCOURIR : on descend dans l'arbre
        if x == a.valeur:         # TESTER
            return True
        if x < a.valeur:          # TESTER
            a = a.gauche          # METTRE A JOUR
        else:
            a = a.droite          # METTRE A JOUR
    return False                  # RENVOYER : sorti de l'arbre, x est absent
```

Au plus **`h` nœuds visités** (un par niveau), où `h` est la hauteur de l'arbre.

Exercice 12 *Insérer dans un ABR* ↑énoncé p. 244

1. `10` : < 15 (gauche), > 8 (droite), < 12 (gauche) → fils gauche de `12`. `18` : > 15, < 20 → fils gauche de `20`. `2` : < 15, < 8, < 5 → fils gauche de `5`.

▷ **Erreur fréquente.** Arrêter la descente trop tôt (placer `10` comme fils de `8`) : une nouvelle valeur descend jusqu'à une place **vide** et devient toujours une feuille.



2. Le parcours **infixe** (gauche, racine, droite) affiche les valeurs par ordre croissant.

Exercice 13 *Programmer l'insertion dans un ABR* ↑énoncé p. 244

1. *Appels attendus, écrits avant de coder :*

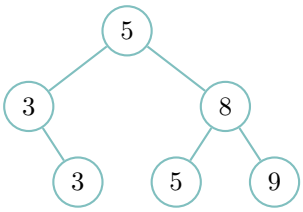
```
assert inserer(Noeud(5), 3).gauche.valeur == 3   # cas normal : plus petit, a gauche
assert inserer(None, 4).valeur == 4              # cas limite : arbre vide, une feuille est
                                                 # creee
assert inserer(Noeud(5), 5).droite.valeur == 5   # cas piege : doublon, a droite (convention)
```

Démarche : trois cas : l'arbre vide (la valeur devient une nouvelle feuille), `x` plus petit que la racine (on insère à gauche), sinon à droite. On réaffecte le sous-arbre modifié, puis on renvoie `a`.

```
def inserer(a, x):
    if a is None:                        # CAS DE BASE : place vide
        return Noeud(x)                  # nouvelle feuille
    if x < a.valeur:                     # TESTER
        a.gauche = inserer(a.gauche, x)  # APPEL RECURSIF : plus petit, a gauche
    else:
        a.droite = inserer(a.droite, x)  # APPEL RECURSIF : plus grand ou egal, a droite
    return a                             # RENVOYER
```

▷ **Erreur fréquente.** Écrire `inserer(a.gauche, x)` sans la réaffectation `a.gauche = ...` : quand `a.gauche` vaut `None`, le nœud créé est perdu.

2. Le second `3` va à droite du premier `3` ; le second `5` va à droite de la racine `5`, puis à gauche de `8`.



Parcours infixe : `3, 3, 5, 5, 8, 9` — trié.

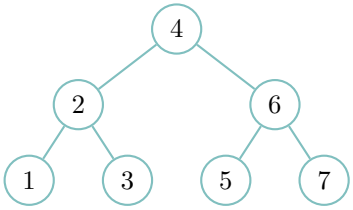
3. Démarche : on insère les valeurs une à une dans un ABR vide ; un parcours infixe qui **renvoie une liste** donne alors les valeurs triées.

```
def infixe(a):
    if a is None:                                          # CAS DE BASE
        return []
    return infixe(a.gauche) + [a.valeur] + infixe(a.droite)   # COMBINER : gauche, racine,
                                                           # droite

def tri_abr(t):
    a = None                                               # INITIALISER : ABR vide
    for v in t:                                            # PARCOURIR
        a = inserer(a, v)                                  # METTRE A JOUR
    return infixe(a)                                       # RENVOYER
```

Par exemple `tri_abr([5, 3, 8, 3, 9, 5, 1])` vaut `[1, 3, 3, 5, 5, 8, 9]`.

4. Avec `1, 2, ..., 7`, chaque valeur devient le fils droit de la précédente : un arbre « en peigne » de hauteur 7 (une chaîne 1–2–…–7). Avec `4, 2, 6, 1, 3, 5, 7`, on obtient l'arbre équilibré ci-dessous, de hauteur 3.



Au pire, une recherche visite autant de nœuds que la hauteur : 7 dans le peigne (comme dans une liste), 3 dans l'arbre équilibré. Mêmes valeurs, même nombre de nœuds : seul l'**ordre d'insertion** a changé.

Pour aller plus loin

Exercice 14 *Défi — l'arbre miroir* ↑énoncé p. 244 *Appels attendus, écrits avant de coder :*

```
a = Noeud(1, Noeud(2, Noeud(4)), Noeud(3))                 # l'arbre de l'enonce
m = miroir(a)
assert m.gauche.valeur == 3 and m.droite.droite.valeur == 4   # cas normal
assert miroir(None) is None                                # cas limite : arbre vide
assert a.gauche.valeur == 2                                # cas piege : a n'est pas modifie
```

```
def miroir(a):
    if a is None:                                          # CAS DE BASE
        return None
    return Noeud(a.valeur, miroir(a.droite), miroir(a.gauche))  # COMBINER : g et d echanges
```

Exercice 15 *Défi — maximum d'un arbre* ↑énoncé p. 244 *Appels attendus, écrits avant de coder :*

```
assert maximum(Noeud(3, Noeud(9), Noeud(1, Noeud(12)))) == 12   # cas normal : pas un ABR
assert maximum(Noeud(5)) == 5                                   # cas limite : un seul noeud
assert maximum(Noeud(-4, Noeud(-2), Noeud(-7))) == -2           # cas piege : que des negatifs
```

```
def maximum(a):
    m = a.valeur                        # INITIALISER : la racine
    if a.gauche is not None:            # TESTER : sous-arbre non vide
        m = max(m, maximum(a.gauche))   # APPEL RECURSIF, METTRE A JOUR
    if a.droite is not None:            # TESTER
        m = max(m, maximum(a.droite))   # APPEL RECURSIF, METTRE A JOUR
    return m                            # RENVOYER
```

Vers le bac

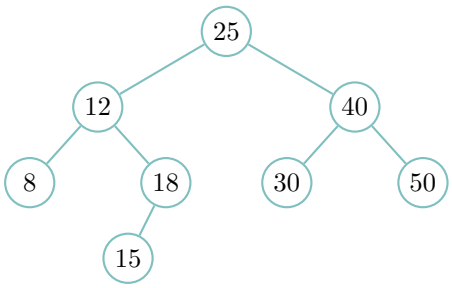
Exercice 16 *ABR : mesures, insertion, recherche (d'après Métropole 2023, jour 2)* ↑énoncé p. 245

Barème indicatif (sur 3) : Q1 : 0,5 ; Q2 : 0,75 ; Q3 : 0,25 ; Q4 : 1,5

1. Taille $= 6$, hauteur $= 3$.

2. `15` : < 25, > 12, < 18 → fils gauche de `18`. `30` : > 25, < 40 → fils gauche de `40`.

▷ **Erreur fréquente.** Placer `15` comme fils droit de `12` : `15` est bien plus grand que `12`, mais il faut continuer à descendre et le comparer à `18`.



3. **(C)** parcours infixe.

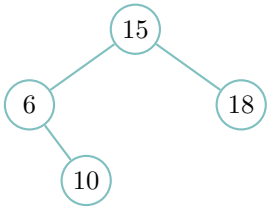
4. ligne 5 : `True` ; ligne 7 : `self.sd().present(x)` ; ligne 9 : `self.sg().present(x)`.

Exercice 17 *Un arbre binaire par une classe (d'après Sujet zéro 2023, sujet B)* ↑énoncé p. 245

Barème indicatif (sur 3) : Q1 : 0,5 ; Q2 : 1 ; Q3 : 1,5

1. Un attribut : `valeur` (ou `enfant_gauche`, `enfant_droit`). Une méthode : `insert_gauche` (ou `get_valeur`, `get_gauche`…).

2. `r` a la valeur `15`, un fils gauche `6` et un fils droit `18` ; puis `6` reçoit un fils droit `10`.



3. *Appels attendus, écrits avant de coder :*

```
t = ArbreBinaire(1)
t.insert_gauche(2)
t.insert_gauche(3)
assert taille(r) == 4          # cas normal : l'arbre de la question 2
assert taille(None) == 0       # cas limite : arbre vide
assert taille(t) == 3          # cas piege : le second insert_gauche n'ecrase pas le premier
```

Démarche : même structure que la fonction `taille` du cours ; seuls changent les accès aux sous-arbres, par les accesseurs.

```
def taille(arbre):
    if arbre is None:                                          # CAS DE BASE
        return 0
    return 1 + taille(arbre.get_gauche()) + taille(arbre.get_droit())    # COMBINER
```

Exercice 18 *Un arbre de décision (d'après Amérique du Nord 2025, jour 1)*

↑énoncé p. 246

Barème indicatif (sur 4) : Q1 : 1 ; Q2 : 0,75 ; Q3 : 1 ; Q4 : 1,25

1. Démarche : chaque question devient un `Noeud` (question, branche oui, branche non), chaque résultat une `Feuille` ; on écrit l'arbre de la racine vers les feuilles.

```
arbre = Noeud("A des plumes ?",
              Feuille("Oiseau"),
              Noeud("Vit dans l'eau ?",
                    Feuille("Poisson"),
                    Feuille("Autre")))
```

2. Dans la classe `Feuille` : `def est_resultat(self): return True`. Dans la classe `Noeud` : `def est_resultat(self): return False`.

3. *Appels attendus, écrits avant de coder :*

```
assert arbre.nb_resultats() == 3                    # cas normal : l'arbre de la question 1
assert Feuille("Oiseau").nb_resultats() == 1        # cas limite : une feuille seule
assert Noeud("?", Feuille("Chat"), Feuille("Chat")).nb_resultats() == 2   # cas piege :
                                                                          # resultats identiques
```

Dans `Feuille` : `def nb_resultats(self): return 1` (le cas de base). Dans `Noeud` :

```
def nb_resultats(self):
    return self.sioui.nb_resultats() + self.sinon.nb_resultats()    # APPEL RECURSIF x 2,
                                                                    # COMBINER
```

Remarque : chaque classe a sa propre version de la méthode ; à l'appel `self.sioui.nb_resultats()`*, Python exécute celle de la classe de l'objet (*`Feuille` *ou* `Noeud`*). C'est le* ***polymorphisme****, au-delà du programme.*

4. Démarche : on part de la racine et l'on descend tant que l'objet courant n'est pas un résultat ; la réponse de l'utilisateur choisit la branche.

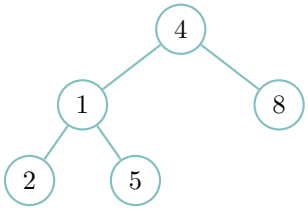
```
def diagnostic(arbre):
    while not arbre.est_resultat():                          # PARCOURIR : on descend
        reponse = input(arbre.question + " (oui/non) ")
        if reponse == "oui":                                 # TESTER
            arbre = arbre.sioui                              # METTRE A JOUR
        else:
            arbre = arbre.sinon                              # METTRE A JOUR
    return arbre.resultat                                    # RENVOYER
```

Exercice 19 *Un arbre représenté par des listes (d'après Amérique du Sud 2023, jour 1)*

↑énoncé p. 247

Barème indicatif (sur 4) : Q1 : 0,75 ; Q2 : 1,25 ; Q3 : 2

1.



2. ligne 7 : `while not est_vide(file):` ; ligne 8 : `arbre = defiler(file)` ; ligne 11 : `p = p + arbre[0]`.

3. *Appels attendus, écrits avant de coder :*

```
assert est_mobile([1, [[4, []], [4, []]]])   # cas normal : deux feuilles de meme poids
assert est_mobile([7, []])                   # cas limite : une feuille
assert not est_mobile(a)                     # cas piege : n1 et n8 pesent 8, mais n1 n'est
                                           # pas un mobile
```

```
def est_mobile(arbre):
    if arbre == []:                          # CAS DE BASE : l'arbre vide est un mobile
        return True
    sous_arbres = arbre[1]
    for sa in sous_arbres:                   # PARCOURIR : meme poids + chacun mobile
        if poids(sa) != poids(sous_arbres[0]):   # TESTER
            return False
        if not est_mobile(sa):               # APPEL RECURSIF
            return False
    return True                              # RENVOYER : aucun defaut trouve
```

Exercice 20 *La plus grande somme racine-feuille (d'après Amérique du Sud 2022, jour 2)*

↑énoncé p. 248

Barème indicatif (sur 3) : Q1 : 1 ; Q2 : 2

1. Les chemins racine-feuille et leurs sommes : `2-7-4-2`= 15 ; `2-7-4-3`= 16 ; `2-7-1-5`= 15 ; `2-5-9-6`= **22**. La plus grande somme est **22**.

2. *Appels attendus, écrits avant de coder :*

```
e = Noeud(2, Noeud(7, Noeud(4, Noeud(2), Noeud(3)), Noeud(1, None, Noeud(5))),
          Noeud(5, None, Noeud(9, Noeud(6))))   # l'arbre de l'enonce
assert somme_max(e) == 22                         # cas normal
assert somme_max(Noeud(5)) == 5                   # cas limite : une feuille seule
assert somme_max(Noeud(1, None, Noeud(-4))) == -3   # cas piege : un seul fils (et negatif)
```

```
def somme_max(a):
    if a.gauche is None and a.droite is None:      # CAS DE BASE : une feuille
        return a.valeur
    if a.gauche is None:                           # TESTER : un seul sous-arbre (droit)
        return a.valeur + somme_max(a.droite)      # APPEL RECURSIF
    if a.droite is None:                           # TESTER : un seul sous-arbre (gauche)
        return a.valeur + somme_max(a.gauche)      # APPEL RECURSIF
    return a.valeur + max(somme_max(a.gauche), somme_max(a.droite))   # COMBINER
```

Exercice 21 *Puissance 4 : l'arbre de coups et l'algorithme min-max (d'après Amérique du Nord 2026, jour 1, parties B et C)* ↑énoncé p. 248

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 1,25 ; Q3 : 2 ; Q4 : 0,75 ; Q5 : 1,5

▶ Solution.

1.
```python
class Noeud:
    def __init__(self, colonne):
        self.colonne = colonne
        self.score = 0
        self.suivants = []
```

2. *Appels attendus, écrits avant de coder :*

```python
def avec_scores(scores):           # outil de test : des fils de colonnes 0, 1, 2...
    n = Noeud(-1)
    for c in range(len(scores)):
        fils = Noeud(c)
        fils.score = scores[c]
        n.suivants.append(fils)
    return n

assert avec_scores([4, -30, 7]).colonne_score_min() == (1, -30)   # cas normal
assert avec_scores([12]).colonne_score_min() == (0, 12)           # cas limite : un seul fils
assert avec_scores([5, 8]).colonne_score_min() == (0, 5)          # cas piege : scores positifs
```

On cherche le minimum en parcourant la liste des fils (partir d'un score 0 ferait échouer le cas piège) :

```python
def colonne_score_min(self):
    meilleur = self.suivants[0]            # INITIALISER : le premier fils
    for fils in self.suivants:             # PARCOURIR
        if fils.score < meilleur.score:    # TESTER
            meilleur = fils                # METTRE A JOUR
    return (meilleur.colonne, meilleur.score)   # RENVOYER
```

3. La méthode est **récursive** : chaque fils calcule son propre score (au niveau suivant et pour l'autre joueur, `3 - joueur`), puis le nœud prend le minimum ou le maximum des scores de ses fils.

```python
def calcule_score(self, niveau, joueur, grille):
    g = grille.gagnant()
    if g == 1:                                  # CAS DE BASE : coup gagnant
        self.score = -(100 + 10 * (niveau_max - niveau))
    elif g == 2:                                # CAS DE BASE : coup gagnant
        self.score = 100 + 10 * (niveau_max - niveau)
    elif niveau == niveau_max:                  # CAS DE BASE : profondeur maximale
        self.score = grille.score()
    else:
        for colonne in range(7):                # PARCOURIR : chaque coup possible
            grille2 = grille.copie_grille()
            if grille2.joue(colonne, joueur):   # TESTER : colonne non pleine
                nouveau_noeud = Noeud(colonne)
                self.suivants.append(nouveau_noeud)
                nouveau_noeud.calcule_score(niveau + 1, 3 - joueur, grille2)   # APPEL RECURSIF
        if joueur == 1:                         # COMBINER : min ou max des fils
            self.score = self.colonne_score_min()[1]
        else:
            self.score = self.colonne_score_max()[1]
```

4. Chaque nœud a jusqu'à 7 fils : l'arbre complet de profondeur 42 compterait de l'ordre de $7^{42} \approx 3 \times 10^{35}$ nœuds. Ni le temps de calcul ni la mémoire ne le permettent ; on limite donc `niveau_max` à quelques coups.

5. On construit l'arbre à partir d'une racine de colonne `-1`, puis on choisit le fils de plus petit score (joueur 1) ou de plus grand score (joueur 2).

```
def choisit_coup(grille, joueur):
    racine = Noeud(-1)
    racine.calcule_score(0, joueur, grille)
    if joueur == 1:
        return racine.colonne_score_min()[0]
    else:
        return racine.colonne_score_max()[0]
```

Vérification : avec trois pions du joueur 2 alignés en bas des colonnes 0, 1 et 2, `choisit_coup` *renvoie* `3` *pour le joueur 2 (il gagne) comme pour le joueur 1 (il bloque), dès que* `niveau_max` *vaut au moins* 2. *Avec* `niveau_max = 1`*, le joueur 1 ne simule pas la réponse de son adversaire : il ne voit pas la menace et ne bloque pas forcément.*

Critiquer une réponse d'IA

Exercice 22 *Un test d'ABR proposé par un assistant d'IA* ↑énoncé p. 250

1. Non. L'arbre proposé a pour racine `8`, avec `12` dans son sous-arbre **gauche** : ce n'est **pas** un ABR (toute valeur du sous-arbre gauche doit être plus petite que `8`). Or la fonction renvoie `True`, car chaque nœud est « localement » correct : `3 < 8`, `10 >= 8`, `1 < 3 <= 12`.

2. L'erreur : la fonction ne compare chaque nœud qu'à ses **fils**, alors que la définition porte sur **toutes les valeurs** du sous-arbre (« *toutes les valeurs de son sous-arbre gauche sont plus petites que sa valeur* »). Il faut propager un encadrement le long de la descente : dans un sous-arbre gauche, toute valeur doit rester **strictement inférieure** à `maxi` ; dans un sous-arbre droit, **supérieure ou égale** à `mini` (convention du cours : doublon à droite) :

```
def est_abr(a, mini=None, maxi=None):
    if a is None:                              # CAS DE BASE
        return True
    if mini is not None and a.valeur < mini:   # TESTER : a droite, >= mini
        return False
    if maxi is not None and a.valeur >= maxi:  # TESTER : a gauche, < maxi
        return False
    return est_abr(a.gauche, mini, a.valeur) and est_abr(a.droite, a.valeur, maxi)   # COMBINER
```

Avec cette version, l'arbre de la question 1 donne `False`, et l'ABR du cours (racine `8`, fils `3` et `10`, feuilles `1`, `6`, `14`) donne bien `True`.

3. Tester la fonction sur un **contre-exemple** construit à la main (un arbre qui n'est *pas* un ABR mais dont chaque nœud respecte l'ordre avec ses seuls fils), et pas seulement sur des arbres « qui marchent » : une fonction qui répond `True` à tout n'a été mise en défaut par aucun test.

S'entraîner à l'oral

Exercice 23 *Expliquer en deux minutes* ↑énoncé p. 251

▶ **Solution.** Pas de texte à apprendre par cœur : voici les **éléments attendus** pour chaque sujet. L'ordre, les mots et l'exemple peuvent être différents ; l'explication est réussie si ces idées y sont, justes et reliées entre elles.

Sujet 1.

- **ABR** : arbre binaire où, pour **chaque** nœud, les valeurs du sous-arbre gauche sont plus petites que la sienne et celles du sous-arbre droit plus grandes.
- Recherche : on compare à la racine, puis on descend d'**un seul** côté ; le nombre de comparaisons est au plus la **hauteur** de l'arbre.
- Exemple : dans l'ABR obtenu en insérant $8, 3, 10, 1, 6$, chercher 6 : 8 (aller à gauche), 3 (aller à droite), 6 trouvé en 3 comparaisons.
- Piège : vérifier la propriété seulement entre un nœud et ses deux fils ; elle porte sur tout le sous-arbre.

Sujet 2.

- Préfixe : racine, puis sous-arbre gauche, puis sous-arbre droit ; infixe : gauche, racine, droit ; suffixe : gauche, droit, racine.
- Exemple : racine 2, fils gauche 1, fils droit 3 : préfixe $2, 1, 3$; infixe $1, 2, 3$; suffixe $1, 3, 2$.
- Le parcours infixe d'un ABR donne les valeurs dans l'**ordre croissant** (c'est un tri).
- Piège : appliquer l'ordre aux seuls nœuds du haut ; la règle s'applique récursivement à **chaque** sous-arbre.

Sujet 3.

- Le coût d'une recherche est lié à la **hauteur**, pas à la taille.
- Arbre bien équilibré de n nœuds : hauteur proche de $\log_2 n$ (un million de valeurs : une vingtaine de niveaux).
- Valeurs insérées déjà triées $(1, 2, 3, \dots)$: chaque nœud n'a qu'un fils droit, l'arbre est **filiforme**, de hauteur n : chercher revient à parcourir une liste.
- Piège : il existe deux conventions de hauteur (arbre réduit à sa racine : 1 ou 0) ; annoncer celle qu'on utilise, comme dans un énoncé.

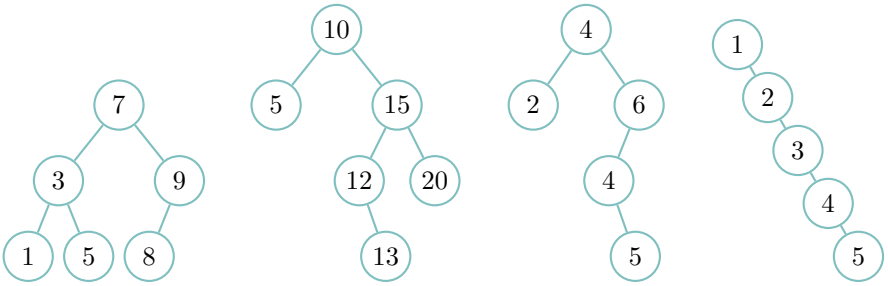
Entraînement

Exercice 24 *Parcours en série* ↑énoncé p. 251

	Préfixe	Infixe	Suffixe	Largeur
a)	A B D E C F	D B E A C F	D E B F C A	A B C D E F
b)	5 9 2 1 7 3 6	9 2 5 7 1 6 3	2 9 7 6 3 1 5	5 9 1 2 7 3 6
c)	M N O P Q	O P N M Q	P O N Q M	M N Q O P

▷ **Erreur fréquente.** En b), commencer le parcours infixe par `2, 9` : 9 n'a pas de fils gauche ; on le visite donc *avant* son fils droit 2 (sous-arbre gauche vide, racine, sous-arbre droit).

Exercice 25 *Insertions en série dans un ABR* ↑énoncé p. 252



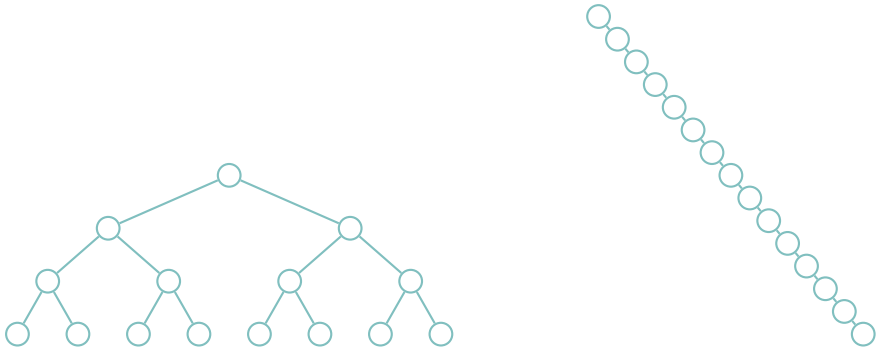
a) hauteur 3 **b)** hauteur 4 **c)** hauteur 4 **d)** hauteur 5

En c), le second `4` va à droite de la racine (égalité), puis à gauche de `6` ; `5` descend ensuite à droite des deux `4`. En d), les valeurs arrivent triées : l'arbre est un « peigne », aussi haut qu'il a de nœuds.

Encadrer la hauteur

Exercice 26 *Encadrer la hauteur* ↑énoncé p. 252

1. D'après l'encadrement du cours, $\log_2(16) = 4 \leqslant h \leqslant 15$. La plus petite hauteur est 4 : l'arbre **parfait**, dont les quatre niveaux sont pleins ($1+2+4+8=15$ nœuds). La plus grande est 15 : l'arbre **filiforme**, une chaîne où chaque nœud a un seul fils.



2. Au plus $1+2+4+8+16 = 2^5 - 1 = 31$ nœuds (tous les niveaux pleins) ; au moins 5 (un nœud par niveau, arbre filiforme).

3. $2^{19} - 1 = 524\,287 < 1\,000\,000 \leqslant 2^{20} - 1 = 1\,048\,575$: la hauteur est comprise entre 20 et 1 000 000. Une recherche descend d'un seul côté et visite au plus un nœud par niveau, soit au plus h nœuds : une **vingtaine** si l'arbre est de hauteur minimale, jusqu'à un **million** s'il est filiforme. D'où l'intérêt des arbres équilibrés (coût logarithmique).

4. Chaque valeur est plus grande que toutes les précédentes : elle va toujours à droite. On obtient une chaîne de 100 nœuds, de hauteur 100, le pire cas de l'encadrement. Pour 1 à 7, insérer d'abord la valeur du milieu, puis les milieux de chaque moitié : `4, 2, 6, 1, 3, 5, 7` donne l'arbre parfait de hauteur $3 = \log_2(8)$. (D'autres ordres conviennent, comme `4, 6, 2, 7, 5, 3, 1` : il suffit que chaque nœud soit inséré avant ses fils.)

5. Avec les arêtes, on retranche 1 : la hauteur d'un arbre de 15 nœuds est comprise entre 3 et 14.

CORRIGÉS DES TP

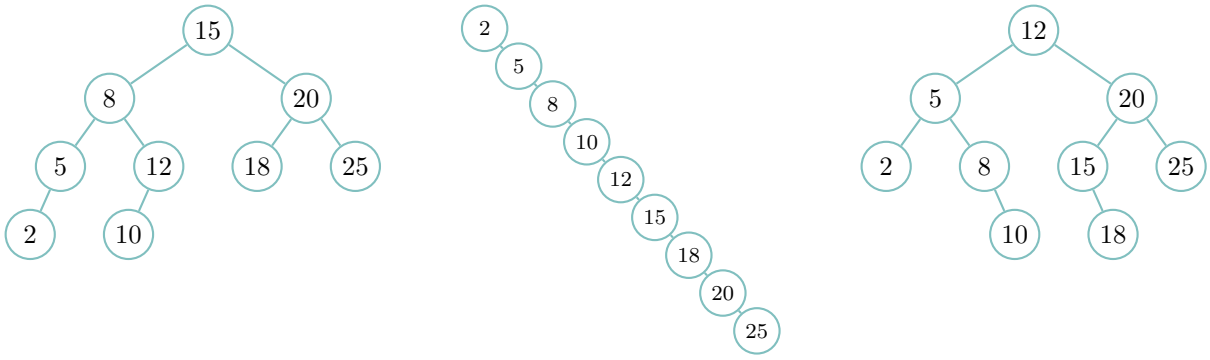
TP 1 Un arbre binaire de recherche, de A à Z

↑énoncé p. 255

Avec ces solutions, les six tests du fichier `tp_arbres_depart.py` affichent `[OK]`.

1 Construire un ABR

Exercice 1 *L'ordre d'insertion compte*



1. Arbre de gauche : taille 9, hauteur 4 (chemin `15--8--5--2` ou `15--8--12--10`).

2. Arbre du milieu : chaque valeur est plus grande que toutes les précédentes, donc elle va toujours à droite. Taille 9, hauteur 9 : chaque nœud n'a qu'un fils, l'arbre est une « dent de peigne », en fait une liste chaînée déguisée.

3. Par exemple `12, 5, 20, 2, 8, 15, 25, 10, 18` (arbre de droite) : hauteur 4. Un bon ordre commence par une valeur « médiane », puis les médianes de chaque moitié.

Exercice 2 *Insérer sans récursivité*

```python
class ABR:
    def __init__(self):
        self._racine = None
        self._taille = 0

    def inserer(self, x):
        nouveau = Noeud(x)
        if self._racine is None:
            self._racine = nouveau
            self._taille = 1
            return True
        n = self._racine
        place = False                    # le nouveau noeud est-il accroche ?
        while not place:
            if x == n.valeur:            # deja present : on ne fait rien
                return False
            if x < n.valeur:
                if n.gauche is None:
                    n.gauche = nouveau
                    place = True
                else:
                    n = n.gauche
```

```
            else:
                if n.droite is None:
                    n.droite = nouveau
                    place = True
                else:
                    n = n.droite
        self._taille = self._taille + 1
        return True
```

L'attribut `_taille` répond **immédiatement**, alors que la fonction `taille` parcourt tout l'arbre (n appels) à chaque fois. En contrepartie, il faut le tenir à jour dans chaque méthode qui modifie l'arbre : c'est un **invariant** que le test vérifie.

2 Chercher

Exercice 3 *Chercher en comptant ses pas*

1.

```
    def contient(self, x):
        n = self._racine
        visites = 0
        while n is not None:
            visites = visites + 1
            if x == n.valeur:
                return True, visites
            if x < n.valeur:
                n = n.gauche
            else:
                n = n.droite
        return False, visites
```

2. Pour `13` : `15` (on va à gauche), `8` (à droite), `12` (à droite) : sous-arbre vide, absent, 3 visites. Chaque visite descend d'un niveau, sur un unique chemin partant de la racine : on visite au plus autant de nœuds que le plus long chemin racine–feuille en contient, c'est-à-dire la hauteur.

Exercice 4 *Le plus petit et le plus grand*

```
    def minimum(self):
        if self._racine is None:
            raise ValueError("arbre vide")
        n = self._racine
        while n.gauche is not None:
            n = n.gauche
        return n.valeur

    def maximum(self):
        if self._racine is None:
            raise ValueError("arbre vide")
        n = self._racine
        while n.droite is not None:
            n = n.droite
        return n.valeur
```

Le minimum est le nœud le plus à gauche : il n'a pas de fils gauche, mais **pas forcément** une feuille (il peut avoir un fils droit, comme `2` dans l'arbre en peigne).

3 Le parcours infixe trie

Exercice 5 *Infixe et tri par ABR*

1. et 3.

```
    def infixe(self):
        res = []
        def parcours(n):
            if n is not None:
                parcours(n.gauche)
                res.append(n.valeur)
                parcours(n.droite)
        parcours(self._racine)
        return res

def tri_abr(tab):
    arbre = ABR()
    for x in tab:
        arbre.inserer(x)
    return arbre.infixe()
```

Autre méthode : sans fonction intérieure, une fonction récursive qui **renvoie** la liste infixe d'un sous-arbre : celle de gauche, puis la racine, puis celle de droite, recollées par concaténation.

```
def infixe_noeud(n):
    if n is None:
        return []
    return infixe_noeud(n.gauche) + [n.valeur] + infixe_noeud(n.droite)

    def infixe(self):                       # dans la classe ABR
        return infixe_noeud(self._racine)
```

2. `[2, 5, 8, 10, 12, 15, 18, 20, 25]`. En chaque nœud, le parcours infixe écrit d'abord tout le sous-arbre gauche (valeurs plus petites), puis le nœud, puis tout le sous-arbre droit (valeurs plus grandes) ; comme c'est vrai à tous les niveaux, la liste obtenue est croissante.

3. Les doublons disparaissent, car `inserer` les refuse : `tri_abr` renvoie `sorted(set(tab))`.

4. Les 200 comparaisons avec `sorted(set(t))` sont toutes égales.

```
for essai in range(200):
    t = [random.randint(0, 50) for i in range(random.randint(0, 30))]
    assert tri_abr(t) == sorted(set(t))
```

4 Mesurer la hauteur

Exercice 6 *Au hasard ou dans l'ordre*

1.

```
def hauteur_triee(n):
    arbre = ABR()
    for v in range(n):
        arbre.inserer(v)
    return arbre.hauteur()

def hauteur_aleatoire(n, essais=20):
    total = 0
    for k in range(essais):
        valeurs = [i for i in range(n)]
        random.shuffle(valeurs)
        arbre = ABR()
        for v in valeurs:
            arbre.inserer(v)
```

```
        total = total + arbre.hauteur()
    return total / essais
```

2. Mesures obtenues (`random.seed(2026)` exécuté une seule fois, puis `hauteur_aleatoire(n)` appelée pour $n = 10$, 100, 1 000, 10 000 dans cet ordre ; sans graine, les moyennes au hasard varient d'une ou deux unités) :

n	10	100	1 000	10 000
$\log_2(n)$	3,3	6,6	10,0	13,3
hauteur moyenne, au hasard	5,45	12,8	22,3	31,05
hauteur, valeurs triées	10	100	1 000	10 000

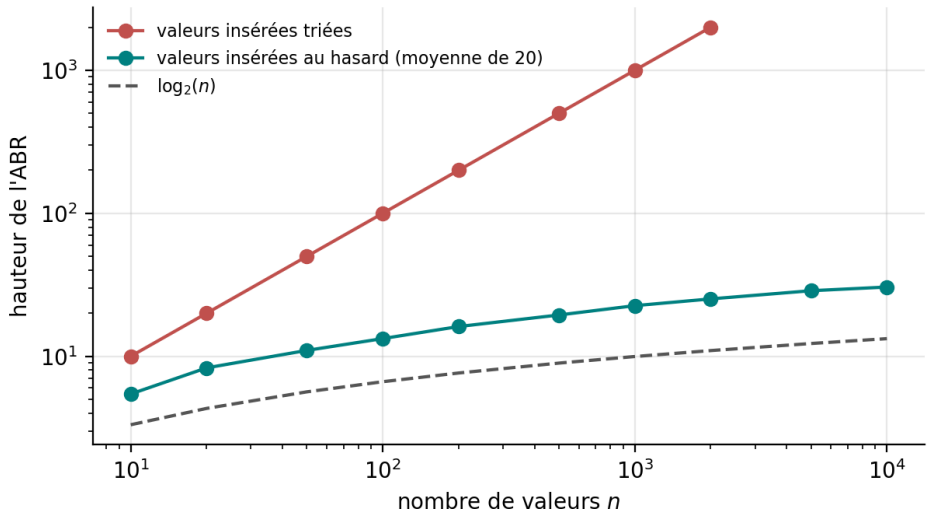
3. Au hasard, la hauteur vaut environ **2 à 2,3 fois** $\log_2(n)$ (un peu moins pour les petites valeurs de n) : elle n'augmente que de 7 à 10 environ quand n est multiplié par 10 (croissance logarithmique). Avec des valeurs triées, la hauteur vaut n : elle est multipliée par 10.

4.

```
import matplotlib.pyplot as plt

def log2(n):
    """Nombre de divisions par 2 pour passer de n a 1 ou moins (log2(n) arrondi au-dessus)"""
    k = 0
    while n > 1:
        n = n / 2
        k = k + 1
    return k

ns = [10, 20, 50, 100, 200, 500, 1000, 2000, 5000, 10000]
nt = [n for n in ns if n <= 2000]
plt.plot(nt, [hauteur_triee(n) for n in nt], "o-", label="valeurs triees")
plt.plot(ns, [hauteur_aleatoire(n) for n in ns], "o-", label="au hasard")
plt.plot(ns, [log2(n) for n in ns], "--", label="log2(n)")
plt.xscale("log")
plt.yscale("log")
plt.xlabel("n")
plt.ylabel("hauteur")
plt.legend()
plt.show()
```



En échelles logarithmiques, la droite rouge (pente 1) traduit une hauteur proportionnelle à n ; la courbe verte reste parallèle à $\log_2(n)$, environ deux fois au-dessus.

5. `hauteur` est récursive : sur un peigne de 1 000 nœuds ou plus, elle s'appelle elle-même 1 000 fois en cascade, ce qui dépasse la limite par défaut de Python (environ 1 000 appels imbriqués) : `RecursionError`. L'insertion, écrite avec une boucle, n'a pas ce problème.

Exercice 7 *Ce que cela coûte vraiment*

```
random.seed(2026)
valeurs = [i for i in range(1000)]
random.shuffle(valeurs)
ar = ABR()
at = ABR()
for v in valeurs:
    ar.inserer(v)
for v in range(1000):
    at.inserer(v)
for arbre in [ar, at]:
    visites = [arbre.contient(v)[1] for v in range(1000)]
    total = 0
    pire = 0
    for nb in visites:
        total += nb
        if nb > pire:
            pire = nb
    print(total / 1000, pire)
```

Arbre au hasard : **11,8** visites en moyenne, **22** au pire (sa hauteur). Arbre trié : **500,5** en moyenne, **1000** au pire — exactement comme une recherche séquentielle dans une liste non triée (500,5 et 1 000). La dichotomie dans une liste triée fait au plus 10 comparaisons. Un ABR n'est intéressant que s'il reste **peu haut** ; il a alors l'avantage sur la liste triée d'accepter des insertions rapides (pas de décalage).

5 Application : l'index d'un texte

Exercice 8 *Construire et interroger l'index*

1. et 2.

```
    def ajouter(self, mot, ligne):
        if self._racine is None:
            self._racine = NoeudIndex(mot)
            self._nb_mots = 1
        n = self._racine
        while n.valeur != mot:
            if mot < n.valeur:
                if n.gauche is None:
                    n.gauche = NoeudIndex(mot)
                    self._nb_mots = self._nb_mots + 1
                n = n.gauche
            else:
                if n.droite is None:
                    n.droite = NoeudIndex(mot)
                    self._nb_mots = self._nb_mots + 1
                n = n.droite
        if n.lignes == [] or n.lignes[-1] != ligne:
            n.lignes.append(ligne)

    def lignes(self, mot):
        n = self._racine
        while n is not None:
            if mot == n.valeur:
```

```
                return n.lignes
            if mot < n.valeur:
                n = n.gauche
            else:
                n = n.droite
        return []

    def afficher(self):
        def parcours(n):
            if n is not None:
                parcours(n.gauche)
                texte = str(n.lignes[0])
                for i in range(1, len(n.lignes)):
                    texte = texte + ", " + str(n.lignes[i])
                print(n.valeur, ":", texte)
                parcours(n.droite)
        parcours(self._racine)
```

3. L'index contient **77** mots différents ; sa hauteur est **15**, pour $\log_2(77) \approx 6{,}3$. Les mots d'un texte n'arrivent ni triés ni vraiment au hasard : l'arbre est plus haut que le minimum possible (7), mais très loin du peigne (77).

4. Ils apparaissent **à la fin**, après `valeurs` : `à`, puis `égale`, `élimine`, `équilibré`, `équilibrés`. Python compare les chaînes caractère par caractère selon leur code : `ord("é")` vaut 233 et `ord("à")` 224, bien plus que `ord("z")` = 122. Pour un vrai ordre alphabétique, on peut ranger les nœuds selon une **clé sans accents** (fonction qui remplace `é`, `è`, `ê` par `e`, `à` par `a`, etc.) tout en affichant le mot d'origine.

Exercice 9 — *Défi — toutes les valeurs d'un intervalle*

```
    def entre(self, a, b):
        res = []
        def parcours(n):                    # renvoie le nombre de noeuds visites
            if n is None:
                return 0
            visites = 1
            if a < n.valeur:                # a gauche, il peut y avoir des valeurs >= a
                visites = visites + parcours(n.gauche)
            if a <= n.valeur <= b:
                res.append(n.valeur)
            if n.valeur < b:                # a droite, il peut y avoir des valeurs <= b
                visites = visites + parcours(n.droite)
            return visites
        nb = parcours(self._racine)
        return res, nb
```

1. Sur l'arbre de l'exercice 1 : `([10, 12, 15, 18], 6)` — les nœuds `5`, `2` et `25` ne sont jamais visités.

2. `entre(500, 509)` visite **24** nœuds dans `ar`, contre **510** dans `at` (il faut descendre le peigne jusqu'à `509`).

3. Un mot commence par `prefixe` s'il est compris entre `prefixe` et les mots qui le prolongent ; on élague donc de la même façon :

```
    def commencant_par(self, prefixe):
        res = []
        def parcours(n):
            if n is None:
                return
            if prefixe < n.valeur:
                parcours(n.gauche)
            commence = n.valeur[:len(prefixe)] == prefixe   # memes premieres lettres ?
            if commence:
                res.append(n.valeur)
```

```
            if n.valeur < prefixe or commence:
                parcours(n.droite)
        parcours(self._racine)
        return res
```

`commencant_par("arb")` renvoie `['arbre', 'arbres']` ; `commencant_par("rech")` renvoie `['recherche']`.

6 Bilan à rédiger

Exercice 10 *Ce que j'ai mesuré*

Éléments attendus :

- Le coût d'une recherche est au plus la **hauteur** ; il dépend donc de la **forme** de l'arbre, c'est-à-dire de l'ordre d'insertion. Pour $n = 1\,000$: hauteur ≈ 22 et ≈ 12 visites en moyenne au hasard ; hauteur 1 000 et 500 visites en moyenne avec des valeurs triées.
- Valeurs triées en pratique : numéros de dossiers ou d'élèves attribués dans l'ordre, dates d'inscription, identifiants croissants d'une base. Les bases de données utilisent des arbres **équilibrés** (B-arbres) qui se réorganisent pour garder une hauteur petite.
- L'index utilise l'**ordre** de l'ABR : la recherche (`lignes`, `ajouter`) descend d'un seul côté à chaque nœud ; l'affichage (`afficher`) repose sur le fait que le parcours infixe donne les clés triées ; `commencant_par` élague grâce à l'ordre.

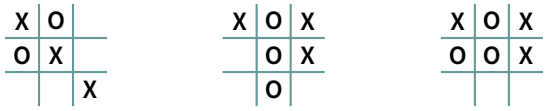
TP 2 Le morpion imbattable

↑ énoncé p. 259

On donne **une** version simple de chaque fonction ; d'autres écritures sont possibles. Avec ces solutions, les sept tests du fichier `tp_morpion_minimax_depart.py` affichent `[OK]`. Les temps de calcul indiqués sont des ordres de grandeur : ils dépendent de la machine.

1 Représenter le jeu

Exercice 1 *Lire une grille*



1. et 2. Grille `"XO.OX...X"` : `X` a gagné grâce à la diagonale `(0, 4, 8)`. Grille `"XOX.OX.O."` : `O` a gagné grâce à la colonne du milieu `(1, 4, 7)`. Grille `"XOXOOX..."` : personne n'a gagné.

3. `X` commence, puis les joueurs alternent. Après chaque paire de coups (un `X` puis un `O`), il y a autant de `X` que de `O` : c'est alors à `X` de jouer. Après un coup de `X` non suivi de la réponse de `O`, il y a un `X` de plus : c'est à `O` de jouer. Dans la troisième grille, il y a trois `X` et trois `O` : c'est à `X` de jouer.

4. Lignes `(0, 1, 2)`, `(3, 4, 5)`, `(6, 7, 8)` ; colonnes `(0, 3, 6)`, `(1, 4, 7)`, `(2, 5, 8)` ; diagonales `(0, 4, 8)` et `(2, 4, 6)`.

Exercice 2 *Les règles en Python*

```
def coups_possibles(grille):
    return [i for i in range(9) if grille[i] == VIDE]


def gagnant(grille):
    for a, b, c in LIGNES:
        if grille[a] != VIDE and grille[a] == grille[b] and grille[b] == grille[c]:
            return grille[a]
    return None


def est_finie(grille):
    return gagnant(grille) is not None or coups_possibles(grille) == []
```

Le test `grille[a] != VIDE` est indispensable : sans lui, trois cases vides alignées feraient de `"."` un « gagnant ».

Exercice 3 *Jouer sans abîmer la grille*

1. La console affiche `['X', 'O', 'X', 'O', 'O', 'X', '.', '.', 'X']` : la grille `g` a été modifiée. L'affectation `nouvelle = grille` ne copie pas la liste ; elle donne un **deuxième nom** à la **même** liste. Modifier `nouvelle[case]`, c'est donc modifier `grille`, et `h` et `g` désignent la même liste.

2.

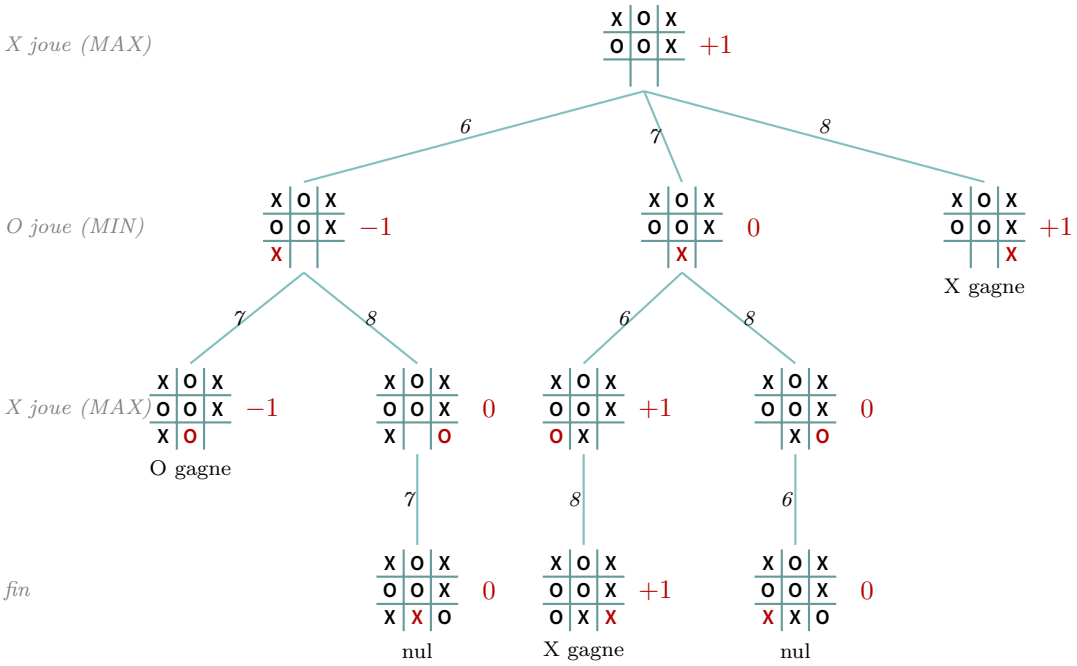
```
def jouer(grille, case, joueur):
    nouvelle = [c for c in grille]   # une vraie copie
    nouvelle[case] = joueur
    return nouvelle
```

3. Avec `jouer_faux`, quand on essaie le coup `6` puis le coup `7` depuis la même position, le symbole posé en `6` serait encore présent lors de l'essai suivant : chaque essai partirait d'une grille déjà modifiée par les précédents, et l'arbre exploré serait faux.

2 L'arbre de jeu

Exercice 4 *Un petit arbre de jeu, à la main*

1. Arbre de jeu de la grille `"XOXOOX..."` (le dernier coup joué est en couleur ; le numéro de la case jouée est sur chaque arête). Les valeurs, en gras à droite des grilles, sont celles de l'exercice 6.



2. Taille : 11 nœuds. Hauteur : 4 (par exemple le chemin racine → `6` → `8` → `7` compte 4 nœuds). Feuilles : 5, donc 5 parties différentes possibles.

3. Voir la figure : `X` gagne dans deux parties (coup `8` immédiat ; `7`, `6`, `8`), `O` gagne dans une (`6` puis `7` : colonne du milieu), deux parties sont nulles.

4. Non : une feuille est une partie **finie**, ce qui peut arriver avant que la grille soit pleine. Ici, la feuille « `X` joue `8` » est à la profondeur 1 et la feuille « `6` puis `7` » à la profondeur 2, alors que la profondeur maximale est 3.

Exercice 5 *L'arbre complet du morpion*

1. La racine a 9 fils (9 cases libres), chacun d'eux a 8 fils, etc. Il y a au plus 9 coups dans une partie : la hauteur de l'arbre est $1 + 9 = 10$ (on compte les nœuds).

2. Il y aurait $9 \times 8 \times 7 \times \cdots \times 1 = 9! = 362\,880$ parties.

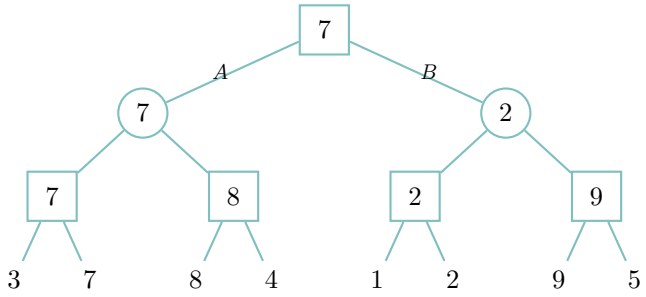
3. Le niveau k contiendrait $9 \times 8 \times \cdots \times (9 - k + 1)$ nœuds, d'où $1 + 9 + 72 + 504 + 3\,024 + 15\,120 + 60\,480 + 181\,440 + 362\,880 + 362\,880 = 986\,410$ nœuds.

4. Les vrais nombres sont plus **petits**, car beaucoup de parties s'arrêtent avant que la grille soit pleine (une victoire est possible dès le 5e coup) : leurs nœuds n'ont pas de descendants. On trouvera 549 946 nœuds et 255 168 parties à l'exercice 8.

3 L'algorithme minimax

Exercice 6 *Remonter les valeurs*

1.



2. MAX joue `A` et il est sûr d'obtenir 7.

3. Non. L'optimiste choisirait `B` (qui contient la feuille 9) ; mais MIN répondrait par le premier fils de `B`, et MAX n'obtiendrait que 2. Dans l'encadré, de même, l'optimiste choisirait `B` (feuille 11) et n'obtiendrait que 2 au lieu de 6. C'est tout l'intérêt de minimax : raisonner sur ce que l'adversaire **laissera** obtenir.

4. Voir les valeurs sur la figure de l'exercice 4 : le fils « `6` » vaut $\min(-1, 0) = -1$, le fils « `7` » vaut $\min(+1, 0) = 0$, le fils « `8` » est une victoire, $+1$. La racine vaut $\max(-1, 0, +1) = +1$: `X` joue `8` et gagne.

Exercice 7 *Programmer minimax*

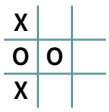
1. et 2.

```
def score_final(grille):
    g = gagnant(grille)
    if g == "X":
        return 1
    if g == "O":
        return -1
    return 0


def minimax(grille, joueur):
    if est_finie(grille):                    # condition d'arret
        return score_final(grille)
    if joueur == "X":
        valeur = -100                        # "moins l'infini"
    else:
        valeur = 100                         # "plus l'infini"
    for c in coups_possibles(grille):
        s = minimax(jouer(grille, c, joueur), adversaire(joueur))
        if joueur == "X":
            valeur = max(valeur, s)
        else:
            valeur = min(valeur, s)
    return valeur
```

La condition d'arrêt (le cas de base) : la partie est finie, c'est-à-dire que l'on est sur une **feuille** de l'arbre de jeu. Elle est toujours atteinte, car chaque appel récursif se fait sur une grille qui a une case vide de moins. Le maximum (ou le minimum) des valeurs des fils se calcule au fur et à mesure, comme une recherche de maximum dans une liste : on part d'une valeur qu'aucun score n'atteint (-100 pour `X`, 100 pour `O`) et on la remplace par `max(valeur, s)` (ou `min(valeur, s)`) à chaque fils.

3. On retrouve la valeur $+1$ de l'exercice 6. Dans la grille `"X..OO.X.."`, `O` menace d'aligner `(3, 4, 5)` : `X` doit jouer `5`, et n'a pas de double menace à créer ; `O` peut ensuite parer toutes les menaces de `X`. Le mieux que `X` puisse garantir est le nul.



4. `minimax(grille_vide(), "X")` vaut 0 : si les deux joueurs jouent parfaitement, le morpion se termine toujours par un **match nul**. Le calcul prend de l'ordre d'une à deux secondes sur un ordinateur, nettement plus dans un éditeur en ligne.

5. `minimax` explore entièrement le sous-arbre du premier coup avant de passer au deuxième : c'est un **parcours en profondeur**. La valeur d'un nœud n'est connue qu'**après** celles de tous ses fils : c'est l'ordre **suffixe**, comme pour le calcul de la taille ou de la hauteur d'un arbre.

Exercice 8 *Compter les positions*

1. et 2.

```python
def minimax_compte(grille, joueur):
    if est_finie(grille):
        return score_final(grille), 1
    if joueur == "X":
        valeur = -100
    else:
        valeur = 100
    visites = 1                                  # la position courante
    for c in coups_possibles(grille):
        s, v = minimax_compte(jouer(grille, c, joueur), adversaire(joueur))
        if joueur == "X":
            valeur = max(valeur, s)
        else:
            valeur = min(valeur, s)
        visites += v
    return valeur, visites


def nb_parties(grille, joueur):
    if est_finie(grille):
        return 1
    total = 0
    for c in coups_possibles(grille):
        total += nb_parties(jouer(grille, c, joueur), adversaire(joueur))
    return total
```

Sur la grille de l'exercice 4 : `minimax_compte` renvoie `(1, 11)` (on retrouve la taille 11) et `nb_parties` renvoie `5`.

3. Grille vide : `(0, 549946)` et `255168`. C'est bien moins que les 986 410 nœuds et 362 880 parties de l'exercice 5, à cause des parties terminées avant que la grille soit pleine.

4. Une même grille peut être atteinte par plusieurs suites de coups : par exemple `X` en `0`, `O` en `4`, `X` en `8`, et `X` en `8`, `O` en `4`, `X` en `0` donnent la même grille. L'arbre de jeu contient un nœud par **suite de coups**, donc la même position y apparaît plusieurs fois, et `minimax` recalcule sa valeur à chaque fois : 549 946 nœuds pour seulement 5 478 grilles distinctes.

Exercice 9 *Choisir son coup, puis jouer*

1.

```python
def meilleur_coup(grille, joueur):
    coups = coups_possibles(grille)
    meilleur = coups[0]
    meilleur_score = minimax(jouer(grille, meilleur, joueur), adversaire(joueur))
    for c in coups:
        s = minimax(jouer(grille, c, joueur), adversaire(joueur))
        if (joueur == "X" and s > meilleur_score) or (joueur == "O" and s < meilleur_score):
            meilleur = c
            meilleur_score = s
```

```
    return meilleur
```

Les comparaisons sont **strictes** : en cas d'égalité, on garde la première case trouvée.

2. On ne peut pas battre la machine : au mieux, la partie est nulle. Comme la valeur de la grille vide est 0, la machine sait toujours garantir au moins le nul ; et dès que l'humain commet une erreur qui lui donne un coup gagnant, elle le trouve.

3. Pour son premier coup sur la grille vide, la machine explore tout l'arbre de jeu (environ 550 000 positions, et même un peu plus, car `meilleur_coup` recalcule la valeur du premier coup). Après deux coups, il ne reste que 7 cases vides : le sous-arbre à explorer compte au plus 13 700 nœuds, et la réponse est quasi immédiate.

4 Gagner vite, perdre tard

Exercice 10 *Une machine qui ne se presse pas*

1. Le coup `6` gagne immédiatement (colonne `(0, 3, 6)`). Pourtant, `meilleur_coup` renvoie `4`. En effet, tous les coups possibles (`4`, `5`, `6`, `7`, `8`) ont la valeur $+1$: `X` gagne de toute façon, car il menace déjà `6`. Toutes les valeurs étant égales, la machine garde la première case, `4`. Ce n'est pas une erreur au sens du score (après `4`, `X` menace à la fois `5`, `6` et `8`, et gagne au coup suivant), mais la partie dure inutilement plus longtemps.

2. `O` menace la colonne `(2, 5, 8)`. Si `X` pare en `8`, `O` joue `4` et menace à la fois `3` (ligne du milieu) et `6` (diagonale) : `X` ne peut parer qu'une menace et perd. Tous les coups de `X` valent donc -1, et la machine joue la première case, `3`, sans même bloquer : `O` gagne tout de suite en `8`. Face à un humain qui pourrait ne pas voir la double menace, il fallait au moins parer en `8`.

3.

```
def score_final(grille):
    g = gagnant(grille)
    vides = len(coups_possibles(grille))
    if g == "X":
        return 10 + vides
    if g == "O":
        return -(10 + vides)
    return 0
```

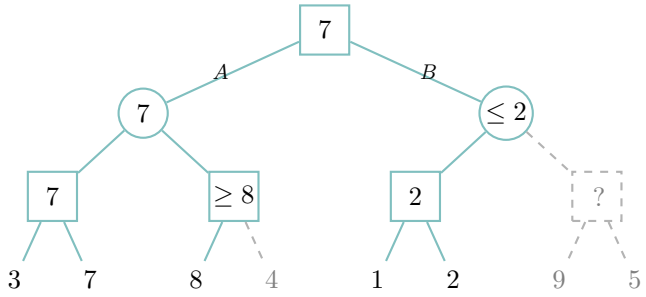
Une victoire avec beaucoup de cases encore vides est une victoire **rapide** ; elle vaut maintenant plus. Sur la grille de gauche, le coup `6` vaut 14 (4 cases vides restent), contre 12 ou moins pour les autres : la machine gagne tout de suite. Sur la grille de droite, parer en `8` vaut -11 (la défaite arrive plus tard) contre -13 pour les autres coups : la machine bloque et oblige l'adversaire à trouver la double menace.

4. Oui, `minimax(grille_vide(), "X")` vaut toujours 0. Le signe garde son sens : positif, `X` gagne s'il joue bien ; négatif, `O` gagne ; nul, match nul. Seule la valeur absolue a changé : elle indique en plus la rapidité de la victoire.

5 Pour aller plus loin

Exercice 11 *Défi — l'élagage alpha-bêta*

1. Feuilles non examinées (en pointillés) : la feuille 4, car le deuxième fils de `A` vaut déjà au moins 8, plus que les 7 que MIN a déjà sous `A` ; et tout le second fils de `B` (feuilles 9 et 5), car `B` vaut déjà au plus 2, moins que les 7 que MAX est sûr d'obtenir avec `A`.



2.

```python
def alphabeta(grille, joueur, alpha, beta):
    if est_finie(grille):
        return score_final(grille), 1
    coups = coups_possibles(grille)
    visites = 1
    i = 0
    if joueur == "X":
        valeur = -100
        while i < len(coups) and alpha < beta:
            s, v = alphabeta(jouer(grille, coups[i], joueur), "O", alpha, beta)
            visites += v
            valeur = max(valeur, s)
            alpha = max(alpha, valeur)
            i += 1
    else:
        valeur = 100
        while i < len(coups) and alpha < beta:
            s, v = alphabeta(jouer(grille, coups[i], joueur), "X", alpha, beta)
            visites += v
            valeur = min(valeur, s)
            beta = min(beta, valeur)
            i += 1
    return valeur, visites
```

Les bornes -100 et 100 jouent le rôle de « moins l'infini » et « plus l'infini » : aucun score ne les atteint.

3. Sur la grille vide : 18 297 positions examinées au lieu de 549 946, soit environ 30 fois moins, pour le **même** score 0. (Avec les scores de la partie 4, on obtient 20 866 positions.)

4. Avec l'ordre centre, coins, bords : 7 275 positions seulement (7 865 avec les scores de la partie 4). Les meilleurs coups sont souvent au centre et dans les coins ; en les essayant d'abord, `alpha` et `beta` se resserrent très tôt, et beaucoup plus de branches sont élaguées ensuite.

Exercice 12 *Défi — quand l'arbre est trop grand*

1.

```
def evaluer(grille):
    ouvertes_x = len([ligne for ligne in LIGNES if "O" not in [grille[k] for k in ligne]])
    ouvertes_o = len([ligne for ligne in LIGNES if "X" not in [grille[k] for k in ligne]])
    return ouvertes_x - ouvertes_o
```

Après un premier `X` : au centre, $8-4=4$ (le centre est sur 4 alignements) ; dans un coin, $8-5=3$; sur un bord, $8-6=2$.

2. L'évaluation n'est qu'une estimation ; une victoire est une certitude. Si une estimation pouvait dépasser le score d'une victoire, la machine pourrait préférer une position « prometteuse » à une victoire immédiate. Ici `evaluer` est comprise entre -8 et 8, et une victoire vaut au moins 10 : c'est bien le cas.

3.

```
def minimax_limite(grille, joueur, profondeur):
    if est_finie(grille):
        return score_final(grille)
    if profondeur == 0:                          # on s'arrete : on estime
        return evaluer(grille)
    if joueur == "X":
        valeur = -100
    else:
        valeur = 100
    for c in coups_possibles(grille):
        s = minimax_limite(jouer(grille, c, joueur), adversaire(joueur), profondeur - 1)
        if joueur == "X":
            valeur = max(valeur, s)
        else:
            valeur = min(valeur, s)
    return valeur


def meilleur_coup_limite(grille, joueur, profondeur):
    coups = coups_possibles(grille)
    meilleur = coups[0]
    meilleur_score = minimax_limite(jouer(grille, meilleur, joueur), adversaire(joueur),
    profondeur - 1)
    for c in coups:
        s = minimax_limite(jouer(grille, c, joueur), adversaire(joueur), profondeur - 1)
        if (joueur == "X" and s > meilleur_score) or (joueur == "O" and s < meilleur_score):
            meilleur = c
            meilleur_score = s
    return meilleur
```

Il y a désormais **deux** conditions d'arrêt : la partie est finie (score exact), ou la profondeur limite est atteinte (estimation).

4. Avec la profondeur 1, la machine ne regarde que son propre coup : elle ne voit pas les menaces. Par exemple : `X` en `0`, la machine en `4` ; `X` en `2` (menace `1`) ; la machine joue `6` et `X` gagne en `1`.

Avec la profondeur 2, elle voit les menaces directes mais pas les doubles menaces : `X` en `0`, machine en `4` ; `X` en `7`, machine en `2` ; `X` en `6` menace à la fois `3` et `8` ; la machine, qui se voit perdue quoi qu'elle fasse, joue la première case libre, `1`, et `X` gagne en `3`.

Une recherche exhaustive (on essaie toutes les parties possibles de `X` contre la machine) montre que la machine, quand elle joue `O`, peut encore être battue jusqu'à la profondeur 5, et devient imbattable à partir de la profondeur 6.

6 Bilan à rédiger

Exercice 13 *Ce que fait vraiment la machine*

Éléments attendus :

- L'arbre de jeu a pour racine la position courante ; chaque nœud est une position, chaque arête un coup, chaque feuille une partie finie. Pour le morpion, depuis la grille vide, sa hauteur est 10 ; il compte 549 946 nœuds et 255 168 feuilles.
- Minimax : sur une feuille (condition d'arrêt), on renvoie le score de la partie ; si c'est à MAX de jouer, on renvoie le maximum des valeurs des fils ; si c'est à MIN, leur minimum. L'algorithme est récursif parce que la valeur d'un nœud se définit à partir des valeurs de ses fils, qui sont des sous-arbres de même nature.
- La machine ne « réfléchit » pas : elle **énumère** toutes les suites possibles et applique une règle de calcul. Aux échecs, l'arbre est beaucoup trop grand pour être exploré en entier ; on combine alors l'élagage **alpha-bêta** (même résultat, beaucoup moins de positions) et une **profondeur limitée** avec une **fonction d'évaluation** (résultat approché).

Pour aller plus loin : 549 946 nœuds, mais combien de positions différentes ?

Dans l'arbre de jeu, une même grille apparaît très souvent : `X` en `0`, `O` en `4`, `X` en `8` donne la même position que `X` en `8`, `O` en `4`, `X` en `0`. Pour compter les positions **différentes**, on parcourt l'arbre comme `minimax`, en notant chaque grille rencontrée dans un dictionnaire dont les clés sont les grilles écrites sous forme de chaînes :

```
def en_texte(grille):
    """Renvoie la grille sous forme d'une chaine de 9 caracteres"""
    texte = ""
    for c in grille:
        texte = texte + c
    return texte


def noter_positions(grille, joueur, vues):
    """Note dans vues les positions de l'arbre de jeu"""
    vues[en_texte(grille)] = True      # cle deja presente : pas de doublon
    if not est_finie(grille):
        for c in coups_possibles(grille):
            noter_positions(jouer(grille, c, joueur), adversaire(joueur), vues)


vues = {}
noter_positions([VIDE] * 9, "X", vues)
print(len(vues))                       # 5478
```

L'arbre a 549 946 nœuds, mais il n'y a que 5 478 positions différentes (dont 958 parties finies) : `minimax` recalcule donc chaque position une centaine de fois en moyenne. Ranger la valeur de chaque position déjà calculée dans un dictionnaire, pour ne jamais la recalculer : ce sera l'idée centrale du chapitre Programmation dynamique.

Un peu d'histoire : un morpion sur l'EDSAC (1952)

En 1952, à l'université de Cambridge, **Alexander « Sandy » Douglas** (1921–2010) programme le morpion sur l'**EDSAC**, l'un des tout premiers ordinateurs à programme enregistré, dans le cadre de sa thèse sur les échanges entre l'homme et la machine. La grille s'affiche sur l'un des petits écrans cathodiques de la machine, prévus à l'origine pour montrer le contenu de la mémoire, et le joueur indique sa case avec un cadran de téléphone ; l'ordinateur joue sans jamais perdre. Avec le jeu de dames de Christopher Strachey, ce programme est l'un des premiers jeux affichés sur un écran électronique. Douglas ne lui avait pas donné de nom : on l'appelle aujourd'hui *OXO*, nom choisi par l'historien Martin Campbell-Kelly quand il a reconstitué l'EDSAC en simulation. Soixante-dix ans plus tard, ce TP fait la même chose en une trentaine de lignes.

Sources : article « OXO (video game) » de Wikipédia (en anglais) ; simulateur EDSAC de M. Campbell-Kelly (université de Warwick).

Réponses des questions de la fiche de révision

↑ questions p. 268

Se tester

1. un nœud sans fils ; le nombre de ses nœuds (§ II et III).
2. `2, 5, 6, 8` (c'est un ABR : valeurs triées) (§ V.A et VII.A).
3. n dans le pire cas ; environ $\log_2 n$ s'il est équilibré (§ VII).
4. erreur `AttributeError` : on lit `.gauche` sur `None` (§ VI).
5. une file (parcours en largeur) (§ V.B).

Et avant ?

1. élu, prêt, bloqué ; on ne passe jamais directement de bloqué à élu (ch. 4, § III).
2. sa valeur de retour ne dépend que de ses paramètres, et elle ne produit aucun effet de bord (ch. 2, § II).
3. non : avant, à la descente (ordre des appels) ; après, à la remontée (ordre inverse) (ch. 1, § II).

CORRIGÉS DU CHAPITRE 6

6 Bases de données et SQL

CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Le grand tableau du secrétariat

↑ énoncé p. 280

Exercice 1 *Le tableau tout-en-un*

1. « M. Rossi » est écrit 4 fois (lignes 1, 2, 4, 5 — dont une fois mal orthographié) ; la salle de la TNSI1, 4 fois aussi (trois fois B12, une fois B21). L'information « la TNSI1 a M. Rossi en salle B12 » est **recopiée** sur chaque ligne d'élève de la classe : c'est la **redondance**.

2. Anomalies (on en attend au moins cinq) :

- **fautes de frappe** : « M. Rosi » (ligne 4), « Bianci » (ligne 5) ;
- **information contradictoire** : salle B21 pour la TNSI1 (ligne 5) alors que les autres lignes disent B12 ;
- **valeur impossible** : la note 23 (ligne 6) ;
- **même chose écrite de deux façons** : « T NSI 2 » au lieu de « TNSI2 », la date « 2026-09-15 » au lieu de « 15/09 » (ligne 7) ;
- **valeur d'une autre nature** : « abs » dans une colonne de nombres (ligne 7) — une absence n'est pas une note.

3. Lignes 1 et 3 : deux classes différentes, donc *probablement* deux élèves homonymes… mais rien ne le garantit (Léa a pu changer de classe). Lignes 1 et 4 : même classe, sans doute la même élève, mais là encore on **devine**. Pour en être sûr, il faudrait un **numéro unique** par élève.

4. 4 cases (lignes 1, 2, 4, 5). Si l'on en oublie une, le tableau donne deux salles différentes pour la même classe : il devient **incohérent**. (Le nom savant, que le cours donnera : *anomalie de mise à jour*.)

5. On ne peut l'inscrire qu'en laissant vides les cases Matière, Note et Date, ce qui n'a pas de sens pour une ligne « de note » — ou en inventant une fausse note. Le tableau mélange deux choses : « qui est l'élève » et « quelle note a-t-elle eue ». (C'est une *anomalie d'insertion*.)

6. En effaçant sa seule note, on efface aussi **l'existence** d'Inès Silva et le fait qu'elle est en TNSI2 : le secrétariat ne sait plus qu'elle est inscrite. (C'est une *anomalie de suppression*.)

Exercice 2 *Deux secrétaires, un seul fichier*

7. Le fichier contient la version de M. Giraud : « Bianchi » est corrigé, mais la note de Zoé **a disparu**. Chacun a travaillé sur sa propre copie du fichier, ouverte avant que l'autre n'enregistre ; le dernier qui enregistre écrase le travail de l'autre.

8. Exemples de règles : bloquer le fichier (ou seulement la ligne) pendant qu'une personne le modifie ; enregistrer modification par modification au lieu du fichier entier ; prévenir la seconde personne qu'une autre version a été enregistrée entre-temps. C'est ce que fait un SGBD (gestion des **accès concurrents**, transactions).

Exercice 3 *Découper le tableau*

9. Un découpage possible (les anomalies sont corrigées ; la note 23 est à vérifier auprès du professeur, l'absence n'est pas une note) :

CLASSE		
Code	**Prof. princ.**	**Salle**
TNSI1	M. Rossi	B12
TNSI2	Mme Costa	C04

ÉLÈVE			
Num.	**Nom**	**Prénom**	**Classe**
1	Martin	Léa	TNSI1
2	Bianchi	Hugo	TNSI1
3	Martin	Léa	TNSI2
4	Durand	Noé	TNSI2
5	Silva	Inès	TNSI2
6	Petit	Zoé	TNSI1

NOTE			
Élève	**Matière**	**Note**	**Date**
1	NSI	15	12/09
2	NSI	11	12/09
3	NSI	17	12/09
1	Maths	13	15/09
2	Maths	9	15/09
4	NSI	?	12/09
5	NSI	14	12/09

La ligne « abs » de Noé disparaît des notes (ou va dans un futur tableau ABSENCE).

10. Le **numéro** de l'élève. Dans NOTE, on écrit ce numéro et non le nom : le nom n'est pas unique (deux Léa Martin) et peut être mal orthographié ; le numéro désigne **une seule** ligne d'ÉLÈVE, où le nom n'est écrit qu'une fois.

11. Déménagement : **une seule** case à changer (dans CLASSE). Zoé : une ligne dans ÉLÈVE, aucune dans NOTE. Supprimer la note d'Inès : elle reste dans ÉLÈVE. Les trois problèmes disparaissent parce que chaque information n'est écrite **qu'une fois**, dans le tableau qui la concerne.

12. Exemples : « une note est un nombre compris entre 0 et 20 » ; « la classe d'un élève doit exister dans CLASSE » ; « le numéro d'élève de NOTE doit exister dans ÉLÈVE » ; « deux élèves n'ont jamais le même numéro » ; « la date s'écrit toujours sous la même forme ».

Exercice 4 *Mettre des mots*

13. Propositions fréquentes, puis mots du cours :

Notion	Propositions fréquentes	Mot du cours
un des petits tableaux	tableau, feuille, fiche	**relation** (ou **table**)
une colonne / une ligne	champ / fiche	**attribut** / **enregistrement**
la colonne qui distingue chaque élève	numéro, identifiant, matricule	**clé primaire**
la colonne qui renvoie vers ÉLÈVE	lien, renvoi, pointeur	**clé étrangère**
« une note est entre 0 et 20 »	règle, condition	**contrainte** (d'intégrité)
le logiciel qui gère tout	gestionnaire, serveur	**SGBD**

14. … restent **cohérentes** ; … puissent les **modifier en même temps** sans se gêner (accès concurrents).

CORRIGÉS DES EXERCICES

Modèle relationnel

Exercice 1 *Lire un schéma* ↑énoncé p. 297

1. **4** relations ; `emprunt` a **4** attributs (`id`, `id_livre`, `id_adherent`, `date_emprunt`).
2. Clé primaire de `livre` : `id`. Clés étrangères de `emprunt` : `id_livre` (référence `livre.id`) et `id_adherent` (référence `adherent.id`).
3. `id_auteur` **relie** chaque livre à son auteur (il pointe vers `auteur.id`) : cela évite de recopier le nom de l'auteur et garantit la cohérence.
4. `annee` : `INTEGER` ; `date_emprunt` : `TEXT` (format `'AAAA-MM-JJ'` — SQLite n'a pas de type date natif).

Exercice 2 *Clés et contraintes* ↑énoncé p. 297

1. Deux livres peuvent porter le **même titre** (rééditions, œuvres homonymes) : `titre` ne garantirait pas l'unicité. Un `id` numérique la garantit simplement.
2. La **contrainte de référence** : `id_livre = 99` ne correspond à aucun `id` de `livre` ⇒ refus.
3. La clé **primaire** identifie de façon unique les lignes de *sa* table ; la clé **étrangère** pointe vers la clé primaire d'une *autre* table pour les relier (et peut, elle, se répéter).

Exercice 3 *Faire évoluer le modèle* ↑énoncé p. 297

On crée une relation `editeur(id, nom, ville)` et l'on ajoute dans `livre` une clé étrangère `#id_editeur` référençant `editeur.id`. Chaque éditeur n'est alors écrit **qu'une seule fois**.

Interroger : SELECT et WHERE

Exercice 4 *Sélections simples* ↑énoncé p. 298 **Démarche :** pour chaque requête, on choisit les colonnes à afficher (`SELECT`), la table (`FROM`), puis la condition que doivent vérifier les lignes (`WHERE`).

```sql
SELECT nom FROM pays WHERE continent = 'Europe';                -- 1.

SELECT nom, capitale FROM pays WHERE population > 100;          -- 2.

SELECT nom FROM pays WHERE continent = 'Asie' AND superficie > 1000; -- 3.
```

▷ **Erreur fréquente.** Écrire `WHERE continent = Europe` sans apostrophes : le SGBD cherche alors une *colonne* nommée `Europe`. Les chaînes de caractères s'écrivent entre apostrophes.

Exercice 5 *Conditions composées* ↑énoncé p. 299 **Démarche :** chaque question combine deux conditions : `OR` si l'une suffit, `AND` si les deux sont exigées (« compris entre ») ; « pas en Europe » se traduit par `<>` (ou par `NOT`).

```sql
SELECT nom FROM pays                                            -- 1.
WHERE continent = 'Afrique' OR continent = 'Océanie';

SELECT capitale FROM pays                                       -- 2.
WHERE population >= 50 AND population <= 150;

SELECT nom FROM pays WHERE continent <> 'Europe';               -- 3.
```

▷ **Erreur fréquente.** Écrire `continent = 'Afrique' AND continent = 'Océanie'` : aucun pays n'est sur les deux continents à la fois, et le résultat est vide. « Afrique ou Océanie » se traduit par `OR`.

Trier, dédoublonner, rechercher un motif

Exercice 6 *Trier et dédoublonner* ↑énoncé p. 299 **Démarche :** `ORDER BY ... DESC` trie par ordre décroissant ; `DISTINCT`, placé juste après `SELECT`, supprime les doublons.

```sql
SELECT nom FROM pays ORDER BY population DESC;    -- 1.
SELECT DISTINCT continent FROM pays;              -- 2.
```

Exercice 7 `LIMIT` *et* `LIKE` ↑énoncé p. 299 **Démarche :** pour les trois plus vastes, on trie par superficie décroissante, puis `LIMIT` ne garde que les premières lignes. Dans un motif `LIKE`, `%` remplace n'importe quelle suite de caractères (même vide).

```sql
SELECT nom, superficie FROM pays                -- 1.
ORDER BY superficie DESC LIMIT 3;

SELECT nom FROM pays WHERE nom LIKE 'A%';        -- 2.

SELECT capitale FROM pays WHERE capitale LIKE '% %';  -- 3.
```

▷ **Erreur fréquente.** Écrire `LIKE 'A'` sans `%` : on ne trouve que les pays dont le nom est exactement « A ». Le `%` représente la suite du nom.

Agréger

Exercice 8 *Fonctions d'agrégation* ↑énoncé p. 299 **Démarche :** une fonction d'agrégation, placée dans le `SELECT`, résume toute une colonne en une seule valeur.

```sql
SELECT COUNT(*) FROM pays;          -- 1.
SELECT AVG(population) FROM pays;   -- 2.
SELECT MAX(superficie) FROM pays;   -- 3.
```

Exercice 9 *Agrégats et filtre* ↑énoncé p. 299 **Démarche :** le `WHERE` filtre d'abord les lignes, l'agrégat est calculé ensuite sur les lignes restantes ; plusieurs agrégats se séparent par des virgules.

```sql
SELECT SUM(population) FROM pays WHERE continent = 'Asie';  -- 1.
SELECT MIN(population), MAX(population) FROM pays;          -- 2.
```

▷ **Erreur fréquente.** Mettre l'agrégat dans le `WHERE` (`WHERE population = MAX(population)`) : le `WHERE` examine les lignes une par une, avant tout calcul ; les agrégats se placent dans le `SELECT`.

Exercice 10 *Agréger la base musique* ↑énoncé p. 299 **Démarche :** mêmes agrégats sur la table `chanson` ; après un `WHERE`, `COUNT(*)` compte les lignes qui vérifient la condition.

```sql
SELECT COUNT(*) FROM chanson WHERE annee = 2013;  -- 1.
SELECT AVG(duree) FROM chanson;                   -- 2.
SELECT SUM(streams) FROM chanson;                 -- 3.
```

Regrouper : `GROUP BY`

Exercice 11 *Regrouper par continent* ↑énoncé p. 300

```sql
SELECT continent, COUNT(*) FROM pays GROUP BY continent;   -- 1.

SELECT continent, SUM(population) FROM pays                -- 2.
GROUP BY continent
ORDER BY SUM(population) DESC;

SELECT continent FROM pays                                 -- 3.  (Europe)
GROUP BY continent
HAVING COUNT(*) >= 4;
```

Croiser des tables : les jointures

Exercice 12 *Jointure de deux tables* ↑énoncé p. 300 **Démarche :** la clé étrangère `livre.id_auteur` référence `auteur.id` : c'est la condition du `ON`. On filtre ensuite avec `WHERE` sur les attributs de `auteur`.

```sql
SELECT livre.titre, auteur.nom                  -- 1.
FROM livre
JOIN auteur ON livre.id_auteur = auteur.id;

SELECT livre.titre                              -- 2.
FROM livre
JOIN auteur ON livre.id_auteur = auteur.id
WHERE auteur.nom = 'Camus';

SELECT livre.titre                              -- 3.
FROM livre
JOIN auteur ON livre.id_auteur = auteur.id
WHERE auteur.pays = 'Royaume-Uni';
```

▷ **Erreur fréquente.** Joindre sur `livre.id = auteur.id` : on associe le livre numéro 1 à l'auteur numéro 1, sans rapport entre eux. La condition relie la clé étrangère (`livre.id_auteur`) à la clé primaire qu'elle référence.

Exercice 13 *Jointure de trois tables* ↑énoncé p. 300

```sql
SELECT adherent.nom, adherent.prenom, livre.titre     -- 1.
FROM emprunt
JOIN adherent ON emprunt.id_adherent = adherent.id
JOIN livre    ON emprunt.id_livre    = livre.id;

SELECT adherent.nom, COUNT(*)                         -- 2.
FROM emprunt
JOIN adherent ON emprunt.id_adherent = adherent.id
GROUP BY adherent.id;       -- par l'identifiant : deux homonymes restent distincts
```

Exercice 14 *Jointure et agrégat (musique)* ↑énoncé p. 300

```sql
SELECT artiste.nom, SUM(chanson.streams)
FROM chanson
JOIN artiste ON chanson.id_artiste = artiste.id
WHERE artiste.pays = 'Belgique'
GROUP BY artiste.id;
```

Modifier la base

Exercice 15 *Ajouter, modifier, supprimer* ↑énoncé p. 300 **Démarche :** une instruction par opération : `INSERT INTO ... VALUES`, `UPDATE ... SET ... WHERE`, `DELETE FROM ... WHERE` ; le `WHERE` désigne la ou les lignes concernées.

```sql
INSERT INTO auteur (id, nom, pays) VALUES (6, 'Hugo', 'France'); -- 1.

UPDATE livre SET genre = 'Dystopie' WHERE id = 1;               -- 2.

DELETE FROM emprunt WHERE id = 6;                               -- 3.
```

3. Sans le `WHERE`, `DELETE FROM emprunt;` effacerait **tous** les emprunts.

▷ **Erreur fréquente.** Écrire `VALUES (6, Hugo, France)` sans apostrophes autour des chaînes : le SGBD prend `Hugo` et `France` pour des noms de colonnes et refuse la requête.

Exercice 16 *Le virement bancaire : transactions et ACID* ↑énoncé p. 300 **1. Démarche :** un virement, c'est un débit et un crédit : deux `UPDATE`, chacun ciblant un compte par son `id`.

```sql
UPDATE compte SET solde = solde - 50 WHERE id = 1;
UPDATE compte SET solde = solde + 50 WHERE id = 2;
```

2. Le compte `1` a été débité (`50` €) mais le compte `2` n'a pas été crédité (toujours `30` €) : `50` € ont **disparu**. L'**atomicité** l'aurait évité : la transaction est faite entièrement ou pas du tout, donc la panne aurait tout annulé (Léa retrouve ses `100` €).

3. Après le `COMMIT` : compte `1` à `50` €, compte `2` à `80` €. La **durabilité** garantit qu'une transaction validée survit à la coupure (elle est écrite sur disque).

4. Le débit `UPDATE compte SET solde = solde - 80 WHERE id = 1;` échoue : le solde deviendrait $50 - 80 = -30$, ce que la contrainte `CHECK (solde >= 0)` interdit. Le crédit de `80` € sur le compte `2`, lui, réussit. Attention : le SGBD (SQLite, par exemple) n'annule que l'**instruction fautive** ; la transaction reste ouverte avec le crédit déjà fait. Si l'on exécutait alors `COMMIT`, le compte `2` passerait à `160` € : `80` € créés de nulle part ! C'est donc à l'**application** de détecter l'erreur et d'exécuter `ROLLBACK;`, qui défait le crédit : les soldes restent `50` € et `80` €. Propriétés : **atomicité** (tout ou rien), qui préserve la **cohérence** (la base ne quitte jamais un état qui respecte les contraintes). *(Dans SQLite, le message d'erreur commence par « CHECK constraint failed ».)*

5. On attend $50 - 20 - 20 =$ **10** €.

Étape	Léa	Père
1	lit 50	
2		lit 50
3	écrit $50 - 20 = 30$	
4		écrit $50 - 20 = 30$

Le compte finit à 30 € : 40 € ont été retirés mais seulement 20 € débités. L'écriture de Léa a été **écrasée** et la banque a perdu 20 €. Le SGBD doit assurer l'**isolation** : la seconde transaction doit attendre que la première soit terminée, comme si elles s'exécutaient l'une après l'autre.

Vers le bac

Exercice 17 — *Location entre particuliers (d'après Polynésie 2023, jour 2)*

↑énoncé p. 301

Barème indicatif (sur 4) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 0,5 ; Q4 : 0,5 ; Q5 : 0,5 ; Q6 : 0,75 ; Q7 : 0,75

Démarche : une sélection avec `WHERE`, une mise à jour avec `UPDATE ... SET ... WHERE`, un ajout avec `INSERT INTO ... VALUES`.

```sql
SELECT nom, prenom FROM Membre WHERE cp = '69003';    -- 1.

UPDATE Objet SET tarif = 15 WHERE id_objet = 1;       -- 2.

INSERT INTO Membre (id_membre, nom, prenom, cp)       -- 3.
VALUES (6, 'Renard', 'Wendie', '69100');
```

4. Mohamed Ali est référencé (clé étrangère `id_membre`) par des lignes de `Reservation` et de `Possede` : le supprimer violerait la **contrainte de référence** ⇒ erreur.
5. On supprime d'abord les lignes qui le référencent, puis le membre :

```sql
DELETE FROM Reservation WHERE id_membre = 1;
DELETE FROM Possede WHERE id_membre = 1;
DELETE FROM Membre WHERE id_membre = 1;
```

▷ **Erreur fréquente.** Supprimer le membre en premier : la contrainte de référence l'interdit tant que des lignes de `Reservation` ou de `Possede` le référencent. On supprime d'abord ce qui le référence.

Démarche : pour les questions 6 et 7, chaque `JOIN ... ON` relie une clé étrangère à la clé primaire qu'elle référence ; `Possede` sert de lien entre `Membre` et `Objet`.

```sql
SELECT COUNT(*)                                       -- 6.
FROM Reservation
JOIN Membre ON Reservation.id_membre = Membre.id_membre
WHERE Membre.nom = 'Alonso' AND Membre.prenom = 'Fernando';

SELECT Membre.nom, Membre.prenom                      -- 7.
FROM Membre
JOIN Possede ON Membre.id_membre = Possede.id_membre
JOIN Objet   ON Possede.id_objet = Objet.id_objet
WHERE Objet.type = 'appareil à raclette';
```

Exercice 18 — *Location de véhicules (d'après La Réunion 2023, jour 2)*

↑énoncé p. 302

Barème indicatif (sur 4) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 0,5 ; Q4 : 0,5 ; Q5 : 0,5 ; Q6 : 0,5 ; Q7 : 1

1. Résultat : 1, 2, 4 (les trois citadines).

```sql
SELECT immatriculation FROM vehicule WHERE carburant = 'diesel'; -- 2.
SELECT COUNT(*) FROM vehicule;                                   -- 3.
```

4. Dans `location`, `id_utilisateur` et `id_vehicule` sont des clés étrangères : ce sont les clés primaires de `utilisateur` et `vehicule`, utilisées ici pour référencer ces relations.
5. L'`id_location` 1 existe déjà : l'insertion violerait la **contrainte d'entité** (unicité de la clé primaire).

```sql
UPDATE utilisateur                                    -- 6.
SET adresse = '50 rue de la Liberté'
WHERE id_utilisateur = 133;

SELECT vehicule.modele, vehicule.immatriculation,     -- 7.
       utilisateur.nom, utilisateur.prenom
FROM location
JOIN vehicule    ON location.id_vehicule    = vehicule.id_vehicule
JOIN utilisateur ON location.id_utilisateur = utilisateur.id_utilisateur
WHERE location.date_debut = '2022-06-21';
```

Exercice 19 *Podcasts de radio (d'après Centres étrangers 2023, jour 1)* ↑énoncé p. 302

Barème indicatif (sur 4) : Q1 : 0,75 ; Q2 : 0,5 ; Q3 : 0,5 ; Q4 : 0,5 ; Q5 : 0,5 ; Q6 : 0,5 ; Q7 : 0,75

1. `description (id_description:INT, resume:TEXT, duree:INT, #id_podcast:INT)`.

2. `('L'enseignement supérieur est-il juste ?', 2022)`.

```sql
SELECT theme FROM podcast WHERE annee = 2019;         -- 3.
```

4. La liste des thèmes de `podcast`, **sans doublon**.

```sql
UPDATE emission                                       -- 5.
SET animateur = 'Emmanuel L.'
WHERE nom = 'Le Temps du débat';

INSERT INTO emission (id_emission, nom, radio, animateur)   -- 6.
VALUES (12850, 'Hashtag', 'France Inter', 'Mathieu V.');

SELECT podcast.theme, emission.nom, description.resume      -- 7.
FROM description
JOIN podcast  ON description.id_podcast = podcast.id_podcast
JOIN emission ON podcast.id_emission    = emission.id_emission
WHERE description.duree < 5;
```

Exercice 20 *Vols spatiaux (d'après Métropole 2023, jour 2)* ↑énoncé p. 303

Barème indicatif (sur 4) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 0,75 ; Q4 : 0,25 ; Q5 : 0,5 ; Q6 : 0,5 ; Q7 : 1

1. Une clé primaire est un attribut (ou groupe d'attributs) qui **identifie de façon unique** chaque enregistrement.

2. `id_astronaute = 1` est déjà attribué : l'insertion violerait la **contrainte d'entité** (unicité de la clé primaire).

3. `Fusee (id_fusee:INT, modele:TEXT, constructeur:TEXT, nb_places:INT)`.

4. Résultat : 2 (Falcon 9 et Starship).

```sql
SELECT modele, constructeur                           -- 5.
FROM Fusee
WHERE nb_places >= 4;

SELECT nom, prenom                                    -- 6.
FROM Astronaute
ORDER BY nom;

SELECT Astronaute.nom, Astronaute.prenom              -- 7.
FROM Astronaute
JOIN Equipe ON Astronaute.id_astronaute = Equipe.id_astronaute
JOIN Vol    ON Equipe.id_vol            = Vol.id_vol
WHERE Vol.date = '25/10/2022';
```

▷ **Erreur fréquente.** Joindre directement `Astronaute` et `Vol` : aucune clé étrangère ne les relie. Il faut passer par la relation `Equipe`, qui contient une clé étrangère vers chacune.

Exercice 21 *Ludothèque (d'après Métropole 2025, jour 2)* ↑énoncé p. 304

Barème indicatif (sur 4) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 0,5 ; Q4 : 0,75 ; Q5 : 0,75 ; Q6 : 0,5 ; Q7 : 0,5

1. Deux adhérents peuvent avoir le **même nom** (homonymes) : `nom` ne serait pas **unique**.

2. Affiche le `nomJeu` et l'`editeur` de tous les jeux, **triés par ordre alphabétique** du nom.

```sql
SELECT nomJeu, categorie                                -- 3.
FROM jeu
WHERE anneeSortie >= 2010 AND ageMinimum < 10;

SELECT nomJeu                                           -- 4.  (en cours d'emprunt)
FROM emprunt
WHERE dateRendu IS NULL;

SELECT adherent.nom, adherent.prenom                    -- 5.
FROM adherent
JOIN emprunt ON adherent.idAdherent = emprunt.idAdherent
WHERE emprunt.nomJeu = 'Catan';

UPDATE emprunt                                          -- 6.
SET dateRendu = '2025-06-03'
WHERE idEmprunt = 1538;
```

7. `participation` a deux clés étrangères : `#nom` (référence `evenement.nom`) et `#idAdherent` (référence `adherent.idAdherent`).

Critiquer une réponse d'IA

Exercice 22 *Une requête proposée par un assistant d'IA* ↑énoncé p. 304

1. Non. On attend **2** (les deux emprunts d'`id_adherent` 1). Sur la mini-base, la requête renvoie **5** : sans condition `ON`, la jointure produit le **produit cartésien** ($5 \times 3 = 15$ lignes, toutes les combinaisons emprunt/adhérent) ; le `WHERE` n'en conserve que celles associées à la ligne « Durand », soit les **cinq** emprunts, quel que soit leur emprunteur.

2. L'erreur est l'**absence de condition de jointure** (`JOIN` sans `ON`). Correction (résultat sur la mini-base : **2**) :

```sql
SELECT COUNT(*)
FROM emprunt
JOIN adherent ON emprunt.id_adherent = adherent.id
WHERE adherent.nom = 'Durand';
```

3. La tester sur une **toute petite base** dont on connaît la réponse, ou afficher les lignes jointes (`SELECT *`) : les emprunts de Martin ou de Petit sur la ligne « Durand » sautent aux yeux. Réflexe : **tout `JOIN` a son `ON`**.

S'entraîner à l'oral

Exercice 23 *Expliquer en deux minutes* ↑énoncé p. 305

▶ **Solution.** Pas de texte à apprendre par cœur : voici les **éléments attendus** pour chaque sujet. L'ordre, les mots et l'exemple peuvent être différents ; l'explication est réussie si ces idées y sont, justes et reliées entre elles.

Sujet 1.

- Les données sont réparties en **relations** (tables) liées entre elles, sans redondance : l'adresse d'un adhérent n'est écrite qu'une fois.
- Les **contraintes d'intégrité** sont vérifiées par le SGBD : il refuse, par exemple, l'emprunt d'un livre qui n'existe pas.
- Le **SGBD** gère les accès simultanés de plusieurs utilisateurs, les droits d'accès, les sauvegardes, et répond aux requêtes SQL sur des millions de lignes.

- Piège : un argument de confort (« c'est plus pratique ») ; les vrais arguments sont la cohérence, la sécurité et le partage.

Sujet 2.

- **Clé primaire** : attribut (ou groupe d'attributs) qui identifie chaque enregistrement de façon **unique** ; deux lignes ne peuvent pas avoir la même.
- **Clé étrangère** : attribut qui fait référence à la clé primaire d'une autre table ; c'est elle qui crée le lien entre les tables.
- Exemple : `Livre(id, titre)` et `Emprunt(id, id_livre, date)`, où `id_livre` fait référence à `Livre.id`.
- Piège : une clé étrangère n'est pas unique (un livre est emprunté plusieurs fois) ; on ne peut pas supprimer un livre encore référencé par un emprunt.

Sujet 3.

- Une jointure (`JOIN ... ON ...`) associe chaque ligne d'une table aux lignes de l'autre qui vérifient la condition, en général « clé étrangère = clé primaire ».
- Exemple :

```sql
SELECT Livre.titre, Emprunt.date
FROM Emprunt
JOIN Livre ON Emprunt.id_livre = Livre.id;
```

- Le résultat est une table qui réunit des colonnes des deux tables : on lit le titre de chaque livre emprunté.
- Piège : oublier la condition `ON` (on obtient toutes les combinaisons) ; préfixer par le nom de la table les attributs de même nom.

Concevoir un bon schéma

Exercice 24 IA IA IA *Un schéma mal conçu : anomalies, structure et contenu* ↑énoncé p. 305

1. Le titre, l'auteur et le pays de l'auteur sont recopiés à chaque emprunt du même livre (*L'Étranger*, Camus, France apparaît deux fois), le pays de Camus à chaque emprunt d'un de ses livres (trois fois), le nom et la ville de Martin à chacun de ses emprunts (deux fois). C'est une **redondance** de données.

2. Les lignes 2 et 3 (tous les emprunts de Martin). Si l'on n'en modifie qu'une, Martin habite à la fois à Menton et à Nice : la base est **incohérente**. C'est une **anomalie de mise à jour**.

3. Non : chaque ligne est un emprunt, il faudrait inventer un emprunt (ou laisser vides l'adhérent et la date) pour enregistrer le livre. De même pour l'adhérent sans emprunt. C'est une **anomalie d'insertion**.

4. Le livre *1984*, son auteur Orwell (et son pays) et l'adhérente Petit (et sa ville) n'apparaissaient que dans cette ligne : la base les « oublie » complètement. C'est une **anomalie de suppression**.

5. Une relation par sorte d'objet, reliées par des clés étrangères : `auteur(id, nom, pays)`

```
livre(id, titre, #id_auteur)
adherent(id, nom, ville)
emprunt(id, #id_livre, #id_adherent, date)
```

Le déménagement de Martin ne modifie plus qu'une ligne d'`adherent` ; un livre ou un adhérent s'enregistre sans emprunt ; supprimer un emprunt ne supprime ni le livre ni l'adhérent. On retrouve le schéma de la base `bibliotheque`. *(Un découpage en trois relations, avec le pays de l'auteur dans `livre`, est accepté ; mais le pays de Camus y reste recopié pour chacun de ses livres.)*

6. *(a)* contenu (`INSERT` d'une ligne dans `emprunt`) ; *(b)* structure (le schéma de `adherent` change) ; *(c)* contenu (`UPDATE`) ; *(d)* structure (`CREATE TABLE` : une relation de plus dans le schéma

de la base) ; *(e)* contenu (`DELETE`). La **structure** est l'ensemble des schémas des relations ; le **contenu**, l'ensemble des enregistrements à un instant donné.

CORRIGÉS DES TP

TP 1 La ligue de futsal des quartiers

↑ énoncé p. 308

Partie 1 — Modéliser

Exercice 1 *Du cahier des charges au schéma*

1. Quatre relations, exactement celles du schéma retenu (encadré du sujet). Toute proposition équivalente convient (par exemple `numero` au lieu de `id`). Pièges : oublier la clé primaire de `but`, ou mettre une liste de buteurs dans `rencontre` (un attribut ne contient qu'**une** valeur).
2. Le nom serait recopié six fois : une faute de frappe créerait une équipe fantôme, et renommer l'équipe obligerait à modifier toutes les lignes. On écrit chaque information **une seule fois** et on relie par une **clé étrangère** (`id_equipe`).
3. `rencontre` référence deux fois `equipe` : une fois pour l'équipe qui reçoit (`id_domicile`), une fois pour celle qui se déplace (`id_exterieur`).
4. Rien : la valeur `NULL` (case vide), que l'on teste avec `IS NULL`.

Partie 2 — Créer des tables avec leurs contraintes

Exercice 2 *Deux tables dans une base vierge*

Ce sont les instructions qui ont créé la base de la ligue (suivies des `INSERT` de deux équipes et de deux joueurs, au choix) :

```sql
CREATE TABLE equipe (
    id             INTEGER PRIMARY KEY,
    nom            TEXT NOT NULL UNIQUE,
    quartier       TEXT NOT NULL,
    annee_creation INTEGER
);

CREATE TABLE joueur (
    id              INTEGER PRIMARY KEY,
    nom             TEXT NOT NULL,
    prenom          TEXT NOT NULL,
    annee_naissance INTEGER CHECK (annee_naissance BETWEEN 1950 AND 2015),
    poste           TEXT CHECK (poste IN ('gardien', 'defenseur', 'milieu', 'attaquant')),
    id_equipe       INTEGER NOT NULL REFERENCES equipe(id)
);
```

On accepte aussi la syntaxe `FOREIGN KEY (id_equipe) REFERENCES equipe(id)` placée en fin de table.

Exercice 3 *La table des rencontres*

```sql
CREATE TABLE rencontre (
    id           INTEGER PRIMARY KEY,
    journee      INTEGER NOT NULL,
    date_match   TEXT NOT NULL,
    id_domicile  INTEGER NOT NULL REFERENCES equipe(id),
    id_exterieur INTEGER NOT NULL REFERENCES equipe(id),
    buts_dom     INTEGER CHECK (buts_dom >= 0),
    buts_ext     INTEGER CHECK (buts_ext >= 0),
    CHECK (id_domicile <> id_exterieur)
);
```

La contrainte « une équipe ne joue pas contre elle-même » porte sur deux attributs : elle s'écrit après la liste des attributs. La table `but` est construite sur le même modèle (`minute` entre 1 et 40 : un match de futsal dure 2×20 minutes).

Exercice 4 *Comparer avec la vraie base*

Différences typiquement relevées : un `NOT NULL` oublié, la contrainte « domicile $\neq$ extérieur » écrite sur une seule colonne (impossible : elle porte sur deux attributs), une clé étrangère sans `REFERENCES`.

Partie 3 — Mettre les contraintes à l'épreuve

Exercice 5 *Le SGBD monte la garde*

Messages réels de SQLite (DB Browser affiche les mêmes) :

	Message	Contrainte
a	`UNIQUE constraint failed: equipe.id`	d'entité : la clé primaire `1` existe déjà
b	`UNIQUE constraint failed: equipe.nom`	`UNIQUE` sur le nom
c	`NOT NULL constraint failed: equipe.quartier`	`NOT NULL` : quartier manquant
d	`FOREIGN KEY constraint failed`	de référence : pas d'équipe `9`
e	`CHECK constraint failed: poste IN (...)`	métier (`CHECK`)
f	`CHECK constraint failed: id_domicile <> id_exterieur`	métier : une équipe contre elle-même
g	`FOREIGN KEY constraint failed`	de référence : des joueurs et des rencontres pointent vers l'équipe `6`
h	*acceptée*	domaine *non* vérifié par SQLite

1. Supprimer l'équipe 6 laisserait des joueurs et des rencontres qui désignent une équipe **inexistante** : la contrainte de référence interdit aussi les suppressions qui créeraient de telles « références pendantes ».

2. SQLite a un typage **souple** : le type déclaré n'est qu'une préférence (« affinité »). `'2021'` serait converti en entier, mais `'hier'` est stocké tel quel, en texte (`typeof(annee_creation)` vaut `'text'`). D'autres SGBD (PostgreSQL, MySQL) refusent. Parade : `annee_creation INTEGER CHECK (typeof(annee_creation) = 'integer')` — avec cette contrainte, 2020 et `'2021'` sont acceptés, `'hier'` est refusé (depuis SQLite 3.37, on peut aussi déclarer la table `STRICT`).

3. Requête :

```sql
DELETE FROM equipe
WHERE id = 8;
```

Partie 4 — Interroger la base de la saison

Exercice 6 *Sélectionner, filtrer, trier*

```sql
-- Q1 (6 lignes) : Rocher Club | Monaco-Ville  en premier (1995)
SELECT nom, quartier
FROM equipe
ORDER BY annee_creation;

-- Q2 (5 lignes)
SELECT prenom, nom, annee_naissance
FROM joueur
WHERE annee_naissance >= 2008
ORDER BY annee_naissance DESC;

-- Q3 (4 lignes)
SELECT nom, prenom, id_equipe
FROM joueur
WHERE poste = 'gardien' AND annee_naissance < 2000;

-- Q4 (4 lignes) : gardien, defenseur, milieu, attaquant
SELECT DISTINCT poste
FROM joueur;

-- Q5 (6 lignes) : Rossi, Bianchi, Ricci, Benali, Moretti, Bellini
SELECT nom, prenom
FROM joueur
WHERE nom LIKE '%i';

-- Q6 (3 lignes) : rencontres 13 et 14 (2026-10-03), 15 (2026-10-04)
SELECT id, journee, date_match
FROM rencontre
WHERE buts_dom IS NULL;
```

Q1 : Rocher Club (1995), AS Fontvieille (1998), Monte-Carlo United (2001), Condamine FC (2004), Moneghetti Futsal (2011), Larvotto Sporting (2016). **Q2** : Gianni Esposito 2010, Sacha Brun 2009, Simone Gallo 2009, Yanis Moreau 2008, Maxime Faure 2008 (l'ordre entre deux joueurs de la même année n'est pas garanti). **Q3** : Rossi Luca (1), Fabre Louis (2), Lefebvre Paul (4), Girard Antoine (5).

Exercice 7 *Agréger et regrouper*

```sql
-- Q7 : 36 | 1990 | 2010
SELECT COUNT(*), MIN(annee_naissance), MAX(annee_naissance)
FROM joueur;

-- Q8 : 56 | 4.666...   (et SELECT COUNT(*) FROM but; donne aussi 56)
SELECT SUM(buts_dom + buts_ext), AVG(buts_dom + buts_ext)
FROM rencontre
WHERE buts_dom IS NOT NULL;

-- Q9 : attaquant 12, defenseur 12, gardien 6, milieu 6
SELECT poste, COUNT(*) AS nb
FROM joueur
GROUP BY poste;

-- Q10 : journee 1 -> 14, 2 -> 14, 3 -> 17, 4 -> 11
SELECT journee, SUM(buts_dom + buts_ext) AS buts
FROM rencontre
WHERE buts_dom IS NOT NULL
GROUP BY journee;
```

Remarque sur Q8 : sans le `WHERE`, `AVG` ignore de toute façon les lignes où la somme vaut `NULL` ; le filtre rend l'intention explicite.

Exercice 8 — *Croiser les tables*

```sql
-- Q11 (6 lignes) : Andrea Bellini (milieu) en premier
SELECT joueur.prenom, joueur.nom, joueur.poste
FROM joueur
JOIN equipe ON joueur.id_equipe = equipe.id
WHERE equipe.nom = 'Rocher Club'
ORDER BY joueur.nom;

-- Q12 : le classement des buteurs
SELECT joueur.prenom, joueur.nom, equipe.nom AS equipe, COUNT(*) AS nb_buts
FROM but
JOIN joueur ON but.id_joueur = joueur.id
JOIN equipe ON joueur.id_equipe = equipe.id
GROUP BY joueur.id
ORDER BY nb_buts DESC, joueur.nom
LIMIT 5;

-- Q13 (3 lignes)
SELECT equipe.nom, COUNT(*) AS buts_marques
FROM but
JOIN joueur ON but.id_joueur = joueur.id
JOIN equipe ON joueur.id_equipe = equipe.id
GROUP BY equipe.id
HAVING COUNT(*) >= 10;
```

Q12 :

prenom	nom	equipe	nb_buts
Yanis	Moreau	AS Fontvieille	7
Karim	Benali	Monte-Carlo United	6
Nicolo	Ferraro	Rocher Club	5
Alessio	Moretti	Monte-Carlo United	5
Tom	Barbier	Moneghetti Futsal	4

Diego Carvalho et Marco Colombo ont eux aussi 4 buts : le tri par nom départage.

Q13 :

nom	buts_marques
AS Fontvieille	10
Condamine FC	10
Monte-Carlo United	15

(ordre non garanti sans `ORDER BY`). `WHERE` ne peut pas filtrer sur `COUNT(*)` : c'est le rôle de `HAVING`.

Exercice 9 — *Les résultats d'une journée*

```sql
-- Q14 : la table equipe jointe deux fois, sous deux alias
SELECT d.nom AS domicile, r.buts_dom, r.buts_ext, e.nom AS exterieur
FROM rencontre AS r
JOIN equipe AS d ON r.id_domicile = d.id
JOIN equipe AS e ON r.id_exterieur = e.id
WHERE r.journee = 3;

-- Q15 : controle de coherence
SELECT rencontre.id, rencontre.buts_dom + rencontre.buts_ext AS score_total,
       COUNT(but.id) AS buts_saisis
FROM rencontre
JOIN but ON but.id_rencontre = rencontre.id
GROUP BY rencontre.id
HAVING rencontre.buts_dom + rencontre.buts_ext <> COUNT(but.id);
```

Q14 : AS Fontvieille 4 – 4 Moneghetti Futsal ; Condamine FC 1 – 0 Rocher Club ; Monte-Carlo United 6 – 2 Larvotto Sporting.

Q15 : résultat **vide** — la base est cohérente pour les 12 rencontres jouées. Limite : un match joué sur le score de 0 à 0 n'a aucune ligne dans `but` et échappe à la jointure ; il n'y en a pas ici.

Partie 5 — Mettre la base à jour

Exercice 10 *Le week-end de la journée 5*

1. à 3. puis tentatives des questions 4 et 5 :

```sql
UPDATE rencontre
SET buts_dom = 1, buts_ext = 1
WHERE id = 14;

INSERT INTO but (id_rencontre, id_joueur, minute)
VALUES (14, 36, 12), (14, 11, 29);

UPDATE rencontre
SET date_match = '2026-10-11'
WHERE id = 15;

UPDATE equipe
SET annee_creation = 1993
WHERE nom = 'Rocher Club';

DELETE FROM joueur
WHERE id = 6;                       -- refusee : FOREIGN KEY constraint failed

DELETE FROM joueur
WHERE nom = 'Petit' AND prenom = 'Raphaël';    -- 1 ligne supprimee
```

4. Refus (`FOREIGN KEY constraint failed`) : 7 lignes de `but` désignent Yanis Moreau ; les supprimer avec lui effacerait l'histoire de la saison. Dans la vraie vie, on garderait le joueur (par exemple avec un attribut « actif »).
5. Raphaël Petit (gardien de Larvotto) n'a marqué aucun but : aucune ligne ne le référence. Il reste 35 joueurs.
6. Elle tenterait de supprimer **tous** les joueurs. Ici, SQLite la refuse en bloc (des buts référencent certains joueurs) : il reste bien 35 joueurs. Mais sans clés étrangères actives, la table serait vidée : « le `WHERE` qui sauve des vies ».
La requête de contrôle (Q15) renvoie toujours un résultat vide.

Partie 6 — Piloter la base depuis Python

Exercice 11 *Premières requêtes depuis Python*

Après la partie 5 (Rocher Club créé en 1993), `equipes_creees_avant(2005)` renvoie :

```
[('Rocher Club', 'Monaco-Ville'), ('AS Fontvieille', 'Fontvieille'),
 ('Monte-Carlo United', 'Monte-Carlo'), ('Condamine FC', 'La Condamine')]
```

```python
def joueurs_de(nom_equipe):
    requete = """SELECT joueur.prenom, joueur.nom, joueur.poste
                 FROM joueur
                 JOIN equipe ON joueur.id_equipe = equipe.id
                 WHERE equipe.nom = ?
                 ORDER BY joueur.nom"""
    return connexion.execute(requete, (nom_equipe,)).fetchall()

def nb_buts(nom, prenom):
    requete = """SELECT COUNT(*)
                 FROM but
                 JOIN joueur ON but.id_joueur = joueur.id
                 WHERE joueur.nom = ? AND joueur.prenom = ?"""
    return connexion.execute(requete, (nom, prenom)).fetchone()[0]
```

Exercice 12 *Pourquoi le point d'interrogation ?*

1. La condition envoyée devient `WHERE equipe.nom = 'x' OR '1'='1'`, toujours vraie : on obtient **tous** les joueurs (35 après la partie 5, 36 sinon). **2.** Sur un site, un utilisateur malveillant peut ainsi lire, modifier ou détruire des données auxquelles il n'a pas droit (*injection SQL*, l'une des failles les plus exploitées du Web). Avec `?`, la valeur est transmise **à part**, comme une simple donnée : elle n'est jamais interprétée comme du SQL ; la version avec `?` renvoie une liste vide (c'est un test du fichier).

Exercice 13 *Le classement du championnat*

Idée. Une rencontre jouée compte pour **deux** équipes, avec les mêmes buts vus de deux côtés : pour l'équipe qui reçoit, les buts « pour » sont `bd` et les buts « contre » `be` ; pour l'équipe qui se déplace, c'est l'inverse. On écrit donc **une seule fois** la mise à jour d'une ligne du classement (fonction `compter_match`), et on l'appelle deux fois par rencontre.

```
def compter_match(s, pour, contre):
    # met a jour la ligne s du classement apres un match
    s["J"] = s["J"] + 1
    s["bp"] = s["bp"] + pour
    s["bc"] = s["bc"] + contre
    if pour > contre:
        s["G"] = s["G"] + 1
        s["pts"] = s["pts"] + 3
    elif pour == contre:
        s["N"] = s["N"] + 1
        s["pts"] = s["pts"] + 1
    else:
        s["P"] = s["P"] + 1
```

Dans `classement()`, à la place des trous :

```
    requete = """SELECT id_domicile, id_exterieur, buts_dom, buts_ext
                 FROM rencontre
                 WHERE buts_dom IS NOT NULL"""
    for (dom, ext, bd, be) in connexion.execute(requete):
        compter_match(stats[dom], bd, be)   # equipe qui recoit
        compter_match(stats[ext], be, bd)   # equipe qui se deplace : buts inverses
```

Autre méthode : sans fonction auxiliaire, la boucle du coup de pouce `for (equipe, pour, contre) in ((dom, bd, be), (ext, be, bd)):` parcourt les deux cas, et son corps contient les mêmes instructions que `compter_match`, appliquées à `s = stats[equipe]`.

Affichage obtenu après la partie 5 (journées 1 à 4 et rencontre 14) :

```
   Equipe               J  G  N  P  bp  bc diff  Pts
1  Monte-Carlo United   4  2  2  0  15   8   +7    8
2  Condamine FC         5  2  2  1  11   7   +4    8
3  AS Fontvieille       4  2  1  1  10   8   +2    7
4  Rocher Club          4  1  2  1   7   7   +0    5
5  Moneghetti Futsal    4  1  1  2   9  14   -5    4
6  Larvotto Sporting    5  0  2  3   6  14   -8    2
```

Chaque but marqué par une équipe est un but encaissé par une autre : sur tout le championnat, la somme des buts pour égale la somme des buts contre. C'est un **invariant** qui détecte une inversion `bd`/`be`.

Exercice 14 *Saisir un résultat en « tout ou rien »*

Idée. Dans le bloc `with connexion:`, on insère chaque but en comptant au passage les buts de chaque équipe (on cherche l'équipe du buteur) ; un buteur étranger au match provoque une erreur, qui annule tout le bloc. Le score n'est écrit qu'à la fin, une fois tous les buts insérés.

```
def saisir_resultat(id_rencontre, buts):
    dom, ext = connexion.execute(
        "SELECT id_domicile, id_exterieur FROM rencontre WHERE id = ?",
        (id_rencontre,)).fetchone()
    bd = 0
    be = 0
    with connexion:
        for (id_joueur, minute) in buts:
            connexion.execute(
                "INSERT INTO but (id_rencontre, id_joueur, minute) VALUES (?, ?, ?)",
                (id_rencontre, id_joueur, minute))
            ligne = connexion.execute("SELECT id_equipe FROM joueur WHERE id = ?",
                                      (id_joueur,)).fetchone()
            if ligne[0] == dom:
                bd = bd + 1
            elif ligne[0] == ext:
                be = be + 1
            else:
                raise ValueError(f"le joueur {id_joueur} ne joue pas ce match")
        connexion.execute(
            "UPDATE rencontre SET buts_dom = ?, buts_ext = ? WHERE id = ?",
            (bd, be, id_rencontre))
```

1. Premier appel : `ValueError: le joueur 1 ne joue pas ce match` ; second : `sqlite3.IntegrityError: FOREIGN KEY constraint failed` (joueur 99 inexistant). Dans les deux cas, le premier but (joueur 30), pourtant inséré, a été **annulé** : 0 but enregistré pour la rencontre 15, score toujours `NULL`. C'est l'**atomicité** (le A de ACID) : tout ou rien.
2. Rencontre 13 : AS Fontvieille 2 – 3 Monte-Carlo United ; rencontre 15 : Moneghetti Futsal 0 – 2 Rocher Club. Classement final :

	Equipe	J	G	N	P	bp	bc	diff	Pts
1	Monte-Carlo United	5	3	2	0	18	10	+8	11
2	Condamine FC	5	2	2	1	11	7	+4	8
3	Rocher Club	5	2	2	1	9	7	+2	8
4	AS Fontvieille	5	2	1	2	12	11	+1	7
5	Moneghetti Futsal	5	1	1	3	9	16	-7	4
6	Larvotto Sporting	5	0	2	3	6	14	-8	2

Meilleurs buteurs : **égalité** entre Karim Benali (Monte-Carlo United) et Yanis Moreau (AS Fontvieille), 8 buts chacun (requête Q12 relancée), devant Nicolo Ferraro et Alessio Moretti (6). La base contient alors 65 buts.

Pour aller plus loin

Exercice 15 *Le classement en une seule requête SQL* [au-delà du programme]

```
SELECT equipe.nom,
       COUNT(*) AS j,
       SUM(CASE WHEN pour > contre THEN 3
                WHEN pour = contre THEN 1
                ELSE 0 END) AS pts,
       SUM(pour) - SUM(contre) AS diff
FROM (SELECT id_domicile AS id_eq, buts_dom AS pour, buts_ext AS contre
      FROM rencontre
      WHERE buts_dom IS NOT NULL
      UNION ALL
```

```sql
      SELECT id_exterieur, buts_ext, buts_dom
      FROM rencontre
      WHERE buts_dom IS NOT NULL) AS resultats
JOIN equipe ON resultats.id_eq = equipe.id
GROUP BY equipe.id
ORDER BY pts DESC, diff DESC;
```

Sur la base finale : Monte-Carlo United 11 (+8), Condamine FC 8 (+4), Rocher Club 8 (+2), AS Fontvieille 7 (+1), Moneghetti Futsal 4 (−7), Larvotto Sporting 2 (−8) — identique au classement Python. `UNION ALL`, `CASE` et les sous-requêtes dépassent le programme : défi réservé aux plus rapides.

Bilan — éléments attendus

- Trois erreurs parmi : clé primaire en double (entité), nom d'équipe en double (`UNIQUE`), quartier absent (`NOT NULL`), équipe ou joueur inexistant (référence), poste inconnu ou équipe contre elle-même (`CHECK`), suppression d'un joueur qui a marqué (référence).
- `WHERE` filtre les **lignes** avant le regroupement ; `HAVING` filtre les **groupes** après `GROUP BY` (Q13 : `HAVING COUNT(*) >= 10`).
- Le `?` empêche l'injection SQL : la valeur reste une donnée, alors qu'une valeur collée dans le texte de la requête peut en changer le sens.
- Classement final ci-dessus ; c'est la **transaction** (atomicité) qui garantit qu'aucun match n'est à moitié saisi.

TP 2 SQL Murder Mystery

↑ énoncé p. 313

Partie 1 — Explorer la base

Exercice 1 *Combien de tables, combien de lignes ?*

1. La requête donne 9 tables. **2.** Par exemple `SELECT COUNT(*) FROM person;`, puis de même pour chaque table :

table	lignes	table	lignes
`crime_scene_report`	1 228	`get_fit_now_check_in`	2 703
`drivers_license`	10 007	`solution`	0
`facebook_event_checkin`	20 011	`income`	7 514
`interview`	4 991	`person`	10 011
`get_fit_now_member`	184		

La plus grosse table est `facebook_event_checkin` ; la table `solution` est vide. **3.** Elle sert à proposer et vérifier la réponse (exercices 8 et 11).

Exercice 2 *Lire la structure et dessiner le schéma*

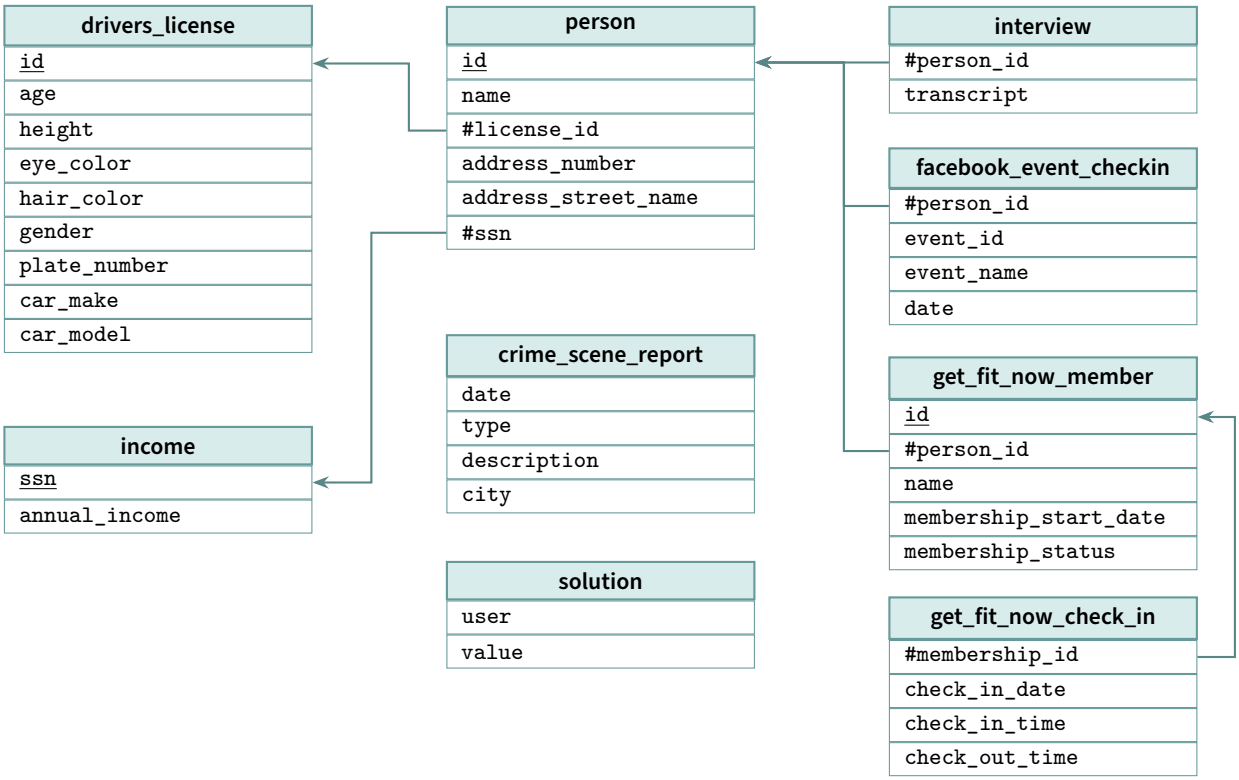
1. Clés primaires : `drivers_license.id`, `person.id`, `income.ssn` et `get_fit_now_member.id` (un **texte**, comme `'48Z55'` : une clé primaire n'est pas forcément un entier). Cinq tables n'en ont pas : `crime_scene_report`, `interview`, `facebook_event_checkin`, `get_fit_now_check_in` et `solution`. Rien n'y empêche donc deux lignes identiques (il n'y en a d'ailleurs aucune : vérifié avec `SELECT DISTINCT`). Le cours dit qu'une relation est un **ensemble** : pas de doublon. Pour le garantir, il faudrait déclarer une clé, éventuellement formée de plusieurs attributs, par exemple (`membership_id`, `check_in_date`, `check_in_time`) pour les passages à la salle. Sans clé, aucune autre table ne peut non plus faire référence à une ligne précise.

2. Six clés étrangères :

- `person.license_id` → `drivers_license.id` et `person.ssn` → `income.ssn` ;
- `interview.person_id`, `facebook_event_checkin.person_id` et `get_fit_now_member.person_id` → `person.id` ;
- `get_fit_now_check_in.membership_id` → `get_fit_now_member.id`.

3. `ssn` est une clé **étrangère** dans `person` (`ssn CHAR REFERENCES income (ssn)`) et la clé **primaire** de `income`.

4. Schéma attendu (les domaines peuvent être ajoutés : entier, texte) :



`person` est au centre : cinq des six clés étrangères y aboutissent ou en partent. `crime_scene_report` et `solution` ne sont reliées à aucune table : le rapport de police ne mène aux personnes qu'à travers le **texte** de sa description (une rue, un prénom), c'est le travail de l'enquêteur de faire le lien.

5. `20180115`. Le format AAAAMMJJ place l'année en tête, puis le mois, puis le jour : l'ordre des nombres est celui du calendrier. Décembre 2017 s'écrit `date >= 20171201 AND date <= 20171231`.

Exercice 3 *Premiers coups de sonde*

```sql
SELECT DISTINCT type
FROM crime_scene_report;

SELECT COUNT(*)
FROM crime_scene_report
WHERE city = 'SQL City';

SELECT MIN(date), MAX(date)
FROM crime_scene_report;
```

1. 9 types : `robbery`, `murder`, `theft`, `fraud`, `arson` (incendie criminel), `bribery` (corruption), `assault` (agression), `smuggling` (contrebande), `blackmail` (chantage). **2.** 9 crimes à SQL City. **3.** De `20170101` (1er janvier 2017) à `20180501` (1er mai 2018).

Partie 2 — L'enquête

Exercice 4 *Le rapport perdu*

```sql
SELECT *
FROM crime_scene_report
WHERE date = 20180115 AND type = 'murder' AND city = 'SQL City';
```

Une seule ligne (sans la condition sur le type, on obtient aussi deux agressions du même jour). Sa description :

> *Security footage shows that there were 2 witnesses. The first witness lives at the last house on "Northwestern Dr". The second witness, named Annabel, lives somewhere on "Franklin Ave".*

Traduction : les images de vidéosurveillance montrent deux témoins. Le premier habite la dernière maison de *Northwestern Drive*. La seconde, prénommée Annabel, habite quelque part sur *Franklin Avenue*.

Exercice 5 *Les deux témoins*

1. « La dernière maison » : le plus grand numéro de la rue.

```sql
SELECT MAX(address_number)
FROM person
WHERE address_street_name = 'Northwestern Dr';

SELECT id, name
FROM person
WHERE address_street_name = 'Northwestern Dr' AND address_number = 4919;
```

Le maximum vaut 4919 ; le premier témoin est **Morty Schapiro**, `id` 14887. Autre méthode : `ORDER BY address_number DESC` sur les 50 habitants de la rue, première ligne (identique). On accepte aussi `LIMIT 1`, au-delà du programme.

2. `name = 'Annabel'` ne renvoie **aucune** ligne : l'attribut `name` contient le prénom **et** le nom, et `=` compare la chaîne entière.

```sql
SELECT id, name, address_number
FROM person
WHERE address_street_name = 'Franklin Ave'
ORDER BY name;
```

45 habitants ; la deuxième ligne est `16371 | Annabel Miller | 103`. Le second témoin est **Annabel Miller**, `id` 16371.

3. [au-delà du programme] `WHERE name LIKE 'Annabel%' AND address_street_name = 'Franklin Ave'` donne directement la même ligne.

Exercice 6 *Les dépositions*

1. `SELECT * FROM interview WHERE person_id = 14887 OR person_id = 16371;` **2.** Avec une jointure :

```sql
SELECT person.name, interview.transcript
FROM person
JOIN interview ON interview.person_id = person.id
WHERE person.id = 14887 OR person.id = 16371;
```

(On peut aussi filtrer sur `person.name`.) Deux lignes :

> *Morty Schapiro : I heard a gunshot and then saw a man run out. He had a "Get Fit Now Gym" bag. The membership number on the bag started with "48Z". Only gold members have those bags. The man got into a car with a plate that included "H42W".*

> *Annabel Miller : I saw the murder happen, and I recognized the killer from my gym when I was working out last week on January the 9th.*

3. Indices et attributs où les vérifier :

- c'est un **homme** : `drivers_license.gender` ;
- il a un sac de la salle *Get Fit Now Gym*, réservé aux abonnés **gold** : `get_fit_now_member.membership_status` ;
- son numéro d'abonnement commence par **48Z** : `get_fit_now_member.id` ;
- sa plaque d'immatriculation contient **H42W** : `drivers_license.plate_number` ;
- il était à la salle le **9 janvier 2018** (Annabel s'y entraînait aussi) : `get_fit_now_check_in.check_in_date`.

Exercice 7 *Le suspect*

1. `SELECT COUNT(*) FROM get_fit_now_check_in WHERE check_in_date = 20180109;` donne 10 passages.

2.

```sql
SELECT get_fit_now_member.id, get_fit_now_member.name, get_fit_now_member.person_id
FROM get_fit_now_member
JOIN get_fit_now_check_in ON get_fit_now_check_in.membership_id = get_fit_now_member.id
WHERE get_fit_now_check_in.check_in_date = 20180109
  AND get_fit_now_member.membership_status = 'gold'
ORDER BY get_fit_now_member.id;
```

id	name	person_id
48Z55	Jeremy Bowers	67318
48Z7A	Joe Germuska	28819
6LSTG	Burton Grippe	83186
90081	Annabel Miller	16371
GE5Q8	Carmen Dimick	92736
XTE42	Sarita Bartosh	55662

Le témoin Annabel Miller est bien dans la liste. Deux abonnements commencent par `48Z` : **deux suspects**, Jeremy Bowers (67318) et Joe Germuska (28819).

3.

```sql
SELECT person.name, drivers_license.gender, drivers_license.plate_number,
       drivers_license.car_make, drivers_license.car_model
FROM person
JOIN drivers_license ON person.license_id = drivers_license.id
WHERE person.id = 67318 OR person.id = 28819;
```

Une seule ligne : `Jeremy Bowers | male | 0H42W2 | Chevrolet | Spark LS`. La plaque contient `H42W` : le meurtrier est **Jeremy Bowers**. Joe Germuska **disparaît** du résultat : son `license_id` (173289) ne correspond à aucun permis, et une jointure ne garde que les lignes qui ont un correspondant.

Pour le professeur. C'est une **violation de la contrainte de référence** : la base déclare la clé étrangère mais ne la respecte pas. `PRAGMA foreign_key_check;` en compte 2 502 (5 permis introuvables, 2 497 numéros `ssn` absents de `income`). SQLite ne vérifie les clés étrangères que sur demande (`PRAGMA foreign_keys = ON`, cf. cours) : bel exemple, à commenter en classe, de ce que le SGBD laisse passer quand on ne lui demande rien.

4. [au-delà du programme] Trois jointures et `LIKE` :

```sql
SELECT person.id, person.name
FROM person
JOIN get_fit_now_member ON get_fit_now_member.person_id = person.id
JOIN get_fit_now_check_in ON get_fit_now_check_in.membership_id = get_fit_now_member.id
JOIN drivers_license ON person.license_id = drivers_license.id
WHERE get_fit_now_check_in.check_in_date = 20180109
```

```
  AND get_fit_now_member.membership_status = 'gold'
  AND get_fit_now_member.id LIKE '48Z%'
  AND drivers_license.plate_number LIKE '%H42W%';
```

Résultat : `67318 | Jeremy Bowers`.

Exercice 8 *Valider la réponse*

`INSERT INTO solution VALUES (1, 'Jeremy Bowers');` puis `SELECT value FROM solution;` :

> *Congrats, you found the murderer! But wait, there's more... If you think you're up for a challenge, try querying the interview transcript of the murderer to find the real villain behind this crime. If you feel especially confident in your SQL skills, try to complete this final step with no more than 2 queries. Use this same INSERT statement with your new suspect to check your answer.*

1. Bravo, vous avez trouvé le meurtrier ! Mais ce n'est pas fini… Si vous aimez les défis, lisez la déposition du meurtrier pour trouver le vrai responsable de ce crime. Si vous êtes sûr de vous en SQL, faites cette dernière étape en deux requêtes au plus. Vérifiez votre réponse avec la même instruction `INSERT`.
2. Une seule ligne, dont la colonne `user` vaut 0 et non 1 : la ligne insérée a été remplacée (explication à l'exercice 11).

Partie 3 — Le vrai commanditaire

Exercice 9 *Ce que sait le meurtrier*

```
SELECT transcript FROM interview WHERE person_id = 67318;
```

> *I was hired by a woman with a lot of money. I don't know her name but I know she's around 5'5" (65") or 5'7" (67"). She has red hair and she drives a Tesla Model S. I know that she attended the SQL Symphony Concert 3 times in December 2017.*

J'ai été engagé par une femme très riche ; je ne connais pas son nom, mais elle mesure entre 5 pieds 5 et 5 pieds 7 (65 à 67 pouces), elle est rousse, conduit une Tesla Model S et a assisté trois fois au *SQL Symphony Concert* en décembre 2017. Indices : femme (`gender`), taille de 65 à 67 pouces, soit environ 165 à 170 cm ($65 \times 2{,}54 \approx 165{,}1$ et $67 \times 2{,}54 \approx 170{,}2$) (`height`), cheveux roux (`hair_color = 'red'`), Tesla Model S (`car_make`, `car_model`), tous dans `drivers_license` ; le concert dans `facebook_event_checkin` (`event_name`, `date`) ; la fortune dans `income` (inutile pour conclure).

Exercice 10 *Démasquer la commanditaire*

1.

```
SELECT person.id, person.name, drivers_license.height
FROM person
JOIN drivers_license ON person.license_id = drivers_license.id
WHERE drivers_license.gender = 'female' AND drivers_license.hair_color = 'red'
  AND drivers_license.car_make = 'Tesla' AND drivers_license.car_model = 'Model S'
  AND drivers_license.height >= 65 AND drivers_license.height <= 67;
```

3 permis, donc trois personnes : Red Korb (78881, 65), Regina George (90700, 66), Miranda Priestly (99716, 66).

2 et 3. En ajoutant une jointure avec `facebook_event_checkin` et les conditions sur le concert, on obtient la **seconde** requête du défi (la première étant la lecture de la déposition) :

```
SELECT person.name, facebook_event_checkin.date
FROM person
JOIN drivers_license ON person.license_id = drivers_license.id
JOIN facebook_event_checkin ON facebook_event_checkin.person_id = person.id
WHERE drivers_license.gender = 'female' AND drivers_license.hair_color = 'red'
  AND drivers_license.car_make = 'Tesla' AND drivers_license.car_model = 'Model S'
```

```
  AND drivers_license.height >= 65 AND drivers_license.height <= 67
  AND facebook_event_checkin.event_name = 'SQL Symphony Concert'
  AND facebook_event_checkin.date >= 20171201 AND facebook_event_checkin.date <= 20171231
ORDER BY facebook_event_checkin.date;
```

Trois lignes, toutes pour **Miranda Priestly** (20171206, 20171212, 20171229). Validation avec `INSERT INTO solution VALUES (1, 'Miranda Priestly');` :

> *Congrats, you found the brains behind the murder! Everyone in SQL City hails you as the greatest SQL detective of all time. Time to break out the champagne!*

(Bravo, vous avez démasqué le cerveau du crime ! Tout SQL City vous acclame comme le plus grand détective SQL de tous les temps. Sortez le champagne !) Pour la petite histoire, son revenu annuel est de 310 000 dollars (jointure `person` – `income` par `ssn`).

4. [au-delà du programme]

```
SELECT person_id, COUNT(*)
FROM facebook_event_checkin
WHERE event_name = 'SQL Symphony Concert'
  AND date >= 20171201 AND date <= 20171231
GROUP BY person_id
HAVING COUNT(*) = 3;
```

Deux personnes : 24556 et 99716 (Miranda Priestly). La première, Bryan Pardo, n'a aucun permis enregistré (encore une clé étrangère sans correspondant) : seul le croisement avec le portrait permet de conclure. C'est l'intérêt d'un indice **recoupé**.

Partie 4 — Comment la base vérifie-t-elle la réponse ?

Exercice 11 *Le déclencheur caché*

La requête affiche (textes des messages abrégés) :

```
CREATE TRIGGER check_solution AFTER INSERT ON solution
    WHEN new.user==1
    BEGIN
        DELETE FROM solution;
        INSERT INTO solution VALUES (0,
        CASE WHEN hex(new.value)=='4A6572656D7920426F77657273' THEN "Congrats, you found the murderer! ..."
             WHEN hex(new.value)=='4D6972616E6461205072696573746C79' THEN "Congrats, you found the brains ..."
             ELSE "That's not the right person. Try again!"
        END
        );
    END
```

1. Il s'exécute **après chaque insertion** dans `solution` dont la colonne `user` vaut 1. Il **vide** la table (`DELETE`), puis y **insère** une ligne dont `user` vaut 0 et dont `value` est le message choisi selon le nom proposé (`CASE ... WHEN`, au-delà du programme). D'où la ligne unique avec `user` à 0 de l'exercice 8 ; et comme la nouvelle ligne a `user` à 0, elle ne redéclenche pas le déclencheur.
2. Pour ne pas gâcher l'enquête : le schéma de la base est lisible par tous (on l'a affiché dès l'exercice 1), un nom écrit en clair y serait trouvé sans aucune requête.
3. `4A 65 72 65` donne `J e r e` ($4A_{16} = 74$, code ASCII de `J`) : c'est le début de « Jeremy Bowers », et le second code donne « Miranda Priestly ». Le nom n'est **pas secret** : il est seulement **codé**, et n'importe qui peut le décoder avec une table publique. **Chiffrer**, c'est rendre le message illisible sans une **clé** secrète (chapitre *Cryptographie*). Pour vérifier une réponse sans pouvoir la retrouver, on stockerait plutôt une **empreinte** (fonction de hachage), au-delà du programme.

Bilan — éléments attendus

- Au programme : `SELECT`, `FROM`, `WHERE` (avec `AND`, `OR`), `JOIN ... ON`, `ORDER BY`, `DISTINCT`, `COUNT`, `MIN`, `MAX`, `INSERT`. Au-delà : `LIKE`, `LIMIT`, `GROUP BY` / `HAVING`, la table `sqlite_master`, le déclencheur.
- Chemin, en trois étapes :
  - rapport de police (par le texte : rue, prénom) → `person` → `interview` (par `person_id`) ;
  - `get_fit_now_check_in` → `get_fit_now_member` (`membership_id`) → `person` (`person_id`) → `drivers_license` (`license_id`) ;
  - `drivers_license` → `person` → `facebook_event_checkin` (`person_id`).
- Domaines : dates entières AAAAMMJJ (pas d'apostrophes, comparaisons possibles), taille en pouces, texte comparé caractère par caractère (`'SQL City'`, prénom et nom réunis dans `name`), identifiant d'abonnement de type texte.
- Réponse personnelle ; le plus souvent : la jointure à trois tables de l'exercice 7 ou l'oubli du préfixe de table (*ambiguous column name : id*).

Pour aller plus loin

Exercice 12 *L'enquête rejouée en Python, sans* `LIKE`

Extrait de `tp_murder_corrige.py`, où `lignes(requete, parametres)` renvoie la liste des lignes du résultat :

```python
requete_gold = """SELECT get_fit_now_member.id, get_fit_now_member.person_id
FROM get_fit_now_member
JOIN get_fit_now_check_in ON get_fit_now_check_in.membership_id = get_fit_now_member.id
WHERE get_fit_now_check_in.check_in_date = ? AND get_fit_now_member.membership_status = ?"""
candidats = []                                          # INITIALISER
for (id_membre, id_personne) in lignes(requete_gold, (20180109, "gold")):  # PARCOURIR
    if id_membre[:3] == "48Z":                          # TESTER
        candidats.append(id_personne)                   # METTRE A JOUR

requete_permis = """SELECT person.name, drivers_license.plate_number
FROM person
JOIN drivers_license ON person.license_id = drivers_license.id
WHERE person.id = ?"""
coupables = []                                          # INITIALISER
for id_personne in candidats:                           # PARCOURIR
    for (nom, plaque) in lignes(requete_permis, (id_personne,)):
        if "H42W" in plaque:                            # TESTER
            coupables.append(nom)                       # METTRE A JOUR
print(coupables)                                        # ['Jeremy Bowers']
```

La tranche `id_membre[:3]` remplace `LIKE '48Z%'` ; `"H42W" in plaque` remplace `LIKE '%H42W%'`. Les valeurs passent par des `?` : jamais collées dans le texte de la requête (cf. cours, injection SQL). Le candidat Joe Germuska, sans permis, ne produit aucune ligne dans la seconde boucle.

Réponses des questions de la fiche de révision

↑ questions p. 320

Se tester

1. elle référence la clé primaire d'une autre table : elle relie les deux tables (§ III.C).
2. elle supprime **toutes** les lignes de `emprunt` (pas de `WHERE`) (§ VI).
3. `SELECT COUNT(*) FROM film WHERE annee > 2010;` : `WHERE` filtre les lignes, puis `COUNT` les compte (§ V.F).
4. 50 : c'est le produit cartésien (§ V.J et Erreurs fréquentes).

5. le « tout ou rien » : le débit et le crédit réussissent ensemble ; en cas de panne, tout est annulé (§ IV).

Et avant ?

1. 1 et 0 (on compte les nœuds) ; en comptant les arêtes : 0 et -1 (ch. 5, § III).
2. c'est la dernière parenthèse ouverte qui doit être refermée la première : politique LIFO (ch. 3, § II.C).
3. 9 : `fib(3)` en déclenche 5 et `fib(2)` 3, plus l'appel initial (ch. 1, § IV.B).

CORRIGÉS DU CHAPITRE 7

Diviser pour régner

7

CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Couper en deux, encore et encore

↑ énoncé p. 333

Exercice 1 *Le dictionnaire de 2000 pages*

1. Au pire 2 000 pages (le mot est sur la dernière page regardée).

2. Tableau complété :

Coups	0	1	2	3	4	5	6	7	8	9	10	11
Pages	2 000	1 000	500	250	125	63	32	16	8	4	2	1

3. 11 coups d'œil au pire. Avec 4 000 pages, un seul de plus : 12. Doubler la taille n'ajoute qu'**un** coup : c'est le comportement du logarithme ($2^{11} = 2\,048 \geqslant 2\,000$).

4. Au plus 7 questions, car $2^7 = 128 \geqslant 100$ (et $2^6 = 64 < 100$). Si l'on dépasse 7, c'est qu'on n'a pas visé le milieu.

Exercice 2 *Fusionner deux paquets triés*

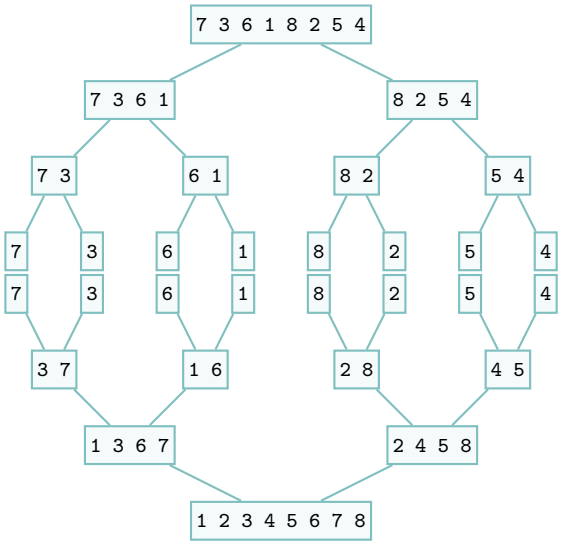
5. On prend dans l'ordre `1, 2, 3, 4, 5, 6, 7, 8`.

6. 7 comparaisons. Quand le paquet D est vide (après la carte `7`), il ne reste que la carte `8` : on la pose sans rien comparer.

7. Au plus 99 comparaisons (une par carte prise, sauf la dernière). Chaque carte n'est prise **qu'une fois** : fusionner coûte de l'ordre du nombre total de cartes.

Exercice 3 *Trier en équipe*

8. Coupes (de haut en bas) puis fusions (de bas en haut) :



9. 3 étages pour 8 cartes ($2^3 = 8$), 4 pour 16, 10 pour 1 024 ($2^{10} = 1\,024$) : c'est $\log_2 n$.

10. Chaque étage de fusions prend chacune des 8 cartes une fois : au plus 8 comparaisons par étage, donc au plus $3 \times 8 = 24$ pour tout le tri. (Sur ce paquet, on en compte en fait 17.)

11. Tableau complété :

Nombre de cartes n	8	16	1 024
Tri par sélection : $\frac{n(n-1)}{2}$	28	120	523 776
Tri en équipe : $n\times$ (nombre d'étages)	24	64	10 240

Sur 8 cartes, la différence est faible ; sur 1 024 cartes, le tri en équipe est environ 50 fois plus économe.

Exercice 4 *Mettre des mots*

12. Propositions possibles puis mots du cours : **diviser** (couper, partager), **régner** (résoudre, faire trier), **combiner** (recoller, fusionner).

13. Dans le dictionnaire, **combiner** n'a rien à faire : on ne garde qu'**une** moitié, et la réponse trouvée dans cette moitié est la réponse au problème entier.

14. « . . .comme une fonction **récursive** ; elle s'arrête quand le paquet n'a **qu'une carte** (condition d'arrêt). »

CORRIGÉS DES EXERCICES

Remarque Plusieurs écritures sont souvent possibles : on donne **une** version simple.

Comprendre et dérouler

★ **Exercice 1** *Recherche dichotomique à la main* ↑énoncé p. 346

▶ **Solution.** `tab = [3, 8, 12, 19, 25, 31, 40, 42, 55, 60, 71]` (indices 0 à 10 ; on note `m = (g + d) // 2`).

1. Recherche de `40` :

- `g=0, d=10, m=5` : `tab[5]=31 < 40` → `g=6` ;
- `g=6, d=10, m=8` : `tab[8]=55 > 40` → `d=7` ;
- `g=6, d=7, m=6` : `tab[6]=40` : **trouvé**.

Valeurs du milieu examinées : `31`, `55`, `40` → **3 valeurs**.

2. Recherche de `7` (absent) :

- `g=0, d=10, m=5` : `31 > 7` → `d=4` ;
- `g=0, d=4, m=2` : `12 > 7` → `d=1` ;
- `g=0, d=1, m=0` : `3 < 7` → `g=1` ;
- `g=1, d=1, m=1` : `8 > 7` → `d=0`.

Maintenant `g=1 > d=0` : l'intervalle de recherche est **vide**, donc `7` est **absent**. Valeurs du milieu examinées : `31`, `12`, `3`, `8` → **4 valeurs**.

▷ **Erreur fréquente.** Garder le milieu dans la zone (`g = 5` au lieu de `g = 6`) : l'élément du milieu vient d'être comparé, on l'exclut. Sinon, la zone peut cesser de rétrécir et la recherche ne s'arrête plus.

3. À chaque valeur du milieu examinée, la zone de recherche est **divisée par deux**. Partant de 1000, on atteint 1 en une dizaine d'étapes : $2^{10} = 1024 > 1000$, donc **au plus 10 valeurs du milieu examinées** (coût logarithmique $\log_2 n$). *(Le code du cours peut faire jusqu'à deux tests `==` et `<` par valeur examinée : on compte les étapes, pas chaque test.)*

★ **Exercice 2** *Maximum d'un tableau par diviser pour régner* ↑énoncé p. 346

▶ **Solution.** **1. Diviser** : `milieu = len(t) // 2` coupe `t` en deux moitiés ; **régner** : les deux appels récursifs calculent `m_g` et `m_d` ; **combiner** : le `if` garde le plus grand des deux. La condition d'arrêt est le tableau d'un seul élément.

2. Démarche : condition d'arrêt : un seul élément, c'est le maximum ; les deux appels récursifs portent sur les tranches `t[:milieu]` et `t[milieu:]`.

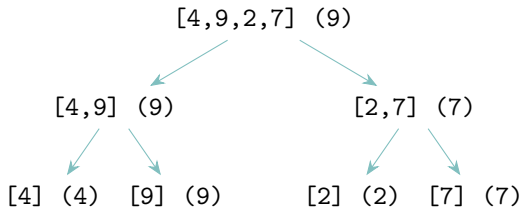
```
1  def maximum(t):
2      if len(t) == 1:                # CAS DE BASE : un seul element
3          return t[0]
4      milieu = len(t) // 2           # DIVISER
5      m_g = maximum(t[:milieu])      # APPEL RECURSIF : moitie gauche
6      m_d = maximum(t[milieu:])      # APPEL RECURSIF : moitie droite
7      if m_g > m_d:                  # COMBINER : garder le plus grand
8          return m_g
9      else:
10         return m_d
```

`maximum([4, 9, 2, 7])` renvoie `9`.

Autre méthode : au lieu de recopier les moitiés avec des tranches, on peut, comme pour la recherche dichotomique du cours, travailler sur la zone `t[debut..fin]` repérée par deux indices ; aucune liste n'est alors recopiée. Premier appel : `maximum_zone(t, 0, len(t) - 1)`.

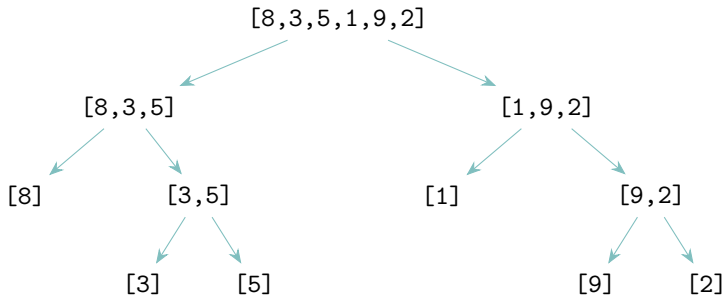
```
def maximum_zone(t, debut, fin):
    if debut == fin:                        # CAS DE BASE : un seul element
        return t[debut]
    milieu = (debut + fin) // 2             # DIVISER
    m_g = maximum_zone(t, debut, milieu)    # APPEL RECURSIF
    m_d = maximum_zone(t, milieu + 1, fin)  # APPEL RECURSIF
    if m_g > m_d:                           # COMBINER
        return m_g
    else:
        return m_d
```

3. Arbre des appels (valeur renvoyée entre parenthèses) :



★★ Exercice 3 *Tri fusion : l'arbre des appels* ↑énoncé p. 346

▶ Solution. 1. Phase « diviser » de `[8, 3, 5, 1, 9, 2]` (coupe à `milieu = len // 2`) :



▷ Erreur fréquente. Couper `[8, 3, 5]` en `[8, 3]` et `[5]` : avec `milieu = 3 // 2 = 1`, la moitié gauche est `tab[:1] = [8]`, la plus courte.

2. Remontée (fusions) :

- `[3]` & `[5]` → `[3,5]` ; puis `[8]` & `[3,5]` → `[3,5,8]` ;
- `[9]` & `[2]` → `[2,9]` ; puis `[1]` & `[2,9]` → `[1,2,9]` ;
- enfin `[3,5,8]` & `[1,2,9]` → `[1,2,3,5,8,9]`.

3. Les deux appels récursifs portent sur les deux moitiés :

```
def tri_fusion(tab):
    if len(tab) <= 1:                   # CAS DE BASE
        return tab
    milieu = len(tab) // 2              # DIVISER
    gauche = tri_fusion(tab[:milieu])   # APPEL RECURSIF
    droite = tri_fusion(tab[milieu:])   # APPEL RECURSIF
    return fusion(gauche, droite)       # COMBINER
```

★ **Exercice 4** *Compter les appels du tri fusion* ↑énoncé p. 347

▶ **Solution.** **1.** `[6,2,7,1]` appelle `[6,2]` et `[7,1]`, qui appellent chacun deux listes d'un élément (`[6]`, `[2]`, `[7]`, `[1]`) : $1+2+4=$ **7 appels**.

▷ **Erreur fréquente.** Oublier l'appel initial, ou ne compter que les feuilles : on trouve 6 ou 4 au lieu de 7. Chaque nœud de l'arbre est un appel.

2. Pour 8 éléments : 1 appel (taille 8), puis 2 (taille 4), 4 (taille 2), 8 (taille 1). Il y a donc **3 niveaux** sous l'appel initial et $1+2+4+8=$ **15 appels** en tout.

3. À chaque niveau, la taille est divisée par 2 : $1024 \to 512 \to \cdots \to 1$, soit **10 niveaux**, et $10 = \log_2 1024$. Le nombre de niveaux de l'arbre est $\log_2 n$: c'est l'origine du facteur $\log_2 n$ dans le coût $n \log_2 n$ du tri fusion.

★★ **Exercice 5** *Écrire la fusion de deux listes triées* ↑énoncé p. 347

▶ **Solution.** **1.** 1 contre 2 → 1 ; 4 contre 2 → 2 ; 4 contre 3 → 3 ; 4 contre 10 → 4 ; 9 contre 10 → 9. La première liste est épuisée : on recopie le reste `[10, 12]`. Résultat `[1, 2, 3, 4, 9, 10, 12]`, en **5 comparaisons**.

2. Démarche : un indice par liste ; tant que les deux listes ont encore des éléments, on compare leurs têtes et l'on recopie la plus petite ; à la fin, on recopie le reste de la liste non épuisée.

Appels attendus, écrits avant de coder :

```
assert fusion([1, 4, 9], [2, 3, 10, 12]) == [1, 2, 3, 4, 9, 10, 12]   # cas normal
assert fusion([], [1, 2]) == [1, 2]                                    # cas limite : une liste vide
assert fusion([5, 6], [1, 2, 3]) == [1, 2, 3, 5, 6]                    # cas piege : recopier le reste de a
```

```
def fusion(a, b):
    resultat = []                       # INITIALISER
    i, j = 0, 0                         # INITIALISER : un indice par liste
    while i < len(a) and j < len(b):    # PARCOURIR
        if a[i] <= b[j]:                # TESTER : la plus petite tete
            resultat.append(a[i])       # METTRE A JOUR
            i = i + 1
        else:
            resultat.append(b[j])       # METTRE A JOUR
            j = j + 1
    return resultat + a[i:] + b[j:]     # RENVOYER : recopier le reste
```

`fusion([1, 4, 9], [2, 3, 10, 12])` renvoie `[1, 2, 3, 4, 9, 10, 12]` ; `fusion([], [1, 2])` renvoie `[1, 2]` (on n'entre pas dans la boucle).

▷ **Erreur fréquente.** Oublier de recopier la fin de la liste non épuisée (`a[i:]` ou `b[j:]`) : sur l'exemple, 10 et 12 disparaîtraient du résultat.

3. Chaque tour de boucle fait **une** comparaison et recopie **un** élément. La boucle s'arrête dès qu'une liste est épuisée, donc au plus quand tous les éléments sauf un ont été recopiés : au plus `len(a) + len(b) - 1` comparaisons. Le coût de la fusion est **linéaire** en la taille totale $n =$ `len(a) + len(b)`.

Vers le bac

★★ **Exercice 6** IA IA IA *Coder un texte avec « diviser pour régner »* (d'après Métropole 2025, jour 2) ↑énoncé p. 347

Barème indicatif (sur 6) : Q1 : 0,75 ; Q2 : 1,25 ; Q3 : 1 ; Q4 : 0,75 ; Q5 : 0,75 ; Q6 : 1 ; Q7 : 0,5

▶ **Solution. 1.** L'effectif total est $15+7+6+6+5=39$, et $39/2=19{,}5$. Le cumul atteint 19,5 dès `"B"` ($15+7=22$). Donc

$$\texttt{separe(tab)} = \big([(\text{"}A\text{"},15),(\text{"}B\text{"},7)]\ ,\ [(\text{"}C\text{"},6),(\text{"}D\text{"},6),(\text{"}E\text{"},5)]\big).$$

`"A"` est dans le premier groupe → son code commence par `"1"` ; `"C"` est dans le second → son code commence par `"0"`.

2. On préfixe le bit puis on recommence sur le *bon* groupe :

```
def shannon(symbole, tab):
    if len(tab) == 1:                          # CAS DE BASE
        return ""
    groupe1, groupe2 = separe(tab)             # DIVISER
    if symbole in [c for (c, _) in groupe1]:   # TESTER : dans quel groupe ?
        return "1" + shannon(symbole, groupe1) # APPEL RECURSIF ; COMBINER : bit devant
    else:
        return "0" + shannon(symbole, groupe2) # APPEL RECURSIF ; COMBINER : bit devant
```

3. Codes obtenus : `A = 11`, `B = 10`, `C = 011`, `D = 010`, `E = 00`. Déroulement de `shannon("C", tab)` :

- `tab` → groupes `[A,B]` et `[C,D,E]` ; `C` est dans le second → `"0" + shannon("C", [C,D,E])` ;
- `[C,D,E]` : total 17, coupe après `C,D` (cumul $12 \geq 8{,}5$) → groupes `[C,D]` et `[E]` ; `C` dans le premier → `"1" + shannon("C", [C,D])` ;
- `[C,D]` : total 12, coupe après `C` (cumul $6 \geq 6$) → groupes `[C]` et `[D]` ; `C` dans le premier → `"1" + shannon("C", [C])` ;
- `[C]` : un seul caractère → `""`.

En remontant : `"1" + "" = "1"`, puis `"1" + "1" = "11"`, puis `"0" + "11" = "011"`. D'où `C = 011`.

4. Les codes `11, 10, 011, 010, 00` forment un **code préfixe** : aucun n'est le début d'un autre. C'est indispensable au **décodage** : en lisant le texte codé bit à bit, dès qu'on reconnaît un code on sait qu'il est *complet* (il ne peut pas être le début d'un code plus long), donc le découpage en symboles est **sans ambiguïté**.

5. Trois temps de « diviser pour régner » : **diviser** = couper la liste en deux groupes d'effectifs à peu près équilibrés (`separe`) ; **régner** = appeler récursivement `shannon` sur le groupe contenant le symbole ; **combiner** = préfixer le bit `"1"` ou `"0"` au code renvoyé.

6. Terminaison. Le variant est `len(tab)` (nombre de caractères). `separe` renvoie deux groupes **non vides** dont la réunion est `tab` (le premier contient au moins `tab[0]` ; comme la liste est triée par effectifs **décroissants**, le dernier effectif vaut au plus la moitié du total, donc le cumul atteint la moitié *avant* le dernier caractère et le second groupe n'est pas vide — sans ce tri, `separe([("A", 1), ("B", 3)])` renverrait un second groupe vide et `shannon("A", ...)` ne s'arrêterait pas) ; le groupe sur lequel on récurse contient donc **strictement moins** de caractères que `tab`. Cet entier positif décroît d'au moins 1 à chaque appel et atteint 1 (cas d'arrêt) : la fonction s'arrête toujours.

7. Démarche : une chaîne vide que l'on complète en parcourant le texte : à chaque caractère, on ajoute son code `shannon`.

Appels attendus, écrits avant de coder :

```
assert encode_shannon("CAD", tab) == "01111010"   # cas normal
assert encode_shannon("", tab) == ""               # cas limite : texte vide
assert encode_shannon("AAE", tab) == "111100"      # cas piege : chaque occurrence est codee
```

```
def encode_shannon(texte, tab):
    code = ""                          # INITIALISER
    for c in texte:                    # PARCOURIR
        code = code + shannon(c, tab)  # METTRE A JOUR
    return code                        # RENVOYER
```

Ainsi `encode_shannon("CAD", tab)` vaut `"011" + "11" + "010" = "01111010"`.

★★ Exercice 7 *Le tri de Stooge* (d'après Amérique du Nord 2024, jour 2)

↑énoncé p. 348

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 1 ; Q3 : 0,5 ; Q4 : 0,5 ; Q5 : 1,25 ; Q6 : 0,75 ; Q7 : 1 ; Q8 : 0,5

▶ **Solution.** **1.** Échange sur place (la liste est modifiée, rien n'est renvoyé) :

```
def echange(tab, i, j):
    temp = tab[i]          # variable temporaire
    tab[i] = tab[j]
    tab[j] = temp
```

2. Deux premiers tiers : de `i` à `j - k` ; deux derniers tiers : de `i + k` à `j` :

```
def triStooge(tab, i, j):
    if tab[i] > tab[j]:              # TESTER : extremites dans le desordre ?
        echange(tab, i, j)
    if (j - i) > 1:                  # TESTER : sinon CAS DE BASE (au plus 2 elements)
        k = (j - i + 1) // 3         # DIVISER : en tiers
        triStooge(tab, i, j - k)     # APPEL RECURSIF : deux premiers tiers
        triStooge(tab, i + k, j)     # APPEL RECURSIF : deux derniers tiers
        triStooge(tab, i, j - k)     # APPEL RECURSIF : deux premiers tiers
```

Avec `A = [5, 6, 4, 2, 3, 1]`, `triStooge(A, 0, 5)` donne bien `[1, 2, 3, 4, 5, 6]`.

3. L'algorithme est **récursif** : la fonction `triStooge` s'appelle elle-même (trois fois) dans son propre corps. La condition d'arrêt (cas de base) est `j - i <= 1` : il reste au plus deux éléments, et un simple échange suffit.

4. `k = (5 - 0 + 1) // 3 = 6 // 3 = 2`. Le premier appel porte sur 6 éléments, chaque tiers en contient 2.

▷ **Erreur fréquente.** Calculer `k = (5 - 0) // 3 = 1` : la formule compte les **éléments** de l'indice `i` à l'indice `j` inclus, soit `j - i + 1 = 6`, d'où `k = 2`.

5. Niveau 1 : 3 appels ; niveau 2 : $3 \times 3 = 9$ appels ; niveau 3 : $9 \times 3 = 27$ appels ; ces 27 appels portent tous sur deux éléments (`j - i = 1`) : ils n'ont pas d'enfants. Total : $3 + 9 + 27 = \mathbf{39}$ appels récursifs.

6. Sur les arêtes partant de la racine, `k = 2` ; sur celles partant du niveau 1 (sous-tableaux de 4 éléments), `k = 1`.

- case **1** : deuxième appel de `triStooge(A,0,3)` (`k = 1`) → `triStooge(A,1,3)` ;
- case **2** : premier appel de `triStooge(A,2,4)` (`k = 1`) → `triStooge(A,2,3)` ;
- case **3** : troisième appel de la racine, identique au premier → `triStooge(A,0,3)`.

7. Avec `A = [5, 6, 4, 2]` : l'appel `triStooge(A,0,3)` échange d'abord 5 et 2, puis `k = 1` et il lance `triStooge(A,0,2)`, `triStooge(A,1,3)`, `triStooge(A,0,2)`.

Appel	A avant l'appel	A après l'appel
`triStooge(A,0,3)`	`[5, 6, 4, 2]`	`[2, 4, 5, 6]`
`triStooge(A,0,2)`	`[2, 6, 4, 5]`	`[2, 4, 6, 5]`
`triStooge(A,1,3)`	`[2, 4, 6, 5]`	`[2, 4, 5, 6]`
`triStooge(A,0,2)`	`[2, 4, 5, 6]`	`[2, 4, 5, 6]`

8. Le **tri fusion**, de coût $n \log_2 n$ dans le pire des cas, est strictement meilleur que $n^{8/3}$ (l'énoncé arrondit : l'exposant exact est $\log 3 / \log 1{,}5 \approx 2{,}71$, ce qui ne change pas la conclusion ; et même les tris par insertion ou par sélection, de coût n^2, font mieux que le tri de Stooge).

★★★ **Exercice 8** IA IA IA *L'élément absolument majoritaire* (d'après Asie 2024, jour 2) ↑énoncé p. 350

Barème indicatif (sur 8) : Q1 : 0,5 ; Q2 : 0,75 ; Q3 : 0,5 ; Q4 : 1 ; Q5 : 0,75 ; Q6 : 0,75 ; Q7 : 0,75 ; Q8 : 0,5 ; Q9 : 0,75 ; Q10 : 0,5 ; Q11 : 1,25

▶ **Solution.** **1.** Il faut apparaître strictement plus de $10/2 = 5$ fois : effectifs possibles $6, 7, 8, 9$ ou 10.

2.

```
def effectif(val, lst):
    n = 0                # INITIALISER
    for x in lst:        # PARCOURIR
        if x == val:     # TESTER
            n = n + 1    # METTRE A JOUR
    return n             # RENVOYER
```

3. Chaque élément de la liste est comparé une fois à `val` : **9 comparaisons**.

4. *Appels attendus, écrits avant de coder :*

```
assert majo_abs1([1, 4, 1, 6, 1, 7, 2, 1, 1]) == 1     # cas normal
assert majo_abs1([7]) == 7                              # cas limite : un seul element
assert majo_abs1([1, 4, 6, 1, 7, 2, 1, 1]) is None      # cas piege : 4 fois sur 8, pas plus de
                                                        # la moitie
```

```
def majo_abs1(lst):
    for val in lst:                          # PARCOURIR
        if effectif(val, lst) > len(lst) / 2:   # TESTER
            return val                       # RENVOYER : trouve
    return None                              # RENVOYER : aucun
```

5. Le premier élément testé est `1`, dont l'effectif 5 dépasse $9/2$: la fonction renvoie `1` dès le premier tour. Un seul appel à `effectif`, soit **9 comparaisons** entre éléments (plus la comparaison `5 > 4.5`). Dans le pire des cas (pas d'élément majoritaire), il y aurait n appels à `effectif`, soit n^2 comparaisons.

6.

```
def eff_dico(lst):
    dico_sortie = {}                                # INITIALISER
    for elt in lst:                                 # PARCOURIR
        if elt in dico_sortie:                      # TESTER
            dico_sortie[elt] = dico_sortie[elt] + 1 # METTRE A JOUR
        else:
            dico_sortie[elt] = 1                    # METTRE A JOUR : premiere apparition
    return dico_sortie                              # RENVOYER
```

Par exemple `eff_dico([1, 4, 1, 6, 1, 7, 2, 1, 1])` renvoie `{1: 5, 4: 1, 6: 1, 7: 1, 2: 1}`.

7.

```
def majo_abs2(lst):
    dico = eff_dico(lst)
    for cle in dico:                    # PARCOURIR
        if dico[cle] > len(lst) / 2:    # TESTER
            return cle                  # RENVOYER
    return None                         # RENVOYER : aucun
```

8. Si $n = 1$, l'unique élément `lst[0]` apparaît $1 > 1/2$ fois : c'est l'élément absolument majoritaire. Le cas de base renvoie `lst[0]`.

9. Notons n_1 et n_2 les tailles de `lst1` et `lst2` ($n_1 + n_2 = n$). Si un élément `x` était absolument majoritaire dans `lst`, il ne le serait dans aucune des deux moitiés, donc il apparaîtrait au plus $n_1/2$ fois dans `lst1` et au plus $n_2/2$ fois dans `lst2`, soit au plus $(n_1 + n_2)/2 = n/2$ fois dans `lst` : ce n'est pas *strictement* plus que la moitié. Contradiction : `lst` n'a pas d'élément absolument majoritaire.

10. Compter les apparitions de `maj1` dans **toute** la liste `lst` (fonction `effectif`) et comparer à $n/2$: si `effectif(maj1, lst) > n/2`, alors `maj1` est l'élément absolument majoritaire de `lst` ; sinon, il ne l'est pas (et il faut examiner l'éventuel majoritaire de `lst2`).

11. Si aucun des deux candidats ne convient, la fonction atteint sa fin sans `return` et renvoie donc `None` (question 9).

```
def majo_abs3(lst):
    n = len(lst)
    if n == 1:                      # CAS DE BASE
        return lst[0]
    else:
        lst_g = lst[:n//2]          # DIVISER
        lst_d = lst[n//2:]          # DIVISER
        maj_g = majo_abs3(lst_g)    # APPEL RECURSIF
        maj_d = majo_abs3(lst_d)    # APPEL RECURSIF
        if maj_g is not None:       # COMBINER : recompter dans toute la liste
            eff = effectif(maj_g, lst)
            if eff > n/2:
                return maj_g
        if maj_d is not None:       # COMBINER
            eff = effectif(maj_d, lst)
            if eff > n/2:
                return maj_d
```

`majo_abs3([1, 4, 1, 6, 1, 7, 2, 1, 1])` renvoie `1` et `majo_abs3([1, 4, 6, 1, 7, 2, 1, 1])` renvoie `None`. Les trois temps : **diviser** (couper en deux moitiés), **régner** (deux appels récursifs), **combiner** (compter les deux candidats dans toute la liste).

★★ Exercice 9 — *Les deux points les plus proches* (d'après Polynésie 2023, jour 1)

↑énoncé p. 351

Barème indicatif (sur 8) : Q1.a : 0,25 ; Q1.b : 0,5 ; Q1.c : 0,5 ; Q2.a : 0,5 ; Q2.b : 1,25 ; Q2.c : 0,75 ; Q2.d : 0,75 ; Q3 : 2 ; Q4.a : 0,5 ; Q4.b : 1

▶ **Solution.** **1. a)** Elle importe la fonction `sqrt` (racine carrée) du module `math`, pour pouvoir l'utiliser directement sous le nom `sqrt`.

1. b) Les nombres flottants sont stockés en binaire avec un nombre limité de bits : `0.1`, `0.2` et `0.3` n'ont pas d'écriture binaire finie et sont arrondis. La somme vaut en réalité `0.30000000000000004`, différente de l'arrondi de `0.3`. On ne doit donc jamais tester l'égalité de deux flottants avec `==`.

1. c) Un tuple est **non modifiable** (immuable) : on peut lire `point_A[0]`, mais pas lui affecter une nouvelle valeur, d'où la `TypeError`.

2. a) Ligne 6 :

```
6         self.longueur = sqrt((point2[0] - point1[0])**2 + (point2[1] - point1[1])**2)
```

2. b) Démarche : on associe chaque point `i` uniquement aux points `j` situés *après* lui, pour ne pas construire $[BA]$ en plus de $[AB]$:

```
def liste_segments(liste_points):
    n = len(liste_points)
    segments = []                                   # INITIALISER
    for i in range(n - 1):                          # PARCOURIR
        for j in range(i + 1, n):                   # PARCOURIR : j apres i
            seg = Segment(liste_points[i], liste_points[j])
            segments.append(seg)                    # METTRE A JOUR
    return segments                                 # RENVOYER
```

▷ **Erreur fréquente.** Faire partir `j` de `0` : on construit alors $[AB]$ et $[BA]$, et même des « segments » $[AA]$ de longueur nulle, qui fausseraient la recherche du plus court.

2. c) Le point d'indice 0 donne $n-1$ segments, le point d'indice 1 en donne $n-2$, …, le point d'indice $n-2$ en donne 1 : la longueur vaut $(n-1)+(n-2)+\cdots+1 = \dfrac{n(n-1)}{2}$.

2. d) On construit $\dfrac{n(n-1)}{2}$ segments, chacun en temps constant : la complexité est **quadratique**, en $O(n^2)$.

3. Démarche : condition d'arrêt : un seul segment ; sinon, deux appels récursifs sur les deux moitiés de la liste, puis on garde le segment de plus petite `longueur` (on renvoie l'objet, pas sa longueur).

Appels attendus, écrits avant de coder :

```
segs = liste_segments([(3, 4), (2, 3), (-3, -1)])      # [AB], [AC], [BC]
assert plus_court_segment(segs) is segs[0]              # cas normal : [AB]
assert plus_court_segment([segs[2]]) is segs[2]         # cas limite : un seul segment
assert plus_court_segment([segs[2], segs[1], segs[0]]) is segs[0]   # cas piege : l'objet, en
                                                                    # dernier
```

```
def plus_court_segment(segments):
    if len(segments) == 1:                              # CAS DE BASE
        return segments[0]
    seg_g = plus_court_segment(moitie_gauche(segments))    # DIVISER, APPEL RECURSIF
    seg_d = plus_court_segment(moitie_droite(segments))    # DIVISER, APPEL RECURSIF
    if seg_g.longueur <= seg_d.longueur:                # COMBINER : garder le plus court
        return seg_g
    else:
        return seg_d
```

Chaque appel réduit strictement la taille de la liste (les deux moitiés sont non vides dès qu'il y a au moins deux segments), ce qui garantit la terminaison.

4. a) `nuage_points = [(3, 4), (2, 3), (-3, -1)]`

4. b) Démarche : on construit la liste de tous les segments, on cherche le plus court par diviser pour régner, puis on affiche ses deux extrémités.

```
segments = liste_segments(nuage_points)
seg = plus_court_segment(segments)
print(seg.p1, seg.p2)
```

Affichage obtenu : `(3, 4) (2, 3)` : les points A et B, à distance $\sqrt{2} \approx 1{,}41$.

★★★ Exercice 10 *Comparer deux classements* (sujet maison, façon bac)

↑énoncé p. 353

Barème indicatif (sur 8) : Q1 : 0,75 ; Q2 : 1 ; Q3 : 1 ; Q4 : 0,75 ; Q5 : 0,75 ; Q6 : 1,25 ; Q7 : 1 ; Q8 : 0,5 ; Q9 : 1

▶ **Solution. 1.** Dans `[2, 4, 1, 3, 5]`, les couples rangés « à l'envers » sont $(2,1)$, $(4,1)$ et $(4,3)$: **3 inversions**. Le classement `[1, 2, 3, 4, 5]` (identique au classement officiel) n'en a aucune. Le classement `[5, 4, 3, 2, 1]` a le maximum : *tous* les couples sont inversés, soit $4+3+2+1 = \mathbf{10}$ inversions.

2. *Appels attendus, écrits avant de coder :*

```
assert inversions_naif([2, 4, 1, 3, 5]) == 3     # cas normal
assert inversions_naif([]) == 0                  # cas limite : liste vide
assert inversions_naif([5, 4, 3, 2, 1]) == 10    # cas piege : tous les couples sont inverses
```

```
def inversions_naif(t):
    nb = 0                                  # INITIALISER
    for i in range(len(t)):                 # PARCOURIR
        for j in range(i + 1, len(t)):      # PARCOURIR : j apres i
            if t[i] > t[j]:                 # TESTER
                nb = nb + 1                 # METTRE A JOUR
    return nb                               # RENVOYER
```

`inversions_naif([2, 4, 1, 3, 5])` renvoie 3.

3. Pour `i = 0`, on fait $n-1$ comparaisons, pour `i = 1`, $n-2$, …, pour `i = n - 1`, aucune : au total $(n-1)+(n-2)+\cdots+1 = \dfrac{n(n-1)}{2}$ comparaisons. Le coût est **quadratique** (de l'ordre de n^2). Pour $n = 10^6$: $\dfrac{10^6 \times (10^6-1)}{2} \approx 5 \times 10^{11}$ comparaisons.

4. `milieu = 3`, `gauche = [5, 2, 6]`, `droite = [1, 4, 3]`.

- inversions de `gauche` : $(5, 2)$, soit 1 ;
- inversions de `droite` : $(4, 3)$, soit 1 ;
- inversions croisées : 5 est plus grand que $1, 4, 3$; 2 que 1 ; 6 que $1, 4, 3$: $3 + 1 + 3 = 7$.

Total : $1 + 1 + 7 = \mathbf{9}$ inversions (`inversions_naif([5, 2, 6, 1, 4, 3])` renvoie bien 9).

5. a) Une inversion croisée ne dépend que des *valeurs* présentes dans chaque moitié (un élément de `gauche` plus grand qu'un élément de `droite`), pas de leur ordre à l'intérieur de la moitié : trier `gauche` et `droite` ne change donc pas leur nombre. **b)** Si `droite[j] < gauche[i]`, comme `gauche` est triée, on a aussi `droite[j] < gauche[i] <= gauche[i+1] <= ... <= gauche[len(gauche)-1]` : `droite[j]` est plus petit que *tous* les éléments restants de `gauche`, soit `len(gauche) - i` éléments. Les éléments `gauche[0]`, …, `gauche[i-1]`, déjà placés, sont `<= droite[j]` : ils ne forment pas d'inversion avec lui.

6.

```
def fusion_compte(gauche, droite):
    resultat = []                                  # INITIALISER
    croisees = 0                                   # INITIALISER
    i = j = 0                                      # INITIALISER
    while i < len(gauche) and j < len(droite):     # PARCOURIR
        if gauche[i] <= droite[j]:                 # TESTER
            resultat.append(gauche[i])             # METTRE A JOUR
            i = i + 1
        else:
            resultat.append(droite[j])             # METTRE A JOUR
            croisees = croisees + len(gauche) - i  # METTRE A JOUR : inversions croisees
            j = j + 1
    resultat = resultat + gauche[i:]
    resultat = resultat + droite[j:]
    return resultat, croisees                      # RENVOYER
```

Par exemple `fusion_compte([2, 5, 6], [1, 3, 4])` renvoie `([1, 2, 3, 4, 5, 6], 7)` : on retrouve les 7 inversions croisées de la question 4.

7.

```
def tri_compte(t):
    if len(t) <= 1:                                # CAS DE BASE
        return t, 0
    milieu = len(t) // 2                           # DIVISER
    gauche, inv_g = tri_compte(t[:milieu])         # APPEL RECURSIF
    droite, inv_d = tri_compte(t[milieu:])         # APPEL RECURSIF
    fusionnee, inv_c = fusion_compte(gauche, droite)  # COMBINER
    return fusionnee, inv_g + inv_d + inv_c        # RENVOYER
```

Diviser : couper `t` en deux moitiés (`t[:milieu]` et `t[milieu:]`) ; **régner** : les deux appels récursifs, qui trient chaque moitié et comptent ses inversions ; **combiner** : `fusion_compte`, qui fusionne et compte les inversions croisées, puis la somme des trois nombres. `tri_compte([5, 2, 6, 1, 4, 3])` renvoie `([1, 2, 3, 4, 5, 6], 9)`.

8. Déroulement de `tri_compte([3, 1, 2])` (`milieu = 1`) :

- `tri_compte([3])` : condition d'arrêt, renvoie `([3], 0)` ;
- `tri_compte([1, 2])` (`milieu = 1`) appelle `tri_compte([1])`, qui renvoie `([1], 0)`, et `tri_compte([2])`, qui renvoie `([2], 0)` ; `fusion_compte([1], [2])` renvoie `([1, 2], 0)` ; l'appel renvoie `([1, 2], 0)` ;
- `fusion_compte([3], [1, 2])` : $1 < 3$, on place 1 et on ajoute $1 - 0 = 1$; $2 < 3$, on place 2 et on ajoute 1 ; puis on recopie 3. Elle renvoie `([1, 2, 3], 2)`.

L'appel initial renvoie `([1, 2, 3], 0 + 0 + 2)`, soit `([1, 2, 3], 2)` : les inversions sont $(3, 1)$ et $(3, 2)$.

9. Terminaison : le variant est la longueur de la liste. Dans le cas récursif, `len(t) >= 2`, donc les deux moitiés sont strictement plus courtes que `t` ; cet entier positif décroît strictement et finit par valoir 0 ou 1 : la condition d'arrêt. **Coût** : comme le tri fusion, $\log_2 n$ niveaux d'appels, et à chaque niveau des fusions de coût total de l'ordre de n : coût de l'ordre de $n \log_2 n$. Pour $n = 10^6$: environ $10^6 \times 20 = 2 \times 10^7$ opérations, contre 5×10^{11} pour la méthode directe, soit environ 25 000 fois moins.

Critiquer une réponse d'IA

★★ **Exercice 11** *Une recherche dichotomique écrite par un assistant d'IA* ↑énoncé p. 355

1. Pour `12` : `(0, 4)`, `milieu = 2`, `tab[2] = 12` → `True` dès le premier appel. Pour `5` : `(0, 4)` milieu 2 (12, on va à gauche) → `(0, 2)` milieu 1 (8, à gauche) → `(0, 1)` milieu 0 (3, à droite) → `(1, 1)` milieu 1 (8, à gauche) → `(1, 1)` milieu 1… L'appel `recherche(tab, 5, 1, 1)` se rappelle **lui-même** indéfiniment ; en console : `RecursionError: maximum recursion depth exceeded`. La réponse est donc **fausse** : toute valeur absente plus petite que le dernier élément provoque cette boucle sans fin (seule une valeur absente plus grande que tous les éléments, qui part toujours à droite, donne bien `False`). Les valeurs présentes, elles, sont toujours trouvées.

2. Dans la branche « moitié gauche », l'appel garde `milieu` dans l'intervalle alors qu'on vient de le tester : l'intervalle `(1, 1)` ne rétrécit plus (le **variant** `fin - debut` ne décroît pas). Ligne corrigée :

```
        return recherche(tab, cible, debut, milieu - 1)  # moitie gauche
```

Avec cette version, `recherche(tab, 5, 0, 4)` renvoie `False` et `recherche(tab, 12, 0, 4)` renvoie toujours `True`.

3. Tester la fonction sur une **valeur absente** (pas seulement sur une valeur présente, qui réussit souvent « par chance ») : une valeur absente force la descente jusqu'à l'intervalle vide et révèle immédiatement les erreurs de bornes. Réflexe du cours : `fin = milieu - 1` ou `debut = milieu + 1`, jamais `milieu` seul.

S'entraîner à l'oral

★★ **Exercice 12** *Expliquer en deux minutes* ↑énoncé p. 355

▶ **Solution.** Pas de texte à apprendre par cœur : voici les **éléments attendus** pour chaque sujet. L'ordre, les mots et l'exemple peuvent être différents ; l'explication est réussie si ces idées y sont, justes et reliées entre elles.

Sujet 1.

- **Diviser** : couper le problème en sous-problèmes de même nature, plus petits ; **régner** : les résoudre récursivement (condition d'arrêt : une liste de 0 ou 1 élément est triée) ; **combiner** : fusionner les solutions.
- Exemple : $[5, 2, 4, 1] \to [5, 2]$ et $[4, 1] \to [2, 5]$ et $[1, 4] \to$ fusion $[1, 2, 4, 5]$.
- La fusion de deux listes triées compare leurs premiers éléments et prend le plus petit : son coût est proportionnel à la longueur totale.
- Piège : oublier l'étape « combiner », où se trouve souvent l'essentiel du travail.

Sujet 2.

- Tri par insertion : dans le pire cas, de l'ordre de n^2 comparaisons (coût **quadratique**).
- Tri fusion : environ $\log_2 n$ niveaux de découpage, chacun coûte de l'ordre de n : coût en $n \log_2 n$.
- En chiffres, pour $n = 10^6$: $n^2 = 10^{12}$ contre $n \log_2 n \approx 2 \times 10^7$, soit environ 50 000 fois moins d'opérations.
- Piège : sur une petite liste, ou une liste presque triée, l'insertion reste rapide ; l'écart se voit quand n devient grand.

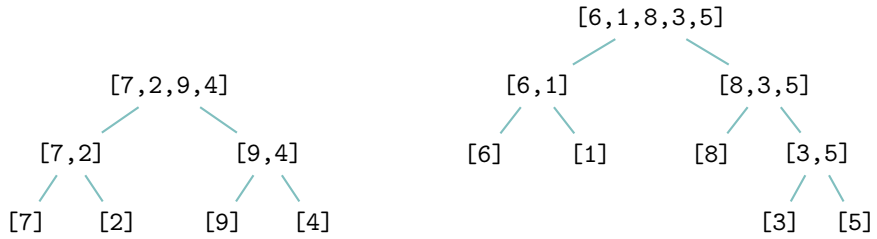
Sujet 3.

- On compare à l'élément du **milieu** et on ne garde qu'**une** moitié : un seul sous-problème, et rien à combiner.
- Exemple : chercher 11 dans $[1, 3, 5, 7, 9, 11, 13]$: milieu $7 < 11$, on garde $[9, 11, 13]$, milieu 11 : trouvé en 2 étapes.
- Coût : environ $\log_2 n$ étapes (une vingtaine pour un million de valeurs), contre n pour une recherche séquentielle.
- Le tri est indispensable : sans lui, rien ne permet d'éliminer une moitié ; piège des bornes (`milieu - 1`, `milieu + 1`) pour que la boucle termine.

Entraînement

Exercice 13 *Arbres d'appels du tri fusion en série* ↑énoncé p. 356

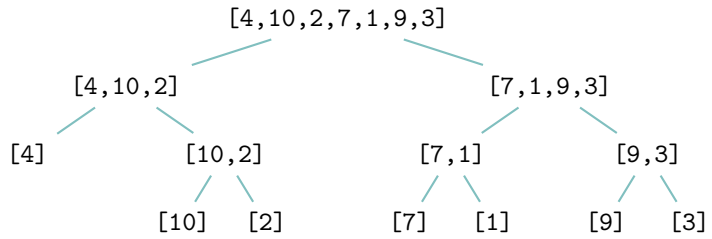
▶ **Solution.** **a)** et **b)** :



a) 7 appels. Remontée : `[7]` & `[2]` → `[2,7]` ; `[9]` & `[4]` → `[4,9]` ; `[2,7]` & `[4,9]` → `[2,4,7,9]`.

b) 9 appels. Remontée : `[6]` & `[1]` → `[1,6]` ; `[3]` & `[5]` → `[3,5]` ; `[8]` & `[3,5]` → `[3,5,8]` ; `[1,6]` & `[3,5,8]` → `[1,3,5,6,8]`.

c)



13 appels. Remontée : `[10]` & `[2]` → `[2,10]` ; `[4]` & `[2,10]` → `[2,4,10]` ; `[7]` & `[1]` → `[1,7]` ; `[9]` & `[3]` → `[3,9]` ; `[1,7]` & `[3,9]` → `[1,3,7,9]` ; enfin `[2,4,10]` & `[1,3,7,9]` → `[1,2,3,4,7,9,10]`. *(Pour une liste de n éléments, on trouve toujours $2n - 1$ appels : n feuilles et $n - 1$ fusions.)*

Exercice 14 *Fusions à la main en série* ↑énoncé p. 356

▶ **Solution.** **a)** 2 contre 1 → 1 ; 2 contre 6 → 2 ; 5 contre 6 → 5 ; 8 contre 6 → 6 ; 8 contre 7 → 7 ; la seconde liste est épuisée, on recopie 8. Résultat `[1, 2, 5, 6, 7, 8]`, **5 comparaisons**.

b) 1 contre 4 → 1 ; 2 contre 4 → 2 ; 3 contre 4 → 3 ; la première liste est épuisée, on recopie `[4, 5]`. Résultat `[1, 2, 3, 4, 5]`, **3 comparaisons**.

c) 3 contre 1 → 1 ; 3 contre 4 → 3 ; 9 contre 4 → 4 ; 9 contre 10 → 9 ; la première liste est épuisée, on recopie `[10, 11]`. Résultat `[1, 3, 4, 9, 10, 11]`, **4 comparaisons**.

▷ **Erreur fréquente.** En b), continuer à comparer 4 et 5 à une liste vide : dès qu'une liste est épuisée, on recopie le reste de l'autre, déjà trié, sans aucune comparaison.

CORRIGÉS DES TP

TP 1 Pourquoi le tri fusion coûte $n \log_2 n$?

↑ énoncé p. 359

Remarque Le corrigé, lu d'un trait, **est** la démonstration rédigée du coût du tri fusion.

Étape 1 — le coût d'une fusion

▶ **Solution.** **1.** À la fin, `resultat` contient **tous** les éléments de `gauche` et `droite`, chacun une fois : elle a donc $p+q$ éléments. Comme chaque `append` ajoute un élément et qu'il n'y a pas d'autre source d'éléments, il y a exactement $p+q$ appels à `append` (et au plus $p+q-1$ comparaisons, une par tour de la première boucle).

2. Si les deux listes totalisent n éléments ($p+q=n$), la fusion effectue de l'ordre de n opérations : son coût est **linéaire**.

Propriété 1 — Coût de la fusion

Fusionner deux listes triées totalisant n éléments coûte de l'ordre de n opérations (coût linéaire).

2 Étape 2 — l'arbre des appels récursifs

▶ **Solution.** **1.** À chaque descente, chaque liste se scinde en deux moitiés : le nombre de sous-listes **double** et leur taille est **divisée par deux**.

Niveau	Nombre de sous-listes	Taille de chacune
0	1	n
1	2	$n/2$
2	4	$n/4$
i	2^i	$n/2^i$

2. Au niveau i, il y a 2^i sous-listes, chacune de taille $n/2^i$. Les fusions de ce niveau reconstruisent ces sous-listes ; d'après l'étape 1, chaque fusion coûte proportionnellement à la taille de son résultat. Le coût total du niveau est donc

$$2^i \times \frac{n}{2^i} = n.$$

Ce coût vaut n à chaque niveau, indépendamment de i : le 2^i (de plus en plus de fusions) et le $n/2^i$ (des fusions de plus en plus petites) se compensent exactement.

3. On s'arrête quand les sous-listes ont une taille de 1, c'est-à-dire quand

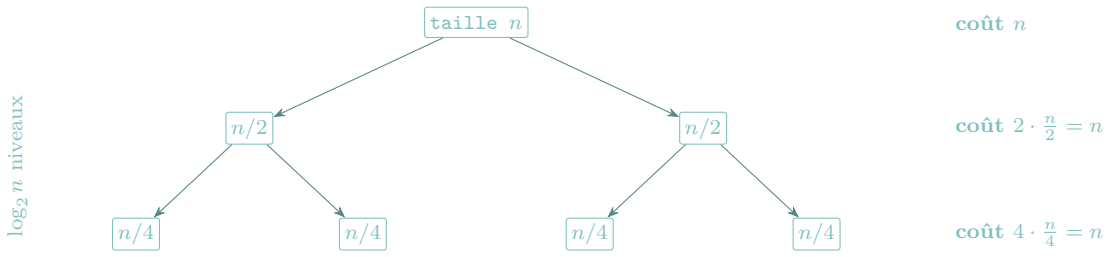
$$\frac{n}{2^i} = 1 \iff 2^i = n \iff i = \log_2 n.$$

Il y a donc $\log_2 n$ niveaux de fusion (on avait posé $n = 2^k$, soit $k = \log_2 n$ niveaux).

4. Le coût total est la somme des coûts de tous les niveaux :

$$\text{coût total} = \underbrace{n}_{\text{par niveau}} \times \underbrace{\log_2 n}_{\text{nombre de niveaux}} = \boxed{n \log_2 n}.$$

Visuellement, chaque *ligne* de l'arbre ci-dessous coûte n, et il y a $\log_2 n$ lignes :



. . . . . (jusqu'aux listes de taille 1) . . . . .

Propriété 2 — Coût du tri fusion

Le tri fusion d'une liste de n éléments a un coût de l'ordre de $n \log_2 n$.

3 Étape 3 — interpréter le résultat

▶ **Solution. 1.** Pour $n = 1024 = 2^{10}$: $\log_2 n = 10$, donc $n \log_2 n = 1024 \times 10 \approx 10^4$ (dix mille), tandis que $n^2 = 1024^2 \approx 10^6$ (un million). Le tri fusion fait donc environ **100 fois** moins d'opérations ici ; et l'écart grandit avec n (rapport $n/\log_2 n$).

2. Le raisonnement n'a utilisé **aucune hypothèse sur l'ordre initial** des éléments : le découpage en deux et le coût des fusions ne dépendent que des *tailles*. Le coût $n \log_2 n$ est donc garanti **dans tous les cas**, y compris le pire. Le tri rapide, lui, est en $n \log_2 n$ *en moyenne* mais peut chuter à n^2 sur des entrées défavorables : le tri fusion offre une **garantie**, ce qui est précieux quand on ne maîtrise pas les données.

Remarque — *La récurrence, hors programme* En notant $T(n)$ le coût, l'algorithme donne $T(n) = 2\,T(n/2) + c\,n$. En développant :

$$T(n) = 2T(n/2) + cn = 4T(n/4) + 2cn = 8T(n/8) + 3cn = \cdots = 2^i\,T(n/2^i) + i\,cn.$$

Pour $i = \log_2 n$, on a $n/2^i = 1$ (coût constant), d'où $T(n) = n \cdot T(1) + cn \log_2 n$, soit un coût de l'ordre de $n \log_2 n$. On retrouve exactement le comptage par niveaux.

TP 2 Tri rapide (*Quicksort*)

↑ énoncé p. 361

2 Implémentation du tri rapide

Exercice 1 *La fonction* `partition`

```
def partition(tab, bas, haut):
    pivot = tab[haut]           # le pivot est le dernier element
    i = bas - 1                 # frontiere des elements <= pivot
    for j in range(bas, haut):
        if tab[j] <= pivot:
            i = i + 1
            temp = tab[i]               # echange de tab[i] et tab[j]
            tab[i] = tab[j]
            tab[j] = temp
    temp = tab[i + 1]           # pivot a sa place : echange
    tab[i + 1] = tab[haut]      # de tab[i + 1] et tab[haut]
    tab[haut] = temp
```

```
    return i + 1                # indice definitif du pivot
```

Sur `[10, 7, 8, 9, 1, 5]` (pivot `5`) : seul `1` est $\leqslant 5$; il est échangé avec `10`, ce qui donne `[1, 7, 8, 9, 10, 5]`, puis le pivot vient en position `1` : `[1, 5, 8, 9, 10, 7]`, et la fonction renvoie `1`. C'est exactement l'exemple de l'énoncé.

Exercice 2 *La fonction récursive* `tri_rapide`

```
def tri_rapide(tab, bas, haut):
    if bas < haut:              # sinon : 0 ou 1 element, deja trie
        p = partition(tab, bas, haut)
        tri_rapide(tab, bas, p - 1)    # regner a gauche du pivot
        tri_rapide(tab, p + 1, haut)   # regner a droite du pivot
```

La **condition d'arrêt** est `bas >= haut` : le sous-tableau a 0 ou 1 élément, il n'y a rien à faire. Les deux appels portent sur les éléments à gauche et à droite du pivot, **sans** le pivot, qui est déjà à sa place définitive : la taille diminue donc d'au moins 1 à chaque appel (variant). Le test affiche `Tableau trie : [1, 5, 7, 8, 9, 10]`.

Autre méthode, qui renvoie une nouvelle liste au lieu de trier `tab` en place : voir l'exercice 8.

Exercice 3 *Rappel : le tri fusion*

Diviser : `milieu`, `gauche = tab[:milieu]`, `droite = tab[milieu:]`. **Régner** : `tri_fusion(gauche)` et `tri_fusion(droite)`. **Combiner** : les trois boucles `while`, qui fusionnent `gauche` et `droite` dans `tab`. Condition d'arrêt : `len(tab) <= 1` (on ne fait rien). Remarque : cette version modifie bien `tab`, mais elle **recopie** les moitiés dans `gauche` et `droite` ; elle n'est donc pas « en place » au sens de la mémoire, contrairement au tri rapide.

3 Comparaison des performances

Exercice 4 *Mesurer et tracer*

On construit la liste **par compréhension** : un entier tiré au hasard entre `0` et `n`, répété `n` fois.

```
def tableau_aleatoire(n):
    return [random.randint(0, n) for i in range(n)]
```

Autre méthode : on part d'une liste vide et on ajoute les `n` valeurs une à une.

```
def tableau_aleatoire(n):
    tab = []
    for i in range(n):
        tab.append(random.randint(0, n))
    return tab
```

Résultats obtenus avec le programme du TP (tableaux aléatoires) ; on a ajouté pour comparaison le tri intégré `list.sort()` :

n	tri rapide	tri fusion	`list.sort()`
100	0,09 ms	0,18 ms	0,003 ms
1 000	1,5 ms	2,4 ms	0,06 ms
10 000	21 ms	32 ms	0,9 ms
100 000	259 ms	400 ms	12 ms

Le graphique `matplotlib` montre deux courbes presque droites (légèrement incurvées vers le haut), celle du tri rapide sous celle du tri fusion. Quand n est multiplié par 10, le temps est multiplié par un peu plus de 10 (de 21 à 259 ms, soit $\times 12$) : c'est la signature d'un coût en $n \log_2 n$, et non en n^2 (on aurait $\times 100$).

4 Analyser les résultats

Exercice 5 *Lire le graphique*

- Petites tailles : le tri rapide est le plus rapide (environ deux fois plus que le tri fusion pour $n = 100$), mais tous les temps sont négligeables.
- Grandes tailles : le tri rapide reste devant, avec environ un tiers de temps en moins (259 ms contre 400 ms pour $n = 100\,000$). Sur des données **aléatoires**, les deux sont en $n \log_2 n$; la différence vient des « constantes » (le tri fusion recopie les moitiés à chaque appel).

Exercice 6 *L'importance du pivot*

Pour tester les stratégies, on place le pivot choisi en dernière position, puis on réutilise `partition` :

```
def tri_rapide_pivot(tab, bas, haut, choix):
    if bas < haut:
        if choix == "premier":
            k = bas
        elif choix == "milieu":
            k = (bas + haut) // 2
        elif choix == "aleatoire":
            k = random.randint(bas, haut)
        else:                                   # "dernier"
            k = haut
        temp = tab[k]                           # le pivot choisi va a la fin :
        tab[k] = tab[haut]                      # echange de tab[k] et tab[haut]
        tab[haut] = temp
        p = partition(tab, bas, haut)
        tri_rapide_pivot(tab, bas, p - 1, choix)
        tri_rapide_pivot(tab, p + 1, haut, choix)
```

Mesures pour $n = 800$ (médiane de 5 essais) :

pivot	tableau aléatoire	tableau déjà trié
dernier élément	1,2 ms	44 ms
premier élément	1,2 ms	16 ms
élément du milieu	1,2 ms	1,0 ms
élément aléatoire	1,6 ms	1,5 ms

Sur un tableau **déjà trié** avec le pivot « dernier élément », le pivot est à chaque fois le **plus grand** : la partition laisse tous les autres éléments d'un seul côté. On ne coupe plus en deux, on enlève un seul élément à chaque appel : $n-1+n-2+\cdots+1 = \frac{n(n-1)}{2}$ comparaisons, soit un coût en n^2. Pour $n = 500$, cela fait 124 750 comparaisons sur le tableau trié, contre quelques milliers seulement sur un tableau aléatoire (par exemple 4 786). De plus, la profondeur de récursion vaut n : dès $n = 1\,000$, Python s'arrête sur `RecursionError: maximum recursion depth exceeded`. Le pivot « premier élément » a le même défaut. Les pivots « milieu » et « aléatoire » évitent ce piège : avec eux, un tableau trié de 100 000 éléments est trié en 0,2 à 0,3 s.

Exercice 7 *Questions de réflexion*

- Le pivot décide de la **coupe** : un pivot proche de la médiane coupe le tableau en deux moitiés équilibrées ($\log_2 n$ niveaux, coût $n \log_2 n$) ; un pivot extrême (le minimum ou le maximum) ne retire qu'un élément (n niveaux, coût n^2).
- On préfère le tri fusion quand il faut une **garantie** au pire cas ($n \log_2 n$ toujours), par exemple sur des données qui peuvent être déjà triées ou fournies par un adversaire, ou quand on veut un tri **stable** (deux éléments égaux gardent leur ordre) : le tri fusion l'est car, en cas d'égalité, la fusion prend l'élément de *gauche* (test `gauche[i] <= droite[j]`) ; le tri rapide ne l'est pas.
- Le tri rapide trie **sur place** : il ne recopie pas de sous-tableaux et n'alloue pas de mémoire supplémentaire, il fait moins d'opérations par élément (« constantes cachées » plus petites) et parcourt le tableau de manière contiguë, ce que la mémoire cache du processeur favorise. Avec un bon pivot, le pire cas devient très improbable.

Exercice 8 *Pour aller plus loin : un tri rapide qui renvoie une nouvelle liste*

1. La partition s'écrit avec deux listes construites **par compréhension**, les éléments `<=` pivot d'un côté, les éléments `>` pivot de l'autre.

```
def tri_rapide_liste(t):
    if len(t) <= 1:                    # condition d'arret
        return t
    pivot = t[len(t) - 1]              # le pivot est le dernier element
    reste = t[:len(t) - 1]
    petits = [x for x in reste if x <= pivot]
    grands = [x for x in reste if x > pivot]
    return tri_rapide_liste(petits) + [pivot] + tri_rapide_liste(grands)
```

`tri_rapide_liste([10, 7, 8, 9, 1, 5])` renvoie `[1, 5, 7, 8, 9, 10]`, et la liste de départ n'est pas modifiée. Variant : la longueur de `t`, car `petits` et `grands` ne contiennent pas le pivot.

2. Cette version est plus **lisible** : elle dit directement « les petits, le pivot, les grands ». Mais chaque appel **construit de nouvelles listes** (`reste`, `petits`, `grands` et le résultat concaténé) : à chaque niveau de récursion, on recopie à nouveau tous les éléments, et l'on utilise une mémoire proportionnelle à n en plus du tableau. `tri_rapide` échange les valeurs dans `tab` lui-même : il ne recopie rien, c'est l'avantage « en place » du tri rapide.

TP 3 Le *Tim sort*, le tri de Python ↑énoncé p. 364

2 Les briques de base

Exercice 1 *Le tri par insertion sur un sous-tableau*

```
def tri_insertion(tab, debut, fin):
    for i in range(debut + 1, fin + 1):
        cle = tab[i]                   # element a inserer
        j = i - 1
        while j >= debut and tab[j] > cle:
            tab[j + 1] = tab[j]        # on decale vers la droite
            j = j - 1
        tab[j + 1] = cle               # on insere a la place liberee
```

On compare à `cle` (l'élément à insérer), on décale `tab[j]` d'un cran vers la droite, puis on dépose `cle` dans la case libérée. Le test affiche `Tableau trie : [5, 6, 11, 12, 13]`. Le test `j >= debut` (et non `j >= 0`) garantit qu'on ne déborde pas du sous-tableau.

Exercice 2 *Fusionner deux sous-tableaux triés*

Les quatre trous se complètent par `gauche[i]`, `droite[j]`, `gauche[i]`, `droite[j]` dans cet ordre (le premier `while` prend la plus petite des deux têtes, les deux suivants recopient le reste) :

```
    while i < nb_gauche and j < nb_droite:
        if gauche[i] <= droite[j]:
            tab[k] = gauche[i]
            i = i + 1
        else:
            tab[k] = droite[j]
            j = j + 1
        k = k + 1
    while i < nb_gauche:
        tab[k] = gauche[i]
        i = i + 1
        k = k + 1
    while j < nb_droite:
        tab[k] = droite[j]
        j = j + 1
        k = k + 1
```

Le test affiche `Tableau fusionne : [5, 6, 7, 11, 12, 13]`. Le `<=` prend l'élément de gauche en cas d'égalité : le tri obtenu est **stable**, comme le vrai Tim sort.

3 Assembler le Tim sort

Exercice 3 *La fonction* `tim_sort`

```
def tim_sort(tab, taille_bloc=64):
    n = len(tab)
    for debut_bloc in range(0, n, taille_bloc):      # 1. trier chaque bloc
        fin_bloc = min(debut_bloc + taille_bloc - 1, n - 1)
        tri_insertion(tab, debut_bloc, fin_bloc)
    taille = taille_bloc                             # 2. fusionner deux a deux
    while taille < n:
        for debut in range(0, n, 2 * taille):
            milieu = min(debut + taille - 1, n - 1)
            fin = min(debut + 2 * taille - 1, n - 1)
            if milieu < fin:
                fusion_en_place(tab, debut, milieu, fin)
        taille = 2 * taille
    return tab
```

Chaque bloc `tab[debut_bloc..fin_bloc]` est trié par insertion, puis les blocs sont fusionnés deux à deux : `tab[debut..milieu]` et `tab[milieu+1..fin]`, avec des blocs de taille `taille` qui double à chaque tour (64, 128, 256…). Le test affiche `Trie : [5, 7, 8, 12, 15, 19, 21, 22, 23, 100]` ; sur ce petit tableau ($10 < 64$), seule l'étape d'insertion travaille. Le jeu de tests force les fusions avec `taille_bloc` égal à 4 sur 200 tableaux aléatoires.

Lien avec le cours : c'est un tri fusion « de bas en haut » (itératif), dont la condition d'arrêt aurait été remplacée par « bloc de 64 éléments, trié par insertion ».

4 Comparaison et analyse

Exercice 4 *Trois tris en course*

Résultats obtenus avec le programme du TP (tableaux aléatoires) ; la dernière colonne est le tri intégré `list.sort()` :

n	Tim sort	tri rapide	tri fusion	`list.sort()`
100	0,20 ms	0,09 ms	0,18 ms	0,003 ms
1 000	3,1 ms	1,5 ms	2,4 ms	0,06 ms
10 000	43 ms	21 ms	32 ms	0,9 ms
100 000	479 ms	259 ms	400 ms	12 ms

Exercice 5 *Lire les résultats*

- Sur des données **aléatoires**, le tri rapide est le plus rapide à toutes les tailles ; notre Tim sort est le plus lent (10 à 35 % de plus que le tri fusion) : sur des données en désordre, les blocs triés par insertion coûtent cher (64 éléments par bloc, coût quadratique dans chaque bloc). Les trois restent en $n \log_2 n$ (temps multiplié par environ 11 à 17 quand n est multiplié par 10 : un peu plus que 10, mais loin du $\times 100$ d'un coût quadratique).
- Sur un tableau **déjà trié**, les rôles s'inversent :

n (tableau trié)	Tim sort	tri fusion	tri rapide
500	0,29 ms	0,87 ms	17 ms
1 000	0,77 ms	2,0 ms	`RecursionError`
100 000	206 ms	310 ms	`RecursionError`

Le Tim sort devient le plus rapide : le tri par insertion d'un bloc déjà trié ne fait qu'un passage (coût linéaire), et chaque fusion se termine vite. Le tri rapide, lui, tombe dans son pire cas (n^2) puis dépasse la profondeur de récursion autorisée.

Exercice 6 *Questions de réflexion*

- Sur un **petit** bloc, le tri par insertion est très rapide (peu d'opérations, pas d'appel récursif, pas de recopie), et il est en $O(n)$ sur un bloc presque trié : il est meilleur que le tri fusion sur quelques dizaines d'éléments, et on garde la fusion pour assembler les blocs.
- Des *runs* trop petits multiplient les étapes de fusion ; trop grands, ils rendent l'insertion (quadratique) très coûteuse. Mesures sur un même tableau aléatoire de 100 000 éléments (une seule mesure par valeur, d'où un écart avec le tableau précédent) :

`taille_bloc`	1	8	32	64	256	1 024
temps	766 ms	595 ms	600 ms	663 ms	1 219 ms	3 642 ms

Il y a un juste milieu (ici entre 8 et 64) ; le vrai Tim sort choisit une taille entre 32 et 64.
- Le vrai Tim sort repère les portions **déjà triées** des données, de longueur quelconque, et les utilise telles quelles comme blocs : sur des données réelles (souvent partiellement triées : une liste déjà triée à laquelle on ajoute quelques éléments, des relevés horodatés…), il évite presque tout le travail. Sur un tableau entièrement trié, il se contente d'un seul passage : coût n au lieu de $n \log_2 n$.

TP 4 Faire tourner une image

↑énoncé p. 368

3 À vous de jouer

Exercice 1 *Sur papier : une petite image* 4×4

Après un quart de tour horaire, la première *colonne* lue de bas en haut devient la première *ligne* :

13	9	5	1
14	10	6	2
15	11	7	3
16	12	8	4

Avec la méthode du TP : on tourne d'abord chaque quadrant 2×2 sur lui-même (`1 2 / 5 6` devient `5 1 / 6 2`, etc.), puis on permute les quatre blocs (HG → HD → BD → BG → HG). On obtient le même tableau.

Exercice 2 *Compléter la fonction récursive*

```python
def rotation(image, ligne, col, taille):
    if taille == 1:              # condition d'arret : un seul pixel
        return
    demi = taille // 2
    rotation(image, ligne,        col,        demi)   # haut-gauche
    rotation(image, ligne,        col + demi, demi)   # haut-droite
    rotation(image, ligne + demi, col,        demi)   # bas-gauche
    rotation(image, ligne + demi, col + demi, demi)   # bas-droite
    for i in range(demi):
        for j in range(demi):
            hg = image[ligne + i][col + j]
            hd = image[ligne + i][col + demi + j]
            bd = image[ligne + demi + i][col + demi + j]
            bg = image[ligne + demi + i][col + j]
            image[ligne + i][col + demi + j]        = hg    # HD <- HG
            image[ligne + demi + i][col + demi + j] = hd    # BD <- HD
            image[ligne + demi + i][col + j]        = bd    # BG <- BD
            image[ligne + i][col + j]               = bg    # HG <- BG
```

Le quadrant haut-droite commence à la ligne `ligne` et à la colonne `col + demi` ; le quadrant bas-gauche à la ligne `ligne + demi` et à la colonne `col`. Dans la permutation horaire, le bloc haut-droite descend en bas-droite (`hd`) et le bloc bas-droite passe en bas-gauche (`bd`).

Les trois temps : **diviser** = calcul de `demi` (quatre quadrants) ; **régner** = les quatre appels récursifs ; **combiner** = la double boucle qui permute les blocs. La condition d'arrêt est `taille == 1` ; le variant est `taille`, divisé par 2 à chaque appel.

Exercice 3 *Lancer et enregistrer le résultat*

1. L'appel initial porte sur toute l'image : coin haut-gauche `(0, 0)`, côté `n`.

```python
rotation(image, 0, 0, n)
```

2. Le fichier `joconde_tournee.png` obtenu montre la Joconde couchée, la tête à droite. La rotation de l'image 256×256 prend environ 0,1 s en Python, et le résultat est identique, pixel par pixel, à celui de `im.rotate(-90)` de Pillow.

Programme complet sur la vraie image :

```python
from PIL import Image

im = Image.open("joconde.png").convert("RGB")
n = im.width
pixels = [p for p in im.getdata()]
image = [pixels[k*n:(k+1)*n] for k in range(n)]

rotation(image, 0, 0, n)          # appel initial : toute l'image

plat = []                          # on remet les pixels a plat
for ligne in image:
    for pixel in ligne:
        plat.append(pixel)
sortie = Image.new("RGB", (n, n))
sortie.putdata(plat)
sortie.save("joconde_tournee.png")
```

Exercice 4 *Pourquoi « en place » ?*

- **Aucune** image supplémentaire n'est créée : tous les pixels sont déplacés à l'intérieur du tableau `image`. Le coût en mémoire, en plus de l'image elle-même, est donc **constant** (il ne dépend pas de la taille de l'image), si l'on met à part la pile des appels.
- Les quatre pixels `hg`, `hd`, `bd`, `bg` sont stockés dans des **variables locales**, le temps d'un échange. Il y en a toujours exactement quatre, quelle que soit la taille de l'image : cette « réserve » est de taille fixe et ne remet pas en cause la mémoire constante. (La pile des appels, elle, ne contient jamais plus de $\log_2 n + 1$ appels imbriqués : 9 pour $n = 256$.)

Exercice 5 *Pour aller plus loin*

- On coupe le côté en deux à chaque appel : pour que les quatre quadrants soient toujours des carrés égaux et que l'on arrive exactement à des carrés de 1 pixel, il faut pouvoir diviser n par 2 jusqu'à 1, donc n doit être une puissance de 2. Sinon, il faudrait gérer des quadrants de tailles différentes (par exemple compléter l'image par des pixels blancs jusqu'à la puissance de 2 suivante, puis rogner).
- Pour le sens **anti-horaire**, on fait tourner les blocs dans l'autre sens (HG → BG → BD → HD → HG) ; les quatre appels récursifs, eux, restent les mêmes (appliquer cette version puis la version horaire redonne l'image de départ) :

```python
            image[ligne + i][col + j]                = hd   # HG <- HD
            image[ligne + demi + i][col + j]         = hg   # BG <- HG
            image[ligne + demi + i][col + demi + j] = bg   # BD <- BG
            image[ligne + i][col + demi + j]         = bd   # HD <- BD
```

- *Bonus* : à chaque niveau de récursion, chaque pixel est déplacé une fois ; il y a $\log_2 n$ niveaux, d'où $n^2 \log_2 n$ écritures de pixels (pour $n = 256$: $256^2 \times 8 = 524\,288$). Une simple recopie dans une nouvelle image coûterait n^2 écritures, mais une deuxième image en mémoire. **Compromis** : on accepte un peu plus de *temps* pour économiser la *mémoire*.
  Autre méthode (la « simple recopie », sans diviser pour régner) : voir l'exercice 6.

Exercice 6 *La simple recopie*

1. Le pixel ligne `0`, colonne `0` de l'image tournée est `13` : il vient de la ligne `3`, colonne `0`. Le pixel ligne `1`, colonne `3` est `2` : il vient de la ligne `0`, colonne `1`. En général, le pixel ligne `i`, colonne `j` de l'image tournée est le pixel ligne `n - 1 - j`, colonne `i` de l'image de départ (la colonne `i` de départ, lue de bas en haut, devient la ligne `i`).

2. On construit la nouvelle image **par compréhension**, ligne par ligne :

```python
def rotation_copie(image):
    n = len(image)
    return [[image[n - 1 - j][i] for j in range(n)] for i in range(n)]
```

Sur l'image 4×4, on obtient `[[13, 9, 5, 1], [14, 10, 6, 2], [15, 11, 7, 3], [16, 12, 8, 4]]`, comme avec la fonction récursive (exercice 1).

3. La recopie fait une seule passe : n^2 écritures, contre $n^2 \log_2 n$ pour la version récursive. Mais elle crée une **deuxième image** de n^2 pixels en mémoire, alors que la version récursive n'utilise qu'une mémoire constante en plus de l'image (les 4 pixels de l'échange). C'est le compromis temps / mémoire de l'exercice 5.

Réponses des questions de la fiche de révision

↑ questions p. 373

Se tester

1. diviser, régner (résoudre récursivement), combiner (§ I).
2. `[1, 2, 3, 6, 9]` (§ III.A).
3. environ 10, car $2^{10} = 1024 \geq 1000$ (§ II).
4. `[1, 2, 3, 6]` : le `9` restant est perdu (§ III.A).
5. le parcours (coût n) ; trier coûte déjà $n \log_2 n$ (§ II et IV).

Et avant ?

1. `SELECT titre FROM film ORDER BY note DESC;` (ch. 6, § V.D).
2. 2 : B attend de l'instant 1 à l'instant 3 que A ait fini (ch. 4, § VI.A).
3. chaque appel en attente occupe un cadre dans la pile d'exécution (ch. 1, § V).

Graphes

8

CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Sans lever le crayon

↑ énoncé p. 385

Exercice 1 *Trois dessins*

1. (a) Possible, par exemple `A-B-C-D-A-C-E-D-B` ; tous les tracés possibles partent de `A` et finissent en `B`, ou l'inverse. (b) Impossible (aucun tracé n'existe). (c) Possible, par exemple `P-Q-M-R-S-M-P`, et l'on revient au départ.

2. En partant de `E` (ou de `C`, `D`), on se retrouve toujours bloqué avec des traits non tracés : le camarade a raison.

3. Oui, on revient au point de départ, et cela marche en partant de n'importe quel point.

Exercice 2 *Compter les traits*

4. Tableau complété :

Dessin	traits en chaque point	points impairs	possible ?	retour ?
(a)	`A` 3, `B` 3, `C` 4, `D` 4, `E` 2	2	oui (de `A` à `B`)	non
(b)	`A` 3, `B` 3, `C` 3, `D` 3, `O` 4	4	non	non
(c)	`M` 4, les quatre autres 2	0	oui	oui

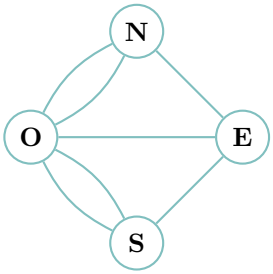
5. On utilise **deux** traits : un pour arriver, un pour repartir. Un point qui n'est ni le départ ni l'arrivée est traversé un certain nombre de fois, en consommant deux traits à chaque passage : il doit avoir un nombre **pair** de traits.

6. Règle conjecturée : 0 **point impair** : possible, et l'on revient au départ (on peut partir de n'importe où) ; 2 **points impairs** : possible, en partant de l'un et en arrivant à l'autre ; **plus de** 2 : impossible. (C'est le théorème d'Euler, valable pour un dessin d'un seul tenant, c'est-à-dire un graphe **connexe** : Euler démontre en 1736 que la condition est nécessaire, et Carl Hierholzer en 1873 qu'elle est suffisante.)

Exercice 3 *Les sept ponts de Königsberg*

7. Personne ne trouve : c'est normal.

8. Le schéma attendu (îles `O` et `E`, berges `N` et `S`) :



9. `O` : 5 ; `N` : 3 ; `S` : 3 ; `E` : 3. Quatre points impairs : la promenade est **impossible**, sans avoir à essayer tous les trajets.

10. Il suffit de relier **deux** des quatre terres (toutes sont « impaires ») : il ne reste alors que deux points impairs. Par exemple, un pont entre `N` et `S` : on part de `O` pour arriver en `E` (ou l'inverse). Avec un pont entre les deux îles, on part d'une berge pour arriver à l'autre. Pour pouvoir *revenir* à son point de départ, il faudrait deux nouveaux ponts.

Exercice 4 *Mettre des mots*

11. Propositions possibles puis mots du cours :

Notion	**Propositions fréquentes**	**Mot du cours**
points reliés par des traits	réseau, schéma, plan	**graphe**
un point / un trait	nœud, lien	**sommet** / **arête**
nombre de traits en un point	nombre de voisins	**degré**
remplacer la carte	simplifier, schématiser	**modéliser**

« Nœud » est aussi employé en informatique (arbres, réseaux).

12. Impossible. Chaque trait a deux extrémités : en additionnant les nombres de traits de tous les points, on compte chaque trait deux fois, donc on obtient un nombre **pair**. Une somme paire ne peut pas contenir un seul nombre impair : le nombre de points impairs est toujours **pair**.

Exercice 5 *Visiter tous les points*

13. `A-B-A-B-A-B-A-B-A-B` : depuis `A`, la plus petite lettre voisine est `B` ; depuis `B`, c'est `A`. Le gardien **tourne en rond** entre deux salles et ne verra jamais `C`, `D`, …, `G` : il ne se souvient pas des salles où il est déjà passé.

14. `A-B-C`, puis plus rien : en `C`, les deux voisines `A` et `B` sont cochées. Le gardien est **bloqué**, alors que `D`, `E`, `F` et `G` n'ont pas été visitées. Cocher évite de tourner en rond, mais ne suffit pas.

15. En `A`, le gardien avait aperçu `C` et `D` sans y aller : il aurait dû **s'en souvenir**.

16. Méthode attendue, avec deux choses à noter :

- on **coche** les salles déjà visitées ;
- on tient une **liste des salles à visiter** : celles que l'on a aperçues (voisines d'une salle visitée) mais pas encore visitées.

Au départ, la liste contient `A`. Tant que la liste n'est pas vide, on en retire une salle, on la visite et on la coche, puis on ajoute à la liste ses voisines qui ne sont ni cochées ni déjà dans la liste. La visite est **terminée quand la liste est vide**.

17. En retirant toujours la salle la plus **ancienne** de la liste :

Salle visitée	`A`	`B`	`C`	`D`	`E`	`F`	`G`
Liste à visiter ensuite	`B C D`	`C D`	`D`	`E F`	`F G`	`G`	(vide)

Ordre de visite : `A`, `B`, `C`, `D`, `E`, `F`, `G`. L'ordre dépend de la salle retirée à chaque étape : en retirant toujours la plus **récente**, on obtient `A`, `D`, `F`, `G`, `E`, `C`, `B`. Dans les deux cas, chaque salle est visitée une fois et une seule. On accepte toute méthode équivalente (par exemple : revenir sur ses pas jusqu'à la dernière salle qui a encore une voisine non cochée).

CORRIGÉS DES EXERCICES

> **Remarque**
> On donne **une** version simple de chaque programme ; d'autres sont possibles. Les ordres de parcours dépendent de l'ordre dans lequel on liste les voisins.

Vocabulaire et modélisation

Exercice 1 ↑ énoncé p. 405 **1.** Voisins de `B` : `A`, `C`, `D`. Degrés : `A` :2, `B` :3, `C` :3, `D` :3, `E` :3, `F` :2.

2. Par exemple `A -- B -- D -- E -- F` (longueur 4).

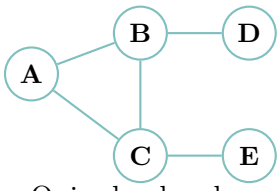
3. Oui, il est connexe : tout sommet est atteignable depuis n'importe quel autre (un seul morceau).

4. Par exemple le cycle `A -- B -- C -- A`, ou `B -- D -- E -- C -- B`.

▷ **Erreur fréquente.** Se tromper d'un degré sans s'en apercevoir : la somme des degrés vaut toujours le double du nombre d'arêtes ($2+3+3+3+3+2 = 16 = 2\times 8$), ce qui permet de se vérifier.

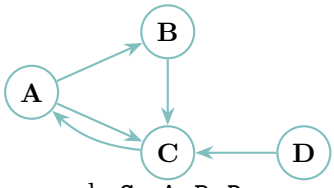
Exercice 2 ↑énoncé p. 405 **1.** Non orienté : « travaille avec » est une relation **symétrique**.

2.



3. « Qui a le plus de partenaires ? » = « quel sommet a le plus grand **degré** ? ». Ici Bilal et Chloé (degré 3).

Exercice 3 ↑énoncé p. 406 **1.**



2. Successeurs de `A` : `B`, `C`. Prédécesseurs de `C` : `A`, `B`, `D`.

3. `D`→`B` : oui (`D`→`C`→`A`→`B`). `B`→`D` : non (aucun arc n'arrive sur `D`). Un chemin peut exister dans un sens sans exister dans l'autre : c'est propre aux graphes **orientés**.

▷ **Erreur fréquente.** Répondre « oui » de `B` vers `D` parce que les sommets sont reliés par une suite d'arcs : dans un graphe orienté, un chemin doit suivre le **sens** des flèches.

4. Oui : `A`→`B`→`C`→`A` est un cycle orienté.

Représenter un graphe

Exercice 4 ↑énoncé p. 406 **1.** Sommets `A`,`B`,`C`,`D`,`E`,`F` ; arêtes `A-B, A-C, B-C, B-D, C-E, D-E, D-F, E-F` :

$$M = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

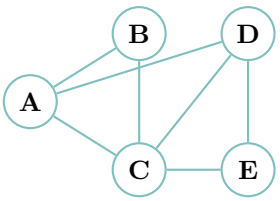
Elle est bien symétrique.

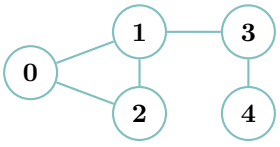
2. Démarche : à chaque sommet (une clé), on associe la liste de ses voisins, par ordre alphabétique.

```
G = {'A': ['B', 'C'], 'B': ['A', 'C', 'D'], 'C': ['A', 'B', 'E'],
     'D': ['B', 'E', 'F'], 'E': ['C', 'D', 'F'], 'F': ['D', 'E']}
```

▷ **Erreur fréquente.** N'écrire chaque arête qu'une fois : dans un graphe non orienté, l'arête `A-B` donne `B` dans les voisins de `A` *et* `A` dans les voisins de `B`.

Exercice 5 ↑énoncé p. 406 **1.** `A-B, A-C, A-D, B-C, C-D, C-E, D-E.` **2.** Sommets 0..4 : `0-1, 0-2, 1-2, 1-3, 3-4`.





Exercice 6

↑énoncé p. 406

1. Les voisins du sommet `i` sont les indices `j` de la ligne `i` où la matrice vaut 1 : on construit cette liste **par compréhension**, ligne après ligne.

Appels attendus, écrits avant de coder :

```
assert matrice_vers_listes([[0, 1, 1], [1, 0, 0], [1, 0, 0]]) == [[1, 2], [0], [0]]   # cas normal
assert matrice_vers_listes([]) == []                                                    # cas limite : aucun sommet
assert matrice_vers_listes([[0, 0], [0, 0]]) == [[], []]                                # cas piege : une liste vide par sommet isole
```

```
def matrice_vers_listes(M):
    n = len(M)
    adj = []                                            # INITIALISER
    for i in range(n):                                  # PARCOURIR : les lignes
        adj.append([j for j in range(n) if M[i][j] == 1])   # METTRE A JOUR : voisins de i
    return adj                                          # RENVOYER
```

Autre méthode : sans compréhension, avec deux boucles imbriquées.

```
def matrice_vers_listes(M):
    adj = []                            # INITIALISER
    for i in range(len(M)):             # PARCOURIR : les lignes
        voisins_i = []                  # INITIALISER
        for j in range(len(M)):         # PARCOURIR : les colonnes
            if M[i][j] == 1:            # TESTER
                voisins_i.append(j)     # METTRE A JOUR
        adj.append(voisins_i)           # METTRE A JOUR
    return adj                          # RENVOYER
```

2. On part d'une matrice $n \times n$ remplie de 0, puis on place un 1 pour chaque voisin `j` de chaque sommet `i`.

```
def listes_vers_matrice(adj):
    n = len(adj)
    M = [[0] * n for i in range(n)]     # INITIALISER : que des 0
    for i in range(n):                  # PARCOURIR : les sommets
        for j in adj[i]:                # PARCOURIR : les voisins de i
            M[i][j] = 1                 # METTRE A JOUR
    return M                            # RENVOYER
```

3. On vérifie que `listes_vers_matrice(matrice_vers_listes(M)) == M` redonne bien la matrice de départ.

4. On crée une liste vide pour chaque sommet, puis, pour chaque arc `s`→`v`, on ajoute `s` aux prédécesseurs de `v`.

```
def predecesseurs(succ):
    pred = {s: [] for s in succ}        # INITIALISER : une liste vide par sommet
    for s in succ:                      # PARCOURIR : les sommets
        for v in succ[s]:               # PARCOURIR : les arcs s -> v
            pred[v].append(s)           # METTRE A JOUR : s precede v
    return pred                         # RENVOYER
```

Sur l'exemple, on obtient `{'A': ['C'], 'B': ['A'], 'C': ['A', 'B', 'D'], 'D': []}` : `D` n'a aucun prédécesseur (aucun arc n'arrive sur lui). Dans un graphe non orienté, successeurs et prédécesseurs sont les mêmes : la fonction renverrait le même dictionnaire.

Exercice 7 ↑énoncé p. 406 **1.** (poids sur la diagonale = 0, et 0 en l'absence d'arête)

$$P = \begin{pmatrix} 0 & 4 & 7 & 0 \\ 4 & 0 & 2 & 5 \\ 7 & 2 & 0 & 3 \\ 0 & 5 & 3 & 0 \end{pmatrix} \quad \text{(ordre A,B,C,D)}$$

2. A-B-C-D $= 4+2+3 = 9$; A-B-D$= 4+5 = 9$; A-C-D$= 7+3 = 10$; A-C-B-D$= 7+2+5 = 14$. Le plus court fait **9 km**, longueur atteinte par deux trajets : A-B-D et A-B-C-D.

La classe `Graphe`

Exercice 8 ↑énoncé p. 407 **Démarche :** on n'utilise que l'interface : `g.voisins(s)` renvoie la liste des voisins, sa longueur est le degré ; pour le nombre d'arêtes, on additionne les degrés de tous les sommets.

Appels attendus, écrits avant de coder (pour `nb_aretes` ; `g` est le graphe G du cours, à 6 arêtes) :

```
h = Graphe(['A', 'B'])
h.ajoute_arete('A', 'B')
assert nb_aretes(g) == 6                  # cas normal
assert nb_aretes(Graphe(['A'])) == 0      # cas limite : aucune arete
assert nb_aretes(h) == 1                  # cas piege : une arete, comptee deux fois dans les
                                  # degres
```

```
def degre(g, s):
    return len(g.voisins(s))

def est_isole(g, s):
    return g.voisins(s) == []

def nb_aretes(g):
    total = 0                             # INITIALISER
    for s in g.sommets:                   # PARCOURIR
        total = total + len(g.voisins(s))   # METTRE A JOUR : somme des degres
    return total // 2                     # RENVOYER : chaque arete est comptee deux fois
```

▷ **Erreur fréquente.** Renvoyer `total` sans le diviser par 2 : chaque arête `a-b` est comptée deux fois, dans les voisins de `a` et dans ceux de `b`.

Parcourir un graphe

Exercice 9 ↑énoncé p. 407 **1.** BFS depuis A (file entre crochets) : on visite A (file [B,C]), B ([C,D]), C ([D]), D ([E,F]), E ([F]), F. Résultat : `[A, B, C, D, E, F]`.

▷ **Erreur fréquente.** Enfiler C une seconde fois en traitant B : C est déjà découvert (il attend dans la file). On teste tous les sommets **découverts**, pas seulement les sommets déjà visités.

2. DFS depuis A : A→B→C→D→E→F. Résultat : `[A, B, C, D, E, F]`.

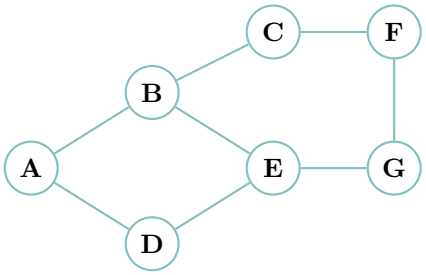
3. Ils visitent les **mêmes** sommets (toute la composante connexe), mais pas forcément dans le même ordre (ici ils coïncident par hasard ; changez une arête et l'ordre diffère).

Exercice 10 ↑énoncé p. 407 *Appels attendus, écrits avant de coder* (pour `BFS` ; `g` est le graphe de l'exercice 9) :

```
h = Graphe(['A', 'B', 'C'])
h.ajoute_arete('A', 'B')
assert BFS(g, 'A') == ['A', 'B', 'C', 'D', 'E', 'F']   # cas normal
assert BFS(Graphe(['A']), 'A') == ['A']                # cas limite : un seul sommet
assert BFS(h, 'A') == ['A', 'B']                       # cas piege : C n'est pas atteignable
```

Voir le cours pour `BFS`, `DFS` (récursif) et `DFS_iteratif`. Pour la file, la classe `File` du chapitre *Structures linéaires* (avec `pop(0)`) convient ici : c'est un choix de **simplicité**, acceptable sur de petits graphes, mais chaque `defiler` coûte alors $O(n)$; sur un grand graphe, on changerait d'implémentation (liste chaînée, deux piles…) sans toucher au BFS. Sur ce graphe, le DFS itératif renvoie par exemple `[A, C, D, F, E, B]` : la **pile** traite en dernier le premier voisin empilé, d'où un ordre différent du DFS récursif — mais c'est bien un parcours en profondeur valide.

Exercice 11 ↑énoncé p. 407 **1.** Arêtes : `A--B`, `A--D`, `B--C`, `B--E`, `C--F`, `D--E`, `E--G`, `F--G`, soit 8 arêtes (les listes contiennent 16 voisins en tout, et chaque arête est écrite deux fois).



2. La file est écrite dans l'ordre de sortie. `A` (file `D, B`) ; `D` (`B, E`) ; `B` (`E, C`) ; `E` (`C, G`) ; `C` (`G, F`) ; `G` (`F`) ; `F` (vide). Résultat : `['A', 'D', 'B', 'E', 'C', 'G', 'F']`.

3. `A` → `D` (premier voisin de `A`) → `E` (`A` est déjà visité) → `G` (`D` est déjà visité) → `F` → `C` → `B`, puis on remonte sans plus rien trouver. Résultat : `['A', 'D', 'E', 'G', 'F', 'C', 'B']`.

4. Pile écrite en ligne, sommet à droite : `A` (pile `D B`) ; `B` (`D E C`) ; `C` (`D E F`) ; `F` (`D E G`) ; `G` (`D E E`) ; `E` (`D E D`) ; `D` (`D E`) ; puis `E` et `D` sont dépilés mais déjà traités, donc ignorés. Résultat : `['A', 'B', 'C', 'F', 'G', 'E', 'D']`. Un sommet peut être empilé **plusieurs fois** (`E` par `B` puis par `G` ; `D` par `A` puis par `E`) : on ne le marque qu'au dépilement, et les exemplaires suivants sont ignorés (règle du cours).

▷ **Erreur fréquente.** Dépiler `D` juste après `A` parce que `D` est le premier voisin de `A` : les voisins sont empilés dans l'ordre `D`, `B`, et c'est le **dernier** empilé, `B`, qui ressort le premier.

5. Oui : avec les listes triées, `BFS` donne `A, B, D, C, E, F, G` et `DFS` donne `A, B, C, F, G, E, D`. En revanche, les **couches** du parcours en largeur ne changent pas : `B` et `D` (distance 1), puis `C` et `E` (distance 2), puis `F` et `G` (distance 3). Seul l'ordre *à l'intérieur* d'une couche dépend de l'ordre des voisins.

6.

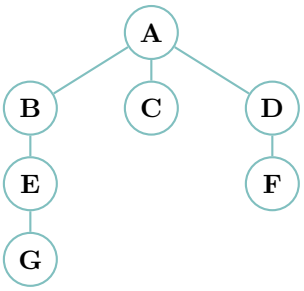
```
g = Graphe(['A', 'B', 'C', 'D', 'E', 'F', 'G'])
g.adj = {'A': ['D', 'B'], 'B': ['E', 'A', 'C'], 'C': ['B', 'F'],
         'D': ['A', 'E'], 'E': ['D', 'G', 'B'], 'F': ['C', 'G'],
         'G': ['F', 'E']}
print(BFS(g, 'A'))           # ['A', 'D', 'B', 'E', 'C', 'G', 'F']
print(DFS(g, 'A'))           # ['A', 'D', 'E', 'G', 'F', 'C', 'B']
print(DFS_iteratif(g, 'A'))  # ['A', 'B', 'C', 'F', 'G', 'E', 'D']
```

Exercice 12 ↑énoncé p. 408 **1.** Distances depuis `A` : `B`, `C`, `D` à 1 ; `E`, `F` à 2 ; `G` à 3.

- **a) BFS** possible (voisins de `A` pris dans l'ordre `C, B, D`) : les couches sont respectées, et `E`, découvert par `C`, passe bien avant `F`, découvert par `D`. Pas un DFS : après `C`, un DFS irait vers un voisin non visité de `C` (`D` ou `E`), pas vers `B`.
- **b) DFS** possible : `A`→`B`→`E`→`C`→`D`→`F`→`G`, chaque sommet étant un voisin du précédent. Pas un BFS : `E` (distance 2) sort avant `C` (distance 1).
- **c) DFS** possible : `A`→`D`→`F`→`G`→`E`→`C` ; `C` n'a plus de voisin non visité, on remonte en `E`, d'où l'on atteint `B`. Pas un BFS (`F` avant `B`).
- **d) Aucun.** Les couches sont respectées, mais cela ne suffit pas : `E` est découvert par `B` et `F` par `D` ; comme `B` sort de la file avant `D`, `E` est enfilé avant `F` et doit sortir avant lui. Pas un DFS : après `B`, il faut aller en `E`, seul voisin non visité de `B`.
- **e) Aucun.** Pas un BFS (`E` avant `C`). Le début `A, B, E, G, F` est un DFS, mais `F` a encore un voisin non visité, `D` : le DFS doit y aller avant de revenir en arrière, alors que la suite passe à `C`.

▷ **Erreur fréquente.** Croire qu'il suffit de respecter les distances pour obtenir un parcours en largeur (suite d) : dans une couche, l'ordre est imposé par l'ordre de sortie des parents dans la couche précédente.

2. `A` (file `B, C, D`) ; `B` (`C, D, E`) ; `C` (`D, E`) ; `D` (`E, F`) ; `E` (`F, G`) ; `F` (`G`) ; `G` (vide). Parents : `B`, `C`, `D` ← `A` ; `E` ← `B` ; `F` ← `D` ; `G` ← `E`.



La profondeur d'un sommet dans cet arbre (en arêtes) est sa **distance** à `A`, et le chemin de l'arbre qui descend de `A` à un sommet est un **plus court chemin** : c'est exactement ce que fait `plus_court_chemin` du cours avec son dictionnaire `parent`. Ici : `A--B--E--G`, de longueur 3. Ce n'est pas le seul : `A--C--E--G` et `A--D--F--G` ont aussi 3 arêtes ; le BFS en fournit un, qui dépend de l'ordre des voisins.

Chemins, cycles, connexité

Exercice 13 ↑énoncé p. 408 **Démarche :** un parcours lancé depuis `u` visite exactement les sommets reliés à `u` par une chaîne ; il suffit de regarder si `v` en fait partie. Le graphe est connexe si un parcours depuis un sommet quelconque visite tous les sommets.

Appels attendus, écrits avant de coder (pour `existe_chemin` ; `g` est le graphe de l'exercice 9) :

```
h = Graphe(['A', 'B', 'C'])
h.ajoute_arete('A', 'B')
assert existe_chemin(g, 'A', 'F')        # cas normal
assert existe_chemin(h, 'C', 'C')        # cas limite : un sommet est relie a lui-meme
assert not existe_chemin(h, 'A', 'C')    # cas piege : C est isole
```

```python
def existe_chemin(g, u, v):
    return v in DFS(g, u)          # ou BFS

def est_connexe(g):
    depart = g.sommets[0]
    return len(BFS(g, depart)) == len(g.sommets)
```

Exercice 14 ↑énoncé p. 408 *Appels attendus, écrits avant de coder* (pour `plus_court_chemin` ; `g` est le graphe de l'exercice 9) :

```python
h = Graphe(['A', 'B', 'C'])
h.ajoute_arete('A', 'B')
assert plus_court_chemin(g, 'A', 'F') == ['A', 'B', 'D', 'F']   # cas normal
assert plus_court_chemin(g, 'A', 'A') == ['A']                  # cas limite : depart = arrivee
assert plus_court_chemin(h, 'A', 'C') is None                   # cas piege : C inaccessible
```

```python
def plus_court_chemin(g, depart, arrivee):
    decouverts = [depart]                   # INITIALISER
    en_attente = File()                     # INITIALISER
    en_attente.enfiler(depart)
    parent = {depart: None}                 # INITIALISER
    while not en_attente.est_vide():        # PARCOURIR
        s = en_attente.defiler()
        if s == arrivee:                    # TESTER : arrive, on reconstruit
            chemin = [arrivee]
            while parent[chemin[0]] is not None:
                chemin = [parent[chemin[0]]] + chemin
            return chemin                   # RENVOYER
        for v in g.voisins(s):              # PARCOURIR
            if v not in decouverts:         # TESTER
                decouverts.append(v)        # METTRE A JOUR
                en_attente.enfiler(v)       # METTRE A JOUR
                parent[v] = s               # METTRE A JOUR
    return None                             # RENVOYER : inaccessible

def distances(g, depart):
    d = {depart: 0}                         # INITIALISER : sert aussi de liste des decouverts
    en_attente = File()                     # INITIALISER
    en_attente.enfiler(depart)
    while not en_attente.est_vide():        # PARCOURIR
        s = en_attente.defiler()
        for v in g.voisins(s):              # PARCOURIR
            if v not in d:                  # TESTER
                d[v] = d[s] + 1             # METTRE A JOUR
                en_attente.enfiler(v)       # METTRE A JOUR
    return d                                # RENVOYER
```

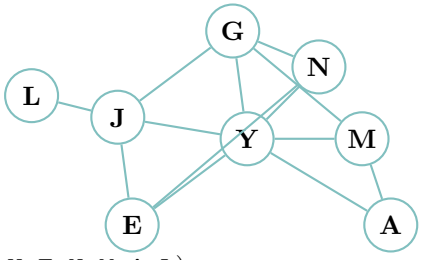
3. Sur le graphe de l'exercice 9, `plus_court_chemin(g, 'A', 'F')` vaut `['A','B','D','F']` : longueur 3, qui est bien la valeur `distances(g,'A')['F']`.

Vers le bac

Exercice 15 *Un cercle d'amis* (d'après Amérique du Nord 2024, jour 1, ex. 2) ↑énoncé p. 409

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 0,5 ; Q4 : 0,75 ; Q5 : 0,25 ; Q6 : 0,5 ; Q7 : 0,5 ; Q8 : 0,5 ; Q9 : 0,5 ; Q10 : 1,5

Partie A. 1.



2. (ordre `G,J,Y,E,N,M,A,L`)

```
matrice_adj = [[0,1,1,0,1,1,0,0],  # G
               [1,0,1,1,0,0,0,1],  # J
               [1,1,0,1,1,1,1,0],  # Y
               [0,1,1,0,1,0,0,0],  # E
               [1,0,1,1,0,0,0,0],  # N
               [1,0,1,0,0,0,1,0],  # M
               [0,0,1,0,0,1,0,0],  # A
               [0,1,0,0,0,0,0,0]]  # L
```

3. `position(sommets,'G')` vaut `0` ; `position(sommets,'Z')` vaut `None`.

4. Démarche : on cherche la ligne du sommet dans la matrice ; elle ne contient que des 0 et des 1, donc la somme de ses éléments est le nombre d'amis.

```
def nb_amis(L, m, s):
    pos_s = position(L, s)
    if pos_s == None:              # TESTER : sommet absent
        return None                # RENVOYER
    amis = 0                       # INITIALISER
    for i in range(len(m)):        # PARCOURIR : la ligne de s
        amis += m[pos_s][i]        # METTRE A JOUR
    return amis                    # RENVOYER
```

5. `nb_amis(sommets, matrice_adj, 'G')` vaut 4.

Partie B. 6. `c` est une **clé** (un sommet), `v` est la **valeur** associée (la liste de ses voisins).

7. Démarche : pour chaque personne (une clé), la liste de ses amis, lue dans l'énoncé.

```
graphe = {'G': ['J','Y','N','M'], 'J': ['G','Y','E','L'],
          'Y': ['G','J','E','N','M','A'], 'E': ['J','Y','N'],
          'N': ['G','Y','E'], 'M': ['G','Y','A'],
          'A': ['Y','M'], 'L': ['J']}
```

8. `def nb_amis(d, s): return len(d[s])`

9. Cercle d'amis de Lou (graphe des fâcheries) : `['L','J','G','Y','E','N']` — Milo et Anas ne sont plus atteignables.

▷ **Erreur fréquente.** Ajouter Milo et Anas au cercle d'amis de Lou : après les fâcheries, aucune chaîne ne relie `L` à `M` ou à `A`.

10. Démarche : on marque d'abord `s` comme visité, puis on relance le parcours sur chacun de ses voisins **pas encore** visités.

```
def parcours_en_profondeur(d, s, visites=None):
    if visites is None:                          # INITIALISER (premier appel)
        visites = []
    visites.append(s)                            # METTRE A JOUR : s est visite
    for v in d[s]:                               # PARCOURIR
        if v not in visites:                     # TESTER
```

```
            parcours_en_profondeur(d, v, visites)   # APPEL RECURSIF
    return visites                                  # RENVOYER
```

Exercice 16 *La recette du pain : un graphe de tâches* (d'après Asie 2024, jour 1, ex. 2) ↑énoncé p. 410

Barème indicatif (sur 4) : Q1 : 0,25 ; Q2 : 0,5 ; Q3 : 0,5 ; Q4 : 0,5 ; Q5 : 1 ; Q6 : 0,5 ; Q7 : 0,75

1. Graphe **orienté** (les flèches donnent l'ordre « avant / après »).

2. `(f)` puis `(g)` : oui (arc `f`→`g`). `(g)` puis `(f)` : non. `(b)` puis `(e)` : oui.

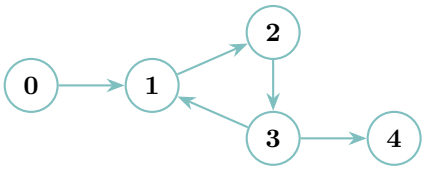
3. Tâches nécessaires avant `(l)` : `b, d, e, f, g, h, i, k`.

▷ **Erreur fréquente.** Ne citer que `g` et `k`, les prédécesseurs directs de `(l)` : il faut remonter tous les arcs, de prédécesseur en prédécesseur.

4. Non, pas de cycle. C'est indispensable : un cycle de dépendances rendrait la recette **impossible** à réaliser (chaque tâche attendrait l'autre).

5. La règle utilisée ne vaut que pour un graphe **non orienté**. Ici, `(f)` a été atteinte depuis `(e)` lors du premier parcours, qui l'a **terminée** (`(f)`, `(g)`, `(l)` entièrement explorés). Arriver de `(d)` sur `(f)` signifie seulement que deux chemins, `(b)`→`(e)`→`(f)` et `(d)`→`(f)`, **se rejoignent** : aucune flèche ne permet de revenir de `(f)` à `(d)`. **Bon critère** pour un graphe orienté : au cours d'un parcours en profondeur, il y a un cycle lorsqu'un arc mène à un sommet **en cours** (encore dans la pile d'appels, pas encore terminé) ; un arc vers un sommet **terminé** ne signale pas de cycle.

6. Arcs : `0`→`1`, `1`→`2`, `2`→`3`, `3`→`1`, `3`→`4`.



7. Impossible : le graphe contient le cycle `1`→`2`→`3`→`1`. On ne peut donc pas ordonner ces tâches (aucun tri topologique n'existe quand il y a un cycle).

Exercice 17 *Un parc d'attractions* (d'après Pays étrangers 2025, sujet PE2, ex. 3 (partie A)) ↑énoncé p. 411

Barème indicatif (sur 4) : Q1 : 0,25 ; Q2 : 0,5 ; Q3 : 0,5 ; Q4 : 0,25 ; Q5 : 0,5 ; Q6 : 0,5 ; Q7 : 0,5 ; Q8 : 1

1. `a4.duree = 12`

2. `a2.voisines[2][1]` vaut `4` : dans le 3e couple `(a4, 4)` de la liste des voisines de `a2`, on lit l'élément d'indice 1, c'est-à-dire la **durée du trajet** (4 min) entre « Petits chevaux » et « Grande roue ».

3. `a4.voisines = [(a2, 4), (a3, 6)]`

4. Le trajet est le même dans les deux sens (aller de x à y prend le même temps que de y à x) : la relation est symétrique, d'où un graphe **non orienté**.

5. Durée de `[a1, a2, a3]` = (durées des attractions) + (durées des trajets) = $(11+6+9)+(7+3) = \mathbf{36}$ min.

6. `[a2, a1, a4, a3]` n'est pas une balade car `a4` n'est pas voisine de `a1` : il n'y a pas d'arête entre « Grand huit » et « Grande roue ».

7–8. On parcourt les couples `(voisine, durée)` de `x` en cherchant `y` ; pour une balade, on vérifie que chaque attraction est voisine de la **suivante**, d'où la boucle jusqu'à l'avant-dernier indice.

Appels attendus, écrits avant de coder (pour `est_balade`) :

```
assert est_balade([a1, a2, a3])          # cas normal
assert est_balade([a4])                  # cas limite : une seule attraction
assert not est_balade([a2, a1, a4, a3])  # cas piege : a1 et a4 ne sont pas voisines
```

```
def sont_voisines(x, y):
    for (v, duree) in x.voisines:                # PARCOURIR
        if v == y:                               # TESTER
            return True                          # RENVOYER : trouvee
    return False                                 # RENVOYER

def est_balade(tab):
    for i in range(len(tab) - 1):                # PARCOURIR : jusqu'a l'avant-dernier
        if not sont_voisines(tab[i], tab[i+1]):  # TESTER
            return False                         # RENVOYER : un maillon manque
    return True                                  # RENVOYER
```

Autre méthode pour `sont_voisines` : construire par compréhension la liste des seules attractions voisines (première composante des couples), puis tester l'appartenance.

```
def sont_voisines(x, y):
    voisines = [v for (v, duree) in x.voisines]
    return y in voisines
```

Exercice 18 IA IA IA

Un réseau d'ordinateurs (d'après Centres étrangers 2024, groupe 1, jour 1, ex. 1) ↑énoncé p. 412

Barème indicatif (sur 4) : Q1 : 0,5 ; Q2 : 0,75 ; Q3 : 0,75 ; Q4 : 2

1. `voisins = [[1,2,3,4], [0,2,3], [0,1], [0,1], [0]]`

2. On ajoute le sommet 5, relié à 0 et 2 : `voisins = [[1,2,3,4,5], [0,2,3], [0,1,5], [0,1], [0], [0,2]]`.

3. Démarche : on tire au hasard un indice valide de la liste `voisins[s]`, puis on renvoie le voisin situé à cet indice.

```
import random
def voisin_alea(voisins, s):
    liste = voisins[s]
    i = random.randrange(len(liste))
    return liste[i]
```

4. C'est un parcours en largeur « par vagues » : à chaque étape, on contamine tous les voisins non encore contaminés, et on compte les étapes.

Appels attendus, écrits avant de coder :

```
assert temps_propagation(voisins, 0) == 1      # cas normal : 0 touche tout le monde
assert temps_propagation([[]], 0) == 0         # cas limite : un seul ordinateur
assert temps_propagation(voisins, 4) == 2      # cas piege : depuis 4, il faut passer par 0
```

```
def temps_propagation(voisins, s):
    contamines = [s]                           # INITIALISER
    frontiere = [s]                            # INITIALISER : contamines a l'etape precedente
    etapes = 0                                 # INITIALISER
    while len(contamines) < len(voisins):      # PARCOURIR : une vague par tour
        nouveaux = []
        for x in frontiere:                    # PARCOURIR
            for v in voisins[x]:               # PARCOURIR
                if v not in contamines:        # TESTER
                    contamines.append(v)       # METTRE A JOUR
                    nouveaux.append(v)         # METTRE A JOUR
        etapes = etapes + 1                    # METTRE A JOUR
        frontiere = nouveaux                   # METTRE A JOUR : vague suivante
    return etapes                              # RENVOYER
```

Depuis le sommet 4 (réseau initial), la contamination prend **2 étapes**.

Exercice 19 *Une agence de voyages* (d'après Polynésie 2024, jour 2, ex. 1) ↑énoncé p. 412

Barème indicatif (sur 6) : Q1 : 0,75 ; Q2 : 0,5 ; Q3 : 0,75 ; Q4 : 0,5 ; Q5 : 0,5 ; Q6 : 1,5 ; Q7 : 1,5

1. Plus court chemin `Mp`→`Nc` : `Mp -- Ar -- Ax -- Nc`, de longueur $80 + 76 + 176 = \mathbf{332}$ km.

2. En visitant le moins de villes possible (3 arêtes) : `Mp -- Ar -- Ax -- Nc` et `Mp -- Ar -- Mr -- Nc`.

3.

```
G = {'Av': ['Ni', 'Mr', 'Ax'],
     'Ni': ['Av', 'Ar', 'Mp'],
     'Mp': ['Ni', 'Ar'],
     'Ar': ['Ni', 'Mp', 'Ax', 'Mr'],
     'Mr': ['Av', 'Ar', 'To', 'Ax', 'Nc'],
     'To': ['Mr', 'Ax', 'Nc'],
     'Ax': ['Av', 'Ar', 'Mr', 'To', 'Nc', 'Di'],
     'Nc': ['Mr', 'To', 'Ax', 'Di'],
     'Di': ['Nc', 'Ax']}
```

4. `LIFO` = *Last In, First Out* (pile) ; `FIFO` = *First In, First Out* (**file**).

5. Parcours en **largeur** : la structure `f` est une **file** (on défile le premier entré), on découvre donc les sommets par distances croissantes.

6. Démarche : on remplace la liste `visite` par un dictionnaire de distances : le départ est à la distance 0, et un sommet découvert depuis `s` est à la distance de `s` plus 1.

Appels attendus, écrits avant de coder :

```
assert distance(G, 'Mp')['Nc'] == 3     # cas normal
assert distance(G, 'Mp')['Mp'] == 0     # cas limite : le depart lui-meme
assert distance(G, 'Mp')['Av'] == 2     # cas piege : Mp-Ni-Av, pas Mp-Ar-Ax-Av
```

```
def distance(graphe, sommet):
    dist = {sommet: 0}                  # INITIALISER
    f = creerFile()                     # INITIALISER
    enfiler(f, sommet)
    while not estVide(f):               # PARCOURIR
        s = defiler(f)
        for v in graphe[s]:             # PARCOURIR
            if v not in dist:           # TESTER
                dist[v] = dist[s] + 1   # METTRE A JOUR
                enfiler(f, v)           # METTRE A JOUR
    return dist                         # RENVOYER
```

Ainsi `distance(G, 'Av')` vaut :

```
{'Av':0, 'Ni':1, 'Mr':1, 'Ax':1, 'Ar':2, 'Mp':2, 'To':2, 'Nc':2, 'Di':2}
```

7. On remplace la file par une **pile** :

```
def parcours2(G, s):
    pile = [s]                          # INITIALISER
    visite = []                         # INITIALISER
    while pile != []:                   # PARCOURIR
        x = pile.pop()
        if x not in visite:             # TESTER
            visite.append(x)            # METTRE A JOUR
            for v in G[x]:              # PARCOURIR
                if v not in visite:     # TESTER
                    pile.append(v)      # METTRE A JOUR
    return visite                       # RENVOYER
```

Un résultat possible : `parcours2(G, 'Av')` = `['Av','Ax','Di','Nc','To','Mr','Ar','Mp','Ni']` (l'ordre dépend de l'ordre des voisins).

Critiquer une réponse d'IA

Exercice 20 *Un parcours en largeur écrit par un assistant d'IA* ↑énoncé p. 413

1. Non. Trace : `en_attente = [A]` ; on retire `A`, on ajoute `B` puis `C` → `[B, C]` ; `pop()` retire le **dernier**, `C`, qui ajoute `E` → `[B, E]` ; on retire `E` ; puis `B`, qui ajoute `D` ; puis `D`. La fonction renvoie donc `['A', 'C', 'E', 'B', 'D']`. Or `E` est à distance 2 de `A` et est traité *avant* `B`, à distance 1 : ce n'est **pas** un parcours en largeur. La liste se comporte en **pile** : le parcours s'enfonce comme un parcours en profondeur (sur ce petit graphe sans cycle, on obtient même l'ordre de `DFS_iteratif`), mais ce n'en est pas tout à fait un en général, puisque les sommets sont marqués dès l'empilement (voir la règle du cours et son contre-exemple). L'ordre attendu est `['A', 'B', 'C', 'D', 'E']`.

2. L'erreur : `en_attente.pop()` retire le **dernier** élément ajouté (comportement de **pile**, LIFO). Pour une **file** (FIFO), il faut retirer le premier :

```
        sommet = en_attente.pop(0)  # on defile (FIFO)
```

Avec cette ligne, la fonction renvoie `['A', 'B', 'C', 'D', 'E']`. C'est la correction la plus courte, et elle suffit sur un petit graphe ; mais c'est un **choix de simplicité** : chaque `pop(0)` décale toute la liste et coûte $O(n)$ (chapitre *Structures linéaires*). La version du cours, avec l'interface `File` (`enfiler`/`defiler`/`est_vide`), évite à la fois l'erreur de l'IA et ce coût caché.

3. Dérouler le code sur un **graphe de quatre ou cinq sommets** où un sommet est à distance 2 : en largeur, tous les sommets à distance 1 doivent sortir *avant* ceux à distance 2. Réflexe du cours : « file (FIFO) = largeur ; pile (LIFO) = profondeur ». Sur une `list`, `pop()` retire le dernier élément : on obtient une pile, et le parcours n'est plus en largeur.

S'entraîner à l'oral

Exercice 21 *Expliquer en deux minutes* ↑énoncé p. 414

▶ Solution. Pas de texte à apprendre par cœur : voici les **éléments attendus** pour chaque sujet. L'ordre, les mots et l'exemple peuvent être différents ; l'explication est réussie si ces idées y sont, justes et reliées entre elles.

Sujet 1.

- **Largeur** : on visite tous les voisins du départ, puis les voisins de ces voisins, « par couches » de distance croissante ; on utilise une **file**.
- **Profondeur** : on avance le plus loin possible le long d'un chemin, puis on revient en arrière ; on utilise une **pile** (ou la récursivité).
- Exemple : arêtes A–B, A–C, B–D, en partant de A : largeur A, B, C, D ; profondeur A, B, D, C.
- Piège : oublier de marquer les sommets déjà visités (boucle sans fin dès qu'il y a un cycle) ; le parcours en largeur donne un plus court chemin en nombre d'arêtes.

Sujet 2.

- **Sommets** : les lieux ou les personnes ; **arêtes** (ou **arcs** si le graphe est orienté) : les routes ou les liens ; un **poids** peut porter une distance.
- Vocabulaire : voisins (adjacents), chemin, cycle, graphe connexe.
- Exemple : un sens unique ou « suivre » quelqu'un sur un réseau social donne un graphe **orienté** ; une amitié réciproque, un graphe non orienté.
- Intérêt : un même algorithme (parcours, plus court chemin) résout des problèmes très différents dès qu'ils sont modélisés par un graphe.

Sujet 3.

- **Matrice d'adjacence** : tableau $n \times n$ où la case $[i][j]$ vaut 1 si i et j sont voisins ; tester une arête est immédiat, mais il faut n^2 cases même s'il y a peu d'arêtes.

- **Liste de successeurs** (un dictionnaire qui associe à chaque sommet la liste de ses voisins) : la mémoire est proportionnelle au nombre d'arêtes, et parcourir les voisins d'un sommet est rapide.
- Exemple : un réseau social de millions de personnes ayant chacune quelques centaines d'amis : la liste s'impose.
- Piège : pour un graphe non orienté, la matrice est symétrique et chaque arête apparaît deux fois dans les listes.

Entraînement

Exercice 22 *Parcours à la main en série* ↑énoncé p. 414 La file est écrite dans l'ordre de sortie (le prochain sommet à défiler en premier).

a) Largeur : `A` (file `B, D`) ; `B` (`D, C, E`) ; `D` (`C, E`) ; `C` (`E`) ; `E` (vide). Ordre `A, B, D, C, E`. Profondeur : `A, B, C, E, D`. Le sommet `F` n'est jamais visité : l'arc `F`→`D` part de `F`, mais aucun arc n'arrive sur `F` ; en suivant les flèches depuis `A`, on ne peut pas l'atteindre.

▷ **Erreur fréquente.** Visiter `F` depuis `D` parce que les deux sommets sont reliés : dans un graphe orienté, on ne suit que les successeurs, et `F` n'est pas un successeur de `D`.

b) Largeur : `A` (`B, C`) ; `B` (`C, D`) ; `C` (`D`) ; `D` (vide). Ordre `A, B, C, D`. Profondeur : `A, B, D, C`. Les sommets `E`, `F`, `G` ne sont pas visités : aucune chaîne ne les relie à `A` (le graphe n'est pas connexe). Pour tout visiter, on relance un parcours depuis un sommet non visité, par exemple `E` (qui donne `E, F, G`), et ainsi de suite tant qu'il reste des sommets non visités.

c) Largeur : `A` (`B, D`) ; `B` (`D, C`) ; `D` (`C`) ; `C` (`E`) ; `E` (`F, G`) ; `F` (`G`) ; `G` (vide). Ordre `A, B, D, C, E, F, G`. Profondeur : `A, B, C, D, E, F, G`. Les parcours ne tournent pas en rond parce qu'un sommet déjà découvert (en largeur) ou déjà visité (en profondeur) n'est jamais repris : en traitant `D`, on ne revient ni vers `A` ni vers `C`. Parents dans le parcours en profondeur : `B` (parent `A`), `C` (parent `B`), `D` (parent `C`), `E` (parent `C`), `F` (parent `E`), `G` (parent `F`). En visitant `D`, son voisin `A` est déjà découvert et n'est pas son parent (`C`) : c'est le cycle `A--B--C--D--A`. En visitant `G`, son voisin `E` est déjà découvert et n'est pas son parent (`F`) : c'est le cycle `E--F--G--E`. (Le voisin `C` de `D`, lui, est son parent : il ne signale rien.)

Modéliser une grille

Exercice 23 *Sortir d'un labyrinthe* (type bac) ↑énoncé p. 415 **1.** On peut toujours passer d'une case libre à sa voisine **et revenir** : la relation « être voisines » est symétrique, d'où un graphe non orienté. Le graphe a 21 sommets (les cases libres : $5+3+5+3+5$ ligne par ligne) et 21 arêtes : 9 « horizontales » (entre deux cases voisines d'une même ligne) et 12 « verticales » (d'une même colonne).

2. Les cases `(0, 5)` et `(4, 4)` n'ont qu'un voisin (degré 1) : ce sont des **culs-de-sac**. Le graphe contient un cycle, qui fait le tour des murs de gauche : `(0,0) -- (0,1) -- (0,2) -- (0,3) -- (1,3) -- (2,3) -- (2,2) -- (3,2) -- (4,2) -- (4,1) -- (4,0) -- (3,0) -- (2,0) -- (1,0) -- (0,0)`. C'est le seul ; il explique qu'il y ait deux façons très différentes de rejoindre la case `(2, 3)`.

3. et 4. *Appels attendus, écrits avant de coder :*

```
assert voisines(lab, 2, 3) == [(1, 3), (2, 2), (2, 4)]   # cas normal
assert voisines(lab, 0, 0) == [(1, 0), (0, 1)]           # cas limite : coin de la grille
assert voisines(lab, 0, 5) == [(1, 5)]                   # cas piege : lab[-1][5] existe, mais
                                                         # hors grille
```

```
def voisines(lab, i, j):
    """Cases libres voisines de (i, j), dans l'ordre haut, bas, gauche, droite."""
    res = []                                                    # INITIALISER
    for (a, b) in [(i - 1, j), (i + 1, j), (i, j - 1), (i, j + 1)]:   # PARCOURIR
```

```
        if 0 <= a < len(lab) and 0 <= b < len(lab[0]) and lab[a][b] == ".":  # TESTER
            res.append((a, b))                                               # METTRE A JOUR
    return res                                                               # RENVOYER

def construire_graphe(lab):
    """Graphe non oriente du labyrinthe : sommets = cases libres (i, j)."""
    libres = [(i, j) for i in range(len(lab)) for j in range(len(lab[0]))
              if lab[i][j] == "."]
    g = Graphe(libres)                                  # INITIALISER
    for (i, j) in libres:                               # PARCOURIR
        for v in voisines(lab, i, j):                   # PARCOURIR
            if not g.sont_voisins((i, j), v):           # TESTER : arete pas encore ajoutee
                g.ajoute_arete((i, j), v)               # METTRE A JOUR
    return g                                            # RENVOYER
```

Dans `voisines`, l'ordre des tests compte : si `a` vaut `-1`, `lab[a]` ne provoque pas d'erreur mais désigne la **dernière** ligne (indice négatif) ; si `a` vaut `len(lab)`, `lab[a]` lève `IndexError`. Comme `and` s'arrête au premier test faux, on vérifie les indices **avant** de lire la case.

▷ **Erreur fréquente.** Ajouter l'arête à chaque fois qu'on rencontre une voisine : chaque arête est alors ajoutée deux fois (une fois depuis chacune de ses extrémités), et chaque voisine apparaît en double dans les listes d'adjacence.

Autre méthode : ne regarder, pour chaque case libre, que sa voisine de **droite** et celle du **bas** ; chaque arête n'est alors rencontrée qu'une fois.

```
    for (i, j) in libres:
        if j + 1 < len(lab[0]) and lab[i][j + 1] == ".":    # voisine de droite
            g.ajoute_arete((i, j), (i, j + 1))
        if i + 1 < len(lab) and lab[i + 1][j] == ".":       # voisine du bas
            g.ajoute_arete((i, j), (i + 1, j))
```

5. Le parcours en largeur découvre les cases par distance croissante ; chaque case reçoit la distance de la case qui l'a découverte, plus 1 :

	0	1	2	3	4	5
0	0	1	2	3		9
1	1			4		8
2	2		6	5	6	7
3	3		7			8
4	4	5	6		10	9

La sortie est à distance 9 : un plus court chemin compte 9 déplacements. En remontant depuis `S` vers une voisine de distance inférieure d'une unité, on obtient (cases colorées) `(0,0)`, `(0,1)`, `(0,2)`, `(0,3)`, `(1,3)`, `(2,3)`, `(2,4)`, `(2,5)`, `(3,5)`, `(4,5)`. Il est **unique** : à chaque étape de la remontée, une seule voisine a la distance qui convient (par exemple, depuis `(2, 3)` à distance 5, seule `(1, 3)` est à distance 4 ; `(2, 2)` est à distance 6).

6. Le parcours en profondeur part vers le **bas** (première voisine de `(0, 0)`), longe le mur de gauche, remonte par `(3, 2)` et `(2, 2)`, atteint `(2, 3)`, explore d'abord le couloir du haut jusqu'à `(0, 1)` (impasse : `(0, 0)` est déjà visitée), revient à `(2, 3)`, puis file vers la droite. En remontant les parents depuis `S` : `(0,0)`, `(1,0)`, `(2,0)`, `(3,0)`, `(4,0)`, `(4,1)`, `(4,2)`, `(3,2)`, `(2,2)`, `(2,3)`, `(2,4)`, `(2,5)`, `(3,5)`, `(4,5)`, soit 13 déplacements au lieu de 9. Le parcours en profondeur trouve **un** chemin, pas forcément le plus court : il s'enfonce dans la première direction venue. Seul le parcours en **largeur** garantit un plus court chemin (en nombre de déplacements), car il atteint toutes les cases à distance k avant celles à distance $k+1$ (cours, « Pourquoi c'est le plus court »).

7.

```python
lab = ["....#.", ".##.#.", ".#....", ".#.##.", "...#.."]
g = construire_graphe(lab)
nb = 0
for s in g.sommets:
    nb = nb + len(g.voisins(s))
print(len(g.sommets), nb // 2)            # 21 21 : chaque arete compte deux fois
print(plus_court_chemin(g, (0, 0), (4, 5)))
print(distances(g, (0, 0))[(4, 5)])       # 9
```

Réponses des questions de la fiche de révision

↑ questions p. 426

Se tester

1. chaque arête `a`–`b` donne à la fois `M[a][b] = 1` et `M[b][a] = 1` (§ III.A).
2. `['A', 'B', 'D', 'C']` (§ V.C).
3. $O(n^2)$; $O(n+m)$ (§ III.A et III.B).
4. il tourne indéfiniment sur le cycle (boucle infinie) (§ V.A).
5. en largeur (BFS, avec une file) (§ V.B).

Et avant ?

1. $n \log_2 n$; $\log_2 n$ (ch. 7, § II et IV).
2. `1, 2, 4, 3` : la racine, puis tout le sous-arbre gauche, puis le droit (ch. 5, § V.A).
3. l'instance (le graphe) sur laquelle la méthode est appelée (ch. 2, § IV.A).

CORRIGÉS DU CHAPITRE 9

Réseaux

9

CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Le voyage d'un paquet

↑énoncé p. 440

1. $28 = 4 \times 6 + 4$: il faut 5 paquets : `RENDEZ`, `-VOUS␣`, `18H␣PO`, `RT␣HER`, `CULE`. Le paquet n°3 contient `18H PO`, le dernier `CULE`.
2. Chaque paquet voyage **indépendamment** : un routeur ne voit passer qu'un paquet à la fois, sans savoir qu'il fait partie d'un message ; il lui faut l'adresse sur **chaque** paquet pour l'aiguiller.
3. À remettre les morceaux **dans l'ordre** à l'arrivée, et à repérer un paquet **manquant**.
4. Léa → A → B → D → F → Hugo : 3 sauts de routeur à routeur (A–B, B–D, D–F).
5. Non : B sait seulement qu'un paquet pour le réseau de Hugo doit être remis à **D**. Aucun routeur ne connaît le chemin complet ; chacun décide **localement** du saut suivant, et c'est l'ensemble des tables qui fait le chemin.
6. F → D → B → A → Léa : ici le chemin inverse. Ce n'est pas obligatoire en général : l'aller et le retour sont décidés par des tables **différentes** (chaque routeur a une ligne pour chaque destination).
7. A → C → D → F ou A → C → E → F (aussi 3 sauts).
8. Rien d'utile : sa table l'envoie vers D, que le câble ne permet plus d'atteindre. Il peut garder le paquet (attendre), le jeter, ou chercher une autre sortie.
9. B l'envoie à A ; la table de A dit « réseau de Hugo → B » : A le renvoie à B, qui le renvoie à A… Le paquet fait du **ping-pong** entre A et B : c'est une **boucle de routage**.
10. Le compteur vaut 7 après A, 6 après B, puis $5, 4, 3, 2, 1, 0$ au fil des allers-retours : le paquet n°3 est **jeté** (en B). Il est perdu, mais le réseau n'est pas encombré pour toujours. (Sur Internet, ce compteur existe dans l'en-tête IP : le *TTL*, *time to live*.)
11. Pour l'aller : A doit envoyer vers **C** (« réseau de Hugo → C ») ; B peut garder « → A » (un paquet qui arriverait en B repartirait par A puis C). Pour le retour : D doit envoyer les paquets pour Léa vers **C** au lieu de B. Nouveau chemin : Léa → A → C → D → F → Hugo, toujours 3 sauts (A → C → E → F convient aussi). Attention à ne pas oublier la table de D : sans elle, la réponse de Hugo ne revient pas.

12. Impossible à la main : il faut que les routeurs **échangent automatiquement** des informations avec leurs voisins (« le lien B–D est coupé », « je peux atteindre tel réseau en tant de sauts ») et recalculent leurs tables tout seuls. Ce sont les **protocoles de routage** (RIP, OSPF dans le cours).

13. Les paquets voyagent indépendamment et peuvent prendre des chemins différents (avant et après la panne) ou attendre plus ou moins longtemps dans un routeur encombré.

14. En triant par numéro : `RENDEZ-VOUS _____RT HERCULE`. Il manque le paquet n°3 (« `18H PO` ») : Hugo le sait car les numéros $1, 2, 4, 5$ sautent le 3. (Si c'était le **dernier** paquet qui manquait, il ne pourrait pas le savoir sans connaître le nombre total de paquets : d'où l'intérêt de l'indiquer.)

15. Hugo doit **redemander** le paquet n°3 à Léa. Inversement, Hugo peut envoyer un **accusé de réception** pour chaque paquet reçu ; si Léa ne reçoit pas d'accusé au bout d'un certain temps, elle **renvoie** le paquet. C'est le rôle de TCP.

16. Propositions possibles, puis mots du cours : morceau → **paquet** ; étiquette → **en-tête** ; machine qui aiguille → **routeur** ; tableau → **table de routage** ; voyage en morceaux → **commutation de paquets** ; règles de mise à jour → **protocole de routage**.

17. … **robuste** (résistant aux pannes), car si un lien ou un routeur tombe, les paquets suivants **contournent** la panne par un autre chemin, et ceux qui sont perdus sont renvoyés.

9

CORRIGÉS DES EXERCICES

Machines, switchs, routeurs

Exercice 1  ↑énoncé p. 454

1. Le **switch** relie les machines *à l'intérieur* d'un même réseau local ; le **routeur** relie *plusieurs* réseaux locaux entre eux. **2.** Non : dans le même réseau local, elles communiquent directement via le switch. **3.** Une interface est une carte réseau (un point de raccordement) ; un routeur en a plusieurs car il est branché sur plusieurs réseaux à la fois. **4.** « … de **réseaux (locaux)** par des **routeurs** ».

Adressage IP

Exercice 2  ↑énoncé p. 454

1. 4 octets, soit $4 \times 8 = \mathbf{32}$ bits.
2. $192 = 128+64$, $168 = 128+32+8$, 1, $37 = 32+4+1$: `11000000.10101000.00000001.00100101`.
3. `/24` : les 24 premiers bits (les trois premiers octets) forment la partie **réseau**, `192.168.1` ; les 8 derniers bits forment la partie **machine**, ici 37.
4. Chaque nombre est un octet (8 bits), donc compris entre 0 et $2^8 - 1 = 255$: 300 ne tient pas sur un octet.

▷ **Erreur fréquente.** Écrire 1 sous la forme `1` au lieu de `00000001` : chaque octet compte exactement 8 bits, zéros de tête compris.

Exercice 3 ↑énoncé p. 454 **1.** `192.168.16.12` et `192.168.16.240` (masque `/24`) : mêmes 24 premiers bits (`192.168.16`) ⇒ **même** réseau. `192.168.16.12` et `192.168.17.3` : le 3e octet diffère ($16 \neq 17$) ⇒ réseaux **différents** (il faut un routeur).
2. Réseau de `192.168.16.12/24` : adresse du réseau `192.168.16.0`, diffusion `192.168.16.255`. Nombre de machines : $2^8 - 2 = \mathbf{254}$ (8 bits d'hôte, moins l'adresse réseau et l'adresse de diffusion).
3. `172.16.5.3/16` : adresse réseau `172.16.0.0` ; hôtes sur $32 - 16 = 16$ bits ⇒ $2^{16} - 2 = \mathbf{65\,534}$ machines.

▷ **Erreur fréquente.** Annoncer $2^8 = 256$ machines pour un `/24` : l'adresse du réseau et l'adresse de diffusion ne peuvent pas être attribuées à une machine, il en reste 254.

Exercice 4 ↑énoncé p. 455 **1.** On prend le plus petit `/n` tel que $2^{32-n} - 2 \geq$ nombre de machines :

Site	Machines	Masque	Adresses dispo.
Siège	100	`/25`	$2^7 - 2 = 126$
Agence Nord	50	`/26`	$2^6 - 2 = 62$
Agence Sud	20	`/27`	$2^5 - 2 = 30$
Boutique	10	`/28`	$2^4 - 2 = 14$
Liaison	2	`/30`	$2^2 - 2 = 2$

2. Un `/30` offre $2^2 - 2 = \mathbf{2}$ adresses de machines : exactement les **deux interfaces** des deux routeurs reliés, sans aucune adresse gaspillée.
3. Un `/24` offre 254 machines. Au Siège, $254 - 100 = \mathbf{154}$ adresses inutilisées ; à la Boutique, $254 - 10 = \mathbf{244}$ inutilisées. C'est un **énorme gaspillage** : CIDR permet justement d'ajuster `/n` à chaque site.

Le modèle en couches

Exercice 5 ↑énoncé p. 455 **1.** De haut en bas : **Application** (HTTP), **Transport** (TCP), **Réseau** (IP), **Lien** (Wi-Fi).
2. En descendant les couches, chacune ajoute **son en-tête** devant ce qu'elle reçoit de la couche du dessus, comme des enveloppes glissées les unes dans les autres : c'est l'**encapsulation**. Bloc émis, de gauche à droite :

en-tête Wi-Fi	en-tête IP	en-tête TCP	**requête HTTP**

3. Il lit l'**en-tête IP**, qui contient l'**adresse IP de destination** (et celle de la source) ; il consulte sa table de routage avec cette adresse. Il n'a **pas** besoin de la requête HTTP : il ne remonte pas au-delà de la couche Réseau.
4. L'en-tête de la couche **Lien** est retiré puis refait à chaque lien (Wi-Fi, puis Ethernet…) : chaque routeur « ouvre » cette enveloppe et en met une nouvelle adaptée au support suivant. Dans l'en-tête **IP**, les adresses IP source et destination restent les mêmes de bout en bout ; seul le compteur TTL est diminué de 1 par chaque routeur.
5. Dans l'ordre inverse de l'émission : en-tête Lien, puis IP, puis TCP. La requête HTTP est remise à l'application concernée : le **serveur web**.
6. Seule la couche **Lien** change (Wi-Fi → 4G) ; le navigateur (couche Application) ne s'adresse qu'à la couche Transport et ignore le support physique. Chaque couche ne dépend que de la couche voisine : on peut en remplacer une sans toucher aux autres. C'est ce qui permet à des réseaux très différents de former un seul Internet.

Tables de routage

Exercice 6 ↑énoncé p. 456 **1.** N1 et N3 (métrique 0, atteints par une interface propre, eth0 et eth2). **2.** A transmet le paquet au **routeur G** (qui saura continuer). **3.** Métrique 0 = réseau **directement raccordé** au routeur.

Le protocole RIP

Exercice 7 ↑énoncé p. 456 **1.** Nombre de sauts depuis A : B → 1 ; C → 1 ; D → 2 (A–B–D) ; E → 2 (A–B–E).
2. Table de routage RIP de A :

Réseau	Sortir vers	Sauts
réseau de B	B	1
réseau de C	C	1
réseau de D	B	2
réseau de E	B	2

▷ **Erreur fréquente.** Écrire dans la colonne « sortir vers » le routeur de destination (D) au lieu du **premier** voisin du chemin (B) : un routeur ne connaît que le prochain saut.

3. La métrique est plafonnée à 15 sauts (au-delà, réseau réputé inaccessible) ; [au-delà du programme] ce plafond est voulu : 16 sert d'« infini », car après une panne des routeurs peuvent se renvoyer une route de plus en plus longue, et le plafond arrête vite ce comptage ; et l'envoi périodique (toutes les 30 s) de toute la table à tous les voisins **sature** un grand réseau.

Le protocole OSPF

Exercice 8 ↑énoncé p. 456 **1.** 1 Gbit/s : $10^8/10^9 = \mathbf{0{,}1}$; 50 Mbit/s : $10^8/(5\times10^7) = \mathbf{2}$; 20 Mbit/s : $10^8/(2\times10^7) = \mathbf{5}$.
2. OSPF préfère le coût le plus **petit** : 1 Gbit/s (coût 0,1), puis 50 Mbit/s (coût 2), puis 20 Mbit/s (coût 5).
3. $0{,}1 + 2 + 2 = \mathbf{4{,}1}$.

▷ **Erreur fréquente.** Diviser 10^8 par le débit exprimé en Mbit/s ($10^8/50$) : il faut d'abord convertir le débit en bit/s (50 Mbit/s $= 5\times10^7$ bit/s).

Exercice 9 ↑énoncé p. 457 **1.** Coûts = 10^8/débit : 100 Mbit/s → 1 ; 10 Mbit/s → 10 ; 1 Mbit/s → 100.
2. Coûts des liaisons : A–B 100 ; B–D 10 ; toutes les autres 1. Table OSPF de A (le lien A–B coûte 100, on l'évite) :

Réseau	Sortir vers	Coût
réseau de B	C	2 (A–C–B)
réseau de C	C	1 (A–C)
réseau de D	C	4 (A–C–B–E–D)
réseau de E	C	3 (A–C–B–E)

Pour D, on compare par exemple A–B–D ($100 + 10 = 110$), A–C–B–D ($1 + 1 + 10 = 12$) et A–C–B–E–D ($1+1+1+1 = 4$) : le chemin le plus long en nombre de liaisons est le moins coûteux.

▷ **Erreur fréquente.** Prendre pour OSPF le chemin qui a le moins de liaisons : OSPF additionne les coûts des liaisons, et ici le meilleur chemin vers D en compte quatre.

3. Les deux protocoles **divergent** pour B, D et E : RIP prend le lien **direct** A–B (1 saut vers B, puis A–B–D et A–B–E en 2 sauts) ; OSPF passe par C (A–C–B, A–C–B–E–D, A–C–B–E). En effet A–B n'a qu'un débit de 1 Mbit/s (coût 100), et B–D seulement 10 Mbit/s (coût 10) : RIP ne « voit » pas cette lenteur, OSPF si. Seule la route vers C est la même (lien direct A–C).

Exercice 10 ↑énoncé p. 457

1. 10 Mbit/s → 10 ; 20 Mbit/s → 5 ; 50 Mbit/s → 2 ; 100 Mbit/s → 1. Soit : A–B 10, A–C 2, B–C 5, B–D 2, C–D 10, B–E 5, D–E 1, D–F 2, E–F 2.

2. Entre parenthèses, le prédécesseur ; ∗ = sommet fixé.

Fixé	A	B	C	D	E	F
—	0	∞	∞	∞	∞	∞
A	0*	10 (A)	2 (A)	∞	∞	∞
C		7 (C)	2*	12 (C)	∞	∞
B		7*		9 (B)	12 (B)	∞
D				9*	10 (D)	11 (D)
E					10*	11 (D)
F						11*

Deux relâchements sont décisifs : en fixant `C`, la distance de `B` tombe de 10 (lien direct, lent) à 7 ; en fixant `B`, celle de `D` tombe de 12 à 9.

3. En remontant les prédécesseurs (par exemple F ← D ← B ← C ← A), toutes les routes partent vers **C** :

Réseau	Sortir vers	Coût
réseau de B	C	7 (A–C–B)
réseau de C	C	2 (A–C)
réseau de D	C	9 (A–C–B–D)
réseau de E	C	10 (A–C–B–D–E)
réseau de F	C	11 (A–C–B–D–F)

4. RIP prend le lien **direct** A–B (1 saut). OSPF passe par C (A–C–B, coût $2+5=7$), car le lien direct A–B, à 10 Mbit/s seulement, coûte 10 : deux sauts rapides valent mieux qu'un saut lent.

5. Sans A–C : on fixe A (0) ; B (10, par A) ; D (12, par B) ; E (13, par D) ; F (14, par D) ; C (15, par B : $10+5$, mieux que $12+10$ par D). Nouvelle table : tout sort vers **B**, avec les coûts B 10, C 15 (A–B–C), D 12 (A–B–D), E 13 (A–B–D–E), F 14 (A–B–D–F). C'est la force d'OSPF : dès qu'une panne est annoncée, chaque routeur recalcule seul ses routes.

6. Avec le graphe saisi sous la forme `{'A': [('B', 10), ('C', 2)], 'B': [('A', 10), ('C', 5), ('D', 2), ('E', 5)], ...}`, l'appel `dijkstra(G, 'A')` du projet renvoie les distances `{'A': 0, 'B': 7, 'C': 2, 'D': 9, 'E': 10, 'F': 11}` et les prédécesseurs correspondants.

Exercices type bac

Exercice 11 *Réseau d'un établissement* ↑énoncé p. 458

Barème indicatif (sur 8) : Q1 : 0,5 ; Q2.a : 0,5 ; Q2.b : 0,5 ; Q2.c : 0,5 ; Q3.a : 1 ; Q3.b : 0,5 ; Q3.c : 0,5 ; Q4.a : 1 ; Q4.b : 1 ; Q4.c : 1 ; Q4.d : 1

1. 162 = 128+32+2 = `10100010` et 4 = `00000100` : `11000000.10101000.10100010.00000100`.

2. (a) Avec `/24`, les trois premiers octets forment la partie réseau : adresse du réseau `192.168.16.0`, masque `255.255.255.0`.

(b) Adresse de diffusion (tous les bits d'hôte à 1) : `192.168.16.255`.

(c) 8 bits d'hôte : $2^8-2=$ **254** machines (on retire l'adresse du réseau et l'adresse de diffusion).

3. (a) Les lignes de distance 1 des tables donnent les liaisons directes : R1–R5, R1–R6, R2–R3, R2–R5, R3–R4, R3–R5, R4–R5, R4–R6 (c'est bien le réseau de la figure 1). On en déduit :

Table de R5

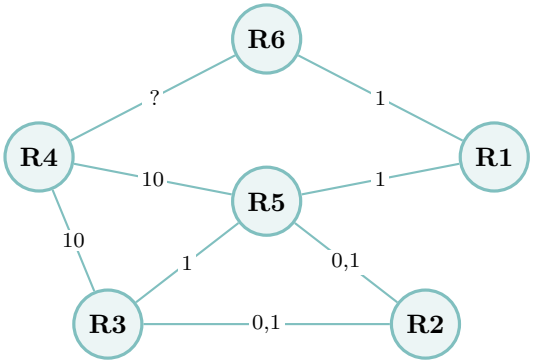
Dest.	passe par	Dist.
R1	R1	1
R2	R2	1
R3	R3	1
R4	R4	1
R6	R1	2

Table de R6

Dest.	passe par	Dist.
R1	R1	1
R2	R1	3
R3	R4	2
R4	R4	1
R5	R1	2

En cas d'égalité, un autre routeur de passage convient aussi : R5 peut joindre R6 par R4 (distance 2) ; R6 peut joindre R5 par R4 (distance 2) et R2 par R4 (distance 3).
(b) Le PC Admin 01 envoie le paquet à son routeur R6. Table de R6 : pour R3, on passe par **R4** ; table de R4 : R3 est un voisin direct. Parcours : **PC Admin 01** → **R6** → **R4** → **R3** → **PC CLG 01** (2 sauts entre routeurs).
(c) Sans R4, les voisins de R6 se réduisent à R1 : la route la plus courte est **R6** → **R1** → **R5** → **R3** (3 sauts).

4. (a) Ethernet : $d = 10^8/10 = 10^7$ bit/s. Fast-Ethernet : $10^8/10^8 = 1$. Fibre : $10^8/10^9 = 0{,}1$.
(b) Ethernet → 10, Fast-Ethernet → 1, Fibre → 0,1 (la liaison R6–R4 reste inconnue) :



(c) Coût(R6–R4) $+10 + 0{,}1 = 11{,}1$, donc coût(R6–R4) $= 1$: c'est une liaison **Fast-Ethernet**.
(d) Le Lycée général est desservi par R2. On compare les coûts des routes de R6 à R2 : R6–R1–R5–R2 : $1 + 1 + 0{,}1 =$ **2,1** ; R6–R1–R5–R3–R2 : 3,1 ; toutes les routes passant par R4 coûtent au moins $1 + 10 + 0{,}1 = 11{,}1$. OSPF choisit donc **PC Admin 01** → **R6** → **R1** → **R5** → **R2** → **PC LG 01**.

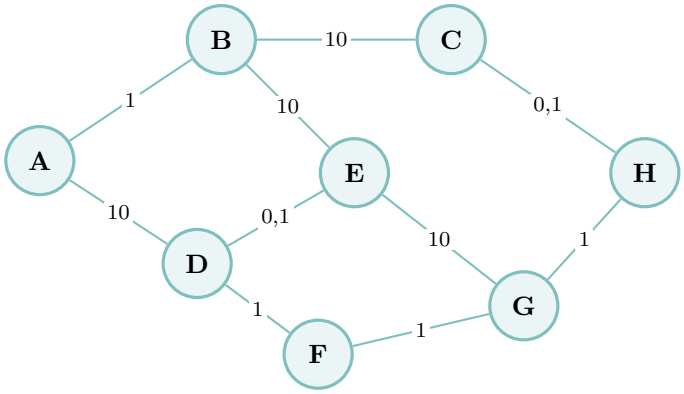
Exercice 12 *Des adresses, RIP puis OSPF* ↑énoncé p. 460

Barème indicatif (sur 4) : Q1.a : 0,5 ; Q1.b : 0,5 ; Q2 : 0,75 ; Q3.a : 0,75 ; Q3.b : 0,75 ; Q3.c : 0,75

1. (a) 2 = `00000010` et 13 = 8+4+1 = `00001101` : `10100100.10110010.00000010.00001101`.
(b) Avec `/24`, les 24 premiers bits (les trois premiers octets) forment la partie réseau ; on met les 8 bits d'hôte à 0 : adresse du réseau `164.178.2.0`.

2. Il faut au minimum 3 sauts, et trois chemins les réalisent : **A**–**B**–**E**–**G**, **A**–**D**–**E**–**G** et **A**–**D**–**F**–**G**.

3. (a) Coûts $10^9/BP$: Ethernet $10^9/10^8 = 10$; Fast-Ethernet $10^9/10^9 = 1$; Fibre $10^9/10^{10} = 0{,}1$.



(b) OSPF choisit le chemin de coût total minimal : **A**–**D**–**F**–**G**, de coût $10 + 1 + 1 = 12$ (devant A–B–C–H–G : $1 + 10 + 0{,}1 + 1 = 12{,}1$).
(c) Sans F, le meilleur chemin est **A**–**B**–**C**–**H**–**G**, de coût 12,1 (A–D–E–G coûte 20,1).

▷ **Erreur fréquente.** Reprendre la formule du cours (10^8/débit) alors que l'énoncé donne coût = $10^9/BP$: on applique toujours la formule de l'énoncé.

Exercice 13 *Bob écrit à Alice* ↑énoncé p. 461

Barème indicatif (sur 4) : Q1 : 0,75 ; Q2 : 0,5 ; Q3 : 1 ; Q4 : 0,75 ; Q5 : 1

1. Avec le masque `/24`, `172.16.2.3` appartient au réseau `172.16.2.0/24`, c'est-à-dire **LAN2** (la passerelle `172.16.2.253` est l'interface de R6 sur ce réseau) : c'est l'ordinateur d'**Alice**.

2. coût = 10 000/1 000 = **10**.

3. Comme dans les autres tables, le coût vers un réseau est celui du chemin le moins cher jusqu'au routeur le plus proche relié à ce réseau. Depuis R6, tout passe par R5 (le lien WAN7 vers R4 coûte 100) :

Dest.	Pass.	Coût
LAN1	R5	21
LAN2	–	–
WAN1	R5	11
WAN2	R5	20
WAN3	R5	11
WAN4	R5	12
WAN5	R5	10
WAN6	–	–
WAN7	–	–
WAN8	R5	10

(Par exemple LAN1 : R6–R5–R2–R1 = 10 + 1 + 10 = 21 ; WAN4, entre R3 et R4 : R6–R5–R2–R4 = 10 + 1 + 1 = 12.)

4. En suivant les tables (R1 envoie vers LAN2 par R2, R2 par R5, R5 par R6) : **R1** → **R2** → **R5** → **R6**, de coût 10 + 1 + 10 = 21.

5. On teste chaque panne : sans R2, le meilleur chemin est R1–R3–R5–R6 ($100+10+10=120$) ; sans R3 ou sans R4, la route R1–R2–R5–R6 (21) reste disponible ; sans **R5**, il reste R1–R2–R4–R6 $= 10+1+100 =$ **111**. Le routeur en panne est **R5**.

Exercice 14 *Du réseau local L1 au réseau local L2* ↑énoncé p. 462

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 0,5 ; Q4 : 0,5 ; Q5 : 0,5 ; Q6 : 0,5 ; Q7 : 1 ; Q8 : 0,75 ; Q9 : 0,5 ; Q10 : 0,75

Partie A. 1. `255.255.0.0`. **2.** On garde les 16 premiers bits de `172.16.8.8` : adresse du réseau `172.16.0.0`. **3.** On met les 16 bits machine à 1 : diffusion `172.16.255.255`. **4.** $2^{16}-2 =$ **65 534** machines (on retire l'adresse du réseau et celle de diffusion).

Partie B. 5. A envoie vers la passerelle `53.10.10.2`, interface de H ; H envoie vers `53.10.10.6`, interface de D, qui est connecté à L2 : **A** → **H** → **D**.

6. Sans la liaison H–D, il faut au minimum 3 sauts de A à D : **A–B–C–D** et **A–H–C–D**.

7. Avec le chemin **A–H–C–D** : A envoie toujours vers H, mais la règle de **H** doit obligatoirement changer (sa passerelle D n'est plus joignable). On examine les autres routeurs : G passe par H (G–H–C–D, 3 sauts, autant que G–F–E–D), F et E passent par E et D, B et C par C et D : leurs règles restent valables. Règle modifiée :

Routeur	Réseau destinataire	Passerelle	Interface
H	L2	53.10.10.10	53.10.10.9

(Avec le chemin A–B–C–D, il faudrait modifier A — passerelle `193.55.24.2`, interface `193.55.24.1` — *et* H, comme ci-dessus.)

8. coût $= 10^9/BP$: 1 Gbit/s → 1 ; 100 Mbit/s → 10 ; 10 Gbit/s → 0,1.

9. Le chemin le moins coûteux est **A–G–F–E–D**, de coût $1+0{,}1+0{,}1+0{,}1 =$ **1,3** (A–H–D coûterait 11, A–B–C–D 3).

10. Sans G–F : **A–H–F–E–D**, de coût $1+1+0{,}1+0{,}1 =$ **2,2** (devant A–B–C–D, 3).

Exercice 15 *Le réseau d'une société* ↑énoncé p. 464

Barème indicatif (sur 4) : Q1 : 0,5 ; Q2 : 0,75 ; Q3 : 1,25 ; Q4 : 0,5 ; Q5 : 1

Partie A. 1. P1 (`192.168.1.10`) et l'interface 1 de R1 (`192.168.1.1/24`) sont dans le réseau L1 `192.168.1.0/24`. P2 peut prendre toute adresse libre de ce réseau, par exemple `192.168.1.11` (ni `.0`, ni `.255`, ni `.1`, ni `.10`).

2. L2 est relié à l'interface 3 de R9, `192.168.18.1/24` : adresse du réseau `192.168.18.0`. Il y a $2^8-2 = 254$ adresses de machines, dont une prise par R9 : **253** adresses possibles pour P3 et P4.

▷ **Erreur fréquente.** Répondre 254 en oubliant que l'interface de R9 occupe déjà une adresse du réseau L2.

Partie B. 3. Table RIP de R1 :

Destination	Suivant	Nombre de sauts
R2	R2	1
R3	R3	1
R4	R4	1
R5	R2 *(ou R3, ou R6)*	2
R6	R6	1
R7	R6	2
R8	R6	2
R9	R6	3

4. Par exemple P1 → R1 → R6 → R7 → R9 → P3 : **3 sauts** entre routeurs (R1–R6–R8–R9 convient aussi).

Partie C. 5. Coûts : 100 Mbit/s → $10^8/10^8 = 1$; 50 Mbit/s → 2 ; 10 Mbit/s → 10. La route de coût minimal est **R1–R2–R5–R6–R7–R9**, de coût $1+1+1+1+2 =$ **6** (les routes R1–R3–R6–R7–R9 ou R1–R4–R6–R7–R9 coûtent 7, et R1–R6 direct coûte déjà 10).

Critiquer une réponse d'IA

Exercice 16 *RIP et OSPF expliqués par un assistant d'IA* ↑énoncé p. 465

1. Non. Avec coût = débit/10^8 : la liaison à 1 Mbit/s coûterait $10^6/10^8 = 0{,}01$ et celle à 100 Mbit/s coûterait 1. OSPF, qui minimise la somme des coûts, **préférerait la liaison lente** — l'inverse du but recherché (le 1 obtenu pour 100 Mbit/s est juste, mais par coïncidence : c'est la bande passante de référence).

2. La formule est **inversée**. Le cours : « *coût* $= 10^8/$ *débit (en bits/s)* » et « *plus le débit est grand, plus le coût est petit* ». Calcul corrigé : 1 Mbit/s $\to 10^8/10^6 = 100$; 100 Mbit/s $\to 10^8/10^8 = 1$: OSPF évite bien la liaison lente. Tout le reste de la réponse (métrique de RIP, échanges périodiques, Bellman-Ford, limite de 15 sauts, Dijkstra, somme des coûts) est exact.

3. Tester la formule sur **deux débits différents** et vérifier le **sens de variation** attendu : une liaison plus rapide doit coûter *moins*. Un seul exemple (celui de l'assistant, 100 Mbit/s) ne suffit pas, car il tombe sur la valeur de référence où les deux formules coïncident.

S'entraîner à l'oral

Exercice 17 *Expliquer en deux minutes* ↑énoncé p. 466

▶ **Solution.** Pas de texte à apprendre par cœur : voici les **éléments attendus** pour chaque sujet. L'ordre, les mots et l'exemple peuvent être différents ; l'explication est réussie si ces idées y sont, justes et reliées entre elles.

Sujet 1.

- Les données sont découpées en **paquets** ; chacun porte l'adresse IP de destination.
- Chaque routeur consulte sa **table de routage** : pour chaque réseau de destination, par où sortir (interface ou routeur voisin) et la **métrique** de la route.
- Le chemin se construit **saut par saut** : chaque routeur ne choisit que le suivant.
- Piège : les paquets d'un même message peuvent emprunter des chemins différents.

Sujet 2.

- **RIP** : métrique = nombre de **sauts** (routeurs traversés) ; chaque routeur échange sa table avec ses voisins.
- **OSPF** : métrique = somme des **coûts** des liaisons, le coût dépendant du débit (par exemple 10^8/débit en bits/s) ; chaque routeur connaît la carte complète et applique Dijkstra.
- Exemple : route 1 de 2 liaisons à 10 Mbit/s (coût $10 + 10 = 20$), route 2 de 3 liaisons à 1 Gbit/s (coût $3 \times 0{,}1 = 0{,}3$) : RIP choisit la route 1, OSPF la route 2.
- Piège : toujours utiliser la formule de coût donnée par l'énoncé (10^8, 10^9…).

Sujet 3.

- Un réseau change sans arrêt : liaisons qui tombent, routeurs ajoutés ; des tables écrites à la main seraient vite fausses.
- Un **protocole de routage** fait échanger automatiquement des informations aux routeurs pour construire et mettre à jour leurs tables.
- Exemple : une liaison est coupée ; les routeurs voisins s'en aperçoivent et recalculent une route qui la contourne.
- Lien avec les graphes : routeurs = sommets, liaisons = arêtes pondérées ; on cherche un plus court chemin.

Entraînement

Exercice 18 *Adresses de réseau en série* ↑énoncé p. 466

a) Réseau `192.168.42.0`, diffusion `192.168.42.255`, $2^8 - 2 = 254$ machines.
b) Réseau `172.20.0.0`, diffusion `172.20.255.255`, $2^{16} - 2 = 65\,534$ machines.
c) Réseau `10.0.0.0`, diffusion `10.255.255.255`, $2^{24} - 2 = 16\,777\,214$ machines.
d) Réseau `192.168.43.0`, diffusion `192.168.43.255`, 254 machines.
e) Non : avec le masque `/24`, leurs parties réseau diffèrent (troisième octet $42 \neq 43$) ; elles sont dans deux réseaux différents, et il faut un routeur.

▷ **Erreur fréquente.** En b), mettre à 0 le seul dernier octet (`172.20.3.0`) : avec `/16`, la partie machine compte **deux** octets, qui passent tous les deux à 0 (ou à 255 pour la diffusion).

Exercice 19 *Tables RIP en série* ↑énoncé p. 466

a)

Dest.	Sortir vers	Sauts
B	B	1
C	C	1
D	C	2

b)

Dest.	Sortir vers	Sauts
B	B	1
C	B ou D	2
D	D	1
E	D	2

c)

Dest.	Sortir vers	Sauts
B	B	1
C	B	2
D	B	2
E	B	3
F	B	4

En c), A n'a qu'un voisin : toutes ses routes sortent vers B.

CORRIGÉS DES TP

TP Construire, router et pinguer un réseau ↑énoncé p. 469

Partie 1 — Un réseau local

Exercice 1 *Construire et adresser*

1. Les trois adresses ont le même masque `/24` et les mêmes 24 premiers bits (`192.168.0`) : elles appartiennent donc au **même réseau local**, et communiquent directement par le switch.

2. 8 bits d'hôte $\Rightarrow 2^8 - 2 =$ **254** machines (les adresses `.0` et `.255` sont réservées au réseau et à la diffusion).

Exercice 2 *Pinguer et observer*

1. `ping 192.168.0.2` réussit ; le paquet va PC-A → Switch → PC-B (pour information, hors programme : il est précédé d'une requête **ARP**, par laquelle PC-A demande « qui a l'adresse IP `192.168.0.2` ? » pour obtenir l'adresse physique, dite MAC, de PC-B ; viennent ensuite les messages **ICMP** *echo* et *reply*, le protocole utilisé par `ping`).

2. Vers une machine **inexistante** (`.9`), personne ne répond à la requête ARP : le `ping` **échoue** (délai dépassé).

Partie 2 — Deux réseaux, un routeur, une passerelle

Exercice 3 *Relier deux réseaux*

Construction seule : routeur à 2 interfaces, `192.168.0.254/24` côté S1 et `192.168.1.254/24` côté S2. Vérifier que chaque câble du routeur est relié au bon switch (erreur fréquente).

Exercice 4 *La passerelle est indispensable*

1. `192.168.0.1` et `192.168.1.1` n'ont pas la même adresse réseau (`192.168.0` ≠ `192.168.1`) : le paquet doit **quitter** le réseau local. Sans **passerelle** configurée, PC-A ne sait pas à qui remettre un tel paquet ⇒ « destination non joignable ».

2. Avec les passerelles réglées sur l'interface du routeur de chaque côté, le `ping` réussit. Trajet : PC-A → S1 → **R** (qui route d'un réseau à l'autre) → S2 → PC-B.

Partie 3 — Le routage manuel entre trois routeurs

Exercice 5 *Construire et adresser*

Construction selon le plan d'adressage de l'énoncé (trois interfaces par routeur, passerelles des trois PC sur l'interface `.254` de leur routeur).

Exercice 6 *Remplir les tables de routage à la main*

Tables de routage à saisir (chaque routeur n'a besoin que des **deux** réseaux locaux qui ne lui sont pas directement rattachés ; les réseaux directs sont gérés automatiquement) :

R1

IP de destination	Masque	Passerelle suivante	Via l'interface
192.168.1.0	255.255.255.0	172.16.12.2	172.16.12.1
192.168.2.0	255.255.255.0	172.16.13.3	172.16.13.1

R2

IP de destination	Masque	Passerelle suivante	Via l'interface
192.168.0.0	255.255.255.0	172.16.12.1	172.16.12.2
192.168.2.0	255.255.255.0	172.16.23.3	172.16.23.2

R3

IP de destination	Masque	Passerelle suivante	Via l'interface
192.168.0.0	255.255.255.0	172.16.13.1	172.16.13.3
192.168.1.0	255.255.255.0	172.16.23.2	172.16.23.3

Exercice 7 *Pinguer de bout en bout, et vérifier*

1. `ping 192.168.1.1` : PC-A l'envoie à sa passerelle R1 ; R1 lit « destination `192.168.1.0` → passerelle suivante `172.16.12.2` », donc transmet à R2 ; R2 est directement relié au réseau B et remet le paquet à PC-B. Trajet : **PC-A → R1 → R2 → PC-B**.

2. `route` affiche la table de routage de la machine (destinations, masques, passerelles, interfaces) ; `traceroute 192.168.2.1` liste les routeurs traversés jusqu'à PC-C, soit R1 puis R3.

3. Reroutage. En changeant, dans R1, la passerelle suivante de la route `192.168.1.0` pour `172.16.13.3` (R3), le paquet vers PC-B part maintenant vers R3, qui possède lui-même une route vers `192.168.1.0` (via R2) : le trajet devient **PC-A → R1 → R3 → R2 → PC-B**. On a montré que c'est **la table de routage seule** qui décide du chemin : la modifier suffit à changer la route, sans toucher au câblage.

Réponses des questions de la fiche de révision

↑ questions p. 473

Se tester

1. Non : les trois premiers octets diffèrent (10.0.3 contre 10.0.4), il faut un routeur (§ III).
2. $2^8 - 2 = 254$ (adresse du réseau et de diffusion exclues) (§ III).
3. $10^8/10^7 = 10$ (§ VIII).
4. Faux : un seul saut par un lien lent peut coûter plus cher que plusieurs sauts rapides (§ IX).
5. OSPF (somme des coûts 10^8/débit) ; RIP ne compte que les sauts (§ VII et VIII).

Et avant ?

1. `A, B, D, C` : d'abord les voisins de `A`, puis ceux à distance 2 (ch. 8, § V.B).
2. `UPDATE film SET note = 8 WHERE id = 5;` (ch. 6, § VI).
3. une file : premier entré, premier sorti (FIFO) (ch. 3, § III).

CORRIGÉS DU CHAPITRE 10

Cryptographie

10

CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Casser le code de César

↑ énoncé p. 481

1. Il y a 26 décalages possibles (25 utiles, puisque $k = 0$ ne change rien). Avec $k = 1$, `FY` redevient `EX` ; avec $k = 2$, `DW` : pas de mot français. Il faudrait en moyenne une douzaine d'essais.

2. Dans la première phrase (`FY XPDDLRP ... OF CZT.`, 47 lettres) : `P` : **11** ; `D` : **6** ; `L` : **3**. Déjà sur une phrase, `P` domine nettement.

3. Dans le message, `P` apparaît 79 fois sur 372 (environ 21 %), loin devant `D` (35). En français, c'est `E` (14,7 %). Comme un décalage envoie toujours la même lettre sur la même lettre, **`P` est très probablement le chiffré de `E`**. (Sur un texte court, les pourcentages ne sont pas exactement ceux du tableau ; seul l'ordre compte.)

4. `E` a pour rang 4 et `P` pour rang 15 : $\mathbf{k = 15 - 4 = 11}$.

5. La deuxième lettre la plus fréquente est `D` (rang 3) : $(3 - 11) \,\%\, 26 = 18$, soit `S`, deuxième lettre du tableau français : l'hypothèse est confirmée. (De même `Y`→`N`, `E`→`T`, `L`→`A`, `W`→`L`.)

6. `UN MESSAGE SECRET A ETE INTERCEPTE PAR LES ESPIONS DU ROI.`

7. Le message complet (sans accents) :

   ```
   UN MESSAGE SECRET A ETE INTERCEPTE PAR LES ESPIONS DU ROI. PERSONNE NE CONNAIT LA
   CLE, MAIS CHACUN SAIT QUE LES LETTRES ONT ETE DECALEES. DANS UNE LANGUE, CERTAINES
   LETTRES REVIENNENT BIEN PLUS SOUVENT QUE LES AUTRES : EN FRANCAIS, LA LETTRE E EST
   DE LOIN LA PLUS FREQUENTE. LE DECALAGE NE CHANGE PAS CES FREQUENCES, IL LES DEPLACE
   SEULEMENT. IL SUFFIT DONC DE COMPTER LES LETTRES DU TEXTE CHIFFRE POUR RETROUVER LA
   CLE ET LIRE CE MESSAGE SANS L'AVOIR JAMAIS RECU.
   ```

8. Lire sans la clé, c'est **décrypter** (le travail de l'attaquant). Le destinataire, qui connaît k, **déchiffre**. L'expéditeur, lui, avait **chiffré**.

9. Le décalage conserve la **fréquence** de chaque lettre (seulement déplacée sur une autre lettre), ainsi que les espaces, la longueur des mots, les lettres doublées (`SS` → `DD`)… Un bon chiffrement doit **effacer ces régularités** : le message chiffré ne doit rien laisser deviner du clair. Et sa sécurité doit tenir à la seule clé, choisie parmi un nombre de possibilités immense (pas 26).

CORRIGÉS DES EXERCICES

Chiffrements symétriques

★ **Exercice 1** *Le chiffre de César, à la main* ↑énoncé p. 495

1. `N`(13)→17 `R`, `S`(18)→22 `W`, `I`(8)→12 `M` : `NSI` devient **`RWM`**.
2. On décale de 3 en arrière : `K`→`H`, `H`→`E`, `O`→`L`, `O`→`L`, `R`→`O` : **`HELLO`**.

   ▷ **Erreur fréquente.** Décaler dans le mauvais sens pour déchiffrer : on *retranche* la clé (`K` → `H`), et l'on ajoute 26 si le rang devient négatif.

3. Il y a **26 clés** (k de 0 à 25). Un ordinateur les teste *toutes* instantanément : le code est cassé en quelques microsecondes.
4. **Force brute** : on déchiffre `YLUKLGCVBZ` avec chacun des 26 décalages ; un seul essai donne un mot français lisible. Ici, pour $k = 7$, on obtient `RENDEZVOUS` : c'est le message clair.

★ **Exercice 2** *Le chiffre de César, en Python* ↑énoncé p. 495

Démarche : pour chaque lettre, on calcule son rang, on le décale de `k` en restant entre 0 et 25 (reste de la division par 26), puis on reconvertit le nouveau rang en lettre.

1. `(a) nouveau = (rang + k) % 26` ; `(b) resultat + chr(nouveau + ord("A"))`.
2. Pour déchiffrer : `cesar(message, -k)` ou, de façon équivalente, `cesar(message, 26 - k)`.

▷ **Erreur fréquente.** Oublier le `% 26` : `cesar("XYZ", 3)` donnerait `[`, `\` et `]`, qui ne sont pas des lettres, au lieu de `ABC`.

★★ **Exercice 3** *Le chiffrement affine* ↑énoncé p. 495

1. Si a n'est pas premier avec 26, la fonction $x \mapsto (ax+b)$ mod 26 n'est pas une bijection : deux lettres différentes peuvent avoir le même chiffré, le déchiffrement devient impossible. Valeurs possibles : 1,3,5,7,9,11,15,17,19,21,23,25.
2. `N`(13)→ $3 \times 13 + 1 = 40 \equiv 14$ `O` ; `S`(18)→ $3 \times 18 + 1 = 55 \equiv 3$ `D` ; `I`(8)→ $3 \times 8 + 1 = 25$ `Z`. Résultat : **`ODZ`**.

   ▷ **Erreur fréquente.** Oublier de réduire modulo 26 : $3 \times 13 + 1 = 40$ ne correspond à aucune lettre ; on retranche 26 autant de fois que nécessaire ($40 - 26 = 14$, soit `O`).

3. $3 \times 9 = 27 = 26 + 1$, donc $3 \times 9 \equiv 1$ [26] : 9 est bien l'inverse de 3.
4. Déchiffrement : $x = 9\,(y - 1)$ mod 26. Sur `ODZ` : `O`(14)→ $9(14 - 1) = 117 \equiv 13$ `N` ; `D`(3)→ $9(3 - 1) = 18$ `S` ; `Z`(25)→ $9(25 - 1) = 216 \equiv 8$ `I`. On retrouve **`NSI`**.

★★ **Exercice 4** *Vigenère : la clé qui se répète* ↑énoncé p. 496

1. `A`+`N`$= 0 + 13 = 13$ `N` ; `N`+`S`$= 13 + 18 = 31 \equiv 5$ `F` ; `A`+`I`$= 0 + 8 = 8$ `I` ; `N`+`N`$= 13 + 13 = 26 \equiv 0$ `A` ; `A`+`S`$= 0 + 18 = 18$ `S` ; `S`+`I`$= 18 + 8 = 26 \equiv 0$ `A`. Résultat : **`NFIASA`**.

2. **Non.** Les trois `A` du clair deviennent `N`, `I` et `S`, et les deux `N` deviennent `F` et `A` : une même lettre du clair donne des lettres chiffrées différentes selon sa position face à la clé. À l'inverse, deux lettres différentes du clair (`N` et `S`) donnent ici le même `A`. Avec César, une lettre est toujours chiffrée pareil. Cela rend l'analyse des fréquences bien plus difficile.

3. La clé se **répète** périodiquement : des groupes de lettres identiques dans le clair, espacés d'un multiple de la longueur de la clé, produisent les mêmes groupes chiffrés. Ces répétitions trahissent la longueur de la clé (méthode de Babbage-Kasiski), après quoi le chiffre se ramène à plusieurs César.

★ **Exercice 5** *Le masque jetable et le XOR* ↑énoncé p. 496

1. $3 \hat{} 66 = 65$ (car $65 \hat{} 66 = 3$, et appliquer $\hat{} 66$ de nouveau annule l'opération). Le XOR est son **propre inverse** : $(m \oplus k) \oplus k = m$.

2. Chiffrer une deuxième fois avec le même masque annule le premier XOR et redonne le message clair : `chiffre(chiffre(msg, masque), masque) == msg`. Une seule fonction suffit.

3. Il est inviolable si le masque est **aléatoire**, **aussi long que le message** et **utilisé une seule fois** (jetable). Démontré par **Claude Shannon** (1949).

4. Il faudrait une clé aussi longue que *tout* ce qu'on veut s'envoyer, vraiment aléatoire et jamais réutilisée : la production et surtout la **distribution** d'une telle clé sont impraticables au quotidien.

Chiffrement asymétrique et RSA

★★ **Exercice 6** *Symétrique ou asymétrique ?* ↑énoncé p. 496

1. **Symétrique** : une *même* clé secrète chiffre et déchiffre. **Asymétrique** : *deux* clés distinctes, une publique pour chiffrer, une privée pour déchiffrer.

2. Le **problème de l'échange de la clé** : il faut avoir partagé la clé secrète *avant* de communiquer, ce qui est difficile sur un canal non sûr.

3. Alice chiffre avec la clé **publique de Bob** ; Bob déchiffre avec sa clé **privée**.

   ▷ **Erreur fréquente.** Faire chiffrer Alice avec *sa propre* clé publique : seule Alice, qui détient la clé privée correspondante, pourrait déchiffrer, pas Bob. Pour la confidentialité, on chiffre avec la clé publique du **destinataire**.

4. Pour signer, Alice chiffre avec sa **propre clé privée** : n'importe qui peut vérifier avec la clé publique d'Alice que le message vient bien d'elle.

★★ Exercice 7 *RSA à la main — fabriquer les clés (1)*

↑énoncé p. 497

1. $n = 3 \times 11 = \mathbf{33}$; $\varphi = (3-1)(11-1) = 2 \times 10 = \mathbf{20}$.
2. PGCD(3,20) $= 1$: $e = 3$ convient. **Clé publique :** $(3, 33)$.
3. On cherche d tel que $3d \equiv 1\ [20]$: $3 \times 7 = 21 = 20 + 1$, donc $\mathbf{d = 7}$. **Clé privée :** $(7, 33)$.

   ▷ **Erreur fréquente.** Chercher l'inverse de e modulo $n = 33$: ici, 3 n'a même pas d'inverse modulo 33 (car 3 divise 33). L'inverse se calcule toujours modulo $\varphi = (p-1)(q-1)$.
4. $C = 5^3 \bmod 33 = 125 \bmod 33 = 125 - 3 \times 33 = \mathbf{26}$.
5. Alice calcule $C^d \bmod n = 26^7 \bmod 33$, qui vaut 5 : on retrouve bien le message $M = 5$.

★★ Exercice 8 *RSA à la main — trouver d (2)*

↑énoncé p. 497

1. $n = 7 \times 11 = \mathbf{77}$; $\varphi = 6 \times 10 = \mathbf{60}$.
2. $7 \times 43 = 301 = 5 \times 60 + 1$, donc $7 \times 43 \equiv 1\ [60]$: $\mathbf{d = 43}$ convient.
3. **Clé publique :** $(7, 77)$; **clé privée :** $(43, 77)$.
4. $C = 2^7 \bmod 77 = 128 \bmod 77 = 128 - 77 = \mathbf{51}$.

★★ Exercice 9 *L'inverse modulaire en Python*

↑énoncé p. 497

1. `pow(3, -1, 20)` renvoie `7` ; `pow(7, -1, 60)` renvoie `43`.
2. `pow(M, e, n)` calcule M^e mod n. On l'écrit ainsi car c'est une **exponentiation modulaire rapide** : elle réduit modulo n à chaque étape et ne manipule jamais le nombre géant M^e (contrairement à `(M ** e) % n`).
3. `n = p * q ;  phi = (p - 1) * (q - 1) ;  d = pow(e, -1, phi).`

★★ Exercice 10 *L'échange de clé de Diffie-Hellman*

↑énoncé p. 497

1. $A = 5^4 \bmod 23 = 625 \bmod 23 = \mathbf{4}$.
2. $B = 5^3 \bmod 23 = 125 \bmod 23 = \mathbf{10}$.
3. Alice : $B^a = 10^4 \bmod 23 = \mathbf{18}$. Bob : $A^b = 4^3 \bmod 23 = 64 \bmod 23 = \mathbf{18}$. Même secret 18, jamais transmis.
4. Pour retrouver le secret, Ève devrait retrouver a (ou b) à partir de $A = g^a \bmod p$: c'est le **problème du logarithme discret**, qui n'a pas d'algorithme rapide connu. Alice et Bob, eux, connaissent leur exposant secret.

HTTPS

★ **Exercice 11** IA IA IA *Le petit cadenas du navigateur* ↑énoncé p. 498

1. L'**asymétrique** sert à *échanger la clé* de session ; le **symétrique** (AES) chiffre ensuite *les données*. Raison : l'asymétrique résout l'échange de clé mais est lent ; le symétrique est rapide mais suppose une clé partagée — on combine leurs forces.

2. Ordre : **(1)** le serveur envoie son certificat et sa clé publique ; **(2)** le client vérifie le certificat, c'est-à-dire la signature de l'autorité de certification, avec la clé publique de cette autorité, déjà connue du navigateur ; **(3)** le client fabrique une clé symétrique, la chiffre avec la clé publique du serveur et l'envoie ; **(4)** les deux échangent leurs données chiffrées en symétrique (AES).

3. Le **certificat** authentifie le serveur (garantit que la clé publique est bien la sienne). Sans lui, un attaquant pourrait se placer au milieu (*man in the middle*) en présentant sa propre clé publique.

Vers le bac

★★ **Exercice 12** IA IA IA *Type bac — La méthode du masque jetable* (d'après Métropole 2025, jour 2) ↑énoncé p. 498

Barème indicatif (sur 8) : Q1 : 0,5 ; Q2 : 0,75 ; Q3 : 0,75 ; Q4 : 1 ; Q5 : 0,75 ; Q6 : 0,75 ; Q7 : 0,5 ; Q8 : 0,5 ; Q9 : 0,5 ; Q10 : 0,75 ; Q11 : 1,25

Partie A.

1. `L`(11)+`E`(4)= 15 `P` ; `I`(8)+`Y`(24)= 32 ≡ 6 `G` ; `B`(1)+`Q`(16)= 17 `R` ; `R`(17)+`M`(12)= 29 ≡ 3 `D` ; `E`(4)+`T`(19)= 23 `X`. Résultat : **`PGRDX`**.

2. **Démarche :** on parcourt les **indices** de la liste (`range(len(L))`) et l'on renvoie le premier indice où se trouve l'élément.

```
def indice(L, element):
    for i in range(len(L)):  # PARCOURIR les indices
        if L[i] == element:  # TESTER
            return i         # RENVOYER des qu'on trouve
```

3. *Appels attendus, écrits avant de coder :*

```
assert lettres_vers_indices('HELLO') == [7, 4, 11, 11, 14]   # cas normal
assert lettres_vers_indices('') == []                        # cas limite : chaine vide
assert lettres_vers_indices('AZ') == [0, 25]                 # cas piege : premiere et
                                                             # derniere lettres
```

Démarche : une liste vide que l'on remplit en parcourant la chaîne : à chaque lettre, on ajoute son indice dans `alphabet`.

```
def lettres_vers_indices(chaine):
    resultat = []                                  # INITIALISER
    for lettre in chaine:                          # PARCOURIR
        resultat.append(indice(alphabet, lettre))  # METTRE A JOUR
    return resultat                                # RENVOYER
```

Autre méthode : on construit directement la liste des indices par compréhension.

```
def lettres_vers_indices(chaine):
    return [indice(alphabet, lettre) for lettre in chaine]
```

4. (a) `ind = indices_msg[k] + indices_cle[k]` ; (b) `ind = ind - 26` ; (c) `indices_vers_lettres(indices_msg_chiffre)`.

5. L'assertion `len(cle) >= len(msg)` est fausse (`len('GFTZ')`$= 4 < 6 =$`len('RESEAU')`) : l'appel **lève une `AssertionError` « impossible »**. Le masque doit être au moins aussi long que le message.

6. Déchiffrement : on *soustrait* le rang de la clé (et on ajoute 26 si le résultat est négatif). `G`(6)−`F`(5)$= 1$ `B` ; `M`(12)−`V`(21)$= -9 \equiv 17$ `R` ; `E`(4)−`E`(4)$= 0$ `A` ; `D`(3)−`I`(8)$= -5 \equiv 21$ `V` ; `H`(7)−`T`(19)$= -12 \equiv 14$ `O`. Message : **`BRAVO`**.

   ▷ **Erreur fréquente.** Oublier d'ajouter 26 quand la différence est négative : `M`(12)−`V`(21)$= -9$ ne correspond à aucune lettre ; $-9 + 26 = 17$ donne `R`.

7. Il suffit de remplacer l'addition par une **soustraction** et le test « ≥ 26, on retranche 26 » par « < 0, on ajoute 26 » :

```python
def dechiffrement(msg, cle):
    assert len(cle) >= len(msg), "impossible"
    indices_msg = lettres_vers_indices(msg)
    indices_cle = lettres_vers_indices(cle)
    n = len(msg)
    indices_msg_dechiffre = []                          # INITIALISER
    for k in range(n):                                  # PARCOURIR
        ind = indices_msg[k] - indices_cle[k]
        if ind < 0:                                     # TESTER
            ind = ind + 26
        indices_msg_dechiffre.append(ind)               # METTRE A JOUR
    return indices_vers_lettres(indices_msg_dechiffre)  # RENVOYER
```

Partie B.

8. Symétrique : *une* clé commune chiffre et déchiffre. Asymétrique : *deux* clés (publique pour chiffrer, privée pour déchiffrer).

9. Bob déchiffre avec sa **clé privée** (le message a été chiffré avec sa clé publique).

10. Une tierce personne peut aussi récupérer la clé *publique* de Bob (elle est publique) et lui envoyer un message signé « Alice » : Bob ne peut pas vérifier l'expéditeur. Il manque l'**authentification** (une signature d'Alice avec *sa* clé privée).

11. **HTTPS** utilise un chiffrement **asymétrique** au début pour échanger de façon sûre une **clé symétrique** de session, puis chiffre toutes les données en **symétrique** (AES). On n'utilise pas que l'asymétrique car il est trop **lent** pour de gros volumes.

★★★ **Exercice 13** IA IA IA *Type bac — Le chiffrement de Polybe* (d'après Centres étrangers 2026, groupe 1, jour 1) ↑ énoncé p. 499

Barème indicatif (sur 8) : Q1 : 0,5 ; Q2 : 0,75 ; Q3 : 0,5 ; Q4 : 0,75 ; Q5 : 0,75 ; Q6 : 1,25 ; Q7 : 2 ; Q8 : 1,5

1. (6,2) →`C`, (3,6) →`O`, (3,5) →`D`, (2,2) →`E` : le message est **`CODE`**.

2. Avec la clé `SECURITY1024`, l'ordre d'insertion est `SECURITY1024ABDFGHJKLMNOPQVWXZ356789`. La grille (remplie ligne par ligne) place `B` en (3,2), `A` en (3,1) et `C` en (1,3). Donc `BAC` se chiffre en (3,2) (3,1) (1,3).

3. C'est **symétrique** : la *même* grille (déterminée par la clé) sert à chiffrer *et* à déchiffrer. Qui connaît la clé peut faire les deux.

4. `generer_ordre('AXU7')` renvoie `'AXU7BCDEFGHIJKLMNOPQRSTVWYZ012345689'` (on ajoute les lettres non présentes dans l'ordre alphabétique, puis les chiffres $\neq 7$).

5. Chaque ligne est une *nouvelle* liste de `n` chaînes vides, et la grille est la liste de ces `n` lignes : deux listes par compréhension imbriquées.

```
def grille_vide(n):
    return [["" for j in range(n)] for i in range(n)]
```

Autre méthode : deux boucles imbriquées, une ligne après l'autre.

```
def grille_vide(n):
    grille = []                # INITIALISER
    for i in range(n):         # PARCOURIR les lignes
        ligne = []             # INITIALISER une ligne neuve
        for j in range(n):     # PARCOURIR les colonnes
            ligne.append("")   # METTRE A JOUR
        grille.append(ligne)   # METTRE A JOUR
    return grille              # RENVOYER
```

6. (a) `range(6)` ; (b) `range(6)` ; (c) `ordre_insertion[indice]`.

7. *Appels attendus, écrits avant de coder :*

```
assert chiffrer('SECURITY1024', 'BAC') == [(3, 2), (3, 1), (1, 3)]   # cas normal
assert chiffrer('SECURITY1024', '') == []                            # cas limite : message vide
assert chiffrer('SECURITY1024', 'S') == [(1, 1)]                     # cas piege : numeros a partir de 1
```

```
def generer_dico(cle):
    grille = generer_grille(cle)
    dico = {}                                  # INITIALISER
    for i in range(6):                         # PARCOURIR les lignes
        for j in range(6):                     # PARCOURIR les colonnes
            dico[grille[i][j]] = (i + 1, j + 1)  # METTRE A JOUR : numeros a partir de 1
    return dico                                # RENVOYER

def chiffrer(cle, message):
    dico = generer_dico(cle)
    return [dico[c] for c in message]          # le couple de chaque caractere
```

8. Symétrique : *une* clé partagée (ici la grille) ; le problème est de **se l'échanger** en sécurité. Asymétrique : *deux* clés (publique/privée), ce qui permet justement d'échanger la clé symétrique quotidienne sans l'avoir jamais transmise en clair.

★ **Exercice 14** IA *Type bac — Transmettre une clé de session* (d'après Métropole 2024, jour 2) ↑énoncé p. 500

Barème indicatif (sur 2) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 1

1. Un algorithme de chiffrement **symétrique** utilise une **même clé**, partagée par l'émetteur et le destinataire, pour chiffrer et pour déchiffrer (exemple : AES).

2. Un algorithme de chiffrement **asymétrique** utilise une **paire de clés** : une clé **publique**, diffusée à tous, qui sert à chiffrer, et une clé **privée**, gardée secrète par son propriétaire, qui seule permet de déchiffrer (exemple : RSA).

3. Le serveur **chiffre la clé C avec la clé publique de Bob** et envoie le résultat. Seul Bob possède la clé privée associée : lui seul peut déchiffrer et retrouver C. Ensuite, client et serveur échangent leurs données chiffrées en symétrique avec C (plus rapide) — c'est le principe de HTTPS.

   ▷ **Erreur fréquente.** Chiffrer C avec une clé *privée* : le serveur ne possède pas celle de Bob, et ce qui est chiffré avec une clé privée se déchiffre avec la clé publique correspondante, connue de tous.

★★★ **Exercice 15** *Type bac — Les mots de passe d'un hôpital* (d'après Polynésie 2024, jour 2) ↑énoncé p. 501

Barème indicatif (sur 6) : Q1 : 1,5 ; Q2 : 1,5 ; Q3 : 1,5 ; Q4 : 1,5

1. Lignes 2, 10, 12 et 14 :

```
1  def mdp_fort(mdp):
2      if len(mdp) < 12:                    # VERIFIER la longueur
3          return False
4      majuscules = 0                       # INITIALISER
5      chiffres = 0                         # INITIALISER
6      symboles = 0                         # INITIALISER
7      for caractere in mdp:                # PARCOURIR
8          if caractere.isupper():          # TESTER
9              majuscules += 1              # METTRE A JOUR
10         if '0' <= caractere <= '9':      # TESTER
11             chiffres += 1                # METTRE A JOUR
12         if caractere in liste_symboles:  # TESTER
13             symboles += 1                # METTRE A JOUR
14     if majuscules < 2 or chiffres < 2 or symboles < 2:
15         return False
16     return True                          # RENVOYER
```

   Ligne 10 : plutôt que la méthode `isdigit` proposée par l'énoncé, on compare le caractère aux bornes `'0'` et `'9'` : les chiffres se suivent dans la table des caractères, donc `'0' <= caractere <= '9'` teste exactement « est-ce un chiffre ? » (les deux écritures sont acceptées). De même, `'A' <= c <= 'Z'` teste une majuscule non accentuée.

2. On continue à tirer des caractères tant que la longueur voulue *ou* l'un des minimums n'est pas atteint (le mot de passe peut donc dépasser `n` caractères si le hasard tarde à fournir les majuscules, chiffres ou symboles demandés) :

```
1      while len(mdp) < n or majuscules < nbr_m or chiffres < nbr_c or symboles < nbr_s:
2          # la variable c contient un caractere
3          # choisi aleatoirement dans la variable caracteres
4          c = choice(caracteres)
5          if 'A' <= c <= 'Z':
6              majuscules += 1
7          if '0' <= c <= '9':
8              chiffres += 1
9          if c in liste_symboles:
10             symboles += 1
11         mdp = mdp + c
```

   Par exemple, tout mot de passe renvoyé par `creation_mdp(12, 2, 2, 2)` est fort.

3. Lignes 5, 6, 15, 16 et 18 :

```
1  def recherche_mot(mdp):
2      mot = transforme(mdp)
3      trouve = []                    # INITIALISER
```

```
    i = 0                           # INITIALISER
    while i < len(mot):             # PARCOURIR
        if '0' <= mot[i] <= '9':    # TESTER : si le caractere est un chiffre
            i = i + 1
        elif mot[i] in liste_symboles:
            i = i + 1
        else:
            # si le caractere est une lettre, on prend les
            # lettres qui la suivent jusqu'au moment ou
            # on trouve un chiffre ou un symbole
            chaine = ''
            while i < len(mot) and not ('0' <= mot[i] <= '9') and mot[i] not in
    liste_symboles:
                chaine = chaine + mot[i]
                i = i + 1
            trouve.append(chaine)   # METTRE A JOUR
    return trouve                   # RENVOYER
```

La condition `i < len(mot)` doit être testée *en premier* (évaluation paresseuse du `and`) pour ne pas lire `mot[i]` hors de la chaîne quand le mot de passe se termine par une lettre.

4. *Appels attendus, écrits avant de coder :*

```
assert mdp_extra_fort('Chef!14Neuro@') == False    # cas normal : fort, mais contient le
                                                   # mot chef
assert mdp_extra_fort('Lit!14Bus@7#') == True      # cas limite : mots de 3 lettres
                                                   # seulement
assert mdp_extra_fort('xq7!zr') == False           # cas piege : aucun mot, mais pas fort
```

Un mot de passe extra fort est d'abord fort, puis ne contient aucun mot d'au moins 4 lettres présent dans `dicoFR` :

```
def mdp_extra_fort(mdp):
    if not mdp_fort(mdp):               # VERIFIER d'abord qu'il est fort
        return False
    for m in recherche_mot(mdp):        # PARCOURIR les mots trouves
        if len(m) >= 4 and m in dicoFR: # TESTER
            return False                # RENVOYER des qu'un mot est trouve
    return True                         # RENVOYER
```

Ainsi `mdp_extra_fort('@20!HôPiTaL&24#')` renvoie `False` (il contient « hôpital »).

★★★ **Exercice 16** *Type bac — Les casiers connectés du lycée* (sujet maison, façon bac) ↑énoncé p. 503

Barème indicatif (sur 8) : Q1 : 0,75 ; Q2 : 0,75 ; Q3 : 0,75 ; Q4 : 1 ; Q5 : 1 ; Q6 : 0,75 ; Q7 : 1 ; Q8 : 0,75 ; Q9 : 0,5 ; Q10 : 0,75

Partie A.

1. $79 =$ `01001111` et $23 =$ `00010111`. Bit à bit : `01001111 ^ 00010111 = 01011000` $= \mathbf{88}$. Puis 88 ^ 23 : `01011000 ^ 00010111 = 01001111` $= \mathbf{79}$. On retrouve le nombre de départ : appliquer deux fois le XOR avec la même clé **annule** l'opération, $(m \text{ ^ } k) \text{ ^ } k = m$.

2. (a) `k = cle[i % len(cle)]` ; (b) `resultat.append(codes[i] ^ k)`.

```
def chiffrer_xor(codes, cle):
    resultat = []                       # INITIALISER
    for i in range(len(codes)):         # PARCOURIR
        k = cle[i % len(cle)]           # la cle est repetee
        resultat.append(codes[i] ^ k)   # METTRE A JOUR
    return resultat                     # RENVOYER
```

Pour `i` $= 0, 1, 2, 3, 4, 5, 6$, l'indice `i % 3` vaut $0, 1, 2, 0, 1, 2, 0$: la clé est reprise depuis le début tous les trois caractères.

3. Chaque code chiffré vaut `codes[i] ^ k` ; le casier calcule `(codes[i] ^ k) ^ k`, qui redonne `codes[i]` (question 1), avec le *même* `k` puisque l'indice `i % len(cle)` est le même. Donc `chiffrer_xor(chiffrer_xor(m, cle), cle)` vaut `m`. C'est un chiffrement **symétrique** : la **même** clé, partagée par l'application et le casier, sert à chiffrer et à déchiffrer.

4. Si $c = m$ ^ k, alors c ^ $m = (m$ ^ $k)$ ^ $m = k$ (le XOR est commutatif, et m s'annule avec lui-même). Ève fait donc le XOR entre le chiffré et le clair connu :

   - 88 ^ 79 : `01011000 ^ 01001111` $=$ `00010111` $= 23$;
   - 80 ^ 85 : `01010000 ^ 01010101` $=$ `00000101` $= 5$;
   - 124 ^ 86 : `01111100 ^ 01010110` $=$ `00101010` $= 42$.

   Elle retrouve la clé `[23, 5, 42]`.

5. *Appels attendus, écrits avant de coder :*

```
assert retrouver_cle([88, 80, 124], [79, 85, 86], 3) == [23, 5, 42]   # cas normal
assert retrouver_cle([88, 80, 124], [79, 85, 86], 0) == []            # cas limite : n = 0
assert retrouver_cle([88, 80, 124, 69], [79, 85, 86, 82], 3) == [23, 5, 42]   # cas piege : s'arreter a n
```

   On applique la question 4 à chaque position : le nombre d'indice `i` de la clé vaut `intercepte[i] ^ debut_connu[i]`.

```
def retrouver_cle(intercepte, debut_connu, n):
    return [intercepte[i] ^ debut_connu[i] for i in range(n)]
```

   Avec `intercepte = [88, 80, 124, 69, 64, 26, 32]`, l'appel `retrouver_cle(intercepte, [79, 85, 86], 3)` renvoie `[23, 5, 42]`. Ève appelle ensuite `chiffrer_xor(intercepte, [23, 5, 42])`, qui renvoie `[79, 85, 86, 82, 69, 48, 55]`, c'est-à-dire `"OUVRE07"`.

6. Le masque jetable exige une clé **aussi longue que le message**, **aléatoire** et **utilisée une seule fois**. Ici la clé est **courte** et **répétée** dans le message, et surtout **réutilisée** pour tous les messages : connaître un morceau du clair (le début `"OUVRE"`, toujours le même) suffit à retrouver la clé, donc à lire — et à fabriquer — tous les messages.

Partie B.

7. $n = 17 \times 19 = 323$ et $\varphi = 16 \times 18 = 288$. Comme $288 = 2^5 \times 3^2$ n'est pas divisible par le nombre premier 5, $e = 5$ est bien premier avec φ. Enfin $5 \times 173 = 865 = 3 \times 288 + 1$, donc $5 \times 173 \equiv 1\ [288]$: $d = 173$ est l'inverse modulaire de e (en Python, `pow(5, -1, 288)` renvoie `173`). Clé publique : $(\mathbf{5}, \mathbf{323})$; clé privée : $(\mathbf{173}, \mathbf{323})$.

8. L'application chiffre avec la clé **publique** du casier : `C = pow(42, 5, 323)`, qui vaut 264. Le casier déchiffre avec sa clé **privée** : `K = pow(264, 173, 323)`, qui redonne 42.

9. Ève connaît la clé publique $(5, 323)$. Elle cherche un diviseur de 323 en essayant $2, 3, 5, \ldots$: elle trouve vite $323 = 17 \times 19$, calcule $\varphi = 288$, puis $d = 173$ comme le casier. Avec un vrai RSA, n est le produit de deux nombres premiers d'environ 300 chiffres chacun (n de 2 048 bits, soit 617 chiffres) : aucun algorithme connu ne sait **factoriser** un tel nombre en un temps raisonnable. La sécurité de RSA repose sur cette difficulté.

Partie C.

10. L'application, croyant parler au casier, chiffre la nouvelle clé avec la clé publique d'**Ève** : Ève la déchiffre avec sa clé privée, peut lire les messages, et même les retransmettre au vrai casier en les rechiffrant avec la vraie clé publique (attaque de l'**homme du milieu**). Il manque l'**authentification** du casier. HTTPS la règle avec un **certificat** : le serveur présente sa clé publique dans un certificat signé par une **autorité de certification** de confiance, que le client vérifie avant d'envoyer la clé symétrique. Un faux casier ne pourrait pas présenter un certificat valide pour sa propre clé.

Critiquer une réponse d'IA

★★ **Exercice 17** *Des clés RSA fabriquées par un assistant d'IA* ↑énoncé p. 504

1. **Non.** Avec $n = 55$: `d = pow(3, -1, 55)` vaut 37 (car $3 \times 37 = 111 = 2 \times 55 + 1$). Puis `chiffrer(7)` $= 7^3 \bmod 55 = 343 \bmod 55 = 13$, et `dechiffrer(13)` $= 13^{37} \bmod 55 =$ **18**, et non 7 : le déchiffrement **ne redonne pas le message**.

2. L'erreur : l'inverse modulaire de e doit être pris **modulo** $\varphi = (p-1)(q-1)$, pas modulo n (cours : « *Calculer d, l'inverse modulaire de e : le nombre tel que $e \times d \equiv 1\ [\varphi]$* »). Lignes corrigées :

```
phi = (p - 1) * (q - 1)     # 40
d = pow(e, -1, phi)         # 27, car 3 * 27 = 81 = 2 * 40 + 1
```

   Avec $d = 27$: `dechiffrer(13)` $= 13^{27} \bmod 55 = 7$. ✓

3. Faire un **aller-retour** sur un petit message : `dechiffrer(chiffrer(7))` doit renvoyer 7. C'est un test d'une ligne, à faire systématiquement sur tout code de chiffrement — ici sur un exemple assez petit pour qu'on puisse vérifier la réponse à la main ($d = 27$, car $3 \times 27 = 81 = 2 \times 40 + 1$).

S'entraîner à l'oral

★★ **Exercice 18** *Expliquer en deux minutes* ↑énoncé p. 505

▶ **Solution.** Pas de texte à apprendre par cœur : voici les **éléments attendus** pour chaque sujet. L'ordre, les mots et l'exemple peuvent être différents ; l'explication est réussie si ces idées y sont, justes et reliées entre elles.

Sujet 1.

- **Symétrique** : une seule clé secrète, partagée, sert à chiffrer et à déchiffrer (César, XOR, AES) ; c'est rapide, mais il faut transmettre la clé sans qu'on l'intercepte.
- **Asymétrique** : deux clés liées ; la clé **publique**, diffusée, sert à chiffrer ; la clé **privée**, gardée secrète, sert à déchiffrer.
- Exemple : Alice publie sa clé publique ; Bob chiffre son message avec ; seule Alice peut le déchiffrer.
- Piège : l'asymétrique est beaucoup plus lent ; on ne l'utilise pas pour chiffrer de gros volumes.

Sujet 2.

- Au début de la connexion (poignée de main TLS), le serveur envoie son **certificat** : sa clé publique, garantie par une autorité de certification.
- Le navigateur s'en sert pour transmettre de façon sûre une **clé de session** (symétrique), que seul le serveur peut retrouver.
- Toute la suite de l'échange est chiffrée en **symétrique** avec cette clé de session : c'est rapide.

- Piège : sans vérification du certificat, une attaque de l'homme du milieu est possible ; HTTPS protège le transport, pas l'honnêteté du site.

Sujet 3.

- Clé publique (e, n) avec $n = p \times q$, où p et q sont de grands nombres premiers gardés secrets.
- Calculer la clé privée d demande $\varphi = (p-1)(q-1)$, donc de connaître p et q, donc de **factoriser** n.
- Multiplier deux nombres est facile ; factoriser un nombre de plusieurs centaines de chiffres est hors de portée : on ne connaît pas d'algorithme efficace.
- Exemple : $n = 55 = 5 \times 11$ se factorise de tête ; la sécurité repose sur la **taille** des clés, pas sur le secret de la méthode.

CORRIGÉS DES TP

TP Chiffrer, casser, signer

↑énoncé p. 508

Partie A — César : chiffrer, puis casser sans la clé

Exercice 1 *César, version complète*

```
def decaler(lettre, k):
    if "A" <= lettre <= "Z":
        return chr((ord(lettre) - 65 + k) % 26 + 65)
    return lettre

def cesar(texte, k):
    resultat = ""
    for c in texte:
        resultat = resultat + decaler(c, k)
    return resultat

clair = normaliser(TEXTE)
chiffre = cesar(clair, 11)
print(chiffre[:50])          # OPAFTD W'LYETBFTEP, WPD SZXXPD NSPCNSPYE L NLNSPC
print(cesar(chiffre, -11) == clair)                    # True
```

Exercice 2 *La force brute*

1. `for k in range(26): print(k, cesar(message, -k))`. Seule la clé **19** donne du français : `RENDEZ-VOUS DEVANT LE MUSEE OCEANOGRAPHIQUE A MIDI`.

2. Compter les mots d'un dictionnaire présents dans chaque candidat, ou comparer les fréquences de ses lettres à celles du français (c'est l'idée de l'exercice suivant).

Exercice 3 — *L'analyse de fréquences, automatisée*

```python
def frequences(texte):
    compte = {}
    for c in texte:
        if "A" <= c <= "Z":
            if c in compte:
                compte[c] += 1
            else:
                compte[c] = 1                # premiere apparition
    return compte

def plus_frequente(texte):
    compte = frequences(texte)
    meilleure = None
    for lettre in compte:
        if meilleure is None or compte[lettre] > compte[meilleure]:
            meilleure = lettre
    return meilleure

def casser_cesar(chiffre):
    return (ord(plus_frequente(chiffre)) - ord("E")) % 26

def score(texte):
    s = 0
    for c in texte:
        if "A" <= c <= "Z":
            s += FREQ_FR[c]
    return s

def casser_cesar_score(chiffre):
    meilleure_cle = 0
    meilleur_score = -1
    for k in range(26):
        sc = score(cesar(chiffre, -k))
        if sc > meilleur_score:
            meilleur_score = sc
            meilleure_cle = k
    return meilleure_cle
```

1. Dans `clair` : `E` (297 fois), `S` (150), `A` (108).

2. `casser_cesar(cesar(clair, 11))` renvoie bien `11` (sur le message intercepté : `19`).

3. Le chiffré est `HAAHXBL H S HBIL` ; sa lettre la plus fréquente est `H` (le `A` du clair) : la fonction renvoie 3 au lieu de 7. Sur un message de 13 lettres, les fréquences n'ont rien de statistique : ici, aucun `E` n'est la lettre la plus fréquente.

4. `casser_cesar_score` trouve 7 sur le message court, 19 sur le message intercepté et 11 sur le long texte : elle utilise *toutes* les lettres et pas seulement la plus fréquente.

Partie B — Vigenère, et comment Babbage l'a cassé

Exercice 4 — *Chiffrer avec une clé-mot*

```python
def vigenere(texte, cle, sens=1):
    resultat = ""
    i = 0                                    # nombre de lettres deja chiffrees
    for c in texte:
        if "A" <= c <= "Z":
            k = ord(cle[i % len(cle)]) - 65
            resultat = resultat + decaler(c, sens * k)
            i += 1
        else:
```

```
            resultat = resultat + c
    return resultat
```

`vigenere("NSI ROCKS", "CLE")` vaut `"PDM TZGMD"`, et `vigenere("PDM TZGMD", "CLE", -1)` redonne `"NSI ROCKS"`.

Exercice 5 *Casser Vigenère quand on connaît la longueur de la clé*

1. Elles sont toutes décalées du même nombre de rangs : c'est un **chiffre de César**, que l'on sait casser.

2. et 3.

```
def colonnes(chiffre, L):
    cols = ["" for j in range(L)]           # L chaines vides
    i = 0                                   # rang de la lettre dans le chiffre
    for c in chiffre:
        if "A" <= c <= "Z":
            cols[i % L] = cols[i % L] + c
            i += 1
    return cols

def casser_vigenere(chiffre, L):
    cle = ""
    for col in colonnes(chiffre, L):
        cle = cle + chr(casser_cesar_score(col) + 65)
    return cle
```

`casser_vigenere(vigenere(clair, "MONACO"), 6)` renvoie `"MONACO"`, et le déchiffrement redonne le texte clair. Chaque colonne compte environ 250 lettres : assez pour que les fréquences parlent.

Exercice 6 *Défi : trouver la longueur de la clé*

```
def indice_coincidence(texte):
    compte = frequences(texte)
    n = 0
    s = 0
    for lettre in compte:
        x = compte[lettre]
        n += x
        s += x * (x - 1)
    return s / (n * (n - 1))

def longueur_cle(chiffre, Lmax=10, seuil=0.065):
    for L in range(1, Lmax + 1):
        total = 0
        for col in colonnes(chiffre, L):
            total += indice_coincidence(col)
        if total / L > seuil:
            return L
    return None
```

1. IC(`clair`) $\approx 0{,}083$; pour 2 000 lettres au hasard, $IC \approx 0{,}038$ à $0{,}039$ selon le tirage (proche de $1/26$). Le français répète beaucoup certaines lettres (`E`, `S`, `A`…), le hasard non.

2. Moyennes pour la clé `MONACO`, $L = 1$ à 8 : 0,052 ; 0,062 ; 0,063 ; 0,062 ; 0,051 ; **0,083** ; 0,052 ; 0,063. Seul $L = 6$ (et ses multiples) redonne l'indice du français : chaque colonne est alors un simple César, qui conserve l'indice ; pour un autre L, une colonne mélange plusieurs décalages et l'indice baisse.

3. Pour `vigenere(clair, "SECRETS")`, `longueur_cle` renvoie 7 et `casser_vigenere(..., 7)` renvoie `"SECRETS"` : Vigenère cassé sans aucune indication.

Partie C — Le masque jetable, et l'erreur fatale

Exercice 7 *Un masque, deux messages*

```python
import secrets

def xor_octets(a, b):
    resultat = [a[i] ^ b[i] for i in range(len(a))]
    return bytes(resultat)

m1 = b"RDV A 18H SOUS LE PONT"
m2 = b"LE CODE EST 4071 MERCI"
masque = secrets.token_bytes(22)
c1 = xor_octets(m1, masque)
c2 = xor_octets(m2, masque)             # ERREUR : masque reutilise
print(xor_octets(c1, masque))            # b'RDV A 18H SOUS LE PONT'
print(xor_octets(c1, c2) == xor_octets(m1, m2))   # True
print(xor_octets(xor_octets(c1, c2), m1))         # b'LE CODE EST 4071 MERCI'
```

2. `random` produit une suite **prévisible** (elle est entièrement déterminée par sa graine) : elle convient aux simulations, pas aux secrets. `secrets` utilise l'aléa du système d'exploitation, imprévisible.

3. $c_1 \oplus c_2 = (m_1 \oplus k) \oplus (m_2 \oplus k) = m_1 \oplus m_2 \oplus (k \oplus k) = m_1 \oplus m_2$: le masque **disparaît**.

4. $(c_1 \oplus c_2) \oplus m_1 = m_2$: connaissant (ou devinant) un message, Ève lit l'autre sans le masque. Le masque jetable n'est inviolable que s'il ne sert **qu'une seule fois**.

Partie D — Un RSA complet

Exercice 8 *Six jeux de clés*

1. $n = 55$, $\varphi = 4 \times 10 = 40$; $3d \equiv 1\ [40]$: $3 \times 27 = 81 = 2 \times 40 + 1$, donc $d = 27$. $C = 7^3 \bmod 55 = 343 \bmod 55 = 13$, et `pow(13, 27, 55)` vaut 7.

2.

```python
jeux = [(3, 11, 3, 4), (5, 11, 3, 7), (5, 7, 5, 10),
        (7, 11, 7, 2), (11, 13, 7, 9), (13, 17, 5, 8)]
for p, q, e, M in jeux:
    n = p * q
    phi = (p - 1) * (q - 1)
    d = pow(e, -1, phi)
    C = pow(M, e, n)
    assert pow(C, d, n) == M
    print(p, q, n, phi, e, d, M, C)
```

Cinquième ligne : $n = 143$, $\varphi = 120$, $d = 103$, $C = 48$. Les six vérifications passent.

3. Si $d = e$, la clé privée est **connue de tous** (elle est égale à la clé publique) : n'importe qui déchiffre. Cela arrive pour de tout petits φ ; il faut le refuser lors de la génération.

Exercice 9 *Calculer d soi-même : l'algorithme d'Euclide étendu*

```python
def pgcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

def euclide_etendu(a, b):
    if b == 0:                              # condition d'arret
        return (a, 1, 0)
    g, u, v = euclide_etendu(b, a % b)
    return (g, v, u - (a // b) * v)

def inverse_modulaire(e, phi):
    g, u, v = euclide_etendu(e, phi)
```

```
    if g != 1:
        return None
    return u % phi
```

1. `euclide_etendu(240, 46)` vaut `(2, -9, 47)` et $240 \times (-9) + 46 \times 47 = -2160 + 2162 = 2$.

2. $eu = 1 - \varphi v$: le reste de la division de eu par φ vaut 1, c'est-à-dire $eu \equiv 1\ [\varphi]$. Comme u peut être négatif (ici `euclide_etendu(3, 40)` vaut `(1, -13, 1)`), on prend `u % phi`, compris entre 0 et $\varphi - 1$: `-13 % 40` vaut 27, et l'on retrouve bien $d = 27$ pour $p = 5$, $q = 11$.

3. `inverse_modulaire` et `pow(e, -1, phi)` coïncident sur les six jeux ; `inverse_modulaire(4, 20)` renvoie `None` (pgcd $= 4$).

Exercice 10 *Fabriquer ses propres clés*

```
def est_premier(n):
    if n < 2:
        return False
    d = 2
    while d * d <= n:
        if n % d == 0:
            return False
        d += 1
    return True

def premier_aleatoire(mini, maxi):
    p = random.randint(mini, maxi)
    while not est_premier(p):
        p = random.randint(mini, maxi)
    return p

def generer_cles(mini=1000, maxi=9999, e=65537):
    p = premier_aleatoire(mini, maxi)
    q = premier_aleatoire(mini, maxi)
    while p == q or pgcd(e, (p - 1) * (q - 1)) != 1:
        p = premier_aleatoire(mini, maxi)       # on recommence
        q = premier_aleatoire(mini, maxi)
    n = p * q
    phi = (p - 1) * (q - 1)
    return (e, n), (inverse_modulaire(e, phi), n)
```

2. Avec `random.seed(2026)` : clé publique `(65537, 32431187)`, clé privée `(18049793, 32431187)` ($n = 4931 \times 6577$).

3. Si $p = q$, $n = p^2$ et l'on retrouve p par une simple racine carrée ; de plus, la formule $\varphi = (p-1)(q-1)$ n'est plus la bonne et le déchiffrement échoue. Si $\text{pgcd}(e, \varphi) \neq 1$, e n'a pas d'inverse modulo φ : pas de clé privée.

Exercice 11 *Chiffrer un vrai message*

```
def chiffrer_message(message, cle_publique):
    e, n = cle_publique
    return [pow(ord(c), e, n) for c in message]

def dechiffrer_message(nombres, cle_privee):
    d, n = cle_privee
    message = ""
    for x in nombres:
        message = message + chr(pow(x, d, n))
    return message
```

1. Le déchiffrement redonne `"Rendez-vous à 18 h"`. RSA travaille modulo n : un nombre $M \geqslant n$ serait ramené à son reste et ne pourrait pas être retrouvé.

2. Début de la liste : `[28766602, 17717906, 4785882, 22928393, 17717906, ...]` : les deux `e` de « Rendez » donnent le même nombre 17 717 906. Caractère par caractère, RSA se

comporte comme un **chiffrement par substitution** : l'analyse de fréquences de la partie A s'applique. D'où le remplissage aléatoire en pratique.

Exercice 12 *Signer*

```python
def signer(message, cle_privee):
    d, n = cle_privee
    return pow(empreinte(message, n), d, n)

def verifier(message, signature, cle_publique):
    e, n = cle_publique
    return pow(signature, e, n) == empreinte(message, n)
```

Avec les clés de l'exercice 10 : la signature vaut 24 670 169 ; `verifier` renvoie `True` pour le message signé et `False` pour « 1000 euros ». La signature garantit l'**authenticité** (seul le détenteur de la clé privée a pu la produire) et l'**intégrité** (le message n'a pas été modifié) ; elle ne le rend pas secret.

Partie E — Factoriser n : l'expérience qui rassure

Exercice 13 *Quand la factorisation devient lente*

```python
def factoriser(n):
    d = 2
    while d * d <= n:
        if n % d == 0:
            return d, n // d
        d += 1
    return None

for n in (1373249, 133763569, 13334100011, 1333361000071, 133333863333859):
    debut = time.perf_counter()
    f = factoriser(n)
    print(n, len(str(n)), f, time.perf_counter() - debut)
```

n	chiffres	facteurs	durée mesurée
1373249	7	$1\,009 \times 1\,361$	0,0001 s
133763569	9	$10\,007 \times 13\,367$	0,001 s
13334100011	11	$100\,003 \times 133\,337$	0,013 s
1333361000071	13	$1\,000\,003 \times 1\,333\,357$	0,13 s
133333863333859	15	$10\,000\,019 \times 13\,333\,361$	1,26 s

3. La durée est multipliée par environ **10** à chaque fois que n gagne deux chiffres. La boucle tourne jusqu'au plus petit facteur $p \approx \sqrt{n}$; quand n est multiplié par 100, $\sqrt{n}$ (donc le nombre de tours) est multiplié par 10. Le coût est exponentiel en le *nombre de chiffres* de n.

4. De 15 à 617 chiffres, on ajoute 602 chiffres, soit 301 fois un facteur 10 : environ $1{,}26 \times 10^{301}$ secondes, soit plus de 10^{283} fois l'âge de l'Univers.

5. Avec des premiers à 4 chiffres, n en a 7 ou 8 : `factoriser(32431187)` renvoie `(4931, 6577)` instantanément, puis $\varphi = 4930 \times 6576$ et `d = pow(65537, -1, phi)` : la clé privée est retrouvée et le message déchiffré en moins d'une seconde. De tels nombres ne protègent rien : la sécurité exige des premiers d'environ 300 chiffres chacun (un n de 2 048 bits a 617 chiffres, comme dans la question 4).

Bilan — éléments attendus

- César et Vigenère conservent les **fréquences** des lettres (globalement, ou colonne par colonne) ; le masque réutilisé laisse fuir $m_1 \oplus m_2$. Un bon chiffrement ne doit rien laisser paraître de la structure du message.
- RSA : choisir p, q premiers (`premier_aleatoire`) ; $n = pq$, $\varphi = (p-1)(q-1)$; e premier avec φ (`pgcd`) ; d inverse de e modulo φ (`euclide_etendu`, `inverse_modulaire`).
- La sécurité repose sur la difficulté de **factoriser** n : chaque paire de chiffres en plus multiplie le temps par 10 (partie E).
- Chiffrer : clé **publique** du destinataire (confidentialité). Signer : clé **privée** de l'expéditeur (authenticité, intégrité).

Réponses des questions de la fiche de révision

↑ questions p. 515

Se tester

1. Déchiffrer : avec la clé ; décrypter : sans la clé (l'attaquant) (§ I).
2. `CRR` ($(25+3) \bmod 26 = 2$, soit `C`) (§ II.A).
3. $n = 35$, $\varphi = 4 \times 6 = 24$ (§ IV.C).
4. Sa **propre clé privée** (§ III.C).
5. Trop lent : RSA n'échange que la clé de session, AES chiffre les données (§ V).

Et avant ?

1. $2^{32-26} - 2 = 2^6 - 2 = 62$ (ch. 9, § III).
2. `len(tab) <= 1` : une liste de 0 ou 1 élément est déjà triée (ch. 7, § III.B).
3. le programme est un texte inerte (un fichier) ; le processus est un programme en cours d'exécution (ch. 4, § II).

CORRIGÉS DU CHAPITRE 11

11 Programmation dynamique

CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Le calcul qui n'en finit pas

↑énoncé p. 528

Exercice 1 *Chronométrer*

1. Temps mesurés sur un ordinateur portable (Python 3.8) :

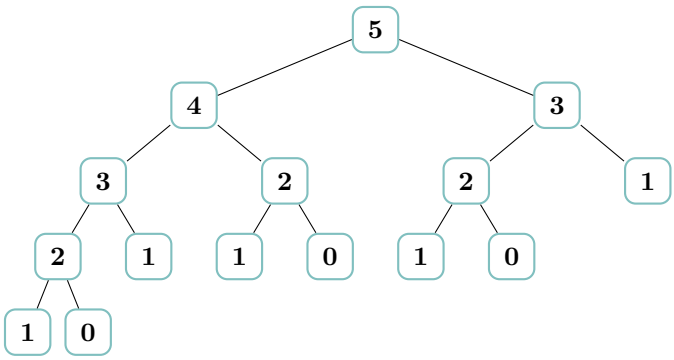
n	20	23	26	29	32	35
temps (s)	0,002	0,007	0,031	0,13	0,56	2,3

2. Le temps est multiplié par environ 4 (ici 4,1 à 4,4) quand n augmente de 3.

3. $n = 38$: environ $4 \times 2{,}3 \approx 10$ s. $n = 50$: cinq fois « +3 » de plus que 35, soit $2{,}3 \times 4^5 \approx 2\,400$ s, environ 40 minutes ; un calcul plus fin (facteur 1,6 par unité) donne de l'ordre d'une heure. Le programme est inutilisable au-delà de $n \approx 40$.

Exercice 2 *Dessiner les appels*

4. Arbre des appels de `fibo(5)` :



5. 15 appels.

6. `fibo(3)` : 2 fois ; `fibo(2)` : 3 fois ; `fibo(1)` : 5 fois (et `fibo(0)` : 3 fois).

7. Le sous-arbre de racine 3 apparaît deux fois (sous 4 et sous 5), celui de racine 2 trois fois. L'ordinateur **refait tout le calcul** à chaque fois : il ne se souvient de rien.

Exercice 3 *Compter sans dessiner*

8. Un appel `fibo(n)` compte pour 1, puis déclenche `fibo(n-1)`, qui produit $A(n-1)$ appels, et `fibo(n-2)`, qui en produit $A(n-2)$.

9. Tableau :

n	0	1	2	3	4	5	6	7	8
$A(n)$	1	1	3	5	9	15	25	41	67

10. $A(n)$ est multiplié par environ 1,6 à chaque pas ($67/41 \approx 1{,}63$), donc par $1{,}6^3 \approx 4{,}2$ quand n augmente de 3 : c'est exactement le facteur mesuré à l'exercice 1. Le temps est proportionnel au nombre d'appels, qui croît de façon **exponentielle** ($A(35) \approx 30$ millions, $A(40) \approx 331$ millions).

Exercice 4 *Imaginer un carnet*

11. Le carnet se remplit dans l'ordre `fibo(2)`, `fibo(3)`, `fibo(4)`, `fibo(5)`, `fibo(6)` : on descend d'abord tout à gauche jusqu'à `fibo(1)` et `fibo(0)`, puis on remonte ; chaque appel de droite (`fibo(2)` sous `fibo(4)`, `fibo(3)` sous `fibo(5)`, `fibo(4)` sous `fibo(6)`) est **lu** dans le carnet.

12. 5 additions avec le carnet (une par valeur $F_2, \dots, F_6$) contre 12 sans. Pour `fibo(40)` : 39 additions avec le carnet, contre plus de 165 millions sans. (En comptant les appels, la version avec carnet en fait 11 pour `fibo(6)` et 79 pour `fibo(40)`.)

13. F_k pour $k = 0$ à 10 : $0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55$. Une seule boucle, aucune récursivité.

Exercice 5 *Mettre des mots*

14. La version de l'exercice 1 est lente parce qu'elle **recalcule sans cesse les mêmes valeurs**. Le carnet permet de **ne calculer chaque valeur qu'une fois**. Avec le carnet, on part du problème **complet** (`fibo(6)`) et on descend ; avec le tableau, on part **des plus petits cas** (F_0, F_1) et on remonte.

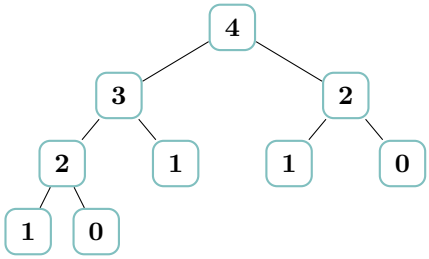
15. Propositions possibles : « prise de notes », « mémoire », « cache ». Le mot du cours est **mémoïsation** (de l'anglais *memo*, un pense-bête) ; le remplissage du tableau s'appelle la **tabulation**.

CORRIGÉS DES EXERCICES

> **Remarque**
> On donne **une** version simple de chaque programme ; d'autres sont possibles. On rappelle le réflexe : écrire la relation de récurrence, repérer le cas de base, puis emballer en mémoïsation ou en tabulation.

Le déclic : ne pas recalculer deux fois

Exercice 1 ↑énoncé p. 543 **1.** Arbre des appels de `fibo(4)` :



2. `fibo(1)` apparaît **3 fois** dans cet arbre.

3. Un appel à `fibo(n)` (pour $n \geq 2$) c'est : *lui-même* (1 appel), plus tous les appels de `fibo(n-1)`, plus tous ceux de `fibo(n-2)`. D'où $A(n) = 1 + A(n-1) + A(n-2)$, avec $A(0) = A(1) = 1$. On calcule :

n	0	1	2	3	4	5	6
$A(n)$	1	1	3	5	9	15	25

▷ **Erreur fréquente.** Trouver $A(4) = 5$ en ne comptant que les feuilles, ou 8 en oubliant l'appel initial : chaque nœud de l'arbre est un appel, d'où 9.

4. $A(n)$ croît comme Fibonacci lui-même, c'est-à-dire **exponentiellement** : `fibo(60)` déclencherait environ 5×10^{12} appels (cinq mille milliards). Au rythme mesuré dans l'activité de découverte (environ 2 secondes pour les 30 millions d'appels de `fibo(35)`), cela représente de l'ordre de 4 jours de calcul ininterrompu.

Exercice 2 ↑énoncé p. 543 **Démarche :** on garde la fonction récursive ; avant de calculer F_n, on regarde s'il est déjà dans le dictionnaire `memo` ; sinon, on le calcule une seule fois et on l'y range. La valeur par défaut `None`, remplacée par un dictionnaire neuf, permet l'appel `fibo_memo(50)`.

```
def fibo_memo(n, memo=None):
    if memo is None:                                    # INITIALISER (premier appel)
        memo = {}
    if n <= 1:                                          # CAS DE BASE
        return n
    if n in memo:                                       # VERIFIER : deja calcule ?
        return memo[n]
    memo[n] = fibo_memo(n - 1, memo) + fibo_memo(n - 2, memo)   # APPEL RECURSIF, METTRE A JOUR
    return memo[n]                                      # RENVOYER
```

`fibo_memo(50)` renvoie bien `12586269025`, instantanément : chaque F_k n'est calculé qu'une fois, le coût est linéaire.

▷ **Erreur fréquente.** Créer `memo = {}` au début de chaque appel sans le transmettre aux appels récursifs : chaque appel repart d'un carnet vide, et l'on retrouve le coût exponentiel.

Exercice 3 ↑énoncé p. 543 **1.** $M(1) = 1$ (un pas de 1). $M(2) = 2$: «1+1» ou «2». $M(3) = 3$: «1+1+1», «1+2», «2+1». $M(4) = 5$.

2. Pour arriver en haut de n marches, le **dernier** pas est soit de 1 (on venait de la marche $n-1$), soit de 2 (on venait de $n-2$). Ces deux familles de chemins sont disjointes et couvrent tout : $M(n) = M(n-1) + M(n-2)$. On reconnaît la suite de **Fibonacci** (décalée).

3. Démarche : même schéma que `fibo_memo`, avec la relation $M(n) = M(n-1) + M(n-2)$ et les conditions d'arrêt $M(0) = M(1) = 1$. *Appels attendus, écrits avant de coder :*

```
assert monter(4) == 5     # cas normal
assert monter(1) == 1     # cas limite : une seule marche
assert monter(2) == 2     # cas piege : pas la valeur de fibo(2), qui vaut 1
```

```python
def monter(n, memo=None):
    if memo is None:                                    # INITIALISER (premier appel)
        memo = {}
    if n <= 1:                                          # CAS DE BASE : M(0) = M(1) = 1
        return 1
    if n in memo:                                       # VERIFIER : deja calcule ?
        return memo[n]
    memo[n] = monter(n - 1, memo) + monter(n - 2, memo)   # APPEL RECURSIF, METTRE A JOUR
    return memo[n]                                      # RENVOYER
```

`monter(10)` renvoie 89.

▷ **Erreur fréquente.** Reprendre telles quelles les conditions d'arrêt de Fibonacci (`return n`) : on aurait $M(0) = 0$, puis $M(2) = 1$ au lieu de 2.

De bas en haut : construire une table

Exercice 4 ↑énoncé p. 544 **1.** On applique $u_n = u_{n-1} + 2\,u_{n-2}$ case après case :

n	0	1	2	3	4	5	6
u_n	2	1	5	7	17	31	65

(par exemple $u_2 = 1 + 2 \times 2 = 5$ et $u_6 = 31 + 2 \times 17 = 65$).

2. On remplit les cases **de gauche à droite**, des petits indices vers les grands : chaque case n'utilise que des cases déjà remplies. La case 6 n'a besoin que des cases 5 et 4.

3. Il faut remplir $n + 1$ cases, chacune en un calcul : le coût est **linéaire** (de l'ordre de n), contre un coût exponentiel pour une récursion naïve qui recalculerait les mêmes termes.

Exercice 5 ↑énoncé p. 544 **Démarche :** un tableau de $n + 1$ cases ; on y place F_0 et F_1, puis une boucle remplit chaque case à partir des deux précédentes, des petits indices vers les grands.

```python
def fibo_tab(n):
    if n <= 1:                                  # CAS DE BASE
        return n
    table = [0] * (n + 1)                       # INITIALISER
    table[1] = 1                                # INITIALISER
    for k in range(2, n + 1):                   # PARCOURIR : petits indices d'abord
        table[k] = table[k - 1] + table[k - 2]  # METTRE A JOUR
    return table[n]                             # RENVOYER
```

Les valeurs coïncident avec `fibo_memo` pour tout n (c'est la même suite, calculée dans l'autre sens).

Autre méthode : chaque case n'utilise que les **deux** précédentes ; il suffit donc de garder deux variables au lieu de tout le tableau (même coût linéaire, mémoire constante).

```python
def fibo_deux_variables(n):
    precedent = 0                       # INITIALISER : F_0
    courant = 1                         # INITIALISER : F_1
    for k in range(n):                  # PARCOURIR
        suivant = precedent + courant   # METTRE A JOUR
        precedent = courant
        courant = suivant
    return precedent                    # RENVOYER : F_n
```

Exercice 6 ↑énoncé p. 544 **Démarche :** même structure que `fibo_tab` ; seules changent les deux premières cases, $M(0) = M(1) = 1$.

```
def monter_tab(n):
    if n <= 1:                                  # CAS DE BASE
        return 1
    table = [1] * (n + 1)                       # INITIALISER : M(0) = M(1) = 1
    for k in range(2, n + 1):                   # PARCOURIR
        table[k] = table[k - 1] + table[k - 2]  # METTRE A JOUR
    return table[n]                             # RENVOYER
```

Exercice 7 ↑énoncé p. 544 **1.** Le meilleur chemin est $3 \to 7 \to 4 \to 9$, de score $3 + 7 + 4 + 9 = \mathbf{23}$.

2. Depuis la case (i, j), on descend vers $(i + 1, j)$ ou $(i + 1, j + 1)$, et on garde le meilleur :

$$S(i, j) = t[i][j] + \max\left(S(i + 1, j),\ S(i + 1, j + 1)\right),$$

avec le cas de base $S(\text{dernier niveau}, j) = t[\text{dernier}][j]$.

3. Démarche : la table a la forme du triangle ; son dernier niveau est connu (cas de base) ; on remonte niveau par niveau avec la relation de la question 2. *Appels attendus, écrits avant de coder :*

```
assert meilleur_score([[3], [7, 4], [2, 4, 6], [8, 5, 9, 3]]) == 23   # cas normal
assert meilleur_score([[5]]) == 5                                      # cas limite : un seul
                                                                       # niveau
assert meilleur_score([[1], [9, 2], [1, 1, 50]]) == 53                 # cas piege : le 9
                                                                       # attire, le 50 gagne
```

```
def meilleur_score(t):
    n = len(t)
    s = [[x for x in niveau] for niveau in t]   # INITIALISER : copie, dernier niveau deja bon
    for i in range(n - 2, -1, -1):              # PARCOURIR : du bas vers le haut
        for j in range(len(t[i])):              # PARCOURIR
            s[i][j] = t[i][j] + max(s[i + 1][j], s[i + 1][j + 1])   # METTRE A JOUR
    return s[0][0]                              # RENVOYER
```

`meilleur_score([[3],[7,4],[2,4,6],[8,5,9,3]])` renvoie 23.

▷ **Erreur fréquente.** Remplir la table du sommet vers la base : la case (i, j) a besoin des deux cases du niveau $i + 1$, qui doivent être déjà remplies. On part du dernier niveau et l'on remonte.

Exercice 8 ↑énoncé p. 545 **1.** `trib_a` procède par **mémoïsation** (*top-down*) : elle reste récursive et range ses résultats dans le dictionnaire `memo`. `trib_b` procède par **tabulation** (*bottom-up*) : une boucle remplit le tableau `table` des petits indices vers les grands, sans aucun appel récursif.

2. Les termes successifs sont $0, 0, 1, 1, 2, 4, 7$: les deux fonctions renvoient 7.

3. Le test `if n not in memo:` : le calcul n'est lancé que si T_n n'est pas déjà dans le carnet ; sinon on relit directement `memo[n]`.

Le rendu de monnaie

Exercice 9 ↑énoncé p. 545 **1.** Glouton pour 6 avec {1,3,4} : 4, puis 1, puis 1 ⇒ **3 pièces**.

2. $3+3=6 \Rightarrow$ **2 pièces**. Le glouton n'est donc **pas optimal**.

▷ **Erreur fréquente.** Croire le glouton toujours optimal parce qu'il l'est avec les pièces en euros : c'est le système de pièces qui décide.

3. $R(0)=0$ et $R(s)=1+\min\limits_{p\le s} R(s-p)$ (on essaie chaque première pièce p et on garde la meilleure suite).

4.

s	0	1	2	3	4	5	6
$R(s)$	0	1	2	1	1	2	2

Par exemple $R(6)=1+\min(R(5),R(3),R(2))=1+\min(2,1,2)=2$ (dernière pièce 3).

Exercice 10 ↑énoncé p. 545 **Démarche :** on tabule la relation $R(s)=1+\min R(s-p)$: `table[0]` vaut 0, les autres cases $+\infty$ au départ (« pas encore de solution ») ; pour chaque somme, de la plus petite à la plus grande, on essaie chaque pièce comme dernière pièce. *Appels attendus, écrits avant de coder :*

```
assert rendu(11, [1, 2, 5]) == 3      # cas normal
assert rendu(0, [1, 2, 5]) == 0       # cas limite : rien a rendre
assert rendu(6, [1, 3, 4]) == 2       # cas piege : le glouton en donne 3
```

```
def rendu(somme, pieces):
    table = [0] + [float("inf")] * somme                # INITIALISER : table[0] = 0, le
                                                        # reste +infini
    for s in range(1, somme + 1):                       # PARCOURIR : sommes croissantes
        for p in pieces:                                # PARCOURIR
            if p <= s and table[s - p] + 1 < table[s]:  # TESTER : mieux avec p ?
                table[s] = table[s - p] + 1             # METTRE A JOUR
    return table[somme]                                 # RENVOYER
```

`rendu(6, [1,3,4]) == 2` et `rendu(11, [1,2,5]) == 3`.

▷ **Erreur fréquente.** Initialiser toute la table à 0 : le test « strictement plus petit » n'est alors jamais vrai, et la fonction renvoie 0 (c'est l'erreur de l'exercice 21). Seule `table[0]` vaut 0.

Exercice 11 ↑énoncé p. 545 On mémorise, pour chaque somme, la dernière pièce utilisée, puis on remonte : *Appels attendus, écrits avant de coder :*

```
assert rendu_pieces(6, [1, 3, 4]) == [3, 3]     # cas normal
assert rendu_pieces(0, [1, 3, 4]) == []         # cas limite : rien a rendre
assert rendu_pieces(8, [1, 4, 5]) == [4, 4]     # cas piege : le glouton donnerait [5, 1, 1, 1]
```

```
def rendu_pieces(somme, pieces):
    table = [0] + [float("inf")] * somme                # INITIALISER
    derniere = [None] * (somme + 1)                     # INITIALISER : derniere piece pour
                                                        # atteindre s
    for s in range(1, somme + 1):                       # PARCOURIR
        for p in pieces:                                # PARCOURIR
            if p <= s and table[s - p] + 1 < table[s]:  # TESTER
                table[s] = table[s - p] + 1             # METTRE A JOUR
                derniere[s] = p                         # METTRE A JOUR
    solution = []                                       # INITIALISER
    s = somme
    while s > 0:                                        # PARCOURIR : on remonte
        solution.append(derniere[s])                    # METTRE A JOUR
        s = s - derniere[s]
    return solution                                     # RENVOYER
```

`rendu_pieces(6, [1,3,4])` renvoie `[3, 3]`.

Des tables à deux dimensions

Exercice 12 ↑énoncé p. 546

1. Pour atteindre (i, j), le robot arrive soit de la case du dessus $(i-1, j)$, soit de celle de gauche $(i, j-1)$: $N(i, j) = N(i-1, j) + N(i, j-1)$. Sur la première ligne et la première colonne, il n'y a qu'un seul chemin (tout droit), donc les valeurs y valent toutes 1.

2. Démarche : une table de `L` lignes et `C` colonnes remplie de 1 (première ligne et première colonne) ; on remplit les autres cases ligne par ligne avec la relation de la question 1. *Appels attendus, écrits avant de coder :*

```
assert nb_chemins(3, 3) == 6     # cas normal
assert nb_chemins(1, 5) == 1     # cas limite : une seule ligne
assert nb_chemins(2, 4) == 4     # cas piege : grille non carree
```

```
def nb_chemins(L, C):
    N = [[1] * C for _ in range(L)]              # INITIALISER : 1re ligne et 1re colonne a 1
    for i in range(1, L):                        # PARCOURIR
        for j in range(1, C):                    # PARCOURIR
            N[i][j] = N[i - 1][j] + N[i][j - 1]  # METTRE A JOUR
    return N[L - 1][C - 1]                       # RENVOYER
```

`nb_chemins(3, 3) == 6` et `nb_chemins(2, 2) == 2`.

▷ **Erreur fréquente.** Mettre des 0 sur la première ligne et la première colonne : il y a exactement *un* chemin jusqu'à chacune de ces cases (tout droit), d'où des 1.

Autre méthode : la même relation en **mémoïsation**, avec un dictionnaire de clé `(i, j)` ; on appelle `nb_chemins_memo(L - 1, C - 1, {})`.

```
def nb_chemins_memo(i, j, memo):
    if i == 0 or j == 0:     # CAS DE BASE : 1re ligne ou 1re colonne
        return 1
    if (i, j) not in memo:   # VERIFIER : pas encore calcule ?
        memo[(i, j)] = nb_chemins_memo(i - 1, j, memo) + nb_chemins_memo(i, j - 1, memo)
    return memo[(i, j)]      # RENVOYER
```

Exercice 13 ↑énoncé p. 546

1. $T[i][c] = \max\left(T[i-1][c],\ \ v_i + T[i-1][c-w_i]\right)$: on *laisse* l'objet i (première branche) ou on le *prend* s'il rentre (seconde branche, avec $w_i \leq c$).

2. *Appels attendus, écrits avant de coder :*

```
assert sac_a_dos([1, 3, 4], [1, 4, 5], 5) == 6              # cas normal
assert sac_a_dos([2, 3], [5, 6], 1) == 0                    # cas limite : aucun objet ne
                                                            # rentre
assert sac_a_dos([1, 3, 4, 5], [1, 4, 5, 7], 7) == 9        # cas piege : le glouton par
                                                            # valeur donne 8
```

```
def sac_a_dos(poids, valeurs, capacite):
    n = len(poids)
    T = [[0] * (capacite + 1) for _ in range(n + 1)]   # INITIALISER
    for i in range(1, n + 1):                          # PARCOURIR : les objets
        for c in range(capacite + 1):                  # PARCOURIR : les capacites
            T[i][c] = T[i - 1][c]                      # METTRE A JOUR : on laisse l'objet
            if poids[i - 1] <= c:                      # TESTER : peut-on le prendre ?
                avec = valeurs[i - 1] + T[i - 1][c - poids[i - 1]]
                if avec > T[i][c]:                     # TESTER
                    T[i][c] = avec                     # METTRE A JOUR
    return T[n][capacite]                              # RENVOYER
```

`sac_a_dos([1,3,4,5], [1,4,5,7], 7) == 9` (on prend les objets de poids 3 et 4 : valeur $4+5=9$).

3. Le glouton par valeur prend d'abord l'objet de valeur 7 (poids 5) ; il reste 2 de capacité. Il essaie ensuite les objets de valeur 5 (poids 4) et 4 (poids 3), qui ne rentrent pas, puis prend l'objet

de poids 1 (valeur 1) : total **8**, **moins bon** que 9. Le glouton n'est pas optimal pour le sac à dos entier.

4.

$T[i][c]$	$c=0$	1	2	3	4	5
$i=0$ (aucun objet)	0	0	0	0	0	0
$i=1$ (poids 1, valeur 1)	0	1	1	1	1	1
$i=2$ (poids 3, valeur 4)	0	1	1	4	5	5
$i=3$ (poids 4, valeur 5)	0	1	1	4	5	**6**

Par exemple $T[3][5] = \max(T[2][5],\ 5 + T[2][1]) = \max(5,\ 6) = 6$. La meilleure valeur est **6** : on prend l'objet de poids 4, puis, dans la capacité restante 1, l'objet de poids 1 ($T[2][1] = 1$ vient de la ligne $i = 1$).

Exercice 14

↑énoncé p. 546

1. `"CHE"`, de longueur **3** (on retrouve C, H, E dans cet ordre dans `CHIEN` et dans `NICHE` ; attention, `"ICE"` ne convient pas : dans `CHIEN`, le I vient après le C).

2. *Appels attendus, écrits avant de coder :*

```
assert plsc("CHIEN", "NICHE") == 3      # cas normal
assert plsc("", "NSI") == 0             # cas limite : un mot vide
assert plsc("ABC", "CBA") == 1          # cas piege : memes lettres, ordre inverse
```

```
def plsc(a, b):
    n, m = len(a), len(b)
    L = [[0] * (m + 1) for _ in range(n + 1)]            # INITIALISER
    for i in range(1, n + 1):                            # PARCOURIR
        for j in range(1, m + 1):                        # PARCOURIR
            if a[i - 1] == b[j - 1]:                     # TESTER
                L[i][j] = L[i - 1][j - 1] + 1            # METTRE A JOUR
            else:
                L[i][j] = max(L[i - 1][j], L[i][j - 1])  # METTRE A JOUR
    return L[n][m]                                       # RENVOYER
```

`plsc("CHIEN","NICHE") == 3` et `plsc("ABCBDAB","BDCAB") == 4`.

Exercice 15

↑énoncé p. 547

1. `"NICHE"` → `"RICHE"` : distance 1 (substituer `N` par `R`). `"CHAT"` → `"CHATON"` : distance 2 (insérer `O` puis `N`).

2. Pour transformer les i premières lettres de `a` en le mot vide, il faut **supprimer** ces i lettres, donc $D[i][0] = i$. Symétriquement, obtenir les j premières lettres de `b` à partir du mot vide demande j **insertions**, donc $D[0][j] = j$.

3. Démarche : on remplit d'abord la première colonne et la première ligne (question 2), puis chaque autre case, ligne par ligne, avec la règle de l'énoncé. *Appels attendus, écrits avant de coder :*

```
assert distance("CHIEN", "CHATS") == 3     # cas normal
assert distance("", "NSI") == 3            # cas limite : que des insertions
assert distance("CHAT", "CHATON") == 2     # cas piege : longueurs differentes
```

```
def distance(a, b):
    n, m = len(a), len(b)
    D = [[0] * (m + 1) for _ in range(n + 1)]   # INITIALISER
    for i in range(n + 1):
        D[i][0] = i                             # INITIALISER
    for j in range(m + 1):
        D[0][j] = j                             # INITIALISER
    for i in range(1, n + 1):                   # PARCOURIR
        for j in range(1, m + 1):               # PARCOURIR
            if a[i - 1] == b[j - 1]:            # TESTER
                D[i][j] = D[i - 1][j - 1]       # METTRE A JOUR
            else:
                D[i][j] = 1 + min(min(D[i - 1][j], D[i][j - 1]), D[i - 1][j - 1])   # METTRE A
                                                                                    # JOUR
    return D[n][m]                              # RENVOYER
```

`distance("CHIEN", "CHATS") == 3` (trois substitutions) et `distance("CHATON", "MOUTON") == 3`.

Exercices type bac

Exercice 16

↑énoncé p. 547

Barème indicatif (sur 4) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 0,75 ; Q4 : 1 ; Q5 : 0,75 ; Q6 : 0,5

1. Sous-séquences strictement croissantes de longueur 2 de `[3,1,8,2,5]` : `[3,8]`, `[3,5]`, `[1,8]`, `[1,2]`, `[1,5]`, `[2,5]`.

2. La plus longue est `[1, 2, 5]`, de longueur **3**.

3. Démarche : on compare chaque élément au précédent ; dès qu'un élément n'est pas strictement plus grand, la réponse est `False`. *Appels attendus, écrits avant de coder :*

```
assert est_strict_croissante([1, 2, 5])            # cas normal
assert est_strict_croissante([7])                  # cas limite : un seul element
assert not est_strict_croissante([1, 3, 3, 8])     # cas piege : egalite, pas strictement
                                                   # croissante
```

```
def est_strict_croissante(seq):
    for k in range(1, len(seq)):    # PARCOURIR
        if seq[k] <= seq[k - 1]:    # TESTER
            return False            # RENVOYER : on sort des le premier defaut
    return True                     # RENVOYER
```

4. Lignes complétées :

```
def llsc_fin(tab, i):
    if i == 0:                                      # CAS DE BASE : ligne 2
        return 1                                    # ligne 3
    max_len = 1                                     # INITIALISER
    for j in range(i):                              # PARCOURIR
        if tab[j] < tab[i]:                         # TESTER : ligne 6
            max_len = max(max_len, llsc_fin(tab, j) + 1)   # APPEL RECURSIF, METTRE A JOUR
    return max_len                                  # RENVOYER
```

5. Lignes complétées :

```
def llsc_dyn(tab):
    n = len(tab)
    dyn = [1] * n                                   # INITIALISER
    for i in range(1, n):                           # PARCOURIR
        for j in range(i):                          # PARCOURIR
            if tab[j] < tab[i]:                     # TESTER
                dyn[i] = max(dyn[i], dyn[j] + 1)    # METTRE A JOUR : ligne 7
    meilleur = dyn[0]                               # INITIALISER : ligne 8, plus grande valeur
                                                    # de dyn
    for k in range(1, n):                           # PARCOURIR
        if dyn[k] > meilleur:                       # TESTER
            meilleur = dyn[k]                       # METTRE A JOUR
    return meilleur                                 # RENVOYER
```

La ligne 8 doit renvoyer la **plus grande** valeur de `dyn` (la plus longue sous-séquence ne se termine pas forcément à la dernière case) : on la cherche par un parcours.

6. La version récursive `llsc_fin` recalcule sans cesse les mêmes valeurs (coût exponentiel), tandis que `llsc_dyn` remplit chaque `dyn[i]` **une seule fois** : son coût est quadratique ($O(n^2)$), bien plus rapide.

Exercice 17 ↑énoncé p. 548

Barème indicatif (sur 6) : Q1 : 2 ; Q2 : 1 ; Q3 : 1,75 ; Q4 : 1,25

1. *Appels attendus, écrits avant de coder :*

```
assert cout(0, 2, liste_mots, 15) == 9            # cas normal
assert cout(1, 3, liste_mots, 14) == 0            # cas limite : la ligne tient tout juste
assert cout(0, 3, liste_mots, 15) == 1000000      # cas piege : 15 lettres, plus 2 espaces
```

```
def cout(i, j, liste_mots, justification):
    mots = liste_mots[i:j]
    nb_car = 0                                  # INITIALISER : nombre de lettres
    for m in mots:                              # PARCOURIR
        nb_car += len(m)                        # METTRE A JOUR
    nb_mots = len(mots)
    if nb_car + (nb_mots - 1) > justification:  # TESTER : la ligne ne tient pas
        return 1000000
    nb_espace_total = justification - nb_car
    supplementaires = nb_espace_total - (nb_mots - 1)
    return supplementaires ** 2                 # RENVOYER
```

On vérifie `cout(0,2,liste_mots,15) == 9`, `cout(0,4,liste_mots,15) == 1000000`, `cout(5,8,liste_mots,15) == 1`.

2. Après chacun des $n-1$ premiers mots on peut couper ou non : cela fait 2^{n-1} découpages possibles. Ce nombre **explose** (exponentiel) : impossible de tous les tester dès que n dépasse quelques dizaines.

3. La fonction remplit **`cout_mini` de la fin vers le début** : quand on calcule `cout_mini[i]`, toutes les cases `cout_mini[j]` avec $j > i$ sont **déjà connues**. La relation est

$$\texttt{cout_mini[i]} = \min_{i<j\leqslant n} \ (\ \texttt{cout(i, j)} + \texttt{cout_mini[j]}\ ),$$

c'est-à-dire : coût de la première ligne (mots i à $j-1$) plus le coût optimal du reste. Pour $j = n$, on adopte la convention `cout_mini[n]` $= 0$ (il ne reste plus aucun mot à justifier) : ce cas, où tous les mots restants tiennent sur une seule ligne, correspond à la valeur initiale `cout(i, n, ...)` du code, puisque la liste `cout_mini` n'a que n cases (indices 0 à $n-1$). Comme on résout les sous-problèmes du plus petit (la fin) au plus grand (le début) en rangeant les résultats dans un tableau, c'est bien de la **tabulation**.

4. Il suffit de renvoyer aussi `cout_mini[0]`, qui contient déjà le coût inesthétique total du meilleur découpage :

```
    return decoupage, cout_mini[0]
```

Exercice 18 ↑énoncé p. 549

Barème indicatif (sur 8) : Q1 : 0,75 ; Q2 : 0,25 ; Q3 : 0,5 ; Q4 : 0,5 ; Q5 : 0,75 ; Q6 : 0,25 ; Q7 : 1 ; Q8 : 0,5 ; Q9 : 1 ; Q10 : 1 ; Q11 : 0,5 ; Q12 : 1

1. Démarche : on vérifie d'abord le volume avec `assert`, puis on range les deux paramètres dans les attributs.

```
class Marchandise:
    def __init__(self, p: int, v: int) -> None:
        assert v > 0, "le volume doit etre strictement positif"
        self.prix = p
        self.volume = v
```

2. `m1 = Marchandise(20, 7)` (le prix d'abord, puis le volume).

3.

```
def ratio(self) -> float:
    return self.prix / self.volume
```

4. Démarche : un accumulateur initialisé à 0, auquel on ajoute l'attribut `prix` de chaque marchandise.

```
def prixListe(tab: list) -> int:
    total = 0                   # INITIALISER
    for m in tab:               # PARCOURIR
        total = total + m.prix  # METTRE A JOUR
    return total                # RENVOYER
```

5. Combinaisons de volume au plus 100 litres (aucune combinaison de trois marchandises ne tient : la plus légère, $m_1 + m_3 + m_4$, fait 110 litres) :

Combinaison	Volume	Prix	Combinaison	Volume	Prix
m_1	20	40	m_1, m_2	90	**250**
m_2	70	210	m_1, m_3	60	200
m_3	40	160	m_1, m_4	70	90
m_4	50	50	m_3, m_4	90	210

La combinaison qui maximise le prix est $\{m_1, m_2\}$, pour **250 €** ; le glouton, lui, n'obtient que 200 €.

6. Glouton (à chaque étape, on fait le choix qui paraît localement le meilleur, sans revenir en arrière).

7. Démarche : c'est le tri par insertion : on décale vers la droite les marchandises de ratio plus petit que celui de `marchandise`, puis on l'insère dans la case libérée.

```
def tri(tab: list) -> None:
    n = len(tab)
    for i in range(1, n):                                    # PARCOURIR
        marchandise = tab[i]
        j = i - 1
        while j >= 0 and marchandise.ratio() > tab[j].ratio():   # TESTER
            tab[j+1] = tab[j]                                # METTRE A JOUR : on decale
            j = j - 1
        tab[j+1] = marchandise                               # METTRE A JOUR : on insere
```

Avec les quatre marchandises de la partie B, on obtient bien l'ordre m_3, m_2, m_1, m_4 (ratios 4 ; 3 ; 2 ; 1).

8. Tri par **insertion**, de coût **quadratique** dans le pire des cas.

9. Démarche : on trie, puis on parcourt les marchandises dans l'ordre en chargeant chacune de celles qui tiennent encore dans le volume restant.

```
def charge(tab: list, volume: int) -> list:
    tri(tab)
    chargement = []                          # INITIALISER
    n = len(tab)
    for i in range(n):                       # PARCOURIR
        if tab[i].volume <= volume:          # TESTER
            chargement.append(tab[i])        # METTRE A JOUR
            volume = volume - tab[i].volume  # METTRE A JOUR
    return chargement                        # RENVOYER
```

Pour 100 litres, `charge` renvoie $[m_3, m_1]$, de prix 200.

10. `option1` correspond à « on n'utilise pas la marchandise `i` » et `option2` à « on l'utilise » :

```
def chargeOptimale(tab: list, v_restant: int, i: int) -> list:
    if i >= len(tab):                                # CAS DE BASE
        return []
    else:
        if tab[i].volume > v_restant:                # TESTER
            return chargeOptimale(tab, v_restant, i+1)   # APPEL RECURSIF
        else:
            option1 = chargeOptimale(tab, v_restant, i+1)   # APPEL RECURSIF
```

```
            option2 = [tab[i]] + chargeOptimale(tab, v_restant - tab[i].volume, i+1)   # APPEL
                                                                                       # RECURSIF
            if prixListe(option1) > prixListe(option2):   # TESTER
                return option1
            else:
                return option2
```

Pour les marchandises de la partie B et 100 litres, on obtient $[m_1, m_2]$, de prix 250 : c'est l'optimum trouvé à la question 5.

11. Depuis `chargeOptimale(tab, 25, 0)`, on atteint l'indice 2 avec 15 litres restants par deux chemins : prendre la marchandise 0 puis ne pas prendre la 1 (`(25, 0)` → `(15, 1)` → `(15, 2)`), ou ne pas prendre la 0 puis prendre la 1 (`(25, 0)` → `(25, 1)` → `(15, 2)`). Le même sous-problème est donc résolu deux fois (de même que `(15, 3)` et `(5, 3)`). Comme chaque appel peut en engendrer deux, le nombre d'appels peut atteindre de l'ordre de 2^n pour n marchandises : les sous-problèmes identiques se multiplient. C'est exactement la situation où la programmation dynamique s'impose.

12. Le résultat ne dépend que du couple `(i, v_restant)` : c'est la clé du dictionnaire. *Appels attendus, écrits avant de coder :*

```
assert prix_max(tab, 100, 0, {}) == 250     # cas normal : les quatre marchandises de la
                                            # partie B
assert prix_max(tab, 10, 0, {}) == 0        # cas limite : rien ne rentre
petit = [Marchandise(100, 10), Marchandise(100, 10), Marchandise(250, 20)]
assert prix_max(petit, 25, 0, {}) == 250    # cas piege : m3 seule bat m1 et m2
```

```
def prix_max(tab, v_restant, i, memo):
    if (i, v_restant) in memo:                          # VERIFIER : deja calcule ?
        return memo[(i, v_restant)]
    if i >= len(tab):                                   # CAS DE BASE
        res = 0
    elif tab[i].volume > v_restant:                     # TESTER
        res = prix_max(tab, v_restant, i + 1, memo)     # APPEL RECURSIF
    else:
        res = max(prix_max(tab, v_restant, i + 1, memo),   # APPEL RECURSIF
                  tab[i].prix + prix_max(tab, v_restant - tab[i].volume, i + 1, memo))
    memo[(i, v_restant)] = res                          # METTRE A JOUR
    return res                                          # RENVOYER
```

`prix_max(tab, 100, 0, {})` renvoie bien 250. Il y a au plus $(n+1) \times (V+1)$ clés différentes (pour un volume initial V), chacune calculée une seule fois : c'est le **sac à dos** du cours, en $O(n \times V)$ (coût **pseudo-polynomial** : voir la remarque du cours).

Exercice 19

↑énoncé p. 551

Barème indicatif (sur 8) : Q1 : 0,75 ; Q2 : 0,5 ; Q3 : 0,75 ; Q4 : 1 ; Q5 : 0,75 ; Q6 : 0,75 ; Q7 : 1 ; Q8 : 1,25 ; Q9 : 0,75 ; Q10 : 0,5

1. Découpes d'une planche de 4 m :

Morceaux	4	$3+1$	$2+2$	$2+1+1$	$1+1+1+1$
Recette (€)	**13**	$10+2=12$	$5+5=10$	$5+2+2=9$	$4\times 2=8$

Donc $V(4) = 13$: on vend la planche de 4 m entière.

2. Non : entière, elle rapporte 14 €, alors que la découpe $3+2$ rapporte $10+5=15$ € (de même $4+1$: $13+2=15$ €).

3. Il y a $n-1$ marques intérieures et, à chacune, deux choix indépendants (scier ou non) : $2\times 2\times\cdots\times 2 = 2^{n-1}$ découpes. Pour 60 m, cela fait $2^{59} \approx 5{,}8\times 10^{17}$ découpes : même à un milliard de découpes par seconde, il faudrait plus de 18 ans. Le nombre de cas croît **exponentiellement**.

4. Une planche de 0 m ne rapporte rien : $V(0) = 0$. Pour $n \geq 1$, une découpe commence par un premier morceau de longueur k, avec $1 \leq k \leq \min(n, 5)$; il reste alors une planche de $n-k$ mètres, qu'il faut découper au mieux, ce qui rapporte au plus $V(n-k)$. On essaie toutes les longueurs k possibles et on garde la meilleure, d'où la relation.

5. Démarche : on traduit la relation de la question 4 : condition d'arrêt $V(0) = 0$, puis on essaie chaque longueur `k` possible du premier morceau et l'on garde la meilleure recette.

```
def valeur_rec(n, prix):
    if n == 0:                                  # CAS DE BASE
        return 0
    meilleur = 0                                # INITIALISER
    for k in range(1, len(prix)):               # PARCOURIR : longueur du premier morceau
        if k <= n:                              # TESTER
            v = prix[k] + valeur_rec(n - k, prix)   # APPEL RECURSIF
            if v > meilleur:
                meilleur = v                    # METTRE A JOUR
    return meilleur                             # RENVOYER
```

6. Chaque appel relance jusqu'à cinq appels récursifs, et les mêmes sous-problèmes sont rencontrés par de nombreux chemins : $V(1)$ est utile pour $V(2)$, $V(3)$, $V(4)$, $V(5)$, $V(6)\ldots$, et il est **recalculé** à chaque fois. Les sous-problèmes **se recouvrent** : le nombre d'appels croît exponentiellement avec n, alors qu'il n'y a que $n+1$ valeurs différentes $V(0), \ldots, V(n)$ à calculer. C'est le signal de la programmation dynamique.

7. Démarche : la fonction de la question 5, avec un dictionnaire : on regarde d'abord si `V(n)` est déjà connu, et l'on y range chaque résultat calculé. *Appels attendus, écrits avant de coder :*

```
assert valeur_memo(8, prix, {}) == 26     # cas normal
assert valeur_memo(0, prix, {}) == 0      # cas limite : planche vide
assert valeur_memo(5, prix, {}) == 15     # cas piege : entiere, elle ne rapporte que 14
```

```
def valeur_memo(n, prix, memo):
    if n == 0:                                      # CAS DE BASE
        return 0
    if n in memo:                                   # VERIFIER : deja calcule
        return memo[n]
    meilleur = 0                                    # INITIALISER
    for k in range(1, len(prix)):                   # PARCOURIR
        if k <= n:                                  # TESTER
            v = prix[k] + valeur_memo(n - k, prix, memo)   # APPEL RECURSIF
            if v > meilleur:
                meilleur = v                        # METTRE A JOUR
    memo[n] = meilleur                              # METTRE A JOUR : on note le
                                                # resultat
    return meilleur                                 # RENVOYER
```

On l'appelle avec un dictionnaire vide : `valeur_memo(8, prix, {})` renvoie 26.

8. Démarche : un tableau `V` rempli des petites longueurs vers les grandes : chaque case utilise la relation de la question 4 et des cases déjà remplies.

```
def valeur_tab(n, prix):
    V = [0] * (n + 1)                           # INITIALISER : V[0] = 0
    for L in range(1, n + 1):                   # PARCOURIR : du plus petit au plus
                                            # grand
        for k in range(1, len(prix)):           # PARCOURIR
            if k <= L and prix[k] + V[L - k] > V[L]:   # TESTER
                V[L] = prix[k] + V[L - k]       # METTRE A JOUR
    return V[n]                                 # RENVOYER
```

n	0	1	2	3	4	5	6	7	8
$V(n)$	0	2	5	10	13	15	20	23	**26**

Par exemple $V(7) = \max(2+20,\ 5+15,\ 10+13,\ 13+10,\ 14+5) = 23$.

9. Ligne 8 : `choix[L] = k` ; ligne 12 : `morceaux.append(choix[L])` ; ligne 13 : `L = L - choix[L]`.

`decoupe_optimale(8, prix)` renvoie `[4, 4]` ($13 + 13 = 26$ €) et `decoupe_optimale(7, prix)` renvoie `[3, 4]` ($10 + 13 = 23$ €).

10. Prix au mètre : 2 ; 2,5 ; 3,33 ; 3,25 ; 2,8. Le glouton coupe d'abord deux morceaux de 3 m (6 m, 20 €) ; il reste 2 m, et parmi les longueurs qui rentrent (1 ou 2 m), la meilleure au mètre est 2 m (5 €). Total : $3 + 3 + 2$, soit **25** €, moins que les 26 € de la découpe $4 + 4$. Le glouton **n'est pas optimal** : il faut la programmation dynamique.

Exercice 20  ↑énoncé p. 553

Barème indicatif (sur 8) : Q1 : 0,75 ; Q2 : 0,5 ; Q3 : 0,75 ; Q4 : 1 ; Q5 : 0,75 ; Q6 : 0,75 ; Q7 : 1 ; Q8 : 0,75 ; Q9 : 1 ; Q10 : 0,75

1. `BAC` est codé par `'213'`. On peut lire `2|1|3` (`BAC`), `21|3` (`UC`) ou `2|13` (`BM`) ; `213` n'est pas une lettre. Il y a donc exactement **trois** décodages.

2. `1|2|2|6` (`ABBF`), `1|2|26` (`ABZ`), `1|22|6` (`AVF`), `12|2|6` (`LBF`), `12|26` (`LZ`).

3. Aucune lettre n'est codée par `0` ni par un nombre commençant par `0`. Dans `'106'`, le `0` doit donc être lu avec le `1` qui le précède : `10|6` (`JF`) est le seul décodage. Dans `'100'`, le premier `0` doit être lu dans `10`, et le second `0` ne peut plus être rattaché à rien (`00` et `0` ne sont pas des lettres) : **aucun** décodage.

4. Les décodages des i premiers chiffres se répartissent en deux familles disjointes selon la dernière lettre :

- elle est codée par le seul chiffre `message[i-1]` : c'est possible si ce chiffre n'est pas `0`, et ce qui précède est alors un décodage quelconque des $i - 1$ premiers chiffres, d'où $D(i-1)$ décodages ;
- elle est codée par les deux chiffres `message[i-2]` et `message[i-1]` : c'est possible s'ils forment un nombre entre 10 et 26, et ce qui précède est un décodage des $i - 2$ premiers chiffres, d'où $D(i-2)$ décodages.

On additionne les deux familles. *(La convention $D(0) = 1$ compte le décodage « vide » : par exemple $D(2) = 2$ pour `'12'`.)*

5. Démarche : on écarte d'abord le cas $i < 2$, puis on forme le nombre des deux chiffres et l'on teste s'il est compris entre 10 et 26.

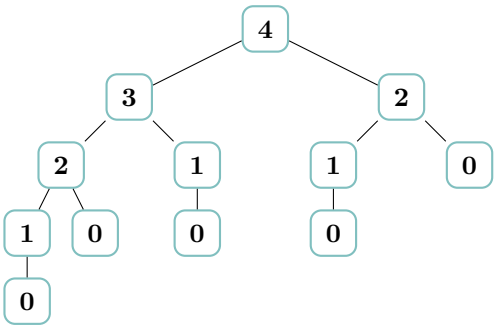
```
def deux_chiffres(message, i):
    if i < 2:
        return False
    nombre = int(message[i - 2]) * 10 + int(message[i - 1])
    return 10 <= nombre and nombre <= 26
```

Le test `10 <= nombre` écarte les nombres commençant par `0` (comme `'06'`).

6. Démarche : on traduit la relation de la question 4 : condition d'arrêt $D(0) = 1$, puis deux termes ajoutés sous condition.

```
def nb_rec(message, i):
    if i == 0:                                  # CAS DE BASE
        return 1
    total = 0                                   # INITIALISER
    if lettre_seule(message, i):                # TESTER
        total = total + nb_rec(message, i - 1)  # APPEL RECURSIF
    if deux_chiffres(message, i):               # TESTER
        total = total + nb_rec(message, i - 2)  # APPEL RECURSIF
    return total                                # RENVOYER
```

7. Arbre des appels de `nb_rec('1111', 4)` (on note seulement la valeur de `i`) :



La fonction renvoie 5 (le nombre de feuilles `0`). L'appel avec `i = 2` est effectué 2 fois, avec `i = 1` 3 fois et avec `i = 0` 5 fois (12 appels en tout). Comme pour Fibonacci, le nombre d'appels croît exponentiellement : plus de 3,5 millions d'appels (exactement 3 524 577) pour 30 chiffres, alors qu'il n'y a que 31 sous-problèmes différents, $D(0)$ à $D(30)$.

8. Démarche : la fonction `nb_rec`, avec un dictionnaire de clé `i` consulté avant tout calcul et rempli avant de renvoyer.

```
def nb_memo(message, i, memo):
    if i == 0:                                          # CAS DE BASE
        return 1
    if i in memo:                                       # VERIFIER
        return memo[i]
    total = 0                                           # INITIALISER
    if lettre_seule(message, i):                        # TESTER
        total = total + nb_memo(message, i - 1, memo)   # APPEL RECURSIF
    if deux_chiffres(message, i):                       # TESTER
        total = total + nb_memo(message, i - 2, memo)   # APPEL RECURSIF
    memo[i] = total                                     # METTRE A JOUR
    return total                                        # RENVOYER
```

Appel : `nb_memo(message, len(message), {})`.

9. Démarche : un tableau `D` rempli de $D(0)$ à $D(n)$; les appels récursifs de `nb_rec` sont remplacés par les cases `D[i - 1]` et `D[i - 2]`, déjà remplies. *Appels attendus, écrits avant de coder :*

```
assert nb_decodages('1226') == 5     # cas normal
assert nb_decodages('7') == 1        # cas limite : un seul chiffre
assert nb_decodages('100') == 0      # cas piege : un 0 qui ne peut pas etre rattache
```

```
def nb_decodages(message):
    n = len(message)
    D = [0] * (n + 1)                   # INITIALISER
    D[0] = 1                            # INITIALISER
    for i in range(1, n + 1):           # PARCOURIR
        if lettre_seule(message, i):    # TESTER
            D[i] = D[i] + D[i - 1]      # METTRE A JOUR
        if deux_chiffres(message, i):   # TESTER
            D[i] = D[i] + D[i - 2]      # METTRE A JOUR
    return D[n]                         # RENVOYER
```

i	0	1	2	3	4	5	6
chiffre `message[i-1]`	—	1	2	3	1	2	3
$D(i)$	1	1	2	3	3	6	**9**

Par exemple $D(4) = D(3) = 3$, car `31` > 26 ; puis $D(5) = D(4) + D(3) = 6$ (`12`) et $D(6) = D(5) + D(4) = 9$ (`23`). Le message `'123123'` admet 9 décodages.

10. Chaque chiffre vaut `1`, qui n'est pas `0`, et `11` est compris entre 10 et 26 : les deux termes de la question 4 sont toujours présents, donc $D(i) = D(i-1) + D(i-2)$ avec $D(0) = D(1) = 1$. On reconnaît la suite de **Fibonacci** (décalée d'un rang). Pour `'111111'` : $1, 1, 2, 3, 5, 8, \mathbf{13}$ décodages.

Critiquer une réponse d'IA

Exercice 21 *Un rendu de monnaie tabulé par un assistant d'IA* ↑énoncé p. 554

1. Non. Toutes les cases valent 0 au départ ; pour `s = 1`, on teste `table[0] + 1 < table[1]`, soit `1 < 0` : faux, la case reste à 0. Il en va de même pour toutes les suivantes : la condition « strictement plus petit que 0 » n'est jamais vraie. Ainsi, `rendu(6, [1, 3, 4])` renvoie `0` (et `rendu(7, [1, 3, 4])` aussi), au lieu de 2.

2. L'erreur : la table est **initialisée à** 0, valeur qui joue le rôle de « meilleur résultat connu » et bat donc toute proposition. Un montant non encore traité doit valoir $+\infty$ (« aucune solution connue »), seule la case `table[0]` valant 0 :

```
    table = [0] + [float("inf")] * somme   # table[0] = 0, le reste : +infini
```

Avec cette ligne, la table pour $\{1, 3, 4\}$ devient `[0, 1, 2, 1, 1, 2, 2]` et `rendu(6, [1, 3, 4])` renvoie 2.

3. Tester sur un **cas trivial** dont on connaît la réponse : `rendu(1, [1])` doit renvoyer `1`, pas `0`. Un résultat nul pour une somme non nulle est impossible et se repère en une ligne.

S'entraîner à l'oral

Exercice 22 *Expliquer en deux minutes* ↑énoncé p. 555

▶ **Solution.** Pas de texte à apprendre par cœur : voici les **éléments attendus** pour chaque sujet. L'ordre, les mots et l'exemple peuvent être différents ; l'explication est réussie si ces idées y sont, justes et reliées entre elles.

Sujet 1.

- `fibo(n) = fibo(n - 1) + fibo(n - 2)` : les mêmes sous-problèmes sont recalculés un très grand nombre de fois (`fibo(2)` est calculé trois fois dès `fibo(5)`).
- Le nombre d'appels croît de façon **exponentielle** : plus de deux millions et demi d'appels pour `fibo(30)`.
- **Mémoïsation** : un dictionnaire garde chaque résultat déjà calculé ; avant de calculer, on regarde s'il y est. Chaque valeur n'est calculée qu'une fois : environ n calculs.
- Piège : le dictionnaire doit être partagé par tous les appels (et non recréé à chaque appel) ; on échange de la **mémoire** contre du **temps**.

Sujet 2.

- **Mémoïsation** (*top-down*) : on garde la fonction récursive et on mémorise ses résultats ; seuls les sous-problèmes nécessaires sont calculés.
- **Tabulation** (*bottom-up*) : une boucle remplit un tableau, des plus petits sous-problèmes jusqu'au problème demandé, sans récursion.
- Exemple : `t[0] = 0`, `t[1] = 1`, puis `t[i] = t[i - 1] + t[i - 2]` pour `i` de 2 à n.
- Les deux reposent sur la **même** relation de récurrence ; la tabulation évite la limite de profondeur de récursion mais oblige à choisir l'ordre de remplissage.

Sujet 3.

- **Glouton** : à chaque étape, on prend la plus grande pièce possible, sans jamais revenir sur ce choix.
- Contre-exemple : pièces $\{1, 6, 10\}$, rendre 12 ; glouton : $10 + 1 + 1$ (3 pièces) ; optimum : $6 + 6$ (2 pièces).
- Programmation dynamique : le nombre minimal de pièces pour s vaut $1+$ le minimum, sur chaque pièce $p \leqslant s$, du nombre minimal pour $s - p$; on examine **toutes** les possibilités en réutilisant les résultats déjà calculés.
- Nuance : pour les pièces en euros, le glouton donne bien l'optimum ; c'est le système de pièces qui décide.

CORRIGÉS DES TP

TP Le plus grand carré blanc

↑ énoncé p. 558

2 Découverte à la main

Exercice 1 *Le coin inférieur droit*

Un carré blanc appuyé sur ce coin ne peut pas être plus grand que le plus **petit** des trois carrés voisins : la valeur est $1 + \min(\underbrace{3}_{\text{haut}}, \underbrace{2}_{\text{gauche}}, \underbrace{4}_{\text{diag.}}) = 1 + 2 = \mathbf{3}$.

Exercice 2 *Remplir toute la grille*

Grille de départ (1 blanc, 0 noir) et table `pgcb` obtenue :

	0	1	2	3	4	5
0	1	1	1	1	1	0
1	1	2	2	2	2	0
2	0	1	2	3	3	1
3	1	1	2	3	4	2
4	1	2	2	3	4	3
5	1	0	1	2	3	4

Le **maximum** de la table vaut **4** (cases surlignées, aux coins `(ligne 3, col 4)`, `(ligne 4, col 4)` et `(ligne 5, col 5)`) : le plus grand carré blanc de cette grille est de **côté 4**.

4 Version récursive

Exercice 3 *Écrire* `pgcb` *en récursif*

```
def pgcb(x, y):
    if est_noir(x, y):
        return 0                        # (a)
    elif x == 0 or y == 0:
        return 1                        # (b)
    else:
        g = pgcb(x - 1, y)                  # (c) a gauche
        h = pgcb(x, y - 1)                  # (d) au-dessus
        d = pgcb(x - 1, y - 1)              # (e) en diagonale
        return 1 + min(min(g, h), d)
```

Exercice 4 *Le problème de la lenteur*

1. Chaque appel `pgcb(x, y)` déclenche **trois** appels sur des pixels voisins, qui se **chevauchent** d'un pixel à l'autre : par exemple `pgcb(x-1, y-1)` est réclamé à la fois par `pgcb(x, y)`, par `pgcb(x-1, y)` et par `pgcb(x, y-1)`. Le **même** sous-problème est donc recalculé un nombre **exponentiel** de fois — exactement comme l'arbre d'appels de `fibo`.

2. On applique la **mémoïsation** : on mémorise chaque résultat déjà calculé pour ne jamais le refaire.

5 Mémoïsation (top-down)

Exercice 5 *Écrire* `pgcb_memo`

```python
def pgcb_memo(x, y, memo=None):
    if memo is None:
        memo = {}                       # carnet neuf a chaque appel principal
    if (x, y) in memo:
        return memo[(x, y)]
    if est_noir(x, y):
        r = 0
    elif x == 0 or y == 0:
        r = 1
    else:
        g = pgcb_memo(x - 1, y, memo)
        h = pgcb_memo(x, y - 1, memo)
        d = pgcb_memo(x - 1, y - 1, memo)
        r = 1 + min(min(g, h), d)
    memo[(x, y)] = r
    return r
```

C'est la construction de `fibo` dans le cours, dans sa version sûre (`memo=None` : le dictionnaire par défaut n'est pas partagé d'un appel à l'autre). L'appel `pgcb_memo(250, 250)` devient instantané et renvoie `21` (sans mémoïsation, `pgcb(250, 250)` dépasse 30 millions d'appels en 10 secondes sans finir). À la question suivante, on passe explicitement un même dictionnaire pour tous les pixels.

6 Sur une vraie image

Exercice 6 *La fonction* `est_noir`

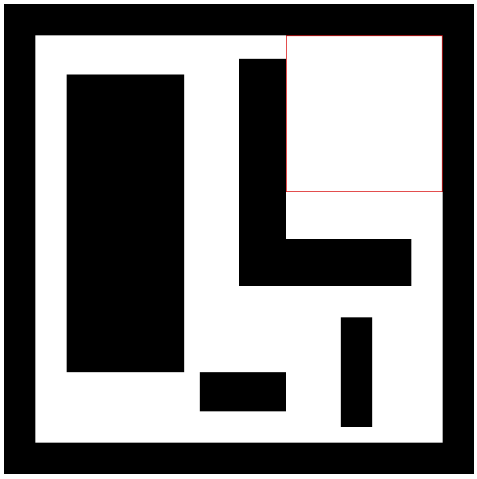
```python
def est_noir(x, y):
    return pixels[x, y] == (0, 0, 0)
```

Exercice 7 *Chercher le plus grand carré de l'image*

On partage **un seul** dictionnaire de mémoïsation pour tous les pixels, créé au début de la fonction et passé à `pgcb_memo`, sinon le calcul reste interminable :

```python
def total_pgcb():
    memo = {}                           # un seul carnet pour tous les pixels
    meilleur = 0
    coin = (0, 0)
    for y in range(hauteur):
        for x in range(largeur):
            taille = pgcb_memo(x, y, memo)
            if taille > meilleur:       # > strict : on garde le premier coin
                meilleur = taille
                coin = (x, y)
    return meilleur, coin
```

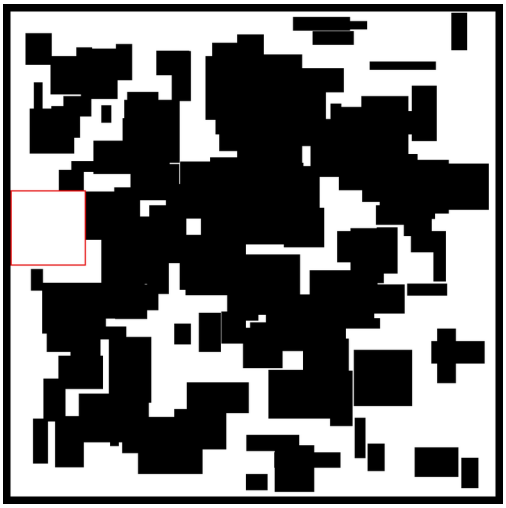
Sur l'image fournie : `total_pgcb()` renvoie **(200, (559, 239))** — un carré blanc de **côté 200**, dont le coin inférieur droit est le pixel `(559, 239)` (le grand bloc blanc en haut à droite).



Le plus grand carré blanc trouvé (contour rouge), côté 200.

Exercice 8 *Défi — la grande image*

Sur `carre_blanc_grand.png` (1200 × 1200), après avoir remplacé `NOM_IMAGE` par `"carre_blanc_grand.png"`, le programme renvoie un carré blanc de **côté 181**, de coin inférieur droit `(198, 627)` — une zone que l'œil n'aurait pas su désigner parmi toutes les taches blanches voisines. C'est précisément ce qu'apporte la programmation dynamique : une réponse **exacte** et **rapide** là où l'intuition est impuissante.



Le plus grand carré blanc de la grande image (contour rouge), côté 181 — indevinable à l'œil.

7 Bonus : tabulation (bottom-up)

Exercice 9 *Écrire* `pgcb_bottom_up`

La table `S` (une ligne de `largeur` zéros pour chaque `y`) se construit par compréhension :

```
def pgcb_bottom_up():
    S = [[0] * largeur for y in range(hauteur)]      # S[y][x]
    meilleur = 0
    coin = (0, 0)
    for y in range(hauteur):
        for x in range(largeur):
            if est_noir(x, y):
                S[y][x] = 0
            elif x == 0 or y == 0:
                S[y][x] = 1
            else:
                S[y][x] = 1 + min(min(S[y][x - 1], S[y - 1][x]), S[y - 1][x - 1])
            if S[y][x] > meilleur:
                meilleur = S[y][x]
                coin = (x, y)
    return meilleur, coin
```

On remplit la table **ligne par ligne**, exactement comme à la main : aucune récursion, aucun dictionnaire. Le résultat est identique : **(200, (559, 239))**.

Mémoïsation et tabulation donnent le même résultat : ce sont les deux faces de la programmation dynamique.

Réponses des questions de la fiche de révision

↑ questions p. 571

Se tester

1. des sous-problèmes qui se recouvrent et une sous-structure optimale (§ IX).
2. 8 (la table vaut `[0, 1, 1, 2, 3, 5, 8]`) (§ III).
3. exponentiel (environ $1{,}6^n$, majoré par 2^n) ; linéaire (n) (§ I et III).
4. elle recalcule tout, comme la version naïve : coût exponentiel (§ II).
5. glouton $4 + 1 + 1$: 3 pièces ; dynamique $3 + 3$: 2 pièces (§ V).

Et avant ?

1. avec la clé **publique d'Alice** ; seule sa clé privée permet de déchiffrer (ch. 10, § III).
2. il l'est si un parcours lancé depuis un sommet quelconque visite tous les sommets (ch. 8, § V.D).
3. 4 : chaque valeur part à droite de la précédente, l'arbre est « en peigne » (ch. 5, § VII.B).

CORRIGÉS DU CHAPITRE 12

Recherche textuelle

12

CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Chercher un mot, à la main

↑énoncé p. 583

1. On essaie les positions 0 à 37, soit **38** positions, pour un total de **45** comparaisons.
2. `SOLEIL` est à la position **37**. Presque toutes les positions coûtent **une seule** comparaison (la première lettre diffère déjà) ; seules les positions 18 et 32 (un `S` suivi d'un espace) en coûtent 2, et la position 37 en coûte 6.
3. Environ **un million** de positions $(n - m + 1)$: on essaie toutes les positions, une par une. Si de plus chaque position coûtait plusieurs comparaisons, on atteindrait jusqu'à $n \times m$ comparaisons.
4. Sous la dernière case (position 5 du texte) se trouve un `T`, qui **n'apparaît pas** dans `SOLEIL`.
5. Non : tant que la bande recouvre ce `T`, une de ses cases devrait contenir `T`, ce qui est impossible. On peut donc faire sauter la bande de **toute sa longueur**, **6** cases, juste après le `T`.
6. À la position 12, la dernière case est sur le `I` (position 17 du texte). Dans la bande, `I` est l'avant-dernière case : on avance de **1** case pour les aligner.
7. Déroulé complet :

Position	Lettre en face de la dernière case	Comparaisons	Saut
0	`T`	1	6
6	`D`	1	6
12	`I`	1	1
13	`S`	1	5
18	`D`	1	6
24	`A`	1	6
30	`U`	1	6
36	`I`	1	1
37	`L`	6	**trouvé**

9 positions essayées et **14** comparaisons. À la position 13, le `S` est dans la bande (première case) : on avance de 5 pour l'aligner.

8. 14 comparaisons au lieu de 45, plus de trois fois moins. Quand la lettre en face de la dernière case n'est pas dans le motif, aucune position recouvrant cette lettre ne peut convenir : les lettres sautées **n'ont jamais besoin d'être lues**. Plus le motif est long, plus les sauts sont grands.

9. Pense-bête : `S`→ 5, `O`→ 4, `L`→ 3, `E`→ 2, `I`→ 1, toute autre lettre → 6. Pour `L`, on ne compte pas la dernière case : c'est le `L` en troisième position qui compte (saut 3). C'est exactement la **table de décalage** de Horspool, calculée une fois pour toutes avant la recherche (le **prétraitement** du motif).

CORRIGÉS DES EXERCICES

La méthode naïve

★ **Exercice 1** *Naïf, à la main* ↑énoncé p. 594

1. `CAAAB` et `CAAAB` : le motif `AA` est trouvé aux positions **1** et **2**.

   ▷ **Erreur fréquente.** S'arrêter à la première occurrence, ou sauter après elle : la méthode naïve teste *toutes* les positions, et deux occurrences peuvent se chevaucher (`AA` en 1 et en 2).

2. Au pire, on compare presque tout le motif (m comparaisons) à chacune des $\approx n$ positions : coût $O(n \times m)$ (quadratique).

★ **Exercice 2** *Naïf, en Python* ↑énoncé p. 594

Démarche : on teste chaque position de départ, de 0 à $n - m$; la boucle `while` avance tant que les lettres coïncident, et le motif est trouvé si elle est allée jusqu'au bout.

`(a) range(n - m + 1)` ; `(b) if j == m:`.

▷ **Erreur fréquente.** Écrire `range(n - m)` : la dernière position de départ, $n - m$, ne serait pas testée, et un motif placé tout à la fin du texte serait manqué.

Boyer-Moore

★★ **Exercice 3** *Les deux idées, et un grand saut* ↑énoncé p. 594

1. **(1)** comparer le motif **de droite à gauche** ; **(2) prétraiter** le motif pour **sauter de plusieurs crans** en cas d'échec (règle du mauvais caractère).

2. `CGGCAG` : `C`→ 3, `G`→ 5, `A`→ 4 (dernières positions ; `C0 G1 G2 C3 A4 G5`).

   ▷ **Erreur fréquente.** Retenir la *première* position d'une lettre répétée (`G`→ 1) : la table de Boyer-Moore garde la **dernière** position, ici `G`→ 5.

3. `T` n'apparaît pas dans le motif : aucun alignement où `T` tomberait *dans* le motif ne peut réussir. Comme ce `T` est aligné avec la dernière lettre du motif, on place le motif juste après lui : on saute de **6 crans** (toute la longueur du motif) d'un seul coup.

   ▷ **Erreur fréquente.** Croire que le saut vaut toujours m quand la lettre est absente : il vaut $j+1$ si l'échec a lieu en position j du motif (exercice 4 : `T` absent, échec en $j=2$, décalage 3). Il ne vaut m que si l'échec a lieu dès la dernière lettre.

★★ **Exercice 4** *Dérouler Boyer-Moore* ↑énoncé p. 595

Table (dernières positions) de `GCAGC` : `G`→ 3, `C`→ 4, `A`→ 2.

position i	fenêtre	échec / trouvé	décalage
$i=0$	`ACGTG`	échec en $j=4$ (`G` du texte)	1
$i=1$	`CGTGC`	échec en $j=2$ (`T`)	3
$i=4$	`GCAGC`	**trouvé**	1
$i=5$	`CAGCT`	échec en $j=4$ (`T`)	5
$i=10$	`GCAGC`	**trouvé**	—

Occurrences aux positions **4** et **10**.

Boyer-Moore-Horspool

★★ **Exercice 5** *Construire la table de décalage* ↑énoncé p. 595

1. `ANANAS` ($m=6$) : `A`→ 1, `N`→ 2, défaut **6**. *(La dernière occurrence de `A` avant la fin est en position 4 → 1 ; de `N` en position 3 → 2.)*
2. `GATTACA` ($m=7$) : `G`→ 6, `A`→ 2, `T`→ 3, `C`→ 1, défaut **7**.
3. `ABRACADABRA` ($m=11$) : `A`→ 3, `B`→ 2, `R`→ 1, `C`→ 6, `D`→ 4, défaut **11**.

▷ **Erreur fréquente.** Donner à la dernière lettre du motif le décalage 0 : on parcourt les lettres **sauf la dernière**. Dans `ANANAS`, `S` n'apparaît qu'en dernière position : il prend le décalage par défaut, 6.

★★ **Exercice 6** *Dérouler Horspool* ↑énoncé p. 595

position i	fenêtre	lettre de fin	décalage
$i=0$	`GTAC`	`C`	2
$i=2$	`ACAC`	`C`	2
$i=4$	`ACGT`	`T` (trouvé)	4 (dépasse la fin)

Le motif `ACGT` est trouvé à la position **4**.

▷ **Erreur fréquente.** Chercher la lettre de fin dans le motif au lieu du texte : c'est la lettre du **texte** alignée avec la dernière case de la fenêtre qui commande le saut.

★★ **Exercice 7** *Horspool en Python* ↑énoncé p. 595

Démarche : la table parcourt les lettres du motif sauf la dernière, la lettre en position k recevant $m-1-k$; une lettre absente de la table autorise le saut maximal.

`(a) range(m - 1)` ; `(b) dec[motif[k]] = m - 1 - k` ; `(c) i += m` (lettre absente de la table : saut maximal).

Coût

★★ **Exercice 8** *Le miracle du sous-linéaire* ↑énoncé p. 596

1. Une seule passe sur le motif : $O(m)$ (les lettres absentes du motif ne sont pas stockées dans la table).
2. À chaque échec, Horspool **saute** de plusieurs cases : les lettres survolées ne sont *jamais lues*. On peut donc conclure en bien moins de n comparaisons.
3. **Faux** : c'est le contraire ! Plus le motif est long, plus les sauts possibles sont grands, donc plus la recherche est **rapide** (jusqu'à $\approx n/m$ comparaisons).
4. **Non** : au **pire cas**, Boyer-Moore reste en $O(n \times m)$, comme le naïf. C'est *en pratique*, sur des textes ordinaires, qu'il est spectaculairement plus rapide.

★★★ **Exercice 9** *Exercice d'entraînement — Chercher un gène dans l'ADN* ↑énoncé p. 596

1. `GATTACA` : `G`→ 6, `A`→ 2, `T`→ 3, `C`→ 1, défaut **7**.
2. Sur `GTAGATTACAGT` :

i	fenêtre	lettre de fin	décalage
0	`GTAGATT`	`T`	3
3	`GATTACA`	`A` (**trouvé**)	2
5	`TTACAGT`	`T`	3 (dépasse la fin)

   Le motif est trouvé à la position **3**.
3. `(a) if j < 0:` ; `(b) m` (le décalage par défaut).
4. Sur 3 milliards de bases, un coût quadratique $O(n \times m)$ serait **prohibitif**. Horspool, souvent **sous-linéaire** (il saute une grande partie du brin sans la lire), rend la recherche réalisable en un temps raisonnable.

Vers le bac

★ **Exercice 10** *Type bac — Parcourir une chaîne de caractères* (d'après Centres étrangers 2023, jour 2) ↑énoncé p. 597

Barème indicatif (sur 2) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 1

1. `bonjour('Alan')` renvoie la chaîne `'Bonjour Alan !'` (concaténation de trois chaînes).
2. `une_chaine[2]` vaut `'n'` et `une_chaine[3]` vaut `'j'` : `x` vaut `False`. `une_chaine[4]` et `une_chaine[1]` valent tous deux `'o'` : `y` vaut `True`. Les deux variables sont de type **booléen** (`bool`).
3. **Démarche :** on parcourt la chaîne caractère par caractère avec un compteur (c'est la méthode naïve pour un motif d'une seule lettre) :

```
1 def occurrences_lettre(une_chaine, une_lettre):
2     nb = 0                  # INITIALISER
3     for c in une_chaine:    # PARCOURIR
4         if c == une_lettre:   # TESTER
5             nb = nb + 1     # METTRE A JOUR
6     return nb               # RENVOYER
```

   Par exemple `occurrences_lettre('Bonjour', 'o')` renvoie 2.

★ **Exercice 11** *Type bac — Le principe de Boyer-Moore* (d'après Asie 2025, jour 1) ↑énoncé p. 597

Barème indicatif (sur 1) : Q1 : 1

1. On aligne le `mot` sous le début du `texte`, puis on compare les caractères **de droite à gauche**, en partant de la **fin** du mot. Si tous les caractères coïncident, le mot est trouvé (`True`). Sinon, on regarde le caractère du texte qui a provoqué la différence (règle du « mauvais caractère ») : grâce à une table calculée à l'avance à partir du mot seul (prétraitement), on **décale le mot vers la droite** de façon à aligner ce caractère avec sa dernière occurrence dans le mot, ou **au-delà de ce caractère** s'il n'apparaît pas dans le mot. On recommence jusqu'à trouver le mot ou dépasser la fin du texte (`False`). Ces sauts permettent de ne pas lire tous les caractères du texte : l'algorithme est en pratique bien plus rapide que la méthode naïve, surtout pour des mots longs.

★★★ **Exercice 12** *Type bac — Rechercher et remplacer dans un éditeur* (sujet maison, façon bac) ↑énoncé p. 598

Barème indicatif (sur 6) : Q1 : 0,75 ; Q2 : 0,5 ; Q3 : 0,75 ; Q4 : 0,75 ; Q5 : 1 ; Q6 : 1,25 ; Q7 : 0,5 ; Q8 : 0,5

1. $n = 20$ (les espaces comptent). `recherche_naive('CHAT', texte)` renvoie `[3, 14]` et `recherche_naive('LE', texte)` renvoie `[0, 11]`.

2. La boucle examine les positions 0 à $n - m = 16$, soit $n - m + 1 = \mathbf{17}$ positions.

3. La première comparaison porte sur `motif[0]`, c'est-à-dire `C`. Or le texte ne contient la lettre `C` qu'aux positions 3 et 14 : partout ailleurs, la première comparaison échoue et la boucle `while` s'arrête aussitôt (1 comparaison). Aux positions 3 et 14, les quatre lettres correspondent (4 comparaisons). Total : $15 \times 1 + 2 \times 4 = \mathbf{23}$ comparaisons.

4. Par exemple le motif `'AAAB'` dans le texte `'AAAAAAAAAAAA'` (douze `A`) : à chacune des $12 - 4 + 1 = 9$ positions, les trois `A` correspondent et l'échec n'a lieu que sur la dernière lettre, soit 4 comparaisons. Total : $9 \times 4 = \mathbf{36}$ comparaisons, c'est-à-dire $(n - m + 1) \times m$: le maximum possible, puisqu'on ne fait jamais plus de m comparaisons par position. *(Le motif `'AAAA'` convient aussi.)*

5. *Appels attendus, écrits avant de coder :*

```
assert est_occurrence('CHAT', texte, 3)          # cas normal
assert est_occurrence('TON', texte, 17)          # cas limite : derniere position
                                                 # possible
assert not est_occurrence('CHAR', texte, 3)      # cas piege : seule la derniere lettre
                                                 # differe
```

   On compare lettre à lettre, comme dans la méthode naïve :

```
def est_occurrence(motif, texte, i):
    for j in range(len(motif)):          # PARCOURIR
        if texte[i + j] != motif[j]:     # TESTER
            return False                 # RENVOYER : des le premier desaccord
    return True                          # RENVOYER
```

6. `(a) resultat = resultat + nouveau` ; `(b) i = i + m` (on saute toute l'occurrence) ; `(c) resultat = resultat + texte[i]` ; `(d) i = i + 1`.

   `remplacer_tout('CHAT', 'LOUP', texte)` renvoie `'LE LOUP ET LE LOUPON'` (le `CHAT` de `CHATON` est remplacé lui aussi).

7. `recherche_naive` teste *toutes* les positions, y compris celles qui chevauchent une occurrence déjà trouvée : `AA` est présent en 0 et en 1. `remplacer_tout`, après une occurrence, saute les m caractères remplacés (ligne `(b)`) : l'occurrence en 0 donne `B`, puis on reprend en position 2, où il ne reste que `A`, recopié tel quel. C'est le comportement attendu : un même caractère du texte ne peut pas être remplacé deux fois.

8. Au pire $(n - m + 1) \times m \approx 10^6 \times 10 = 10^7$ comparaisons. L'algorithme de **Boyer-Moore** (ou sa version simplifiée, **Horspool**) est bien plus rapide en pratique : il compare le motif de droite à gauche et, grâce à une table calculée à l'avance à partir du motif, **saute** plusieurs positions d'un coup en cas d'échec, sans lire les caractères survolés.

★★ Exercice 13 IA IA IA

Type bac — Un antivirus à la recherche de signatures (sujet maison, façon bac) ↑énoncé p. 599

Barème indicatif (sur 6) : Q1 : 0,75 ; Q2 : 0,75 ; Q3 : 1,25 ; Q4 : 0,75 ; Q5 : 1 ; Q6 : 0,75 ; Q7 : 0,75

1. On parcourt les lettres du motif **sauf la dernière** : la lettre en position k reçoit le décalage $m - 1 - k$ (si une lettre apparaît plusieurs fois, c'est sa dernière position qui l'emporte) ; toute autre lettre reçoit le décalage par défaut m.

   `'4D5A90'` ($m = 6$) : `4`→ 5, `D`→ 4, `5`→ 3, `A`→ 2, `9`→ 1, défaut **6**.

2. `'E8FFFF'` ($m = 6$) : `E`→ 5, `8`→ 4, `F`→ 1, défaut **6**. On n'exclut que la *dernière case* du motif, pas la lettre `F` elle-même : `F` apparaît aussi aux positions 2, 3 et 4, et c'est la position 4 (la plus à droite hors dernière case) qui donne le décalage $6 - 1 - 4 = 1$.

3. Déroulé de Horspool :

position i	fenêtre	lettre de fin	décalage
$i = 0$	`00E84D`	`D`	4
$i = 4$	`4D5A00`	`0` (échec en $j = 4$)	6
$i = 10$	`90FF4D`	`D`	4
$i = 14$	`4D5A90`	`0` (**trouvé**)	6 (dépasse la fin)

   La fonction renvoie `[14]`.

4. Horspool a testé **4** alignements, contre $22 - 6 + 1 = \mathbf{17}$ pour la méthode naïve. Comparaisons : 1 (en $i = 0$) +2 (en $i = 4$: `0` correspond, puis `0` $\neq$ `9`) +1 (en $i = 10$) +6 (en $i = 14$) = **10** comparaisons. *(La méthode naïve en effectue 26.)*

5. *Appels attendus, écrits avant de coder :*

```
assert est_infecte(contenu, ['E8FFFF', '4D5A90'])     # cas normal
assert not est_infecte(contenu, [])                   # cas limite : aucune signature
assert not est_infecte(contenu, ['E8FFFF'])           # cas piege : E8 et FF presents,
                                                      # pas a la suite
```

   On essaie les signatures une à une :

```
def est_infecte(contenu, signatures):
    for s in signatures:                  # PARCOURIR
        if len(horspool(s, contenu)) > 0: # TESTER
            return True                   # RENVOYER : une signature suffit
    return False                          # RENVOYER
```

   Par exemple, avec le contenu de la question 3 :

   `est_infecte(contenu, ['E8FFFF', '4D5A90'])` renvoie `True` ;

   `est_infecte(contenu, ['E8FFFF'])` renvoie `False`.

6. Chaque appel à `horspool` reconstruit la table : la table d'une signature est donc construite jusqu'à 100 000 **fois** (une fois par fichier analysé), alors qu'elle ne dépend que de la signature. Il suffit de construire **une seule fois**, avant l'analyse, un dictionnaire qui associe à chaque signature sa table de décalage (1 000 constructions en tout), puis d'utiliser une version de `horspool` qui reçoit la table en paramètre au lieu de la recalculer. On dépense un peu de mémoire pour gagner du temps.

7. **Oui**, c'est plausible : la table d'une signature longue autorise des décalages plus grands (jusqu'à m quand la lettre de fin de fenêtre est absente du motif). En pratique, Horspool teste de l'ordre de n/m alignements : plus m est grand, moins il y a d'alignements, et la recherche est plus rapide (au pire des cas, le coût reste toutefois en $O(n \times m)$).

★★★ Exercice 14 — *Type bac — Repérer des amorces dans un brin d'ADN* (sujet maison, façon bac)

↑énoncé p. 600

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 1 ; Q3 : 0,75 ; Q4 : 0,5 ; Q5 : 1 ; Q6 : 1 ; Q7 : 0,5 ; Q8 : 0,75

Partie A

1. `TGCATG` = `T0 G1 C2 A3 T4 G5` : `table_dernier` renvoie `{'T': 4, 'G': 5, 'C': 2, 'A': 3}`.

2. Déroulé (le brin est `C0 A1 T2 G3 G4 A5 T6 G7 C8 A9 T10 G11 C12 C13`) :

position i	fenêtre	échec / trouvé	décalage
$i = 0$	`CATGGA`	échec en $j = 5$, `c = A`	$\max(1, 5-3) = 2$
$i = 2$	`TGGATG`	échec en $j = 2$, `c = G`	$\max(1, 2-5) = 1$
$i = 3$	`GGATGC`	échec en $j = 5$, `c = C`	$\max(1, 5-2) = 3$
$i = 6$	`TGCATG`	**trouvé**	1
$i = 7$	`GCATGC`	échec en $j = 5$, `c = C`	3 (dépasse la fin)

   La fonction renvoie `[6]` (5 alignements testés).

3. En $i = 2$: le caractère fautif `G` a sa dernière position (5) **à droite** de l'échec ($j = 2$), donc `j - dernier['G']` $= -3$. Sans le `max(1, ...)`, on ajouterait -3 à i : le motif **reculerait** en $i = -1$. Sur cet exemple, i prend ensuite les valeurs -2, -3, -2, -3… (en Python, un indice négatif lit le texte depuis la fin) : le programme **boucle sans fin**. Le `max` garantit qu'on avance toujours d'au moins un cran, donc que l'algorithme termine.

Partie B

4. Lettres sauf la dernière (`T0 G1 C2 A3 T4`) : `T`→ 1, `G`→ 4, `C`→ 3, `A`→ 2, défaut **6**.

5. Déroulé de Horspool :

position i	fenêtre	lettre de fin	décalage
$i = 0$	`CATGGA`	`A`	2
$i = 2$	`TGGATG`	`G` (échec en $j = 2$)	4
$i = 6$	`TGCATG`	`G` (**trouvé**)	4 (dépasse la fin)

   Horspool trouve l'amorce en position 6 avec **3** alignements, contre **5** pour Boyer-Moore (règle du mauvais caractère seule). En $i = 2$, Boyer-Moore ne peut avancer que d'un cran (question 3), alors que Horspool, qui regarde toujours la lettre de fin de fenêtre, saute de 4. Ce n'est pas une règle générale : les deux algorithmes sont aussi rapides en pratique.

Partie C

6. *Appels attendus, écrits avant de coder :*

```
assert cherche_amorces(['TGCATG', 'GGA', 'AAAA'], brin) == {'TGCATG': [6], 'GGA': [3], 'AAAA': []}   # cas normal
assert cherche_amorces([], brin) == {}                                  # cas limite : aucune amorce
assert cherche_amorces(['ATG'], brin) == {'ATG': [1, 5, 9]}    # cas piege : plusieurs positions
```

On range la liste des positions de chaque amorce dans le dictionnaire :

```
def cherche_amorces(amorces, brin):
    resultat = {}                          # INITIALISER
    for a in amorces:                      # PARCOURIR
        resultat[a] = horspool(a, brin)    # METTRE A JOUR
    return resultat                        # RENVOYER
```

7. On réutilise le dictionnaire renvoyé par `cherche_amorces`, et on construit par compréhension la liste des amorces qui n'ont aucune occurrence :

```
def amorces_absentes(amorces, brin):
    occurrences = cherche_amorces(amorces, brin)
    return [a for a in amorces if len(occurrences[a]) == 0]
```

`amorces_absentes(['TGCATG', 'GGA', 'AAAA'], brin)` renvoie `['AAAA']`.

Autre méthode : une boucle qui ajoute à la liste chaque amorce absente.

```
def amorces_absentes(amorces, brin):
    absentes = []                                  # INITIALISER
    occurrences = cherche_amorces(amorces, brin)
    for a in amorces:                              # PARCOURIR
        if len(occurrences[a]) == 0:               # TESTER
            absentes.append(a)                     # METTRE A JOUR
    return absentes                                # RENVOYER
```

8. Méthode naïve : $n - m + 1 \approx 3 \times 10^9$ alignements (3 milliards). Horspool, si chaque saut vaut $m = 20$: environ $n/m = 3 \times 10^9/20 = 1{,}5 \times 10^8$ alignements (150 millions), soit vingt fois moins.

Critiquer une réponse d'IA

★★ **Exercice 15** *Une table de décalage construite par un assistant d'IA* ↑énoncé p. 601

1. **Non.** La table obtenue est `{'A': 1, 'N': 2, 'S': 0}` : la dernière lettre `S` reçoit le décalage 0. Déroulé : $i = 0$, fenêtre `ANANES`, comparaison de droite à gauche : `S = S`, puis `E` $\neq$ `A` ; la lettre de fin de fenêtre est `S`, décalage `dec['S'] = 0`, donc i reste à 0… et la même fenêtre est examinée indéfiniment : le programme **boucle sans fin** (sortie réelle : aucune, il faut l'interrompre).

2. L'erreur : la **dernière lettre** du motif ne doit *pas* figurer dans la table (cours : « *on parcourt ses lettres sauf la dernière* »), sinon son décalage vaut 0. Ligne corrigée :

```
    for k in range(m - 1):          # on ignore la DERNIERE lettre
```

La table devient `{'A': 1, 'N': 2}` (`S` absent, donc décalage 6). Déroulé corrigé : $i = 0$ (fin de fenêtre `S` → saut 6), $i = 6$ (fenêtre « espace + `ANANA` », fin `A` → saut 1), $i = 7$: `ANANAS` trouvé. La fonction renvoie 7.

3. Regarder la table produite : un décalage de 0 est **toujours** une erreur (on n'avancerait jamais). Plus généralement, vérifier qu'un algorithme de recherche termine sur un texte où le motif est *absent* ou précédé d'une fausse piste.

S'entraîner à l'oral

★★ **Exercice 16** *Expliquer en deux minutes* ↑énoncé p. 602

▶ **Solution.** Pas de texte à apprendre par cœur : voici les **éléments attendus** pour chaque sujet. L'ordre, les mots et l'exemple peuvent être différents ; l'explication est réussie si ces idées y sont, justes et reliées entre elles.

Sujet 1.

- On place le motif (de longueur m) à chaque position du texte (de longueur n) et on compare caractère par caractère : une **fenêtre glissante** qui avance d'un cran.
- Exemple : chercher `NSI` dans `LES NSI` : positions $0, 1, 2, 3$ échouent dès la première lettre, la position 4 réussit.
- Coût dans le pire cas : de l'ordre de $n \times m$ comparaisons (texte `AAAA...A`, motif `AAB`).
- Piège : les bornes ; la dernière position possible est $n - m$.

Sujet 2.

- On compare le motif au texte en partant de la **fin** du motif.
- Une **table de décalage** est préparée à l'avance : pour chaque caractère du motif (sauf le dernier), la distance entre sa dernière apparition et la fin du motif ; pour un caractère absent, la longueur du motif.
- En cas d'échec, on décale selon le caractère du texte aligné avec la fin du motif ; s'il n'apparaît pas dans le motif, on saute de m positions d'un coup.
- Exemple : motif `CHAT` : décalages `C` $\to 3$, `H` $\to 2$, `A` $\to 1$, tout autre caractère $\to 4$.

Sujet 3.

- Chaque saut de m positions évite de lire des caractères du texte : on en lit souvent bien moins que n (coût **sous-linéaire**, de l'ordre de n/m dans le meilleur cas).
- Exemple : un motif de dix lettres rares dans un texte en français avance le plus souvent de dix positions à la fois.
- Pire cas : petit alphabet et nombreuses répétitions (ADN, `AAAA...`) ; les décalages valent 1, et l'on retrouve le coût de la méthode naïve.
- On paie un peu de mémoire (la table) pour gagner beaucoup de temps ; usages : « Rechercher » d'un éditeur, antivirus, bio-informatique.

Entraînement

★ **Exercice 17** *Tables de décalage en série* ↑énoncé p. 602

a) `CHAT` ($m = 4$) : `C`$\to 3$, `H`$\to 2$, `A`$\to 1$, défaut **4**.
b) `MAMAN` ($m = 5$) : `M`$\to 2$, `A`$\to 1$, défaut **5** (`M` en position 2 et `A` en position 3 remplacent leurs valeurs précédentes).
c) `RADAR` ($m = 5$) : `R`$\to 4$, `A`$\to 1$, `D`$\to 2$, défaut **5** (le `R` final est ignoré, seul compte celui de la position 0).
d) `BONBON` ($m = 6$) : `B`$\to 2$, `O`$\to 1$, `N`$\to 3$, défaut **6**.

▷ **Erreur fréquente.** En c), donner à `R` le décalage 0 à cause du `R` final : la dernière case du motif est ignorée. `R` garde la valeur de sa position 0, soit $5 - 1 - 0 = 4$.

★ **Exercice 18** *Horspool à la main en série* ↑énoncé p. 602

a) Table de `TAC` : `T`→ 2, `A`→ 1, défaut 3.

position i	fenêtre	lettre de fin	décalage
$i = 0$	`TIC`	`C`	3
$i = 3$	`TAC`	`C` (**trouvé**)	3
$i = 6$	`TOC`	`C`	3 (dépasse la fin)

Occurrence en position **3**.

b) Table de `GAG` : `G`→ 2, `A`→ 1, défaut 3.

position i	fenêtre	lettre de fin	décalage
$i = 0$	`TGA`	`A`	1
$i = 1$	`GAG`	`G` (**trouvé**)	2
$i = 3$	`GCG`	`G`	2
$i = 5$	`GAG`	`G` (**trouvé**)	2 (dépasse la fin)

Occurrences en positions **1** et **5**.

c) Table de `CODE` : `C`→ 3, `O`→ 2, `D`→ 1, défaut 4.

position i	fenêtre	lettre de fin	décalage
$i = 0$	`DECO`	`O`	2
$i = 2$	`CODE`	`E` (**trouvé**)	4
$i = 6$	`RLEC`	`C`	3
$i = 9$	`CODE`	`E` (**trouvé**)	4 (dépasse la fin)

Occurrences en positions **2** et **9**.

CORRIGÉS DES TP

TP La course des algorithmes de recherche ↑énoncé p. 605

Partie A — Compter le travail

Exercice 1 *La méthode naïve, avec compteur*

1. $i = 0$: 1 (`A`≠`C`) ; $i = 1$: 2 ; $i = 2$: 2 ; $i = 3$: 1 ; $i = 4$: 3 (trouvé). Total : **9 comparaisons**.

2.

```python
def naive_compte(motif, texte):
    n = len(texte)
    m = len(motif)
    positions = []
    comp = 0
    for i in range(n - m + 1):
        j = 0
        while j < m and texte[i + j] == motif[j]:
            comp += 1                    # comparaison reussie
            j += 1
        if j < m:
            comp += 1                    # la comparaison qui a echoue
```

```
        else:
            positions.append(i)
    return positions, comp
```

Exercice 2 *Horspool, avec compteur*

1. $i = 0$: fenêtre `CAA`, `G`≠`A` (1 comparaison), fin `A` → saut 2 ; $i = 2$: fenêtre `AGA`, 1 comparaison, fin `A` → saut 2 ; $i = 4$: `ACG`, 3 comparaisons, trouvé ; fin `G` (absente de la table) → saut 3, $i = 7$: fin. **3 alignements, 5 comparaisons**.

2.

```
def horspool_compte(motif, texte):
    n = len(texte)
    m = len(motif)
    dec = table_decalage(motif)
    positions = []
    comp = 0
    i = 0
    while i <= n - m:
        j = m - 1
        while j >= 0 and texte[i + j] == motif[j]:
            comp += 1                    # comparaison reussie
            j -= 1
        if j >= 0:
            comp += 1                    # la comparaison qui a echoue
        else:
            positions.append(i)
        c = texte[i + m - 1]
        if c in dec:
            i += dec[c]
        else:
            i += m
    return positions, comp
```

Exemple `MOTIF` du cours : **9** comparaisons pour Horspool (4 échecs immédiats puis 5), **26** pour la méthode naïve.

Exercice 3 *Une référence, et mille tests*

```
def toutes_positions(motif, texte):
    n = len(texte)
    m = len(motif)
    positions = []
    for i in range(n - m + 1):           # toutes les positions : chevauchements permis
        if texte[i:i + m] == motif:
            positions.append(i)
    return positions

def mot_aleatoire(longueur):
    mot = ""
    for k in range(longueur):
        mot = mot + random.choice("AB")
    return mot

for k in range(1000):
    texte = mot_aleatoire(random.randint(0, 30))
    motif = mot_aleatoire(random.randint(1, 5))
    ref = toutes_positions(motif, texte)
    assert naive_compte(motif, texte)[0] == ref
    assert horspool_compte(motif, texte)[0] == ref
```

Avec deux lettres seulement, le motif apparaît **souvent**, avec beaucoup de correspondances partielles et de chevauchements : ce sont les situations délicates, celles où se cachent les erreurs d'indice. Sur un alphabet de 26 lettres, presque tous les tests seraient des « absent » sans intérêt.

Partie B — Un vrai roman

Exercice 4 *À la recherche de Phileas Fogg*

1. Mesures sur `verne.txt` ($n = 440\,824$ caractères, fichier Gutenberg complet) :

Motif (m)	Occ.	Comp. naïf	Comp. Horspool	Rapport
`Fix` (3)	275	$1{,}00\,n$	$0{,}35\,n$	2,9
`Fogg` (4)	655	$1{,}01\,n$	$0{,}27\,n$	3,8
`Aouda` (5)	136	$1{,}00\,n$	$0{,}23\,n$	4,4
`Passepartout` (12)	423	$1{,}01\,n$	$0{,}13\,n$	7,6
`quatre-vingts jours` (19)	18	$1{,}02\,n$	$0{,}10\,n$	10,3

2. Le naïf fait toujours un peu plus de n comparaisons, quel que soit le motif : presque toutes les positions échouent dès la première lettre (ici une majuscule ou une lettre peu fréquente), quelques-unes à la deuxième. Horspool en fait de moins en moins quand le motif s'allonge (de l'ordre de n/m, un peu plus pour les motifs courts).

3. À chaque échec, il saute souvent de m caractères (ou presque) : les caractères **survolés** lors des sauts ne sont jamais comparés.

4. `fogg` n'est pas trouvé (ou presque pas) : la recherche distingue majuscules et minuscules (`"F" != "f"`). On cherche `"fogg"` dans le texte mis en minuscules (et on écrit aussi le motif en minuscules) :

```
def minuscules(texte):
    resultat = ""
    for c in texte:
        if "A" <= c <= "Z":
            resultat = resultat + chr(ord(c) + 32)   # majuscule -> minuscule
        else:
            resultat = resultat + c
    return resultat
```

Dans le roman : `fogg` 0 fois, `Fogg` 655 fois, et 685 fois dans `minuscules(texte)` (les 30 de plus sont écrits en capitales, `FOGG`, dans les titres de chapitres). Les capitales accentuées (`É…`) ne sont pas converties : sans effet ici.

Exercice 5 *Un mini-*`grep`

`grep` *filtre* les lignes : on construit par compréhension la liste des couples `(numéro, ligne)` pour les seules lignes où Horspool trouve au moins une position (on peut aussi partir d'une liste vide et ajouter chaque couple avec `append`).

```
def grep(motif, texte):
    lignes = texte.split("\n")
    return [(k + 1, lignes[k]) for k in range(len(lignes))
            if horspool_compte(motif, lignes[k])[0] != []]

def surligner(motif, ligne):
    positions = horspool_compte(motif, ligne)[0]
    resultat = ""
    i = 0
    for p in positions:
        if p >= i:              # on ignore une occurrence qui chevauche la precedente
            resultat = resultat + ligne[i:p] + "[" + motif + "]"
            i = p + len(motif)
    return resultat + ligne[i:]

for numero, ligne in grep("Passepartout", texte)[:5]:
    print(numero, surligner("Passepartout", ligne))
```

2. On ne peut pas mettre de crochets qui se chevauchent : on garde la première occurrence et on ignore celles qui la recouvrent, d'où `"[aa]a"` (c'est aussi le choix de `grep` et des éditeurs).

3. Le texte sans fins de ligne se construit comme `minuscules` :

```
def sans_fins_de_ligne(texte):
    resultat = ""
    for c in texte:
        if c == "\n":
            resultat = resultat + " "
        else:
            resultat = resultat + c
    return resultat
```

Dans le roman : 282 lignes, 282 occurrences dans le texte entier, 301 après le remplacement. `grep` compte des *lignes* : une ligne qui contient deux fois le motif ne compterait qu'une fois (ce n'est jamais le cas ici, d'où 282 = 282). Surtout, quand `Phileas` est en fin de ligne et `Fogg` au début de la suivante, le texte contient `"Phileas\nFogg"` : ni `grep`, ni la recherche dans le texte entier ne trouvent le motif (le caractère de fin de ligne n'est pas une espace). En remplaçant les fins de ligne par des espaces, on retrouve ces occurrences coupées (19 ici), au prix de la perte des numéros de ligne. *(Sur un petit texte :* `grep("Fogg voyage", "Phileas Fogg\nvoyage")` *renvoie* `[]`*, mais le motif est trouvé après le remplacement.)*

Partie C — La longueur du motif, et la taille de l'alphabet

Exercice 6 *Plus le motif est long…*

1.

```
def moyenne_comparaisons(algo, texte, m, essais=5, graine=1):
    random.seed(graine)
    total = 0
    for k in range(essais):
        p = random.randint(0, len(texte) - m)
        total += algo(texte[p:p + m], texte)[1]
    return total / essais / len(texte)

adn = genome(1_000_000)
for m in (2, 4, 8, 16, 32, 64):
    print(m, moyenne_comparaisons(naive_compte, adn, m),
             moyenne_comparaisons(horspool_compte, adn, m))
```

2. Valeurs obtenues (graines fixées : exactes et reproductibles) :

m	**2**	**4**	**8**	**16**	**32**	**64**
Naïf, roman	1,08	1,10	1,10	1,10	1,10	1,10
Horspool, roman	0,56	0,30	0,16	0,10	0,07	0,05
Naïf, génome	1,25	1,33	1,33	1,33	1,33	1,33
Horspool, génome	0,71	0,54	0,40	0,41	0,35	0,45

3. Sur le roman, le travail de Horspool est à peu près **divisé par deux** quand m double (0,30 → 0,16 → 0,10), comme $1/m$ (un peu au-dessus) : c'est bien le comportement **sous-linéaire** en n/m du cours. Pour $m = 64$, on ne compare qu'environ 1 caractère sur 20.

4. Dans un motif de 32 bases, chacune des quatre lettres apparaît presque à coup sûr **près de la fin**. Par exemple, `table_decalage(adn[1000:1032])` vaut :
`{'T': 9, 'A': 1, 'C': 2, 'G': 4}`. Le saut est donc toujours petit (ici au plus 9, souvent 1 à 4), quelle que soit la longueur du motif : le gain plafonne. Plus l'alphabet est **petit**, moins Horspool est efficace.

5. À une position quelconque, la première comparaison est toujours faite ; elle réussit avec probabilité 1/4, et l'on fait alors une deuxième comparaison, qui réussit avec probabilité 1/4… En

moyenne : $1 + \frac{1}{4} + \frac{1}{16} + \cdots \approx \frac{4}{3} \approx 1{,}33$ comparaison par position. Sur un texte français, la première lettre du motif est plus rare (alphabet plus grand), d'où $\approx 1{,}1$.

6. Avec `matplotlib` : pour chaque ligne, `plt.plot(longueurs, valeurs, label=...)` ; puis `plt.xscale("log")`, `plt.legend()`, `plt.show()`.

Partie D — Les cas défavorables

Exercice 7 *Construire le pire*

1. Mesures ($n = 20\,000$) :

```
texte = "A" * 20000
for m in (5, 10, 20, 40):
    m1 = "A" * (m - 1) + "B"
    m2 = "B" + "A" * (m - 1)
    print(m, naive_compte(m1, texte)[1], horspool_compte(m1, texte)[1],
             naive_compte(m2, texte)[1], horspool_compte(m2, texte)[1], 20000 * m)
```

m	naïf, `A...AB`	Horspool, `A...AB`	naïf, `BA...A`	Horspool, `BA...A`	$n \times m$
5	99 980	19 996	19 996	99 980	100 000
10	199 910	19 991	19 991	199 910	200 000
20	399 620	19 981	19 981	399 620	400 000
40	798 440	19 961	19 961	798 440	800 000

2. Motif `A...AB` : le **naïf** est piégé : il compare de gauche à droite, réussit $m-1$ fois et n'échoue que sur le `B`, puis ne décale que de 1 : m comparaisons à chacune des $n-m+1$ positions. Horspool commence par la fin : échec immédiat sur le `B` (1 comparaison)… mais il ne saute que de 1 (la lettre de fin de fenêtre est `A`, à distance 1 de la fin du motif) : $n-m+1$ comparaisons. Motif `BA...A` : c'est l'inverse, **Horspool** est piégé (m comparaisons de droite à gauche, puis saut de 1), le naïf échoue tout de suite.

3. Les deux sont en $O(n \times m)$ dans le pire des cas (coût **quadratique** si m est de l'ordre de n). Ce pire cas exige un texte très répétitif (une seule lettre) : il n'arrive pratiquement jamais en français, mais il peut arriver sur des données binaires ou des séquences d'ADN très répétées.

4. Un motif sans aucun `A`, par exemple `"B" * 10` : la lettre de fin de fenêtre est toujours `A`, absente de la table, donc saut de 10 après une seule comparaison : $20\,000/10 = 2\,000$ comparaisons.

Partie E — Face à Python

Exercice 8 *Contre la montre*

1. Durées mesurées (meilleure de trois exécutions, motifs de 16 caractères absents du texte : `genome(16, 1)` pour le génome, `"le gentleman dit"` pour le roman) :

	`naive_compte`	`horspool_compte`	`present` (`in`)
Génome (10^6 bases)	200 ms	63 ms	4,6 ms
Roman ($4{,}4 \times 10^5$ car.)	72 ms	15 ms	0,31 ms

L'opérateur `in` est environ 14 fois plus rapide que notre Horspool sur le génome, 50 fois sur le roman (l'ordre de grandeur dépend de l'ordinateur).

2. Notre Horspool est **interprété** : chaque tour de boucle Python coûte des dizaines d'instructions machine (et nous incrémentons en plus un compteur). L'opérateur `in` est compilé en langage machine, avec un algorithme du même type. Le nombre de comparaisons mesure le travail **de l'algorithme**, indépendamment de la machine et du langage ; la durée mesure aussi le coût de **l'implémentation**. Les deux mesures se complètent.

Exercice 9 *Défi : Boyer-Moore réduit au mauvais caractère*

```python
def table_dernier(motif):
    dernier = {}
    for k in range(len(motif)):
        dernier[motif[k]] = k
    return dernier

def boyer_moore_compte(motif, texte):
    n = len(texte)
    m = len(motif)
    dernier = table_dernier(motif)
    positions = []
    comp = 0
    i = 0
    while i <= n - m:
        j = m - 1
        while j >= 0 and texte[i + j] == motif[j]:
            comp += 1                    # comparaison reussie
            j -= 1
        if j < 0:
            positions.append(i)
            i += 1
        else:
            comp += 1                    # la comparaison qui a echoue
            c = texte[i + j]
            if c in dernier:
                k = dernier[c]
            else:
                k = -1
            i += max(1, j - k)
    return positions, comp
```

Moyennes obtenues (comparaisons divisées par n, $m = 2$ à 64) : génome 0,83 ; 0,58 ; 0,50 ; 0,53 ; 0,33 ; 0,49 — roman 0,58 ; 0,31 ; 0,16 ; 0,10 ; 0,07 ; 0,05. Réduit au seul mauvais caractère, Boyer-Moore ne fait **pas mieux** que Horspool : quasi identique sur le roman, souvent un peu moins bon sur le génome (le caractère fautif est souvent une lettre présente juste à côté dans le motif, d'où des sauts de 1). C'est ce qui justifie la simplification de Horspool ; le vrai gain de Boyer-Moore vient de la règle du bon suffixe [au-delà du programme].

Bilan — éléments attendus

- Texte ordinaire : naïf $\approx n$ comparaisons (1,0 à 1,1 n en français, 1,33 n sur l'ADN) ; Horspool $\approx n/m$ (sous-linéaire). Pire cas : les deux en $n \times m$ (tableau de la partie D).
- Plus le motif est long, plus les sauts (jusqu'à m) sont grands. Sur l'ADN, les quatre lettres sont toutes près de la fin du motif : les sauts restent petits et le gain plafonne vers 0,4 n.
- Le prétraitement (la table) coûte une seule passe sur le motif (m opérations), négligeable devant la recherche dans un texte de $n \gg m$ caractères : on dépense un peu de mémoire pour gagner beaucoup de temps.
- L'opérateur `in` est compilé et optimisé : on l'utilise en pratique. Programmer Horspool sert à **comprendre** pourquoi il est rapide, et quand il ne l'est pas.

Réponses des questions de la fiche de révision

↑ questions p. 611

Se tester

1. $O(n \times m)$ (§ II).
2. `M`→ 2, `A`→ 1, défaut 5 (§ IV).
3. De 4 (lettre absente : toute la longueur) (§ IV).
4. Elle recevrait 0 : on ne bougerait plus (§ IV).
5. Boyer-Moore : plus le motif est long, plus les sauts sont grands (§ V).

Et avant ?

1. les résultats déjà calculés, pour ne jamais résoudre deux fois le même sous-problème (ch. 11, § II).
2. `192.168.1.0` : les 24 premiers bits sont gardés, les bits d'hôte mis à 0 (ch. 9, § III).
3. `SELECT DISTINCT id_real FROM film;` (ch. 6, § V.E).

CORRIGÉS DU CHAPITRE 13

13 Calculabilité et décidabilité

CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Un programme peut en manger un autre

↑ énoncé p. 625

Exercice 1 *La recette de crêpes*

Situation	Qui travaille ?	Ce qu'il reçoit	Exécutée ou lue ?
cuisinier	le cuisinier	la recette	**exécutée**
photocopieuse	la photocopieuse	la recette	lue (copiée)
traducteur	le traducteur	la recette	lue (traduite)
correcteur orthographique	le correcteur	la recette	lue (vérifiée)
compter les lignes	l'éditeur	la recette	lue (mesurée)

1. Oui : le **même** texte est une suite d'instructions pour le cuisinier, et une simple donnée pour tous les autres. Tout dépend de **qui** le reçoit et de ce qu'il en fait.

Exercice 2 *Python lit du Python*

2. Il s'affiche trois fois `bonjour`. Le programme qui travaille est `python3` (l'**interpréteur**) ; sa donnée est le fichier `test.py`, c'est-à-dire le **texte** d'un programme. (Et `python3` est lui-même lancé par le terminal, qui tourne sur le système d'exploitation…)

3. `compter_lignes(source)` renvoie 3 (trois passages à la ligne) ; aucun `bonjour` n'est affiché : le texte de `accueil` est **lu**, pas exécuté — comme la recette par la photocopieuse.

4. Deux `bonjour` : `exec` **exécute** le texte reçu (définition de `accueil`, puis appel `accueil(2)`) — comme le cuisinier. `exec` est un interpréteur Python *dans* Python.

5. Oui : le texte de `compter_lignes` est une chaîne comme une autre ; elle renverrait 6. Un programme peut recevoir **son propre texte** en entrée : rien de magique, c'est une donnée.

Exercice 3 *Le devin et le contrariant*

6. Le devin a eu raison **zéro** fois. Aucune prédiction n'est gagnante : quelle que soit celle qu'il écrit, le contrariant la lit et fait l'inverse. Le devin n'est pas « mauvais » : sa tâche est **impossible** dès que sa prédiction est lue par celui qu'elle concerne.

7. Le contrariant ne pourrait plus contredire : le devin aurait raison une fois sur deux au hasard, et la contradiction disparaîtrait. Tout le piège vient de ce que la prédiction **porte sur** celui qui la lit et lui est **donnée**.

8. Ni l'un ni l'autre : si elle est vraie, ce qu'elle dit est vrai, donc elle est fausse ; si elle est fausse, ce qu'elle dit est faux, donc elle est vraie (**paradoxe du menteur**).

9. S'il se rase, il fait partie de ceux qui se rasent eux-mêmes, donc il ne devrait pas se raser ; s'il ne se rase pas, il devrait se raser. Aucune réponse ne tient : un tel barbier **ne peut pas exister**.

10. Les trois parlent **d'eux-mêmes** (ou reçoivent un énoncé sur eux-mêmes) et le **contredisent** : c'est l'**auto-référence**.

Exercice 4 *Mettre des mots*

11. Mots attendus : **une donnée** (une chaîne de caractères) ; un **interpréteur** ; **son propre texte** (le programme peut se prendre lui-même en entrée) ; **lit la prédiction qui le concerne et fait le contraire**.

12. Intuition attendue : non. Si un tel devin existait, on pourrait écrire un programme « contrariant » qui demande au devin ce qu'il va faire lui-même, puis fait l'inverse (boucler si le devin prédit l'arrêt, s'arrêter sinon). C'est exactement la preuve du cours. On laisse la question ouverte : le cours y répond.

CORRIGÉS DES EXERCICES

Un programme est une donnée

★ **Exercice 1** *Qui est la machine, qui est la donnée ?* ↑énoncé p. 639

1. Machine : `python3`. Donnée : le fichier `tri.py`.
2. Machine : `gcc`. Donnée : le fichier `jeu.c`.
3. Machine : l'antivirus. Donnée : le fichier `photo.exe`.
4. Machine : l'éditeur de texte. Donnée : le fichier `code.py`.

Dans chaque cas, la « donnée » est un **programme** : un programme peut donc être avalé par un autre.

★ **Exercice 2** *Un programme n'est qu'une chaîne de caractères* ↑énoncé p. 639

1. Un programme est écrit avec des caractères (lettres, chiffres, symboles, retours à la ligne) : son code source **est** une chaîne de caractères, que l'on peut stocker dans une variable, dans un fichier, transmettre, etc.

2. Puisqu'un programme est une simple donnée (une chaîne), rien n'empêche de la passer en argument à un autre programme (un interpréteur, un compilateur…) — ni de passer le code d'un programme **à ce programme lui-même** : c'est ce qui rendra possible le raisonnement du problème de l'arrêt.

Décidable ou indécidable ?

★ **Exercice 3** *Tri sélectif* ↑énoncé p. 639

1. **Décidable** (tester les diviseurs ; l'algorithme s'arrête toujours).

2. **Décidable** (parcourir la liste une fois).

3. **Indécidable** : c'est le **problème de l'arrêt**.

4. **Décidable** (rechercher le mot ; le dictionnaire est fini).

5. **Indécidable** : décider si deux programmes calculent la même fonction est une question sur leur *comportement* (théorème de Rice).

▷ **Erreur fréquente.** Répondre « décidable : il suffit d'exécuter `P` et `Q` et de comparer leurs résultats » : il y a une infinité d'entrées, et une exécution peut ne jamais s'arrêter.

★★ **Exercice 4** *Un décideur qui s'arrête toujours* ↑énoncé p. 640

1. **Démarche :** on teste tous les diviseurs possibles, de 2 à $n-1$; dès qu'on en trouve un, on peut conclure « pas premier » ; on ne conclut « premier » qu'après la boucle.

```
1 def est_premier(n):
2     for d in range(2, n):  # PARCOURIR : (a) on teste 2, 3, ..., n-1
3         if n % d == 0:     # TESTER
4             return False   # RENVOYER : (b) un diviseur -> pas premier
5     return True            # RENVOYER : (c) aucun diviseur -> premier
```

   ▷ **Erreur fréquente.** Placer `return True` dans la boucle (dans un `else`) : la fonction conclurait « premier » dès le premier entier testé qui ne divise pas n (`est_premier(9)` renverrait `True`).

2. La boucle `for` parcourt un intervalle **fini** (2 à $n-1$) : elle se termine forcément, donc la fonction **s'arrête toujours** et renvoie la bonne réponse. Le problème « n est-il premier ? » est donc **décidable**.

Le problème de l'arrêt

★★ **Exercice 5** *Va-t-il s'arrêter ?* ↑énoncé p. 640

1. `compte(5)` : `n` vaut 5,4,3,2,1,0 → **s'arrête** (renvoie `"fini"`). `compte(-3)` : `n` vaut $-3, -4, -5, \ldots$ et ne vaut **jamais** 0 → **ne s'arrête pas**.

   ▷ **Erreur fréquente.** Conclure que `compte(-3)` s'arrête parce que `n` diminue : `n` s'*éloigne* de 0. Un variant doit être un entier positif qui décroît vers la condition d'arrêt.

2. `mystere(6)` : `n` vaut 6,3,10,5,16,8,4,2,1 → **s'arrête**.

3. On a là un programme d'une **extrême simplicité** dont **personne** ne sait s'il s'arrête pour toute entrée. C'est une illustration frappante : prédire l'arrêt d'un programme peut être arbitrairement difficile — et le théorème de l'arrêt dit que, *dans le cas général*, aucune méthode automatique ne peut le faire.

★★ **Exercice 6** *Refaire la preuve du problème de l'arrêt* ↑énoncé p. 640

1. **Cas 1** : `halt(paradoxe, paradoxe)` vaut `True`, donc le test du `if` est vrai : on exécute `while True` → l'appel **ne s'arrête pas**. Or `halt` avait annoncé qu'il s'arrêtait : **contradiction**.

   ▷ **Erreur fréquente.** S'arrêter à « on entre dans `while True` » : il faut encore confronter ce comportement à la prédiction de `halt`, qui annonçait l'arrêt, pour obtenir la contradiction.

2. **Cas 2** : `halt(paradoxe, paradoxe)` vaut `False`, donc le test est faux : on exécute le `else`, qui fait `return` → l'appel **s'arrête**. Or `halt` avait annoncé qu'il ne s'arrêtait pas : **contradiction**.

3. Dans les deux cas, `halt` se trompe : la fonction `halt` **ne peut pas exister**. Le problème de l'arrêt est **indécidable** (Turing, 1936).

★★ **Exercice 7** *Cousins du paradoxe* ↑énoncé p. 641

1. « Cette phrase est fausse » : si on la suppose vraie, elle affirme sa propre fausseté (donc fausse) ; si on la suppose fausse, alors ce qu'elle dit est faux, donc elle est vraie. Aucune valeur de vérité ne tient.

2. Le barbier : s'il se rase lui-même, il fait partie de ceux *qui se rasent*, donc il ne devrait pas se raser ; s'il ne se rase pas, il devrait. Impasse identique.

Lien : dans les deux cas, un objet **parle de lui-même** et se retourne contre sa propre affirmation. C'est exactement le ressort de `paradoxe(paradoxe)`, qui fait *le contraire* de ce que `halt` prédit de lui : l'**auto-référence**.

★★★ **Exercice 8** *La contagion de l'indécidable* ↑énoncé p. 641

1. On raisonne par l'absurde : on suppose que `renvoie42` existe, et on montre qu'il permettrait alors de décider l'arrêt. Soit `prog` un programme quelconque. On fabrique le programme `prog2` suivant.

```
1 def prog2(x):
2     prog(x)      # on execute prog sur x, en ignorant son resultat
3     return 42
```

Si `prog(x)` s'arrête, `prog2(x)` passe à la ligne suivante et renvoie 42 ; si `prog(x)` ne s'arrête pas, `prog2(x)` ne renvoie jamais rien. Donc `prog2(x)` renvoie 42 **si et seulement si** `prog(x)` s'arrête. On aurait alors `halt(prog, x) = renvoie42(prog2, x)` : `renvoie42` déciderait le problème de l'arrêt, ce qui est impossible (théorème de l'arrêt). Donc `renvoie42` n'existe pas : la question « `prog(x)` renvoie-t-il 42 ? » est, elle aussi, indécidable. On dit qu'on a **réduit** (ramené) le problème de l'arrêt à cette question.

2. C'est le **théorème de Rice**.

Calculabilité et Turing-complétude

★ **Exercice 9** IA IA IA *Vrai ou faux* ↑énoncé p. 641

1. **Faux** : C et Python sont tous deux Turing-complets, ils calculent *les mêmes* fonctions.
2. **Vrai** : c'est la définition d'une fonction calculable.
3. **Faux** : la thèse de Church-Turing n'est pas un théorème (elle relie l'intuition de « calcul » à un modèle formel) ; elle est *admise*, jamais démentie.

   ▷ **Erreur fréquente.** Parler du « théorème » de Church-Turing : une notion intuitive (« calculer ») ne peut pas entrer dans une démonstration ; la thèse est admise parce qu'elle n'a jamais été démentie.
4. **Faux** : le nombre d'instructions ne fait pas la puissance ; Brainfuck est Turing-complet, donc aussi puissant que Python (juste beaucoup moins commode).
5. **Faux** : l'indécidabilité ne dépend pas du langage ; le problème de l'arrêt est indécidable dans *tout* langage Turing-complet.

P, NP et le million de dollars [au-delà du programme]

★ **Exercice 10** IA IA IA *Vérifier, est-ce facile ?* ↑énoncé p. 641

1. Trier : **trouver** est facile (algorithme en $n \log n$) et **vérifier** aussi. Problème de classe **P**.
2. Sudoku : **vérifier** une grille remplie est facile ; **trouver** le remplissage semble difficile. (Une grille 9×9 reste à la portée d'un ordinateur ; c'est le Sudoku **généralisé**, sur des grilles $n^2 \times n^2$ de plus en plus grandes, qui est NP-complet.)
3. Voyageur de commerce : **vérifier** qu'une tournée proposée mesure ≤ 500 est facile (on additionne) ; **trouver** la meilleure semble très difficile.

★★ **Exercice 11** IA IA IA *Colorier un graphe* ↑énoncé p. 642

1. On parcourt les arêtes : A–B (rouge/vert ok), A–C (rouge/bleu ok), **B–C (vert/bleu ok)**, B–D (vert/rouge ok), **C–D (bleu/rouge ok)**, B–E (vert/rouge ok), D–E **(rouge/rouge !)**. Deux voisins D et E sont **tous deux rouges** : le coloriage proposé est **incorrect**. La vérification est **rapide** (une passe sur les arêtes).
2. Chaque sommet reçoit 1 couleur parmi 3 : $3^5 = 243$ possibilités pour 5 sommets, et 3^n pour n sommets — une croissance **exponentielle**.

3. **Oui**, il est dans **NP** : on *vérifie* une solution proposée en temps polynomial (une passe sur les arêtes). On **ne sait pas** s'il est dans **P** : aucun algorithme polynomial n'est connu pour *trouver* une 3-coloration (le problème est même NP-complet : en trouver un prouverait P = NP). Attention, NP ne s'oppose pas à P : tout problème de P est aussi dans NP.

★★ **Exercice 12** *Et si P = NP ?* ↑énoncé p. 642

1. **P** : problèmes résolubles en temps polynomial. **NP** : problèmes dont une solution *oui* se *vérifie* en temps polynomial. On a P ⊆ NP car, si l'on sait *résoudre* vite, on sait *vérifier* vite (résoudre puis comparer).

2. Conséquences de P = NP : (sécurité) la **cryptographie RSA** s'effondrerait, car factoriser un grand nombre deviendrait rapide ; (créativité) **trouver** deviendrait aussi facile que **vérifier** — inventer une preuve, une mélodie, une solution ne serait plus « plus dur » que la reconnaître.

3. La quasi-totalité des chercheurs pense que P ≠ NP, mais ce n'est **pas démontré** : c'est un problème **ouvert** (l'un des sept problèmes du millénaire, doté d'un million de dollars).

Vers le bac

★★ **Exercice 13** *Terminaison et problème de l'arrêt (d'après Métropole septembre 2024, jour 2)* ↑énoncé p. 642

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 0,5 ; Q4 : 0,5 ; Q5 : 0,5 ; Q6 : 0,5 ; Q7 : 0,75 ; Q8 : 0,5 ; Q9 : 0,5 ; Q10 : 0,5 ; Q11 : 0,75

Partie A.

1. `f1(7)` : `i` vaut $7, 8, 9, 10 \to$ le `while` s'arrête, **`f1(7)` termine** et renvoie 10.

2. `f1(-2)` : `i` vaut $-2, -1, 0, \ldots, 10 \to$ **termine**, renvoie 10.

3. `f1(12)` : `i` vaut $12, 13, 14, 15, 16, \ldots$ (les 5 premières : 12,13,14,15,16) ; `i` ne vaut **jamais** 10 $\to$ **ne termine pas**.

4. `f1(n)` termine si et seulement si $n \leq 10$ (on peut alors atteindre 10 en incrémentant).

Partie B.

5. `f2(4)` : $4 + f2(2) = 4 + 2 + f2(0) = 4 + 2 + 0 = 6 \to$ **termine**, renvoie 6.

6. `f2(5)` : $5 + f2(3) = 5 + 3 + f2(1) = 5 + 3 + 1 + f2(-1) = \ldots$: on décroît de 2 à partir d'un impair, on **saute** 0 et `n` devient négatif sans jamais valoir 0 $\to$ **ne termine pas**.

   ▷ **Erreur fréquente.** Répondre que `f2(5)` renvoie $5 + 3 + 1 = 9$: après 1, l'appel suivant porte sur -1, et la condition d'arrêt `n == 0` n'est jamais atteinte (en pratique, Python s'interrompt sur une `RecursionError`).

7. `f2(n)` termine (pour n entier naturel) si et seulement si n **est pair**.

8. **Démarche :** une fonction récursive sans aucune condition d'arrêt, dont l'appel récursif ne se rapproche de rien.

```
def infini(n):
    return infini(n)  # APPEL RECURSIF, sans aucun CAS DE BASE
```

(aucune valeur de `n` ne fait terminer l'appel.)

Partie C.

9. Si `arret(code_paradoxe, code_paradoxe)` renvoie `True` : le `if` est vrai, on exécute `infini(42)` → l'appel `paradoxe(code_paradoxe)` **ne termine pas**. Or `arret` affirmait qu'il terminait : **contradiction**.

10. Si `arret(...)` renvoie `False` : le `if` est faux, on exécute `return 0` → l'appel **termine**. Or `arret` affirmait qu'il ne terminait pas : **contradiction**.

11. Dans les deux cas, `arret` donne une réponse démentie par les faits : une telle fonction `arret` **ne peut pas exister**.

★★★ **Exercice 14** IA IA IA *Un programme comme chaîne de caractères (d'après Asie 2025, jour 1)* ↑énoncé p. 643

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 0,5 ; Q3 : 1 ; Q4 : 0,75 ; Q5 : 0,75 ; Q6 : 0,75 ; Q7 : 1,25 ; Q8 : 0,5

1. À la fin : `x` passe de 10 à 0 (dix tours), et `y` monte de 10 à 20. Donc **`x` vaut** 0 **et `y` vaut** 20.

2. Un programme Python est un **texte** (des caractères) : on peut donc le stocker dans une chaîne, la manipuler, et l'exécuter avec `exec`.

3. `programme3` : `x` vaut 10,8,6,4,2,0 → **termine**. `programme4` : `x` croît (10,12,14,...), le test `x>0` reste vrai → **ne termine pas**. `programme5` : `x=10` et le test `x<0` est **faux** d'emblée, la boucle ne s'exécute pas → **termine** aussitôt. `programme6` : `x` vaut 10,6,2, -2, -6, ... et ne vaut jamais 0 → **ne termine pas**.

4. `arret_essai1` **exécute** le programme puis renvoie `True`. Cela ne décide **rien** : si le programme passé **ne s'arrête pas**, alors `exec(programme)` ne s'arrête pas non plus, et la fonction **elle-même ne termine pas** — donc ne renverra jamais `False`. Or un *décideur* doit s'arrêter dans **tous** les cas. Ce code ne convient pas.

5. *Appels attendus, écrits avant de coder :*

```
assert arret_essai2("x = 0\nwhile x > 0:\n    x = x - 1") == False   # cas normal
assert arret_essai2("") == True                                      # cas limite : programme vide
assert arret_essai2("# while") == False                              # cas piege : mot en toute fin
```

```
def arret_essai2(programme):
    m = len("while")
    for i in range(len(programme) - m + 1):  # PARCOURIR les positions
        if programme[i:i + m] == "while":    # TESTER : fenetre glissante
            return False                     # RENVOYER des qu'on le trouve
    return True                              # RENVOYER
```

Autre méthode : avec l'opérateur `in` des chaînes (`present` du TP du chapitre 12), `return "while" not in programme`.

Contre-exemples. (i) `True` alors que ça ne s'arrête pas : un programme **sans `while`** dont la boucle ne finit jamais, par exemple `L = [0]` puis `for x in L: L.append(x)` (la liste s'allonge à chaque tour, le parcours n'atteint jamais la fin, jusqu'à épuisement de la mémoire) ; il n'y a pas `while`, donc `arret_essai2` renvoie `True`, pourtant il ne s'arrête pas. (Une récursion infinie, `def g(): g()` puis `g()`, ne convient qu'en théorie, avec une pile illimitée : en pratique Python l'interrompt par une `RecursionError`.) (ii) `False` alors que ça s'arrête : `while False: pass` *contient* le mot `while` (donc `arret_essai2` renvoie `False`), pourtant le programme s'arrête immédiatement. La présence du mot « while » ne dit rien de la terminaison.

6.
```python
def terminaison_inverse(programme):
    if arret(programme):      # si le programme termine...
        while True:           # ... on boucle pour toujours
            pass
    else:
        return                # sinon on termine
```

7. Notons P la chaîne `programme_paradoxal`, c'est-à-dire `"terminaison_inverse(programme_paradoxal)"`. Exécuter P revient à évaluer `terminaison_inverse`(P). Si `arret`(P) dit que P **termine**, alors `terminaison_inverse` **boucle** $\to P$ ne termine pas : contradiction. Si `arret`(P) dit que P **ne termine pas**, alors `terminaison_inverse` **termine** $\to P$ termine : contradiction. Dans les deux cas `arret` se trompe : une telle fonction `arret` **ne peut pas exister** (problème de l'arrêt).

8. **Non** : l'impossibilité n'a rien à voir avec Python. Tout langage assez expressif (Turing-complet) permet d'écrire ce même paradoxe : le problème de l'arrêt est indécidable **quel que soit le langage**.

★★ Exercice 15 *Un correcteur automatique d'exercices (sujet maison, façon bac)*

↑énoncé p. 644

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 1 ; Q3 : 0,75 ; Q4 : 0,75 ; Q5 : 0,75 ; Q6 : 0,75 ; Q7 : 1 ; Q8 : 0,5

Partie A.

1. Le premier paramètre, `fonction`, est lui-même une **fonction**, c'est-à-dire un programme : `verifier` est un programme qui prend **un autre programme en argument** (un programme est une donnée).

2. `verifier(carre_a, tests)` renvoie `3` ; `verifier(carre_b, tests)` renvoie `2` ($2 \times 0 = 0$ et $2 \times 2 = 4$ sont justes, mais $2 \times 3 = 6 \neq 9$) ; `verifier(carre_c, tests)` renvoie `3` (pour $n \geq 0$, la boucle ajoute n exactement n fois) ; `verifier(carre_d, tests)` renvoie `3`.

3. `carre_d` réussit les trois tests mais renvoie 9 pour toute autre entrée : `carre_d(5)` renvoie 9 au lieu de 25. Des tests réussis ne **prouvent pas** qu'une fonction est correcte : ils ne vérifient qu'un nombre **fini** d'entrées, alors qu'il y en a une infinité.

Partie B.

4. `i` vaut $0, 1, 2, 3, \ldots$: il augmente et ne vaudra **jamais** -3. L'appel `carre_c(-3)` **ne termine pas**, donc `verifier(carre_c, tests)` **ne termine pas non plus** : la plateforme reste bloquée.

5. Par exemple :

```python
def carre_c(n):
    resultat = 0                    # INITIALISER
    i = 0                           # INITIALISER
    while i < abs(n):
        resultat = resultat + abs(n)  # METTRE A JOUR
        i = i + 1
    return resultat                 # RENVOYER
```

On ajoute $|n|$ exactement $|n|$ fois, donc on obtient $|n| \times |n| = n^2$. **Variant** : $|n| - i$ est un entier positif tant qu'on est dans la boucle et il **diminue de** 1 à chaque tour ; il ne peut pas décroître indéfiniment, donc la boucle s'arrête (après $|n|$ tours).

6. **Non.** Ce n'est pas une décision de l'arrêt : une fonction **correcte mais lente** (par exemple `carre_c(10**9)`, qui fait un milliard de tours de boucle) dépasse les 2 secondes et serait déclarée « non terminante » alors qu'elle termine. La règle se trompe dans ce sens-là ; seule la réponse « termine » est sûre.

7. C'est le **problème de l'arrêt**, **indécidable** (Turing, 1936). Idée : si `arret` existait, on écrirait un programme `paradoxe` qui, appliqué à lui-même, **boucle** si `arret` prédit qu'il s'arrête et **s'arrête** si `arret` prédit qu'il boucle ; dans les deux cas `arret` se trompe : contradiction, donc `arret` n'existe pas.

8. **Non.** Savoir si deux fonctions renvoient toujours le même résultat est une question non triviale sur leur **comportement** : elle est indécidable d'après le **théorème de Rice**. (Et les tests n'y suffisent pas, cf. question 3.)

★★★ **Exercice 16** *Un antivirus infaillible ? (sujet maison, façon bac)* ↑énoncé p. 645

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 1 ; Q3 : 0,5 ; Q4 : 1 ; Q5 : 0,75 ; Q6 : 0,75 ; Q7 : 0,75 ; Q8 : 0,75

Partie A.

1. **Oui** : la donnée d'entrée est le code source d'un programme (une chaîne de caractères), et la réponse est *oui* (`True`) ou *non* (`False`).

2. *Appels attendus, écrits avant de coder :*

```
assert contient("x = 1\nformater_disque()", "formater_disque") == True   # cas normal
assert contient("abc", "") == True                                        # cas limite : motif vide
assert contient("disq", "disque") == False                                # cas piege : motif trop long
```

Par exemple :

```
def contient(texte, motif):
    n = len(motif)
    for i in range(len(texte) - n + 1):  # PARCOURIR les positions
        if texte[i:i + n] == motif:      # TESTER la tranche
            return True                  # RENVOYER des qu'on le trouve
    return False                         # RENVOYER
```

3. **Oui**, elle s'arrête toujours : la boucle `for` parcourt un nombre **fini** de positions. Mais elle décide le problème « le **texte** du programme contient-il le mot `formater_disque` ? », qui porte sur le *texte* du programme et non sur son *comportement*.

4. `code1` : `True`, mais `1 > 2` est faux, l'appel n'a **jamais** lieu → **fausse alerte**. `code2` : `True`, mais le mot n'apparaît que dans un **commentaire** → **fausse alerte**. `code3` : `False` (le mot complet n'apparaît pas, il est fabriqué par concaténation), pourtant `exec` exécute bien `formater_disque()` → **virus non détecté**.

Partie B.

5. `piege` exécute d'abord `programme`, qui n'appelle jamais `formater_disque()`, puis l'appel ajouté. Si `programme` **termine**, on arrive à la dernière ligne et `formater_disque()` est appelée. S'il **ne termine pas**, on n'atteint jamais cette ligne et la fonction n'est jamais appelée. Donc `piege` est dangereux **si et seulement si** `programme` termine.

6. On peut écrire :

```
def arret(programme):
    return dangereux(programme + "\nformater_disque()\n")
```

Elle s'arrête toujours (car `dangereux` s'arrête toujours) et donne la bonne réponse d'après la question précédente.

7. On aurait écrit une fonction qui **décide le problème de l'arrêt**, ce qui est impossible (Turing). L'hypothèse de départ est donc fausse : la fonction `dangereux` parfaite **ne peut pas exister**.

8. Le **théorème de Rice** : toute question non triviale sur le **comportement** d'un programme est indécidable. Les antivirus réels restent très utiles (signatures de virus connus, analyse des comportements suspects, exécution surveillée…), mais ils doivent accepter de **se tromper parfois** : des fausses alertes et des virus non détectés. Aucun ne peut être infaillible.

★★★ Exercice 17 — *Un interpréteur miniature (sujet maison, façon bac)*

↑énoncé p. 646

Barème indicatif (sur 8) : Q1 : 0,5 ; Q2 : 1 ; Q3 : 1 ; Q4 : 1 ; Q5 : 1,25 ; Q6 : 1 ; Q7 : 1 ; Q8 : 1,25

1. Le programme est donné sous forme d'une **liste de chaînes de caractères** : c'est une simple **donnée**. `executer` est un **interpréteur**, exactement comme `python3`, qui lit et exécute le texte d'un programme Python.

2. On complète ainsi :

```
        elif mots[0] == "dec":
            if x > 0:
                x = x - 1
            i = i + 1
        elif mots[0] == "sinul":
            if x == 0:
                i = int(mots[1])
            else:
                i = i + 1
```

3. Déroulé (numéro de l'instruction exécutée, valeur de `x` *avant* l'instruction) : $(0,2)$, $(1,2)$, $(2,1)$, $(0,1)$, $(1,1)$, $(2,0)$, $(0,0)$; à cette dernière étape, `x` vaut 0, on saute à l'instruction 3, qui n'existe pas : le programme s'arrête. `executer(P1, 2, 100)` renvoie `0`, après 7 **instructions**. `P1` **remet `x` à zéro** (en le diminuant de 1 à chaque tour).

4. `P2` augmente `x` puis retourne à l'instruction 0, **indéfiniment** : il n'y a aucune instruction de saut vers la fin, le numéro d'instruction vaut toujours 0 ou 1. Il **ne s'arrête jamais**, et `executer(P2, 0, 1000)` renvoie `None` (limite atteinte).

5. Un passage dans les instructions 0 à 4 : si `x` vaut 0, on sort (saut en 6) ; sinon on retire 1 ; si `x` vaut alors 0, on saute en 5 ; sinon on retire encore 1 et on recommence. Chaque tour retire donc 2. Si `x` est **pair**, on finit par trouver `x` $= 0$ à l'instruction 0 : le programme **s'arrête**. Si `x` est **impair**, on trouve `x` $= 0$ à l'instruction 2 et l'on saute en 5, où `"aller 5"` renvoie **sans fin** sur elle-même : le programme **ne s'arrête pas**. `P3` s'arrête **si et seulement si `x` est pair**.

6. `P1` avec $x = 10^6$ exécute $3 \times 10^6 + 1$ instructions, plus que la limite de 10^6 : `executer` renvoie `None`, donc `arret_limite(P1, 10**6)` renvoie `False`. C'est **faux** : `P1` s'arrête toujours. Augmenter la limite ne règle rien en général : quelle que soit la limite L, il suffit de prendre x tel que $3x + 1 > L$ pour la mettre en défaut.

7. **Oui** : si `executer` renvoie une valeur, c'est que le programme **s'est effectivement arrêté** (on l'a vu s'arrêter). En revanche, `None` signifie seulement « pas encore arrêté » : on ne peut pas savoir s'il s'arrêterait un peu plus tard ou jamais. Exécuter permet de **constater** l'arrêt, jamais de **prouver** qu'il n'aura pas lieu.

8. **Non.** Pour un langage Turing-complet, le **problème de l'arrêt est indécidable** : le raisonnement de Turing (programme `paradoxe` qui se prend lui-même en donnée) s'y transpose, car l'indécidabilité ne dépend pas du langage.

Critiquer une réponse d'IA

★★ **Exercice 18** *Le problème de l'arrêt expliqué par un assistant d'IA* ↑énoncé p. 647

1. **Non.** Le programme proposé s'arrête, et on le *prouve* sans l'exécuter : `n` est un entier positif qui **décroît strictement** à chaque tour (un **variant** de boucle, vu en Première), donc la boucle fait exactement 10 tours. On sait donc parfaitement, *au cas par cas*, prouver qu'un programme s'arrête.

2. La phrase fautive est « *il est impossible de prouver qu'un programme donné s'arrête ; la seule chose que l'on puisse faire est de l'exécuter et d'attendre* ». Elle contredit le cours : « *on peut très bien, au cas par cas, prouver qu'un programme donné s'arrête (avec un variant de boucle). Ce qui est impossible, c'est une unique méthode automatique et universelle qui marcherait sur tous les programmes* ». Correction : *il n'existe aucun algorithme général qui décide l'arrêt de tous les programmes ; pour un programme particulier, une preuve (variant, raisonnement) reste possible.* Tout le reste de la réponse (énoncé, date, schéma de la preuve, indépendance du langage, contagion aux autres questions de comportement) est exact.

3. Confronter chaque affirmation générale à un **exemple concret** que l'on maîtrise : le premier programme à boucle bornée venu réfute « impossible de prouver ». Plus généralement, relire la définition (« *pour toute entrée* », « *un algorithme* ») : indécidable signifie « pas de méthode universelle », pas « jamais de réponse ».

S'entraîner à l'oral

★★ **Exercice 19** *Expliquer en deux minutes* ↑énoncé p. 648

► Solution. Pas de texte à apprendre par cœur : voici les **éléments attendus** pour chaque sujet. L'ordre, les mots et l'exemple peuvent être différents ; l'explication est réussie si ces idées y sont, justes et reliées entre elles.

Sujet 1.

- Un programme est un **texte** (une suite de caractères, donc de bits) : un autre programme peut le lire, le transformer ou l'exécuter.
- Exemples : l'interpréteur Python (il reçoit un programme et l'exécute), un compilateur, un antivirus, un correcteur automatique d'exercices.
- Exemple simple : un programme Python qui compte les lignes d'un fichier `.py`.
- Conséquence : on peut se demander si un programme pourrait analyser *tous* les programmes ; c'est l'origine du problème de l'arrêt.

Sujet 2.

- Problème : écrire `arret(prog, entree)` qui renvoie `True` si `prog` s'arrête sur `entree` et `False` sinon, pour **tous** les programmes.
- Par l'absurde : on suppose qu'elle existe et on construit `paradoxe(prog)`, qui boucle sans fin si `arret(prog, prog)` vaut `True` et s'arrête sinon.
- Question : `paradoxe(paradoxe)` s'arrête-t-il ? S'il s'arrête, `arret` répond `True`, donc il boucle ; s'il boucle, `arret` répond `False`, donc il s'arrête : contradiction.
- Conclusion (Turing, 1936) : aucun programme ne peut décider l'arrêt de tous les programmes.

Sujet 3.

- Un problème de décision est **décidable** s'il existe un algorithme qui répond correctement (oui ou non), en un temps fini, pour **toute** entrée.
- **Indécidable** : aucun algorithme **unique** ne fonctionne pour toutes les entrées.
- Exemple : on prouve facilement qu'un programme précis, avec une seule boucle `for`, s'arrête ; c'est la méthode universelle qui n'existe pas.
- Piège : confondre avec « trop long à calculer » (question de coût, P et NP) ; indécidable signifie impossible même avec un temps illimité.

CORRIGÉS DES TP

TP La machine de Turing et le prix de la force brute

↑ énoncé p. 651

Partie 1 — Un simulateur de machine de Turing

Exercice 1 *Faire tourner une machine à la main*

1. Trace obtenue (`executer(INVERSER, "1011", trace=True)`, le ^ désigne la case lue) :

```
    q0 | 1011          q0 | 0101
         ^                   ^
    q0 | 0011          q0 | 0100_
          ^                   ^
    q0 | 0111         fin | 0100_
           ^                   ^
```

5 étapes ; le ruban contient `0100`.

2. La machine **inverse chaque bit** (complément à un) et s'arrête au premier blanc.

Exercice 2 *Le cœur du simulateur*

```python
def une_etape(regles, ruban, tete, etat):
    lu = lire_case(ruban, tete)
    ecrit, deplacement, nouvel_etat = regles[(etat, lu)]
    ruban[tete] = ecrit
    if deplacement == "D":
        tete = tete + 1
    else:
        tete = tete - 1
    return tete, nouvel_etat
```

Exercice 3 *Une machine qui ne s'arrête pas*

`None` dans les deux cas. **1.** Sans limite, `executer(FUITE, "1")` ne rendrait jamais la main : le simulateur hériterait de la non-terminaison de la machine simulée. **2.** Non : `None` signifie seulement « pas arrêtée *au bout de* `limite` étapes ». Une machine peut s'arrêter après $10^6 + 1$ étapes ; quelle que soit la limite choisie, il existe des machines qui s'arrêtent juste après. Décider l'arrêt pour *toutes* les machines est impossible : c'est le **problème de l'arrêt** (indécidable, Turing 1936). En revanche, une réponse « arrêtée » est **sûre**.

Partie 2 — Programmer des machines

Exercice 4 *Ajouter 1 en binaire*

```
INCREMENT = {
    ("q0", "0"): ("0", "D", "q0"),          # aller jusqu'au bout
    ("q0", "1"): ("1", "D", "q0"),
    ("q0", BLANC): (BLANC, "G", "retenue"),
    ("retenue", "1"): ("0", "G", "retenue"),  # 1 + 1 = 10 : la retenue continue
    ("retenue", "0"): ("1", "G", "fin"),      # 0 + 1 = 1 : termine
    ("retenue", BLANC): ("1", "G", "fin"),    # 111 -> 1000 : nouveau chiffre a gauche
}
```

`1011` → `1100` en 8 étapes ; `111` → `1000` en 8 étapes (4 pour aller au bout, 3 pour propager la retenue, 1 pour poser le `1`). Le test du fichier, qui porte sur les 300 premiers entiers, affiche alors `OK`.

Exercice 5 *Un nombre pair de 1 ?*

```
PARITE = {
    ("q0", "0"): ("0", "D", "q0"),          # q0 : nombre pair de 1 lus
    ("q0", "1"): ("1", "D", "impair"),
    ("q0", BLANC): (BLANC, "G", "oui"),
    ("impair", "0"): ("0", "D", "impair"),  # impair : nombre impair de 1 lus
    ("impair", "1"): ("1", "D", "q0"),
    ("impair", BLANC): (BLANC, "G", "non"),
}
```

1. Deux états suffisent : l'un « retient » qu'on a vu un nombre pair de `1`, l'autre un nombre impair (exemples : `10110` → `non`, `1001` → `oui`). **2.** Dans son **état** (une mémoire finie) et sur le **ruban** (une mémoire illimitée).

Exercice 6 *Reconnaître un palindrome*

Règles à ajouter :

```
    ("compare_a", "a"): (BLANC, "G", "retour"),
    ("compare_a", "b"): ("b", "G", "non"),
    ("compare_a", BLANC): (BLANC, "G", "oui"),   # longueur impaire : symbole central
    ("compare_b", "b"): (BLANC, "G", "retour"),
    ("compare_b", "a"): ("a", "G", "non"),
    ("compare_b", BLANC): (BLANC, "G", "oui"),
    ("retour", "a"): ("a", "G", "retour"),
    ("retour", "b"): ("b", "G", "retour"),
    ("retour", BLANC): (BLANC, "D", "q0"),
```

Les 511 mots de longueur 0 à 8 sont correctement classés (même réponse que `mot == mot[::-1]`, qui compare le mot à son inverse). Sur `abba` : `oui` en 15 étapes.

Exercice 7 *Combien d'étapes ?*

Mesures (exactes, elles ne dépendent pas de l'ordinateur) :

Longueur n	20	40	80	160	320
Étapes	231	861	3 321	13 041	51 681

1. Quand n double, le nombre d'étapes est multiplié par environ 4 : coût **quadratique**. Pour un palindrome de longueur paire n, on trouve exactement $\frac{(n+1)(n+2)}{2}$ étapes : la tête fait environ $n/2$ allers-retours de longueur décroissante. **2.** Non : Python et la machine de Turing calculent **exactement les mêmes** fonctions — c'est un fait démontré (Python est Turing-complet : notre simulateur montre qu'il sait imiter n'importe quelle machine de Turing). La thèse de Church-Turing ajoute que rien de « calculable » au sens intuitif n'échappe à ce modèle. Ce qui diffère d'un modèle à l'autre, c'est la **commodité** et la **vitesse** (ici quadratique contre linéaire), jamais ce qui est **calculable**.

Partie 3 — Trouver ou vérifier ? [au-delà du programme]

Exercice 8 *Vérifier une solution proposée*

```
def verifier(nombres, cible, choix):
    deja = [False] * len(nombres)       # deja[i] : indice i deja utilise ?
    total = 0
    for i in choix:
        if deja[i]:                      # un indice utilise deux fois
            return False
        deja[i] = True
        total = total + nombres[i]
    return total == cible
```

1. k additions (plus la création du tableau `deja`, de taille n, et un test par indice) : coût **linéaire**, donc polynomial. **2.** Mesuré : environ 0,05 s pour 500 000 indices dans une liste d'un million de nombres.

Exercice 9 *Chercher par force brute*

1. `13` s'écrit `1101` : bits 0, 2 et 3 à 1, d'où `[0, 2, 3]` ; `sous_ensemble(0, 4)` renvoie `[]` (aucun nombre choisi). Chaque nombre est pris ou non : 2^n choix, numérotés par les entiers de 0 à $2^n - 1$ (tous les mots de n bits).

```
def chercher(nombres, cible):
    n = len(nombres)
    essais = 0
    for k in range(2 ** n):
        choix = sous_ensemble(k, n)
        essais = essais + 1
        if verifier(nombres, cible, choix):
            return choix, essais
    return None, essais
```

2. `chercher([8, 3, 12, 5, 7], 15)` renvoie `([1, 2], 7)` ($3 + 12$, trouvé au 7e essai, $k = 6$) ; avec la cible 100 : `(None, 32)`, les 2^5 choix ont été essayés.

Exercice 10 *Mesurer l'explosion*

Durées mesurées :

n	10	12	14	16	18	20
Durée (s)	0,003	0,016	0,075	0,35	1,5	6,9

1. Environ $\times 4{,}5$ quand n augmente de 2, donc à peu près $\times 2$ quand n augmente de 1 : coût **exponentiel** (2^n essais ; le léger excès vient de ce que chaque essai coûte lui-même environ n opérations). **2.** $n = 40$: $6{,}9 \times 2^{20} \approx 7 \times 10^6$ s, soit environ 3 mois (plutôt 5 à 6 en tenant compte du facteur n). $n = 60$: encore $\times 2^{20}$, environ 7×10^{12} s, soit plus de 200 000 ans. **3.** $1\,000 \approx 2^{10}$: un ordinateur mille fois plus rapide ne gagne qu'**une dizaine** d'éléments. Face à une croissance exponentielle, le matériel ne sauve pas : il faudrait un meilleur **algorithme**. **4.** Vérifier une solution proposée est rapide (linéaire) ; en trouver une, avec les seuls algorithmes connus, demande un temps exponentiel. « P = NP » affirmerait qu'il existe un algorithme **polynomial** pour trouver (pour ce problème NP-complet, donc pour tous ceux de NP) ; on pense que non, sans savoir le démontrer.

Partie 4 — Une expérience sur l'arrêt

Exercice 11 *La suite de Syracuse*

1.

```python
def syracuse_etapes(n, limite):
    etapes = 0
    while n != 1:
        if etapes == limite:
            return None
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
        etapes = etapes + 1
    return etapes
```

2. Vérifié : aucun `None` de 1 à 100 000 avec une limite de 1 000. Le record est 77 031, avec 350 étapes (trouvé par une boucle sur `n` de 1 à 100 000 qui mémorise le plus grand nombre d'étapes rencontré).
3. `syracuse_etapes(27, 100)` renvoie `None`, mais `syracuse_etapes(27, 1000)` renvoie 111 (la suite monte jusqu'à 9 232 avant de redescendre). `None` ne veut donc **pas** dire « ne s'arrête jamais » : seulement « pas encore ». Même dissymétrie qu'à l'exercice 3.
4. Le programme s'arrête **si et seulement si** la conjecture de Goldbach est **fausse** (il renvoie alors le premier contre-exemple). Comme la fonction `halt` du cours prend deux arguments, un programme et sa donnée, on donne à `chercher_contre_exemple` un paramètre `x` qu'elle n'utilise pas ; il suffirait alors de calculer `halt(chercher_contre_exemple, 0)` : `True` signifierait « conjecture fausse », `False` « conjecture vraie ». Un détecteur d'arrêt universel résoudrait mécaniquement d'innombrables problèmes ouverts des mathématiques : c'est une raison supplémentaire de ne pas être surpris qu'il ne puisse pas exister. (La conjecture tient pour tous les entiers pairs jusqu'à 20 000, et bien au-delà, ce qui ne prouve rien.)

Bilan — éléments attendus

- Quelques lignes de Python simulent **n'importe quelle** machine de Turing : Python est Turing-complet. Le simulateur lit la table de règles comme une **donnée** : c'est une machine **universelle** au sens de Turing (cours, § IV.A). `PALINDROME` calcule la même chose que `mot == mot[::-1]`, en $\frac{(n+1)(n+2)}{2}$ étapes au lieu d'un parcours : la puissance de calcul est la même, la vitesse non.
- Vérifier : quelques centièmes de seconde pour un million de nombres ; chercher : plusieurs secondes dès 20 nombres, doublement à chaque nombre ajouté.
- Une limite ne donne qu'une réponse sûre dans un sens (« s'arrête ») ; pour l'autre, aucune limite ne suffit. Un vrai détecteur d'arrêt universel est impossible (problème de l'arrêt).

Réponses des questions de la fiche de révision

↑ questions p. 657

Se tester

1. Il existe un algorithme qui s'arrête toujours et répond juste pour toute entrée (§ II).
2. Non : $7, 5, 3, 1, -1, \ldots$ jamais 0 (§ III.A).
3. Il s'arrête (`return`) : `halt` s'est trompé, contradiction (§ III.C).
4. par exemple l'interpréteur `python3`, un compilateur, un antivirus, un éditeur qui colore la syntaxe (§ I).
5. P versus NP : trouver une solution est-il aussi facile que la vérifier ? On croit que non (§ VI).

Et avant ?

1. `N`→ 2, `S`→ 1, défaut 3 (la dernière lettre n'est pas inscrite) (ch. 12, § IV).
2. symétrique : une seule clé, partagée ; asymétrique : une clé publique pour chiffrer, une clé privée pour déchiffrer (ch. 10, § II et III).
3. `[1, 2, 3, 4, 9]` : une fois `[2, 3]` épuisée, on recopie le reste `[9]` (ch. 7, § III.A).

CORRIGÉS DU CHAPITRE 14

14 Systèmes sur puce et informatique embarquée

CORRIGÉS DES ACTIVITÉS PRÉPARATOIRES

ACTIVITÉ 1 Un ordinateur dans une carte de 5 cm

↑énoncé p. 670

Exercice 1 *Un bouton, une lumière*

▶ **Solution.** **1.** Tant que A est enfoncé, un cœur s'affiche ; dès qu'on le relâche, l'écran s'éteint (au plus 0,1 s plus tard).

2. Percevoir : `button_a.is_pressed()` (ligne 4, on lit l'état du bouton). **Décider** : le `if` / `else` (lignes 4 et 6). **Agir** : `display.show(Image.HEART)` et `display.clear()` (lignes 5 et 7).

3. Le programme s'exécute **une seule fois**, au lancement : le bouton n'est pas enfoncé à cet instant, l'écran est effacé, puis le programme se termine. Appuyer ensuite sur A ne fait plus rien. Un système embarqué doit surveiller le monde **en permanence** : d'où la boucle sans fin.

4. `if button_a.is_pressed() and button_b.is_pressed():`

Exercice 2 *Un thermomètre*

▶ **Solution.** **5.** La carte fait défiler la température choisie avec le curseur (dans le simulateur, le curseur va d'environ −5 à 50 degrés). `temperature()` renvoie un **entier** (`int`), en degrés Celsius ; `str(t)` le transforme en texte pour l'afficher.

6. Capteur : le capteur de température de la carte (il mesure, c'est une **entrée**). **Actionneur** : l'écran de LED (il agit, c'est une **sortie**).

Exercice 3 *Un thermostat*

▶ **Solution.** **7.** Condition : `t < consigne`. À 15 degrés, la LED centrale s'allume (radiateur en marche) ; à 25 degrés, elle s'éteint.

```
1  from microbit import *
2
3  consigne = 19
4  while True:
5      t = temperature()
6      if t < consigne:
7          display.set_pixel(2, 2, 9)    # radiateur allume
8      else:
9          display.clear()               # radiateur eteint
10     sleep(1000)
```

8. Avec dix minutes d'attente, le programme ne verrait un changement de température qu'avec jusqu'à dix minutes de retard : la pièce pourrait refroidir longtemps avant que le radiateur ne s'allume. Pour un radiateur, c'est inconfortable mais sans danger. Pour un **airbag**, un retard de quelques millisecondes est déjà inacceptable : la réponse doit arriver **à temps**, c'est une **contrainte de temps réel**.

Exercice 4 *Ce qu'on a découvert*

▶ **Solution. 9.**

- … **lit un capteur, décide, puis commande un actionneur** (percevoir → décider → agir).
- … **mesurer** une grandeur physique (entrée) ; … **agir** sur le monde physique (sortie).
- … **exécute directement son programme**, sans système d'exploitation. Le processeur et la mémoire sont **tous dans la petite puce noire** de la carte.

CORRIGÉS DES EXERCICES

★ **Exercice 1** *Le bon vocabulaire* ↑énoncé p. 680

(1)–(c) ; **(2)**–(e) ; **(3)**–(b) ; **(4)**–(a) ; **(5)**–(d).

▷ **Erreur fréquente.** Inverser capteur et actionneur : le capteur *mesure* (il fournit une entrée au programme), l'actionneur *agit* (il exécute une commande de sortie).

★ **Exercice 2** *SoC ou pas SoC ?* ↑énoncé p. 680

(a) carte bancaire → **microcontrôleur** (une tâche précise, minuscule, sobre) ; *(b)* smartphone → **SoC** (ordinateur complet intégré) ; *(c)* PC de jeu évolutif → **carte mère classique** (on remplace/ajoute des composants) ; *(d)* thermostat connecté → **microcontrôleur** (embarqué, capteur + actionneur) ; *(e)* montre connectée → **SoC** (petit, mais ordinateur complet avec OS).

▷ **Erreur fréquente.** Ranger la montre connectée parmi les microcontrôleurs parce qu'elle est petite : elle fait tourner un système d'exploitation et des applications ; c'est un ordinateur complet, donc un SoC.

★★ **Exercice 3** *Microcontrôleur contre SoC* ↑énoncé p. 680

1. Par exemple : puissance (quelques dizaines de MHz contre plusieurs GHz multi-cœurs) ; mémoire (quelques Ko contre plusieurs Go) ; consommation (minuscule contre modérée) ; prix (quelques euros contre quelques dizaines) ; **système d'exploitation** (aucun contre un OS complet).

2. Un microcontrôleur est trop peu puissant et a trop peu de mémoire pour un OS ; sans OS, il exécute *directement* son programme, de façon **prévisible**. Pour un airbag, on veut une réponse *garantie à temps* : un OS pourrait interrompre le programme au mauvais moment.

3. Tous deux **intègrent sur une seule puce** les composants d'un ordinateur (processeur, mémoire, périphériques) ; sur le SoC d'un téléphone, la mémoire vive principale est une puce empilée dans le même boîtier.

★★ **Exercice 4** *La loi de Moore, chiffres en main* ↑énoncé p. 680

1. $50 \div 2 = \mathbf{25}$ doublements.

   ▷ **Erreur fréquente.** Compter 50 doublements en 50 ans : la loi prévoit un doublement tous les **deux** ans.

2. $2\,300 \times 2^{25} \approx 2\,300 \times 33{,}5$ millions $\approx \mathbf{7{,}7 \times 10^{10}}$, soit environ **80 milliards** de transistors.

3. Non : la loi prévoit ≈ 80 milliards, la réalité (A15, 2021) est ≈ 15 milliards, soit environ 5 **fois moins**. La loi de Moore donne le bon *ordre de grandeur* et la bonne *tendance*, mais elle a **ralenti** ces dernières années (elle reste une prédiction empirique, pas une loi physique).

4. $3 \div 0{,}2 = \mathbf{15}$ atomes environ : c'est un ordre de grandeur, car « 3 nm » est un nom commercial (les motifs réels sont plusieurs fois plus larges, de l'ordre de la dizaine de nanomètres), mais on reste à quelques dizaines d'atomes. On approche des limites **physiques** : on ne peut pas graver plus fin qu'un atome. La loi de Moore ne pourra donc pas continuer éternellement.

★★ **Exercice 5** *Dérouler un thermostat* ↑énoncé p. 681

1. **Capteur** : le capteur de température (entrée). **Actionneur** : le radiateur (sortie).

2. **Démarche :** un tour de boucle lit le capteur, décide (trop froid ou non), puis commande l'actionneur. `(a) t = lire_temperature()` ; `(b) allumer_radiateur()` ; `(c) eteindre_radiateur()`.

3. **Non** : quelques secondes de retard sur un chauffage sont sans danger, alors qu'un freinage ABS doit réagir en quelques millisecondes. La contrainte de temps réel est bien plus **faible** ici (d'où le `attendre(30)`).

▷ **Erreur fréquente.** Confondre « temps réel » et « très rapide » : un système temps réel doit répondre dans un délai **garanti**, adapté à sa tâche ; pour un thermostat, ce délai peut être de plusieurs secondes.

★★ **Exercice 6** *Von Neumann ou Harvard ?* ↑énoncé p. 681

1. **Von Neumann** : programme *et* données dans la **même** mémoire. **Harvard** : programme (mémoire morte, Flash) et données (mémoire vive, RAM) dans **deux mémoires séparées**.

   ▷ **Erreur fréquente.** Inverser les deux modèles : c'est l'architecture de von Neumann qui range programme et données dans une **même** mémoire ; celle de Harvard les sépare.

2. Avec deux bus, le processeur peut charger **une instruction et une donnée en même temps** (au même cycle), au lieu de faire deux accès successifs : c'est plus rapide.

3. Un microcontrôleur vise la **simplicité et la vitesse** sur une tâche fixe : Harvard (deux mémoires dédiées) y est efficace. Un SoC vise la **polyvalence** (exécuter n'importe quel programme, chargé en mémoire) : le modèle de von Neumann, plus souple, s'y prête mieux. *(Nuance : au plus près de chaque cœur, un SoC a des mémoires locales, des caches séparés pour les instructions et pour les données : une touche de Harvard à l'intérieur d'une architecture de von Neumann.)*

★★ **Exercice 7** *Le prix de l'intégration* ↑énoncé p. 681

1. **Intérêts** (deux au choix) : faible consommation d'énergie, petite taille, faible coût, grande vitesse (courtes distances). **Limites** (deux au choix) : non réparable, non évolutif, obsolescence, déchets électroniques.

2. Tout étant intégré et soudé, on **ne peut pas remplacer** la seule batterie : il faut changer tout le téléphone (non réparable / non évolutif). Cela alimente la **montagne de déchets électroniques** et motive les lois sur le **droit à la réparation** et l'indice de réparabilité.

3. L'énergie consommée par un circuit finit en **chaleur**, dissipée dans les transistors et dans les connexions ; transporter des signaux sur de longs fils entre puces séparées en consomme beaucoup. En consommant peu (composants proches, sans long câblage), un SoC chauffe peu, donc n'a souvent **pas besoin de ventilateur** (il est aussi *silencieux*).

★★ **Exercice 8** *Exercice d'entraînement — un microcontrôleur dans la voiture* ↑énoncé p. 681

1. **Entrée** : le capteur de vitesse (**capteur**). **Sortie** : le dispositif d'accélération/freinage (**actionneur**).
2. Par exemple : **faible consommation** (autonomie, peu de chaleur), **faible coût** (des dizaines par voiture), **petite taille** (s'intègre partout), **fiabilité / temps réel** (pas d'OS qui pourrait retarder la réponse). (Deux suffisent.)
3. Un régulateur ou un freinage doit réagir en une fraction de seconde : un retard mettrait en danger les passagers. La **garantie de répondre à temps** est donc une exigence de **sécurité**, pas seulement de confort.
4. `v`= 120 < `consigne`= 130 : la condition `v < consigne` est vraie, donc le système **accélère** (`accelerer()`), puis attend 0,1 s avant de re-mesurer.

★ **Exercice 9** *Type bac — Plusieurs processeurs sur une puce* (d'après Amérique du Nord 2026, jour 2) ↑énoncé p. 682

Barème indicatif (sur 2) : Q1 : 1 ; Q2 : 1

1. Avec un seul processeur, un seul processus s'exécute à un instant donné : le parallélisme n'est qu'**apparent** (alternance). Avec plusieurs processeurs (ou plusieurs cœurs), plusieurs processus s'exécutent **réellement en même temps**, un par processeur : plus de calculs sont réalisés par seconde, les processus attendent moins leur tour et la machine reste réactive même quand l'un d'eux calcule beaucoup (par exemple décoder de la musique pendant que l'interface répond à l'utilisateur).
2. **Avantage** (un seul suffit) : les composants (processeurs, GPU, modules de communication, et la mémoire dans le même boîtier) sont regroupés sur une même puce, donc très proches : communications rapides, **faible consommation** (autonomie de la batterie) et peu de chaleur, faible encombrement, coût réduit en grande série.
   Inconvénient (un seul suffit) : le SoC n'est **ni réparable ni évolutif** : si un composant tombe en panne ou devient insuffisant (mémoire par exemple), on ne peut pas le remplacer seul, il faut changer toute la puce, voire l'appareil ; sa conception est aussi très coûteuse.

★★★ **Exercice 10** *Type bac — Choisir la puce d'une montre connectée* (sujet maison, façon bac) ↑énoncé p. 682

Barème indicatif (sur 6) : Q1 : 0,75 ; Q2 : 0,5 ; Q3 : 0,75 ; Q4 : 0,75 ; Q5 : 1,25 ; Q6 : 1,25 ; Q7 : 0,75

1. **A** et **C** sont des **microcontrôleurs** : fréquence de quelques dizaines à une centaine de MHz, mémoire de quelques centaines de Ko à 1 Mo, **aucun système d'exploitation**, consommation et prix très faibles. **B** est un **SoC** de type smartphone : plusieurs cœurs à plus d'un GHz, 2 Go de mémoire, un système d'exploitation.
2. Dans les deux cas, **les composants d'un ordinateur** (processeur, mémoire, entrées-sorties) sont intégrés sur **un seul circuit intégré** (pour un SoC de téléphone, la mémoire vive principale est empilée dans le même boîtier).
3. Puce A : 300 ÷ 2 = **150** h ; puce B : 300 ÷ 20 = **15** h ; puce C : 300 ÷ 8 = **37,5** h.
4. Les puces **A** et **C** tiennent au moins 24 h (la puce B seulement 15 h). Pourtant, seule la puce **B** peut faire tourner les applications : elles nécessitent un **système d'exploitation**, et beaucoup de mémoire et de puissance, ce qu'un microcontrôleur n'a pas.

5. *Appels attendus, écrits avant de coder :*

```
assert puces_compatibles(puces, 300, 24) == ["A", "C"]   # cas normal
assert puces_compatibles(puces, 300, 200) == []           # cas limite : aucune puce
assert puces_compatibles(puces, 300, 150) == ["A"]        # cas piege : 150 h tout juste
                                                          # suffit
```

Par exemple :

```
def autonomie(capacite_mAh, conso_mA):
    return capacite_mAh / conso_mA

def puces_compatibles(puces, capacite_mAh, heures_min):
    return [puce["nom"] for puce in puces
            if autonomie(capacite_mAh, puce["conso_mA"]) >= heures_min]
```

Autre méthode : une boucle qui ajoute le nom de chaque puce assez autonome.

```
def puces_compatibles(puces, capacite_mAh, heures_min):
    resultat = []                          # INITIALISER
    for puce in puces:                     # PARCOURIR
        if autonomie(capacite_mAh, puce["conso_mA"]) >= heures_min:
            resultat.append(puce["nom"])   # METTRE A JOUR
    return resultat                        # RENVOYER
```

`puces_compatibles(puces, 300, 72)` renvoie `["A"]` (seule la puce A dépasse 72 h ; la puce C n'atteint que 37,5 h).

6. Puce A : $2 \times 24 = 48$ mAh ; puce B : $20 \times 2 = 40$ mAh ; total **88 mAh par jour**. Autonomie : $300 \div 88 \approx$ **3,4 jours** (environ 3 jours complets).

7. C'est une **contrainte de temps réel** : la mesure doit être faite à intervalle régulier, sans retard. Le microcontrôleur, **sans système d'exploitation**, exécute directement son programme de façon **prévisible**, alors que l'OS de la puce B partage le processeur entre de nombreux processus et pourrait retarder la lecture. De plus, la puce A consomme 10 fois moins, ce qui compte pour une tâche permanente.

★★★ **Exercice 11** *Type bac — Une station météo au lycée* (sujet maison, façon bac) ↑énoncé p. 683

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 0,75 ; Q3 : 0,5 ; Q4 : 0,75 ; Q5 : 1 ; Q6 : 1 ; Q7 : 0,75 ; Q8 : 0,75

1. **Capteurs** : température, humidité, vitesse du vent (anémomètre). **Actionneur** : la LED rouge d'alerte.

2. Le **programme** est dans la mémoire **Flash** (mémoire morte) ; les **mesures** (données) sont dans la mémoire **vive** (RAM). La mémoire Flash est **non volatile** : elle conserve son contenu sans alimentation, contrairement à la RAM.

3. Sur 10 bits : $2^{10} =$ **1 024** valeurs, de **0** à **1 023**.

4. Par exemple :

```
def convertir(valeur):
    return valeur * 0.1 - 40
```

`convertir(512)` renvoie $51{,}2 - 40 =$ **11,2** °C (Python affiche une valeur approchée, comme `11.200000000000003`, à cause du codage des flottants). Plus haute température mesurable : $1\,023 \times 0{,}1 - 40 =$ **62,3** °C.

5. *Appels attendus, écrits avant de coder :*

```
assert alerte_gel([412, 398, 420]) == True   # cas normal
assert alerte_gel([400]) == False            # cas limite : 0 degre, pas strictement
                                             # negatif
assert alerte_gel([420, 410, 390]) == True   # cas piege : le gel n'arrive qu'a la fin
```

Par exemple :

```
def alerte_gel(mesures):
    for valeur in mesures:          # PARCOURIR
        if convertir(valeur) < 0:   # TESTER
            return True             # RENVOYER des qu'il gele
    return False                    # RENVOYER : aucune mesure negative
```

Les températures sont 1,2 ; 0,5 ; −0,2 ; 0,1 ; 2,0 °C : la valeur 398 donne −0,2 °C, donc la fonction renvoie `True`.

6. (a) `valeur = lire_capteur_temperature()` ; (b) `convertir(valeur) < 0` ; (c) `eteindre_led()`.

7. 6 relevés par heure × 24 h × 3 capteurs = **432** mesures par jour, soit 432 × 2 = **864** octets. 2 048 ÷ 864 ≈ 2,37 : la RAM peut conserver au plus 2 **jours complets** de mesures. Or la liste `mesures` grossit **sans fin** (`while True`) : au bout de quelques jours, la mémoire est saturée. Il faut régulièrement **transmettre** les mesures (par exemple au serveur du club) puis les effacer, ou ne garder que les plus récentes.

8. Avec deux mémoires et **deux bus**, le processeur lit **une instruction et une donnée en même temps**, ce qui est plus rapide. Pas besoin d'OS : le microcontrôleur exécute **une seule tâche**, toujours la même, directement dès la mise sous tension ; un OS n'apporterait rien et ne tiendrait d'ailleurs pas dans 2 Ko de RAM.

★★ Exercice 12 IA IA IA *Type bac — Une salle informatique à renouveler* (sujet maison, façon bac)

↑ énoncé p. 684

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 0,75 ; Q3 : 1 ; Q4 : 1,25 ; Q5 : 0,75 ; Q6 : 0,75 ; Q7 : 1

1. Sont intégrés sur la puce : le **CPU**, les **mémoires caches**, le **GPU** et le **contrôleur Wi-Fi**. L'alimentation électrique n'est pas sur la puce, et il n'y a pas de ventilateur.

2. Les signaux parcourent de **très courtes distances**, sans longs fils de cuivre : il faut moins d'énergie, donc il se dégage **moins de chaleur**, et un ventilateur devient inutile. Autre avantage : la machine est **silencieuse** (et beaucoup plus petite).

3. Solution 1 : 150 × 30 × 8 × 160 = 5 760 000 Wh = **5 760 kWh**. Solution 2 : 15 × 30 × 8 × 160 = 576 000 Wh = **576 kWh**. Économie : 5 760 − 576 = **5 184 kWh**, soit 5 184 × 0,20 = **1 036,80 € par an**.

4. *Appels attendus, écrits avant de coder :*

```
assert energie_salle(["mini", "tour", "mini"], puissances, 10) == 1.8   # cas normal
assert energie_salle([], puissances, 10) == 0                           # cas limite :
                                                                        # aucun poste
assert energie_salle(["tour"], puissances, 1) == 0.15                   # cas piege : en
                                                                        # kWh, pas en Wh
```

Par exemple :

```
def energie_salle(postes, puissances, heures):
    total = 0                                       # INITIALISER
    for poste in postes:                            # PARCOURIR
        total = total + puissances[poste] * heures  # METTRE A JOUR : en Wh
    return total / 1000                             # RENVOYER : en kWh
```

`energie_salle(["mini", "tour", "mini"], puissances, 10)` : $(15 + 150 + 15) \times 10 =$ 1 800 Wh, la fonction renvoie `1.8` (kWh).

5. Solution 1 : on **ajoute ou remplace des barrettes** de mémoire sur la carte mère (composants séparés, **évolutifs**). Solution 2 : la mémoire vive est **soudée** contre la puce, **impossible d'en ajouter** : il faudra **remplacer les mini-PC** entiers.

6. **Non** : le SoC est **non réparable**, toute la puce est perdue (et en pratique tout le mini-PC). Cela alimente la **montagne de déchets électroniques** ; la société y répond par le **droit à la réparation** et l'**indice de réparabilité**.

7. *Exemple.* Solution 2 : consommation divisée par 10 (environ 1 000 € économisés par an), silence, faible encombrement, faible chaleur dans la salle. Solution 1 : machines **réparables** et **évolutives** (mémoire, carte graphique), donc durée de vie potentiellement plus longue et moins de déchets. Une recommandation raisonnable : la solution 2 pour un usage bureautique courant, en surveillant sa durée de vie ; la solution 1 si la salle sert à des usages exigeants appelés à évoluer. *(Une recommandation argumentée dans l'autre sens est aussi correcte.)*

★★★ **Exercice 13** IA IA IA *Type bac — Au cœur d'un smartphone* (sujet maison, façon bac) ↑énoncé p. 685

Barème indicatif (sur 6) : Q1 : 0,5 ; Q2 : 0,75 ; Q3 : 0,5 ; Q4 : 0,5 ; Q5 : 0,5 ; Q6 : 0,75 ; Q7 : 1,25 ; Q8 : 0,5 ; Q9 : 0,75

Partie A.

1. C'est un **ordinateur complet et puissant** sur une seule puce : processeur multi-cœur à plusieurs GHz, 8 **Go** de mémoire dans le boîtier (et non quelques Ko), GPU, processeurs spécialisés, modules de communication ; il fait tourner un **système d'exploitation** complet, ce qu'un microcontrôleur ne fait pas.

2. **Unité de calcul** : CPU, GPU, NPU, module cryptographique. **Mémoire** : RAM. **Entrées-sorties** : modem 5G, Wi-Fi, GPS. (Le bus relie le tout.)

3. *(a)* Musique écran éteint : tâche **légère** et **longue** → un cœur **économe** (il suffit largement et préserve la batterie). *(b)* Jeu 3D : tâche **lourde** → les cœurs **rapides** (avec le GPU pour l'affichage), quitte à consommer davantage.

4. C'est le **système d'exploitation** (son ordonnanceur). Avec 8 cœurs, jusqu'à 8 processus s'exécutent **réellement en même temps** (au lieu d'une simple alternance) : le téléphone reste réactif et fait plus de calculs par seconde.

5. Un processeur spécialisé réalise *sa* tâche (graphisme, calcul d'IA, chiffrement des données) **beaucoup plus vite** et en **consommant moins** qu'un CPU généraliste ; il libère en outre le CPU pour le reste.

Partie B.

6. De 2015 à 2025 : 10 ans, soit 5 doublements : $1 \times 2^5 =$ **32 milliards** prévus. Réalité : 16 milliards, soit **deux fois moins**. La tendance (croissance exponentielle) reste juste, mais la loi de Moore **ralentit** : c'est une observation empirique, et l'on approche des limites physiques de la gravure.

7. Par exemple :

```
1 def annee_prevue(transistors, annee, objectif):
2     while transistors < objectif:       # TESTER : objectif atteint ?
3         transistors = transistors * 2   # METTRE A JOUR : un doublement...
4         annee = annee + 2               # METTRE A JOUR : ... tous les deux ans
5     return annee                        # RENVOYER
```

8. Il faut passer de 1 à 16 milliards : $16 = 2^4$, soit 4 doublements, donc 8 ans : la fonction renvoie 2023. L'Azur X1 (2025) arrive avec 2 **ans de retard** sur la prévision.

9. $10^9 \div 2\,300 \approx 434\,783$. Or $2^{18} = 262\,144 < 434\,783 \leq 2^{19} = 524\,288$: il faut 19 doublements, soit 38 ans. La fonction renvoie $1971 + 38 = 2009$.

★ **Exercice 14** *L'informatique invisible* ↑énoncé p. 686

1. Par exemple : montre connectée, ampoule connectée, thermostat, enceinte vocale, caméra de surveillance, voiture connectée.

2. **Sécurité** : chaque objet connecté est une machine sur le réseau, donc une **porte d'entrée** potentielle pour un attaquant (souvent mal protégée). **Vie privée** : chaque objet est un **capteur** qui collecte des données sur nous (déplacements, habitudes, voix…), qui peuvent être exploitées.

★★ **Exercice 15** *Le système sur puce expliqué par un assistant d'IA* ↑énoncé p. 686

1. **Non.** 50 ans = 25 doublements ; 0,7 MHz $\times\, 2^{25} \approx 0{,}7 \times 33{,}5$ millions de MHz $\approx 2{,}3 \times 10^7$ MHz, soit environ 23 000 GHz (23 THz). C'est **plusieurs milliers de fois** la fréquence réelle d'un processeur actuel (2 à 5 GHz) : la loi telle qu'énoncée par l'assistant est fausse.

2. La phrase fautive est « *la fréquence d'horloge des processeurs double environ tous les deux ans* ». Le cours : « *le nombre de transistors sur une puce double environ tous les deux ans* ». C'est le **nombre de transistors** qui suit la loi de Moore ; la fréquence a certes augmenté grâce à la miniaturisation, mais beaucoup moins vite, et elle **plafonne** depuis les années 2000 (chaleur, consommation). La fin de la phrase (« *passer de* 2 300 *transistors… aux dizaines de milliards* ») parle d'ailleurs bien de transistors, ce qui trahit l'incohérence. Le reste de la réponse (composants d'un SoC, avantages, inconvénient, usages) est exact.

3. Faire un **calcul d'ordre de grandeur** sur toute affirmation chiffrée : une croissance exponentielle sur cinquante ans donne des nombres énormes, qu'il suffit de confronter à une valeur connue (quelques GHz). Relire aussi la phrase pour vérifier qu'elle est **cohérente avec elle-même** (fréquence annoncée, transistors comptés).

★★ **Exercice 16** *Expliquer en deux minutes* ↑énoncé p. 687

▶ Solution. Pas de texte à apprendre par cœur : voici les **éléments attendus** pour chaque sujet. L'ordre, les mots et l'exemple peuvent être différents ; l'explication est réussie si ces idées y sont, justes et reliées entre elles.

Sujet 1.

- Un **SoC** regroupe sur un seul circuit intégré processeur(s) et leurs caches, processeur graphique, contrôleurs d'entrées-sorties et modules de communication (Wi-Fi, Bluetooth, réseau mobile) ; la mémoire vive est empilée dans le même boîtier.
- Avantages : composants très proches donc échanges rapides, faible consommation (autonomie, souvent sans ventilateur), petite taille, coût réduit en grande série.
- Exemple : un smartphone doit être fin, léger et tenir une journée sur batterie : impossible avec des composants séparés.
- Limites : ni réparable ni évolutif (on ne change pas la mémoire), ce qui pose la question de l'obsolescence.

Sujet 2.

- **Microcontrôleur** : mini-ordinateur minuscule, sobre et bon marché, conçu pour **une** tâche précise dans un système embarqué ; quelques dizaines de MHz, quelques Ko de mémoire, en général **pas** de système d'exploitation complet.
- **SoC de smartphone** : plusieurs cœurs à plusieurs GHz, des Go de mémoire, un système d'exploitation complet ; c'est un ordinateur polyvalent.
- Exemple : un régulateur de vitesse répète sans fin « lire les capteurs → décider → commander les actionneurs ».
- Piège : « plus petit, donc moins bien » ; le microcontrôleur est choisi pour sa sobriété et pour le temps réel (un airbag ne peut pas attendre).

Sujet 3.

- Énoncé : le nombre de **transistors** sur une puce double environ tous les deux ans (Gordon Moore prévoit en 1965 un doublement chaque année, puis révise en 1975 : tous les deux ans) ; c'est une prédiction empirique, pas une loi physique.
- Chiffres : 2 300 transistors pour l'Intel 4004 (1971), des dizaines de milliards aujourd'hui.
- Piège : ce n'est **pas** la fréquence d'horloge qui double ; elle plafonne à quelques GHz.
- Limites : la taille des atomes (gravure à quelques nanomètres), la chaleur, le coût des usines ; on multiplie désormais les cœurs et les processeurs spécialisés.

Entraînement

★ **Exercice 17** *Systèmes embarqués en série* ↑énoncé p. 687

a) Capteur : capteur de luminosité ; actionneur : la lampe. Tours : 120 → `eteindre_lampe()` ; 60 → `eteindre_lampe()` ; 40 → `allumer_lampe()` ; 35 → `allumer_lampe()` ; 80 → `eteindre_lampe()`. La lampe est donc successivement éteinte, éteinte, allumée, allumée, éteinte (une commande est envoyée à chaque tour, même si l'état ne change pas).

b) Capteur : capteur d'humidité du sol ; actionneur : la pompe. Tours : 45 → aucune commande, pompe arrêtée ; 28 → `demarrer_pompe()`, en marche ; 35 → aucune commande, la pompe **reste en marche** ; 62 → `arreter_pompe()`, arrêtée ; 50 → aucune commande, arrêtée.

▷ **Erreur fréquente.** En b), considérer que la pompe s'arrête à 35 % parce que l'humidité dépasse 30 % : entre 30 et 60, aucun test n'est vrai, aucune commande n'est envoyée, et la pompe continue de tourner jusqu'à dépasser 60 %.

c) Capteur : capteur de température ; actionneur : le ventilateur. Tours : 45 → `ventiler(0)`, arrêt ; 55 → `ventiler(1)`, lent ; 72 → `ventiler(2)`, rapide ; 68 → `ventiler(1)`, lent ; 49 → `ventiler(0)`, arrêt.

★ **Exercice 18** *La loi de Moore en série* ↑énoncé p. 688

a) 10 ans = 5 doublements : $5\,000 \times 2^5 = 5\,000 \times 32 = \mathbf{160\,000}$ transistors.
b) 20 ans = 10 doublements : $10^6 \times 2^{10} = 1\,024 \times 10^6$, soit environ **un milliard** de transistors.
c) 6 ans = 3 doublements : $3 \times 2^3 = \mathbf{24\ milliards}$ de transistors.

Compléments

★★ **Exercice 19** *Lire le schéma d'un SoC de téléphone* ↑énoncé p. 688

1. **Calculer** : CPU, GPU, NPU. **Mémoriser** : le cache de 4 Mo (dans le bloc CPU). **Communiquer** : modem 5G, Wi-Fi, GPS (**radio**) ; USB, liaisons caméra et capteurs (**filaire**). **Faire fonctionner la puce** : gestion d'énergie, contrôleur d'affichage (il envoie l'image à l'écran), contrôleur mémoire (il organise les échanges avec la RAM). *(On accepte le contrôleur d'affichage parmi les entrées-sorties.)*

2. C'est un **réseau sur puce** ; les carrés sont des **routeurs**. Sur un bus unique, un seul échange a lieu à la fois : avec des milliards de transistors et des dizaines de blocs, ce bus serait un **goulot d'étranglement**. Le réseau achemine les données par paquets, comme les routeurs d'Internet (chapitre 9), et plusieurs échanges ont lieu **en même temps**.

3. Le cache est gravé **tout près** des cœurs, sur la puce : les données y arrivent beaucoup plus vite que de la RAM, qui est une autre puce (plus loin, plus lente d'accès). Le processeur y garde les données qu'il vient d'utiliser, qu'il a de bonnes chances de réutiliser.

4. $8 \times 2^{30} \times 8 = 2^{36} \approx 6{,}9 \times 10^{10}$ bits, donc au moins **69 milliards** de transistors : plus de **quatre fois** toute la Corail C2. Graver la RAM sur la puce la rendrait énorme et hors de prix : on la fabrique à part et on l'empile dans le même boîtier, au plus près.

5. Par exemple le **GPU** et le **contrôleur d'affichage** (écran éteint), le **NPU**, la caméra ; le CPU peut aussi ralentir. Intérêt : la batterie dure plus longtemps (**autonomie**), et le téléphone chauffe moins.

6. **Vitesse** : les blocs sont à quelques millimètres les uns des autres, les signaux arrivent plus tôt et les échanges sont rapides. **Consommation** : de courtes liaisons demandent moins d'énergie que les pistes d'une carte mère et ses connecteurs, et chaque bloc inutile peut être éteint. **Inconvénient** : rien n'est remplaçable ni évolutif (on ne peut pas ajouter de mémoire vive, ni réparer un seul bloc) : il faut changer tout l'appareil.

★ Exercice 20 IA IA IA *Logiciel ou matériel : situer dans le temps* ↑énoncé p. 689

1. *(f)* Turing, machine universelle : **1936** ; *(b)* ENIAC recâblé : **1945** ; *(c)* programme enregistré à Manchester : **1948** ; *(e)* circuit intégré : **1958–1959** ; *(a)* Intel 4004 : **1971** ; *(d)* puce A13 : **2019**.

2. *(a)* **matériel** (on change les câblages) ; *(b)* **logiciel** ; *(c)* **matériel dédié** (le module cryptographique de la puce) ; *(d)* **logiciel** (même machine, autre programme).

3. Avec un programme **enregistré en mémoire**, une même machine fait n'importe quel calcul : il suffit de **charger un autre programme**, sans toucher au matériel ; changer de tâche devient rapide et bon marché, et c'est le logiciel qui porte les fonctions. Aujourd'hui, certaines tâches sont si fréquentes et si lourdes (graphisme, IA, chiffrement) qu'un circuit **spécialisé** les fait beaucoup **plus vite** et en **consommant moins** qu'un processeur généraliste : la loi de Moore offre assez de transistors pour les graver sur la puce.

4. On appelle une fonction prête à l'emploi, en lui passant un paramètre, sans savoir comment elle est écrite : c'est une **API**, comme pour une bibliothèque. Derrière, du **matériel** : l'écran de 25 LED, ses circuits de commande, et le processeur du micro:bit qui exécute le code.

CORRIGÉS DES TP

TP Programmer un système embarqué

↑ énoncé p. 692

Partie A — La boucle et son rythme

Exercice 1 *Combien de tours par seconde ?*

1. Le nombre de passages dans la boucle pendant une seconde, c'est-à-dire sa **fréquence** (de quelques milliers à quelques dizaines de milliers de tours par seconde ; valeur variable selon le simulateur et la carte).

2. Aucun capteur au sens du cours (seulement l'horloge interne `running_time()`), aucun actionneur ; il ne fait rien d'utile : il mesure seulement la vitesse de la boucle.

3. Non : le simulateur ne reproduit pas la vitesse réelle du processeur ; il émule la carte dans le navigateur, avec la puissance de l'ordinateur. Sur une vraie carte, la valeur dépendrait du processeur à 64 MHz et de l'interpréteur MicroPython.

Exercice 2 *Ce qui ralentit la boucle*

Ligne ajoutée	Tours par seconde (ordre de grandeur)
(rien)	des milliers, voire des dizaines de milliers
lecture de l'accéléromètre	moins (il faut interroger le capteur à chaque tour)
`sleep(20)`	un peu moins de 50
`display.scroll("Hi")`	1 affiché (chaque tour dure plus d'une seconde)
`display.scroll("Hi", wait=False)`	de nouveau beaucoup (le défilement ne bloque plus)

1. Chaque tour dure au moins 20 ms, donc au plus $1000/20 = 50$ tours par seconde ; on mesure un peu moins, car le reste du tour prend aussi du temps.

2. Non : pendant le défilement, le programme est **bloqué** dans `scroll` et ne lit plus aucun capteur. Un airbag réagirait avec plus d'une seconde de retard ; un podomètre raterait les pas faits pendant l'affichage.

3. Avec `wait=False`, le défilement se fait « en arrière-plan » et la boucle continue aussitôt (relancé à chaque tour, le texte n'a d'ailleurs pas le temps de défiler : on ne lance un affichage qu'en cas de besoin). Règle : **dans la boucle d'un système temps réel, aucune instruction ne doit bloquer longtemps** ; la durée d'un tour doit rester petite devant le temps de réaction exigé.

Partie B — Trois systèmes embarqués

Exercice 3 *Une veilleuse qui s'allume avec la nuit*

1. $(255 - 128) \times 25 \,//\, 255 = 3175 \,//\, 255 = 12$ LED ; $(255 - 200) \times 25 \,//\, 255 = 1375 \,//\, 255 = 5$ LED.

2.

```
from microbit import *

while True:
    lumiere = display.read_light_level()
    nb = (255 - lumiere) * 25 // 255
    display.clear()
    for k in range(nb):
        display.set_pixel(k % 5, k // 5, 9)
    sleep(200)
```

3. Capteur : le capteur de lumière (les LED elles-mêmes) ; traitement : le calcul de `nb` ; actionneur : les LED de l'écran. La lumière ambiante varie lentement : la regarder 5 fois par seconde suffit largement. Un frein ABS doit réagir en quelques millisecondes : 200 ms de pause serait beaucoup trop.

Exercice 4 *Une alarme de température qui ne panique pas*

1.

Température	26	29	30	31	29	30	29	28	27
Alarme	non	non	**oui**	oui	oui	oui	oui	non	non

À 29 (entre 28 et 30), l'alarme garde son état : éteinte la première fois, allumée ensuite.

2.

```
from microbit import *
import music

en_alarme = False
son_coupe = False
mini = temperature()
maxi = mini
while True:
    t = temperature()
    if t < mini:
        mini = t
    if t > maxi:
        maxi = t
    if t >= 30:
        en_alarme = True
    elif t <= 28:
        en_alarme = False
        son_coupe = False            # le son reviendra a la prochaine alarme
    if button_a.was_pressed() and en_alarme:
        son_coupe = True
    if button_b.was_pressed():
        display.scroll(str(mini) + "/" + str(maxi))
    if en_alarme:
        display.show(Image.NO)
        if not son_coupe:
            music.pitch(880, 100)
    else:
        display.clear()
    sleep(200)
```

À noter : sans le test `and en_alarme`, un appui sur A hors alarme couperait par avance le son de la prochaine alarme ; `display.scroll` bloque la boucle plusieurs secondes (acceptable ici : la température varie lentement).

3. Une liste qui grandit à chaque tour (5 mesures par seconde, soit 432 000 par jour) saturerait vite les 128 Ko de mémoire vive, et le programme planterait au bout de quelques heures. Deux variables `mini` et `maxi` suffisent et occupent une place constante, quelle que soit la durée de fonctionnement.

Exercice 5 — *Un podomètre*

1. $\sqrt{0+0+1024^2} = 1024$; $\sqrt{300^2+400^2+1000^2} = \sqrt{1\,250\,000} \approx 1118$.

2.

```
from microbit import *
import math

pas = 0
en_haut = False
while True:
    x, y, z = accelerometer.get_values()
    a = math.sqrt(x * x + y * y + z * z)
    if not en_haut and a > 1250:
        en_haut = True
        pas = pas + 1
    elif en_haut and a < 1050:
        en_haut = False
    if button_a.was_pressed():
        display.scroll(str(pas), wait=False)
    if button_b.was_pressed():
        pas = 0
        display.clear()
    sleep(20)                          # environ 50 mesures par seconde
```

3. Dans le simulateur, chaque passage du curseur **z** au-delà de 1250 (en valeur absolue) puis retour vers 1000 compte un pas ; un aller-retour qui reste entre 1050 et 1250 n'en compte pas.

Partie C — Tester la logique sans la carte

Exercice 6 — *Les fonctions de traitement*

```
def veilleuse(lumiere, allumee, seuil_bas=40, seuil_haut=80):
    if lumiere < seuil_bas:
        return True
    if lumiere > seuil_haut:
        return False
    return allumee

def nb_basculements(etats):
    c = 0
    for i in range(1, len(etats)):
        if etats[i] != etats[i - 1]:
            c += 1
    return c

def nb_led(lumiere):
    return (255 - lumiere) * 25 // 255

def alarme(temp, en_alarme, seuil=30, retour=28):
    if temp >= seuil:
        return True
    if temp <= retour:
        return False
    return en_alarme

def mise_a_jour(mini, maxi, temp):
    if temp < mini:
        mini = temp
    if temp > maxi:
        maxi = temp
    return (mini, maxi)

```

```python
def norme(x, y, z):
    return math.sqrt(x * x + y * y + z * z)

def compter_pas_naif(normes, seuil=1200):
    pas = 0
    for i in range(1, len(normes)):
        if normes[i - 1] <= seuil < normes[i]:
            pas += 1
    return pas

def compter_pas(normes, haut=1250, bas=1050):
    pas = 0
    en_haut = False
    for a in normes:
        if not en_haut and a > haut:
            en_haut = True
            pas += 1
        elif en_haut and a < bas:
            en_haut = False
    return pas
```

Exercice 7 *Pourquoi deux seuils ? Les mesures parlent*

1. Veilleuse naïve : **15** basculements ; à hystérésis : **1** seul. Au crépuscule, la lumière mesurée hésite autour de 60 (entre 45 et 75 avec le bruit) : avec un seul seuil, chaque petite variation fait changer d'état. Avec deux seuils écartés (40 et 80), il faut une vraie variation pour basculer.

2. Naïf : **26** pas ; à hystérésis : **20** pas. La marche dure 10 s à 2 pas par seconde : il y a 20 pas, c'est `compter_pas` qui a raison. (Avec les graines 1 à 10, le naïf trouve entre 21 et 28 pas, l'autre toujours 20.)

3. On attend 10 pas ; le naïf en compte **24**, `compter_pas` exactement **10**. Quand le signal monte lentement, il passe longtemps près du seuil, et le **bruit** le fait franchir le seuil plusieurs fois au cours d'un même pas : chaque franchissement est compté.

4.

```python
repos = [random.randint(940, 1060) for i in range(500)]
print(compter_pas_naif(repos), compter_pas(repos))      # 0 0
```

Partie D — Consommation : combien de temps sur deux piles ?

Exercice 8 *Faire durer les piles*

1. $1\,000/30 \approx 33$ h, soit environ **1,4 jour**.

2. $I = 0{,}3 + 0{,}0099 = 0{,}3099$ mA ; autonomie $1\,000/0{,}3099 \approx 3\,227$ h, soit environ **134 jours**. On a gagné un facteur de l'ordre de **100** : un objet embarqué passe l'essentiel de son temps à *dormir*.

3. $4\,000/24 \approx 167$ mA, environ 5,5 fois le courant de la carte active (et environ 500 fois celui de la veilleuse économe).

4. Sur l'ordinateur de préparation (Python 3), la boucle vide fait environ **8 millions** de tours par seconde, bien plus que la carte. Écart : processeur à plusieurs GHz contre 64 MHz, architecture plus performante (caches, plusieurs instructions par cycle). En échange : une consommation des milliers de fois plus grande, un prix et un encombrement bien supérieurs, un ventilateur…

Partie E — Analyse : la puce du micro:bit face à un PC

Exercice 9 *Fiche d'analyse*

1.

	nRF52833 (micro:bit)	PC de bureau	Rapport PC / micro:bit
Fréquence	64 MHz	3,5 GHz = 3 500 MHz	≈ 55
Nombre de cœurs	1	8	8
Mémoire vive	128 Ko	16 Go	131 072
Puissance électrique	≈ 0,1 W	≈ 100 W	≈ 1 000
Système d'exploitation	aucun	Windows, Linux, macOS	—
Composants	une seule puce	pièces séparées	—

Mémoire : 16 Go = 16 × 1024 × 1024 Ko = 16 777 216 Ko, et 16 777 216/128 = 131 072. Composants du PC : carte mère, processeur, barrettes de mémoire, disque, carte graphique…

2. Processeur : le cœur Cortex-M4 ; mémoire : la Flash (programme) et la mémoire vive (données) ; entrées-sorties : la radio Bluetooth, le convertisseur analogique-numérique (capteurs), les minuteries. Tout est gravé sur **un seul circuit intégré** : c'est un système sur puce. Il s'agit d'un **microcontrôleur** : fréquence de quelques dizaines de MHz, mémoire de quelques centaines de Ko (128 Ko de mémoire vive et 512 Ko de Flash ici), aucun système d'exploitation, très faible consommation, une tâche embarquée.

3. L'**architecture de Harvard** (programme et données dans deux mémoires, deux bus) : le processeur peut lire une instruction et une donnée **en même temps**.

4. L'alarme n'a qu'**une** tâche : un OS (ordonnanceur, gestion de plusieurs processus) consommerait mémoire et énergie, et rendrait le temps de réaction moins prévisible. MicroPython est **interprété** : chaque instruction est analysée à l'exécution, ce qui est bien plus lent qu'un programme compilé (en C par exemple) et occupe de la mémoire pour l'interpréteur ; les produits industriels sont programmés en C ou en assembleur.

5. Sur le micro:bit, la radio est *dans* la puce : il faut changer toute la puce, en pratique toute la carte. Sur un PC, on change seulement la carte Wi-Fi. Intérêt de l'intégration : taille, consommation, coût ; limite : **non réparable**, non évolutif.

Bilan — éléments attendus

- Podomètre : **acquisition** (accéléromètre, 50 mesures par seconde), **contrôle** (compter, afficher à l'écran), **temps** (la boucle doit tourner assez vite pour ne pas rater un pas : pas d'instruction bloquante).
- Hystérésis : deux seuils écartés pour ignorer le bruit autour d'un seuil unique (15 basculements contre 1 ; 26 pas comptés au lieu de 20, 24 au lieu de 10).
- Une veilleuse ou un podomètre n'a besoin ni de GHz, ni de Go : la puce consomme environ 1 000 fois moins qu'un PC (des mois sur deux piles en mode veille), coûte quelques euros et tient dans un objet de quelques centimètres.

Réponses des questions de la fiche de révision

↑ questions p. 699

Se tester

1. Un ordinateur complet (processeur et caches, GPU, périphériques) intégré sur un seul circuit, la mémoire vive étant dans le même boîtier (§ III).
2. Capteur : la sonde de température (entrée) ; actionneur : le chauffage (sortie) (§ IV.A).
3. 3 doublements, donc 8 milliards (§ I).
4. Non : elle est soudée dans le boîtier de la puce (non évolutif) ; il faut changer l'appareil (§ V).
5. Microcontrôleur : sans OS, il répond à temps, de façon prévisible (temps réel) (§ IV).

Et avant ?

1. aucun programme ne peut décider, pour tout programme et toute entrée, si ce programme s'arrête (ch. 13, § III).
2. $R(0) = 0$ et $R(s) = 1 + \min R(s - p)$ sur les pièces $p \leq s$ (ch. 11, § V).
3. `{'A': ['B', 'C'], 'B': ['A', 'C'], 'C': ['A', 'B']}` (ch. 8, § III.B).

Problèmes de synthèse et exercices type bac

> **Remarque**
> On donne une version de chaque fonction ; d'autres écritures correctes sont possibles. Le fichier complet, `corrige_synthese_terminale.py`, est en ligne : `https://github.com/MathThibaud/nsi-fichiers-eleves/tree/main/corriges/terminale/synthese`.

Après les chapitres 1 à 3 (récursivité, programmation objet, structures linéaires)

★★ **Exercice 1**  *Dérécursiver avec une pile* ↑énoncé p. 706

1. Appels successifs : `afficher_binaire(13)` → `(6)` → `(3)` → `(1)`. L'appel `(1)` ne relance rien ($1 < 2$) et affiche `1` ; puis, en remontant, `(3)` affiche `1`, `(6)` affiche `0` et `(13)` affiche `1`. Affichage : `1101`, l'écriture **binaire** de 13 ($8 + 4 + 1$).

2. Au moment de l'appel le plus profond, la pile contient, du sommet vers le bas : `afficher_binaire(1)`, `(3)`, `(6)`, `(13)`. Chaque appel affiche son chiffre **après** le retour de l'appel qu'il a lancé : les affichages se font donc dans l'ordre où les appels quittent la pile, c'est-à-dire du dernier lancé au premier (LIFO). Or le dernier appel porte le chiffre de **poids fort** : on obtient les chiffres dans le bon ordre, alors que les divisions par 2 les calculent du poids faible au poids fort.

3.

```
def binaire(n):
    p = Pile()
    p.empiler(n % 2)
    n = n // 2
    while n > 0:
        p.empiler(n % 2)
        n = n // 2
    resultat = ""
    while not p.est_vide():
        resultat = resultat + str(p.depiler())
    return resultat
```

Le premier reste est empilé avant la boucle : ainsi `binaire(0)` renvoie bien `"0"`. La pile remplace la pile d'appels : elle garde les chiffres « en attente » et les rend dans l'ordre inverse.

4. Avec une file, les chiffres ressortent dans l'ordre où ils ont été calculés (FIFO), du poids faible au poids fort : on obtient `"1011"`, l'écriture binaire **à l'envers**. C'est précisément le caractère LIFO de la pile qui remet les chiffres dans l'ordre.

★★ **Exercice 2** *Annuler et rétablir* ↑énoncé p. 707

1.

```
    def __init__(self):
        self.texte = ""
        self.annulations = Pile()
        self.retablissements = Pile()
```

2. Les piles sont dessinées verticalement, **sommet en haut**.

Instruction	e.texte	annulations	retablissements
Editeur()	""	vide	vide
ecrire("Bon")	"Bon"	""	vide
ecrire("jour")	"Bonjour"	"Bon" ""	vide
annuler()	"Bon"	""	"Bonjour"
retablir()	"Bonjour"	"Bon" ""	vide
annuler()	"Bon"	""	"Bonjour"
ecrire("soir")	"Bonsoir"	"Bon" ""	vide

3.

```
    def ecrire(self, mot):
        self.annulations.empiler(self.texte)
        self.texte = self.texte + mot
        self.retablissements = Pile()

    def annuler(self):
        if not self.annulations.est_vide():
            self.retablissements.empiler(self.texte)
            self.texte = self.annulations.depiler()

    def retablir(self):
        if not self.retablissements.est_vide():
            self.annulations.empiler(self.texte)
            self.texte = self.retablissements.depiler()
```

4. Après `ecrire("soir")`, le texte annulé (`"Bonjour"`) appartient à une « histoire » abandonnée : le rétablir effacerait sans prévenir ce qu'on vient d'écrire. Tous les éditeurs se comportent ainsi. C'est la ligne `self.retablissements = Pile()` de `ecrire` qui vide la pile des rétablissements à chaque nouvelle saisie.

★★★ **Exercice 3** *Listes chaînées récursives* ↑énoncé p. 707

1.

2. On suit la définition récursive : liste vide, ou une cellule suivie d'une liste.

```
def longueur(lst):
    if lst is None:
        return 0
    return 1 + longueur(lst.suivante)

def somme(lst):
    if lst is None:
        return 0
    return lst.valeur + somme(lst.suivante)
```

3.

```python
def contient(lst, x):
    if lst is None:
        return False
    if lst.valeur == x:
        return True
    return contient(lst.suivante, x)
```

Dès que `x` est trouvé, la fonction renvoie `True` sans faire d'appel récursif : inutile de regarder la suite.

4.

```python
def renverser_dans(lst, acc):
    if lst is None:
        return acc
    return renverser_dans(lst.suivante, Cellule(lst.valeur, acc))

def renverser(lst):
    return renverser_dans(lst, None)
```

Pour `lst` $= 3 \to 1 \to 4$: `acc` vaut successivement `None`, puis 3, puis $1 \to 3$, puis $4 \to 1 \to 3$. On crée de nouvelles cellules : `lst` n'est pas modifiée.

Autre méthode : la même idée avec une boucle : on parcourt `lst` et on pose chaque valeur *en tête* du résultat, qui joue le rôle de `acc`.

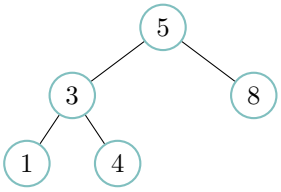
```python
def renverser_iteratif(lst):
    resultat = None
    while lst is not None:
        resultat = Cellule(lst.valeur, resultat)
        lst = lst.suivante
    return resultat
```

5. Pour ajouter en queue, il faut d'abord atteindre la dernière cellule en suivant les n liens : environ n opérations (coût **linéaire**), contre une seule en tête. C'est pourquoi une liste chaînée fait une excellente **pile** (on empile et dépile en tête, en une opération), mais une file médiocre si l'on ne garde pas aussi un accès direct à la dernière cellule.

Après le chapitre 5 (arbres)

★★★ **Exercice 4** *Trier avec un arbre binaire de recherche* ↑énoncé p. 708

1. Racine 5 ; à gauche 3 (avec 1 à sa gauche et 4 à sa droite) ; à droite 8.



2.

```python
def infixe_dans(a, resultat):
    if a is not None:
        infixe_dans(a.gauche, resultat)
        resultat.append(a.valeur)
        infixe_dans(a.droite, resultat)
```

Pour l'arbre de la question 1, `resultat` contient `[1, 3, 4, 5, 8]` : le parcours infixe d'un ABR donne les valeurs **triées**.

3.

```python
def tri_abr(tab):
    a = None
```

```
    for x in tab:
        a = inserer(a, x)
    resultat = []
    infixe_dans(a, resultat)
    return resultat
```

4. `tri_abr([3, 1, 3])` renvoie `[1, 3, 3]` : les doublons sont conservés. Le premier `3` est la racine, `1` va à sa gauche ; le second `3` n'est pas plus petit que la racine, il va donc à **droite** (convention du cours : égal à droite) et devient le fils droit de la racine. Le parcours infixe visite la racine, puis son sous-arbre droit : les deux `3` sortent côte à côte. Pour un tri **sans doublon**, il suffit de ne rien faire quand `x == a.valeur` : on remplace `else:` par `elif x > a.valeur:`, et `tri_abr([3, 1, 3])` renvoie alors `[1, 3]`. C'est une autre convention, que l'on rencontre dans certains sujets (ABR « sans doublon ») : toujours lire celle de l'énoncé.

5. Chaque valeur est plus grande que toutes les précédentes : elle va toujours à droite. L'arbre est un « peigne » de hauteur 100, en réalité une liste chaînée. L'insertion du dernier élément compare 100 à 99 valeurs. Au total, $0 + 1 + \cdots + 99 = 4\,950$ comparaisons, soit environ $n^2/2$: coût **quadratique**, comme un tri par insertion. Pour un tableau mélangé, chaque insertion coûte au plus une douzaine de comparaisons ici, de l'ordre de $\log_2(n)$: environ $n \log_2(n)$ au total. On retrouvera ce coût au chapitre 7, avec le tri fusion.

6. `inserer` s'appelle récursivement une fois par niveau traversé : avec 10 000 valeurs triées, l'arbre a une hauteur de 10 000, donc il faut 10 000 appels emboîtés. Cela dépasse la limite de la pile d'appels de Python (environ 1 000 par défaut) : `RecursionError`. La forme de l'arbre a donc un coût en temps *et* en mémoire.

★★ Exercice 5 *La largeur d'un arbre* ↑énoncé p. 709

1. Niveau 1 : 1 nœud (1) ; niveau 2 : 2 (2, 3) ; niveau 3 : 3 (4, 5, 6) ; niveau 4 : 3 (7, 8, 9). Largeur : **3**.

2.

```
def noeuds_par_niveau(a):
    compte = {}
    if a is None:
        return compte
    f = File()
    f.enfiler((a, 1))
    while not f.est_vide():
        n, niveau = f.defiler()
        if niveau in compte:
            compte[niveau] = compte[niveau] + 1
        else:
            compte[niveau] = 1
        if n.gauche is not None:
            f.enfiler((n.gauche, niveau + 1))
        if n.droite is not None:
            f.enfiler((n.droite, niveau + 1))
    return compte
```

3.

```
def largeur_max(a):
    compte = noeuds_par_niveau(a)
    maxi = 0
    for niveau in compte:
        if compte[niveau] > maxi:
            maxi = compte[niveau]
    return maxi
```

Pour l'arbre vide, le dictionnaire est vide et la fonction renvoie 0.

4. Le dictionnaire est le **même** : chaque nœud est visité une fois, avec son niveau, quelle que soit la structure utilisée. Seul l'**ordre de visite** change : avec une file, on visite niveau par niveau

(largeur) ; avec une pile, on s'enfonce d'abord dans une branche (profondeur). Ici, l'ordre ne compte pas, car on se contente de compter.

Après les chapitres 6 à 8 (bases de données, diviser pour régner, graphes)

★★ **Exercice 6** *Une médiathèque, de la table à l'objet* ↑énoncé p. 709

1. Un même livre peut être emprunté plusieurs fois : le livre `10` apparaît dans les emprunts `1` et `4`. Les valeurs de `id_livre` ne sont donc pas toutes distinctes, alors qu'une clé primaire doit identifier une seule ligne. Les clés étrangères sont `id_adherent`, qui fait référence à `adherent(id_adherent)`, et `id_livre`, qui fait référence à `livre(id_livre)`.

2.

```sql
SELECT livre.titre
FROM emprunt
JOIN adherent ON emprunt.id_adherent = adherent.id_adherent
JOIN livre ON emprunt.id_livre = livre.id_livre
WHERE adherent.nom = 'Durand' AND adherent.prenom = 'Lina';
```

Résultat : *Le Petit Prince* et *Les Misérables*. Deux adhérents s'appellent Durand : avec la seule condition `nom = 'Durand'`, on obtiendrait aussi le livre emprunté par Paul Durand (emprunt `4`). On peut aussi écrire `WHERE emprunt.id_adherent = 1`.

3. La requête compte les emprunts non rendus : `4` (emprunts `2`, `3`, `4` et `5`).

4.

```sql
SELECT livre.auteur, COUNT(*)
FROM emprunt
JOIN livre ON emprunt.id_livre = livre.id_livre
GROUP BY livre.auteur;
```

Résultat : Hugo 1, Saint-Exupéry 2, Verne 2.

5. On repère l'emprunt par sa clé primaire :

```sql
UPDATE emprunt
SET rendu = 1
WHERE id_emprunt = 2;
```

6.

```python
class Emprunt:
    def __init__(self, numero, titre, jour):
        self.numero = numero
        self.titre = titre
        self.jour = jour

    def en_retard(self, aujourdhui):
        return aujourdhui - self.jour > 21
```

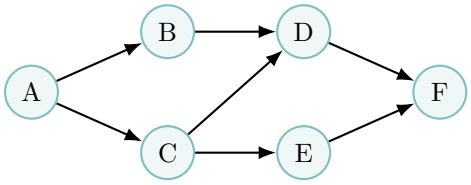
7.

```python
def titres_en_retard(lignes, aujourdhui):
    emprunts = [Emprunt(numero, titre, jour) for (numero, titre, jour) in lignes]
    return [e.titre for e in emprunts if e.en_retard(aujourdhui)]
```

Le jour 47, les prêts durent depuis 27, 22 et 7 jours : la fonction renvoie `["Les Misérables", "Le Petit Prince"]`. La base stocke les données, l'objet regroupe ces données et le comportement qui les concerne (la règle des 21 jours).

★★★ **Exercice 7** *Ordonnancer des tâches qui dépendent les unes des autres* ↑énoncé p. 710

1. Les arcs sont A→B, A→C, B→D, C→D, C→E, D→F et E→F.



2.

```
def successeurs(prerequis):
    succ = {}
    for t in prerequis:
        succ[t] = []
    for t in prerequis:
        for p in prerequis[t]:
            succ[p].append(t)
    return succ
```

3. Au départ, les nombres de prérequis restants sont A 0, B 1, C 1, D 2, E 1, F 2, et la file contient A.

Tâche exécutée	Effet sur les successeurs	File (tête à gauche)
A	B et C passent à 0 : enfilées	B C
B	D passe à 1	C
C	D et E passent à 0 : enfilées	D E
D	F passe à 1	E
E	F passe à 0 : enfilée	F
F	aucun successeur	vide

Ordre d'exécution : A, B, C, D, E, F.

4.

```
def ordre_execution(prerequis):
    succ = successeurs(prerequis)
    attente = {}
    for t in prerequis:
        attente[t] = len(prerequis[t])
    f = File()
    for t in prerequis:
        if attente[t] == 0:
            f.enfiler(t)
    ordre = []
    while not f.est_vide():
        t = f.defiler()
        ordre.append(t)
        for s in succ[t]:
            attente[s] = attente[s] - 1
            if attente[s] == 0:
                f.enfiler(s)
    if len(ordre) < len(prerequis):
        return None
    return ordre
```

5. La fonction renvoie `None` : plus aucune tâche n'est sans prérequis, la file est vide dès le départ et aucune tâche n'est exécutée. Le graphe contient désormais des **cycles**, par exemple A→B→D→F→A. C'est un **interblocage** : comme des processus qui attendent chacun une ressource détenue par un autre, chaque tâche du cycle attend la fin d'une autre tâche du cycle. Le contrôle `len(ordre) < len(prerequis)` le détecte : si un cycle existe, ses tâches ne sont jamais enfilées.

6. Une tâche est enfilée au plus une fois (au moment où son nombre de prérequis restants tombe à 0, ce qui n'arrive qu'une fois). Chaque dépendance est examinée une fois dans `successeurs`, puis une fois quand son prérequis est exécuté. Le coût est donc **linéaire** en (nombre de tâches + nombre de dépendances).

Après les chapitres 9 et 10 (réseaux, cryptographie)

★★ **Exercice 8** *Router des paquets avec un dictionnaire* ↑énoncé p. 711

1.

```
def cout(debit):
    return 10**8 / (debit * 10**6)
```

10 Mbit/s : coût 10 ; 50 Mbit/s : coût 2 ; 100 Mbit/s : coût 1.

2. C'est le parcours en largeur du chapitre *Graphes*, qui calcule les distances en nombre d'arêtes.

```
def sauts(reseau, depart):
    distance = {depart: 0}
    f = File()
    f.enfiler(depart)
    while not f.est_vide():
        r = f.defiler()
        for voisin in reseau[r]:
            if voisin not in distance:
                distance[voisin] = distance[r] + 1
                f.enfiler(voisin)
    return distance
```

Depuis R1 : R2 et R3 à 1 saut, R4 et R5 à 2 sauts. La route RIP de R1 vers R5 est R1–R2–R5 (2 sauts, la seule de cette longueur).

3.

```
def est_chemin(reseau, chemin):
    for i in range(len(chemin) - 1):
        if chemin[i + 1] not in reseau[chemin[i]]:
            return False
    return True

def cout_chemin(reseau, chemin):
    total = 0
    for i in range(len(chemin) - 1):
        total = total + cout(reseau[chemin[i]][chemin[i + 1]])
    return total
```

4.

```
def meilleur_chemin(reseau, chemins):
    meilleur = None
    for c in chemins:
        if est_chemin(reseau, c):
            if meilleur is None or cout_chemin(reseau, c) < cout_chemin(reseau, meilleur):
                meilleur = c
    return meilleur
```

Coûts : R1–R2–R5 : $10 + 1 = 11$; R1–R3–R4–R5 : $1 + 2 + 1 = 4$; R1–R2–R4–R5 : $10 + 2 + 1 = 13$; R1–R3–R4–R2–R5 : $1 + 2 + 2 + 1 = 6$; R1–R4–R5 n'est pas un chemin (R1 et R4 ne sont pas reliés). OSPF choisit **R1–R3–R4–R5** (coût 4). RIP choisit R1–R2–R5, plus court en sauts, mais qui emprunte la liaison lente R1–R2 (10 Mbit/s) : RIP ignore les débits, OSPF en tient compte.

5. La liaison figure dans les dictionnaires des deux routeurs :

```
del reseau["R3"]["R4"]
del reseau["R4"]["R3"]
```

La route RIP ne change pas (R1–R2–R5, 2 sauts). Pour OSPF, les candidats qui passent par R3–R4 ne sont plus des chemins ; il reste R1–R2–R5 (coût 11) et R1–R2–R4–R5 (coût 13) : le paquet passe désormais par **R1–R2–R5**, comme avec RIP.

★★ **Exercice 9** *RSA miniature et exponentiation rapide* ↑énoncé p. 712

1. $n = 11 \times 17 = 187$ et $\varphi = 10 \times 16 = 160$. $e \times d = 7 \times 23 = 161 = 160 + 1$, donc $e \times d \bmod \varphi = 1$.

2. $42^2 = 1\,764 = 9 \times 187 + 81$, donc $42^2 \bmod 187 = 81$. Puis $81 \times 42 = 3\,402 = 18 \times 187 + 36$: $42^3 \bmod 187 = 36$. Puis $36^2 = 1\,296 = 6 \times 187 + 174$: $42^6 \bmod 187 = 174$. Enfin $174 \times 42 = 7\,308 = 39 \times 187 + 15$: le chiffré est $c = \mathbf{15}$.

3. Il faut $k - 1$ multiplications ($a \times a \times \cdots \times a$). Avec un exposant d'environ 600 chiffres, même à un milliard de multiplications par seconde, le calcul durerait des milliards de fois l'âge de l'Univers : c'est impossible.

4.

```
def expo_mod(a, k, n):
    if k == 0:
        return 1
    moitie = expo_mod(a, k // 2, n)
    carre = (moitie * moitie) % n
    if k % 2 == 0:
        return carre
    return (carre * a) % n
```

Un seul appel récursif, sur `k // 2` : **diviser** l'exposant par deux, **régner** par l'appel récursif, **combiner** en élevant au carré (et en multipliant par a si k est impair). Le variant `k` est un entier positif qui diminue strictement, jusqu'à la condition d'arrêt $k = 0$.

5. Les appels portent sur $k = 23, 11, 5, 2, 1$, puis 0. Chaque appel avec k impair fait 2 multiplications (un carré et un produit par a), avec k pair une seule : $2 + 2 + 2 + 1 + 2 = \mathbf{9}$ multiplications, contre 22 pour la méthode naïve. Le nombre d'appels est de l'ordre de $\log_2 k$, donc il faut au plus environ $2 \log_2 k$ multiplications : pour un exposant de 2 048 bits, quelques milliers de multiplications suffisent.

6. `expo_mod(15, 23, 187)` renvoie `42` : on retrouve bien le message. Le message `[72, 105]` se chiffre en `[expo_mod(m, 7, 187) for m in [72, 105]]`, soit `[30, 96]`.

Après les chapitres 11 à 14 (programmation dynamique, recherche textuelle, calculabilité, systèmes sur puce)

★★★ **Exercice 10** *Détecter un plagiat : la plus longue sous-chaîne commune* ↑énoncé p. 712

1. `"RITHME"` (6 caractères), à la fin des deux mots.

2.

```
def plus_longue_naive(a, b):
    meilleure = ""
    for i in range(len(a)):
        for j in range(i + 1, len(a) + 1):
            if j - i > len(meilleure) and recherche_naive(a[i:j], b) != []:
                meilleure = a[i:j]
    return meilleure
```

Pour un texte `a` de n caractères, il y a $n + (n-1) + \cdots + 1 = \dfrac{n(n+1)}{2}$ couples (i, j), soit environ $n^2/2$ sous-chaînes. Chacune peut déclencher une recherche naïve, qui examine jusqu'à n positions de départ (et compare jusqu'à $j - i$ caractères à chacune) : le coût total est **au moins** de l'ordre de n^3. Inutilisable sur deux textes de quelques milliers de caractères.

3. Si `a[i-1] == b[j-1]`, une sous-chaîne commune qui se termine par ces deux caractères est une sous-chaîne commune qui se termine par `a[i-2]` et `b[j-2]` (éventuellement vide), prolongée par ce caractère : la plus longue mesure donc $L[i-1][j-1]+1$. Si les deux caractères sont différents, aucune sous-chaîne commune ne peut se terminer à la fois par l'un et par l'autre : $L[i][j] = 0$.

4. Lignes : `a = "ABAB"` ; colonnes : `b = "BABA"`.

		B	A	B	A
	0	0	0	0	0
A	0	0	1	0	1
B	0	1	0	2	0
A	0	0	2	0	**3**
B	0	1	0	**3**	0

La longueur cherchée est le **maximum** de la table, 3. La case $L[3][4] = 3$ correspond à la sous-chaîne qui se termine par `a[2]` : `a[0:3] = "ABA"` ; la case $L[4][3]$ donne `"BAB"`.

5. On retient la longueur maximale et la position `fin` où elle se termine dans `a`.

```
def plus_longue_dyn(a, b):
    n, m = len(a), len(b)
    L = [[0 for j in range(m + 1)] for i in range(n + 1)]
    lmax, fin = 0, 0
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                L[i][j] = L[i - 1][j - 1] + 1
                if L[i][j] > lmax:
                    lmax, fin = L[i][j], i
    return a[fin - lmax:fin]
```

`plus_longue_dyn("ALGORITHME", "LOGARITHME")` renvoie `"RITHME"`. Chaque case se calcule en temps constant à partir de sa voisine en haut à gauche : coût de l'ordre de $n \times m$, au lieu d'au moins n^3 pour la méthode naïve.

★★ Exercice 11 *Une machine minuscule* ↑énoncé p. 713

1. Avec `x = 2` :

Instruction exécutée	`position` après	`x` après
`("saute_si_nul", 3)` ($x \neq 0$)	1	2
`("ajoute", -1)`	2	1
`("saute", 0)`	0	1
`("saute_si_nul", 3)` ($x \neq 0$)	1	1
`("ajoute", -1)`	2	0
`("saute", 0)`	0	0
`("saute_si_nul", 3)` ($x = 0$)	3	0

La machine atteint `stop` avec `x = 0` après 7 instructions. Chaque tour de boucle exécute 3 instructions et diminue `x` de 1 : pour une valeur de départ `x` positive, il y a $3x + 1$ instructions avant `stop`.

2. Avec `x = 4`, `x` prend les valeurs 4, 2, 0 : la machine s'arrête avec `x = 0`. Avec `x = 3`, `x` prend les valeurs 3, 1, -1, $-3\ldots$: il reste impair, ne vaut jamais 0, et la machine ne s'arrête jamais.

3.

```
def executer(programme, x, max_pas):
    position = 0
    pas = 0
    while programme[position][0] != "stop" and pas < max_pas:
        instr = programme[position]
        if instr[0] == "ajoute":
            x = x + instr[1]
            position = position + 1
        elif instr[0] == "saute_si_nul":
            if x == 0:
```

```
                position = instr[1]
            else:
                position = position + 1
        elif instr[0] == "saute":
            position = instr[1]
        pas = pas + 1
    if programme[position][0] == "stop":
        return x
    return None
```

4. Avec `x = 50`, `prog_a` a besoin de $3 \times 50 + 1 = 151$ instructions, plus que `max_pas = 100` : `executer` abandonne trop tôt. Quelle que soit la valeur choisie pour `max_pas`, un programme qui s'arrête un peu plus tard sera pris à tort pour un programme qui boucle : `executer` ne sait pas distinguer « s'arrêtera plus tard » de « ne s'arrêtera jamais ». Et aucun autre programme ne peut le faire pour *tous* les programmes : c'est le **problème de l'arrêt**, indécidable. Enfin, `executer` reçoit le programme comme une simple **liste**, qu'elle lit et exécute : un programme est une donnée pour un autre programme, exactement comme Python lit et exécute nos fichiers `.py`.

5. Par exemple `[("ajoute", 1), ("saute", 0)]` (ou même `[("saute", 0)]`) : aucune instruction `stop` n'est jamais atteinte. Un programme qui ne s'arrête pas n'est pas forcément faux : un système embarqué est **conçu** pour tourner tant qu'il est alimenté. Ce qu'on exige alors, c'est que chaque tour de boucle se termine, pour que le système réagisse à temps.

Corrigé du sujet n° 24 — Arbre d'une expression arithmétique

↑ énoncé p. 727

> **Remarque**
> On donne une version simple ; d'autres réponses sont possibles. Le fichier complet, `corrige_sujet_24.py`, est en ligne : `github.com/MathThibaud/nsi-fichiers-eleves/tree/main/corriges/terminale/pratique-24`.

Question 1

Dessiner `e4` *et* `e5`*, donner l'expression et la valeur de chacun. Pourquoi un arbre n'a-t-il pas besoin de parenthèses ?*

▶ **Solution.** `e4 = Noeud('-', Noeud('-', Noeud(8), Noeud(3)), Noeud(1))` : la racine `-` a pour fils gauche le nœud `-` (de fils `8` et `3`) et pour fils droit la feuille `1`. Elle représente $(8-3)-1=4$.

`e5 = Noeud('-', Noeud(8), Noeud('-', Noeud(3), Noeud(1)))` : la racine `-` a pour fils gauche la feuille `8` et pour fils droit le nœud `-` (de fils `3` et `1`). Elle représente $8-(3-1)=6$.

Les deux arbres ont les mêmes nombres et les mêmes opérateurs, mais pas la même forme, et pas la même valeur. C'est la **forme de l'arbre** qui dit dans quel ordre les calculs sont faits : un opérateur s'applique aux valeurs de ses deux sous-arbres, qu'il faut donc calculer avant lui. Les parenthèses (et les règles de priorité) ne servent qu'à retrouver cette structure à partir d'une écriture sur une ligne ; dans l'arbre, elle est déjà explicite.

▶ Appel professeur. dessiner les deux arbres, justifier les valeurs 4 et 6 et expliquer que la structure de l'arbre remplace les parenthèses.

Question 2

Expliquer pourquoi `en_chaine(e1)` *est ambigu, puis corriger la fonction.*

▶ **Solution.**

```
>>> en_chaine(e1)
'3 + 5 * 7 - 2'
```

Sans parenthèses, avec les règles de priorité habituelles, cette chaîne vaut $3+35-2=36$ et non 40 : on a perdu la structure de l'arbre (et `e4` et `e5` donneraient la même chaîne `"8 - 3 - 1"`). Il suffit d'entourer de parenthèses chaque opération, c'est-à-dire le résultat de chaque nœud qui n'est pas une feuille :

```
1  def en_chaine(a):
2      if est_feuille(a):
3          return str(a.valeur)
4      return "(" + en_chaine(a.gauche) + " " + a.valeur + " " + en_chaine(a.droite) + ")"
```

```
1      assert en_chaine(e2) == "((2 * (1 + 9)) - 4)"
2      assert en_chaine(e4) == "((8 - 3) - 1)"
3      assert en_chaine(e5) == "(8 - (3 - 1))"
```

Question 3

Écrire la fonction récursive `evaluer`.

▶ **Solution.** Une feuille vaut son nombre (cas de base). Sinon, on évalue récursivement les deux sous-arbres, puis on applique l'opérateur de la racine, **dans l'ordre** gauche puis droite (important pour la soustraction).

```
def evaluer(a):
    if est_feuille(a):
        return a.valeur
    valeur_gauche = evaluer(a.gauche)
    valeur_droite = evaluer(a.droite)
    if a.valeur == '+':
        return valeur_gauche + valeur_droite
    elif a.valeur == '-':
        return valeur_gauche - valeur_droite
    else:
        return valeur_gauche * valeur_droite
```

```
    assert evaluer(e2) == 16
    assert evaluer(e4) == 4
    assert evaluer(e5) == 6
```

▶ Appel professeur. identifier le cas de base et l'appel récursif, justifier la terminaison (chaque appel porte sur un sous-arbre strictement plus petit) et montrer que `e4` et `e5` donnent bien 4 et 6.

Question 4

Écrire la fonction récursive `nb_operateurs`.

▶ **Solution.** Une feuille ne contient aucun opérateur ; un nœud opérateur compte pour `1`, plus les opérateurs de ses deux sous-arbres.

```
def nb_operateurs(a):
    if est_feuille(a):
        return 0
    return 1 + nb_operateurs(a.gauche) + nb_operateurs(a.droite)
```

```
    assert nb_operateurs(e2) == 3
    assert nb_operateurs(Noeud('+', Noeud(1), Noeud(2))) == 1
```

▶ Appel professeur. faire le lien avec le calcul de la taille d'un arbre vu en cours (même schéma récursif, seul le cas de base change).

Corrigé du sujet n° 25 — Arbre binaire de recherche

↑ énoncé p. 729

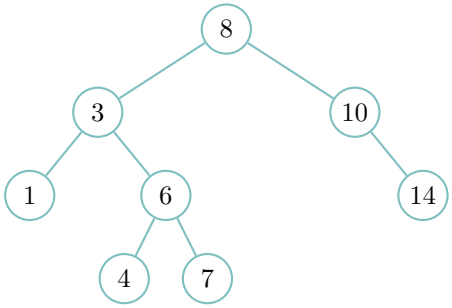
> **Remarque**
> On donne une version simple ; d'autres réponses sont possibles. Le fichier complet, `corrige_sujet_25.py`, est en ligne : `github.com/MathThibaud/nsi-fichiers-eleves/tree/main/corriges/terminale/pratique-25`.

Question 1

Dessiner l'ABR obtenu par `construire_abr([8, 3, 10, 1, 6, 14, 4, 7])` *et donner sa hauteur.*

► Solution. On insère chaque valeur en descendant depuis la racine, à gauche si elle est plus petite, à droite si elle est plus grande, jusqu'à une place vide :

- `8` est la racine ; `3` va à gauche de `8` ; `10` à droite de `8` ;
- `1` à gauche de `3` ; `6` à droite de `3` ; `14` à droite de `10` ;
- `4` à gauche de `6` ; `7` à droite de `6`.



La hauteur est `4` (chemin `8`, `3`, `6`, `4`) :

```
>>> hauteur(construire_abr([8, 3, 10, 1, 6, 14, 4, 7]))
4
```

► Appel professeur. refaire à voix haute l'insertion de deux valeurs (par exemple `4` puis `7`) et vérifier la propriété d'ABR en un nœud.

Question 2

Écrire la fonction récursive `recherche(a, x)`.

► Solution. Arbre vide : `x` est absente. Sinon on compare `x` à la racine et on ne descend que du côté où `x` peut se trouver.

```
1 def recherche(a, x):
2     if a is None:
3         return False
4     if x == a.valeur:
5         return True
```

```
    if x < a.valeur:
        return recherche(a.gauche, x)
    else:
        return recherche(a.droite, x)
```

Autre méthode : comme on ne descend que d'un seul côté, une boucle suffit : on remplace l'arbre par le sous-arbre gauche ou droit jusqu'à trouver `x` ou tomber sur un arbre vide.

```
def recherche(a, x):
    while a is not None:
        if x == a.valeur:
            return True
        if x < a.valeur:
            a = a.gauche
        else:
            a = a.droite
    return False
```

```
    assert recherche(a, 8)
    assert not recherche(a, 20)
```

Question 3

Écrire `infixe(a)` *puis* `tri_abr(valeurs)`.

▶ **Solution.**

```
def infixe(a):
    if a is None:
        return []
    return infixe(a.gauche) + [a.valeur] + infixe(a.droite)


def tri_abr(valeurs):
    return infixe(construire_abr(valeurs))
```

Dans un ABR, tout le sous-arbre gauche est plus petit que la racine, elle-même plus petite que tout le sous-arbre droit : le parcours infixe donne donc les valeurs **dans l'ordre croissant**. Les doublons disparaissent car `inserer` ne modifie pas l'arbre si la valeur est déjà présente. Attention : ce sujet fixe sa propre convention (inégalités **strictes**, ABR sans doublon), différente de celle du cours, où une valeur égale va à droite et les doublons sont conservés ; toujours appliquer la convention de l'énoncé.

```
    assert infixe(None) == []
    assert tri_abr([]) == []
```

▶ Appel professeur. expliquer pourquoi le parcours infixe d'un ABR est trié et d'où vient la suppression des doublons.

Question 4

Exécuter `experience(200)` *plusieurs fois, décrire les deux arbres et la conséquence sur le coût de* `recherche`.

▶ **Solution.**

```
>>> experience(200)
insertion dans l'ordre croissant : 200
insertion dans un ordre aléatoire : 15
```

La première hauteur vaut toujours 200 ; la seconde change à chaque exécution mais reste petite (une quinzaine en moyenne, entre 13 et 25 environ).

- Dans l'ordre croissant, chaque nouvelle valeur est plus grande que toutes les précédentes : elle va toujours à droite. L'arbre est un **peigne** (une « liste » de 200 nœuds, sans aucun fils gauche), de hauteur n.
- Dans un ordre aléatoire, l'arbre est « touffu », proche d'un arbre équilibré, de hauteur de l'ordre de $\log_2(n)$ (à un facteur près ; $\log_2(200) \approx 7{,}6$).

La recherche descend d'un niveau à chaque appel, donc fait au plus **hauteur** comparaisons : jusqu'à 200 dans le peigne (autant qu'une recherche séquentielle dans une liste), une quinzaine dans l'arbre aléatoire. L'efficacité d'un ABR dépend donc de sa forme : il faut qu'il soit (à peu près) **équilibré**.

▶ Appel professeur. décrire le peigne, relier le coût de la recherche à la hauteur, et conclure qu'insérer des données déjà triées est le pire cas.

Pour aller plus loin : réparer un peigne

Si les valeurs sont déjà triées, on peut construire directement un ABR **équilibré** : la valeur du milieu devient la racine, la moitié gauche le sous-arbre gauche, la moitié droite le sous-arbre droit, et l'on recommence récursivement sur chaque moitié.

```
def construire_equilibre(valeurs):
    """Renvoie un ABR de hauteur minimale contenant les
    valeurs de la liste valeurs (triee, sans doublon)"""
    if valeurs == []:
        return None
    milieu = len(valeurs) // 2
    return Noeud(valeurs[milieu],
                 construire_equilibre(valeurs[:milieu]),
                 construire_equilibre(valeurs[milieu + 1:]))
```

```
>>> hauteur(construire_equilibre([i for i in range(1, 201)]))
8
```

Les deux moitiés ont la même taille à une unité près : la hauteur est la plus petite possible, 8 pour 200 valeurs ($2^7 - 1 = 127 < 200 \leq 255 = 2^8 - 1$). Pour réparer le peigne de la question 4, il suffit donc d'écrire `construire_equilibre(infixe(a))`. La recherche passe alors de 200 comparaisons au plus à 8.

Un peu d'histoire : les arbres AVL (1962)

Reconstruire tout l'arbre après chaque insertion coûterait trop cher. En 1962, deux chercheurs soviétiques de Moscou, **Georgi Adelson-Velski** et **Evguéni Landis**, publient une méthode qui garde un ABR équilibré *au fil des insertions* : en chaque nœud, les hauteurs des deux sous-arbres ne diffèrent jamais de plus de 1, et une insertion qui casse cette règle est réparée par quelques « rotations » locales, près de l'endroit modifié. La hauteur reste alors de l'ordre de $\log_2(n)$ quel que soit l'ordre d'arrivée des valeurs, même triées. Ces arbres portent les initiales de leurs inventeurs : ce sont les **arbres AVL**, premiers d'une famille d'arbres « auto-équilibrés » encore utilisés dans les bibliothèques des langages et les bases de données.
Source : G. Adelson-Velski et E. Landis, « Un algorithme pour l'organisation de l'information » (en russe), *Doklady Akademii Nauk SSSR*, 146 (1962), p. 263–266 ; D. Knuth, *The Art of Computer Programming*, vol. 3, § 6.2.3.

Corrigé du sujet n° 26 — Sortir d'un labyrinthe

↑ énoncé p. 731

> **Remarque**
> On donne une version simple ; d'autres réponses sont possibles. Le fichier complet, `corrige_sujet_26.py`, est en ligne : `github.com/MathThibaud/nsi-fichiers-eleves/tree/main/corriges/terminale/pratique-26`.

Question 1

Écrire `cases_voisines(grille, case)` *(ordre : haut, bas, gauche, droite).*

▶ **Solution.** On écrit les quatre cases candidates dans l'ordre demandé, puis on construit **par compréhension** la liste de celles qui sont dans la grille et ne sont pas des murs.

```
def cases_voisines(grille, case):
    ligne, colonne = case
    candidates = [(ligne - 1, colonne), (ligne + 1, colonne),
                  (ligne, colonne - 1), (ligne, colonne + 1)]
    return [(l, c) for (l, c) in candidates
            if 0 <= l < len(grille) and 0 <= c < len(grille[l])
            and grille[l][c] != "#"]
```

Autre méthode : on part d'une liste vide et on ajoute chaque candidate valable avec `append`.

```
    voisines = []
    for (l, c) in candidates:
        if 0 <= l < len(grille) and 0 <= c < len(grille[l]) and grille[l][c] != "#":
            voisines.append((l, c))
    return voisines
```

Le test des bornes n'est pas indispensable ici (les labyrinthes sont entourés de murs), mais il rend la fonction sûre pour n'importe quelle grille.

```
    assert cases_voisines(grille, (1, 3)) == [(1, 2), (1, 4)]
```

▶ Appel professeur. expliquer le calcul des quatre voisines (quelle coordonnée change, dans quel sens) et l'ordre imposé.

Question 2

Compléter `plus_court_chemin` *(parcours en largeur).*

▶ **Solution.** Quand on découvre un voisin, il faut : le marquer comme découvert, l'enfiler, et noter son parent.

```
        for voisin in graphe[sommet]:
            if voisin not in decouverts:
                decouverts.append(voisin)
                en_attente.append(voisin)
                parent[voisin] = sommet
```

```
    assert plus_court_chemin(graphe, (1, 1), (1, 1)) == [(1, 1)]
    assert plus_court_chemin(graphe, (1, 1), (3, 3)) == [(1, 1), (2, 1), (3, 1), (3, 2), (3, 3)
    ]
```

Question 3

Utiliser `sortir` *sur* `labyrinthe.txt`.

▶ Solution.

```
>>> sortir("labyrinthe.txt")
######################
#Eoooooooo#         #
#########o####### # #
#       #ooooooo  # #
# # # # #######o### #
# #       #    o    #
# ##### ### ###o#####
#     #     #  ooooo#
### # ####### #####o#
#             #    S#
######################
26
```

Il faut **26 déplacements** (le chemin compte 27 cases, entrée et sortie comprises).

▶ Appel professeur. expliquer pourquoi le parcours en largeur donne un plus court chemin : il découvre les cases par distance croissante à l'entrée, et le parent d'une case est toujours à distance un de moins.

Question 4

Remplacer `pop(0)` *par* `pop()` *: obtient-on encore un chemin ? le plus court ?*

▶ Solution.

```
>>> sortir("labyrinthe.txt")
######################
#Eoooooooo#      ooo#
#########o#######o#o#
#ooo    #ooooooooo#o#
#o#o# # ####### ###o#
#o#ooooo  #ooooooooo#
#o#####o###o### #####
#ooooo#ooooo#ooooooo#
### #o#######o#####o#
#    ooooooooo#    S#
######################
78
```

On obtient encore un chemin **valide** de l'entrée à la sortie (chaque case a pour parent une case voisine), mais de **78** déplacements au lieu de 26 : ce n'est plus le plus court.

Avec `pop()`, `en_attente` n'est plus une file mais une **pile** : on repart toujours de la dernière case découverte et l'on s'enfonce loin avant de revenir en arrière, comme dans un parcours en profondeur. Ce n'en est pas tout à fait un, car les cases sont marquées dès qu'on les empile (voir la règle du cours de Graphes) ; surtout, le parcours ne découvre plus les cases par distance croissante. Comme ce labyrinthe contient des **cycles** (plusieurs chemins possibles), le parent retenu pour une case n'est pas forcément sur un plus court chemin.

▶ Appel professeur. faire le lien file ↔ largeur, pile ↔ profondeur, et expliquer que seul le parcours en largeur garantit un plus court chemin (en nombre d'arêtes).

Corrigé du sujet n° 27 — Calculatrice en notation polonaise inverse

↑énoncé p. 733

> **Remarque**
> On donne une version simple ; d'autres réponses sont possibles. Le fichier complet, `corrige_sujet_27.py`, est en ligne : `github.com/MathThibaud/nsi-fichiers-eleves/tree/main/corriges/terminale/pratique-27`.

Question 1

Évaluer à la main `5 1 2 + 4 * + 3 -` *en dessinant la pile.*

▶ **Solution.** La pile est dessinée verticalement, **sommet en haut**.

Élément	Action	Pile
5	empiler	5
1	empiler	1 5
2	empiler	2 1 5
+	dépiler 2 et 1, empiler $1+2$	3 5
4	empiler	4 3 5
*	dépiler 4 et 3, empiler 3×4	12 5
+	dépiler 12 et 5, empiler $5+12$	17
3	empiler	3 17
-	dépiler 3 et 17, empiler $17-3$	14

La valeur est `14` : c'est l'expression $5+(1+2)\times 4-3$.

▶ Appel professeur. dérouler la pile à voix haute, en insistant sur l'ordre des opérandes pour `-`.

Question 2

Écrire `evaluer_npi(expression)` *avec une* `Pile`.

▶ **Solution.**

```
1 def evaluer_npi(expression):
2     p = Pile()
3     for element in expression.split():
4         if element in OPERATEURS:
5             droite = p.depiler()      # le dernier empile est l'operande de DROITE
6             gauche = p.depiler()
```

```
            if element == "+":
                p.empiler(gauche + droite)
            elif element == "-":
                p.empiler(gauche - droite)
            else:
                p.empiler(gauche * droite)
        else:
            p.empiler(int(element))
    return p.depiler()
```

Erreur fréquente : dépiler dans le mauvais ordre, ce qui donne $2 - 7 = -5$ pour `7 2 -` (le test le détecte).

```
    assert evaluer_npi("5 1 2 + 4 * + 3 -") == 14
    assert evaluer_npi("2 3 4 * +") == 14
```

▶ Appel professeur. expliquer pourquoi une pile convient (l'opérateur s'applique aux deux derniers résultats obtenus) et l'ordre des deux dépilements.

Question 3

Renvoyer `None` *pour une expression mal formée.*

▶ **Solution.** Deux situations à détecter :

- à la lecture d'un opérateur, la pile contient **moins de deux** nombres (`"3 +"`, `"+"`) : dépiler provoquerait une erreur ;
- à la fin, la pile ne contient **pas exactement un** nombre : il reste des nombres inutilisés (`"3 4"`) ou la pile est vide (expression vide `""`).

```
def evaluer_npi(expression):
    p = Pile()
    for element in expression.split():
        if element in OPERATEURS:
            if p.taille() < 2:       # il manque un operande
                return None
            droite = p.depiler()
            gauche = p.depiler()
            if element == "+":
                p.empiler(gauche + droite)
            elif element == "-":
                p.empiler(gauche - droite)
            else:
                p.empiler(gauche * droite)
        else:
            p.empiler(int(element))
    if p.taille() != 1:              # expression vide ou operateur manquant
        return None
    return p.depiler()
```

```
    assert evaluer_npi("") is None
    assert evaluer_npi("+") is None
    assert evaluer_npi("1 2 + +") is None
```

Question 4

Traduire $(2 + 3) \times (7 - 4)$ *en NPI et vérifier.*

▶ **Solution.** On écrit l'expression de gauche, puis celle de droite, puis l'opérateur : `2 3 +` puis `7 4 -` puis `*`, soit `2 3 + 7 4 - *`.

```
>>> evaluer_npi("2 3 + 7 4 - *")
15
```

On retrouve bien $5 \times 3 = 15$. C'est l'ordre du parcours **suffixe** de l'arbre de l'expression : voilà pourquoi la NPI n'a pas besoin de parenthèses.

► Appel professeur. justifier la traduction et, si possible, faire le lien avec le parcours suffixe d'un arbre.

Corrigé du sujet n° 28 — Ordonnancement de tâches : le tourniquet

↑énoncé p. 735

> **Remarque**
> On donne une version simple ; d'autres réponses sont possibles. Le fichier complet, `corrige_sujet_28.py`, est en ligne : `github.com/MathThibaud/nsi-fichiers-eleves/tree/main/corriges/terminale/pratique-28`.

Question 1

Tourniquet de quantum 2 sur `[("A", 3), ("B", 1), ("C", 4)]` *: frise, file, instants de fin.*

▶ **Solution.** (tête de file à gauche)

Intervalle	Tâche	Ce qui se passe	File ensuite
$[0;2]$	A	A s'exécute 2, il reste 1 : réenfilée	`B, C, A(1)`
$[2;3]$	B	B s'exécute 1 : **finie à 3**	`C, A(1)`
$[3;5]$	C	C s'exécute 2, il reste 2 : réenfilée	`A(1), C(2)`
$[5;6]$	A	A s'exécute 1 : **finie à 6**	`C(2)`
$[6;8]$	C	C s'exécute 2 : **finie à 8**	(vide)

On obtient `[("B", 3), ("A", 6), ("C", 8)]`.

▶ Appel professeur. dérouler la frise en montrant qu'une tâche non terminée repart en **fin** de file.

Question 2

Écrire `tourniquet(taches, quantum)` *avec une* `File`.

▶ **Solution.**

```
def tourniquet(taches, quantum):
    f = File()
    for tache in taches:
        f.enfiler(tache)
    instant = 0
    fins = []
    while not f.est_vide():
        nom, reste = f.defiler()
        execution = min(quantum, reste)    # au plus quantum unites de temps
        instant = instant + execution
        reste = reste - execution
        if reste > 0:
            f.enfiler((nom, reste))        # pas finie : retour en fin de file
        else:
            fins.append((nom, instant))
    return fins
```

On enfile un **nouveau** couple `(nom, reste)` : la liste `taches` de départ n'est pas modifiée.

```
    assert tourniquet(taches, 1) == [("B", 2), ("A", 6), ("C", 8)]
    assert tourniquet(taches, 10) == premier_arrive(taches)
    assert tourniquet([], 2) == []
```

Question 3

Écrire `temps_moyen(fins)`.

▶ **Solution.**

```
def temps_moyen(fins):
    total = 0
    for nom, instant in fins:
        total = total + instant
    return total / len(fins)
```

Pour l'exemple : premier arrivé donne $(3+4+8)/3 = 5$, le tourniquet de quantum 2 donne $(3+6+8)/3 \approx 5{,}67$.

▶ Appel professeur. présenter `tourniquet` et le rôle de la file, puis vérifier les tests.

Question 4

Exécuter `comparer("taches.csv")` *et interpréter.*

▶ **Solution.**

```
>>> comparer("taches.csv")
premier arrivé : 62.375
tourniquet, quantum 1 : 27.0
tourniquet, quantum 2 : 27.75
tourniquet, quantum 5 : 30.0
tourniquet, quantum 10 : 36.875
tourniquet, quantum 100 : 62.375
```

- En premier arrivé, premier servi, la `sauvegarde` (40 unités), placée en tête, fait attendre toutes les autres : même la `calculatrice` (durée 1) ne se termine qu'à l'instant 78. Le tourniquet fait passer les **tâches courtes** rapidement (quantum 5 : `navigateur` finie à 8 au lieu de 43) ; les longues (`sauvegarde`, `antivirus`) finissent un peu plus tard, mais le temps moyen est divisé par plus de deux.
- Quand le quantum dépasse la durée de toutes les tâches (ici 100), aucune tâche n'est interrompue : on retrouve exactement premier arrivé, premier servi.
- En pratique, chaque changement de tâche (**commutation de contexte**) a un coût pour le processeur : sauvegarder et restaurer l'état de la tâche. Un quantum trop petit multiplie ces commutations et fait perdre du temps ; on choisit un compromis.

▶ Appel professeur. interpréter les résultats et faire le lien avec l'ordonnancement des processus vu en cours (interactivité, coût des commutations).

Corrigé du sujet n° 29 — Recherche dichotomique

↑énoncé p. 737

> **Remarque**
> On donne une version simple ; d'autres réponses sont possibles. Le fichier complet, `corrige_sujet_29.py`, est en ligne : `github.com/MathThibaud/nsi-fichiers-eleves/tree/main/corriges/terminale/pratique-29`.

Question 1

Écrire `recherche_dichotomique(tab, x)` *avec une boucle* `while`.

▶ Solution.

```
def recherche_dichotomique(tab, x):
    debut = 0
    fin = len(tab) - 1
    while debut <= fin:
        milieu = (debut + fin) // 2
        if tab[milieu] == x:
            return milieu
        elif tab[milieu] < x:
            debut = milieu + 1
        else:
            fin = milieu - 1
    return -1
```

Pour un tableau vide, `fin` vaut `-1` : on n'entre pas dans la boucle et on renvoie `-1`.

▶ Appel professeur. justifier la terminaison : la longueur `fin - debut + 1` de la zone de recherche est un entier qui diminue strictement à chaque tour (grâce aux `+ 1` et `- 1`).

Question 2

Compléter `comparaisons_dichotomique`, *exécuter* `comparer(1000)` *et* `comparer(1000000)`, *expliquer*.

▶ Solution. Même boucle que la question 1, avec un compteur incrémenté à chaque élément du milieu examiné :

```
def comparaisons_dichotomique(tab, x):
    nb = 0
    debut = 0
    fin = len(tab) - 1
    while debut <= fin:
        milieu = (debut + fin) // 2
        nb = nb + 1
        if tab[milieu] == x:
            return nb
        elif tab[milieu] < x:
            debut = milieu + 1
        else:
            fin = milieu - 1
    return nb
```

```
>>> comparer(1000)
n = 1000
  séquentielle : 1000
  dichotomique : 10
>>> comparer(1000000)
n = 1000000
  séquentielle : 1000000
  dichotomique : 20
```

La valeur cherchée est absente : la recherche séquentielle examine les n éléments (coût **linéaire**). La recherche dichotomique divise la zone par deux à chaque étape ; le nombre d'étapes est le nombre de divisions par 2 pour passer de n à 0, environ $\log_2(n)$: $2^{10} = 1024$ et $2^{20} \approx 10^6$ (coût **logarithmique**). Multiplier la taille par 1000 ajoute seulement une dizaine de comparaisons.

▶ Appel professeur. expliquer le lien entre « diviser par deux » et le logarithme en base 2.

Question 3

Corriger `premiere_occurrence` *(plus petit indice de* `x`*).*

▶ Solution.

```
>>> premiere_occurrence([1, 2, 2, 2, 3], 2)
2
```

L'erreur : dès que `tab[milieu] == x`, la fonction renvoie `milieu`, qui est **une** occurrence de `x`, pas forcément la première : d'autres `x` peuvent se trouver à gauche. Correction : quand on trouve `x`, on **mémorise** l'indice et on **continue** à chercher dans la moitié gauche.

```
def premiere_occurrence(tab, x):
    resultat = -1
    debut = 0
    fin = len(tab)          # debut inclus, fin exclu
    while debut < fin:
        milieu = (debut + fin) // 2
        if tab[milieu] < x:
            debut = milieu + 1
        elif tab[milieu] > x:
            fin = milieu
        else:
            resultat = milieu    # une occurrence, peut-etre pas la premiere :
            fin = milieu         # on continue a chercher a gauche
    return resultat
```

La zone continue d'être divisée par deux à chaque tour : le coût reste logarithmique.

```
    assert premiere_occurrence([], 4) == -1
    assert premiere_occurrence([1, 1, 2, 2, 2, 2, 2, 2, 3], 2) == 2
```

▶ Appel professeur. expliquer l'erreur sur l'exemple, puis pourquoi la version corrigée renvoie bien le plus petit indice et reste une dichotomie.

Question 4

Écrire la fonction récursive `recherche_rec(tab, x, debut, fin)`.

▶ Solution. La condition d'arrêt est la zone vide (`debut > fin`) ; sinon, on examine le milieu et on relance la recherche sur une seule moitié.

```
def recherche_rec(tab, x, debut, fin):
    if debut > fin:
        return -1
    milieu = (debut + fin) // 2
    if tab[milieu] == x:
```

```
        return milieu
    elif tab[milieu] < x:
        return recherche_rec(tab, x, milieu + 1, fin)
    else:
        return recherche_rec(tab, x, debut, milieu - 1)
```

```
assert recherche_rec(tab, 2, 0, len(tab) - 1) == 0
assert recherche_rec([], 4, 0, -1) == -1
```

Erreurs fréquentes

- `milieu = (debut + fin) / 2` : la division `/` donne un flottant, et `tab[milieu]` provoque une `TypeError`. Il faut la division entière `//`.
- `debut = milieu` au lieu de `milieu + 1` : avec `tab = [1, 3]` et `x = 3`, on a `debut = 0`, `fin = 1`, `milieu = 0`, puis à nouveau `debut = 0` : la zone ne diminue plus et la boucle ne s'arrête jamais (en récursif : `RecursionError`).
- **Oublier le `return`** devant l'appel récursif : la valeur trouvée est perdue et la fonction renvoie `None`.
- **Travailler sur des tranches** au lieu de passer `debut` et `fin` :

```
def recherche_tranches(tab, x):
    """Version a eviter : les tranches copient la liste"""
    if tab == []:
        return -1
    milieu = len(tab) // 2
    if tab[milieu] == x:
        return milieu
    elif tab[milieu] < x:
        return recherche_tranches(tab[milieu + 1:], x)
    else:
        return recherche_tranches(tab[:milieu], x)
```

`recherche_tranches([1, 3, 5, 7, 9], 9)` renvoie `1` et non `4` : c'est l'indice de `9` dans la tranche `[7, 9]`, pas dans le tableau. De plus, chaque tranche est une **copie** : copier la moitié du tableau à chaque appel coûte un temps proportionnel à n, et l'on perd tout le bénéfice du coût logarithmique.

Un peu d'histoire : une idée simple, longtemps mal programmée

D'après Donald Knuth, la recherche dichotomique est décrite pour la première fois en 1946 par **John Mauchly**, l'un des concepteurs de l'ENIAC, lors d'une série de cours à l'université de Pennsylvanie ; mais il faut attendre 1960 pour qu'une version publiée, due à Derrick Lehmer, fonctionne pour *toutes* les tailles de tableau. En 2006, **Joshua Bloch**, de Google, révèle que la recherche dichotomique de la bibliothèque standard de Java contenait une erreur depuis neuf ans : sur un tableau de plus d'un milliard d'éléments, la somme `debut + fin` dépassait le plus grand entier représentable et devenait négative. En Python, les entiers n'ont pas de taille maximale : cette erreur-là ne peut pas se produire, mais celles de l'encadré précédent, si.

Sources : D. Knuth, *The Art of Computer Programming*, vol. 3, § 6.2.1 ; J. Bloch, « Extra, Extra – Read All About It : Nearly All Binary Searches and Mergesorts are Broken », blog de Google Research, 2 juin 2006.

Corrigé du sujet n° 30 — Messages secrets

↑énoncé p. 739

> **Remarque**
> On donne une version simple ; d'autres réponses sont possibles. Le fichier complet, `corrige_sujet_30.py`, est en ligne : `github.com/MathThibaud/nsi-fichiers-eleves/tree/main/corriges/terminale/pratique-30`.

Question 1

Écrire `cesar(message, k)` *(les caractères autres que* `A`–`Z` *restent inchangés).*

▶ Solution.

```
1  def cesar(message, k):
2      resultat = ""
3      for lettre in message:
4          if "A" <= lettre <= "Z":
5              rang = ord(lettre) - ord("A")
6              nouveau = (rang + k) % 26
7              resultat = resultat + chr(nouveau + ord("A"))
8          else:
9              resultat = resultat + lettre
10     return resultat
```

Le `% 26` rend le décalage circulaire (`Z` décalé de 1 donne `A`). Il fonctionne aussi pour une clé négative : en Python, `-3 % 26` vaut 23. C'est pourquoi `cesar(c, -k)` déchiffre.

▶ Appel professeur. expliquer le rôle du modulo et pourquoi le même code déchiffre avec `-k`.

Question 2

Compléter `occurrences`*, puis retrouver la clé de* `message_secret.txt` *et afficher le clair.*

▶ Solution.

```
1  def occurrences(texte):
2      occ = {}
3      for lettre in texte:
4          if "A" <= lettre <= "Z":
5              if lettre in occ:
6                  occ[lettre] = occ[lettre] + 1
7              else:
8                  occ[lettre] = 1
9      return occ
```

```
>>> secret = lire_fichier("message_secret.txt")
>>> lettre_plus_frequente(secret)
'L'
```

La lettre la plus fréquente, `L` (rang 11), est très probablement un `E` (rang 4) chiffré : la clé vaut $11 - 4 = 7$. On déchiffre avec `cesar(secret, -7)` :

```
>>> cesar(secret, -7)
'EN MIL NEUF CENT QUARANTE, DANS LE PARC DE BLETCHLEY, UNE EQUIPE ...'
```

Le texte parle d'Alan Turing et des attaques par **force brute**. On pouvait aussi essayer les 26 clés et repérer à l'œil le seul résultat lisible. La clé se calcule aussi en Python : `(ord(lettre) - ord("E")) % 26`.

▶ Appel professeur. expliquer pourquoi l'analyse des fréquences casse César : une même lettre est **toujours** chiffrée de la même façon, donc les fréquences sont conservées.

Question 3

Trouver et corriger l'erreur de `vigenere`.

▶ **Solution.**

```
>>> vigenere("NSIROCKS", "CLE")
'CQZGMTZQ'
```

L'erreur est à la ligne `decalage = ord(cle[i % len(cle)])` : elle prend le **code** de la lettre de la clé (`ord("C")` vaut 67) au lieu de son **rang** (2). Il faut retirer `ord("A")`, comme pour le message :

```
1         decalage = ord(cle[i % len(cle)]) - ord("A")
```

▶ Appel professeur. montrer comment l'erreur a été localisée (calcul à la main de la première lettre, affichage de `decalage`).

Question 4

Écrire `dechiffre_vigenere(message, cle)`*, puis expliquer pourquoi la méthode de la question 2 ne suffit plus.*

▶ **Solution.** On refait le même parcours en **retranchant** le décalage :

```
1 def dechiffre_vigenere(message, cle):
2     resultat = ""
3     for i in range(len(message)):
4         decalage = ord(cle[i % len(cle)]) - ord("A")
5         rang = ord(message[i]) - ord("A")
6         nouveau = (rang - decalage) % 26
7         resultat = resultat + chr(nouveau + ord("A"))
8     return resultat
```

```
1     assert dechiffre_vigenere("", "CLE") == ""
2     assert dechiffre_vigenere(vigenere("ZZZ", "B"), "B") == "ZZZ"
```

Avec Vigenère, une même lettre du clair est chiffrée de façons **différentes** selon sa position (`ANANAS` chiffré avec `CLE` donne `CYEPLW` : les trois `A` deviennent `C`, `E` et `L`). Les `E` du clair se répartissent sur plusieurs lettres chiffrées : la lettre la plus fréquente du chiffré ne correspond plus à un seul `E`, et un seul décalage ne suffit pas à tout déchiffrer. Si l'on connaît la **longueur** L de la clé, on peut en revanche refaire l'analyse des fréquences sur les lettres d'indices $0, L, 2L, \ldots$ (toutes décalées pareil), puis sur $1, L+1, \ldots$: c'est l'idée de Babbage et Kasiski.

▶ Appel professeur. expliquer la différence avec César et, si possible, l'idée de l'attaque par longueur de clé.

Corrigé du sujet n° 31 — Recherche d'un motif

↑ énoncé p. 741

> **Remarque**
> On donne une version simple ; d'autres réponses sont possibles. Le fichier complet, `corrige_sujet_31.py`, est en ligne : `github.com/MathThibaud/nsi-fichiers-eleves/tree/main/corriges/terminale/pratique-31`.

Question 1

Écrire `recherche_naive(motif, texte)`, *puis compter les occurrences de* `"GATTACA"` *dans* `adn.txt`.

▶ Solution.

```
1  def recherche_naive(motif, texte):
2      n = len(texte)
3      m = len(motif)
4      positions = []
5      for i in range(n - m + 1):
6          j = 0
7          while j < m and texte[i + j] == motif[j]:
8              j = j + 1
9          if j == m:
10             positions.append(i)
11     return positions
```

Le `range(n - m + 1)` évite de dépasser la fin du texte (dernière position possible : `n - m`). Si le motif est plus long que le texte, la boucle ne fait aucun tour.

```
>>> adn = lire_fichier("adn.txt")
>>> len(recherche_naive("GATTACA", adn))
13
```

▶ Appel professeur. expliquer le rôle de `n - m + 1` et l'ordre des deux conditions du `while` (on teste `j < m` avant de lire `motif[j]`).

Question 2

Écrire `table_decalage(motif)`.

▶ Solution.

```
1  def table_decalage(motif):
2      m = len(motif)
3      dec = {}
4      for k in range(m - 1):        # toutes les lettres sauf la derniere
5          dec[motif[k]] = m - 1 - k
6      return dec
```

On parcourt le motif de gauche à droite : pour une lettre répétée, la dernière affectation (la plus proche de la fin, donc la plus petite valeur) écrase les précédentes. Pour `"GATTACA"`, le `A` d'indice 1 reçoit d'abord 5, puis celui d'indice 4 lui donne 2. La dernière lettre n'est pas dans la table : sinon elle recevrait 0, et le motif ne bougerait plus.

▶ Appel professeur. expliquer pourquoi on garde la plus petite distance (ne jamais sauter par-dessus une occurrence possible).

Question 3

Trouver et corriger l'erreur de `horspool`.

▶ Solution.

```
>>> horspool("ABA", "ABABABA")
[0, 4]
```

Après une occurrence trouvée, la fonction fait `i = i + m` : elle saute toute la longueur du motif et manque les occurrences qui **chevauchent** celle qu'on vient de trouver (ici celle en position 2). Après une occurrence, il faut décaler comme après un échec, avec la table, sans dépasser une autre occurrence possible :

```
        if j < 0:
            positions.append(i)
        c = texte[i + m - 1]
        if c in dec:
            i = i + dec[c]
        else:
            i = i + m
```

Sur `adn.txt`, `horspool("GATTACA", adn)` renvoie alors la même liste que `recherche_naive` (13 positions).

▶ Appel professeur. montrer l'erreur sur `"AA"` dans `"AAAA"` et justifier le décalage après une occurrence.

Question 4

Compléter `comparaisons_naive` *et* `comparaisons_horspool`*, exécuter* `comparer()` *et commenter.*

▶ Solution. On reprend la boucle de `recherche_naive`, puis celle de `horspool` corrigée, en comptant chaque comparaison, y compris celle qui échoue :

```
def comparaisons_naive(motif, texte):
    n = len(texte)
    m = len(motif)
    nb = 0
    for i in range(n - m + 1):
        j = 0
        while j < m and texte[i + j] == motif[j]:
            nb = nb + 1                # comparaison reussie
            j = j + 1
        if j < m:
            nb = nb + 1                # la comparaison qui a echoue
    return nb

def comparaisons_horspool(motif, texte):
    n = len(texte)
    m = len(motif)
    dec = table_decalage(motif)
    nb = 0
    i = 0
    while i <= n - m:
        j = m - 1
        while j >= 0 and texte[i + j] == motif[j]:
            nb = nb + 1                # comparaison reussie
            j = j - 1
        if j >= 0:
            nb = nb + 1                # la comparaison qui a echoue
        c = texte[i + m - 1]
```

```
28          if c in dec:
29              i = i + dec[c]
30          else:
31              i = i + m
32      return nb
```

```
>>> comparer()
motif de longueur 7 dans un texte de longueur 100000
  naïve    : 133510
  horspool : 47069
motif de longueur 13 dans un texte de longueur 105000
  naïve    : 113985
  horspool : 13497
motif de longueur 10 dans un texte de longueur 100000
  naïve    : 999910
  horspool : 99991
```

- **ADN** : Horspool fait environ 3 fois moins de comparaisons. Avec seulement 4 lettres, presque toutes figurent dans le motif, près de sa fin : les sauts restent courts.
- **Texte français** : avec un alphabet plus grand, beaucoup de caractères (`P`, `U`, `S`...) sont absents du motif et provoquent un saut de toute sa longueur (13). Horspool fait environ 8 fois moins de comparaisons : il ne lit même pas la plupart des caractères du texte (coût **sous-linéaire**).
- **Pire cas de la méthode naïve** : à chaque position, elle compare 9 `A` avant d'échouer sur le `B`, soit environ $n \times m$ comparaisons. Horspool, qui commence par la fin, échoue dès la première comparaison (`B` différent de `A`) et n'en fait qu'une par position.

▶ Appel professeur. expliquer pourquoi la taille de l'alphabet et la longueur du motif influent sur l'efficacité de Horspool.

Corrigé du sujet n° 32 — Le grand escalier

↑énoncé p. 743

> **Remarque**
> On donne une version simple ; d'autres réponses sont possibles. Le fichier complet, `corrige_sujet_32.py`, est en ligne : `github.com/MathThibaud/nsi-fichiers-eleves/tree/main/corriges/terminale/pratique-32`.

Question 1

Justifier la relation, puis écrire `effort_rec(couts, i)`.

▶ **Solution.** On arrive sur la marche `i` soit depuis la marche `i - 1` (un pas d'une marche), soit depuis la marche `i - 2` (un pas de deux marches). Dans les deux cas, on paie ensuite l'effort de la marche `i`. On a intérêt à venir de celle des deux où l'on arrive avec le moins d'effort, d'où `E(i) = couts[i] + min(E(i - 1), E(i - 2))`. Les marches 0 et 1 s'atteignent directement depuis le sol : ce sont les deux cas de base.

```
1 def effort_rec(couts, i):
2     if i == 0 or i == 1:
3         return couts[i]
4     return couts[i] + min(effort_rec(couts, i - 1), effort_rec(couts, i - 2))
```

▶ Appel professeur. justifier la terminaison : `i` est un entier positif qui diminue strictement à chaque appel, jusqu'à 0 ou 1.

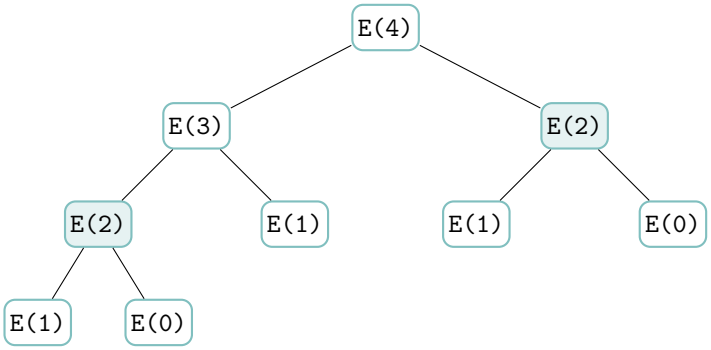
Question 2

Exécuter `compter_appels` *pour* 10, 20 *et* 25 *marches, expliquer la croissance.*

▶ **Solution.** Après l'ajout de la ligne `APPELS[0] = APPELS[0] + 1` en tête de `effort_rec` :

```
>>> compter_appels(10), compter_appels(20), compter_appels(25)
(176, 21890, 242784)
```

L'arbre des appels de $E(4)$ (les deux appels $E(2)$, qui refont exactement le même calcul, sont grisés) :



L'appel $E(4)$ lance $E(3)$ et $E(2)$; or $E(3)$ relance lui-même $E(2)$ et $E(1)$, soit 9 appels en tout. $E(2)$ est donc calculé **deux fois**, $E(1)$ trois fois, et ainsi de suite : les sous-problèmes **se recouvrent**.

Chaque marche supplémentaire multiplie environ par 1,6 le nombre d'appels (comme la suite de Fibonacci) : le coût est **exponentiel**. Pour 100 marches, il faudrait de l'ordre de 10^{21} appels : c'est impossible en pratique.

▶ Appel professeur. dessiner l'arbre des appels de $E(4)$ et y repérer les calculs répétés.

Question 3

Écrire `effort_memo(couts, i, memo)`*, puis calculer l'effort pour* `escalier.txt`*.*

▶ Solution.

```python
def effort_memo(couts, i, memo):
    if i == 0 or i == 1:
        return couts[i]
    if i in memo:                    # deja calcule : on le relit
        return memo[i]
    memo[i] = couts[i] + min(effort_memo(couts, i - 1, memo),
                             effort_memo(couts, i - 2, memo))
    return memo[i]
```

```
>>> effort_total_memo(lire_escalier("escalier.txt"))
185
```

Chaque $E(i)$ n'est plus calculé qu'une fois : le coût devient **linéaire** en le nombre de marches.

▶ Appel professeur. expliquer pourquoi le même dictionnaire `memo` doit être partagé par tous les appels.

Question 4

Écrire `effort_tab(couts)`*, puis comparer avec la stratégie gloutonne.*

▶ Solution. On remplit la liste du bas vers le haut : quand on calcule `e[i]`, les valeurs `e[i - 1]` et `e[i - 2]` sont déjà connues.

```python
def effort_tab(couts):
    n = len(couts)
    e = [0] * n
    e[0] = couts[0]
    e[1] = couts[1]
    for i in range(2, n):
        e[i] = couts[i] + min(e[i - 1], e[i - 2])
    return min(e[n - 1], e[n - 2])
```

```
>>> effort_tab([10, 15, 20]), effort_glouton([10, 15, 20])
(15, 25)
>>> couts = lire_escalier("escalier.txt")
>>> effort_tab(couts), effort_glouton(couts)
(185, 213)
```

Sur `[10, 15, 20]`, le glouton choisit la marche 0 (effort 10, la moins pénible des deux premières), puis la marche 1 (15) : il paie 25. Or monter directement sur la marche 1 ne coûte que 15. Le glouton fait à chaque pas le choix qui semble le meilleur sur le moment, sans jamais revenir en arrière. La programmation dynamique, elle, compare **toutes** les possibilités grâce à la relation de récurrence, sans les énumérer une à une : elle trouve toujours le minimum.

▶ Appel professeur. citer un autre problème du cours où le glouton n'est pas optimal (rendu de monnaie avec le système $\{1, 3, 4\}$).

IV

ANNEXES

Rappels de Première et lexique

Les indispensables de Première

Mode d'emploi

Ces pages rassemblent les notions de Première dont la Terminale se sert sans les réexpliquer. Elles s'adressent à ceux qui ont suivi la Première ailleurs ou qui l'ont oubliée. La ligne « Prérequis » de chaque cours de Terminale renvoie aux rubriques utiles : relisez-les avant d'aborder le chapitre, tapez les exemples dans la console Python, et vérifiez que vous retrouvez les résultats affichés. Chaque rubrique se termine par le **piège** le plus fréquent.

Les bases de Python

Types de base. `int` (entier : `15`, `-3`), `float` (flottant, nombre à virgule : `12.5`), `str` (chaîne de caractères : `"Ada"`), `bool` (booléen : `True`, `False`). La fonction `type` donne le type d'une valeur ; `int(...)`, `float(...)`, `str(...)` convertissent. `input(...)` renvoie **toujours** une chaîne.

```
>>> 7 / 2        # division : un flottant
3.5
>>> 7 // 2       # quotient entier
3
>>> 7 % 2        # reste
1
>>> 2 ** 10      # puissance
1024
```

```
>>> type(3.0)
<class 'float'>
>>> "3" + "4"      # deux chaînes : on colle
'34'
>>> int("3") + 4   # conversion, puis calcul
7
```

Variables et affectation. `x = 5` se lit « `x` reçoit `5` » : on évalue la partie droite, puis on range le résultat sous le nom de gauche. `x = x + 1` augmente `x` de 1. Le test d'égalité s'écrit `==` (`!=` pour « différent »).

Conditions. `if`, `elif`, `else`, suivis de `:` et d'un bloc **indenté**. On combine les conditions avec `and`, `or`, `not`. Ces opérateurs sont **séquentiels** : dans `a and b`, si `a` est faux, `b` n'est pas évalué (d'où `n != 0 and total / n > 1`, sans risque de division par zéro).

Boucles. La boucle **bornée** `for i in range(a, b)` fait prendre à `i` les valeurs `a`, `a + 1`, …, `b - 1` (`b` est exclu ; `range(n)` va de 0 à `n - 1`). La boucle **non bornée** `while condition:` répète tant que la condition est vraie : on l'emploie quand on ne connaît pas le nombre de tours à l'avance.

Fonctions. `def` définit une fonction, ses **paramètres** reçoivent les valeurs passées à l'appel (les **arguments**), et `return` **renvoie** le résultat et termine l'appel. Les variables créées dans une fonction sont **locales** : elles n'existent pas en dehors.

```
def nb_chiffres(n):
    """Renvoie le nombre de chiffres de l'entier n >= 1."""
    compteur = 0
    while n > 0:
        n = n // 10          # on retire le dernier chiffre
        compteur = compteur + 1
    return compteur

for i in range(1, 4):
    print(i, i ** 2)
print(nb_chiffres(2026))
```

Affichage :

```
1 1
2 4
3 9
4
```

`print` n'est pas `return`. `print` *affiche* un texte à l'écran, pour l'utilisateur ; `return` *renvoie* une valeur au programme, qui peut la ranger dans une variable et la réutiliser. Une fonction sans

`return` renvoie `None`.

```
def double_affiche(x):
    print(2 * x)
def double(x):
    return 2 * x
r1 = double_affiche(5)  # affiche 10
r2 = double(5)          # n'affiche rien
print(r1, r2)           # affiche None 10
```

`r1` vaut `None` : la première fonction a *affiché* le double, mais ne l'a pas *renvoyé*. Seule `double` peut servir dans un calcul, comme `double(5) + 1`.

▷ **Piège classique.** écrire `=` au lieu de `==` dans un test, oublier le `:` ou mal indenter un bloc, et surtout « répondre » avec `print` quand l'énoncé demande une fonction qui *renvoie*.

Les chaînes de caractères

Une chaîne (`str`) est une suite de caractères, **numérotés à partir de 0**. Si elle compte `n` caractères, les indices vont de `0` à `n - 1` ; les **indices négatifs** comptent depuis la fin (`mot[-1]` est le dernier caractère). Une **tranche** `mot[a:b]` est la sous-chaîne des indices `a` à `b - 1`. Une chaîne est **immuable** : on ne modifie pas un caractère, on fabrique une nouvelle chaîne.

```
>>> mot = "bonjour"
>>> len(mot)
7
>>> mot[0], mot[-1]
('b', 'r')
>>> mot[1:4]          # indices 1, 2, 3
'onj'
>>> mot[:3]           # depuis le début
'bon'
>>> "j" in mot        # appartenance
True
```

```
>>> mot + " !"        # concaténation
'bonjour !'
>>> mot[0] = "B"
TypeError: 'str' object does not
support item assignment
>>> "B" + mot[1:]     # nouvelle chaîne
'Bonjour'
>>> ord("A"), chr(66) # code du caractère
(65, 'B')
```

On parcourt une chaîne caractère par caractère avec `for lettre in mot`, ou par indice avec `for i in range(len(mot))`. Pour tester un début ou une fin de chaîne, on compare une tranche : `mot[:3] == "bon"`. Pour mêler texte et valeurs, on utilise une **f-string** : `f"{prenom} a {age} ans"` donne `'Ada a 17 ans'` si `prenom` vaut `"Ada"` et `age` vaut 17.

```
def inverser(mot):
    """Renvoie la chaîne mot écrite à l'envers."""
    resultat = ""                  # chaîne vide au départ
    for lettre in mot:
        resultat = lettre + resultat
    return resultat
```

```
>>> inverser("NSI")
'ISN'
>>> inverser("radar")
'radar'
```

▷ **Piège classique.** comparer des nombres écrits en chaînes. Les chaînes se comparent caractère par caractère, dans l'ordre du dictionnaire : `"10" < "9"` vaut `True`, car `"1"` vient avant `"9"`. On convertit avec `int` avant de comparer ou de calculer.

Les tableaux (listes Python)

Un **tableau** (type `list`) est une suite **ordonnée** et **modifiable** (*mutable*) d'éléments, écrite entre crochets. Mêmes règles d'indices que pour les chaînes : de `0` à `len(t) - 1`, indices négatifs depuis la fin, tranches `t[a:b]` (`b` exclu) qui fabriquent un *nouveau* tableau. `t.append(x)` ajoute `x` à la fin.

```
>>> notes = [12, 8, 15, 9]
>>> notes[0], notes[-1]
(12, 9)
>>> notes[1:3]
[8, 15]
>>> notes[1] = 20      # modification
>>> notes.append(17)   # ajout à la fin
>>> notes
[12, 20, 15, 9, 17]
>>> len(notes)
5
```

```
>>> [n + 1 for n in notes if n < 16]
[13, 16, 10]
>>> [0 for i in range(4)]
[0, 0, 0, 0]
>>> M = [[1, 2, 3], [4, 5, 6]]
>>> M[1][2]            # ligne 1, colonne 2
6
>>> len(M), len(M[0])  # lignes, colonnes
(2, 3)
```

Deux parcours. Par valeur, `for x in t`, quand seules les valeurs comptent ; **par indice**, `for i in range(len(t))`, quand on a besoin de la position ou qu'on veut *modifier* le tableau (`t[i] = ...`). Réaffecter la variable `x` d'un parcours par valeur ne change pas le tableau.

Construction par compréhension. `[expression for x in itérable if condition]` construit un tableau en une ligne (la partie `if` est facultative).

Tableaux de tableaux. Une **matrice** de `n` lignes et `p` colonnes est un tableau de `n` lignes, chacune un tableau de longueur `p` : `M[i]` est la ligne `i`, `M[i][j]` la case ligne `i`, colonne `j`. On la parcourt avec deux boucles imbriquées, et on la crée par compréhension : `[[0 for j in range(p)] for i in range(n)]`.

Une variable désigne un objet. `b = a` ne copie pas le tableau : `a` et `b` deviennent deux noms (des **alias**) du *même* tableau. Pour une vraie copie : `b = [x for x in a]` (ou `a[:]`).

```
>>> a = [1, 2, 3]
>>> b = a              # alias
>>> b[0] = 99
>>> a                  # a a changé aussi !
[99, 2, 3]
```

```
>>> c = [x for x in a] # vraie copie
>>> c[0] = 0
>>> a                  # a n'a pas bougé
[99, 2, 3]
```

▷ **Piège classique.** sortir du tableau : le dernier indice est `len(t) - 1`, et `t[len(t)]` provoque une `IndexError`. Autre piège : `[[0] * p] * n` crée `n` fois la *même* ligne ; modifier une case modifie alors toute la colonne. On écrit une compréhension.

Les p-uplets

Un **p-uplet** (type `tuple`) est une suite **ordonnée** et **immuable** : une fois créé, on ne peut plus le modifier. Ce sont les **virgules** qui le créent (les parenthèses sont conseillées) ; un p-uplet d'un seul élément s'écrit `(3,)`. On y accède comme à un tableau : indices, tranches, `len`, `in`. Il sert à regrouper des données figées (coordonnées, date) et à **renvoyer plusieurs valeurs** d'une fonction. Comme il est immuable, il peut servir de clé de dictionnaire.

```
>>> point = (3, 4)
>>> x, y = point        # déballage
>>> x + y
7
>>> point[0] = 5
TypeError: 'tuple' object does not
support item assignment
>>> a, b = 1, 2
>>> a, b = b, a         # échange
>>> a, b
(2, 1)
```

```
def division(a, b):
    """Renvoie le couple (quotient, reste)
    de la division entière de a par b."""
    return a // b, a % b

q, r = division(25, 7)
print(q, r)          # affiche 3 4
```

▷ **Piège classique.** écrire `(3)` pour un p-uplet d'un élément : c'est simplement l'entier `3`. Et vouloir modifier un p-uplet : si les données doivent changer, on choisit un tableau.

Les dictionnaires

Un **dictionnaire** (type `dict`) est un ensemble de couples **clé** → **valeur**, écrit entre accolades. On accède à une valeur par sa **clé**, et non par une position. Les clés sont **uniques** et **immuables** (chaînes, entiers, p-uplets ; jamais un tableau). Affecter `d[cle] = valeur` crée la clé si elle n'existe pas, et remplace sa valeur sinon.

```
>>> age = {"Alice": 17, "Bob": 16}
>>> age["Alice"]
17
>>> age["David"] = 15     # nouvelle clé
>>> age["Bob"] = 17       # valeur remplacée
>>> age
{'Alice': 17, 'Bob': 17, 'David': 15}
>>> "Bob" in age          # une clé ?
True
>>> 17 in age             # in teste les CLÉS
False
>>> age["Zoé"]
KeyError: 'Zoé'
```

Parcourir. `for cle in d` parcourt les **clés** ; `d.keys()`, `d.values()` et `d.items()` donnent les clés, les valeurs et les couples `(clé, valeur)`.

```
for prenom, a in age.items():
    print(f"{prenom} a {a} ans")
```

Affichage :

```
Alice a 17 ans
Bob a 17 ans
David a 15 ans
```

Le cas d'école : compter des apparitions. La clé est l'objet compté, la valeur son compteur.

```
def compter(mot):
    """Renvoie le dictionnaire lettre -> nombre d'apparitions."""
    compteur = {}
    for lettre in mot:
        if lettre in compteur:
            compteur[lettre] = compteur[lettre] + 1
        else:
            compteur[lettre] = 1
    return compteur
```

```
>>> compter("abracadabra")
{'a': 5, 'b': 2, 'r': 2,
 'c': 1, 'd': 1}
```

▷ **Piège classique.** lire `d[cle]` sans savoir si la clé existe : `KeyError`. Le réflexe : tester `if cle in d` d'abord, comme dans `compter`.

Le parcours séquentiel

Un **parcours séquentiel** examine les éléments d'un tableau un par un, du premier au dernier. Presque tous suivent le même patron : **initialiser** une variable, la **mettre à jour** à chaque élément, la **renvoyer** à la fin. L'initialisation dépend du calcul : `0` pour une somme ou un compteur, le **premier élément** pour un maximum ou un minimum (le « champion »).

```
def somme(t):
    """Renvoie la somme des nombres de t."""
    s = 0
    for x in t:
        s = s + x
    return s

def maximum(t):
    """Renvoie le plus grand élément de t,
    supposé non vide."""
    m = t[0]            # jamais 0 !
    for x in t:
        if x > m:
            m = x
    return m
```

```
def indice(t, x):
    """Renvoie l'indice de la première
    occurrence de x dans t, -1 si absent."""
    for i in range(len(t)):
        if t[i] == x:
            return i    # trouvé : on s'arrête
    return -1           # parcours fini : absent

def nb_occurrences(t, x):
    """Renvoie le nombre d'apparitions de x."""
    n = 0
    for element in t:
        if element == x:
            n = n + 1
    return n
```

Avec `t = [-3, -1, -7, -1]` : `somme(t)` renvoie `-12`, `maximum(t)` renvoie `-1`, `indice(t, -1)` renvoie `1`, `indice(t, 5)` renvoie `-1` et `nb_occurrences(t, -1)` renvoie `2`. Pour signaler un élément absent, beaucoup d'énoncés renvoient `None` plutôt que `-1` : on applique la convention de l'énoncé.

À retenir — Le coût linéaire

Le **coût** d'un algorithme est le nombre d'opérations élémentaires qu'il effectue, exprimé en fonction de la taille `n` des données. Un parcours fait un nombre d'opérations **proportionnel à `n`** : son coût est **linéaire**. Si `n` double, le temps double. Une recherche s'arrête dès qu'elle trouve (**meilleur cas** : 1 comparaison), mais dans le **pire cas** (élément absent) elle lit tout le tableau : n comparaisons. C'est le pire cas que l'on retient.

Invariant de boucle. Pour prouver qu'une boucle calcule le bon résultat, on exhibe une propriété vraie avant la boucle et conservée à chaque tour. Pour `maximum` : « `m` est le plus grand des éléments déjà examinés ». Après le dernier tour, tous ont été examinés : `m` est le maximum du tableau.

▷ **Piège classique.** initialiser le maximum à `0` : avec `[-3, -1, -7, -1]`, la fonction renverrait `0`, une valeur absente du tableau. Autre piège : écrire `return -1` (ou `return False`) *dans* la boucle, sous un `else` : la recherche s'arrête alors dès le premier élément.

Les tris par sélection et par insertion

Trier un tableau, c'est ranger ses éléments dans l'ordre croissant. Les deux tris de Première trient **en place** : ils modifient le tableau reçu, par comparaisons et échanges, et ne renvoient rien.

Sélection. Pour chaque position `i`, on cherche le **minimum** de la partie non triée (de `i` à la fin) et on l'**échange** avec la case `i`.

```
def tri_selection(t):
    """Trie t en place, ordre croissant."""
    n = len(t)
    for i in range(n):
        i_min = i
        for j in range(i + 1, n):
            if t[j] < t[i_min]:
                i_min = j
        temp = t[i]           # échange
        t[i] = t[i_min]
        t[i_min] = temp
```

Insertion. On prend les éléments un à un et on **insère** chacun à sa place dans la partie gauche, déjà triée, en **décalant** vers la droite les plus grands.

```
def tri_insertion(t):
    """Trie t en place, ordre croissant."""
    n = len(t)
    for i in range(1, n):
        x = t[i]        # élément à insérer
        j = i - 1
        while j >= 0 and t[j] > x:
            t[j + 1] = t[j]   # décalage
            j = j - 1
        t[j + 1] = x
```

Dérouler sur `[7, 3, 9, 1, 5]` (état du tableau à la fin de chaque tour de la boucle `for`) :

	après 1 tour	après 2 tours	après 3 tours	après 4 tours
Sélection	`[1, 3, 9, 7, 5]`	`[1, 3, 9, 7, 5]`	`[1, 3, 5, 7, 9]`	`[1, 3, 5, 7, 9]`
Insertion	`[3, 7, 9, 1, 5]`	`[3, 7, 9, 1, 5]`	`[1, 3, 7, 9, 5]`	`[1, 3, 5, 7, 9]`

Invariants. Sélection : après `i` tours, les `i` premières cases contiennent les `i` plus petits éléments, triés et à leur place **définitive**. Insertion : après le tour d'indice `i`, les cases `0` à `i` contiennent les éléments de départ de ces cases, rangés dans l'ordre. Dans le `while`, `j >= 0` est testé *avant* `t[j]` : l'évaluation séquentielle de `and` évite de lire une case qui n'existe pas.

Coût quadratique. Les deux tris placent une boucle dans une boucle : le nombre de comparaisons est de l'ordre de n^2 (exactement $\frac{n(n-1)}{2}$ pour la sélection). Leur coût est **quadratique** : si `n` double, le temps est multiplié par 4. Atout de l'insertion : sur un tableau déjà (presque) trié, le `while` ne tourne presque pas, et le coût devient linéaire. En pratique, Python fournit `t.sort()` (en place) et `sorted(t)` (nouveau tableau), bien plus rapides.

▷ **Piège classique.** échanger sans variable temporaire : `t[i] = t[j]` puis `t[j] = t[i]` perd l'ancienne valeur de `t[i]`. On passe par `temp`, ou on écrit `t[i], t[j] = t[j], t[i]`.

La recherche dichotomique

Précondition : le tableau est trié. On compare l'élément cherché à celui du **milieu** de la zone de recherche, et l'on élimine la moitié où il ne peut pas être ; on recommence jusqu'à le trouver ou jusqu'à ce que la zone soit vide.

```python
def recherche_dichotomique(t, x):
    """Renvoie True si x est dans le tableau
    TRIÉ t, False sinon."""
    deb = 0
    fin = len(t) - 1
    while deb <= fin:          # zone non vide
        mil = (deb + fin) // 2
        if t[mil] == x:
            return True
        elif x > t[mil]:
            deb = mil + 1      # moitié droite
        else:
            fin = mil - 1      # moitié gauche
    return False               # zone vide : absent
```

Dérouler. Recherche de `23` dans `[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]` :

deb	fin	mil	t[mil]
0	9	4	16
5	9	7	56
5	6	5	23

Trois tours, puis `True`. Pour renvoyer la position, on renvoie `mil` au lieu de `True`, et `-1` (ou `None`) au lieu de `False`.

Terminaison. La quantité `fin - deb` est un **variant** : c'est un entier qui diminue strictement à chaque tour. Elle finit par devenir négative (`deb > fin`), et la boucle s'arrête si l'élément n'a pas été trouvé avant.

Coût logarithmique. Chaque tour divise la zone par deux : sur `n` éléments, il y a au plus de l'ordre de $\log_2 n$ tours (le nombre de fois qu'on peut diviser `n` par 2 avant d'arriver à 1). Le coût est **logarithmique** : doubler la taille du tableau n'ajoute qu'**un** tour.

Taille `n`	10	1 000	1 000 000	1 000 000 000
Recherche séquentielle (coût linéaire), pire cas	10	1 000	1 000 000	1 000 000 000
Recherche dichotomique (coût logarithmique), pire cas	4	10	20	30

Pour comparer les ordres de grandeur : un coût **quadratique** (n^2) ferait 10^{12} opérations pour un million d'éléments ; un coût **exponentiel** (2^n, qui n'est pas au programme de Première et que l'on rencontre dès le chapitre 1 de cette année avec la suite de Fibonacci) devient inaccessible dès que `n` dépasse quelques dizaines.

▷ **Piège classique.** appliquer la dichotomie à un tableau non trié : `recherche_dichotomique([5, 1, 8, 3, 9], 1)` renvoie `False`, alors que `1` est présent. Autre piège : oublier le `+ 1` ou le `- 1` dans `deb = mil + 1` et `fin = mil - 1`, ce qui peut créer une boucle infinie.

Représenter les données en binaire

Bit, octet, bases. Un **bit** vaut `0` ou `1` ; un **octet** regroupe 8 bits. Avec n bits, on écrit 2^n valeurs : un octet code les entiers de 0 à 255. En base b, chaque rang pèse une puissance de b. En binaire (base 2), les poids sont, de droite à gauche, $1, 2, 4, 8, 16, \ldots$: $\texttt{101101}_2 = 32+8+4+1 = 45$. Pour passer du décimal au binaire, on fait des **divisions successives par 2** et on lit les restes **de bas en haut**. L'**hexadécimal** (base 16) utilise les chiffres `0` à `9` puis `A` à `F` (`A` vaut 10, …, `F` vaut 15) : un chiffre hexadécimal correspond à **4 bits**, on regroupe donc les bits par paquets de 4 : $\texttt{1101 1001}_2 = \texttt{D9}_{16} = 217$.

```
>>> bin(13)            # vers le binaire
'0b1101'
>>> 0b1101
13
>>> hex(217)           # vers l'hexadécimal
'0xd9'
>>> int("D9", 16)
217
```

Entiers relatifs : le complément à deux. Pour coder $-x$ sur n bits : écrire x sur n bits, **inverser** tous les bits, puis **ajouter 1**. Sur 8 bits :

$$12 = \texttt{00001100} \xrightarrow{\text{inverser}} \texttt{11110011} \xrightarrow{+1} \texttt{11110100} = -12.$$

Le bit de poids fort donne le signe (`1` : négatif). Sur n bits, on code de -2^{n-1} à $2^{n-1}-1$: de -128 à 127 sur un octet. Pour décoder `11111010` : il commence par `1`, on inverse et on ajoute 1, on obtient `00000110`, soit 6 : il code -6.

Flottants. Un nombre à virgule est stocké en binaire sur un nombre fini de bits, souvent de façon **approchée** : 0,1 n'a pas d'écriture binaire finie. Ainsi `0.1 + 0.2` vaut `0.30000000000000004` en Python, et `0.1 + 0.2 == 0.3` vaut `False`. On ne teste donc jamais l'égalité de deux flottants avec `==` : on vérifie que leur écart est très petit, par exemple `abs(a - b) < 1e-9`.

Booléens et portes logiques. Une valeur booléenne vaut `1` (vrai) ou `0` (faux). Les opérateurs **non**, **et**, **ou** (en Python `not`, `and`, `or`) et le **ou exclusif xor** (« exactement un des deux ») se résument par leur **table de vérité**. Dans le processeur, chacun est réalisé par une **porte logique** de quelques transistors ; en assemblant des portes, on obtient par exemple le **demi-additionneur** (somme = a xor b, retenue = a et b). Une expression à n variables a une table de 2^n lignes.

a	b	non a	a et b	a ou b	a xor b
0	0	1	0	0	0
0	1	1	0	1	1
1	0	0	0	1	1
1	1	0	1	1	0

▷ **Piège classique.** se tromper de sens : le bit de *droite* pèse 1, et les restes des divisions successives se lisent de bas en haut. Et confondre ou et xor : 1 ou 1 vaut 1, mais 1 xor 1 vaut 0.

Les données en tables

Une **table** range des données en lignes et en colonnes. Chaque colonne porte un **descripteur** (son nom) ; chaque ligne est un **enregistrement**, qui donne une valeur pour chaque descripteur. Un fichier **CSV** est un simple fichier texte : la première ligne (l'**en-tête**) donne les descripteurs, chaque ligne suivante un enregistrement, les valeurs étant séparées par un **séparateur** (`,` ou `;`). En Python, une table est une **liste de dictionnaires** : un enregistrement est un dictionnaire `{descripteur: valeur}`.

Le fichier `prenoms.csv` *:*

```
prenom,sexe,annee,nombre
Louise,F,2023,3177
Ambre,F,2023,3168
Alba,F,2023,3088
Jade,F,2023,2891
Gabriel,G,2023,4527
```

```
import csv

def charger(fichier):
    """Renvoie la table du fichier CSV,
    sous forme de liste de dictionnaires."""
    with open(fichier, encoding="utf-8", newline="") as f:
        lecteur = csv.DictReader(f)
        table = [ligne for ligne in lecteur]
    return table
```

`csv.DictReader` utilise l'en-tête comme clés ; pour un séparateur point-virgule, on écrit `csv.DictReader(f, delimiter=";")`. Ensuite, **sélectionner** des lignes, c'est un parcours qui garde celles qui vérifient un critère ; **trier** selon une colonne se fait avec `sorted` et le paramètre `key`, qui indique la valeur à comparer pour chaque ligne (`reverse=True` : ordre décroissant ; `sorted` renvoie une nouvelle liste).

```
def selection(table, sexe):
    """Renvoie les lignes de table dont
    le descripteur sexe vaut sexe."""
    resultat = []
    for ligne in table:
        if ligne["sexe"] == sexe:
            resultat.append(ligne)
    return resultat

def trier_par_nombre(table):
    """Renvoie la table triée par nombre
    décroissant."""
    return sorted(table, reverse=True,
        key=lambda ligne: int(ligne["nombre"]))
```

```
>>> prenoms = charger("prenoms.csv")
>>> len(prenoms)
5
>>> prenoms[0]
{'prenom': 'Louise', 'sexe': 'F',
 'annee': '2023', 'nombre': '3177'}
>>> prenoms[2]["nombre"]
'3088'
>>> [l["prenom"] for l in
...  selection(prenoms, "F")]
['Louise', 'Ambre', 'Alba', 'Jade']
>>> [l["prenom"] for l in
...  trier_par_nombre(prenoms)][:2]
['Gabriel', 'Louise']
```

La sélection s'écrit aussi par compréhension : `[l for l in prenoms if l["sexe"] == "F"]`. On accède à une valeur en deux temps : la ligne par son **indice**, puis la valeur par son **descripteur** (`prenoms[0]["prenom"]` vaut `'Louise'`). Calculer une moyenne, un maximum ou un effectif sur une colonne, c'est le patron du parcours séquentiel, de coût linéaire.

▷ **Piège classique.** oublier que tout ce qui sort d'un CSV est du **texte** : `prenoms[0]["nombre"] + prenoms[1]["nombre"]` vaut `'31773168'` (concaténation). On convertit avec `int(...)` avant de calculer ou de trier ; sans conversion, `"900"` serait classé après `"3070"` dans l'ordre croissant.

Spécifier et tester un programme

Spécifier une fonction, c'est décrire son contrat sans dire comment elle s'y prend : son **prototype** (nom, paramètres, ce qu'elle renvoie), ses **préconditions** (ce qui doit être vrai sur les arguments avant l'appel) et ses **postconditions** (ce qui est garanti sur le résultat après l'appel). La spécification s'écrit dans la **docstring**, une chaîne entre triples guillemets placée juste après le `def` et affichée par `help`.

`assert`. L'instruction `assert condition, "message"` interrompt le programme avec une `AssertionError` si la condition est fausse. Elle sert à **vérifier une précondition** à l'entrée d'une fonction (programmation défensive : on échoue tôt et clairement), et à écrire des **tests**. Autre stratégie défensive : renvoyer une valeur sentinelle, `None`, que l'appelant teste avec `is None`.

```
def moyenne(notes):
    """Renvoie la moyenne des nombres de notes.
    Précondition : notes est non vide."""
    assert len(notes) > 0, "la liste est vide"
    total = 0
    for note in notes:
        total = total + note
    return total / len(notes)
```

```
>>> moyenne([12, 15, 9])
12.0
>>> moyenne([])
AssertionError: la liste est vide
```

```
def indice_maximum(t):
    """Renvoie l'indice du plus grand élément
    de t (le premier en cas d'égalité),
    ou None si t est vide."""
    if len(t) == 0:
        return None
    ind_max = 0
    for i in range(1, len(t)):
        if t[i] > t[ind_max]:
            ind_max = i
    return ind_max
```

Un jeu de tests se place dans le fichier, après la fonction. Il couvre chaque cas de la spécification (dont le tableau **vide**), les **valeurs limites** (premier et dernier indice, nombres négatifs, 0) et les **cas particuliers** (égalités, doublons). Si un test échoue, on corrige, puis on relance tous les tests.

```
1 assert indice_maximum([2, 3, 1]) == 1
2 assert indice_maximum([]) is None
3 assert indice_maximum([8, 3, 3, 7]) == 0        # maximum au début
4 assert indice_maximum([3, 1, 3, 7]) == 3        # maximum à la fin
5 assert indice_maximum([5, 9, 9]) == 1           # égalité : le premier
```

```
6 assert indice_maximum([-3, -1, -3, -7]) == 1    # tous négatifs
```

Des tests réussis ne prouvent pas qu'un programme est correct : il peut échouer sur un cas non testé. Seul un raisonnement le prouve. Deux outils :

- un **variant de boucle** prouve la **terminaison** d'une boucle `while` : c'est une quantité entière, positive, qui **décroît strictement** à chaque tour. Une telle quantité ne peut pas décroître indéfiniment, donc la boucle s'arrête. Dans `nb_chiffres`, `n` est un variant (`n // 10 < n` dès que `n` $\geqslant 1$) ;
- un **invariant de boucle** prouve la **correction** : vrai avant la boucle et conservé à chaque tour, il est encore vrai à la sortie (exemple du maximum, rubrique « Le parcours séquentiel »).

▷ **Piège classique.** confondre précondition (sur les arguments, avant l'appel) et postcondition (sur le résultat, après l'appel), et oublier le cas du tableau vide dans les tests. Avec la convention `-1` pour « absent », attention : `t[-1]` est un indice valide (le dernier élément), il faut tester le résultat avant de s'en servir.

Le Web et le réseau

Internet et le Web. Internet est le réseau mondial qui relie les machines ; le **Web** est l'un des services qui l'utilisent : des pages reliées par des liens, consultées avec un navigateur. Un **protocole** est un ensemble de règles que deux machines respectent pour communiquer.

Client et serveur. Dans le modèle **client-serveur**, le client *demande* une ressource, le serveur la *fournit* (le navigateur est un client, il interroge des serveurs web). Une ressource est désignée par son **URL**, par exemple `https://www.lycee.mc/nsi/cours.html` : le protocole (`https`), le nom de domaine du serveur (`www.lycee.mc`), le chemin de la ressource (`/nsi/cours.html`). Le **DNS** est l'annuaire qui traduit le nom de domaine en adresse IP.

HTTP. Le client envoie une **requête**, le serveur renvoie une **réponse** qui commence par un **code de statut** : `200` (succès), `404` (ressource introuvable), `500` (erreur du serveur)… Deux méthodes à connaître :

- `GET` **demande** une ressource ; les paramètres éventuels sont **visibles dans l'URL** ;
- `POST` **envoie** des données au serveur (un formulaire), qui ne figurent **pas** dans l'URL.

`https` est la version **chiffrée** de HTTP. Afficher une page demande souvent des dizaines de requêtes (HTML, CSS, scripts, images).

Paquets et protocoles TCP/IP. Un message est découpé en **paquets** qui voyagent indépendamment et sont réassemblés à l'arrivée. Chaque couche de protocoles ajoute son en-tête au message qu'elle reçoit : c'est l'**encapsulation**.

Couche (protocole)	Ce que son en-tête apporte
Application (HTTP)	la requête elle-même, par exemple `GET /index.html`
Transport (TCP)	les **ports** (le service visé : 80 pour HTTP, 443 pour HTTPS), un numéro d'ordre ; TCP renvoie les paquets perdus grâce aux **accusés de réception**
Réseau (IP)	les **adresses IP** de la source et de la destination
Liaison (Ethernet, Wi-Fi)	les adresses **MAC** (physiques) des deux machines voisines sur le trajet

Adresses IP et routeurs. Une **adresse IP** identifie une machine sur le réseau ; en IPv4, ce sont quatre nombres de 0 à 255 (quatre octets), par exemple `193.51.208.14`. Une adresse **publique** est unique sur Internet ; une adresse **privée** (`192.168.x.x`, `10.x.x.x`) ne l'est que dans un réseau local. Un **commutateur** (*switch*) relie les machines d'un même réseau local ; un **routeur** relie des réseaux entre eux : il lit l'adresse IP de destination de chaque paquet et l'envoie vers le voisin qui le rapproche du but. Internet est l'ensemble de ces routeurs interconnectés.

▷ **Piège classique.** envoyer un mot de passe avec `GET` : il apparaît dans l'URL et dans l'historique. On utilise `POST`, sur une connexion `https`. Autre piège : croire qu'une vérification faite dans le navigateur (en JavaScript, côté client) est sûre ; le serveur doit toujours vérifier à nouveau.

Architecture et système d'exploitation

Le modèle de von Neumann (1945), suivi par presque tous les ordinateurs, comporte quatre unités :

- l'**unité de commande**, qui lit les instructions du programme, les décode et ordonne leur exécution ;
- l'**unité arithmétique et logique** (UAL), qui calcule (additions, comparaisons, opérations logiques) ;
- la **mémoire**, qui range à la fois le **programme** et les **données**, dans des cases repérées par une adresse ;
- les **entrées-sorties**, qui relient la machine au monde (clavier, écran, disque, réseau, capteurs et actionneurs).

L'unité de commande et l'UAL forment le **processeur** (CPU) ; les unités sont reliées par des **bus**. Le processeur répète le cycle **chercher** l'instruction, la **décoder**, l'**exécuter**. Il n'exécute que du **langage machine** (des instructions simples codées en binaire ; l'assembleur en est la version lisible). Plus une mémoire est rapide, plus elle est petite : les **registres** (dans le processeur), puis la **mémoire vive** (RAM, volatile : effacée à l'extinction), puis le **disque** (permanent, plus lent).

Le système d'exploitation (OS : Linux, Windows, macOS, Android…) est l'ensemble des programmes qui gèrent le matériel et le mettent à disposition des applications : il gère le **processeur** (quel programme s'exécute, et quand), la **mémoire**, les **fichiers**, les **périphériques** et les **utilisateurs** (comptes, droits d'accès).

La ligne de commande Linux. Les fichiers forment un **arbre** dont la racine est `/`. Un chemin **absolu** part de la racine (`/home/eleve/Documents/cours.txt`) ; un chemin **relatif** part du répertoire courant (`../Images/logo.png`). `.` désigne le répertoire courant, `..` le répertoire parent, `~` le répertoire personnel.

Commande	Rôle	Commande	Rôle
`pwd`	afficher le répertoire courant	`cat f`	afficher le contenu du fichier `f`
`ls`, `ls -l`	lister (en détail : droits, taille)	`cp a b`	copier `a` vers `b`
`cd chemin`	changer de répertoire	`mv a b`	déplacer ou renommer
`mkdir rep`	créer un répertoire	`rm f`, `rm -r rep`	supprimer (définitivement)
`touch f`	créer un fichier vide	`man cmd`	afficher le manuel d'une commande
`chmod`	modifier les droits		

Droits. Chaque fichier porte trois droits, `r` (lecture), `w` (écriture), `x` (exécution), accordés séparément à trois catégories : `u` (le propriétaire), `g` (le groupe), `o` (les autres). `ls -l` les affiche après le type (`-` pour un fichier, `d` pour un répertoire), par groupes de trois.

```
-rwxr-xr--  1 eleve  eleve   58  8 sep 09:12 script.sh
```

Ici le propriétaire a `rwx`, le groupe `r-x`, les autres `r--`. `chmod u+x f` ajoute l'exécution au propriétaire, `chmod o-r f` retire la lecture aux autres. En chiffres, `r` vaut 4, `w` vaut 2, `x` vaut 1, et chaque catégorie s'écrit avec la somme de ses droits : `chmod 754 script.sh` donne `rwxr-xr--` ($7 = 4+2+1$, $5 = 4+1$, 4). L'administrateur `root` a tous les droits.

▷ **Piège classique.** oublier le `/` initial d'un chemin absolu (ou en mettre un à un chemin relatif), et oublier que Linux distingue majuscules et minuscules : `Documents` et `documents` sont deux répertoires différents. Et `rm` ne passe pas par une corbeille : ce qui est supprimé est perdu.

Lexique

Les définitions sont résumées ; le renvoi (chapitre et page, cliquable) mène à la définition complète dans le cours. Les mots du langage Python sont regroupés à part, en fin de lexique.

A

Accesseur Méthode qui lit un attribut et renvoie sa valeur sans le modifier (souvent nommée `get_...` dans les sujets de bac). ch. 2, p. 76

Accumulateur Paramètre supplémentaire qui transporte d'un appel à l'autre le résultat en cours de construction ; on lui donne en général une valeur par défaut. ch. 1, p. 25

Actionneur Composant par lequel un système embarqué agit sur le monde physique (moteur, chauffage, LED, frein...). ch. 14, p. 676

Adjacents (voisins) Deux sommets reliés par une arête sont adjacents, ou voisins. Dans un graphe orienté, les voisins d'un sommet sont ses successeurs. ch. 8, p. 390

Adresse IP Adresse d'une interface réseau ; en IPv4, quatre octets (32 bits), par exemple `172.16.5.3`, formés d'une partie réseau et d'une partie hôte. ch. 9, p. 443

AES *Advanced Encryption Standard* (2001) : chiffrement symétrique standard d'aujourd'hui, par blocs de 128 bits, avec une clé de 128, 192 ou 256 bits. ch. 10, p. 486

Algorithme de Boyer-Moore Recherche textuelle (1977) qui compare le motif de droite à gauche et, après prétraitement du motif, saute plusieurs crans d'un coup (règle du mauvais caractère). ch. 12, p. 587

Algorithme de Dijkstra (Hors programme.) Recherche du plus court chemin dans un graphe pondéré, en fixant les sommets par coût total croissant ; utilisé par le protocole OSPF. ch. 8, p. 402 ; ch. 9, p. 450

Algorithme glouton Algorithme qui fait à chaque étape le choix qui semble le meilleur (la plus grosse pièce) ; rapide, mais pas toujours optimal. ch. 11, p. 535

Anomalies (d'un mauvais schéma) Incohérences dues à la redondance (une même information recopiée plusieurs fois) : de mise à jour (une copie oubliée), d'insertion (impossible d'enregistrer une donnée seule), de suppression (perte d'information). ch. 6, p. 286

API (interface de programmation) Liste de ce qu'une bibliothèque met à disposition (fonctions, classes, constantes) avec leur spécification : on s'en sert sans avoir besoin de lire leur code. ch. 2, p. 81

Arbre Structure hiérarchique de nœuds reliés par des arêtes, issue d'une racine. Vu comme graphe, c'est un graphe connexe sans cycle. ch. 5, p. 230 ; ch. 8, p. 392

Arbre binaire Arbre dont chaque nœud a au plus deux fils, appelés fils gauche et fils droit (l'un ou les deux peuvent manquer). ch. 5, p. 230

Arbre binaire de recherche (ABR) Arbre binaire où, pour chaque nœud, les valeurs du sous-arbre gauche sont strictement plus petites et celles du droit plus grandes ou égales (doublon à droite). ch. 5, p. 235

Arbre des appels Schéma de tous les appels déclenchés par un appel récursif ; celui de `fib(5)` montre que les mêmes calculs sont refaits un grand nombre de fois. ch. 1, p. 27

Arc Lien orienté d'un sommet vers un autre dans un graphe orienté : il ne se parcourt que dans le sens de la flèche. ch. 8, p. 390

Architecture de Harvard (Hors programme.) Architecture où programme et données sont dans deux mémoires séparées, reliées par deux bus distincts : on lit une instruction et une donnée en même temps. ch. 14, p. 676

Arête Lien entre deux nœuds d'un arbre (ou branche) ; dans un graphe non orienté, lien entre deux sommets, qui se parcourt dans les deux sens. ch. 5, p. 230 ; ch. 8, p. 390

Attaque par force brute Attaque qui essaie toutes les clés possibles ; elle est imparable quand les clés sont peu nombreuses (26 pour César). ch. 10, p. 485

Attribut (d'un objet) Donnée attachée à une instance, qui décrit son état ; on y accède par la notation pointée (`gollum.vie`), et dans la classe par `self.vie`. ch. 2, p. 72

Attribut (d'une relation) Colonne d'une relation : il porte un nom et un domaine (son type : entier, texte, date...). ch. 6, p. 283

B

Boyer-Moore-Horspool Version simplifiée (1980) : le saut ne dépend que du caractère du texte aligné avec la dernière case de la fenêtre, lu dans une seule table de décalage. ch. 12, p. 589

C

Capteur Composant qui mesure une grandeur physique (température, vitesse, lumière…) et la transmet au système embarqué. ch. 14, p. 676

Cas récursif Cas où la fonction s'appelle elle-même sur un problème plus petit, qui se rapproche de la condition d'arrêt, puis combine le résultat de cet appel. ch. 1, p. 21

Certificat « Carte d'identité » d'un serveur, contenant sa clé publique et signée par une autorité de certification que le navigateur connaît déjà. ch. 10, p. 491

Chaîne Dans un graphe non orienté, suite de sommets reliés de proche en proche par des arêtes ; sa longueur est son nombre d'arêtes. ch. 8, p. 390

Chemin Dans un graphe orienté, suite de sommets reliés par des arcs pris dans le bon sens. ch. 8, p. 391

Chiffre de César Chiffrement par décalage : la lettre de rang x devient celle de rang `(x + k) % 26`. Seulement 26 clés : il cède à la force brute. ch. 10, p. 484

Chiffre de Vigenère Chiffrement (XVIe siècle) par décalages variables, donnés par une clé-mot répétée : une même lettre n'est plus toujours chiffrée de la même façon. ch. 10, p. 485

Chiffrement affine La lettre de rang x devient celle de rang `(a * x + b) % 26`, avec a premier avec 26 ; on déchiffre grâce à l'inverse modulaire de a. ch. 10, p. 485

Chiffrement asymétrique (à clé publique) Chiffrement où chacun possède une paire de clés : ce que chiffre la clé publique, seule la clé privée associée le déchiffre. Connaître la clé publique ne livre pas la privée. ch. 10, p. 487

Chiffrement symétrique Chiffrement où la même clé sert à chiffrer et à déchiffrer ; il est rapide, mais les deux interlocuteurs doivent d'abord partager ce secret commun. ch. 10, p. 484

Chiffrer, déchiffrer, décrypter Chiffrer : rendre un message clair illisible à l'aide d'une clé. Déchiffrer : le retrouver avec la clé. Décrypter : le retrouver sans la clé (cryptanalyse, le travail de l'attaquant). ch. 10, p. 483

Classe Modèle (le « plan ») qui décrit les attributs et les méthodes communs à toute une famille d'objets ; en Python, elle se déclare avec `class`. ch. 2, p. 72

Classe NP (Hors programme.) Problèmes de décision dont on peut vérifier une solution proposée en temps polynomial (« non déterministe polynomial ») ; P = NP ? reste une question ouverte. ch. 13, p. 634

Classe P (Hors programme.) Problèmes que l'on sait résoudre par un algorithme en temps polynomial (n^2, n^3…), c'est-à-dire en un temps raisonnable. ch. 13, p. 634

Clé (de chiffrement) Information secrète (ou publique, en asymétrique) qui paramètre le chiffrement ; la sécurité d'un code ne doit reposer que sur le secret de la clé. ch. 10, p. 483

Clé étrangère Attribut dont la valeur doit être celle d'une clé primaire d'une autre (ou de la même) relation : elle relie deux tables et garantit que la référence existe. ch. 6, p. 285

Clé primaire Attribut (ou groupe d'attributs) dont la valeur identifie de façon unique chaque enregistrement ; elle ne se répète jamais et n'est jamais vide (contrainte d'entité). ch. 6, p. 284

Clé privée Clé de la paire asymétrique gardée secrète et jamais partagée : elle déchiffre les messages reçus et sert à signer. ch. 10, p. 487

Clé publique Clé de la paire asymétrique que l'on distribue à qui veut (comme un cadenas ouvert) : elle sert à chiffrer un message pour son propriétaire, ou à vérifier sa signature. ch. 10, p. 487

Commutation de contexte Passage du processeur d'un processus à un autre ; répétée des milliers de fois par seconde, elle donne l'illusion que tout s'exécute en même temps. ch. 4, p. 183

Commutation de paquets Principe d'Internet : les messages sont découpés en paquets indépendants qui empruntent chacun leur chemin ; les liens sont partagés et les pannes contournées. ch. 9, p. 445

Condition d'arrêt (cas de base) Cas où une fonction récursive répond directement, sans s'appeler. Elle est obligatoire et doit être réellement atteinte, sinon les appels ne s'arrêtent jamais. ch. 1, p. 21

Conditions de Coffman (Hors programme.) Les quatre conditions simultanées d'un interblocage : exclusion mutuelle, détention et attente, non-préemption, attente circulaire ; en casser une suffit. ch. 4, p. 186

Constructeur Méthode spéciale `__init__`, appelée automatiquement à la création d'une instance, qui met en place ses attributs. ch. 2, p. 73

Contrainte de domaine Chaque valeur d'un attribut doit appartenir à son domaine, c'est-à-dire à son type : un attribut `annee` de type entier refuse la valeur `"hier"`. ch. 6, p. 284

Contrainte de référence Une clé étrangère ne peut désigner qu'une valeur existante : impossible d'ajouter une ligne qui référence une clé absente, ou de supprimer une ligne encore référencée. ch. 6, p. 285

Contraintes d'intégrité Règles qui gardent les données cohérentes : contraintes de domaine, d'entité (clé primaire), de référence (clé étrangère), plus `NOT NULL`, `UNIQUE` et règles métier. ch. 6, p. 284

Coût logarithmique, linéaire, quasi-linéaire, quadratique Coûts en $O(\log_2 n)$, $O(n)$, $O(n \log_2 n)$, $O(n^2)$: recherche dichotomique, parcours d'une liste, tri fusion, tri par insertion. Diviser pour régner fait souvent gagner une classe. ch. 7, p. 342

Cycle Chaîne (non orientée) ou chemin (orienté) qui revient à son point de départ sans réemprunter deux fois la même arête. ch. 8, p. 392

D

Décidable Se dit d'un problème de décision pour lequel il existe un algorithme qui, pour toute entrée, s'arrête en un temps fini et renvoie la bonne réponse. ch. 13, p. 628

Degré d'un sommet Nombre de voisins d'un sommet (dans un graphe non orienté) ; un sommet de degré nul est isolé. ch. 8, p. 390

Dictionnaire Structure qui associe chaque valeur à une clé ; chercher une clé (`c in dico`) coûte à peu près $O(1)$, contre $O(n)$ pour `x in liste`. ch. 3, p. 138

Distance d'édition (Levenshtein) Nombre minimal d'opérations élémentaires (insérer, supprimer ou substituer une lettre) pour transformer un mot en un autre. ch. 11, p. 538

`DISTINCT` Mot-clé placé après `SELECT` qui supprime les doublons du résultat : chaque valeur n'apparaît qu'une fois. ch. 6, p. 289

Diviser pour régner Méthode de conception d'algorithmes en trois temps : diviser le problème en sous-problèmes du même type plus petits, les résoudre (souvent récursivement), combiner leurs solutions. ch. 7, p. 336

Domaine Type des valeurs d'un attribut (entier, texte, date…) ; la contrainte de domaine impose que chaque valeur lui appartienne. ch. 6, p. 283

E

Échange de Diffie-Hellman Méthode (1976) qui permet à deux personnes de fabriquer une clé secrète commune en public, grâce à l'exponentiation modulaire. ch. 10, p. 488

Effet de bord Modification, par une fonction, de quelque chose en dehors d'elle-même (variable globale, liste reçue en paramètre, affichage, fichier…), autre que sa valeur de retour. ch. 2, p. 70

Empiler, dépiler Empiler : poser un élément au sommet d'une pile. Dépiler : retirer l'élément du sommet et le renvoyer. On n'accède jamais directement au milieu d'une pile. ch. 3, p. 131

Encapsulation (objet) Principe qui consiste à cacher l'état interne d'un objet et à ne le laisser manipuler qu'à travers ses méthodes, sans toucher directement aux attributs. ch. 2, p. 75

Encapsulation (réseau) À l'émission, chaque couche ajoute son en-tête (son étiquette) à la donnée, comme des enveloppes emboîtées ; à l'arrivée, chaque couche retire le sien. ch. 9, p. 444

Enfiler, défiler Enfiler : ajouter un élément à la queue d'une file. Défiler : retirer l'élément de tête (le plus ancien) et le renvoyer. ch. 3, p. 133

Enregistrement Ligne d'une relation : une entité décrite par une valeur pour chaque attribut. ch. 6, p. 283

États d'un processus (élu, prêt, bloqué) Élu : le processus utilise le processeur (un seul à la fois par cœur). Prêt : il attend son tour. Bloqué : il attend une ressource (disque, clavier…) et ne peut pas avancer. ch. 4, p. 180

Exponentiation modulaire Calcul de $x^e \bmod n$, facile à faire mais très difficile à « défaire » ; au cœur de Diffie-Hellman et de RSA (`pow(x, e, n)` en Python). ch. 10, p. 488

F

Famine Situation d'un processus qui attend indéfiniment parce que d'autres, plus prioritaires, passent sans cesse avant lui ; on l'évite en remontant sa priorité. ch. 4, p. 184

Feuille Nœud d'un arbre qui n'a aucun fils ; dans un arbre binaire en Python, ses deux fils valent `None`. ch. 5, p. 230

File Structure linéaire où l'on ajoute d'un côté (la queue) et retire de l'autre (la tête) : le premier entré est le premier sorti (FIFO). Opérations : `enfiler`, `defiler`, `est_vide`. ch. 3, p. 133

Fils, père (arbre) Les nœuds situés juste sous un nœud sont ses fils ; ce nœud est leur père. Chaque nœud a un seul père, sauf la racine. ch. 5, p. 230

Fonction calculable Fonction pour laquelle il existe un algorithme (une suite finie d'opérations élémentaires) qui, pour toute entrée x, produit $f(x)$ en un temps fini. ch. 13, p. 631

Fonction pure Fonction dont le résultat ne dépend que de ses paramètres et qui ne produit aucun effet de bord : avec les mêmes arguments, elle renvoie toujours le même résultat. ch. 2, p. 70

Fonction récursive Fonction qui s'appelle elle-même dans son propre corps, sur un problème plus petit, jusqu'à atteindre une condition d'arrêt. ch. 1, p. 21

Fonctions d'agrégation Fonctions qui résument une colonne en une seule valeur : `COUNT` (compter), `SUM` (somme), `AVG` (moyenne), `MIN`, `MAX`. ch. 6, p. 289

Fusion de deux listes triées Construction d'une liste triée à partir de deux listes déjà triées, en prenant à chaque étape le plus petit des deux éléments de tête ; coût linéaire. ch. 7, p. 339

G

Graphe Donnée $G = (S,A)$ d'un ensemble fini de sommets et d'arêtes (non orienté) ou d'arcs (orienté). Son ordre est son nombre de sommets, sa taille son nombre d'arêtes. ch. 8, p. 390

Graphe connexe Graphe « d'un seul tenant » : toute paire de sommets peut être reliée par une chaîne. Sinon, il se décompose en composantes connexes. ch. 8, p. 391

Graphe d'attente Graphe orienté où $P \to R$ signifie « P attend R » et $R \to P$ « R est détenue par P » ; un cycle y signale un interblocage. ch. 4, p. 185

Graphe non orienté Graphe dont les arêtes se parcourent dans les deux sens (un réseau d'amitiés, une carte routière à double sens). ch. 8, p. 390

Graphe orienté Graphe dont les liens sont des arcs, à parcourir dans le sens de la flèche (le graphe des abonnements d'un réseau social). ch. 8, p. 391

Graphe pondéré Graphe dont chaque arête porte une valeur numérique, son poids (distance, coût, temps, débit…). ch. 8, p. 391

H

Hauteur d'un arbre Dans ce livre, nombre de nœuds du plus long chemin de la racine à une feuille : arbre vide 0, racine seule 1. En comptant les arêtes : racine seule 0, arbre vide -1. ch. 5, p. 231

HTTPS HTTP chiffré (le cadenas du navigateur) : TLS utilise l'asymétrique au début pour échanger une clé de session, puis tout le reste est chiffré en symétrique. ch. 10, p. 491

I

Implémentation Réalisation concrète d'une structure dans un langage (avec un tableau, une liste chaînée…) qui respecte l'interface ; une même interface admet plusieurs implémentations. ch. 3, p. 129

Indécidable Se dit d'un problème de décision qu'aucun algorithme ne peut résoudre correctement pour toutes les entrées, comme le problème de l'arrêt. ch. 13, p. 628

`INSERT`, `UPDATE`, `DELETE` Requêtes qui ajoutent (`INSERT INTO ... VALUES`), modifient (`UPDATE ... SET ... WHERE`) ou suppriment (`DELETE FROM ... WHERE`) des lignes. Ne jamais oublier le `WHERE`. ch. 6, p. 292

Instance Objet construit à partir d'une classe (`bilbo = Personnage("Bilbo", 15)`) ; deux instances d'une même classe ont chacune leurs propres attributs. ch. 2, p. 72

Interblocage (*deadlock*) Situation figée où des processus bloqués attendent chacun une ressource détenue par un autre : aucun ne peut plus avancer. ch. 4, p. 185

Interface Liste des opérations d'une structure de données, avec leur nom et leur effet ; on peut écrire un programme en n'utilisant que l'interface. ch. 3, p. 129

Inverse modulaire Inverse de a modulo n : le nombre a^{-1} tel que $a \times a^{-1}$ a pour reste 1 dans la division par n ; il sert au chiffrement affine et à RSA. ch. 10, p. 485

J

Jointure (`JOIN ... ON`) Opération qui recolle deux tables : `FROM A JOIN B ON A.cle_etrangere = B.cle_primaire` associe chaque ligne de `A` à la ligne de `B` correspondante. ch. 6, p. 291

L

Liste chaînée Structure formée de maillons dispersés en mémoire, chacun contenant une valeur et un lien vers le suivant (`None` à la fin) ; on y accède par le premier maillon, la tête. ch. 3, p. 136

Liste d'adjacence Représentation qui associe à chaque sommet la liste de ses voisins, en Python par un dictionnaire ; économe en mémoire, idéale si le graphe est creux. ch. 8, p. 394

Liste (type abstrait) Suite finie d'éléments manipulée par cinq opérations : `liste_vide`, `est_vide`, `ajouter_tete`, `tete` et `queue`. ch. 3, p. 138

Loi de Moore Prévision de Gordon Moore (1965, révisée en 1975) : le nombre de transistors sur une puce double environ tous les deux ans. ch. 14, p. 672

M

Machine de Turing Modèle théorique d'ordinateur (1936) : un ruban infini découpé en cases, une tête de lecture-écriture, un état parmi un nombre fini et une table de règles. ch. 13, p. 632

Machine universelle Machine de Turing qui, lisant sur son ruban la table de règles d'une autre machine et son entrée, la simule pas à pas : l'idée de l'ordinateur à programme enregistré. ch. 13, p. 632

Maillon (cellule) Élément d'une liste chaînée : une valeur et un lien vers le maillon suivant. En Python, une classe `Cellule` à deux attributs, `valeur` et `suivante`. ch. 3, p. 136

Masque jetable Chiffrement par une clé aussi longue que le message, totalement aléatoire et utilisée une seule fois, combinée par XOR : le seul code inviolable. ch. 10, p. 486

Masque, notation CIDR Écriture `/n` du masque : les n premiers bits forment la partie réseau, les $32 - n$ autres la partie hôte. Bits d'hôte tous à 0 :

adresse du réseau ; tous à 1 : adresse de diffusion. ch. 9, p. 443

Matrice d'adjacence Tableau carré M (sommets numérotés) où $M[i][j] = 1$ si les sommets i et j sont voisins, 0 sinon ; symétrique pour un graphe non orienté. Idéale si le graphe est dense. ch. 8, p. 393

Mémoïsation Technique qui munit une fonction récursive d'un dictionnaire (le « carnet ») des résultats déjà calculés : résolution de haut en bas (*top-down*), chaque sous-problème une seule fois. ch. 11, p. 533

Méthode Fonction définie à l'intérieur d'une classe, qui agit sur l'instance grâce à `self` ; on l'appelle avec la notation pointée : `bilbo.perd_vie(4)`. ch. 2, p. 74

Métrique Valeur qui mesure le « coût » d'une route dans une table de routage : plus elle est petite, mieux c'est ; un réseau directement raccordé a une métrique nulle. ch. 9, p. 445

Microcontrôleur Mini-ordinateur sur une puce, minuscule, bon marché et sobre, conçu pour une tâche précise ; sans système d'exploitation complet, il exécute directement son programme. ch. 14, p. 676

Modèle de von Neumann Architecture d'un ordinateur : une unité de calcul (processeur), une mémoire contenant à la fois programmes et données, des entrées-sorties, reliés par des bus. ch. 14, p. 673

Modèle en couches (TCP/IP) Organisation de la communication en couches empilées (application, transport, réseau, lien), chacune ne s'occupant que d'une chose. ch. 9, p. 444

Modèle relationnel Organisation des données en tables (relations) reliées entre elles, proposée par Edgar F. Codd (IBM) en 1970. ch. 6, p. 283

Module, bibliothèque Un module est un fichier Python (`.py`) regroupant les fonctions et les classes d'un même thème, que l'on peut importer ; une bibliothèque est un ensemble de modules. ch. 2, p. 81

Motif Petite chaîne de caractères, de longueur m, que l'on cherche dans un texte. ch. 12, p. 585

Mutateur Méthode qui modifie un attribut, en vérifiant au besoin que la nouvelle valeur est correcte (souvent nommée `set_...` dans les sujets de bac). ch. 2, p. 76

N

Notation O (grand O) $O(f(n))$ désigne un coût au plus proportionnel à $f(n)$ quand la taille n des données devient grande ; les constantes et les termes plus petits ne comptent pas. ch. 7, p. 342

NP-complet (Hors programme.) Se dit d'un problème parmi les plus durs de NP : tout problème de NP peut s'y ramener (SAT, voyageur de commerce, coloration à 3 couleurs). ch. 13, p. 635

Nœud Chacun des éléments d'un arbre ; il porte une valeur et peut avoir des fils. ch. 5, p. 230

O

Objet Entité informatique qui réunit des attributs (des données qui décrivent son état) et des méthodes (des fonctions qui décrivent son comportement). ch. 2, p. 72

`ORDER BY` Clause qui trie le résultat d'une requête selon une colonne : `ASC` croissant (par défaut), `DESC` décroissant. ch. 6, p. 289

Ordonnancement Stratégie par laquelle le système choisit, parmi les processus prêts, celui qui obtient le processeur, et pour combien de temps. ch. 4, p. 183

Ordonnancement par priorités Chaque processus reçoit une valeur de priorité ; on élit le prêt le plus urgent (au bac, souvent la plus petite valeur : vérifier l'énoncé). ch. 4, p. 184

Ordonnancement préemptif Ordonnancement qui peut interrompre le processus élu, par exemple quand arrive un processus plus court ou plus urgent. ch. 4, p. 184

Ordonnanceur Rouage du système d'exploitation (*scheduler*) qui choisit, parmi les processus prêts, celui qui obtient le processeur. ch. 4, p. 183

OSPF Protocole de routage à état de liens : chaque routeur connaît toute la carte, la métrique d'une route est la somme des coûts de ses liaisons. ch. 9, p. 447

OU exclusif (XOR) Opération booléenne bit à bit (`^` en Python) qui vaut 1 si les deux bits diffèrent ; elle est son propre inverse, d'où son usage en chiffrement. ch. 10, p. 486

P

Paradigme de programmation Manière de concevoir et d'organiser un programme (impératif, fonctionnel, objet...). Un même langage peut en autoriser plusieurs, et un programme les mélanger. ch. 2, p. 69

Paradigme fonctionnel Style fondé sur des fonctions pures, sans effet de bord, et sur des données immuables ; la création de liste par compréhension en relève. ch. 2, p. 70

Paradigme impératif Style où le programme est une suite d'ordres (affectations, boucles, tests) qui font évoluer pas à pas l'état de la mémoire ; l'ordre d'exécution compte. ch. 2, p. 69

Parcours en largeur (BFS) Parcours niveau par niveau (arbre) ou par distance croissante au départ (graphe), à l'aide d'une file ; sur un graphe, il donne un plus court chemin en nombre d'arêtes. ch. 5, p. 234 ; ch. 8, p. 397

Parcours en profondeur (DFS) Parcours qui va « le plus loin possible » avant de revenir en arrière, par récursivité ou avec une pile ; sur un arbre binaire : préfixe, infixe, suffixe. ch. 5, p. 233 ; ch. 8, p. 397

Parcours infixe Parcours en profondeur : le sous-arbre gauche, puis la racine, puis le droit ; dans un ABR, il donne les valeurs dans l'ordre croissant. ch. 5, p. 233

Parcours préfixe Parcours en profondeur d'un arbre binaire : la racine, puis le sous-arbre gauche, puis le droit. ch. 5, p. 233

Parcours suffixe Parcours en profondeur : le sous-arbre gauche, puis le droit, puis la racine ; utile pour évaluer une expression. ch. 5, p. 233

Père, fils (processus) Le processus qui en crée un autre (par l'appel système `fork` sous Unix) est le père, le processus créé est le fils : les processus forment un arbre. ch. 4, p. 181

PID, PPID Le PID est l'identifiant unique d'un processus ; le PPID est celui de son père. Le processus `init` (PID 1) est la racine de tous les autres. ch. 4, p. 181

Pile Structure linéaire où l'on n'ajoute et ne retire un élément qu'au sommet : le dernier entré est le premier sorti (LIFO). Opérations : `empiler`, `depiler`, `sommet`, `est_vide`. ch. 3, p. 130

Pile d'exécution (pile d'appels) Structure où le système enregistre les fonctions en cours d'exécution : celle du sommet s'exécute, celles du dessous sont en pause ; une fonction terminée est dépilée. ch. 1, p. 20

Plus court chemin Chemin de longueur minimale entre deux sommets ; le parcours en largeur le trouve en nombre d'arêtes, l'algorithme de Dijkstra en tenant compte des poids. ch. 8, p. 398

Plus court d'abord (SJF) Ordonnancement qui élit, chaque fois que le processeur se libère, le processus prêt dont la durée est la plus courte ; il réduit le temps d'attente moyen. ch. 4, p. 183

Plus longue sous-séquence commune Plus long mot obtenu à partir de deux mots en effaçant des lettres sans changer l'ordre des autres ; calculée par une table $L[i][j]$. ch. 11, p. 537

Premier arrivé, premier servi (FCFS) Ordonnancement qui sert les processus dans leur ordre d'arrivée, chacun jusqu'au bout, sans jamais l'interrompre. ch. 4, p. 183

Prétraitement Calcul fait une fois sur le motif avant la recherche (la table de décalage, en $O(m)$) : on dépense un peu de mémoire pour gagner du temps. ch. 12, p. 591

Problème de décision Question dont la réponse est « oui » ou « non », posée sur une donnée d'entrée. ch. 13, p. 628

Problème de l'arrêt Savoir si un programme P lancé sur une entrée E s'arrête. Turing (1936) a prouvé qu'aucun programme ne peut le décider à coup sûr : il est indécidable. ch. 13, p. 630

Problème de l'échange de clé Difficulté du chiffrement symétrique : comment partager la clé secrète sur un réseau où tout peut être intercepté, sans s'être jamais rencontrés ? ch. 10, p. 487

Problème du sac à dos Choisir des objets (poids, valeur) pour maximiser la valeur totale sans dépasser la capacité du sac ; on le résout par une table à deux entrées. ch. 11, p. 536

Processus Programme en cours d'exécution : le programme est un texte inerte rangé sur le disque, le processus est ce texte « en train de vivre » en mémoire, avec ses données et ses ressources. ch. 4, p. 180

Profondeur d'un nœud Nombre d'arêtes du chemin de la racine jusqu'au nœud : la racine est à la profondeur 0 (autre convention, en nœuds : racine à 1 ; suivre l'énoncé). ch. 5, p. 231

Programmation dynamique Méthode qui résout un problème à partir de ses sous-problèmes en retenant chaque résultat, pour ne jamais calculer deux fois la même chose (mémoïsation ou tabulation). ch. 11, p. 532

Programme (comme donnée) Un programme est une simple suite de caractères : il peut être la donnée d'entrée d'un autre programme (interpréteur, compilateur, antivirus). ch. 13, p. 627

`ps`, `top`, `kill` Commandes du terminal : `ps -ef` liste tous les processus avec leur PID et leur PPID, `top` les affiche en temps réel, `kill` suivi d'un PID demande l'arrêt d'un processus. ch. 4, p. 182

Q

Quantum Petit intervalle de temps accordé à un processus avant que le processeur puisse être donné à un autre (tourniquet). ch. 4, p. 183

R

Racine Nœud du sommet d'un arbre, le seul à n'avoir pas de père ; on dessine les arbres « la tête en bas », racine en haut. ch. 5, p. 230

Recherche dichotomique Recherche dans un tableau trié : on compare la valeur cherchée à celle du milieu et on recommence sur la moitié qui reste ; coût logarithmique ($\log_2 n$). ch. 7, p. 337

Recherche naïve (fenêtre glissante) On superpose le motif au texte, on compare lettre à lettre et, en cas d'échec, on décale d'un seul cran ; jusqu'à $n \times m$ comparaisons. ch. 12, p. 586

Recherche textuelle Recherche des occurrences d'un motif (petite chaîne, de longueur m) dans un texte (grande chaîne, de longueur $n \geq m$). ch. 12, p. 585

Règle du mauvais caractère En cas d'échec sur un caractère du texte, on décale le motif pour aligner ce caractère avec sa dernière occurrence dans le motif, ou on le dépasse s'il en est absent. ch. 12, p. 587

Relation de récurrence Formule qui exprime la solution d'un problème à partir de celles de sous-problèmes plus petits, avec ses cas de base ; c'est le cœur de toute programmation dynamique. ch. 11, p. 535

Relation (table) Tableau à deux dimensions d'une base de données : chaque colonne est un attribut, chaque ligne un enregistrement. ch. 6, p. 283

Rendu de monnaie Problème : rendre une somme avec le moins de pièces possible. Avec $\{1, 3, 4\}$, le glouton rend 6 en trois pièces, la programmation dynamique en deux. ch. 11, p. 535

Réseau local (LAN) Ensemble de machines proches qui partagent la même adresse réseau ; elles y sont reliées par des switchs. ch. 9, p. 443

RIP Protocole de routage dont la métrique est le nombre de sauts (routeurs traversés), vecteur de distance appuyé sur Bellman-Ford ; plafonné à 15 sauts. ch. 9, p. 446

Routage statique, routage dynamique Statique : chaque ligne de la table est écrite à la main. Dynamique : des protocoles (RIP, OSPF) découvrent les routes automatiquement. ch. 9, p. 445

Routeur Appareil qui relie plusieurs réseaux locaux entre eux, avec une interface (carte réseau) par réseau raccordé ; il aiguille les paquets grâce à sa table de routage. ch. 9, p. 443

RSA Chiffrement asymétrique (Rivest, Shamir, Adleman, 1977) dont la clé publique contient $n = p \times q$, produit de deux grands nombres premiers ; sa sûreté repose sur la difficulté de factoriser n. ch. 10, p. 489

S

Schéma relationnel Nom d'une relation avec, pour chaque attribut, son nom et son domaine (clé primaire soulignée, clé étrangère précédée de #) ; le schéma d'une base réunit ceux de ses relations. ch. 6, p. 283

`SELECT ... FROM ... WHERE` Requête de base : `SELECT` liste les colonnes (`*` : toutes), `FROM` indique la table, `WHERE` garde les lignes qui satisfont une condition (`AND`, `OR`, `NOT`). ch. 6, p. 288

SGBD Système de gestion de bases de données : logiciel qui stocke, protège et interroge une base (persistance, cohérence, efficacité, accès concurrents, sécurité), piloté en SQL. ch. 6, p. 286

Signature numérique Message chiffré avec la clé privée de l'expéditeur : chacun peut le déchiffrer avec sa clé publique, ce qui prouve qu'il vient bien de lui (authentification). ch. 10, p. 489

Sommet (d'un graphe) Chacun des « points » d'un graphe (*vertex* en anglais), reliés entre eux par des arêtes ou des arcs. ch. 8, p. 390

Sous-arbre Arbre formé d'un nœud et de tous ses descendants : tout nœud est la racine d'un sous-arbre (sous-arbres gauche et droit en binaire). ch. 5, p. 230

Sous-problèmes qui se recouvrent Premier ingrédient de la programmation dynamique : la solution fait appel plusieurs fois aux mêmes sous-calculs (sinon, mémoïser ne servirait à rien). ch. 11, p. 540

Sous-structure optimale Second ingrédient de la programmation dynamique : la solution optimale du problème se construit à partir des solutions optimales de ses sous-problèmes. ch. 11, p. 540

SQL *Structured Query Language* : langage universel des bases relationnelles, pour interroger (lire) et modifier les données. ch. 6, p. 287

Switch (commutateur) Appareil qui relie les machines à l'intérieur d'un même réseau local. ch. 9, p. 443

Système d'exploitation (OS) Programme qui gère la machine et fait tourner tous les autres : il partage le processeur et les ressources entre les processus, gère la mémoire, les fichiers et les droits. ch. 4, p. 179

Système embarqué Système informatique intégré à un objet pour une tâche précise (carte bancaire, thermostat, drone, voiture), en dialogue avec le monde physique. ch. 14, p. 676

Système sur puce (SoC) Circuit intégré unique qui rassemble processeur, mémoires caches, processeur graphique, gestion d'énergie et périphériques (radio, filaires), reliés par un réseau sur puce. ch. 14, p. 674

T

Table de décalage Table construite en prétraitant le motif : la lettre en position k (sauf la dernière) reçoit le décalage $m - 1 - k$; toute lettre absente reçoit m. ch. 12, p. 589

Table de routage Table d'un routeur qui indique, pour chaque réseau de destination, par où sortir (interface ou routeur voisin) et la métrique de la route. ch. 9, p. 445

Tableau Structure qui range ses éléments dans des cases mémoire contiguës : accès direct à la case d'indice i, mais taille fixée (insérer au milieu oblige à tout décaler). ch. 3, p. 136

Tableau dynamique Nature réelle de la `list` de Python : un tableau qui s'agrandit seul. Elle peut

simuler une pile ou une file, mais n'est pas le type abstrait « liste ». ch. 3, p. 136

Tabulation Technique qui résout les sous-problèmes du plus petit au plus grand en rangeant leurs résultats dans un tableau : résolution de bas en haut (*bottom-up*). ch. 11, p. 534

Taille d'un arbre Nombre de nœuds de l'arbre ; l'arbre vide a pour taille 0. Récursivement : 1+ taille(gauche) + taille(droit). ch. 5, p. 231

TCP Protocole de la couche transport qui garantit que tout arrive, dans l'ordre et sans perte : il numérote les paquets et redemande ceux qui manquent (accusés de réception). ch. 9, p. 451

Temps d'attente Temps qu'un processus passe à ne pas s'exécuter alors qu'il est arrivé ; temps d'attente = temps de séjour − durée de calcul. Sa moyenne mesure la qualité d'un ordonnancement. ch. 4, p. 183

Temps de séjour Instant de fin d'un processus moins son instant d'arrivée (certains sujets disent temps de rotation ou temps d'exécution). ch. 4, p. 183

Temps réel Contrainte d'un système qui doit réagir dans un délai garanti (un airbag) ; d'où des microcontrôleurs sans système d'exploitation lourd. ch. 14, p. 676

Terminaison Propriété d'un algorithme qui s'arrête en un temps fini, quelle que soit l'entrée ; on la justifie en exhibant un variant. ch. 1, p. 26

Théorème de Rice (Hors programme.) Toute question non triviale sur le comportement d'un programme est indécidable : aucun antivirus ne peut garantir de tout détecter. ch. 13, p. 631

Thèse de Church-Turing Tout ce qui est calculable par un algorithme est calculable par une machine de Turing. ch. 13, p. 632

Tourniquet (*round-robin*) Ordonnancement où les processus prêts tournent dans une file : chacun s'exécute un quantum, puis repart en fin de file s'il n'a pas fini. ch. 4, p. 184

Transaction Suite d'opérations qu'un SGBD exécute en entier ou pas du tout (un virement : débit et crédit) ; en cas de panne, elle est annulée. Garanties résumées par l'acronyme ACID. ch. 6, p. 282

Tri fusion Tri « diviser pour régner » (von Neumann, 1945) : on coupe le tableau en deux, on trie récursivement chaque moitié, puis on fusionne ; coût $n \log_2 n$, même au pire. ch. 7, p. 338

Tri rapide (*quicksort*) Tri « diviser pour régner » : on choisit un pivot, on partitionne (plus petits à gauche, plus grands à droite) et on trie chaque côté ; $n \log_2 n$ en moyenne, n^2 au pire. ch. 7, p. 343

Turing-complet Se dit d'un langage ou d'un système qui peut simuler une machine de Turing, donc calculer tout ce qui est calculable ; les langages de programmation se valent ainsi. ch. 13, p. 632

Type abstrait de données Description d'une structure par les opérations qu'on peut lui appliquer (son « quoi »), sans dire comment elles sont réalisées (son « comment »). ch. 3, p. 129

V

Variant Quantité entière positive qui décroît strictement à chaque appel récursif (ou à chaque tour de boucle) : son existence prouve que la condition d'arrêt est atteinte. ch. 1, p. 26

Python : mots-clés, fonctions et syntaxe

% (reste, modulo) `x % n` est le reste de la division de `x` par `n` : il fait « boucler » l'alphabet dans les chiffrements (`(x + k) % 26`). ch. 10, p. 484

^ (OU exclusif) Opérateur XOR bit à bit de Python, au cœur du masque jetable : appliqué deux fois avec la même clé, il redonne le message, $(m \oplus k) \oplus k = m$. ch. 10, p. 486

`append`, `pop` Méthodes de `list` : `append(x)` ajoute à la fin, `pop()` retire et renvoie le dernier élément (une pile), `pop(0)` le premier (une file, mais en $O(n)$: tout est décalé). ch. 3, p. 134

`assert` `assert condition, "message"` vérifie, pendant la mise au point et les tests, ce qui doit être vrai ; sinon il lève `AssertionError`. ch. 2, p. 78

Carnet `memo` (mémoïsation) `if n in memo: return memo[n]` lit un résultat déjà calculé, `memo[n] = resultat` l'écrit. Piège : `memo={}` par défaut est partagé entre tous les appels. ch. 11, p. 534

`class` Mot-clé qui déclare une classe, suivi de son nom (par convention avec une majuscule) : `class Personnage:`. On crée une instance par `Personnage()`. ch. 2, p. 73

Docstring Texte entre triples guillemets placé en tête d'un module, d'une fonction ou d'une méthode : ce qu'elle reçoit, ce qu'elle renvoie, les erreurs qu'elle lève. ch. 2, p. 82

`exec` Fonction qui exécute un programme donné sous forme de texte : le même texte peut être lu comme une donnée ou exécuté. ch. 13, p. 628

`float("inf")` Infini de Python, plus grand que tout nombre : valeur de départ d'un minimum à calculer (rendu de monnaie, Dijkstra). ch. 11, p. 536

help `help(nom)` affiche dans la console la docstring d'une fonction, d'un module ou d'une classe. ch. 2, p. 81

if __name__ == "__main__": Bloc exécuté quand on lance le fichier lui-même, mais pas quand on l'importe : on y place les tests d'un module. ch. 2, p. 82

import, from ... import `import math` charge tout un module (ses noms s'écrivent avec le préfixe `math.`) ; `from random import randint` importe un seul nom, utilisable sans préfixe. ch. 2, p. 81

in (liste, dictionnaire) `x in liste` compare `x` aux éléments un par un, en $O(n)$; `c in dico` cherche une clé en $O(1)$ environ ; `v in dico.values()` reste linéaire. ch. 3, p. 139

__init__ Constructeur : méthode spéciale appelée automatiquement à la création d'une instance, où l'on crée tous ses attributs (`self.vie = 20`). ch. 2, p. 73

is None, is not None Test qui compare à `None` ; schéma de parcours d'une liste chaînée : `while c is not None:` puis `c = c.suivante`. ch. 3, p. 137

lambda Fonction anonyme écrite en une expression : `lambda x: x * x` est la fonction qui, à `x`, associe `x * x`. ch. 2, p. 71

Liste par compréhension Expression `[f(x) for x in L if condition]` qui construit une nouvelle liste sans modifier celle qu'elle parcourt : un geste du style fonctionnel. ch. 2, p. 71

None Valeur spéciale « rien » : elle marque la fin d'une liste chaînée, un sous-arbre vide, ou sert de valeur par défaut. ch. 3, p. 136

Notation pointée (objet.attribut, objet.methode()) Accès à un attribut (`gollum.vie`) ou appel d'une méthode (`bilbo.perd_vie(4)`) d'une instance ; `self` n'est pas écrit entre les parenthèses. ch. 2, p. 73

Paramètre par défaut (def f(n, acc=0):) Valeur écrite dans l'en-tête, utilisée quand l'appel ne fournit pas ce paramètre ; pour une liste, mettre `None` par défaut et créer la liste dans la fonction. ch. 1, p. 25

pass Instruction qui ne fait rien ; elle permet d'écrire une classe ou une fonction pour l'instant vide. ch. 2, p. 73

pow(x, e, n), pow(a, -1, n) `pow(x, e, n)` calcule x^e mod n par exponentiation modulaire rapide ; `pow(a, -1, n)` renvoie l'inverse modulaire de a modulo n. ch. 10, p. 485

raise Mot-clé qui lève une erreur (une exception) : `raise ValueError("message")`. La fonction s'arrête immédiatement ; cela fait partie de son comportement prévu. ch. 2, p. 77

RecursionError Erreur levée quand la pile d'appels devient trop profonde : le plus souvent, une condition d'arrêt oubliée ou jamais atteinte. ch. 1, p. 21

__repr__, __str__ Méthodes spéciales d'affichage : `__repr__` sert quand on tape le nom de l'objet dans la console, `__str__` sert à `print` (à défaut, `print` utilise `__repr__`). ch. 2, p. 79

self Premier paramètre de toute méthode : il désigne l'instance en train d'être manipulée ; dans la classe, un attribut se lit `self.attribut`. ch. 2, p. 73

sorted, list.sort Tri de Python (Tim sort, mélange de tri fusion et de tri par insertion) ; `key=` reçoit la fonction qui donne la valeur à comparer. ch. 2, p. 71 ; ch. 7, p. 343

Tiret bas (_solde) Convention : un attribut préfixé d'un tiret bas est interne et ne doit pas être utilisé hors de la classe ; Python ne l'interdit pas techniquement. ch. 2, p. 76

type `type(x)` renvoie la classe d'une valeur : `type("radar")` renvoie `<class 'str'>`. `str`, `int` et `list` sont des classes. ch. 2, p. 73

ValueError, IndexError, TypeError Types d'erreurs courants : valeur incorrecte, indice hors limites (pile vide…), type incorrect ; `AttributeError` signale un attribut inexistant. ch. 2, p. 78

while True Boucle sans fin : celle d'un microcontrôleur qui lit ses capteurs, décide et commande ses actionneurs tant qu'il est sous tension. ch. 14, p. 677

Crédits iconographiques

Illustrations d'ouverture de chapitre — dessins originaux de l'auteur.

Présentation et panorama de l'année

Capture d'écran du logiciel Spyder (logiciel libre, licence MIT) — capture réalisée par l'auteur — licence MIT pour le logiciel
`https://www.spyder-ide.org`

Leonhard Euler (portrait de 1753) — Jakob Emanuel Handmann — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Leonhard_Euler.jpg`

Les tours de Hanoï, gravure de 1883 — Édouard Lucas — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:LucasTourHanoi_crop.jpg`

Alan Turing à 16 ans — auteur inconnu (peut-être A. R. Chaffin) — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Alan_Turing_at_age_16.jpg`

Photo du badge de John von Neumann à Los Alamos — Los Alamos National Laboratory — Los Alamos National Laboratory (mention obligatoire) — p. 15
`https://commons.wikimedia.org/wiki/File:John_von_Neumann_Los_Alamos_identity_badge_photo.jpg`

Friedrich L. Bauer — Hcii — CC BY-SA 3.0 — p. 15
`https://commons.wikimedia.org/wiki/File:FriedrichLudwigBauer.jpg`

Edsger W. Dijkstra — Hamilton Richards — CC BY-SA 3.0 — p. 15
`https://commons.wikimedia.org/wiki/File:Edsger_Wybe_Dijkstra.jpg`

Microprocesseur Intel C4004 — Thomas Nguyen — CC BY-SA 4.0 — p. 15
`https://commons.wikimedia.org/wiki/File:Intel_C4004.jpg`

Whitfield Diffie — Duncan Hull — CC BY-SA 4.0 — p. 15
`https://commons.wikimedia.org/wiki/File:Whitfield_Diffie_Royal_Society.jpg`

Chapitre 1 — Récursivité

Les tours de Hanoï, gravure de 1883 — Édouard Lucas — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:LucasTourHanoi_crop.jpg`

John McCarthy à Stanford (2006) — null0 — CC BY 2.0 — p. 29
`https://commons.wikimedia.org/wiki/File:John_McCarthy_(computer_scientist)_Stanford_2006_(272020300).jpg`

Chapitre 3 — Structures linéaires : piles, files, listes chaînées

Friedrich L. Bauer — Hcii — CC BY-SA 3.0 — p. 15
`https://commons.wikimedia.org/wiki/File:FriedrichLudwigBauer.jpg`

Chapitre 4 — Processus et ordonnancement

Grace Hopper en 1984 — James S. Davis (U.S. Navy) — domaine public — p. 188
`https://commons.wikimedia.org/wiki/File:Commodore_Grace_M._Hopper,_USN_(covered).jpg`

Edsger W. Dijkstra — Hamilton Richards — CC BY-SA 3.0 — p. 15
`https://commons.wikimedia.org/wiki/File:Edsger_Wybe_Dijkstra.jpg`

Microprocesseur Intel C4004 — Thomas Nguyen — CC BY-SA 4.0 — p. 15
`https://commons.wikimedia.org/wiki/File:Intel_C4004.jpg`

Chapitre 5 — Arbres

Arthur Cayley — Herbert Beraud — domaine public — p. 238
`https://commons.wikimedia.org/wiki/File:Arthur_Cayley.jpg`

Figuier « tête en bas », thermes romains de Baïes — Ralf Roletschek (RaBoe) — CC BY-SA 3.0 — p. 238
`https://commons.wikimedia.org/wiki/File:Baia-Complesso_Termal_Romano_2010-by-RaBoe-109.jpg`

Chapitre 6 — Bases de données et SQL

Mécanisme du disque IBM 350 (RAMAC) — Arnold Reinhold — CC BY-SA 3.0 — p. 294
`https://commons.wikimedia.org/wiki/File:IBM_350_RAMAC_disk_mechanism.agr.jpg`

Exploits of a Mom (xkcd n° 327) — Randall Munroe — CC BY-NC 2.5 — p. 292
`https://xkcd.com/327/`

Chapitre 7 — Diviser pour régner

Photo du badge de John von Neumann à Los Alamos — Los Alamos National Laboratory — Los Alamos National Laboratory (mention obligatoire) — p. 15
`https://commons.wikimedia.org/wiki/File:John_von_Neumann_Los_Alamos_identity_badge_photo.jpg`

La Joconde (recadrage carré) — Léonard de Vinci — domaine public — p. 370
`https://commons.wikimedia.org/wiki/File:Mona_Lisa,_by_Leonardo_da_Vinci,_from_C2RMF_retouched.jpg`

Chapitre 8 — Graphes

Königsberg en 1652, gravure — Merian-Erben — domaine public — p. 389
`https://commons.wikimedia.org/wiki/File:Image-Koenigsberg,_Map_by_Merian-Erben_1652.jpg`

Leonhard Euler (portrait de 1753) — Jakob Emanuel Handmann — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Leonhard_Euler.jpg`

Chapitre 9 — Réseaux

Croquis du réseau ARPANET, décembre 1969 — DARPA — domaine public — p. 452
`https://commons.wikimedia.org/wiki/File:A_sketch_of_the_ARPANET_in_December_1969.png`

Louis Pouzin — ITU Pictures — CC BY 2.0 — p. 452
`https://commons.wikimedia.org/wiki/File:Louis_Pouzin_(26191247583).jpg`

Chapitre 10 — Cryptographie

Whitfield Diffie — Duncan Hull — CC BY-SA 4.0 — p. 15
`https://commons.wikimedia.org/wiki/File:Whitfield_Diffie_Royal_Society.jpg`

Machine Enigma à trois rotors — Daderot — CC0 — p. 494
`https://commons.wikimedia.org/wiki/File:Enigma_machine,_3_rotor_-_National_Electronics_Museum_-_DSC00197.JPG`

Chapitre 11 — Programmation dynamique

Page du Liber abbaci (problème des lapins) — Biblioteca Nazionale Centrale di Firenze — domaine public — p. 532
`https://commons.wikimedia.org/wiki/File:Liber_abbaci_magliab_f124r.jpg`

Chapitre 12 — Recherche textuelle

Donald Knuth — orcmid — CC BY 2.0 — p. 593
`https://commons.wikimedia.org/wiki/File:2012-06-14_1974_Turing_Laureate_Donald_Knuth.jpg`

Chapitre 13 — Calculabilité et décidabilité

Alan Turing à 16 ans — auteur inconnu (peut-être A. R. Chaffin) — domaine public — p. 15
`https://commons.wikimedia.org/wiki/File:Alan_Turing_at_age_16.jpg`

Kurt Gödel étudiant (1925) — auteur inconnu — domaine public — p. 633
`https://commons.wikimedia.org/wiki/File:Young_Kurt_G%C3%B6del_as_a_student_in_1925.jpg`

Chapitre 14 — Systèmes sur puce et informatique embarquée

Microprocesseur Intel C4004 — Thomas Nguyen — CC BY-SA 4.0 — p. 672
`https://commons.wikimedia.org/wiki/File:Intel_C4004.jpg`

Puce Apple A13 Bionic (illustration) — Henriok — CC0 (domaine public) — p. 675
`https://commons.wikimedia.org/wiki/File:Apple_A13_Bionic.jpg`